OMAR DEV NOTES · COMPLETE EDITION

مرجع البرمجة وتطوير الويب الكامل

كل محتوى الدوكس الحالي في كتاب واحد: 140 درسًا لكل لغة، بالعربية والإنجليزية، من أساسيات الكمبيوتر وC++ إلى PHP والتشغيل والأمان وقواعد البيانات.

6 مسارات مترابطة

Arabic + English

إصدار ٢٠ أكتوبر ٢٠٢٦

CONTENTS · المحتويات

فهرس الكتاب

يبدأ كل مسار وكل درس في صفحة مستقلة لتسهيل القراءة والطباعة والمراجعة.

النسخة العربية

1. أساسيات الكمبيوتر والبرمجة والشبكات — 60 درسًا
2. لغة C++ — 14 درسًا
3. لغة PHP — 18 درسًا
4. تشغيل PHP والإنتاج — 20 درسًا
5. الأمان والهوية والصلاحيات — 22 درسًا
6. قواعد البيانات وPDO — 6 درسًا

English edition

1. Computer, programming, and networking fundamentals — 60 lessons
2. The C++ language — 14 lessons
3. The PHP language — 18 lessons
4. PHP runtime and production — 20 lessons
5. Security, identity, and authorization — 22 lessons
6. Databases and PDO — 6 lessons

OMAR DEV NOTES

النسخة العربية

نسخة منظمة للطباعة تشمل كل المسارات والدروس المنشورة في الدوكس.

01

المسار · 60 درسًا

أساسيات الكمبيوتر والبرمجة والشبكات

شرح متدرج، أمثلة عملية، وتصحيحات تساعدك على الانتقال من الفكرة إلى التطبيق.

أساسيات المبرمج: الكمبيوتر والبرامج والشبكات

افهم جهازك ونظام التشغيل وتنفيذ الكود والذاكرة والملفات، وبعدها الشبكات والويب وحل المشكلات.

أنت عايز تتعلم برمجة، فمحتاج تعرف إيه اللي بيحصل لما تشغّل كودك: مين بينفذ التعليمات؟ البيانات بتتحفظ فين؟ ليه البرنامج ممكن يستنى؟ وإزاي يوصل لبرنامج تاني على جهاز بعيد؟ ده هدف الكارد ده.

هنبدأ بمقدمة عن مكونات الكمبيوتر بالقدر اللي يخدم المبرمج، بعدها التركيز هيكون على السوفتوير والخدمات اللي برنامجك بيعتمد عليها. أمثلة الكود العامة بـPHP، وLaravel لما نحتاج تطبيق ويب أكبر. مسار C++ بيستخدم لغة المسار نفسه.

1. الكمبيوتر ونظام التشغيل من عين المبرمج

ابدأ من الكمبيوتر ونظام التشغيل.

هتعرف علاقة CPU وRAM والتخزين، وبعدها الفرق بين الكود والبرنامج والعملية اللي شغالة. هتمشي وراء تشغيل سكربت، وطلب فتح ملف، وتوزيع وقت المعالج، واستخدام الذاكرة والصلاحيات.

الترتيب: **هاردوير مختصر → سوفتوير وRuntime → نظام التشغيل → تنفيذ الكود → المعالج وتمثيل البيانات → الذاكرة والملفات → أدوات التشغيل والتصحيح**.

2. الشبكات والويب

انتقل إلى الشبكات والويب لما تبقى فاهم يعني إيه عملية وملف ومنفذ اتصال.

هتفهم إزاي الأجهزة بتوصل لبعض، وإزاي الاسم بيتحول لعنوان، وبعدها نقل البيانات وطلبات الويب والتشفير والخوادم. الهدف إنك تعرف تفسر اللي حصل وتحدد مكان العطل.

3. التفكير والحساب اللي تحتاجه في الكود

الرياضيات وحل المشكلات قسم ترجع له حسب احتياجك: المنطق والشروط، القسمة والباقي، دقة الأرقام، وصف خطوات الحل ومقارنة تكلفتها. مش مطلوب تخلص كل الرياضيات قبل ما تبدأ تتعلم لغة.

تذاكر إزاي؟

اقرأ الموقف الأول، وتابع مثالًا من المدخلات لحد النتيجة. قبل تشغيل المثال توقع اللي هيحصل، وبعده غيّر حالة واحدة وشوف هل تفسيرك فضل صحيح. المصطلح الجديد لازم يتشرح في السياق، والمتطلب السابق نذكّرك بمعناه مع رابط له.

بعد أساس التنفيذ تقدر تكمل PHP لتطبيقات الويب، أو C++ لو دي اللغة اللي بتذاكرها.

الكمبيوتر ونظام التشغيل للمبرمج

مقدمة هاردوير مختصرة، بعدها السوفتوير ونظام التشغيل وتنفيذ البرامج والذاكرة والملفات وأدوات التطوير.

لما برنامج يتأخر أو يستهلك ذاكرة أو يفشل يفتح ملف، حفظ قواعد اللغة مش كفاية علشان تعرف السبب. محتاج تعرف البرنامج بيطلب إيه من الجهاز ونظام التشغيل بيتعامل مع الطلب إزاي.

هنبدأ بالهاردوير بالقدر اللي يخدم المبرمج، وبعدها ندخل في السوفتوير والتنفيذ. درس المقدمة بيفرق بين الكود والبرنامج والعملية والـRuntime والمكتبة والـFramework قبل ما تحتاج تستخدم المصطلحات دي في نظام التشغيل.

امشِ بالترتيب ده

- مكوّنات الهاردوير واللوحة الأم
- السوفتوير: من الكود لبرنامج شغال
- نظام التشغيل: البنية والأنواع وطريقة العمل
- من خطوات الحل إلى برنامج يعمل
- كيف ينفّذ المعالج التعليمات؟
- تمثيل الأعداد والحروف في الذاكرة
- الصورة والصوت ووحدات الحجم
- العناوين والذاكرة الافتراضية
- حجز الذاكرة وعمر البيانات
- نقل البيانات ومتى يصبح الحفظ ثابتًا
- إمتى يفيد تشغيل أعمال كثيرة معًا؟
- اقرأ ملفاتك واكتبها بالطرفية
- قنوات الأوامر والصلاحيات والبيئة
- اختبر السلوك وحدّد سبب الخطأ

اربط اللي بتشوفه بالسبب

قبل تشغيل المثال توقع النتيجة، وبعده غيّر مدخلًا واحدًا وراجع تفسيرك. في مثال الملف، حدد العملية وهويتها والمسار والناتج المطلوب. في مثال التأخير، اسأل هل البرنامج بيحسب فعلًا ولا مستني موردًا؟

بعد فهم العملية والذاكرة والموارد، انتقل إلى الشبكات والويب علشان تتبع التواصل بين البرامج.

مكوّنات الهاردوير واللوحة الأم

اللوحة الأم والناقلات والتخزين وأجهزة الإدخال والإخراج والطاقة والتبريد ورحلة البيانات داخل الجهاز.

كل وصلة لها وظيفة

الجزء	شرح قريب من وظيفته	مثال
Socket — مقبس المعالج	مكان يثبت فيه المعالج ويوصله باللوحة	المعالج لازم يناسب المقبس؛ تشابه الحجم وحده لا يكفي
Bus — ناقل	طريق اتصال لنقل بيانات وإشارات بين أجزاء	USB يصل الجهاز الخارجي بالمتحكم
Seek — تحريك رأس القرص	انتقال الرأس لمكان البيانات في HDD	قراءة أماكن متناثرة تتطلب حركة أكثر؛ SSD لا يملك رأسًا متحركًا
PSU — Power Supply Unit	مزوّد يحوّل كهرباء المصدر إلى جهود تحتاجها القطع	قدرته وجودته والتوصيلات تؤثر في استقرار الجهاز

تخيل فتح ملف: وسيط التخزين يقرأ، الناقل ينقل، الذاكرة تحمل، والمعالج يستخدم البيانات. الطاقة والتبريد يسمحان للأجزاء بالعمل؛ لا يخزنان نسخة الملف.

الهاردوير كنظام مترابط

الكمبيوتر ليس CPU (Central Processing Unit؛ المعالج الرئيسي الذي ينفذ التعليمات) محاطًا بقطع مستقلة. اللوحة الأم تربط المعالج والذاكرة والتخزين وبطاقات التوسعة عبر مسارات كهربائية وبروتوكولات. البرنامج لا يرسل أمرًا مباشرًا لكل ترانزستور؛ يطلب خدمة من نظام التشغيل، والـDriver (تعريف الجهاز: برنامج يتيح للنظام التعامل معه) يتعامل مع Controller (متحكم: مكوّن يدير تعامل الجهاز مع جزء مادي) الجهاز.

اللوحة الأم وChipset

تحدد اللوحة Socket المعالج، نوع RAM (Random Access Memory؛ ذاكرة العمل التي تحمل بيانات وتعليمات البرامج النشطة) وعدد القنوات، منافذ التخزين، مسارات PCIe (Peripheral Component Interconnect Express؛ اتصال سريع بين مكونات الجهاز عبر مسارات بيانات)، ومنافذ الإدخال والإخراج. الـChipset (مجموعة دوائر توفر وصلات ووحدات تحكم للمكونات) يضيف Controllers ومسارات لا يوفرها المعالج مباشرة. التوافق لا يتحدد بشكل الموصل فقط؛ Firmware (برنامج منخفض المستوى محفوظ مع الجهاز لتهيئته والتحكم فيه) والطاقة والإصدار وعدد Lanes (مسارات بيانات في وصلة مثل PCIe) عوامل مهمة.

Buses وPCIe وUSB

الـBus ينقل Data وAddresses وControl Signals. PCIe اتصال تسلسلي Point-to-point يستخدم Lanes؛ بطاقة `x16` لديها مسارات أكثر من `x1` لكن السرعة الفعلية تعتمد أيضًا على جيل PCIe والجهاز. USB (Universal Serial Bus؛ معيار توصيل أجهزة لنقل بيانات وتوفير طاقة بحسب الدعم) يربط أجهزة متنوعة ويجمع الطاقة والبيانات وفق الإصدار والمنفذ؛ شكل USB-C لا يضمن وحده سرعة

أوقدرة معينة.

التخزين: HDD وSSD وNVMe

- HDD (Hard Disk Drive؛ وحدة تخزين بأقراص مغناطيسية تدور) مغناطيسي ميكانيكي، سعته كبيرة وتكلفته منخفضة لكن Seek أبطأ.
- SATA (Serial ATA؛ معيار توصيل تخزين) SSD (Solid-State Drive؛ وحدة تخزين إلكترونية بلا أجزاء ميكانيكية متحركة) بلا أجزاء متحركة وزمن وصوله أقل.
- NVMe (Non-Volatile Memory Express؛ بروتوكول اتصال بتخزين غير متطاير، غالبًا عبر PCIe) SSD يتصل عادة عبر PCIe ويستفيد من Queues متوازية.

التخزين دائم نسبيًا، بينما RAM أسرع ومتطايرة. فتح ملف لا يعني نقل القرص كله إلى RAM؛ النظام يقرأ Blocks/Pages حسب الحاجة ويستخدم Cache (نسخة محفوظة لتقليل تكرار القراءة أو الحساب).

أجهزة الإدخال والإخراج

Keyboard وMouse وCamera وNetwork Card ترسل بيانات أوEvents. الشاشة والطابعة والسماعات تستقبل نتائج. بعض الأجهزة تفعل الأمرين. Controller يدير الجهاز، وDriver يقدم واجهة يفهمها Kernel (نواة نظام التشغيل التي تدير الموارد والوصول المحمي). Interrupt (مقاطعة: إشعار يلفت المعالج إلى حدث يحتاج معالجة) يخبر CPU بحدث بدل Polling (فحص دوري بدل انتظار إشعار من الطرف الآخر) مستمر، وقد يستخدم الجهاز DMA (Direct Memory Access؛ نقل بيانات بين جهاز والذاكرة من غير نسخ المعالج لكل بايت) لنقل Blocks إلى RAM بكلفة CPU أقل.

PSU والتبريد

Power Supply تحول الكهرباء إلى جهود مناسبة وتحتاج قدرة وجودة وحمايات ملائمة. الحرارة ترفع احتمال Throttling (تقليل معدل العمل، مثل خفض سرعة المعالج لحماية الحرارة أو الطاقة) وعدم الاستقرار؛ المشتت والمراوح وتدفق الهواء وThermal Interface (مادة أو سطح يحسن انتقال الحرارة بين الشريحة والمشتت) أجزاء وظيفية وليست شكلًا تجميليًا.

رحلة البيانات

```
SSD/HDD → Storage controller → RAM → CPU cache/registers
                                       ↓
                                  CPU executes
                                       ↓
RAM ← result ← GPU/NIC/display/storage controller
```

ليست كل خطوة نسخة كاملة. Cache وDMA وMemory Mapping (ربط ملف أو مورد بنطاق عناوين يستطيع البرنامج الوصول إليه) وBuffers تقلل النقل أو تؤجله. سرعة النظام تحددها أبطأ مرحلة في المسار الفعلي، لا أكبر رقم على قطعة منفردة.

إقلاع الجهاز وتشخيص البطء

Firmware — برمجيات الجهاز الأساسية تهيئ المكونات، و**Unified Extensible Firmware Interface) UEFI؛ واجهة برمجيات بدء تشغيل الجهاز)** واجهة بدء التشغيل التي تدير اختيارات الإقلاع وتشغّل **Bootloader — برنامج تحميل نظام التشغيل**. دول مش نظام التشغيل الكامل.

Bottleneck — الجزء الذي يحد الأداء يعتمد على العمل الفعلي. **Queue depth** عدد طلبات التخزين التي تنتظر أو يجري التعامل معها وفق الأداة، و**Utilization** نسبة انشغال مورد. **Thermal throttling** خفض سرعة المعالج عندما يصل لحد الحرارة؛ راقب الحرارة والسرعة معًا. قرص مشغول وطابور قراءة طويل يشيران لاتجاه مختلف عن معالج يخفض سرعته بسبب الحرارة.

Redundant Array of Independent Disks) RAID؛ تنظيم عدة أقراص لتحسين خواص مثل التوافر أو الأداء حسب النوع) تنظيم عدة أقراص بطرق تختلف في تحمل العطل أو الأداء. ليس نسخة احتياطية؛ حذف خاطئ أو تشفير ملفات ضار قد يؤثر في مجموعة الأقراص كلها.

تدريب وحل: فتح ملفات بطيء ومعالج قليل الانشغال. افحص زمن وصول التخزين وطابور الطلبات وحالة القرص، ولا تشترِ معالجًا اعتمادًا على البطء وحده. لو المشكلة تتزامن مع ارتفاع الحرارة وهبوط تردد المعالج، راجع التبريد والقياس تحت نفس الحمل.

تأكد من فهمك

01 **لماذا شكل USB-C لا يخبرك بالسرعة وحده؟**

الإجابة: الشكل Connector فقط؛ البروتوكول والإصدار والكابل والجهاز تحدد السرعة والطاقة والميزات.

02 **ما الفرق بين Controller وDriver؟**

الإجابة: Controller مكوّن هاردوير يدير الجهاز، وDriver برنامج داخل نظام التشغيل يتحدث معه ويعرض واجهة موحدة.

03 **لماذا NVMe ليس مجرد اسم أسرع لـSSD؟**

الإجابة: SSD يصف وسيط التخزين، بينما NVMe بروتوكول مصمم للتخزين غير المتطاير عبر PCIe وQueues متوازية.

04 **ما فائدة DMA؟**

الإجابة: يسمح للجهاز بنقل Blocks إلى RAM أو منها دون أن ينسخ CPU كل Byte بنفسه، مع بقاء الإعداد والتنسيق للنظام.

السوفتوير: من الكود لبرنامج شغال

افهم الفرق بين الكود والبرنامج والعملية وبيئة التنفيذ والمكتبات ونظام التشغيل، وامشِ وراء سكربت PHP من الملف لحد النتيجة.

أنت كتبت `echo 'Hello';` في ملف. الملف موجود على القرص، لكن لسه مفيش حاجة اتنفذت. إيه اللي بيحوّل الكلام ده لشغل فعلي على الجهاز؟ في الدرس ده هنفصل الأجزاء اللي بتدخل في التنفيذ، ونشغّل مثال صغير نقدر نفسر نتيجته.

محتاج تعرف إن المعالج بينفذ تعليمات، وإن RAM ذاكرة الشغل المؤقتة والتخزين بيحتفظ بالملفات. مش محتاج تكون درست لغة قبل كده.

الهاردوير موجود، مين بيحدد له يعمل إيه؟

الهاردوير، Hardware هو الأجزاء المادية زي المعالج والذاكرة والقرص. **السوفتوير، Software** هو البرامج والتعليمات والبيانات والإعدادات اللي بتحدد الشغل المطلوب من الجهاز. نفس الجهاز ممكن يشغّل متصفح أو برنامج حسابات؛ اللي اتغير هنا هو البرنامج اللي بيتنفذ والبيانات اللي بيعالجها.

السوفتوير مش نوع واحد. فيه تطبيقات بتستخدمها مباشرة زي المتصفح، ونظام تشغيل بيدير الموارد زي Windows أو نظام مبني على Linux، وأدوات للمبرمج زي PHP وCompiler ومحرر الكود. وفيه **Firmware**: برنامج بيكون مرتبط بالجهاز وبيشارك في تهيئته أو التحكم فيه، زي UEFI عند بدء تشغيل الكمبيوتر. الأدوار مختلفة، ومش لازم كل برنامج يكون له نافذة.

الكود والبرنامج والعملية: ثلاث حاجات مختلفة

Source code هو النص اللي المبرمج بيكتبه بلغة مفهومة لأدوات اللغة. ملف `hello.php` كود مصدر. **البرنامج** هو التعليمات والأجزاء اللي تحقق مهمة معينة؛ ممكن يتكون من ملف أو آلاف الملفات. **Process، عملية** هي نسخة شغالة من برنامج ولها هوية وذاكرة وموارد بيديرها نظام التشغيل.

لو شغّلت نفس سكربت PHP مرتين من نافذتين، عندك نفس كود المصدر لكن عمليتين منفصلتين. تغيير متغير في عملية مش بيغيّر تلقائيًا متغير العملية التانية. علشان يتبادلوا بيانات محتاجين وسيلة مشاركة واضحة، زي ملف أو قاعدة بيانات أو اتصال.

ملف البرنامج على القرص مش لازم يكون شغال. وبرنامج شغال مش لازم يكون بيحسب طول الوقت؛ ممكن يكون مستني المستخدم أو القرص أو الشبكة.

ليه المعالج مش بيقرأ PHP مباشرة؟

المعالج بينفذ تعليمات آلة مناسبة لمعمارية الجهاز. لغات البرمجة محتاجة أدوات تحول كودها أو تنفذه. **Compiler، مُترجم** بيحوّل كودًا لصيغة أخرى؛ الصيغة دي ممكن تكون تعليمات آلة أو تمثيل وسيط. **Interpreter، مفسر** بينفذ برنامجًا أو تمثيلًا له تحت إدارة برنامج آخر.

مش دقيق نقول كل لغة إما مترجمة فقط أو مفسرة فقط. تنفيذ PHP المعتاد مثلًا بيحلل المصدر ويحوّله لتعليمات وسيطة اسمها **Opcodes**، وبعدها محرك التنفيذ يشغّلها. **Runtime، بيئة التنفيذ** بتقدم خدمات وقت التشغيل، زي إدارة القيم والذاكرة وتنفيذ استدعاءات اللغة والتعامل مع الأخطاء.

PHP نفسه برنامج مكتوب ومبني علشان يشتغل على نظام التشغيل. لما كود PHP يطلب قراءة ملف، بيئة التنفيذ بتستخدم خدمات النظام للوصول للملف. مش سكربت PHP اللي بيتحكم مباشرة في القرص.

```
hello.php على القرص
        ↓ يقرأه برنامج PHP
تحليل الكود وتجهيزه للتنفيذ
        ↓
ينفذ التعليمات محرك PHP
        ↓ عند الحاجة
نظام التشغيل يدير ملفات وذاكرة وأجهزة
        ↓
المعالج ينفذ تعليمات البرامج والنظام
```

الرسم ده بيوضح المسؤوليات، مش مراحل زمنية منفصلة تمامًا. النظام حاضر طول التنفيذ، والمعالج بينفذ كود المحرك والنظام، مش مستنيهم يخلصوا الأول.

شغّل أصغر برنامج وتابع أثره

المثال محتاج PHP CLI، يعني إصدار PHP اللي بتشغله من سطر الأوامر. اتأكد إنه متاح:

```
php -v
```

لو الأمر مش معروف، ده معناه إن الطرفية مش لاقية برنامج PHP في مسارات البحث أو إنه مش متثبت. لسه السكربت نفسه ما بدأش؛ تغيير `echo` مش هيصلح المشكلة دي.

احفظ الملف باسم `hello.php`:

```
<?php
declare(strict_types=1);

$name = 'Omar';
echo 'Hello ' . $name . PHP_EOL;
```

افتح الطرفية في نفس مجلد الملف وشغّل:

```
php hello.php
```

الناتج المتوقع:

```
Hello Omar
```

`<?php` بيبدأ كود PHP. المتغير `$name` اسم بنوصل بيه للقيمة `Omar` وقت التشغيل. النقطة `.` بتوصل نصين. `echo` بتكتب النتيجة، و`PHP_EOL` بتضيف نهاية سطر مناسبة للبيئة. `strict_types` بيأثر على قواعد أنواع قيم معينة عند استدعاء الدوال؛ مش معناه إنك أعلنت نوع كل متغير.

نظام التشغيل بدأ عملية PHP، وPHP قرأ الملف وشغّل الكود وكتب الناتج على **Standard output، قناة الخرج العادية** اللي الطرفية بتعرضها. بعد انتهاء السكربت العملية بتنتهي. الملف على القرص بيفضل موجود؛ قيمة `$name` اللي كانت داخل العملية مش قاعدة بيانات محفوظة.

غيّر الاسم إلى `Mona` وشغّل تاني. النتيجة الجديدة تظهر في التشغيل الجديد لأن PHP قرأ المصدر المعدل. حفظ الملف مش هو نفسه تشغيله.

المكتبة والـFramework والامتداد

Library، مكتبة كود جاهز برنامجك بيستخدمه لمهمة محددة. **Framework، إطار عمل** بيقدم تنظيمًا وأجزاء لدورة تشغيل التطبيق، وبيستدعي كودك في أماكن محددة. Laravel مثال لإطار عمل PHP: يستقبل الطلب ويوجهه للكود المناسب ويوفر أدوات للتعامل مع البيانات والخدمات.

Extension، امتداد PHP بيضيف وظائف لمحرك PHP، زي PDO للتعامل مع قواعد البيانات. وجود ملف مكتبة في المشروع مش هو نفسه تفعيل امتداد في PHP. **Dependency، اعتماد** جزء خارجي محتاجه برنامجك علشان يشتغل. Composer بيدير حزم PHP؛ مش بديل عن تثبيت PHP نفسه أو تفعيل امتداداته.

أنت تقدر تتعلم المتغيرات والملفات والدوال بـPlain PHP الأول. بعد ما تفهمها، Laravel يوضح استخدام المفاهيم دي في تطبيق أكبر. مش محتاج تدخل Framework علشان تطبع سطرًا.

المثبت على الجهاز مش بالضرورة شغال

Installation، تثبيت بيجهز ملفات وأجزاء البرنامج وإعداداته. **Configuration، إعداد** بيحدد اختيارات التشغيل، زي مكان ملف البيانات أو عنوان قاعدة البيانات. **Execution، تنفيذ** هو تشغيل التعليمات فعلًا.

ممكن PHP متثبت، لكن الامتداد المطلوب مش متاح. وممكن التطبيق يبدأ لكن يفشل في فتح ملف بسبب الصلاحيات. وممكن قاعدة البيانات تكون متثبتة لكن خدمتها متوقفة. كل حالة لها دليل مختلف؛ عبارة «مش شغال» لوحدها مش تشخيص.

في الويب هتلاقي **Service، خدمة**: عملية أو مجموعة عمليات بتفضل مستعدة لشغل جديد. خادم الويب مثال. سكربت CLI قصير ممكن يبدأ وينتهي لكل تشغيل، بينما Worker قد يفضل شغال يعالج مهام متتالية. العمر ده مهم: المتغير اللي كان بيختفي مع نهاية تشغيل قصير ممكن يفضل في ذاكرة عملية طويلة العمر لو التطبيق احتفظ بيه.

أخطاء هتقابلك وإزاي تفرّق بينها

اللي ظهر	معناه الأقرب	أول حاجة تفحصها
أمر `php` مش معروف	مشكلة تثبيت أو مسار بحث	وجود PHP وPATH
`Could not open input file`	PHP اشتغل لكنه مش لاقي السكربت	اسم الملف والمجلد الحالي
`Parse error`	الكود مش مطابق لقواعد اللغة	موضع الخطأ والسطر اللي قبله
الناتج قديم	ممكن ملف غير اللي عدلته أو تشغيل مختلف	المسار والأمر وحفظ الملف
`Permission denied`	العملية ممنوعة من الوصول للمورد	هوية التشغيل وصلاحيات المورد

الجدول نقطة بداية، مش ضمان لسبب واحد. اقرأ الرسالة كاملة وحدد أي مرحلة وصلت لها قبل ما تغيّر الكود.

جرّب بنفسك

افتح طرفيتين وشغّل `hello.php` في كل واحدة بعد تغيير الاسم. هل التشغيل الأول هيحتفظ بالمتغير بعد انتهائه؟ وهل اسم الملف وحده يضمن إنك بتشغل نفس النسخة من مجلدين مختلفين؟

▼ راجع التفكير والإجابة

كل تشغيل عملية مستقلة، وقيم المتغيرات داخلها بتنتهي بانتهاء العملية. الملف نفسه محفوظ لكن القيم المؤقتة مش محفوظة فيه تلقائيًا. وملفان بنفس الاسم في مجلدين مختلفين ممكن يحتويان كودًا مختلفًا، لذلك افحص المسار الحالي والمسار اللي مررته لـPHP.

دلوقتي عندك خريطة واضحة: كود مصدر محفوظ، أداة تنفذه، عملية لها موارد، ونظام تشغيل بيدير الموارد دي. درس نظام التشغيل بيتابع مين بيسمح للعملية تعمل إيه، وإزاي برامج كتير تتشارك نفس الجهاز.

مراجع للمراجعة

- PHP CLI
- تعريفات الأنواع في PHP
- مقدمة Composer

نظام التشغيل: البنية والأنواع وطريقة العمل

Kernel وUser Mode وSystem Calls والعمليات والخيوط والجدولة والذاكرة والملفات والتعريفات وأنواع أنظمة التشغيل.

أنت فاتح محرر الكود والمتصفح، وعندك سكربت PHP بيقرأ ملفًا. الجهاز نفسه فيه معالج وذاكرة وتخزين، لكن البرامج دي مش كل واحدة ماسكة الجهاز لوحدها. مين يوزع وقت التنفيذ؟ مين يسمح للسكربت يقرأ الملف؟ وإزاي خطأ في برنامج ما يبوّظش ذاكرة كل البرامج التانية؟

نظام التشغيل، Operating System هو مجموعة البرامج اللي بتدير الموارد وبتقدم للتطبيقات طريقة منظمة لاستخدامها. في الدرس ده هنمشي وراء برنامج واحد: يبدأ، يطلب ملفًا، يستنى، يرجع ينفذ، وبعدها ينتهي. من الرحلة دي هنفهم النواة والعمليات والخيوط والجدولة والذاكرة والملفات والصلاحيات.

المطلوب قبل البداية: تعرف إن CPU بينفذ تعليمات، وRAM ذاكرة شغل مؤقتة، وإن ملف البرنامج مش هو العملية اللي شغالة. لو الفرق الأخير جديد، افتح الكود والبرنامج والـRuntime الأول.

شوف الفرق بين برنامج موجود وبرنامج شغال

على Windows افتح المفكرة، ثم Task Manager بـCtrl+Shift+Esc. في Details ابحث عن `notepad.exe`؛ قد يختلف الاسم في بعض الإصدارات. **Process — عملية** نسخة برنامج تعمل ولها موارد؛ **Thread — خيط** مسار تنفيذ داخلها.

من اختيار أعمدة Details أظهر PID وThreads إذا كانت متاحة. PID رقم يميز العملية حاليًا، وThreads عدد خيوطها. سجّل العدد، واكتب في المفكرة، ولاحظ استخدام CPU. صفر تقريبًا لا يعني أن البرنامج مقفول؛ قد ينتظر إدخالًا. عمود Status مثل Running في الواجهة لا يكشف حالة كل خيط لحظة بلحظة.

لا تنه أي عملية أثناء التدريب. **سؤال:** هل عدد خيوط أكبر يثبت سرعة أعلى؟ لا؛ نحتاج نوع العمل وقياس الوقت، والخيوط قد تكون منتظرة.

ليه بدأنا بالملاحظة دي؟ لأن نظام التشغيل بيدير **حالة التنفيذ**، مش قائمة ملفات البرامج فقط. عملية مستنية إدخالًا ممكن تكون موجودة في الذاكرة ومش محتاجة وقت CPU دلوقتي. ده هيفسر بعد شوية ليه برنامج يتأخر رغم إن المعالج مش مشغول.

النواة والنظام والتوزيعة

Kernel — النواة الجزء الذي يدير المعالج والذاكرة والأجهزة، وله صلاحيات أوسع من التطبيق العادي. النظام الكامل يشمل النواة وأدوات الإدارة والمكتبات والخدمات وواجهة الاستخدام. **Library — المكتبة** كود جاهز يستخدمه أكثر من برنامج، و**Service — الخدمة** برنامج يؤدي وظيفة في الخلفية مثل استقبال طلبات شبكة.

Linux اسم نواة. **Distribution — توزيعة** مثل Ubuntu تجمع النواة مع حزم برامج وأدوات وإعدادات وتحديثات. Windows وmacOS نظامان كاملان بأنوية ومكونات مختلفة. كلمة Linux تُستخدم أحيانًا في الكلام اليومي للدلالة على النظام كله، فحدد السياق.

إزاي التطبيق يطلب خدمة محمية؟

التطبيق يعمل عادة في **User mode — وضع مستخدم** بصلاحيات محدودة. النواة تنفذ الأجزاء الحساسة في **Kernel mode — وضع النواة**. لما التطبيق يحتاج يفتح ملفًا، يستخدم **System call — طلب خدمة من النظام**. النظام يفحص الطلب والصلاحيات ثم ينفذه أو يرفضه.

مثلًا قارئ النصوص يطلب فتح ملف؛ لا يُسمح له لمجرد أنه يعمل على الجهاز أن يقرأ ذاكرة برنامج الحسابات. الحماية تسمح بمشاركة آمنة حين يمنح النظام الإذن، وليست منعًا لكل تبادل.

برنامج واحد، وعملية، وخيوط تنفيذ

Process — عملية نسخة برنامج شغالة، لها ذاكرة وملفات مفتوحة وهوية وصلاحيات. **Address space — مساحة عناوين** هي العناوين التي تراها العملية وتستخدمها للوصول إلى ذاكرتها.

Thread — خيط تنفيذ مسار تعليمات داخل العملية. خيوط العملية تشترك عادة في مساحة العناوين والملفات، لكن لكل خيط حالة تنفيذ و**Stack — مكدس استدعاءات** يحتفظ بما يحتاجه للعودة من الدوال التي استدعاها. مشاركة الذاكرة تحتاج تنسيقًا لو أكثر من خيط يعدّل نفس البيانات.

Scheduler — المجدول جزء يختار أي خيط يأخذ وقت المعالج. **Context switch — تبديل سياق** حفظ حالة التنفيذ الحالية واسترجاع حالة أخرى. زيادة الخيوط قد تزيد التنافس والتبديل بدل التسريع، خصوصًا لو العمل محدود بمعالج أو مورد واحد.

الحالة	معناها
Ready — جاهز	يستطيع العمل وينتظر دوره على المعالج
Running — يعمل	ينفذ الآن
Blocked أو Sleeping — منتظر	ينتظر بيانات أو موردًا أو موعدًا
Terminated — انتهى	أكمل عمله أو أُنهي

بعض الأنظمة تصف هذه الحالات على مستوى الخيط وبعض العروض تجمعها للعملية. لو خيط ينتظر القرص أو الشبكة، يقدر المجدول يشغّل خيطًا جاهزًا آخر.

الذاكرة: العنوان مش مكانًا ثابتًا في الشريحة

Virtual memory — ذاكرة افتراضية تعطي العملية مساحة عناوين يديرها النظام. **Page — صفحة** جزء بحجم محدد من هذه المساحة. **Page tables — جداول الصفحات** تربط العناوين الافتراضية بالذاكرة الفعلية أو تصف حالتها.

Page fault — حدث يحتاج تدخل النظام يحصل عند وصول لا يمكن إكماله مباشرة بالترجمة الحالية. قد يعني تحميل صفحة لم تُحضَر بعد، وقد يعني وصولًا غير مسموح؛ ليس دائمًا انهيارًا. **Paging** إدارة ونقل صفحات، مش بالضرورة نقل العملية كلها.

Shared memory — ذاكرة مشتركة تسمح لعمليتين بالوصول إلى بيانات مشتركة مع تنسيق. **Memory-mapped file — ملف مربوط بالذاكرة** يربط جزءًا من الملف بعناوين يمكن للبرنامج الوصول إليها. **Swap** مساحة تخزين قد يضع فيها النظام بعض محتوى الذاكرة مؤقتًا. التفاصيل والحسابات في درس الذاكرة.

صلاحيات الصفحات تحدد القراءة والكتابة والتنفيذ. مثلًا صفحة تعليمات يمكن أن تكون قابلة للتنفيذ وغير قابلة للكتابة، وصفحة بيانات غير قابلة للتنفيذ. **OOM، Out Of Memory — نفاد الذاكرة المتاحة للطلب** يحتاج سياسة: قد يفشل طلب الحجز أو ينهي النظام عملية، حسب النظام والإعدادات.

الملف والجهاز والإشعار

Filesystem — نظام ملفات ينظم الأسماء والمجلدات ويربطها بمساحات تخزين تسمى **Blocks — كتل**. يحتفظ بـ**Metadata — بيانات وصفية** مثل الحجم والمالك والصلاحيات. **Page cache — نسخة صفحات محفوظة في الذاكرة** تقلل الرجوع للقرص، لكن انتهاء الكتابة في الذاكرة لا يساوي دائمًا وصولها للتخزين الدائم.

I/O، Input/Output — إدخال وإخراج يشمل قراءة ملف أو استقبال شبكة أو عرض نتيجة. **Driver — تعريف جهاز** برنامج يسمح للنظام بالتعامل مع **Controller — متحكم مادي** يدير الجهاز.

Interrupt — مقاطعة إشعار بحدث يحتاج معالجة. **Polling — فحص متكرر** يسأل النظام الجهاز عن الحالة بدل انتظار إشعار. الاختيار يعتمد على الحمل والتصميم.

File descriptor أوHandle — معرّف مورد مفتوح قيمة يستخدمها البرنامج للرجوع إلى ملف أو مورد فتحه. أغلق المورد عند انتهاء الحاجة، أو استخدم آلية موثقة في **Runtime — بيئة تنفيذ اللغة** تدير إغلاقه؛ لا تفترض أن كل مورد سيُغلق في الوقت المناسب من نفسه.

المستخدم والصلاحيات

User — مستخدم هوية، و**Group — مجموعة** تجمع هويات لتسهيل منح الإذن. **Permissions — صلاحيات** تحدد العمليات المسموحة، مثل قراءة ملف وكتابته. مبدأ **Least privilege — أقل صلاحية لازمة** معناه إعطاء البرنامج ما يحتاجه فقط.

حساب الإدارة اسمه Administrator في Windows، وRoot في أنظمة مثل Linux. خدمة ويب لا تحتاج عادة العمل بحساب إدارة كامل. **Service account — حساب خدمة** مخصص لبرنامج، وتقتصر صلاحياته على ملفاته ومنافذه المطلوبة. الصلاحيات طبقة حماية؛ تشفير الأسرار وعزل الشبكات لهما دور آخر أيضًا.

أنواع الاستخدام

- **Desktop — نظام مكتبي:** نوافذ وتطبيقات وتفاعل مباشر.
- **Server — نظام لخدمات الخادم:** خدمات مستمرة وإدارة عن بعد وقدرة على تحمل طلبات كثيرة.
- **Mobile — نظام هاتف:** يهتم بالطاقة والحساسات و**Sandbox — عزل التطبيقات** بحيث تكون لكل منها حدود وصول.
- **Embedded — نظام مدمج:** يشغّل وظيفة محددة داخل جهاز، وقد تكون موارده قليلة.
- **Real-time — نظام بزمن استجابة محدد:** يهتم بإكمال العمل قبل موعد معروف. في **Hard real-time** تجاوز الموعد يُعد فشلًا حتى لو ظهرت النتيجة لاحقًا.
- **Hypervisor — مدير أجهزة افتراضية:** يشغّل **Virtual machines**، بيئات تظهر كأجهزة مستقلة لأنظمتها. قد يعمل فوق العتاد مباشرة أو فوق نظام آخر؛ دي طبقة افتراضية وليست مرادفًا لكل نظام خادم.

الأنظمة تتشابه في إدارة الذاكرة والملفات والتنفيذ، وتختلف في **API، Application Programming Interface — واجهات الخدمات التي يطلبها البرنامج**، والأدوات وطريقة تنظيم الملفات والترخيص. اختار بناءً على متطلبات البرنامج والأجهزة.

امشِ وراء تشغيل سكربت وقراءة ملف

واجهة النوافذ اسمها **GUI، Graphical User Interface**. والطرفية تشغّل **Shell**، برنامج يفسر الأوامر. سواء نقرت أيقونة أو كتبت أمرًا، المسار العام هو:

1. النظام ينشئ عملية.
2. **Loader — محمّل البرنامج** يجهز الملف التنفيذي والمكتبات في مساحة العناوين.
3. يجهز النظام الذاكرة ومكدس الاستدعاءات ومعرّفات الموارد.
4. المجدول يبدأ تشغيل خيط، والبرنامج يطلب خدمات الملفات والشبكة حسب حاجته.

خلينا نجرب الرحلة بكود PHP. محتاج PHP CLI ومجلد تدريب تقدر تكتب فيه. احفظ الملف التالي باسم `os-file-lab.php`:

```
<?php
declare(strict_types=1);

$path = __DIR__ . '/os-lab-note.txt';
$data = "Hello from PHP\n";
$written = file_put_contents($path, $data);
if ($written !== strlen($data)) {
    throw new RuntimeException('Could not write the complete file');
}
$loaded = file_get_contents($path);
if ($loaded === false) {
    throw new RuntimeException('Could not read the file');
}
echo $loaded;
```

شغّل:

```
php os-file-lab.php
```

المتوقع يظهر `Hello from PHP` ويتكوّن ملف `os-lab-note.txt` بجوار السكربت. إعادة التشغيل تستبدل محتوى ملف التدريب بنفس النص؛ شغّله في مجلد المعمل فقط.

`__DIR__` بيحدد مجلد السكربت بدل الاعتماد على مكان فتح الطرفية. المتغير `$path` يحمل اسم الملف، و`file_put_contents` تطلب كتابة البيانات وترجع عدد البايتات أو `false` عند الفشل. بنقارن العدد علشان ما نعتبرش كتابة ناقصة نجاحًا.
`file_get_contents` تقرأ المحتوى أو ترجع `false`، و`echo` تكتب النتيجة للطرفية.

تحت الاستدعاءات دي، Runtime PHP بيستخدم خدمات نظام التشغيل. النظام بيحل المسار ويفحص صلاحيات هوية العملية، ويستخدم نظام الملفات والذاكرة المؤقتة والتعريفات عند الحاجة. مش لازم كل استدعاء PHP يقابل System Call واحدة بالضبط؛ المكتبات والـBuffers ممكن تجمع العمل أو تنفذه على مراحل.

لو البيانات جاهزة في Cache ممكن القراءة ترجع بسرعة. لو محتاجة وصولًا أبطأ، خيط التنفيذ ممكن ينتظر والنظام يشغّل خيطًا جاهزًا آخر. ده الفرق بين **الانتظار** و**استهلاك المعالج**. نجاح الكتابة هنا معناه إن واجهة الكتابة قبلت البيانات المطلوبة؛ مش تجربة تثبت النجاة من انقطاع الكهرباء. ثبات البيانات له ضمانات إضافية في الكتابة والـBuffers.

لما السكربت يفشل، تحدد الطبقة إزاي؟

الموقف	التفسير اللي نختبره	ليه تشغيل كل حاجة كـAdministrator مش حل؟
ملف مش موجود	المسار أو الاسم أو مجلد العمل	الصلاحيات مش هتخلق الملف الصحيح
فتح الملف مرفوض	هوية العملية وصلاحيات المورد	ممكن يخفي السبب ويوسع ضرر التطبيق
العملية بتستنى وCPU منخفض	قرص أو شبكة أو قفل أو مؤقت	الانتظار مش محتاج صلاحيات أعلى تلقائيًا
استخدام ذاكرة بيزيد	بيانات محتفظ بيها أو تخصيص زائد	هوية الإدارة مش بتصلح عمر البيانات

المتصفح مش هوية تنفيذ PHP على الخادم. خدمة PHP ممكن تشتغل بحساب مختلف عن حسابك اللي فاتح الطرفية؛ علشان كده نفس المسار ممكن يشتغل من CLI ويفشل من خدمة الويب. افحص هوية العملية ومسارها وصلاحياتها في الحالتين.

تواصل العمليات والانتظار المتبادل

IPC، Inter-Process Communication يعني تبادل بيانات وإشارات بين العمليات. **Pipe — قناة بيانات** تمرر خرجًا إلى عملية أخرى. **Socket — واجهة اتصال** قد تكون محلية أو عبر شبكة. الذاكرة المشتركة وسيلة أخرى، و**Signal — إشارة** تبلغ العملية بحدث. ده معنى مختلف لاختصار IPC عن «تعليمات لكل دورة» في قياس المعالج.

Deadlock — انتظار متبادل لا يتقدم: العملية A تمسك المورد1 وتنتظر2، وB تمسك2 وتنتظر1. الشروط المعتادة: مورد لا يشاركه اثنان معًا، والاحتفاظ بمورد أثناء انتظار آخر، وعدم سحب المورد بالقوة، ودائرة انتظار. فرض ترتيب واحد لحجز الموارد يكسر دائرة الانتظار في هذا النموذج.

تدريب محلول: برنامج يقرأ ملفًا. لو النسخة المطلوبة موجودة في ذاكرة التخزين المؤقت قد يرجع الرد سريعًا. لو يحتاج القرص، قد ينتظر خيط التنفيذ ويشغّل النظام خيطًا آخر. ده انتظار طبيعي، مش دليل Deadlock؛ ابحث عن دائرة اعتماد لا تتقدم قبل تشخيص التعطل المتبادل.

تأكد من فهمك

01 **هل Linux وUbuntu الشيء نفسه؟**

الإجابة: Linux Kernel، أما Ubuntu فتوزيعة تجمعه مع أدوات وحزم وخدمات وسياسات.

02 **لماذا يحتاج التطبيق إلى System Call؟**

الإجابة: لأنه لا يملك في User Mode صلاحية إدارة الجهاز والذاكرة المحمية مباشرة؛ Kernel يتحقق وينفذ العملية.

03 ما الذي تشاركه Threads داخل Process؟

الإجابة: Address Space وموارد مثل الملفات غالبًا، مع Stack وحالة تنفيذ مستقلة لكل Thread.

04 ما الفرق بين Server OS وReal-time OS؟

الإجابة: Server يركز على الخدمات والسعة والاعتمادية، بينما Real-time يركز على ضمان حدود زمن الاستجابة وقابليتها للتنبؤ.

من خطوات الحل إلى برنامج يعمل

من خطوات الحل إلى برنامج يعمل

بعد فهم تمثيل القيم، هنتتبع تعليمات البرنامج: مين يحوّلها ومين يشغّلها، وليه البرنامج قد يحتاج مكتبة خارجية.

من الخوارزمية إلى البرنامج

1. حدّد المدخلات والمخرجات والقيود.
2. اكتب خطوات مستقلة عن اللغة باستخدام Pseudocode (وصف شبه برمجي للخطوات؛ للتفكير وليس صياغة لغة جاهزة للتشغيل) أو Flowchart (مخطط انسيابي يعرض الخطوات والقرارات بأسهم).
3. اختبر الخطوات يدويًا بحالة عادية وحدّية وغير صالحة.
4. ترجمها إلى كود، ثم قارن السلوك بالنتيجة المتوقعة.
5. قِس الوضوح والصحة أولًا، ثم حسّن الأداء عندما يثبت القياس وجود مشكلة.

Compiler: مترجم يحول كودًا إلى تعليمات آلة أو تمثيل آخر قابل للتنفيذ بأداة مناسبة. Interpreter: مفسر ينفذ الكود أو تمثيله أثناء التشغيل.

من المشكلة إلى تعليمات ينفذها الجهاز

Compiler وInterpreter وRuntime وJIT

يحول Compiler المصدر قبل التشغيل إلى كود آلة أو تمثيل وسيط. ينفذ Interpreter التعليمات أو التمثيل أثناء التشغيل. يوفر Runtime (بيئة تنفيذ اللغة والخدمات التي تحتاجها؛ غير معنى وقت التشغيل الزمني) خدمات تحتاجها اللغة مثل إدارة الذاكرة والاستثناءات والمكتبات. يجمع JIT (Just-In-Time compilation؛ ترجمة أجزاء من البرنامج أثناء تشغيله) أجزاء أثناء التشغيل اعتمادًا على الاستخدام الفعلي. كثير من المنصات تمزج أكثر من نموذج، لذلك لا تختصر اللغة في وصف «مترجمة» أو «مفسرة» فقط.

Library وDependency ونسخة التشغيل

المكتبة كود قابل لإعادة الاستخدام، والـDependency شيء يعتمد عليه المشروع. قد تكون المكتبة Static فتدخل في الملف التنفيذي، أو Dynamic تُحمّل وقت التشغيل. اختلاف النسخ قد يغير API (Application Programming Interface؛ اتفاق يسمح لبرنامج بطلب بيانات أو عملية من مكوّن آخر) أو السلوك، ولذلك تسجل المشاريع الإصدارات وتستخدم Lock Files أو أدوات بناء قابلة لإعادة الإنتاج.

كيف ينفّذ المعالج التعليمات؟

كيف ينفّذ المعالج التعليمات؟

تعليمتان ثم مكان البيانات

افترض تعليمتين مبسطتين: ضع 5 في مكان عمل داخل المعالج، ثم أضف 3 إلى نفس القيمة. المكان الصغير داخل المعالج اسمه **Register — سجل**. المعالج يجلب التعليمة، يفهم العملية المطلوبة، ثم ينفذها؛ الناتج بعد الأولى 5 وبعد الثانية 8. دي صورة تعليمية، مش صياغة لغة آلة بعينها.

```
CPU registers ⇔ L1 cache ⇔ L2 cache ⇔ L3 cache ⇔ RAM
```

Cache — ذاكرة تحتفظ بنسخة قريبة تقلل انتظار البيانات المتكررة. L1 وL2 وL3 مستويات؛ هذا ترتيب شائع مبسط، وقد يختلف التنظيم والمشاركة بين الأنوية. السهم يعني طلب بيانات أو نقلها، وليس ضرورة مرور كل قراءة بكل مستوى. بعد ثبات الصورة نضيف تفاصيل التوازي والتنبؤ.

داخل CPU

ينفذ المعالج دورة Fetch–Decode–Execute. Program Counter (سجل يحدد موضع التعليمة التالية حسب تنفيذ المعالج) يحدد التعليمة التالية، Control Unit (وحدة تنظم فك التعليمات وتنسيق تنفيذها) تفكها وتنسق التنفيذ، ALU (Arithmetic Logic Unit؛ وحدة الحساب والمنطق داخل المعالج) تنفذ الحساب والمنطق، Registers (سجلات: أماكن صغيرة وسريعة جدًا لحمل قيم قريبة من التنفيذ) تحمل القيم الأقرب للتنفيذ، ووحدات أخرى تعالج Floating Point وVectors وBranches وMemory Access.

المعالجات الحديثة تنفذ عدة تعليمات في مراحل متداخلة وتستخدم Branch Prediction (توقع الفرع التالي لتجهيز تعليماته قبل معرفة نتيجة الشرط) وOut-of-order Execution (تنفيذ تعليمات جاهزة بترتيب داخلي مختلف مع الحفاظ على النتائج المطلوبة). لذلك "تعليمة واحدة كل Clock (إيقاع دورات المعالج؛ لا يساوي عدد التعليمات)" تبسيط، والأداء لا يقاس بالـGHz (جيجاهرتز؛ مليار دورة في الثانية، وليس عدد تعليمات منجزة) وحده.

Cores وThreads وClock وIPC

Core (نواة معالجة تملك موارد تنفيذ فعلية) يستطيع تنفيذ Stream تعليمات مستقلًا. Hardware Thread (سياق تنفيذ عتادي يشارك موارد النواة وقد لا يملك نواة كاملة مستقلة) يسمح بمشاركة بعض موارد Core لتحسين الاستفادة، لكنه ليس Core كاملًا. Clock يحدد عدد الدورات، وIPC (Instructions Per Cycle؛ متوسط عدد التعليمات المنجزة في دورة للمعالج) يصف مقدار العمل المنجز في الدورة. الأداء يعتمد على الاثنين إضافة إلى الذاكرة والكود والحرارة وحدود الطاقة.

زيادة الأنوية تفيد فقط عندما يمكن تقسيم العمل. جزء Serial أوLock (قفل ينظم الوصول المشترك حتى لا تتداخل عمليات متعارضة مشترك قد يحد التسارع وفق Amdahl's Law (قانون يبين كيف يحد الجزء الذي لا يمكن تقسيمه من تسريع البرنامج كله).

Registers وL1/L2/L3 Cache

Registers أصغر وأسرع تخزين داخل Core. L1 صغيرة وسريعة جدًا، L2 أكبر وأبطأ، وL3 أكبر وغالبًا مشتركة. Cache (نسخة محفوظة لتقليل تكرار القراءة أو الحساب) تنقل البيانات في Cache Lines (كتل تنقلها ذاكرة المعالج المؤقتة بدل بايت منفرد كل مرة)؛ الوصول المتجاور يستفيد من Locality (استخدام بيانات قريبة مكانيًا أو إعادة استخدامها قريبًا زمنيًا)، بينما القفز العشوائي يسبب Cache Misses (مرات عدم وجود البيانات المطلوبة في المستوى المفحوص من الذاكرة المؤقتة) وانتظار RAM (Random Access Memory؛ ذاكرة العمل التي تحمل بيانات وتعليمات البرامج النشطة).

Cache ليست RAM إضافية يديرها البرنامج عادة، ولا تضمن كتابة البيانات فورًا إلى التخزين. Cache Coherence (قواعد اتساق نسخ الذاكرة المؤقتة بين الأنوية) تحافظ على رؤية معقولة بين Cores لكنها لا تستبدل Locks وAtomic Operations (عمليات تظهر كوحدة غير قابلة للتجزئة بالنسبة للتزامن المحدد) في البرامج المتزامنة.

فك أسماء التنفيذ وجرّب القياس

Fetch–Decode–Execute يعني جلب التعليمة، وفهم نوع العملية والبيانات المطلوبة، وتنفيذها. **Floating point** تمثيل يسمح بأعداد كسرية بدقة محدودة. **Vector** مجموعة قيم تنفذ عليها عملية معًا، و**Branch** انتقال لمسار تعليمات حسب شرط. **Serial** جزء تنفذ خطواته بالتتابع.

SIMD (Single Instruction Multiple Data؛ تطبيق تعليمة على عدة عناصر بيانات) تعليمة واحدة تعمل على عدة عناصر. ده مختلف عن تشغيل أنوية متعددة لتدفقات تعليمات مستقلة. في GPU، كلمة **Kernel** تعني برنامج حساب نرسله لوحداته، مش نواة نظام التشغيل. **Branch divergence** اختلاف مسار عناصر تعمل معًا مما قد يقلل الاستفادة من التوازي. **Unified memory** طريقة تعرض ذاكرة بصورة موحدة للبرنامج، لكنها لا تلغي كلفة انتقال البيانات دائمًا.

لو 50% من وقت عمل لا يمكن تقسيمه، تسريع النصف الآخر بلا حد لن يجعل البرنامج أسرع من الضعف: الزمن يبدأ 100، ويبقى على الأقل 50. ده مثال **قانون أمدال** الذي يبين حد التسريع.

تدريب قياس محلول: شغّل مهمة تمهيدًا Warm-up (تشغيل تمهيدي قبل القياس لتقليل أثر البداية)، ثم سجل خمس مدد تحت نفس الشروط: 9،10،10،11،30 مللي ثانية. الوسيط Median (الوسيط: القيمة الوسطى بعد ترتيب القياسات) هو 10 بعد الترتيب؛ المتوسط 14. اختلافهما ينبهك لتذبذب أو قيمة بعيدة. لا تحذف قياس30 بلا تفسير، وسجّل حجم البيانات وكلفة نقلها أيضًا.

تأكد من فهمك

01 **لماذا معالج 4GHz ليس دائمًا أسرع من 3.5GHz؟**

الإجابة: تختلف IPC والمعمارية والـCache وعدد الأنوية وحدود الطاقة والحمل الفعلي.

02 **ما الفرق بين Core وHardware Thread؟**

الإجابة: Core يملك وحدات تنفيذ فعلية، أما Hardware Thread فيشارك جزءًا من موارد Core لتحسين الاستفادة وليس Core مستقلًا كاملًا.

03 **متى يصبح GPU اختيارًا سيئًا؟**

الإجابة: عندما يكون العمل صغيرًا أوSerial أوكثير التفرع أوتكلفة نقل البيانات أكبر من الحساب.

04 **كيف تؤثر Locality في الأداء؟**

الإجابة: الوصول المتجاور يعيد استخدام Cache Lines، بينما الوصول العشوائي يزيد Cache Misses والانتظار على RAM.

الخطوة التالية

كمّل في إمتى يفيد تشغيل أعمال كثيرة معًا؟ بعد تنفيذ التجربة هنا.

تمثيل الأعداد والحروف في الذاكرة

تمثيل الأعداد والحروف في الذاكرة

الفكرة العامة

كل البيانات والتعليمات تنتهي داخل الجهاز في صورة بِتّات. لغات البرمجة تمنحنا مستوى أوضح لكتابة الخوارزمية ثم يحولها مترجم أو مفسر إلى خطوات قابلة للتنفيذ.

المفاهيم الأساسية

- البت Bit يأخذ 0 أو 1، وثمانية بِتّات تكوّن Byte.
- العدد الثنائي تمثيل موضعي أساسه 2؛ قيمة الخانة تتحدد بقوة 2.
- النص مش حروف مرسومة في الذاكرة؛ بنستخدم ترميزًا متفقًا عليه. في النصوص الحديثة معيار Unicode يحدد أرقام الرموز، وترميز زي UTF-8 (Unicode Transformation Format) بوحدات 8 بت؛ ترميز أرقام رموز Unicode كبايتات) يحدد البايتات. توجد ترميزات أخرى، فلا نفترض أن كل ملف نص UTF-8.
- الخوارزمية خطوات محددة ومنتهية لحل مشكلة، وليست مرتبطة بلغة بعينها.
- اختيار اللغة يعتمد على المجال والمنظومة والأداء والفريق، لا على وجود "أفضل لغة" مطلقًا.

مثال تطبيقي

الخوارزمية لتحديد نجاح طالب: اقرأ الدرجة، تحقق أنها بين 0 و100، ثم اطبع "ناجح" إذا كانت 50 فأكثر وإلا "راسب". التحقق يسبق القرار حتى لا نعالج قيمة غير صالحة.

تصحيح مفاهيم وأخطاء شائعة

- الكود ليس الخوارزمية نفسها؛ هو تنفيذ لها بلغة معينة.
- لا يكفي أن تعمل الخوارزمية مع مثال واحد؛ اختبر الحدود والمدخلات غير الصحيحة.

من الثنائي إلى السداسي عشري

يجعل النظام السداسي عشري قراءة البتات أقصر، لأن كل رقم Hex يمثل أربع بتات. مثلًا `1111 1010₂` يساوي `FA₁₆`. يظهر Hex في عناوين الذاكرة، والألوان مثل `#22C55E`، وعرض محتوى الملفات الثنائية، وBit Masks.

Unicode وUTF-8 عمليًا

Unicode يمنح كل محرف **Code Point** مثل `U+0639` للحرف «ع»، بينما UTF-8 يحدد كيف تتحول هذه القيمة إلى بايتات. الحروف الإنجليزية الأساسية تستخدم بايتًا واحدًا، وحروف العربية تستخدم غالبًا بايتين، وقد تستخدم رموز أخرى حتى أربعة بايتات.

لا تفترض أن عدد المحارف يساوي عدد البايتات، ولا أن فهرسة البايت تصلح لقص النص. يجب أن تعرف دوال اللغة والمكتبة التي تتعامل مع Unicode عند التحقق من طول النص أو تقسيمه.

Signed وUnsigned وTwo’s Complement

يحدد النوع كيف نفسر نمط البتات. في ثمانية بتات يمثل unsigned القيم من `0` إلى `255`، بينما يمثل signed الشائع بطريقة Two’s Complement القيم من `-128` إلى `127`. للحصول على سالب قيمة موجبة، تعكس البتات ثم تضيف واحدًا. يلتف unsigned حسابيًا داخل مداه، لكن الالتفاف قد يكون خطأ منطقيًا.

Floating Point ولماذا `0.1 + 0.2` ليست دقيقة دائمًا

تمثل الأعداد العائمة عادة بإشارة وأس وجزء دال وفق IEEE 754 (معيار شائع يحدد تمثيل الأعداد العائمة وعملياتها، أي أعداد تسمح بكسور ضمن دقة محدودة). كثير من الكسور العشرية لا يملك تمثيلًا ثنائيًا منتهيًا، فتُخزن قيمة تقريبية. لذلك لا تستخدم مساواة مباشرة دائمًا لنتائج العمليات، ولا تستخدم Floating Point للأموال الحرجة من غير سياسة تقريب واضحة.

Endianness

عند تخزين قيمة متعددة البايتات تحدد Endianness ترتيب البايتات. تضع Little-endian البايت الأقل أهمية أولًا في الذاكرة، وتضع Big-endian الأعلى أولًا. لا يظهر الفرق عند قراءة القيمة بالنوع نفسه، لكنه مهم عند الشبكات والملفات الثنائية وتحويل البيانات بين الأنظمة.

اختبر التمثيل بأرقام فعلية

راجع أولًا تحويل 5 إلى 101 وتمثيل النص والصورة والصوت. **Hex** سداسي عشري يستخدم0–9 ثم A–F للقيم10–15. `FA` معناه `15×16+10=250`، وبالثنائي **Bit mask** . `11111010` — **قناع بتات** نمط نستخدمه لاختيار بتات محددة، مثل قناع ينظر لآخر بت لتحديد الزوجي والفردي.

في تمثيل **Two’s complement** — **المتمم لاثنين** ذي8 بتات: 5 هو00000101؛ نعكس كل بت فنحصل على11111010، ثم نضيف1 فنحصل على11111011، أي -5 بهذا التفسير. نفس النمط يساوي251 إذا قرأناه unsigned. نمط10000000 يساوي-128، وأكبر موجب01111111 يساوي127. لا نعمّم التفاف الحساب غير السالب على signed في كل لغة؛ مثلًا تجاوز signed في C++ ليس التفافًا مضمونًا.

Floating point — **عدد عائم** يستخدم إشارة وأسًا وجزءًا دالًا؛ الأس يحدد مقياس القيمة والجزء الدال يحدد دقتها. مثلما1/3 لا تنتهي كتابته بالعشري،0.1 لا تنتهي كتابته بالثنائي. في `0.1+0.2` ،JavaScript يُعرض عادة `0.30000000000000004`. اختَر سياسة دقة وتقريب حسب المسألة؛ لا تستخدم هامشًا عشوائيًا لإخفاء خطأ. تمثيل الأموال بوحدات صحيحة صغيرة مناسب في حالات، مع قواعد تحويل وتقريب واضحة.

Endianness — ترتيب البايتات لا يقلب البتات داخل كل بايت. للقيمة ذات البايتين `0x1234`، حيث0x يدل على سداسي عشري: Big-endian يضع12 ثم34 من عنوان الذاكرة الأقل، وLittle-endian يضع34 ثم12. **تدريب وحل:** البايتان35 و00 بالسداسي يساويان53 كعدد16 بتlittle-endian، لكن الأول وحده يُقرأ الحرف5 في UTF-8؛ لازم تحدد النوع والترتيب والترميز.

Library — مكتبة كود نعيد استخدامه، و**Dependency — اعتماد** شيء يحتاجه المشروع. **Static linking** يضم كود المكتبة في ناتج البناء، و**Dynamic linking** يعتمد على تحميلها أثناء تشغيل البرنامج. **Lock file** يسجل نسخ الاعتمادات المحددة للمساعدة في تكرار البناء.

تأكد من فهمك

01 **مثّل العدد 45 بالثنائي ثم اشرح لماذا لا تتغير قيمته.**

> **الإجابة المشروحة:** 45 = 32+8+4+1، لذلك تمثيله 101101. القيمة واحدة؛ الذي تغير هو الأساس والرموز المستخدمة لكتابتها.

02 **لماذا لا يمكن اعتبار UTF-8 "لغة برمجة" رغم أنه يحول النص إلى بايتات؟**

> **الإجابة المشروحة:** UTF-8 ترميز يحدد تمثيل المحارف، ولا يصف تعليمات أو تحكمًا أو خوارزميات قابلة للتنفيذ مثل لغة البرمجة.

03 **هل الخوارزمية التي تنجح مع مثال واحد تعد صحيحة؟ وضح باختبار مضاد.**

> **الإجابة المشروحة:** لا. خوارزمية القسمة قد تنجح مع 10÷2 وتفشل عند المقام صفر؛ يجب تغطية الحدود والمدخلات غير الصالحة.

04 **متى تختار مترجمًا Compiler ومتى يهمك Interpreter أو JIT؟**

> **الإجابة المشروحة:** الاختيار تابع للمنصة والأداء ودورة التطوير. الترجمة المسبقة تناسب الأداء والتوزيع الثنائي، والتفسير/JIT قد يمنح مرونة وتشغيلًا ديناميكيًا.

الخطوة التالية

كمّل في من خطوات الحل إلى برنامج يعمل بعد تنفيذ التجربة هنا.

الصورة والصوت ووحدات الحجم

الصورة والصوت ووحدات الحجم

بعد تمثيل العدد والحرف، هنستخدم نفس البايتات لتمثيل صورة وصوت. اقرأ جزءًا وطبّق حسابه قبل الجزء التالي؛ المطلوب هنا ضرب وقسمة بسيطة.

والصورة والصوت؟

الصورة الرقمية ممكن نمثلها بشبكة **بكسلات — Pixels**، نقاط صغيرة لكل منها لون. مثال لون **RGB، Red Green Blue** يصف كمية الأحمر والأخضر والأزرق. في تمثيل شائع كل قناة لون بايت: `255,0,0` أحمر، و `0,0,0` أسود. صورة 2×2 فيها 4 بكسلات؛ بثلاثة بايتات لكل بكسل تحتاج 12 بايتًا لبيانات اللون الخام. حجم ملف الصورة الفعلي قد يختلف بسبب معلومات الملف والضغط أو تفاصيل تمثيل أخرى.

طبّق قبل الصوت: صورة3×2، وثلاثة بايتات لكل بكسل:6×3=18بايتًا خامًا.

عينات الصوت

الصوت ممكن نمثله **بعينات — Samples**: قياسات متتابعة لقيمة الإشارة في لحظات محددة. **معدل أخذ العينات — Sample rate** عدد القياسات في الثانية. في مثال قناة صوت واحدة، 8000 عينة في الثانية و16 بتًا لكل عينة: الحجم الخام لثانية هو `8000×16÷8 =` `16000` بايت. دي أرقام توضيحية، مش قاعدة لكل تسجيل.

الضغط — Compression يقلل مساحة التمثيل. الضغط بدون فقد يسمح باستعادة البيانات الأصلية نفسها، والضغط بفقد يتنازل عن بعض التفاصيل لتقليل الحجم. ملف نص وبرنامج يحتاجان الحفاظ على البايتات الأصلية؛ بعض صور وصوت الاستخدام اليومي تقبل فقد تفاصيل حسب المطلوب.

وحدات الحجم والسرعة

`b` الصغيرة تعني bit، و `B` الكبيرة تعني byte. `kB = 1000 B` و `MB = 1,000,000 B` . فيه وحدات ثنائية بأسماء مختلفة: `KiB = 1024` `B` و `MiB = 1,048,576 B` . بعض الواجهات تخلط التسميات؛ راجع المقصود.

سرعة `8 Mb/s` تعني ثمانية ملايين بت في الثانية، أي `1 MB/s` نظريًا قبل تكلفة نقل الرسائل وأي قيود أخرى. حجم الملف مقدار بيانات، أما السرعة فمقدار بيانات في زمن؛ الاتنين مش نفس الوحدة.

افصل الحسابات

1. صوت 4000 عينة، و16بتًا للعينة، لقناة واحدة وثانية واحدة: `4000×16÷8=8000` بايت خام. ضاعف المدة فقط؛ الحجم16000.
2. اختر ضغطًا لملف برنامج: لازم يستعيد نفس البايتات، إذن ضغط بدون فقد. صورة للعرض قد تقبل ضغطًا بفقد لو التفاصيل المتبقية مناسبة.

3. سرعة16Mb/s تعادل نظريًا2MB/s قبل أي تكلفة إضافية. ملف10MB يحتاج على الأقل5ثوانٍ عند هذا المعدل النظري الثابت، وقد يحتاج أكثر في الواقع.

اكتب الوحدات بجوار كل قيمة؛ خلط بت ببايت يغيّر الحساب ثماني مرات.

العناوين والذاكرة الافتراضية

العناوين والذاكرة الافتراضية

عنوانان لنفس البيانات

العنوان الافتراضي رقم يستخدمه البرنامج داخل مساحة عناوينه، و**العنوان الفعلي** مكان يقابله في الذاكرة. النظام والمعالج يترجمان بينهما. افترض أن الذاكرة مقسمة لصفحات4096بايت: العنوان5000 يعني الصفحة1 والباقي904 من بدايتها. لو موجودة في الإطار7 فعليًا، نصل إلى `7×4096+904=29576` .

```
Virtual: page 1 + offset 904 → page table: 1 maps to 7 → Physical: frame 7 + offset 904
```

Page table — جدول صفحات يحفظ الربط، و**offset — إزاحة** مكان البايت داخل الصفحة. ده حجم صفحة افتراضي للتجربة، مش قاعدة لكل جهاز. **جرّب:** العنوان4096 إزاحته0؛ والعنوان4095 في الصفحة0 وإزاحته4095.

ما RAM؟

RAM (Random Access Memory؛ ذاكرة العمل التي تحمل بيانات وتعليمات البرامج النشطة) تخزين سريع متطاير يحمل الكود والبيانات النشطة. CPU (Central Processing Unit؛ المعالج الرئيسي الذي ينفذ التعليمات) لا يتعامل معها كقائمة متغيرات؛ يقرأ ويكتب عناوين، غالبًا عبر Cache (نسخة محفوظة لتقليل تكرار القراءة أو الحساب) Lines. Memory Controller يدير الاتصال بالشرائح، وتؤثر القنوات والتردد والـLatency في Bandwidth والزمن.

Physical وVirtual Memory

كل Process (عملية: نسخة برنامج تعمل بذاكرة وحالة وموارد) يرى Address Space (مجموعة العناوين التي يمكن للعملية استخدامها) افتراضية خاصة. تقسم الذاكرة إلى Pages (صفحات ذاكرة: وحدات ثابتة الحجم يديرها النظام)، وتترجم MMU (Memory Management Unit؛ وحدة ترجمة عناوين الذاكرة وفحص الوصول) العنوان الافتراضي إلى Physical Frame (وحدة ذاكرة فعلية تقابل حجم صفحة) باستخدام Page Tables (جداول تربط الصفحات الافتراضية بمواقعها أو حالاتها) يديرها Kernel. هذا يوفر Isolation ويسمح بتحميل الصفحات عند الحاجة ومشاركة Pages آمنة بين العمليات.

Page Fault (حدث وصول لصفحة تحتاج تدخل النظام؛ قد يكون تحميلًا عاديًا أو وصولًا غير مسموح) لا يعني Crash دائمًا؛ قد يعني أن Page صحيحة لم تُحمّل بعد. إذا اضطر النظام إلى نقل Pages باردّة إلى Swap (مساحة تخزين قد يستخدمها النظام لإبعاد بعض بيانات الذاكرة عن RAM مؤقتًا) يصبح الوصول أبطأ كثيرًا. Thrashing (انشغال النظام بتبديل الصفحات أكثر من إنجاز العمل) يحدث عندما يقضي النظام وقته في تبديل الصفحات بدل العمل.

تتبع عنوانًا بدل تخيل شرائح منفصلة

افترض صفحات حجمها 4096 بايتًا. العنوان الافتراضي 5000 يقع في الصفحة رقم1 إذا بدأنا العد من صفر، وإزاحته داخلها `5000-4096=904` . لو جدول الصفحات يربطها بالإطار الفعلي رقم7، فالعنوان الفعلي التوضيحي **Page** . `7×4096+904=29576` وحدة في العناوين الافتراضية، و**Frame** وحدة مقابلة في الذاكرة الفعلية؛ المثال لتوضيح الترجمة، وليس قاعدة أن كل نظام يستخدم هذا الحجم.

TLB (Translation Lookaside Buffer)؛ مخزن سريع لترجمات عناوين الذاكرة) مخزن سريع لترجمات العناوين. عدم وجود الترجمة فيه لا يعني بالضرورة Page fault؛ قد يجدها المعالج في جدول صفحات صالح. **Demand paging** تجهيز الصفحة عند الحاجة، و**Overcommit** سياسة قد تسمح بحجوزات افتراضية أكبر من الموارد المتاحة فورًا؛ مش ضمانًا بنجاح كل كتابة لاحقًا.

تأكد من فهمك

01 **هل Page Fault خطأ قاتل دائمًا؟**

> **الإجابة:** لا؛ قد يكون طلبًا طبيعيًا لتحميل Page صحيحة، ويصبح خطأ فقط إذا كان الوصول غير صالح أوتعذر التعامل معه.

03 **لماذا زيادة Swap لا تعادل زيادة RAM؟**

> **الإجابة:** التخزين أبطأ كثيرًا؛ Swap تساعد النجاة من الضغط لكنها قد تسبب Thrashing وLatency مرتفعة.

الخطوة التالية

كمّل في حجز الذاكرة وعمر البيانات بعد تنفيذ التجربة هنا.

كمّل في نقل البيانات ومتى يصبح الحفظ ثابتًا بعد تنفيذ التجربة هنا.

حجز الذاكرة وعمر البيانات

حجز الذاكرة وعمر البيانات

هنتتبع قيمة محلية، وبعدها بيانات نحتاجها لمدة أطول. المطلوب فهم معنى عنوان في الذاكرة من الدرس السابق.

قيمة قصيرة العمر وبيانات نحتفظ بها

تخيل دالة تحسب سعرًا: المدخل والنتيجة الوسيطة مطلوبان أثناء الاستدعاء. عند انتهائه يمكن إنهاء عمر البيانات المحلية حسب قواعد اللغة. أما قائمة المشتريات فتحتاجها بعد انتهاء دالة الإضافة. **Allocation — حجز ذاكرة** توفير مكان للبيانات؛ **Lifetime — عمر البيانات** الفترة التي تكون فيها صالحة للاستخدام. المكان والعمر ليسا نفس السؤال.

Stack، أو المكدس، ينظم عادة بيانات استدعاءات الدوال. Heap، أو مساحة الحجز الديناميكي، تسمح بأعمار أكثر مرونة. اللغة والتنفيذ قد يحسنان مكان التخزين؛ لا تفترض أن كل قيمة محلية حتمًا موجودة فعليًا على المكدس.

سؤال: انتهاء دالة الإضافة لا يعني حذف قائمة ما زال البرنامج يحتاجها. مَن يقرر انتهاء عمرها؟ قواعد اللغة وطريقة إدارة الذاكرة والمراجع إليها، كما نفصّل تحت.

Heap وStack

Stack Frames (سجلات استدعاء تحمل حالة الدالة وما يلزم العودة منها) تحفظ معلومات استدعاءات الدوال ومتغيرات محلية وفق Runtime (بيئة تنفيذ اللغة والخدمات التي تحتاجها؛ غير معنى وقت التشغيل الزمني) واللغة، وتُزال عادة عند العودة. Heap (مساحة تخصيص بيانات لا يرتبط عمرها مباشرة باستدعاء واحد) تستخدم للكائنات والبيانات ذات العمر الديناميكي. Stack (مكدس: تنظيم آخر داخل أول خارج، ويُستخدم غالبًا لتتبع استدعاءات الدوال) Overflow ينتج غالبًا من Recursion (استدعاء الدالة لنفسها مباشرة أو عبر دوال أخرى) عميق أوإطار ضخم؛ Heap قد تعاني Leak (تسرب: موارد تظل محجوزة بعد انتهاء الحاجة إليها) أوFragmentation (تجزؤ: مساحة موزعة أو مهدرة تصعّب تلبية طلبات الحجز) أوOOM (Out Of Memory؛ عدم كفاية الذاكرة المطلوبة).

هذه أسماء لمناطق من Address Space، وليست شرائح RAM منفصلة. نظام التشغيل قد لا يمنح Physical Memory لحظة الحجز نفسها بسبب Demand Paging وOvercommit policies.

Allocation وLeak وFragmentation وOOM

Allocator (جزء برمجي يدير حجز كتل الذاكرة وإعادة استخدامها) يدير Blocks ويعيد استخدامها. Leak يعني بقاء Reference (مرجع يتيح الوصول لكائن أو بيانات؛ وجوده قد يبقيها مطلوبة) أوحجز لم يعد مفيدًا. Fragmentation تعني وجود مساحة لكنها موزعة بما يصعب تلبية Block مناسب أو يزيد الهدر. عند نفاد الذاكرة قد يفشل Allocation (حجز مساحة ذاكرة للاستخدام) أو يقتل النظام Process وفق سياسته.

راقب Working Set (الصفحات التي يستخدمها البرنامج بنشاط خلال فترة القياس) وResident Memory (الجزء الموجود حاليًا في الذاكرة الفعلية وفق حساب النظام) وPeak Usage (أعلى استهلاك رُصد خلال فترة)، لا حجم المتغيرات النظري فقط. Garbage Collector (جامع المهملات الذي يسترجع ذاكرة لم تعد قابلة للوصول) يساعد في لغات Managed لكنه لا يصلح Retention مقصودة أوCaches بلا حدود.

Managed memory ذاكرة تدير اللغة أجزاء من عمرها، و**Native memory** حجز خارج العدادات المعتادة لذاكرة كائنات اللغة.
Retention احتفاظ بمراجع لكائنات لا نحتاجها؛ جامع المهملات لا يحذف كائنًا ما زال البرنامج يحتفظ به. **RSS (Resident Set Size)؛ مقدار من ذاكرة العملية موجود حاليًا في RAM وفق قياس النظام)** يقيس الذاكرة المقيمة وفق النظام؛ قد يزيد بسبب مكدسات خيوط أو حجز مكتبة أو صفحات مرتبطة بملف حتى لو عداد heap ثابت.

نقل البيانات ومتى يصبح الحفظ ثابتًا

نقل البيانات ومتى يصبح الحفظ ثابتًا

البرنامج ممكن ينتج بيانات أسرع من الجهاز اللي يستقبلها. هنفهم مكان الانتظار، ثم الفرق بين إرسال الكتابة وثباتها على وسيط التخزين.

صندوق انتظار أم نسخة سريعة؟

برنامج يكتب100رسالة كل ثانية، والجهة الأخرى تقرأ60. تتراكم40رسالة كل ثانية. **Buffer — مخزن مؤقت للنقل** يحمل البيانات في الطريق. **Queue — طابور** ينظم ترتيب الانتظار. أما **Cache — نسخة لإعادة الاستخدام** فتحفظ نتيجة يمكن غالبًا جلبها أو حسابها مرة أخرى.

بعد10ثوانٍ يزيد الانتظار400رسالة. **Backpressure — إبطاء المنتج استجابة لبطء المستهلك** تمنع النمو بلا حد، أو نرفض عملًا جديدًا عند حد واضح. بعد وصول الكتابة للجهاز يأتي سؤال آخر: هل ستبقى بعد انقطاع الطاقة؟ ده معنى **Durability — ثبات البيانات بعد الفشل ضمن ضمان محدد**.

Buffer وCache وQueue وStream وPool

المفهوم	الهدف
Buffer (مساحة تحتفظ ببيانات مؤقتًا أثناء انتقالها بين طرفين مختلفي السرعة)	امتصاص فرق السرعة أوتجميع البيانات قبل النقل
Cache	الاحتفاظ بنسخة لتجنب حساب أوقراءة متكررة
Queue (طابور يعالج العناصر عادة بترتيب وصولها)	تنظيم وحدات عمل تنتظر المعالجة
Stream (تدفق بيانات متتابعة قد لا نعرف حجمه مقدمًا)	تدفق متتابع قد لا نعرف حجمه مقدمًا
Pool (مجموعة موارد جاهزة يعاد استخدامها بدل إنشائها كل مرة)	موارد جاهزة لإعادة الاستخدام مثل Connections أوWorkers

قد تستخدم بنية واحدة لأكثر من دور، لكن سياسة الصحة مختلفة: Cache يمكن إسقاطها وإعادة بنائها، بينما Buffer غير المرسل قد يحتوي بيانات لا يجوز فقدها.

أنواع الـBuffers

- Keyboard/Input buffer يجمع Events حتى يقرأها البرنامج.
- File buffer يقلل System Calls الصغيرة.
- Socket send/receive buffers تمتص اختلاف سرعة الشبكة والتطبيق.
- `stdout` قد يكون Line-buffered في Terminal وFully buffered عند التحويل إلى ملف.

- Ring buffer يعيد استخدام مساحة ثابتة عبر مؤشري قراءة وكتابة.
- Double buffering يبني Frame خلفية بينما تُعرض الحالية لتقليل Flicker.

Buffer Overflow يعني الكتابة بعد الحد في بيئة لا تمنعها، وهو خطر أمني. Buffer Underflow في الصوت/الفيديو يعني أن المستهلك احتاج بيانات قبل وصولها. Backpressure تمنع المنتج السريع من ملء الذاكرة بلا حد.

Durability و Flush

`flush` قد يدفع البيانات من مكتبة إلى Kernel فقط؛ Kernel نفسه قد يحتفظ بها في Page Cache، وController أوDevice قد يملك Cache أخرى. إذا كان المطلوب Durability (ضمان بقاء البيانات المحفوظة وفق عقد محدد حتى بعد العطل) بعد انقطاع الكهرباء فاحتج إلى API (Application Programming Interface؛ اتفاق يسمح لبرنامج بطلب بيانات أو عملية من مكوّن آخر) وسياسة Filesystem/Database صريحة، لا مجرد طباعة أوإغلاق Stream.

DMA و Zero-copy

DMA (Direct Memory Access؛ نقل بيانات بين جهاز والذاكرة من غير نسخ المعالج لكل بايت) يسمح للجهاز بنقل Blocks من RAM أوإليها دون نسخ CPU لكل Byte. Zero-copy اسم لمجموعة تقنيات تقلل النسخ بين Buffers وطبقات Kernel/User Space؛ لا تعني عدم وجود أي حركة بيانات. استخدمها عندما يثبت القياس أن Copy Cost مؤثرة.

تدريب وحل: منتج يضيف 100 رسالة في الثانية ومستهلك يعالج60؛ الطابور يزيد40 كل ثانية. بعد10 ثوانٍ يزيد400 رسالة. حد حجم مع رفض أو إبطاء المنتج Backpressure يعالج النمو؛ زيادة RAM وحدها تؤجل الامتلاء. وflush، دفع البيانات من مخزن لآخر، لا يضمن وحده الحفظ بعد قطع الطاقة.

02 **ما الفرق الجوهري بين Buffer وCache؟**

> **الإجابة:** Buffer يحمل بيانات في طريقها بين طرفين، أما Cache فتحفظ نسخة يمكن عادة إعادة إنتاجها لتسريع الوصول.

04 **لماذا flush لا يضمن Durability دائمًا؟**

> **الإجابة:** قد تنقل البيانات إلى Buffer أخرى في Kernel أوالجهاز؛ الضمان يحتاج Sync وFilesystem/Database contract مناسبًا.

إمتى يفيد تشغيل أعمال كثيرة معًا؟

إمتى يفيد تشغيل أعمال كثيرة معًا؟

هنقارن خطوات تعتمد على بعضها بآلاف العمليات المستقلة. المطلوب فهم المعالج والذاكرة من الدروس السابقة.

كيف يختلف GPU؟

CPU (Central Processing Unit؛ المعالج الرئيسي الذي ينفذ التعليمات) يحتوي عددًا أقل من Cores قوية محسنة للـLatency (زمن انتظار عملية واحدة) والتفرعات والمهام العامة. GPU (Graphics Processing Unit؛ معالج مناسب لعمليات كثيرة متشابهة على بيانات متعددة) يحتوي عددًا كبيرًا من وحدات أبسط لتنفيذ نفس العملية على بيانات كثيرة، فيبرع في الرسومات والمصفوفات وMachine Learning وScientific Computing.

Branching غير المنتظم ونقل البيانات الصغير المتكرر قد يقللان فائدة GPU. يجب نقل البيانات إلى ذاكرة GPU المنفصلة في البطاقات Discrete، ثم تشغيل Kernel وإعادة النتائج؛ كلفة النقل قد تكون أكبر من الحساب الصغير.

Integrated وDiscrete GPU

Integrated GPU تشارك غالبًا ذاكرة النظام وميزانية الطاقة، وهي مناسبة للعرض وأعمال متوسطة. Discrete GPU تملك عادة ذاكرة رسومية خاصة وميزانية طاقة وتبريدًا مناسبًا؛ أداؤها يعتمد على الموديل والحمل، لكن الوصول إلى RAM وVRAM (Video RAM؛ ذاكرة بيانات الرسوم، غالبًا خاصة ببطاقة منفصلة) ليس مجانيًا. Unified Memory قد تبسط البرمجة ولا تلغي كلفة الحركة الفيزيائية دائمًا.

اختيار المعالج المناسب

العمل	الأنسب غالبًا	السبب
Web request وقواعد عمل	CPU	فروع وI/O (Input/Output؛ القراءة والكتابة والتعامل مع الأجهزة) وLatency
ضغط ملف واحد صغير	CPU	كلفة الإرسال للـGPU لا تستحق
ضرب مصفوفات ضخمة	GPU	Data parallelism مرتفع
رسم ملايين Pixels	GPU	عمليات متشابهة على بيانات كثيرة
Database transaction	CPU	تحكم وذاكرة وI/O أكثر من حساب متجانس

اقرأ ملفاتك واكتبها بالطرفية

اقرأ ملفاتك واكتبها بالطرفية

الطرفية مفيدة لما عايز تكرر أوامر أو تشوف نتيجة عملية بوضوح. هنستخدم PowerShell على Windows ونشتغل في مجلد تدريب جديد. المطلوب تعرف تحفظ ملفًا وتحدد مكانه؛ لا تحتاج مشروعًا جاهزًا.

الطرفية غير مفسّر الأوامر

Terminal — الطرفية نافذة تعرض إدخالًا وخرجًا نصيًا. **Shell — مفسّر أوامر** البرنامج الذي يقرأ الأمر ويشغله ويربطه بأوامر أخرى. PowerShell وBash وzsh وcmd أمثلة لمفسّرات أوامر، ولكل واحد قواعده.

Process — عملية نسخة برنامج تعمل الآن. مفسّر الأوامر قد ينفذ الأمر بنفسه أو يبدأ عملية أخرى. **Escaping — حماية رمز من التفسير الخاص** تختلف بين المفسّرات؛ مثلًا كتابة علامة اقتباس داخل نص لها صياغة تعتمد على الأداة. لا تنقل صياغة أمر من Bash إلى PowerShell بافتراض التطابق.

مكانك ومسار الملف

Working directory — مجلد العمل الحالي نقطة البداية للمسار النسبي. **Path — مسار** يحدد مكان ملف، مش اسمه وحده.

- **Absolute path — مسار مطلق** يبدأ من جذر النظام أو القرص، مثل `C:\Users\Learner\note.txt`.
- **Relative path — مسار نسبي** يبدأ من مجلدك الحالي، مثل `.\note.txt`.
- `.` تعني المجلد الحالي، و`..` المجلد الأب الذي يحتويه.
- **Extension — امتداد** نهاية الاسم مثل .txt؛ لا يثبت وحده نوع البيانات الفعلي.

لو الأمر قال إن الملف مش موجود، اعرض مجلد العمل أولًا؛ يمكن الملف موجود في مكان مختلف، فلا تغيّر اسمه عشوائيًا.

مساحة تدريب وأوامر تقرأ نتيجتها

افتح PowerShell على Windows. الأوامر التالية تنشئ مجلد تدريب جديدًا داخل مجلد المستخدم، ثم ملفًا نصيًا. لا تعتمد على وجود مجلد صاحب الشرح. لو المجلد موجود بالفعل، اختر اسمًا جديدًا:

```
Set-Location $HOME
New-Item -ItemType Directory -Name terminal-practice
Set-Location .\terminal-practice
Set-Content -LiteralPath '.\note.txt' -Value 'Hello'
Get-Location
Get-ChildItem
Get-Content -LiteralPath '.\note.txt'
```

Set-Location يغيّر مجلد العمل، و$HOME متغير جاهز يشير لمجلد حسابك. New-Item ينشئ عنصرًا، وItemType يحدد أنه مجلد. Set-Content يكتب النص؛ لو استخدمته على ملف موجود يستبدل محتواه، لذلك اشتغل في مجلد التدريب الجديد. Get-Location يعرض المكان، وGet-ChildItem يعرض عناصره، وGet-Content يقرأ الملف. المتوقع رؤية note.txt ثم Hello. Tab يساعد في إكمال الاسم.

Quoting — وضع النص بين علامات اقتباس يحافظ على مسار فيه مسافات. **Wildcard — نمط اختيار** مثل النجمة قد يطابق ملفات كثيرة؛ LiteralPath يعامل المسار كما هو. **Recursive — شامل المجلدات الداخلية** يوسع نطاق العملية، لذلك راجع المسار قبل نقل أو حذف.

قبل تجربة أمر واسع

اعرض المكان ومحتوياته، واقرأ المساعدة بواسطة Get-Help أوالخيار –help إذا كانت الأداة تدعمه. اقتبس المسار الذي فيه مسافات. **Bulk operation — عملية على عناصر كثيرة** و**Recursive — شاملة المجلدات الداخلية** توسع نطاق التعديل؛ جرّب على ملفات التدريب أولًا وراجع المسار قبل النقل والحذف.

لا تضع أسرارًا داخل نص الأمر إذا كان سيظهر في تاريخ الأوامر أو قائمة العمليات. **Script — ملف أوامر** يشغّل خطوات متتابعة؛ عليه فحص رمز الانتهاء والأخطاء عند الاعتماد على برامج أخرى.

تأكد من فهمك

01 **ما الفرق بين Terminal وShell؟**

الإجابة: Terminal تعرض الجلسة، وShell تفسر الأوامر وتبدأ العمليات وتربط الـStreams.

02 **لماذا يفشل Relative Path أحيانًا رغم وجود الملف؟**

الإجابة: لأنه يُفسر من Working Directory الحالية، وقد تختلف عن مجلد Script أوالمشروع.

03 **لماذا نفصل stdout عن stderr؟**

الإجابة: حتى تظل البيانات القابلة للمعالجة نظيفة بينما يمكن عرض أوتسجيل التشخيص منفصلًا.

04 **هل Environment Variable خزنة أسرار؟**

الإجابة: لا؛ هي قناة إعداد وقد تتسرب. استخدم Secret Manager وصلاحيات وتدويرًا ومنعًا للتسجيل.

الخطوة التالية

كمّل في قنوات الأوامر والصلاحيات والبيئة بعد تنفيذ التجربة هنا.

قنوات الأوامر والصلاحيات والبيئة

قنوات الأوامر والصلاحيات والبيئة

ابدأ بعد إنشاء ملف وقراءته بالطرفية. هنا نفصل البيانات العادية عن رسالة الخطأ، ثم نعرف كيف يرث البرنامج إعداداته وما الذي تسمح به هويته.

البيانات والاسم والصلاحيات

الملف يحتوي بايتات، والبايت8 بتات تمثل بيانات. **Metadata — بيانات وصفية** مثل الحجم والأوقات والمالك. **Filesystem — نظام ملفات** ينظم ربط أسماء المجلدات والملفات بالتخزين.

إعادة الاسم داخل نفس نظام الملفات قد تعدّل البيانات الوصفية بسرعة. النقل بين أقراص قد يحتاج نسخ البيانات ثم حذف الأصل، فيأخذ وقتًا مختلفًا. **Symbolic link — رابط رمزي** يشير إلى مسار آخر؛ ليس نسخة احتياطية من محتواه.

User — مستخدم هوية، و**Group — مجموعة** تجمع هويات لمنح الإذن. **Permissions — صلاحيات** تحدد القراءة والكتابة والتنفيذ. في Unix قد تراها موزعة على المالك owner والمجموعة group والباقين others. في Windows توجد **ACLs، Access Control Lists — قوائم تحكم في الوصول** تحدد من يسمح له بأي عملية.

صلاحية التنفيذ لا تثبت أن الملف آمن. وحق قراءة ملف لا يعني أنه مناسب للتنفيذ داخل تطبيقك. حساب Administrator أوRoot هو حساب إدارة واسع الصلاحيات؛ استخدم الصلاحية التي يحتاجها العمل فقط.

متغيرات البيئة: اسم وقيمة

Environment variable — متغير بيئة اسم مرتبط بقيمة، مثل اسم وضع التشغيل. **Parent process — العملية الأصلية** تبدأ **Child process — عملية فرعية** ترث عادة نسخة من إعدادات بيئتها.

```
$env:APP_ENV = 'development'
Get-ChildItem Env:APP_ENV
```

السطر الأول يجعل قيمة APP_ENV هي development داخل هذه الجلسة. الثاني يعرض الاسم وقيمته. لا يغيّر ذلك إعدادات الجهاز كله، ولا بيئة عمليات بدأت قبل التغيير.

متغير البيئة قناة إعداد، مش خزنة أسرار: أدوات فحص العمليات أو تقارير الأعطال أو السجلات قد تكشفه. **Secret manager — خدمة إدارة أسرار** تحفظ القيم الحساسة وتتحكم في الوصول إليها في التشغيل الفعلي. افحص وجود إعداداتك ونوعها وحدودها عند بدء التطبيق.

ثلاث قنوات ورقم انتهاء

- **stdin — Standard input:** الإدخال المعتاد للبرنامج، مثل نص من لوحة المفاتيح أو من برنامج قبله.

- **stdout — Standard output:** النتيجة العادية التي قد يقرأها برنامج آخر.
- **stderr — Standard error:** رسائل التشخيص والأخطاء، منفصلة عن النتيجة.
- **Exit code — رمز انتهاء:** رقم يعيده البرنامج؛ الصفر يعني نجاحًا عادة، وغير الصفر يحدد فشلًا حسب توثيق البرنامج.

CI، Continuous Integration — دمج تغييرات مع فحوص آلية يستخدم رمز الانتهاء ليعرف هل الفحص نجح. لا يحتاج تخمين النجاح من جملة معروضة.

Pipe — أنبوب أوامر، ورمزه `|`، يمرر نتيجة أمر إلى التالي. **Stream — تدفق** بيانات متتابعة؛ لا يحتاج دائمًا ملفًا وسيطًا. في PowerShell يمكن للأنبوب تمرير **Objects — قيم لها حقول**، زي اسم الملف وحجمه، بدل نص فقط.

Redirection — إعادة توجيه ترسل قناة إلى ملف أو وجهة أخرى. افصل رسائل التقدم والتشخيص عن البيانات التي سيعالجها برنامج آخر. **Backpressure — انتظار المنتج للمستهلك** يحصل لما الجانب الذي يقرأ أبطأ من الذي يكتب وتمتلئ مساحة الانتظار.

جرّب القنوات بعد تعريفها

في PowerShell، pipe بين أوامره المعتادة يمرر كائنات، أي قيم لها حقول مثل الاسم والحجم، مش مجرد نص دائمًا:

```
Get-ChildItem -File | Select-Object Name, Length
```

Select-Object يختار حقلي Name وLength. أما البرامج الخارجية فلها إدخال وخروج وأخطاء وفق واجهة النظام. مثال يكتب ملفين داخل مجلد التدريب:

```
cmd /c 'echo result & echo diagnostic 1>&2 & exit /b 3' 1> output.txt 2> error.txt
$LASTEXITCODE
```

ده مثال خاص بـWindows: شغّلنا cmd لكتابة result للخروج العادي وdiagnostic للأخطاء، ثم إنهاء بكود3. إعادة التوجيه1> تحفظ الخروج، و2> تحفظ الأخطاء. المتوقع LASTEXITCODE يساوي3؛ هذا متعمد في التجربة وليس عطلًا. لا تستخدم نفس صياغة cmd في Bash أو PowerShell على أنها متطابقة.

في مثال `/c`، cmd يشغّل النص ثم ينهي المفسّر، و`&` داخل النص يفصل أوامر cmd، و`echo` يكتب رسالة، و`1>&2` يرسل الخروج العادي لقناة الخطأ، و`exit /b 3` ينهي بالأرقام المتفق عليها هنا. هذه الرموز داخل النص تخص cmd؛ إعادة التوجيه خارج الاقتباس ينفذها PowerShell.

اختبر السلوك وحدّد سبب الخطأ

اختبر السلوك وحدّد سبب الخطأ

اختبار الكود يقارن سلوكه بوعده، وGit يحفظ تاريخ الملفات. هنفصل المهمتين: اكتب المتوقع مستقلًا، شغّل الحالة، ثم ابحث عن أول خطوة تختلف.

اكتشاف سبب الخطأ

Debugging — تشخيص الخطأ وإصلاحه يبدأ بتجربة تقدر تكررها:

1. أعد المشكلة بمدخل ثابت.
2. اكتب النتيجة المتوقعة Expected والنتيجة الفعلية Actual.
3. حدد آخر نقطة كانت صحيحة.
4. اجمع دليلًا من Logs، سجلات الأحداث، وDebugger، أداة توقف التنفيذ وتعرض القيم، وTests، اختبارات تقارن النتائج بالمطلوب.
5. ضع فرضية واحدة واختبرها بتغيير صغير.
6. أصلح السبب وأضف Regression Test (اختبار لحالة عطّل سابق حتى نلاحظ رجوعه).

`git bisect` يستطيع البحث الثنائي في التاريخ إذا عرفت Commit جيدة وأخرى سيئة، بشرط وجود طريقة تميّز النجاح من الفشل.

أنواع الاختبارات

- **Unit test — اختبار وحدة:** يفحص منطقًا صغيرًا معزولًا.
- **Integration test — اختبار تكامل:** يفحص تعاون مكونات، مثل البرنامج وقاعدة البيانات أو نظام الملفات.
- **End-to-end test — اختبار رحلة كاملة:** ينفذ خطوات مستخدم عبر النظام.
- **Regression test — اختبار عدم رجوع عطل:** يعيد حالة كشفت عطلًا سابقًا ليلفت النظر لو رجع؛ نجاحه ليس برهانًا أن كل الأعطال مستحيلة.

الاختبار الجيد يرتب **Arrange–Act–Assert**، أي جهز البيانات، نفذ العملية، ثم قارن بالمطلوب، وله سبب فشل واحد واضح، ولا يعتمد على وقت أوشبكة عشوائية بلا تحكم.

الشبكات والويب

امشِ وراء طلب من الشبكة المحلية، للعناوين ونقل البيانات، ثم اتصال ويب محمي. تفاصيل تطبيقات الخوادم تأتي بعد المسار الأساسي.

هنمشي من اتصال جهازين إلى عنوان موقع وطلب صفحة، وبعدها نجرب خادمًا محليًا ونفهم الكاش والتحديثات ومراقبة الطلب. شغّل معمل Node.js قبل تطبيقات الخوادم؛ أمثلة PHP الإضافية لها رابط تجهيز لغتها داخل الدرس.

الاتصال ونقل البيانات

- الإنترنت والويب ورحلة الطلب
- كيف تتواصل الأجهزة داخل الشبكة المحلية؟
- تقسيم الشبكة ومنع دوران الإطارات
- اتصل بشبكة Wi-Fi وافهم مشاكلها
- إعدادات عنوان الجهاز والاتصال
- احسب حدود الشبكة من عنوانها
- اختيار الطريق وقراءة عناوين IPv6
- كيف يتحول اسم الموقع إلى عنوان؟
- أخطاء البحث عن العناوين وحماية DNS
- TCP: الاتصال من أول SYN لحد الإغلاق
- UDP: رسائل مستقلة
- MTU: حجم الحزم وتشخيص المسار

الويب والاتصال الآمن

- عنوان الويب والمنافذ وبروتوكول HTTP
- الحروف والرموز داخل عنوان الويب
- اقرأ طلب HTTP وردّه
- تذكّر المستخدم وحماية التعديلات
- تشفير الاتصال وشهادات المواقع
- كيف يتفق الطرفان على اتصال مشفّر؟
- كيف يعرض المتصفح الصفحة؟
- تحميل البرامج وترتيب مهام المتصفح

تطبيقات الويب والخوادم

- جهّز معمل الطلب والرد على جهازك

- رحلة الطلب داخل الخادم
- الخوادم الوسيطة وتوزيع الطلبات
- توزيع الحمل وسياسات الوسيط
- التخزين المؤقت وضغط بيانات الويب
- السماح بقراءة البيانات بين المواقع
- سياسات تحميل الموارد وعزل النوافذ
- اسأل دوريًا أو استقبل بثًّا من الخادم
- استقبل إشعار خادم وتعامل مع تكراره
- تصميم واجهات تبادل البيانات
- كيف يختلف HTTP/2 عن HTTP/3؟
- وزّع المحتوى وافهم الخادم الأصلي
- حماية الطلبات وتحديد معدلها
- تتبّع الطلب وفسّر القياسات

الإنترنت والويب ورحلة الطلب

افهم الفرق بين الإنترنت والويب، وأدوار العميل والخادم، ورحلة الطلب والاستجابة من الرابط إلى الصفحة.

لما تكتب عنوان موقع وتضغط Enter، الصفحة بتظهر بعد سلسلة خطوات. هنمشي وراء طلب واحد: نعرف عنوان الوجهة، نفتح اتصالًا، نبعت طلبًا، ونفهم الرد. مش مطلوب تكون كتبت برنامج خادم؛ المطلوب تعرف تفتح متصفحًا وتكتب عنوانًا.

أربع خطوات قبل التفاصيل

تكتب اسم الموقع؛ **DNS** نظام يسأل عن معلومات الاسم ومنها عنوانه. بعدها المتصفح ينشئ اتصالًا بالجهة المناسبة، ويرسل **طلبًا** للصفحة، ويتلقى **ردًا** يعرضه.

```
Browser ── name question ──> DNS
Browser <─ address answer ── DNS
Browser ── connection + HTTP request ──> Server
Browser <──────── HTTP response ──────── Server
```

DNS يجيب عن العنوان؛ محتوى الصفحة لا يمر بداخله. ده أول تصور تحتاجه. أسماء الوسطاء وطرق التشفير تفاصيل تأتي في دروسها. **توقع:** اسم غير موجود يفشل قبل طلب الصفحة؛ عنوان صحيح مع مورد غير موجود يمكن أن يرجع404 من خادم وصلنا إليه.

الفكرة قبل المصطلحات

تخيّل الإنترنت كشبكة طرق عالمية، والويب كخدمة توصيل تعمل فوق هذه الطرق. الطرق تستطيع حمل أشياء أخرى أيضًا؛ لذلك **الإنترنت ليس هو الويب**:

- **الإنترنت** هو البنية التي تربط الشبكات والأجهزة باستخدام بروتوكولات مثل IP (Internet Protocol؛ قواعد عنونة الحزم وتوجيهها بين الشبكات، وعنوان IP رقم يحدد واجهة في هذا السياق).
- **الويب** خدمة تستخدم هذه البنية للوصول إلى موارد، يعني صفحات أو صور أو بيانات، بعناوين URL (Uniform Resource Locator؛ عنوان يحدد موردًا وطريقة الوصول إليه) عبر HTTP (Hypertext Transfer Protocol؛ قواعد طلب الموارد والرد عليها في الويب) أو HTTPS (Hypertext Transfer Protocol Secure؛ طلبات ويب عبر اتصال محمي بالتشفير والتحقق من هوية الطرف الآخر).
- البريد الإلكتروني، ومكالمات الإنترنت، والألعاب خدمات تستخدم الإنترنت من غير أن تكون صفحات ويب.

هذا الفرق مفيد عند تشخيص الأعطال: نجاح الاتصال بالإنترنت لا يضمن أن موقعًا بعينه أو خدمة HTTP تعمل.

العميل والخادم: دوران لا نوعان من الأجهزة

الـ**Client** برنامج يبدأ الطلب: متصفح، تطبيق هاتف، أو أداة مثل `curl` ترسل طلبًا وتعرض الرد نصيًا. والـ**Server** برنامج يستقبل الطلب ويعيد استجابة. قد يعمل الاثنان على جهازك أثناء التطوير، وقد تمر الاستجابة في الإنتاج عبر عدة خوادم.

```
المستخدم → Client → HTTP Request → Server
                                      ↓
                                برنامج الموقع ⇔ بياناته
                                      ↓
عرض الصفحة ← المستخدم ← HTTP Response
```

كلمة «خادم» قد تعني الجهاز أو البرنامج حسب السياق. الأدق هنا هو الدور: برنامج ينتظر الطلبات على عنوان ومنفذ؛ المنفذ رقم يحدد جهة الاستقبال داخل الجهاز.

اقرأ الرابط قبل أن تتبعه

```
https://shop.example/products/42?currency=EGP#reviews
```

- `https` هو الـscheme (طريقة الوصول في بداية العنوان، مثل https) أو البروتوكول المطلوب.
- `shop.example` هو اسم المضيف الذي يحتاج إلى DNS (Domain Name System؛ نظام يجيب عن أسئلة أسماء النطاقات، ومنها عناوينها).
- `/products/42` هو المسار الذي يرسله المتصفح للخادم.
- `currency=EGP` هي query string (معاملات بعد علامة ? في العنوان، مثل name=Omar) وتصل عادة إلى الخادم.
- `#reviews` هو fragment (جزء بعد # يستخدمه العميل محليًا، مثل موضع عنوان في الصفحة) يستخدمه المتصفح، ولا يُرسل في HTTP Request (طلب: رسالة يرسلها العميل ليطلب موردًا أو عملية) عادة.

جرّب طلبًا حقيقيًا

```
curl.exe -i "https://example.com/"
```

سترى نتيجة شبيهة بالآتي، وقد تختلف بعض الرؤوس:

```
HTTP/2 200
content-type: text/html

<!doctype html>
...
```

- `curl` هو العميل بدل المتصفح.
- `-i` يطلب إظهار response headers مع الـbody.
- `HTTP/2 200` يعني أن الطلب نجح على مستوى HTTP؛ لا يعني أن كل منطق العمل صحيح.
- `content-type` يخبر العميل كيف يفسر الـbody.
- ما بعد السطر الفارغ هو جسم الاستجابة.

لرؤية خطوات الاتصال بتفصيل أكبر:

```
curl.exe -v "https://example.com/" -o NUL
```

استخدم `/dev/null` بدل `NUL` على Linux وmacOS. الخيار `-v` يعرض معلومات الاتصال وTLS والطلب، و`-o` يمنع طباعة جسم الصفحة كي تركز على الرحلة.

أخطاء شائعة

- **«DNS يفتح الصفحة»:** DNS يعيد عنوانًا؛ HTTP هو الذي يطلب المورد.
- **«Nginx يشغّل PHP بنفسه»:** غالبًا يمرر الطلب إلى PHP-FPM.
- **«ظهور `200` يعني أن كل شيء صحيح»:** قد تكون النتيجة ناقصة أو خاطئة مع status ناجح.
- **«كل زيارة تصل إلى قاعدة البيانات»:** قد تجيب طبقة cache أو ملف ثابت قبل ذلك.
- **«Server يعني جهازًا ضخمًا»:** هو دور يمكن أن يؤديه برنامج صغير على جهازك.

طبّق الفكرة

▼ فتحت موقعًا فظهرت رسالة DNS error. أي أجزاء من الرحلة لم تبدأ بعد؟

لم يحصل العميل على IP مناسب، لذلك لا يستطيع عادة بدء TCP أو QUIC ولا TLS ولا HTTP مع الخادم. ابدأ بفحص الاسم وDNS، لا كود PHP.

▼ الأمر `curl` يعيد HTML (Hypertext Markup Language؛ لغة وصف بنية الصفحة وعناصرها) صحيحًا لكن الصفحة في المتصفح لا تعمل. ماذا تستنتج؟

مسار DNS والاتصال وHTTP الأساسي يعمل على الأقل من بيئة `curl`. افحص JavaScript (لغة برمجة تستخدم مثلًا لتنفيذ تفاعل الصفحة داخل المتصفح)، وطلبات الموارد التابعة، وCORS (Cross-Origin Resource Sharing؛ قواعد تسمح للمتصفح بقراءة رد مصدر مختلف عندما يوافق الخادم)، وcookies، وcache. الاستنتاج يضيّق البحث لكنه لا يثبت أن البيئتين متطابقتان.

▼ هل يصل الجزء `#reviews` إلى PHP؟

لا يصل عادة؛ المتصفح يستخدم الـfragment محليًا. يصل المسار والـquery string، مثل `/products/42?currency=EGP`.

تأكد من فهمك

1. ما الفرق العملي بين الإنترنت والويب؟
   - **الإجابة:** الإنترنت بنية النقل العامة، والويب خدمة تستخدمها عبر HTTP/HTTPS. لذلك قد يعمل الإنترنت بينما تتعطل خدمة ويب محددة.
2. لماذا يمكن أن يكون جهازك Client وServer في الوقت نفسه؟
   - **الإجابة:** لأنهما دوران برمجيان؛ المتصفح يبدأ الطلب، وخادم التطوير المحلي يستقبله، حتى لو عملا على الجهاز نفسه.
3. أين تبحث إذا نجح DNS وTLS لكن الخادم أعاد `403`؟

- **الإجابة:** الاتصال وصل إلى HTTP، و `403` يعني أن الخادم فهم الطلب ورفض السماح به. افحص الهوية والصلاحيات وقواعد web server أو WAF.

كيف تتواصل الأجهزة داخل الشبكة المحلية؟

كيف تنتقل بيانات التطبيق عبر طبقات الشبكة، وكيف تتواصل الأجهزة داخل LAN باستخدام Frames وMAC وSwitch وARP.

جهازان وراوتر

```
Laptop A ── Switch ── Laptop B
              |
           Router ── other networks
```

Switch — مُبدّل يصل الأجهزة داخل الشبكة، و**Router — موجّه** ينقل بينها وبين شبكات أخرى. افترض A=192.168.1.20 وB=192.168.1.30، وكلاهما /24، أي أول24بت تحدد الشبكة. A يعرف من القناع أن B محلي. يستخدم ARP، سؤالًا محليًا عن عنوان واجهة B، ثم يرسل **إطارًا** يحمل البيانات وعنوان واجهة B عبر المبدّل. **MAC** عنوان الواجهة على الوصلة.

لو الوجهة خارج الشبكة، إطار A المحلي يتجه إلى واجهة الراوتر، بينما عنوان IP للوجهة يظل الوجهة البعيدة في المثال الذي لا يطبق ترجمة عناوين. **جرّب على الورق:** أرسل من B إلى A، ثم من B إلى شبكة أخرى؛ مَن يستقبل الإطار المحلي في كل مرة؟ A، ثم الراوتر.

نبدأ من رحلة بسيطة داخل البيت

افترض إن لابتوبك عايز يفتح صفحة موجودة على جهاز آخر. البرنامج يعرف URL (Uniform Resource Locator؛ عنوان يحدد موردًا وطريقة الوصول إليه)، لكن كارت الشبكة لا يرسل URL كما هو. البيانات تنزل خلال طبقات، وكل طبقة تضيف معلومات تساعد المرحلة التالية:

```
بيانات التطبيق
  ↓ معلومات النقل
TCP segment أو UDP datagram
  ↓ معلومات IP
IP packet
  ↓ معلومات الوصلة المحلية
Ethernet frame على وصلة Ethernet، أو Wi-Fi frame على وصلة Wi-Fi
  ↓
إشارات مناسبة للوصلة
```

الطبقات مش أجهزة منفصلة بالضرورة؛ هي تقسيم للمسؤوليات. HTTP (Hypertext Transfer Protocol؛ قواعد طلب الموارد والرد عليها في الويب) يهتم بمعنى الطلب، TCP (Transmission Control Protocol؛ نقل بايتات بترتيب مع معالجة الفقد، مع احتمال فشل الاتصال) يهتم بالاتصال والترتيب، IP (Internet Protocol؛ قواعد عنونة الحزم وتوجيهها بين الشبكات، وعنوان IP رقم يحدد واجهة في هذا السياق) يهتم بالوصول بين الشبكات، وEthernet (معيار شائع للاتصال على شبكة محلية سلكية)/Wi-Fi (اتصال لاسلكي بالشبكة المحلية؛ لا يضمن اتصالها بالإنترنت) يهتمان بالنقلة الحالية داخل الـLink.

لو الوجهة خارج شبكتك المحلية، جهازك لا يبحث عن MAC (Media Access Control؛ عنوان تستخدمه واجهة الشبكة للتواصل على الوصلة المحلية) الخاص بالسيرفر البعيد. هو يحتاج MAC الخاص بالـDefault Gateway (البوابة الافتراضية: الراوتر الذي نرسل له الوجهات التي لا نملك لها مسارًا أدق)، ويرسل له الـFrame (إطار: وحدة بيانات تخص الوصلة المحلية مثل Ethernet)، وبعدها كل Router يبني Frame جديدة للنقلة التالية. عنوان IP يصف الوجهة عبر الرحلة، بينما MAC تتغير من Link لآخر.

ARP (Address Resolution Protocol؛ سؤال عن عنوان MAC المقابل لعنوان IPv4 على الشبكة المحلية) هو السؤال المحلي: «مين عنده عنوان IPv4 (Internet Protocol version 4؛ إصدار عناوين الشبكة ذي32 بتّا) ده؟ ابعتلي MAC بتاعك». النتيجة تدخل Cache (نسخة محفوظة لتقليل تكرار القراءة أو الحساب) لمدة، لكنها غير موثوقة أمنيًا بمفردها؛ ARP Spoofing ممكن يربط IP بعنوان مهاجم داخل نفس الشبكة.

لماذا نقسم الاتصال إلى طبقات؟

كل طبقة تحل مشكلة محددة وتقدم خدمة للطبقة الأعلى. تطبيق HTTP لا يحتاج إلى معرفة نوع كابل Ethernet، وTCP لا يحتاج إلى فهم معنى JSON (JavaScript Object Notation؛ صيغة نصية لترتيب البيانات في أسماء وقيم وقوائم). يسمح هذا الفصل بتغيير تقنية في طبقة من غير إعادة تصميم النظام كله.

نموذج TCP/IP	أمثلة	وحدة البيانات
Application	HTTP وDNS (Domain Name System؛ نظام يجيب عن أسئلة أسماء النطاقات، ومنها عناوينها) وTLS (Transport Layer Security؛ قواعد حماية الاتصال بالتشفير والتحقق)	Message/Data
Transport	TCP وUDP (User Datagram Protocol؛ نقل رسائل مستقلة بلا ضمان مدمج لوصولها أو ترتيبها)	Segment (مقطع TCP يحمل جزءًا من تدفق البايتات)/Datagram (رسالة مستقلة لها حدود في بروتوكول النقل)
Internet	IPv4 وIPv6 (Internet Protocol version 6؛ إصدار عناوين الشبكة ذي128 بتًا)	Packet (حزمة: وحدة بيانات IP التي تُوجّه بين الشبكات)
Link	Ethernet وWi-Fi	Frame

تضيف كل طبقة Header (حقل أو مقدمة معلومات تضاف للبيانات بحسب الطبقة) عند الإرسال، وتزيل الطبقة المقابلة هذا الـHeader عند الاستقبال. تسمى العملية Encapsulation.

LAN وEthernet وMAC Address

الـLAN (Local Area Network؛ شبكة أجهزة في مساحة محلية زي بيت أو مكتب) شبكة محلية داخل منزل أو مكتب أو مركز بيانات. يحدد Ethernet طريقة بناء Frame وإرسالها داخل الشبكة المحلية. يحتوي الـFrame على MAC Address للمصدر والوجهة، ونوع البيانات، وPayload، وقيمة لاكتشاف التلف.

عنوان MAC يعمل داخل نطاق الشبكة المحلية ولا يحل محل IP. يقرأ Switch عناوين المصدر ويبني جدولًا يربط كل MAC بالمنفذ الذي ظهر منه، ثم يرسل الـFrame إلى المنفذ المناسب بدل بثه دائمًا إلى الجميع.

ARP والوصول إلى الـGateway

عندما يعرف الجهاز IP داخل شبكته لكنه لا يعرف MAC المقابل، يرسل ARP Request (طلب: رسالة يرسلها العميل ليطلب موردًا أو عملية) بالبث: «من يملك هذا IP؟». يرد الجهاز المطلوب بعنوان MAC ويحفظ المرسل النتيجة مؤقتًا في ARP Cache.

إذا كان IP الهدف خارج الشبكة المحلية، لا يبحث الجهاز عن MAC الخادم البعيد. يرسل الـFrame إلى MAC الخاص بالـDefault Gateway، بينما يبقى IP الوجهة داخل Packet هو عنوان الخادم البعيد.

Switch وRouter

يحوّل Switch الـFrames داخل LAN اعتمادًا على MAC. يربط Router شبكات IP مختلفة ويختار Next Hop اعتمادًا على Routing Table. قد يجمع الجهاز المنزلي وظائف Router وSwitch وWi-Fi Access Point وDHCP (Dynamic Host Configuration Protocol؛ خدمة تمنح الجهاز إعدادات الشبكة تلقائيًا) وNAT (Network Address Translation؛ تغيير عناوين IP أثناء مرور الحزم بين شبكات) في صندوق واحد، لكن كل وظيفة تظل مختلفة منطقيًا.

تدريب عملي

▼ ليه MAC السيرفر البعيد مش موجودة في Frame الخارجة من اللابتوب؟

لأن Ethernet تعمل على الـLink المحلية فقط. اللابتوب يرسل للـDefault Gateway، فيستخدم MAC الراوتر المحلية. الراوتر يزيل Frame ويبني واحدة جديدة للنقلة التالية.

▼ رتب: Segment وPacket وFrame

بيانات التطبيق تدخل Segment في طبقة النقل، ثم Packet في طبقة IP، ثم Frame على الـLink. عند الاستقبال تُفك الطبقات بالعكس.

تأكد من فهمك

01 **لماذا يحتاج الجهاز إلى MAC وIP معًا؟**

> **الإجابة:** يحدد IP الجهاز عبر الشبكات، بينما يوجه MAC الـFrame على الوصلة المحلية الحالية.

02 **إلى أي MAC يرسل جهاز Packet متجهة إلى الإنترنت؟**

> **الإجابة:** إلى MAC الخاص بالـDefault Gateway، مع بقاء IP الوجهة هو IP الخادم البعيد.

الخطوة التالية

كمّل في تقسيم الشبكة ومنع دوران الإطارات بعد تنفيذ التجربة هنا.

تقسيم الشبكة ومنع دوران الإطارات

تقسيم الشبكة ومنع دوران الإطارات

بعد تتبع إطار بين جهازين، تخيل شبكة مكتب فيها مجموعات تحتاج فصلًا وروابط احتياطية. **الإطار** وحدة نقل البيانات على الوصلة المحلية. هنا نفهم سبب التقسيم قبل أدواته.

تقسيم الشبكة ومنع الدوران

ICMP، Internet Control Message Protocol ينقل رسائل تشخيص وأخطاء، مثل عدم الوصول لوجهة؛ ping أحد استخداماته. **VLAN، Virtual Local Area Network** تقسيم منطقي للشبكة المحلية: أجهزة تشترك في نطاق بث واحد، حتى لو نفس أجهزة التوصيل تحمل أكثر من شبكة. **Broadcast — بث عام** رسالة تصل لأجهزة نطاق البث؛ لا تعبر راوترًا عاديًا لكل الإنترنت.

STP، Spanning Tree Protocol يمنع حلقات توصيل Ethernet من إبقاء الإطارات تدور وتتكاثر، عن طريق تعطيل بعض المسارات الزائدة منطقيًا. **ARP spoofing** إرسال ربط كاذب بين IP وMAC؛ لأن ARP وحده لا يثبت هوية المجيب. الإصدار السادس IPv6 يستخدم **NDP، Neighbor Discovery Protocol** لاكتشاف الجيران ومعلومات محلية أخرى؛ له مخاطر تلاعب أيضًا.

تدريب محلول: جهاز في VLAN 10 يريد جهازًا في VLAN 20. حتى لو الجهازان على نفس صندوق التوصيل، يحتاج المرور عادة عبر راوتر أو جهاز تحويل يدعم التوجيه بين الشبكتين وبحسب سياسة السماح. ينتهي إطار الوصلة عند جهة التوجيه، وتخرج حزمة IP داخل إطار يناسب الوصلة التالية. وعند غياب MAC معروف قد يرسل switch الإطار لعدة منافذ داخل نفس نطاق البث؛ مش دائمًا منفذًا واحدًا.

اتصل بشبكة Wi-Fi وافهم مشاكلها

فرّق بين الاتصال بالشبكة ووصول الإنترنت، وافهم اسم الشبكة ونقطة الوصول والإشارة والحماية والتشخيص.

الهاتف بيعرض علامة Wi-Fi، لكن الموقع مش بيفتح. هل المشكلة في الهاتف، ولا الشبكة داخل البيت، ولا اتصال البيت بالإنترنت؟ هنفصل المراحل ونجرّب تشخيصًا من غير تغيير إعدادات عشوائي. ابدأ بعد الشبكة المحلية، وهي الأجهزة التي تتواصل داخل نطاق قريب زي البيت.

مين بيعمل إيه داخل البيت؟

Wi-Fi طريقة اتصال لاسلكي بالشبكة المحلية باستخدام موجات راديوية. الاسم مش اختصارًا تقنيًا مطلوب نفكّه. **Access point — نقطة وصول** الجزء اللي الجهاز بيتصل به لاسلكيًا ليدخل الشبكة. **Router — راوتر** يوجّه البيانات بين شبكتك وشبكات أخرى. ممكن نفس الصندوق يؤدي الوظيفتين.

Modem — مودم يربطك بنوع خط مقدم الخدمة، مثل خط الهاتف أو الكابل، ويحوّل الإشارات المناسبة له. في الألياف قد توجد **ONT، Optical Network Terminal — وحدة طرفية للشبكة الضوئية**. مش كل بيت عنده نفس الأجهزة أو نفس تقسيم الوظائف؛ صندوق واحد قد يجمع أكثر من وظيفة.

```
Phone --Wi-Fi--> Access point --local network--> Router --> Provider --> Internet
```

كل سهم جزء من الطريق. قوة أول سهم لا تثبت سلامة بقيته. ممكن تتصل بنقطة الوصول وتتبادل ملفات محلية بينما وصلة الإنترنت متوقفة. وكابل **Ethernet**، الاتصال السلكي المحلي، لا يضمن الإنترنت وحده أيضًا.

اسم الشبكة والدخول إليها

SSID، Service Set Identifier هو اسم الشبكة الذي يظهر في قائمة Wi-Fi، زي `Home-Learning`. الاسم وحده مش إثباتًا لصاحب الشبكة؛ ممكن شبكتان تعلنا نفس الاسم. استخدم شبكة تملكها أو مسموح لك بها، وخد الاسم وكلمة المرور من صاحبها.

على Windows افتح قائمة الشبكات من شريط المهام أو إعدادات الشبكة، اختار الاسم، واضغط اتصال واكتب كلمة المرور. أسماء القوائم تختلف حسب الإصدار. ما تنسخش كلمة المرور داخل تقرير تشخيص أو صورة للشاشة.

شبكات الفنادق والمطارات قد تعرض **Captive portal — صفحة سماح بالدخول** قبل إتاحة الإنترنت. نجاح الاتصال اللاسلكي لا يعني إكمال هذه الخطوة. افتح إشعار تسجيل الدخول الخاص بالنظام إن ظهر، وتأكد إن الشبكة فعلًا تخص المكان.

الإشارة والسرعة مش نفس الحاجة

موجات Wi-Fi تعمل في نطاقات تردد، منها 2.4 و5 و6 **GHz — جيجاهرتز**؛ الجيجاهرتز مليار دورة في الثانية. ده وصف للموجة، مش سرعة تنزيل ملف، ومش المقصود شبكة المحمول 5G.

2.4 GHz غالبًا تصل لمسافة أكبر عبر العوائق، بينما 5 و6 قد يوفران سعة أكبر قريبًا من نقطة الوصول مع أجهزة وظروف مناسبة. النتيجة تعتمد على الحوائط والمكان والتداخل والجهاز؛ لا تختار من الرقم وحده.

Channel — قناة جزء من نطاق التردد تستخدمه الشبكة. شبكات قريبة وأجهزة أخرى قد تشارك الهواء أو تتداخل، فتزيد فترات الانتظار والمحاولات. شرائط الإشارة تقرّب قوة الاستقبال، ولا تقيس وحدها الازدحام أو سرعة اشتراك الإنترنت.

مقوّي الإشارة يعيد نقل الاتصال؛ لو وصلة المقوّي بنقطة الوصول ضعيفة، قرب الهاتف منه وحده لا يصلح الجزء الضعيف. جرّب مكانًا أفضل قبل شراء أو تغيير أجهزة.

حماية الاتصال

WPA، Wi-Fi Protected Access اسم عائلة وسائل لحماية الاتصال اللاسلكي. WPA2 وWPA3 إصدارات منها؛ استخدم إعدادًا حديثًا تدعمه أجهزتك، مثل WPA3-Personal أو وضع التوافق WPA2/WPA3 عند الحاجة. لا تعتبر شبكة مفتوحة بلا حماية مناسبة لبيانات حساسة.

كلمة مرور Wi-Fi تسمح بالدخول للشبكة. كلمة مرور إدارة الراوتر تسمح بتغيير إعداداته؛ الاثنين مختلفين. غيّر بيانات الإدارة الافتراضية واتبع تحديثات الشركة المصنعة. **Guest network — شبكة ضيوف** قد تفصل أجهزة الزوار عن أجهزتك وفق إعداداتها؛ تأكد من خيار العزل بدل افتراضه من الاسم.

حماية Wi-Fi تخص الوصلة المحلية. **HTTPS** حماية اتصال الويب بين المتصفح والموقع، وله درس لاحق؛ تشغيل واحدة لا يغني عن الثانية، ولا يضمن أمانة صاحب الموقع.

شخّص بالمقارنة

الملاحظة	الخطوة التالية والسبب
اسم الشبكة لا يظهر	راجع تشغيل Wi-Fi ووضع الطيران والمسافة ودعم الجهاز للنطاق
الاسم ظاهر لكن كلمة المرور مرفوضة	راجع الشبكة المختارة وكلمة المرور وحالة الأحرف
يعمل قريبًا ويتعطل بعيدًا	قارن نفس الجهاز ونفس الاختبار قرب نقطة الوصول؛ افحص أثر المسافة والعوائق
كل الأجهزة متصلة ولا تصل للإنترنت	راجع حالة وصلة مقدم الخدمة؛ الاحتمال ليس مشكلة جهاز واحد فقط
جهاز واحد يفشل والآخر يعمل في نفس المكان	راجع إعدادات الجهاز الفاشل؛ المقارنة ضيّقت نطاق البحث ولم تثبت سببًا نهائيًا
موقع واحد يفشل ومواقع أخرى تعمل	افحص الموقع أو تحويل اسمه لعنوان؛ الاتصال العام قد يكون سليمًا

تجربة محلولة: بنفس الهاتف افتح نفس الصفحة قريبًا من الراوتر ثم في الغرفة البعيدة. لو القريب يعمل والبعيد يتقطع، نبدأ بفحص التغطية. لو الجهازان في المكان القريب لا يفتحان أي موقع، نراجع مسار الإنترنت. لا نبدأ بإعادة ضبط المصنع لأنها تمسح إعدادات وقد تحتاج بيانات مقدم الخدمة.

في درس العناوين التالي هنعرف معنى إعدادات العنوان والبوابة، وبعده نجرب أدوات تميّز مراحل الاتصال. الهدف هنا إنك تعرف أي جزء تختبره، وما تخلطش علامة Wi-Fi باتصال إنترنت كامل.

مراجع: تشخيص Wi-Fi على Windows، إعدادات الراوتر ونقاط الوصول.

سجل مقارنة بدل الاعتماد على الإحساس

استخدم نفس الصفحة وفي وقت متقارب؛ لا تغيّر أكثر من عامل معًا. القيم دي خانات تملأها، مش نتائج مفترضة:

التجربة	الجهاز	المكان	اسم الشبكة	قوة الإشارة الظاهرة	فتح صفحة جديدة	ملاحظتك
1	الأول	قريب	…	…	نجح/فشل والمدة	…
2	الأول	بعيد	…	…	…	…
3	الثاني	قريب	…	…	…	…

لو القريب يعمل والبعيد يفشل على نفس الجهاز، افحص التغطية والتداخل. لو الجهازان يفشلان قريبًا مع إشارة قوية، افحص الاتصال بعد نقطة الوصول. نتيجة واحدة ليست إثباتًا نهائيًا؛ أعد نفس المقارنة وحدد ما تغير.

إعدادات عنوان الجهاز والاتصال

إعدادات عنوان الجهاز والاتصال

اقرأ إعداداتك قبل الحساب

على Windows اكتب `ipconfig /all` في PowerShell. ابحث عن المحول المتصل، واقرأ أربعة أشياء فقط: IPv4 Address عنوان الجهاز، Subnet Mask قناع يحدد جزء الشبكة، Default Gateway الراوتر الذي نرسل له عند غياب طريق أدق، وDNS Servers خوادم سؤال أسماء المواقع. لا تغيّر الإعدادات في التجربة.

DHCP اختصار Dynamic Host Configuration Protocol: طريقة يحصل بها الجهاز على إعدادات من خادم لمدة قابلة للتجديد. مثال: عنوان192.168.1.20، قناع255.255.255.0، بوابة192.168.1.1؛ القيم الفعلية على جهازك قد تختلف. الحساب التفصيلي والتوجيه والإصدار السادس لكل منها درس تالٍ.

DHCP

يمنح DHCP الجهاز إعداداته آليًا: IP (Internet Protocol؛ قواعد عنونة الحزم وتوجيهها بين الشبكات، وعنوان IP رقم يحدد واجهة في هذا السياق) Address وSubnet Mask وDefault Gateway وDNS Servers ومدة الإيجار. تبدأ العملية غالبًا برسائل Discover، سؤال عن خادم إعدادات، ثم Offer، عرض إعدادات، ثم Request، طلب استخدام العرض المختار، ثم Acknowledgement، تأكيد الإعدادات. الإيجار مؤقت ويمكن تجديده، لذلك لا تفترض أن عنوان جهاز العميل ثابت دائمًا.

Private IP وPublic IP وNAT

تستخدم الشبكات المحلية نطاقات Private مثل `10.0.0.0/8` و`172.16.0.0/12` و`192.168.0.0/16`. لا توجه هذه العناوين مباشرة على الإنترنت. يغير NAT عنوان المصدر عند خروج الاتصال، ويستخدم PAT المنافذ لتمييز اتصالات أجهزة كثيرة خلف Public IP واحد.

Port Forwarding قاعدة تسمح لاتصال وارد بالانتقال إلى جهاز داخلي محدد. أما CGNAT (Carrier-Grade NAT؛ ترجمة عناوين ينفذها مزود الخدمة لعدة مشتركين) فيطبقه مزود الخدمة على عدة عملاء، وقد يمنع استقبال الاتصالات المباشرة حتى لو أعددت Router المنزل.

تدريب عملي

▼ الجهاز عنده IP لكن مفيش Default Gateway. إيه اللي يشتغل؟

غالبًا يقدر يكلم نفس الـSubnet، لكنه لا يملك طريقًا للشبكات الخارجية من غير Route أخرى.

▼ ليه NAT مش Firewall؟

NAT يترجم عناوين وPorts؛ Firewall يطبق سياسة السماح والمنع. ممكن NAT تكون موجودة مع قواعد ضعيفة أوPort Forwarding مفتوح.

تأكد من فهمك

01 **كيف يقرر الجهاز أن الوجهة داخل شبكته؟**

> **الإجابة:** يطبق الـSubnet Mask على عنوانه وعنوان الوجهة ويقارن Network Prefix.

02 **ما الفرق بين NAT وPort Forwarding؟**

> **الإجابة:** NAT يترجم الاتصالات، بينما Port Forwarding يضيف قاعدة لاتصال وارد نحو مضيف وخدمة داخلية.

الخطوة التالية

كمّل في احسب حدود الشبكة من عنوانها بعد تنفيذ التجربة هنا.

كمّل في اختيار الطريق وقراءة عناوين IPv6 بعد تنفيذ التجربة هنا.

احسب حدود الشبكة من عنوانها

احسب حدود الشبكة من عنوانها

جهازان آخر عنوانيهما .20 و.70: هل هما في نفس الشبكة؟ العنوان وحده مش كفاية؛ نحتاج القناع، أي البتات التي تحدد جزء الشبكة. هنحسب نفس المثال مع /24 ثم /26، بعد مراجعة تحويل العدد إلى ثنائي.

Subnet Mask وCIDR

Subnet — شبكة فرعية مجموعة عناوين تشترك في بداية معينة. **Subnet mask — قناع الشبكة** يحدد البتات التي تمثل هذه البداية. **CIDR، Classless Inter-Domain Routing** يكتب طول البداية بعد شرطة مائلة بدل الاعتماد على فئات عناوين قديمة. راجع الثنائي والبايت قبل الحساب.

IPv4 عنوان 32 بتًا، مكتوب في أربع مجموعات؛ كل مجموعة 8 بتات وقيمتها من 0 إلى 255. في /24 نخصص 24 بتًا للشبكة، فيتبقى 8 بتات، أي 256 عنوانًا إجمالًا. في شبكة IPv4 عادية بهذا الحجم، أول عنوان للشبكة وآخر عنوان للبث العام، فيتبقى 254 عنوان مضيف. دي مش قاعدة نطرح بها اثنين من كل شبكة؛ /31 للوصلات بين طرفين و/32 لحالة عنوان واحد لهما استخدامات مختلفة.

مثال كامل على /24 و/26

`192.168.1.0/24` قناعه `255.255.255.0`: عنوان الشبكة .0 والبث .255 وعناوين الأجهزة المعتادة .1 إلى .254. الجهاز `192.168.1.20/24` يرى `192.168.1.70` محليًا.

في `192.168.1.0/26` يتبقى 6 بتات: `2⁶=64` عنوانًا. القناع `255.255.255.192`؛ آخر بايت 192 يساوي `11000000`، أي أول بتين للشبكة. النطاق .0 إلى .63؛ الأجهزة .1 إلى .62 والبث .63. العنوان .70 يقع في الشبكة التالية .64/26، فيحتاج مسارًا إليها.

`AND — البتية و` تعطي 1 فقط لو البتان 1. تطبيق القناع على آخر بايت من .20:

```
20:   00010100
192:  11000000
AND:  00000000  → 0. الشبكة
```

آخر بايت من .70 هو `01000110`، وبعد القناع يعطي `01000000` أي .64؛ إذن مختلف. **تدريب وحل:** .62 محلي للجهاز .20/26، و.65 غير محلي. نحتاج المسار وقاعدة السماح، لا تغيير MAC (Media Access Control؛ عنوان تستخدمه واجهة الشبكة للتواصل على الوصلة المحلية) إلى عنوان الخادم البعيد.

الكتابة `/24` تعني أن أول 24 بت تحدد الشبكة والبقية للمضيفين. في `192.168.1.0/24` يكون نطاق العناوين داخل شبكة واحدة، ويحدد الحساب الثنائي Network Address وBroadcast Address والعناوين القابلة للاستخدام.

لا يكفي حفظ `/24`. افهم أن زيادة Prefix تقلل عدد عناوين المضيفين وتنتج شبكات أكثر. يستخدم الجهاز الـMask ليقرر هل يرسل مباشرة داخل LAN (Local Area Network؛ شبكة أجهزة في مساحة محلية زي بيت أو مكتب) أم إلى Default Gateway.

اختيار الطريق وقراءة عناوين IPv6

اختيار الطريق وقراءة عناوين IPv6

بعد تحديد هل الوجهة محلية، نحتاج نعرف الجهاز التالي في الطريق. **التوجيه — Routing** اختيار طريق للحزمة من جدول. **Next hop — النقلة التالية** الجهاز الذي نستلمه له الآن. هنبدأ بجدول صغير قبل توسيع العناوين إلى IPv6.

Routing

تحتوي Routing Table على شبكات وNext Hop وواجهة وربما Metric. يختار Router المسار الأكثر تحديدًا وفق Longest Prefix Match. وإذا لم توجد قاعدة أدق يستخدم Default Route مثل `0.0.0.0/0` .

IPv6

يستخدم IPv6 عناوين 128 بت مثل `2001:db8::1` . يوفر مساحة هائلة ويستخدم Neighbor Discovery بدل ARP. توجد عناوين Link-local تبدأ غالبًا بـ `fe80::` ، وLoopback هو `::1` . لا يعني وجود IPv6 اختفاء الجدران النارية أو الحاجة إلى سياسات وصول واضحة.

اختيار الطريق وقراءة IPv6

Prefix — بادئة العنوان البتات المشتركة التي تحدد الشبكة. **Next hop — النقلة التالية** الجهاز التالي الذي نرسل إليه. **Metric — كلفة مسار** تساعد الاختيار بين مسارات مرشحة وفق النظام والسياسة؛ أطول بادئة مطابقة أسبق من مقارنة الكلفة بين مسارات مختلفة التحديد في جدول توجيه عادي.

تدريب محلول: الجدول يحتوي `0.0.0.0/0 → A` ، و `192.168.0.0/16 → B` ، و `192.168.1.0/24 → C` . الوجهة 192.168.1.70 تختار C، و192.168.2.8 تختار B، و203.0.113.8 تختار A. اخترنا أطول تطابق لا أصغر رقم راوتر.

IPv6 يكتب 128 بتًا عادة في ثماني مجموعات سداسية عشرية، كل منها 16 بتًا. يمكن حذف أصفار بداية المجموعة، و `::` تختصر مجموعات صفرية متتالية مرة واحدة في العنوان. `2001:db8::1` عنوان أمثلة، و `::1` للجهاز نفسه. نطاق العناوين المحلية للوصلة هو `fe80::/10` ؛ لا يخرج تلقائيًا للإنترنت.

SLAAC، Stateless Address Autoconfiguration تكوين عنوان تلقائي اعتمادًا على معلومات يعلنها الراوتر. **Router Advertisement — إعلان الراوتر** رسالة معلومات الشبكة المحلية؛ تعرّف الأجهزة أيضًا بالراوتر الافتراضي. **DHCPv6** نسخة إعدادات IPv6 من بروتوكول التهيئة، قد تضيف عنوانًا أو إعدادات حسب الشبكة؛ المسار الافتراضي لا يأتي منها بالطريقة المعتادة لـDHCPv4. اكتشاف الجيران NDP (Neighbor Discovery Protocol؛ اكتشاف الجيران والراوترات وإعدادات الوصلة في IPv6) يحل دور معرفة عناوين الوصلة. يبقى **Firewall — جدار ناري** بقواعد سماح ومنع مطلوبًا؛ NAT لا يغني عنه.

الجهاز محتاج أكتر من IP علشان يتصل

لما تتصل بالـWi-Fi (اتصال لاسلكي بالشبكة المحلية؛ لا يضمن اتصالها بالإنترنت)، جهازك محتاج عنوانه، وحجم الشبكة المحلية، والـDefault Gateway (البوابة الافتراضية: الراوتر الذي نرسل له الوجهات التي لا نملك لها مسارًا أدق)، وDNS (Domain Name System؛ نظام يجيب عن أسئلة أسماء النطاقات، ومنها عناوينها) Resolver. غالبًا DHCP (Dynamic Host Configuration Protocol؛ خدمة تمنح الجهاز إعدادات الشبكة تلقائيًا) بيوفرهم تلقائيًا:

```
Discover → Offer → Request → Acknowledge
```

الـSubnet Mask أوCIDR (Classless Inter-Domain Routing؛ كتابة طول بداية الشبكة بعد شرطة مائلة) يحدد العناوين المحلية. لو الوجهة Local، الجهاز يتواصل معها على الـLink. لو خارجها، يرسل للـGateway. الراوتر يقرأ جدول Routes ويختار أكثر Prefix تحديدًا يطابق الوجهة.

NAT (Network Address Translation؛ تغيير عناوين IP أثناء مرور الحزم بين شبكات) مش هو Routing. Routing يختار الطريق، أما NAT فيغيّر عناوين أوPorts (منفذ منطقي برقم يميز جهة استقبال في بروتوكول النقل) أثناء المرور. في البيت، PAT (Port Address Translation؛ تمييز اتصالات أجهزة خلف عنوان واحد باستخدام المنافذ) يسمح لأجهزة كثيرة تشارك Public IPv4 (Internet Protocol version 4؛ إصدار عناوين الشبكة ذي32 بتًا) واحدة باستخدام Ports. ده مش بديلًا عن Firewall، ولا يجعل الشبكة آمنة تلقائيًا.

IPv6 (Internet Protocol version 6؛ إصدار عناوين الشبكة ذي128 بتًا) وسّع مساحة العناوين ويستخدم Neighbor Discovery بدل ARP (Address Resolution Protocol؛ سؤال عن عنوان MAC المقابل لعنوان IPv4 على الشبكة المحلية). شكل العنوان اتغير، لكن Prefix وRoute وNext Hop ما زالوا أساس القرار.

كيف يتحول اسم الموقع إلى عنوان؟

افهم لماذا نحتاج الاسم والعنوان، وكيف يصل DNS من الـcache إلى الإجابة الموثوقة.

في الدرس السابق احتاج العميل إلى عنوان الخادم قبل بدء الاتصال. هنا سنفصل بين شيئين: **IP (Internet Protocol)؛ قواعد عنونة الحزم وتوجيهها بين الشبكات، وعنوان IP رقم يحدد واجهة في هذا السياق) هو العنوان الذي يمكن توجيه الحزم إليه، وDNS (Domain Name System؛ نظام يجيب عن أسئلة أسماء النطاقات، ومنها عناوينها) هو النظام الذي يجد العنوان المناسب للاسم.**

اقرأ شكل نتيجة البحث

شغّل `nslookup example.com` . ده **مثال توضيحي لشكل** الناتج؛ القيم الحقيقية تتغير:

```
Server:  router.home
Address: 192.168.1.1
Non-authoritative answer:
Name:    example.com
Address: 203.0.113.10
```

أول Server وAddress يخصان الجهة التي أجابت عن السؤال، مش الموقع. Name وAddress بعد Answer يخصان الاسم المطلوب. العنوان203.0.113.10 مخصص للتوثيق هنا، فلا تتوقع الوصول للموقع به. **Non-authoritative** تعني أن الإجابة ليست مباشرة من الخادم صاحب بيانات النطاق؛ لا تعني وحدها أن الإجابة خاطئة. قد ترى أكثر من عنوان أو إصدارًا سادسًا بدل هذه القيم.

الفكرة: دفتر عناوين موزّع

البشر يتذكرون `example.com` أسهل من رقم، والخدمة قد تنقل خوادمها من غير تغيير الاسم. لكن DNS ليس ملفًا عالميًا واحدًا؛ هو نظام موزّع، وكل جهة تجيب عن الجزء المسؤول عنه.

عنوان IP يعرّف واجهة على شبكة ويسمح للراوترات بتوجيه الحزم:

- `192.168.1.20` : مثال IPv4 (Internet Protocol version 4؛ إصدار عناوين الشبكة ذي32 بتًا) خاص داخل شبكة محلية.
- `203.0.113.10` : عنوان IPv4 مخصص للأمثلة والتوثيق.
- `2001:db8::10` : عنوان IPv6 (Internet Protocol version 6؛ إصدار عناوين الشبكة ذي128 بتًا) مخصص للأمثلة.
- `127.0.0.1` وloopback : `::1` ، أي الجهاز نفسه.
- `localhost` : اسم يُحل عادة إلى loopback.

العنوان الخاص داخل المنزل ليس عنوانك العام بالضرورة؛ الراوتر يجري NAT (Network Address Translation؛ تغيير عناوين IP أثناء مرور الحزم بين شبكات) غالبًا. وIP لا يحدد شخصًا أو جهازًا بصورة دائمة، فقد يتغير أو تشترك فيه أجهزة.

ما الذي يعيده DNS؟

يعيد DNS سجلات ذات أنواع مختلفة:

Record	معناه
`A`	اسم إلى IPv4
`AAAA`	اسم إلى IPv6
`CNAME`	اسم مستعار يشير إلى اسم آخر
`MX`	خوادم استقبال البريد للنطاق
`TXT`	نصوص تحقق وسياسات
`NS`	خوادم الأسماء المسؤولة عن zone (منطقة DNS: جزء من الأسماء تديره جهة وتنشر سجلاته)

قد يعيد الاسم أكثر من IP لتوزيع الحمل أو اختيار موقع قريب. DNS يجد وجهة محتملة؛ لا يحمل HTML (Hypertext Markup Language؛ لغة وصف بنية الصفحة وعناصرها) ولا يشغّل PHP (اسم لغة برمجة تستخدم كثيرًا لمعالجة طلبات الويب على الخادم).

رحلة الاستعلام عند غياب الـcache

1. يفحص المتصفح إجاباته المخزنة.
2. يفحص نظام التشغيل ملف hosts والـOS cache (نسخة محفوظة لتقليل تكرار القراءة أو الحساب).
3. يسأل **recursive resolver (خادم DNS ينفذ البحث عن الإجابة بالنيابة عن جهازك)** يوفره الـISP (Internet Service Provider؛ شركة أو جهة توفر الاتصال بالإنترنت) أو المؤسسة أو خدمة عامة.
4. إن لم يملك الإجابة، يسأل resolver خادم **Root**: من يعرف `.com`؟
5. يرشده Root إلى خوادم **TLD (Top-Level Domain؛ الجزء الأعلى من اسم النطاق، زي com)** الخاصة بـ`.com`.
6. يرشده TLD إلى **authoritative name server (خادم الأسماء المسؤول عن نشر الإجابات المعتمدة لجزء من نظام الأسماء)** المسؤول عن النطاق.
7. يعيد authoritative server (الخادم المسؤول عن بيانات النطاق في هذه المرحلة) السجل النهائي.
8. يعيد resolver الإجابة للعميل ويخزنها حسب TTL (Time To Live؛ عمر النسخة المخزنة في DNS، وفي IPv4 حقل منفصل يحد عدد مرات تمرير الحزمة).

```
Browser/OS cache
      ↓ cache miss
Recursive resolver
      ↓
Root → TLD → Authoritative
                  ↓
               IP + TTL
```

⚠

تصحيح مهم

المتصفح لا يسأل Root وTLD بنفسه عادة، والرحلة الكاملة لا تتكرر لكل طلب. الـrecursive resolver يقوم بها عند غياب إجابة صالحة من الـcache.

TTL: السرعة مقابل سرعة التغيير

الـTTL عدد ثوانٍ يسمح خلالها بتخزين الإجابة. TTL طويل يقلل الاستعلامات لكنه يجعل تغيير السجل أبطأ في الظهور عند بعض العملاء. TTL قصير يسرّع الانتقال بعد انتهاء القيم المخزنة، لكنه يزيد عدد الاستعلامات.

«انتشار DNS» ليس زرًا ينتظر العالم كله بعده مدة ثابتة. كل cache يحتفظ بالإجابة القديمة حتى تنتهي مدتها، وقد تطبق بعض الأنظمة سياسات إضافية.

تجربة عملية وقراءة النتيجة

```
nslookup -type=A example.com
nslookup -type=AAAA example.com
nslookup -type=MX example.com
```

نتيجة `A` ستكون شبيهة بالآتي، مع اختلاف العنوان والـresolver:

```
Name:    example.com
Address: 93.184.216.34
```

- `-type=A` يطلب IPv4 تحديدًا.
- `Name` هو الاسم الذي جرى حله.
- `Address` هو أحد العناوين المعادة، وليس ضمانًا بأن خدمة HTTP (Hypertext Transfer Protocol؛ قواعد طلب الموارد والرد عليها في الويب) تعمل.
- طلب `AAAA` قد يعيد IPv6 أو لا يجد سجلًا.
- طلب `MX` يبحث عن خوادم البريد، لا خادم الويب.

أخطاء شائعة

- اعتبار DNS اتصالًا بالخادم؛ هو خطوة لاكتشاف العنوان.
- افتراض أن `localhost` يعني جهازًا بعيدًا؛ يعني الجهاز الذي ينفذ الأمر.

- تعديل سجل ثم توقع ظهوره فوريًا رغم وجود caches وTTL.
- استخدام IP قديم مباشرة وتجاوز الاسم، ثم كسر TLS (Transport Layer Security؛ قواعد حماية الاتصال بالتشفير والتحقق) أو الاستضافة التي تعتمد على اسم المضيف.
- اعتبار عدم وجود `AAAA` دليلًا على أن `A` لا يعمل؛ النوعان مستقلان.

مسائل عملية

▼ غيّرت سجل A وما زال زميلك يصل إلى الخادم القديم. هل التعديل فشل؟

ليس بالضرورة. افحص الإجابة من الـauthoritative server ثم قيمة TTL والـresolver الذي يستخدمه الزميل. قد تكون لديه إجابة قديمة صالحة في cache.

▼ نجح `nslookup` لكن `curl` فشل. ماذا أثبت الاختبار؟

أثبت أن مسار DNS أعاد عنوانًا فقط. قد يكون الفشل لاحقًا في التوجيه أو TCP (Transmission Control Protocol؛ نقل بايتات بترتيب مع معالجة الفقد، مع احتمال فشل الاتصال) أو TLS أو HTTP أو الخدمة نفسها.

▼ لماذا لا نستبدل DNS بكتابة IP في كل تطبيق؟

لأن العنوان قد يتغير، وقد توجد عدة عناوين، كما تعتمد TLS والاستضافة الافتراضية على اسم المضيف. الاسم يفصل هوية الخدمة المنطقية عن موقعها الحالي.

تأكد من فهمك

1. من يسأل Root وTLD عادة؟ **الـrecursive resolver عند غياب الإجابة من cache، لا المتصفح مباشرة.**
2. ماذا يحدث عند انتهاء TTL؟ **تصبح الإجابة المخزنة غير صالحة ويحتاج العميل أو resolver إلى استعلام جديد.**
3. هل يعني نجاح DNS أن الموقع سيفتح؟ **لا؛ هو يحدد عنوانًا، وما زالت مراحل الاتصال وTLS وHTTP والتطبيق قائمة.**

الخطوة التالية

كمّل في أخطاء البحث عن العناوين وحماية DNS بعد تنفيذ التجربة هنا.

أخطاء البحث عن العناوين وحماية DNS

أخطاء البحث عن العناوين وحماية DNS

بعد ما قرأت نتيجة بحث ناجحة، هتفرق بين اسم غير موجود وخادم لا يستطيع إكمال البحث، ثم بين إخفاء الاستعلام والتحقق من صحة بياناته.

إجابة غير موجودة وحماية DNS

Negative caching — تخزين الإجابة السلبية معناه حفظ رد أن الاسم أو نوع السجل غير موجود مدة محددة، بدل السؤال نفسه كل مرة. لذلك إضافة سجل جديد قد لا تظهر فورًا لعميل عنده رد سلبي محفوظ.

DNSSEC (Domain Name System Security Extensions)؛ امتدادات توقع بيانات DNS للتحقق من سلامتها ومصدرها)، Domain Name System Security Extensions يضيف توقيعات لسجلات DNS ليتحقق العميل الداعم من مصدرها وسلامتها عبر سلسلة ثقة. لا يشفّر السؤال والرد. أما **DoH (DNS over HTTPS)؛ نقل سؤال DNS وردّه عبر HTTPS)، DNS over HTTPS (Hypertext Transfer Protocol Secure؛ طلبات ويب عبر اتصال محمي بالتشفير والتحقق من هوية الطرف الآخر)** و**DoT (DNS over TLS)؛ نقل سؤال DNS وردّه عبر TLS)، DNS over TLS** فيحميان الاتصال بين العميل وخادم الاستعلام بتشفيره. حماية هذا الطريق لا تثبت أمان الموقع الذي أعاد DNS عنوانه.

تدريب محلول: اسم له سجل A، لكن سؤال AAAA لا يعيد عنوانًا. نستنتج أن عنوان الإصدار السادس لم يظهر في هذا الاستعلام؛ لا نستنتج تعطل سجل الإصدار الرابع أو الموقع كله. اسأل كل نوع مستقلًا.

لمشاهدة تفاصيل قد تشمل TTL، أي مدة حفظ الإجابة بالثواني، استخدم `nslookup -debug -type=A example.com`. راجع حقول answer، أي الإجابة، لأن أداة الاستعلام قد تعرض أقسامًا أخرى أيضًا. كرر مع AAAA وMX وNS؛ الإجابات والوقت تختلف حسب النطاق وخادم الاستعلام.

TCP: الاتصال من أول SYN لحد الإغلاق

افهم اتصال TCP والـbyte stream والترتيب وإعادة الإرسال والتحكم في التدفق والازدحام والإغلاق، وجرّب اتصالًا محليًا بـPHP.

أنت بتبعت ملف أو طلب لخادم. الطريق ممكن يضيّع جزء من البيانات، أو يوصل جزء متأخر، أو يكرر بيانات. برنامجك محتاج يشوف البايتات بالترتيب اللي اتبعتت بيه، ويعرف إن الاتصال فشل لو النقل ما قدرش يكمل. دي المشكلة اللي TCP بيحلها.

محتاج تعرف **IP**: بروتوكول عنونة وتوجيه الحزم بين الأجهزة. و**Port، منفذ**: رقم بيساعد نظام التشغيل يحدد نقطة استقبال البيانات داخل الجهاز. DNS بيبحث عن العنوان؛ TCP نفسه ممكن يتصل بعنوان مباشر من غير بحث DNS.

TCP بيقدم إيه لبرنامجك؟

Transmission Control Protocol بروتوكول نقل بيقدم **Byte stream**: تدفق بايتات مرتب بين طرفين. البايت وحدة من 8 بتات. TCP بيرتب البيانات ويعالج الفقد والتكرار داخل الاتصال، لكنه مش بيحافظ على حدود رسائل التطبيق.

لو أرسلت النصين `Hello` و`World` في استدعاءين، المستقبل ممكن يقرأ `HelloWorld` مرة واحدة، أو يقرأ أجزاء أصغر. عدد مرات الكتابة مش لازم يساوي عدد مرات القراءة. التطبيق هو اللي يحدد نهاية الرسالة، مثلًا بنهاية سطر أو بطول مكتوب قبل المحتوى.

الاعتمادية هنا مش وعد إن الشبكة عمرها ما هتقع. فيه مهلات وحدود وموارد؛ لو ما قدرش يكمل، الاتصال ممكن يفشل. وكلمة «اتصال» مش معناها إن فيه سلك محجوز ليك على طول الطريق؛ الطرفان بيحتفظوا بحالة مشتركة للنقل.

مين متصل بمين؟

في حالة TCP المعتادة، الاتصال بيتحدد بعنوان ومنفذ كل طرف:

```
Client 127.0.0.1:51024  <-->  Server 127.0.0.1:9099
```

عنوان `127.0.0.1` بيرجع لنفس الجهاز. العميل عادة بياخد منفذًا مؤقتًا من النظام، والخادم بيستنى على منفذ معروف. عميل تاني يقدر يكلم نفس الخادم بمنفذ مصدر مختلف. الأرقام هنا مثال؛ المنفذ المؤقت الفعلي ممكن يختلف.

Socket هي واجهة بيستخدمها البرنامج للتواصل مع طبقة الشبكة. الخادم بيعمل **Listen**، يعني يستعد لقبول اتصالات، وبعدها **Accept** يرجع له موردًا يمثل اتصالًا مقبولًا. ده مختلف عن رسالة التطبيق اللي هتتبعت بعد كده.

الاتصال بيبدأ إزاي؟

في البداية المعتادة بتحصل مصافحة ثلاثية، **Three-way handshake**:

```
Client                              Server
SYN, sequence=x  ------------------->
                 <------------------- SYN+ACK, sequence=y, ack=x+1
ACK, ack=y+1   ---------------------->
              الاتصال يقدر ينقل بيانات
```

SYN بيبدأ مزامنة أرقام التتابع. **ACK** بيأكد التقدم المتوقع من الطرف الآخر. كل اتجاه له أرقام تتابع مستقلة؛ الطرفان مش بيتشاركوا عداد بايتات واحد. SYN نفسه بيستهلك رقمًا في مساحة التتابع، علشان كده التأكيد في الرسم زاد واحدًا.

نجاح المصافحة معناه إن نقطة TCP قبلت الاتصال. لسه ما يثبتش إن المستخدم مسموح له يدخل أو إن قاعدة البيانات شغالة. المصادقة ومنطق التطبيق بييجوا فوق النقل.

ترتيب البايتات وتأكيد وصولها

تخيل نطاق بيانات يبدأ عند الرقم 1000، وفيه `ABCDEF`. أول ثلاثة بايتات هي 1000 و1001 و1002، والثلاثة اللي بعدهم تبدأ عند 1003. رقم التتابع بيحدد موضع البايت، مش «رقم الحزمة الرابعة» ولا «رقم طلب الشراء».

Cumulative ACK المعتاد بيشير لرقم البايت التالي المتوقع: تأكيد 1003 معناه إن البايتات السابقة وصلت بترتيب متصل. لو جزء ناقص وفيه بيانات بعده، TCP ممكن يحتفظ باللي وصل ويستعيد الناقص، وبعدها يسلّم للتطبيق تدفقًا مرتبًا.

```
وصل:        ABC     [جزء ناقص]     GHI
للتطبيق:    ABC
بعد التعويض: بالترتيب ABCDEF GHI
```

Retransmission معناها إعادة إرسال بيانات استُنتج إنها ضاعت، اعتمادًا على المؤقتات ومعلومات التأكيد. استنتاج الفقد ممكن يكون غلط بسبب تأخير أو إعادة ترتيب؛ التنفيذ بيتعامل مع الاحتمالات دي. آليات زي **SACK** تسمح للطرف المستقبل يبلغ عن نطاقات إضافية وصلت، فتساعد المرسل يعرف الفجوات.

لو بيانات جديدة وصلت لكن فيه فجوة قبلها، التطبيق ممكن يستنى سد الفجوة. ده اسمه **Head-of-line blocking** في تدفق TCP: البيانات موجودة لكن مش جاهزة للتسليم المرتب.

حجم الجزء بيتحدد إزاي؟

بيانات التطبيق بتنزل لطبقات النقل، بعدها IP، بعدها الوصلة المحلية:

```
Application bytes → TCP segment → IP packet → Ethernet/Wi-Fi frame
```

كل طبقة بتضيف **Header**: معلومات علشان الطرف المقابل يعرف يتعامل مع الوحدة. TCP فيه المنافذ وأرقام التتابع والأعلام؛ IP فيه العناوين ومعلومات التوجيه. عنوان MAC يخص الوصلة المحلية، والراوتر يختار الخطوة التالية لحزمة IP.

MTU أقصى حجم حمولة طبقة أعلى تقدر الوصلة تحملها. **MSS** أقصى مقدار بيانات TCP يعلن الطرف إنه مستعد يستقبله في مقطع، باستثناء رؤوس TCP وIP. MSS وMTU مش نفس الرقم، ومعلومات المسار والخيارات بتأثر على التقسيم الفعلي. نقل رسالة كبيرة مش معناه إرسال حزمة واحدة بنفس حجمها.

Path MTU Discovery بيحاول يحدد الحجم المناسب للمسار. لو رسائل التحكم المرتبطة بالحجم اتحجبت، ممكن اتصال صغير ينجح ونقل بيانات أكبر يتعطل. ده احتمال تشخيصي، مش حكم تلقائي على كل رفع ملف فاشل.

قيمة **Checksum** بتساعد تكشف فسادًا عرضيًا في البيانات، لكنها مش حماية ضد مهاجم يزوّر بيانات. TCP لوحده مش تشفير؛ حماية محتوى الاتصال تحتاج بروتوكولًا زي TLS.

نافذة المستقبل وازدحام الشبكة

فيه مشكلتان مختلفتان:

- المستقبل ممكن يكون أبطأ من المرسل. **Flow control** يستخدم نافذة الاستقبال المعلنة، `rwnd`، علشان ما يغرقش المستقبل.
- الطريق نفسه ممكن يكون مزدحمًا. **Congestion control** يضبط كمية البيانات الموجودة في الطريق بنافذة ازدحام عند المرسل، `cwnd`.

الكمية المسموح بوجودها من غير تأكيد بتتأثر بالأصغر من النافذتين، مع خصم البيانات اللي لسه في الطريق. زيادة سرعة المعالج مش هتحل تلقائيًا اختناق شبكة أو مستقبل مش بيقرأ.

الخوارزميات المختلفة ليها سياسات مختلفة. لا تفترض إن كل فقد معناه نفس رد الفعل أو إن كل نظام بيستخدم نفس خوارزمية الازدحام. المهم تفهم الفصل بين قدرة المستقبل وقدرة الطريق.

الاتصال بينتهي إزاي؟

كل اتجاه يقدر ينتهي لوحده. **FIN** معناه «مش هبعت بيانات جديدة في الاتجاه ده». الطرف التاني يؤكد FIN، ولسه ممكن يبعت بياناته الباقية. الإغلاق المعتاد بيتضمن FIN وACK لكل اتجاه، وممكن بعض الرسائل تندمج؛ مش لازم تشوف أربع حزم منفصلة دائمًا.

Half-close يعني إنهاء الإرسال مع استمرار الاستقبال. أما **RST** فيعيد ضبط الاتصال بصورة مفاجئة بدل الإغلاق المنظم. برنامجك لازم يتعامل مع EOF أو خطأ، ومش يفترض إن كل انتهاء يعني اكتمال الرسالة المطلوبة.

TIME_WAIT حالة انتظار بعد الإغلاق المنظم عند الطرف المناسب، تساعد في التعامل مع المقاطع المتأخرة وإعادة تأكيد نهاية الاتصال. وجودها طبيعي. حذفها أو تقليلها عشوائيًا علشان «ننظف الاتصالات» ممكن يغيّر سلوك النظام بصورة غير محسوبة.

Keepalive في TCP ممكن يساعد يكتشف بعض الاتصالات الخاملة اللي مات طرفها حسب الإعدادات. مش هو نفسه إعادة استخدام اتصال HTTP، ومش بديل عن مهلة عمل واضحة لطلب التطبيق.

تجربة محلية بـPHP: رسالتك مش هي اتصالك

هتشغّل ملف خادم وملف عميل في طرفيتين. محتاج PHP CLI. افتح مجلد تدريب فاضي على جهازك، واستعمل منفذ `9099` محليًا؛ مش محتاج تفتح المنفذ للإنترنت.

احفظ الملف باسم `tcp-server.php`:

```php
<?php
declare(strict_types=1);

$server = stream_socket_server('tcp://127.0.0.1:9099', $code, $message);
if ($server === false) {
    throw new RuntimeException($message, $code);
}
echo 'Listening', PHP_EOL;
$client = stream_socket_accept($server, 15);
if ($client === false) {
    fclose($server);
    throw new RuntimeException('No client connected');
}
stream_set_timeout($client, 5);
$line = fgets($client, 128);
if ($line !== "Hello\n") {
    fclose($client);
    fclose($server);
    throw new RuntimeException('Expected one short newline-terminated message');
}
$response = "Received: Hello\n";
while ($response !== '') {
    $written = fwrite($client, $response);
    if ($written === false || $written === 0) {
        throw new RuntimeException('Write failed');
    }
    $response = substr($response, $written);
}
fclose($client);
fclose($server);
```

شغّل الخادم:

```
php tcp-server.php
```

هيظهر `Listening` ويستنى عميلًا لحد 15 ثانية. بعد قبول العميل، القراءة لها مهلة 5 ثوانٍ. احفظ الملف التاني باسم `tcp-client.php` وشغّله من طرفية تانية أثناء الانتظار:

```php
<?php
declare(strict_types=1);

$socket = stream_socket_client('tcp://127.0.0.1:9099', $code, $message, 5);
if ($socket === false) {
    throw new RuntimeException($message, $code);
}
stream_set_timeout($socket, 5);
$request = "Hello\n";
while ($request !== '') {
    $written = fwrite($socket, $request);
    if ($written === false || $written === 0) {
        throw new RuntimeException('Write failed');
    }
    $request = substr($request, $written);
}
$response = fgets($socket, 128);
fclose($socket);
if ($response !== "Received: Hello\n") {
    throw new RuntimeException('Missing or incomplete response');
}
echo $response;
```

```
php tcp-client.php
```

الناتج المتوقع عند العميل:

```
Received: Hello
```

`fgets` هنا بتقرأ لحد نهاية سطر أو حد القراءة أو انتهاء المورد. اخترنا نهاية السطر علشان تحدد **رسالة التطبيق**. حلقات `fwrite` موجودة لأن الكتابة قد تقبل عدد بايتات أقل من المطلوب؛ لازم نكمّل الباقي. المثال بيرفض رسالة طويلة أو ناقصة؛ مش بروتوكول إنتاج يقبل أي حجم.

الخادم يقبل عميلًا واحدًا ثم ينتهي. العميل يبدأ اتصال TCP، وبعده يرسل الرسالة، وبعده يستقبل رد التطبيق. فرق بين نجاح الاتصال وبين تطابق الرد المتوقع. المثال مشفرش البيانات؛ على شبكة غير محلية محتاج تصميم حماية مناسب.

تتأكد من إيه وقت العطل؟

لو ظهر `Connection refused`، افحص إن فيه برنامج بيستمع على العنوان والمنفذ المطلوبين. رفض الاتصال ممكن كمان يكون نتيجة سياسة شبكة بترد بالرفض. لو فيه Timeout، السبب ممكن يكون إسقاط مرور أو مشكلة توجيه أو طرف متأخر؛ الرسالة وحدها مش بتحسم السبب.

في PowerShell تقدر تراقب اتصالًا محليًا أثناء تشغيل المثال:

```
Get-NetTCPConnection -LocalPort 9099 -ErrorAction SilentlyContinue
```

وعلى Linux:

```
ss -tn
```

الاتصال قصير، فممكن ينتهي قبل ما تلحق تشوفه. استخدم **Packet capture** لو محتاج تتبع المصافحة والتأكيد والإغلاق؛ ده تسجيل لحركة الشبكة وقد يكشف بيانات حساسة، فخليه في معملك المحلي. `ping` يستخدم ICMP، وبالتالي نجاحه مش إثبات إن خادم TCP على منفذ معين بيقبل اتصالات.

TCP وصل البيانات، هل الطلب اتنفذ؟

TCP ACK يثبت تقدم النقل، مش حفظ الطلب في قاعدة البيانات. وممكن الخادم ينفذ دفعًا، وبعدها ينقطع الاتصال قبل ما العميل يستلم الرد. العميل مش هيعرف من انقطاع الاتصال وحده هل الدفع حصل.

إعادة نفس الطلب ممكن تعمل أثرًا تجاريًا تاني. **Idempotency** سياسة تطبيق تمنع تضاعف الأثر المقصود عند إعادة نفس العملية، مثل مفتاح طلب فريد ونتيجة محفوظة. TCP يعالج تكرار بايتات داخل الاتصال؛ التطبيق مسؤول عن تكرار طلبات كاملة عبر اتصالات مختلفة.

راجع فهمك

▼ كتبت رسالتين، لكن المستقبل قرأهما مع بعض. ده خطأ في TCP؟
لا. TCP بيقدم byte stream. محتاج تحدد حدود الرسالة بنهاية سطر أو طول أو صيغة بروتوكول واضحة.
▼ جالك ACK وبعده فشل الدفع. فيه تناقض؟
لا. ACK يخص استلام طبقة النقل. الدفع له تحقق ومعاملة ورد تطبيق مستقل.
▼ المصافحة نجحت لكن رفع ملف كبير اتوقف. تبدأ منين؟
افصل الفرضيات: هل التطبيق بيقرأ؟ هل فيه مهلة؟ هل فيه إعادة إرسال أو مشكلة Path MTU؟ اجمع دليلًا من التطبيق والاتصال بدل افتراض سبب واحد.
▼ إيه الفرق بين FIN وRST؟
FIN ينهي الإرسال في اتجاه بطريقة منظمة، مع إمكانية استمرار الاتجاه الآخر. RST يعيد ضبط الاتصال بصورة مفاجئة.

مراجع الدرس

- مواصفة TCP — RFC 9293
- PHP stream_socket_server
- PHP stream_socket_client
- PHP fwrite: عدد البايتات المكتوبة

UDP: رسائل مستقلة من غير اتصال TCP

افهم حدود رسالة UDP والفقد والترتيب والتكرار، ومتى نحتاجه وإيه اللي لازم التطبيق يعمله فوقه.

تخيل برنامج بيبعت مكان لاعب كل لحظة. لو تحديث قديم اتأخر، أحيانًا الأهم إنك تعرض المكان الجديد بدل ما توقف كل حاجة لحد ما القديم يرجع. هنا محتاج تفهم خدمة النقل اللي بتاخدها، وإيه المسؤولية اللي هتفضل على برنامجك.

UDP، User Datagram Protocol بروتوكول نقل فوق IP. بيقدم **Datagrams**: رسائل منفصلة لها حدود. محتاج تعرف إن IP بيوجّه الحزم بين الأجهزة، وإن المنفذ رقم بيحدد نقطة استقبال داخل الجهاز.

الرسالة هنا لها حدود

لو أرسلت رسالتين، كل واحدة datagram مستقلة. مش معنى ده إن كل واحدة هتوصل؛ لكن UDP مش بيجمعهم في byte stream واحد زي TCP. التطبيق بيقرأ رسائل، ومحتاج يجهز مساحة تكفي الرسالة. لو الواجهة بتقص الرسالة الأكبر من مساحة الاستقبال، الجزء اللي اتقص مش بالضرورة يبقى متاحًا في القراءة التالية؛ راجع سلوك الـAPI اللي بتستخدمها.

رأس UDP الأساسي 8 بايتات: منفذ المصدر، منفذ الوجهة، طول الوحدة، وقيمة فحص. **Checksum** بتساعد تكتشف فسادًا عرضيًا، لكنها مش مصادقة ولا تشفير. استخدام الفحص وقواعده يختلف حسب IP والإضافات؛ ما تبنيش تصميم حماية على مجرد وجوده.

إيه اللي UDP مش بيعمله تلقائيًا؟

مفيش مصافحة إنشاء اتصال TCP، ولا إعادة إرسال مدمجة للبيانات المفقودة، ولا ضمان ترتيب، ولا منع تكرار، ولا تحكم في ازدحام الطريق مثل اللي بيقدمه TCP. ممكن رسالة تضيع، أو رسالتان يوصلا بترتيب معكوس، أو يتكرر نفس المحتوى. التطبيق لازم يكون مستعد للحالات دي.

بعض واجهات البرمجة بتسمح بـ `connect` على UDP لتحديد الطرف الافتراضي وتصفية الاستقبال أو تحسين التعامل مع الأخطاء. اسم الدالة مش معناه إن فيه مصافحة TCP حصلت أو إن الإرسال بقى مضمونًا.

مثال تتبّع بدل حفظ التعريف

العميل بعت تحديثات مكان اللاعب بالشكل ده:

```
sent:     seq=10 x=40 | seq=11 x=42 | seq=12 x=45
received: seq=10 x=40 | seq=12 x=45 | seq=11 x=42
```

`seq` رقم تتابع يضيفه **التطبيق** هنا، مش حقل في UDP. لو البرنامج طبق الرسائل بالترتيب اللي وصلت بيه، اللاعب هيرجع من 45 إلى 42 بسبب تحديث قديم.

حل مناسب لبعض ألعاب الحالة الآنية: احتفظ بآخر رقم مطبق، وتجاهل تحديثًا أقدم. بكده آخر حالة هي `x=45`. لكن ده مش حل لخصم رصيد أو إنشاء طلب؛ العملية المالية لازم تعرف هل اتنفذت وتمنع التكرار. اختيار سياسة النقل بيتبع معنى البيانات.

ليه حجم الرسالة مهم؟

UDP فوق IP، ووصلة الشبكة ليها حد حجم اسمه **MTU**. رسالة كبيرة ممكن تحتاج تجزئة في الطبقة الأدنى أو تفشل وفق البروتوكول والطريق والإعدادات. فقد جزء قد يمنع إعادة تجميع الرسالة كلها. علشان كده صغر الرأس مش معناه إنك تبعت ملفات ضخمة كرسالة واحدة.

لا تختار حجمًا آمنًا عالميًا من غير معرفة المسار والبروتوكول. صمّم حدود حجم وسياسة رسائل تناسب التطبيق، واختبر الفقد والتكرار والترتيب.

إمتى تختاره؟

DNS يستخدم UDP في كثير من التبادلات، لكنه يدعم TCP أيضًا؛ مفيش قاعدة إن DNS لا يعمل إلا بـUDP. بعض الألعاب والوسائط تستخدم UDP مع بروتوكولات أعلى تضيف توقيتًا وترتيبًا أو تعويض فقد حسب الحاجة.

HTTP/3 يستخدم QUIC فوق **QUIC**. UDP. يضيف تدفقات موثوقة وتشفيرًا وتحكمًا في الازدحام؛ ده مش معناه إن UDP نفسه اكتسب الخصائص دي. لازم تميز خدمة الطبقة الأساسية عن البروتوكول اللي فوقها.

أخطاء تخلي الاختيار مضللًا

- «UDP أسرع دائمًا»: قارن زمن الاستجابة والتسليم المفيد والحمل، مش حجم الرأس فقط.
- «الرسالة اتبعتت، يبقى وصلت»: نجاح استدعاء الإرسال محليًا مش إقرارًا من التطبيق البعيد.
- «هنعيد لحد ما توصل»: إعادة بلا مهلة وحدود ومعدل ممكن تزود الازدحام وتكرر الأثر.
- «عنوان المصدر يثبت الهوية»: UDP نفسه مش بيثبت هوية المرسل؛ عناوين المصدر ممكن تتزوّر في بعض الظروف.

الخدمات العامة كمان لازم تفكر في **Amplification**: مهاجم يرسل طلبًا صغيرًا بعنوان ضحية مزوّر فترد الخدمة برد أكبر على الضحية. امنع تحويل خدمتك لأداة إغراق، وراجع التحقق من العنوان وحجم الردود وحدود المعدل حسب البروتوكول.

جرّب تصميمك على الورق

مستقبل طبق الرسالة `seq=12`، ثم وصلت `seq=12` مرة تانية، وبعدها `seq=11`. إيه النتيجة لو الرسالة حالة لاعب؟ وإيه اللي يتغير لو الرسالة «خصم 10 جنيه»؟

▼ التفكير والإجابة

لو الحالة الكاملة مرقمة وتقبل سياسة «الأحدث فقط»، تجاهل المكرر والأقدم. في الخصم المالي، الرقم وحده مش بروتوكول عمل كامل: محتاج هوية موثوقة ومعرّف عملية فريد وحفظ نتيجة وضمانات معاملة، ولازم تعرف حالة التنفيذ عند انقطاع الرد. ما تختارش UDP لمجرد جملة «أسرع».

مراجع

- UDP — RFC 768
- إرشادات استخدام UDP — RFC 8085

MTU: ليه البيانات الصغيرة تعدّي والكبيرة تتعطل؟

افهم حجم الحزمة وMTU وMSS واكتشاف حجم المسار، وإزاي تختبر فرضية مشكلة الحجم من غير خلطها بمشاكل التطبيق.

الاتصال اتفتح والرد الصغير وصل، لكن رفع ملف كبير وقف. فيه أكتر من احتمال: التطبيق مستني، التخزين بطيء، فيه مهلة، أو بعض أحجام الحزم مش بتعدّي من الطريق. الدرس ده يشرح الاحتمال الأخير وإزاي تجمع دليل عليه.

محتاج تعرف إن IP بيحمل البيانات في حزم، وإن TCP بيقسم تدفق البايتات لمقاطع ويعيد إرسال الناقص. إغلاق الاتصال بـFIN أو RST جزء من درس TCP؛ هنا موضوعنا هو حجم البيانات في الطريق.

حجم الملف مش حجم الحزمة

ملف حجمه 10 MB مش بيتبعت بالضرورة كحزمة واحدة. التطبيق يكتب بايتات، وطبقات الشبكة تقسمها وتضيف معلوماتها.

MTU، Maximum Transmission Unit هو حد حجم حمولة الطبقة الأعلى اللي الوصلة تقدر تحملها. مثلًا لما تكون MTU لوصلة معينة 1500 بايت لبيانات IP، رؤوس IP وTCP بتاخد جزءًا من المساحة؛ مش كل الـ1500 بيانات ملف.

MSS، Maximum Segment Size يخص مقدار بيانات TCP في المقطع من غير رؤوس TCP وIP. مثال حسابي فقط، لو MTU = 1500 ورأس IPv4 = 20 ورأس TCP = 20 ومن غير خيارات، مساحة البيانات = 1460 بايت. وجود خيارات أو تغليف إضافي يغير الحساب، فلا تعتبر 1460 رقمًا مناسبًا لكل اتصال.

الطريق فيه أكتر من وصلة

تخيل أول وصلة تقبل 1500، لكن نفقًا في النص لا يسمح إلا بـ1400. **Path MTU** بيتحدد بالوصلة الأضيق للحجم على المسار. المسار ممكن يتغير، علشان كده قيمة واجهة جهازك وحدها مش وصف كامل للطريق.

التعامل مع الحزمة الأكبر يختلف حسب IP والرايات. في IPv4، بعض الظروف تسمح بالتجزئة، وراية **Don’t Fragment** تمنعها. في IPv6 الراوتر مش بيجزّئ الحزمة بالنيابة عن المرسل؛ الطرف المرسل لازم يتعامل مع حدود المسار.

Path MTU Discovery مجموعة آليات علشان المرسل يعرف الحجم المناسب. بعض الآليات تعتمد على رسائل ICMP المرتبطة بالحجم، وفيه آليات تحقق على مستوى الحزم من غير الاعتماد فقط على الرسالة دي. حجب رسائل التحكم عشوائيًا ممكن يعطل الاكتشاف ويعمل حالة اتصال يبدو مفتوحًا لكن بياناته تتعطل.

إزاي تثبت الفرضية؟

ابدأ بتجربة محكومة على نظام تملكه، وقارن أحجامًا صغيرة وكبيرة مع ثبات الطرفين. اجمع توقيت التطبيق وسجل استلام البيانات وسجل الحزم إن كان متاحًا. **Packet capture** تسجيل لحركة الشبكة؛ ممكن يحتوي بيانات حساسة، فاستعمله في المعمل وبحدود واضحة.

على Windows، مثال لتجربة ICMP عبر IPv4 مع منع التجزئة:

```
ping -4 -f -l 1472 example.com
```

1472 هنا حجم بيانات ICMP في التجربة، ومع 20 بايت IPv4 و8 بايت ICMP يبقى المثال 1500 بايت قبل طبقة الوصلة. الرؤوس والوجهة والطريق بيحددوا هل الفرضية دي مناسبة. خروج رسالة تفيد الحاجة للتجزئة دليل على حد حجم في التجربة. Timeout وحده مش دليل؛ الطرف قد يمنع ICMP أصلًا.

الأمر مش اختبار TCP مباشر، ومش تصريح بتغيير MTU في جهاز الإنتاج. لو عايز تختبر TCP، راقب إعادة الإرسال ورسائل الحجم وحالة التطبيق أثناء نقل محدد.

افصل مشكلة الطريق عن مشكلة التطبيق

لو التطبيق سجّل إنه استلم كل بايتات الرفع وبعدين دخل في ضغط أو تحويل فيديو، التعطل الظاهر ممكن يكون حسابًا أو تخزينًا بعد النقل. لو البيانات الكبيرة لا تتقدم وفيه إعادة إرسال متكررة مع إشارات للحجم، فرضية المسار أقوى.

Checksum يكشف بعض الفساد العرضي، و**Retransmission** يعيد بيانات استُنتج فقدها. الاتنين مش تشفيرًا، ولا بيشرحوا وحدهم ليه حصل الفقد.

تمرين تشخيص

طلب صغير نجح، وملف كبير فشل. زميلك اقترح تصغير MTU فورًا. قبل التعديل، اكتب دليلًا تميّز بيه بين مشكلة المسار وخادم خلص استقبال الملف لكنه مشغول بمعالجته.

▼ إجابة مشروحة

قارن مقدار البيانات اللي استلمها التطبيق وتوقيتها مع تقدم اتصال النقل. استلام كامل ثم وقت CPU أو قرص مرتفع يوجهك للمعالجة. توقف النقل قبل اكتماله مع إشارات حجم أو إعادة إرسال يدعم فحص المسار. المقارنة بين أحجام مضبوطة مفيدة، لكن نجاح ping أو فشله وحده لا يحسم TCP. غيّر إعدادًا واحدًا في معمل وأعد القياس.

مراجع

- IPv4 — RFC 1191J Path MTU Discovery
- IPv6 — RFC 8201J Path MTU Discovery
- Microsoft ping

عنوان الويب والمنافذ وبروتوكول HTTP

تشريح الرابط، معنى Port وSocket، نطاقات المنافذ، وبروتوكول HTTP.

قبل التفاصيل

قبل التفاصيل: الرابط يحدد **ماذا** تريد و**أين** تطلبه، والمنفذ يحدد البرنامج الذي سيستقبل الاتصال على الجهاز. أما HTTP (Hypertext Transfer Protocol؛ قواعد طلب الموارد والرد عليها في الويب) فيحدد شكل الحوار بين العميل والخادم.

HTTP

HTTP = Hypertext Transfer Protocol، يعني قواعد الحوار بين برامج التطبيق. **Request — طلب** رسالة يسأل بها العميل عن مورد أو عملية، و**Response — رد** رسالة النتيجة. **Resource — مورد** شيء له عنوان، زي صفحة أو صورة أو بيانات منتج. HTTP ليس اتصال الشبكة نفسه؛ يعمل فوق وسائل نقل مثل TCP (Transmission Control Protocol؛ نقل بايتات بترتيب مع معالجة الفقد، مع احتمال فشل الاتصال) أو QUIC (اسم بروتوكول نقل فوق UDP يضيف تدفقات موثوقة وحماية اتصال).

HTTP في أصله عديم الحالة: كل request رسالة مستقلة. استمرار تسجيل الدخول أو السلة يحتاج معلومة تربط الطلبات. **Cookie** قيمة صغيرة يحفظها المتصفح ويرسلها مع الطلبات المناسبة. **Session — جلسة** بيانات يحتفظ بها التطبيق عن تفاعل المستخدم، و**Token — رمز اعتماد** قيمة يقدمها العميل لإثبات إذن أو هوية. هنفصلها في درس الرسائل.

تشريح URL

```
https://user:pass@example.com:443/products/42?currency=EGP#reviews
└─┘   └──────┘ └──────────┘ └┘└──────┘└──────┘└──────┘└─────┘
scheme  userinfo     host    port  path     query    fragment
```

الجزء	المعنى
Scheme (طريقة الوصول في بداية العنوان، مثل https)	البروتوكول/طريقة الوصول مثل `http` و `https`
User info	جزء مستخدم وكلمة مرور يسمح به شكل بعض معرّفات الموارد، لكنه غير مناسب لحفظ الأسرار
Hostname	اسم المضيف مثل `example.com`
Port (منفذ منطقي برقم يميز جهة استقبال في بروتوكول النقل)	الخدمة على الجهاز؛ يُحذف إذا استُخدم الافتراضي
Path (مسار المورد داخل العنوان، مثل /products)	المورد داخل الموقع
Query	أزواج إضافية بعد `?`
Fragment (جزء بعد # يستخدمه العميل محليًا، مثل موضع عنوان في الصفحة)	موضع محلي بعد `#` ولا يُرسل عادة إلى الخادم

تحذير

لا تضع كلمة مرور أو token أو بيانات حساسة في URL (Uniform Resource Locator؛ عنوان يحدد موردًا وطريقة الوصول إليه). قد تظهر في تاريخ التصفح، وسجلات الخادم، وأدوات تحليل الزيارات. حقل `Referer` في بعض الطلبات قد ينقل عنوان الصفحة السابقة حسب سياسة المتصفح.

ما المنفذ؟

عنوان IP (Internet Protocol؛ قواعد عنونة الحزم وتوجيهها بين الشبكات، وعنوان IP رقم يحدد واجهة في هذا السياق) يحدد الواجهة/الجهاز على الشبكة، والمنفذ يحدد خدمة منطقية داخل نظام التشغيل. رقم المنفذ يُكتب باستخدام 16 بت، والبت خانة قيمتها صفر أو واحد؛ يعني 2^{16} احتمالًا من `0` إلى `65535`.

- `0–1023`: منافذ خدمات معروفة، اسمها System أو Well-Known.
- `1024–49151`: منافذ يمكن تسجيلها لخدمات، اسمها User أو Registered.
- `49152–65535`: نطاق Dynamic/Private — منافذ ديناميكية أو خاصة؛ النطاق 49152–65535 غير مخصص لخدمات مسجلة لدى IANA، وغالبًا تختار منه الأنظمة منافذ مؤقتة للاتصالات للاستخدام الديناميكي أو الخاص. التقسيم صادر عن **IANA**، **Internet Assigned Numbers Authority**، الجهة المنسقة لسجلات أرقام ومعرّفات الإنترنت. نطاق المنافذ المؤقتة الفعلي قد يختلف حسب النظام.

الخدمة	المنفذ الشائع
FTP (File Transfer Protocol؛ بروتوكول نقل ملفات) control	21
SSH (Secure Shell؛ بروتوكول اتصال آمن للتحكم في جهاز بعيد)	22
SMTP (Simple Mail Transfer Protocol؛ بروتوكول إرسال البريد بين الأنظمة)	25
DNS (Domain Name System؛ نظام يجيب عن أسئلة أسماء النطاقات، ومنها عناوينها)	53
HTTP	80
HTTPS (Hypertext Transfer Protocol Secure؛ طلبات ويب عبر اتصال محمي بالتشفير والتحقق من هوية الطرف الآخر)	443
MySQL، برنامج لإدارة قواعد البيانات	3306
PostgreSQL، برنامج لإدارة قواعد البيانات	5432

المنفذ الشائع اتفاق وإعداد، مش وسيلة حماية. يمكن تشغيل HTTP محليًا على `8000` أو `8080`.

Connection و Socket

Socket — نقطة اتصال برمجية واجهة يستخدمها البرنامج للإرسال والاستقبال. لو بنتكلم عن عنوان نقطة شبكة، نلخصه ببروتوكول النقل وعنوان IP والمنفذ؛ مثل TCP مع 203.0.113.8 والمنفذ443. اتصال TCP يُميز غالبًا بهذه الرباعية:

```
(client IP, client port, server IP, server port)
192.0.2.5:53014 → 203.0.113.8:443
```

يستخدم العميل عادة منفذًا مؤقتًا، بينما يستمع الخادم على منفذ معروف. لذلك يمكن لآلاف العملاء الاتصال بالمنفذ 443 نفسه دون أن تختلط اتصالاتهم.

مثال PHP محلي

ده تطبيق اختياري بعد تثبيت PHP حسب تجهيز المعمل. أنشئ مجلد تدريب، واحفظ داخله ملف `hello.php` بمحتوى `<?php echo 'Hello';`. علامة `<?php` تبدأ كود PHP، و`echo` تعرض النص. افتح الطرفية داخل المجلد، ثم شغّل:

```
php -S localhost:8000
```

الخيار `-S` يبدأ خادم التطوير المحلي. اترك الطرفية مفتوحة وافتح العنوان التالي في المتصفح؛ المتوقع كلمة `Hello`. `localhost` اسم جهازك نفسه. أوقف الخادم بـCtrl+C عند الانتهاء؛ الخادم ده للتجربة المحلية.

في `http://localhost:8000/hello.php`:

- scheme = `http`

- host = `localhost`
- port = `8000`
- path = `/hello.php`

عدم كتابة المنفذ في HTTPS يعني عادة 443، وفي HTTP يعني 80؛ لا يعني عدم وجود منفذ.

مسائل عملية

▼ هل يرسل المتصفح الجزء `#section` إلى الخادم؟

لا. الـfragment للاستخدام داخل المتصفح عادة، بينما يُرسل path وquery string (معاملات بعد علامة ? في العنوان، مثل name=Omar).

▼ الخدمة تعمل على `localhost:8080` لكن الرابط بلا منفذ لا يصل إليها. لماذا؟

لأن HTTP يستخدم 80 وHTTPS يستخدم 443 افتراضيًا. يجب كتابة `:8080` ما دامت الخدمة تستمع عليه.

▼ هل المنفذ 443 يعني أن الاتصال مشفر حتمًا؟

لا. الرقم اتفاق شائع؛ TLS (Transport Layer Security؛ قواعد حماية الاتصال بالتشفير والتحقق) الفعلي هو ما ينشئ التشفير. يمكن لبرنامج غير مشفر أن يستمع على 443 بإعداد خاطئ.

الخطوة التالية

كمّل في الحروف والرموز داخل عنوان الويب بعد تنفيذ التجربة هنا.

الحروف والرموز داخل عنوان الويب

الحروف والرموز داخل عنوان الويب

اكتب عنوانًا فيه اسم عربي أو مسافة: إزاي يبقى له شكل يصلح للنقل؟ ابدأ بعد معرفة أجزاء عنوان الويب.

الرموز داخل العنوان

Percent-encoding — الترميز بعلامة % يكتب بايتًا برقمين سداسيين بعد %؛ مثل `%20` لمسافة. الرقم السداسي طريقة مختصرة لكتابة البتات كما شرحنا في تمثيل البيانات. فك الترميز مرتين بالخطأ قد يغيّر المعنى؛ مثل تحويل `%252F` أولًا إلى `%2F` ثم إلى شرطة مائلة.

URI، Uniform Resource Identifier اسم عام لمعرّف مورد، وURL نوع يصف الوصول إليه. **IRI، Internationalized Resource Identifier** يسمح بمحارف دولية في كتابة المعرّف. **IDNA، Internationalized Domain Names in Applications** قواعد التعامل مع أسماء نطاقات دولية وتحويلها لصيغة مناسبة لنظام الأسماء. ده مختلف عن ترميز المسار والبحث بعلامة %.

تدريب محلول: في `http://[::1]:8080/search?q=hello%20world&tag=a&tag=b#results`، المضيف عنوان جهازك المحلي IPv6 (Internet Protocol version 6؛ إصدار عناوين الشبكة ذي128 بتًا) بين أقواس، والمنفذ 8080. الهدف المرسل في HTTP/1.1 هو `/search?q=hello%20world&tag=a&tag=b`، والجزء بعد # لا يدخل فيه. المفتاح tag مكرر؛ تفسيره كقائمة أو اختيار قيمة واحدة يعتمد على مكتبة التطبيق، فلا نفترض سلوكًا واحدًا.

اقرأ طلب HTTP وردّه

اقرأ طلب HTTP وردّه

قبل التفاصيل

قبل تفاصيل جلسة المستخدم ورمز دخوله، افصل بين **رسالة HTTP (Hypertext Transfer Protocol)؛ قواعد طلب الموارد والرد عليها في الويب) واحدة** وبين حالة المستخدم عبر عدة رسائل. الطلب يحمل نية العميل، والاستجابة تحمل نتيجة الخادم.

الطلب: العميل بيطلب إيه؟

في HTTP/1.1 النصي يتكون الطلب من **Start line — سطر البداية**، ثم **Headers — حقول تصف الرسالة**، ثم سطر فارغ، ثم **Body — جسم الرسالة** لو فيه محتوى. ده مثال مكتوب للفهم، مش أمر تلصقه في الطرفية:

```
POST /api/orders?notify=1 HTTP/1.1
Host: shop.example
Accept: application/json
Content-Type: application/json
Authorization: Bearer ey...
Cookie: session=abc123
Content-Length: 27

{"product_id":42,"count":2}
```

- **Method — طريقة الطلب:** العملية المطلوبة؛ POST هنا يرسل طلب معالجة أو إنشاء.
- **Target — هدف الطلب:** المسار `/api/orders` مع معاملات البحث `notify=1`، ومعناها هنا طلب إشعار وفق تصميم التطبيق.
- **Version — الإصدار:** HTTP/1.1 يحدد نسخة طريقة الحوار.
- **Headers — حقول الرسالة:** معلومات تصف المحتوى وطريقة التعامل معه؛ تسمى أحيانًا Metadata، أي بيانات عن البيانات.
- **Body — الجسم:** بيانات الطلب نفسها؛ هنا رقم المنتج42 والكمية2.

HTTP/2 وHTTP/3 يمثلان الرسائل بصيغة ثنائية بدل الأسطر النصية نفسها؛ التمثيل الثنائي غير التشفير الأمني، لكن معاني العملية وحقول الرسالة ومحتواها تظل نفسها.

قبل الجدول: **Safe — قراءة لا يقصد بها تغيير حالة المورد**، زي جلب صفحة؛ تسجيل الزيارة في السجل لا يناقض المعنى.
Idempotent — تكرار نفس الطلب له نفس الأثر المقصود على المورد: حذف مورد ثم إعادة حذفه لا يحذفه مرتين، لكن رمز الرد قد يختلف. الطلب ممكن يكون idempotent ويغيّر الحالة، زي PUT. والـ**Representation — تمثيل المورد** نسخة بيانات تصفه، مثل معلومات منتج بصيغة JSON (JavaScript Object Notation؛ صيغة نصية لترتيب البيانات في أسماء وقيم وقوائم).

JSON، JavaScript (لغة برمجة تستخدم مثلًا لتنفيذ تفاعل الصفحة داخل المتصفح) Object Notation صيغة نصية لبيانات منظمة؛ `{"count":2}` يربط الاسم count بالقيمة 2. جسم المثال أعلاه 27 بايتًا بترميز UTF-8 (Unicode Transformation Format بوحدات 8 بت؛ ترميز أرقام رموز Unicode كبايتات)، من غير سطر جديد في نهايته. **Content-Length** يعد البايتات لا عدد الحروف؛ تغيير الجسم يحتاج إعادة حسابه. `Bearer ey...` قيمة توضيحية مختصرة وليست رمز دخول صالحًا.

Methods

Method	الدلالة المعتادة	Safe؟	Idempotent؟
`GET`	جلب تمثيل المورد، مثل بيانات منتج	نعم	نعم
`HEAD`	مثل GET لكن من غير جسم الرد	نعم	نعم
`POST`	معالجة خاصة بالمورد/إنشاء شائعًا	لا	لا غالبًا
`PUT`	استبدال الحالة الكاملة	لا	نعم
`PATCH`	تعديل جزئي	لا	يعتمد على التصميم
`DELETE`	إزالة المورد	لا	نعم دلاليًا
`OPTIONS`	إمكانات الاتصال	نعم	نعم

Safe وIdempotent دلالات يجب أن يحترمها التطبيق، وليستا آلية صلاحيات. لا يجوز تنفيذ حذف حقيقي عبر GET، وعلى الخادم التحقق من الهوية والصلاحية في كل Method.

Headers شائعة

- `Host` : اسم الموقع المطلوب. **Virtual hosts — مواقع افتراضية** أسماء مواقع مختلفة يخدمها نفس الخادم، فيستخدم الاسم لاختيار الموقع.
- `Accept` : أنواع الاستجابة المقبولة.
- `Content-Type` : نوع جسم الرسالة المرسل.
- `Authorization` : بيانات اعتماد. **Bearer token** رمز يكفي تقديمه كما هو لاستخدام الإذن المرتبط به، لذلك نحافظ على سريته.
- `Cookie` : قيم صغيرة سبق أن حفظها المتصفح ويحق إرسالها إلى هذا العنوان.
- `User-Agent` : معلومات العميل.
- `Cache-Control` : سياسة **Cache (نسخة محفوظة لتقليل تكرار القراءة أو الحساب) — التخزين المؤقت**: هل يمكن حفظ نسخة من الرد وإعادة استخدامها ومتى؟

الرد: الخادم رجّع إيه؟

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Location: /api/orders/901
Cache-Control: no-store

{"id":901,"status":"pending"}
```

سطر البداية يذكر الإصدار ورمز الحالة **Status code (رقم يصف نتيجة معالجة طلب HTTP)**، مثل201 بمعنى إنشاء مورد. عبارة `Created` وصف مقروء، لا نعتمد عليها بدل الرقم. بعدها حقول الرسالة وجسم اختياري. `Location` عنوان المورد الجديد، و`no-store` تطلب عدم تخزين الرد في مخزن HTTP مؤقت.

الفئة	المعنى	أمثلة
`1xx`	معلومات مؤقتة	`100 Continue`
`2xx`	نجاح	`200 OK`, `201 Created`, `204 No Content`
`3xx`	إعادة توجيه أو إعادة تحقق من نسخة مخزنة	`301`, `302`, `304`
`4xx`	مشكلة في الطلب	`400`, `401`, `403`, `404`, `422`
`5xx`	فشل جهة الخادم	`500`, `502`, `503`

`401` يعني غالبًا أن المصادقة مطلوبة أو غير صالحة، و`403` أن الطلب مفهوم لكن غير مسموح. لا تستخدم `200` لكل شيء؛ الحالة جزء من عقد الـAPI (Application Programming Interface؛ اتفاق يسمح لبرنامج بطلب بيانات أو عملية من مكوّن آخر).

مسائل عملية

▼ ما الفرق بين `401` و`403` ؟

401 يعني أن الطلب يحتاج بيانات اعتماد صالحة، و403 يعني أن الخادم رفض الإذن؛ قد يكون الرفض بسبب الهوية أو قاعدة أخرى، فلا يثبت وحده أن المستخدم سجل الدخول.

▼ كيف يكون HTTP stateless مع وجود تسجيل دخول؟

كل طلب مستقل عند البروتوكول، لكن cookie أو token يحمل معرّفًا يسمح للتطبيق باسترجاع الحالة من مخزن بيانات الجلسات.

▼ هل يناسب GET عملية تحوّل رصيدًا؟

لا. GET يفترض القراءة الآمنة وقد تعيد المتصفحات أو برامج فهرسة المواقع تشغيله. استخدم method تغيّر الحالة مع حماية وإعادة آمنة.

الخطوة التالية

كمّل في تذكّر المستخدم وحماية التعديلات بعد تنفيذ التجربة هنا.

تذكّر المستخدم وحماية التعديلات

تذكّر المستخدم وحماية التعديلات

بعد قراءة طلب ورد منفردين، نحتاج ربط طلبات المستخدم ومنع تعديل قديم من مسح تعديل أحدث.

Stateless لا يعني بلا حالة في التطبيق

HTTP لا يتذكر تلقائيًا الطلب السابق. التطبيق يبني الاستمرارية بواسطة:

- **Cookie:** قيمة يخزنها المتصفح ويرسلها وفق اسم الموقع والمسار وقواعد الأمان المحددة لها.
- **Session:** بيانات يحتفظ بها الخادم، مرتبطة غالبًا بمعرّف في Cookie؛ مثل رقم المستخدم الذي سجل الدخول.
- **Token:** رمز اعتماد يرسله العميل، شائع عند تبادل البيانات بين البرامج. ليس كل token JWT (JSON Web Token؛ صيغة رمز تحمل بيانات ادعاءات، وليست كل الرموز من هذا النوع)، وJWT ليس Session بديلًا تلقائيًا.

مثال PHP

التطبيق اختياري بعد تجهيز PHP وتشغيل الخادم المحلي. احفظ الكود في `hello.php` داخل مجلد التدريب، وشغّل `php -S localhost:8000` من المجلد. ده ملف كامل يرد على طلب GET، أي طلب قراءة، ويعيد رسالة ترحيب بصيغة JSON.

```
<?php
declare(strict_types=1);

header('Content-Type: application/json; charset=utf-8');

if (($_SERVER['REQUEST_METHOD'] ?? '') !== 'GET') {
    http_response_code(405);
    header('Allow: GET');
    echo json_encode(['error' => 'Method not allowed']);
    exit;
}

$rawName = $_GET['name'] ?? 'زائر';
if (!is_string($rawName)) {
    http_response_code(400);
    echo json_encode(['error' => 'name must be text']);
    exit;
}
$name = trim($rawName);

echo json_encode(
    ['message' => "مرحبًا {$name}"],
    JSON_UNESCAPED_UNICODE | JSON_INVALID_UTF8_SUBSTITUTE | JSON_THROW_ON_ERROR
);
```

جرّب `curl.exe -i "http://localhost:8000/hello.php?name=Omar"` على Windows، أو `curl` على الأنظمة الأخرى. المتوقع رمز200 وجسم `{"message":"مرحبًا Omar"}` . افتح لوحة Network، أي الشبكة، في أدوات المطور DevTools لمشاهدة الرسالة أيضًا.

`$_SERVER` يحمل معلومات الطلب، و`$_GET` يحمل معاملات الرابط. `??` يستعمل البديل لو القيمة غير موجودة، و`!==` تقارن مع النوع. `header` تكتب حقل رد، و`http_response_code` تختار الحالة، و`json_encode` تحول بيانات PHP إلى نص JSON. `is_string` تتأكد أن الاسم نص، و`trim` تزيل المسافات من طرفيه. `exit` ينهي الطلب بعد رسالة الخطأ. الثوابت في آخر الملف تمنع تحويل الحروف العربية إلى تسلسلات هروب، وتستبدل بايتات النص غير الصالحة، وتطلب خطأ استثنائيًا لو فشل الترميز لسبب آخر.

اختبر الحدود: `?name[]=Omar` يرسل قائمة بدل نص، فيُرفض بـ400. طلب POST يُرفض بـ405 ومعه `Allow: GET` لتوضيح الطريقة المسموحة. الاسم غير المرسل يأخذ «زائر». أوقف الخادم بـCtrl+C بعد التجربة.

اللغة والنسخة والتعديل المتزامن

Content negotiation — اختيار تمثيل يناسب العميل يستخدم مثلًا Accept لنوع البيانات، وAccept-Language للغة، و-Accept Encoding للضغط المدعوم. إذا تغير الرد حسب أحد هذه الحقول، **Vary** يحدد الحقول التي يجب أن يميز المخزن المؤقت نسخها.

ETag — Entity Tag معرّف لنسخة المورد يرسله الخادم. **If-None-Match** يقول «أرسل المحتوى لو النسخة اختلفت» ويفيد في إعادة فحص النسخة المحفوظة. **If-Match** يقول «نفّذ التعديل فقط لو النسخة ما زالت مطابقة» ويحمي من **Lost update — ضياع تعديل بسبب كتابة عميل فوق تعديل أحدث**.

تدريب وحل: عميل قرأ نسخة `"v7"`، ثم عميل آخر غيّرها إلى `"v8"`. الأول يرسل تعديلًا مع `If-Match: "v7"`. الخادم يفحص الشرط وينفذ الكتابة كوحدة لا يتدخل بينهما تعديل آخر؛ عند عدم التطابق يرفض بـ`412 Precondition Failed`، أي فشل شرط مسبق. يرجع العميل يقرأ الأحدث ويقرر كيف يعالج التعارض.

تشفير الاتصال وشهادات المواقع

إزاي المتصفح يتأكد من اسم الموقع ويحمي البيانات، وإيه معنى TLS وCA وDV وOV وEV.

تخيّل إنك بتكتب كلمة السر في موقع. البيانات بتعدّي على أجهزة وشبكات قبل ما توصل. محتاج تمنع حد في الطريق من قراءتها أو تغييرها، وتتأكد إنك بتكلم الموقع اللي كتبت عنوانه. ده دور **TLS، اختصار Transport Layer Security**: قواعد لإنشاء اتصال محمي بين برنامجين. هنفهم الاتصال، وبعده نفك معنى شهادة الموقع واختصاراتها.

قبل الدرس راجع رسائل HTTP: الطلب رسالة من العميل، والرد رسالة من الخادم. العميل هنا هو المتصفح، والخادم هو البرنامج اللي بيرد عليه.

إيه الفرق بين HTTP وHTTPS؟

HTTP، اختصار Hypertext Transfer Protocol، قواعد الطلب والرد على الويب. **HTTPS (Hypertext Transfer Protocol Secure؛ طلبات ويب عبر اتصال محمي بالتشفير والتحقق من هوية الطرف الآخر)** هو استخدام HTTP عبر اتصال محمي بـTLS. معنى طلب القراءة `GET` أو رمز النجاح `200` ما بيتغيرش؛ اللي اتغير هو حماية الرسائل أثناء انتقالها.

TLS بيوفر ثلاث حاجات:

- **سرية البيانات — Confidentiality:** اللي بيراقب الطريق ما يقدرش يقرأ المحتوى المحمي.
- **سلامة البيانات — Integrity:** التغيير غير المسموح في الرسالة يتكشف.
- **التحقق من الهوية — Authentication:** المتصفح يفحص دليلًا يربط اسم الموقع بمفتاح يملكه الخادم.

الحماية تخص الطريق بين طرفي الاتصال. الخادم المخترق يقدر يقرأ البيانات بعد فكها. HTTPS ما يصلحش أخطاء التطبيق، زي **SQL Injection**، إدخال نص يغيّر أمر قاعدة البيانات، أو **XSS، Cross-Site Scripting**، تشغيل كود مؤذٍ داخل صفحة المستخدم، أو السماح لشخص بقراءة حساب غيره. عنوان **IP (Internet Protocol؛ قواعد عنونة الحزم وتوجيهها بين الشبكات، وعنوان IP رقم يحدد واجهة في هذا السياق)**، عنوان وجهة الاتصال على الشبكة، قد يظل ظاهرًا، واسم الموقع قد يظهر في بعض خطوات الاتصال.

شهادة الموقع بتقول إيه؟

النطاق — Domain name اسم زي `example.com`. **الشهادة الرقمية — Digital certificate** مستند إلكتروني يربط أسماء موقع بمفتاح عام وفترة صلاحية، وعليه توقيع من جهة إصدار.

CA، اختصار Certificate Authority، معناها جهة إصدار الشهادات. بتتحقق من طالب الشهادة حسب نوعها، وبعدها توقّعها. المتصفح ما يصدقش أي جهة من نفسه؛ عنده أو عند نظام التشغيل **Trust store — قائمة جهات موثوقة**. يفحص سلسلة توقيعات، غالبًا من شهادة الموقع إلى جهة وسيطة، ثم **Root CA — جهة جذر موثوقة** موجودة في القائمة.

المتصفح يفحص مثلًا:

1. هل الاسم اللي زرته موجود في **SAN، Subject Alternative Name**؟ ده حقل بيسرد الأسماء التي تغطيها الشهادة.
2. هل التاريخ داخل فترة الصلاحية؟ وهل ساعة جهازك صحيحة؟
3. هل سلسلة التوقيعات تصل لجهة يثق بها؟

4. هل حقول **Key Usage** و**Extended Key Usage**، الاستخدامات المسموحة للمفتاح، تسمح بهذا الغرض؟
5. هل الخادم أثبت امتلاك المفتاح الخاص؟ وهل الشهادة ملغاة وفق وسائل الفحص المتاحة للعميل؟

شهادة صحيحة لـ `shop.example` لا تكفي لزيارة `bank.example`. التشفير من غير فحص الاسم مش ضمان إنك وصلت للجهة المطلوبة.

DV وOV وEV: الجهة بتتحقق من إيه؟

الاختصارات دي بتوضح **جهة إصدار الشهادة بتتأكد من إيه قبل ما تديها لصاحب الموقع**. كلمة «النطاق» هنا معناها اسم الموقع، زي `shop.example`. علشان تثبت إن الاسم تحت تحكمك، الجهة تطلب منك تعمل تغيير محدد في إعداداته أو تنشر قيمة تحقق في مكان تحدده على الموقع. نجاح الخطوة دي ما يضمنش أمان كل صفحة أو أمانة صاحبها.

الاختصار والاسم الكامل	المعنى بلغة عادية	مثال على ما يثبته
DV — Domain Validation، التحقق من اسم الموقع	الجهة تتأكد إن اللي طلب الشهادة يقدر يتحكم في اسم الموقع	مثلًا، يقدر ينشر قيمة التحقق اللي طلبتها الجهة. ده لوحده ما يثبتش هو شركة إيه
OV — Organization Validation، التحقق من الشركة أو المؤسسة	الجهة تتحقق من اسم الموقع، وكمان تراجع بيانات الجهة اللي طلبت الشهادة	تتأكد من اسم الشركة ووجودها من أوراق ومصادر تقدر تعتمد عليها
EV — Extended Validation، تحقق بفحوص إضافية	الجهة تراجع بيانات الشركة بخطوات إضافية محددة في قواعد إصدار النوع ده	من الفحوص: هل الشركة مسجّلة وموجودة وبتشتغل فعلًا؟ وهل الشخص اللي طلب الشهادة له حق يطلبها باسمها؟

OV وEV مش تشفيرًا أقوى من DV. الأنواع الثلاثة ممكن تستخدم نفس إعدادات TLS. الفرق في التحقق من هوية الجهة، مش ضمان منع كل احتيال. شكل عرض الشهادة يختلف بين المتصفحات؛ ما تعتمدش على لون أو رمز واجهة قديم لتحديد نوعها.

جرّب فحص شهادة

افتح `https://example.com`، ثم معلومات الاتصال بجوار العنوان وابحث عن عرض الشهادة؛ مكان الاختيار يختلف حسب المتصفح. سجّل اسمًا من SAN، و**Issuer — جهة الإصدار**، وبداية الصلاحية ونهايتها. دي معلومات عامة، مش المفتاح الخاص.

على Windows افتح PowerShell من Start وشغّل:

```
curl.exe -v https://example.com/ -o NUL
```

`curl.exe` عميل يطلب العنوان، و`-v` يعرض تفاصيل الاتصال، و`-o NUL` يهمل جسم الصفحة. في Linux وmacOS استخدم `curl` و`/dev/null` بدل `NUL`. التفاصيل تعتمد على نسخة الأداة؛ قد ترى TLS وALPN. لا تتجاوز تحذير الشهادة أو تعطّل التحقق لإكمال التجربة، ولا تطبع كلمات سر أو رموز دخول أثناء التشخيص.

تدريب محلول: ظهر خطأ اختلاف اسم الشهادة. هل شراء EV يصلحه؟ لا؛ المطلوب شهادة تغطي الاسم المطلوب وإعداد خادم صحيح. ولو الشهادة سليمة لكن الرد `500`، فالطلب وصل لخطأ في المعالجة بعد نجاح الاتصال؛ افحص التطبيق.

أخطاء شائعة وراجع فهمك

- إرسال كلمة سر عبر `http://` لا يحميها TLS؛ استخدم عنوان HTTPS الصحيح.
- حفظ المفتاح الخاص في **Git**، أداة تاريخ الملفات، يعرّضه لمن يقرأ التاريخ. لو اتسرّب يحتاج استبدالًا وإلغاء الشهادة المناسبة، مش مجرد حذف السطر.
- **Base64** طريقة تمثيل بيانات بحروف يمكن عكسها من غير مفتاح؛ **Encoding — ترميز**، مش **Encryption — تشفير**.

▼ هل HTTPS سليم يعني أن صاحب الموقع أمين؟

لا. يثبت اتصالًا محميًا بالاسم الذي فُحص. موقع خداع يقدر يملك شهادة صحيحة لاسمه؛ راجع الاسم والجهة التي تتعامل معها.

▼ ليه نفصل شهادة الموقع عن مفاتيح بيانات الاتصال؟

الشهادة تربط هوية بمفتاح عام. المصافحة تستخدم إثبات الهوية واتفاق الأسرار، وبعدها مفاتيح نقل متماثلة تحمي البيانات بكفاءة.

▼ ما الفرق بين DV وOV وEV في جملة؟

DV يتأكد إن طالب الشهادة يتحكم في اسم الموقع. OV يراجع بيانات الشركة كمان. EV يطلب فحوص زيادة حسب قواعده. الفرق في المعلومات اللي الجهة راجعتها، مش في قوة التشفير.

بعد الدرس تقدر تفرق بين مشكلة الوصول للشبكة، ومشكلة الشهادة، وخطأ HTTP بعد نجاح الاتصال. الخطوة التالية إزاي المتصفح يعرض الصفحة.

مراجع للتفاصيل

- رسائل مصافحة TLS 1.3.
- متطلبات شهادات المواقع.
- متطلبات التحقق الموسع EV.

الخطوة التالية

كمّل في كيف يتفق الطرفان على اتصال مشفّر؟ بعد تنفيذ التجربة هنا.

كيف يتفق الطرفان على اتصال مشفّر؟

كيف يتفق الطرفان على اتصال مشفّر؟

هنا تفاصيل ما وراء الاتصال المشفّر. اقرأ أولًا معنى التشفير والشهادة والجهة المصدرة في درس HTTPS، ثم تتبع الرسائل واحدة واحدة.

مفتاح التشفير مش كلمة السر اللي بتكتبها

مفتاح التشفير — Encryption key قيمة تستخدمها خوارزمية، أي خطوات حسابية محددة، لحماية البيانات أو فكها.

- **التشفير المتماثل — Symmetric cryptography:** الطرفان عندهما السر اللازم لحماية الرسائل وقراءتها. مناسب لنقل بيانات كثيرة بكفاءة. **AES-GCM** و**ChaCha20-Poly1305** اسمان لطريقتين بتجمعا تشفير المحتوى وكشف تعديله.
- **التشفير غير المتماثل — Asymmetric cryptography:** فيه مفتاح عام **Public key** يمكن مشاركته، ومفتاح خاص **Private key** لازم صاحبه يحافظ على سريته. من استخداماته **التوقيع الرقمي**: دليل يصنعه المفتاح الخاص ويقدر المفتاح العام يفحصه.

مش بنشفّر كل الصفحة بالمفتاح العام. خطوات البداية بتثبت الهوية وتساعد الطرفين يتفقوا على أسرار، وبعدها يستخدموا **مفاتيح نقل — Traffic keys** لحماية البيانات. أحيانًا تسمع **Session keys**؛ المقصود مفاتيح اتصال، مش جلسة تسجيل الدخول المخزنة في التطبيق.

في طريقة قديمة اسمها **RSA key exchange** كان العميل يشفّر سرًا أوليًا بالمفتاح العام للخادم، ثم تُشتق منه مفاتيح الاتصال. دي مش طريقة TLS 1.3. في مصافحة TLS 1.3 المعتادة بشهادة، ممكن يُستخدم **ECDHE، Elliptic Curve Diffie–Hellman Ephemeral**: اتفاق على سر باستخدام قيم مؤقتة. كل طرف يرسل جزءًا علنيًا ويحتفظ بجزء سري؛ الطرفان يحسبان السر نفسه من غير إرسال السر النهائي.

مع الاتفاق المؤقت السليم وحذف الأسرار المؤقتة، تسريب مفتاح الشهادة لاحقًا لا يكفي وحده لفك تسجيلات الاتصالات القديمة. دي **Forward secrecy — حماية سرية الاتصالات السابقة**. استئناف اتصال قديم له تفاصيل مختلفة، فلا نعمّم رسم البداية على كل اتصال.

خطوات بدء الاتصال

Handshake — المصافحة هي رسائل البداية للاتفاق والتحقق. ده رسم لمصافحة TLS 1.3 كاملة باستخدام شهادة خادم:

```
المتصفح → ClientHello: الإصدارات المدعومة وجزء اتفاق المفتاح واسم الموقع
الخادم  → ServerHello: الإعدادات المختارة وجزء اتفاق المفتاح
الطرفان يشتقان مفاتيح لحماية بقية المصافحة
الخادم  → EncryptedExtensions ثم Certificate ثم CertificateVerify ثم Finished
المتصفح يفحص الشهادة والتوقيع ورسالة إتمام المصافحة
المتصفح → Finished
الطرفان يتبادلان HTTP بمفاتيح حماية بيانات التطبيق
```

`Hello` رسالة بدء، و`EncryptedExtensions` إعدادات إضافية محمية. `Certificate` الشهادة، و`CertificateVerify` توقيع يثبت امتلاك مفتاحها الخاص ويربطه بالمصافحة، و`Finished` فحص يؤكد سلامة ما اتفق عليه الطرفان. **الشهادة مش جزءًا من ServerHello نفسها.**

Cipher suites قائمة طرق حماية مدعومة؛ في TLS 1.3 اختيارها لا يحدد وحده طريقة اتفاق المفتاح أو نوع الشهادة. عند استئناف اتصال سابق قد تقل الرسائل. وفي HTTP/3 يستخدم النقل **QUIC (اسم بروتوكول نقل فوق UDP يضيف تدفقات موثوقة وحماية اتصال)**، بروتوكول يدمج مصافحة TLS 1.3؛ له درس مستقل.

انتهاء الصلاحية والإلغاء

Expiry — انتهاء الصلاحية وصول التاريخ لنهاية المدة. **Revocation — الإلغاء** سحب الثقة قبل نهاية المدة، مثلًا بعد تسريب المفتاح.

- **CRL، Certificate Revocation List:** قائمة شهادات ألغتها جهة الإصدار.
- **OCSP، Online Certificate Status Protocol:** طريقة للاستعلام عن حالة شهادة.
- **OCSP stapling:** الخادم يرفق رد حالة موقّعًا من الجهة بدل أن يجلبه العميل منفصلًا.

طريقة الفحص والتصرف عند تعذره تختلف حسب العميل وإعداداته؛ مش كل متصفح ينفذ كل وسيلة بنفس الطريقة.

إضافات هتقابلها مع الاتصال المشفّر

HSTS، HTTP Strict Transport Security سياسة يتعلمها المتصفح من الموقع عبر HTTPS سليم، فتطلب استخدام HTTPS معه مستقبلًا خلال مدة معينة. فيه كمان قوائم تجهيز مسبق لبعض المواقع. السياسة لا تصلح شهادة منتهية أو باسم مختلف.

ALPN، Application-Layer Protocol Negotiation تفاوض داخل TLS على بروتوكول التطبيق، مثل اختيار HTTP/2. هو اختيار طريقة الحوار، مش نوع شهادة إضافيًا.

mTLS، Mutual TLS تحقق متبادل: الخادم يطلب شهادة من العميل كمان. مفيد لتمييز خدمات تتواصل مع بعضها. شهادة العميل لا تعفي التطبيق من تحديد العمليات المسموحة للخدمة.

كيف يعرض المتصفح الصفحة؟

Navigation وDOM وCSSOM وRender Tree وتنفيذ JavaScript واستخدام DevTools لفهم الأداء.

غيّر حجم نص وقِس عرضه

احفظ المثال باسم `render.html`، أو شغّله. **CSS** قواعد شكل الصفحة؛ `font-size` حجم الخط، و`px` وحدة بكسل CSS وليست وعدًا بنقطة مادية واحدة في كل شاشة. `span` جزء نص، و`style` يحدد شكله. الدالة `getBoundingClientRect()` تقرأ صندوق العنصر بعد التخطيط، و`width` عرضه.

```
<!doctype html>
<html lang="en">
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>Measure a visible change</title>
<p><span id="sample" style="font-size:16px;display:inline-block">Hello</span></p>
<button id="grow">Grow text</button>
<p id="output"></p>
<script>
function measure() {
  const width = document.querySelector("#sample").getBoundingClientRect().width;
  document.querySelector("#output").textContent = "Width: " + width + " CSS pixels";
}
function grow() {
  document.querySelector("#sample").style.fontSize = "32px";
  measure();
}
document.querySelector("#grow").addEventListener("click", grow);
measure();
</script>
</html>
```

سجّل العرض الأول ثم اضغط الزر: النص يكبر من16 إلى32، والعرض المقاس يزيد. القيمة الدقيقة تعتمد على الخط والمتصفح، فلا نحفظ رقمًا ثابتًا. **DOM** تمثيل عناصر الصفحة، و**Layout — تخطيط** حساب مواضعها وأحجامها، و**Paint — رسم** تجهيز شكلها. تغيير الحجم يحتاج حسابًا جديدًا؛ مجرد تغيير لون لا يطلب نفس تغيّر الحجم.

قبل التفاصيل

وصول ملف الصفحة مش نهاية الرحلة. **HTML، Hypertext Markup Language** يصف العناصر، زي عنوان وفقرة. **CSS، Cascading Style Sheets** يصف شكلها، زي اللون والحجم. **JavaScript (لغة برمجة تستخدم مثلًا لتنفيذ تفاعل الصفحة داخل المتصفح)** لغة تضيف حسابًا وتفاعلًا. هنشوف المتصفح يحوّل المحتوى والشكل لصورة على الشاشة، ونحدد سبب التأخير بدل تخمينه.

DOM، Document Object Model تمثيل العناصر كشجرة: صفحة تحتوي عنوانًا وفقرات. **CSSOM، CSS Object Model** تمثيل قواعد التنسيق بعد قراءتها. **Pixel — بكسل** نقطة لون صغيرة على الشاشة، و**Render tree — شجرة العرض** المعلومات التي يحتاجها المتصفح لترتيب العناصر التي سيعرضها.

في الرسم، decode يعني قراءة البايتات بحسب ترميز النص، وtokens أجزاء ذات معنى مثل بداية وسم، وparse تحليل هذه الأجزاء لبناء تمثيل منظم. الأسهم ترتيب تعليمي للمسؤوليات؛ المتصفح ممكن يعيد بعض الخطوات عندما تتغير الصفحة.

من الاستجابة إلى Pixels

```
HTML bytes -> decode -> tokens -> DOM
CSS bytes  -> parse  -> CSSOM
DOM + CSSOM -> Render Tree -> Layout -> Paint -> Composite
```

- **DOM:** شجرة العناصر والمحتوى.
- **CSSOM:** القواعد المحللة التي تحدد الشكل.
- **Layout:** حساب الأحجام والمواقع.
- **Paint:** رسم النصوص والألوان والحدود.
- **Composite:** تركيب الطبقات لإظهار الإطار النهائي.

CSS اللازمة للعرض قد تؤخر الرسم، وJavaScript التقليدية قد توقف تحليل HTML حتى تُحمّل وتنفّذ.

DevTools

1. افتح **Network** وفعّل Disable cache (نسخة محفوظة لتقليل تكرار القراءة أو الحساب) أثناء التجربة.
2. راقب DNS (Domain Name System؛ نظام يجيب عن أسئلة أسماء النطاقات، ومنها عناوينها) وConnection وTLS (Transport Layer Security؛ قواعد حماية الاتصال بالتشفير والتحقق) وWaiting/TTFB (Time To First Byte؛ مدة الانتظار حتى وصول أول بايت من الرد بحسب أداة القياس) وDownload.
3. افحص headers (حقل أو مقدمة معلومات تضاف للبيانات بحسب الطبقة) والحجم المنقول ونوع البروتوكول.
4. استخدم **Performance** لتحديد long tasks وlayout وpaint.
5. استخدم **Elements** لرؤية DOM والـcomputed styles.

نصيحة

TTFB مرتفع قد يكون شبكة أو CDN (Content Delivery Network؛ شبكة خوادم تقدم المحتوى من نقاط موزعة قرب المستخدمين) أو PHP (اسم لغة برمجة تستخدم كثيرًا لمعالجة طلبات الويب على الخادم) أو Database (قاعدة بيانات: تخزين منظم يمكن البحث فيه وتعديله بقواعد). اربط Network trace مع server logs (سجلات أحداث وتشخيص يكتبها البرنامج) وprofiling قبل تحديد السبب.

مسائل عملية

▼ لماذا قد تظهر الصفحة بلا تنسيق لحظة؟

قد يصل HTML قبل CSS أو يتأخر stylesheet؛ يستطيع المتصفح بناء DOM لكنه يحتاج CSSOM لتنسيق الرسم.

▼ أين تبحث عن ملف JavaScript أعاد 404؟

ابدأ من Network لمعرفة URL (Uniform Resource Locator؛ عنوان يحدد موردًا وطريقة الوصول إليه) والحالة والمبادر، ثم Console لرؤية أثر فشل التحميل.

▼ هل `DOMContentLoaded` يعني أن كل الصور اكتملت؟

لا. يعني أن HTML حُلّل وأن DOM جاهز؛ قد تظل الصور وموارد أخرى قيد التحميل.

قِس ما يشعر به المستخدم

LCP، Largest Contentful Paint وقت ظهور أكبر عنصر محتوى مناسب للقياس في الجزء المرئي. **INP، Interaction to Next Paint** يقيس تأخر الاستجابة المرئية لتفاعلات المستخدم خلال الزيارة. **CLS، Cumulative Layout Shift** يقيس مقدار تحرك العناصر بشكل غير متوقع. دي مقاييس مختلفة؛ تحميل سريع لا يمنع زرًا بطيئًا أو نصًا يقفز مكانه.

Main thread — خيط العمل الرئيسي ينفذ كثيرًا من عمل الصفحة. **Event loop — دورة الأحداث** تنظم تنفيذ المهام. **Microtasks — مهام صغيرة مؤجلة** تُنفذ عند نقاط محددة بعد انتهاء التنفيذ الحالي وقبل فرصة العرض التالية؛ سلسلة لا تنتهي منها قد تؤخر الواجهة. مش لازم يحدث رسم بعد كل مهمة.

Layout thrashing تكرار قراءة مواقع العناصر وتغييرها بشكل يجبر المتصفح على إعادة الحساب كثيرًا. اجمع القراءات ثم التغييرات عندما يناسب. **Accessibility — إتاحة الاستخدام** تشمل إمكانية قراءة المحتوى واستخدامه بلوحة المفاتيح وأدوات المساعدة.

تدريب وحل: أكبر صورة تتأخر بينما HTML وصل بسرعة. افحص وقت بدء طلبها، حجمها، وأي كود يؤخر إظهارها. غيّر عاملًا واحدًا وقارن عدة تحميلات بنفس ظروف الشبكة. تحسن زمن الخادم وحده لا يثبت تحسن ظهور الصورة.

الخطوة التالية

كمّل في تحميل البرامج وترتيب مهام المتصفح بعد تنفيذ التجربة هنا.

تحميل البرامج وترتيب مهام المتصفح

تحميل البرامج وترتيب مهام المتصفح

ابدأ بعد تجربة تغيير صفحة وقياس حجمها. **Module — وحدة كود** ملف يعلن ما يتيحه وما يحتاجه من ملفات أخرى. **Dependency graph — رسم الاعتماد** يربط الملف بما يحتاجه؛ لو A يستورد B، لازم تتجهز B وفق قواعد التحميل قبل تنفيذ ما يعتمد عليها. هنربط التفاصيل باستجابة الصفحة.

JavaScript والتحميل

```
<script src="/app.js" defer></script>
<script type="module" src="/main.js"></script>
```

`defer` يسمح باستمرار parsing وينفذ بعد اكتمال DOM وبالترتيب. الـmodules مؤجلة افتراضيًا ولها dependency graph. أما `async` فينفذ عند اكتمال التحميل دون ضمان ترتيب.

تغيير layout بصورة متكررة داخل loop قد يسبب layout thrashing. اجمع القراءات ثم الكتابات، وحرّك العناصر غالبًا عبر `transform` عندما يناسب.

دورة حدث مبسطة

JavaScript يعمل عادة على main thread (خيط تنفيذ: مسار تعليمات داخل عملية) مع event loop. المهام الطويلة تؤخر input والرسم. قس Largest Contentful Paint وInteraction to Next Paint وCumulative Layout Shift.

ليه التحميل الثاني ممكن يختلف؟

Redirect — إعادة توجيه رد يطلب من المتصفح زيارة عنوان آخر؛ فتبدأ رسالة طلب جديدة، لكن مش لازم اتصال جديد. **Connection reuse — إعادة استخدام الاتصال** معناها إرسال طلب آخر على اتصال موجود بدل تكرار البداية. **Service Worker (ملف JavaScript يسجله الموقع لدى المتصفح؛ يستجيب لأحداث مثل طلب ملف، ويمكنه تقديم نسخة محفوظة وفق قواعد الموقع)** برنامج يسجله الموقع في المتصفح، ويمكنه التعامل مع بعض الطلبات واستخدام نسخ محلية حسب كوده.

في أدوات المطور **DevTools**، افتح لوحة **Network — الشبكة** قبل إعادة التحميل. رسم **Waterfall — التوقيت المتتابع** يعرض شريطًا لكل طلب يوضح متى بدأ وكم استغرق. شوف أول تحميل والثاني، وافتح تفاصيل طلب الصفحة. **Cookie** قيمة صغيرة يحفظها المتصفح ويرسلها مع الطلبات المناسبة؛ و**CORS** قواعد سماح لقراءة رد موقع آخر داخل JavaScript، هنفصلها في درسها.

تدريب وحل: اختفت مرحلة الاتصال في الطلب الثاني؛ هل ده يثبت أن الصفحة جاءت من التخزين المؤقت؟ لا. ممكن الاتصال اتعاد استخدامه والرد جاء من الخادم. ابحث عن عبارة تشير لنسخة محلية أو رؤوس الرد، وهي معلومات تصف الرسالة، ولا تستنتج من السرعة وحدها.

جهّز معمل الطلب والرد على جهازك

جهّز معمل الطلب والرد على جهازك

هنشغّل عميلًا وخادم بيانات على جهازك. **العميل** يطلب، و**الخادم** يرد. كل واحد له منفذ مختلف: رقم يحدد البرنامج المستقبل. نستخدم ملفات كاملة جاهزة، ونغيّر تجربة واحدة كل مرة.

التجهيز والتشغيل

1. ثبّت Node.js واختبر `node --version` كما في تجهيز الخادم المحلي. لا توجد حزم إضافية تحتاج تثبيتًا.
2. أنشئ مجلدًا جديدًا باسم `network-practice` . احفظ ملف المعمل الكامل lab.mjs فيه باستخدام حفظ الرابط باسم. تأكد أنه ليس `lab.mjs.txt` .
3. افتح PowerShell في المجلد من شريط عنوان مستكشف الملفات، ونفّذ:

```
node lab.mjs
```

4. افتح `http://127.0.0.1:8765/` . تظهر أزرار التجارب. الخادم الثاني على `http://127.0.0.1:8766` . اترك الطرفية مفتوحة؛ كل طلب يكتب سطرًا فيه مساره ورمزه ومدته ومعرّفه.
5. للإيقاف اضغط Ctrl+C في الطرفية. لو المنفذ مشغول أوقف نسخة المعمل السابقة. إعادة التشغيل تصفّر بيانات الإشعارات وعدّاد الطلبات.

ابدأ برد واحد

افتح `http://127.0.0.1:8766/api/products/1` . المتوقع بيانات دفتر: الاسم Notebook والسعر20 والعملةEGP. افتح أدوات المطور F12 ثم Network، وأعد التحميل. اختَر الطلب لتجد الحالة200، وهي نجاح تقديم الرد. غيّر آخر جزء إلى2؛ الحالة404 وتعني أن المنتج المطلوب غير موجود في المعمل.

لتشوف الرد ورؤوسه في **PowerShell على Windows**، افتح نافذة ثانية ونفّذ:

```
curl.exe -i http://127.0.0.1:8766/api/products/1
```

`-i` يظهر رؤوس الرد قبل محتواه. نكتب `curl.exe` لتشغيل البرنامج نفسه؛ بعض نسخ PowerShell تستخدم `curl` اسمًا مختصرًا لأداة أخرى. على macOS أو Linux استخدم `curl` بدل `curl.exe` .

خريطة التجارب

التجربة	المسار أو الزر	الدليل المتوقع
خادم واحد	`/api/products/1` على8766	200 وجسم JSON
وسيط واحد	زر Through one proxy	نفس المنتج؛ رأسان يحملان نفس معرّف الطلب
قراءة بين أصلين	زرا CORS	رد يصل في الحالتين، لكن JavaScript لا يقرأ إلا المسموح
نسخة مخزنة	`/cache` على8766	`max-age=2` ووسم نسخة؛ طلب شرطي يعيد304
سؤال دوري	Poll three times	ثلاثة طلبات وثلاثة أوقات
بث من الخادم	Receive three events	طلب ممتد يحمل ثلاثة أحداث
مسار مفقود	`/missing` على8766	404 ومعرّف طلب
فشل مقصود	`/failure` على8766	500 ومعرّف طلب؛ خطأ تعليمي متعمد
حد طلبات	`/limited` على8766 ثلاث مرات سريعًا	أول اثنين200 والثالث429 في نافذة10ثوانٍ

المعمل يستمع على جهازك فقط، ويشرح السلوك من غير حسابات أو بيانات شخصية. إعداداته تعليمية؛ لا يمثل خدمة منشورة أو نظام دفع. ملفه كامل ومتاح للقراءة، وكل درس يشرح الجزء المرتبط بتجربته.

رحلة الطلب داخل الخادم

رحلة الطلب داخل البنية الخلفية من الشبكة إلى الخادم والتطبيق وقاعدة البيانات ثم الاستجابة.

خادم واحد يوضح الفكرة

جهّز المعمل المحلي. افتح `/api/products/1` على منفذ8766: الخادم يقرأ المسار، يختار بيانات المنتج، ويكوّن JSON، ثم يرسل الحالة200 والجسم. غيّر1 إلى2 فتجد404. **Route — مسار معالجة** قاعدة تربط عنوانًا بعمل ينفذه البرنامج.

```
Browser → one server → choose route → build response → Browser
```

افحص سطر الطلب في الطرفية واربطه بالحالة في Network. دلوقتي عندنا سبب لفهم كل مرحلة. الوسيط في الدرس التالي يضيف جهة أخرى أمام هذا الخادم؛ مش شرطًا لعمل أول طلب.

قبل التفاصيل

هذا الدرس يتتبع الطلب بعد وصوله إلى البنية الخلفية. الهدف ليس حفظ أسماء الأدوات، بل معرفة حدود المسؤولية حتى تبحث عن العطل في الطبقة الصحيحة.

ما السيرفر؟

كلمة **Server** تصف دورًا، لا شكل جهاز بعينه. قد تعني:

- **جهازًا فعليًا أو Virtual Machine (جهاز افتراضي: بيئة تحاكي جهازًا وتشغل نظامًا وتطبيقات)** يوفّر CPU (Central Processing Unit؛ المعالج الرئيسي الذي ينفذ التعليمات) وRAM (Random Access Memory؛ ذاكرة العمل التي تحمل بيانات وتعليمات البرامج النشطة) وشبكة وتخزينًا.
- **برنامجًا** يستمع على عنوان IP (Internet Protocol؛ قواعد عنونة الحزم وتوجيهها بين الشبكات، وعنوان IP رقم يحدد واجهة في هذا السياق) ومنفذ ويجيب وفق Protocol، مثل Nginx أو PostgreSQL.
- **دورًا في تفاعل محدد:** البرنامج الذي يطلب خدمة هو Client، والذي يقدمها Server. وقد يكون التطبيق نفسه Client لخادم قاعدة البيانات.

يبدأ Server Process (عملية: نسخة برنامج تعمل بذاكرة وحالة وموارد) عادة بفتح Socket (واجهة اتصال ينشئها البرنامج؛ نقطة شبكة منها ترتبط عادة ببروتوكول وعنوان ومنفذ) وربطه بعنوان ومنفذ ثم الاستماع. عندما يصل اتصال، يقرأ الطلب ويتحقق منه وينفذ العمل ويرسل استجابة. تشغيل `localhost:8000` لا يعني وجود جهاز مستقل؛ يعني أن Process على جهازك يستمع محليًا على المنفذ `8000`.

يمكن لجهاز واحد تشغيل Web Server (خادم ويب يستقبل طلبات الويب ويعيد ملفات أو يمرر العمل للتطبيق) وApplication Server (برنامج يشغّل منطق التطبيق ويبني نتيجته) وDatabase (قاعدة بيانات: تخزين منظم يمكن البحث فيه وتعديله بقواعد) في التطوير، بينما تفصلها بيئة الإنتاج لأسباب التوسع والعزل والأمان. وعبارة «السيرفر وقع» غير دقيقة حتى نحدد: هل الجهاز متوقف، أم العملية، أم الشبكة، أم خدمة تابعة؟

من الإنترنت إلى التطبيق

خلّينا نتتبع طلبًا من أول دخوله للبنية الخلفية لحد رجوع النتيجة:

كل طبقة اختيارية ويمكن دمج عدة أدوار على جهاز واحد في مشروع صغير.

توزيع الشغل بين الخوادم

Load Balancer — موازن الأحمال وسيط يختار نسخة من الخدمة لكل اتصال أو طلب. **Backend** أو**Upstream** اسم للخادم الذي يمرر إليه الوسيط الطلب. **Layer 4** هنا مستوى النقل الذي يتعامل مع TCP (Transmission Control Protocol)؛ نقل بايتات بترتيب مع معالجة الفقد، مع احتمال فشل الاتصال) وUDP (User Datagram Protocol)؛ نقل رسائل مستقلة بلا ضمان مدمج لوصولها أو ترتيبها)، و**Layer 7** مستوى التطبيق الذي يفهم HTTP (Hypertext Transfer Protocol)؛ قواعد طلب الموارد والرد عليها في الويب)؛ الأرقام من نموذج OSI (Open Systems Interconnection)؛ نموذج تعليمي يقسم أدوار الاتصال إلى سبع طبقات)، نموذج تعليمي يقسم الاتصال لسبع مسؤوليات.

Health check — فحص الجاهزية طلب صغير للتأكد أن الخدمة تستجيب. **Round-robin** يوزع بالتناوب، و**Least-connections** يختار الأقل اتصالات نشطة. **Sticky sessions** تحاول إبقاء المستخدم على نفس النسخة؛ **Shared session store** يحفظ بيانات الجلسات في مكان تشترك فيه النسخ.

TLS termination — إنهاء TLS عند الوسيط (Transport Layer Security)؛ قواعد حماية الاتصال بالتشفير والتحقق) يعني أن الوسيط يفك حماية الاتصال الوارد. الاتصال التالي يحتاج قرار حماية مستقلًا؛ تشفير المتصفح لا يعني تلقائيًا تشفير كل الوصلات الداخلية.

موازن الحمل قد يرفع السعة والتوافر، ويضيف forwarding headers، أي حقولًا توضح معلومات الطلب الأصلي.

> ⚠
>
> **تنبيه**
>
> لا تثق في `X-Forwarded-For` من أي عميل مباشرة. اضبط قائمة trusted proxies (وسطاء محددون تثق في المعلومات التي يضيفونها)، وإلا يمكن تزوير عنوان العميل.

أمثلة: Nginx/HAProxy أو خدمات سحابية مثل AWS Elastic Load Balancing. والـLoad Balancer ليس بديلًا لقاعدة البيانات أو كود التطبيق.

Virtual Host و Web Server

يستقبل Nginx أو Apache الاتصال ويختار الموقع باستخدام IP/port (منفذ منطقي برقم يميز جهة استقبال في بروتوكول النقل) واسم `Host` أو SNI (Server Name Indication؛ حقل في بداية TLS يحدد اسم الخادم المطلوب)/TLS configuration.

- **Static file:** صورة أو CSS (Cascading Style Sheets؛ قواعد مظهر عناصر الصفحة) أو JS (اختصار JavaScript، لغة برمجة لتعليمات وتفاعل الصفحة) يمكن إعادته مباشرة، مع Cache (نسخة محفوظة لتقليل تكرار القراءة أو الحساب) وCompression (ضغط: تمثيل نفس المحتوى ببيانات أقل في ضغط الويب المعتاد).
- **Dynamic (ديناميكي: نتيجة يبنيها البرنامج عند الطلب) PHP (اسم لغة برمجة تستخدم كثيرًا لمعالجة طلبات الويب على الخادم):** يُمرر إلى PHP runtime (بيئة تنفيذ اللغة والخدمات التي تحتاجها؛ غير معنى وقت التشغيل الزمني)، غالبًا PHP-FPM (PHP FastCGI Process Manager؛ برنامج يدير عمليات PHP التي تستقبل عملًا من خادم الويب) عبر FastCGI.
- **Reverse proxy:** قد يمرر الطلب إلى خدمة أخرى.

> **تصحيح**
>
> Nginx لا ينفذ PHP داخل عمليته. يمرر الطلب إلى PHP-FPM ثم يستقبل الناتج. Apache يمكن أن يستخدم PHP-FPM أيضًا أو إعدادات أخرى.

PHP لغة تُستخدم كثيرًا لبرامج الخادم. **PHP-FPM، PHP FastCGI Process Manager** برنامج يدير عمليات PHP الجاهزة لمعالجة الطلبات. **FastCGI** قواعد تبادل الطلب والرد بين خادم الويب وبيئة تنفيذ التطبيق؛ مش نفس بروتوكول المتصفح. **Worker (وحدة تنفيذ تتولى مهمة أو طلبًا؛ قد تكون عملية أو خيطًا حسب النظام)** هنا عملية جاهزة لمعالجة طلب، ومش لازم عامل طابور في الخلفية.

PHP-FPM والتطبيق

يدير PHP-FPM مجموعة workers. يستقبل FastCGI parameters والطلب، يشغّل entry point مثل `public/index.php`، ثم يعيد headers (حقل أو مقدمة معلومات تضاف للبيانات بحسب الطبقة)/body (جسم الرسالة: المحتوى المرسل مثل نص صفحة أو بيانات طلب) لخادم الويب.

Entry point — نقطة الدخول الملف الذي تبدأ منه معالجة التطبيق. **Route — المسار البرمجي** قاعدة تربط عنوان الطلب بالكود المناسب. **Middleware — خطوة وسيطة** تنفذ فحصًا مشتركًا قبل المعالج أو بعده، مثل التحقق من تسجيل الدخول. **Business logic — قواعد التطبيق** تحدد ما الذي يفترض أن يحصل، مثل حساب سعر طلب. **Queue — طابور عمل** يحفظ مهام لمعالجتها لاحقًا.

التطبيق بدوره:

1. يطابق Route.
2. يشغّل Middleware.

3. يتحقق من المدخلات والمصادقة والصلاحيات.
4. ينفذ business logic.
5. يستخدم Database/Cache/Queue/API (Application Programming Interface؛ اتفاق يسمح لبرنامج بطلب بيانات أو عملية من مكوّن آخر) عند الحاجة.
6. يبني Response (رد: رسالة النتيجة التي يعيدها الخادم).

```
<?php
declare(strict_types=1);

header('Content-Type: application/json; charset=utf-8');

try {
    $product = ['id' => 42, 'name' => 'Keyboard'];
    echo json_encode($product, JSON_THROW_ON_ERROR);
} catch (Throwable $e) {
    error_log($e);
    http_response_code(500);
    echo json_encode(['error' => 'Internal server error']);
}
```

لا ترسل stack trace (سلسلة استدعاءات توضح أين مر التنفيذ عند الخطأ) أو كلمات مرور قاعدة البيانات للمستخدم.

رجوع الاستجابة

تعود النتيجة إلى Web Server ثم Load Balancer/CDN (Content Delivery Network؛ شبكة خوادم تقدم المحتوى من نقاط موزعة قرب المستخدمين) ثم الشبكة والمتصفح. قد تضيف طبقات وسيطة headers أو compression أو caching.

راقب:

- `Content-Type` الصحيح.
- Status code (رقم يصف نتيجة معالجة طلب HTTP) الصحيح.
- `Cache-Control` للبيانات الخاصة والعامة.
- Security headers (حقول رد تضبط سلوك حماية في المتصفح حسب نوع كل حقل) الملائمة.
- Request ID (معرّف يميز طلبًا لتتبع أحداثه) لتتبع نفس الطلب في logs (سجلات أحداث وتشخيص يكتبها البرنامج) عبر الخدمات.
- زمن كل طبقة، لا زمن PHP فقط.

حالات مهمة

Cache hit

```
Browser/CDN Cache → response
PHP and database are skipped
```

Static file

```
Nginx → /public/app.css → response
PHP is skipped
```

Dynamic request

```
Nginx → PHP-FPM → App → Database → response
```

فشل backend

قد يعيد proxy (وسيط يستقبل اتصالًا أو طلبًا ويمرره لجهة أخرى) حالة `502 Bad Gateway` عندما لا يحصل على استجابة صحيحة من upstream، أو `504 Gateway Timeout` عند انتهاء المهلة. أما `500` فعادة خطأ داخل التطبيق/الخادم الذي عالج الطلب.

تدريب نهائي

1. افتح Network tab وسجّل method (طريقة الطلب: نوع العملية المطلوبة مثل القراءة) وstatus وcontent type.
2. أعد تحميل الصفحة ولاحظ `304` أو `from memory cache`.
3. أضف ملف CSS ثابتًا وendpoint (عنوان يقدم عملية أو بيانات من التطبيق) PHP ديناميكيًا وقارن.
4. أرسل request ID في header وسجله داخل PHP.
5. ارسم مسارك الفعلي وحدد الطبقات غير الموجودة بدل نسخ المخطط حرفيًا.

مسائل عملية

▼ أعاد Nginx خطأ `502`. هل يعني أن PHP نفّذ الطلب وأخطأ؟

ليس بالضرورة. 502 يعني أن الـgateway لم يحصل على استجابة صالحة من upstream؛ افحص PHP-FPM والاتصال والمهلة قبل منطق التطبيق.

▼ متى يستطيع load balancer إرسال طلبين للمستخدم نفسه إلى خادمين مختلفين؟

عند توزيع الطلبات دون sticky sessions. لذلك يجب ألا تعتمد الحالة المهمة على ذاكرة خادم واحد فقط.

▼ من يخدم ملف CSS ثابتًا غالبًا؟

يمكن للـweb server أو CDN خدمته مباشرة، فلا حاجة لتمريره إلى PHP في الإعداد المعتاد.

المهلات واستهلاك الموارد

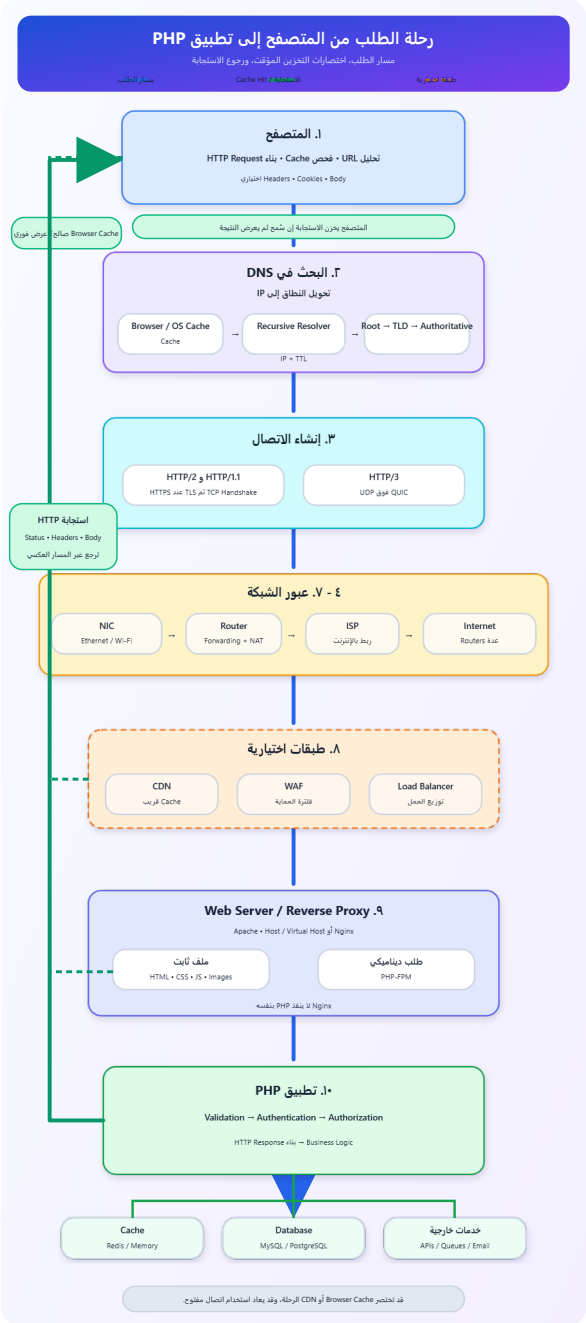
Timeout — مهلة انتظار حد زمني للعمل أو انتظار الرد. لو مهلة العميل ثانيتين، مثال ميزانية ممكنة: نخصص 0.6 ثانية لاستعلام البيانات، وننهي عمل التطبيق خلال 1.3 ثانية، ويوقف الوسيط الانتظار عند 1.6 ثانية، فيتبقى وقت لنقل الرد. دي حدود تصميم توضيحية تُراجع بالقياس، ومش مدد نجمعها كلها كأنها خطوات متتابعة؛ جزء منها يحيط بعمل أجزاء أخرى. إغلاق اتصال العميل لا يوقف عمل الخادم دائمًا تلقائيًا.

لو كل عمليات PHP-FPM مشغولة، طلب جديد قد ينتظر في طابور. زيادة عددها بلا حساب RAM قد تنتهي بـ**OOM، Out Of Memory**، أي عدم كفاية الذاكرة. **Unix socket** قناة اتصال محلية بين عمليات على نفس النظام، بينما TCP يصلح لاتصال محلي أو بين أجهزة. الحاوية Container بيئة تشغيل معزولة نسبيًا لتطبيق، وقد تتصل بحاوية أخرى عبر TCP.

تدريب محلول: الوسيط يعطي 504 بعد ثانية بينما التطبيق يحتاج 1.5 ثانية. أول سؤال هل المعالجة نفسها بطيئة أم المهلة أقصر من العقد المطلوب؟ راجع التوقيت في السجلات ثم اضبط الميزانية؛ رفع كل المهلات بلا قياس قد يخفي عطلًا ويزيد الطلبات المنتظرة.

ماذا يحدث بعد الضغط على Enter؟

1. يحلل المتصفح الرابط ويفحص الـcache (نسخة محفوظة لتقليل تكرار القراءة أو الحساب) والسياسات المحلية.
2. يحوّل DNS اسم المضيف إلى عنوان IP إذا لم توجد إجابة مخزنة.
3. يختار نظام التشغيل واجهة الشبكة والمسار، وتتحرك البيانات عبر Wi-Fi (اتصال لاسلكي بالشبكة المحلية؛ لا يضمن اتصالها بالإنترنت) أو Ethernet (معيار شائع للاتصال على شبكة محلية سلكية) والراوتر والـISP (Internet Service Provider؛ شركة أو جهة توفر الاتصال بالإنترنت).
4. يُنشأ TCP (Transmission Control Protocol؛ نقل بايتات بترتيب مع معالجة الفقد، مع احتمال فشل الاتصال) connection لـHTTP/1.1 أو HTTP/2، أو اتصال QUIC (اسم بروتوكول نقل فوق UDP يضيف تدفقات موثوقة وحماية اتصال) فوق UDP (User Datagram Protocol؛ نقل رسائل مستقلة بلا ضمان مدمج لوصولها أو ترتيبها) لـHTTP/3.
5. مع HTTPS تتم TLS (Transport Layer Security؛ قواعد حماية الاتصال بالتشفير والتحقق) handshake (مصافحة: رسائل البداية للاتفاق على الاتصال) للتحقق من هوية الخادم وإنشاء قناة مشفرة.
6. يرسل المتصفح HTTP Request يحتوي method (طريقة الطلب: نوع العملية المطلوبة مثل القراءة) وpath (مسار المورد داخل العنوان، مثل /products) وheaders (حقل أو مقدمة معلومات تضاف للبيانات بحسب الطبقة)، وقد يحتوي body (جسم الرسالة: المحتوى المرسل مثل نص صفحة أو بيانات طلب).
7. قد يستقبل الطلب CDN (Content Delivery Network؛ شبكة خوادم تقدم المحتوى من نقاط موزعة قرب المستخدمين) أو WAF (Web Application Firewall؛ نظام يفحص طلبات الويب ويطبق قواعد حماية) أو load balancer (موازن أحمال يوزع العمل على نسخ خدمة) قبل الخادم الأصلي.
8. يعيد web server (خادم ويب يستقبل طلبات الويب ويعيد ملفات أو يمرر العمل للتطبيق) ملفًا ثابتًا، أو يمرر الطلب الديناميكي إلى تطبيق مثل PHP (اسم لغة برمجة تستخدم كثيرًا لمعالجة طلبات الويب على الخادم) عبر PHP-FPM (PHP FastCGI Process Manager؛ برنامج يدير عمليات PHP التي تستقبل عملًا من خادم الويب).
9. يتحقق التطبيق من المدخلات والهوية والصلاحيات، وقد يقرأ cache أو database (قاعدة بيانات: تخزين منظم يمكن البحث فيه وتعديله بقواعد).
10. تعود HTTP Response (رد: رسالة النتيجة التي يعيدها الخادم) وفيها status code (رقم يصف نتيجة معالجة طلب HTTP) وheaders وbody، ثم يفسر المتصفح المحتوى ويعرضه.



افتح الرسم بالحجم الكامل

ملاحظة

هذه خريطة شاملة وليست طريقًا إجباريًا. قد يجيب browser cache أو CDN من غير الوصول إلى PHP، وقد لا يملك التطبيق الصغير load balancer أو database أصلًا.

من المسؤول عن ماذا؟

المكوّن	مسؤوليته الأساسية
Browser أو Client	بناء الطلب، وإدارة cache/cookies، وتفسير الاستجابة
DNS	تحويل الاسم إلى عناوين مناسبة
TCP أو QUIC	نقل البيانات وخصائص الاتصال
TLS	التشفير، وسلامة البيانات، والتحقق من هوية الخادم
HTTP	معنى الطلب والاستجابة
Web server	استقبال HTTP، وخدمة الملفات الثابتة، وتمرير العمل الديناميكي
PHP	منطق التطبيق وبناء الاستجابة
Database	حفظ البيانات واسترجاعها

الخوادم الوسيطة وتوزيع الطلبات

الفرق العملي بين الخادم والوسيط وموازن الأحمال وبوابة APIs، ومسؤوليات كل طبقة ومخاطر الإعداد الخاطئ.

جرّب وسيطًا واحدًا

في المعمل المحلي، اضغط Through one proxy. المتصفح يطلب `/proxy` من8765؛ هذا الخادم يطلب المنتج من8766 ثم يرجع الرد. **Upstream — الخادم التالي الذي يطلب منه الوسيط** هو8766 هنا.

في Network افتح رد `/proxy` : `X-Lab-Proxy: forwarded` يوضح المرور، و `X-Request-Id` و `X-Upstream-Request-Id` يحملان نفس القيمة لربط الطلبين. القيمة فريدة للتجربة؛ مش كلمة تحفظها. قارن سطري الطرفية بنفس المعرف. افهم هذا الطريق قبل توزيع العمل على عدة خوادم.

قبل التفاصيل

كل طبقة أمام التطبيق يجب أن تحل مشكلة واضحة. زيادة proxies بلا حاجة تضيف نقاط فشل، بينما توزيع المسؤوليات الصحيح يحسن الأمان والتوسع.

لماذا توجد طبقات أمام التطبيق؟

يمكن للعميل الاتصال بتطبيق واحد مباشرة، لكن الأنظمة الحقيقية تحتاج TLS (Transport Layer Security؛ قواعد حماية الاتصال بالتشفير والتحقق) وحماية وتوزيع حمل وتوجيهًا ومراقبة وسياسات موحدة. لذلك توضع طبقات أمام التطبيق. قد يجمع منتج واحد أكثر من دور، لكن هذا لا يلغي الفرق بين المسؤوليات.

```
Client ⇄ DNS (name lookup only)
Client → CDN/WAF → Load Balancer → Reverse Proxy → API Gateway
                                                    ├─ Identity
                                                    ├─ Orders
                                                    └─ Catalog → Cache / DB / Queue
```

ليست كل الطبقات إلزامية. تطبيق صغير قد يستخدم Nginx كتأمين TLS وReverse Proxy (وسيط يستقبل الطلبات نيابة عن خوادم خلفه) أمام تطبيق واحد، بينما منظومة خدمات متعددة قد تحتاج Gateway.

Forward Proxy

يعمل **Forward Proxy (وسيط يستقبل اتصالًا أو طلبًا ويمرره لجهة أخرى)** نيابة عن العميل. يعرف العميل أنه سيرسل اتصالاته إلى الوسيط، ثم يتصل الوسيط بالخارج باسمه. يُستخدم في شبكات الشركات لتطبيق سياسة خروج، وحجب وجهات، وتسجيل الاستخدام، أو التخزين المؤقت.

```
Employees → Corporate Forward Proxy → Internet servers
```

قد يرى الخادم الخارجي عنوان الوسيط، لكن هذا لا يجعل الاتصال مجهولًا؛ الوسيط نفسه قد يسجل الهوية والوجهة، ويحدد إعداد TLS ما يستطيع فحصه.

Reverse Proxy

يعمل **Reverse Proxy** نيابة عن الخوادم. يطلب العميل نطاق التطبيق طبيعيًا، فيصل إلى وسيط يختار Upstream ويرسل إليه الطلب. من وظائفه:

- TLS termination وتجديد الشهادات مركزيًا.
- توجيه Paths (مسار المورد داخل العنوان، مثل /products) أو Hosts إلى خدمات مختلفة.
- Compression (ضغط: تمثيل نفس المحتوى ببيانات أقل في ضغط الويب المعتاد) وCaching وحدود حجم الطلب.
- إضافة Request ID (معرّف يميز طلبًا لتتبع أحداثه) وSecurity Headers (حقول رد تضبط سلوك حماية في المتصفح حسب نوع كل حقل).
- إخفاء العناوين الداخلية ومنع الوصول المباشر إليها.

Nginx وHAProxy وEnvoy أمثلة تستطيع أداء بعض هذه الأدوار. الـReverse Proxy لا يصلح تلقائيًا أخطاء Validation والمصادقة والصلاحيات داخل التطبيق.

Upstream الخدمة التي يمرر إليها الوسيط الطلب، و**TLS termination** فك اتصال TLS عنده؛ الوصلة التالية تحتاج حماية مستقلة. **Request ID** قيمة تربط سجلات الطلب نفسه. **Compression** ضغط لتقليل الحجم، و**Validation** فحص أن المدخلات مناسبة قبل استخدامها.

مقارنة الأدوار

المكوّن	يعمل نيابة عن	القرار الأساسي	مثال
Server	الخدمة	كيف يعالج الطلب	تطبيق أو قاعدة بيانات
Forward Proxy	العميل	هل وإلى أين يخرج الاتصال	شبكة شركة
Reverse Proxy	الخوادم	لأي Upstream يمر الطلب	TLS وتوجيه النطاق
Load Balancer	نسخ الخدمة	أي نسخة سليمة تستقبل الحمل	توزيع Workers (وحدة تنفيذ تتولى مهمة أو طلبًا؛ قد تكون عملية أو خيطًا حسب النظام)
API Gateway (بوابة تجمع دخول عدة واجهات خدمة وتطبق سياسات مشتركة)	منظومة APIs	أي API وهوية وسياسة تنطبق	مدخل Microservices

قد يكون المنتج Reverse Proxy وLoad Balancer معًا، وقد تحتوي منصة Gateway على الدورين. راجع المسؤولية وحدود الثقة والفشل، لا اسم المنتج فقط.

مسائل عملية

▼ ما الفرق المركزي بين forward وreverse proxy؟

الـforward proxy يعمل نيابة عن العملاء نحو الإنترنت؛ الـreverse proxy يقف أمام الخوادم ويتلقى طلبات العملاء نيابة عنها.

▼ لماذا لا نثق بأي `X-Forwarded-For` ؟

لأن العميل يستطيع تزويره. ثق فقط في header أعاد بناءه proxy موثوق، واضبط قائمة الـproxies الموثوقة.

▼ هل API gateway بديل لمنطق الصلاحيات داخل الخدمة؟

لا. يمكنه تطبيق سياسات مشتركة، لكن الخدمة تظل مسؤولة عن قواعد الوصول الخاصة بمواردها.

الخطوة التالية

كمّل في توزيع الحمل وسياسات الوسيط بعد تنفيذ التجربة هنا.

توزيع الحمل وسياسات الوسيط

توزيع الحمل وسياسات الوسيط

ابدأ بعد تجربة وسيط واحد. لما تضيف أكثر من نسخة للخدمة، نحتاج قاعدة لاختيار نسخة، وحدودًا للانتظار وإعادة المحاولة، ومعرفة أي معلومات واردة من وسيط نثق فيه.

اربط الاسم بقرار فعلي

لو عندك نسختان من التطبيق، **Load balancing** يختار مين يستقبل الطلب. **Timeout — مهلة** أقصى وقت ننتظره، و**Retry — إعادة محاولة** إرسال جديد بعد فشل، و**Budget — حد للمحاولات** يمنع الاستمرار بلا نهاية. **Health check — فحص حالة** يختبر قدرة نسخة على العمل.

Token — قيمة اعتماد يحملها العميل لإثبات حق وفق تحقق الخادم، و**Quota — حصة** إجمالي استخدام مسموح خلال فترة. **Business logic — قواعد التطبيق** مثل ملكية فاتورة أو صلاحية تعديلها؛ مكان القرار النهائي الخدمة التي تملك البيانات. **Rollback — رجوع لإعداد سابق** وسيلة تراجع عندما يسبب التعديل مشكلة. هنستخدم الأسماء دي لشرح قرار، مش كقائمة أدوات مطلوب تثبيتها.

Load Balancer

يوزع **Load Balancer (موازن أحمال يوزع العمل على نسخ خدمة)** العمل بين نسخ متعددة لرفع السعة والتوافر:

```
Load Balancer
  ├─ App A: healthy
  ├─ App B: healthy
  └─ App C: unhealthy → removed by health check
```

قد يعمل في Layer 4 على TCP (Transmission Control Protocol)؛ نقل بايتات بترتيب مع معالجة الفقد، مع احتمال فشل الاتصال)/UDP (User Datagram Protocol)؛ نقل رسائل مستقلة بلا ضمان مدمج لوصولها أو ترتيبها)، أو Layer 7 ويفهم HTTP (Hypertext Transfer Protocol)؛ قواعد طلب الموارد والرد عليها في الويب). من خوارزمياته Round Robin وLeast Connections وConsistent Hashing. ميّز بين **Liveness** التي تسأل هل العملية حية و**Readiness** التي تسأل هل تستطيع استقبال عمل الآن.

إعادة المحاولة بلا حدود تضاعف الضغط على خدمة متعبة. اضبط Timeout وRetry (إعادة محاولة عملية بعد فشل أو تأخر، مع حساب أثر التكرار) Budget، واجعل العملية Idempotent أو استخدم Idempotency Key عندما يمكن تكرارها.

API Gateway

الـ**API (Application Programming Interface؛ اتفاق يسمح لبرنامج بطلب بيانات أو عملية من مكوّن آخر) Gateway** نقطة دخول منظمة لواجهات عدة خدمات، ويركز على سياسات الـAPI. يمكنه:

- مطابقة Routes وإصدارات `/v1` و`/v2`.

- التحقق الأولي من Token وتمرير هوية موثوقة.
- Rate Limits وQuotas حسب المستخدم أو التطبيق.
- تحويل Protocol أو شكل Request (طلب: رسالة يرسلها العميل ليطلب موردًا أو عملية)/Response (رد: رسالة النتيجة التي يعيدها الخادم).
- تجميع نتائج عدة خدمات لعميل محدد.
- توحيد Metrics وTraces وAccess Logs (سجلات أحداث وتشخيص يكتبها البرنامج).

تبقى صلاحية الوصول إلى المورد داخل الخدمة المالكة للبيانات. ولا تضع كل Business Logic في الـGateway؛ وإلا يتحول إلى تطبيق مركزي ضخم وعنق زجاجة للنشر والاختبار.

العناوين الحقيقية والثقة

يضيف الوسطاء `Forwarded` أو `X-Forwarded-For` و`X-Forwarded-Proto`. لا تثق بها من أي اتصال؛ يستطيع العميل تزويرها. امسح القيم غير الموثوقة عند الحافة، واضبط التطبيق على Trusted Proxies (وسطاء محددون تثق في المعلومات التي يضيفونها) معروفة فقط.

لا تمرر Authorization أو Cookies إلى Upstream غير مقصود، وحدد أحجام Headers (حقل أو مقدمة معلومات تضاف للبيانات بحسب الطبقة) وBody (جسم الرسالة: المحتوى المرسل مثل نص صفحة أو بيانات طلب) والمهل، واحم لوحة إدارة الـGateway، واستخدم TLS داخليًا عندما يتطلب Threat Model ذلك.

الفشل والمراقبة

كل طبقة تضيف زمنًا ونقطة فشل. شغّل أكثر من نسخة للطبقات الحرجة، واستخدم Health Checks واقعية، وانشر Configuration تدريجيًا مع Rollback. مرر Correlation/Trace ID وقس زمن كل Hop ومعدل الأخطاء ورفض Rate Limit وتشبع الاتصالات.

`502 Bad Gateway` يعني غالبًا أن الوسيط لم يحصل على استجابة صالحة من Upstream، و`504 Gateway Timeout` يعني انتهاء مهلة الانتظار. اربط الرمز بسجلات الطرفين والـTrace قبل الحكم.

اختار طريقة توزيع وفكّر في إعادة المحاولة

Round robin يوزع بالتناوب؛ مناسب كبداية لطلبات متقاربة الكلفة. **Least connections** يميل للنسخة الأقل اتصالات نشطة؛ يفيد مع اتصالات طويلة متفاوتة، لكنه لا يقيس حمل كل اتصال بدقة. **Consistent hashing** يربط قيمة مثل معرّف مستخدم بخادم بطريقة تقلل تغيّر التوزيع عند إضافة خوادم.

Session affinity محاولة إرسال العميل لنفس النسخة. ده قد يخفي اعتمادًا على حالة في ذاكرة خادم واحد؛ لو وقع الخادم، الحالة لا تصبح مشتركة من نفسها. **Liveness** يسأل هل العملية حية، و**Readiness** يسأل هل تقدر تستقبل طلبًا الآن.

Retry budget — ميزانية إعادة المحاولة حد لعدد المحاولات أو زمنها. لو ثلاث طبقات تعمل ثلاث محاولات لكل منها، قد يصل العمل في أعمق طبقة إلى 27 محاولة لطلب أصلي واحد. **Idempotency key** معرّف للعملية يحتفظ الخادم بنتيجته لمنع تكرار أثرها.

تدريب وحل: طلبات قراءة قصيرة مستقلة يمكن توزيعها بالتناوب. اتصال WebSocket، قناة مستمرة في الاتجاهين، يظل عادة على النسخة التي قبلته طول حياته؛ اختيار نسخة أقل اتصالات قد يفيد للاتصالات الجديدة. لو انقطع واتصل العميل من جديد فلا نفترض استعادة الرسائل أو الحالة تلقائيًا.

التخزين المؤقت وضغط بيانات الويب

Freshness وValidation وCache-Control وETag وVary وCDN وضغط gzip وBrotli.

ثلاث لحظات لنفس النسخة

Cache — نسخة محفوظة لإعادة الاستخدام تقلل طلب البيانات من جديد. في المعمل `/cache` يرد بوسم `ETag: "lesson-v1"`، أي اسم نسخة، و `Cache-Control: public, max-age=2`، أي السماح بالتخزين لمدة صلاحية ثانيتين.

اللحظة	ما يحدث؟	النتيجة
أول طلب	لا توجد نسخة	200 مع المحتوى
طلب عادي خلال الصلاحية	المتصفح قد يستخدم النسخة إن كانت سياسته تسمح	لا نحتاج تنزيل الجسم من الخادم
بعد انتهاء الصلاحية	نسأل هل تغير وسم النسخة؟	304 إذا لم تتغير، فنستخدم الجسم المحفوظ

جهّز المعمل ثم نفّذ:

```
curl.exe -i http://127.0.0.1:8766/cache
Set-Content -LiteralPath cache-headers.txt -Value 'If-None-Match: "lesson-v1"' -Encoding ascii
curl.exe -i -H "@cache-headers.txt" http://127.0.0.1:8766/cache
```

`Set-Content` يكتب رأس الطلب في ملف تدريب، و `-H "@cache-headers.txt"` يقرأه كما هو؛ ده يحافظ على علامات التنصيص في إصدارات PowerShell المختلفة.

الأول200، والثاني304 بلا جسم. `If-None-Match` يرسل وسم النسخة الموجودة لدينا. curl لا يعمل مخزن متصفح هنا؛ الأمر الثاني **يحاكي سؤال التحقق** صراحة. في المتصفح افتح Network واترك Disable cache غير محددة، ولا تعتمد على Refresh وحده كطلب عادي؛ إعادة التحميل قد تفرض تحققًا. بعد ما تشرح التسلسل، اقرأ السياسات والضغط تحت.

قبل التفاصيل

الـcache (نسخة محفوظة لتقليل تكرار القراءة أو الحساب) يعيد استخدام نتيجة سابقة لتقليل الزمن والعمل. السؤال الآمن ليس «هل نخزن؟» فقط، بل من يستطيع التخزين، وكم المدة، وكيف نتحقق من التغيير.

لماذا نخزن الاستجابة؟

الـcache تقلل latency (زمن انتظار عملية واحدة) واستهلاك الشبكة والحمل على PHP (اسم لغة برمجة تستخدم كثيرًا لمعالجة طلبات الويب على الخادم) وقاعدة البيانات. قد توجد نسخ في المتصفح أو proxy (وسيط يستقبل اتصالًا أو طلبًا ويمرره لجهة أخرى)/CDN (Content Delivery Network؛ شبكة خوادم تقدم المحتوى من نقاط موزعة قرب المستخدمين) أو التطبيق، ولكل طبقة سياسة مستقلة.

Validationو Freshness

```
Cache-Control: public, max-age=60, s-maxage=300
ETag: "product-42-v7"
Vary: Accept-Encoding, Accept-Language
```

- `max-age` عمر صلاحية النسخة بالثواني؛ يسري على المخازن الخاصة والمشتركة ما لم تخصص سياسة أخرى.
- `s-maxage` يحدد عمر النسخة في المخازن المشتركة ويتقدم فيها على max-age.
- `private` يسمح بالتخزين الخاص ولا يسمح عادة لـCDN بالمشاركة.
- `no-store` يطلب عدم التخزين.
- `no-cache` لا يعني عدم التخزين؛ يعني إعادة التحقق قبل الاستخدام.
- `Vary` يجعل مفتاح النسخة يعتمد على headers (حقل أو مقدمة معلومات تضاف للبيانات بحسب الطبقة) محددة.

بعد انتهاء freshness يرسل العميل `If-None-Match`. إذا لم يتغير المورد يعيد الخادم `304 Not Modified` بلا body (جسم الرسالة: المحتوى المرسل مثل نص صفحة أو بيانات طلب). ويمكن استخدام `Last-Modified` و`If-Modified-Since` عندما يناسب.

مثال PHP

```
<?php
// Save inside the downloadable php-labs folder.
require __DIR__ . '/http-router.php';
```

لا تجعل بيانات مستخدم خاصة `public`، ولا تعتمد على query string (معاملات بعد علامة ? في العنوان، مثل name=Omar) عشوائي كسياسة invalidation.

الضغط

gzip وBrotli يقللان الحجم النصي مثل HTML (Hypertext Markup Language؛ لغة وصف بنية الصفحة وعناصرها) وCSS (Cascading Style Sheets؛ قواعد مظهر عناصر الصفحة) وJSON (JavaScript Object Notation؛ صيغة نصية لترتيب البيانات في أسماء وقيم وقوائم). طبّق الضغط غالبًا في Web Server (خادم ويب يستقبل طلبات الويب ويعيد ملفات أو يمرر العمل للتطبيق)/CDN، ولا تضغط صورًا مضغوطة أصلًا بلا قياس. يجب أن تراعي cache قيمة `Accept-Encoding` عادة عبر `Vary`.

Checklist

- حدّد من يستطيع التخزين ومدة الصلاحية.
- استخدم fingerprinted filenames للأصول الثابتة طويلة العمر.
- اختبر invalidation قبل مدة طويلة.
- لا تخزن استجابة فيها `Set-Cookie` أو بيانات شخصية بصورة عامة.
- راقب hit ratio والحجم والزمن.

مرجع

- RFC 9111: HTTP Caching

مسائل عملية

▼ لماذا لا نخزن صفحة الحساب في shared cache؟

لأنها مخصصة لمستخدم وقد تتسرب لغيره. استخدم سياسة private/no-store المناسبة ولا تجعل CDN يشاركها.

▼ ماذا يعني رد `304` ؟

أن النسخة المخزنة ما زالت صالحة وفق validator؛ لا يرسل الخادم body كاملًا ويستخدم العميل نسخته.

▼ متى نحتاج `Vary: Accept-Encoding` ؟

عندما تختلف الاستجابة حسب الترميز مثل gzip أو br، كي لا يعيد cache نسخة لا يستطيع العميل فكها.

نسخة قديمة وقت التحديث أو العطل

stale — نسخة انتهى عمر استخدامها المباشر لا يعني أنها تغيرت فعلًا، بل أنها تحتاج سياسة أو تحققًا قبل إعادة استخدامها. **stale-while-revalidate** يسمح ضمن مهلة محددة بإرجاع النسخة القديمة أثناء التحقق من تحديثها. **stale-if-error** يسمح، حسب السياسة والعميل، بنسخة قديمة عند عطل بدل إظهار الخطأ. ما نطبقش ده بلا حدود على قرارات أو بيانات شخصية حساسة.

Cache key — مفتاح النسخة يحدد أي طلبات تشترك في الرد المحفوظ. إهمال مدخل يغيّر الرد قد يسبب **Cache poisoning — تلويث التخزين المؤقت**، بحيث تُحفظ نتيجة غير مناسبة ويأخذها آخرون. **Side channel — قناة تسريب جانبية** استنتاج سر من أثر مثل تغير حجم الرد؛ ضغط سر بجوار نص يتحكم فيه مهاجم قد يخلق هذا النوع من المخاطر.

تدريب وحل: صفحة أسعار عامة لها نسختان عربية وإنجليزية وضغط مختلف. ميّز اللغة والضغط في سياسة النسخ، مثل Vary المناسب، ولا تشارك صفحة حساب شخصية بين المستخدمين. اختيار no-store للبيانات الحساسة يمنع التخزين المطلوب امتثاله؛ وجود Set-Cookie وحده لا يمنع كل مخزن مشترك تلقائيًا.

امشِ مع الطلب الشرطي خطوة خطوة

افتح المختبر المحلي، وشغّل `php -S 127.0.0.1:8097 http-router.php` ، وبعدها `php http-client-lab.php` في طرفية ثانية. اقرأ `http-router.php` و `src/Http.php` : المسار يقبل GET وHEAD فقط، يحسب ETag للتمثيل الحالي، يحلل قائمة وسوم، ويستخدم المقارنة الضعيفة مع If-None-Match. يعني `W/"abc"` ممكن يطابق `"abc"` ، والنجمة `*` تطابق موردًا موجودًا. التقسيم عند كل فاصلة غلط لأن الوسم داخل التنصيص ممكن يحتوي فاصلة. تطابق GET/HEAD يرجع 304 بلا body، وHEAD غير المطابق يرجع 200 بالهيدرز بلا body. المثال بيرفض الطرق الأخرى بـ405؛ نقطة كتابة عامة تستخدم 412 عند فشل شرط If-None-Match. الهيدر غير الصحيح يرجع 400 حسب سياسة المثال.

تدريب: ابعت وسمًا مختلفًا، ووسمًا ضعيفًا مطابقًا داخل قائمة، و `*` . المتوقع 200 ثم 304 ثم 304. الوسم يخص التمثيل المختار فعلًا؛ لو الضغط أو اللغة غيروا البايتات، اضبط استراتيجية النسخ وهيدرز Vary المناسبة.

RFC 9110, If-None-Match

السماح بقراءة البيانات بين المواقع

تعريف Origin وقيود المتصفح وSimple Requests وPreflight والـCredentials والإعداد الآمن.

نفس الرد: مرة ممنوع قراءته ومرة مسموح

جهّز المعمل المحلي وافتح صفحته على8765. الأصل هو البروتوكول والمضيف والمنفذ؛ اختلاف8765 عن8766 يجعل الأصلين مختلفين.

1. افتح Network واضغط CORS: blocked read. الطلب إلى `/cors/no` قد يظهر200، لكن الصفحة تعرض Blocked لأن كودها لا يملك إذن قراءة الرد.
2. اضغط `/cors/yes` . CORS: allowed read يعيد نفس البيانات مع `Access-Control-Allow-Origin:` `http://127.0.0.1:8765`؛ فتستطيع الصفحة قراءتها.
3. قارن رؤوس الردين، وليس رمز200 وحده. افتح الرد مباشرة في تبويب: ده تنقل، مختلف عن محاولة كود صفحة قراءة بيانات أصل آخر.

حاول تفسر: السماح هنا لا يثبت تسجيل الدخول ولا ملكية المنتج؛ هو إذن قراءة للمتصفح. المثال GET بسيط، أما الطلب الذي يحتاج Preflight فهنفهم طلبه المسبق تحت. المصدر: دليل CORS.

قبل التفاصيل

Same-Origin Policy (سياسة الأصل الواحد: قيود متصفح على قراءة بيانات مصدر مختلف) تحمي المستخدم من أن تقرأ صفحة خبيثة بيانات موقع آخر بصلاحيات متصفحه. CORS (Cross-Origin Resource Sharing؛ قواعد تسمح للمتصفح بقراءة رد مصدر مختلف عندما يوافق الخادم) استثناء يعلنه الخادم، وليس وسيلة لمنع الطلبات من كل الأدوات.

ما Origin؟

الـorigin (أصل الصفحة: طريقة الوصول واسم المضيف والمنفذ معًا) يتكوّن من **scheme (طريقة الوصول في بداية العنوان، مثل https) + host + port**:

```
https://app.example:443
```

ويختلف عن `http://app.example` أو `https://api.example` أو `https://app.example:8443` .

Same-Origin Policy تمنع JavaScript (لغة برمجة تستخدم مثلًا لتنفيذ تفاعل الصفحة داخل المتصفح) من قراءة كثير من موارد origin أخرى بلا سماح. لا تمنع إرسال كل الطلبات، ولا تحمي server-to-server (تواصل مباشر بين برامج خادم، خارج سياسة قراءة صفحات المتصفح) clients.

CORS

```
Access-Control-Allow-Origin: https://app.example
Access-Control-Allow-Credentials: true
Vary: Origin
```

لا يمكن جمع credentials (بيانات اعتماد مثل رمز دخول أو ملف تعريف ارتباط مناسب) مع wildcard (قيمة عامة مثل النجمة * التي تطابق أي أصل في هذا الحقل) `*`. طابق origin مع allow-list (قائمة سماح تحدد القيم المقبولة صراحة) صريحة، ولا تعكس أي قيمة مرسلة بلا تحقق.

CORS، Cross-Origin Resource Sharing سماح محدد يعلنه الخادم لقراءة ردوده من صفحات ذات أصل آخر. **Credentials — بيانات إثبات هوية** تشمل مثلًا cookies التي يرسلها المتصفح حسب إعداد الطلب والسياسات. **Allow-list — قائمة السماح** أسماء أصول تقبلها صراحة، مش أي قيمة جاء بها الطلب.

Simple request — طلب مستوفٍ لشروط الإرسال دون فحص مسبق: ليست كل GET بسيطة وليست كل POST تحتاج فحصًا. توجد قيود على الطريقة والرؤوس ونوع المحتوى؛ POST بصيغة application/json (JavaScript Object Notation؛ صيغة نصية لترتيب البيانات في أسماء وقيم وقوائم) عادة يحتاج فحصًا، بينما إرسال نموذج بنوع مسموح قد لا يحتاجه. حتى عند إرسال الطلب دون فحص، قراءة الرد تظل خاضعة لـCORS.

Preflight

```
OPTIONS /api/orders HTTP/1.1
Origin: https://app.example
Access-Control-Request-Method: POST
Access-Control-Request-Headers: Content-Type, Authorization
```

يرد الخادم بالـmethods (طريقة الطلب: نوع العملية المطلوبة مثل القراءة) والـheaders (حقل أو مقدمة معلومات تضاف للبيانات بحسب الطبقة) المسموحة ومدة حفظ القرار. تعامل مع `OPTIONS` قبل auth middleware (خطوة مشتركة في معالجة الطلب لفحص الهوية أو الصلاحيات) التي تتطلب credential غير موجودة في preflight.

Preflight — فحص مسبق طلب OPTIONS يسأل عن السماح قبل الطلب الفعلي. نتيجة 204 تعني نجاحًا بلا جسم رد.
Authentication التحقق من الهوية، و**Authorization** تحديد العمليات المسموحة لها. **CSRF، Cross-Site Request Forgery** خداع المتصفح لإرسال عملية غير مقصودة ببيانات دخوله؛ السماح بقراءة الرد مش هو منع هذه العملية.

مثال PHP مبسط

المقطع التالي جزء من معالج طلب PHP (اسم لغة برمجة تستخدم كثيرًا لمعالجة طلبات الويب على الخادم)، وليس تطبيق دخول كاملًا. يُوضع بعد بداية ملف PHP وقبل أي إخراج. أمثلة اللغة تحتاج تجهيز PHP. هو يوضح رؤوس CORS؛ التحقق من هوية الطلب الفعلي وصلاحيته مسؤولية التطبيق:

```php
$allowed = ['https://app.example', 'https://admin.example'];
$origin = $_SERVER['HTTP_ORIGIN'] ?? '';

if (in_array($origin, $allowed, true)) {
    header("Access-Control-Allow-Origin: {$origin}");
    header('Vary: Origin');
    header('Access-Control-Allow-Credentials: true');
}

if (($_SERVER['REQUEST_METHOD'] ?? '') === 'OPTIONS') {
    header('Access-Control-Allow-Methods: GET, POST');
    header('Access-Control-Allow-Headers: Content-Type, Authorization');
    http_response_code(204);
    exit;
}
```

تحذير

CORS ليست Authentication أو Authorization وليست بديلًا عن CSRF protection. عميل غير متصفح يستطيع تجاهلها، والخادم يجب أن يتحقق من كل طلب.

مسائل عملية

▼ هل CORS يمنع `curl` من إرسال الطلب؟

لا. CORS سياسة يطبقها المتصفح على JavaScript؛ يجب أن يعتمد الخادم على authentication وauthorization للحماية.

▼ لماذا لا يجتمع wildcard مع credentials؟

لأن الاستجابة ذات cookies أو credentials يجب أن تسمح origin محددًا، لا أي موقع، مع `Access-Control-Allow-Credentials` .

▼ متى يرسل المتصفح preflight؟

قبل طلب cross-origin غير بسيط، ليسأل عبر OPTIONS إن كان method والـheaders المطلوبة مسموحة.

شخّص الرفض من الرسالتين

الفحص المسبق له مخزن قرارات مستقل نسبيًا عن تخزين جسم HTTP (Hypertext Transfer Protocol؛ قواعد طلب الموارد والرد عليها في الويب). افتح Network في أدوات المطور: هل OPTIONS فشل؟ ولا نجح ووصل الطلب الفعلي لكن رده لا يحتوي السماح الصحيح؟ أداة curl لا تطبق سياسة قراءة JavaScript في المتصفح، لذلك نجاحها وحده لا يثبت صحة CORS.

no-cors في طلب المتصفح مش طريقة لتجاوز المنع وقراءة البيانات. قد ينتج **Opaque response — رد لا يكشف كوده أو جسمه لبرنامج الصفحة**.

تدريب وحل: OPTIONS نجح لكن الرد الفعلي نسي Access-Control-Allow-Origin. الحل إصلاح رؤوس الرد الفعلي أيضًا، بما فيها الردود المناسبة للأخطاء، وفق قائمة السماح. تعطيل سياسة المتصفح أو إضافة no-cors لا يحول الرد لبيانات مقروءة.

سياسات تحميل الموارد وعزل النوافذ

سياسات تحميل الموارد وعزل النوافذ

اقرأ تجربة CORS أولًا: هي تسمح لكود الصفحة بقراءة رد من أصل مختلف. السياسات هنا تجيب أسئلة أخرى:

السياسة	الاسم الكامل	سؤالها
CORP	Cross-Origin Resource Policy	هل يسمح المورد باستخدامه في طلبات معينة من أصل آخر، ومنها تحميل no-cors؟
COEP	Cross-Origin Embedder Policy	هل تشترط الصفحة أذونات مناسبة للموارد التي تضمها؟
COOP	Cross-Origin Opener Policy	هل تُفصل علاقة النافذة بالنوافذ التي فتحتها عند اختلاف السياسة؟

no-cors وضع طلب يجعل الرد غير قابل للقراءة العادية بواسطة JavaScript، وليس تعطيلًا للحماية. إعداد هذه الرؤوس معًا قد يتيح عزلًا تحتاجه قدرات متصفح معينة؛ لا تضفها عشوائيًا لأنها قد تمنع تحميلات أو تغيّر تواصل النوافذ.

تطبيق فكري: ظهور صورة من موقع آخر لا يثبت أن كودك يستطيع قراءة بيانات ردها. فسّر الفرق بين التضمين وقراءة الرد قبل تغيير أي سياسة.

سياسات أخرى لها وظائف مختلفة: **CORP، Cross-Origin Resource Policy** يحدد السماح بتحميل المورد في سياقات معينة؛ **COEP، Cross-Origin Embedder Policy** يفرض شروطًا على الموارد التي تضمها الصفحة؛ **COOP، Cross-Origin Opener Policy** يفصل سياق النافذة عن نوافذ أخرى وفق السياسة. دي أدوات عزل إضافية، مش بدائل متبادلة.

اسأل دوريًا أو استقبل بثًّا من الخادم

اسأل دوريًا أو استقبل بثًّا من الخادم

شغّل السؤال الدوري ثم البث

في المعمل المحلي افتح Network:

1. اضغط **Polling** — **سؤال دوري**: نرسل طلبًا، نقرأ الوقت، ننتظر قليلًا، ثم نكرر. تظهر ثلاثة أوقات وثلاثة طلبات `/proxy-clock` . Poll three times.
2. امسح قائمة Network للعرض فقط، واضغط **SSE — Server-Sent Events** قناة يرسل فيها الخادم أحداثًا نصية للمتصفح. يظهر طلب `/proxy-events` واحد يحمل الأحداث1 و2 و3، ثم يغلق المثال الاتصال. Receive three events.
3. `EventSource` أداة المتصفح لفتح هذه القناة؛ `onmessage` تتلقى رسالة، و `close()` تغلقها. من غير الإغلاق قد تعاود الأداة الاتصال بعد انقطاعه. الكود الكامل للعميل والخادم داخل ملف المعمل؛ لا تحتاج تخمين جزء مفقود.

قارن: الأول يسأل مرارًا حتى لو لم يحدث جديد؛ الثاني يحتفظ باتصال ليستقبل. لم نجرب WebSocket هنا؛ قناته تسمح للطرفين بالإرسال. شرح SSE.

قبل التفاصيل

اختر قناة التحديث حسب اتجاه البيانات وتكرارها وتحمل التأخير. Polling وSSE (Server-Sent Events؛ إرسال أحداث نصية من الخادم للمتصفح عبر HTTP) وWebSocket وwebhooks ليست درجات أفضلية؛ كل منها يحل شكلًا مختلفًا من التواصل.

Polling — سؤال دوري: المتصفح يسأل كل عدة ثوانٍ هل فيه جديد. **Long polling — سؤال ينتظر**: الخادم يحتفظ بالطلب حتى يأتي حدث أو تنتهي المهلة، ثم يعيد العميل السؤال. **SSE، Server-Sent Events** تدفق أحداث نصية من الخادم للمتصفح عبر HTTP (Hypertext Transfer Protocol؛ قواعد طلب الموارد والرد عليها في الويب). **WebSocket** قناة مستمرة تسمح للطرفين بإرسال رسائل. **Webhook** طلب يرسله خادم لخادم آخر عند حدث، مثل تأكيد دفع.

ليست كل التحديثات Request/Response تقليدية

النمط	الاتجاه	مناسب لـ
Polling	العميل يسأل دوريًا	تحديثات قليلة وحل بسيط
Long Polling	طلب ينتظر حدثًا	توافق واسع مع latency (زمن انتظار عملية واحدة) أقل
SSE	Server → Browser	إشعارات وfeeds نصية
WebSocket	اتجاهان مستمران	دردشة وألعاب وتعاون لحظي
Webhook	Server → Server	إشعار نظام خارجي بحدث

اختيار WebSocket لمجرد أنه "لحظي" يضيف إدارة connections وheartbeat وbackpressure (إبطاء المنتج عندما لا يستطيع المستهلك متابعة معدل البيانات) وتوسّع أفقي. ابدأ بأبسط نمط يحقق المطلوب.

Heartbeat رسالة دورية لفحص بقاء الاتصال، و**Backpressure** إبطاء المنتج عندما لا يقدر المستهلك يلحق بالرسائل بدل تراكمها بلا حد. **التوسع الأفقي** إضافة نسخ خدمة لتوزيع الحمل.

Server-Sent Events

المقطع التالي يوضح صيغة حدث واحد داخل ملف PHP (اسم لغة برمجة تستخدم كثيرًا لمعالجة طلبات الويب على الخادم) بعد سطر البداية، وليس خادم بث كاملًا. حقل event اسم الحدث وdata بياناته؛ السطر الفارغ ينهي الحدث. X-Accel-Buffering يطلب من Nginx عدم تجميع الخرج في هذا المسار، وflush يحاول دفع الخرج؛ النتيجة تعتمد أيضًا على مخازن الإخراج وإعدادات الوسطاء:

```
header('Content-Type: text/event-stream');
header('Cache-Control: no-cache');
header('X-Accel-Buffering: no');

echo "event: order.updated\n";
echo 'data: ' . json_encode(['id' => 42, 'status' => 'paid']) . "\n\n";
flush();
```

SSE يعيد الاتصال تلقائيًا في المتصفح ويمكن استخدام `id` و`Last-Event-ID` للاستئناف. لا تشغل FPM (FastCGI Process Manager؛ مدير عمليات معالجة طلبات PHP) workers (وحدة تنفيذ تتولى مهمة أو طلبًا؛ قد تكون عملية أو خيطًا حسب النظام) بلا حدود؛ راجع بنية الخادم والمهلات قبل streams (تدفق بيانات متتابعة قد لا نعرف حجمه مقدمًا) طويلة.

EventSource واجهة المتصفح المعتادة لاستقبال SSE وإعادة الاتصال. حقل id يعطي الحدث معرّفًا؛ المتصفح يستطيع إرسال Last-Event-ID في إعادة الاتصال، لكن الخادم لازم يحتفظ بما يحتاجه لإعادة الأحداث. PHP-FPM (PHP FastCGI Process Manager؛ برنامج يدير عمليات PHP التي تستقبل عملًا من خادم الويب) يدير عمليات PHP؛ الاتصال الطويل قد يشغل عاملًا لمدة طويلة.

التشغيل

- ضع حدودًا لعدد الاتصالات والرسائل والحجم.
- استخدم ping/heartbeat واكتشف الاتصالات الميتة.
- طبّق authentication عند الاتصال وauthorization لكل قناة/حدث.
- راقب reconnect rate وqueue lag وdelivery failures.

مسائل عملية

▼ متى يكون SSE أبسط من WebSocket؟

عندما يحتاج المتصفح تحديثات مستمرة من الخادم فقط؛ SSE أحادي الاتجاه ويعمل فوق HTTP مع إعادة اتصال مدمجة.

▼ كيف يتحقق مستقبِل webhook من المرسل؟

يتحقق من توقيع مبني على body الخام وsecret، مع timestamp وحد زمني، ثم يمنع التكرار بمعرّف الحدث.

▼ لماذا يجب أن يكون webhook idempotent؟

لأن المرسل قد يعيد المحاولة عند timeout حتى لو عولج الحدث؛ يجب ألا تكرر الإعادة الأثر التجاري.

الخطوة التالية

كمّل في استقبل إشعار خادم وتعامل مع تكراره بعد تنفيذ التجربة هنا.

استقبل إشعار خادم وتعامل مع تكراره

استقبل إشعار خادم وتعامل مع تكراره

Webhook — إشعار بين خادمين طلب يرسله نظام عند حدث اتفقت عليه، مثل اكتمال عملية. العميل هنا برنامج خادم، مش تبويب المتصفح. هنبدأ بإشعار محلي واحد ثم نرسله مرتين لنفهم خطر تكرار الأثر.

أرسل حدثًا مرتين

شغّل المعمل. في PowerShell اكتب:

```
Set-Content -LiteralPath event.json -Value '{"id":"order-1"}' -Encoding ascii
curl.exe -i -H "Content-Type: application/json" --data-binary "@event.json" http://127.0.0.1:8766/webhook
```

الأمر الأول يكتب ملف نص فيه معرّف حدث للتدريب. `--data-binary` ترسل محتواه كما هو في طلب POST. المتوقع `duplicate:false` و `processedCount:1` . كرر أمر curl: يصبح `duplicate:true` والعدد يظل1؛ الخادم تذكر المعرّف فلم يكرر الأثر. غيّر المعرف إلى order-2 في الملف وأرسل: العدد2.

هذا مستقبل محلي يفقد ذاكرته بإعادة التشغيل، ولا يتحقق من توقيع مرسل خارجي. في خدمة حقيقية نحتاج تحقق المصدر وحفظ منع التكرار بشكل دائم؛ التفاصيل التالية تشرح ليه، من غير خلطها بأول رسالة نجربها.

Webhooks

Raw body — الجسم الخام هو البايتات كما وصلت قبل تغيير المسافات أو تحويل JSON (JavaScript Object Notation؛ صيغة نصية لترتيب البيانات في أسماء وقيم وقوائم) إلى قيم. **HMAC، Hash-based Message Authentication Code** قيمة تحقق محسوبة من الرسالة ومفتاح سري مشترك. **Timestamp** وقت إنشاء الرسالة. **Replay** إعادة إرسال رسالة صحيحة قديمة لتنفيذ أثرها مرة أخرى. تختلف تفاصيل توقيع كل مزود، فاتبع عقده بدل افتراض صيغة واحدة.

المقطع البرمجي أسفل القائمة جزء توضيحي فقط: يفترض أن timestamp وrawBody وsecret وsignature قُرئت وفُحصت وفق عقد المزود. SHA-256 دالة تلخيص مستخدمة داخل HMAC، وhash_equals تقارن قيم التحقق بطريقة تتجنب تسريب الفارق من زمن المقارنة. المقطع وحده لا يفحص عمر الرسالة ولا يمنع تكرار الحدث:

المستقبل يجب أن:

1. يقرأ raw body.
2. يتحقق من توقيع HMAC وtimestamp قبل parsing الموثوق.
3. يرفض replay خارج نافذة زمنية.
4. يسجل event ID ويمنع المعالجة المكررة.
5. يعيد نجاحًا سريعًا ثم ينقل العمل الثقيل إلى queue (طابور يعالج العناصر عادة بترتيب وصولها).

```
$expected = hash_hmac('sha256', $timestamp . '.' . $rawBody, $secret);
if (!hash_equals($expected, $signature)) {
    http_response_code(401);
    exit;
}
```

المُرسل يعيد المحاولة عند الفشل؛ لذلك idempotency ليست تحسينًا اختياريًا.

الوصول مرة لا يعني المعالجة مرة

إعادة الاتصال لا تسترجع الرسائل تلقائيًا. **Sequence** رقم ترتيب، و**Cursor** علامة آخر موضع قرأه العميل، و**Snapshot** صورة للحالة الحالية؛ تختار منهم ما يسمح باستكمال العمل.

Idempotent — أثر لا يتكرر عند إعادة نفس العملية مهم للـwebhook. بعد فحص التوقيع والوقت، سجّل معرّف الحدث مع تغيير بيانات العمل في عملية قاعدة بيانات واحدة إما تنجح كلها أو تفشل كلها، أو استخدم آلية موثوقة تحقق نفس الضمان. لا تكتفِ بفحص «هل المعرّف موجود؟» ثم كتابة منفصلة؛ قد يصل طلبان معًا.

تدريب محلول: حدث دفع بمعرّف pay-42 وصل مرتين. اجعل المعرف فريدًا في سجل المعالجة؛ أول معالجة تسجل الدفع، والثانية تكتشف أنه عولج وتعيد ردًا مناسبًا بلا إضافة رصيد ثانية. لو شغلك يُرسل لطابور، أقرّ بالاستلام بعد حفظه بشكل موثوق، لا قبل حفظه ثم فقده عند توقف الخادم. الوصول المتأخر أو المختلف الترتيب يحتاج أيضًا مقارنة حالة الطلب وقواعد انتقالها.

تصميم واجهات تبادل البيانات

REST وRPC وGraphQL وتصميم الموارد والحالات والأخطاء والصفحات والإصدارات والعقود.

عقد منتج واحد قبل أنواع الواجهات

API — واجهة برمجة التطبيقات اتفاق يحدد كيف يطلب برنامج بيانات أو عملية. **العقد** وصف الطلبات المسموحة والردود. في المعمل اختبر العقد الآتي على8766:

الطلب	الحالة	المحتوى
GET `/api/products/1`	200	id عدد، name نص، price عدد20، currency النصEGP
GET `/api/products/2`	404	error.code يساوي NOT_FOUND
GET `/api/products/abc`	400	error.code يساوي INVALID_ID
POST `/api/products/1`	405	Allow يذكر GET وHEAD

`GET` يطلب قراءة، و`POST` يطلب معالجة لا يقبلها هذا المسار. الرد JSON: حقول لها أسماء وقيم، مثل كائن المنتج في مشروع المشتريات. افتح أول رابط بالمتصفح، ثم جرّب `curl.exe -i -X POST http://127.0.0.1:8766/api/products/1` . `-X` يحدد نوع الطلب.

اكتب المتوقع قبل التشغيل. النجاح مش مجرد200: افحص نوع كل حقل ومعناه. هذا عقد قراءة محدود؛ إضافة منتج تحتاج عقدًا آخر يحدد المدخلات والتحقق والصلاحيات قبل المقارنة بين REST وRPC وGraphQL.

قبل التفاصيل

API (Application Programming Interface؛ اتفاق يسمح لبرنامج بطلب بيانات أو عملية من مكوّن آخر) جيدة هي عقد يمكن للعميل توقعه، لا مجرد endpoints (عنوان يقدم عملية أو بيانات من التطبيق) تعمل اليوم. صمّم الموارد والحالات والأخطاء والصفحات والتوافق قبل أن تتكاثر الاعتمادات.

API، Application Programming Interface — واجهة برمجة تطبيقات اتفاق على طلبات ونتائج يفهمها برنامج آخر. **Endpoint — نقطة وصول** عنوان وعملية محددة، مثل GET لطلب منتج. **العقد — Contract** يحدد الحقول وأنواعها والنتائج والأخطاء، بحيث يعرف العميل ماذا يرسل وكيف يتعامل مع الرد.

CRUD، Create Read Update Delete إنشاء وقراءة وتعديل وحذف بيانات. **REST، Representational State Transfer** أسلوب تنظيم حول موارد وتمثيلاتها وقيود منها فصل العميل والخادم واستقلال الطلبات؛ مجرد وضع JSON (JavaScript Object Notation؛ صيغة نصية لترتيب البيانات في أسماء وقيم وقوائم) وراء عنوان مش كفاية ليكون التصميم REST كاملًا. **RPC، Remote Procedure Call** طلب تنفيذ عملية مسماة عن بعد. **GraphQL** لغة استعلام ونظام تنفيذ يسمح للعميل باختيار حقول ضمن وصف أنواع اسمه schema؛ **Resolver** كود يجلب قيمة حقل، وحدود complexity تمنع استعلامًا بالغ الكلفة.

اختر الأسلوب حسب المشكلة

- **REST:** موارد وعقود HTTP (Hypertext Transfer Protocol؛ قواعد طلب الموارد والرد عليها في الويب)؛ مناسب لمعظم CRUD والـpublic APIs.
- **RPC:** عمليات صريحة مثل `calculateShipping` ؛ مناسب عندما يكون الفعل أهم من المورد.
- **GraphQL:** العميل يحدد الحقول عبر schema؛ قوي للواجهات المتنوعة لكنه يحتاج حدود complexity وauthorization لكل resolver.

الاسم لا يضمن الجودة. العقد الواضح، التوافق، الأمان والمراقبة أهم من الشعار.

موارد وHTTP

```
GET    /api/orders/42
POST   /api/orders
PATCH  /api/orders/42
DELETE /api/orders/42
```

- استخدم nouns متسقة.
- أعد `201` مع `Location` عند الإنشاء، و`204` عندما لا يوجد body (جسم الرسالة: المحتوى المرسل مثل نص صفحة أو بيانات طلب).
- ميّز conflict `409`، absence `404`، permission `403`، authentication `401`، parsing `400`، وvalidation `422` وفق عقدك.
- لا تجعل كل نتيجة `200`.

Validation وProblem Details

```
{
  "type": "https://docs.example/errors/validation",
  "title": "Validation failed",
  "status": 422,
  "errors": {"email": ["Invalid format"]},
  "request_id": "req_01J..."
}
```

استخدم شكل خطأ ثابتًا، ولا تكشف stack trace (سلسلة استدعاءات توضح أين مر التنفيذ عند الخطأ) أو SQL. معيار Problem Details يعرّف حقولًا قابلة للامتداد.

Problem Details صيغة موحدة لوصف خطأ HTTP. في المثال type رابط لتعريف نوع الخطأ، وtitle وصف قصير، وstatus يكرر رمز الرد. errors وrequest_id حقول إضافية في عقدنا: الأول أخطاء المدخلات، والثاني معرّف تتبع الطلب. **Stack trace** قائمة مواضع التنفيذ وقت العطل، و**SQL** لغة أوامر قاعدة البيانات؛ لا ترسلهما للعميل كبديل لرسالة مفهومة.

Filtering وPagination

Pagination — تقسيم النتائج لصفحات يمنع إرجاع آلاف السجلات مرة واحدة، و**Filtering — التصفية** يختار المطابق لشرط. **Cursor** علامة موضع لاستكمال القراءة وفق ترتيب ثابت، مثل تاريخ الطلب ومعرّفه، و**Offset** عدد العناصر المتخطاة. يحتاج المؤشر فحصًا وحدود وصول؛ إخفاؤه بترميز Base64 لا يجعله موثوقًا.

```
GET /api/orders?status=paid&limit=20&cursor=eyJpZCI6OTAwfQ
```

ضع حدًا أقصى للصفحة. Cursor pagination أفضل غالبًا للبيانات الكبيرة المتغيرة، بينما offset أبسط للتنقل المحدود.

التوافق والإصدارات

- فضّل التغيير الإضافي backward-compatible.
- لا تغيّر معنى حقل قائم بصمت.
- أعلن deprecation وموعد الإزالة.
- اختبر consumers بعقود آلية.
- يمكن وضع الإصدار في path (مسار المورد داخل العنوان، مثل /products) أو header (حقل أو مقدمة معلومات تضاف للبيانات بحسب الطبقة)؛ الاتساق أهم من الاختيار.

العقد والتشغيل

وثّق OpenAPI أو schema قابلة للاختبار، وأضف idempotency، rate limits، authentication للعمليات الحساسة، request IDs (معرّف يميز طلبًا لتتبع أحداثه)، timeouts وobservability. صمّم الـAPI للفشل الجزئي وإعادة المحاولة لا للمسار السعيد فقط.

مراجع

- RFC 9110: HTTP Semantics
- RFC 9457: Problem Details for HTTP APIs

مسائل عملية

▼ متى نعيد `404` ومتى `403`؟

404 لعدم وجود المورد، و403 لوجوده مع منع الوصول. أحيانًا تختار السياسة 404 لإخفاء وجود مورد حساس.

▼ لماذا cursor pagination أنسب أحيانًا من offset؟

تتعامل أفضل مع بيانات تتغير أثناء التصفح وتتجنب كلفة القفز لمسافات كبيرة، لكنها تحتاج ترتيبًا ثابتًا وcursor غير قابل للتلاعب.

▼ هل كل تغيير يحتاج إصدار API جديدًا؟

لا. الإضافات المتوافقة غالبًا لا تحتاجه، أما حذف حقل أو تغيير معناه فيحتاج خطة إصدار وترحيل واضحة.

اختبر العقد عند التكرار والتغيير

Backward compatible — متوافق مع العملاء السابقين يعني أن العميل القديم يظل يعمل حسب عقده. **Deprecation — إعلان الاستغناء التدريجي** يعطي مهلة انتقال قبل إزالة ميزة. **OpenAPI** صيغة لوصف نقاط الوصول والمدخلات والردود بحيث تقرأها الأدوات وتتحقق منها.

Idempotency-Key معرّف يرسله العميل لعملية قابلة لإعادة المحاولة. الخادم يربطه بالمستخدم والعملية والمدخلات والنتيجة، ويمنع المعالجة المتزامنة المكررة أيضًا. مدة الاحتفاظ بالمفتاح جزء من العقد، مش ضمانًا أبديًا.

Rate limit حد لعدد الطلبات في زمن، و**Quota** حصة استهلاك خلال فترة. عند الحد المناسب يرجع الخادم 429، ويمكن أن يرسل **Retry-After**، وقت الانتظار قبل إعادة المحاولة. **Observability — قابلية فهم التشغيل** استخدام السجلات والقياسات لتحديد ما حدث ولماذا.

تدريب محلول: نفس المفتاح مع جسم مختلف: ارفض التعارض برسالة ورمز محددين في العقد، مثل 409، ولا تنفذ عملية جديدة بصمت. اختبر أيضًا أن إضافة حقل اختياري لا تكسر العميل القديم، وأن العميل لا يعتمد على ترتيب حقول JSON.

كيف يختلف HTTP/2 عن HTTP/3؟

تطور نقل رسائل HTTP من الاتصالات المتعددة إلى Multiplexing وQUIC فوق UDP.

نفس ثلاثة طلبات عند فقد بيانات

Stream — تدفق منطقي قناة لرسائل طلب داخل اتصال مشترك. **Frame — إطار بروتوكول** قطعة تحمل نوعًا ورقم تدفق وبيانات بحسب البروتوكول؛ مش هو إطار Ethernet بالضرورة.

تخيل A وB وC موارد مطلوبة، وفقدت بيانات تخص A. الأسهم ترتيب مبسط، مش قياس سرعة:

النسخة والافتراض	قبل الفقد	أثر الفقد
HTTP/1.1، اتصال واحد وطلبات متتابعة	A ثم B ثم C	انتظار A يؤخر بدء الطلبين في هذا النموذج؛ المتصفحات قد تفتح اتصالات إضافية
HTTP/2، تدفقات على TCP واحد	أجزاء A وB وC تتداخل	فقد مقطع TCP يؤخر تسليم البايتات اللاحقة لكل التدفقات على الاتصال حتى الاستعادة
HTTP/3، تدفقات QUIC	أجزاء A وB وC تتداخل	استعادة بيانات A لا تفرض ترتيب بايتاتها على B وC؛ لكن الازدحام والموارد المشتركة ما زالت تؤثر

راجع فهمك: هل HTTP/3 يلغي كل تأخير؟ لا. ميّز إزالة انتظار ترتيب النقل بين التدفقات عن سعة الشبكة وزمن عمل التطبيق.

ليه ظهر أكتر من إصدار لـHTTP؟

معنى `GET` و`404` وHeaders (حقل أو مقدمة معلومات تضاف للبيانات بحسب الطبقة) ما اختفاش. اللي اتغير هو طريقة نقل الرسائل بكفاءة. HTTP/1.1 (Hypertext Transfer Protocol؛ قواعد طلب الموارد والرد عليها في الويب) كان غالبًا يستخدم أكتر من اتصال لتقليل الانتظار. HTTP/2 سمح لطلبات كثيرة تشارك اتصال TCP (Transmission Control Protocol؛ نقل بايتات بترتيب مع معالجة الفقد، مع احتمال فشل الاتصال) واحد باستخدام Streams (تدفق بيانات متتابعة قد لا نعرف حجمه مقدمًا) وFrames:

```
One TCP connection
  ├─ Stream 1: HTML
  ├─ Stream 3: CSS
  └─ Stream 5: image
```

لكن TCP يقدم Byte Stream (تتابع بايتات مرتب لا يحفظ تلقائيًا حدود رسائل التطبيق) مرتبة واحدة. لو Packet (حزمة: وحدة بيانات IP التي تُوجّه بين الشبكات) ضاعت، ممكن Streams كلها تستنى لحد استعادة البايتات الناقصة. ده Transport Head-of-line Blocking (انتظار على مستوى النقل بسبب نقص بيانات سابقة).

QUIC (اسم بروتوكول نقل فوق UDP يضيف تدفقات موثوقة وحماية اتصال) مبني فوق UDP (User Datagram Protocol؛ نقل رسائل مستقلة بلا ضمان مدمج لوصولها أو ترتيبها) لكنه يضيف الاعتمادية والتشفير وإعادة الإرسال والتحكم في الازدحام، ويعطي Streams استقلالًا أكبر. HTTP/3 يستخدم QUIC؛ «فوق UDP» لا تعني إن بيانات التطبيق غير موثوقة.

المتصفح والخادم يتفاوضوا ويقدروا يرجعوا لإصدار أقدم. الإصدار الأحدث وحده مش ضمان سرعة؛ المسافة والفقد وإعداد السيرفر والـCaching كلها عوامل.

HTTP Semantics ونسخة النقل

تبقى معاني Methods (طريقة الطلب: نوع العملية المطلوبة مثل القراءة) وStatus Codes (رقم يصف نتيجة معالجة طلب HTTP) وHeaders متشابهة بين الإصدارات، لكن طريقة تمثيل الرسائل ونقلها تتغير. HTTP/1.1 نصي في صورته المعروفة، بينما يستخدم HTTP/2 Frames ثنائية، ويعمل HTTP/3 فوق QUIC.

HTTP/2 وMultiplexing

يقسم HTTP/2 الاتصال إلى Streams مستقلة تحمل Frames، فيستطيع المتصفح إرسال طلبات متعددة على اتصال TCP واحد بدل انتظار كل استجابة. ويضغط Headers باستخدام HPACK.

يحل ذلك مشكلة انتظار الطلبات على مستوى HTTP/1.1، لكن فقد Packet واحدة في TCP قد يؤخر كل Streams حتى يعيد TCP ترتيب البايتات. تسمى هذه المشكلة Transport Head-of-line Blocking.

QUIC وHTTP/3

يبني QUIC نقلًا موثوقًا ومشفرًا فوق UDP، ويدمج TLS (Transport Layer Security؛ قواعد حماية الاتصال بالتشفير والتحقق) 1.3 في Handshake (مصافحة: رسائل البداية للاتفاق على الاتصال). لكل Stream ترتيب مستقل، لذلك لا يوقف فقد البيانات في Stream كل Streams الأخرى. كما يستخدم Connection ID ليساعد الاتصال على الاستمرار عند تغير شبكة الهاتف من Wi-Fi (اتصال لاسلكي بالشبكة المحلية؛ لا يضمن اتصالها بالإنترنت) إلى بيانات الهاتف.

لا يعني استخدام UDP أن HTTP/3 غير موثوق؛ QUIC نفسه ينفذ التأكيد وإعادة الإرسال والتحكم في الازدحام. يستخدم UDP كواجهة للنظام كي يطبق هذه الآليات في User Space (مساحة تشغيل برامج المستخدم خارج نواة النظام) بصورة أسرع قابلية للتطوير.

اختيار الإصدار

يتفاوض العميل والخادم على البروتوكول، وقد يعود المتصفح إلى HTTP/2 أو HTTP/1.1 إذا تعذر QUIC. تقيس الأدوات النتيجة الفعلية بدل افتراض أن الإصدار الأحدث أسرع دائمًا؛ المسافة وفقد الحزم وإعداد الخادم وحجم الموارد كلها مؤثرة.

تدريب عملي

▼ إيه اللي اتغير في HTTP/2: المعنى ولا النقل؟

Semantics (دلالات: معنى العملية والنتيجة بغض النظر عن شكل نقلها) الأساسية بقيت، بينما النقل استخدم Binary Frames وMultiplexing (دمج تدفقات: إرسال أجزاء من عدة رسائل على اتصال واحد) وضغط Headers.

▼ ليه HTTP/3 موثوق فوق UDP؟

لأن QUIC ينفذ ACK (Acknowledgment؛ إقرار باستلام بيانات وفق قواعد البروتوكول) وإعادة الإرسال والترتيب والتحكم في الازدحام والتشفير.

▼ إزاي تعرف الإصدار المستخدم فعلًا؟

شوف عمود Protocol في Network DevTools أوأداة قياس، وما تفترضش من إعداد السيرفر وحده.

تأكد من فهمك

01 **ما الذي يقدمه Multiplexing؟**

> **الإجابة:** يسمح لعدة طلبات واستجابات بمشاركة اتصال واحد عبر Streams مستقلة منطقيًا.

02 **هل HTTP/3 غير موثوق لأنه يستخدم UDP؟**

> **الإجابة:** لا؛ QUIC يضيف الاعتمادية وإعادة الإرسال والتحكم في الازدحام فوق UDP.

ضغط الرؤوس وإعادة الاتصال

HPACK طريقة ضغط حقول HTTP/2، و**QPACK** طريقة ضغط حقول HTTP/3. ضغط الحقول يقلل البيانات المتكررة. QPACK مصمم ليحد مشكلة انتظار التدفقات على معلومات جدول الضغط، لكنه لا يلغي كل انتظار؛ بعض التدفقات قد تحتاج تحديث جدول قبل قراءة رؤوسها.

RTT، Round-Trip Time زمن رحلة ذهاب وعودة. **RTT-0** إرسال بيانات مبكرة عند استئناف اتصال مناسب، لتقليل الانتظار؛ قد يعيد مهاجم إرسالها، لذلك عملية تغير حالة حساسة لا تُقبل بلا سياسة تمنع أثر التكرار. مش معنى التشفير أن كل طلب مبكر آمن من الإعادة.

Connection ID — معرّف اتصال يساعد QUIC يربط الاتصال رغم تغيّر بعض عناوينه؛ **Migration — انتقال مسار الاتصال** قد يسمح بانتقال الهاتف من Wi-Fi لشبكة الهاتف حسب الدعم والتحقق من الطريق. جدار ناري يمنع UDP قد يفرض استخدام نقل آخر.

تدريب محلول: فقدت بيانات تدفق صورة في HTTP/3. تدفق مستقل عنده بيانات كاملة قد يستمر؛ لكن فقد الحزم والتحكم المشترك في الازدحام أو اعتماد الرؤوس قد يؤثر في الأداء أيضًا. في HTTP/2 حزمة TCP المفقودة قد تعطل تسليم بايتات لاحقة لكل التدفقات على الاتصال نفسه. قارن تحت نفس ظروف الشبكة بدل الحكم من اسم الإصدار.

وزّع المحتوى وافهم الخادم الأصلي

وزّع المحتوى وافهم الخادم الأصلي

CDN وOrigin وPoint of Presence

توزع **CDN (Content Delivery Network)؛ شبكة خوادم تقدم المحتوى من نقاط موزعة قرب المستخدمين)** نقاط حضور قريبة من المستخدمين. تستقبل Edge/PoP (Point of Presence؛ موقع توجد فيه خوادم شبكة تقديم المحتوى) الطلب وتعيد نسخة مخزنة إن كان Cache (نسخة محفوظة لتقليل تكرار القراءة أو الحساب) Key صالحًا، وإلا تطلب المحتوى من **Origin server — الخادم الأصلي الذي تجلب منه شبكة توزيع المحتوى الملفات أو الردود؛ ده معنى مختلف عن أصل صفحة الويب المكوّن من البروتوكول والمضيف والمنفذ** ثم قد تحفظه. النتيجة زمن أقل وحمل أقل على الخادم الأصلي.

يتكوّن Cache Key عادة من Host وPath (مسار المورد داخل العنوان، مثل /products) وQuery محددة، وقد يتأثر بـ `Vary`. إدخال كل Cookie في المفتاح يخفض نسبة Cache Hits، بينما تجاهل Cookie أو Authorization في محتوى شخصي قد يسرّب استجابة مستخدم لآخر.

سياسات مهمة:

- `Cache-Control: public` للمحتوى القابل للمشاركة، و`private` للمحتوى الخاص بالمتصفح.
- `max-age` لعمر النسخة ما لم تخصص سياسة أخرى و`s-maxage` لعمر النسخة المشتركة.
- أسماء ملفات تحمل Hash للأصول الثابتة كي تستخدم عمرًا طويلًا.
- Purge/Invalidation عند الحاجة، مع فهم أن الانتشار قد لا يكون لحظيًا.
- حماية Origin بحيث لا يستطيع مهاجم تجاوز طبقة الحافة بسهولة.

الخطوة التالية

كمّل في حماية الطلبات وتحديد معدلها بعد تنفيذ التجربة هنا.

كمّل في تتبّع الطلب وفسّر القياسات بعد تنفيذ التجربة هنا.

حماية الطلبات وتحديد معدلها

حماية الطلبات وتحديد معدلها

خادم قادر على تنفيذ طلب سليم قد يتعرض لطلبات ضارة أو أكثر من قدرته. هنفرق بين فحص الطلب وتنظيم عدد الطلبات.

ثلاث طرق للعد بأرقام صغيرة

Rate limit — حد معدل الطلبات يقرر كم طلبًا نقبل خلال وقت. لو الحد2 كل10ثوانٍ:

الطريقة	الفكرة	أثرها
Fixed window — نافذة ثابتة	عداد يبدأ من جديد كل10ثوانٍ	قد يقبل طلبين آخر نافذة وطلبين أول التالية قريبين جدًا
Sliding window — نافذة متحركة	انظر للعشر ثواني السابقة للحظة الحالية	يتجنب نفس القفزة عند الحد، وله أكثر من طريقة تنفيذ
Token bucket — مخزن سماح	مخزن سعته2، وكل5ثوانٍ نضيف سماحًا حتى السعة	طلبان فورًا لو ممتلئ، ثم انتظار التجديد؛ كل طلب يستهلك سماحًا

في المعمل افتح `/limited` على8766 ثلاث مرات سريعًا:200،200،429. `Retry-After` يقترح زمن انتظار. هذه نافذة ثابتة مشتركة للتجربة، مش سياسة عادلة لكل مستخدم. حماية محتوى الطلب موضوع مختلف عن عدد الطلبات.

WAF ليس بديلًا لأمان التطبيق

يفحص **Web Application Firewall** خصائص الطلب ويطبق Managed Rules أو قواعد مخصصة ضد أنماط معروفة. يستطيع حظر أو Challenge أو Log، لكنه لا يعرف قواعد المجال كاملة: قد يكون طلب تعديل فاتورة صالح الصياغة لكن غير مسموح لهذا المستخدم. لذلك تظل Validation والمصادقة والصلاحيات وParameterized Queries داخل التطبيق.

اختبر القواعد في وضع مراقبة قبل الحظر إن أمكن؛ القاعدة الواسعة قد تنتج False Positives وتوقف عملاء حقيقيين. وثّق الاستثناءات بمدة ومالك وسبب بدل تعطيل الحماية دائمًا.

Rate Limiting وإدارة الاندفاع

يمكن حساب الحد حسب IP (Internet Protocol؛ قواعد عنونة الحزم وتوجيهها بين الشبكات، وعنوان IP رقم يحدد واجهة في هذا السياق) أو User ID أو API (Application Programming Interface؛ اتفاق يسمح لبرنامج بطلب بيانات أو عملية من مكوّن آخر) أو Key أو Tenant أو Route. من النماذج Fixed Window وSliding Window وToken Bucket. يسمح Token Bucket باندفاع قصير حتى سعة الدلو ثم يحد المعدل المستمر.

أعد `429 Too Many Requests`، وأرسل `Retry-After` عندما تستطيع إعطاء قيمة صحيحة. لا تعتمد على IP وحده في كل الحالات؛ قد يشترك آلاف المستخدمين خلف NAT (Network Address Translation)؛ تغيير عناوين IP أثناء مرور الحزم بين شبكات)، وقد يبدل المهاجم عناوينه.

فرّق بين Rate Limit لمنع الإساءة، وConcurrency Limit لمنع تشبع Workers (وحدة تنفيذ تتولى مهمة أو طلبًا؛ قد تكون عملية أو خيطًا حسب النظام)، وQuota لاستهلاك خطة خلال يوم أو شهر. عند الضغط الشديد استخدم Backpressure (إبطاء المنتج عندما لا يستطيع المستهلك متابعة معدل البيانات) أو Load Shedding بدل قبول عمل لن يكتمل.

تتبّع الطلب وفسّر القياسات

تتبّع الطلب وفسّر القياسات

لو الطلب بطيء، محتاج دليل عن الجزء اللي أخّر الرد. هنقرأ سجلّا ومدة ومعرّف طلب، ثم نربط قياسات أكثر من خدمة.

اقرأ قياسًا ومعرّفًا من تجربة تملكها

في المعمل نفّذ طلب المنتج ثم `/missing` ثم `/failure` . كل رد يحمل `X-Request-Id` ، وفي الطرفية سطر بنفس المعرف وحالة200 أو404 أو500. ده **Log — سجل حدث**، والمعرف يربط الرد بالسجل. عبر `/proxy` تجد نفس المعرف عند الطرفين.

Span — جزء محدد من رحلة العمل له بداية ونهاية مثل زمن استعلام قاعدة البيانات. مجموعة أجزاء مرتبطة تكوّن Trace، أي تتبع الرحلة. معرّف الطلب وحده ليس تتبعًا كاملًا؛ المعمل يثبت الربط بين طلبين، ولا يدّعي قياس قاعدة بيانات غير موجودة.

Percentile — قيمة يقع عندها أو تحتها جزء من القياسات. بافتراض20مدة مرتبة 10،20،...،200مللي ثانية، وبطريقة أقرب رتبة: p50 عند الرتبة10 فيساوي100؛ p95 عند19 فيساوي190؛ p99 عند20 فيساوي200. برامج القياس قد تستخدم استيفاءً مختلفًا، والعينة الصغيرة ضعيفة لتقدير ذيل توزيع كبير. اذكر الطريقة وحجم العينة بدل التعامل مع p95 كمتوسط.

تسليمك: ثلاث حالات مع معرفاتها وسجلاتها، وطلب عبر الوسيط، وتفسير200/404/500. بعده جرّب المشروع العام تحت كتوسع، مع قبول غياب حقول لا يعرضها الموقع العام.

Logs وMetrics وTraces

- **Log:** حدث تفصيلي مثل فشل تسجيل دخول، مع وقت ومستوى وحقول منظمة.
- **Metric:** قياس مجمع مثل معدل الطلبات وError Rate وLatency (زمن انتظار عملية واحدة) Percentiles وتشبع الاتصالات.
- **Trace:** رحلة طلب عبر Spans في CDN وGateway والتطبيق وقاعدة البيانات.

مرر Trace/Correlation ID ولا تسجل كلمات مرور أو Tokens أو بيانات شخصية بلا ضرورة. قس `p50` و `p95` و `p99` ؛ المتوسط وحده يخفي بطء نسبة صغيرة من الطلبات. اربط التنبيه بعرض مستخدم متأثر، لا بكل ارتفاع لحظي عديم الأثر.

```
Client timing: DNS → Connect → TLS → TTFB → Download
Server trace:  Edge → Gateway → App → DB → External API
```

إذا ارتفع TTFB (Time To First Byte؛ مدة الانتظار حتى وصول أول بايت من الرد بحسب أداة القياس) بينما زمن التطبيق طبيعي، افحص Queueing والحافة والشبكة. إذا كان Span قاعدة البيانات بطيئًا، انتقل إلى الاستعلام والفهرس والاتصالات بدل اتهام المتصفح.

تأكد من فهمك

01 **كيف قد يؤدي Cache Key خاطئ إلى تسريب بيانات؟**

الإجابة: إذا تجاهل المفتاح الهوية أو Header (حقل أو مقدمة معلومات تضاف للبيانات بحسب الطبقة) يغير الاستجابة، قد تعيد CDN نسخة مستخدم إلى مستخدم آخر.

02 **لماذا لا يكفي WAF (Web Application Firewall؛ نظام يفحص طلبات الويب ويطبق قواعد حماية) لحماية Endpoint (عنوان يقدم عملية أو بيانات من التطبيق) لتعديل فاتورة؟**

الإجابة: يستطيع كشف أنماط ضارة، لكنه لا يعرف وحده ملكية الفاتورة وصلاحية المستخدم وقواعد المجال.

03 **ما الفرق بين Rate Limit وConcurrency Limit وQuota؟**

الإجابة: الأول يحد معدل الوصول، والثاني عدد الأعمال المتزامنة، والثالث إجمالي الاستهلاك خلال فترة.

04 **لماذا نحتاج Log وMetric وTrace معًا؟**

الإجابة: Metric يكشف الاتجاه، وTrace يحدد المرحلة البطيئة أو الفاشلة، وLog يقدم تفاصيل الحدث اللازمة لفهم السبب.

مشروع تراكمي: تتبّع طلب واحد من الاسم إلى الـOrigin

اختر URL (Uniform Resource Locator؛ عنوان يحدد موردًا وطريقة الوصول إليه) عامًا غير حساس، وأنشئ «بطاقة رحلة» لطلب واحد. لا تكتفِ بصورة DevTools؛ اربط كل ملاحظة بطبقة وبدليل قابل لإعادة التنفيذ.

1. الاسم والعنوان

```
nslookup example.com
```

سجّل اسم الـresolver والعناوين والـTTL (Time To Live؛ عمر النسخة المخزنة في DNS، وفي IPv4 حقل منفصل يحد عدد مرات تمرير الحزمة) إن أظهرته أداتك. تغيّر العنوان بين تشغيلين قد يكون نتيجة CDN أو DNS (Domain Name System؛ نظام يجيب عن أسئلة أسماء النطاقات، ومنها عناوينها) load balancing، وليس خطأ.

2. الاتصال وHTTP

```
curl.exe -sS -o NUL -D headers.txt -w "status=%{http_code} remote=%{remote_ip} connect=%{time_connect} ttfb=%
{time_starttransfer} total=%{time_total}\n" https://example.com/
```

مثال لشكل الناتج، وليس قيمة يجب نسخها:

```
status=200 remote=203.0.113.10 connect=0.042 ttfb=0.118 total=0.121
```

على macOS أو Linux استبدل `NUL` بـ `/dev/null` . افحص `headers.txt` وابحث عن `cache-control` و `age` و `via` وأي request ID (معرّف يميز طلبًا لتتبع أحداثه)؛ غياب Header لا يثبت غياب CDN.

3. كرر وقارن

نفّذ الطلب خمس مرات واحفظ status وTTFB وAge. ثم أضف query parameter آمنًا ولاحظ هل غيّر cache key. لا تستنتج cache hit من السرعة وحدها؛ اطلب دليلًا من headers أو منصة الخدمة.

4. مسار الفشل

في المعمل اختبر مسارًا غير موجود وتأكد أن 404 تحمل معرّف الطلب. في الموقع العام سجل هل المعرف وسياسة التخزين ظاهرين؛ غيابهما ليس فشلًا منك. إذا تملك بيئة تجريبية، ولّد 500 مراقبة وتحقق أن الـtrace يصل من الحافة إلى التطبيق وأن السجل لا يحتوي Cookie أو Authorization.

التسليم

جدول من خمس محاولات، لقطة من Network waterfall، تفسير لفرق TTFB، وفرضية واحدة قابلة للدحض عن موضع التأخير. ميّز بوضوح بين «ما قسته» و«ما استنتجته».

راجع الاستنتاج قبل تعديل الخدمة

Purge أو Invalidation — إبطال نسخة مخزنة يطلب حذفها أو اعتبارها غير صالحة. التغيير قد لا يصل لكل نقطة خدمة لحظيًا. عنوان أصل ثابت يحمل رقم نسخة أو بصمة محتوى يساعد على طلب الملف الجديد صراحة؛ كلمة hash هنا بصمة من المحتوى، وليست تشفيرًا يخفيه.

False positive — إنذار خاطئ حظر طلب سليم ظنًا أنه هجوم. راقب قاعدة WAF قبل تعميمها، وحدد طريقة رجوع **Rollback** لإعداد معروف. **Trace context — معلومات ربط التتبع** لازم تنتقل بين الخدمات والمهام المؤجلة حتى نقدر نجمع رحلة الطلب. معرّف الطلب مفيد في ردود 404 و500 كمان، لكن لا تفترض أن أي موقع عام يعرضه.

تدريب محلول: الرد أسرع في المحاولة الثانية بلا دليل cache. لا تعتبرها إصابة مؤكدة؛ قد يكون السبب إعادة استخدام اتصال أو تغير الحمل. في خدمة تملكها، اربط رؤوس Age وحالة المخزن وسجلاته. لمقارنة الإبطال من نقطتين جغرافيتين، تحتاج نقطتي قياس فعليتين؛ تشغيل نفس الأمر مرتين من جهاز واحد لا يثبت انتشار التغيير عالميًا.

الرياضيات وحل المشكلات

ابدأ بالحساب والمنطق اللي تحتاجه المسألة، ثم اكتب خطوات الحل وتتبعها. بعد كده قارن طرق الحل واتوسع في الرياضيات التطبيقية.

ابدأ بالحساب والمنطق اللي تحتاجه المسألة، ثم اكتب خطوات الحل وتتبعها. بعد كده قارن طرق الحل واتوسع في الرياضيات التطبيقية.

قارن واتعمّق

تفاصيل نمو كلفة الحل وتتبع الرسوم والبرمجة الديناميكية تبني على القوائم والحلقات والدوال. جرّب تتبّعها بعد تطبيقات البرمجة؛ البداية من مسألة صغيرة تفهمها، مش حفظ أسماء طرق الحل.

امشِ بالترتيب وطبّق كل خطوة

- النسب والمتوسط والقوى
- القسمة والباقي ودقة الحساب
- المتغيرات والمعادلات والمنطق البولياني
- المجموعات والعلاقات ومدخلات الدالة
- التفكير الحاسوبي وتحليل المتطلبات
- الخوارزميات وPseudocode وأشجار القرار
- المخططات الانسيابية والحلقات والتصحيح
- مقارنة الخوارزميات وطرق تنظيم البيانات
- ليه الحلقة صحيحة وبتنتهي؟
- حوّل أنماط الحل إلى خطوات
- اختَر بين الحل الجشع وحفظ النتائج
- القياس والوحدات والأعداد الأولية
- المتتابعات والعد والاحتمال
- ارسم العلاقات وابحث عن طريق

النسب والمتوسط والقوى

النسب والمتوسط والقوى

استخدم القسم بعد القيم والقرارات والحلقات. ارجع للحساب حسب الحاجة؛ الرسوم والتعقيد والبرمجة الديناميكية توسعات لاحقة وليست شروطًا لأول برنامج.

حساب واحد بمعنى واضح

ثلاث كراسات سعر الواحدة 20: الإجمالي `3×20=60` . خصم 10% يعني `60×10÷100=6` ، فتدفع 54. **تمرين:** لو السعر 30 والكمية 2، هل يتغير الإجمالي؟ لا؛ يفضل 60. القوة `2³` تساوي `2×2×2=8` ، والجذر `√25=5` لأن `5×5=25` . لا تخلط رمز النسبة % في الكلام بمعامل الباقي في كود لغة معينة.

الفكرة العامة

العمليات الحسابية الأساسية تظهر مباشرة في البرامج: الباقي، النسب، المتوسطات، القوى، الجذور، وترتيب التنفيذ. المهم هو ربط الصيغة بمعنى المسألة قبل حسابها.

المفاهيم الأساسية

- الباقي يساعد في الزوجي والفردي، الدورات، وتقسيم الوقت.
- النسبة المئوية جزء من 100؛ قيمة الخصم = السعر × النسبة ÷ 100.
- المعدل يقارن كميتين بوحدتين مختلفتين، مثل كم/ساعة.
- المتوسط الحسابي = مجموع القيم ÷ عددها، ويتأثر بالقيم المتطرفة.
- الأقواس ثم القوى ثم الضرب والقسمة ثم الجمع والطرح؛ الضرب والقسمة في المستوى نفسه يُجمعان من اليسار، وكذلك الجمع والطرح. لكن القوى ومعاملات البرمجة لها قواعد ارتباط مختلفة؛ اكتب أقواسًا عندما يلتبس المقصود.

مثال تطبيقي

سعر منتج 800 وخصمه 15%: الخصم 120 والسعر النهائي 680. لا تطرح 15 مباشرة لأن 15 هنا نسبة وليست مبلغًا.

تصحيح مفاهيم وأخطاء شائعة

- لا تستخدم قسمة صحيحة عندما تحتاج كسورًا.
- اكتب الوحدات بجوار القيم لتكتشف الصيغ غير المنطقية.

خريطة الدرس

تأكد من فهمك

01 **لماذا يعطي 9/5 نتيجة خاطئة في بعض اللغات عند تحويل الحرارة؟**

الإجابة المشروحة: إذا كان الطرفان صحيحين تنفذ قسمة صحيحة فتصبح النتيجة 1. استخدم 9.0/5.0 أو حوّل أحد الطرفين إلى نوع عشري.

02 **منتج سعره 800 خُصم 15% ثم أضيفت ضريبة 15%. هل يعود إلى 800؟**

الإجابة المشروحة: لا؛ الخصم يعطي 680، والضريبة عليه 102، فيصبح 782. النسبتان تطبقان على أساسين مختلفين.

03 **متى يكون المتوسط الحسابي مضلِّلًا؟**

الإجابة المشروحة: عند وجود قيم متطرفة أو توزيع غير متوازن؛ قد يكون الوسيط أو توزيع القيم أكثر تعبيرًا من المتوسط وحده.

04 **حوّل 10,000 ثانية إلى ساعات ودقائق وثوانٍ باستخدام القسمة والباقي.**

الإجابة المشروحة: الساعات 10000÷3600 = 2، والباقي 2800. الدقائق 2800÷60 = 46، والباقي 40؛ النتيجة 2:46:40.

الخطوة التالية

كمّل في القسمة والباقي ودقة الحساب بعد تنفيذ التجربة هنا.

القسمة والباقي ودقة الحساب

القسمة والباقي ودقة الحساب

معاك 10 قطع وعلب تسع كل منها 3. ثلاث علب لا تكفي؛ تحتاج 4. لكن لو بتسأل عن عدد العلب الممتلئة فقط فالإجابة 3 والباقي قطعة. السؤال هو اللي يحدد نوع القسمة، وبعده نختار الرمز.

احسب المثال قبل المصطلح

`floor(10/3)=3` تعني اختيار أقرب عدد صحيح أقل من الناتج أو يساويه. `ceil(10/3)=4` تختار أقرب عدد صحيح أكبر منه أو يساويه. مع −2.3، الأرضية −3 والسقف −2؛ ارسم القيم على خط الأعداد قبل تعميم قاعدة الموجب.

عند توزيع 12 و18 قطعة في مجموعات متساوية بلا باقي، أكبر حجم مناسب هو 6. ده **GCD — Greatest Common Divisor**، أكبر قاسم مشترك. لو جرس يدق كل 4 دقائق وآخر كل 6، أول اجتماع بعد البداية عند 12؛ ده **LCM — Least Common Multiple**، أصغر مضاعف مشترك موجب للعددين الموجبين.

جرّب: 17 قطعة وعلب تسع 5: كم علبة ممتلئة، وكم قطعة متبقية، وكم علبة تحتاج لكل القطع؟ اكتب 3 و2 و4، وفسّر كل نتيجة قبل متابعة القواعد.

القسمة الصحيحة وFloor وCeil

القسمة الصحيحة ترجع خارج قسمة صحيحًا؛ تقريب الأعداد السالبة يعتمد على اللغة، زي الأمثلة التالية. للحصول على عدد مجموعات تسع كل منها `size` عنصرًا نستخدم القسمة السقفية للأعداد الموجبة: `(count + size - 1) / size`. أما `floor` فيختار العدد الصحيح الأقل أو المساوي، و`ceil` يختار الأكبر أو المساوي، و`round` يقرب وفق قواعد الدالة.

GCD وLCM وخوارزمية إقليدس

القاسم المشترك الأكبر `GCD` أكبر عدد يقسم القيمتين. تعتمد خوارزمية إقليدس على أن `gcd(a, b) = gcd(b, a % b)` حتى يصبح الباقي صفرًا. والمضاعف المشترك الأصغر يساوي `abs(a / gcd(a,b) * b)` مع الانتباه للصفر وتجاوز المدى.

Modular Arithmetic

العلاقة `a % m` تضع القيم في دورة من البواقي. نستخدمها للزوجية والساعات والتكرار والتحقق من القابلية للقسمة. انتبه إلى أن باقي العدد السالب يختلف سلوكه بين الصيغ الرياضية وبعض اللغات، لذلك طبّع النتيجة عند الحاجة بصيغة مثل `((value % m) + m) % m` عندما يكون `m` موجبًا.

الدقة والخطأ العددي

الخطأ المطلق هو مقدار الفرق بين القيمة المحسوبة والحقيقية، والخطأ النسبي يقارن هذا الفرق بحجم القيمة. عند مقارنة نتائج Floating Point اختر Tolerance يناسب المجال، ولا تستخدم قيمة ثابتة واحدة لكل المقاييس.

احسب بدل حفظ اسم العملية

القوة ضرب متكرر:2^3=2×2×2=8. **الجذر التربيعي** عدد غير سالب مربعه يساوي القيمة:√25=5. المتوسط للقائمة2،4،9 هو15÷3=5؛ الوسيط4 هو القيمة الوسطى بعد الترتيب. لا نحسب متوسط قائمة فارغة بقسمة على صفر.

الباقي في17÷5 يساوي2 لأن17=3×5+2. رمز% يستعمله كثير من اللغات للباقي. القسمة الصحيحة ليست لها قاعدة سالب واحدة في كل اللغات: في C++ قسمة-7 على3 تحذف الجزء الكسري نحو الصفر فتعطي-2، بينما Python مع// تقرّب لأسفل فتعطي-3. وفي JavaScript،7/3 قسمة عادية مش قسمة صحيحة. اذكر اللغة قبل توقع النتيجة.

لعدد10 عناصر في مجموعات تسع3، القسمة السقفية تعطي4. الصيغة(count+size−1)÷size تحتاج **قسمة صحيحة** وcount غير سالب وsize موجب؛ انتبه لتجاوز مدى الجمع. صفر عناصر يحتاج صفر مجموعات، وحجم مجموعة صفر مدخل مرفوض.

GCD (Greatest Common Divisor)؛ أكبر قاسم مشترك لعددين) — أكبر قاسم مشترك: لـ48 و18، البواقي:48%18=18،18%12=12،12%6=6،6%6=0؛ آخر قاسم غير صفري6. **LCM (Least Common Multiple)؛ أصغر مضاعف مشترك غير سالب وفق اتفاق المثال) — أصغر مضاعف مشترك** يساوي48÷6×18=144. إذا أحد العددين صفر نعيد0 وفق الاتفاق البرمجي الشائع، ونعالج(0،0) منفصلًا قبل القسمة. abs تعني القيمة المطلقة، أي المسافة عن الصفر.

تدريب وحل: الساعة23 وبعدها5 ساعات: (23+5)%24=4. ده حساب دوري. **Tolerance — هامش قبول فرق عددي** نختاره وفق القياس؛ **Overflow — تجاوز المدى** قيمة لا يسعها النوع، مش خطأ تقريب نعالجه بالهامش.

المتغيرات والمعادلات والمنطق البولياني

المتغير اسم لقيمة قد تتغير، والمعادلة تصف علاقة بين قيم. المنطق البولياني يحول شروط المسألة إلى قرارات يمكن للبرنامج تقييمها.

استخدم القسم بعد القيم والقرارات والحلقات. ارجع للحساب حسب الحاجة؛ الرسوم والتعقيد والبرمجة الديناميكية توسعات لاحقة وليست شروطًا لأول برنامج.

جملة منطقية نجربها

الدخول مسموح لو معاك تذكرة **ومش محظور**. المنطق البولياني يستخدم نتيجتين: `true` صحيح و`false` غير صحيح. `AND` معناها لازم الشرطان، و`NOT` تعكس الإجابة.

تذكرة؟	محظور؟	مسموح؟
لا	لا	لا
لا	نعم	لا
نعم	لا	نعم
نعم	نعم	لا

ده **جدول حقيقة**: كل تركيبات المدخلات ونتيجتها. اشرح الصف الأخير قبل متابعة الرموز. المجموعات والعلاقات لها درس مستقل بعد تثبيت القرار البسيط.

الفكرة العامة

المتغير اسم لقيمة قد تتغير، والمعادلة تصف علاقة بين قيم. المنطق البولياني يحول شروط المسألة إلى قرارات يمكن للبرنامج تقييمها.

المفاهيم الأساسية

- التعبير ينتج قيمة، بينما المعادلة أو المقارنة تسأل عن علاقة.
- الإسناد يحدّث حالة المتغير؛ لذلك ترتيب التعليمات قد يغيّر النتيجة.
- AND يتطلب تحقق الشروط كلها، وOR يكفيه شرط واحد، وNOT يعكس القيمة.
- XOR يساوي true عندما يختلف الشرطان، لكنه أقل استخدامًا في شروط الأعمال اليومية.
- اكتب كل شرط بسيط منفصلًا ثم اجمعه لتقليل أخطاء المنطق.

مثال تطبيقي

أهلية قرض قد تكون: عمر مناسب AND دخل كافٍ AND لا توجد مديونية خطرة. حوّل كل عبارة إلى Boolean باسم واضح ثم اجمعها في قرار نهائي.

تصحيح مفاهيم وأخطاء شائعة

- لا تخلط بين "يساوي" و"ضع قيمة في المتغير".
- انتبه للأقواس عند دمج AND وOR حتى يكون المعنى صريحًا.

خريطة الدرس

خريطة مفاهيم: المتغيرات والمعادلات والمنطق البولياني

Implicationو Truth Tables

يسجل Truth Table نتيجة التعبير لكل تركيب من القيم المنطقية. يكون `A && B` صحيحًا فقط عندما يصح الاثنان، و`A || B` صحيحًا عندما يصح أحدهما على الأقل، وXOR صحيحًا عندما يختلفان.

العبارة `A -> B` تكون خاطئة فقط عندما يصح A وتفشل B. لا تخلطها بعلاقة سببية؛ إنها صياغة منطقية. وتقول قوانين De Morgan إن نفي `A && B` يساوي `!A || !B`، ونفي `A || B` يساوي `!A && !B`.

تأكد من فهمك

01 **ما الفرق بين x = x + 1 في البرمجة والمعادلات الرياضية؟**

> **الإجابة المشروحة:** برمجيًا هي إسناد: احسب القيمة القديمة ثم خزّن الأكبر بواحد. رياضيًا المعادلة x=x+1 مستحيلة على الأعداد العادية.

02 **حوّل "مقبول إذا كان موظفًا أو دخله كافيًا، وبشرط ألا يكون محظورًا" إلى منطق واضح.**

> **الإجابة المشروحة:** eligible = (isEmployee OR hasEnoughIncome) AND NOT isBlocked. الأقواس ضرورية لأن الحظر يجب أن يرفض الحالتين.

03 **متى تختلف عدة شروط if مستقلة عن سلسلة else-if؟**

> **الإجابة المشروحة:** الشروط المستقلة قد تنفذ أكثر من فرع، بينما else-if تختار أول شرط صحيح فقط؛ الاختيار يعتمد على كون النتائج متزامنة أم حصرية.

04 **كيف تختبر قاعدة منطقية مركبة دون تجربة كل القيم عشوائيًا؟**

> **الإجابة المشروحة:** أنشئ جدول حقيقة أو جدول قرار يغطي كل شرط منفردًا، التركيبات المهمة، وحدود المقارنات، ثم حدد النتيجة المتوقعة مسبقًا.

الخطوة التالية

كمّل في المجموعات والعلاقات ومدخلات الدالة بعد تنفيذ التجربة هنا.

المجموعات والعلاقات ومدخلات الدالة

المجموعات والعلاقات ومدخلات الدالة

خلّينا نبدأ بدالة تربيع. نسمح فقط بالمدخلين 1 و2. الناتجان 1 و4. **Domain — مجموعة المدخلات المسموحة** هنا {1,2}. لو اتفقنا أن الناتج عدد صحيح غير سالب، فدي **Codomain — مجموعة النتائج المسموح بها**. أما **Range — النتائج التي حصلت فعلًا** فهي {1,4}. الفرق بين المسموح والمتحقق مهم حتى لو الدالة واضحة.

جرّب قبل القراءة: لو أضفنا المدخل 3، ما الذي يزيد في النتائج الفعلية؟ 9. هل كل عدد غير سالب أصبح ناتجًا؟ لا. بعد كده نستخدم نفس فكرة المجموعة لشرح العلاقات و«كل» و«يوجد».

المجموعات والعلاقات والدوال

المجموعة عناصر فريدة بلا اعتماد ضروري على الترتيب. الـSubset مجموعة تقع عناصرها كلها داخل أخرى. يجمع Union العناصر الموجودة في أي من المجموعتين، ويحتفظ Intersection بالعناصر المشتركة، ويزيل Difference عناصر مجموعة من أخرى.

العلاقة مجموعة أزواج تربط عناصر. يمكن أن تكون انعكاسية أو متناظرة أو متعدية. أما الدالة فتربط كل عنصر من Domain بنتيجة واحدة في Codomain، وتسمى القيم الناتجة فعليًا Range. يظهر هذا الفرق في تصميم APIs والتحقق من أن لكل مدخل صالح نتيجة محددة.

Predicates وQuantifiers

الـPredicate عبارة تعتمد على قيمة مثل `isEven(x)`. يعني «لكل» أن الشرط ينجح لكل عنصر، ويعني «يوجد» أن عنصرًا واحدًا على الأقل يحققه. تقابلها في البرمجة عمليات مثل `all_of` و`any_of` وفحوص القوائم.

جدول حقيقة وعلاقات بمثال

Boolean — قيمة منطقية إما true بمعنى صح أوfalse بمعنى خطأ. الرموز&& و|| و! في لغات كثيرة تعني AND وOR وNOT؛ الرمز البرمجي الفعلي يتبع اللغة.

A	B	A AND B	A OR B	A XOR B	A يؤدي إلى B
خطأ	خطأ	خطأ	خطأ	خطأ	صح
خطأ	صح	خطأ	صح	صح	صح
صح	خطأ	خطأ	صح	صح	خطأ
صح	صح	صح	صح	خطأ	صح

«يؤدي إلى» علاقة منطقية تفشل فقط لو المقدمة صحيحة والنتيجة خاطئة؛ لا تثبت سببية. **De Morgan** اسم قاعدتي نفي: ليس(A وB) يساوي(ليسA أو ليسB)، وليس(A أوB) يساوي(ليسA و ليسB). جرّب الصفوف الأربعة للتحقق بدل الحفظ.

لو{1،2}=A وB={2،3} ، **Union — الاتحاد** {1،2،3}، و**Intersection — التقاطع** {2}، و**Difference — الفرق** {1}=B−A. المجموعة{1} **Subset — مجموعة جزئية** منA.

العلاقة **انعكاسية** لو كل عنصر مرتبط بنفسه، و**متناظرة** لو ارتباطb بـa يستلزمb بـb، و**متعدية** لوa مرتبط بـb وb بـc يستلزمa بـc. المساواة مثال يحقق الثلاثة.

في الدالة f(x)=x×x للمدخلات{1،2}، **Domain — مجال المدخلات** {1،2}. لو اخترنا الأعداد الصحيحة غير السالبة **Codomain — مجالًا مستهدفًا**، فالنتائج الفعلية **Range** هي{1،4}. كل مدخل له نتيجة واحدة.

Predicate شرط يعتمد على قيمة، زي «x زوجي». **Quantifier — محدد كمي** يقول لكل عنصر أو يوجد عنصر. في{2،3} يوجد زوجي، لكن ليس كل عنصر زوجيًا. دي إجابة تدريب توضح أن تبديل«يوجد» بـ«لكل» يغيّر المعنى.

التفكير الحاسوبي وتحليل المتطلبات

التفكير الحاسوبي طريقة منظمة لتحويل مسألة غير واضحة إلى أجزاء وقواعد وخطوات قابلة للتنفيذ والاختبار.

استخدم القسم بعد القيم والقرارات والحلقات. ارجع للحساب حسب الحاجة؛ الرسوم والتعقيد والبرمجة الديناميكية توسعات لاحقة وليست شروطًا لأول برنامج.

مطلب واحد يتحول لاختبارات

مطلب ماكينة السحب: «اسمح بسحب مبلغ موجب، من مضاعفات 50، ولا يزيد على الرصيد». افترض رصيدًا 200 وعدم وجود رسوم. **القيود** الحدود التي يجب أن يلتزم بها المدخل.

المدخل	المتوقع	أي قاعدة نختبر؟
150	قبول، الباقي 50	حالة عادية
200	قبول، الباقي 0	الحد الأعلى نفسه
250	رفض	أكبر من الرصيد
0 أو −50	رفض	المبلغ لازم موجب
75	رفض	ليس مضاعفًا لـ50
نص غير رقمي	رفض قبل الحساب	نوع المدخل

قسم الحل إلى قراءة، تحقق، ثم خصم وعرض. الرفض لا يغيّر الرصيد. **حاول:** أضف رسمًا ثابتًا 10 لكل سحب؛ حالة 200 تصبح مرفوضة لأن التكلفة 210. تغيير المطلب يغيّر الاختبار قبل الكود.

الفكرة العامة

التفكير الحاسوبي طريقة منظمة لتحويل مسألة غير واضحة إلى أجزاء وقواعد وخطوات قابلة للتنفيذ والاختبار.

المفاهيم الأساسية

- التفكيك Decomposition يقسم المشكلة إلى مهام أصغر يمكن فهمها.
- التعرف على الأنماط يعيد استخدام حلول سابقة بدل البدء من الصفر.
- التجريد يحتفظ بالتفاصيل المؤثرة ويؤجل التفاصيل غير المهمة.
- تصميم الخوارزمية يرتب الخطوات ويحدد القرارات والتكرار.
- تحليل المتطلبات يحدد المدخلات والقواعد والحالات الحدية والمخرجات ومعايير النجاح.

مثال تطبيقي

في ماكينة ATM (Automated Teller Machine؛ ماكينة الصراف الآلي): افصل التحقق من البطاقة، التحقق من الرقم السري، اختيار العملية، فحص الرصيد والحد اليومي، تنفيذ السحب، ثم تحديث الرصيد وإصدار الإيصال.

تصحيح مفاهيم وأخطاء شائعة

- لا تبدأ بالكود قبل معرفة حالات الرفض والحدود.
- المتطلب الغامض مثل "سريع" يحتاج معيارًا قابلًا للقياس.

حوّل طلبًا صغيرًا لحاجة تقدر تختبرها

في قائمة المشتريات، عبارة «اقبل كمية» مش كفاية. نحدد عددًا صحيحًا من 1 إلى 100. نقسم المهمة: قراءة النص، رفض الفراغ، التحويل لعدد، فحص الحدود، ثم الإضافة. جرّب 0 و1 و100 و101 و2.5؛ المقبول 1 و100 فقط. فصل الخطوات يخليك تعرف أول مرحلة حصل فيها الخطأ.

مقارنة كلفة التشغيل وأنماط الحل لها مكان لاحق في مقارنة الخوارزميات. نتيجة الدرس الحالي مسألة واضحة واختبارات تتوقع ناتجها بنفسك.

تأكد من فهمك

01 **ما الفرق بين التفكيك والتجريد؟**

الإجابة المشروحة: التفكيك يقسم النظام إلى مشكلات أصغر، أما التجريد فيختار التفاصيل المؤثرة ويخفي ما لا يحتاجه الحل الحالي.

02 **في نظام حجز، ما متطلب غامض يجب تحويله إلى معيار قابل للاختبار؟**

الإجابة المشروحة: "النظام سريع" غامض. صياغة قابلة للاختبار: 95% من عمليات البحث تستجيب خلال أقل من 500ms تحت حمل محدد.

03 **كيف يمنع اكتشاف الأنماط تكرار الحل؟**

الإجابة المشروحة: يكشف بنية مشتركة مثل التحقق ثم القرار ثم التسجيل، فتُبنى وظيفة أو قاعدة عامة بدل نسخ منطق منفصل لكل حالة.

04 **حلل سحب ATM إلى وحدات وحدد أهم حالة حدية.**

الإجابة المشروحة: الوحدات: تحقق البطاقة والPIN (Personal Identification Number؛ رقم سري للتحقق من صاحب البطاقة أو الجهاز)، اختيار المبلغ، فحص الرصيد والحد، الصرف، التحديث، الإيصال. حالة حدية مهمة: مبلغ يساوي الرصيد أو الحد اليومي بالضبط.

الخوارزميات وPseudocode وأشجار القرار

الـPseudocode وشجرة القرار تمثيلان للتفكير قبل الكود: الأول يوضح تسلسل الخطوات، والثانية توضح مسارات القرارات المتفرعة.

استخدم القسم بعد القيم والقرارات والحلقات. ارجع للحساب حسب الحاجة؛ الرسوم والتعقيد والبرمجة الديناميكية توسعات لاحقة وليست شروطًا لأول برنامج.

الفكرة العامة

الـPseudocode وشجرة القرار تمثيلان للتفكير قبل الكود: الأول يوضح تسلسل الخطوات، والثانية توضح مسارات القرارات المتفرعة.

المفاهيم الأساسية

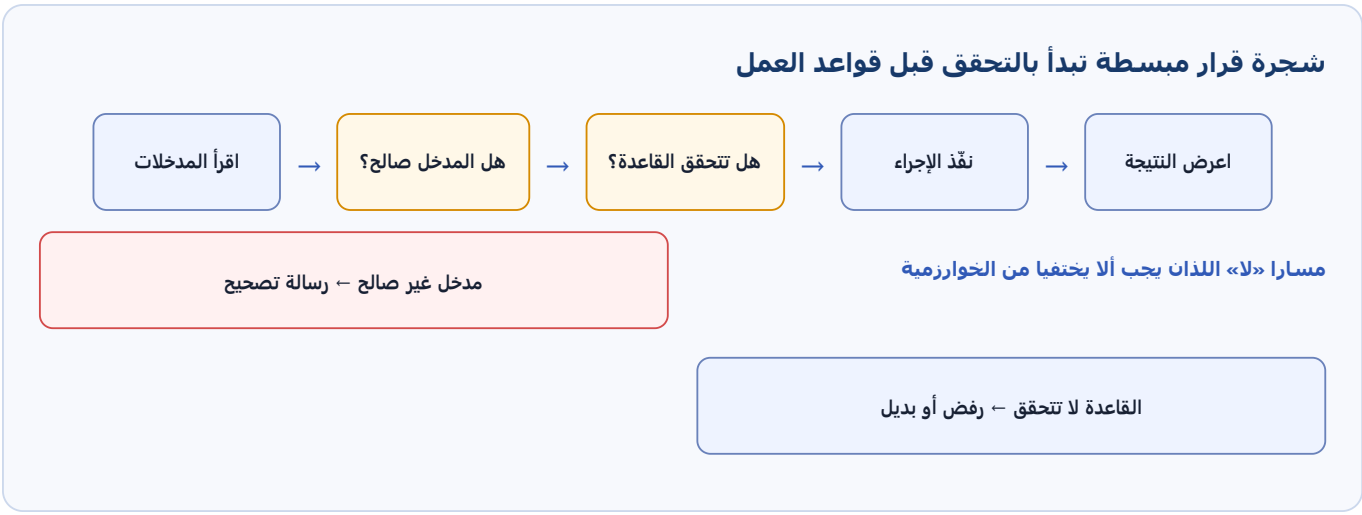
- الخوارزمية الجيدة لها مدخلات واضحة، خطوات غير ملتبسة، نهاية، ومخرجات صحيحة.
- استخدم أسماء تعبّر عن المعنى بدل x وy في مسائل الأعمال.
- الـPseudocode لا يلتزم بصياغة لغة، لكنه يجب أن يظل دقيقًا.
- شجرة القرار مناسبة عندما تقود الإجابات إلى حالات نهائية متعددة.
- رتّب التحقق من العام إلى الخاص، ومن رفض المدخل غير الصالح إلى قواعد العمل.

مثال تطبيقي

إيجاد الأكبر بين ثلاثة أعداد: ابدأ بالعدد الأول كأكبر قيمة، قارنه بالثاني وحدّثه عند الحاجة، ثم كرر مع الثالث. هذه الطريقة أبسط من كتابة كل التركيبات الممكنة.

تصحيح مفاهيم وأخطاء شائعة

- شجرة القرار الكبيرة قد تصبح صعبة الصيانة؛ اجمع القواعد المتشابهة وسمّ الشروط.
- لا تجعل مسارًا بلا نتيجة أو تترك حالة ممكنة دون معالجة.



اكتب الخوارزمية وتتبعها

Pseudocode — خطوات شبه برمجية وصف دقيق للحل من غير الالتزام بلغة تشغيل. المثال يستقبل ثلاثة أعداد صالحة ويرجع أكبر قيمة:

```
read a, b, c
largest = a
if b > largest:
    largest = b
if c > largest:
    largest = c
print largest
```

read معناها اقرأ، وprint اطبع، وif اختبر الشرط، وعلامة = هنا تعيين قيمة. مع a=−7 وb=−2 وc=−5 نبدأ بـlargest=−7، ثم نحدثها إلى−2، ثم نتركها لأن−5 أصغر. الناتج−2. لو بدأنا largest بصفر هنعطي نتيجة غلط لأن الصفر مش ضمن المدخلات.

Precondition — شرط قبل البدء: القيم أعداد صالحة. **Postcondition — شرط بعد الانتهاء**: الناتج واحد من القيم ولا توجد قيمة أكبر منه. عند تعميم الحل لقائمة، **Loop invariant — حقيقة محفوظة أثناء التكرار** هي أن largest أكبر القيم التي فحصناها حتى الآن. **Termination — الانتهاء** مضمون لأن كل دورة تفحص عنصرًا جديدًا وعدد العناصر محدود. القائمة الفارغة تحتاج قرارًا صريحًا، مثل إرجاع «لا توجد قيمة»، بدل قراءة أول عنصر غير موجود.

نفس مثال الأكبر على شكل شجرة

نفترض أن a وb وc أعداد صالحة. كل «نعم/لا» تختار فرعًا، وكل نهاية ترجع قيمة:

```
a >= b?
├─ Yes: a >= c?
│        ├─ Yes → a
│        └─ No  → c
└─ No:  b >= c?
         ├─ Yes → b
         └─ No  → c
```

`>=` أكبر من أو يساوي. جرّب −7 و−2 و−5: أول إجابة لا، ثم نعم، فنرجع b أي−2. جرّب تساوي الثلاثة: أي واحدة تحمل نفس أكبر قيمة. قارن الشجرة بخطوات تحديث `largest`؛ النتيجة نفسها، لكن تكرار التحديث يتوسع لقائمة بسهولة.

تأكد من فهمك

01 **لماذا طريقة "ابدأ بالأول كأكبر قيمة ثم حدّثه" أفضل من تعداد ترتيبات ثلاثة أعداد؟**

الإجابة المشروحة: تحتاج مقارنتين فقط وتتوسع لأي عدد من القيم، بينما تعداد الترتيبات يزداد بسرعة ويصعب التحقق منه.

02 **متى تصبح شجرة القرار اختيارًا سيئًا؟**

الإجابة المشروحة: عندما تتكاثر الفروع والقواعد المشتركة فتتكرر العقد. عندها يفيد جدول قرار أو قواعد مسماة أو State Machine.

03 **ما الخصائص الأربع التي يجب إثباتها في خوارزمية جيدة؟**

الإجابة المشروحة: وضوح المدخلات والمخرجات، دقة الخطوات، الانتهاء في وقت محدود، والصحة لكل الحالات الواقعة داخل المتطلبات.

04 **رتب فحوص خصم يعتمد على صلاحية السعر والعضوية وقيمة الطلب.**

الإجابة المشروحة: تحقق من صلاحية السعر أولًا، ثم احسب الأهلية، ثم طبق قاعدة العضوية والحد المالي. لا تسمح لمدخل غير صالح بالدخول في حساب الخصم.

الجشع والبرمجة الديناميكية: اختار بالدليل

مثال العملات المحلول موجود في درس مقارنة الخوارزميات، بعد شرح تنظيم البيانات ونمو عدد الخطوات. هنا ركّز أولًا على كتابة خطوات حل واضح وتتبعها.

لما القرار يعتمد على حالة سابقة

في شجرة القرار، كل **ورقة** نهاية مسار. لو شرط الخصم «عضو وسعره100 أو أكثر»، اختبر عضوًا عند99 و100، وغير عضو عند100، وسعرًا سالبًا مرفوضًا. **State Machine — نموذج حالات وانتقالات** ينفع لما القرار يعتمد على الحالة السابقة، مثل طلب «جديد ← مدفوع ← مشحون». لا نسمح بانتقال جديد مباشرة إلى مشحون وفق القاعدة دي.

المخططات الانسيابية والحلقات والتصحيح

المخطط الانسيابي يوضح حركة التنفيذ بصريًا، والحلقات تمثل التكرار، والتتبّع Debugging يقارن ما حدث بما كان يجب أن يحدث.

استخدم القسم بعد القيم والقرارات والحلقات. ارجع للحساب حسب الحاجة؛ الرسوم والتعقيد والبرمجة الديناميكية توسعات لاحقة وليست شروطًا لأول برنامج.

الفكرة العامة

المخطط الانسيابي يوضح حركة التنفيذ بصريًا، والحلقات تمثل التكرار، والتتبّع Debugging يقارن ما حدث بما كان يجب أن يحدث.

المفاهيم الأساسية

- البداية/النهاية بيضاوية، العملية مستطيل، القرار مُعيّن، والإدخال/الإخراج متوازي أضلاع.
- كل قرار يجب أن يوضح فروعه، وغالبًا يكونان نعم/لا.
- الحلقة تحتاج بداية وشرط استمرار وتحديثًا؛ غياب التحديث سبب شائع للحلقة اللانهائية.
- Counter يعدّ العناصر، وAccumulator يجمع قيمها، وSentinel ينهي الإدخال بقيمة خاصة.
- التتبّع اليدوي يسجل قيمة المتغيرات عند كل دورة ويكشف الأخطاء المنطقية.

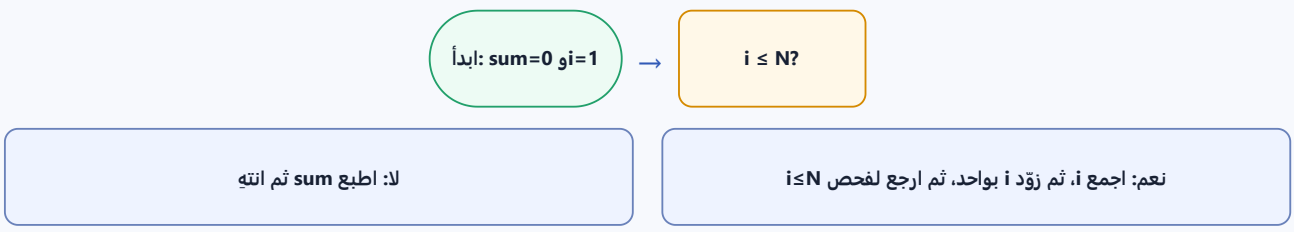
مثال تطبيقي

لحساب مجموع `1..N`: ابدأ بـ `sum = 0` و `i = 1`، وكرر ما دام `i <= N`: أضف `i` إلى `sum` ثم زد `i`. بعد انتهاء الحلقة اطبع `sum`.

تصحيح مفاهيم وأخطاء شائعة

- المخطط يشرح المنطق لكنه لا يعوض اختبار الكود.
- البرنامج الذي يعمل بلا رسالة خطأ قد يظل خطأ منطقيًا.

مسارات واضحان من الشرط



امشِ مع الحلقة دورة بدورة

Counter — عدّاد يعد مرات، زي attempts لعدد المحاولات. **Accumulator — قيمة تتجمع** زي sum لمجموع الأسعار. **Sentinel — علامة توقف** قيمة متفق عليها، مثل كلمة "stop" لإنهاء الإدخال؛ لازم نختار علامة لا تتعارض مع البيانات المسموحة.

في جمع1 إلىN، نفترض أن N عدد صحيح غير سالب:

قبل فحص الشرط	i	sum	هل i≤3؟	بعد الدورة
البداية	1	0	نعم	sum=1، i=2
الثانية	2	1	نعم	sum=3، i=3
الثالثة	3	3	نعم	sum=6، i=4
الرابعة	4	6	لا	اخرج واطبع6

التحديث يرجع لفحص الشرط؛ الخروج فرع «لا» من الشرط نفسه. مع N=0 لا ندخل جسم الحلقة والناتج0؛ مع N=1 الناتج1. **Off-by-one — خطأ بزيادة أو نقص دورة واحدة** مثل استخدام i<N بدل i≤N؛ عند N=3 الناتج3 لأننا أسقطنا3.

تدريب محلول: لو نسينا i=i+1 وكان N≥1، يظل الشرط صحيحًا ويتكرر الجسم بلا نهاية. راقب i في جدول التتبع أو بأداة إيقاف التنفيذ خطوة بخطوة، ثم أعد اختبار0 و1 و3.

تأكد من فهمك

01 **ما العناصر الثلاثة التي يجب مراجعتها لمنع حلقة لا نهائية؟**

الإجابة المشروحة: قيمة البداية، وشرط الاستمرار، والتحديث في كل مسار. يجب أن يجعل التحديث الحالة أقرب إلى توقف الشرط.

02 **ما الفرق بين Counter وAccumulator وSentinel؟**

الإجابة المشروحة: Counter يعد الأحداث، وAccumulator يجمع قيمًا، وSentinel قيمة خاصة تنهي الإدخال ولا تدخل عادة في الحساب.

03 **حلقة تجمع 1..N أعادت قيمة أكبر من المتوقع بـN. ما الاشتباه الأول؟**

الإجابة المشروحة: راجع البداية والحدود: ربما بدأ sum بقيمة N أو جُمعت N مرتين. الشرط i<=N+1 يضيف N+1، مش N؛ لما N=3 الصحيح 6، ومع الدورة الرابعة الناتج 10 والزيادة 4. جدول التتبع يكشف أول دورة انحرفت.

04 **صمّم تسجيل دخول بثلاث محاولات دون خطأ off-by-one.**

الإجابة المشروحة: ابدأ attempts=0، وبعد كل فشل زدها، وكرر ما دام غير ناجح وattempts<3. اختبر النجاح الأول والثالث وثلاثة إخفاقات.

مقارنة الخوارزميات وطرق تنظيم البيانات

تحويل المشكلة إلى خطوات، كتابة Pseudocode، اختيار بنية البيانات، وفهم التعقيد بصورة عملية.

ابدأ بعد التطبيق على المتغيرات والحلقات والقوائم في اللغة التي اخترتها.

قبل التفاصيل

البرمجة تبدأ قبل الكود: حدّد المدخلات والمخرجات والقيود، واكتب خطوات يمكن اختبارها، ثم اختر بنية البيانات والتعقيد المناسبين للحجم.

من المشكلة إلى برنامج

لا تبدأ بالكود. اكتب أولًا:

1. **المدخلات:** ما البيانات القادمة؟ وما حدودها؟
2. **المخرجات:** ما النتيجة المطلوبة بدقة؟
3. **القواعد:** ما الحالات العادية والاستثنائية؟
4. **التقسيم:** حوّل المشكلة إلى خطوات صغيرة قابلة للاختبار.
5. **التحقق:** جرّب مثالًا عاديًا، حدًا أدنى، حدًا أقصى، ومدخلًا غير صالح.

مثال: حساب إجمالي سلة بعد خصم.

```
INPUT items, discountPercent
IF discountPercent < 0 OR discountPercent > 100
    RETURN error
total = 0
FOR EACH item IN items
    IF item.price < 0 OR item.quantity < 1
        RETURN error
    total = total + item.price * item.quantity
RETURN total * (1 - discountPercent / 100)
```

الـPseudocode يشرح المنطق بلا ارتباط بصياغة لغة، ثم يمكن تحويله إلى PHP (اسم لغة برمجة تستخدم كثيرًا لمعالجة طلبات الويب على الخادم) أو JavaScript أو غيرهما.

اختيار بنية البيانات

الحاجة	بنية مناسبة
ترتيب عناصر والسماح بالتكرار	List/Array
الوصول بقيمة مفتاح	Map/Associative Array
منع التكرار	Set
أول داخل أول خارج	Queue
آخر داخل أول خارج	Stack

اختيار البنية يؤثر في وضوح الحل وكلفته. لا تستخدم nested loops تلقائيًا إذا كان Map يستطيع تحويل البحث المتكرر إلى lookup مباشر.

التعقيد Big O

Big O يصف نمو الزمن أو الذاكرة مع نمو حجم الإدخال، وليس عدد الثواني الفعلي.

التعقيد	مثال
`O(1)`	قراءة موضع معلوم في مصفوفة تدعم الوصول المباشر
`O(log n)`	Binary search في بيانات مرتبة
`O(n)`	المرور مرة على القائمة
`O(n log n)`	خوارزميات فرز فعالة شائعة
`O(n²)`	مقارنة كل عنصر بكل عنصر

الأولوية للصحة والوضوح، ثم القياس قبل التحسين. خوارزمية `O(n)` قد تكون أبطأ على بيانات صغيرة من حل أبسط بسبب الثوابت.

تمرين

لديك قائمة طلبات، وكل طلب يحوي `customer_id` و`total`. صمّم خوارزمية تعيد إجمالي كل عميل:

- اكتب المدخل والمخرج.
- عالج القائمة الفارغة والقيم السالبة.
- استخدم جدول تجميع؛ حلل الافتراض الذي يجعل زمنه المتوقع `O(n)` كما في الحل.
- اكتب ثلاث حالات اختبار قبل كتابة الكود.

مسائل عملية

▼ متى تكون قائمة أفضل من set؟

عندما تحتاج الترتيب أو التكرار أو المرور المتتابع، ولا تحتاج اختبار العضوية السريع الذي يبرر set.

▼ لماذا لا يكفي أن تعمل الخوارزمية على ثلاثة عناصر؟

لأن التعقيد قد يجعلها غير عملية على مليون عنصر. اختبر الصحة ثم قدّر نمو الزمن والذاكرة مع حجم الإدخال.

▼ ما فائدة pseudocode؟

يفصل منطق الحل عن syntax اللغة، فيسهل كشف الحالات الناقصة قبل كتابة التفاصيل التنفيذية.

حل تمرين العملاء ومعنى اختيار البنية

List/Array — قائمة أو مصفوفة تحتفظ بعناصر يمكن الرجوع إليها بموضع، مثل [8,3,8]. **Map — جدول مفاتيح وقيم** يربط مفتاحًا بقيمة، مثل رقم العميل وإجماليه. **Set — مجموعة** تمنع تكرار القيمة. **Queue — طابور** يخرج الأقدم أولًا زي صف الانتظار، و**Stack — مكدس** يخرج الأحدث أولًا زي رصة أطباق. تفاصيل الترتيب والأداء تعتمد على تنفيذ البنية في اللغة.

Lookup يعني البحث عن قيمة. الوصول لموضع معلوم في مصفوفة عادة O(1). البحث بالمفتاح في **Hash map — جدول يستخدم دالة لتحويل المفتاح إلى مكان بحث** متوسطه المتوقع O(1) بافتراض توزيع مناسب وإدارة سعة؛ الأسوأ قد يكون O(n) في تنفيذ تقليدي عند تزاحم مفاتيح كثيرة. جدول مرتب بشجرة قد يوفر O(log n) بدلًا منه. لذلك جملة «كل Map ثابت الوقت» غلط.

Binary search — بحث ثنائي يستبعد نصف النطاق المرتب كل مرة: للبحث عن7 في [1,3,5,7,9] نفحص5، ثم نكمل في الجزء الأكبر. $\log_2 n$ يسأل كم مرة نقسم n على2 حتى يبقى1؛ مثلًا1024 تحتاج10 مرات. الفرز يعني إعادة ترتيب العناصر حسب قاعدة، مثل من الأصغر للأكبر.

عقد تمرين العملاء: كل طلب له رقم عميل صحيح موجب وإجمالي غير سالب، ونرفض غير ذلك. هنفترض الأرقام هنا بوحدات عملة صحيحة صغيرة، مثل القروش، لتجنب تقريب الكسور. القائمة الفارغة ترجع جدولًا فارغًا.

```
totals = empty map
for each order in orders:
    validate order.customer_id and order.total
    old = totals.get(order.customer_id, default=0)
    totals[order.customer_id] = old + order.total
return totals
```

get تقرأ القيمة وتستعمل0 لو العميل جديد، وvalidate تطبق قواعد الصلاحية المذكورة وليست دالة جاهزة في لغة. للطلبات (عميل1،100)، (عميل2،50)، (عميل1،20)، الناتج {1:120,2:50}. إجمالي −1 مرفوض. مع Hash map مناسبة، الزمن المتوقع O(n) والذاكرة O(k) لعدد k من العملاء المختلفين، غير مساحة المدخل نفسه.

Precondition شرط صالح قبل التنفيذ، و**Postcondition** ما يجب أن يحققه الناتج. **Invariant** هنا: بعد كل طلب، الجدول يحتوي مجموع الطلبات التي فحصناها لكل عميل. **Termination** انتهاء الحلقة مضمون لأن القائمة محدودة. **Recursion — استدعاء ذاتي** تحتاج حالة توقف؛ مثل جمع القائمة بإضافة أول عنصر لمجموع الباقي، والتوقف عند القائمة الفارغة. كل استدعاء ينتظر نتيجة التالي ويستهلك مساحة في مكدس الاستدعاءات، لذلك التكرار الطويل قد يكون أبسط وأقل استهلاكًا.

تدريب محلول: أكبر قيمة في [−4] هي −4، وفي قائمة فارغة نرجع «لا توجد قيمة» وفق العقد. بعد فحص كل عنصر، المتغير largest يجب أن يمثل أكبر ما فحصناه؛ تهيئته بصفر تفشل مع القوائم السالبة.

Best وAverage وWorst Case

قد يختلف عدد العمليات باختلاف المدخل. يجد البحث الخطي الهدف في أول عنصر في أفضل حالة، وقد يفحص العناصر كلها في أسوأ حالة. عند مقارنة الخوارزميات اذكر الحالة المقصودة ولا تكتب Big O بلا تفسير.

Time وSpace Complexity

يقيس Time Complexity نمو عدد العمليات، ويقيس Space Complexity الذاكرة الإضافية. قد نستبدل الزمن بذاكرة إضافية، مثل استخدام Set لتسريع اكتشاف التكرار. تجاهل الثوابت لا يعني أنها غير مهمة عمليًا، لكنه يوضح السلوك عند نمو `n`.

تصميم الاختبارات

قسّم المدخلات إلى فئات متكافئة، ثم اختبر الحدود والفارغ والقيمة الواحدة والتكرار والقيم الكبيرة والمدخل غير الصالح. اختبر Overflow والعمليات التي تغير الفرع. اكتب النتيجة المتوقعة يدويًا قبل تشغيل البرنامج حتى لا تفسر خرجه على أنه صحيح لمجرد أنه يبدو معقولًا.

الخطوة التالية

كمّل في اختَر بين الحل الجشع وحفظ النتائج بعد تنفيذ التجربة هنا.

كمّل في حوّل أنماط الحل إلى خطوات بعد تنفيذ التجربة هنا.

ليه الحلقة صحيحة وبتنتهي؟

ليه الحلقة صحيحة وبتنتهي؟

ارجع لجدول جمع 1 إلى 3 في درس التتبع: قبل كل دورة اسأل «إيه اللي اتحسب لحد دلوقتي؟». **حقيقة الحلقة** جملة تفضل صحيحة عند موضع محدد في كل دورة، وليست مجرد نتيجة تجربة واحدة.

ابدأ بالجدول ثم برّر

قبل i=1 المجموع 0، وقبل i=2 المجموع 1، وقبل i=3 المجموع 3. كل مرة sum يساوي مجموع الأعداد السابقة لـi. إضافة i تحافظ على الجملة للدورة التالية. عدد القيم المتبقية يقل حتى صفر، فيتوقف التنفيذ. ده فرق اختبار أمثلة عن تفسير سبب صحة الخطوات.

حقيقة **Loop invariant** هنا: قبل كل فحص، sum مجموع الأعداد من1 إلى i−1. البداية تحققها لأن المجموع الفارغ0، وكل دورة تحافظ عليها بإضافة i. **Variant** — **مقدار يقترب من التوقف** هو عدد القيم المتبقية N−i+1 أثناء استمرار الحلقة؛ يقل واحدًا في كل دورة.

Nested loops — حلقات داخل بعضها لا تعني تلقائيًا n^2. لو الخارجية تتكرر n مرة والداخلية3 مرات، الإجمالي3n وينمو خطيًا O(n). لو الداخلية n مرة كل مرة، الإجمالي n^2. احسب عدد الخطوات الفعلي قبل وصف النمو.

تمرين: لو الخارجية 4 دورات والداخلية 3، فكم تنفيذًا للجسم؟ 12. لو العددان يتغيران، سمّهما n وm فيكون n×m؛ لا تفترض أنهما نفس العدد.

حوّل أنماط الحل إلى خطوات

حوّل أنماط الحل إلى خطوات

الاسم وحده مش طريقة حل. هنستخدم قائمة صغيرة لنعد التكرار ونحسب مجموع جزء منها، ثم نوضح متى يسمح الترتيب بتحريك مؤشرين. اكتب النتيجة يدويًا قبل اختيار الطريقة.

أنماط الحل

يجرب Brute Force كل الاحتمالات الممكنة وقد يكون البداية الصحيحة إذا كانت القيود صغيرة. تحاكي Simulation القواعد خطوة بخطوة. يستخدم Frequency Map عدادًا لكل قيمة، وتحسب Prefix Sum مجموع كل بادئة للإجابة السريعة عن نطاقات. يحرك Two Pointers مؤشرين وفق خاصية مرتبة، بينما يختار Greedy أفضل قرار محلي ويحتاج برهانًا أنه يؤدي إلى حل صحيح.

من اسم النمط إلى خطواته

خُد القائمة [2,1,2,3]. **Frequency Map — جدول تكرار** يربط كل قيمة بعدد ظهورها: {2:2,1:1,3:1}. علامة النقطتين هنا تربط القيمة بعدّادها. **Set — مجموعة قيم مميزة** يحتفظ بـ{1,2,3} من غير عدّاد؛ ينفع لسؤال «القيمة ظهرت قبل كده؟».

Prefix Sum — مجموع من البداية للقائمة [2,1,2,3] هو P=[0,2,3,5,8]. بدأنا بصفر لتمثيل عدم جمع أي عنصر. لو ترقيم العناصر من1، مجموع العناصر من2 إلى4 هو P[4]−P[1]=8−2=6. تجهيز P يحتاج المرور على القائمة مرة؛ كل سؤال بعد كده يحتاج عمليتي وصول وطرح بدل إعادة الجمع.

Two Pointers — مؤشرين: في قائمة مرتبة [1,3,5,8] عايزين قيمتين مجموعهما9. نبدأ من الطرفين:1+8=9 فنجد الحل. لو المجموع أقل من المطلوب نحرك الطرف الأصغر يمينًا، ولو أكبر نحرك الأكبر يسارًا. الترتيب هو اللي يبرر إسقاط الاحتمالات؛ نفس القاعدة لا تصلح لقائمة غير مرتبة.

Brute Force — تجربة كل الاحتمالات هنا يجرب كل زوج. مع n عناصر فيه $n(n-1)/2$ زوجًا. لما الحجم يزيد من10 إلى100، العدد يزيد من45 إلى4950. ده نمو تربيعي نكتبه $O(n^2)$. المرور مرة على n عنصر نمو خطي O(n). العملية الثابتة O(1) لا يزيد عدد خطواتها مع n؛ مش معناها أنها تستغرق ثانية.

Greedy — اختيار جشع يختار أفضل خطوة حالية؛ مثال العملات في مثال العملات المحلول تحت يوضح ليه محتاج دليل صحة.
Dynamic Programming — البرمجة الديناميكية تحفظ نتائج مسائل أصغر متكررة، وفي مسائل أفضل حل تعتمد على إمكان بناء الحل الأفضل من حلول أصغر مناسبة. **Recursion — استدعاء ذاتي** أن تستدعي الدالة نفسها لمدخل أصغر مع حالة توقف؛ مش كل استدعاء ذاتي برمجة ديناميكية. **Divide and Conquer — تقسيم ودمج** يحل أجزاء أصغر ثم يدمجها، زي ترتيب نصفي قائمة ودمجهما.

تدريب محلول: في [4,4,2] عدد مرات4 هو2، وعدد القيم المميزة2، ومجموع القائمة10. القائمة الفارغة مجموعها0 وفق العقد ده. قيمة مثل "hello" مرفوضة لأن المدخل المطلوب أعداد. **Overflow — تجاوز السعة** يحصل لما الناتج لا يناسب نوع العدد؛ اختبر الحدود قبل الاعتماد على النتيجة.

اختَر بين الحل الجشع وحفظ النتائج

اختَر بين الحل الجشع وحفظ النتائج

المطلوب أقل عدد عملات لتكوين مبلغ. قبل أي اسم خوارزمية، جرّب تكوين 6 من 1 و3 و4: اختيار الأكبر أولًا يعطي 4+1+1؛ البديل 3+3 أفضل. هنفهم ليه الاختيار السريع لا يكفي، وإزاي نبني جدولًا لنتائج أصغر. اقرأ مقارنة عدد الخطوات وأنماط الحل قبل الدرس.

الجشع والبرمجة الديناميكية: اختار بالدليل

في مثال العملات التالي، dp اسم جدول للنتائج، وdp[x] خانة المبلغ x، وmin تختار القيمة الأصغر، و!= معناها «لا يساوي». O(Tk) يعني أن خطوات الحل تنمو مع حاصل ضرب مبلغ الهدف T في عدد أنواع العملات k؛ O(T) يعني مساحة جدول تتناسب مع الهدف.

عايز تكوّن 6 بأقل عملات من {1,3,4} مع السماح بالتكرار. الجشع Greedy يختار أكبر عملة تنفع دلوقتي: 4+1+1، ثلاث عملات. لكن 3+3 تحتاج عملتين. الاختيار المحلي الأفضل مش ضمان لأفضل حل كامل؛ لازم برهان يخص المسألة.

البرمجة الديناميكية Dynamic Programming بتحفظ حلول المسائل الأصغر. dp[x] أقل عدد عملات يكوّن x. البداية dp[0]=0، والباقي «غير قابل للوصول». نجرب آخر عملة c؛ الباقي x−c يحتاج dp[x−c] ثم نضيف واحدة. العملات لازم أعدادًا صحيحة موجبة، والهدف عددًا صحيحًا غير سالب.

```
coins = [1, 3, 4]
dp[0] = 0
for x from 1 through target:
    dp[x] = infinity
    for c in coins:
        if c <= x and dp[x-c] != infinity:
            dp[x] = min(dp[x], dp[x-c] + 1)
```

x	0	1	2	3	4	5	6
dp[x]	0	1	2	1	1	2	2

عند 6 بنقارن dp[5]+1=3 وdp[3]+1=2 وdp[2]+1=3، فنختار 2. أي حل ينتهي بعملة جربناها، والباقي مسألة أصغر اتحلت بالفعل؛ دي حجة صحة العلاقة. `infinity` علامة في pseudocode مش متغير جاهز: استخدم قيمة خاصة أو حدًا كبيرًا مع منع جمع الواحد عليه. لعدد k عملات وهدف T، الوقت O(Tk) والذاكرة O(T)، فالهدف مليار غير مناسب للجدول حتى لو k صغير.

تدريب محلول: مع {3,4} والهدف 2، النتيجة «غير ممكن»، مش صفرًا. الهدف 0 يحتاج صفر عملات. لو كل عملة متاحة مرة واحدة، الحل السابق لا يحقق العقد لأنه يسمح بإعادة استخدامها؛ لازم حالة أو ترتيب تحديث يعبّر عن المخزون المحدود.

القياس والوحدات والأعداد الأولية

القياس والوحدات والأعداد الأولية

استخدم القسم بعد القيم والقرارات والحلقات. ارجع للحساب حسب الحاجة؛ الرسوم والتعقيد والبرمجة الديناميكية توسعات لاحقة وليست شروطًا لأول برنامج.

الفكرة العامة

الرياضيات التطبيقية تربط الأرقام بوحدات وتمثيلات. التحويل والتقريب والإحداثيات والأعداد الأولية أدوات لحل مسائل عملية لا مجرد قوانين منفصلة.

المفاهيم الأساسية

- ثبت وحدة كل قيمة قبل الحساب وحوّل القيم إلى وحدة مشتركة.
- floor يتجه لأسفل وceil لأعلى، حتى مع الأعداد السالبة؛ أما round فيختار الأقرب.
- الإحداثي (x,y) يحدد نقطة، والرسم يوضح كيف تتغير قيمة مع أخرى.
- في الدالة y=2n+3، الميل 2 يعني زيادة y بمقدار 2 لكل زيادة واحدة في n.
- لاختبار أولية n يكفي تجربة القواسم حتى √n لأن العوامل تأتي في أزواج.

مثال تطبيقي

لاختبار 29، جرّب القسمة على الأعداد الأولية حتى √29≈5.38: أي 2 و3 و5. لا يقسمه أي منها، إذن 29 أولي.

تصحيح مفاهيم وأخطاء شائعة

- ceil(-2.3) يساوي -2 وليس -3.
- العدد 1 ليس أوليًا، وكل عدد زوجي أكبر من 2 غير أولي.

خريطة الدرس

خريطة مفاهيم: الرياضيات التطبيقية والقياس والرسوم والأعداد الأولية

تأكد من فهمك

01 لماذا ceil(-2.3) = -2 بينما floor(-2.3) = -3؟

الإجابة المشروحة: ceil يختار أصغر عدد صحيح لا يقل عن القيمة، وهو -2. floor يختار أكبر عدد صحيح لا يزيد عنها، وهو -3.

02 أثبت لماذا يكفي اختبار القواسم حتى $\sqrt{n}$ عند فحص الأولية.

الإجابة المشروحة: إذا كان $n=a\times b$ وكان العاملان أكبر من $\sqrt{n}$ فسيكون حاصل ضربهما أكبر من n؛ إذن أي تحليل مركب يملك عاملًا واحدًا على الأقل لا يتجاوز الجذر.

03 ماذا يعني الميل 2 في $y=2n+3$ عمليًا؟

الإجابة المشروحة: كل زيادة واحدة في n ترفع y بمقدار 2، بينما 3 هي القيمة الابتدائية عندما $n=0$.

04 ما خطأ اختبار العدد 1 كعدد أولي لأنه لا يقبل القسمة على 2؟

الإجابة المشروحة: تعريف الأولي يتطلب عاملين موجبين بالضبط: 1 والعدد نفسه. العدد 1 له عامل واحد فقط، لذلك ليس أوليًا.

الخطوة التالية

كمّل في المتتابعات والعد والاحتمال بعد تنفيذ التجربة هنا.

كمّل في ارسم العلاقات وابحث عن طريق بعد تنفيذ التجربة هنا.

المتتابعات والعد والاحتمال

المتتابعات والعد والاحتمال

عندك قميصان وثلاثة سراويل؛ اكتب كل زوج ممكن فتحصل على 6 اختيارات. ده بداية العد. بعده هتفرق بين ترتيب أشخاص واختيار مجموعة، ثم تحسب احتمالًا مع توضيح شروطه. افهم المجموعات أولًا من درسها السابق.

المتتابعات والعلاقات التكرارية

يزيد الحد في المتتابعة الحسابية بمقدار ثابت، بينما يضرب في نسبة ثابتة في المتتابعة الهندسية. يمكن حساب الحد مباشرة بدل توليد كل الحدود عندما تتوفر الصيغة. أما Recurrence فتعرف الحد من حدود سابقة مثل Fibonacci، ويجب تحديد Base Cases حتى لا يستمر التعريف بلا نهاية.

مبدأ العد والتباديل والتوافيق

إذا كان اختيار أول له `a` احتمالات ثم لكل اختيار أول اختيار ثانٍ له `b` احتمالات فعدد التركيبات `a*b` . التبديل يهتم بالترتيب، بينما التوافق يختار عناصر من غير اعتبار ترتيبها. لا تحسب Factorial مباشرة للقيم الكبيرة من غير فهم حدود النوع وإمكانية تبسيط الصيغة.

أساسيات الاحتمالات

فضاء العينة هو مجموعة النتائج الممكنة. احتمال الحدث عدد بين صفر وواحد. الحدثان مستقلان إذا لم يغير وقوع أحدهما احتمال الآخر. الاحتمال الشرطي يقيس وقوع A مع معرفة وقوع B. الأرقام العشوائية في البرامج غالبًا Pseudorandom وتحتاج Seed ومولدًا مناسبًا، ولا تستخدم مولدًا عاديًا لأسرار أمنية.

أمثلة للمتتابعات والعد والعشوائية

في متتابعة حسابية زي 3،5،7،9 بنضيف2 كل مرة؛ الحد رقم n، مع بداية العد من1، هو 3+2(n−1). في متتابعة هندسية زي 3،6،12،24 بنضرب في2؛ الحد هو 3×2^(n−1). علامة ^ هنا كتابة نصية للأس، مش معناها في كل لغة برمجة.

Recurrence — علاقة تكرارية بتحدد قيمة من قيم سابقة. في Fibonacci نحدد F(0)=0 وF(1)=1، ثم F(n)=F(n−1)+F(n−2) عندما n≥2. دي **Base cases — حالات البداية**؛ منها نحسب 0،1،1،2،3،5. من غير تحديدها ما نعرفش نبدأ.

Factorial — المضروب ورمزه n! هو حاصل ضرب الأعداد من1 إلى n، و0!=1. مثلًا4!=24 طريقة لترتيب أربعة أشخاص مختلفين. الأعداد بتكبر بسرعة؛20! أكبر من قدرة عدد صحيح32بت. ما تستخدمش نوعًا محدودًا قبل حساب أكبر قيمة ممكنة.

Pseudorandom — عشوائية زائفة سلسلة تنتج من خوارزمية وحالة بداية اسمها **Seed — بذرة**. نفس الخوارزمية والبذرة يعيدان نفس السلسلة، وده مفيد لإعادة اختبار محاكاة. كلمات السر والمفاتيح تحتاج مولدًا مصممًا للأمان؛ مجرد تغيير البذرة بالوقت لا يكفي.

تدريب محلول: الحد الرابع للمتتابعتين فوق هو9 و24. تقريب −2.3 لأسفل يعطي−3 ولأعلى يعطي−2. عند تعادل المسافة في round، مثل2.5، القاعدة تختلف بين اللغات والإعدادات؛ افحص القاعدة المطلوبة بدل افتراض نتيجة واحدة.

مجموعات وعدّ واحتمال بحساب فعلي

المجموعة عناصر مميزة بلا تكرار. لو A={1,2,3} للطلاب الناجحين في الاختبار الأول وB={3,4} للثاني، فالاتحاد A∪B={1,2,3,4} للناجحين في واحد على الأقل، والتقاطع A∩B={3} للناجحين في الاثنين، والفرق A−B={1,2} للناجحين في الأول فقط. عدد الاتحاد 3+2−1=4؛ طرحنا التقاطع لأننا حسبناه مرتين.

اختيار قائد ونائب من 4 أشخاص بلا تكرار يعطي 4×3=12 لأن الأدوار مرتبة. اختيار عضوين بلا أدوار يعطي 12÷2=6 لأن كل زوج اتعد مرتين. مبدأ الضرب يتتبع عدد الاختيارات في كل خطوة، ولا يشترط استقلالًا احتماليًا.

في نرد عادل احتمال الزوجي 3/6=1/2. لو عرفت أن النتيجة أكبر من 3، النتائج الممكنة تصبح {4,5,6}، واحتمال الزوجي 2/3. قسمة النتائج الملائمة على الكل تفترض تساوي احتمالات النتائج.

تدريب محلول: كيس فيه كرتان حمراوان وكرة زرقاء. سحب كرتين بلا إرجاع يعطي احتمال حمراوين (2/3)×(1/2)=1/3؛ بعد الحمراء الأولى يتبقى كرتان منهما واحدة حمراء. مع الإرجاع النتيجة (2/3)×(2/3)=4/9. دي حالة توضح ليه لازم نحدد افتراضات الاحتمال.

ارسم العلاقات وابحث عن طريق

ارسم العلاقات وابحث عن طريق

Graph — رسم علاقات فيه نقاط تمثل أشياء وخطوط تمثل روابط. هنا كل خط اتصال في الاتجاهين وتكلفته خطوة واحدة. F منفصلة:

الخطوط هي A–B وA–C وB–D وC–E وD–E. **Queue — طابور** يخرج أول عنصر دخل. للبحث الأقرب أولًا نضع A في الطابور، ونعلّم كل نقطة وقت إضافتها حتى لا تتكرر.

النقطة التي خرجت	الطابور بعد إضافة الجيران الجدد
البداية	A
A	B, C
B	C, D
C	D, E
D	E
E	فارغ

وصلنا E عن طريق C: المسار A→C→E خطوتان. **جرّب:** ابدأ من B؛ أقصر طريق لـE هو B→D→E. لا يوجد طريق إلى F. بعد المثال نسمّي الأسلوب BFS ونحسب عمله.

Graphs وTrees

يتكون Graph من Vertices وEdges. قد تكون الحافة موجهة أو غير موجهة، وموزونة أو بلا وزن. الـPath سلسلة حواف، والـCycle يعود إلى نقطة سابقة. والـTree رسم متصل بلا دورات وله مسار وحيد بين أي عقدتين. تستخدم Graphs في الطرق والشبكات والتبعيات والعلاقات الاجتماعية.

تتبع Graph بالطابور

العقدة Vertex تمثل مكانًا، والحافة Edge تربط مكانين. خد رسمًا غير موجه وكل حافة تكلف خطوة: A متصلة بـB وC، وB بـD، وC بـE، وD بـE؛ والعقدة F منفصلة.

البحث بالعرض BFS (Breadth-First Search؛ بحث يزور العقد مستوى بعد مستوى) يزور مستوى بعد مستوى. الطابور Queue يخرج أقدم عنصر دخل أولًا. ابدأ بـ[A]، أخرج A وأضف B وC، ثم أخرج B وأضف D، ثم أخرج C وأضف E. أقل مسار من A إلى E هو A→C→E بطول حافتين. سجل العقدة كمكتشفة وقت إدخالها للطابور حتى لا تضيفها ثانية بسبب دورة. سجل parent[E]=C وparent[C]=A لاستعادة المسار بالعكس.

F لا يمكن الوصول إليها. BFS يقلل عدد الحواف، مش تكلفة أوزان عامة. مع V عقد وE حواف وقائمة جيران، الوقت O(V+E) والذاكرة الإضافية O(V). الرسم مش شجرة: فيه دورة A→B→D→E→C→A، وكمان غير متصل بسبب F. الشجرة غير الموجهة البسيطة متصلة وبلا دورات، ولها V−1 حافة.

في الرسم الذي تتبعناه، V عدد العقد وE عدد الحواف. كتابة **O(V+E)** تعني إن عدد الخطوات ينمو مع مجموعهما، مع افتراض تخزين جيران كل عقدة في قائمة. **O(V)** للذاكرة يعني مساحة تتناسب مع عدد العقد. دي طريقة لوصف النمو، مش مدة بالثواني. راجع مقارنة الحلول لتفاصيل القياس.

02

المسار · 14 درسًا

لغة C++

شرح متدرج، أمثلة عملية، وتصحيحات تساعدك على الانتقال من الفكرة إلى التطبيق.

C++

دروس مرتبة تبدأ من التفكير وفهم الكمبيوتر، ثم بناء برنامج C++ وترجمته، ثم البيانات والقرارات والحلقات والمشروعات.

يبدأ هذا القسم بطريقة تحليل المشكلة وتمثيل الأعداد داخل الكمبيوتر، ثم يشرح أدوات C++ وبنية البرنامج ومراحل الـCompiler بالتفصيل. بعد ذلك تنتقل الدروس إلى الأنواع والمتغيرات والترميز والمعاملات والإدخال، ثم الشروط و `switch` والحلقات. تتضمن الدروس أمثلة كاملة ومشروعات ومسائل تدريبية وأخطاء شائعة وأسئلة بإجابات مشروحة.

خريطة الدروس

1. **حل المشكلات والمخططات والتصحيح** — Decision Trees وFlowcharts وPseudocode والاختبارات وتتبع الأخطاء المنطقية.
2. **أنظمة الأعداد والكمبيوتر والذاكرة** — Binary وOctal وHex ومكوّنات الكمبيوتر والـCPU والذاكرة ونظام التشغيل.
3. **مقدمة C++ والأدوات وأول برنامج** — خصائص اللغة والمعايير وIDE والمحرر وبنية البرنامج و `cout` .
4. **الـCompiler ومراحل البناء والتشخيص** — Preprocessor وTokens وAST وIR والتحسين وAssembly وObject files والربط والأخطاء.
5. **الأنواع والمتغيرات والنطاق والذاكرة** — الأنواع الأساسية والأحجام والمدى والتهيئة والثوابت وScope وLifetime.
6. **ترميز النصوص والمحارف والتحويلات** — ASCII وUnicode وUTF-8 و `char` والتحويلات الضمنية والصريحة.
7. **المعاملات والتعبيرات والعمليات البتية** — الأولوية والإسناد والمقارنة والمنطق وShort Circuit والعمليات على البتات.
8. **الإدخال والتحويلات والحدود والرياضيات** — `cin` وفشل الإدخال وCasting و `numeric_limits` و `cmath` والصيغ.
9. **if وelse والتحقق والتداخل** — الفروع الحصرية والمستقلة وترتيب الشروط والنطاقات والشروط المتداخلة.
10. **switch وتدفق التحكم والحاسبة** — `case` و `break` و `default` وFall-through والنطاق المحلي.
11. **الحلقات وأنماط التكرار** — `while` و `for` و `do-while` والعدادات والمجمّعات وSentinel والحلقات المتداخلة.
12. **المدخلات وقواعد العمل** — تحويل المتطلبات إلى متغيرات وقواعد Boolean وحالات اختبار. مادة **التدريب على المسائل** موجودة دلوقتي داخل درس حل المشكلات، ومسائل الشروط والصيغ موجودة داخل درس `if` و `else` . الصفحتان القديمتان محفوظتان في المشروع كأرشيف فقط، لكنهما مش جزء من ترتيب المذاكرة.

طريقة المذاكرة

نفّذ كل مثال ثم غيّر المدخلات وتوقع الناتج قبل التشغيل. في درس الـCompiler ارسم مراحل البناء بيدك، وفي دروس التحكم اكتب Test Cases قبل كتابة الشرط أو الحلقة.

13. الدوال والحاويات والمراجع
14. الكائنات وSTL وRAII والاختبارات

1. حل المشكلات والمخططات والتصحيح

قبل كتابة C++ ثبّت منطق الحل: حدّد المدخلات والنتيجة، اكتب الخطوات، مثّل القرارات، ثم اختبر الخوارزمية يدويًا. اللغة تنفذ التفكير ولا تستبدله.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Loop:** حلقة تكرار تعيد تنفيذ مجموعة تعليمات وفق شرط.

تحليل المشكلة قبل كتابة الكود

قبل كتابة C++ ثبّت منطق الحل: حدّد المدخلات والنتيجة، اكتب الخطوات، مثّل القرارات، ثم اختبر الخوارزمية يدويًا. اللغة تنفذ التفكير ولا تستبدله.

Pseudocode وFlowchart وجدول التتبّع

الـ `Pseudocode` وصف منظم للحل بلغة قريبة من الإنسان. نستخدم فيه كلمات مثل `READ` و `IF` و `REPEAT` من غير التقيد بقواعد C++. فائدته أن منطق الحل يصبح قابلًا للمراجعة قبل أن تدخل تفاصيل الأقواس والأنواع والمكتبات.

الـ `Flowchart` يمثل الخطوات بأشكال وأسهم. المستطيل يعني عملية، والمعيّن يعني قرارًا له مساران أو أكثر، ومتوازي الأضلاع يمثل إدخالًا أو إخراجًا. يجب أن يقود كل سهم إلى خطوة محددة وألا يوجد مسار ينتهي بلا نتيجة واضحة.

جدول التتبّع يسجل قيم المتغيرات بعد كل خطوة. إذا كان البرنامج يحسب مجموعًا، فاكتب قيمة العداد والمجموع بعد كل دورة. ستعرف عندها هل الخطأ بدأ من التهيئة أم من الشرط أم من التحديث.

ابدأ بحالة عادية صحيحة، ثم اختبر القيم عند الحدود والمدخلات غير الصالحة. الخطأ النحوي يمنع المترجم من بناء البرنامج، وخطأ وقت التشغيل يظهر أثناء التنفيذ، أما الخطأ المنطقي فينتج قيمة خاطئة رغم أن البرنامج يعمل.

مثال: القسمة مع فحص المقام

```
READ a, b
IF b = 0 THEN
  PRINT "Division is undefined"
ELSE
  PRINT a / b
END IF
```

أخطاء شائعة وتصحيحات

- لا تختبر بالقيم السهلة فقط؛ جرّب الصفر والحدود والقيم السالبة.
- لا تحول الرسم إلى زينة؛ يجب أن يطابق مسارات التنفيذ فعلًا.

أشجار القرار وقواعد العمل

شجرة القرار تعرض سؤالًا عند كل عقدة، وفرعًا لكل إجابة ممكنة، ونتيجة نهائية عند كل Leaf. تفيد عندما تعتمد النتيجة على شروط مترابطة أو متداخلة، وتكشف المسارات الناقصة والقواعد المتعارضة قبل كتابة `if`.

في مثال الموافقة على قرض يمكن أن يبدأ القرار بالعمر، ثم الدخل، ثم الضامن أو Credit score. لا يكفي رسم المسار الناجح؛ يجب أن ينتهي كل مسار إلى موافقة أو رفض أو مراجعة يدوية. حوّل الشجرة بعد ذلك إلى شروط مرتبة، مع اختبار حالة الحد مثل عمر 21 بالضبط.

Pseudocode وFlowchart

يشرح Flowchart سير العمل بصريًا، بينما يصف Pseudocode الخطوات بنص منظم مستقل عن اللغة. استخدم الشكل البيضاوي للبداية والنهاية، والمستطيل للعملية، والمعين للقرار، ومتوازي الأضلاع للإدخال والإخراج، والأسهم لاتجاه التنفيذ. Connector يربط أجزاء الرسم دون خطوط متقاطعة طويلة.

```
INPUT number
IF number > 0
    OUTPUT "Positive"
ELSE IF number < 0
    OUTPUT "Negative"
ELSE
    OUTPUT "Zero"
END IF
```

عند رسم حاسبة بسيطة، تتفرع العملية حسب العامل، ثم يفحص فرع القسمة المقام قبل الحساب. تعود الفروع الناجحة إلى نقطة إخراج واحدة، بينما يسلك العامل غير المدعوم مسار خطأ واضحًا.

Test cases قبل الكود

أنشئ حالات عادية، وحدّية، وغير صالحة. قاعدة خصم تبدأ من 500 تحتاج 499 و500 و501. إذا كان الشرط مكتوبًا `subtotal > 500` فستكشف حالة 500 الخطأ فورًا. ويساعد جدول القرار عندما توجد عدة شروط، لأنه يسجل كل Combination والنتيجة المقابلة.

تتبع الخطأ المنطقي

الـLogic error يسمح للبرنامج بالعمل لكنه ينتج قيمة خاطئة أو لا ينتهي. ابحث عن أول حالة تختلف فيها القيمة الفعلية عن المتوقعة:

1. أعد الخطأ بمدخل ثابت.
2. تتبع المسار خطوة بخطوة.
3. سجل قيم المتغيرات بعد كل خطوة أو Iteration.
4. أصلح أول سبب، لا النتائج التابعة له.
5. أعد الحالة الفاشلة والحالات المجاورة.

في مجموع Running total إذا كان المتوقع بعد المدخلين `5` و`3` هو `8` لكن الفعلي `10`، فابدأ بالـIteration الثانية. الخطأ في الـIteration الثالثة قد يكون مجرد أثر للقيمة السابقة.

تعريف المشكلة والمدخلات والمخرجات والقيود

المشكلة البرمجية عقد يحدد ما يدخل إلى البرنامج وما يجب أن يخرج منه. اكتب المدخلات بأسمائها وأنواعها ووحداتها. `price` وحده غامض؛ هل هو بالجنيه أم بالقرش؟ وهل يقبل كسرًا؟ وهل يمكن أن يكون صفرًا؟ واكتب المخرجات بنفس الدقة: هل المطلوب المجموع فقط أم فاتورة تفصيلية ورسالة خطأ عند فشل الإدخال؟

القيود تحدد الحل الممكن. إذا كان عدد العناصر عشرة، فقد يعمل حل يقارن كل عنصر بالآخر. إذا وصل العدد إلى مليون، قد يصبح الحل نفسه بطيئًا جدًا. وتشمل القيود قواعد المجال، مثل أن العمر بين 12 و80، والكمية موجبة، والمقام لا يساوي صفرًا.

اكتب الافتراضات التي لم يحددها السؤال. إذا كان السعر سالبًا، هل ترفضه أم تعتبره مرتجعًا؟ إذا تساوى متسابقان، هل توجد نتيجة تعادل؟ تحويل الغموض إلى قرار مكتوب يمنع ظهور سلوك مختلف في كل جزء من البرنامج.

الخوارزمية وصحة الخطوات

الخوارزمية سلسلة خطوات محدودة وغير غامضة تحول المدخلات إلى المخرجات. «محدودة» تعني أن التنفيذ يصل إلى نهاية، ولذلك يجب أن تتغير حالة أي تكرار باتجاه التوقف. و«غير غامضة» تعني أن كل خطوة قابلة للتنفيذ، فلا نكتب «احسب الحل المناسب» من غير تعريف طريقة الحساب.

نختبر صحة الخوارزمية بسؤالين: هل تعطي النتيجة الصحيحة لكل مدخل صالح؟ وهل تتوقف؟ تتبع مثال يدوي مفيد، لكنه لا يثبت الصحة وحده. قسّم مجال المدخلات إلى فئات، واختر حالة من كل فئة، ثم اختبر الحدود التي تتغير عندها القرارات.

رموز الـFlowchart

الشكل	معناه	مثال
بيضاوي	بداية أو نهاية	Start، End
متوازي أضلاع	إدخال أو إخراج	Read age، Print total
مستطيل	عملية حسابية أو إسناد	`total = price * quantity`
معيّن	قرار	`quantity > 0?`
سهم	اتجاه التنفيذ	الانتقال من الفحص إلى الحساب

يجب أن يحمل كل فرع خارج من المعيّن معنى مثل نعم/لا، وأن يعود مسار التكرار إلى القرار الصحيح. إذا ازدحم الرسم، قسّم العمل إلى خوارزميات فرعية بدل تقاطع الأسهم بلا دلالة.

حالات الاختبار المتكافئة والحدود

لا تحتاج إلى اختبار كل رقم ممكن. قسّم القيم إلى مجموعات تتعامل معها الخوارزمية بالطريقة نفسها. لشرط `score >= 60` توجد فئة أقل من 60 وفئة من 60 فأكثر، بالإضافة إلى القيم غير الصالحة خارج `0..100`. اختبر `59` و`60` و`61` لكشف أخطاء الحد، ثم `-1` و`101` لاختبار التحقق.

اكتب لكل اختبار المدخلات والنتيجة المتوقعة وسبب اختيار الحالة. إذا ظهر خطأ، احتفظ بالاختبار بعد إصلاحه حتى لا يعود لاحقًا. يسمى ذلك Regression Test لأنه يحمي سلوكًا سبق أن تعطل.

طبّق طريقة حل المشكلة على منصات التدريب

ما البرمجة التنافسية؟

البرمجة التنافسية تدريب مركز على تحليل القيود واختيار الخوارزمية وكتابة حل صحيح سريع. المنصة أداة قياس وليست بديلًا عن الفهم.

قراءة المسألة والقيود والتعقيد

المسألة التنافسية تحدد صيغة الإدخال والنتيجة المطلوبة والقيود. اقرأ هذه الأجزاء قبل الأمثلة لأن المثال يوضح حالة واحدة، بينما القيود تحدد ما إذا كان الحل ممكنًا. إذا كان `n` يصل إلى `100000`، فإن مقارنة كل عنصر بكل عنصر قد تنفذ نحو عشرة مليارات عملية، ولذلك يكون `O(n²)` غير مناسب غالبًا.

حل مثالًا صغيرًا يدويًا واكتب العلاقة الرياضية أو حالة التكرار قبل الكود. اختر النوع من أكبر نتيجة وسيطة، لا من شكل العينة. بعد الانتهاء اكتب تعقيد الزمن والذاكرة حتى تتأكد أن الحل يناسب الحدود.

نتيجة `Wrong Answer` تعني أن البرنامج أنهى التنفيذ لكنه أعاد نتيجة مختلفة. راجع فهم النص والحدود ومواضع `1-based` و`0-based` ونوع البيانات والتقريب. `Runtime Error` يشير غالبًا إلى وصول غير صالح أو قسمة على صفر، و`Time Limit Exceeded` يعني أن عدد العمليات أكبر من المسموح. تغيير الأسطر عشوائيًا لا يشخص السبب.

مثال: مسألة Elephant والقسمة السقفية

```
// Elephant: minimum moves of length at most 5
int moves = (distance + 4) / 5; // integer ceiling
```

برنامج كامل قابل للتشغيل

```
#include <iostream>

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);

    int distance{};
    if (!(std::cin >> distance) || distance < 0) {
        return 1;
    }

    const int moves = (distance + 4) / 5;
    std::cout << moves << '\n';
}
```

إذا كان الإدخال `12` فالخروج `3`: خطوتان بطول 5 وخطوة أخيرة بطول 2. الزمن `O(1)` والذاكرة `O(1)`. لا يطبع البرنامج رسالة مثل `Enter` لأن الـOnline Judge يقارن الخرج حرفيًا. فحص الإدخال هنا مفيد عند تشغيل المثال محليًا، بينما تضمن المسألة الأصلية عادة أن `distance` الإدخال يطابق القيود.

أخطاء شائعة وتصحيحات

- قبول العينة لا يعني قبول كل الحالات.
- لا تبدأ بأصعب منصة؛ ابنِ عادة حل يومية تدريجية.

المنصات وخطة التدريب

توفر Codeforces مسابقات وتصنيفات ومجموعات مسائل، بينما تركز LeetCode كثيرًا على أنماط المقابلات وهياكل البيانات. ابدأ بمسائل تنفيذ وحساب وشروط، ثم الحلقات والحاويات، وسجل بعد كل مسألة: الفكرة، سبب الخطأ، التعقيد، وحالة اختبار لم تتوقعها.

أنماط المسائل المستخدمة في التدريب

المسألة	الفكرة الأساسية
617A Elephant	Ceiling division صحيحة بالأعداد الصحيحة
581A Vasya the Hipster	`min` ثم حساب الباقي
281A Word Capitalization	تعديل أول محرف مع فحص النص الفارغ
4A Watermelon	زوجية العدد مع استبعاد 2
1A Theatre Square	Ceiling لكل بعد واستخدام `long long`
959A Mahmoud and Ehab	Parity وشرط بسيط
486A Calculating Function	اشتقاق صيغة بدل Loop طويل
835A Key Races	حساب زمنين ثم مقارنة ثلاثية
1173A Nauuo and Votes	ضمان النتيجة رغم القيم المجهولة
318A Even Odds	تقسيم الترتيب إلى كتلة فردية وزوجية
459A Pashmak and Garden	حالات هندسية لمحاور ومربع

اقرأ الحل بعد محاولة حقيقية، ثم أغلقه وأعد كتابة الفكرة من الذاكرة على مثال مختلف. الهدف التعرف إلى النمط واشتقاقه، لا حفظ ترتيب الأسطر.

خريطة الدرس

دورة عمل الـOnline Judge

يرسل المتسابق Source Code، فتترجمه المنصة بإصدار وخيارات محددة ثم تشغله على حالات مخفية داخل حدود للوقت والذاكرة. تقارن المنصة الخرج المتوقع بالفعلي وفق Checker. أي نص إضافي مثل `Enter n:` قد يسبب Wrong Answer لأن المطلوب تنسيق دقيق.

تصنيف النتائج

`Compilation Error` يعني أن المصدر لم يُبنَ. `Wrong Answer` يعني أن التنفيذ انتهى لكن الخرج خاطئ. `Runtime Error` يشمل الانهيار والقسمة على صفر والوصول غير الصالح. `Time Limit Exceeded` يدل على زمن زائد، و `Memory Limit Exceeded` على استهلاك ذاكرة أكبر من الحد.

بعد القبول، لا تكتف بالكود. سجل الفكرة والتعقيد والحالة التي أخطأت فيها، ثم أعد الحل بعد أيام من الذاكرة. الهدف بناء نمط تفكير يمكن نقله إلى مسألة جديدة.

تأكد من فهمك

01 **لماذا يجب التحقق من المقام قبل القسمة لا بعدها؟**

الإجابة المشروحة: لأن العملية غير الصالحة تكون قد نُفذت بالفعل. التحقق المبكر يمنع الخطأ ويحدد مسارًا واضحًا للرفض.

02 **برنامج يعمل لكنه يعطي الأكبر خطأ عند تساوي قيمتين. ما نوع العيب؟**

الإجابة المشروحة: خطأ منطقي في المقارنات أو ترتيب الفروع؛ البناء والتشغيل لا يثبتان صحة الخوارزمية.

03 **متى يكون Flowchart أفضل من Pseudocode؟**

الإجابة المشروحة: عند الحاجة لرؤية التفرعات والعودة بصريًا؛ أما Pseudocode فأفضل لتفاصيل التسلسل القريبة من الكود.

04 **كيف تثبت أن خوارزمية أكبر ثلاثة أعداد تغطي كل الحالات؟**

الإجابة المشروحة: اختبر كل عنصر كأكبر، وحالات التساوي، والسالب، وحدود النوع، ثم قارن بتوقع مكتوب قبل التنفيذ.

اربط النقاط ببعض

بعد رسم الحل، اكتب loop invariant أو شرطًا يبقى صحيحًا عند كل خطوة، وحدد termination بدل الاكتفاء بأن الأمثلة تعمل. حوّل مثال القسمة إلى برنامج C++ كامل باختبارات للصفر والقيم السالبة والحدود، وقارن trace بالتوقع.

جرّب بنفسك

أدخل خطأ منطقيًا لا يمنع compilation وتأكد أن test case حدية تكشفه.

2. أنظمة الأعداد والكمبيوتر والذاكرة

تفهم أنظمة الأعداد كيف تُمثّل القيم، وتفهم مكونات الحاسوب أين يعيش البرنامج وكيف ينتقل من التخزين إلى التنفيذ.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Cache:** نسخة مؤقتة من البيانات هدفها تقليل وقت الانتظار والعمل المتكرر.

العدد وقاعدة تمثيله

تفهم أنظمة الأعداد كيف تُمثّل القيم، وتفهم مكونات الحاسوب أين يعيش البرنامج وكيف ينتقل من التخزين إلى التنفيذ.

العشري والثنائي والثماني والسداسي عشر

نظام العد يحدد الرموز المتاحة وقيمة كل موضع. النظام العشري أساسه `10` ويستخدم الأرقام من `0` إلى `9`. الثنائي أساسه `2` ويستخدم `0` و`1`. الثماني أساسه `8`، والسداسي عشر أساسه `16` ويضيف الرموز `A` إلى `F` للقيم من عشرة إلى خمس عشرة.

في أي نظام موضعي تساوي قيمة الخانة الرقم مضروبًا في قوة الأساس. العدد الثنائي `101101` يعني `1×2⁵ + 0×2⁴ + 1×2³ + 1×2² + 0×2¹ + 1×2⁰`. وكل رقم Hex يختصر أربع بتات بالضبط، لذلك تتحول المجموعة `1101` مباشرة إلى `D`.

المعالج `CPU` ينفذ التعليمات، و`RAM` تحتفظ بالكود والبيانات المستخدمة الآن، وذاكرة `Cache` الصغيرة والسريعة تقلل انتظار المعالج للبيانات المتكررة. التخزين مثل SSD يحتفظ بالملفات بعد إيقاف الجهاز. عند فتح برنامج، ينشئ نظام التشغيل `Process` ويحمّل الأجزاء المطلوبة إلى الذاكرة ثم يخصص له موارد التنفيذ.

مثال: كتابة القيمة نفسها بثلاث قواعد

```
45₁₀ = 32 + 8 + 4 + 1 = 101101₂ = 2D₁₆
```

أخطاء شائعة وتصحيحات

- الذاكرة RAM ليست التخزين الدائم.
- التحويل لا يغير القيمة؛ يغير طريقة كتابتها فقط.

خريطة الدرس

الأساس والقيمة الموضعية

يحدد Base عدد الرموز وقيمة كل خانة. يستخدم Decimal الرموز 0–9، وBinary الرمزين 0 و1، وOctal الرموز 0–7، وHexadecimal الرموز 0–9 وA–F. العدد 325_{10} يعني $3\times10^2 + 2\times10^1 + 5\times10^0$، بينما 1011_2 يعني $8 + 0 + 2 + 1 = 11_{10}$.

احفظ قوى 2 الأساسية من $2^0=1$ حتى $2^8=256$. للتحويل من Binary إلى Decimal اجمع قيم الخانات التي تحتوي 1. وللتحويل من Decimal إلى Binary استخدم القسمة المتكررة على 2 واقرأ البواقي من الأسفل، أو فكك العدد إلى مجموع قوى 2.

Hexadecimal والـNibble

كل رقم Hex يمثل أربع بتات، لذلك $1011\ 0110_2$ يساوي $B6_{16}$. للتحويل إلى Hex قسّم من اليمين إلى مجموعات من أربع بتات وأضف أصفارًا إلى اليسار عند الحاجة. وللعكس استبدل كل رقم Hex بنمط من أربع بتات مع الحفاظ على الأصفار داخل المجموعة.

يظهر Hex في عناوين الذاكرة، وDebuggers، وعرض Bytes الملفات، وألوان RGB، وBit masks. يظهر Octal في سياقات مثل صلاحيات Unix، لكنه أقل استخدامًا في التطبيقات العامة.

CPU وRAM والتخزين

يحمل التخزين ملفات البرنامج، ثم يحمّل نظام التشغيل الصفحات المطلوبة إلى RAM. يجلب CPU التعليمات والبيانات النشطة ويحللها وينفذها. Registers داخل المعالج هي الأسرع والأصغر، ثم Cache بمستوياتها، ثم RAM، ثم SSD/HDD الأكبر والأبطأ.

Cache لا تستبدل RAM، وRAM لا تستبدل التخزين. لكل مستوى هدف وعمر وسعة وتكلفة وصول مختلفة. انقطاع الكهرباء يمحو RAM عادة، بينما يحتفظ التخزين بالملفات.

بقية المكوّنات

- Motherboard تربط المعالج والذاكرة والتخزين وبطاقات التوسع.
- GPU يعالج الرسوم والعمليات المتوازية المناسبة.
- Power supply يحول الطاقة ويوفر الفولت المناسب.
- Network adapter يربط الجهاز بالشبكات.
- Cooling يبعد الحرارة حتى لا تخفض المكونات سرعتها أو تتضرر.
- أجهزة الإدخال ترسل بيانات، وأجهزة الإخراج تعرض النتائج.

نظام التشغيل وفتح البرنامج

يدير نظام التشغيل العمليات والذاكرة والملفات والأجهزة والمستخدمين والصلاحيات. عند فتح برنامج: يستقبل الإدخال، يجد الملف على التخزين، ينشئ Process، يحمّل أجزاءه إلى الذاكرة، يبدأ التنفيذ، ثم ترسل النتائج إلى الشاشة أو ملف أو شبكة. البرنامج المخزن ليس Process حتى يبدأ التشغيل.

التحويل بين القواعد خطوة بخطوة

للتحويل من قاعدة إلى العشري، اضرب كل رقم في قوة الأساس الخاصة بموضعه ثم اجمع. في `101101₂` تبدأ القوى من الصفر جهة اليمين، فتكون القيمة `1×2⁵ + 1×2³ + 1×2² + 1×2⁰ = 45`.

للتحويل من عدد عشري صحيح موجب إلى الثنائي، اقسم على اثنين وسجل الباقي، ثم كرر بالقيمة الناتجة. تُقرأ البواقي من الأخير إلى الأول. مثلًا: `13 ÷ 2 = 6` والباقي `1`، ثم `6 ÷ 2 = 3` والباقي `0`، ثم `3 ÷ 2 = 1` والباقي `1`، وأخيرًا `1 ÷ 2 = 0` والباقي `1`، فتكون النتيجة `1101₂`.

للتحويل بين Binary وHex قسّم البتات إلى مجموعات من أربع من اليمين. المجموعة `1010` هي `A`، و`1111` هي `F`. أما Octal فيجمع كل ثلاث بتات. هذه التحويلات لا تغير القيمة الرياضية، بل تغير طريقة كتابتها.

البت والبايت والكلمة

الـBit أصغر وحدة ثنائية وله حالتان `0` أو `1`. يجمع الـByte عادة ثمانية بتات، ولذلك يملك 256 نمطًا مختلفًا. لا يعني ذلك دائمًا القيم من صفر إلى 255؛ معنى النمط يعتمد على النوع الذي يفسره. النمط نفسه قد يمثل عددًا أو جزءًا من محرف أو تعليمة آلة.

كلمة Word هي حجم طبيعي يتعامل معه المعالج، وغالبًا ترتبط بعرض السجلات والمنصة مثل 32 أو 64 بت. لا تخلط بينها وبين Byte، ولا تفترض أن كل نوع في C++ يساوي حجم كلمة كاملة.

الأعداد السالبة وTwo’s Complement

تمثل أغلب الأجهزة الأعداد الصحيحة السالبة بطريقة Two’s Complement. في تمثيل من ثمانية بتات، تتراوح القيمة signed عادة من `-128` إلى `127`. للحصول على تمثيل `-5` تعكس بتات `5` ثم تضيف واحدًا. تسمح هذه الطريقة لدوائر الجمع نفسها بالتعامل مع الموجب والسالب.

يمثل `unsigned` المجال غير السالب، وفي ثمانية بتات يكون من `0` إلى `255`. لكن استخدام unsigned لا يجعل البيانات أكثر أمانًا. طرح قيمة أكبر من أصغر يلتف ضمن المجال، وقد ينتج رقمًا كبيرًا بدل قيمة سالبة متوقعة.

هرم الذاكرة

كلما اقترب التخزين من المعالج أصبح أسرع وأصغر وأغلى. السجلات داخل CPU هي الأسرع، ثم مستويات Cache، ثم RAM، ثم SSD أو القرص. ينقل النظام البيانات في كتل لأن الوصول المتجاور أكثر كفاءة غالبًا من القفز بين أماكن متباعدة.

RAM متطايرة فتفقد محتواها عند انقطاع الطاقة، بينما التخزين الدائم يحتفظ بالملفات. الـCache لا تحل محل RAM؛ إنها تحتفظ بنسخ صغيرة من البيانات المتوقع استخدامها قريبًا. فهم هذا الفرق يفسر لماذا يؤثر ترتيب البيانات وطريقة المرور عليها في الأداء.

CPU والتعليمات والعملية

يجلب المعالج التعليمة من الذاكرة، ويفك ترميزها، ثم ينفذها. تستخدم التعليمات سجلات ووحدات حساب ومنطق، وقد تقرأ من الذاكرة أو تكتب إليها. هذه دورة مبسطة، لأن المعالجات الحديثة تستخدم Pipeline وتنفيذًا متوازيًا وتوقع الفروع.

البرنامج هو ملف وتعليمات مخزنة، أما Process فهي نسخة تعمل الآن ولها ذاكرة وحالة وموارد. يمكن تشغيل البرنامج نفسه مرتين فتوجد عمليتان مستقلتان. وينظم نظام التشغيل وقت CPU والذاكرة والملفات والأجهزة بين العمليات.

تأكد من فهمك

01 **حوّل 0x2D إلى عشري وثنائي.**

الإجابة المشروحة: 2D = 2×16+13 = 45، وكل رقم Hex أربع بتات: 2=0010 وD=1101، إذن 00101101.

02 **لماذا تستخدم Hex في عناوين الذاكرة والألوان؟**

الإجابة المشروحة: تمثيل أقصر من الثنائي مع تحويل مباشر لكل أربع بتات، لذلك يجمع الدقة وسهولة القراءة.

03 **ماذا يحدث تقريبًا عند فتح برنامج من SSD؟**

الإجابة المشروحة: يحمّل نظام التشغيل الكود والبيانات المطلوبة إلى RAM، ينشئ Process، ثم يجلب CPU التعليمات مع الاستفادة من Cache.

04 **هل زيادة RAM تجعل CPU أسرع؟**

الإجابة المشروحة: لا ترفع سرعة المعالج نفسها، لكنها تقلل اللجوء للتخزين البطيء عند ضغط الذاكرة وقد تحسن أداء النظام.

اربط النقاط ببعض

Endianness يحدد ترتيب bytes متعددة داخل الذاكرة ولا يغير ترتيب البتات المكتوبة داخل byte. IEEE-754 يقسم floating value إلى sign وexponent وfraction مع قيم خاصة. العنوان pointer إلى موقع افتراضي لا «رقم الخانة الفيزيائية».

جرّب بنفسك

افحص bytes لعدد صحيح وfloat على جهازك وفسرها دون افتراض portability.

3. مقدمة C++ والأدوات وأول برنامج

C++ لغة مترجمة تمنح تحكمًا عاليًا بالأداء والذاكرة. تعلّم بنية البرنامج وخط البناء أهم من ربط التعلم ببيئة تطوير واحدة.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Compiler:** المترجم: برنامج يحوّل كود C++ إلى ملف يقدر الكمبيوتر يشغّله.
- **Runtime:** وقت التشغيل: الفترة اللي البرنامج بيكون شغال فيها فعلًا.
- **Debugger:** أداة تتبّع الأخطاء: بتوقف البرنامج خطوة خطوة عشان تشوف القيم ومسار التنفيذ.
- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.

ما لغة C++؟

C++ لغة مترجمة تمنح تحكمًا عاليًا بالأداء والذاكرة. تعلّم بنية البرنامج وخط البناء أهم من ربط التعلم ببيئة تطوير واحدة.

المحرر والـCompiler والـIDE

المحرر يغيّر الملفات النصية فقط. الـCompiler يقرأ كود C++ ويفحص قواعده وأنواعه ثم ينتج كودًا أقرب إلى الآلة. الـIDE يجمع المحرر وأوامر البناء والمصحح `Debugger` وإدارة المشروع في واجهة واحدة، لكنه لا يستبدل المترجم.

أصغر وحدة لها معنى في المصدر تسمى `Token`، مثل اسم متغير أو رقم أو عامل `+`. مجموعة Tokens يمكن أن تكوّن `Expression` ينتج قيمة، مثل `price * quantity`. وعندما يصبح التعبير جزءًا من أمر كامل ينتهي غالبًا بفاصلة منقوطة نحصل على `Statement`.

يبدأ البرنامج المستقل من الدالة `main`. إرجاع `0` يخبر نظام التشغيل تقليديًا أن التنفيذ نجح. الرأس `<iostream>` يصرّح بأدوات الإدخال والإخراج، و`std` هي مساحة الأسماء التي تضع فيها المكتبة القياسية أسماء مثل `cout`.

يمر المصدر عادة بالـPreprocessor، ثم التحليل والترجمة، ثم توليد ملف Object، وأخيرًا يربط الـLinker الملفات والمكتبات ليصنع الملف التنفيذي. درس الـCompiler التالي يشرح كل مرحلة بالتفصيل.

أول برنامج C++

```
#include <iostream>

int main() {
    std::cout << "Hello, C++!\n";
    return 0;
}
```

أخطاء شائعة وتصحيحات

- فعّل التحذيرات واقرأ أول رسالة مرتبطة بكودك.
- لا تضع using namespace std في ملفات header أو المشاريع الكبيرة بلا حاجة.

الأدوات والمعيار

المحرر يغيّر النص، والـIDE يضيف إدارة المشروع والبناء وDebugger، لكن المترجم الحقيقي يكون MSVC أو GCC أو Clang. اختر معيارًا واضحًا للمشروع مثل C++20 أو C++23، ولا تفترض أن كل ميزة جديدة مدعومة في كل Toolchain.

مراحل البناء بالتفصيل

1. يعالج Preprocessor أوامر `#include` و`#define` والـConditional compilation.
2. يحلل Compiler الـTokens والصياغة والأنواع ويبني تمثيلًا وسيطًا ثم يولد كودًا.
3. يحول Assembler التعليمات إلى Object file يحتوي Machine code ورموزًا غير محسومة.
4. يجمع Linker ملفات Object والمكتبات، ويبلغ عن التعريفات المفقودة أو المكررة.

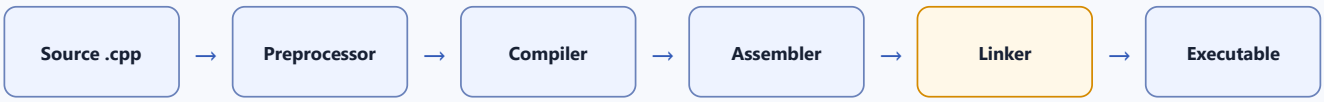
يمكن ترجمة ملفات `.cpp` بصورة منفصلة ثم ربطها. لذلك قد ينجح Compile لملف ويظهر الخطأ لاحقًا في Link إذا استدعيت دالة صرحت بها ولم توفر تعريفها.

تشخيص الأخطاء

- **Syntax/compile error:** صياغة أو نوع غير صالح، مثل فاصلة منقوطة مفقودة.
- **Link error:** تصريح موجود لكن التعريف مفقود أو مكرر.
- **Runtime error:** بدأ البرنامج ثم فشل أثناء التنفيذ.
- **Logic error:** انتهى البرنامج دون انهيار لكن الناتج خاطئ.

اقرأ أول خطأ حقيقي، وافصل رسالة المترجم عن الأسطر التابعة لها. أنشئ أصغر مثال يعيد المشكلة، وأصلح سببًا واحدًا ثم أعد البناء.

مراحل تحويل كود C++ إلى ملف تنفيذي



لماذا C++؟

C++ لغة عالية المستوى تمنح وصولًا مباشرًا نسبيًا إلى الذاكرة والموارد. تستخدم في المحركات والألعاب والمتصفحات والأنظمة المضمنة والبرامج التي تحتاج أداءً وتحكمًا. هذا التحكم يجعل أخطاء العمر والحدود والتحويلات أخطر، لذلك تعلمها يبني انضباطًا في الأنواع والملكية.

تطورت اللغة من C++98 إلى تحديث C++11 الكبير، ثم C++14 وC++17 وC++20 وC++23. يحدد خيار `-std=` أو إعداد المشروع المعيار المستخدم. المعيار يصف اللغة والمكتبة، أما Compiler فيطبق هذا الوصف بدرجات دعم مختلفة.

IDE والمحرر والـToolchain

المحرر يغير الملفات، والـIDE يضيف Projects وDebugger وتكامل البناء. Visual Studio يوفر MSVC عند تثبيت Desktop development with C++، وVS Code يحتاج Compiler وإضافات منفصلة، وCode::Blocks قد يأتي مع Toolchain في بعض الحزم. اختيار الأداة لا يغير قواعد اللغة.

في Visual Studio أنشئ Console App، وافتح ملف `.cpp`، واستخدم `Ctrl+F5` للتشغيل دون Debugger و`F5` معه. على الطرفية يمكن بناء الملف بـ `g++ main.cpp -o app` ثم تشغيل الناتج.

Tokens وExpressions وStatements

`price + tax` تعبير ينتج قيمة. Statement `total = price + tax;` تنفذ إسنادًا وتنتهي بفاصلة منقوطة. تتكون الصياغة من Tokens مثل الكلمات المحجوزة والأسماء والـLiterals والمعاملات وعلامات الترقيم.

تشريح أول برنامج

```
#include <iostream>

int main()
{
    std::cout << "Hello, world!\n";
    return 0;
}
```

يجعل Header التصريحات متاحة، و`main` نقطة دخول البرنامج، وStandard output stream `std::cout`، و`<<` يرسل البيانات إلى Stream، و`return 0` يبلغ عن إنهاء ناجح. يسمح المعيار بالوصول ضمنيًا إلى نهاية `main` كإرجاع صفر، لكن كتابة القيمة توضح الفكرة للمبتدئ.

النصوص وEscape sequences

داخل String literal تعني `\n` سطرًا جديدًا وTab `\t` و`\"` علامة اقتباس وBackslash `\\`. يختلف Character literal مثل `'A'` عن String literal مثل `"A"`. لا تستخدم `std::endl` لمجرد سطر جديد لأنه يضيف Flush غالبًا؛ استخدم `\n` ثم Flush عندما تحتاجه فعلًا.

Namespaces والمكتبة القياسية

تمنع Namespace تعارض الأسماء. ينتمي `cout` و`cin` و`string` إلى `std`. كتابة `std::cout` توضح الاسم الكامل، بينما `using namespace std;` يدخل أسماء كثيرة وقد يصنع تعارضات، خصوصًا داخل Headers.

```
namespace school { int students{120}; }
namespace company { int students{45}; }

std::cout << school::students << ' ' << company::students;
```

ملفات المصدر وHeader Files

يحمل ملف `.cpp` تعريفات قابلة للترجمة. ويحتوي Header عادة على تصريحات الأنواع والدوال التي تحتاجها أكثر من Translation Unit. الـDeclaration يخبر المترجم باسم الدالة ونوع مدخلاتها ونتيجتها، أما الـDefinition فيحتوي الجسم الذي سينفذ.

إذا وضعت تعريف دالة عادية في Header وضمّنته في أكثر من ملف فقد تحصل على Multiple Definition عند الربط. تستخدم Include Guards أو `#pragma once` لمنع معالجة Header نفسه أكثر من مرة داخل Translation Unit واحدة، لكنها لا تلغي قاعدة التعريف الواحد.

calculator.hpp

```
#pragma once
int add(int left, int right);

// calculator.cpp
#include "calculator.hpp"
int add(int left, int right) {
    return left + right;
}
```

أوامر البناء والتحذيرات

```
g++ -std=c++20 -Wall -Wextra -Wpedantic main.cpp -o app
```

يحدد `-std=c++20` معيار اللغة. وتفعل الخيارات `-Wall -Wextra -Wpedantic` تحذيرات تكشف قيمًا غير مهيأة وتحويلات وشروطًا مشكوكًا فيها. يحدد `-o app` اسم الملف التنفيذي. التحذير لا يوقف البناء دائمًا، لكنه يستحق الفهم والإصلاح بدل إخفائه.

يمكن فصل الترجمة عن الربط:

```
g++ -std=c++20 -Wall -Wextra -c calculator.cpp -o calculator.o
g++ -std=c++20 -Wall -Wextra -c main.cpp -o main.o
g++ main.o calculator.o -o app
```

ينتج الخيار `-c` ملف Object ولا يشغّل الـLinker. تسمح هذه الطريقة بإعادة ترجمة الملف المتغير فقط ثم ربط الملفات من جديد.

main وقيمة الخروج ووسائط التشغيل

يمكن أن تستقبل `main` وسائط من سطر الأوامر. يمثل `argc` عدد العناصر، ويحتوي `argv[0]` عادة على اسم البرنامج. تعني قيمة الخروج صفر النجاح تقليديًا، بينما تعبر قيمة أخرى عن فشل يستطيع Shell أو Script اكتشافه.

Standard Output وStandard Error والـBuffer

يرسل `std::cout` النتائج العادية إلى Standard Output، بينما يناسب `std::cerr` رسائل الخطأ. قد يجمع الـStream النص في Buffer قبل إرساله لتحسين الأداء. يضيف `\n` سطرًا جديدًا، أما `std::endl` فيضيف السطر ويجبر الـStream على Flush، لذلك لا تستخدمه بعد كل سطر من غير حاجة.

تأكد من فهمك

01 **ما الفرق بين IDE وCompiler؟**

الإجابة المشروحة: الـIDE واجهة تجمع التحرير والبناء والتصحيح، بينما Compiler أداة تحول المصدر وتفحصه؛ يمكن استخدام أحدهما دون الآخر.

02 **لماذا قد تنجح Compilation ثم يفشل Linking؟**

الإجابة المشروحة: كل ملف قد يكون صحيحًا منفردًا، لكن Linker لا يجد تعريف رمز مستخدم أو يجد تعريفات متكررة.

03 **ما وظيفة main وما معنى return 0؟**

الإجابة المشروحة: main نقطة دخول البرنامج المستضاف، و0 إشارة تقليدية لنظام التشغيل بأن التنفيذ انتهى بنجاح.

04 **كيف تتعامل مع عشر رسائل مترجم ناتجة من خطأ واحد؟**

الإجابة المشروحة: ابدأ بأول رسالة مرتبطة بمصدرٍ، أصلحها وأعد البناء؛ الأخطاء التالية قد تكون آثارًا متسلسلة.

اربط النقاط ببعض

استخدم debugger لوضع breakpoint وفحص stack بدل إضافة prints فقط. فعّل AddressSanitizer وUndefinedBehaviorSanitizer في build تطويري، واجعل CMake أو build script يحفظ flags نفسها لكل الملفات. مدير الحزم يحل الاعتماد لكنه لا يعفيك من تثبيت النسخ ومراجعة المصدر.

جرّب بنفسك

ابنِ البرنامج بتحذيرات وsanitizers ثم شخّص out-of-bounds متعمدًا.

4. الـCompiler ومراحل البناء والتشخيص

رحلة برنامج C++ من Source Code إلى Translation Unit وTokens وAST وIR وAssembly وObject Files ثم الربط والتحميل، مع تفسير الأخطاء والتحذيرات.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Compiler:** المترجم: برنامج يحوّل كود C++ إلى ملف يقدر الكمبيوتر يشغّله.
- **Runtime:** وقت التشغيل: الفترة اللي البرنامج بيكون شغال فيها فعلًا.
- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.
- **Scope:** النطاق: المكان اللي يقدر الكود داخله يشوف اسمًا أو متغيرًا.
- **Loop:** حلقة تكرار تعيد تنفيذ مجموعة تعليمات وفق شرط.

لماذا يحتاج كود C++ إلى ترجمة؟

المعالج ينفذ تعليمات آلة خاصة بمعماريته، بينما نكتب C++ بصياغة مفهومة للبشر. يحول Toolchain البرنامج إلى ملف مناسب لنظام التشغيل والمعالج المستهدف. لا يقوم برنامج واحد بكل العمل بالضرورة؛ كلمة «Compiler» تستخدم أحيانًا للواجهة التي تنسق مجموعة مراحل وأدوات.

```
Source files
  ↓ Preprocessor
Translation units
  ↓ Compiler frontend + optimizer + backend
Assembly or machine code
  ↓ Assembler
Object files
  ↓ Linker + libraries
Executable
  ↓ Operating-system loader
Running process
```

قد يدمج Toolchain بعض المراحل، وقد لا يحفظ ملف Assembly على القرص، لكن النموذج يفسر نوع كل خطأ ومكانه.

Compiler وInterpreter وJIT

النموذج	وقت الترجمة	الناتج المعتاد
Ahead-of-time compilation	قبل التشغيل	Object files وملف تنفيذي للمنصة
Interpretation	أثناء التشغيل	ينفذ Runtime المصدر أو تمثيلًا وسيطًا
Bytecode + VM + JIT	قبل التشغيل وأثناءه	Bytecode ثم كود آلة للمسارات الساخنة

C++ تستخدم عادة ترجمة مسبقة إلى Native code. Java وC# تجمعان بين Intermediate code وVirtual machine، ومحركات JavaScript قد تفسر ثم تستخدم JIT. لذلك تقسيم اللغات إلى «مترجمة» و«مفسرة» فقط تبسيط، لأن التنفيذ الفعلي تحدده المنظومة.

أشهر Toolchains

- **GCC** ويستخدم الأمر `g++` عادة لربط مكتبة C++ تلقائيًا.
- **Clang** بواجهة `clang++` وبنية LLVM.
- **MSVC** ويستخدم `cl` على ويندوز.

المترجم والمكتبة القياسية ونظام التشغيل والمعمارية وإعدادات البناء تؤثر جميعًا في الميزات والـABI وشكل الملف التنفيذي والرسائل.

```
g++ -std=c++20 -Wall -Wextra -Wpedantic main.cpp -o app
clang++ -std=c++20 -Wall -Wextra -Wpedantic main.cpp -o app
cl /std:c++20 /W4 /EHsc main.cpp
```

استخدم إعداد Debug أثناء التعلم ليحتفظ بمعلومات التصحيح، ثم قارن Release بعد تفعيل التحسينات. لا تجعل سرعة Release عذرًا لإخفاء Undefined Behavior.

البرنامج الذي سيتحرك عبر المراحل

```
#include <iostream>
#define TAX_RATE 0.14

int add(int a, int b)
{
    return a + b;
}

int main()
{
    int price{100};
    double total = price + price * TAX_RATE;
    std::cout << add(10, 20) << '\n';
    std::cout << total << '\n';
}
```

الناتج المتوقع `30` ثم `114` . الماكرو يوضح Preprocessing، لكن `constexpr double taxRate{0.14};` أفضل لقيمة عادية لأنه يملك نوعًا ونطاقًا ويخضع لقواعد C++.

1. Preprocessing

يعالج Preprocessor الأسطر التي تبدأ غالبًا بـ `#` قبل تحليل C++ الطبيعي:

- `#include` يضم تصريحات Header في عملية الترجمة.
- `#define` يجري Token substitution.
- `#if` و `#ifdef` و `#ifndef` تختار أجزاء من المصدر حسب الإعداد.
- `#pragma once` أو Header Guards يمنعان تضمين التعريفات نفسها أكثر من مرة داخل Translation Unit.

بعد التوسيع تصبح `TAX_RATE` قريبة من `0.14`. يظل الناتج C++، ولم ينتج Machine code بعد.

Translation Unit

كل ملف `.cpp` بعد Preprocessing مع محتوى Headers المضمّن يشكل Translation Unit مستقلة. لذلك قد يطول البناء عندما يتغير Header واسع الاستخدام، لأن ملفات كثيرة تحتاج إعادة ترجمة.

Headers تقدم تصريحات وأنواعًا مشتركة، أما التعريفات غير المناسبة للتكرار فتوضع في `.cpp`. تضمين ملف `.cpp` بدل Header يسبب غالبًا تعريفات مكررة وتنظيمًا خاطئًا للبناء.

2. Lexical analysis وTokens

يقسم Compiler Frontend النص إلى وحدات ذات معنى:

```
int x = 10 + 20;
```

ينتج بصورة مبسطة: الكلمة المحجوزة `int`، والاسم `x`، وعامل الإسناد، والعددين، وعامل الجمع، والفاصلة المنقوطة. Tokenization تتعرف إلى القطع لكنها لا تثبت أن ترتيبها أو معناها صحيح.

التعليقات لا تصبح تعليمات تنفيذ. يزيلها مسار الترجمة المبكر عادة، لذلك لا تزيد سرعة البرنامج ولا تقللها.

3. Parsing وAST

يفحص Parser ترتيب Tokens وفق Grammar اللغة ويبني **Abstract Syntax Tree** تمثل البنية. التعبير:

```
10 + 20 * 3
```

يضع الضرب أعمق من الجمع، لذلك يحسب `20 * 3` أولًا. يعكس هذا أولوية المعاملات حتى لو لم تكتب أقواسًا. الأقواس توثق نيتك وتجعل المراجعة أسهل.

خطأ مثل `int x = ;` يفشل نحويًا لأن التعبير بعد `=` مفقود. وقد يشير Diagnostic إلى Token تالٍ لأن المترجم لم يتأكد من الخطأ إلا عند الوصول إليه.

4. Semantic analysis

بعد صحة البنية يفحص المترجم المعنى القانوني:

- هل الاسم معرّف وفي Scope مناسب؟
- هل الأنواع متوافقة؟
- أي Overload يجب اختياره؟
- هل التحويل مسموح؟
- هل قواعد الوصول و`const` والقوالب صحيحة؟

```
int count = "Hello"; // البنية واضحة، لكن النوع غير متوافق
unknown = 10;        // الاسم غير معرّف
```

قد ينتج خطأ واحد رسائل تابعة كثيرة. أصلح أول سبب مرتبط بكودك ثم أعد البناء بدل معالجة كل سطر كأنه مشكلة مستقلة.

5. Intermediate Representation

قد يحول المترجم AST إلى **IR** أسهل للتحليل والتحسين وأكثر استقلالًا عن المعالج:

```
temp1 = b * c
temp2 = a + temp1
x = temp2
```

هذا Pseudocode تعليمي وليس LLVM IR حقيقيًا. تختلف الصيغة بين المترجمات، وقد يستخدم Compiler أكثر من IR في مراحل مختلفة.

6. Optimization

يحاول Optimizer تحسين البرنامج دون تغيير السلوك الملحوظ المطلوب لبرنامج سليم:

- **Constant folding:** يحسب `10 * 20` أثناء البناء.
- **Dead-code elimination:** يحذف فرعًا ثابتًا غير قابل للوصول.
- **Inlining:** يدمج جسم دالة صغيرة في موضع الاستدعاء عندما يكون مناسبًا.
- **Loop optimizations:** يعيد ترتيب أو يوسع بعض أعمال الحلقة وفق القيود.

لا يستطيع المترجم الحفاظ على معنى برنامج يعتمد على Undefined Behavior. مثلًا تجاوز مجال Signed integer أو استخدام كائن بعد انتهاء عمره يسمح بافتراضات تحسين قد تنتج سلوكًا غير متوقع.

7. Code generation والـAssembly

يختار Backend تعليمات المعمارية المستهدفة. قد تتحول دالة جمع على x86-64 إلى تعليمات تشبه:

```
mov eax, edi
add eax, esi
ret
```

هذا مثال توضيحي؛ أسماء Registers وطريقة تمرير المعاملات تتبع **Calling Convention** والـABI. Windows x64 وSystem V x86-64 وARM64 لا تستخدم الترتيب نفسه بالضرورة.

Assembly لغة نصية بأسماء مثل `mov` و`add`. أما Assembler فهو الأداة التي ترمز هذه التعليمات إلى Bytes يفهمها المعالج.

8. Object File

ينتج Assembler ملف `.o` أو `.obj`. غالبًا لا يمكن تشغيله وحده، وقد يحتوي:

- أقسامًا لكود الآلة والبيانات الثابتة والمتغيرة.
- **Symbols** للتعريفات التي يوفرها وللمراجع التي يحتاجها.
- **Relocation entries** لمواضع لا تُعرف عناوينها النهائية بعد.
- Debug information تربط التعليمات بأسطر المصدر.
- Metadata خاصة بصيغة الملف مثل ELF أو COFF.

وجود Machine code داخل Object file لا يعني أنه برنامج كامل، لأن عناوين ودوالّا ومكتبات قد تظل غير محسومة.

9. Separate Compilation

math.cpp

```
int add(int a, int b) { return a + b; }
```

main.cpp

```
int add(int, int); // declaration
int main() { return add(10, 20); }
```

يترجم كل ملف إلى Object file مستقل. يعرف `main.obj` شكل الدالة من التصريح لكنه يحتاج تعريفها، بينما يوفر `math.obj` التعريف. تسمح هذه الآلية بإعادة ترجمة الملفات المتغيرة فقط، مع إعادة بناء الملفات المتأثرة إذا تغير Header.

10. Linking

يجمع Linker ملفات Object والأجزاء المطلوبة من المكتبات ويحل المراجع:

```
main.obj: needs add(int, int)
math.obj: provides add(int, int)
Linker: matches the required symbol with its definition
```

أخطاء Linker الشائعة

- **Undefined reference / unresolved external:** التصريح معروف لكن التعريف غير موجود، أو ملفه/مكتبته لم تدخل أمر الربط، أو Signature غير مطابق.
- **Multiple definition:** ظهر تعريف غير `inline` أكثر من مرة، غالبًا بسبب وضعه في Header بصورة غير صحيحة.
- **Library order أو architecture mismatch:** المكتبة غير متوافقة أو لم تُربط بالطريقة المطلوبة.

قد ينتج الربط ملفًا تنفيذيًا مكتفيًا بأجزاء مكتبات ثابتة، أو ملفًا يعتمد على Dynamic libraries تُحمّل عند التشغيل.

11. Loading والتشغيل

بعد نجاح البناء يقرأ نظام التشغيل صيغة الملف التنفيذي، وينشئ Process وVirtual address space، ويحمّل أقسام البرنامج والمكتبات المطلوبة، ويجهز Stack وHeap، ثم ينقل التحكم إلى كود البداية الذي يصل في النهاية إلى `main`.

نجاح Build لا يضمن نجاح Run. قد يفشل تحميل مكتبة ديناميكية، أو تظهر مشكلة في الإدخال، أو يحدث Undefined Behavior، أو يعمل البرنامج لكنه يطبق صيغة خاطئة.

أنواع المشكلات

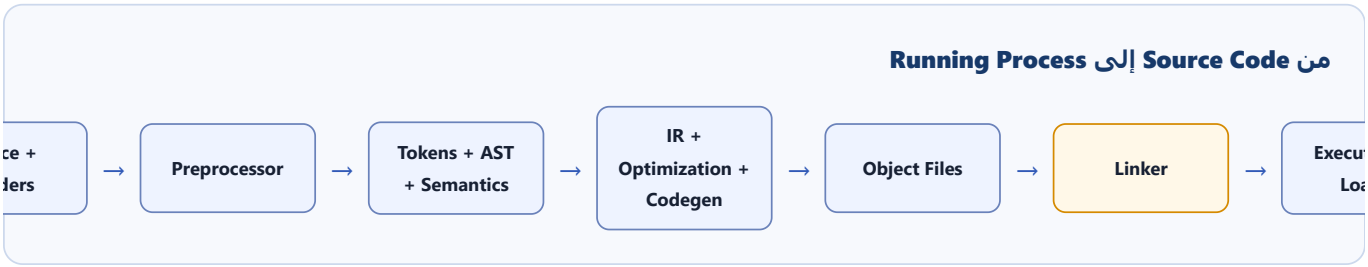
النوع	المثال	المرحلة
Preprocessor error	Header غير موجود أو `#if` غير مغلق	Preprocessing
Syntax error	فاصلة منقوطة مفقودة	Parsing
Semantic/type error	إسناد String إلى `int`	Semantic analysis
Link error	تعريف دالة مفقود	Linking
Load error	مكتبة ديناميكية مطلوبة غير موجودة	Loading
Runtime failure	قسمة صحيحة على صفر أو وصول غير صالح	Execution
Logic error	`area = width + height`	البرنامج يعمل لكن النتيجة خاطئة
Warning	تضييق `double` إلى `int` أو متغير غير مستخدم	Build ينجح غالبًا

التحذير ليس ضمان فشل، لكنه Finding يجب فهمه. فعّل مستوى تحذير قويًا، وتعامل مع التحذيرات الجديدة كأخطاء في المشروع التعليمي.

طريقة تشخيص عملية

1. أعد المشكلة بأصغر Input ثابت.
2. حدد هل الفشل في Build أم Link أم Load أم Run أم النتيجة.
3. اقرأ أول Diagnostic متعلق بملفك، مع اسم الملف والسطر ونوع المشكلة.
4. قلّل الكود حتى يبقى السبب وحده.
5. أصلح تغييرًا واحدًا وأعد البناء من الحالة النظيفة عند الشك في ملفات قديمة.
6. أضف Test يمنع رجوع الخطأ.

خريطة الدرس



تأكد من فهمك

01 **ما الفرق بين Source file وTranslation Unit وObject file؟**

الإجابة المشروحة: Source file هو الملف المكتوب. Translation Unit هي صورته بعد Preprocessing والتضمينات. Object file ناتج مترجم يحوي كود آلة وSymbols وRelocations لكنه قد لا يكون قابلًا للتشغيل وحده.

02 **لماذا قد تكون الصياغة صحيحة ويظهر Semantic error؟**

الإجابة المشروحة: Grammar قد تقبل شكل الإسناد، لكن تحليل الأنواع أو الأسماء يثبت أن القيمة لا تناسب النوع أو أن الاسم غير معروف.

03 **ماذا يضيف IR بين AST وكود الآلة؟**

الإجابة المشروحة: تمثيلًا داخليًا أسهل للتحليل والتحسين وأقل ارتباطًا بصياغة المصدر أو معمارية واحدة.

04 **لماذا يظهر Undefined reference بعد نجاح ترجمة كل ملف؟**

الإجابة المشروحة: لأن Compiler احتاج التصريح فقط، لكن Linker لم يجد تعريف الرمز في أي Object file أو مكتبة مربوطة.

05 **هل التحسين يستطيع تغيير سلوك برنامج سليم؟ وماذا عن Undefined Behavior؟**

الإجابة المشروحة: يجب أن يحافظ على السلوك الملحوظ المطلوب لبرنامج ذي سلوك معرّف. عند Undefined Behavior لا تقدم اللغة الضمان نفسه، وقد تبدو نتيجة التحسين غريبة.

06 **صنّف: Header مفقود، متغير غير معرّف، دالة بلا تعريف، مكتبة DLL مفقودة، صيغة مساحة خاطئة.**

الإجابة المشروحة: Header في Preprocessing، المتغير في Semantic analysis، الدالة في Linking، المكتبة في Loading، وصيغة المساحة Logic error أثناء الاختبار.

اربط النقاط ببعض

قسّم دراسة الدرس إلى preprocessing/compilation، ثم object files/linking، ثم loading/runtime. استخدم خيار حفظ الـpreprocessed source، و `nm` أو أداة المنصة لفحص symbols، و `objdump` لفحص sections. أضف ODR وtemplate instantiation كسببين شائعين لأخطاء linking المعقدة.

جرّب بنفسك

احتفظ بثلاثة artifacts من برنامج واحد واربط كل artifact بمرحلة وخطأ محتمل.

5. الأنواع والمتغيرات والنطاق والذاكرة

النوع يحدد شكل القيمة والعمليات الممكنة عليها وحدود تمثيلها. المتغير يجمع اسمًا ونوعًا وقيمة وعمرًا ونطاق رؤية.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **API:** واجهة محددة تسمح لبرنامج يطلب بيانات أو ينفّذ عملية عند برنامج آخر.
- **UTF-8:** طريقة شائعة لتحويل أرقام Unicode إلى بايتات تُحفظ وتُنقل.
- **Scope:** النطاق: المكان اللي يقدر الكود داخله يشوف اسمًا أو متغيرًا.
- **Function:** دالة: جزء كود له اسم ومهمة محددة ويمكن استدعاؤه أكثر من مرة.

النوع والمتغير

النوع يحدد شكل القيمة والعمليات الممكنة عليها وحدود تمثيلها. المتغير يجمع اسمًا ونوعًا وقيمة وعمرًا ونطاق رؤية.

اختيار النوع والتهيئة

النوع يحدد مجموعة القيم الممكنة، وطريقة تمثيلها في الذاكرة، والعمليات التي يسمح بها المترجم. اختر `int` عندما تمثل عددًا صحيحًا يقع داخل مداه، و`double` عندما تحتاج قيمة كسرية تقريبية، و`bool` لحقيقة منطقية، و`std::string` لنص قابل للتعامل كوحدة واحدة.

لا تختَر النوع من حجمه فقط. اسأل عن معنى القيمة وأكبر وأصغر قيمة متوقعة. رقم الهوية مثلًا قد يتكون من أرقام لكنه ليس كمية نجري عليها جمعًا، ولذلك يناسبه النص غالبًا.

التهيئة تضع أول قيمة للمتغير. الصيغة ذات الأقواس مثل `int count{0};` تساعد المترجم على رفض كثير من تحويلات التضييق. `const` تمنع تعديل القيمة بعد تهيئتها، بينما `constexpr` تطلب أن تكون القيمة قابلة للحساب وقت الترجمة عندما تسمح العبارة بذلك.

النطاق `Scope` يحدد أين يمكن استخدام الاسم، والعمر `Lifetime` يحدد متى يوجد الكائن فعلًا. المتغير المحلي ينتهي عادة عند مغادرة كتلته. أما `static` المحلي فيحتفظ بقيمته بين استدعاءات الدالة. تعريف اسم محلي يطابق اسمًا خارجيًا يسمى `Shadowing` وقد يخفي القيمة التي كنت تقصدها.

أمثلة على التصريح والتهيئة

```
#include <limits>
#include <string>

int age{20};
double price{49.95};
const std::string country{"Egypt"};
constexpr int daysPerWeek{7};
```

أخطاء شائعة وتصحيحات

- أحجام الأنواع قد تختلف بين المنصات؛ استخدم sizeof وnumeric_limits عند الحاجة.
- float وdouble تمثيلان تقريبيان، فلا تعتمد دائمًا على مساواة عشرية مباشرة.

خريطة الدرس

خريطة مفاهيم: الأنواع والمتغيرات والنطاق وترميز النص

فئات الأنواع

تضم الأنواع الأساسية `bool` وCharacter types وSigned/unsigned integers وFloating-point types و `void` . وتبني اللغة والمكتبة أنواعًا مركبة مثل Arrays وPointers وReferences، وأنواعًا معرفة من المستخدم مثل Classes وEnums، وأنواع مكتبة مثل `std::string` .

لا يضمن المعيار الحجم نفسه لكل Integer type على كل منصة، لكنه يضمن ترتيبًا أدنى للعروض. على MSVC الحديث يكون `int` غالبًا 4 Bytes و `long long` ثمانية، لكن استخدم `sizeof` و `std::numeric_limits` بدل حفظ افتراضات المنصة.

Signed وUnsigned والمدى

يخصص النوع Signed جزءًا من التمثيل للقيم السالبة، بينما يستخدم Unsigned كل الأنماط من صفر فأعلى ويطبق Arithmetic modulo `2^N` . خلط Signed وUnsigned قد يحول قيمة سالبة إلى عدد كبير. لا تستخدم Unsigned كعلاج تلقائي للمدخل السالب؛ تحقق من المجال أولًا.

```
std::cout << sizeof(int) << '\n';
std::cout << std::numeric_limits<int>::min() << '\n';
std::cout << std::numeric_limits<int>::max() << '\n';
```

Floating point

`float` و`double` يمثلان مدى واسعًا بدقة محدودة. كثير من الكسور العشرية لا تملك تمثيلًا ثنائيًا دقيقًا، لذلك لا تعتمد على `0.1 + 0.2 == 0.3`. استخدم Tolerance مناسبة للسياق، وللأموال استخدم أصغر وحدة صحيحة أو نوعًا عشريًا مناسبًا.

التصريح والتهيئة والإسناد

```
int a;       // قد يكون غير مهيأ محليًا
int b = 5;   // copy initialization
int c(5);    // direct initialization
int d{5};    // list initialization وتمنع narrowing الواضح
int e{};     // value initialization إلى صفر
```

يفضل تعريف المتغير قريبًا من أول استخدام وتهيئته فورًا. تستخدم `auto` النوع المستنتج من Initializer، لكنها لا تعني «بلا نوع». راجع النوع المستنتج عندما تكون المراجع و`const` والتحويلات مؤثرة.

الثوابت والذاكرة

`const` يمنع تعديل الاسم بعد التهيئة، و`constexpr` يطلب قيمة قابلة للاستخدام وقت الترجمة عند تحقق الشروط. لا يعني وجود متغير أن اللغة تعدك بعنوان ذاكرة ثابت؛ قد يحتفظ Optimizer بالقيمة في Register أو يزيلها إذا لم تؤثر في السلوك.

Scope وLifetime وShadowing

Block scope يبدأ وينتهي مع `{}`، وFunction scope يتعلق بأجزاء محددة مثل Labels، وNamespace scope يستمر عبر الملفات وفق التصريحات. قد يخفي متغير داخلي اسمًا خارجيًا، وهو Shadowing يربك القراءة. Lifetime يحدد متى يوجد الكائن، بينما Scope يحدد أين يمكن استخدام الاسم. الخلط بينهما يؤدي لاحقًا إلى References معلقة.

Integer types والمدى

توجد الأنواع `short` و`int` و`long` و`long long` بنسخ signed وunsigned. يضمن المعيار ترتيبًا أدنى للأحجام لكنه لا يضمن أن `int` دائمًا 32 بت على كل منصة. استخدم `sizeof` لمعرفة الحجم في بيئة معينة، و`std::numeric_limits<T>::min()` و`max()` لمعرفة المدى.

اختيار `long long` لا يصلح منطق البرنامج إذا كانت العملية نفسها تنفذ أولًا بنوع أصغر. في التعبير `int a * int b` يحدث الضرب كـ`int` قبل إسناده إلى `long long`. حوّل أحد الطرفين أو عرّف المدخلات بالنوع الواسع عند احتمال تجاوز المدى.

Literals وSuffixes وauto

العدد Integer literal `10` من نوع مناسب يبدأ عادة بـ`int`، وFloating literal `10.0` من نوع `double`. تضيف اللاحقة `LL` معنى `long`، وتضيف `f` معنى `float`. أما `auto` فيطلب من المترجم استنتاج النوع من القيمة المهيئة.

لا يعني `auto` أن المتغير بلا نوع أو أن نوعه يتغير لاحقًا. الاستنتاج يحدث وقت الترجمة ثم يبقى النوع ثابتًا. استخدمه عندما يكون النوع واضحًا من الجهة اليمنى أو طويلًا جدًا، ولا تستخدمه إذا كان يخفي معنى مهمًا مثل الفرق بين signed وunsigned.

التخزين والعمر والنطاق

المتغير المحلي العادي ينشأ عند وصول التنفيذ إلى تعريفه وينتهي عند مغادرة الكتلة. المتغير `static` المحلي ينشأ مرة واحدة ويستمر حتى نهاية البرنامج، لكنه يظل مرئيًا داخل الدالة فقط. المتغير العام يملك عمر البرنامج كله ويزيد الترابط بين الأجزاء، لذلك يفضل تقليل استخدامه.

النطاق Scope يتعلق بإمكانية الوصول إلى الاسم، والعمر Lifetime يتعلق بوجود الكائن. قد ينتهي نطاق اسم بينما يستمر كائن أشير إليه بطريقة أخرى، وقد يبقى اسم مرئيًا لكنه يشير إلى كائن انتهى عمره إذا استخدمت مرجعًا أو مؤشرًا بطريقة خاطئة.

القيمة غير المهيأة وUndefined Behavior

قراءة متغير محلي أساسي قبل تهيئته خطأ خطير، وقد تنتج قيمة مختلفة من تشغيل إلى آخر. التهيئة الصفرية بالأقواس تمنع ذلك: `int attempts{};` يبدأ من صفر و `bool finished{};` يبدأ بقيمة false.

لا تعتمد على أن الذاكرة كانت صفرًا أثناء التجربة. بعض الأخطاء تؤدي إلى Undefined Behavior، أي أن معيار C++ لا يفرض نتيجة محددة، وقد يبدو البرنامج صحيحًا ثم يتغير مع التحسين أو بيئة أخرى.

برنامج كامل: اختيار النوع والتحقق من المدى

```
#include <iostream>
#include <string>

int main() {
    std::string name;
    int age{};
    long long balancePiasters{};

    std::cout << "Name age balance-in-piasters: ";
    if (!(std::cin >> name >> age >> balancePiasters)) {
        std::cerr << "Invalid input\n";
        return 1;
    }
    if (age < 0 || age > 130 || balancePiasters < 0) {
        std::cerr << "Value outside the accepted range\n";
        return 1;
    }

    const bool adult = age >= 18;
    std::cout << name << " | age=" << age
              << " | adult=" << std::boolalpha << adult
              << " | balance=" << balancePiasters << " piasters\n";
}
```

مدخل `Omar 24 12550` يطبع بيانات صحيحة، بينما `Omar -2 100` يمر من ناحية النوع لكنه يفشل قاعدة المجال. النوع لا يغني عن Validation، و`long long` يختار من الحد الأقصى المحتمل لا من القيمة التجريبية الصغيرة.

تأكد من فهمك

01 لماذا **int** ليس مناسبًا دائمًا للعمر رغم أن القيم صغيرة؟

الإجابة المشروحة: قد يكون مناسبًا حسابيًا، لكن الاختيار يشمل معنى القيمة والتحقق منها؛ النوع وحده لا يمنع عمرًا سالبًا أو غير منطقي.

02 ما فائدة **Brace initialization**؟

الإجابة المشروحة: تجعل التهيئة واضحة وترفض كثيرًا من تحويلات التضييق مثل تحويل 3.7 مباشرة إلى int.

03 ما الفرق بين **const** و**constexpr**؟

الإجابة المشروحة: const يمنع التعديل بعد التهيئة، أما constexpr فيطلب إمكان التقييم وقت الترجمة عند استخدامه بهذه الصفة.

04 لماذا لا يمثل **char** حرفًا عربيًا كاملًا في **UTF-8** غالبًا؟

الإجابة المشروحة: لأن الحرف العربي يُرمز عادة بعدة بايتات، وchar يخزن وحدة بايت واحدة لا Code Point كاملًا.

05 **توقع الناتج:** `int x{3}; double y{x / 2};` **ثم طباعة** `y` **. لماذا؟**

الإجابة المشروحة: الناتج 1 لأن القسمة حدثت بين عددين صحيحين قبل التحويل إلى double. استخدم `double y = x / 2.0;` للحصول على 1.5.

06 **صحح تعريفًا يخزن عدد زوار قد يتجاوز مليار زائر وقيمة لا يجوز أن تتغير بعد التهيئة.**

الإجابة المشروحة: استخدم نوعًا موثق المجال مثل `const std::int64_t visitors{value};` بعد التحقق من المدخل، بدل افتراض أن `int` يكفي.

اربط النقاط ببعض

Reference اسم بديل لكائن موجود ولا يجوز أن تكون null بصورة عادية، بينما pointer قيمة عنوان قد تكون null وتتغير. enum class يمنع تسرب الأسماء والتحويل الضمني للأعداد. استخدم `std::span` أو range عندما تحتاج view مع حجم بدل pointer منفرد.

جرّب بنفسك

اكتب API مرة بـpointer ومرة بـreference ومرة بـspan وحدد العقد الذي يعبّر عنه كل اختيار.

6. ترميز النصوص والمحارف والتحويلات

النص والتحويلات والدوال الرياضية تلتقي كثيرًا في المسائل. المطلوب معرفة ما تمثله القيمة قبل تحويلها أو إجراء حساب عليها.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Unicode:** معيار بيعطي الحروف والرموز من لغات مختلفة أرقامًا موحدة.
- **UTF-8:** طريقة شائعة لتحويل أرقام Unicode إلى بايتات تُحفظ وتُنقل.

المحرف والنص والترميز

النص والتحويلات والدوال الرياضية تلتقي كثيرًا في المسائل. المطلوب معرفة ما تمثله القيمة قبل تحويلها أو إجراء حساب عليها.

ASCII وUnicode وUTF-8 وUTF-16

`ASCII` جدول قديم يربط الأرقام من `0` إلى `127` بمحارف إنجليزية ورموز تحكم. `Unicode` أوسع بكثير؛ فهو يعطي كل محرف مجرد `Code Point` مثل `U+0627` لحرف الألف. الـCode Point ليس هو البايتات المخزنة، لذلك نحتاج إلى Encoding.

`UTF-8` يشفّر الـCode Point في بايت واحد إلى أربعة بايتات، ويتطابق مع ASCII في أول 128 قيمة. `UTF-16` يستخدم وحدة بطول 16 بت، وقد يحتاج زوجًا من الوحدات لبعض المحارف. لهذا لا يساوي عدد البايتات عدد الحروف دائمًا، وقد يتكون الرمز المرئي من أكثر من Code Point.

نوع `char` يمثل وحدة صغيرة من البيانات ولا يضمن احتواء محرف Unicode كامل. التحويل الموسّع، مثل نقل `unsigned char` إلى `int`، يحافظ غالبًا على القيمة. التحويل إلى نوع أضيق قد يفقد جزءًا منها أو يغيّر معناها. `static_cast` يجعل نية التحويل صريحة لكنه لا يفحص المدى نيابة عنك.

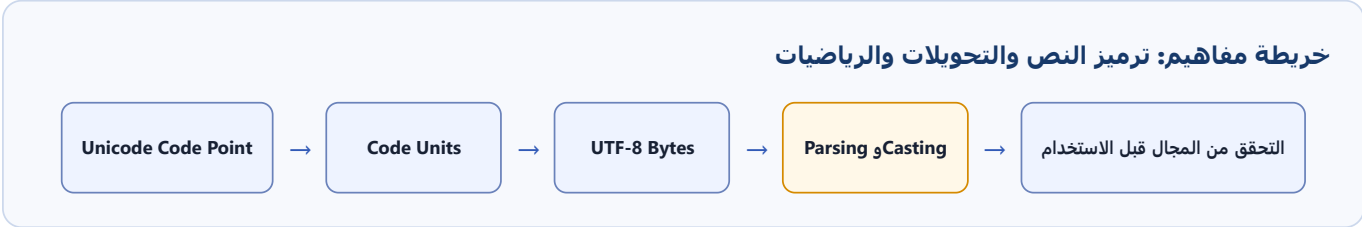
مثال: توسيع القيمة والتحقق قبل الجذر

```
unsigned char raw{255};
int widened = raw;
double root = value >= 0 ? std::sqrt(value) : 0.0;
```

أخطاء شائعة وتصحيحات

- عدد البايتات ليس عدد الحروف المرئية دائمًا.
- static_cast لا يثبت أن القيمة داخل المجال؛ تحقق أولًا.

خريطة الدرس



من المحرف إلى البايتات

المحرف فكرة نصية، وCode point رقم في Unicode، وEncoding يحول هذا الرقم إلى Code units ثم Bytes. ASCII يغطي مجموعة إنجليزية صغيرة من 0 إلى 127. UTF-8 يحافظ على هذه القيم ببايت واحد ويستخدم أكثر من بايت لمحارف أخرى.

`char` في C++ يخزن Byte صغيرًا، وليس «حرف Unicode كاملًا» بالضرورة. قد يتكون الحرف العربي أو Emoji من عدة Bytes في UTF-8، وقد يتكون Grapheme المرئي من أكثر من Code point. لذلك `std::string::size()` تعيد عدد Bytes لا عدد الحروف التي يراها المستخدم.

Character literals والقيم العددية

```
char letter{'A'};
std::cout << letter << '\n';
std::cout << static_cast<int>(letter) << '\n';
```

يطبع السطر الأول المحرف، والثاني قيمته العددية في Execution character set. لا تعتمد على ترتيب محارف خارج المجموعات التي يضمنها المعيار أو دون معرفة Encoding.

أنواع التحويل

- Integral promotion يرفع أنواعًا صغيرة قبل الحساب.
- Usual arithmetic conversions تختار نوعًا مشتركًا للمعاملين.
- `static_cast<T>` يوضح تحويلًا عدديًا أو تحويلًا مسموحًا وقت الترجمة.
- `const_cast` يغير Qualifiers ولا يجعل تعديل كائن Const أصليًا آمنًا.
- `dynamic_cast` يعمل في Hierarchies متعددة الأشكال ويدرس لاحقًا مع OOP.
- `reinterpret_cast` منخفض المستوى وخطير، ولا يحول معنى البتات إلى قيمة سليمة تلقائيًا.

تضييق `double` إلى `int` يقتطع الجزء الكسري، والتحويل خارج المجال قد يكون غير صالح أو Implementation-defined حسب الحالة. افحص المجال قبل التحويل.

Parsing ليس Casting

تحويل النص `"123"` إلى رقم عملية Parsing تحتاج فحص النجاح، وليست `static_cast<int>`. استخدم أدوات مثل `std::from_chars` أو Stream مع التحقق من الحالة. والعكس Formatting يحتاج تحديد الشكل والدقة.

Code Point وCode Unit والرمز المرئي

الـCode Point رقم مجرد يعرّف عنصرًا في Unicode. الـCode Unit هي الوحدة التي يستخدمها الترميز، مثل Byte في UTF-8 أو 16 بت في UTF-16. أما الرمز الذي يراه المستخدم فقد يتكون من أكثر من Code Point، مثل حرف مع علامة تشكيل أو Emoji مركب.

لهذا توجد أطوال مختلفة للنص: عدد البايتات، وعدد Code Points، وعدد الرموز المرئية. `std::string::size()` يعيد عدد البايتات المخزنة، ولا يضمن عدد الحروف التي يراها المستخدم.

بنية UTF-8 والتعامل الآمن

يستخدم محرف ASCII بايتًا واحدًا في UTF-8، وتستخدم القيم الأخرى بايتين أو ثلاثة أو أربعة. لا يجوز قص النص عند بايت عشوائي لأنك قد تفصل تسلسل محرف في منتصفه. كما أن UTF-8 لا يدير اتجاه العربية أو تشكيلها؛ تتولى ذلك مكتبات Unicode ومحركات العرض.

قد يملك نصان الشكل البصري نفسه وتسلسلين مختلفين من Code Points، ولهذا تحتاج المقارنة المتقدمة أحيانًا إلى Unicode Normalization قبل البحث أو التخزين الفريد.

Integer Promotions والتحويلات الحسابية

يرفع المترجم الأنواع الصحيحة الصغيرة مثل `char` و`short` غالبًا إلى `int` قبل الحساب. وعند جمع `int` مع `double` يتحول الصحيح إلى `double`. لكن `double result = 7 / 2;` يعطي `3.0` لأن القسمة الصحيحة حدثت قبل الإسناد. الصواب عند الحاجة إلى الكسر هو `static_cast<double>(7) / 2`.

static_cast وdynamic_cast وconst_cast وreinterpret_cast

يستخدم `static_cast` للتحويلات المعروفة وقت الترجمة. ويستخدم `dynamic_cast` مع الأنواع متعددة الأشكال لفحص التحويل وقت التشغيل. أما `const_cast` فيغير صفة `const`، و`reinterpret_cast` يعيد تفسير التمثيل منخفض المستوى؛ كلاهما يحتاج سببًا واضحًا وخبرة أكبر. تجنب C-style cast لأنه يخفي نوع التحويل المقصود.

تحويل نص مثل `"123"` إلى عدد ليس Cast بل Parsing. يجب فحص أن النص صالح كاملًا وأن النتيجة داخل المدى. توفر المكتبة `std::stoi` التي قد ترمي Exception و`std::from_chars` التي تعيد حالة فشل قابلة للفحص.

تأكد من فهمك

01 **ما الفرق بين Unicode Code Point وUTF-8 bytes؟**

الإجابة المشروحة: Code Point هو هوية مجردة للمحرف، وUTF-8 طريقة لترميزه إلى بايت واحد أو أكثر.

02 **لماذا text.size() قد لا يساوي عدد الحروف الظاهرة؟**

الإجابة المشروحة: في UTF-8 تعيد كثير من المكتبات عدد البايتات، وقد يتكون المحرف المرئي من عدة Code Points أيضًا.

03 متى يكون **Narrowing** مقبولًا؟

> **الإجابة المشروحة:** عندما تتحقق من المجال والدقة المطلوبة وتوثق فقد البيانات المقصود؛ cast وحده لا يكفي.

04 كيف تتعامل مع **sqrt** لقيمة مستخدم سالبة؟

> **الإجابة المشروحة:** ارفضها أو استخدم مجال الأعداد المركبة إن كان مطلوبًا؛ لا تحول NaN إلى نتيجة تبدو صحيحة.

اربط النقاط ببعض

C++ standard library لا تقدم معالجة grapheme كاملة؛ استخدم مكتبة Unicode متخصصة عندما تحتاج case folding أو normalization أو segmentation. `dynamic_cast` يحتاج polymorphic base وRTTI، وC-style cast يخفي أنواع تحويل متعددة. افحص integer promotions قبل الحساب على الأنواع الصغيرة.

جرّب بنفسك

اختبر نصًا عربيًا وemoji وقارن bytes وcode points وgraphemes بأداة مناسبة.

7. المعاملات والتعبيرات والعمليات البتية

العوامل تبني التعبيرات، والتحويلات تحدد النوع الذي تُحسب به النتيجة. افهم القواعد بدل الاعتماد على التجربة العشوائية.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Unicode:** معيار بيعطي الحروف والرموز من لغات مختلفة أرقامًا موحدة.
- **Boolean:** قيمة منطقية لها حالتان فقط: صح أو خطأ.

التعبير والعامل والنتيجة

العوامل تبني التعبيرات، والتحويلات تحدد النوع الذي تُحسب به النتيجة. افهم القواعد بدل الاعتماد على التجربة العشوائية.

الحساب والمقارنة والمنطق

التعبير `Expression` مجموعة قيم وعوامل تنتج قيمة جديدة. العوامل الحسابية هي `+` و`-` و`*` و`/` و`%`. عندما يكون طرفا القسمة عددين صحيحين، تكون النتيجة صحيحة أيضًا ويُحذف الجزء الكسري. لهذا تعطي `7 / 2` القيمة `3`، بينما يعطي `static_cast<double>(7) / 2` القيمة `3.5`.

عامل `%` يعيد باقي قسمة صحيحة، لذلك يستخدم لاختبار الزوجية وتقسيم القيم إلى دورات. العوامل `==` و`!=` و`<` و`<=` و`>` و`>=` تنتج `bool`. العوامل المنطقية `&&` و`||` تجمع شروطًا، و`!` يعكس الحقيقة. يستخدم `&&` و`||` خاصية Short Circuit، فقد لا يقيّمان الطرف الثاني إذا حُسمت النتيجة من الطرف الأول.

`static_cast` يوضح أن التحويل مقصود، لكنه لا يجعل القيمة مناسبة للنوع الجديد. افحص المجال قبل التحويل إلى نوع أصغر. تجاوز مدى signed integer يؤدي إلى سلوك غير معرّف، ويمكن قراءة الحدود من `std::numeric_limits`.

العوامل `&` و`|` و`^` و`~` و`<<` و`>>` تعمل على البتات نفسها. تستخدم في الأقنعة والرايات والبروتوكولات منخفضة المستوى، ولا تحل محل العوامل المنطقية.

مثال: متوسط لا يفقد الجزء الكسري

```
int total{7};
int count{2};
double average = static_cast<double>(total) / count; // 3.5
bool valid = count > 0 && average >= 0.0;
```

أخطاء شائعة وتصحيحات

- لا تخلط `=` مع `==`.
- ضع أقواسًا عندما تؤثر أولوية العوامل في وضوح المعنى.

الإدخال والإخراج

يعيد `std::cin >> value` الـStream نفسه، لذلك يمكن فحص نجاح القراءة. المدخل النصي الذي يحتوي مسافات يحتاج `std::getline`، وعند مزجه مع `>>` قد تحتاج إلى استهلاك نهاية السطر المتبقية.

```
double price{};
int quantity{};
if (!(std::cin >> price >> quantity) || price < 0 || quantity < 0) {
    std::cerr << "Invalid input\n";
    return 1;
}
```

عائلات المعاملات

- الحسابية: `+ - * / %`.
- المقارنة: `== != < <= > >=`.
- المنطقية: `! || &&` مع Short Circuit.
- الإسناد: `= += -= *= /=`.
- الزيادة والنقصان: فرّق بين القيمة الناتجة من `i++` و`++i` عندما تدخل في تعبير.
- البتية: `~ << >> ^ | &`، وتستخدم مع الأقنعة والتمثيل الثنائي لا مع شروط Boolean.
- الشرطية: `condition ? first : second` عندما يكون الاختيار تعبيرًا بسيطًا.

التحويلات

التحويل الضمني قد يرفع نوعًا صغيرًا أو يفقد جزءًا من القيمة. استخدم `static_cast` لإظهار النية، وابتعد عن C-style casts. `char` نوع عددي صغير يمثل Code unit، وتحويله إلى `int` يظهر القيمة العددية لا معنى المحرف الكامل في Unicode.

مكتبة الرياضيات والصيغ

توفر `<cmath>` دوالًا مثل `sqrt` و`pow` و`round` و`ceil` و`floor` و`abs`. انتبه إلى أن `ceil` و`floor` يتحركان نحو اتجاهين مختلفين مع القيم السالبة. اكتب الصيغة إلى خطوات مسماة، ووحّد الوحدات قبل الحساب.

```
double radius{};
std::cin >> radius;
constexpr double pi = 3.141592653589793;
double area = pi * radius * radius;
```

التعبير والقيمة والآثار الجانبية

قد ينتج Expression قيمة ويغيّر حالة في الوقت نفسه. الإسناد والزيادة والقراءة من Stream أمثلة لها Side effects. لا تضع عدة تعديلات على المتغير نفسه في تعبير معقد؛ افصل الخطوات حتى يكون ترتيبها واضحًا ومضمونًا.

الأولوية والترابط

تحدد Precedence كيف تُجمع المعاملات، وتحدد Associativity اتجاه التجميع عند تساوي الأولوية. الضرب قبل الجمع، و`&&` قبل `||`، والإسناد يرتبط من اليمين. لا تحفظ الجدول كاملًا؛ استخدم أقواسًا عند جمع عائلات مختلفة، لأن صحة الكود وحدها لا تكفي إن كان القارئ سيفهم قاعدة أخرى.

Postfix وPrefix

يزيد `++i` القيمة ثم ينتج الجديدة، بينما ينتج `i++` القيمة القديمة ثم يزيد. داخل Statement مستقل تكون النتيجة العملية نفسها للأنواع البسيطة، لكن الفرق يظهر داخل التعبير. تجنب تعبيرات تعتمد على عدة زيادات لأنها سهلة الخطأ ولا تضيف وضوحًا.

العمليات البتية

يعمل `&` و`|` و`^` و`~` و`<<` و`>>` على تمثيل العدد. القناع `permissions & writeMask` يختبر Bit، و`permissions | writeMask` يفعله، و`permissions & ~writeMask` يلغيه. لا تخلط `&` مع `&&` أو `|` مع `||`؛ الثانية تعمل على Boolean وتستخدم Short Circuit.

Ternary

المعامل Expression `condition ? a : b` تختار قيمة واحدة، ولذلك يناسب تعيين Label أو قيمة قصيرة. لا تحوله إلى شجرة شروط متداخلة طويلة؛ استخدم `if/else` عندما تحتاج خطوات أو حالات كثيرة.

خريطة الدرس

الإسناد المركب والزيادة والنقصان

يختصر `total += value` التعبير `total = total + value` مع تقييم الجانب الأيسر مرة واحدة. توجد صور مشابهة مثل `/=` و`*=` و`-=` و`%=`. أما `++i` فيزيد القيمة ثم ينتج الجديدة، بينما `i++` ينتج القديمة ثم يزيد المتغير. عندما لا تحتاج القيمة الناتجة، يوضح `++i` أن الهدف هو الزيادة نفسها.

لا تضع أكثر من تعديل معقد لنفس المتغير في تعبير واحد. افصل الخطوات إلى Statements واضحة حتى لا تعتمد على ترتيب تقييم يصعب فهمه أو يسبب سلوكًا غير معرّف في صيغ معينة.

Short Circuit كأداة أمان

في `count != 0 && total / count > 10` لن تنفذ القسمة إذا كان `count` صفرًا. وفي `index >= size || values[index] == 0` لن يحدث الوصول إلى العنصر إذا كان الفهرس خارج النهاية. يجب أن يأتي فحص الأمان قبل العملية التي يعتمد عليها.

Overflow وUnderflow

إذا تجاوز signed integer مداه فالسلوك غير معرّف. أما unsigned فيلتف وفق حساب modulo، لكن الالتفاف قد يظل خطأ منطقيًا. افحص قبل الجمع أو الضرب عندما تأتي القيم من مصدر غير موثوق، واستخدم نوعًا أوسع للنتائج الوسيطة.

Floating-point overflow قد ينتج Infinity، والقيم الصغيرة جدًا قد تصبح Subnormal أو صفرًا. لا تقارن نتائج الحسابات العشرية دائمًا بـ `==`؛ استخدم سماحية مناسبة لطبيعة المسألة.

أقنعة البتات

يمكن لكل بت أن يمثل Flag مستقلة. يستخدم `mask | flag` لتفعيل راية، و`mask & flag` لاختبارها، و`mask & ~flag` لإزالتها، و`mask ^ flag` لقلبها. يجب استخدام نوع unsigned مناسب، وتوثيق معنى كل بت بثابت مسمى بدل أرقام سحرية.

برنامج كامل: العمليات والتحويل والقسمة الآمنة

```
#include <iostream>
int main() {
    long long total{}; int count{};
    std::cout << "Total and count: ";
    if (!(std::cin >> total >> count) || count <= 0) {
        std::cerr << "Count must be positive\n"; return 1;
    }
    const double average = static_cast<double>(total) / count;
    std::cout << "Average: " << average << '\n'
              << "High average: " << std::boolalpha << (average >= 85.0) << '\n';
}
```

النسخة `double average = total / count;` تنفذ قسمة صحيحة أولًا. اختبر `7 2` لترى الفرق بين `3` و`3.5`، واختبر Count صفرًا.

تأكد من فهمك

01 **ما ناتج 7/2 و7.0/2 ولماذا؟**

> **الإجابة المشروحة:** الأول 3 بقسمة صحيحة، والثاني 3.5 لأن وجود double يرفع الحساب إلى قسمة عشرية.

02 لماذا static_cast<int>(largeDouble) ليس ضمان أمان؟

الإجابة المشروحة: هو يعلن نية التحويل فقط؛ قد تضيع الكسور أو تكون القيمة خارج مجال int، لذا يجب التحقق أولًا.

03 ما الفرق بين && و& مع قيم Boolean؟

الإجابة المشروحة: && منطقي ويستخدم Short-circuit، أما & بتّي ويقيّم الطرفين ويعمل على البتات؛ قد تتشابه نتيجة bool لكن السلوك مختلف.

04 كيف تمنع Overflow في n*(n+1)/2؟

الإجابة المشروحة: وسّع أحد العوامل قبل الضرب مثل 1LL*n، وتحقق أن المجال الأوسع يكفي للحد الأقصى المتوقع.

05 توقع قيم `7 / 2` و`7 / 2.0` وفسر اختلافهما.

الإجابة المشروحة: الأولى 3 لأن المعاملين صحيحان، والثانية 3.5 لأن وجود 2.0 يرفع العملية إلى floating point قبل القسمة.

06 ما الخطأ في حساب `long long total = a * b;` إذا كان `a` و`b` من نوع `int`؟

الإجابة المشروحة: قد يحدث overflow أثناء ضرب int قبل الإسناد الواسع. صححه إلى `1LL * a * b` وتحقق من أن long long نفسه يكفي.

اربط النقاط ببعض

ترتيب تقييم operands ليس دائمًا ترتيب كتابتها، فلا تعدل القيمة نفسها عدة مرات داخل expression. Signed overflow undefined behavior، بينما unsigned يلتف modulo. Bit shift على قيمة سالبة أو بعدد خارج المدى خطر؛ استخدم unsigned type وحقق عدد البتات.

جرّب بنفسك

اختبر expression ذات side effects ثم أعد كتابتها إلى statements واضحة.

8. الإدخال والتحويلات والحدود والرياضيات

الإدخال والإخراج جزء من تصميم البرنامج، وليس مجرد سطرين. تحقق من نجاح القراءة، وافهم حدود cin وgetline، وافصل خطأ البناء عن خطأ التشغيل والمنطق.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Runtime:** وقت التشغيل: الفترة اللي البرنامج بيكون شغال فيها فعلًا.
- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.

قراءة البيانات وكتابة النتائج

الإدخال والإخراج جزء من تصميم البرنامج، وليس مجرد سطرين. تحقق من نجاح القراءة، وافهم حدود cin وgetline، وافصل خطأ البناء عن خطأ التشغيل والمنطق.

cin وgetline وحالة الـStream

يقرأ `std::cin >> value` الرموز بعد تخطي المسافات، ثم يحاول تحويلها إلى نوع المتغير. إذا أدخل المستخدم نصًا في مكان عدد، يدخل الـStream حالة فشل ولا تتغير القراءات التالية حتى تمسح الحالة وتتخلص من الإدخال غير الصالح.

تقرأ `std::getline` السطر كاملًا بما فيه المسافات حتى نهاية السطر. بعد استخدام `>>` تبقى محرف نهاية السطر غالبًا، ولذلك يمكن كتابة `std::getline(std::cin >> std::ws, name)` لتخطي المسافة البيضاء المتبقية قبل قراءة الاسم.

التعليق الجيد يشرح سبب قرار أو قيد لا يظهر من الكود. أما `#include` فهو توجيه يعالجه الـPreprocessor قبل الترجمة. ينتج المترجم ملف Object، ثم يربط الـLinker هذا الملف بالتعريفات المطلوبة ليصنع الملف التنفيذي.

يوفر `<cmath>` دوال مثل `sqrt` و`pow` و`floor` و`ceil` و`round`. يجب فحص مجال الدالة ونوع القيمة المعادة. `sqrt` لا يعطي نتيجة حقيقية لعدد سالب، و`pow` قد يستخدم حسابًا عائمًا حتى عندما تبدو المدخلات صحيحة.

مثال: قراءة رقم وسطر بأمان

```
#include <iostream>
#include <string>

int main() {
    int age{};
    std::string name;
    if (!(std::cin >> age)) return 1;
    std::getline(std::cin >> std::ws, name);
    std::cout << name << " is " << age << "\n";
}
```

أخطاء شائعة وتصحيحات

- رسالة linker تختلف عن syntax error؛ ابحث عن تعريف مفقود أو تكرار رموز.
- sqrt لقيمة سالبة في الأعداد الحقيقية لا يعطي نتيجة صالحة.

خريطة الدرس

خريطة مفاهيم: الإدخال وبنية المصدر والأخطاء ومكتبة الرياضيات

حالة الـStream

تعيد عملية `>>` الـStream نفسه، ويتحول إلى False عند فشل القراءة. بعد الفشل تبقى القيمة الجديدة غير مقروءة وتظل حالة الخطأ حتى تستدعي `clear()`، وقد تحتاج `ignore()` لتجاوز النص المسبب للمشكلة.

```
int age{};
while (!(std::cin >> age) || age < 0 || age > 120) {
    std::cout << "Invalid age\n";
    std::cin.clear();
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}
```

عند استخدام `getline` بعد `>>` قد تقرأ نهاية السطر المتبقية كسطر فارغ. استخدم `std::ws` عند ملاءمته أو نظف الـDelimiter بوضوح.

التحويل وحدود النوع

قسمة Integer على Integer تنتج Integer. حوّل أحد الطرفين قبل القسمة لحفظ الكسر. لا يحمي `static_cast` من Overflow أو قيمة خارج المجال. استخدم `numeric_limits<T>::min()` و`max()` و`lowest()` وخصائص Floating point لفهم النوع على المنصة الحالية.

cmath والتقريب

توفر `<cmath>` دوال `sqrt` و`pow` و`abs` و`round` و`ceil` و`ceil` و`floor`. `floor` يتجه إلى موجب اللانهاية و`floor` إلى سالبها، لذلك يختلفان مع الأعداد السالبة. لحساب Ceiling division لأعداد موجبة يمكن استخدام `(a + b - 1) / b` بعد فحص Overflow أو `a / b + (a % b != 0)`.

ترجمة الصيغة إلى كود

حدد الرموز ووحداتها أولًا، ثم اكتب القيم الوسيطة بأسماء. مساحة دائرة هي `πr²` وليست `2πr`، ومساحة مستطيل ضرب الطول في العرض لا جمعهما. اختبر صفرًا وقيمة سالبة إن كانت غير صالحة وقيمة كبيرة قد تتجاوز المجال.

Source structure

توضع التصريحات المشتركة في Header والتعريفات في `.cpp`. استخدم Namespace لتنظيم الأسماء، ولا تعتمد على ترتيب تضمين عشوائي. اجعل كل ملف يضم مباشرة ما يحتاجه كي لا ينجح البناء بالصدفة بسبب Header آخر.

إصلاح Stream بعد إدخال غير صالح

عندما تفشل `std::cin` في تحويل النص، يظل الرمز غير الصالح في الإدخال وتُضبط Failbit. يجب مسح الحالة ثم تجاهل بقية السطر قبل إعادة المحاولة:

```
std::cin.clear();
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
```

ضع القراءة داخل حلقة تحقق إذا كان البرنامج تفاعليًا. أما في برنامج تنافسي أو أداة تقرأ ملفًا محدد البنية، فقد يكون إنهاء التنفيذ برسالة خطأ أنسب من طلب الإدخال مرة أخرى.

تنسيق الإخراج

يوفر `<iomanip>` أدوات مثل `std::fixed` و`std::setprecision` و`std::setw`. مع `fixed` تحدد `setprecision(2)` عدد الخانات بعد العلامة. هذا يغير طريقة العرض فقط ولا يحول `double` إلى قيمة مالية دقيقة.

مجال الدوال الرياضية

تحتاج `sqrt` إلى قيمة غير سالبة في الحساب الحقيقي، وتحتاج القسمة إلى مقام غير صفر. وقد تفقد `pow` الدقة أو تنتج Overflow. استخدم الضرب المباشر مثل `x * x` للقوى الصحيحة الصغيرة عندما يكون أوضح وأدق، وافحص `std::isfinite` إذا كانت الحسابات قد تنتج Infinity أو NaN.

Logic Error و Runtime Error و Link Error و Compile Error

يظهر Compile Error أثناء تحليل ملف مصدر، مثل فاصلة منقوطة مفقودة أو نوع غير متوافق. يظهر Link Error بعد نجاح ترجمة الملفات عندما لا يجد الـLinker تعريف دالة أو يجد تعريفات متكررة. يظهر Runtime Error أثناء التشغيل، مثل وصول غير صالح للذاكرة. أما Logic Error فيعطي نتيجة خاطئة من غير أن يتوقف البرنامج.

ابدأ بأول رسالة خطأ حقيقية في كودك؛ كثير من الرسائل التالية تكون نتيجة لها. فعّل التحذيرات، وقلّص المثال، وثبت المدخل الذي يعيد المشكلة قبل تعديل الكود.

تأكد من فهمك

01 **لماذا يقرأ getline سطرًا فارغًا بعد cin >> age؟**

الإجابة المشروحة: يبقى محرف newline في Stream؛ استخدم getline(cin >> ws, name) أو تعامل مع الباقي صراحة.

02 **كيف تتحقق من فشل إدخال رقم؟**

الإجابة المشروحة: اختبر حالة Stream مثل if (!(cin >> value))، ثم عالج الخطأ ونظف الإدخال قبل الاستمرار.

03 **صنّف: header مفقود، قسمة على صفر، ومعادلة خصم خاطئة.**

الإجابة المشروحة: الأول خطأ بناء، والثاني خطأ تشغيل أو سلوك غير صالح حسب النوع، والثالث خطأ منطقي.

04 **لماذا sqrt يحتاج تحققًا رغم أن الكود يُترجم؟**

الإجابة المشروحة: المترجم يتحقق من النوع لا من قيمة التشغيل؛ الجذر الحقيقي لقيمة سالبة خارج المجال ويعطي NaN.

اربط النقاط ببعض

`std::from_chars` يفصل parsing عن locale ويعطي موضع التوقف والخطأ دون exceptions. Stream exceptions اختيارية ويجب ألا تستبدل فحص الحالة بلا خطة. قراءة الملفات تضيف open failure وEOF وpartial data، وlocale قد يغير الفاصل العشري.

جرّب بنفسك

اقرأ القيمة نفسها عبر cin وfrom_chars وملف، وقارن عقود الفشل.

9. if وelse والتحقق والتداخل

if وelse توجّهان التنفيذ. اكتب الشروط بحيث يقرأها الإنسان كقواعد واضحة، وقلل التعشيق بالتحقق المبكر وتجميع المنطق المتشابه.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Compiler:** المترجم: برنامج يحوّل كود C++ إلى ملف يقدر الكمبيوتر يشغّله.
- **Unicode:** معيار بيعطي الحروف والرموز من لغات مختلفة أرقامًا موحدة.
- **Boolean:** قيمة منطقية لها حالتان فقط: صح أو خطأ.

كيف تختار if مسار التنفيذ؟

if وelse توجّهان التنفيذ. اكتب الشروط بحيث يقرأها الإنسان كقواعد واضحة، وقلل التعشيق بالتحقق المبكر وتجميع المنطق المتشابه.

if وelse وelse if

تفحص `if` تعبيرًا في سياق منطقي. المقارنة الصريحة مثل `age >= 18` أوضح من الاعتماد على تحويل رقم إلى `bool`. إذا كانت النتيجة `true` ينفذ جسم `if`، وإذا كانت `false` ينتقل التنفيذ إلى `else` إن وجدت.

استخدم الأقواس المعقوفة حتى مع تعليمة واحدة. عند إضافة تعليمة لاحقًا ستبقى حدود الفرع واضحة. ترتبط `else` بأقرب `if` لم تحصل على `else`، ولذلك تزيل الأقواس أي التباس في الشروط المتداخلة.

سلسلة `else if` تختار أول شرط صحيح فقط. أما عدة تعليمات `if` مستقلة فيمكن أن تنفذ كلها. استخدم السلسلة للتصنيفات الحصرية مثل التقدير، واستخدم الشروط المستقلة عندما يستطيع المستخدم الحصول على أكثر من ميزة.

رتب الحالات غير الصالحة أولًا، ثم الحالات الخاصة، ثم القاعدة العامة. ويتيح Short Circuit فحص الأمان قبل العملية، كما في `denominator != 0 && numerator / denominator > 2`، لأن القسمة لن تنفذ إذا كان المقام صفرًا.

مثال: تصنيف الدرجة بعد التحقق

```
if (score < 0 || score > 100) {
    std::cout << "Invalid score\n";
} else if (score >= 85) {
    std::cout << "Excellent\n";
} else if (score >= 50) {
    std::cout << "Pass\n";
} else {
    std::cout << "Fail\n";
}
```

أخطاء شائعة وتصحيحات

- if (x = 5) يسند قيمة بدل المقارنة.
- فاصلة منقوطة بعد if تصنع جسمًا فارغًا.

من قاعدة العمل إلى شرط

اكتب القاعدة أولًا بلغة واضحة، وحدد متغيراتها وحدودها، ثم حوّلها إلى Boolean expression. قاعدة «يقبل الطلب إذا كانت الكمية موجبة والمخزون كافيًا» تصبح `quantity > 0 && quantity <= stock` .

اختبر كل حد: الكمية صفر، واحد، تساوي المخزون، وتتجاوزه. الأخطاء تظهر غالبًا في الفرق بين `<` و `<=` أو في جمع شرطين بـ `&&` بدل `||` .

ترتيب القرارات والتحقق المبكر

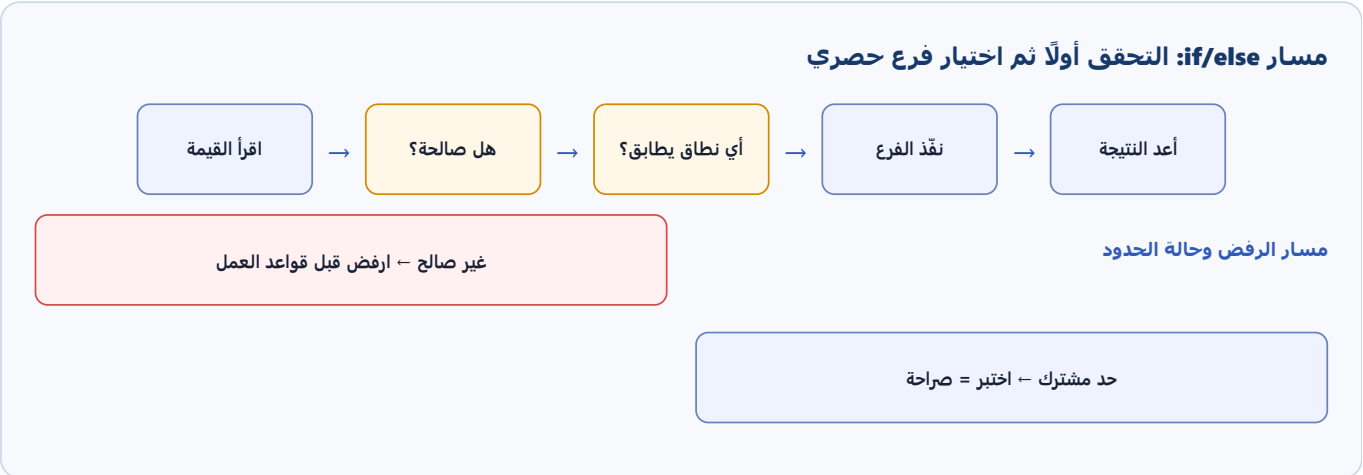
تحقق من الإدخال غير الصالح أولًا، ثم الحالات الخاصة، ثم القاعدة العامة. هذا يقلل التعشيق ويمنع تنفيذ الحساب على بيانات غير موثوقة.

```
if (age < 0 || age > 120) {
    std::cout << "Invalid age";
} else if (hasDebt) {
    std::cout << "Manual review";
} else if (income >= 30000 && creditScore >= 700) {
    std::cout << "Approved";
} else {
    std::cout << "Rejected";
}
```

استخدم `if` مستقلة عندما يمكن أن تتحقق عدة نتائج معًا، مثل عدّ الرقم موجبًا وزوجيًا. استخدم `else if` عندما يجب اختيار نتيجة واحدة فقط. وللنصوص استخدم مقارنة `std::string` ، مع الانتباه إلى حالة الأحرف والمسافات إن كانت القاعدة تتطلب التطبيع.

Short Circuit وأولوية المنطق

في `denominator != 0 && numerator / denominator > 2` يمنع الشرط الأول القسمة غير الآمنة. تعمل `&&` قبل `||` ، لكن الأقواس أفضل عندما تجمع قواعد متعددة لأنها توثق المقصود وتمنع القراءة الخاطئة.



Boolean context والأقواس

يمكن تحويل الصفر إلى `false` وغير الصفر إلى `true`، لكن كتابة المقارنة الصريحة توضح المقصود عندما لا يكون المتغير Boolean أصلًا. استخدم الأقواس حتى لجسم سطر واحد لتجنب أن تنفصل تعليمة جديدة عن الشرط عند التعديل.

عدة if أم سلسلة واحدة؟

عدة `if` مستقلة مناسبة عندما يمكن للرقم أن يكون موجبًا وزوجيًا معًا. أما تصنيف الدرجة إلى Band واحدة فيحتاج `if/else if/else`. ترتيب الحدود من الأعلى إلى الأقل يمنع شرطًا عامًا مثل `score >= 50` من ابتلاع الحالات الأعلى.

الشروط المتداخلة

يكون Nesting مناسبًا عندما لا معنى للسؤال الثاني إلا بعد نجاح الأول، مثل فحص Credit score بعد اجتياز العمر. لكن Guard clauses أو دمج شروط واضحة قد يقللان العمق. لا تدمج قواعد مستقلة في تعبير ضخم يصعب اختباره.

Strings والمدخلات المركبة

تدعم `std::string` المقارنة بـ`==`، لكن `"Admin"` لا يساوي `"admin"`. قرر هل النظام حساس للحالة، ونظف المسافات وفق قاعدة موثقة. لا تحول النص عشوائيًا دون مراعاة Locale وUnicode.

Guard Clauses وتقليل التداخل

عندما تكون حالة غير صالحة معروفة، عالجها مبكرًا ثم اخرج من الدالة. هذا يحافظ على المسار الصحيح في مستوى تداخل واحد بدل وضع كل خطوة داخل `if` جديدة. لا تستخدم الخروج المبكر عشوائيًا؛ اجعله لحالات واضحة برسائل ونتائج محددة.

الشروط الحصرية والشروط المستقلة

التقدير الدراسي نتيجة واحدة، ولذلك يناسبه `if/else if/else`. أما الشارات مثل «حضور ممتاز» و«مشروع متميز» فقد تجتمع، ولذلك تحتاج `if` مستقلة. اختيار البناء الخاطئ قد يمنع نتيجة صحيحة حتى لو كانت كل مقارنة منفردة سليمة.

De Morgan وتبسيط النفي

نفي `(age >= 18 && hasId)` يساوي `(age < 18 || !hasId)`. تساعد قوانين De Morgan في كتابة سبب الرفض مباشرة، لكن لا تحول الشرط إلى صيغة أصعب. سمّ الأجزاء المنطقية عندما يزيد عدد العوامل.

برنامج كامل: التحقق ثم التصنيف

```
#include <iostream>
int main() {
    int score{};
    std::cout << "Score from 0 to 100: ";
    if (!(std::cin >> score)) { std::cerr << "Score must be an integer\n"; return 1; }
    if (score < 0 || score > 100) std::cout << "Invalid score\n";
    else if (score >= 85) std::cout << "Excellent\n";
    else if (score >= 50) std::cout << "Pass\n";
    else std::cout << "Fail\n";
}
```

اختبر `-1` و`0` و`49` و`50` و`84` و`85` و`100` و`101` ومدخلًا نصيًا. وضع `score >= 50` أولًا يجعل فرع الممتاز غير قابل للوصول.

مسائل تطبّق على الشروط والصيغ

خريطة المسائل والحلول

959A Mahmoud and Ehab

المدخل: عدد صحيح موجب `n`. **المخرج:** اسم الفائز. لا نحتاج محاكاة أدوار اللعبة؛ النتيجة تعتمد على Parity فقط. إذا كان `n` زوجيًا يستطيع Mahmoud فرض الفوز، وإلا يفوز Ehab.

```
#include <iostream>

int main() {
    int n{};
    std::cin >> n;
    std::cout << (n % 2 == 0 ? "Mahmoud" : "Ehab") << '\n';
}
```

عند `n = 6` يكون الباقي صفرًا فتظهر `Mahmoud`. التعقيد `O(1)` والذاكرة `O(1)`. اختبر `1` و`2` وقيمة كبيرة. الخطأ الشائع هو كتابة Loop لمحاكاة لعبة حُسمت بخاصية رياضية.

486A Calculating Function

الدالة تجمع `-1 + 2 - 3 + 4 ...`. كل زوج يساوي `1`. إذا كان `n` زوجيًا توجد `n / 2` أزواج. وإذا كان فرديًا نحصل على أزواج حتى `n - 1` ثم نطرح `n`.

```cpp
#include <iostream>

int main() {
    long long n{};
    std::cin >> n;

    const long long answer = (n % 2 == 0)
        ? n / 2
        : -(n + 1) / 2;

    std::cout << answer << '\n';
}
```

لـ `-1 + 2 - 3 + 4 = 2` : `n = 4` . ولـ `n = 5` تصبح النتيجة `-3` . التعقيد `O(1)` . استخدم `long long` لأن القيود أكبر من مجال `int` . الحل البديل بحلقة صحيح للقيم الصغيرة لكنه يفشل زمنيًا مع حد كبير.

4A Watermelon

نريد تقسيم الوزن إلى جزأين زوجيين موجبين. لذلك يجب أن يكون الوزن زوجيًا، وأن يكون أكبر من 2 لأن `2 = 0 + 2` لا يعطي جزأين موجبين.

```cpp
#include <iostream>

int main() {
    int weight{};
    std::cin >> weight;

    if (weight > 2 && weight % 2 == 0) {
        std::cout << "YES\n";
    } else {
        std::cout << "NO\n";
    }
}
```

اختبارات الحدود: `4 → YES` ، `3 → NO` ، `2 → NO` . التعقيد `O(1)` . الخطأ الأشهر هو الاكتفاء بالزوجية فيقبل البرنامج الوزن 2.

835A Key Races

زمن كل لاعب يساوي وقت الكتابة `s * v` مضافًا إليه زمن الذهاب والعودة `2 * t` . احسب الزمنين مرة واحدة ثم قارنهما.

```cpp
#include <iostream>

int main() {
    long long s{}, v1{}, v2{}, t1{}, t2{};
    std::cin >> s >> v1 >> v2 >> t1 >> t2;

    const long long first = s * v1 + 2 * t1;
    const long long second = s * v2 + 2 * t2;

    if (first < second) {
        std::cout << "First\n";
    } else if (second < first) {
        std::cout << "Second\n";
    } else {
        std::cout << "Friendship\n";
    }
}
```

مثلًا `s=5, v1=1, v2=2, t1=1, t2=1` يعطي الزمنين 7 و12، فيفوز First. التعقيد `O(1)`. لا تكرر الصيغة داخل كل فرع لأن ذلك يزيد احتمال تعديل نسخة ونسيان الأخرى.

1173A Nauuo and Votes

لدينا أصوات موجبة `x` وسالبة `y` ومجهولة `z`. تكون `+` مضمونة إذا ظل `x` أكبر حتى بعد ذهاب كل المجهول إلى الطرف الآخر: `x > y + z`. وتكون `-` مضمونة بالعكس. التعادل مؤكد فقط إذا لم توجد أصوات مجهولة وكان الطرفان متساويين.

```
#include <iostream>

int main() {
    long long x{}, y{}, z{};
    std::cin >> x >> y >> z;

    if (x > y + z) {
        std::cout << "+\n";
    } else if (y > x + z) {
        std::cout << "-\n";
    } else if (z == 0 && x == y) {
        std::cout << "0\n";
    } else {
        std::cout << "?\n";
    }
}
```

اختبر `3 2 10` و`3 10 2` و`0 5 5` و`1 5 5`. التعقيد `O(1)`. الخطأ الشائع هو اعتبار `x > y` كافيًا رغم أن الأصوات المجهولة قد تقلب النتيجة.

318A Even Odds

يحتوي الجزء الفردي `(n + 1) / 2` عنصرًا. الموضع الفردي `k` يعطي `2*k - 1`. إذا تجاوز `k` الجزء الفردي، نطرح حجمه ثم نحول الموضع المتبقي إلى العدد الزوجي المقابل.

```
#include <iostream>

int main() {
    long long n{}, k{};
    std::cin >> n >> k;

    const long long oddCount = (n + 1) / 2;
    const long long answer = (k <= oddCount)
        ? 2 * k - 1
        : 2 * (k - oddCount);

    std::cout << answer << '\n';
}
```

لـ`n = 10` يصبح الترتيب `1 3 5 7 9 2 4 6 8 10`. اختبر `k=5` و`k=6` لأنهما نقطة الانتقال. التعقيد `O(1)` بدل بناء التسلسل في `O(n)`.

459A Pashmak and Garden

نريد نقطتين تكملان مربعًا موازيًا للمحاور. إذا تشارك المدخلان قيمة `x` فهما ضلع رأسي، وإذا تشاركا `y` فهما ضلع أفقي. وإلا يجب أن يكون الفرق المطلق في المحورين متساويًا ليكونا قطرًا صالحًا.

```cpp
#include <cstdlib>
#include <iostream>

int main() {
    int x1{}, y1{}, x2{}, y2{};
    std::cin >> x1 >> y1 >> x2 >> y2;

    if (x1 == x2 && y1 != y2) {
        const int side = std::abs(y1 - y2);
        std::cout << x1 + side << ' ' << y1 << ' '
                  << x2 + side << ' ' << y2 << '\n';
    } else if (y1 == y2 && x1 != x2) {
        const int side = std::abs(x1 - x2);
        std::cout << x1 << ' ' << y1 + side << ' '
                  << x2 << ' ' << y2 + side << '\n';
    } else if (std::abs(x1 - x2) == std::abs(y1 - y2)) {
        std::cout << x1 << ' ' << y2 << ' '
                  << x2 << ' ' << y1 << '\n';
    } else {
        std::cout << "-1\n";
    }
}
```

اختبر ضلعًا رأسيًا، ضلعًا أفقيًا، قطرًا صحيحًا، نقطتين متطابقتين، ومستطيلًا غير مربع. التعقيد `O(1)`. الخطأ الشائع هو قبول أي قطر دون مقارنة الفرقين المطلقين.

617A Elephant

يقطع الفيل من 1 إلى 5 وحدات في الخطوة. أقل عدد خطوات هو القسمة السقفية للمسافة على 5.

```cpp
#include <iostream>

int main() {
    int distance{};
    std::cin >> distance;
    std::cout << (distance + 4) / 5 << '\n';
}
```

للمسافة 12 نحتاج 3 خطوات: `5 + 5 + 2`. اختبر 1 و5 و6. التعقيد `O(1)`. الحلقة التي تطرح 5 صحيحة لكنها أطول وأقل تعبيرًا عن الصيغة.

581A Vasya the Hipster

يستخدم جوارب بلونين مختلفين مدة `min(a,b)`. بعد نفاد اللون الأقل، تُستخدم الجوارب المتبقية في أزواج من اللون نفسه.

```cpp
#include <algorithm>
#include <cstdlib>
#include <iostream>

int main() {
    int red{}, blue{};
    std::cin >> red >> blue;

    const int different = std::min(red, blue);
    const int same = std::abs(red - blue) / 2;
    std::cout << different << ' ' << same << '\n';
}
```

لـ `3 7` توجد 3 أيام مختلفة ثم يومان من اللون المتبقي. التعقيد `O(1)`. لا تجمع اللونين ثم تقسم؛ الأيام المختلفة تستهلك جوربًا من كل لون معًا.

281A Word Capitalization

المطلوب تغيير الحرف الأول فقط إلى Uppercase. تحقّق من أن النص غير فارغ، ومرر قيمة `unsigned char` إلى دوال `<cctype>` لتجنب سلوك غير معرّف مع القيم السالبة لـ `char`.

```
#include <cctype>
#include <iostream>
#include <string>

int main() {
    std::string word;
    std::cin >> word;

    if (!word.empty()) {
        const auto first = static_cast<unsigned char>(word.front());
        word.front() = static_cast<char>(std::toupper(first));
    }

    std::cout << word << '\n';
}
```

`hello` تصبح `Hello`، و`World` تبقى كما هي. التعقيد `O(1)` بالنسبة للعمل المطلوب بعد القراءة. المثال يتعامل مع محارف المسألة اللاتينية؛ لا يمثل حلًا عامًا لتحويل Unicode.

1A Theatre Square

نحسب عدد البلاطات على كل بُعد بقسمة سقفية، ثم نضرب العددين. يجب توسيع الحساب لأن الناتج قد يتجاوز `int`.

```
#include <iostream>

int main() {
    long long n{}, m{}, a{};
    std::cin >> n >> m >> a;

    const long long rows = (n + a - 1) / a;
    const long long columns = (m + a - 1) / a;
    std::cout << rows * columns << '\n';
}
```

لساحة `6 × 6` وبلاطة ضلعها 4 نحتاج `2 × 2 = 4` بلاطات. التعقيد `O(1)`. الخطأ الشائع هو استخدام القسمة الصحيحة مباشرة أو تخزين حاصل الضرب في `int`.

لكل مسألة اكتب القيود والتعقيد وحالة حدية قبل الكود. الحل القصير لا يعفيك من شرح سبب الصيغة.

تأكد من فهمك

01 لماذا يجب ترتيب `score>=85` قبل `score>=50`؟

الإجابة المشروحة: لأن else-if تتوقف عند أول شرط صحيح؛ وضع >=50 أولًا يلتقط درجات الممتاز ويجعل الفرع الخاص غير قابل للوصول.

02 ما أثر `if(x=5)`؟

الإجابة المشروحة: يسند 5 إلى x ثم تتحول القيمة غير الصفرية إلى true؛ استخدم `==` وفعّل تحذيرات المترجم.

03 متى تستخدم if مستقلة بدل else-if؟

الإجابة المشروحة: عندما يمكن أن تتحقق نتائج متعددة في الوقت نفسه، مثل منح أكثر من Badge؛ else-if للحالات الحصرية.

04 كيف تقلل Nested if في التحقق؟

الإجابة المشروحة: استخدم Guard clauses لرفض المدخل غير الصالح مبكرًا، ثم اترك المسار الرئيسي أقل تعشيقًا.

05 لديك الفروع `score >= 50` ثم `score >= 85` . لماذا لن يصل البرنامج إلى تقدير ممتاز؟

الإجابة المشروحة: لأن درجة 85 تحقق الفرع الأول في سلسلة else-if ويتوقف الاختيار. رتب الحدود من الأكثر تحديدًا والأعلى إلى الأقل: 85 ثم 50 ثم الباقي.

06 صحح `if (age = 18)` واذكر وسيلة تمنع مرور الخطأ بصمت.

الإجابة المشروحة: استخدم `if (age == 18)` . فعّل تحذيرات المترجم مثل `-Wall -Wextra -Werror` واكتب اختبارين لقيمتي 17 و18.

اربط النقاط ببعض

بعد إتقان الأمثلة اليدوية، أضف property-based checks للحواف: الناتج يجب أن يبقى داخل النطاق، والقيمة الأكبر لا تنتقل إلى تصنيف أقل دون قاعدة صريحة. استخدم compiler warnings لكشف الشروط الثابتة أو الفروع غير القابلة للوصول.

جرّب بنفسك

ولّد قيمًا حول كل حد وتأكد من عدم وجود gap أو overlap غير مقصود.

10. switch وتدفق التحكم والحاسبة

switch مناسب لاختيار فرع حسب قيمة منفصلة واحدة. استخدمه عندما تكون الحالات ثابتة وواضحة، ويفضل if للنطاقات والشروط المركبة.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Compiler:** المترجم: برنامج يحوّل كود C++ إلى ملف يقدر الكمبيوتر يشغّله.
- **Scope:** النطاق: المكان اللي يقدر الكود داخله يشوف اسمًا أو متغيرًا.

الاختيار بقيمة منفصلة

switch مناسب لاختيار فرع حسب قيمة منفصلة واحدة. استخدمه عندما تكون الحالات ثابتة وواضحة، ويفضل if للنطاقات والشروط المركبة.

selector وcase وbreak وdefault

تفحص `switch` قيمة واحدة تسمى `selector`، ثم تقفز إلى `case` المطابقة. يناسبها عدد صحيح أو `char` أو `enum`. لا تستخدم مباشرة مع `double` أو `std::string`، ولا تعبّر عن مدى مثل `score >= 50` لأن كل `case` يجب أن تكون قيمة ثابتة ومميزة وقت الترجمة.

بعد الوصول إلى حالة يستمر التنفيذ في الحالات التالية ما لم يقابله `break` أو `return` أو نهاية `switch`. هذا السلوك يسمى Fall-through، وقد يكون خطأ إذا نسيت `break`. عندما تقصده فعلًا استخدم `[[fallthrough]]` حتى يفهم القارئ والمترجم نيتك.

تعالج `default` أي قيمة لم تطابق الحالات المكتوبة. ويمكن وضع عدة `case` متتالية قبل جسم واحد عندما تشترك القيم في السلوك. إذا عرّفت متغيرًا محليًا داخل حالة، ضع جسم الحالة بين `{}` حتى لا يقفز التنفيذ عبر تهيئته من حالة أخرى.

مثال: قائمة أوامر

```
switch (choice) {
case 1: runReport(); break;
case 2: saveFile(); break;
case 0: std::cout << "Bye\n"; break;
default: std::cout << "Unknown option\n";
}
```

أخطاء شائعة وتصحيحات

- لا يختبر `switch` الشرط `score >= 50` مباشرة.

- نسيان break قد يشغّل أكثر من حالة.

ما الذي يقبله switch؟

يعمل Selector مع الأنواع الصحيحة و`char` وEnums والتحويلات المناسبة إليها. لا يقبل `double` أو `std::string` مباشرة. ويجب أن تكون كل `case` قيمة ثابتة ومميزة، وليست شرطًا ينفذ أثناء التشغيل.

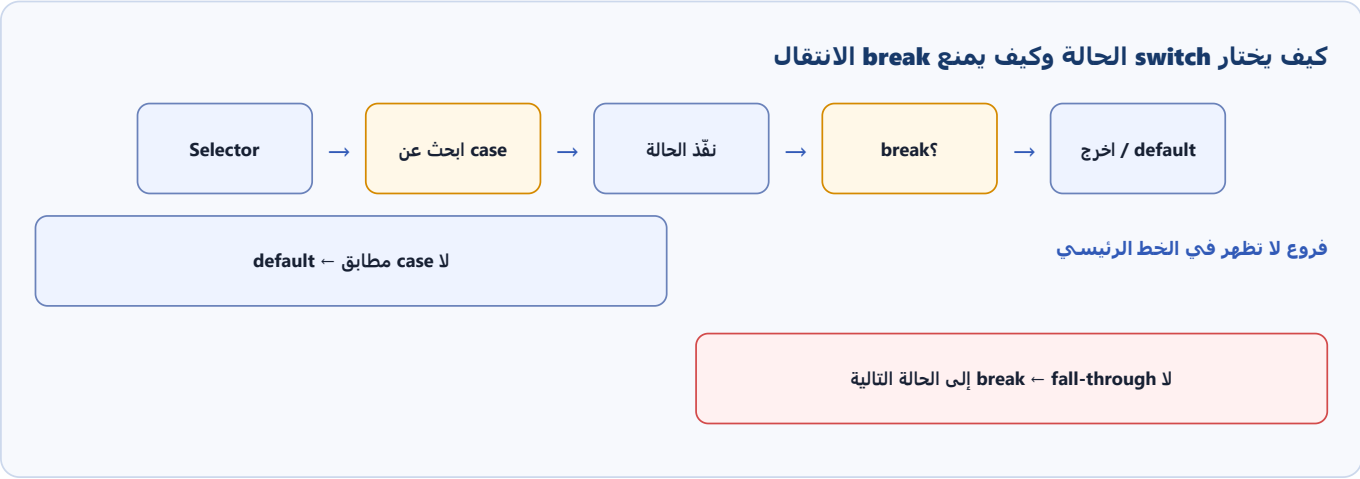
قد تتشارك حالات متعددة جسمًا واحدًا، ويمكن توثيق الاستمرار المقصود باستخدام `[[fallthrough]]` . ضع أقواسًا حول جسم الحالة إذا عرّفت متغيرًا محليًا حتى لا يقفز التنفيذ فوق تهيئته.

حاسبة آمنة

```
double a{}, b{};
char op{};
if (!(std::cin >> a >> op >> b)) return 1;

switch (op) {
case '+': std::cout << a + b; break;
case '-': std::cout << a - b; break;
case '*': std::cout << a * b; break;
case '/':
    if (b == 0.0) std::cout << "Cannot divide by zero";
    else std::cout << a / b;
    break;
default: std::cout << "Unsupported operator";
}
```

هذا المثال يوضح أن `switch` يختار العملية، بينما يعالج `if` قاعدة النطاق داخل حالة القسمة. اختر البنية حسب نوع القرار لا حسب الرغبة في استخدام صياغة جديدة.



Fall-through و Grouped cases

```
switch (answer) {
case 'Y':
case 'y':
    std::cout << "Confirmed";
    break;
case 'N':
case 'n':
    std::cout << "Cancelled";
    break;
default:
    std::cout << "Use Y or N";
}
```

تجمع الحالات المتتالية قيمًا مختلفة على جسم واحد. أما الاستمرار بعد تنفيذ تعليمات حالة فيوثق بـ `[[fallthrough]]` منذ C++17 عندما يكون مقصودًا. لا يختبر `switch` الحالة التالية من جديد؛ يبدأ عند Label المطابق ويستمر للأمام حتى `break` أو نهاية البنية.

Scope داخل case

لا تنشئ Labels نطاقًا مستقلًا. ضع جسم الحالة داخل `{}` عند تعريف متغير حتى لا يقفز التنفيذ فوق Initialization، وحتى يمكنك استخدام الاسم نفسه في حالة أخرى.

enum بدل الأرقام السحرية

إذا كانت الحالات تمثل مفهومًا معروفًا، فاستخدم `enum class` بدل `1` و`2` و`3`. يعطي كل اختيار اسمًا ونوعًا مستقلًا ويمنع خلطه بسهولة مع أعداد أخرى. ما زلت تحتاج إلى تحويل الإدخال الخام والتحقق منه قبل استخدام القيمة.

switch أم if أم جدول بيانات؟

استخدم `switch` لقيمة منفصلة واحدة، و`if` للنطاقات والشروط المركبة. إذا زاد عدد الحالات وكانت كل حالة تربط مفتاحًا بقيمة فقط، فقد يكون جدول أو Map أو Array أوضح من عشرات أسطر `case`. وإذا اختلف السلوك المعقد لكل حالة فقد تحتاج لاحقًا إلى دوال منفصلة.

كيف تنفّذ switch القرار خطوة بخطوة؟

يقيّم البرنامج تعبير `switch` مرة واحدة، ثم يحوّل النتيجة إلى النوع المناسب ويبحث عن Label تحمل القيمة نفسها. عند العثور عليها يقفز التنفيذ إلى أول تعليمة بعدها. لا يعيد البرنامج فحص الحالات التالية، لذلك لا تعني `case` شرطًا جديدًا مثل `if`؛ إنها نقطة دخول داخل بنية التحكم.

```
char action{'P'};

switch (action) {
case 'R':
    std::cout << "Registration\n";
    break;
case 'P':
    std::cout << "Payment\n";
    break;
case 'S':
    std::cout << "Status\n";
    break;
default:
    std::cout << "Unknown action\n";
}
```

القيمة هي `'P'`، لذلك يبدأ التنفيذ من حالة الدفع ويخرج عند `break`. لا يختبر البرنامج `'R'` ثم `'P'` بالتتابع كما تفعل سلسلة `if/else if` من ناحية الصياغة، وإن كانت النتيجة المنطقية متشابهة في هذا المثال.

الأنواع المسموح بها للـSelector

النوع	صالح مباشرة؟	الملاحظة
`char` والأنواع الصحيحة	نعم	مثل أرقام القوائم وحروف الأوامر
`bool`	نعم تقنيًا	غالبًا تكون `if` أوضح لأن هناك احتمالين فقط
`enum` و`enum class`	نعم	اختيار جيد للحالات المسماة بعد معالجة الإدخال
`float` و`double`	لا	القيم العشرية لا تصلح Labels منفصلة موثوقة
`std::string`	لا	استخدم `if/else if` أو خريطة أو حوّل الإدخال إلى Enum

قد تقبل البنية Class لها تحويل مناسب إلى نوع صحيح أو Enum، لكن التحويل الضمني هنا قد يخفي معنى القرار. في الكود التعليمي والتطبيقي اجعل نوع الـSelector واضحًا.

شروط case: قيمة ثابتة وفريدة

كل Label يجب أن تكون Integral Constant Expression معروفة وقت الترجمة، وأن تكون فريدة بعد التحويل إلى نوع الـSelector.

```
constexpr int registerCode = 1;
constexpr int reportCode = 1 + 1;

int choice{};
std::cin >> choice;

switch (choice) {
case registerCode:
    std::cout << "Register\n";
    break;
case reportCode:
    std::cout << "Report\n";
    break;
}
```

لا تصلح قيمة يحددها المستخدم ولا مقارنة وقت التشغيل كLabel:

```
// case choice:       // ليست ثابتة وقت الترجمة
// case choice > 2:   // Predicate وليست قيمة منفصلة
// case 2:            // تساوي 2 بالفعل reportCode خطأ إذا كانت
```

استخدام `constexpr` يعلن بوضوح أن الاسم قيمة ثابتة وقت الترجمة. وقد يكون `const int` صالحًا في بعض الحالات عندما يحقق قواعد Constant Expression، لكن `constexpr` يعبّر عن القصد بدقة أكبر.

تتبع Fall-through بدل تخمينه

```
int choice{1};

switch (choice) {
case 1:
    std::cout << "One\n";
case 2:
    std::cout << "Two\n";
    break;
default:
    std::cout << "Other\n";
}
```

التتبّع الصحيح:

1. يجد البرنامج `case 1` ويطبع `One`.
2. لا توجد `break`، فيكمل إلى التعليمات التالية ويطبع `Two`.
3. تقابله `break` فيخرج من `switch`.

لم يقارن البرنامج القيمة مع `case 2` مرة ثانية. إذا كان الانتقال مقصودًا بين جسمين، اكتب `[[fallthrough]];` في C++17 أو أحدث. أما وضع Labels متتالية بلا تعليمات بينها، مثل `'Y'` و `'y'`، فهو تجميع قيم على الجسم نفسه ولا يحتاج Attribute.

default ليست شرطًا إلزاميًا لكنها شبكة أمان

يمكن حذف `default`، وعندها لا ينفذ `switch` شيئًا إذا لم توجد حالة مطابقة. وجودها أفضل غالبًا مع مدخل المستخدم لأنها تكشف القيمة غير المدعومة. يجوز وضعها في الوسط، لكن وضعها في النهاية يجعل التدفق أسهل في القراءة. وإذا وضعتها قبل حالات أخرى فتذكّر أن التنفيذ قد يستمر منها إلى ما بعدها عند غياب `break`.

مثال كامل: حاسبة مع التحقق من الإدخال

```
#include <iostream>

int main() {
    double left{}, right{};
    char operation{};

    std::cout << "Enter: number operator number: ";
    if (!(std::cin >> left >> operation >> right)) {
        std::cerr << "Invalid input\n";
        return 1;
    }

    switch (operation) {
    case '+':
        std::cout << "Result: " << left + right << '\n';
        break;
    case '-':
        std::cout << "Result: " << left - right << '\n';
        break;
    case '*':
        std::cout << "Result: " << left * right << '\n';
        break;
    case '/':
        if (right == 0.0) {
            std::cerr << "Cannot divide by zero\n";
            return 1;
        }
        std::cout << "Result: " << left / right << '\n';
        break;
    default:
        std::cerr << "Unsupported operator\n";
        return 1;
    }
}
```

اختبر البرنامج بالمدخلات `8 / 2` و`8 / 0` و`5 * 3` و`5 ? 3` . كل اختبار يغطي مسارًا مختلفًا: نجاح القسمة، قاعدة منع الصفر، عملية عادية، والـ`default` . التحقق من نجاح `std::cin` يسبق القرار لأن `switch` لا يستطيع إصلاح Input Stream فاشل.

تأكد من فهمك

01 **لماذا لا يصلح switch مباشرة لـscore>=50؟**

الإجابة المشروحة: case يطابق قيمًا ثابتة منفصلة، لا نطاقات أو تعبيرات مقارنة؛ استخدم if/else للنطاقات.

02 **ماذا يحدث عند نسيان break؟**

الإجابة المشروحة: يستمر التنفيذ في الحالات التالية حتى break أو نهاية switch، وهو Fall-through قد يكون خطأ أو مقصودًا موثقًا.

03 **متى تجمع عدة case؟**

الإجابة المشروحة: عندما تشترك قيم متعددة في السلوك نفسه، مثل عدة حروف تؤدي إلى الأمر نفسه.

04 **لماذا تحتاج أقواسًا داخل case عند تعريف متغير؟**

الإجابة المشروحة: لإنشاء نطاق واضح وتجنب القفز فوق تهيئة متغير بين labels مختلفة.

05 **إذا بدأت القيمة من case 1 ولا توجد break قبل case 2، هل يعيد البرنامج مقارنة الـSelector مع 2؟**

الإجابة المشروحة: لا. بعد القفز إلى Label المطابقة يستمر التنفيذ للأمام مثل أي تعليمات متتابعة حتى break أو return أو نهاية switch.

06 **لماذا يستخدم مثال الحاسبة switch للعملية if داخل القسمة؟**

الإجابة المشروحة: العملية قيمة حرفية منفصلة تناسب switch، أما منع القسمة فيعتمد على Predicate وقت التشغيل وهو `right == 0.0`، لذلك تناسبه if.

اربط النقاط ببعض

C++17 يسمح init-statement قبل selector لتضييق scope. مع enum class فعّل تحذير عدم تغطية الحالات، وقرر هل default مطلوب: حذفه قد يساعد compiler على كشف قيمة جديدة، بينما وجوده مناسب للمدخل الخارجي بعد validation.

جرّب بنفسك

أضف قيمة enum جديدة وتأكد أن build أو test يكشف switch غير المكتملة.

11. الحلقات وأنماط التكرار

الحلقة تصف تكرارًا له حالة بداية وشرط استمرار وتحديث. اختيار while أو for أو do-while يعتمد على شكل التكرار لا على التفضيل الشخصي.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Loop:** حلقة تكرار تعيد تنفيذ مجموعة تعليمات وفق شرط.

مكوّنات الحلقة

الحلقة تصف تكرارًا له حالة بداية وشرط استمرار وتحديث. اختيار while أو for أو do-while يعتمد على شكل التكرار لا على التفضيل الشخصي.

for وwhile وdo-while

كل حلقة تحتاج حالة بداية، وشرط استمرار، وتحديثًا يجعل الحالة تقترب من التوقف. غياب التحديث أو تحركه في الاتجاه الخطأ ينشئ Infinite Loop.

تجمع `for` التهيئة والشرط والتحديث في سطر واحد، لذلك تناسب عدادًا معروف المسار. تستخدم `while` عندما يعتمد التوقف على إدخال أو حالة تتغير ولا نعرف عدد الدورات مقدمًا. تنفذ `do-while` الجسم مرة على الأقل لأنها تختبر الشرط بعد التنفيذ، ولذلك تناسب قائمة يجب عرضها مرة أو محاولة إدخال أولى إلزامية.

الـCounter متغير يعد مرات تحقق شرط. الـAccumulator يدمج القيم، فيبدأ المجموع من صفر والضرب من واحد. الـSentinel قيمة خاصة تنهي سلسلة غير معروفة الطول ولا تدخل ضمن البيانات. `break` ينهي أقرب حلقة، بينما `continue` يتجاوز بقية الدورة الحالية ويعود إلى الاختبار أو التحديث.

الحلقتان المتداخلتان لا تعنيان `O(n²)` دائمًا، لكن إذا نفذت الداخلية `n` مرة لكل واحدة من `n` دورات خارجية فالإجمالي يقارب `n²`. احسب عدد العمليات الفعلي ولا تعتمد على شكل الكود وحده.

مثال: جمع الأعداد الزوجية

```
long long sum{};
for (int i = 1; i <= n; ++i) {
    if (i % 2 != 0) continue;
    sum += i;
}
```

أخطاء شائعة وتصحيحات

- تأكد أن كل مسار يقترب من شرط التوقف.
- تحديث العداد في المكان الخطأ قد يسبب دورة زائدة أو ناقصة.

تتبع الحلقة يدويًا

اكتب جدولًا يحوي قيمة متغير التحكم، ونتيجة الشرط، والتغيير في الحالة بعد كل Iteration. حدد **البداية، وشرط الاستمرار، والخطوة** قبل كتابة `while` أو `for` . إذا كانت الخطوة لا تقرّب الحالة من التوقف فالحلقة قد تصبح لا نهائية.

أنماط التكرار

- Counter يبدأ من صفر ويزيد عند تحقق قاعدة.
- Accumulator يبدأ بقيمة محايدة، مثل صفر للجمع وواحد للضرب.
- Sentinel قيمة خاصة تنهي الإدخال ولا تدخل ضمن البيانات.
- Validation loop يكرر القراءة حتى تصبح القيمة صالحة.
- Nested loops تمثل صفوفًا وأعمدة، وعدد تنفيذ الجسم الداخلي يساوي حاصل ضرب حدود الحلقتين غالبًا.

```
int positive{}, negative{}, even{}, odd{};
for (int i = 0; i < n; ++i) {
    int value{};
    std::cin >> value;
    if (value > 0) ++positive;
    else if (value < 0) ++negative;
    if (value % 2 == 0) ++even;
    else ++odd;
}
```

لا تحتاج Array عندما تريد ملخصًا فقط ويمكن تحديثه فور وصول كل قيمة. تحتاج تخزين العناصر إذا كنت ستعود إليها لاحقًا أو ترتبها أو تقارنها معًا.

دورة الحلقة ومكان حدوث أخطاء البداية والحد والتحديث

while وfor وdo-while

تفحص `while` الشرط قبل الجسم وقد تنفذ صفر مرة. تجمع `for` التهيئة والشرط والتحديث في Header وتناسب العد المعروف. تنفذ `do-while` الجسم مرة قبل الفحص، فتناسب قائمة يجب عرضها أو إدخالًا يجب طلبه مرة على الأقل.

```
int number{};
do {
    std::cin >> number;
} while (number <= 0);
```

Infinite loops

قد تكون الحلقة اللانهائية خطأ بسبب Update مفقود أو اتجاه خاطئ أو فاصلة منقوطة بعد `while` . وقد تكون مقصودة في Server loop أو Event loop أو Menu، لكنها تحتاج Exit strategy واضحة واستجابة للفشل أو الإلغاء.

Sentinel والتحقق

قيمة Sentinel مثل `-1` تنهي التسلسل ولا تدخل في البيانات. يجب قراءة القيمة التالية داخل الحلقة، وإلا لن تتغير الحالة. في Validation loop صف القيم المرفوضة بالشرط: `age < 0 || age > 120` .

continue وbreak

ينهي `break` أقرب Loop، ويتجاوز `continue` بقية الـIteration الحالية. الإفراط فيهما قد يخفي تدفقًا معقدًا، لكنهما أوضح عندما يعبران عن توقف أو Skip محدد.

Nested loops

في مستطيل نجوم تمثل الحلقة الخارجية الصفوف والداخلية الأعمدة. إذا كان عدد الصفوف `r` والأعمدة `c` ينفذ الجسم الداخلي `r*c` مرة. أعد تهيئة متغير الحلقة الداخلية لكل صف، وتتبع صفًا كاملًا قبل تعميم النتيجة.

Invariant الحلقة

الـInvariant حقيقة تبقى صحيحة قبل كل دورة وبعدها. عند جمع أول `i` أعداد، يجب أن يمثل `sum` مجموع الأعداد التي تمت معالجتها فقط. يساعد ذلك على اختيار قيمة البداية وتحديث المتغير وتفسير النتيجة بعد التوقف.

البحث والحد الأدنى والأقصى

عند البحث يمكن التوقف بـ `break` بعد العثور على القيمة إذا كنت تحتاج أول ظهور فقط. وعند حساب الحد الأدنى لا تبدأ بقيمة سحرية قد تكون داخل البيانات؛ اقرأ أول قيمة صالحة أو احتفظ بمتغير يوضح أن البرنامج لم يرَ قيمة بعد.

معالجة أرقام العدد

يعطي `number % 10` الرقم الأخير في عدد موجب، وتزيله `number /= 10` . يمكن بهذه الحلقة حساب عدد الأرقام أو مجموعها أو عكسها. انتبه إلى حالة الصفر وإشارة العدد وأكبر قيمة عند بناء العدد المعكوس.

التعقيد الفعلي للحلقات المتداخلة

إذا دارت الحلقة الداخلية من الصفر إلى `n` لكل دورة خارجية فالتعقيد `O(n²)`. أما إذا تحرك مؤشر داخلي إلى الأمام فقط عبر التنفيذ كله، فقد يكون مجموع عمله `O(n)`. احسب عدد مرات تنفيذ الجسم بدل الحكم من عدد مستويات الأقواس.

نموذج الحلقة: البداية والشرط والجسم والتحديث

قبل أن تختار `while` أو `for`، حوّل المطلوب إلى أربع إجابات واضحة:

1. **البداية:** ما أول قيمة أو حالة؟
2. **شرط الاستمرار:** متى يسمح البرنامج بدورة أخرى؟
3. **الجسم:** ما العمل الذي يتكرر؟
4. **التحديث:** ما الذي يتغير حتى نقترب من التوقف؟

الدورة الواحدة تسمى Iteration. في الحلقات ذات الاختبار المسبق، يفحص البرنامج الشرط قبل كل Iteration؛ إذا كان خطأ من البداية فلن ينفذ الجسم مطلقًا.

while بالتتبّع اليدوي

```
int i = 1;

while (i <= 5) {
    std::cout << i << '\n';
    ++i;
}
```

قبل الدورة	نتيجة `i <= 5`	المطبوع	القيمة بعد `++i`
1	true	1	2
2	true	2	3
3	true	3	4
4	true	4	5
5	true	5	6
6	false	لا شيء	لا يوجد تحديث

يساعد الجدول على كشف الـOff-by-one قبل التشغيل. لو كتبت `i < 5` فآخر قيمة مطبوعة ستكون 4. ولو حذفت `++i` ستظل `i` تساوي 1 ويستمر البرنامج في طباعة 1.

العد التصاعدي والتنازلي والخطوات المختلفة

```
// من 0 إلى 9: عشر دورات بالضبط
int i = 0;
while (i < 10) {
    std::cout << i << ' ';
    ++i;
}

// من 10 إلى 1
int countdown = 10;
while (countdown >= 1) {
    std::cout << countdown << ' ';
    --countdown;
}

// الأعداد الزوجية من 0 إلى 20 مباشرة
int even = 0;
while (even <= 20) {
    std::cout << even << ' ';
    even += 2;
}
```

في العد التنازلي يجب أن تتحرك الخطوة نحو الحد. استخدام `++countdown` مع شرط `countdown >= 1` يبعد القيمة عن التوقف وقد ينتهي بسلوك غير معرّف عند تجاوز مجال النوع الصحيح.

تحديثات الضرب والقسمة

لا يشترط أن تكون الخطوة `+1` أو `-1` . قد تكبر القيمة أو تصغر بنسبة ثابتة:

```
int value = 1;
while (value <= 64) {
    std::cout << value << ' ';
    value *= 2;
}
// 1 2 4 8 16 32 64
```

```
int value{};
std::cin >> value;

while (value > 0) {
    std::cout << value << ' ';
    value /= 2;
}
```

إذا كان الإدخال `100` فالتتبّع هو `100, 50, 25, 12, 6, 3, 1` ثم تصبح القيمة صفرًا بسبب القسمة الصحيحة. هذا النمط ينفذ تقريبًا `log₂(value)` دورة، لا عددًا مساويًا للقيمة نفسها. عند الضرب راقب Overflow لأنه قد يمنع الشرط من الوصول إلى الحالة المتوقعة.

الفاصلة المنقوطة بعد while

```
int i = 1;
while (i <= 5);  // جسم فارغ
{
    std::cout << i;
    ++i;
}
```

الفاصلة المنقوطة هي جسم الحلقة الفارغ، ولذلك لا يصل التنفيذ إلى Block التالي ما دام الشرط صحيحًا، كما أن `i` لا تتغير داخل الحلقة. اكتب الأقواس في سطر واضح وفعّل تحذيرات المترجم كي يلفت انتباهك إلى Empty body.

Sentinel: إدخال لا نعرف طوله

```
int number{};
std::cin >> number;

while (number != 0) {
    std::cout << "Accepted: " << number << '\n';
    std::cin >> number;
}
```

الصفر هنا إشارة توقف وليس عنصرًا من البيانات. نحتاج قراءة أولية قبل الحلقة حتى يستطيع الشرط فحص قيمة حقيقية، ثم قراءة جديدة في نهاية كل دورة. حذف القراءة الثانية يجعل الشرط يفحص القيمة نفسها إلى الأبد. الصيغة الأكثر متانة تربط الاستمرار بنجاح Stream أيضًا:

```
while (std::cin >> number && number != 0) {
    std::cout << "Accepted: " << number << '\n';
}
```

do-while عندما يجب تنفيذ الجسم مرة

```
int number{};

do {
    std::cout << "Enter a positive number: ";
    std::cin >> number;
} while (number <= 0);
```

يطلب البرنامج الإدخال أولًا، ثم يكرر عند القيمة غير الصالحة. الفاصلة المنقوطة بعد `while (condition);` جزء إلزامي من الصياغة هنا، بعكس الخطأ السابق في حلقة `while` العادية.

قائمة تعمل مرة واحدة على الأقل

```
int choice{};

do {
    std::cout << "1. Play\n2. Settings\n0. Exit\n";
    std::cin >> choice;

    if (choice == 1) {
        std::cout << "Playing\n";
    } else if (choice == 2) {
        std::cout << "Settings\n";
    } else if (choice != 0) {
        std::cout << "Unknown choice\n";
    }
} while (choice != 0);
```

عرض القائمة يسبق اختبار الاختيار، لذلك تعبّر `do-while` عن المشكلة مباشرة. في تطبيق حقيقي عالج فشل `std::cin` أيضًا حتى لا يبقى البرنامج في حلقة تستخدم قيمة قديمة.

for وترتيب التنفيذ الحقيقي

```
for (int i = 1; i <= 5; ++i) {
    std::cout << i << '\n';
}
```

ترتيب التنفيذ هو:

1. تنفذ `int i = 1` مرة واحدة.
2. يفحص البرنامج `i <= 5` .
3. إذا كان الشرط صحيحًا ينفذ الجسم.
4. ينفذ `++i` بعد الجسم.
5. يعود إلى الشرط، ولا يعيد Initialization.

المعادلة المكافئة تقريبًا باستخدام `while` هي:

```
int i = 1;
while (i <= 5) {
    std::cout << i << '\n';
    ++i;
}
```

إذا كان لدينا `n` عناصر تبدأ فهارسها من صفر، فالنمط المعتاد هو `i < n` . عند `n = 5` تكون الفهارس `0, 1, 2, 3, 4` . استخدام `i <= n` ينتج ست دورات ويحاول لاحقًا الوصول إلى فهرس خارج الحدود.

Counter وAccumulator والـAverage

الـCounter يجيب عن سؤال «كم عنصرًا؟»، والـAccumulator يجيب عن «ما مجموع أو ناتج العناصر؟».

```cpp
int evenCount = 0;

for (int i = 1; i <= 10; ++i) {
    if (i % 2 == 0) {
        ++evenCount;
    }
}

std::cout << evenCount; // 5
```

```cpp
int n{};
std::cin >> n;

long long sum = 0;
for (int i = 0; i < n; ++i) {
    int number{};
    std::cin >> number;
    sum += number;
}

if (n > 0) {
    double average = static_cast<double>(sum) / n;
    std::cout << "Sum: " << sum << "\nAverage: " << average;
}
```

بدأ `sum` من صفر لأن الصفر هو العنصر المحايد للجمع. وحوّلنا قبل القسمة حتى لا نفقد الجزء العشري. استخدم `long long` عندما يمكن أن يتجاوز مجموع قيم صحيحة مجال `int` حتى لو كان كل مدخل منفردًا داخل المجال.

Product Accumulator

```cpp
long long product = 1;

for (int i = 0; i < 5; ++i) {
    int number{};
    std::cin >> number;
    product *= number;
}

std::cout << product;
```

يبدأ الضرب من 1 لأن `1 * x = x`. البدء من صفر يجعل الناتج صفرًا بعد أول عملية مهما كانت المدخلات. ما زال احتمال Overflow قائمًا لأن حاصل الضرب يكبر بسرعة.

تصنيف القيم أثناء قراءتها

```
int positive = 0;
int negative = 0;
int even = 0;
int odd = 0;

for (int i = 0; i < n; ++i) {
    int number{};
    std::cin >> number;

    if (number > 0) {
        ++positive;
    } else if (number < 0) {
        ++negative;
    }

    if (number % 2 == 0) {
        ++even;
    } else {
        ++odd;
    }
}
```

الصفر ليس موجبًا ولا سالبًا، لكنه عدد زوجي لأن باقي قسمته على 2 يساوي صفرًا. لا نحتاج Array هنا لأننا نستخرج كل الملخصات لحظة وصول الرقم. سنحتاج تخزين القيم إذا أردنا ترتيبها أو طباعتها لاحقًا أو مقارنة كل عنصر بعناصر أخرى.

أكبر قيمة دون افتراض أن الصفر أكبر

```
int maximum{};

if (!(std::cin >> maximum)) {
    return 1;
}

for (int i = 1; i < 5; ++i) {
    int number{};
    std::cin >> number;

    if (number > maximum) {
        maximum = number;
    }
}

std::cout << maximum;
```

نبدأ بأول قيمة حقيقية ثم نقرأ أربع قيم إضافية. لو بدأنا `maximum = 0` وكانت المدخلات كلها سالبة، سيطبع البرنامج صفرًا رغم أنه لم يظهر في البيانات. عند احتمال عدم وجود أي مدخل استخدم حالة «لا توجد قيمة بعد» بدل افتراض عنصر أول.

البحث باستخدام Flag أو التوقف المبكر

```
bool found = false;

for (int i = 0; i < 10; ++i) {
    int number{};
    std::cin >> number;

    if (number == 7) {
        found = true;
    }
}

std::cout << std::boolalpha << found;
```

تبقى `found` صحيحة بعد أول تطابق. إذا لم نحتج قراءة بقية القيم يمكن التوقف مبكرًا:

```
if (number == 7) {
    found = true;
    break;
}
```

لكن في مسألة Input منظمة قد تظل بقية القيم جزءًا من البيانات المطلوبة للخطوة التالية؛ لا تستخدم `break` قبل أن تفهم أثر تركها داخل Stream.

break وcontinue بالتتبّع

```
for (int i = 1; i <= 10; ++i) {
    if (i == 5) {
        break;
    }
    std::cout << i << ' ';
}
// 1 2 3 4
```

عند `i == 5` يخرج `break` من أقرب حلقة، فلا يزور القيم من 6 إلى 10.

```
for (int i = 1; i <= 10; ++i) {
    if (i == 5) {
        continue;
    }
    std::cout << i << ' ';
}
// 1 2 3 4 6 7 8 9 10
```

يتجاوز `continue` بقية جسم الدورة الحالية فقط. في `for` ينتقل إلى التحديث ثم الشرط. في `while` ينتقل إلى الشرط مباشرة، لذلك قد يتجاوز Update مكتوبًا في نهاية الجسم ويسبب Infinite Loop.

Nested Loops: صفوف وأعمدة

```
for (int row = 1; row <= 3; ++row) {
    for (int column = 1; column <= 4; ++column) {
        std::cout << "* ";
    }
    std::cout << '\n';
}
```

تكمل الحلقة الداخلية أعمدتها الأربعة لكل صف خارجي، فينفذ الطبع 12 مرة. الصيغة نفسها باستخدام `while` تحتاج إعادة تهيئة العمود داخل دورة الصف:

```
int row = 1;

while (row <= 3) {
    int column = 1;

    while (column <= 4) {
        std::cout << "* ";
        ++column;
    }

    std::cout << '\n';
    ++row;
}
```

لو وضعت `column` خارج الحلقة الخارجية ولم تعدها إلى 1، ستنتهي بعد الصف الأول وتفشل بقية الصفوف. ويمكن دمج نوعين مختلفين من الحلقات؛ المهم أن يكون دور كل واحدة واضحًا، مثل `while` لعدد الصفوف الذي تحكمه حالة خارجية و`for` لأعمدة ذات عدد معروف.

قائمة تفاعلية تتعافى من فشل الإدخال

```
#include <iostream>
#include <sstream>
#include <string>

int main() {
    std::string line;
    while (true) {
        std::cout << "1. Continue\n0. Exit\nChoice: ";
        if (!std::getline(std::cin, line)) {
            if (std::cin.eof()) return 0;
            std::cerr << "Input read failed\n";
            return 1;
        }
        std::istringstream input(line);
        int choice{};
        char extra{};
        if (!(input >> choice) || (input >> extra)) {
            std::cout << "Enter one integer only.\n";
            continue;
        }
        if (choice == 0) return 0;
        if (choice == 1) std::cout << "Continuing\n";
        else std::cout << "Unknown choice\n";
    }
}
```

بنقرأ سطرًا كاملًا بـ `getline` ، وبنحلله في `istringstream` ، يعني stream مصدره نص في الذاكرة. بنرفض النص الزائد زي `1x` ، ونعيد الطلب بعد المدخل الخطأ من غير فحص اختيار قديم. نهاية الإدخال EOF تنهي البرنامج بنجاح؛ فشل القراءة الآخر يرجع 1. في نهج القراءة المباشرة القديم، `clear()` يمسح حالة الفشل و `ignore()` يتخلص من بقية السطر، لكن لازم تفحص EOF قبل المحاولة من جديد.

تأكد من فهمك

01 **متى تختار do-while بدل while؟**

الإجابة المشروحة: عندما يجب تنفيذ الجسم مرة واحدة قبل اختبار الاستمرار، مثل عرض قائمة أو قراءة أول محاولة.

02 **ما سبب Off-by-one الأكثر شيوعًا؟**

الإجابة المشروحة: عدم الاتفاق على هل الحد مشمول، أو بداية العداد، أو موضع التحديث؛ اكتب أول وآخر قيمة متوقعة قبل الحلقة.

03 **كيف تحسب متوسط سلسلة تنتهي بـ-1؟**

الإجابة المشروحة: اجمع القيم وعدها دون إدخال -1، ثم اقسم بعد التأكد أن count>0.

04 **ماذا يحدث لتعقيد حلقتين متداخلتين كل منهما n؟**

الإجابة المشروحة: عادة $O(n^2)$، لكن افحص الحدود الفعلية؛ إذا الداخلية تتناقص أو تتقدم إجمالًا فقد يختلف التحليل.

05 **ما ناتج حلقة تبدأ من 100 وتنفذ `i /= 2` ما دام `i > 0` ؟ ولماذا تختفي الكسور؟**

الإجابة المشروحة: تطبع 100، 50، 25، 12، 6، 3، 1. المتغير int، لذلك القسمة الصحيحة تحذف الجزء الكسري؛ بعد 1 تصبح القيمة 0 فتتوقف الحلقة.

06 **لماذا يبدأ Product Accumulator من 1، ولماذا لا يجوز بدء Maximum من 0 دائمًا؟**

الإجابة المشروحة: الواحد هو العنصر المحايد للضرب، أما الصفر فيصفّر الناتج. والـMaximum يجب أن يبدأ بأول عنصر حقيقي لأن كل المدخلات قد تكون سالبة.

07 **ما الفرق بين break وcontinue داخل حلقة for، وما الخطر الخاص بـcontinue داخل while؟**

الإجابة المشروحة: break ينهي أقرب حلقة، وcontinue يتجاوز بقية الدورة. في while قد يتجاوز continue تحديث الحالة المكتوب آخر الجسم، فتظل الحالة ثابتة وتصبح الحلقة لا نهائية.

08 **لماذا يجب إعادة column إلى 1 داخل كل دورة من حلقة الصفوف؟**

> **الإجابة المشروحة:** لأن الحلقة الداخلية تستهلك قيم العمود حتى النهاية في الصف الأول. تهيئته داخل الحلقة الخارجية تنشئ عداد أعمدة جديدًا لكل صف.

اربط النقاط ببعض

الدرس فائق الكثافة؛ ادرسه في ثلاث وحدات: بناء الحلقة والتتبع، ثم counters/accumulators/search، ثم nested loops وcomplexity. حلقة range-based for تمر على العناصر مباشرة. تعديل حجم حاوية أثناء المرور عليها قد يبطل الـiterator، أي الموضع الذي تعتمد عليه الحلقة؛ اقرأ التطبيق في درس الحاويات. اختبر كل نمط قبل الانتقال للوحدة التالية.

جرّب بنفسك

أنشئ ثلاثة ملفات تدريب مستقلة بدل تشغيل أمثلة الدرس كلها كبرنامج واحد.

اختبر عقد إدخال القائمة

البرنامج الكامل بيحلل سطرًا كاملًا، فبيرفض `abc` والسطر الفارغ و`1x` والعدد خارج حدود int. المسافات حول عدد واحد مقبولة. جرّب السطور `abc` ثم `1` ثم `0`: رسالة خطأ، ثم `Continuing`، ثم خروج. اقفل الإدخال فورًا، ومرة ثانية بعد `1`: الحالتان لازم ينتهوا من غير دوران. مقتطف do-while السابق بيفترض مدخلات صحيحة. `continue` فيه لسه بيفحص شرط الحلقة، ففشل التحويل اللي يترك صفرًا ممكن يخرج بالخطأ. استخدم البرنامج الكامل لما تحتاج التعافي من المدخل غير الصحيح.

احفظه باسم `menu.cpp` وترجمه بالأمر `g++ -std=c++17 -Wall -Wextra -pedantic menu.cpp -o menu`. شغّل على `./menu` Linux/macOS أو `.\menu.exe` في PowerShell.

12. المدخلات وقواعد العمل

قواعد العمل تتحول إلى شروط واضحة بعد تسمية كل حقيقة مستقلة والتحقق من صحة المدخلات. افصل صلاحية البيانات عن قرار القبول.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Boolean:** قيمة منطقية لها حالتان فقط: صح أو خطأ.

من نص المتطلبات إلى شروط

قواعد العمل تتحول إلى شروط واضحة بعد تسمية كل حقيقة مستقلة والتحقق من صحة المدخلات. افصل صلاحية البيانات عن قرار القبول.

صلاحية البيانات وقرار العمل

ابدأ بجدول يحدد اسم كل مدخل ونوعه والمدى المقبول. الدرجة مثلًا عدد من `0` إلى `100`. هذه قاعدة صلاحية بيانات، وليست قرار نجاح. بعد التأكد من صحة القيمة يمكن تطبيق سياسة النجاح، مثل اشتراط درجة لا تقل عن `60` وحضور لا يقل عن `75`.

سمّ كل حقيقة مستقلة باسم يشرح معناها، مثل `hasPassingGrade` أو `attendanceIsValid`. الاسم يمنع تكرار تعبير طويل ويجعل الشرط قريبًا من لغة المتطلبات.

تجمع `&&` الشروط المطلوبة معًا، لذلك تفشل النتيجة إذا فشل أي شرط. تجمع `||` بدائل يكفي تحقق أحدها. عند مزج العاملين استخدم الأقواس لتوثيق المقصود حتى لو كانت أولوية العوامل ستعطي النتيجة نفسها.

رسالة «مرفوض» وحدها لا تساعد المستخدم أو الاختبار. حدد هل السبب بيانات غير صالحة، أم درجة منخفضة، أم حضور غير كافٍ. واختبر دائمًا قيمة أقل من الحد والحد نفسه وقيمة أعلى منه لتكتشف الفرق بين `>` و `>=`.

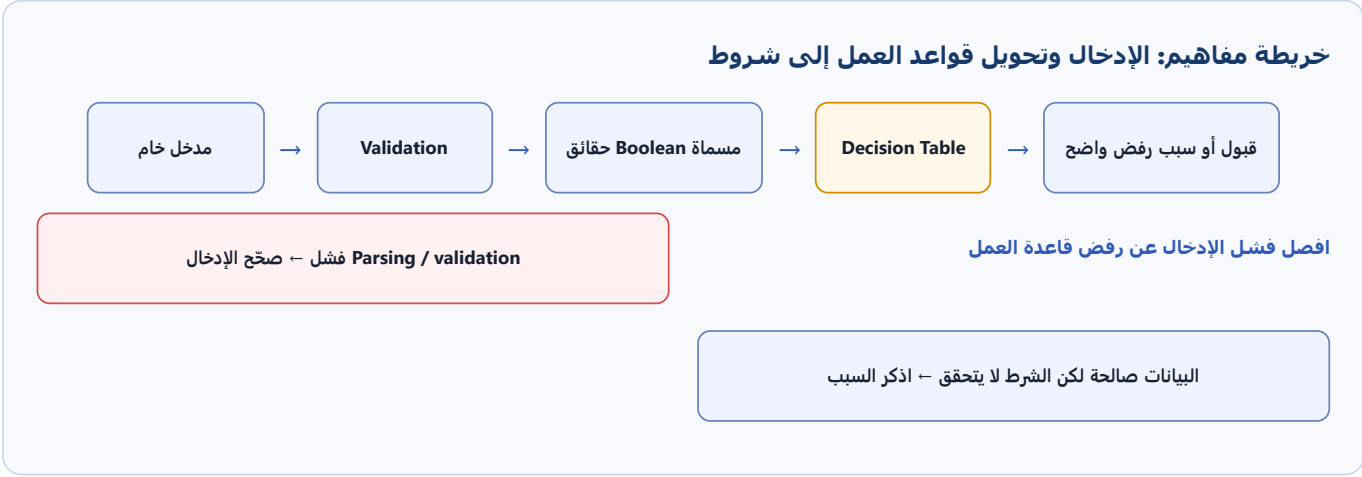
مثال: أهلية الطالب

```
bool validScore = score >= 0 && score <= 100;
bool eligible = validScore && score >= 60 && attendance >= 75;
```

أخطاء شائعة وتصحيحات

- لا تجعل قيمة غير صالحة تدخل في قرار الأعمال.
- انتبه هل الحد مشمول: `>` أم `>=`.

خريطة الدرس



استخراج المتطلبات

حوّل نص المسألة إلى أربعة أقسام: Inputs، Validity rules، Business decisions، Outputs. «اقرأ العمر والدخل، ارفض العمر الأقل من 21، واطلب ضامنًا عند انخفاض الدخل» يحتوي تحققًا من العمر، وقرارًا مستقلًا عن كفاية الدخل، ومسار مراجعة لا يساوي الرفض.

سمِّ الحقائق بدل تكرار تعبير طويل:

```
bool adult = age >= 21;
bool incomeEnough = income >= 30000;
bool strongCredit = creditScore >= 700;
bool canApprove = adult && incomeEnough && strongCredit;
```

الأسماء تجعل مراجعة القاعدة أسهل، لكن لا تنس التحقق من أن القيم نفسها داخل المجال المعقول قبل استخدامها.

Truth table وDecision table

يساعد Truth table في فهم `&&` و`||` و`!`. أما Decision table فيجمع عدة شروط وقواعد عمل. اكتب كل Combination المهم وحدد النتيجة حتى تكتشف حالة بلا Outcome أو قاعدتين تعطيان نتيجتين متعارضتين.

تصميم الحالات

اختبر Minimum وMaximum والقيمة عند الحد وما قبلها وما بعدها. أضف مدخلًا يفشل Parsing ومدخلًا صحيح النوع لكنه غير صالح في المجال. إذا استخدمت Floating point فلا تختبر التساوي الحرفي لنتائج حسابية حساسة دون سياسة تقريب واضحة.

ترتيب التحقق ورسائل الرفض

افصل فشل قراءة القيمة عن كونها خارج المجال وعن عدم استيفاء قاعدة العمل. النص بدل الرقم Input Error، ودرجة Validation Error `120`، ودرجة `55` رفض طبيعي وفق السياسة. هذا الفصل يمنع عرض «غير مؤهل» لبيانات لم تكن صالحة أصلًا.

إذا أمكن فشل أكثر من قاعدة، حدد هل يعرض النظام أول سبب أم كل الأسباب. عرض كل الأسباب يناسب نموذج تسجيل، بينما قد يتعمد نظام أمني عدم كشف تفاصيل حساسة. اكتب القرار في المتطلبات واختبر ترتيب الرسائل.

القواعد المشتقة وعدم تكرار المنطق

احسب الحقيقة المسماة مرة واحدة ثم استخدمها. إذا كررت `score >= 60 && attendance >= 75` في ثلاثة أماكن، فقد تعدل حدًا وتنسى الآخرين. يمكن لاحقًا نقل القاعدة إلى دالة نقية تستقبل البيانات وتعيد النتيجة من غير قراءة أو طباعة، فيسهل اختبارها.

برنامج كامل: صلاحية البيانات ثم قرار الأهلية

```
#include <iostream>
int main() {
    int score{}, attendance{}; char paidAnswer{};
    std::cout << "Score attendance paid(y/n): ";
    if (!(std::cin >> score >> attendance >> paidAnswer)) {
        std::cerr << "Invalid input format\n"; return 1;
    }
    if (score < 0 || score > 100 || attendance < 0 || attendance > 100 ||
        (paidAnswer != 'y' && paidAnswer != 'Y' && paidAnswer != 'n' && paidAnswer != 'N')) {
        std::cerr << "Input outside the accepted domain\n"; return 1;
    }
    const bool paid = paidAnswer == 'y' || paidAnswer == 'Y';
    const bool eligible = score >= 60 && attendance >= 75 && paid;
    std::cout << (eligible ? "Eligible\n" : "Not eligible\n");
}
```

اختبر `59/60` و `74/75` وإجابتي الدفع ومدخلًا خارج المجال. الـValidation يسبق قرار الأهلية ولا يختلط به.

تأكد من فهمك

01 **لماذا تفصل validScore عن eligible؟**

الإجابة المشروحة: الأول يثبت صلاحية البيانات، والثاني يطبق قاعدة العمل. الفصل يمنع اعتبار قيمة فاسدة رفضًا عاديًا ويعطي رسالة أدق.

02 **كيف تختبر حد حضور 75%؟**

الإجابة المشروحة: استخدم 74.99 أو 74 حسب النوع، ثم 75، ثم قيمة أعلى، إضافة إلى قيم غير صالحة تحت 0 وفوق 100.

03 **متى تستخدم && ومتى || في أهلية؟**

الإجابة المشروحة: && للمتطلبات التي يجب اجتماعها، و|| للمسارات البديلة؛ استخدم الأقواس عند دمجهما لتثبيت السياسة.

04 **لماذا رسالة "غير مؤهل" وحدها ضعيفة؟**

الإجابة المشروحة: لا تساعد المستخدم أو الاختبار أو الدعم على معرفة القاعدة الفاشلة؛ أعد سببًا محددًا دون كشف معلومات حساسة.

05 **صمم أقل مجموعة اختبارات لقاعدة: العمر من 18 إلى 60 شاملًا والحضور 75% على الأقل.**

الإجابة المشروحة: اختبر 17 و18 و60 و61 مع حضور صالح، ثم 74.99 و75 مع عمر صالح، وأضف مدخلًا غير رقميًا. هذه تغطي الحدود وكل سبب رفض مستقل.

06 **كيف تكشف Decision Table تعارض قاعدتين قبل كتابة الكود؟**

الإجابة المشروحة: تسرد تركيبات الحقائق ونتيجة كل تركيب؛ إذا حصل الصف نفسه على نتيجتين أو بقي صف بلا نتيجة يظهر الغموض قبل تحويله إلى if.

اربط النقاط ببعض

في تطبيق أكبر اجعل parsing يحول النص إلى type موثوق، وvalidation يثبت قواعد الشكل، وdomain service يقرر الأهلية. لا تكرر Boolean facts بين controller وواجهة وقاعدة بيانات؛ اجعل القرار في موضع واحد واختبره بجدول حالات.

جرّب بنفسك

استخرج القرار إلى دالة نقية وتأكد أن واجهتين تستخدمان العقد نفسه.

الدوال والحاويات والمراجع

قسّم البرنامج إلى دوال واضحة، واختر array أو vector أو string حسب شكل البيانات، ومرّر القيم بأمان باستخدام القيمة أو المرجع.

الدرس ده بعد الشروط والحلقات. المقتطفات الصغيرة بتوضح الصياغة داخل برنامج مع الترويسات المناسبة؛ المثال الأخير تحت برنامج كامل بمعيار C++17. المرجع اسم آخر لكائن موجود، والحاوية كائن يحتفظ بمجموعة قيم.

لماذا نحتاج الدوال؟

عندما يكبر البرنامج لا يجب أن تبقى كل الخطوات داخل `main`. الدالة تسمي مهمة واحدة، وتحدد مدخلاتها وناتجها، وتسمح باختبارها وإعادة استخدامها.

```
double finalPrice(double subtotal, double discountRate)
{
    return subtotal * (1.0 - discountRate);
}
```

يفصل **التصريح** اسم الدالة وأنواعها عن **التعريف** الذي يحتوي الجسم. يحتاج المترجم إلى معرفة التصريح قبل أول استدعاء، بينما يجد Linker التعريف النهائي عند ربط الملفات.

القيمة والمرجع وconst

التمرير بالقيمة ينسخ المعامل، لذلك لا يغير الأصل. المرجع `T&` يسمح بتعديل الأصل. المرجع الثابت `const T&` يمنع التعديل ويتجنب نسخ كائن كبير.

```
void addTax(double& total, double rate) { total *= 1.0 + rate; }
double average(const std::vector<int>& values);
```

استخدم قيمة للأنواع الصغيرة عندما تريد استقلالًا، و`const&` للقراءة من كائن كبير، و`&` فقط عندما يكون تعديل الوسيط جزءًا واضحًا من عقد الدالة. تجنب إعادة Reference لمتغير محلي لأنه ينتهي عند خروج الدالة.

النطاق والعمر

المتغير المحلي يعيش حتى نهاية Block. والمتغير `static` المحلي يحتفظ بقيمته بين الاستدعاءات. افصل بين **Scope** الذي يحدد أين يظهر الاسم و**Lifetime** الذي يحدد متى يوجد الكائن فعليًا.

array وvector وstring

النوع	الحجم	الاستخدام
`std::array<T, N>`	ثابت ومعروف وقت الترجمة	عدد عناصر ثابت مع واجهة آمنة
`std::vector<T>`	يتغير أثناء التشغيل	القائمة العامة الافتراضية
`std::string`	نص قابل للتعديل	معالجة النصوص والبحث والدمج

```
std::vector<int> scores{75, 91, 60};
scores.push_back(84);

int total = 0;
for (int score : scores) total += score;
double avg = scores.empty() ? 0.0
                            : static_cast<double>(total) / scores.size();
```

الفهرس يبدأ من صفر. `operator[]` لا يفحص الحدود، بينما `at()` يرمي استثناءً عند فهرس غير صالح. قد يعيد `vector` حجز ذاكرته عند النمو، لذلك قد تصبح المؤشرات والمراجع لعناصره القديمة غير صالحة.

تقسيم الملفات

ضع التصريحات العامة في Header محمي بـ `#pragma once` أو Header Guards، والتعريفات في ملفات `.cpp`. لا تستخدم `using namespace std;` في Header لأنه يلوث مساحة أسماء كل ملف يضمه.

مثال مترابط

ابنِ برنامج درجات بثلاث دوال: `readScores` للإدخال، و`average` للحساب، و`printReport` للعرض. بهذا يصبح اختبار الحساب مستقلًا عن الطرفية، ويمكن لاحقًا استبدال الإدخال بملف دون إعادة كتابة المنطق.

تأكد من فهمك

01 **متى تختار const T& بدل T؟**

الإجابة: عند قراءة كائن كبير دون نسخه ودون السماح بتعديله. أما الأنواع الصغيرة مثل int فتمريرها بالقيمة أبسط غالبًا.

02 **لماذا قد يصبح Reference لعنصر vector غير صالح بعد push_back؟**

الإجابة: قد ينقل vector عناصره إلى مساحة أكبر، فيشير المرجع القديم إلى الذاكرة السابقة.

03 **كيف تجعل دالة حساب المتوسط قابلة للاختبار دون cin أو cout؟**

> **الإجابة:** اجعلها تستقبل الحاوية وتعيد النتيجة فقط، واترك القراءة والعرض لدوال أخرى.

برنامج كامل واختبار الحدود

```
#include <iostream>
#include <stdexcept>
#include <vector>

double average(const std::vector<int>& scores) {
    if (scores.empty()) throw std::invalid_argument("No scores");
    double total = 0;
    for (int score : scores) {
        if (score < 0 || score > 100) throw std::invalid_argument("Score outside 0..100");
        total += score;
    }
    return total / static_cast<double>(scores.size());
}
int main() {
    const std::vector<int> scores{75, 90, 60};
    std::cout << average(scores) << '\n';
    try { std::cout << average({}) << '\n'; }
    catch (const std::invalid_argument& error) { std::cout << error.what() << '\n'; }
}
```

احفظه باسم `scores.cpp` وترجمه بـ `g++ -std=c++17 -Wall -Wextra -pedantic scores.cpp -o scores` ثم شغّله. الناتج `75` ثم `No` . `const&` `scores` يتجنب نسخ vector ويمنع الدالة من تغييره. المجمّع double مناسب لدرجات محدودة وحساب تقريبي، مش تمثيل أموال. القائمة الفارغة تُرفض بدل اختراع متوسط صفر. جرّب `{0}` و `{100}` و `{101}`: أول حالتين ترجعان الدرجة الوحيدة، والثالثة ترمي استثناءً. `average({})` ينشئ vector مؤقتًا يعيش حتى انتهاء النداء، لكن إرجاع مرجع إلى vector محلي يترك مرجعًا لكائن انتهى عمره.

مرجع محفوظ: التدريب على حل المسائل

نسخة مرجعية محفوظة من مادة التدريب التي أصبحت جزءًا من درس حل المشكلات.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Runtime:** وقت التشغيل: الفترة اللي البرنامج بيكون شغال فيها فعلًا.
- **Loop:** حلقة تكرار تعيد تنفيذ مجموعة تعليمات وفق شرط.
- **Function:** دالة: جزء كود له اسم ومهمة محددة ويمكن استدعاؤه أكثر من مرة.

ما البرمجة التنافسية؟

البرمجة التنافسية تدريب مركز على تحليل القيود واختيار الخوارزمية وكتابة حل صحيح سريع. المنصة أداة قياس وليست بديلًا عن الفهم.

قراءة المسألة والقيود والتعقيد

المسألة التنافسية تحدد صيغة الإدخال والنتيجة المطلوبة والقيود. اقرأ هذه الأجزاء قبل الأمثلة لأن المثال يوضح حالة واحدة، بينما القيود تحدد ما إذا كان الحل ممكنًا. إذا كان `n` يصل إلى `100000`، فإن مقارنة كل عنصر بكل عنصر قد تنفذ نحو عشرة مليارات عملية، ولذلك يكون `O(n²)` غير مناسب غالبًا.

حل مثالًا صغيرًا يدويًا واكتب العلاقة الرياضية أو حالة التكرار قبل الكود. اختر النوع من أكبر نتيجة وسيطة، لا من شكل العينة. بعد الانتهاء اكتب تعقيد الزمن والذاكرة حتى تتأكد أن الحل يناسب الحدود.

نتيجة `Wrong Answer` تعني أن البرنامج أنهى التنفيذ لكنه أعاد نتيجة مختلفة. راجع فهم النص والحدود ومواضع `1-based` و`0-based` ونوع البيانات والتقريب. `Runtime Error` يشير غالبًا إلى وصول غير صالح أو قسمة على صفر، و`Time Limit Exceeded` يعني أن عدد العمليات أكبر من المسموح. تغيير الأسطر عشوائيًا لا يشخص السبب.

مثال: مسألة Elephant والقسمة السقفية

```
// Elephant: minimum moves of length at most 5
int moves = (distance + 4) / 5; // integer ceiling
```

برنامج كامل قابل للتشغيل

```
#include <iostream>

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);

    int distance{};
    if (!(std::cin >> distance) || distance < 0) {
        return 1;
    }

    const int moves = (distance + 4) / 5;
    std::cout << moves << '\n';
}
```

إذا كان الإدخال `12` فالخروج `3`: خطوتان بطول 5 وخطوة أخيرة بطول 2. الزمن `O(1)` والذاكرة `O(1)`. لا يطبع البرنامج رسالة مثل `Enter` لأن الـOnline Judge يقارن الخرج حرفيًا. فحص الإدخال هنا مفيد عند تشغيل المثال محليًا، بينما تضمن المسألة الأصلية عادة أن `distance` الإدخال يطابق القيود.

أخطاء شائعة وتصحيحات

- قبول العينة لا يعني قبول كل الحالات.
- لا تبدأ بأصعب منصة؛ ابنِ عادة حل يومية تدريجية.

المنصات وخطة التدريب

توفر Codeforces مسابقات وتصنيفات ومجموعات مسائل، بينما تركز LeetCode كثيرًا على أنماط المقابلات وهياكل البيانات. ابدأ بمسائل تنفيذ وحساب وشروط، ثم الحلقات والحاويات، وسجل بعد كل مسألة: الفكرة، سبب الخطأ، التعقيد، وحالة اختبار لم تتوقعها.

أنماط المسائل المستخدمة في التدريب

المسألة	الفكرة الأساسية
617A Elephant	Ceiling division صحيحة بالأعداد الصحيحة
581A Vasya the Hipster	`min` ثم حساب الباقي
281A Word Capitalization	تعديل أول محرف مع فحص النص الفارغ
4A Watermelon	زوجية العدد مع استبعاد 2
1A Theatre Square	Ceiling لكل بعد واستخدام `long long`
959A Mahmoud and Ehab	Parity وشرط بسيط
486A Calculating Function	اشتقاق صيغة بدل Loop طويل
835A Key Races	حساب زمنين ثم مقارنة ثلاثية
1173A Nauuo and Votes	ضمان النتيجة رغم القيم المجهولة
318A Even Odds	تقسيم الترتيب إلى كتلة فردية وزوجية
459A Pashmak and Garden	حالات هندسية لمحاور ومربع

اقرأ الحل بعد محاولة حقيقية، ثم أغلقه وأعد كتابة الفكرة من الذاكرة على مثال مختلف. الهدف التعرف إلى النمط واشتقاقه، لا حفظ ترتيب الأسطر.

خريطة الدرس

دورة عمل الـOnline Judge

يرسل المتسابق Source Code، فتترجمه المنصة بإصدار وخيارات محددة ثم تشغله على حالات مخفية داخل حدود للوقت والذاكرة. تقارن المنصة الخرج المتوقع بالفعلي وفق Checker. أي نص إضافي مثل `Enter n:` قد يسبب Wrong Answer لأن المطلوب تنسيق دقيق.

تصنيف النتائج

`Compilation Error` يعني أن المصدر لم يُبنَ. `Wrong Answer` يعني أن التنفيذ انتهى لكن الخرج خاطئ. `Runtime Error` يشمل الانهيار والقسمة على صفر والوصول غير الصالح. `Time Limit Exceeded` يدل على زمن زائد، و `Memory Limit Exceeded` على استهلاك ذاكرة أكبر من الحد.

بعد القبول، لا تكتف بالكود. سجل الفكرة والتعقيد والحالة التي أخطأت فيها، ثم أعد الحل بعد أيام من الذاكرة. الهدف بناء نمط تفكير يمكن نقله إلى مسألة جديدة.

تأكد من فهمك

01 **لماذا تحدد القيود الخوارزمية قبل الكود؟**

الإجابة المشروحة: لأن n=10^5 قد يستبعد $O(n^2)$ مهما كان التنفيذ مرتبًا، بينما O(n log n) أو O(n) قد يناسب.

02 **ما معنى Wrong Answer بعد نجاح العينات؟**

الإجابة المشروحة: العينات ليست إثباتًا؛ راجع تفسير المسألة والحدود والأنواع والتقريب وأنشئ حالات مضادة.

03 **لماذا صيغة (d+4)/5 تحل Elephant؟**

الإجابة المشروحة: إنها Ceiling للقسمة الصحيحة على 5: إضافة 4 ترفع أي باقي إلى خطوة إضافية دون استخدام floating point.

04 **كيف تبني تدريبًا مفيدًا بعد قبول الحل؟**

الإجابة المشروحة: اكتب التعقيد، راجع حلًا بديلًا، أعد التنفيذ لاحقًا، وسجل الخطأ الذي منعك أول مرة.

اربط النقاط ببعض

جهز template صغيرًا لا يخفي الأنواع أو الأخطاء، وفعّل fast I/O فقط عندما تثبت القيود الحاجة. افصل local debug عن output المرسل، وأضف generator لحالات صغيرة تقارن الحل السريع بـbrute force. خطة التدريب تقيس نوع الأخطاء والزمن حتى الحل لا عدد المسائل فقط.

جرّب بنفسك

اكتب stress test يولد مدخلات صغيرة ويوقف عند أول اختلاف.

الكائنات وSTL وRAII والاختبارات

حوّل البيانات والقواعد إلى أنواع واضحة، واستخدم مكتبة C++ القياسية وإدارة الموارد التلقائية واختبارات صغيرة لبناء برامج قابلة للصيانة.

اقرأ الأول الدوال والحاويات. مقتطفات الأصناف والخوارزميات تحتاج البرنامج المحيط والترويسات؛ المثال الأخير كامل. الـiterator بيمثل موضعًا داخل حاوية؛ النطاق `[begin, end)` يشمل البداية ويستبعد الموضع اللي بعد آخر عنصر.

من المتغيرات إلى نوع يمثل المجال

يجمع `class` الحالة والعمليات التي تحافظ على صحتها. اجعل التفاصيل الداخلية `private`، وقدّم واجهة عامة لا تسمح بحالة غير صالحة.

```
class Order {
public:
    explicit Order(double subtotal) : subtotal_{subtotal} {
        if (!std::isfinite(subtotal) || subtotal < 0) throw std::invalid_argument{"negative subtotal"};
    }

    double total(double discount) const {
        if (!std::isfinite(discount) || discount < 0 || discount > 1)
            throw std::invalid_argument{"discount outside 0..1"};
        return subtotal_ * (1.0 - discount);
    }

private:
    double subtotal_{};
};
```

الـConstructor ينشئ كائنًا صالحًا. والكلمة `const` بعد الدالة تعني أنها لا تعدل الحالة الظاهرة للكائن. استخدم Composition لبناء نوع من أنواع أصغر قبل التفكير في الوراثة.

RAII وإدارة الموارد

يربط RAII عمر المورد بعمر كائن: يكتسب المورد في Constructor ويحرره Destructor. تستخدم `std::vector` و`std::string` هذه الفكرة لإدارة الذاكرة تلقائيًا، وتستخدمها Streams للملفات.

تجنب `new` و`delete` المباشرين في كود التطبيق. إذا احتجت ملكية ديناميكية، فابدأ بـ`std::unique_ptr`. استخدم `std::shared_ptr` فقط عندما تكون الملكية المشتركة حقيقية ومفهومة، لأن دورات المراجع قد تمنع التحرير.

خوارزميات STL

افصل **ما تريد** عن تفاصيل الحلقة. تعمل خوارزميات المكتبة على نطاق يحدده Iterators.

```
std::vector<int> scores{75, 42, 91, 60};
std::sort(scores.begin(), scores.end());

auto passed = std::count_if(scores.begin(), scores.end(),
    [](int score) { return score >= 50; });
```

تعرف أيضًا على `find` و`transform` و`accumulate`. لا تستخدم خوارزمية لمجرد قصر الكود؛ اخترها عندما تجعل النية أوضح وتحافظ على صحة الحدود.

الأخطاء والاستثناءات

فرّق بين إدخال متوقع أن يكون غير صالح، ويمكن تمثيله بنتيجة تحقق، وبين فشل يمنع العملية من إكمال عقدها وقد يناسبه Exception. التقط الاستثناء في مستوى يستطيع إصلاح المشكلة أو تحويلها إلى رسالة مفيدة، لا عند كل سطر.

الاختبارات والتصحيح

اختبر الدالة بوحدات صغيرة تغطي الحالة العادية والحدود والمدخل غير الصالح. استخدم Assertions لفرض افتراضات داخلية في التطوير، ولا تجعلها بديلًا عن التحقق من بيانات المستخدم.

```
#include <cassert>
#include <cmath>

constexpr double tolerance = 1e-9;
assert(std::abs(finalPrice(100.0, 0.10) - 90.0) < tolerance);
assert(std::abs(finalPrice(0.0, 0.10) - 0.0) < tolerance);
```

المقارنة بسماحية تناسب الحسابات العشرية التقريبية، لكن `assert` قد يزال عند تعريف `NDEBUG`. استخدم إطار اختبارات حقيقيًا لاختبارات المشروع، واترك Assertions لافتراضات التطوير الداخلية.

فعّل تحذيرات المترجم، واستخدم Debugger وSanitizers لاكتشاف القراءة خارج الحدود واستخدام الذاكرة بعد انتهاء عمرها. اجعل كل إصلاح Bug مصحوبًا باختبار كان يفشل قبله.

مشروع ختامي

أعد بناء مشروع الطلب الإلكتروني: `Product` يمثل المنتج، و`Cart` يحتفظ بالعناصر، و`PricingPolicy` يحسب الخصم، ودالة مستقلة تتحقق من الإدخال. استخدم `vector` للعناصر وAlgorithms للحساب، ثم اختبر سلة فارغة، وحد الخصم، وكمية غير صالحة.

تأكد من فهمك

01 **لماذا يفضّل RAII على تحرير المورد يدويًا في كل مسار؟**

الإجابة: لأن Destructor يعمل عند انتهاء العمر حتى مع return مبكر أو Exception، فيمنع التسرب وتكرار منطق التنظيف.

02 **متى يكون Composition أوضح من Inheritance؟**

الإجابة: عندما يحتوي النوع على تعاون بين أجزاء ولا توجد علاقة is-a حقيقية قابلة للاستبدال.

03 **ما الاختبار الذي تضيفه بعد إصلاح خطأ عند subtotal يساوي 500؟**

الإجابة: اختبار Regression للقيمة 500 مع حالتي 499 و501 حتى يثبت سلوك الحد ولا يعود الخطأ.

برنامج كامل واختبار الحدود

```
#include <algorithm>
#include <fstream>
#include <iostream>
#include <stdexcept>
#include <vector>

void savePassingScores(const std::vector<int>& scores) {
    std::ofstream file("passing-scores.txt");
    if (!file) throw std::runtime_error("Cannot open output");
    for (int score : scores) {
        if (score < 0 || score > 100) throw std::invalid_argument("Invalid score");
        if (score >= 50) file << score << '\n';
    }
    file.flush();
    if (!file) throw std::runtime_error("Write failed");
    file.close();
    if (!file) throw std::runtime_error("Close failed");
}
int main() {
    std::vector<int> scores{75, 42, 91, 60};
    std::sort(scores.begin(), scores.end());
    const auto passed = std::count_if(scores.begin(), scores.end(),
        [](int score) { return score >= 50; });
    try {
        savePassingScores(scores);
        std::cout << "passed=" << passed << '\n';
    } catch (const std::exception& error) {
        std::cerr << error.what() << '\n';
        return 1;
    }
}
```

احفظه باسم `raii.cpp` وترجمه بـ `g++ -std=c++17 -Wall -Wextra -pedantic raii.cpp -o raii` ، وشغله في مجلد تجارب قابل للكتابة. هيستبدل `passing-scores.txt` بالدرجات 60 و75 و91 كل واحدة في سطر، ويطبع `passed=3` . `sort` تغير vector، و `count_if` تقرأه باستخدام lambda، وهي دالة صغيرة في موضع استخدامها. الـstream يملك مقبض الملف ويحرره عند انتهاء النطاق حتى أثناء فك المكدس بسبب استثناء. فحص flush وclose صراحة يظهر فشل الكتابة؛ الـdestructor مش هيبلغ عنه عبر نتيجة العملية العادية. RAII يحرر المورد لكنه مش بيلغي البايتات المكتوبة: درجة غير صالحة ممكن تترك ملفًا جزئيًا. تحقق أولًا واستخدم بروتوكول ملف مؤقت ثم استبدال عندما تحتاج كتابة كاملة أو لا شيء. اختبر قائمة فارغة ومجلدًا لا يسمح بالكتابة.

مقتطف Order يحتاج `<cmath>` و `<stdexcept>` . بيرفض القيم غير المنتهية زي NaN، وكمان المجموع السالب والخصم خارج 0..1. double هنا لتوضيح بناء الصنف بحساب تقريبي؛ نموذج تسعير فعلي يحتاج وحدات مالية صحيحة محدودة أو نوعًا عشريًا مناسبًا.

مرجع محفوظ: مسائل الشروط والصيغ

نسخة مرجعية محفوظة من المسائل التي أصبحت جزءًا من درس if وelse.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Compiler:** المترجم: برنامج يحوّل كود C++ إلى ملف يقدر الكمبيوتر يشغّله.
- **Unicode:** معيار بيعطي الحروف والرموز من لغات مختلفة أرقامًا موحدة.
- **Loop:** حلقة تكرار تعيد تنفيذ مجموعة تعليمات وفق شرط.
- **Function:** دالة: جزء كود له اسم ومهمة محددة ويمكن استدعاؤه أكثر من مرة.

تحويل نص المسألة إلى حالات ومعادلات

مسائل الشروط تدربك على تحويل نص قصصي إلى معادلات وحالات حصرية. الصعوبة غالبًا في فهم الحدود، لا في كتابة if نفسها.

الحدود والحالات والحل الرياضي

عرّف كل رمز قبل كتابة الشرط. إذا كان `n` طول تسلسل و`k` موضعًا، فحدد هل يبدأ الموضع من واحد أم من صفر، وما أصغر وأكبر قيمة ممكنة. بعد ذلك قسّم الحل إلى حالات لا تتداخل، أو اكتب ترتيبًا واضحًا عندما تستطيع أكثر من قاعدة الانطباق.

لا تحاكِ عملية طويلة إذا كانت هناك صيغة مباشرة. في مجموع متناوب، يمكن جمع كل زوج بدل الدوران حتى `n`. وفي القسمة السقفية للأعداد الموجبة يمكن استخدام `(value + step - 1) / step`. الحل الرياضي يقلل الزمن ويجعل حدود الحالات أوضح.

اختر النوع بناءً على أكبر عملية وسيطة. قد يدخل كل رقم داخل `int` لكن حاصل ضرب رقمين يتجاوزه، ولذلك تستخدم مسائل كثيرة `long`. لا تستخدم `double` عندما تحتاج نتيجة صحيحة دقيقة.

في `Even Odds` توضع الأعداد الفردية أولًا ثم الزوجية. عدد الأعداد الفردية حتى `n` هو `(n + 1) / 2`. إذا وقع `k` في هذا النصف فالقيمة `2*k - 1`، وإلا نطرح حجم النصف الأول ثم نحول الموضع المتبقي إلى عدد زوجي.

مثال: اشتقاق Even Odds

```
long long oddCount = (n + 1) / 2;
long long answer = (k <= oddCount)
    ? 2 * k - 1
    : 2 * (k - oddCount);
```

أخطاء شائعة وتصحيحات

- انتبه هل k فهرس يبدأ من 1 أم 0.
- لا تستخدم double لحساب يحتاج دقة صحيحة كاملة.

خريطة الدرس

خريطة مفاهيم: الشروط وتمارين Codeforces

خريطة المسائل والحلول

959A Mahmoud and Ehab

المدخل: عدد صحيح موجب `n`. **المخرج:** اسم الفائز. لا نحتاج محاكاة أدوار اللعبة؛ النتيجة تعتمد على Parity فقط. إذا كان `n` زوجيًا يستطيع Mahmoud فرض الفوز، وإلا يفوز Ehab.

```
#include <iostream>

int main() {
    int n{};
    std::cin >> n;
    std::cout << (n % 2 == 0 ? "Mahmoud" : "Ehab") << '\n';
}
```

عند `n = 6` يكون الباقي صفرًا فتظهر `Mahmoud`. التعقيد `O(1)` والذاكرة `O(1)`. اختبر `1` و`2` وقيمة كبيرة. الخطأ الشائع هو كتابة Loop لمحاكاة لعبة حُسمت بخاصية رياضية.

486A Calculating Function

الدالة تجمع `1 - 2 + 3 - 4 + ...`. كل زوج يساوي `-1`. إذا كان `n` زوجيًا توجد `n / 2` أزواج. وإذا كان فرديًا نحصل على أزواج حتى `n - 1` ثم نطرح `n`.

```
#include <iostream>

int main() {
    long long n{};
    std::cin >> n;

    const long long answer = (n % 2 == 0)
        ? n / 2
        : -(n + 1) / 2;

    std::cout << answer << '\n';
}
```

لـ `-1 + 2 - 3 + 4 = 2` : `n = 4` . ولـ `n = 5` تصبح النتيجة `-3` . التعقيد `O(1)` . استخدم `long long` لأن القيود أكبر من مجال `int` . الحل البديل بحلقة صحيح للقيم الصغيرة لكنه يفشل زمنيًا مع حد كبير.

4A Watermelon

نريد تقسيم الوزن إلى جزأين زوجيين موجبين. لذلك يجب أن يكون الوزن زوجيًا، وأن يكون أكبر من 2 لأن `2 = 0 + 2` لا يعطي جزأين موجبين.

```
#include <iostream>

int main() {
    int weight{};
    std::cin >> weight;

    if (weight > 2 && weight % 2 == 0) {
        std::cout << "YES\n";
    } else {
        std::cout << "NO\n";
    }
}
```

اختبارات الحدود: `4 → YES` ، `3 → NO` ، `2 → NO` . التعقيد `O(1)` . الخطأ الأشهر هو الاكتفاء بالزوجية فيقبل البرنامج الوزن 2.

835A Key Races

زمن كل لاعب يساوي وقت الكتابة `s * v` مضافًا إليه زمن الذهاب والعودة `2 * t` . احسب الزمنين مرة واحدة ثم قارنهما.

```
#include <iostream>

int main() {
    long long s{}, v1{}, v2{}, t1{}, t2{};
    std::cin >> s >> v1 >> v2 >> t1 >> t2;

    const long long first = s * v1 + 2 * t1;
    const long long second = s * v2 + 2 * t2;

    if (first < second) {
        std::cout << "First\n";
    } else if (second < first) {
        std::cout << "Second\n";
    } else {
        std::cout << "Friendship\n";
    }
}
```

مثلًا `s=5, v1=1, v2=2, t1=1, t2=1` يعطي الزمنين 7 و12، فيفوز First. التعقيد `O(1)` . لا تكرر الصيغة داخل كل فرع لأن ذلك يزيد احتمال تعديل نسخة ونسيان الأخرى.

1173A Nauuo and Votes

لدينا أصوات موجبة `x` وسالبة `y` ومجهولة `z` . تكون `+` مضمونة إذا ظل `x` أكبر حتى بعد ذهاب كل المجهول إلى الطرف الآخر: `x > y + z` . وتكون `-` مضمونة بالعكس. التعادل مؤكد فقط إذا لم توجد أصوات مجهولة وكان الطرفان متساويين.

```cpp
#include <iostream>

int main() {
    long long x{}, y{}, z{};
    std::cin >> x >> y >> z;

    if (x > y + z) {
        std::cout << "+\n";
    } else if (y > x + z) {
        std::cout << "-\n";
    } else if (z == 0 && x == y) {
        std::cout << "0\n";
    } else {
        std::cout << "?\n";
    }
}
```

اختبر `3 2 10` و`3 10 2` و`0 5 5` و`1 5 5`. التعقيد `O(1)`. الخطأ الشائع هو اعتبار `x > y` كافيًا رغم أن الأصوات المجهولة قد تقلب النتيجة.

318A Even Odds

يحتوي الجزء الفردي `(n + 1) / 2` عنصرًا. الموضع الفردي `k` يعطي `2*k - 1`. إذا تجاوز `k` الجزء الفردي، نطرح حجمه ثم نحول الموضع المتبقي إلى العدد الزوجي المقابل.

```cpp
#include <iostream>

int main() {
    long long n{}, k{};
    std::cin >> n >> k;

    const long long oddCount = (n + 1) / 2;
    const long long answer = (k <= oddCount)
        ? 2 * k - 1
        : 2 * (k - oddCount);

    std::cout << answer << '\n';
}
```

لـ`n = 10` يصبح الترتيب `1 3 5 7 9 2 4 6 8 10`. اختبر `k=5` و`k=6` لأنهما نقطة الانتقال. التعقيد `O(1)` بدل بناء التسلسل في `O(n)`.

459A Pashmak and Garden

نريد نقطتين تكملان مربعًا موازيًا للمحاور. إذا تشارك المدخلان قيمة `x` فهما ضلع رأسي، وإذا تشاركا `y` فهما ضلع أفقي. وإلا يجب أن يكون الفرق المطلق في المحورين متساويًا ليكونا قطرًا صالحًا.

```cpp
#include <cstdlib>
#include <iostream>

int main() {
    int x1{}, y1{}, x2{}, y2{};
    std::cin >> x1 >> y1 >> x2 >> y2;

    if (x1 == x2 && y1 != y2) {
        const int side = std::abs(y1 - y2);
        std::cout << x1 + side << ' ' << y1 << ' '
                  << x2 + side << ' ' << y2 << '\n';
    } else if (y1 == y2 && x1 != x2) {
        const int side = std::abs(x1 - x2);
        std::cout << x1 << ' ' << y1 + side << ' '
                  << x2 << ' ' << y2 + side << '\n';
    } else if (std::abs(x1 - x2) == std::abs(y1 - y2)) {
        std::cout << x1 << ' ' << y2 << ' '
                  << x2 << ' ' << y1 << '\n';
    } else {
        std::cout << "-1\n";
    }
}
```

اختبر ضلعًا رأسيًا، ضلعًا أفقيًا، قطرًا صحيحًا، نقطتين متطابقتين، ومستطيلًا غير مربع. التعقيد `O(1)`. الخطأ الشائع هو قبول أي قطر دون مقارنة الفرقين المطلقين.

617A Elephant

يقطع الفيل من 1 إلى 5 وحدات في الخطوة. أقل عدد خطوات هو القسمة السقفية للمسافة على 5.

```cpp
#include <iostream>

int main() {
    int distance{};
    std::cin >> distance;
    std::cout << (distance + 4) / 5 << '\n';
}
```

للمسافة 12 نحتاج 3 خطوات: `5 + 5 + 2`. اختبر 1 و5 و6. التعقيد `O(1)`. الحلقة التي تطرح 5 صحيحة لكنها أطول وأقل تعبيرًا عن الصيغة.

581A Vasya the Hipster

يستخدم جوارب بلونين مختلفين مدة `min(a,b)`. بعد نفاد اللون الأقل، تُستخدم الجوارب المتبقية في أزواج من اللون نفسه.

```cpp
#include <algorithm>
#include <cstdlib>
#include <iostream>

int main() {
    int red{}, blue{};
    std::cin >> red >> blue;

    const int different = std::min(red, blue);
    const int same = std::abs(red - blue) / 2;
    std::cout << different << ' ' << same << '\n';
}
```

لـ `3 7` توجد 3 أيام مختلفة ثم يومان من اللون المتبقي. التعقيد `O(1)`. لا تجمع اللونين ثم تقسم؛ الأيام المختلفة تستهلك جوربًا من كل لون معًا.

281A Word Capitalization

المطلوب تغيير الحرف الأول فقط إلى Uppercase. تحقّق من أن النص غير فارغ، ومرر قيمة `unsigned char` إلى دوال `<cctype>` لتجنب سلوك غير معرّف مع القيم السالبة لـ `char`.

```
#include <cctype>
#include <iostream>
#include <string>

int main() {
    std::string word;
    std::cin >> word;

    if (!word.empty()) {
        const auto first = static_cast<unsigned char>(word.front());
        word.front() = static_cast<char>(std::toupper(first));
    }

    std::cout << word << '\n';
}
```

`hello` تصبح `Hello`، و`World` تبقى كما هي. التعقيد `O(1)` بالنسبة للعمل المطلوب بعد القراءة. المثال يتعامل مع محارف المسألة اللاتينية؛ لا يمثل حلًا عامًا لتحويل Unicode.

1A Theatre Square

نحسب عدد البلاطات على كل بُعد بقسمة سقفية، ثم نضرب العددين. يجب توسيع الحساب لأن الناتج قد يتجاوز `int`.

```
#include <iostream>

int main() {
    long long n{}, m{}, a{};
    std::cin >> n >> m >> a;

    const long long rows = (n + a - 1) / a;
    const long long columns = (m + a - 1) / a;
    std::cout << rows * columns << '\n';
}
```

لساحة `6 × 6` وبلاطة ضلعها 4 نحتاج `2 × 2 = 4` بلاطات. التعقيد `O(1)`. الخطأ الشائع هو استخدام القسمة الصحيحة مباشرة أو تخزين حاصل الضرب في `int`.

لكل مسألة اكتب القيود والتعقيد وحالة حدية قبل الكود. الحل القصير لا يعفيك من شرح سبب الصيغة.

استخراج الصيغة من القيود

إذا سمحت القيود بقيمة `n` تصل إلى `10^18` فلن تنجح حلقة حتى `n`، ويصبح وجود صيغة `O(1)` أو خوارزمية لوغاريتمية ضرورة. وإذا كان الناتج حاصل ضرب بعدين كبيرين، استخدم `long long` قبل الضرب لا بعده.

اختبار الانتقال بين الحالات

أهم الاختبارات تقع عند النقطة التي يغير عندها البرنامج الفرع. في Even Odds اختبر آخر موضع فردي وأول موضع زوجي. وفي Watermelon اختبر `2` و`3` و`4`. وفي قسمة سقفية اختبر عددًا يقبل القسمة تمامًا وعددًا يزيد عنه بواحد.

لا تحفظ كود الحل. اكتب سبب كل حالة والمعادلة التي تربط الموضع بالقيمة. إذا عجزت عن شرحها على مثال صغير، فلن يكشف لك الـCompiler الخطأ الرياضي.

تأكد من فهمك

01 **اشتق صيغة الجزء الفردي في Even Odds.**

الإجابة المشروحة: عدد الفرديات حتى n هو (n+1)/2. إذا k داخله فالقيمة ذات الموضع k هي 2k-1.

02 **لماذا نستخدم long long في مسائل تبدو مدخلاتها int؟**

الإجابة المشروحة: قد يكون كل مدخل داخل int لكن ضربها أو جمعها يتجاوز المجال؛ نوع التعبير الوسيط هو المهم.

03 **كيف تعرف أن الحل الرياضي أفضل من المحاكاة؟**

الإجابة المشروحة: إذا أمكن حساب الموضع أو المجموع بصيغة ثابتة، تصبح O(1) بدل تكرار قد يتجاوز الزمن.

04 **ما اختبار حدّي يكشف خطأ 0-based/1-based؟**

الإجابة المشروحة: اختبر k=1، وآخر موضع في الجزء الأول، وأول موضع في الجزء الثاني، وk=n.

05 **لماذا الشرط `w % 2 == 0` وحده لا يحل Watermelon؟**

الإجابة المشروحة: لأن 2 زوجي لكنه لا ينقسم إلى عددين زوجيين موجبين. الشرط الكامل هو `w > 2 && w % 2 == 0`.

06 **في Even Odds، ما آخر موضع فردي وما أول موضع زوجي عندما `n = 7`؟**

الإجابة المشروحة: عدد الفرديات `(7 + 1) / 2 = 4`؛ الموضع 4 يحمل 7، والموضع 5 هو أول الزوجيات ويحمل 2.

07 **لماذا تحتاج Theatre Square إلى `long long` حتى لو كان كل بُعد يقبل int؟**

الإجابة المشروحة: لأن حاصل ضرب عددي البلاطات على المحورين قد يتجاوز int. يجب أن تكون القيم الوسيطة والضرب من النوع الواسع، لا المتغير النهائي فقط.

08 **ما الحالات الهندسية الثلاث التي يجب فصلها في Pashmak and Garden؟**

الإجابة المشروحة: ضلع رأسي، ضلع أفقي، أو قطر مائل 45 درجة حيث `abs(dx) == abs(dy)` . أي حالة مائلة أخرى مستحيلة.

اربط النقاط ببعض

حوّل sample tests إلى اختبار آلي، ثم أضف الحدود والانتقال بين الحالات وحالات عشوائية تقارن بصيغة بطيئة موثوقة. سجّل مصدر المسألة وقيودها لأن صحة الصيغة تعتمد على تلك القيود، ولا تعتمد على نجاح العينات كبرهان.

جرّب بنفسك

اختر مسألة واختبر كل نقطة يتغير عندها branch أو formula.

1. تأسيس C++ والكمبيوتر

الأساس المنطقي وأنظمة الأعداد والأدوات والترجمة والأنواع والمتغيرات والمعاملات والإدخال.

تشرح هذه المجموعة التفكير قبل الكود، وتمثيل الأعداد داخل الكمبيوتر، وأدوات C++، والأنواع والمتغيرات والمعاملات والإدخال. اتبع ترتيب الدروس لأن كل درس يستخدم المصطلحات التي شُرحت قبله.

2. الشروط والتحقق والمشروعات

قواعد العمل ومسار البناء والمعاملات والشروط وswitch ومشروعان كاملان.

تربط هذه المجموعة بين المدخلات وقواعد العمل ومراحل بناء البرنامج والشروط و`switch`. ثم تطبق المفاهيم في مشروع طلب إلكتروني ونظام تسجيل لمركز تدريبي.

3. الترميز والحلقات

ترميز النصوص والتحويلات والحلقات والعدادات والمجاميع وأنماط التكرار.

تشرح هذه المجموعة ترميز النصوص والتحويلات، ثم تنتقل إلى الحلقات والعدادات والمجمّعات والحلقات المتداخلة. تدريب قراءة المسائل وقيودها بقى موجود داخل درس حل المشكلات.

03

المسار · 18 درسًا

لغة PHP

شرح متدرج، أمثلة عملية، وتصحيحات تساعدك على الانتقال من الفكرة إلى التطبيق.

PHP

مسار شامل يبدأ بأساسيات PHP ويتدرج إلى بناء برامج ويب قابلة للفهم والتجربة.

المسار ده يبدأ معاك من أساسيات PHP، وبعدها يبني كل فكرة على اللي قبلها لحد ما تقدر تقرأ برنامج ويب وتفهم أجزاؤه. كل درس فيه أمثلة عملية وحالات شائعة وأخطاء لازم تعرف سببها وطريقة إصلاحها.

خريطة الدروس

1. مقدمة وصياغة PHP — ما PHP؟ وكيف تعمل؟ والوسوم والتعليقات والدمج مع HTML.
2. المتغيرات والنطاق وSuperglobals — الإسناد بالقيمة والمرجع والمتغيرات المتغيرة ونطاق المتغير.
3. أنواع البيانات ونظام الأنواع — الأنواع الأساسية والمركبة و`callable` و`iterable` وUnion/Intersection والكتابة الصارمة.
4. الإخراج والتصحيح والثوابت — `echo` و`print` و`var_dump` و`const` و`define`.
5. الشروط والحلقات — تتبع القرارات والحلقات خطوة خطوة، حدود القيم وcontinue وbreak والمراجع.
6. التعبيرات والمؤثرات — الأولوية والمقارنة والمنطق والمصفوفات والتنفيذ وPipe Operator.
7. الدوال والـ Callbacks وتضمين الملفات — Call Stack وRecursion وCallbacks ومشروع ملفين مع require.
8. الملفات والـ Streams وJSON وCSV — أوضاع `fopen` والصلاحيات والـ wrappers والـ contexts والفلاتر.
9. رفع الملفات والكوكيز والجلسات — رفع آمن، إعدادات Cookies، ودورة حياة Session.
10. Namespaces وComposer وأدوات الجودة — مشروع PSR-4 كامل، lock وSemVer وscripts وaudit وPHPUnit وPHPStan وPHP-CS-Fixer.
11. الأخطاء والاستثناءات — Throwable والاستثناءات المخصصة وحدود المعالجة.
12. المصفوفات وأدوات التحويل — map وfilter وreduce والفرز والذاكرة.
13. النصوص وUnicode وUTF-8 — Regex وmbstring والتطبيع والـpatterns الآمنة.
14. التاريخ والوقت — UTC وDateTimeImmutable وtimezones وDST.
15. Request وRouter وResponse — Front Controller وJSON وMiddleware.
16. خريطة ميزات PHP الحديثة — PHP 8.0–8.5 وWeakMap وFibers وDNF وReadonly وHooks.
17. برامج مترابطة وDebugging عملي — دفتر ملاحظات كامل يربط Form وValidation وSession وFiles وRouter، مع Xdebug واختبارات.

طريقة المذاكرة

نفّذ كل مثال، ثم غيّر المدخلات وتوقع النتيجة قبل التشغيل. الأمثلة الأمنية ليست إضافات اختيارية؛ هي جزء من الاستخدام الصحيح لـ PHP.

أول برنامج

```php
<?php

declare(strict_types=1);

function greet(string $name): string
{
    return "مرحبًا {$name}";
}

echo greet('عمر');
```

> الأمثلة مكتوبة لـ PHP 8.x، وأي ميزة تحتاج إصدارًا أحدث مذكور إصدارها بجوارها.

ابدأ بتجهيز المختبرات والأمثلة القابلة للتنزيل.

مشروع يتطور معك

ابدأ بنموذج الاسم في الدرس 9، وافهم مشروع Composer في 10، وتتبع API في 15، ثم ابنِ دفتر الملاحظات في 17. حمّل الملفات الكاملة: برامج الدروس، نموذج الجلسة والرفع، مشروع Composer مع lock واختبارات، ودفتر الملاحظات. الدروس ما زالت 17 درسًا مرقمًا، بالإضافة لصفحة تجهيز المختبر؛ السيدبار والأعداد تُحسب من الملفات تلقائيًا.

تجهيز المختبرات وتشغيل الأمثلة

.PHP labs, fixtures, reproducible tests and expected results

هنا هتجهز ملفات تقدر تشغّلها فعلًا، وتعرف الفرق بين اختبار دالة محلي واختبار خدمة كاملة. ابدأ بأساسيات PHP لو لسه مش عارف المتغيرات والدوال؛ ارجع للملفات بالتدريج مع كل درس.

التجهيز

نزّل ملفات المختبر وفك الضغط، ثم افتح الطرفية داخل مجلد `php-labs`. لو بتستخدم نسخة المشروع، الملفات في `examples/php-labs`. ثبّت PHP 8.4 أو أحدث، ثم افحص:

```
php -v
php -m
php tests.php
```

المختبر الأساسي يحتاج `pdo_sqlite` و`sodium` و`mbstring`، واختبار HTTP يحتاج `curl`. مافيش مكتبات خارجية مطلوبة للاختبارات الأساسية. PHP 8.5 مطلوب للأمثلة المكتوب عليها 8.5 تحديدًا في درس الميزات الحديثة. لو ظهر "could not find driver"، فعّل `pdo_sqlite` في ملف الإعداد الذي يعرضه `php --ini` لنفس PHP اللي بتشغله.

الناتج مجموعة سطور `PASS`، وفي النهاية `failures=0`. كود الخروج 0 يعني نجاح؛ أي إخفاق يرجع 1. الاختبارات تستخدم استثناءات صريحة، مش `assert` ممكن يتعطل من إعدادات PHP. لقراية كود الخروج استخدم `$LASTEXITCODE` في PowerShell أو `echo $?` في Bash.

اختار الاختبار المناسب

الأمر	ماذا يختبر؟
`php tests.php values`	صيغة المبالغ وحدود الجمع والتحويل وقوائم المعرّفات
`php tests.php php`	Attributes وCSV والمراجع وUnicode والتاريخ
`php tests.php security`	سياسة Claims بعد التوقيع وCSRF والسياق المشفّر وسياسة الـhash
`php tests.php inventory`	معاملة شراء وتكرار الطلب وrollback والقيود
`php locking-lab.php`	عمليتان تتنافسان على نفس مخزون SQLite
`php total.php 12.50 3.25`	جمع يطبع `15.75`
`php orders.php fixtures/orders.json`	قراءة طلبات JSON صحيحة مع حدود المدخلات
`php stream-lab.php fixtures/large.csv`	قراءة 10000 سجل CSV تدريجيًا
`php orm-query-lab.php`	مقارنة 21 استعلامًا باستعلامين لتحميل العلاقات
`php restore-verify.php`	إنشاء لقطة SQLite وفتحها والتحقق من قيودها وبياناتها

قاعدة SQLite في الذاكرة بتبدأ من جديد كل تشغيل. اختبار التزامن والاستعادة ينشئ ملفًا مؤقتًا خاصًا به ويحذفه بعد الانتهاء؛ مابيتصلش بقاعدة مشروعك.

HTTP حقيقي على جهازك

في طرفية أولى، من مجلد المختبر:

```
php -S 127.0.0.1:8097 http-router.php
```

وفي طرفية ثانية من نفس المجلد:

```
php http-client-lab.php
```

المتوقع `status=200` و`server_error=500` و`conditional=304` و`timeout_handled=yes`. افتح `http-router.php` لفهم كل مسار، و`src/Http.php` لفهم تحليل ETag. أوقف خادم PHP بـCtrl+C. ده خادم تطوير محلي، والاختبار لا يقيس FPM أو HTTPS أو حمل الإنتاج.

اقرأ الملفات بالترتيب

`bootstrap.php` يحمل الدوال بمسارات تبدأ من `__DIR__` ، وهو مجلد الملف نفسه؛ كده الاستدعاء مايعتمدش على مكان الطرفية. `src/Values.php` لقواعد القيم، و `schema.sql` للجداول وبيانات البداية، و `src/Inventory.php` للمعاملة، و `tests.php` للأمثلة المتوقعة والفاشلة.

الـfixture ملف بمدخلات ثابتة معروفة علشان تقدر تكرر التجربة. `fixtures/broken.json` خطأ متعمد: شغّل مستورد الطلبات عليه وتوقع رفض صيغة JSON، مش نجاحًا بلا بيانات. غيّر نسخة من المدخلات، لا النتيجة المتوقعة، وشوف أي قاعدة انكسرت.

حدود الدليل العملي

`cache-lab.php` يشرح ترتيب أحداث السباق داخل نموذج محلي؛ لا يشغل `profiling-lab.php` Redis. يحسب percentiles لعينة ثابتة؛ لا يرسل traces. اختبارات JWT هنا تفحص claims بعد التحقق، ولا تنفذ بروتوكول هوية كاملًا. نجاح SQLite لا يثبت درجات العزل أو deadlocks في PostgreSQL/MySQL. أكمل تجارب التشغيل الخاصة بكل خدمة في بيئة اختبار معزولة بالمحرك والإعداد الحقيقيين.

1. مقدمة وصياغة PHP

بداية هادية من الصفر: يعني إيه PHP، الكود بيتشغّل فين، وإزاي تكتب وتشغّل أول برنامج وتدمجه مع HTML بأمان.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Runtime:** وقت التشغيل: الفترة اللي البرنامج بيكون شغال فيها فعلًا.
- **HTTP:** قواعد تبادل الطلبات والردود بين المتصفح والخادم.
- **URL:** العنوان الكامل لمورد على الويب، زي صفحة أو صورة أو نقطة API.
- **API:** واجهة محددة تسمح لبرنامج يطلب بيانات أو ينفّذ عملية عند برنامج آخر.
- **Queue:** طابور مهام تنتظر عاملًا ينفذها في الخلفية.
- **CLI:** واجهة تتعامل معها بكتابة أوامر نصية بدل الضغط على أزرار.
- **UTF-8:** طريقة شائعة لتحويل أرقام Unicode إلى بايتات تُحفظ وتُنقل.
- **Scope:** النطاق: المكان اللي يقدر الكود داخله يشوف اسمًا أو متغيرًا.

هنفهم إيه في الدرس ده؟

قبل ما نحفظ أوامر، محتاجين نفهم الصورة الكبيرة. لما تفتح موقع وتضغط على زر «تسجيل الدخول»، المتصفح بيبعت طلب للسيرفر. السيرفر محتاج برنامج يستقبل البيانات، يتأكد منها، يكلم قاعدة البيانات، وبعد كده يرجّع نتيجة. **PHP واحدة من اللغات اللي ممكن تكتب بيها البرنامج ده.**

في نهاية الدرس هتكون فاهم:

- PHP بتشتغل فين، والفرق بينها وبين HTML وJavaScript.
- إزاي تشغّل ملف PHP من الـTerminal أو من خلال متصفح.
- شكل تعليمات PHP، وإمتى نكتب `;`.
- إزاي نخلط PHP مع HTML من غير ما نعرّض الصفحة لمشكلة أمنية.
- إزاي تقرأ رسالة الخطأ بدل ما تعتبرها حاجة مخيفة.

يعني إيه PHP؟

PHP لغة برمجة عامة الاستخدام، لكنها مشهورة جدًا في تطوير الـBackend، يعني الجزء اللي بيشتغل على السيرفر. الاسم بدأ تاريخيًا بـ**Personal Home Page**، وبعد كده بقى الاسم الرسمي المتكرر **PHP: Hypertext Preprocessor**.

خلّينا نفرّق بين ثلاث حاجات:

- **HTML** بيصف شكل ومحتوى الصفحة اللي المتصفح هيعرضها.
- **JavaScript في المتصفح** يضيف تفاعل بعد وصول الصفحة للمستخدم.
- **PHP** غالبًا بتشتغل على السيرفر قبل ما الاستجابة توصل للمتصفح.

مثلًا لو عندنا الكود ده على السيرفر:

```
<?php
$name = 'Omar';
echo "Hello {$name}";
```

المتصفح مش بيستلم `$name` ولا `echo` . هو بيستلم الناتج فقط:

```
Hello Omar
```

دي نقطة مهمة جدًا: **كود PHP نفسه المفروض يفضل على السيرفر**. لو ملف المصدر أو أسرار الاتصال بقاعدة البيانات وصلوا للمتصفح، فدي مشكلة في إعداد السيرفر وليست طريقة عمل PHP الطبيعية.

الطلب بيمشي إزاي؟

تخيّل إنك فتحت الرابط `/products.php` :

1. المتصفح يبعت HTTP Request.
2. Web Server زي Nginx أو Apache يستقبل الطلب.
3. لو المطلوب ملف PHP ديناميكي، السيرفر يمرره إلى PHP Runtime.
4. PHP تقرأ الكود وتنفذه. ممكن أثناء التنفيذ تقرأ ملفًا أو تكلم قاعدة بيانات أو API.
5. PHP تنتج Body ومعاه Headers وStatus Code.
6. Web Server يرجع HTTP Response للمتصفح.
7. المتصفح يعرض HTML أو يتعامل مع JSON حسب نوع الاستجابة.

```
Browser → HTTP Request → Web Server → PHP → Database/File/API
Browser ← HTTP Response ← Web Server ← Result
```

مش لازم كل طلب يكلم قاعدة بيانات، ومش لازم الناتج يكون HTML. PHP ممكن ترجع JSON لتطبيق موبايل، أو تنشئ ملفًا، أو تشتغل من غير متصفح أصلًا عن طريق سطر الأوامر.

هل PHP «بتتفسر سطرًا سطرًا»؟

الجملة دي مفيدة كتقريب أولي لكنها مش دقيقة تقنيًا. محرك PHP يحوّل الكود إلى Opcodes ثم ينفذها، وOPcache ممكن يحتفظ بالنتيجة دي، وفي الإصدارات الحديثة توجد JIT. اللي يهمك الآن إنك في التطوير المعتاد تعدّل الملف وتشغّله من غير Build تقليدي بعد كل تعديل.

PHP بتستخدم في إيه؟

تقدر تستخدمها في:

- استقبال Forms والتحقق من البيانات.
- إنشاء صفحات ديناميكية وREST APIs.
- قراءة وكتابة قواعد البيانات والملفات.
- إدارة Sessions وتسجيل الدخول والصلاحيات.
- تشغيل أوامر CLI وJobs مجدولة وQueue Workers.
- إرسال بريد أو التعامل مع JSON وخدمات خارجية.
- إنشاء صور أو PDFs عند تثبيت المكتبات المناسبة.
- كتابة كود Procedural أو Object-Oriented.

ممكن بأدوات خارجية تبني Desktop Apps، لكن ده مش الاستخدام الأشهر ولا أول اختيار منطقي للغة.

جهّز البيئة واتأكد إن PHP شغالة

ممكن تثبّت PHP لوحدها، أو تستخدم حزمة جاهزة:

- **XAMPP:** متاح لأكثر من نظام.
- **WAMP:** شائع على Windows.
- **MAMP:** شائع على macOS.
- **LAMP:** وصف شائع لـ Linux + Apache + MySQL/MariaDB + PHP.

افتح Terminal واكتب:

```
php --version
```

لو التثبيت وPATH مضبوطين، هتشوف رقم الإصدار. لو ظهر إن الأمر غير معروف، فده معناه غالبًا إن PHP مش متثبتة أو إن مكان الملف التنفيذي مش مضاف إلى `PATH`.

أول برنامج من سطر الأوامر

أنشئ ملفًا اسمه `hello.php`:

```
<?php

$name = 'Omar';
$lessonCount = 1;

echo "Hello {$name}!", PHP_EOL;
echo "You finished lesson {$lessonCount}.", PHP_EOL;
```

شغّله:

```
php hello.php
```

الناتج المتوقع:

```
Hello Omar!
You finished lesson 1.
```

نفهم المثال بهدوء:

- `<?php` بتقول إن اللي بعدها كود PHP.
- `$name` متغير خزّنا فيه نصًا.
- `$lessonCount` متغير خزّنا فيه عددًا.
- `echo` بتخرج قيمة.
- `{}` حول اسم المتغير بتخلي حدوده واضحة داخل النص.
- `PHP_EOL` معناها نهاية سطر مناسبة لنظام التشغيل.
- `;` بتنهي التعليمة البسيطة.

في ملف PHP خالص، الأفضل غالبًا ما تكتبش وسم الإغلاق `?>`. ده يمنع مسافات غير مقصودة بعده من إنها تخرج قبل الـHeaders.

تشغيل PHP في المتصفح

اعمل مجلدًا اسمه `public` وحط جواه `index.php`:

```
<?php

echo '<h1>My first PHP page</h1>';
echo '<p>The server generated this HTML.</p>';
```

من المجلد اللي يحتوي `public` شغّل:

```
php -S localhost:8000 -t public
```

وبعد كده افتح:

```
http://localhost:8000
```

الخادم المدمج مناسب للتعلم والتطوير المحلي فقط. مش معمول علشان يتحط مباشرة على الإنترنت كخادم إنتاج.

تعليمات PHP والـBlocks

التعليمة البسيطة غالبًا بتنتهي بـ `;`:

```
$price = 150;
$quantity = 2;
$total = $price * $quantity;
echo $total;
```

لكن الـBlock نفسه لا نضع بعد قوسه الأخير فاصلة منقوطة:

```
if ($total >= 300) {
    echo 'Free shipping';
} else {
    echo 'Shipping fee applies';
}
```

لو نسيت `;`، ممكن رسالة الخطأ تشير للسطر التالي، لأن PHP استمرت في القراءة لحد ما اكتشفت إن تركيب الجملة لم يعد ممكنًا. لذلك بص كمان على السطر اللي قبل مكان الخطأ.

دمج PHP مع HTML

PHP اتصممت من البداية بحيث تقدر تدخل وتخرج من وضع PHP داخل قالب HTML:

```php
<?php
$title = 'متجري';
$products = ['Keyboard', 'Mouse', 'Monitor'];
?>
<!doctype html>
<html lang="ar" dir="rtl">
<head>
    <meta charset="utf-8">
    <title><?= htmlspecialchars($title, ENT_QUOTES, 'UTF-8') ?></title>
</head>
<body>
    <h1><?= htmlspecialchars($title, ENT_QUOTES, 'UTF-8') ?></h1>

    <?php if ($products === []): ?>
        <p>مفيش منتجات دلوقتي.</p>
    <?php else: ?>
        <ul>
            <?php foreach ($products as $product): ?>
                <li><?= htmlspecialchars($product, ENT_QUOTES, 'UTF-8') ?></li>
            <?php endforeach; ?>
        </ul>
    <?php endif; ?>
</body>
</html>
```

الاختصار `<?= $value ?>` معناه «اطبع القيمة». والصياغة `if: ... endif;` و `foreach: ... endforeach;` بتكون أوضح داخل HTML من كثرة الأقواس.

استخدمنا `htmlspecialchars()` لأن أي قيمة ممكن يظهر فيها نص غير موثوق لازم تتحول قبل وضعها في HTML. لو اسم المنتج كان يحتوي `<script>`، إحنا عايزين نعرضه كنص، مش نخلي المتصفح ينفذه ككود.

> ⚠
>
> **مش كل Escape يصلح لكل مكان**
>
> `htmlspecialchars()` مناسب غالبًا للنص داخل HTML. لكن URL وJavaScript وCSS لهم سياقات مختلفة. القاعدة هي: اعمل Encoding وقت الإخراج وبالطريقة المناسبة للمكان اللي هتحط فيه القيمة.

الـHeaders لازم تخرج قبل الـBody

الـHTTP Response فيها Headers ثم Body. علشان كده لازم تستدعي `header()` و `setcookie()` و `session_start()` قبل ما تطبع HTML أو نصًا:

```php
<?php

header('Content-Type: application/json; charset=utf-8');

$response = ['status' => 'ok'];
echo json_encode($response, JSON_UNESCAPED_UNICODE | JSON_THROW_ON_ERROR);
```

الناتج:

```
{"status":"ok"}
```

لو طبعت حاجة قبل `header()`، ممكن تقابل رسالة `headers already sent`. وقتها دور على `echo` أو HTML أو حتى مسافة خرجت قبل `<?php`.

التعليقات وحساسية الحروف

التعليق نص PHP بيتجاهله، أما قواعد كتابة الكود فاسمها Syntax. المتغير اسم بنوصل بيه لقيمة. Composer بيدير مكتبات PHP، وOPcache بيحفظ التعليمات المترجمة، وNginx بيستقبل طلبات الويب ويمرر شغل PHP لمشغّلها؛ هنشرح الأدوات دي في مسار التشغيل.

```
// تعليق لسطر واحد

# تعليق لسطر واحد، لكنه أقل استخدامًا
/*
   تعليق لأكثر من سطر
*/

$userName = 'Omar';
echo $userName;
// echo $username; // متغير مختلف لأن حالة الحروف مختلفة
```

أسماء المتغيرات Case-sensitive، يعني `$userName` غير `$username`. كلمات اللغة مثل `if` لا تعتمد عمليًا على حالة الحروف، وأسماء الدوال والأصناف تُطابق من غير حساسية، لكن ما تعتمدش على ده: اكتب الاسم بنفس الطريقة اللي اتعرّف بها والتزم بمعايير PSR علشان الكود يفضل واضحًا.

التعليق الجيد يشرح **سبب القرار**، مش يعيد قراءة الكود:

```
// Keep money in cents to avoid binary floating-point rounding.
$priceCents = 1999;
```

مثال كامل صغير

البرنامج ده يقرأ اسمًا من Query String ويعرض ترحيبًا آمنًا. شغّله بالخادم المدمج وافتح `http://localhost:8000/?name=Omar`:

```php
<?php

$rawName = $_GET['name'] ?? 'Guest';
$name = is_string($rawName) ? trim($rawName) : 'Guest';

if ($name === '') {
    $name = 'Guest';
}

$safeName = htmlspecialchars($name, ENT_QUOTES, 'UTF-8');
?>
<!doctype html>
<html lang="en">
<head><meta charset="utf-8"><title>Welcome</title></head>
<body>
    <h1>Hello <?= $safeName ?></h1>
</body>
</html>
```

غيّر الرابط إلى `?name=<b>Omar</b>` . المفروض تشوف العلامات كنص، مش اسمًا عريضًا؛ وده يثبت إن الإخراج اتعمل له Encoding.

أخطاء شائعة في أول يوم

- فتحت ملف PHP بالنقر عليه فظهر الكود أو صفحة فارغة: شغّله من PHP CLI أو من Web Server، مش كملف عادي.
- الأمر `php` غير معروف: راجع التثبيت و `PATH` .
- ظهر `Parse error` : اقرأ رقم الملف والسطر وراجع السطر السابق أيضًا.
- ظهر `Undefined variable` : الاسم مكتوب غلط أو المتغير لم يأخذ قيمة قبل الاستخدام.
- ظهر `headers already sent` : خرج Body قبل تعديل Headers.
- الصفحة تعرض مدخل المستخدم مباشرة: استخدم Encoding مناسب وقت الإخراج.

خريطة رحلة أول برنامج

تأكد من فهمك

01 **المتصفح طلب ملف PHP. هل يستلم الكود نفسه؟ اشرح الرحلة باختصار.**

الإجابة: لا. Web Server يمرر الملف إلى PHP، وPHP تنفذه وترجع الناتج. المتصفح يستلم Response فيها HTML أو JSON أو نوع محتوى آخر، وليس كود المصدر.

02 **إيه الناتج المتوقع من `echo 'PHP', ' ', 8;` ؟**

الإجابة: الناتج هو `PHP 8` . `echo` تقدر تستقبل أكثر من قيمة مفصولة بفواصل وتخرجهم بالترتيب.

03 **الكود بيقول إن الخطأ في السطر 8، لكن السطر شكله سليم. فين تبص بعد كده؟**

الإجابة: راجع السطر السابق؛ ممكن يكون ناقصه فاصلة منقوطة أو علامة اقتباس أو قوس، ولم يكتشف المحرك استحالة التركيب إلا عند السطر 8.

04 **ليه ماينفعش نكتب `echo $_GET['name'];` مباشرة داخل HTML؟**

الإجابة: لأن المستخدم يقدر يرسل HTML أو JavaScript. لازم نتحقق من المدخل حسب قواعدنا، وعند عرضه كنص نستخدم `htmlspecialchars()` بالسياق المناسب.

05 **صلّح الكود: `$name = 'Omar' echo $name;`**

الإجابة: أضف `;` بعد الإسناد: `$name = 'Omar'; echo $name;` . كل واحدة تعليمة مستقلة.

06 **مهمة تطبيقية: اعمل صفحة تستقبل `product` و `price` من الرابط وتعرضهما بأمان.**

خط الحل: اقرأ القيم باستخدام `??` لقيمة افتراضية، تحقق أن السعر رقم صالح، وحوّل اسم المنتج باستخدام `htmlspecialchars()` وقت عرضه. جرّب قيمة فارغة وسعرًا غير رقمي واسمًا يحتوي علامات HTML.

ملخص الدرس

PHP بتشتغل غالبًا على السيرفر، والمتصفح بياخد الناتج فقط. تقدر تشغلها من CLI أو Web Server محلي. ابدأ كل ملف بوسم PHP، أنهِ التعليمات البسيطة بـ `;` ، واعمل Encoding لأي بيانات غير موثوقة وقت عرضها. في الدرس الجاي هنفهم المتغيرات بشكل أعمق: القيمة بتتخزن إزاي، وإيه معنى Scope، وإزاي بيانات الطلب بتدخل البرنامج.

شغّل وتحقق

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

نفّذ نقطة التحقق التالية داخل بيئة الدرس:

```
php hello.php
```

معيار النجاح: يطبع البرنامج السطر المتوقع وينتهي بكود 0؛ ثم أضف خطأ syntax متعمدًا مرة واحدة وتأكد أن الرسالة تحدد الملف والسطر.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

اربط النقاط ببعض

لمنع حمل زائد، ادرس الدرس في مسارين: CLI أولًا لفهم syntax وexit code، ثم Web لفهم headers وbody. Composer ليس مطلوبًا لأول ملف، لكنه يصبح نقطة بدء المشروع الحقيقي عبر autoload واعتماديات مثبتة. لا تخلط بين PHP built-in server وبيئة إنتاج.

جرّب بنفسك

شغّل المثال في CLI ثم Web وحدد اختلاف input/output والعملية المستضيفة.

2. المتغيرات والنطاق وSuperglobals

شرح من الصفر للمتغيرات والإسناد بالقيمة والمرجع والنطاق، وإزاي بيانات الطلب بتوصل للبرنامج عن طريق Superglobals.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **HTTP:** قواعد تبادل الطلبات والردود بين المتصفح والخادم.
- **Cache:** نسخة مؤقتة من البيانات هدفها تقليل وقت الانتظار والعمل المتكرر.
- **Session:** بيانات مؤقتة تساعد الخادم يميّز المستخدم بين أكثر من طلب.
- **UTF-8:** طريقة شائعة لتحويل أرقام Unicode إلى بايتات تُحفظ وتُنقل.
- **Scope:** النطاق: المكان اللي يقدر الكود داخله يشوف اسمًا أو متغيرًا.
- **Function:** دالة: جزء كود له اسم ومهمة محددة ويمكن استدعاؤه أكثر من مرة.

ليه بنحتاج متغيرات؟

البرنامج بيتعامل مع قيم بتتغير: اسم المستخدم، سعر المنتج، عدد القطع، أو نتيجة حساب. بدل ما نكرر القيمة في كل مكان، بنديها اسمًا واضحًا ونخزنها في **متغير**.

فكّر في المتغير كاسم بنستخدمه علشان نوصل لقيمة أثناء تشغيل البرنامج. التشبيه بالصندوق مفيد في البداية، لكن خلي بالك إن طريقة التخزين الحقيقية تعتمد على نوع القيمة وإدارة الذاكرة داخل PHP.

```
<?php

$productName = 'Keyboard';
$priceCents = 150000;
$inStock = true;

echo $productName, PHP_EOL;
echo $priceCents, PHP_EOL;
```

الجزء `$` بيقول لـPHP إن ده اسم متغير. علامة `=` هنا مش معناها «يساوي» في الرياضيات؛ معناها **خد القيمة اللي على اليمين واسندها للاسم اللي على الشمال**.

قواعد تسمية المتغير

بعد `$` لازم الاسم يبدأ بحرف أو underscore، وبعد كده ممكن يحتوي أرقامًا:

```
$name = 'Omar';       // صحيح
$_count = 3;          // صحيح
$price2 = 19.5;       // صحيح
// $2price = 19.5;    // خطأ: بدأ برقم
```

الأسماء Case-sensitive:

```
$userName = 'Omar';
$username = 'Ali';

echo $userName; // Omar
echo $username; // Ali
```

اختار اسمًا يشرح المعنى. `$priceCents` أوضح من `$p`، و`$isEmailVerified` أوضح من `$flag`. الاسم الطويل المعقول أرخص من وقت تضيعُه وأنت بتحاول تفهم الكود بعد شهر.

المتغير بيأخذ نوعه من القيمة

PHP لغة Dynamically Typed، يعني مش لازم تكتب نوع المتغير وقت إنشائه:

```
$value = 10;       // int
$value = 'ten';    // string بعد الإسناد الجديد
```

ده مش معناه إن PHP «من غير أنواع». كل قيمة لها نوع، وPHP بتطبّق قواعد تحويل ومقارنة حسب النوع. هنشرح الأنواع بالتفصيل في الدرس التالي، وبعدها هنستخدم Type Declarations علشان نخلي حدود الدوال أوضح.

تقدر تشوف النوع والقيمة أثناء التعلم أو التصحيح:

```
$price = 19.5;

var_dump($price);            // float(19.5)
echo get_debug_type($price); // float
```

طباعة المتغير داخل النص

الـDouble Quotes تسمح باستبدال المتغير، أما Single Quotes فتعرض النص كما هو في أغلب الحالات:

```
$name = 'Omar';
$price = 20;

echo "Hello {$name}, the price is {$price} pounds.", PHP_EOL;
echo 'Hello $name', PHP_EOL;
```

الناتج:

```
Hello Omar, the price is 20 pounds.
Hello $name
```

استخدام `{$name}` بيحدد بداية الاسم ونهايته بوضوح. الصيغة القديمة `${name}` داخل النصوص Deprecated، فما تعتمدش عليها.

الإسناد بالقيمة

شوف المثال ده وتوقع الناتج قبل ما تشغله:

```
$a = 10;
$b = $a;
$b++;

echo $a, PHP_EOL;
echo $b, PHP_EOL;
```

الناتج:

```
10
11
```

وقت `$b = $a` أخذت `$b` قيمة مستقلة منطقيًا. تعديل `$b` بعد كده لم يغير `$a`.

بالنسبة للمصفوفات والنصوص، PHP تستخدم داخليًا تحسينًا اسمه **Copy-on-write**: مش لازم تنسخ كل البايتات فورًا، لكنها تتصرف قدامك كأن القيم مستقلة، وتعمل النسخة الفعلية عند التعديل. المهم كمبرمج مبتدئ هو السلوك، مش تفاصيل التحسين.

الإسناد بالمرجع

علامة `&` تخلي اسمين مرتبطين بنفس الحاوية المنطقية:

```
$score = 10;
$alias =& $score;

$alias++;

echo $score; // 11
```

لما عدّلنا `$alias` اتغيرت `$score` لأنها مرتبطة بها بالمرجع.

```
unset($alias);
echo $score; // 11 ما زالت
```

`unset($alias)` فك الاسم `$alias`، لكنه ما حذفش القيمة من `$score`.

المرجع في PHP مش Pointer يدوي زي C، ومش محتاج تستخدمه في أغلب الكود. استخدامه من غير داعي يخلي السؤال «مين غيّر القيمة؟» أصعب جدًا. ابدأ بالإسناد العادي، واستخدم المرجع فقط لما تكون فاهم السبب والسلوك.

الكائنات لها سلوك مختلف

عند إسناد Object لمتغير آخر، المتغيران يشيران عادة إلى نفس الكائن:

```
$first = new stdClass();
$first->name = 'Omar';

$second = $first;
$second->name = 'Ali';

echo $first->name; // Ali
```

لو محتاج كائنًا منفصلًا تستخدم `clone` ، وتفاصيل النسخ العميق والسطحي هتيجي في مسار OOP.

المتغيرات المتغيرة

PHP تقدر تستخدم قيمة متغير كاسم لمتغير آخر:

```
$field = 'email';
$$field = 'omar@example.com';

echo $email;
```

ده معناه إن `$$field` تحولت إلى `$email` . الميزة موجودة، لكن غالبًا المصفوفة أو الكائن أوضح:

```
$user = [
    'email' => 'omar@example.com',
];

echo $user['email'];
```

ماتستخدمش مدخل المستخدم مباشرة كاسم متغير؛ ده يصعب التحقق والتتبع وقد يغير بيانات ماكنتش ناوي تسمح بتغييرها.

يعني إيه Scope؟

Scope أو النطاق هو المكان اللي يقدر الكود يشوف فيه اسم المتغير. المتغير المكتوب خارج الدوال في نطاق الملف، والمتغير اللي يتعمل داخل دالة يكون Local للدالة.

```php
$taxRate = 0.14;

function showLocalScope(): void
{
    $message = 'I exist inside this function';
    echo $message;

    // echo $taxRate; // غير متاح تلقائيًا هنا
}

showLocalScope();
// echo $message; // غير متاح خارج الدالة
```

وجود نطاقات منفصلة بيقلل التصادم والتغييرات المفاجئة. الدالة الأفضل تستقبل اللي تحتاجه صراحة:

```php
function priceWithTax(int $priceCents, float $taxRate): int
{
    return (int) round($priceCents * (1 + $taxRate));
}

echo priceWithTax(10000, 0.14); // 11400
```

دلوقتي الدالة واضحة وسهل اختبارها: نفس المدخلات تعطي نفس النتيجة.

`global` و`$GLOBALS`

PHP تسمح للدالة توصل لمتغير خارجي، لكن الأفضل تتجنب ده في أغلب الحالات:

```php
$total = 100;

function addTaxUsingGlobal(float $rate): float
{
    global $total;
    return $total * (1 + $rate);
}
```

وتقدر توصل لنفس القيمة عن طريق:

```php
function addTaxUsingGlobals(float $rate): float
{
    return $GLOBALS['total'] * (1 + $rate);
}
```

المشكلة إن اعتماد الدالة بقى مخفيًا. لو `$total` مش موجود أو اتغير في مكان بعيد، النتيجة تتغير. تمرير القيمة كـArgument يجعل الاعتماد ظاهرًا وأسهل في الاختبار.

المتغير `static` داخل الدالة

المتغير المحلي العادي يبدأ من جديد كل استدعاء. أما `static` المحلي فيتهيأ مرة واحدة ويحفظ قيمته داخل الـProcess الحالية:

```php
function nextId(): int
{
    static $id = 0;
    return ++$id;
}

echo nextId(), PHP_EOL; // 1
echo nextId(), PHP_EOL; // 2
echo nextId(), PHP_EOL; // 3
```

ده مش تخزين دائم. لو العملية انتهت وبدأت عملية جديدة، القيمة تبدأ من الأول. كمان الحالة المخفية ممكن تصعّب الاختبار، فلا تستخدمها بدل قاعدة البيانات أو Cache أو Dependency واضحة.

نطاق الملفات المضمّنة

الملف اللي بتضمه باستخدام `include` أو `require` يرث نطاق السطر اللي اتضم فيه:

```php
function loadConfig(): array
{
    $environment = 'development';
    return require __DIR__ . '/config.php';
}
```

لو `config.php` استخدم `$environment`، هيكون متاحًا لأنه اتضم داخل الدالة. السلوك ده مفيد أحيانًا، لكنه ممكن يعمل اعتمادًا مخفيًا؛ الأفضل إن ملفات الإعداد ترجع قيمة واضحة بدل تعديل متغيرات كثيرة.

Superglobals: بيانات متاحة في كل النطاقات

في تطبيق الويب، PHP بتحط أجزاء من الطلب والبيئة في مصفوفات خاصة اسمها **Superglobals**. متاحة داخل الدوال من غير `global`:

المتغير	بيحتوي غالبًا على إيه؟
`$_GET`	Query String الموجودة بعد `?` في الرابط
`$_POST`	حقول نموذج أُرسل بترميز Forms مناسب
`$_SERVER`	معلومات عن الطلب والخادم والـHeaders المتاحة
`$_FILES`	بيانات الملفات المرفوعة
`$_COOKIE`	Cookies التي أرسلها العميل
`$_SESSION`	بيانات Session بعد `session_start()`
`$GLOBALS`	جدول المتغيرات العامة

لو فتحت:

```
http://localhost:8000/?page=2&category=books
```

تقدر تقرأ:

```
$page = $_GET['page'] ?? '1';
$category = $_GET['category'] ?? 'all';
```

لكن كل بيانات جاية من المستخدم **غير موثوقة** حتى لو شكلها طبيعي.

الوجود، والتحقق، والتحويل، والإخراج

دي أربع خطوات مختلفة:

1. **اقرأ القيمة** مع التعامل مع عدم وجودها.
2. **تحقق** إنها تحقق قواعد البرنامج.
3. **حوّلها** لنوع داخلي مناسب.
4. **اعمل Encoding** وقت إخراجها حسب السياق.

```
$rawPage = $_GET['page'] ?? null;
$page = filter_var($rawPage, FILTER_VALIDATE_INT, [
    'options' => ['min_range' => 1, 'max_range' => 1000],
]);

if ($page === false) {
    $page = 1;
}
```

ليه استخدمنا `$page === false` ؟ لأن `filter_var` ترجع `false` عند الفشل، والمقارنة الصارمة تمنع خلطها بقيم أخرى.

قائمة Checkboxes ممكن توصل كمصفوفة:

```
?order_ids[]=10&order_ids[]=20
```

```
function positiveIds(mixed $input, int $maximumCount = 100): array
{
    if (!is_array($input) || !array_is_list($input) || count($input) > $maximumCount) {
        throw new InvalidArgumentException('Expected a bounded list of IDs');
    }
    $ids = [];
    foreach ($input as $raw) {
        if ((!is_string($raw) && !is_int($raw))
            || preg_match('/\A[1-9][0-9]*\z/', (string) $raw) !== 1) {
            throw new InvalidArgumentException('Invalid ID');
        }
        $id = filter_var($raw, FILTER_VALIDATE_INT, ['options' => ['min_range' => 1]]);
        if ($id === false) {
            throw new InvalidArgumentException('ID exceeds integer range');
        }
        $ids[] = $id;
    }
    return $ids;
}

try {
    $orderIds = positiveIds($_GET['order_ids'] ?? []);
} catch (InvalidArgumentException) {
    http_response_code(422);
    exit('Invalid order IDs');
}
```

التحويل مش تحقق: `intval("12x")` بيطلع 12 رغم إن المدخل مش رقم صحيح. هنا بنقبل قائمة فيها 100 معرّف بحد أقصى، وكل معرّف عدد صحيح موجب بلا أصفار بادئة. بنرفض المصفوفات المتداخلة والصفر وتجاوز الحد. القائمة الفارغة مسموحة. بعد التحقق لسه لازم تتأكد إن المستخدم المسجل يقدر يصل لكل طلب اختاره.

التحقق مش هو الـEscaping. ممكن الاسم يكون صحيحًا حسب قواعدك، لكن يظل محتاج `htmlspecialchars()` عند عرضه في HTML، وPrepared Statement عند استخدامه كقيمة في SQL.

ليه `eval()` خطيرة؟

`eval()` تأخذ نصًا وتنفذه ككود PHP:

```
$code = 'echo "hello";';
eval($code);
```

لو النص جاء من مستخدم، فأنت سمحت له يكتب كودًا على السيرفر. في التطبيقات العادية مش محتاج `eval()` أصلًا. استخدم دوالًا أو مصفوفة Callbacks أو `match`:

```
$operations = [
    'trim' => static fn (string $value): string => trim($value),
    'upper' => static fn (string $value): string => strtoupper($value),
];

$operation = 'trim';
$result = $operations[$operation]('  hello  ');
```

برنامج كامل: ترحيب من الرابط

```
<?php
declare(strict_types=1);

$rawName = $_GET['name'] ?? 'Guest';

if (!is_string($rawName)) {
    $rawName = 'Guest';
}

$name = trim($rawName);
if ($name === '' || mb_strlen($name) > 50) {
    $name = 'Guest';
}

$safeName = htmlspecialchars($name, ENT_QUOTES, 'UTF-8');
?>
<!doctype html>
<html lang="en">
<head><meta charset="utf-8"><title>Greeting</title></head>
<body><h1>Hello <?= $safeName ?></h1></body>
</html>
```

جرّب الحالات دي:

```
/?name=Omar
/?name=
/?name[]=Omar
/?name=<script>alert(1)</script>
```

كل حالة بتختبر حاجة مختلفة: قيمة طبيعية، قيمة فاضية، نوع غير متوقع، ومحاولة إدخال HTML.

خريطة حركة القيمة

```
Request value
     ↓
Read from $_GET
     ↓
Check type and rules
     ↓
Store in a clear local variable
     ↓
Encode for HTML output
```

تأكد من فهمك

01 **توقع الناتج:** `$a = 5; $b = $a; $b++; echo "$a,$b";`

الإجابة: `5,6` لأن الإسناد العادي أعطى `$b` قيمة مستقلة منطقيًا.

02 **أضف `&` في المثال السابق عند الإسناد. إيه اللي هيتغير؟**

الإجابة: لو كتبنا `$b =& $a` فالناتج `6,6` لأن الاسمين مرتبطان بنفس القيمة المنطقية.

03 **ليه الدالة اللي تستقبل `$total` كـArgument أسهل في الاختبار من دالة تستخدم `global` ؟**

الإجابة: لأن كل اعتماد ظاهر في التوقيع، فنقدر نعطيها مدخلات محددة ونتوقع النتيجة. `global` يخلي النتيجة مرتبطة بحالة خارجية قد تتغير من مكان آخر.

04 **الرابط يحتوي `?page=abc` . ليه التحويل المباشر إلى `int` مش كفاية؟**

الإجابة: التحويل قد ينتج صفرًا ويخفي إن المدخل كان غير صالح. الأفضل نتحقق أنه Integer وفي النطاق المطلوب، ثم نقرر هل نستخدم Default أم نرجع خطأ.

05 **إيه الفرق بين Validation وHTML Encoding؟**

الإجابة: Validation تقرر هل القيمة تحقق قواعد البرنامج. Encoding يحول القيمة وقت إخراجها بحيث يتعامل معها المتصفح كبيانات في السياق المحدد، لا ككود.

06 **اكتب Counter باستخدام `static` ، ثم اشرح ليه ماينفعش تستخدمه لحفظ عدد زيارات الموقع بشكل دائم.**

الإجابة: المتغير يحفظ قيمته بين استدعاءات الدالة داخل العملية الحالية فقط. الزيارات قد تذهب لعمليات أو خوادم مختلفة، والقيمة تضيع عند انتهاء العملية؛ نحتاج مخزنًا مشتركًا ودائمًا مثل Database أو Cache مناسب.

ملخص الدرس

المتغير اسم لقيمة أثناء تشغيل البرنامج. الإسناد العادي والمرجع مش نفس الشيء، والكائنات لها سلوك مشاركة مختلف. الـScope يحدد مين يقدر يشوف المتغير، وتمرير القيم للدوال أوضح من `global` . الـSuperglobals بتدخل بيانات الطلب للبرنامج، لكن وجود البيانات لا يجعلها صحيحة أو آمنة: اقرأها، تحقق منها، حوّلها، واعمل Encoding وقت الإخراج.

شغّل وتحقق

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

نفّذ نقطة التحقق التالية داخل بيئة الدرس:

```
php scope-lab.php
```

معيار النجاح: تثبت المخرجات الفرق بين local وglobal وstatic، ولا يقرأ من superglobal قبل التحقق من وجود المفتاح وشكله.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

اربط النقاط ببعض

Superglobals حدود ثقة وليست بيانات جاهزة: افحص الوجود والشكل والحجم ثم حول إلى type داخل request object. Variable variables و `eval` ليستا أدوات تصميم معتادة؛ غالبًا map أو callable registry أو parser صريح أوضح وأأمن. لا تجعل global state اعتمادًا مخفيًا.

جرّب بنفسك

حوّل endpoint يقرأ $_GET في كل مكان إلى قيمة محققة تمرر للدوال.

3. أنواع البيانات ونظام الأنواع

نفهم النوع من البداية، والفرق بين النص والعدد والقيمة المنطقية وnull، ثم المصفوفات والكائنات وتصريحات الأنواع والأنواع المتقدمة.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Boolean:** قيمة منطقية لها حالتان فقط: صح أو خطأ.
- **Loop:** حلقة تكرار تعيد تنفيذ مجموعة تعليمات وفق شرط.
- **Function:** دالة: جزء كود له اسم ومهمة محددة ويمكن استدعاؤه أكثر من مرة.

يعني إيه Data Type؟

القيمة `10` مش زي النص `'10'`. شكلهم قريب، لكن الأولى عدد نقدر نجمعه، والثانية حروف ممكن تكون جاية من Form. **نوع البيانات** بيقول لـPHP وللمبرمج القيمة دي معناها إيه، وإيه العمليات المنطقية اللي تنفع معها.

```
$quantity = 10;       // int
$input = '10';        // string
$price = 19.95;       // float
$isAvailable = true;  // bool
$discount = null;     // لا توجد قيمة حاليًا
```

لو تجاهلت الأنواع، ممكن تجمع نصًا غير صالح، أو تعتبر القيمة الفارغة رقمًا، أو تقارن قيمتين بطريقة تعطي نتيجة غير متوقعة. الهدف مش حفظ قائمة الأنواع؛ الهدف إنك تعرف **القيمة الموجودة معاك، ومنين جت، وإيه العمليات الآمنة عليها**.

خريطة بسيطة قبل التفاصيل

هنقسم الأنواع بالشكل ده:

- **Scalar:** قيمة واحدة، وتشمل `bool` و`int` و`float` و`string`.
- **Compound:** تجمع قيمًا أو سلوكًا، مثل `array` و`object`.
- **Special:** مثل `null` و`resource`.
- **أنواع تستخدم في توقيع الدوال:** مثل `callable` و`iterable` و`mixed` و`void` و`never`.
- **أنواع نعرّفها بنفسنا:** Classes وInterfaces وEnums.
- **تركيبات أنواع:** Union مثل `int|string` وIntersection مثل `Countable&Iterator`.

مش لازم تستخدم كل الأنواع المتقدمة من أول يوم. ابدأ بالـScalar والمصفوفات، وبعد ما تفهم الدوال والكائنات ارجع لأجزاء `callable` وIntersection Types.

`bool`: صح أو غلط

الـBoolean له قيمتان فقط: `true` و`false`. بنستخدمه في الأسئلة والقرارات:

```
$isLoggedIn = true;
$hasPermission = false;

if ($isLoggedIn && $hasPermission) {
    echo 'Allowed';
} else {
    echo 'Denied';
}
```

أسماء Boolean الأفضل تبدأ بكلمة توضح إنها سؤال: `$isActive` أو `$hasAccess` أو `$canEdit`.

PHP تعتبر بعض القيم Falsey داخل الشرط:

- `false`
- `0` و`0.0`
- النص الفارغ `''` والنص `'0'`
- المصفوفة الفارغة `[]`
- `null`

```
if ('0') {
    echo 'will not run';
}
```

علشان كده ما تعتمدش على Truthiness لما يكون الفرق بين صفر وقيمة مفقودة مهمًا.

`null`: مفيش قيمة

`null` معناها إن مفيش قيمة حاليًا. ده مختلف عن صفر، والنص الفارغ، و`false`.

```
$middleName = null;

if ($middleName === null) {
    echo 'No middle name was provided';
}
```

المتغير قد يكون `null` لو أسندت له القيمة، أو قد يصبح غير معرّف بعد `unset()`:

```php
$value = 'hello';
unset($value);
```

المتغير غير المعرّف مش مطابق تمامًا لمتغير موجود وقيمته `null` في كل عمليات الفحص. استخدم `isset()` لما تريد تعرف إن المفتاح موجود وقيمته ليست `null`، و`array_key_exists()` لما وجود المفتاح نفسه مهم حتى لو قيمته `null`.

المقارنة العادية والصارمة

`==` تسمح بتحويل الأنواع قبل المقارنة. `===` تقارن النوع والقيمة معًا:

```php
var_dump(0 == false);    // true
var_dump(0 === false);   // false
var_dump('10' == 10);    // true
var_dump('10' === 10);   // false
```

في أغلب كود التطبيقات، ابدأ بالمقارنة الصارمة `===` و`!==`. استخدم المقارنة المرنة فقط لو أنت فاهم قواعد التحويل ومحتاجها فعلًا.

`int`: الأعداد الصحيحة

```php
$decimal = 42;
$octal = 0o52;
$hex = 0x2A;
$binary = 0b101010;

var_dump($decimal, $octal, $hex, $binary);
```

كل القيم السابقة تساوي 42، لكن طريقة كتابتها مختلفة. الأشكال الثنائية والسداسية تظهر في Flags والألوان والبروتوكولات، لكن العدد داخل العمليات يظل عددًا.

القسمة العادية قد ترجع `float`:

```php
echo 7 / 2;        // 3.5
echo intdiv(7, 2); // 3
echo 7 % 2;        // 1، باقي القسمة
```

التحويل إلى `int` يقتطع الجزء العشري، ولا يقربه:

```php
echo (int) 3.9; // 3
echo round(3.9); // 4
```

حجم `int` يعتمد على المنصة، وتقدر تشوف الحدود باستخدام `PHP_INT_MAX` و`PHP_INT_MIN`. لو تجاوز الحساب المجال، ممكن تتحول النتيجة إلى `float` وتفقد دقة.

`float`: الأعداد العشرية

الـFloat تمثل كسورًا ثنائية، ولذلك مش كل كسر عشري له تمثيل دقيق:

```
$result = 0.1 + 0.2;

var_dump($result);          // 0.3 قيمة قريبة من
var_dump($result === 0.3);  // false غالبًا
```

ده مش عيب خاص بـPHP؛ دي طبيعة IEEE 754 المستخدمة في لغات كثيرة. للمقارنات العلمية استخدم Tolerance:

```
$expected = 0.3;
$epsilon = 0.000001;

if (abs($result - $expected) < $epsilon) {
    echo 'Close enough';
}
```

للأموال، أبسط اختيار آمن في أمثلة كثيرة هو أصغر وحدة صحيحة:

```
$priceCents = 1999;
$quantity = 3;
$totalCents = $priceCents * $quantity; // 5997
```

لو المجال يحتاج كسورًا عشرية دقيقة أو أرقامًا كبيرة، استخدم امتدادًا أو مكتبة Decimal مناسبة بدل `float`.

`string`: النصوص والبايتات

```
$name = 'Omar';
$message = "Hello {$name}";
$joined = 'PHP' . ' ' . '8';
```

عامل دمج النصوص هو النقطة `.`، مش `+`.

```
echo $joined; // PHP 8
```

PHP String هي سلسلة Bytes. الوصول بالفهرس يرجع Byte، وده قد ينجح مع ASCII لكنه مش طريقة آمنة لتقسيم النص العربي:

```
$english = 'PHP';
echo $english[0];  // P
echo $english[-1]; // P
```

للنصوص متعددة البايتات استخدم دوال `mb_*` عند توفر `mbstring`، مثل `mb_strlen()` و`mb_substr()`.

Nowdoc وHeredoc

للنصوص متعددة الأسطر:

```
$name = 'Omar';

$heredoc = <<<TEXT
Hello $name
This value is interpolated.
TEXT;

$nowdoc = <<<'TEXT'
$name stays exactly as written.
TEXT;
```

Heredoc تتصرف قريبًا من Double Quotes، وNowdoc قريب من Single Quotes.

النص الرقمي مش رقمًا مضمونًا

بيانات Forms وQuery Strings بتوصل غالبًا كنصوص:

```
$rawAge = $_GET['age'] ?? null;
```

ما تعتمدش على إن PHP هتحولها تلقائيًا في الحساب. تحقق ثم حوّل:

```
$age = filter_var($rawAge, FILTER_VALIDATE_INT, [
    'options' => ['min_range' => 1, 'max_range' => 120],
]);

if ($age === false) {
    echo 'Invalid age';
} else {
    echo "Next year you will be ", $age + 1;
}
```

`is_numeric()` مفيدة لمعرفة إن النص له شكل رقمي، لكنها لا تطبق قواعد مجال عملك. مثلًا عمر `-20` قد يكون Numeric لكنه غير صالح كتاريخ عمر بشري.

`array`: قائمة وMap في نفس النوع

مصفوفة PHP Ordered Map، ولذلك تقدر تستخدمها كقائمة أو كخريطة مفاتيح وقيم:

```
$colors = ['red', 'blue'];

echo $colors[0]; // red
```

```php
$user = [
    'id' => 7,
    'name' => 'Omar',
    'active' => true,
];

echo $user['name'];
```

وممكن تكون متعددة الأبعاد:

```php
$orders = [
    ['id' => 101, 'total_cents' => 5000],
    ['id' => 102, 'total_cents' => 7500],
];

echo $orders[1]['total_cents']; // 7500
```

لو استخدمت مفتاحًا غير موجود يظهر Warning. استخدم `??` لقيمة افتراضية لما يكون الغياب متوقعًا:

```php
$country = $user['country'] ?? 'Unknown';
```

`object` وClass

الكائن Instance من Class تجمع حالة وسلوكًا:

```php
final class Product
{
    public function __construct(
        public string $name,
        public int $priceCents,
    ) {}
}

$product = new Product('Keyboard', 150000);
echo $product->name;
```

مش لازم تفهم كل تفاصيل المثال الآن. المهم تعرف إن النوع هنا `Product`، وإن الكائن يقدر يحمل Properties وMethods لها معنى مرتبط بالمجال. مسار OOP هيبني الفكرة من البداية.

`enum`: حالات محدودة بالاسم

بدل نص حر ممكن يتكتب غلط، Enum تحدد الحالات المسموح بها:

```php
enum OrderStatus: string
{
    case Pending = 'pending';
    case Paid = 'paid';
    case Cancelled = 'cancelled';
}

$status = OrderStatus::Paid;
echo $status->value; // paid
```

دلوقتي ماينفعش الحالة تكون `'paied'` بالخطأ من غير ما يظهر خلل واضح في التحويل.

`resource`: مقبض لمورد خارجي

بعض الدوال ترجع `resource` يمثل اتصالًا أو Stream مفتوحًا:

```php
$handle = fopen(__FILE__, 'rb');

if ($handle === false) {
    throw new RuntimeException('Could not open the file');
}

echo get_debug_type($handle); // resource (stream)
fclose($handle);
```

الـResource مش محتوى الملف نفسه؛ هو Handle نستخدمه للتعامل مع المورد. امتدادات حديثة كثيرة أصبحت تعيد Objects بدل Resources، لذلك راجع نوع القيمة في التوثيق.

تصريحات الأنواع في الدوال

نقدر نوضح ما تستقبله الدالة وما ترجعه:

```php
function calculateTotal(int $priceCents, int $quantity): int
{
    return $priceCents * $quantity;
}

echo calculateTotal(1500, 3); // 4500
```

ده يجعل عقد الدالة واضحًا ويساعد PHP وأدوات التحليل تكتشف أخطاء بدري.

اكتب في بداية الملف:

```php
<?php
declare(strict_types=1);
```

بدون Strict Types قد تحول PHP بعض قيم الـScalar عند استدعاء الدالة. مع Strict Types، تمرير `'3'` إلى Parameter من نوع `int` يرمي `TypeError` بدل التحويل التلقائي في أغلب الحالات.

القرار يُطبّق من الملف **المستدعي** للدالة. ويوجد استثناء عملي: يمكن قبول `int` حيث المطلوب `float` لأن التحويل لا يفقد الجزء الكسري الموجود أصلًا.

Union Types و Nullable

لو الدالة قد ترجع مستخدمًا أو لا تجد شيئًا:

```
function findUser(int $id): ?array
{
    return $id === 7 ? ['id' => 7, 'name' => 'Omar'] : null;
}
```

`?array` معناها `array|null`. ويمكن كتابة Union أوسع:

```
function normalizeId(int|string $id): int
{
    if (is_string($id) && !ctype_digit($id)) {
        throw new InvalidArgumentException('Invalid ID');
    }

    return (int) $id;
}
```

ما توسعش النوع لمجرد الراحة. كل نوع إضافي بيزود الحالات اللي لازم تختبرها.

`mixed` و `void` و `never`

- `mixed` يعني إن القيمة ممكن تكون من أي نوع، ومنها `null`. استخدم نوعًا أدق لو تقدر.
- `void` يعني إن الدالة لا ترجع قيمة مفيدة.
- `never` يعني إن الدالة لا ترجع للمستدعي أصلًا، لأنها ترمي Exception أو تستدعي `exit` أو لا تنتهي.

```
function logMessage(string $message): void
{
    error_log($message);
}

function fail(string $message): never
{
    throw new RuntimeException($message);
}
```

`callable` و Closure

الـCallable قيمة PHP تقدر تستدعيها كدالة. Callback هي Callable بنمررها علشان تتنفذ لاحقًا:

```php
$double = function (int $number): int {
    return $number * 2;
};

$shortDouble = fn (int $number): int => $number * 2;

echo $double(4);      // 8
echo $shortDouble(5); // 10
```

الـClosure كائن يمثل دالة مجهولة. وفي PHP الحديثة نقدر نأخذ First-class Callable:

```php
function clean(string $value): string
{
    return trim($value);
}

$cleaner = clean(...);
echo $cleaner('  hello  ');
```

والكائن يصبح Callable لو عرّف `__invoke()` :

```php
final class Formatter
{
    public function __invoke(string $value): string
    {
        return strtoupper(trim($value));
    }
}

$formatter = new Formatter();
echo $formatter(' hello '); // HELLO
```

`iterable` وGenerator

`iterable` يقبل Array أو Object يطبق `Traversable` . ده مفيد لما الدالة محتاجة تلف على عناصر من غير ما تهتم بمصدرها:

```php
function printValues(iterable $values): void
{
    foreach ($values as $value) {
        echo $value, PHP_EOL;
    }
}
```

الـGenerator ينتج القيم واحدة واحدة باستخدام `yield` بدل بناء مصفوفة كاملة:

```
function numbers(int $maximum): iterable
{
    for ($number = 1; $number <= $maximum; $number++) {
        yield $number;
    }
}

printValues(numbers(3));
```

الناتج:

```
1
2
3
```

`yield` توقف الدالة مؤقتًا وتحفظ حالتها، ثم تكمل من نفس المكان عند طلب القيمة التالية. كل Generator يطبق Iterator، لكن مش كل Iterator معمول باستخدام Generator.

Intersection وDNF Types — للقراءة الآن

Intersection Type مثل `Countable&Iterator` يعني إن الكائن لازم يحقق النوعين معًا. وDNF Types تسمح بتجميع Unions وIntersections بأقواس وفق قواعد محددة، مثل `(A&B)|null` .

الأنواع دي مهمة في تصميم مكتبات وعقود متقدمة، لكن ما تحتاجش تستخدمها دلوقتي. ارجع لها بعد Interfaces وOOP؛ وجودها هنا علشان خريطة الأنواع تكون كاملة من غير ما نخلط مستوى البداية بالمستوى المتقدم.

برنامج كامل: حساب إجمالي طلب

```php
<?php
declare(strict_types=1);

function readPositiveInt(mixed $value): ?int
{
    $result = filter_var($value, FILTER_VALIDATE_INT, [
        'options' => ['min_range' => 1],
    ]);

    return $result === false ? null : $result;
}

$priceCents = readPositiveInt($_GET['price_cents'] ?? null);
$quantity = readPositiveInt($_GET['quantity'] ?? null);

if ($priceCents === null || $quantity === null) {
    http_response_code(400);
    echo 'price_cents and quantity must be positive integers';
    exit;
}

$totalCents = $priceCents * $quantity;

echo "Total: {$totalCents} cents";
```

جرّب:

```
/?price_cents=1999&quantity=3
/?price_cents=abc&quantity=3
/?price_cents=1999&quantity=0
```

المثال بيربط بين أنواع البيانات، و`mixed` على حدود المدخل غير الموثوق، والتحقق، وNullable Return، والمقارنة الصارمة، والحساب بأعداد صحيحة.

أخطاء شائعة

- استخدام `==` ثم الاستغراب من تحويل الأنواع.
- استخدام `float` للأموال من غير فهم الدقة.
- افتراض إن كل قيمة في `$_GET` نص واحد؛ المهاجم يقدر يرسل Array.
- استخدام `$text[0]` لتقسيم العربية.
- جعل كل الدوال تقبل `mixed` بدل تعريف عقد واضح.
- إضافة Union واسع لإخفاء تصميم غير واضح.
- استخدام `fetchAll()` أو Array ضخمة حين يمكن إنتاج العناصر تدريجيًا.

تأكد من فهمك

01 **إيه الفرق بين `0` و `'0'` و `false` و `null` ؟**

الإجابة: القيم أنواعها مختلفة: Integer وString وBoolean وNull. قد تتشابه في شرط أو مقارنة مرنة، لكنها لا تتساوى بالمقارنة الصارمة، ولكل واحدة معنى مختلف في البرنامج.

02 **توقع الناتج:** `var_dump('5' === 5); var_dump('5' == 5);`

الإجابة: الأولى `false` لاختلاف النوع، والثانية `true` لأن المقارنة المرنة تحول القيم وفق قواعد PHP.

03 **ليه `0.1 + 0.2 === 0.3` قد تكون False؟**

الإجابة: لأن الكسور تُخزن بتمثيل ثنائي محدود، وبعض القيم العشرية لا تُمثل بالضبط. نقارن بتسامح مناسب أو نستخدم تمثيلًا عشريًا دقيقًا حسب المجال.

04 **متى تستخدم `array_key_exists()` بدل `isset()` ؟**

الإجابة: لما يهمك تعرف إن المفتاح موجود حتى لو قيمته `null` . `isset()` ترجع False للمفتاح المفقود وللقيمة Null.

05 **صلّح تصميم دالة سعر تقبل `mixed` من داخل التطبيق كله.**

الإجابة: اقبل `mixed` فقط عند حد خارجي لو كان المصدر غير موثوق، ثم تحقق وحوّل إلى `int` يمثل القروش. خلي دوال المجال الداخلية تقبل `int` واضحًا بدل إعادة التحقق في كل مكان.

06 **اكتب Generator ينتج الأعداد الزوجية من 2 إلى حد أقصى، ثم اطبع أول أربع قيم.**

خط الحل: ابدأ Loop من 2 وزوّد 2 كل مرة واستخدم `yield` . عند حد أقصى 8 سيكون الناتج 2 و4 و6 و8 من غير بناء Array مسبقًا.

ملخص الدرس

النوع هو معنى القيمة والعمليات المناسبة لها. استخدم المقارنة الصارمة، وتحقق من النصوص قبل تحويلها، وما تستخدمش Float للأموال من غير قرار واعٍ. Arrays مناسبة للقوائم والخرائط، وObjects وEnums تضيف أنواعًا لها معنى. Type Declarations وStrict Types تجعل حدود الدوال أوضح، والأنواع المتقدمة موجودة علشان نستخدمها لما نوصل لمشكلتها، مش لمجرد استعراضها.

شغّل وتحقق

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

نفّذ نقطة التحقق التالية داخل بيئة الدرس:

```
php types-lab.php
```

معيار النجاح: تمر الحالات الصحيحة، وترفض الدالة النوع الخاطئ في strict mode بدل تحويله بصمت؛ وثّق نوع الخطأ وكود الخروج.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

اربط النقاط ببعض

قسّم الدراسة إلى scalar/null، ثم arrays/objects/enums، ثم advanced declarations مثل union/intersection/never. أضف اختبارات للحدود الرقمية وnumeric strings وNaN/INF، ولا تستخدم type متقدمًا قبل وجود عقد يحتاجه. Serialization موضوع تخزين وثقة منفصل عن type declaration.

جرّب بنفسك

اكتب مصفوفة حالات تبين القيمة والنوع قبل وبعد كل تحويل.

4. الإخراج والتصحيح والثوابت

نطبع النتيجة، نفحص القيم وقت حدوث مشكلة، ونفهم إمتى نستخدم ثابتًا بدل متغير.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Debugger:** أداة تتبّع الأخطاء: بتوقف البرنامج خطوة خطوة عشان تشوف القيم ومسار التنفيذ.
- **HTTP:** قواعد تبادل الطلبات والردود بين المتصفح والخادم.
- **API:** واجهة محددة تسمح لبرنامج يطلب بيانات أو ينفّذ عملية عند برنامج آخر.
- **Session:** بيانات مؤقتة تساعد الخادم يميّز المستخدم بين أكثر من طلب.
- **Cookie:** قيمة صغيرة يحفظها المتصفح ويرسلها مع الطلبات المناسبة.
- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.
- **CLI:** واجهة تتعامل معها بكتابة أوامر نصية بدل الضغط على أزرار.
- **UTF-8:** طريقة شائعة لتحويل أرقام Unicode إلى بايتات تُحفظ وتُنقل.
- **Boolean:** قيمة منطقية لها حالتان فقط: صح أو خطأ.
- **Function:** دالة: جزء كود له اسم ومهمة محددة ويمكن استدعاؤه أكثر من مرة.

قبل ما نبدأ

وأنت بتتعلم، محتاج تشوف نتيجتين مختلفتين: **ناتج البرنامج للمستخدم**، ومعلومات مؤقتة تساعدك أنت تفهم البرنامج بيعمل إيه. `echo` مناسبة للإخراج، أما `var_dump()` وأدوات الـDebugging فبتكشف النوع والقيمة أثناء التشخيص. خلط الاثنين في الإنتاج ممكن يعرض بيانات حساسة أو يفسد JSON وHTML.

في الدرس ده هنفهم كمان الثوابت: أسماء لقيم مش المفروض تتغير أثناء التشغيل، زي اسم التطبيق أو HTTP Status معروف.

`echo`: أبسط طريقة للإخراج

```php
<?php

$name = 'Omar';
echo 'Hello ', $name, PHP_EOL;
```

الناتج:

```
Hello Omar
```

`echo` عبارة لغوية Language Construct، مش دالة عادية. علشان كده الأقواس اختيارية، وتقدر تبعت لها أكتر من قيمة مفصولة بفواصل:

```
echo 'Subtotal: ', 100, ' EGP', PHP_EOL;
```

لو محتاج تركّب قيمة واحدة الأول، استخدم دمج النصوص بالنقطة:

```
$message = 'Hello ' . $name;
echo $message;
```

داخل HTML فيه اختصار واضح:

```
<h1><?= htmlspecialchars($name, ENT_QUOTES, 'UTF-8') ?></h1>
```

الاختصار `<?= ... ?>` معناه «اطبع نتيجة التعبير». ما تطبعش مدخل مستخدم مباشرة؛ اعمل Encoding مناسب للمكان اللي هتضع فيه القيمة.

`print`: قريب من `echo` مع فرق صغير

```
$result = print 'Printed';
var_dump($result); // int(1)
```

الخاصية	`echo`	`print`
القيمة المرجعة	لا توجد	دائمًا `1`
عدد القيم	يقبل عدة قيم بلا أقواس	قيمة واحدة
الاستخدام داخل Expression	لا	ممكن بسبب القيمة المرجعة
اختصار القوالب	`<?= ... ?>`	لا يوجد

فرق السرعة مش قرار عملي. استخدم `echo` في أغلب الحالات لأنها أبسط ومتوقعة، واعرف `print` علشان تفهم الكود القديم أو المثال اللي قدامك.

إخراج Terminal غير إخراج Web

في CLI، `PHP_EOL` ينزل سطرًا جديدًا:

```
echo 'First line', PHP_EOL;
echo 'Second line', PHP_EOL;
```

في HTML، نهاية السطر داخل المصدر لا تعمل بالضرورة كسطر مرئي. استخدم عناصر HTML مناسبة:

```
echo '<p>First paragraph</p>';
echo '<p>Second paragraph</p>';
```

وفي JSON لازم تحدد نوع المحتوى وتطبع JSON صالحًا فقط، من غير رسائل Debug قبله:

```
header('Content-Type: application/json; charset=utf-8');
echo json_encode(['status' => 'ok'], JSON_THROW_ON_ERROR);
```

لو أضفت `var_dump()` قبل JSON، العميل هيستلم Response غير صالحة حتى لو البيانات الصحيحة موجودة بعدها.

ليه `echo` مش أداة Debug كاملة؟

لو طبعت `false` باستخدام `echo` مش هتشوف شيئًا تقريبًا، ولو طبعت Array هتظهر مشكلة أو كلمة `Array` بدل تفاصيلها. أثناء التشخيص محتاج النوع والحجم والبنية.

```
$value = false;

echo $value;      // لا يوضح الحقيقة جيدًا
var_dump($value); // bool(false)
```

`var_dump()` و`print_r()` و`get_debug_type()`

```
$user = [
    'id' => 7,
    'active' => true,
    'roles' => ['editor', 'reviewer'],
];

var_dump($user);
print_r($user);
echo get_debug_type($user); // array
```

- `var_dump()` تعرض النوع والقيمة والطول والتفاصيل المتداخلة.
- `print_r()` أسهل بصريًا للمصفوفات والكائنات، لكنها أقل دقة في الأنواع.
- `print_r($value, true)` ترجع العرض كنص بدل طباعته.
- `get_debug_type()` ترجع اسم نوع أوضح، خصوصًا للكائنات والـResources.

```
$debugText = print_r($user, true);
file_put_contents(__DIR__ . '/debug.txt', $debugText);
```

ده مثال تعليمي فقط. ما تكتبش بيانات مستخدم حساسة في ملف عام، وامسح ملفات الـDebug بعد الانتهاء.

Debugging بخطوات بدل الطباعة العشوائية

افترض إن السعر النهائي طلع غلط:

```
$priceCents = 10000;
$discountPercent = 10;
$finalCents = $priceCents - $discountPercent;
```

المشكلة إننا طرحنا `10` قروش بدل 10%. قبل ما تغير الكود عشوائيًا:

1. اكتب النتيجة المتوقعة: `9000`.
2. افحص كل قيمة ونوعها.
3. اختبر العملية في سطر صغير.
4. صحح المعادلة.
5. أضف Test يمنع رجوع الخطأ.

```
$discountCents = (int) round($priceCents * ($discountPercent / 100));
$finalCents = $priceCents - $discountCents;

var_dump($priceCents, $discountPercent, $discountCents, $finalCents);
```

المفروض تشوف `10000` و`10` و`1000` و`9000`.

`dump()` و`dd()` والـLogger

Frameworks كثيرة توفر `dump()` لعرض قيمة بشكل أحسن، و`dd()` بمعنى Dump and Die: تعرض القيم وتوقف التنفيذ. الأدوات دي للتطوير، وممكن توقف Request أو تكشف Token وCookie وبيانات عميل لو اتسابت في الإنتاج.

في الإنتاج استخدم Logger ببيانات منظمة:

```
error_log(json_encode([
    'event' => 'checkout_failed',
    'order_id' => 42,
    'reason' => 'payment_timeout',
], JSON_THROW_ON_ERROR));
```

ما تسجلش Password أوSession ID أوAccess Token. خلي رسالة المستخدم بسيطة، والتفاصيل التقنية في سجل محمي.

يعني إيه ثابت؟

المتغير ممكن يتغير:

```
$attempts = 1;
$attempts++;
```

الثابت اسم لقيمة المفروض تفضل كما هي بعد تعريفها:

```
const APP_NAME = 'Omar Notes';
echo APP_NAME;
```

الثابت لا يبدأ بـ `$`. استخدمه لما المعنى فعلًا ثابت في الكود، مش لمجرد منع التغيير. بيانات تختلف بين البيئات، زي Password قاعدة البيانات، مكانها Configuration/Environment مش ثابت مكتوب في Repository.

`const` و`define()`

```
const APP_NAME = 'Omar Notes';
define('APP_VERSION', '1.0.0');
```

- `const` تصريح لغوي، مناسب للثوابت المعروفة وقت تعريف الكود.
- `define()` استدعاء وقت التشغيل، ولذلك ممكن يظهر داخل شرط.
- `define()` ينشئ ثابتًا عامًا، ولا يعرّف Class Constant.
- ما تقدرش تضع `const` المحلي داخل Function أوBlock شرطي.

```
if (getenv('APP_ENV') === 'testing') {
    define('FAKE_EXTERNAL_SERVICES', true);
}
```

حتى لو ده مسموح، Configuration Object غالبًا أوضح في التطبيقات الكبيرة من ثوابت عامة بتظهر حسب مسار التنفيذ.

Class Constants

```
final class HttpStatus
{
    public const OK = 200;
    public const NOT_FOUND = 404;
}

echo HttpStatus::NOT_FOUND;
```

وجود الثابت داخل Class بيربط الاسم بالمفهوم بدل نشر أسماء عامة. ولو الحالات تمثل مجموعة مغلقة لها سلوك وقيمة، Enum قد تكون أنسب.

Magic Constants

PHP توفر أسماء تتغير قيمتها حسب مكانها في الملف:

```
echo __LINE__, PHP_EOL;
echo __FILE__, PHP_EOL;
echo __DIR__, PHP_EOL;
echo __FUNCTION__, PHP_EOL;
echo __CLASS__, PHP_EOL;
echo __METHOD__, PHP_EOL;
echo __NAMESPACE__, PHP_EOL;
```

اسمها "Magic" لأن المحرك يحسب قيمتها من السياق. أهم مثال عملي هو `__DIR__`:

```
$config = require __DIR__ . '/../config/app.php';
```

المسار هنا يبدأ من مجلد الملف نفسه، مش من Current Working Directory اللي شغلت منه الأمر. ده يخلي `require` أكثر ثباتًا.

Predefined Constants

PHP والـExtensions تعرف ثوابت جاهزة:

```
echo PHP_VERSION, PHP_EOL;
echo PHP_OS_FAMILY, PHP_EOL;
echo PHP_INT_MAX, PHP_EOL;
echo PHP_EOL;
```

بعض الثوابت موجودة فقط لو Extension معينة متثبتة. استخدم `defined('CONSTANT_NAME')` لو الوجود اختياري:

```
if (defined('JSON_THROW_ON_ERROR')) {
    echo 'JSON exception flag is available';
}
```

برنامج صغير قابل للتجربة

```php
<?php
declare(strict_types=1);

const TAX_RATE = 0.14;

$priceCents = 25000;
$taxCents = (int) round($priceCents * TAX_RATE);
$totalCents = $priceCents + $taxCents;

if (getenv('APP_DEBUG') === '1') {
    var_dump([
        'file' => __FILE__,
        'price_cents' => $priceCents,
        'tax_cents' => $taxCents,
    ]);
}

echo "Total: {$totalCents} cents", PHP_EOL;
```

شغّله مرة عادي، ومرة مع `APP_DEBUG=1` حسب طريقة ضبط Environment Variables في نظامك. الناتج الأساسي يفضل ثابتًا، بينما معلومات التشخيص تظهر فقط في بيئة التطوير.

أخطاء شائعة

- استخدام `echo` لفحص Boolean أوArray ثم فهم النتيجة غلط.
- ترك `var_dump()` أو `dd()` في API إنتاجية.
- تسجيل أسرار بحجة التشخيص.
- اعتبار قيمة تختلف بين Development وProduction ثابتًا داخل الكود.
- بناء مسار نسبي يعتمد على المجلد الحالي بدل `__DIR__`.
- تعريف اسم ثابت عام جدًا قد يتصادم مع مكتبة أخرى.

تأكد من فهمك

01 **إيه الناتج والقيمة الموجودة في `$result` بعد `$result = print 'Hi';` ؟**

الإجابة: يتم إخراج `Hi`، وتأخذ `$result` القيمة الصحيحة `1`.

02 **ليه `echo false;` مش كفاية علشان تعرف قيمة المتغير؟**

الإجابة: لأنها تخرج كنص فارغ تقريبًا. `var_dump(false)` تعرض النوع والقيمة بوضوح: `bool(false)`.

03 **API ترجع JSON، لكن العميل يقول إن JSON غير صالحة. إيه أول حاجة تدور عليها؟**

الإجابة: دور على أي `var_dump` أوWarning أوHTML خرج قبل أو بعد JSON، وتأكد من Content-Type ومن إن المسار يطبع Document JSON واحدة فقط.

04 **إمتى تستخدم `__DIR__` في مسار ملف؟**

الإجابة: لما المسار لازم يكون محسوبًا بالنسبة لمكان الملف الحالي، بغض النظر عن المجلد اللي بدأ منه تشغيل البرنامج.

05 **هل كلمة مرور قاعدة البيانات مناسبة كـ `const` داخل الكود؟ ليه؟**

الإجابة: لا. هي Secret وتختلف بين البيئات وتحتاج Rotation؛ تُحقن من Environment أوSecret Manager ولا تُحفظ في Repository.

06 **صحح معادلة خصم كتبت `$final = $priceCents - $percent;` .**

الإجابة: احسب الخصم أولًا: `$discount = (int) round($priceCents * ($percent / 100));` ثم اطرحه. اختبر 0% و100% وقيمة عادية وحدد سياسة التقريب.

ملخص الدرس

`echo` و`print` للإخراج، لكن أدوات الـDebugging تكشف النوع والبنية وقت التشخيص. افصل ناتج المستخدم عن معلومات المطور، وما تعرضش أسرارًا. الثابت مناسب لقيمة لا تتغير داخل تصميم البرنامج، و`__DIR__` مثال عملي مهم لبناء مسار موثوق.

شغّل وتحقق

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

نفّذ نقطة التحقق التالية داخل بيئة الدرس:

```
php debug-lab.php 2> debug.log
```

معيار النجاح: يذهب الناتج المقصود إلى stdout والتشخيص إلى stderr أو logger، ولا تظهر قيمة سرية في أي منهما.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

اربط النقاط ببعض

في التطوير استخدم Xdebug أو debugger لفحص stack وbreakpoints، وفي الإنتاج عطّل display_errors ووجّه التفاصيل إلى logger محمي مع correlation ID. الثابت مناسب لقيمة بنيوية لا تتغير أثناء التشغيل، بينما configuration البيئية ليست class constant لمجرد أنها «ثابتة الآن».

جرّب بنفسك

اختبر خطأً في التطوير والإنتاج وتأكد أن المستخدم لا يرى stack trace.

5. الشروط والحلقات

if وswitch وmatch وfor وwhile وdo-while وforeach مع أهم الفروق والأخطاء.

المشكلة: قرار واحد ثم تكراره

عندك كشف درجات: 95 تستحق A، و40 تستحق F، و150 لازم نرفضها. بعد كده نكرر القرار لكل طالب. **Control Flow** هو طريق تنفيذ التعليمات؛ **Condition** سؤال نتيجته `true` أو `false`، و**Loop** حلقة تعيد مجموعة تعليمات. ذاكر اختيار الطريق أولًا، وبعدها تتبع التكرار على ورقة. كل برنامج كامل هنا يعمل على PHP 8.0+ في ملف مستقل.

if وelseif وelse: أول فرع صحيح

Block مجموعة تعليمات بين `{ }`، و**Branch** طريق يختاره الشرط. `if` تعني «لو»، و`elseif` «وإلا لو»، و`else` «في باقي الحالات». PHP تختبر من فوق لتحت وتتوقف عند أول فرع صحيح في السلسلة. احفظ `grade.php` وشغّل `php grade.php`:

```
<?php
$score = 75;
if ($score < 0 || $score > 100) {
    $grade = 'invalid';
} elseif ($score >= 90) {
    $grade = 'A';
} elseif ($score >= 75) {
    $grade = 'B';
} elseif ($score >= 50) {
    $grade = 'C';
} else {
    $grade = 'F';
}
echo "Score {$score}: {$grade}", PHP_EOL;
```

```
Score 75: B
```

`$score` المدخل. `||` معناها «أو»، فنرفض ما دون 0 أو فوق 100. عند 75 الشرط الأول false، ثم شرط 90 false، ثم شرط 75 true. نخزن B ونتجاوز الفروع التالية. `echo` خارج السلسلة، فيعمل بعد أي فرع.

المدخل	الناتج
-1 / 101	invalid
0 / 49	F
50 / 74	C
75 / 89	B
90 / 100	A

غلط وتصحيحه: البدء بـ `$score >= 50` يجعل 95 تأخذ C. رتب الحدود من الأعلى للأقل. واستبدال `elseif` بـ `if` منفصلة يسمح لعدة فروع بتغيير النتيجة؛ خلي البدائل في سلسلة واحدة، واستخدم شروطًا مستقلة فقط لما تحتاج إجراءات مستقلة. `>=` يشمل الحد نفسه؛ `>` يستبعده.

اختصارات اختيار قيمة

`$active ? 'active' : 'paused'` هي Ternary: الشرط، نتيجة الصح، نتيجة الغلط. استخدمها لقيمة قصيرة. `$input['name'] ?? 'guest'` تختار البديل لو المفتاح مفقود أو `null` فقط؛ تحافظ على `0` و `''`. أما `?:` فتستخدم التحويل لمنطقية، فتستبدل `'0'` أيضًا.

`$user?->address?->city` تستخدم Nullsafe: تتوقف عند `null` في سلسلة كائنات، مش عند Exception أو مفتاح Array مفقود. تفاصيل الكائنات في OOP. في HTML ممكن `if (...): ... else: ... endif;` بدل الأقواس، لكن لا تخلط الصيغتين في نفس الكتلة.

switch وmatch

`switch` تختار `case` حسب قيمة. `break` تمنع الانتقال للحالة التالية؛ استمرار التنفيذ بدونها اسمه **fall-through**. `match` تعبير يعيد قيمة من PHP 8.0، يقارن بالنوع والقيمة `===`، ولا يسقط لفرع آخر. شغّل `status.php`:

```
<?php
$status = '200';
switch ($status) {
    case 200:
        echo "switch: success", PHP_EOL;
        break;
    default:
        echo "switch: other", PHP_EOL;
}
$message = match ($status) {
    200, 201 => 'success',
    default => 'other',
};
echo "match: {$message}", PHP_EOL;
```

```
switch: success
match: other
```

`switch` تستخدم مقارنة مرنة فتطابق النص `'200'` بالعدد `200`. `match` لا تطابقهما. كل **arm** قيمة ونتيجتها بعد `=>`، وممكن عدة قيم تشترك في نتيجة. بدون تطابق أو `default` يحصل `UnhandledMatchError`. `match (true)` مع شروط مثل `$age < 18` ممكنة للنطاقات، لكن `if` غالبًا أوضح. تحقق من المدخل قبل تحويل نوعه.

for: البداية ثم الاختبار ثم الجسم ثم الخطوة

Counter عداد للدورات، و**accumulator** قيمة تجمع النتائج. احفظ `sum.php` وشغّله:

```
<?php
$total = 0;
for ($i = 1; $i <= 3; $i++) {
    $total += $i;
    echo "i={$i}, total={$total}", PHP_EOL;
}
echo "after: i={$i}, total={$total}", PHP_EOL;
```

```
i=1, total=1
i=2, total=3
i=3, total=6
after: i=4, total=6
```

`$total = 0` تحصل مرة؛ لو دخلت الجسم المجموع يبدأ من الصفر كل دورة. `$i = 1` تبدأ مرة، ثم `$i <= 3` تُفحص قبل كل دورة. الجسم يجمع ويطبع، و`$i++` تزيد واحدًا بعده. عند 4 نخرج. لا تضع `;` بعد `for (...)` لأنها تصنع جسمًا فارغًا.

قبل الجسم	المجموع قبله	بعد الجمع	بعد الزيادة
i=1	0	1	i=2
i=2	1	3	i=3
i=3	3	6	i=4

قائمة من 3 عناصر مفاتيحها 0 و1 و2، فشرطها `$i < count($names)` وليس `<=`. دورة زيادة أو ناقصة اسمها **off-by-one**. أجزاء `for` اختيارية؛ `for (;;)` لا تنتهي من نفسها. للقوالب `for (...): ... endfor;`.

while وdo-while: إمتى تفحص؟

`while` تفحص قبل الجسم؛ ممكن تعمل صفر مرة. `do-while` بعده؛ تعمل مرة على الأقل. في `attempts.php` عندنا مدخلات ثابتة عشان نعرف نكرر التجربة:

```php
<?php
$replies = ['', '', 'Omar'];
$attempt = 0;
do {
    $input = $replies[$attempt] ?? '';
    $attempt++;
    echo "attempt={$attempt}", PHP_EOL;
} while ($input === '' && $attempt < 3);
echo $input === '' ? "No name\n" : "Hello {$input}\n";

$remaining = 0;
while ($remaining > 0) {
    echo "work", PHP_EOL;
    $remaining--;
}
```

```
attempt=1
attempt=2
attempt=3
Hello Omar
```

نقرأ عنصرًا ثم نزود المحاولة قبل الاختبار. `&&` تعني إن الشرطين لازم يصحوا: الاسم فاضي ولسه أقل من 3 محاولات. الحلقة الأخيرة لا تعمل. لو هتقرأ بـ `fgets(STDIN)` افحص `false` لنهاية الإدخال. كل Loop محتاجة تقدم وحد خروج؛ مجرد وجود زيادة في مسار لا يضمن إن كل المسارات توصل لها.

foreach وcontinue وbreak

`foreach` تزور العناصر بدون إدارة فهرس. `continue` تتخطى باقي الدورة الحالية، و `break` تنهي الحلقة. شغّل `scores.php` :

```php
<?php
$scores = ['Ali' => 0, 'Mona' => -1, 'Omar' => 100, 'Nour' => 80];
foreach ($scores as $name => $score) {
    if ($score < 0) {
        continue;
    }
    echo "{$name}: {$score}", PHP_EOL;
    if ($score === 100) {
        break;
    }
}
```

```
Ali: 0
Omar: 100
```

`$name => $score` تقرأ المفتاح والقيمة. الصفر درجة صحيحة؛ Mona تُتخطى، وNour لا تُزار. مع Rows ممكن `foreach ($users as ['id' => $id, 'name' => $name])` لو الحقول موجودة. `break 2` تخرج من مستويين؛ ارسم التداخل قبل استخدامها. `continue` داخل `switch` متداخلة قد لا تقصد بها المستوى الصحيح؛ بسّط التصميم أو حدد المستوى بعناية.

`foreach ($prices as &$price)` تسمح بتعديل الأصل بالمرجع. بعد النهاية تظل `$price` مرتبطة بآخر عنصر: استخدم `unset($price)` فورًا. ابدأ بالتمرير بالقيمة، واستعمل المرجع لما يكون التعديل مقصودًا.

توقع، شخّص، كمّل

▼ توقع الدرجات 49 و50 و75 و90، وحلقة تبدأ بـ4 وشرطها 3 => i

F و C و B و A. الحلقة تعمل صفر مرة لأن الشرط يُفحص قبل الجسم. اختبر الحدود على الجانبين.

▼ Debugging: $i = 0; while ($i < 3) { if ($i === 1) continue; $i++; }

القيمة تعلق عند 1 لأن continue تتخطى الزيادة. حرّك العداد قبل مسار التخطي، أو استخدم for والزيادة في الجزء الثالث؛ for تنفذ خطوة الزيادة بعد continue.

▼ كمّل حلقة تجمع الأعداد الزوجية من 1 إلى 6

ابدأ total=0 وكرر من 1 إلى 6 شاملًا، ثم `if ($i % 2 === 0) { $total += $i; }`. الباقي صفر يعني زوجي، فالناتج 12.

▼ بعد foreach بالمرجع كتبت $price = 99 دون unset. إيه اللي يتغير؟

آخر عنصر يصبح 99. ضع `unset($price)` بعد الحلقة؛ حذف اسم المرجع لا يحذف عنصر المصفوفة.

خطوة للمشروع

دفتر الملاحظات في الدرس 17 سيختار بين خطأ وحفظ عند استقبال نموذج، ويعرض العناصر بحلقة. اكتب حالات قيمة مفقودة وفارغة وصحيحة الآن. للتدريب الإضافي شغّل `php control-flow-lab.php` من المختبر الموجود، واختبر صفر/واحد/عدة عناصر.

6. التعبيرات والمؤثرات

قيم التعبيرات، الأولوية، الحساب والإسناد والمقارنة والمنطق والمصفوفات والتنفيذ والـ Pipe.

المشكلة: الحساب صحيح شكليًا والنتيجة غلط

السعر 1500 قرش والكمية 3؛ عايزين إجماليًا، وخصمًا، وقرار شحن مجاني. **Expression** تعبير ينتج قيمة، و**Operator** رمز عملية، و**Operand** قيمة تعمل عليها العملية. في `$price * $quantity` الرمز `*` معامل، والمتغيران طرفاه. قبل كتابة تعبير اسأل عن نوع مدخلاته ونوع نتيجته.

الأمثلة الأساسية PHP 8.0+. احفظ `checkout.php` وشغّله بـ `php checkout.php`:

```
<?php
declare(strict_types=1);

$price = 1500;
$quantity = 3;
$subtotal = $price * $quantity;
$discount = intdiv($subtotal * 10, 100);
$afterDiscount = $subtotal - $discount;
$shipping = $afterDiscount >= 4000 ? 0 : 500;
$total = $afterDiscount + $shipping;
echo "subtotal={$subtotal}", PHP_EOL;
echo "discount={$discount}", PHP_EOL;
echo "shipping={$shipping}", PHP_EOL;
echo "total={$total}", PHP_EOL;
```

```
subtotal=4500
discount=450
shipping=0
total=4050
```

المتغيران الأولان أعداد صحيحة بالقرش. الضرب ينتج 4500. `intdiv` قسمة صحيحة؛ المثال يختار تقريب الخصم لأسفل للقيم الموجبة، ودي قاعدة معلنة مش مصادفة. نطرح 450، ونقارن 4050 بالحد 4000؛ الشرط true فيختار Ternary شحنًا صفرًا. نجمع ونطبع. المثال بمبالغ صغيرة؛ للأرقام الكبيرة افحص overflow قبل الضرب، وللسياسات المالية الأخرى استخدم تقريبًا معلنًا أو Decimal.

غلط: `$subtotal - 10` يطرح 10 قروش، مش 10%. و`$shipping = $subtotal >= 4000` يختبر قبل الخصم؛ صحح المتغير المستخدم حسب قاعدة المتجر. غيّر الكمية إلى 2: الإجمالي بعد الخصم 2700 والشحن 500 والناتج 3200.

الأولوية مش ترتيب تنفيذ مضمون

Precedence تحدد تجميع التعبير: `2 + 3 * 4` تعني `2 + (3 * 4)` فتنتج 14. `(2 + 3) * 4` تنتج 20. لا تعتمد على ترتيب تقييم تعبيرات تغير المتغير نفسه، مثل جمع زيادتين لنفس العداد؛ افصل كل تعديل في سطر.

احفظ `precedence.php`:

```php
<?php
$a = true && false;
$b = true and false;
var_dump($a, $b);
$x = 5;
echo $x++, PHP_EOL;
echo ++$x, PHP_EOL;
```

```
bool(false)
bool(true)
5
7
```

`&&` أعلى أولوية من الإسناد، فـ `$a` تأخذ false. `and` أقل منه، فكأن السطر `($b = true) and false` ؛ النتيجة الأخيرة لا تُخزن في b. استخدم `&&` و `||` وأقواسًا واضحة. `$x++` تعيد القديمة ثم تزود؛ `++$x` تزود ثم تعيد الجديدة. الإسناد نفسه يعيد قيمة، ولذلك `$b =` `$a = 5` يضع 5 في الاثنين، لكن سطرين أوضح غالبًا.

خريطة العمليات

المجموعة	الرموز والمعنى
حساب	`+ - * /` ، باقي القسمة `%` ، القوة `**`
إسناد مختصر	`+= -= *= /= %= **= .= ??=`
مقارنة	`== != <>` مرنة، `=== !==` صارمة، `< > <= >=` ترتيب
ترتيب ثلاثي	`<=>` تعيد -1 أو 0 أو 1 للأعداد البسيطة
منطق	`&&` و، `` ` ``
نصوص	`.` دمج، و `.=` دمج ثم إسناد

`/` قد تنتج float؛ `10 / 4` تساوي 2.5، و `intdiv(10, 4)` تساوي 2، و `10 % 4` تساوي 2. المقسوم عليه صفر يرمي `DivisionByZeroError` . كسور float تقريبية؛ لا تختبر ناتج حساب عشري متكرر بمساواة تامة بدون سياسة دقة.

النوع جزء من المقارنة

شغّل **Short circuit** . `compare.php` معناها عدم تقييم الطرف الثاني لما الأول حسم النتيجة:

```php
<?php
var_dump('0' == 0);
var_dump('0' === 0);
var_dump('0' != 0);
var_dump('0' !== 0);
$value = '0';
echo $value ?? 'missing', PHP_EOL;
echo $value ?: 'empty', PHP_EOL;
$divisor = 0;
var_dump($divisor !== 0 && 10 / $divisor > 2);
```

```
bool(true)
bool(false)
bool(false)
bool(true)
0
empty
bool(false)
```

الأربع مقارنات توضح قيمة ونوع النص والعدد. `??` لا تعتبر النص `'0'` مفقودًا؛ `?:` تعتبره falsy. في آخر سطر الطرف الأول false، فلا تحدث القسمة. ما تحطش عملية ضرورية للحفظ أو العد داخل طرف قد لا يُنفذ. و`$input ??= 'guest'` إسناد البديل فقط عند الغياب أو null.

نفس الرمز مع Array أو Object

`+` بين المصفوفات اتحاد حسب المفتاح، والقيمة اليسرى تفوز عند التعارض. `array_merge` تستبدل المفاتيح النصية بقيمة اليمين، وتعيد ترقيم العددية. جرّب:

```php
<?php
$left = ['timeout' => 3, 0 => 'A'];
$right = ['timeout' => 5, 0 => 'B'];
echo json_encode($left + $right, JSON_THROW_ON_ERROR), PHP_EOL;
echo json_encode(array_merge($left, $right), JSON_THROW_ON_ERROR), PHP_EOL;
```

```
{"timeout":3,"0":"A"}
{"timeout":5,"0":"A","1":"B"}
```

مساواة Arrays بـ`===` تشمل الأنواع وترتيب أزواج المفاتيح؛ `==` لا تطلب نفس الترتيب. `instanceof` يفحص هل كائن يحقق صنفًا/واجهة. إسناد كائن لمتغير ثانٍ لا ينسخه؛ `clone` نسخة سطحية، والكائنات الداخلية قد تظل مشتركة. التفاصيل في OOP.

رموز ما تستخدمهاش لإخفاء مشكلة

`@` قد يكتم تشخيص التعبير، لكنه لا يصلح الفشل. افحص return value أو تعامل مع Exception. العلامات الخلفية Backticks في PHP تنفذ أمر shell مثل `shell_exec`؛ لا تستخدمها لعرض نص، ولا تدخل بيانات مستخدم في `exec` أو `system` أو `proc_open`. اختار API مباشرة للمهمة، أو قائمة أوامر ثابتة وarguments منفصلة لو التنفيذ ضروري.

Pipe من PHP 8.5 فقط

Callable قيمة تشير لدالة قابلة للاستدعاء، وهنفهمها في الدرس 7. `|>` يمرر نتيجة اليسار كوسيط واحد لدالة اليمين. شغّل `pipe.php` باستخدام PHP 8.5؛ الملف لا يُحلل على 8.4:

```
<?php
$slug = ' Hello PHP '
    |> trim(...)
    |> strtolower(...)
    |> (fn (string $s): string => str_replace(' ', '-', $s));
echo $slug, PHP_EOL;
```

```
hello-php
```

كل مرحلة ترجع قيمة جديدة. بديل 8.0 هو `str_replace(' ', '-', strtolower(trim(' Hello PHP ')))`؛ البديل لا يحتاج First-class Callable. مرجع Pipe الرسمي.

توقع، شخّص، كمّل

▼ توقع: 2 + 3 * 4، ثم (2 + 3) * 4

14 ثم 20. الأقواس تغير تجميع العمليات؛ لا تغير قيمة الأعداد.

▼ Debugging: if ($quantity = 0) بدل المقارنة

الإسناد يضع صفرًا ثم الشرط false. استخدم `$quantity === 0` بعد التحقق من أن المدخل عدد صحيح. لا تصلحها بـ== لمجرد إخفاء اختلاف النوع.

▼ كمّل شرط قسمة آمنة مع divisor=0

`$divisor !== 0 && $amount / $divisor > 2` . فحص الصفر أولًا يمنع تقييم القسمة؛ تبديل الطرفين يفقد الحماية.

▼ لماذا ['timeout' => 3] + ['timeout' => 5] ليست تحديثًا إلى 5؟

الاتحاد يحافظ على قيمة اليسار. لو القصد override استخدم array_merge أو unpacking وفق قواعد المفاتيح.

جرّب `php operators-lab.php` من المختبر. في مشروع الدفتر هنستخدم `===` للـMethod و `??` للمفتاح المفقود؛ لا نستبدل التحقق من النوع بـtruthiness.

7. الدوال والـ Callbacks وتضمين الملفات

تعريف الدوال والوسائط والمراجع والـ closures والـ arrow functions وinclude وrequire.

المشكلة: نفس الحساب مكتوب في خمس أماكن

لو قاعدة السعر اتغيرت، تعديل خمس نسخ ممكن ينسينا واحدة. **Function** دالة تجمع مهمة تحت اسم، تستقبل بيانات، وترجع نتيجة. مش لازم تحفظ مصطلحاتها مرة واحدة: **parameter** اسم المدخل في التعريف، **argument** القيمة عند الاستدعاء، و**return value** النتيجة الراجعة.

ابدأ بحساب يرجع قيمة بدل طباعتها؛ كده نقدر نعرضه في CLI أو HTML ونختبره. احفظ PHP 8.0 ، `functions.php` +، وشغّل `php functions.php`:

```
<?php
declare(strict_types=1);

function subtotal(int $price, int $quantity = 1): int
{
    if ($price < 0 || $quantity < 1) {
        throw new InvalidArgumentException('Invalid order');
    }
    return $price * $quantity;
}
function receipt(int $price, int $quantity): string
{
    $amount = subtotal($price, $quantity);
    return "Total: {$amount}";
}
echo receipt(1500, 3), PHP_EOL;
echo subtotal(quantity: 2, price: 500), PHP_EOL;
```

```
Total: 4500
1000
```

`declare` يجعل هذا الملف يستدعي دوال الأنواع scalar بصرامة. `function subtotal` تعريف؛ الجسم لا يعمل لمجرد قراءته. `int` قبل الاسم نوع المدخل وبعد القوس نوع الناتج. `= 1` قيمة افتراضية لو لم نمرر quantity. شرط البداية يرفض القيم خارج العقد؛ `throw` توقف المسار بفشل واضح، وهنفصلها في الدرس 11. `return` تنهي الاستدعاء وترسل حاصل الضرب. الدالة الثانية تركب رسالة من النتيجة. السطر الأخير **named arguments**: القيم مرتبطة بالأسماء، لذلك إعادة تسمية parameter قد تكسر المستدعي.

PHP لا تسمح بتعريف دالتين بنفس الاسم في Namespace واحدة حسب اختلاف عدد الوسائط. استخدم defaults أو Union Types أو variadic حسب المعنى. الأعداد هنا صغيرة؛ مراجعة حدود الضرب جزء من توسيع التطبيق.

Call Stack: فين بيروح التنفيذ ويرجع إزاي؟

Call Stack رصّة الاستدعاءات النشطة. كل استدعاء يضيف **frame** فيه متغيراته ومكان الرجوع. آخر دالة دخلت هي أول واحدة ترجع. تتبع السطر `receipt(1500, 3)`:

```
main
main → receipt(price=1500, quantity=3)
main → receipt → subtotal(price=1500, quantity=3)
main → receipt(amount=4500)
main → echo("Total: 4500")
```

متغير `$amount` محلي لـreceipt؛ المتغيرات المتشابهة في Frames مختلفة مش نفس المتغير. `return` ترجع للمستدعي، مش لبداية البرنامج. **Recursion** دالة تستدعي نفسها: كل مرة Frame جديد، فلازم حالة توقف وحد للمدخل. في `countdown.php` :

```php
<?php
function countdown(int $n): void
{
    if ($n < 0 || $n > 10) {
        throw new InvalidArgumentException('Use 0..10');
    }
    if ($n === 0) {
        echo "go", PHP_EOL;
        return;
    }
    echo $n, PHP_EOL;
    countdown($n - 1);
}
countdown(3);
```

```
3
2
1
go
```

`void` تعني لا نتيجة مفيدة راجعة. فحص 0 هو **base case**، وطرح 1 يقربنا له؛ حذف أي منهما قد يستهلك الذاكرة. الدوال الصغيرة الواضحة أو Loop أنسب للتكرار البسيط، والـStack يساعد في فهم Trace عند الخطأ.

القيمة والمرجع وvariadic

التمرير العادي لا يسمح للدالة بإعادة إسناد متغير المستدعي. `&` يجعلها تعدله مباشرة؛ استخدمه بسبب واضح. `...` في تعريف الدالة يجمع عددًا متغيرًا من arguments في Array. جرّب `arguments.php` :

```php
<?php
function increment(int $n): int { return $n + 1; }
function incrementInPlace(int &$n): void { $n++; }
function sum(int ...$numbers): int { return array_sum($numbers); }

$n = 4;
echo increment($n), ':', $n, PHP_EOL;
incrementInPlace($n);
echo $n, PHP_EOL;
echo sum(...[2, 3, 4]), PHP_EOL;
```

```
5:4
5
9
```

الأول يرجع 5 ويترك n=4؛ الثاني يغير n إلى 5. `...` وقت الاستدعاء يفرد القائمة إلى arguments. إسناد object مختلف عن إعادة إسناد المتغير: ممكن دالة تعدل حالة نفس الكائن حتى بدون `&`؛ التفاصيل في OOP.

Callback: ابعت طريقة الشغل، مش ناتجها

Callable قيمة قابلة للاستدعاء. **Callback** دالة تمررها لكود آخر ليقرر وقت استدعائها. لا تعني تلقائيًا شغلًا مؤجلًا أو متوازيًا. `array_map` مثلًا تستدعيها فورًا لكل عنصر. في `callbacks.php` نكتب المستدعي بأنفسنا عشان الفكرة تبقى واضحة:

```
<?php
declare(strict_types=1);

function transform(array $values, callable $operation): array
{
    $result = [];
    foreach ($values as $value) {
        $result[] = $operation($value);
    }
    return $result;
}
$factor = 2;
$double = function (int $n) use ($factor): int {
    return $n * $factor;
};
$factor = 10;
$values = transform([1, 2, 3], $double);
echo implode(',', $values), PHP_EOL;
$plusOne = fn (int $n): int => $n + 1;
echo implode(',', transform($values, $plusOne)), PHP_EOL;
```

```
2,4,6
3,5,7
```

`transform` تستقبل Array ودالة؛ لا تعرف هل العملية ضرب أم جمع. كل دورة تستدعي `$operation($value)` وترتب النتيجة. `function (...) use ($factor)` دالة مجهولة اسمها **Closure**، وتلتقط factor بالقيمة عند إنشائها: 2 حتى بعد تغييره إلى 10. `use (&$factor)` تراقب نفس المتغير؛ استخدمها فقط لو ده مقصود. `fn` هي Arrow Function، تعبير واحد بعودة ضمنية والتقاط خارجي بالقيمة.

غلط: تمرير `$double(3)` إلى transform يمرر العدد 6، مش دالة. الصحيح `$double` بدون استدعاء. دالة مسماة ممكن تشير لها بنص (variable function) `'trim'`، أو `trim(...)` من PHP 8.1 كـFirst-class Callable؛ بعدها تستدعي القيمة. استخدم `is_callable` لو المصدر غير مضمون، ولا تجعل المستخدم يختار اسم دالة عشوائيًا. Closure محلية أنسب من إعلان function مسماة داخل function؛ الإعلان الداخلي يحدث بعد استدعاء الخارجية ويشارك نطاق أسماء الدوال وقد يتكرر.

مشروع ملفين: require ينفذ الملف

اعمل مجلدًا فيه `config.php` و`main.php`. ده مشروع كامل بدون Composer. الملف الأول يرجع بيانات؛ الثاني يحتاجها:

config.php

```php
<?php
return ['name' => 'Notebook', 'limit' => 3];
```

main.php

```php
<?php
$config = require __DIR__ . '/config.php';
echo $config['name'], ': ', $config['limit'], PHP_EOL;
```

```
Notebook: 3
```

شغّل `php main.php` . `__DIR__` مجلد الملف الحالي؛ يحميك من اختلاف مجلد التشغيل. `require` يقرأ وينفذ الملف ثم يأخذ return. بدون return صريحة نتيجة التضمين الناجح عادة 1. الملف يرث نطاق مكان التضمين؛ لذلك تجنب اعتمادًا خفيًا على متغيرات خارجية.

`include` تصدر Warning عند الفشل وغالبًا تكمل. `require` ترمي Error في PHP الحديثة، فلا يكمل المسار بدون معالجة. نسختا `_once` تمنعان إعادة تحميل التعريفات. لا تستخدم require_once لقراءة Config ثم تتوقع نفس Array في كل استدعاء؛ التحميل الثاني قد يرجع true. لا تبن مسار التضمين من Input. الدرس 10 ينقل تحميل Classes إلى Composer.

`goto` تقفز إلى label داخل الملف والنطاق نفسه، ولا تدخل بها Loop أو switch من الخارج. مثال `goto done; echo 'skip';` يطبع done، لكن دالة صغيرة أو break غالبًا أسهل في التتبع. `done: echo 'done';`

توقع، شخّص، كمّل

▼ توقع: غير factor من 2 إلى 10 بعد إنشاء Closure

مع use ($factor) تظل النتائج 2,4,6 لأن الالتقاط حدث بالقيمة. مع use (&$factor) تصبح 10,20,30. Arrow تلتقط بالقيمة أيضًا.

▼ Debugging: الدالة تعمل echo ثم أحاول جمع ناتجها

الطباعة لا ترجع القيمة للحساب. استخدم return للقيمة، وخلي echo عند عرض النتيجة؛ كده يمكن اختبار الدالة بدون التقاط الإخراج.

▼ كمّل Callback تعيد مربع العدد

`fn (int $n): int => $n * $n` ؛ تمريرها إلى transform مع [1,2,3] يعطي [1,4,9]. لا تستدعها قبل تمريرها.

▼ شخّص recursion تستدعي countdown($n) بلا تغيير

كل استدعاء يضيف Frame بنفس n فلا يصل للصفر. مرر n-1، واحتفظ بحالة توقف وحد للمدخل؛ مجرد كتابة base case لا يكفي بدون تقدم نحوها.

في مشروع الدفتر سنفصل التحقق والحفظ والعرض إلى دوال، ثم نمرر Handler كـCallback للـRouter. جرّب أيضًا `php functions-lab.php` من المختبر.

8. الملفات والـ Streams وJSON وCSV

نظام الملفات، أوضاع الفتح، الصلاحيات، streams وwrappers وcontexts وJSON وCSV.

المشكلة: البيانات بتختفي لما البرنامج يقفل

المتغيرات في الذاكرة مؤقتة. لو عايزين الملاحظة تفضل موجودة بين تشغيلتين، نكتبها في ملف. الملف Bytes على التخزين؛ البرنامج يفتحه ويقرأ أو يكتب ثم يقفله. **Stream** واجهة للتعامل مع البيانات وهي بتتحرك، زي شباك تقرأ منه جزءًا في كل مرة. **Handle** المقبض اللي يرجع من `fopen`، مش محتوى الملف.

Cursor موضع القراءة/الكتابة الحالي. **Chunk** قطعة Bytes بحجم محدد. **Buffer** مساحة وسيطة لتجميع البيانات. فهم الثلاثة يمنع تحميل ملف ضخم كله لمجرد عد سطوره. الأمثلة PHP 8.0+، و `finally` قسم تنظيف ينفذ بعد المحاولة حتى لو فشلت؛ التفاصيل في الدرس 11.

أول تجربة كاملة بدون ملفات على جهازك

احفظ `stream.php` وشغّل Stream مؤقت يبدأ في الذاكرة ويمكن أن ينتقل لملف مؤقت عند تجاوز حد الذاكرة الخاص به: `php://temp` . `php stream.php`

```
<?php
$stream = fopen('php://temp', 'w+b');
if ($stream === false) {
    throw new RuntimeException('Cannot open stream');
}
try {
    $text = "Ali\nMona\n";
    $offset = 0;
    while ($offset < strlen($text)) {
        $written = fwrite($stream, substr($text, $offset));
        if ($written === false || $written === 0) {
            throw new RuntimeException('Write made no progress');
        }
        $offset += $written;
    }
    echo 'position=', ftell($stream), PHP_EOL;
    if (!rewind($stream)) {
        throw new RuntimeException('Cannot rewind');
    }
    $count = 0;
    while (($line = fgets($stream)) !== false) {
        $count++;
        echo $count, ': ', rtrim($line, "\r\n"), PHP_EOL;
    }
    if (!feof($stream)) {
        throw new RuntimeException('Read failed before EOF');
    }
    echo "count={$count}", PHP_EOL;
} finally {
    fclose($stream);
}
```

```
position=9
1: Ali
2: Mona
count=2
```

السطر الأول يفتح موردًا؛ المقارنة الصارمة تميز الفشل. النص Bytes: 3 9 للاسم الأول وnewline و4 للثاني و `fwrite` .newline تعيد عدد Bytes المكتوبة؛ حلقة offset تعيد محاولة **الباقي فقط** وتوقف لو العدد صفر حتى لا تعلق. `ftell` يعرض المؤشر عند النهاية. `rewind` يرجعه للبداية، وإلا القراءة بعد الكتابة قد لا تجد شيئًا.

`fgets` تقرأ سطرًا، و `!== false` تميز السطر `'0'` أو الفارغ عن الفشل. `rtrim` هنا تزيل نهايات السطور فقط، مش المسافات المقصودة. `feof` بعد القراءة يميز نهاية طبيعية عن خطأ. `fclose` في finally يحرر المورد. الذاكرة في القراءة السطرية تعتمد على أطول سطر، مش ثابتة لو سطر واحد ضخم؛ `fread($stream, 8192)` تناسب قطعًا محدودة.

اختار وضع الفتح قبل ما تكتب

Mode	البداية والنتيجة
`r / r+`	الملف لازم يوجد؛ قراءة / قراءة وكتابة من البداية
`w / w+`	يمسح المحتوى فور الفتح أو ينشئ الملف
`a / a+`	ينشئ إن لزم؛ الكتابة دائمًا في النهاية
`x / x+`	إنشاء حصري؛ يفشل لو موجود
`c / c+`	ينشئ إن لزم بلا مسح؛ المؤشر في البداية

`+` تضيف القراءة والكتابة معًا، و `b` تمنع تحويلات النص الخاصة ببعض الأنظمة. **خطأ خطير:** فتح الملف بـ `w` ثم محاولة أخذ قفل؛ المسح سبق القفل. لو بتعدل بيانات موجودة ابدأ بـ `c+` وخذ القفل قبل القراءة/التعديل. وجود الملف أو نتيجة `is_writable` فحص مبدئي فقط؛ العملية نفسها قد تفشل.

JSON: شكل تخزين، مش تحقق من المعنى

Serialization تحويل بيانات إلى تمثيل قابل للحفظ. JSON مناسبة لقيم بسيطة مشتركة بين اللغات. احفظ `notes-json.php` في مجلد تجارب قابل للكتابة؛ المثال يستبدل `notes-demo.json` في نفس المجلد:

```php
<?php
$path = __DIR__ . '/notes-demo.json';
$notes = [['name' => 'Omar', 'text' => 'Learn streams']];
$json = json_encode($notes, JSON_THROW_ON_ERROR);
$bytes = file_put_contents($path, $json, LOCK_EX);
if ($bytes === false || $bytes !== strlen($json)) {
    throw new RuntimeException('Save failed');
}
$raw = file_get_contents($path);
if ($raw === false) {
    throw new RuntimeException('Read failed');
}
$loaded = json_decode($raw, true, flags: JSON_THROW_ON_ERROR);
if (!is_array($loaded) || !is_string($loaded[0]['text'] ?? null)) {
    throw new RuntimeException('Unexpected data shape');
}
echo $loaded[0]['text'], PHP_EOL;
```

```
Learn streams
```

`__DIR__` يثبت المسار. `json_encode` تحول Array إلى نص؛ `JSON_THROW_ON_ERROR` تمنع فشلًا صامتًا. `file_put_contents` مناسبة لملف صغير وتعيد عدد Bytes. `LOCK_EX` تنظم الكتابة بين مشاركين يستخدمون نفس القفل؛ لا تجعل دورة read→modify→write كلها آمنة تلقائيًا، ولا تمنع قارئًا لا يأخذ قفلًا من رؤية بيانات جزئية.

`json_decode(..., true)` تعيد Objects كـArrays، لكن JSON صالحة قد تكون عددًا أو null؛ لذلك فحص الشكل منفصل. `json_validate` من 8.3 مفيدة لو محتاج التحقق النحوي فقط؛ لو هتفك JSON لا تفحصها ثم تفكها مرتين بلا داعٍ. `JSON_UNESCAPED_UNICODE` تغير شكل النص المعروض لا معنى البيانات. لا تستخدم `unserialize` مع مدخل غير موثوق؛ `allowed_classes` ليست ضمانًا لجعل بيانات مهاجم آمنة.

CSV: الفاصلة ممكن تكون جوه الحقل

CSV جدول نصي؛ الحقول التي تحتوي فاصلة أو تنصيصًا تحتاج قواعد اقتباس. `explode(',')` لا تفهمها. احفظ `csv.php` :

```php
<?php
$stream = fopen('php://temp', 'w+b');
if ($stream === false) {
    throw new RuntimeException('Cannot open CSV');
}
try {
    $expected = ['a,b', 'say "hi"', 'back\\slash', 'عمر'];
    if (fputcsv($stream, $expected, escape: '') === false) {
        throw new RuntimeException('CSV write failed');
    }
    if (!rewind($stream)) {
        throw new RuntimeException('Cannot rewind');
    }
    $actual = fgetcsv($stream, escape: '');
    if ($actual !== $expected) {
        throw new RuntimeException('CSV round-trip failed');
    }
    echo "CSV round-trip OK", PHP_EOL;
} finally {
    fclose($stream);
}
```

```
CSV round-trip OK
```

البرنامج يكتب ثم يعيد القراءة ويقارن الأنواع والقيم. `escape: ''` صريحة لأن الاعتماد على الافتراضي deprecated من 8.4، ولأننا نريد مضاعفة التنصيص القياسية. `str_getcsv` لتحليل صف موجود كنص. اتفق على encoding والفاصل، وافحص عدد الأعمدة وأنواعها بعد التحليل. لو الملف سيفتح في Spreadsheet، قيم تبدأ بصيغة قد تُنفذ كـFormula؛ طبّق سياسة تصدير مناسبة.

Wrapper وContext وFilter من غير خلط

Wrapper يربط عنوانًا بمصدر: `file://` ملفات، `php://memory` ذاكرة فقط، `php://temp` ذاكرة ثم تخزين مؤقت، `php://input` جسم طلب HTTP الخام. توجد أيضًا `http://` و`ftp://` و`data://` و`compress.zlib://` حسب الإضافات والإعدادات؛ يمكن تسجيل Wrapper بـ`stream_wrapper_register`، لكن مش مطلوب للمبتدئ.

Context إعدادات العملية مثل timeout وHTTP headers؛ لا يحول الفشل لنجاح. **Filter** يحول Bytes أثناء مرورها. تجربة `filter.php` كاملة:

```php
<?php
$stream = fopen('php://temp', 'w+b');
if ($stream === false) {
    throw new RuntimeException('Open failed');
}
try {
    if (fwrite($stream, 'hello') !== 5 || !rewind($stream)) {
        throw new RuntimeException('Prepare failed');
    }
    $filter = stream_filter_append($stream, 'string.toupper', STREAM_FILTER_READ);
    if ($filter === false) {
        throw new RuntimeException('Filter failed');
    }
    $text = stream_get_contents($stream);
    if ($text === false) {
        throw new RuntimeException('Read failed');
    }
    echo $text, PHP_EOL;
} finally {
    fclose($stream);
}
```

```
HELLO
```

نربط الفلتر بالقراءة فقط، فنقرأ نسخة محولة؛ فلتر ASCII ده مش بديلًا لتحويل Unicode. لإنشاء إعداد HTTP مثلًا `stream_context_create(['http' => ['timeout' => 3, 'header' => "Accept: application/json\r\n"]])`، ثم تمرره إلى دالة القراءة. الطلب الحقيقي يحتاج فحص status وTLS وحد حجم؛ `ignore_errors` يسمح بقراءة جسم رد فاشل ولا يجعله ناجحًا. `allow_url_fopen` يتحكم في URL wrappers؛ اترك `allow_url_include` معطلًا، ولا تستخدم عنوان مستخدم للقراءة الحرة من شبكة الخادم.

الحفظ والصلاحيات

في Unix الأرقام قراءة=4 وكتابة=2 وتنفيذ=1 لكل مالك/مجموعة/آخرين؛ `0600` للمالك فقط، `0644` تسمح للآخرين بالقراءة، `0755` شائعة للمجلدات. `chmod($path, 0640)` ليست بديلًا عن ضبط مستخدم الخدمة، وWindows تعتمد ACLs؛ لا تعالج خطأ الصلاحيات بـ `0777`.

لإضافة سجلات خذ `flock(..., LOCK_EX)`، اكتب كل Bytes، ثم `fflush` وحرر القفل في finally. القارئ المتعاون يأخذ `LOCK_SH`. لمنع رؤية نسخة جزئية عند الاستبدال استخدم ملفًا مؤقتًا في نفس filesystem ثم rename مع فحص الفشل واختبار المنصة؛ `fflush` وحدها لا تضمن النجاة من انقطاع الكهرباء. القفل المنفصل الثابت مطلوب لو بتستبدل الملف نفسه وعايز تنسيقًا بين عمليات متعددة.

مسار التخزين يحدده التطبيق، لا اسم يرسله المستخدم. `realpath` مفيد للموجود، لكنه يعيد false لملف جديد؛ تحقق من المجلد الأب والحد الفاصل، وانتبه للروابط الرمزية. JSON ضخمة تحتاج parser تدفقيًا، مش مجرد fopen قبل json_decode.

توقع، شخّص، كمّل

▼ توقع: حذف rewind من stream.php

المؤشر بعد الكتابة عند 9، فتبدأ القراءة من النهاية ولا تجد السطور؛ count=0. المقبض لا يرجع للبداية تلقائيًا.

▼ Debugging: if (!$raw) بعد file_get_contents

يخلط الفشل بالنص الفارغ أو '0'. الصحيح `$raw === false`، ثم افحص إن كان الفراغ مسموحًا حسب العقد.

▼ كمّل شرط يوقف حلقة الكتابة بدون تقدم

`$written === false || $written === 0`. فحص false وحدها يترك احتمال دوران بلا نهاية عند صفر.

▼ هل LOCK_EX على الحفظ وحده يمنع ضياع تحديثين قرآ نفس الملف؟

لا. قد يقرأ الاثنان الحالة القديمة ثم يستبدل الثاني تعديل الأول. اقفل دورة القراءة والتعديل والكتابة كلها، أو استخدم قاعدة بيانات بمعاملة عند تطور التطبيق.

شغّل أيضًا `php stream-lab.php fixtures/large.csv` من المختبر. في الدرس 17 سنحفظ كل ملاحظة في ملف مستقل تحت مجلد خاص؛ اختبر ملفًا فارغًا وآخر مفقودًا وJSON تالفة كحالات مختلفة.

مراجع: CSV، Streams، الكتابة.

9. رفع الملفات والكوكيز والجلسات

رفع الملفات بأمان، خيارات Cookies، CSRF، ودورة حياة الجلسة وحمايتها.

المشكلة: كل Request بتبدأ من جديد

المتصفح يفتح صفحة ثم يرسل نموذجًا، لكن المتغيرات المحلية في الطلب الأول لا تنتقل للثاني. نحتاج طريقة تربط الطلبات. **Cookie** قيمة صغيرة يحفظها المتصفح ويرسلها مع الطلب المناسب. **Session** بيانات على الخادم مرتبطة بمعرّف عشوائي؛ المتصفح يحمل المعرّف فقط في الحالة المعتادة. **Upload** نقل Bytes ملف إلى الخادم؛ دي مشكلة مختلفة هنطبق عليها نفس التحقق من المدخلات.

الدرس لا يبني نظام تسجيل دخول؛ هدفه فهم نقل البيانات واستمرار الحالة. تفاصيل الهوية والصلاحيات في مسار الأمان. الأمثلة PHP 8.1+، مع mbstring وfileinfo.

دورة الجلسة خطوة خطوة

1. المتصفح يرسل أول GET بدون Session Cookie.
2. `session_start` تطلب من مخزن الجلسات فتح حالة. بدون معرّف مقبول تنشئ PHP معرّفًا جديدًا.
3. الخادم يرسل `Set-Cookie` في **Response header**، فيحفظه المتصفح.
4. الطلب التالي يحمل `Cookie` في **Request header**.
5. `session_start` تقرأ البيانات إلى `$_SESSION` ؛ مع file handler الافتراضي تأخذ قفل الجلسة أثناء الاستخدام.
6. `session_write_close` أو نهاية الطلب تحفظ التغييرات وتحرر القفل.

`setcookie` لا تغير `$_COOKIE` في الطلب الحالي؛ دي صورة لما وصل بالفعل. و `session_start` لازم تسبق HTML وأي echo لأن الجلسة قد تحتاج إرسال headers.

برنامج كامل: النموذج يتذكر اسمك

أنشئ مجلدًا فيه `public/preferences.php` . شغّل `php -S 127.0.0.1:8081 -t public` وافتح النموذج المحلي. السيرفر التعليمي محلي فقط. هذا المثال يضبط Secure=false عمدًا لاتصال HTTP المحلي؛ عند نشر HTTPS اضبطها true من إعداد موثوق، لا من Header يرسله المستخدم.

CSRF إرسال موقع آخر طلب تغيير باسم متصفحك. **Token** هنا قيمة عشوائية في الجلسة والنموذج نطابقهما قبل التغيير؛ ليست Password ولا Session ID.

```php
<?php
declare(strict_types=1);

if (!session_start([
    'use_strict_mode' => true,
    'use_only_cookies' => true,
    'cookie_httponly' => true,
    'cookie_secure' => false,
    'cookie_samesite' => 'Lax',
    'cookie_path' => '/',
])) {
    throw new RuntimeException('Session unavailable');
}
$_SESSION['csrf'] ??= bin2hex(random_bytes(32));
$error = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $token = $_POST['csrf'] ?? null;
    if (!is_string($token) || !hash_equals($_SESSION['csrf'], $token)) {
        http_response_code(403);
        exit('Invalid form token');
    }
    $name = $_POST['name'] ?? null;
    if (!is_string($name) || !mb_check_encoding($name, 'UTF-8')) {
        $error = 'Name must be UTF-8 text';
    } else {
        $name = trim($name);
        if ($name === '' || mb_strlen($name, 'UTF-8') > 40) {
            $error = 'Use 1 to 40 code points';
        } else {
            $_SESSION['name'] = $name;
            session_write_close();
            header('Location: /preferences.php', true, 303);
            exit;
        }
    }
    http_response_code(422);
}
$name = $_SESSION['name'] ?? 'Guest';
$csrf = $_SESSION['csrf'];
session_write_close();
function escape(string $value): string
{
    return htmlspecialchars($value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}
?>
<!doctype html>
<html lang="en"><meta charset="utf-8"><title>Preferences</title>
<p>Hello <?= escape($name) ?></p>
<p><?= escape($error) ?></p>
<form method="post">
  <input type="hidden" name="csrf" value="<?= escape($csrf) ?>">
  <label>Name <input name="name" maxlength="40" required></label>
  <button>Save</button>
</form>
</html>
```

توقع أول زيارة `Hello Guest`. اكتب Omar، فالـPOST ترجع 303 ثم المتصفح يعمل GET وتظهر `Hello Omar`. افتح نافذة خاصة: ترجع Guest لأنها جلسة مختلفة. أدخل مسافات فقط: 422 ولا يتغير الاسم. احذف token من الطلب: 403 قبل الحفظ.

اقرأ البرنامج بالترتيب: نضبط الجلسة ونبدأها، ننشئ CSRF مرة، نميز POST، نفحص النوع قبل hash_equals، ثم UTF-8 والطول. `maxlength` مساعدة في المتصفح وليست تحقق الخادم؛ المتصفح وPHP قد يعدان وحدات النص بشكل مختلف، وهنفصل Unicode في الدرس 13. `303` تطبق **Post/Redirect/Get** فتحديث الصفحة يعيد GET، لكنه ليس حماية عامة من تكرار POST. ننسخ القيم

المطلوبة ثم نقفل الجلسة مبكرًا. `htmlspecialchars` تحمي سياق HTML النصي والـattribute المقتبس عند العرض، ولا تغير البيانات المخزنة.

Cookie flags وحدودها

الإعداد	ما يفعله
Secure	إرسال عبر HTTPS؛ المثال المحلي فقط يستعمل false
HttpOnly	يمنع قراءة Cookie عبر JavaScript، لكنه لا يمنع طلبات XSS
SameSite=Lax	يقلل الإرسال في سياقات cross-site؛ لا يستبدل CSRF token
SameSite=None	يحتاج Secure؛ لا تستخدمه بلا سبب
Path/Domain	تحدد متى يرسلها المتصفح؛ ليستا نظام صلاحيات
expires / cookie_lifetime	عمر Cookie، وليس وحده عمر صلاحية بيانات الخادم

Cookie تفضيل مثل theme ممكن يغيرها المستخدم؛ تحقق من قيمة ضمن `['light', 'dark']`. لا تخزن Password أو صلاحية إدارية خامًا فيها. خزن البيانات الحساسة في مخزن مناسب وخلي المعرّفات نفسها أسرارًا.

Session Fixation: المهاجم يعرف التذكرة قبل الدخول

Fixation يعني إقناع الضحية باستخدام Session ID يعرفه المهاجم، ثم يظل المعرّف صالحًا بعد نجاح تسجيل الدخول. السرقة مختلفة: الحصول على معرّف ضحية موجود بالفعل. `use_strict_mode` ترفض معرّفات غير مهيأة، لكنها لا تحل كل سيناريوهات التثبيت لجلسة موجودة.

بعد التحقق الحقيقي من بيانات الدخول، غيّر المعرّف **قبل** إضافة هوية المستخدم الجديدة. المقطع التالي خاص بحد تسجيل الدخول في نظام مجهز، وليس Login جاهزًا:

```
// After credentials were verified and a session was started:
if (!session_regenerate_id(false)) {
    throw new RuntimeException('Cannot rotate session');
}
$_SESSION['user_id'] = $verifiedUserId;
$_SESSION['authenticated_at'] = time();
```

`false` تحتفظ ببيانات الجلسة القديمة لتجنب قطع الطلبات المتزامنة؛ لكن ده مش تصميم إبطال مكتمل. النظام الفعلي يحتاج علامة obsolete/timestamp في السجل القديم، فترة انتقال قصيرة وسياسة تمنع استخدامه بصلاحيات جديدة، وانتهاء خمول وعمر كلي يطبقهما الخادم. حذف القديم فورًا بـ `true` قد يناسب مثالًا متسلسلًا، لكنه يسبب فقد جلسات وسباقات على شبكة غير مستقرة. لا تعرض المعرّف ولا تسجله في logs. اتبع إدارة أمان الجلسات ودرس الجلسات.

إنهاء الجلسة غير مسح متغير

في مسار POST محمي بـCSRF وبعد session_start: امسح `$_SESSION = []` ، واحذف Cookie بنفس Path وDomain وflags بوضع expires في الماضي، ثم `session_destroy` . الدالة وحدها لا تمسح Cookie ولا المصفوفة المحلية. `session_unset` تمسح المتغيرات فقط. `session_write_close` تحفظ وتحرر القفل؛ التعديل على `$_SESSION` بعدها لا يُحفظ تلقائيًا. `session_start(['read_and_close' => true])` للقراءة فقط، لكن انتبه لسياسة انتهاء المخزن.

مخزن Redis أو Database يحتاج `SessionHandlerInterface` أو `session_set_save_handler` وسياسة قفل وانتهاء واضحة. Garbage collection تنظيف احتمالي للتخزين، مش قرار صلاحية الطلب.

رفع صورة: من نموذج لملف خاص

أنشئ `public/upload.php` في نفس المجلد؛ الكود ينشئ `storage/uploads` خارج public. **MIME** وصف لنوع المحتوى؛ نوع المتصفح وامتداد الاسم غير موثوقين. `multipart/form-data` طريقة إرسال حقول وملفات. المثال يسمح PNG/JPEG حتى 2 MiB، ويتطلب CSRF:

```php
<?php
declare(strict_types=1);
session_start([
    'use_strict_mode' => true,
    'use_only_cookies' => true,
    'cookie_httponly' => true,
    'cookie_secure' => false,
    'cookie_samesite' => 'Lax',
]);
$_SESSION['upload_csrf'] ??= bin2hex(random_bytes(32));
$csrf = $_SESSION['upload_csrf'];
session_write_close();
$message = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $token = $_POST['csrf'] ?? null;
    if (!is_string($token) || !hash_equals($csrf, $token)) {
        http_response_code(403);
        exit('Invalid form token');
    }
    $file = $_FILES['avatar'] ?? null;
    if (!is_array($file)
        || ($file['error'] ?? null) !== UPLOAD_ERR_OK
        || !is_string($file['tmp_name'] ?? null)
        || !is_uploaded_file($file['tmp_name'])) {
        http_response_code(422);
        exit('Upload failed');
    }
    $size = filesize($file['tmp_name']);
    $mime = (new finfo(FILEINFO_MIME_TYPE))->file($file['tmp_name']);
    $extensions = ['image/png' => 'png', 'image/jpeg' => 'jpg'];
    if ($size === false || $size < 1 || $size > 2 * 1024 * 1024
        || !is_string($mime) || !isset($extensions[$mime])) {
        http_response_code(422);
        exit('Unsupported file');
    }
    $directory = dirname(__DIR__) . '/storage/uploads';
    if (!is_dir($directory) && !mkdir($directory, 0700, true) && !is_dir($directory)) {
        throw new RuntimeException('Storage unavailable');
    }
    $name = bin2hex(random_bytes(16)) . '.' . $extensions[$mime];
    if (!move_uploaded_file($file['tmp_name'], $directory . '/' . $name)) {
        throw new RuntimeException('Save failed');
    }
    $message = 'Saved privately';
}
?>
<!doctype html>
<html lang="en"><meta charset="utf-8"><title>Upload</title>
<p><?= $message ?></p>
<form method="post" enctype="multipart/form-data">
  <input type="hidden" name="csrf" value="<?= htmlspecialchars($csrf, ENT_QUOTES, 'UTF-8') ?>">
  <input type="file" name="avatar" accept="image/png,image/jpeg" required>
  <button>Upload</button>
</form>
</html>
```

افتح `/upload.php`: الصورة المقبولة تعطي `Saved privately` وملفًا باسم مولد؛ نص باسم .jpg يُرفض بـ422. نتحقق من error وشكل tmp_name قبل استخدامه؛ ملف برقم خطأ قد لا يوجد أصلًا. `filesize` و`finfo` يفحصان الملف المؤقت على الخادم. `move_uploaded_file` تنقل ملف Upload فعلي. رسالة النجاح ثابتة، واسم العميل لا يدخل المسار.

`accept` اختيار واجهة فقط. MIME لا تثبت أن الصورة آمنة لكل استخدام؛ قبل العرض الفعلي طبق فك/إعادة ترميز وحد أبعاد وفحصًا مناسبًا. اضبط `upload_max_filesize` و`post_max_size` (الأخير أكبر لاستيعاب تغليف multipart) وحد جسم الطلب بالخادم. تجاوز post_max_size قد يترك POST وFILES فارغتين؛ ميّز 413 في طبقة حد الطلب كما سنعمل في المشروع. مرجع الرفع.

توقع، شخّص، كمّل

▼ توقع: ضغط Save ثم فتح نافذة خاصة

النافذة الأولى تحتفظ بالاسم؛ الخاصة لها Cookie jar مختلفة فتبدأ Guest. الاسم على الخادم، والمعرّف هو الرابط.

▼ Debugging: session_start بعد طباعة HTML

قد تكون headers اتبعتت فلا تستطيع PHP إرسال Cookie. انقل بدء الجلسة قبل كل إخراج، وراجع مسافات/BOM قبل الوسم؛ output buffering مش إصلاحًا لترتيب غير واضح.

▼ كمّل الحماية من name[]=Omar

افحص `is_string($name)` قبل trim أو mb_strlen. اسم الحقل لا يضمن نوعه؛ الطلب قد يحمل Array.

▼ مهاجم يعرف Session ID قبل الدخول. هل HttpOnly تكفي؟

لا؛ HttpOnly تخص قراءة JavaScript. تحتاج strict mode وتغيير المعرّف عند انتقال الصلاحية وإبطال القديم بسياسة صحيحة؛ راجع fixation.

▼ ملف avatar.php.jpg، والحقل type يقول image/jpeg. نقبله؟

لا بناءً على الاسمين. افحص الخطأ والحجم والمحتوى، ولّد الاسم، واحفظه خارج public. غير المسموح لا يصل لمرحلة النقل.

برنامج preferences هو مرحلة Session→Validation→Form. احتفظ به؛ الدرس 17 يضيف Files و`php tests.php` Router. `security` من المختبر اختبار مساعد للسياسات وليس بديلًا لتجربة multipart الفعلية.

10. Namespaces وComposer وأدوات الجودة

تنظيم الأسماء والاستيراد والـaliases وحل الأسماء وربط Namespace بمجلد عبر Composer PSR-4.

مشكلتان مختلفتان: اسم متكرر وملف مش متحمّل

مشروعك ومكتبة خارجية عندهم Class اسمها User. **Namespace** عنوان للاسم يمنع التصادم، مثل محافظتين فيهم نفس اسم الشارع. **Autoloading** حل لمشكلة أخرى: عند طلب Class، مين يحمّل ملفها؟ **Composer** مدير حزم: يجلب المكتبات المتوافقة ويولد محمّلًا للأسماء.

هنستخدم Class صغيرة كحاوية للدالة فقط. تفاصيل تصميم Classes في OOP. تحتاج PHP 8.3+ وComposer 2 للمشروع التالي. تحقق بـ `php -v` و`composer --version`، وشغّل كل الأوامر من مجلد المشروع، لا من مجلد الموقع.

المشروع كاملًا

النسخة الجاهزة موجودة في `examples/php-course/composer-demo` داخل المستودع، وكمان في تحميل أمثلة المنهج. أو أنشئ الملفات الأربعة التالية بنفس الحروف:

```
composer-demo/
  composer.json
  bin/demo.php
  src/Billing/PriceCalculator.php
  tests/PriceCalculatorTest.php
```

1. composer.json: وصف المشروع

`require` احتياجات التشغيل، و`require-dev` أدوات التطوير. الإصدارات هنا نطاقات متوافقة مقصودة للمثال، وليست وعدًا بأنها أحدث إصدار:

```
{
  "name": "learning/price-demo",
  "type": "project",
  "require": {
    "php": "^8.3"
  },
  "require-dev": {
    "phpunit/phpunit": "^11.5",
    "phpstan/phpstan": "^2.1",
    "friendsofphp/php-cs-fixer": "^3.0"
  },
  "autoload": {
    "psr-4": {
      "App\\": "src/"
    }
  },
  "scripts": {
    "demo": "@php bin/demo.php",
    "test": "@php vendor/bin/phpunit --bootstrap vendor/autoload.php tests",
    "analyse": "@php vendor/bin/phpstan analyse src --level=6 --no-progress",
    "style": "@php vendor/bin/php-cs-fixer fix src --dry-run --diff --rules=@PSR12",
    "check": [
      "@test",
      "@analyse",
      "@style"
    ]
  },
  "config": {
    "platform": {
      "php": "8.3.0"
    }
  },
  "description": "A small PSR-4 teaching project with boundary tests and development tools"
}
```

`App\\` في JSON تمثل النص `App\` لأن backslash تُهرب. **PSR-4** اتفاق يربط Prefix بمجلد. Composer يحذف `App\` من الاسم، ويحوّل الباقي لمسار: `App\Billing\PriceCalculator` ← `src/Billing/PriceCalculator.php`. اللغة نفسها لا تفرض علاقة الاسم بالمجلد؛ الاتفاق هو اللي يفرضها.

2. الصنف: src/Billing/PriceCalculator.php

```
<?php

declare(strict_types=1);

namespace App\Billing;

final class PriceCalculator
{
    public function subtotal(int $price, int $quantity): int
    {
        if ($price < 0 || $quantity < 1 || $price > intdiv(PHP_INT_MAX, $quantity)) {
            throw new \InvalidArgumentException('Invalid order');
        }

        return $price * $quantity;
    }
}
```

`namespace` تأتي بعد declare وقبل التعريفات. الاسم الكامل للصنف أصبح `public function` .App\Billing\PriceCalculator دالة نستدعيها على الكائن. الشرط يرفض سعرًا سالبًا وكمية أقل من 1. `||` تتوقف مبكرًا، فلا نقسم على كمية صفر. مقارنة السعر بـ `intdiv(PHP_INT_MAX, $quantity)` ترفض الضرب الذي سيتجاوز الحد **قبل** تنفيذه. `\InvalidArgumentException` اسم كامل من النطاق العام.

3. نقطة التشغيل: bin/demo.php

```
<?php
declare(strict_types=1);

use App\Billing\PriceCalculator;

require dirname(__DIR__) . '/vendor/autoload.php';

$calculator = new PriceCalculator();
echo $calculator->subtotal(1500, 3), PHP_EOL;
```

`use` اختصار للاسم في الملف الحالي؛ لا يقرأ ملفًا ولا ينشئ كائنًا. `require vendor/autoload.php` تسجل المحمّل، و `new` تطلب الصنف فيحمّله Composer. الأمر `composer demo` يشغّل `@php` بنفس PHP التي تستخدمها Composer. النتيجة `4500` .

4. اختبار السلوك: tests/PriceCalculatorTest.php

Test تجربة آلية لها مدخل ومتوقع. **Assertion** تحقق يفشل برسالة لو خالف الفعلي المتوقع. PHPUnit تكتشف دوال test؛ القيد 11.5 هنا يدعم خط PHP المستخدم في المثال:

```php
<?php
declare(strict_types=1);

use App\Billing\PriceCalculator;
use PHPUnit\Framework\TestCase;

final class PriceCalculatorTest extends TestCase
{
    public function testValidOrder(): void
    {
        self::assertSame(4500, (new PriceCalculator())->subtotal(1500, 3));
    }

    public function testZeroPriceIsAllowed(): void
    {
        self::assertSame(0, (new PriceCalculator())->subtotal(0, 1));
    }

    public function testZeroQuantityIsRejected(): void
    {
        $this->expectException(InvalidArgumentException::class);
        (new PriceCalculator())->subtotal(100, 0);
    }

    public function testNegativePriceIsRejected(): void
    {
        $this->expectException(InvalidArgumentException::class);
        (new PriceCalculator())->subtotal(-1, 1);
    }

    public function testOverflowIsRejected(): void
    {
        $this->expectException(InvalidArgumentException::class);
        (new PriceCalculator())->subtotal(PHP_INT_MAX, 2);
    }
}
```

`assertSame` تفحص النوع والقيمة؛ expectedException تُكتب قبل العملية المتوقع فشلها. الاختبارات تثبت قواعد السعر والحدود، مش أسماء المتغيرات. التشغيل الأول لإنشاء lock:

```
composer update
composer demo
composer check
composer audit
```

composer.lock: نفس النسخ عند زميلك

`composer.json` يعلن النطاقات؛ `composer.lock` يسجل النسخ الدقيقة المحلولة. **install** مع lock يثبت تلك النسخ. **update** يعيد حل النطاقات وقد يغير lock. في تطبيق مثل ده التزم بالملفين في Git، ولا تعدل lock يدويًا. زميلك يستخدم `composer install`، لا update لمجرد بدء العمل. `vendor` مخرجات قابلة لإعادة الإنشاء؛ لا تلتزم بها.

SemVer شكل `major.minor.patch` : تغيير كاسر، إضافة متوافقة، إصلاح متوافق بحسب التزام الحزمة. `^2.1` تسمح `>=2.1.0` `~2.1.0` . `^0.3.2` تسمح `<3.0.0` . `>=2.1.0 <2.2.0` تتوقف قبل `0.4.0` لأن ما قبل 1.0 أكثر تحفظًا. النطاق وعد توافق من الناشر، مش بديلًا للاختبار.

`composer require vendor/package` تضيف اعتمادًا وتحدث lock؛ `composer require --dev ...` لأداة تطوير. راجع diff والأصل قبل اعتماد الحزمة. `composer audit` يفحص تنبيهات منشورة وفق بيانات وقت التشغيل؛ نجاحه لا يثبت خلو البرنامج من كل ثغرة. حدّث اعتمادًا محددًا بـ `composer update vendor/package` واختبر بعده.

scripts والتحليل والتنسيق

Script أمر مسمى في composer.json. `composer check` يشغل test ثم analyse ثم style؛ الفشل يوقف السلسلة بكود خروج غير صفر. scripts وpluginsيمكنها تنفيذ كود؛ اقرأ ما يجلبه المشروع قبل تشغيله.

Static analysis فحص الأنواع ومسارات الكود بدون تشغيل كل المدخلات. PHPStan هنا تفحص src عند level 6. جرّب مؤقتًا إعادة نص بدل int في subtotal: التحليل يجب أن يفشل، ثم ارجع الإصلاح. التحليل لا يغني عن اختبار حساب صحيح بنوع صحيح.

PHP-CS-Fixer يراجع شكل الكود. `--dry-run --diff` يعرض التغييرات بدون تطبيق. لتطبيقها عن قصد استخدم `composer exec -- php-cs-fixer fix src --rules=@PSR12`. التنسيق لا يصلح المنطق. Pest وPsalm وPint بدائل، لكن لا نثبت أداتين لنفس الدور في هذا المعمل.

تشخيص Class not found

اتبع الاسم خطوة بخطوة: use، ثم Namespace في الملف، ثم Prefix في Composer، ثم المسار وحالة الحروف، ثم وجود require للمحمّل. Linux غالبًا حساس للحروف؛ نجاح اسم `billing` بدل `Billing` على جهازك لا يثبت صحته. بعد تغيير خريطة autoload شغّل `composer dump-autoload` . إضافة Class صحيحة تحت PSR-4 عادة لا تحتاج تحديث المحمّل العادي؛ الـclassmap المحسن/authoritative يحتاج إعادة توليد حسب إعدادك.

الأسماء التي تبدأ بـ `\` كاملة. `namespace\Name` نسبية للمكان الحالي. الاسم القصير لصنف يتبع imports ثم Namespace الحالية؛ دوال وثوابت غير مؤهلة قد ترجع للنطاق العام عند غياب المحلي، فلا تعمم قواعد الأصناف عليها. يمكن `use X\User as ExternalUser` و `use function X\helper` وimports . `use const X\LIMIT` وقت التحليل وعلى مستوى الملف، ليست use الخاصة بالتقاط Closure.

Composer تدعم `classmap` لكود لا يتبع PSR-4، و `files` لدوال يجب تحميلها دائمًا، و `autoload-dev` لتعريفات الاختبارات. لا تجعل ملف Class ينفذ طلبات شبكة عند تحميله.

إعداد التطوير والإنتاج

التطوير يثبت require-dev ويشغّل check. النشر من lock باستخدام `composer install --no-dev --optimize-autoloader` ثم `composer check-platform-reqs --no-dev` على بيئة الإنتاج الفعلية. `config.platform` تساعد حل الاعتمادات لمنصة مستهدفة، لكنها لا تثبت امتدادًا ولا تغير PHP الحقيقية. لا تشغّل update عشوائيًا أثناء النشر. إعدادات الأخطاء وXdebug في الدرسين 11 و17.

توقع، شخّص، كمّل

▼ توقع: حذف vendor ثم composer install مع lock

تعاد نفس النسخ المقفلة ضمن المنصة المتوافقة؛ install ليست بحثًا عن أحدث نسخة. لو المنصة ناقصة امتدادًا يفشل بدل حلها بالتخمين.

▼ Debugging: use صحيح لكن الصنف غير موجود

use لا يحمل الملف. تأكد من vendor/autoload.php والخريطة واسم الملف وNamespace والحروف؛ ثم dump-autoload عند تعديل الخريطة.

▼ كمّل PSR-4 للاسم App\Billing\PriceCalculator

Prefix هو `App\` ومجلده src/، والباقي Billing/PriceCalculator.php. في JSON تكتب backslash مرتين لتمثيل واحدة.

▼ اختبار pass وتنسيق pass، هل ده يضمن صحة كل المدخلات؟

لا. كل أداة تثبت جزءًا: اختبارات حالات محددة، وتحليل عقود، وتنسيق شكل. أضف حالات حدود نابعة من القاعدة، لا اختبارات تكرر التنفيذ.

المراجع: Composer، القيود، PHP-CS-Fixer، PHPStan، PHPUnit.

11. الأخطاء والاستثناءات

Throwable وError وException وtry/catch/finally والاستثناءات المخصصة وحدود المعالجة والتسجيل الآمن.

المشكلة: الفشل مش نتيجة عادية

حساب رصيد ممكن ينجح، أو يرفض مبلغًا سالبًا، أو يرفض مبلغًا أكبر من الرصيد. وقراءة ملف ممكن تفشل لأن القرص غير متاح. لو كل الحالات ترجع `null`، المستدعي مش هيعرف هل دي «لا توجد بيانات» ولا Bug.

Exception إعلان إن العملية لم تنتج نتيجتها الطبيعية. `throw` ترمي الفشل؛ `try` تحدد المنطقة التي نراقبها، و`catch` تستقبل نوعًا تعرف تعالجه. **Throwable** المظلة التي تجمع Exception وError. الأخيرة تشمل أخطاء مثل TypeError وDivisionByZeroError؛ مش كل خطأ في PHP Exception: Warning قد تطبع تشخيصًا وترجع false.

الدرس PHP 8.1+. ارجع لـCall Stack في الدرس 7: الاستثناء يمشي عكس سلسلة الاستدعاءات لحد catch مناسبة.

برنامج كامل بثلاث حالات

احفظ `withdraw.php` وشغّل **Domain**. `php withdraw.php` هنا قواعد السحب، وDomainException فشل قاعدة عمل رغم أن النوع صحيح:

```
<?php
declare(strict_types=1);

function withdraw(int $balance, int $amount): int
{
    if ($balance < 0 || $amount <= 0) {
        throw new InvalidArgumentException('Use a nonnegative balance and positive amount');
    }
    if ($amount > $balance) {
        throw new DomainException('Insufficient funds');
    }
    return $balance - $amount;
}
foreach ([200, 1200, -1] as $amount) {
    echo "request={$amount}", PHP_EOL;
    try {
        $remaining = withdraw(1000, $amount);
        echo "remaining={$remaining}", PHP_EOL;
    } catch (InvalidArgumentException $error) {
        echo "invalid input", PHP_EOL;
    } catch (DomainException $error) {
        echo "declined", PHP_EOL;
    } finally {
        echo "finished attempt", PHP_EOL;
    }
}
```

```
request=200
remaining=800
finished attempt
request=1200
declined
finished attempt
request=-1
invalid input
finished attempt
```

كل دورة تبدأ برصيد 1000؛ المثال يقارن حالات مستقلة، مش كشف حساب متراكم. 200 ترجع 800؛ 1200 ترمي قبل return، فيُتخطى echo remaining وتعمل catch المجال. -1 تدخل فحص المدخل. finally تعمل في الحالات الثلاث. لا تجعل catch عامة قبل الخاصة لأنها ستلتقط النوع مبكرًا.

لو دالة داخل withdraw رمت، يُلغى مسارها ثم يبحث PHP خارجها. catch لا تعيد التنفيذ للسطر الذي فشل؛ تكمل بعد الكتلة المعالجة. بدون catch يصل الفشل للحد العام وينتهي الطلب/السكربت عادة.

finally للتنظيف، مش لتغيير النتيجة

المورد المفتوح يحتاج إغلاقًا في النجاح والفشل. تجربة `cleanup.php`:

```php
<?php
$stream = fopen('php://temp', 'w+b');
if ($stream === false) {
    throw new RuntimeException('Open failed');
}
try {
    try {
        throw new RuntimeException('Simulated read failure');
    } finally {
        fclose($stream);
        echo "closed", PHP_EOL;
    }
} catch (RuntimeException $error) {
    echo "handled", PHP_EOL;
}
echo is_resource($stream) ? "open\n" : "not open\n";
```

```
closed
handled
not open
```

الـfinally الداخلية تعمل أثناء خروج الاستثناء، وبعدها catch الخارجية. finally تعمل أيضًا مع return العادية، لكن لا تعتمد عليها بعد `exit` أو إنهاء العملية بالقوة. **خطأ:** return داخل finally قد تخفي نتيجة أو Exception؛ اتركها للتنظيف الذي لا يخفي سبب الفشل الأصلي.

استثناء مخصص مثل `final class InsufficientStock extends DomainException {}` يعطي نوعًا واضحًا، لكن لا تحتاج صنفًا لكل رسالة. عند تغليف فشل احتفظ بـ `throw new RuntimeException('Could not load notes', 0, $error);` previous:. قرارات البرنامج تعتمد على النوع، لا على مقارنة نص الرسالة.

Warning مش هتدخل catch تلقائيًا

`file_get_contents` قد تصدر Warning وترجع false. ضع فحصًا صريحًا كما في درس الملفات. `@` يخفي التشخيص ولا يصلح القراءة. إذا احتجت تحويل Warning إلى Exception عند حد I/O محدد، ثبّت handler مؤقتًا وأعد السابق في finally، واحترم `error_reporting()`. لا تحول كل Notice في تطبيق كامل عشوائيًا ثم ترجع نجاحًا.

في worker طويل العمر، catch على حد الرسالة تقرر retry أو رفض المهمة وإعادة تنظيف الحالة. retry مناسبة فقط لفشل مؤقت وعملية يمكن تكرارها بدون تنفيذ أثر مرتين؛ الاسم التقني TypeError. **idempotent** داخل الحساب Bug يحتاج إصلاحًا، مش خمس محاولات.

التطوير والإنتاج: نفس الخطأ وعرض مختلف

Development بيئة تجرب فيها، و**production** خدمة يستخدمها الناس. اعرف الملف المحمل بـ `php --ini`؛ إعداد CLI قد يختلف عن FPM/Apache. مثال ملف ini للتطوير:

```
error_reporting=E_ALL
display_errors=On
display_startup_errors=On
log_errors=On
```

وفي الإنتاج، مع error_log في مسار محمي قابل لكتابة الخدمة:

```
error_reporting=E_ALL
display_errors=Off
display_startup_errors=Off
log_errors=On
zend.exception_ignore_args=On
```

الهدف الاحتفاظ بالتشخيص في سجل محمي بدل إرساله للمستخدم. لا تسجل Password أو Session ID أو Body كاملة. حجب arguments من trace يقلل تسربًا محتملًا لكنه لا ينقح رسالة كتبت أسرارًا يدويًا. خطأ parsing في نفس الملف قد يحدث قبل ini_set؛ إعداد php.ini يسبق التنفيذ.

حد تطبيق كامل يعيد JSON

احفظ `boundary.php` وشغّله بخادم PHP محلي. الطلب يرجع 500 وJSON بها request_id مختلف كل مرة. السجل يحتوي النوع والمكان فقط؛ ده مثال لتقليل البيانات، ويمكن إضافة سياق منقح حسب الحاجة:

```php
<?php
declare(strict_types=1);

set_exception_handler(static function (Throwable $error): void {
    $id = bin2hex(random_bytes(8));
    error_log(json_encode([
        'event' => 'unhandled_failure',
        'request_id' => $id,
        'type' => get_class($error),
        'file' => basename($error->getFile()),
        'line' => $error->getLine(),
    ], JSON_THROW_ON_ERROR));
    http_response_code(500);
    header('Content-Type: application/json; charset=utf-8');
    header('Cache-Control: no-store');
    echo json_encode(['error' => 'Internal error', 'request_id' => $id], JSON_THROW_ON_ERROR);
});
throw new RuntimeException('Demonstration failure');
```

المعالج شبكة أمان للفشل غير المتوقع؛ لا يستبدل 422 عند فشل Validation. لو سبق إرسال body، مش هتقدر تصلح الاستجابة بالكامل؛ خلي الإخراج في نقطة واحدة. catch فارغة أو `catch (Throwable) { return null; }` تخفي الأعطال. سجل الخطأ مرة في الحد المسؤول، مش في كل طبقة مر بها.

توقع، شخّص، كمّل

▼ توقع ترتيب cleanup.php لو العملية تفشل

closed ثم handled ثم not open. التنظيف يحصل أثناء خروج الاستثناء وقبل catch الخارجية.

▼ Debugging: catch (Exception) لا تمسك TypeError

TypeError تحت Error، والاثنان تحت Throwable. امسك الأنواع المتوقعة قرب العملية وThrowable عند الحد العام؛ لا تحول Bug إلى نجاح.

▼ كمّل تغليف خطأ مع الاحتفاظ بالسبب

`throw new RuntimeException('Load failed', 0, $error);` . المعامل الثالث previous يحفظ سلسلة السبب للتشخيص.

▼ مستخدم يرى مسار ملف داخلي في JSON الإنتاج

راجع display_errors وoutput العشوائية ورسائل exceptions الخام. افصل رسالة عامة ومعرّف تتبع عن سجل محمي، ثم أعد اختبار الرد للتأكد أنه JSON واحدة.

جرّب `php error-lab.php` من المختبر. في الدفتر، بيانات نموذج غير صحيحة 422، وفشل تخزين غير متوقع 500؛ لا نخلطهما.

12. المصفوفات وأدوات التحويل

Lists وMaps وdestructuring وunpacking وmap وfilter وreduce والفرز والأداء.

المشكلة: عندك مجموعة بيانات، محتاج تختار وتحول وتجمع

قائمة أسعار مش مجرد متغير واحد. **List** عناصر مفاتيحها 0 إلى n-1 بالترتيب. **Map** تربط مفتاحًا ذا معنى بقيمة مثل name→Omar. PHP تستخدم `array` للاثنين: خريطة مرتبة، فلا تفترض شكل المفاتيح من اسم المتغير.

map تحول كل عنصر، **filter** تختار بعض العناصر، **reduce** تجمع المجموعة في نتيجة واحدة. دي طرق للتعبير عن حركة البيانات، مش شرطًا لجعل الكود «احترافيًا». الدرس PHP 8.1+ بسبب array_is_list وunpacking المفاتيح النصية.

برنامج كامل: إجمالي الطلبات المدفوعة

احفظ `paid.php` وشغّل `php paid.php`. نفترض أن Rows اجتازت التحقق من النوع والشكل قبل هذه المرحلة:

```
<?php
declare(strict_types=1);

$orders = [
    ['id' => 'A', 'status' => 'paid', 'total' => 1200],
    ['id' => 'B', 'status' => 'pending', 'total' => 500],
    ['id' => 'C', 'status' => 'paid', 'total' => 800],
];
$paid = array_filter($orders, static fn (array $row): bool => $row['status'] === 'paid');
echo 'keys=', implode(',', array_keys($paid)), PHP_EOL;
$totals = array_map(static fn (array $row): int => $row['total'], $paid);
echo json_encode($totals, JSON_THROW_ON_ERROR), PHP_EOL;
$totals = array_values($totals);
echo json_encode($totals, JSON_THROW_ON_ERROR), PHP_EOL;
$sum = array_reduce($totals, static fn (int $carry, int $n): int => $carry + $n, 0);
echo "sum={$sum}", PHP_EOL;
echo array_is_list($totals) ? "list\n" : "map\n";
```

```
keys=0,2
{"0":1200,"2":800}
[1200,800]
sum=2000
list
```

`array_filter` تستدعي callback تسأل عن status، وتحافظ على المفاتيح 0 و2؛ حذف B لا يعيد الترقيم. `array_map` مع Array واحدة تحافظ على مفاتيحها، فيتحول كل Row لعدد. JSON ترى فجوة فتعرض object. `array_values` تعيد الترقيم، فتظهر قائمة JSON. `reduce` تبدأ carry=0، ثم 1200، ثم 2000. لو القائمة فاضية النتيجة 0؛ بدون initial قد تصبح null. `array_sum` أبسط لجمع أعداد فقط؛ هنا reduce لشرح الآلية.

المصفوفة الأصلية لم تتغير. العمليات تبني Arrays جديدة، لكن لو العناصر Objects قد تشير النتائج لنفس الكائنات؛ نسخ الحاوية مش نسخًا عميقًا.

حلقة مكافئة، ومتى تكون أوضح؟

```php
<?php
$orders = [
    ['status' => 'paid', 'total' => 1200],
    ['status' => 'pending', 'total' => 500],
    ['status' => 'paid', 'total' => 800],
];
$sum = 0;
foreach ($orders as $row) {
    if ($row['status'] !== 'paid') {
        continue;
    }
    $sum += $row['total'];
}
echo $sum, PHP_EOL;
```

```
2000
```

نفس الاختيار والجمع بلا Arrays وسيطة؛ لو عندك توقف مبكر أو أكثر من إحصائية قد تكون Loop أوضح. مع بيانات كثيرة جدًا استخدم Generator أو pagination بدل `fetchAll` أو بناء عدة Arrays. الـGenerator تنتج عنصرًا عند الطلب؛ الدرس 16 يشرحها. PHP Arrays تحمل metadata للمفاتيح والقيم، مش مصفوفة أرقام مضغوطة. قس الذاكرة بـ `memory_get_peak_usage(true)` على نفس حجم البيانات قبل تغيير التصميم. `SplQueue` مناسبة لطابور و `SplHeap` لأولوية عندما تحتاج سلوكهما فعلًا.

المفاتيح لها أنواع وقواعد

احفظ **Destructuring** . `keys.php` استخراج عدة قيم بأسمائها/مواضعها، و**unpacking** فرد العناصر في Array أو arguments:

```php
<?php
$keys = [0 => 'first', '0' => 'second', '01' => 'third'];
echo json_encode($keys, JSON_THROW_ON_ERROR), PHP_EOL;
$user = ['id' => 7, 'name' => 'Omar'];
['id' => $id, 'name' => $name] = $user;
[$first, $second] = [10, 20];
echo "{$id}:{$name}:{$first}:{$second}", PHP_EOL;

$defaults = ['timeout' => 3, 'retries' => 1];
$environment = ['timeout' => 5];
echo json_encode([...$defaults, ...$environment], JSON_THROW_ON_ERROR), PHP_EOL;
echo json_encode($defaults + $environment, JSON_THROW_ON_ERROR), PHP_EOL;
```

```
{"0":"second","01":"third"}
7:Omar:10:20
{"timeout":5,"retries":1}
{"timeout":3,"retries":1}
```

`'0'` تتحول لمفتاح integer فتستبدل 0، لكن `'01'` تظل نصًا. فك بنية حقل غير موجود يسبب Warning؛ افحص شكل المدخل أولًا. في unpacking النصي القيمة اللاحقة تفوز؛ في `+` اليسار تفوز، والمفاتيح العددية في unpacking يعاد ترقيمها.
و `sum(...$numbers)` يفرد القائمة لوسائط دالة، مش نفس عملية تعريف `int ...$numbers` التي تجمعها.

`isset($row['x'])` لا تفرق الغياب عن null، و `array_key_exists('x', $row)` تميز وجود المفتاح حتى لو null. `in_array($needle, $values, true)` تبحث بالقيمة والنوع؛ النسخة بلا true مرنة. `array_filter($values)` بدون callback تحذف كل falsy ومنها `0` و `'0'`؛ اكتب predicate صريحة لو الصفر صالح.

الفرز يغيّر المصفوفة نفسها

Comparator دالة ترجع سالبًا لو الأول قبل الثاني، صفرًا لو متساويين، وموجبًا لو بعده. لا ترجع Boolean. برنامج `sort.php`:

```
<?php
$orders = [
    ['id' => 2, 'total' => 800],
    ['id' => 3, 'total' => 1200],
    ['id' => 1, 'total' => 800],
];
usort($orders, static fn (array $a, array $b): int =>
    [$a['total'], $a['id']] <=> [$b['total'], $b['id']]
);
echo implode(',', array_column($orders, 'id')), PHP_EOL;
```

```
1,2,3
```

نقارن total ثم id لكسر التعادل، فـ1 تسبق 2 رغم نفس السعر. `usort` تعيد ترقيم المفاتيح؛ قيمة عودتها نجاح العملية وليست Array مرتبة. **غلط:** `$sorted = sort($values)` تجعل sorted Boolean. انسخ القيم أولًا لو عايز الحفاظ على الأصل، ثم رتب النسخة.

الدالة	ما يرتب؟	يحافظ على المفاتيح؟
sort / rsort	القيم تصاعدي/تنازلي	لا
asort / arsort	القيم	نعم
ksort / krsort	المفاتيح	نعم
usort	القيم بمقارن مخصص	لا

للتنازلي اعكس طرفي المقارنة. قارن أنواعًا متجانسة؛ خلْط نصوص وأعداد قد يجعل ترتيبًا غير مقصود.

توقع، شخّص، كمّل

▼ توقع: array_filter([0, 5, 0, 8]) ثم JSON

المفاتيح تبقى 1 و3، فتظهر object. لو الصفر مطلوب، استخدم callback لا تحذفه؛ لو المطلوب list بعد الفلترة استخدم array_values.

▼ Debugging: comparator ترجع $a['total'] > $b['total']

ترجع true/false فتفقد التفريق بين «قبل» و«متساوي». استخدم `$a['total'] <=> $b['total']` ونوع int.

▼ كمّل reduce لضرب [2,3,4]، بما فيها القائمة الفارغة

ابدأ بقيمة 1، واستخدم `fn (int $carry, int $n): int => $carry * $n` . الناتج 24؛ الفارغة ترجع 1 حسب عقد حاصل الضرب، مش 0.

▼ إيه الأنسب لحساب مجموع وعدد مع توقف عند حد معين؟

حلقة واضحة غالبًا؛ تقدر تحدث العدد والمجموع ثم break بدون Arrays وسيطة أو callback مع آثار جانبية مخفية.

في الدفتر ستعرض قائمة الملاحظات وتفحص شكلها عند القراءة. شغّل `php arrays-lab.php` من المختبر وغيّر القيم لصفر وقائمة فارغة.

13. النصوص وUnicode وRegular Expressions

Bytes وUTF-8 وmbstring والتطبيع والمقارنة والـRegex الآمن في PHP.

المشكلة: الاسم ثلاث حروف، لكن strlen تقول ستة

`string` في PHP سلسلة **Bytes**، والـByte وحدة تخزين من 8 bits. **Unicode** يعطي الرموز أرقامًا اسمها **code points**. **UTF-8** يحول code point إلى 1–4 Bytes. **Grapheme cluster** مجموعة code points يتعامل معها المستخدم غالبًا كحرف ظاهر واحد. الثلاثة قياسات مختلفة؛ لازم تختار حسب السؤال، مش حسب اسم الدالة.

مثلًا عمر ثلاثة code points وستة Bytes. حرف e مع علامة نبرة منفصلة يبدو رمزًا واحدًا لكنه code point للحرف وآخر للعلامة. Emoji العائلة تجمع عدة رموز وروابط غير ظاهرة. الدرس 8.1 PHP+، مع امتدادي `mbstring` و`intl`؛ تحقق بـ`php -m`.

برنامج يقارن الوحدات الثلاث

احفظ `unicode.php` كـUTF-8 وشغّل `php unicode.php`:

```
<?php
declare(strict_types=1);

$samples = [
    'ASCII' => 'Omar',
    'Arabic' => 'عمر',
    'accent' => "e\u{0301}",
    'family' => "👨‍👩‍👧",
];
foreach ($samples as $label => $text) {
    if (!mb_check_encoding($text, 'UTF-8')) {
        throw new InvalidArgumentException('Invalid UTF-8');
    }
    printf("%s: bytes=%d points=%d graphemes=%d\n",
        $label, strlen($text), mb_strlen($text, 'UTF-8'), grapheme_strlen($text));
}
$text = 'عمر';
echo 'first byte=', bin2hex($text[0]), PHP_EOL;
echo 'first point=', mb_substr($text, 0, 1, 'UTF-8'), PHP_EOL;
```

```
ASCII: bytes=4 points=4 graphemes=4
Arabic: bytes=6 points=3 graphemes=3
accent: bytes=3 points=2 graphemes=1
family: bytes=25 points=7 graphemes=1
first byte=d8
first point=ع
```

`\u{0301}` كتابة code point للعلامة في النص ذي التنصيص المزدوج. `mb_check_encoding` تتأكد أن Bytes تشكل UTF-8 صحيحة قبل تفسيرها. `strlen` مناسبة لحجم ملف/طلب، و`mb_strlen` تعد code points، و`grapheme_strlen` تعد المجموعات الظاهرة وفق Unicode/ICU المثبتة. قواعد تجميع الرموز الحديثة قد تختلف مع إصدار ICU؛ الأمثلة هنا رموز معروفة.

Byte `$text[0]` واحدة d8، وهي جزء ناقص من الحرف ع؛ `substr` قد يقطع UTF-8 في المنتصف أيضًا. `mb_substr` تقطع حسب code points، لكنها قد تفصل علامة تشكيل؛ `grapheme_substr` أنسب لحد حروف ظاهر. لا تستخدم عدد Graphemes وحده كحد لحجم الطلب، لأن المجموعة قد تحمل علامات كثيرة؛ ضع حد Bytes أيضًا.

شكلان لنفس الحرف: Normalization

Normalization توحيد تمثيلات Unicode المتكافئة وفق قاعدة محددة. NFC تفضل الشكل المركب حيث يوجد. احفظ `normalize.php`:

```
<?php
$composed = "\u{00E9}";
$decomposed = "e\u{0301}";
var_dump($composed === $decomposed);
$normalized = Normalizer::normalize($decomposed, Normalizer::FORM_C);
if ($normalized === false) {
    throw new RuntimeException('Normalization failed');
}
var_dump($composed === $normalized);
echo strlen($decomposed), ' -> ', strlen($normalized), PHP_EOL;
```

```
bool(false)
bool(true)
3 -> 2
```

أول مقارنة بين Bytes مختلفة، رغم تقارب الشكل. بعد NFC تتطابق. التطبيع لا يحول كل حروف متشابهة بصريًا لنفس المعنى، ولا يصلح XSS. قرر سياسة identifiers والبحث قبل التطبيع، واحتفظ بالأصل للعرض إذا مطلوب. لا تطبع أو تغير Password من نفسك. `mb_strtolower` أنسب من strtolower للنص المتعدد، لكن case folding والترتيب اللغوي لهما قواعد؛ `Collator` من intl للترتيب المحلي عند الحاجة.

Regex: وصف شكل صغير، مش Parser لكل شيء

Regular expression نمط يصف شكل النص. ابدأ بشرط طول ثم pattern بسيطة. `\A` بداية النص الحقيقية، و `\z` نهايته الحقيقية. `[0-9]` رقم ASCII، و `{4}` أربع مرات. `(?<id>...)` مجموعة باسم لالتقاط جزء. حرف `u` بعد delimiter يفعل UTF-8 mode؛ لا يغير كل قواعد المجال تلقائيًا.

برنامج `code.php` يقبل ORD وأربع أرقام فقط:

```php
<?php
function orderId(string $input): ?string
{
    if (strlen($input) > 32) {
        return null;
    }
    $matched = preg_match('/\AORD-(?<id>[0-9]{4})\z/u', $input, $matches);
    if ($matched === false) {
        throw new RuntimeException(preg_last_error_msg());
    }
    return $matched === 1 ? $matches['id'] : null;
}
foreach (['ORD-1234', 'xORD-1234', "ORD-1234\n", 'ORD-0000'] as $input) {
    echo json_encode($input), ' => ', orderId($input) ?? 'invalid', PHP_EOL;
}
```

```
"ORD-1234" => 1234
"xORD-1234" => invalid
"ORD-1234\n" => invalid
"ORD-0000" => 0000
```

الدالة تعيد النص لا int فتحافظ على الأصفار. فحص الطول قبل المحرك يحد العمل. `preg_match` تعيد 1 للتطابق و0 لعدم التطابق وfalse للخطأ؛ لا تخلط 0 وfalse. المجموعة لا تُقرأ إلا بعد نجاح المطابقة. قبول 0000 شكليًا لا يعني وجود طلب بهذا الرقم؛ التحقق من البيانات خطوة أخرى.

غلط: `/^ORD-[0-9]{4}$/` قد يقبل موضع نهاية قبل newline أخيرة؛ `\z` أدق لعقد النص الكامل. وأنماط مثل `(a+)+` قد تسبب **backtracking** مكلفًا: المحرك يجرب تقسيمات كثيرة عند قرب الفشل. استبدلها بنمط أبسط، ضع حدود طول، وافحص أخطاء PCRE. لا تشغّل pattern مستخدم غير موثوق بحرية. JSON وHTML لهما Parsers؛ Regex ليست بديلًا عامًا.

العرض والترميز حسب المكان

Validation تقرر هل البيانات مسموحة. **Encoding** تجعل البيانات نصًا في سياق العرض بدل أن تُفهم كتعليمات. لا توجد دالة تعقيم عامة. احفظ `escaping.php`:

```php
<?php
$name = '<Omar & Mona>';
echo htmlspecialchars($name, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8'), PHP_EOL;
echo '/search?q=', rawurlencode($name), PHP_EOL;
echo json_encode(['name' => $name], JSON_THROW_ON_ERROR), PHP_EOL;
```

```
&lt;Omar &amp; Mona&gt;
/search?q=%3COmar%20%26%20Mona%3E
{"name":"<Omar & Mona>"}
```

htmlspecialchars تحتاج HTML text/quoted attributes، ومعامل URL يحتاج rawurlencode أو http_build_query؛ لو الرابط داخل HTML attribute يحتاج ترميز HTML بعد بناء URL. JavaScript سياق آخر؛ استخدم استجابة JSON منفصلة أو آلية embedding آمنة، مش interpolation/`sprintf` للعرض، لا لبناء SQL. الأسرار الثابتة الطول مثل CSRF tokens تقارن بـ `hash_equals`؛ دي وظيفة مختلفة عن مقارنة نصوص المستخدم.

توقع، شخّص، كمّل

▼ توقع قياسات e مع U+0301

3 Bytes و2 code points و1 grapheme. الأول حرف ASCII بايت والثاني علامة بايتين؛ الشاشة تجمعهما.

▼ Debugging: قص اسم عربي بـ substr($name, 0, 1)

تأخذ Byte واحدة وقد تنتج UTF-8 تالفة. استخدم mb_substr للـcode points أو grapheme_substr لحرف ظاهر، حسب عقدك.

▼ كمّل Regex تقبل AB-123456 كاملة

`/\A[A-Z]{2}-[0-9]{6}\z/` . اختبر بداية زائدة ونهاية newline. بعدها افحص إن كان كود البلد مسموحًا؛ الشكل وحده مش قاعدة المجال.

▼ preg_match رجعت false؛ هل أقول فقط إن الكود غير مطابق؟

لا. 0 هي عدم التطابق. false خطأ نمط أو encoding أو حد محرك؛ افحص preg_last_error_msg وتعامل معه عند حد التطبيق.

شغّل `php text-lab.php` في المختبر. دفتر الملاحظات سيحد Bytes وcode points ويتحقق من UTF-8 ثم يرمز عند العرض. مرجع Grapheme.

14. التاريخ والوقت والمناطق الزمنية

DateTimeImmutable وUTC وtimezones وDST وParsing وIntervals والتخزين الصحيح.

المشكلة: «الساعة 9» مش موعد كامل

ناقص التاريخ والمكان: 9 في القاهرة مش 9 في لندن. **Instant** لحظة واحدة عالميًا. **Local date/time** التاريخ والساعة في مكان. **Timezone** قواعد المكان، مثل Africa/Cairo، وتشمل تغييرات التوقيت الصيفي. **Offset** فرق عن UTC في لحظة، مثل +02:00؛ مش بديلًا عن اسم المنطقة وقواعدها. **Duration** مدة، مش تاريخًا.

نخزن لحظة حدث مضى غالبًا بـUTC، ونعرضها بمنطقة المستخدم. موعد متكرر «كل يوم 9 بالقاهرة» يحتاج الوقت المحلي واسم المنطقة وسياسة DST، مش رقم UTC ثابتًا إلى الأبد. الأمثلة PHP 8.1+، وتستخدم تواريخ محددة بدل now عشان نقدر نراجع الناتج.

لحظة واحدة، عرضان

احفظ `zones.php` وشغّل `php zones.php`. **Immutable** تعني أن العملية ترجع كائنًا جديدًا ولا تغير الأصل:

```
<?php
$utc = new DateTimeImmutable('2024-01-15 12:00:00', new DateTimeZone('UTC'));
$cairo = $utc->setTimezone(new DateTimeZone('Africa/Cairo'));
echo $utc->format('Y-m-d H:i:s P'), PHP_EOL;
echo $cairo->format('Y-m-d H:i:s P'), PHP_EOL;
var_dump($utc->getTimestamp() === $cairo->getTimestamp());
$tomorrow = $utc->modify('+1 day');
echo $utc->format('Y-m-d'), ' / ', $tomorrow->format('Y-m-d'), PHP_EOL;
```

```
2024-01-15 12:00:00 +00:00
2024-01-15 14:00:00 +02:00
bool(true)
2024-01-15 / 2024-01-16
```

ننشئ تاريخًا في UTC، ثم setTimezone تحول العرض. `getTimestamp` عدد الثواني منذ Unix epoch لنفس اللحظة، فيتساوى. `Y-m-d` سنة-شهر-يوم، `H:i:s` ساعة-دقيقة-ثانية؛ الحرف i للدقائق، مش m. `P` يطبع offset. `DateTimeInterface::ATOM` صيغة جاهزة للنقل مع offset. آخر سطر يثبت أن modify لم تغير utc.

غلط: استدعاء `$utc->modify('+1 day');` وتجاهل العودة ثم توقع تاريخ جديد. الصحيح إسناد النتيجة. اختار DateTimeImmutable عشان مشاركة الكائن بين الدوال لا تسبب تغييرًا مفاجئًا.

Parsing صارم: المحرك ممكن يصلح تاريخًا أنت عايز ترفضه

Parsing قراءة نص وتحويله لقيمة تاريخ. Parser مرنة قد تحول 30 فبراير إلى مارس. احفظ `dates.php`:

```php
<?php
declare(strict_types=1);
function parseDate(string $input): DateTimeImmutable
{
    if (preg_match('/\A[0-9]{4}-[0-9]{2}-[0-9]{2}\z/', $input) !== 1) {
        throw new InvalidArgumentException('Use YYYY-MM-DD');
    }
    $date = DateTimeImmutable::createFromFormat('!Y-m-d', $input, new DateTimeZone('UTC'));
    $errors = DateTimeImmutable::getLastErrors();
    if ($date === false
        || ($errors !== false && ($errors['warning_count'] > 0 || $errors['error_count'] > 0))
        || $date->format('Y-m-d') !== $input) {
        throw new InvalidArgumentException('Invalid calendar date');
    }
    return $date;
}
foreach (['2024-02-29', '2023-02-29', '2024-2-9'] as $input) {
    try {
        echo parseDate($input)->format('Y-m-d H:i:s'), PHP_EOL;
    } catch (InvalidArgumentException) {
        echo "invalid: {$input}", PHP_EOL;
    }
}
```

```
2024-02-29 00:00:00
invalid: 2023-02-29
invalid: 2024-2-9
```

Regex تضبط الشكل وطول المكونات، مش صحة التقويم. `!` تصفر الحقول غير المذكورة بدل أخذ الوقت الحالي. نحفظ getLastErrors فور التحليل؛ من PHP 8.2 قد ترجع false عند عدم وجود أخطاء، فلا نفترض أنها Array. نفحص warnings أيضًا لأن إصلاح تاريخ مستحيل قد يكون Warning، ثم نعيد format للمقارنة. كل `||` توقف مبكرًا، فلا نستدعي format على false.

التاريخ فقط مثل يوم ميلاد ليس بالضرورة Instant؛ تخزينه كـDATE محلية له معنى مختلف عن created_at. لا تضف timezone لمجرد أن كل قيم الوقت في النظام لها timezone.

يوم تقويمي غير 24 ساعة دائمًا

Interval وصف فرق، مثل `P1D` يوم تقويمي و`PT24H` 24 ساعة. شغّل `dst.php` بتوقيت لندن التاريخي المعروف في مارس 2024:

```php
<?php
$start = new DateTimeImmutable('2024-03-30 12:00:00', new DateTimeZone('Europe/London'));
$calendar = $start->add(new DateInterval('P1D'));
$elapsed = $start->add(new DateInterval('PT24H'));
foreach (['calendar' => $calendar, 'elapsed' => $elapsed] as $label => $end) {
    $seconds = $end->getTimestamp() - $start->getTimestamp();
    echo $label, ': ', $end->format('Y-m-d H:i P'), " seconds={$seconds}", PHP_EOL;
}
```

```
calendar: 2024-03-31 12:00 +01:00 seconds=82800
elapsed: 2024-03-31 13:00 +01:00 seconds=86400
```

النقطة الأولى تحافظ على 12 محليًا، لكن الفرق 23 ساعة بسبب تقديم الساعة. الثانية تحافظ على 24 ساعة فتصبح 13 محليًا. وقت محلي قد يكون غير موجود عند تقديم الساعة، أو يتكرر عند تأخيرها؛ قرر رفضه أو اختيار offset بوضوح، ولا تقبل اختيار Parser الصامت في نظام مواعيد حساس.

`P1M` شهر مش 30 يومًا: إضافة شهر لـ31 يناير قد تتجاوز فبراير. قرر هل المطلوب آخر يوم في الشهر التالي أم إضافة تقويمية كما هي. اختبر leap day ونهاية الشهر/السنة. قواعد المناطق تتغير سياسيًا؛ حدث بيانات timezone وأعد تقييم المواعيد المستقبلية المحلية حسب سياسة التطبيق.

اختبار «الآن» بدون الانتظار

Clock injection معناها تمرير الوقت الحالي كمدخل بدل استدعائه عميقًا داخل المنطق. نسخة بسيطة بدون Interfaces:

```
<?php
function expired(DateTimeImmutable $deadline, DateTimeImmutable $now): bool
{
    return $now >= $deadline;
}
$deadline = new DateTimeImmutable('2024-01-01T12:00:00+00:00');
foreach (['11:59:59', '12:00:00', '12:00:01'] as $time) {
    $now = new DateTimeImmutable("2024-01-01T{$time}+00:00");
    echo $time, ': ', expired($deadline, $now) ? 'expired' : 'valid', PHP_EOL;
}
```

```
11:59:59: valid
12:00:00: expired
12:00:01: expired
```

الحد نفسه منتهٍ وفق `>=` ؛ دي قاعدة واضحة قابلة للاختبار. قياس زمن تنفيذ كود يفضل ساعة monotonic مثل `hrtime` ، لا wall clock التي قد تُصحح أثناء القياس. خزّن event instant بوضوح، واحتفظ بالـtimezone المطلوبة للمواعيد المحلية، وحول عند العرض.

توقع، شخّص، كمّل

▼ توقع: UTC 12:00 والقاهرة 14:00 في المثال، أيهما أحدث؟

نفس اللحظة، والدليل timestamp متساوية. النص المختلف عرض محلي فقط.

▼ Debugging: createFromFormat رجعت Object لتاريخ 29-02-2023 فاعتبرته صحيحًا

المحرك قد يصحح التاريخ. افحص warnings/errors وround-trip، لأن وجود Object لا يثبت قبول المدخل حرفيًا.

▼ كمّل صلاحية تنتهي عند deadline نفسها

استخدم `$now >= $deadline` . اختبر ثانية قبل وحدًا مساويًا وثانية بعد؛ > فقط تسمح باللحظة الحدية.

▼ موعد يومي 9 محليًا؛ نخزن offset +02:00 وحده؟

لا. احتفظ بالوقت المحلي واسم المنطقة وسياسة الأيام الغامضة، لأن offset قد تتغير مع DST أو القانون.

شغّل `php datetime-lab.php` من المختبر. في الدفتر created_at لحظة UTC، ثم ننسقها للعرض. مرجع التحليل.

15. من HTTP Request إلى Router وResponse

Front Controller وقراءة JSON وRouting وMiddleware وبناء استجابة HTTP صحيحة دون Framework.

المشكلة: العنوان مش لازم يكون اسم ملف

بدل `save.php` و`list.php` عايزين `GET /health` و`POST /notes`، مع طريقة واحدة للتعامل مع الأخطاء والإخراج. هنفصل خمس مسؤوليات قبل أي Framework:

المصطلح	مسؤوليته
Request	method وpath وheaders وbody جاية من العميل
Front Controller	نقطة الدخول الواحدة، غالبًا public/index.php
Middleware	عمل مشترك يحيط بالخطوات التالية أو يرفض قبلها
Router	يختار Handler حسب method وpath
Handler	ينفذ حالة واحدة ويرجع Response
Response / Emitter	بيانات status/headers/body؛ وEmitter ترسلها مرة واحدة

الطلب يدخل كما يلي؛ Middleware خاصة بمسار قد تأتي بعد Router، لكن middleware العامة في المثال تغلفها:

```
Request → Front Controller → Middleware → Router → Handler
                                    ↑          ↓
Response ← Emitter ← Middleware ←──────────┘ Response
```

ملف كامل يوضح الحدود

PHP 8.1+ وmbstring. أنشئ `public/index.php` كما يلي، أو استخدم `http-demo` في حزمة الأمثلة. من مجلد http-demo شغّل `php -S 127.0.0.1:8082 -t public public/index.php`. المعامل الأخير يجعل كل الطلبات تمر عبر Front Controller، مش مجرد ملفات موجودة. المثال API تعليمية تعيد الملاحظة ولا تحفظها؛ الحفظ في الدرس 17.

```php
<?php
declare(strict_types=1);

function response(array $data, int $status = 200, array $headers = []): array
{
    return [
        'status' => $status,
        'headers' => $headers + [
            'Content-Type' => 'application/json; charset=utf-8',
            'Cache-Control' => 'no-store',
            'X-Content-Type-Options' => 'nosniff',
        ],
        'body' => json_encode($data, JSON_THROW_ON_ERROR | JSON_UNESCAPED_UNICODE),
    ];
}
function createNote(array $request): array
{
    if ($request['type'] !== 'application/json') {
        return response(['error' => 'Unsupported media type'], 415);
    }
    try {
        $payload = json_decode($request['body'], false, 32, JSON_THROW_ON_ERROR);
    } catch (JsonException) {
        return response(['error' => 'Invalid JSON'], 400);
    }
    if (!$payload instanceof stdClass
        || !is_string($payload->text ?? null)
        || trim($payload->text) === ''
        || mb_strlen($payload->text, 'UTF-8') > 200) {
        return response(['error' => 'Use a text field with 1 to 200 code points'], 422);
    }
    return response(['text' => trim($payload->text)], 200);
}
function route(array $request): array
{
    $routes = [
        '/health' => ['GET' => static fn (array $r): array => response(['status' => 'ok'])],
        '/notes' => ['POST' => 'createNote'],
    ];
    $methods = $routes[$request['path']] ?? null;
    if ($methods === null) {
        return response(['error' => 'Not found'], 404);
    }
    $handler = $methods[$request['method']] ?? null;
    if ($handler === null) {
        return response(['error' => 'Method not allowed'], 405, ['Allow' => implode(', ', array_keys($methods))]);
    }
    return $handler($request);
}
function middleware(array $request, callable $next): array
{
    $id = bin2hex(random_bytes(8));
    try {
        if (strlen($request['body']) > 4096) {
            $reply = response(['error' => 'Body too large'], 413);
        } else {
            $reply = $next($request);
        }
    } catch (Throwable $error) {
        error_log(json_encode(['request_id' => $id, 'type' => get_class($error)], JSON_THROW_ON_ERROR));
        $reply = response(['error' => 'Internal error'], 500);
    }
    $reply['headers']['X-Request-ID'] = $id;
    return $reply;
}
```

```
$body = file_get_contents('php://input', false, null, 0, 4097);
if ($body === false) {
    $reply = response(['error' => 'Body unavailable'], 500);
} else {
    $path = parse_url($_SERVER['REQUEST_URI'] ?? '/', PHP_URL_PATH);
    $request = [
        'method' => $_SERVER['REQUEST_METHOD'] ?? 'GET',
        'path' => is_string($path) ? $path : '',
        'type' => strtolower(trim(explode(';', $_SERVER['CONTENT_TYPE'] ?? '')[0])),
        'body' => $body,
    ];
    $reply = middleware($request, 'route');
}
http_response_code($reply['status']);
foreach ($reply['headers'] as $name => $value) {
    header("{$name}: {$value}");
}
echo $reply['body'];
```

تتبع البرنامج من أسفل لأعلى ثم مع طلب واحد

تعريف الدوال لا ينفذ أجسامها. التنفيذ الحقيقي يبدأ بقراءة body في الأسفل. نقرأ بحد 4097 Bytes عشان نكتشف تجاوز 4096 بدون تحميل كل الجسم في ذاكرة التطبيق. الخادم نفسه يحتاج حدًا لأن PHP أو الخادم قد يكون استقبل/خزن الجسم قبل وصول الكود. `parse_url(..., PHP_URL_PATH)` يستبعد query string، فـ `/health?check=1` يختار نفس المسار.

نبني Request Array ثم نستدعي middleware مع `'route'` كـMiddleware. Callback تنشئ request ID وتفحص الحجم قبل parsing. لو الحجم مقبول تستدعي next؛ Router يبحث المسار أولًا، ثم method، ثم يستدعي Handler. الرد يرجع بالعكس؛ Middleware تضيف header، وEmitter ترسل status ثم headers ثم body.

`response` تبني بيانات فقط، ولا تعمل echo أو `json_encode` أو exit. هنا قبل إرسال headers، فيمكن للحد العام تحويل فشل encoding إلى 500. لا تخلط `var_dump` مع الرد. read failure خارج middleware له 500 مستقلة هنا؛ لو أردت request ID لكل خطأ ضع إنشاء الطلب نفسه داخل boundary أوسع.

JSON صحيحة مش معناها طلب صحيح

createNote تفصل ثلاثة أسئلة: هل Content-Type مناسب؟ هل النص JSON سليمة؟ هل الجذر Object وله حقل text نصي غير فارغ وبطول مسموح؟ `[]` و`null` وJSON `42` سليمة لكن ليست العقد المطلوب. `json_decode` بدون true تعطينا stdClass للجذر object، فيسهل تمييزها عن list.

`trim` تنظف الأطراف العادية؛ لا تدّعي تنظيف كل أنواع المسافات في `mb_strlen`. Unicode يقيس code points كما اتفقنا. البرنامج يرد 200 لنتيجة التحقق وإعادة النص، لأنه لا ينشئ موردًا دائمًا؛ تطبيق إنشاء حقيقي يضيف حفظًا ويرد 201 وLocation عند وجود عنوان للمورد.

عناوين Router هنا حرفية وثابتة؛ لا تنفذ decoding متكررًا ولا تحوّل path إلى include. Router إنتاجية تحتاج سياسة parameters وURL decoding وHEAD/OPTIONS وcontent negotiation؛ اتبع Framework عند الحاجة. لا تثق في X-Forwarded-* إلا من Proxy معروف ومضبوط.

جرّب العقد من الطرفية

في Windows استعمل `curl.exe` بدل alias PowerShell. لتفادي فروق quoting احفظ `note.json` بمحتوى `{"text":"Learn routing"}`، و`broken.json` بمحتوى `{` فقط، ثم:

```
curl -i http://127.0.0.1:8082/health
curl -i -X POST -H "Content-Type: application/json" --data-binary @note.json http://127.0.0.1:8082/notes
curl -i -X POST -H "Content-Type: application/json" --data-binary @broken.json http://127.0.0.1:8082/notes
```

الطلب	النتيجة
GET /health	200 و `{"status":"ok"}`
GET /missing	404
GET /notes	405 وAllow: POST
POST /notes بدون JSON Content-Type	415
POST /notes مع JSON مكسورة	400
POST /notes مع [] أو text فارغة/Array	422
body أكبر من 4096 Bytes	413 قبل parsing
text صحيحة	200 و `{"text":"Learn routing"}`

Header X-Request-ID عشوائي، فلا تثبت قيمته في Test؛ تحقق من وجوده وشكله. الـContent-Type وstatus جزء من الاختبار، مش body فقط. لا تقرأ مسارًا فيه تخزين أو vendor من Document Root؛ اجعل public فقط مكشوفًا.

توقع، شخّص، كمّل

▼ توقع: GET /notes مقابل GET /unknown

الأول مسار معروف بطريقة غير مسموحة: 405 وAllow. الثاني مسار غائب: 404. ترتيب البحث عن path ثم method هو السبب.

▼ Debugging: Router نفذت echo قبل Middleware تضيف header

الإرسال المبكر قد يرسل headers ويمنع التعديل أو يفسد JSON. خلي Handler ترجع Response وEmitter واحدة ترسل بعد رجوع السلسلة.

▼ كمّل خطوة تمنع body كبيرة قبل json_decode

اقرأ حد+1 ثم قارن الطول بالحد وأعد 413. أضف حد الخادم أيضًا؛ Content-Length وحدها ليست دليلًا على الحجم الفعلي لكل طرق النقل.

▼ رتب authentication وauthorization وvalidation

بعد اختيار Route، اعرف الهوية ثم الصلاحية قبل تنفيذ أثر محمي، وتحقق من المدخل قبل business logic. حد الأخطاء يحيط بالسلسلة، وحد الجسم يسبق parsing. الترتيب التفصيلي يعتمد مسؤولية كل middleware.

المشروع التالي ينقل نفس الحدود إلى ملفات ويضيف نموذجًا وجلسة وتخزينًا. للمزيد من HTTP جرّب المعمل كما هو موضح في تجهيز المختبر.

16. خريطة ميزات PHP الحديثة 8.0–8.5

خريطة زمنية من PHP 8.0 إلى 8.5 تشمل WeakMap وFibers وDNF Types وReadonly Classes وProperty Hooks.

المشكلة: المثال صحيح، لكن السيرفر مش بيفهمه

ميزة جديدة قد تغير Syntax فيفشل الملف **قبل التنفيذ** على إصدار أقدم. `if (PHP_VERSION_ID >= ...)` داخل نفس الملف لا تحمي صياغة لا يفهمها Parser. اختار الحد الأدنى في Composer، واختبره فعلًا، ثم تعلم الميزة لأنها تحل مشكلة محددة.

الدرس خريطة قراءة على مرحلتين: أولًا Generators وWeakMap وFibers لفهم دورة التنفيذ والذاكرة، ثم أنواع وميزات كائنات للتعرف عليها. تفاصيل OOP في مسارها، مش مطلوب تبني Framework هنا.

خريطة زمنية لإصدارات صادرة

دي تواريخ الإصدار الأول لكل سلسلة، مش توصية بتثبيت النسخة .0 القديمة. PHP 8.5 صدرت فعلًا في 20 نوفمبر 2025؛ ليست RFC مستقبلية. راجع إصدار patch مدعوم عند التثبيت.

Version	Release date	Features
8.0	2020-11-26	Named arguments, attributes, constructor promotion, union types, match, nullsafe, WeakMap, ValueError, mixed, JIT
8.1	2021-11-25	Enums, fibers, first-class callables, intersection types, never, readonly properties, array_is_list
8.2	2022-12-08	DNF types, readonly classes, standalone true/false/null, SensitiveParameter, dynamic-property deprecation
8.3	2023-11-23	Typed class constants, Override, json_validate, readonly reinitialization during cloning, dynamic class-constant access
8.4	2024-11-21	Property hooks, asymmetric visibility, lazy objects, array_find/array_any/array_all, Deprecated attribute
8.5	2025-11-20	Pipe, URI extension, clone with, NoDiscard, array_first/array_last, partitioned cookie option

التواريخ موثقة في سجل الإصدارات. الجدول يعرف مكان الميزة؛ مش شرط تحفظه. **مهم تاريخيًا:** Generators و `yield` من 5.5، و `yield from` من 7.0؛ وجودها هنا لا يجعلها ميزة **JIT** .8 PHP ترجمة وقت التشغيل، ومش ضمانًا أن موقعك أسرع؛ قياس التطبيق هو الحكم.

`require: {"php":"^8.3"}` يعني `>=8.3.0 <9.0.0` ، فلا تدخل Hooks من 8.4 في ملف يدعي دعم 8.3. `composer check-platform-reqs` يفحص المنصة الفعلية. راجع migration/deprecations واختبر أقل وأعلى إصدار تدعمه؛ رقم إصدار جديد لا يلغي اختباراتك.

Generator: عنصر عند الطلب، مش Array كاملة

Generator دالة تحتوي yield؛ استدعاؤها يرجع كائن تكرار، وجسمها يتقدم أثناء foreach. تحتفظ بمكانها والمتغيرات بين العناصر. `yield from` تفوض إنتاج العناصر لمصدر آخر. احفظ `generator.php` ؛ الكود متوافق مع PHP 7.0+ لكن استخدم بيئة PHP مدعومة:

```
<?php
function batches(): Generator
{
    echo "begin", PHP_EOL;
    yield from [10, 20];
    yield from [30];
    echo "end", PHP_EOL;
}
$items = batches();
echo "created", PHP_EOL;
foreach ($items as $key => $value) {
    echo "{$key}:{$value}", PHP_EOL;
}
echo implode(',', iterator_to_array(batches(), false)), PHP_EOL;
```

```
created
begin
0:10
1:20
0:30
end
begin
end
10,20,30
```

created قبل begin تثبت أن الاستدعاء لم يبن القائمة. المفتاح 0 يتكرر لأن yield from تحافظ على مفاتيح المصدر؛ `iterator_to_array(..., false)` يعيد الترقيم فلا يفقد 10. الخيار الافتراضي يحافظ على المفاتيح وقد يستبدل المتكرر. التحويل إلى Array يحمل كل العناصر، فيفقد ميزة الذاكرة. لإعادة المرور أنشئ Generator جديدة؛ لا تفترض rewind بعد استهلاكها.

في ملفات كبيرة اجعل Generator تقرأ سطرًا داخل try/finally وتقفل المقبض عند انتهائها أو تدميرها. الاحتفاظ بـGenerator متوقفة يحتفظ بمواردها. `yield from` لا تجعل القراءة غير متزامنة.

WeakMap من 8.0: معلومات تتبع عمر كائن

Strong reference مرجع يبقي الكائن حيًا. WeakMap تستخدم كائنات كمفاتيح دون أن يكون المفتاح سببًا لإبقائها حية؛ مناسبة لـmetadata، أي معلومات إضافية مؤقتة. احفظ `weak.php` ، PHP 8.0+:

```php
<?php
$map = new WeakMap();
$request = new stdClass();
$map[$request] = 'checked';
$alias = $request;
echo count($map), PHP_EOL;
unset($request);
echo count($map), PHP_EOL;
unset($alias);
echo count($map), PHP_EOL;
```

```
1
1
0
```

حذف request وحدها لا يكفي لأن alias مرجع قوي. بعد حذف الاثنين يختفي المدخل في هذا المثال بدون cycles. لو القيمة المخزنة نفسها تحتفظ بمرجع للمفتاح قد تبقيه حيًا؛ الضعف للمفتاح مش وعدًا بإزالة كل graph. لا تستخدم توقيت garbage collection لتقرير سداد أو حذف مهم، ولا تعتبر WeakMap cache دائمة. المرجع.

Fiber من 8.1: وقف واستئناف سلسلة استدعاءات

Fiber سياق تنفيذ يمكنه تعليق Call Stack ثم استكمالها. **Cooperative** يعني أن الكود يسلّم التحكم صراحة؛ ليست Thread ولا CPU parallelism. احفظ PHP 8.1 ، `fiber.php` +:

```php
<?php
$fiber = new Fiber(function (): string {
    echo "fiber entered", PHP_EOL;
    $reply = Fiber::suspend('need input');
    echo "fiber resumed", PHP_EOL;
    return strtoupper($reply);
});
echo "main before", PHP_EOL;
echo $fiber->start(), PHP_EOL;
echo "main between", PHP_EOL;
$fiber->resume('done');
echo $fiber->getReturn(), PHP_EOL;
```

```
main before
fiber entered
need input
main between
fiber resumed
DONE
```

new Fiber لا تشغلها. start تدخل الجسم لحد suspend؛ قيمة need input ترجع للمستدعي. resume ترسل done لتصبح ناتج suspend في `$reply` . بعد النهاية getReturn تقرأ DONE؛ return النهائية مش بالضرورة قيمة resume. لا تستأنف قبل start أو بعد النهاية. دوال `isStarted/isSuspended/isTerminated` توضح الحالة.

Event loop مكتبة تنظم مَن يستأنف ومتى عند جاهزية I/O؛ Fiber وحدها لا تصنع loop ولا تجعل file_get_contents غير blocking. استخدم مكتبة موثوقة لو احتجت async. مرجع Fibers.

DNF Types وReadonly Classes من 8.2

Union تقبل واحدًا من الأنواع، مثل A أو B. **Intersection** تشترط نفس الكائن يحقق النوعين A وB. **DNF** تجمعهما بصيغة «(A وB) أو C». الأقواس حول intersection مطلوبة. مثال `dnf.php` كامل على 8.2+:

```
<?php
function describe((Countable&Stringable)|array $value): string
{
    return is_array($value) ? 'array:' . count($value) : (string) $value;
}
$items = new class implements Countable, Stringable {
    public function count(): int { return 2; }
    public function __toString(): string { return $this->count() . ' items'; }
};
echo describe([10, 20]), PHP_EOL;
echo describe($items), PHP_EOL;

readonly class Amount
{
    public function __construct(public int $minorUnits) {}
}
$amount = new Amount(500);
echo $amount->minorUnits, PHP_EOL;
try {
    $amount->minorUnits = 600;
} catch (Error) {
    echo "readonly blocked reassignment", PHP_EOL;
}
```

```
array:2
2 items
500
readonly blocked reassignment
```

Array تمر من فرع union، والكائن يحقق Countable وStringable معًا. readonly class تجعل Instance properties المقيدة بالنوع readonly وتمنع dynamic properties؛ ليست deep immutability: كائن داخل property قد تتغير حالته. constructor promotion اختصار لتعريف property واستقبالها، ظهرت من 8.0، لكن readonly class نفسها 8.2.

Enums من 8.1 مجموعة حالات مغلقة؛ backed enum تستخدم from التي ترمي ValueError لو القيمة غير موجودة أو tryFrom التي ترجع null. readonly property من 8.1، وnever لدالة لا ترجع طبيعيًا، مثل التي تنهي أو ترمي دائمًا. اقرأ التفاصيل في OOP ونظام الأنواع.

Attributes: بيانات وصفية تحتاج مَن يقرأها

Reflection فحص تعريفات الكود أثناء التشغيل. مثال PHP 8.0، `attribute.php`+:

```php
<?php
#[Attribute(Attribute::TARGET_FUNCTION)]
final class Label
{
    public function __construct(public string $text) {}
}
#[Label('Preview')]
function preview(): void {}
$definition = new ReflectionFunction('preview');
$label = $definition->getAttributes(Label::class)[0]->newInstance();
echo $label->text, PHP_EOL;
```

```
Preview
```

Attribute لا تنفذ إذنًا أو مسارًا تلقائيًا؛ Reflection هنا قرأتها ثم استخدمنا `#[SensitiveParameter]` من 8.2 تخفي قيمة parameter في traces، ولا تنقح logs كتبتها بنفسك. `#[Override]` من 8.3 تساعد في كشف method يفترض أنها تعيد تعريف أصل غير موجود. Typed class constants من 8.3 تصرح النوع، مثل `public const int LIMIT = 10;`؛ والوصول الديناميكي `ClassName::{$name}` من 8.3.

8.4: Hooks وAsymmetric Visibility وLazy Objects

Hook كود عند قراءة/كتابة property. **Asymmetric visibility** صلاحية قراءة تختلف عن الكتابة. مثال `contact.php` على 8.4+:

```php
<?php
final class Contact
{
    public private(set) string $email {
        set {
            $clean = trim($value);
            if (filter_var($clean, FILTER_VALIDATE_EMAIL) === false) {
                throw new InvalidArgumentException('Invalid email');
            }
            $this->email = strtolower($clean);
        }
    }
    public function __construct(string $email) { $this->email = $email; }
}
$contact = new Contact(' OMAR@EXAMPLE.COM ');
echo $contact->email, PHP_EOL;
```

```
omar@example.com
```

constructor مسموح لها بالكتابة، والقراءة public؛ كتابة خارجية تُرفض. lowercase البريد سياسة تعليمية هنا وليست قاعدة لكل نظم البريد. **Lazy object** تؤجل initialization لحين الحاجة للحالة؛ Reflection توفر ghost/proxy. تستخدمها DI/ORM غالبًا، ومش معنى lazy أن كل method تشغّل initializer فورًا. احتفظ بدالة مسماة للعمليات المعقدة بدل منطق مخفي في property. مرجع lazy objects.

8.5: برنامج صغير بميزات صدرت بالفعل

احفظ `features85.php` وشغّله بـPHP 8.5. **URI** عنوان مورد؛ الامتداد الجديد يوفر APIs وفق RFC 3986 وWHATWG. Clone with تنسخ كائنًا وتعدل properties أثناء النسخ؛ لا تعيد تشغيل constructor تلقائيًا، فلا تعتمد عليها لإعادة التحقق:

```
<?php
readonly class Page
{
    public function __construct(public string $title) {}
    public function withTitle(string $title): self
    {
        return clone($this, ['title' => $title]);
    }
}
$draft = new Page('Draft');
$published = $draft->withTitle('Published');
$slug = ' Learn PHP ' |> trim(...) |> strtolower(...);
$uri = new Uri\Rfc3986\Uri('https://example.com/notes?sort=new');
echo $draft->title, ' / ', $published->title, PHP_EOL;
echo $slug, PHP_EOL;
echo $uri->getHost(), PHP_EOL;
echo array_first(['A', 'B']), '/', array_last(['A', 'B']), PHP_EOL;
```

```
Draft / Published
learn php
example.com
A/B
```

الأصل Draft لم يتغير. عملية clone with داخل method لها صلاحية الكتابة؛ readonly من 8.4 لها protected(set) افتراضيًا، فلا تفترض أن تعديلها من global scope مسموح. Pipe تمرر وسيطًا واحدًا لكل callable؛ تفاصيلها في الدرس 6. array_first/last تعيدان null للفارغ؛ null قد تكون قيمة عنصر أيضًا، فلا تخلط الحالتين لو العقد يفرقهما.

`#[NoDiscard]` تصدر Warning عند تجاهل نتيجة معلّمة؛ `(void)` تجاهل صريح في 8.5. توجد أيضًا attributes على constants، callable/static closures في constant expressions، وتوسيع asymmetric visibility، وخيار `partitioned` Cookie. راجع الإعلان الرسمي لـ8.5 قبل اختيار API، ولا تحول وجودها إلى إلزام باستخدامها.

توقع، شخّص، كمّل

▼ توقع عدد WeakMap بعد unset(request) مع بقاء alias

1؛ alias مرجع قوي. الضعف في المفتاح لا يلغي المراجع الأخرى.

▼ Debugging: وضعت pipe داخل if لفحص الإصدار على PHP 8.4

يفشل parsing قبل if. ضع المثال في ملف يعمل فقط على 8.5 أو استخدم الصياغة القديمة وارفع الحد الأدنى بخطة.

▼ كمّل نوع يقبل Array أو كائنًا Countable وStringable

`(Countable&Stringable)|array` من 8.2. نفس الكائن لازم يحقق الواجهتين؛ union بينهما ستغير العقد.

▼ توقع وقت ظهور begin عند استدعاء batches فقط

لن يظهر حتى تبدأ التكرار. Generator مؤجلة؛ تحويلها إلى Array يستهلكها بالكامل وقد يستهلك ذاكرة كبيرة.

▼ هل Fiber تقلل وقت حساب CPU طويل تلقائيًا؟

لا. لا Thread ولا parallelism؛ من غير suspend يظل الحساب ممسكًا بالتحكم. فائدتها في تنظيم تعليق العمل مع scheduler وI/O مناسبين.

شغّل كل ملف على الحد المعلن، و `php -l` على الأقل قبل التنفيذ. `php modern-features-lab.php` من المختبر يضيف enum وreadonly وmatch failures. راجع الدعم الحالي عند النشر.

17. مشروع دفتر الملاحظات وDebugging عملي

برامج PHP كاملة تربط الأنواع والدوال والملفات والطلبات والاستثناءات، مع تمارين توقع الناتج وتصحيح الأخطاء.

المشكلة: الأجزاء تعمل وحدها، لكن هل الطلب الكامل يعمل؟

الدالة قد تكون صحيحة، لكن النموذج يرسل Array بدل نص، أو الحفظ يفشل، أو echo مبكرة تكسر redirect. عشان كده هنربط Form وValidation وSession وFiles وRouter في **دفتر ملاحظات محلي**. لكل جلسة متصفح دفترها؛ لا يوجد Login أو Database في المشروع. لما تحتاج حسابات أو تخزينًا متعدد المستخدمين انتقل إلى الأمان وقواعد البيانات بدل اختراعها هنا.

كل الملفات كاملة تحت `examples/php-course/notebook` أو حزمة أمثلة المنهج. تحتاج PHP 8.1+ وmbstring ومخزن جلسات قابلًا للكتابة. اقرأ على ثلاث جلسات: الطلب والتحقق؛ الحفظ والعرض؛ الاختبارات والتصحيح.

مراحل البناء التي وصلنا لها

المرحلة	ما تعلمته	الدليل العملي
الدرس 5–7	شروط ودوال وCallbacks	اختيار خطأ أو متابعة وتنفيذ Handler
الدرس 8	ملفات وJSON وفشل I/O	كتابة ملف والتحقق من Bytes وقراءة شكله
الدرس 9	Form→Validation→Session	preferences.php تتذكر الاسم وCSRF
الدرس 10	Composer وأدوات الجودة	composer-demo له lock واختبارات فعلية
الدرس 13–15	Unicode والوقت ومسار الطلب	UTF-8 وUTC وRouter وEmitter
هنا	ربط الحدود	حفظ ملاحظة ثم عرضها عبر GET

لا تنسخ fragments متفرقة فوق بعضها. أنشئ الشجرة التالية، ثم اكتب كل ملف كما هو:

```
notebook/
  config.php
  public/index.php
  src/notebook.php
  views/notebook.php
  storage/              (created on first save)
```

1. إعداد واضح بدل افتراض البيئة

`config.php`: environment قيمة من إعداد الخدمة، وليست من Request:

```php
<?php
$environment = getenv('APP_ENV') ?: 'development';
if (!in_array($environment, ['development', 'production'], true)) {
    throw new RuntimeException('Invalid APP_ENV');
}
return [
    'secure_cookie' => $environment === 'production',
    'storage' => getenv('NOTEBOOK_STORAGE') ?: __DIR__ . '/storage',
];
```

المحلي HTTP فيختار Cookie بدون Secure. الإنتاج يختار Secure=true ويحتاج HTTPS فعليًا. NOTEBOOK_STORAGE اختيار لمسار تخزين موثوق في الاختبار أو النشر؛ الافتراضي خارج public. إعداد غير معروف يفشل بدل تشغيل سياسة غير مقصودة. حد الطلب في الخادم وphp.ini يكمّل حد التطبيق؛ أخطاء الإنتاج تسجل ولا تعرض.

2. التحقق والتخزين والعرض: src/notebook.php

```php
<?php
declare(strict_types=1);

function validateNote(array $input): array
{
    $data = [];
    $errors = [];
    foreach (['name' => 40, 'text' => 200] as $field => $limit) {
        $value = $input[$field] ?? null;
        if (!is_string($value) || strlen($value) > $limit * 4
            || !mb_check_encoding($value, 'UTF-8')) {
            $errors[$field] = 'Use valid UTF-8 text';
            $data[$field] = '';
            continue;
        }
        $value = trim($value);
        $data[$field] = $value;
        if ($value === '' || mb_strlen($value, 'UTF-8') > $limit) {
            $errors[$field] = "Use 1 to {$limit} code points";
        }
    }
    return ['data' => $data, 'errors' => $errors];
}
function ownerDirectory(string $root, string $owner): string
{
    if (preg_match('/\A[a-f0-9]{32}\z/', $owner) !== 1) {
        throw new RuntimeException('Invalid storage identifier');
    }
    return $root . '/' . $owner;
}
function readNotes(string $root, string $owner): array
{
    $directory = ownerDirectory($root, $owner);
    if (file_exists($root) && (!is_dir($root) || !is_readable($root))) {
        throw new RuntimeException('Storage unavailable');
    }
    if (!is_dir($directory)) {
        return [];
    }
    $paths = glob($directory . '/*.json');
    if ($paths === false || count($paths) > 100) {
        throw new RuntimeException('Cannot list notes');
    }
    $notes = [];
    foreach ($paths as $path) {
        $raw = file_get_contents($path, false, null, 0, 8193);
        if ($raw === false || strlen($raw) > 8192) {
            throw new RuntimeException('Cannot read note');
        }
        $note = json_decode($raw, true, 32, JSON_THROW_ON_ERROR);
        if (!is_array($note)
            || !is_string($note['name'] ?? null)
            || !is_string($note['text'] ?? null)
            || !is_string($note['created_at'] ?? null)
            || !is_string($note['id'] ?? null)) {
            throw new RuntimeException('Invalid stored note');
        }
        $notes[] = $note;
    }
    usort($notes, static fn (array $a, array $b): int =>
        [$a['created_at'], $a['id']] <=> [$b['created_at'], $b['id']]);
```

```
        return $notes;
    }
    function saveNote(string $root, string $owner, array $data, DateTimeImmutable $now): void
    {
        $directory = ownerDirectory($root, $owner);
        if (!is_dir($directory) && !mkdir($directory, 0700, true) && !is_dir($directory)) {
            throw new RuntimeException('Cannot create storage');
        }
        $id = bin2hex(random_bytes(16));
        $note = [
            'id' => $id,
            'name' => $data['name'],
            'text' => $data['text'],
            'created_at' => $now->setTimezone(new DateTimeZone('UTC'))->format(DateTimeInterface::ATOM),
        ];
        $json = json_encode($note, JSON_UNESCAPED_UNICODE | JSON_THROW_ON_ERROR);
        $temporary = tempnam($directory, '.pending-');
        if ($temporary === false) {
            throw new RuntimeException('Cannot create temporary file');
        }
        if (realpath(dirname($temporary)) !== realpath($directory)) {
            unlink($temporary);
            throw new RuntimeException('Temporary file outside storage');
        }
        try {
            if (file_put_contents($temporary, $json) !== strlen($json)) {
                throw new RuntimeException('Cannot write complete note');
            }
            if (!rename($temporary, $directory . '/' . $id . '.json')) {
                throw new RuntimeException('Cannot publish note');
            }
        } finally {
            if (is_file($temporary)) {
                unlink($temporary);
            }
        }
    }
    function escapeHtml(string $value): string
    {
        return htmlspecialchars($value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
    }
    function renderNotes(array $notes, array $errors, array $old, string $csrf, string $flash): string
    {
        ob_start();
        try {
            require dirname(__DIR__) . '/views/notebook.php';
            return (string) ob_get_contents();
        } finally {
            ob_end_clean();
        }
    }
```

اقرأ الدوال واحدة واحدة

`validateNote` تبني بيانات منظفة وأخطاء. افحص is_string قبل strlen وmb_check_encoding؛ `name[]=x` لا يكسر الدالة. حد Bytes يسبق العمل على Unicode، ثم trim وحد code points. `continue` تمنع تنفيذ باقي التحقق على قيمة مرفوضة. الصفر النصي `'0'` صالح؛ لا نستخدم empty التي ترفضه.

`ownerDirectory` تسمح فقط بمعرّف مولد من 32 حرف hex. المتصفح لا يحدد اسم الملف؛ owner تأتي من مخزن Session على الخادم. ده فصل دفاتر الجلسات، وليس نظام حسابات يمكن استعادتها.

`readNotes` تعتبر مجلدًا لم ينشأ دفترًا فارغًا. `glob` تبحث عن ملفات json المنشورة فقط، ثم نقرأ بحد حجم ونفك JSON بفشل واضح. لا نحول ملفًا تالفًا إلى «لا ملاحظات»؛ ده فقد بيانات متخفٍ. نفحص البنية، ونرتب بالوقت ثم id لكسر التعادل؛ المعرّف العشوائي لا يعني ترتيب إنشاء داخل نفس الثانية.

`saveNote` تنشئ المجلد الخاص وتولد id، وتحول now المحقونة إلى UTC. `tempnam` تنشئ ملفًا مؤقتًا؛ نتحقق من كتابة العدد الكامل، ثم rename لاسم json نهائي داخل نفس filesystem. القارئ لا يرى ملفات pending. اختبر ضمان rename على منصة النشر؛ المثال لا يَعِد بمتانة عند انقطاع الكهرباء. finally تنظف المؤقت عند الفشل.

`renderNotes` تستخدم output buffer عشان القالب يرجع string بدل إرسالها فورًا؛ finally تنظف الـbuffer حتى لو القالب فشل. `escapeHtml` تعمل عند العرض، لا عند التخزين.

3. القالب: views/notebook.php

```
<!doctype html>
<html lang="en">
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>My learning notebook</title>
<h1>My learning notebook</h1>
<p>This browser session owns this notebook. This demo has no login.</p>
<p role="status"><?= escapeHtml($flash) ?></p>
<?php foreach ($errors as $field => $message): ?>
  <p role="alert"><?= escapeHtml($field . ': ' . $message) ?></p>
<?php endforeach; ?>
<form method="post" action="/notes">
  <input type="hidden" name="csrf" value="<?= escapeHtml($csrf) ?>">
  <p><label>Name <input name="name" required value="<?= escapeHtml($old['name'] ?? '') ?>"></label></p>
  <p><label>Note <textarea name="text" required><?= escapeHtml($old['text'] ?? '') ?></textarea></label></p>
  <button>Save note</button>
</form>
<h2>Saved notes</h2>
<?php if ($notes === []): ?><p>No notes yet.</p><?php endif; ?>
<ol>
<?php foreach ($notes as $note): ?>
  <li><strong><?= escapeHtml($note['name']) ?></strong>:
    <span class="note-text"><?= escapeHtml($note['text']) ?></span>
    <time datetime="<?= escapeHtml($note['created_at']) ?>"><?= escapeHtml($note['created_at']) ?></time>
  </li>
<?php endforeach; ?>
</ol>
</html>
```

hidden csrf من الجلسة، لكن المستخدم يقدر يغير أي hidden input؛ الحماية من مقارنة الخادم، مش اختفاء الحقل. نعرض errors جنب النموذج ونحافظ على input الصحيحة بعد 422. كل اسم ونص وزمن يمر عبر escapeHtml حتى لو جاء من ملفنا؛ الثقة في مكان التخزين لا تحول النص إلى HTML مسموح. لو كتبت `<b>hello</b>` لازم تشوفها حروفًا، مش خطًا عريضًا.

الواجهة الإنجليزية هنا مقصودة لتثبيت نفس المثال في اللغتين؛ الاسم والملاحظة يقبلان العربية وUTF-8. تغيير نصوص الواجهة لا يغير منطق الدرس.

4. نقطة الدخول والـMiddleware والـRouter

`public/index.php` ؛ ده الملف الوحيد الذي يستقبل كل المسارات:

```
<?php
declare(strict_types=1);
require dirname(__DIR__) . '/src/notebook.php';

function reply(string $body, int $status = 200, array $headers = []): array
{
    return ['status' => $status, 'headers' => $headers, 'body' => $body];
}
function routeNotebook(string $method, string $path, array $config): array
{
    $routes = ['/' => 'GET', '/notes' => 'POST'];
    if (!isset($routes[$path])) {
        return reply('Not found', 404);
    }
    if ($routes[$path] !== $method) {
        return reply('Method not allowed', 405, ['Allow' => $routes[$path]]);
    }
    $owner = $_SESSION['owner'];
    $csrf = $_SESSION['csrf'];
    if ($method === 'POST') {
        $type = strtolower(trim(explode(';', $_SERVER['CONTENT_TYPE'] ?? '')[0]));
        if ($type !== 'application/x-www-form-urlencoded') {
            return reply('Unsupported media type', 415);
        }
        $token = $_POST['csrf'] ?? null;
        if (!is_string($token) || !hash_equals($csrf, $token)) {
            return reply('Invalid form token', 403);
        }
        $result = validateNote($_POST);
        $notes = readNotes($config['storage'], $owner);
        if ($result['errors'] !== []) {
            return reply(renderNotes($notes, $result['errors'], $result['data'], $csrf, ''), 422);
        }
        if (count($notes) >= 100) {
            return reply('Notebook is full', 409);
        }
        saveNote($config['storage'], $owner, $result['data'], new DateTimeImmutable('now', new
DateTimeZone('UTC')));
        $_SESSION['name'] = $result['data']['name'];
        $_SESSION['flash'] = 'Note saved';
        return reply('', 303, ['Location' => '/']);
    }
    $flash = $_SESSION['flash'] ?? '';
    unset($_SESSION['flash']);
    return reply(renderNotes(
        readNotes($config['storage'], $owner),
        [],
        ['name' => $_SESSION['name'] ?? '', 'text' => ''],
        $csrf,
        $flash,
    ));
}
function sessionMiddleware(callable $next, array $config): array
{
    $raw = file_get_contents('php://input', false, null, 0, 4097);
    if ($raw === false) {
        throw new RuntimeException('Cannot read body');
    }
    if (strlen($raw) > 4096 || (int) ($_SERVER['CONTENT_LENGTH'] ?? 0) > 4096) {
        return reply('Body too large', 413);
    }
    if (!session_start([
        'use_strict_mode' => true,
        'use_only_cookies' => true,
        'cookie_secure' => $config['secure_cookie'],
```

```
            'cookie_httponly' => true,
            'cookie_samesite' => 'Lax',
            'cookie_path' => '/',
        ])) {
            throw new RuntimeException('Session unavailable');
        }
        try {
            $_SESSION['owner'] ??= bin2hex(random_bytes(16));
            $_SESSION['csrf'] ??= bin2hex(random_bytes(32));
            return $next();
        } finally {
            session_write_close();
        }
    }

    ini_set('display_errors', '0');
    ini_set('log_errors', '1');
    $requestId = bin2hex(random_bytes(8));
    try {
        $config = require dirname(__DIR__) . '/config.php';
        $method = $_SERVER['REQUEST_METHOD'] ?? 'GET';
        $path = parse_url($_SERVER['REQUEST_URI'] ?? '/', PHP_URL_PATH);
        $response = sessionMiddleware(
            static fn (): array => routeNotebook($method, is_string($path) ? $path : '', $config),
            $config,
        );
    } catch (Throwable $error) {
        error_log(json_encode(['request_id' => $requestId, 'type' => get_class($error)], JSON_THROW_ON_ERROR));
        $response = reply('Internal error. Reference: ' . $requestId, 500);
    }
    http_response_code($response['status']);
    header('Content-Type: text/html; charset=utf-8');
    header('Cache-Control: no-store');
    header('X-Content-Type-Options: nosniff');
    header('X-Request-ID: ' . $requestId);
    foreach ($response['headers'] as $name => $value) {
        header("{$name}: {$value}");
    }
    echo $response['body'];
```

رحلة الحفظ سطرًا بسطر على مستوى القرار

نقرأ الإعداد ونحدد method/path، ثم sessionMiddleware تقرأ body بحد وتبدأ الجلسة. owner عشوائي ثابت داخل الجلسة، وcsrf عشوائي آخر؛ لا ترسل owner في النموذج. تستدعي next فتصل routeNotebook: تبحث المسار ثم method، ثم نوع body، ثم token، ثم validateNote. **ولا ملف يُكتب قبل نجاح الخطوات دي.**

البيانات المرفوضة ترجع 422 مع نفس النموذج. لو وصل الدفتر 100 ملاحظة يرجع 409. في النجاح نقرأ القائمة ونحفظ، ثم نخزن الاسم وflash ونرجع 303. الـfinally تغلق Session قبل الإرسال. مع file session handler الافتراضي يظل القفل ممسوكًا خلال count ثم save؛ طلبان لنفس الجلسة لا يتجاوزان الحد بسبب قراءة نفس العدد. تغيير handler يحتاج ضمان قفل مكافئ.

المتصفح يتبع Location إلى GET /. handler تقرأ flash ثم تمسحها، لذلك رسالة Note saved تظهر مرة. القراءة لا تعيد الحفظ. Emitter واحدة تطبع status والـheaders ثم body. حد الأخطاء يعطي 500 ومعرّفًا للسجل؛ logs هنا لا تحتوي النص أو Cookie.

الـ303 لا تمنع إعادة إرسال POST عمدًا ولا Double-click في كل ظرف؛ النسخة دي قد تحفظ ملاحظة ثانية عند POST ثانية. مفتاح idempotency مرحلة منفصلة عند الحاجة. انتهاء Session يفقد ربط المتصفح بالدفتر؛ تنظيف الملفات القديمة واستعادة الحسابات خارج نطاق هذه التجربة ومذكوران في README.

التشغيل ودليل النجاح

من مجلد notebook، شغّل `php tests.php` أولًا: المتوقع `PASS: 15 notebook checks` للتحقق من المدخلات والتخزين والعزل وUTC والترميز. بعدها شغّل الخادم:

```
php -S 127.0.0.1:8083 -t public public/index.php
```

افتح الدفتر المحلي. اكتب Omar وLearn PHP. المتوقع POST 303 ثم GET 200، رسالة Note saved وملاحظة محفوظة. Refresh ثانية: الرسالة تختفي والملاحظة تظل. نافذة خاصة: دفتر فاضي. بعد إيقاف السيرفر وتشغيله بنفس Session Cookie ومخزن جلسات ساري، الملفات ما زالت موجودة.

التجربة	المطلوب
name فارغ أو name[]	422، لا ملف جديد
نص عربي وemoji	حفظ وعرض صحيحان
نص يحتوي HTML	يظهر كنص، لا يتنفذ
CSRF ناقص/غلط	403، لا حفظ
GET /notes	405 وAllow: POST
missing/	404
JSON بدل form-urlencoded	415
جسم أكثر من 4096 Bytes	413
ملف JSON مخزن تالف	500 مع reference، لا ادعاء دفتر فارغ
GET /storage/...	404؛ Document Root هي public فقط

اختبر فساد الملفات في نسخة تجارب منفصلة باستخدام NOTEBOOK_STORAGE، مش ملاحظاتك الحقيقية. الاختبارات الآلية المرفقة في المستودع تتحقق من الجلسات والعزل وCSRF وUTF-8 والإخراج والحفظ ومسارات الخطأ.

Debugging: دليل قبل التعديل

1. ثبّت Request صغيرة تعيد العيب، وسجّل expected وactual.
2. حدد الحد: parsing أم validation أم حفظ أم عرض؟
3. افحص قيمة ونوعًا عند الحد، بدون طباعة داخل Response.
4. اختبر فرضية واحدة وعدل سببًا واحدًا.
5. أضف Regression Test تفشل قبل الإصلاح وتنجح بعده.

Breakpoint نقطة توقف في debugger قبل تنفيذ سطر. Xdebug امتداد PHP، والـIDE عميل يعرض المتغيرات والـCall Stack. ثبت الامتداد المطابق لبنية وإصدار PHP وفق التثبيت الرسمي، ثم `php --ri xdebug` للتأكد. إعداد محلي، مع سطر zend_extension لمسار ملفك الفعلي:

```
xdebug.mode=debug
xdebug.start_with_request=trigger
xdebug.client_host=127.0.0.1
xdebug.client_port=9003
```

افتح listener في IDE واضبط path mapping لو PHP داخل container. ضع breakpoint على `$result =` ثم أرسل النموذج مع XDEBUG_TRIGGER عن طريق browser helper/Cookie. **Step Into** تدخل الدالة، **Step Over** تنفذ السطر، و**Step Out** تكمل حتى تعود للمستدعي. راقب `$_POST['name']` ، ثم errors، ثم Call Stack: `validateNote($_POST);` index→middleware→route→validateNote.

من CLI على `$env:XDEBUG_TRIGGER='1'` :PowerShell ثم شغّل الملف، وبعد التجربة `Remove-Item Env:XDEBUG_TRIGGER` . لو لا تتوقف: راجع PHP المستخدمة وملف ini وlistener وtrigger وmapping؛ امتداد CLI لا يعني أن FPM حملته. مرجع step debugging. في الإنتاج لا تفعّل اتصال debugger عام؛ استخدم logs محمية وrequest ID وdisplay_errors=Off. إعدادات 11 تشرح الفرق. Log منظمة بمعلومات منقحة، لا dump لكل SESSION.

الاختبارات والأدوات على مشروع فعلي

ارجع إلى composer-demo من الدرس 10: `composer install` ثم `composer check` ثم PHPUnit . `composer audit` تتحقق من النتائج، PHPStan من العقود، وPHP-CS-Fixer من التنسيق. الـlock يثبت النسخ؛ لا تحذفها لحل خطأ test. أثناء نشر التطبيق استخدم install من lock وcheck-platform-reqs على المنصة المستهدفة، مع PHP ini منفصل للتطوير والإنتاج.

امتداد التدريب: البرامج الكاملة السابقة ما زالت متاحة

في مختبر PHP الأصلي ملفات `total.php` و`orders.php` و`stream-lab.php` كاملة وليست أجزاء ناقصة:

- `15.75` → `php total.php 12.50 3.25` . تحويل المبلغ يتم كنص لوحدات صحيحة: ضبط الصيغة بـ `\z` / `\A` ، تكملة الكسر لرقمين، مقارنة طول الأرقام ثم ترتيبها مع PHP_INT_MAX **قبل التحويل**. الجمع يفحص `left <= PHP_INT_MAX - right` . اختبر 0 و12 و12.5 و12.50، وارفض -1 و1.234 وnewline وقيمة ضخمة. `php tests.php values` يثبت الحدود.
- `orders=2` → `php orders.php fixtures/orders.json` و`items=3` و`total_minor=2999` ، للطلبين 1250×2 و499×1. JSON المكسورة ترجع `ERROR invalid JSON` وكود خروج 2. افصل القراءة والتحقق والحساب والعرض، واختبر كمية صفرًا وسعرًا سالبًا وoverflow.
- `php stream-lab.php fixtures/large.csv` يعالج بالتتابع؛ اختبر الملف الفارغ والمفقود والسطر الكبير. Generator أو SplFileObject لا تجعل ذاكرة السطر الواحد غير محدودة آمنة. راقب العدد والذاكرة، ولا تخلط فشل القراءة بقائمة فاضية.

حالة JSON endpoint وفروق 400/422/415/413 موجودة كاملة في http-demo بالدرس 15. خلي لكل برنامج README وfixtures واختبارًا؛ الشغل من نسخة نظيفة يكشف ملفات محلية منسية.

توقع، شخّص، كمّل

▼ توقع: Save ناجحة ثم Refresh مرتين

POST تحفظ مرة وترجع 303. أول GET تظهر flash وتمسحها؛ التالية تعرض الملاحظة بدون flash ولا حفظ جديد. إعادة POST نفسها حالة أخرى قد تضيف ملاحظة.

▼ Debugging: name[]=Omar يسبب TypeError

يعني وصلت Array لدالة نص قبل التحقق. ضع is_string في validateNote قبل trim/mb_strlen، واختبر أن الرد 422 وأن عدد الملفات لم يتغير.

▼ كمّل اختبار يمنع XSS من ملف مخزن

احفظ نصًا مثل <img src=x onerror=alert(1)> ثم افحص HTML: يظهر مرمزًا داخل note-text ولا يوجد عنصر img ناتج عنه. الاختبار لازم يقرأ الرد الحقيقي، مش يختبر escapeHtml وحدها فقط.

▼ ملف JSON تالف أدى لدفتر فاضي ورسالة نجاح. إيه الخطأ؟

الفشل اتبلع. ارم خطأ عند decode/shape failure، وسجل reference وارجع 500 بلا تفاصيل حساسة. لا تعدل المتوقع ليتوافق مع فقد البيانات.

▼ المجموع المالي صحيح للمدخلات العادية ويفشل عند الحد الأعلى. أول خطوة؟

ثبّت الحد وحالة تتجاوزه في Test، وافحص قبل الضرب أو الجمع؛ overflow قد يحول النتيجة إلى float قبل أن تلحق تتحقق منها.

الانتهاء من الدرس يعني أنك تقدر تتبع طلب واحد من النموذج حتى الملف والرد، وتثبت مسار رفض بلا أثر، وتشرح سبب كل حالة HTTP.

04

المسار · 20 درسًا

تشغيل PHP والإنتاج

شرح متدرج، أمثلة عملية، وتصحيحات تساعدك على الانتقال من الفكرة إلى التطبيق.

تشغيل PHP وأدوات الإنتاج

من Composer وOPcache وPHP-FPM إلى الاختبارات وRedis والطوابير والنشر والمراقبة والمرونة والتعافي.

المسار ده يشرح الأدوات والطبقات اللي بتحيط بتطبيق PHP بعد كتابة الكود. هتتعرف بالتدريج على مسؤولية مدير الحزم وعميل HTTP وخادم الويب وPHP-FPM، وإزاي تفرّق بين دور كل جزء.

لماذا هو قسم مستقل؟

هذه الموضوعات ليست جزءًا من OOP، وليست مجرد صياغة للغة PHP. إنها طبقة **Runtime & Production**: تثبيت الاعتماديات، تحميل الأصناف، تنفيذ PHP بكفاءة، الاتصال بخدمات خارجية، إدارة العمليات، وتمرير الإعدادات بأمان.

خريطة الدروس

1. Composer وإدارة الاعتماديات — `composer.json` و`composer.lock` و`install` و`update` وPSR-4.
2. cURL وعملاء HTTP — libcurl وامتداد PHP وGuzzle والمهلات والتحقق من TLS.
3. OPcache وPreloading — Opcode cache، الإعداد للإنتاج، القياس، وحدود Preloading.
4. ذاكرة PHP وzval — Garbage Collection وReference Counting وCopy-on-write والدورات المرجعية.
5. PHP-FPM وإدارة العمليات — Master وWorkers وPools وأنماط `static` و`dynamic` و`ondemand`.
6. Apache وNginx وFastCGI — مسار الطلب والـMPM والـEvent Loop والاتصال بـFPM.
7. متغيرات البيئة وإدارة الإعدادات — `getenv()` و`$_ENV` و`.env` والأسرار.
8. PHP CLI وphp.ini والامتدادات — SAPIs واكتشاف الإعداد وextensions.
9. Logging وPSR-3 — Observability وstructured logs وmetrics وtraces.
10. الاختبارات وجودة الكود — PHPUnit وdoubles وcoverage والتحليل الساكن.
11. Data Caching وRedis — Cache-aside وTTL وinvalidation وstampede.
12. Queues وIdempotency — Workers وretry وDLQ وscheduling.
13. Deployment وCI/CD — artifacts وContainers وhealth checks وrollout وrollback.
14. أمان الاعتماديات — Composer audit والسياسات وplugins وCI.
15. Profiling وSLI/SLO — Flame Graphs وOpenTelemetry وPercentiles وأهداف الخدمة وانتقال Trace Context.
16. المرونة وإدارة Workers — Circuit Breaker وBulkhead وBackpressure وSignals وsystemd.
17. Disaster Recovery واختبار الاستعادة — RPO وRTO والنسخ والاسترجاع وGame Days.
18. Sessions وShared State عند التوسع — التخزين المشترك والقفل وTTL والتوافق بين النسخ.
19. اتصالات قاعدة البيانات وتشغيلها — ميزانية الاتصالات والمهلات والـpooling والـreplicas.
20. PHP طويلة العمر وWorker Runtimes — FrankenPHP وRoadRunner وSwoole وتسرب الحالة وإعادة الضبط.

بروتوكول التجربة في كل درس

كل تمرين يمر بخمس خطوات: توقّع النتيجة، شغّل الأمر، أحدث عطلًا مقصودًا، اجمع دليلًا من الخروج أو logs أو metrics، ثم أصلح السبب وأعد الاختبار. نجاح الأمر وحده ليس دليلًا كافيًا من دون معيار قبول يمكن مراجعته.

الخيط الذي يربط الدروس

المتصفح يتصل بخادم الويب، والخادم يمرر طلب PHP إلى FPM، والـWorker يشغّل كودًا حمّله Composer ويستفيد من OPcache، وقد يستدعي خدمة خارجية عبر cURL. الإعدادات والأسرار تصل من البيئة، والذاكرة تُدار داخل كل Worker.

مسار طلب إنتاجي مختصر

```
Client
  -> Nginx / Apache (TLS, static files, routing)
  -> FastCGI
  -> PHP-FPM pool
  -> Composer autoloader + OPcache
  -> Application / Database / External API
  <- HTTP response
```

الأمثلة تستهدف PHP 8.x. إعدادات الخادم أمثلة تعليمية ويجب تكييف المستخدمين والمسارات والسعة مع نظام التشغيل والموارد الفعلية.

مراجع رسمية

- الاستخدام الأساسي لـComposer وتحسين الـAutoloader
- توثيق cURL وlibcurl
- OPcache وPreloading
- Garbage Collection في PHP
- PHP-FPM
- إعداد الجلسات وإدارة اتصالات PDO
- FrankenPHP Worker Mode
- Nginx FastCGI وApache MPM
- getenv() و `$_ENV`

1. Composer وإدارة الاعتماديات

composer.json وcomposer.lock وinstall وupdate وrequire-dev وPSR-4 وتجهيز الإنتاج.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.
- **Namespace:** مساحة أسماء تمنع تعارض الأسماء بين أجزاء المشروع.
- **Autoloading:** تحميل ملف الكلاس تلقائيًا عند استخدامه بدل include اليدوي.
- **Composer:** مدير حزم PHP يثبت المكتبات ويجهز التحميل التلقائي.

Composer: مدير اعتماديات PHP

Composer أداة تدير مكتبات المشروع وإصداراتها وتولّد Autoloader. هو لا يثبت "PHP نفسها"، بل يحل القيود الموجودة في `composer.json` ويضع الحزم غالبًا داخل `vendor/`.

الملفان الأساسيان

- `composer.json`: ما يحتاجه المشروع والقيود المسموحة والإعدادات والـscripts.
- `composer.lock`: الإصدارات الدقيقة التي حُلّت بالفعل، مع hashes وmetadata تجعل التثبيت قابلًا للتكرار.

```
{
  "require": {
    "php": "^8.3",
    "guzzlehttp/guzzle": "^7.9"
  },
  "require-dev": {
    "phpunit/phpunit": "^11.0"
  },
  "autoload": {
    "psr-4": {
      "App\\": "src/"
    }
  }
}
```

⚠

تصحيح مهم

`composer.lock` لا "يتابع ما حدث" فقط؛ في التطبيقات يجب رفعه إلى Git لأنه يثبت الإصدارات الدقيقة. أما المكتبة المنشورة للآخرين فالمستهلك يحل اعتمادياتها ضمن مشروعه، لذلك lock الخاص بالمكتبة لا يتحكم في تثبيته.

install أم update؟

الأمر	مع وجود lock	النتيجة
`composer install`	يقرأ الإصدارات الدقيقة	تثبيت متكرر ومتوقع، وهو المناسب للـCI والإنتاج
`composer update`	يعيد حل القيود	يغيّر `composer.lock` إلى أحدث إصدارات مسموحة
`composer update vendor/package`	تحديث محدود	يقلل نطاق التغيير لكنه قد يحدّث اعتماديات مرتبطة

```
composer install
composer require monolog/monolog
composer require --dev phpunit/phpunit
composer update guzzlehttp/guzzle --with-all-dependencies
```

`composer require` يعدّل `composer.json` ويحدّث الـlock ويثبّت الحزمة عادة. الخيار `--dev` يضعها في `require-dev`.

Autoloading وPSR-4

بعد تحديد Namespace إلى directory يشغّل Composer:

```
composer dump-autoload
```

ثم يكفي تحميل ملف واحد عند نقطة الدخول:

```
<?php
declare(strict_types=1);

require dirname(__DIR__) . '/vendor/autoload.php';

$service = new App\Billing\InvoiceService();
```

في PSR-4 يجب أن يتوافق الاسم `App\Billing\InvoiceService` عادة مع:

```
src/Billing/InvoiceService.php
```

Linux حساس لحالة الأحرف؛ خطأ مثل `invoiceService.php` قد يعمل محليًا على Windows ثم يفشل في الإنتاج.

تثبيت إنتاجي آمن

```
composer validate --strict
composer install --no-dev --prefer-dist --optimize-autoloader --no-interaction
composer audit --locked
composer check-platform-reqs --lock --no-dev
```

- `--no-dev` يستبعد أدوات التطوير، لكنه لا يعني أن `require-dev` غير مهم أثناء الاختبار.
- `--optimize-autoloader` يحوّل قواعد PSR إلى class map محسّنة؛ استخدمه في الإنتاج لا أثناء التطوير المتغير بسرعة.
- يمكن استعمال `--classmap-authoritative` بعد الاختبار، لكنه يكسر الأصناف التي تُولّد وقت التشغيل إذا لم تدخل الخريطة.
- لا تستخدم `--ignore-platform-reqs` كحل دائم؛ قد تثبّت كودًا لا يدعمه إصدار PHP أو Extension على الخادم.
- راجع Composer scripts/plugins لأنها كود قابل للتنفيذ أثناء التثبيت. لا تثبّت مشروعًا غير موثوق بصلاحيات عالية.

سير عمل موصى به

1. أضف أو حدّث الحزم في فرع تطوير.
2. راجع فرق `composer.json` و `composer.lock` .
3. شغّل الاختبارات و `composer audit` .
4. ارفع الملفين إلى Git، ولا ترفع `vendor/` غالبًا.
5. في النشر شغّل `composer install` لا `update` .

بهذا تكون عملية اختيار الإصدارات منفصلة عن عملية نشرها، فلا يفاجئك إصدار جديد أثناء الإنتاج.

مسألة تشغيلية

▼ ليه لازم ترفع composer.lock في التطبيق؟

لتثبيت نفس الإصدارات المختبرة في كل بيئة؛ composer.json يصف القيود بينما lock يسجل الحل الفعلي.

شغّل وتحقق

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

نفّذ نقطة التحقق التالية داخل `examples/php-labs` أو النسخة المستخرجة من الحزمة:

```
composer validate --strict
composer audit --locked
composer check-platform-reqs --lock --no-dev
```

معيار النجاح: تنتهي الأوامر بكود 0؛ يكون `composer.lock` صالحًا، ولا توجد advisory معروفة في محتواه، وتطابق نسخة PHP والامتدادات الفعلية متطلبات الإنتاج. وجود lock هو الذي يجعل فحص `-locked` قابلًا للتكرار؛ `validate` وحده لا ينشئ lock ولا يضمن وجوده.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

اربط النقاط ببعض

افهم SemVer كعقد توافق لا ضمان جودة: `^` و`~` يفتحان نطاقات مختلفة، والـlock يثبت الاختيار الفعلي. تحقق من platform requirements وإضافات PHP، واضبط private repositories واعتماديّاتها دون تخزين token. راجع scripts لأنها كود ينفذ أثناء Composer.

دورة التجربة

قبل التنفيذ اكتب توقعك، ثم شغّل المثال وسجّل الخروج. أحدث فشلًا واحدًا مقصودًا، اجمع الدليل من logs أو metrics، أصلح السبب، وأعد التشغيل لإثبات أن الإصلاح يعالج العطل ولا يخفيه.

جرّب بنفسك

اشرح لماذا install على lock يختلف عن update في production.

2. cURL وعملاء HTTP

libcurl وامتداد PHP وGuzzle والمهلات وTLS وإرسال JSON ومعالجة الأخطاء.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **HTTP:** قواعد تبادل الطلبات والردود بين المتصفح والخادم.
- **DNS:** خدمة بتحوّل اسم الموقع إلى عنوان رقمي يقدر الجهاز يوصل له.
- **IP:** عنوان رقمي بيميز جهازًا أو واجهة شبكة.
- **URL:** العنوان الكامل لمورد على الويب، زي صفحة أو صورة أو نقطة API.
- **TLS:** طبقة تشفير بتحمي البيانات وهي ماشية بين طرفين.
- **API:** واجهة محددة تسمح لبرنامج يطلب بيانات أو ينفّذ عملية عند برنامج آخر.
- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.
- **Worker:** برنامج يعمل في الخلفية ويسحب المهام من الطابور وينفذها.

من cURL إلى عميل HTTP موثوق

مشروع cURL ينتج أداتين مختلفتين:

- `curl` : برنامج لسطر الأوامر.
- `libcurl` : مكتبة لنقل البيانات عبر بروتوكولات متعددة.

امتداد `ext-curl` في PHP يربط PHP بـlibcurl. وهو **عميل** يبدأ اتصالات بخوادم أخرى؛ ليس خادم ويب ولا Socket server.

دورة الطلب في PHP

```php
<?php
declare(strict_types=1);

$handle = curl_init('https://api.example.com/v1/users/42');

curl_setopt_array($handle, [
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_HTTPHEADER => [
        'Accept: application/json',
        'User-Agent: OmarDevNotes/1.0',
    ],
    CURLOPT_CONNECTTIMEOUT => 3,
    CURLOPT_TIMEOUT => 10,
    CURLOPT_FOLLOWLOCATION => false,
]);

$body = curl_exec($handle);

if ($body === false) {
    $message = curl_error($handle);
    $code = curl_errno($handle);
    curl_close($handle);
    throw new RuntimeException("Network error {$code}: {$message}");
}

$status = curl_getinfo($handle, CURLINFO_RESPONSE_CODE);
curl_close($handle);

if ($status < 200 || $status >= 300) {
    throw new RuntimeException("Unexpected HTTP status: {$status}");
}

$data = json_decode($body, true, flags: JSON_THROW_ON_ERROR);
```

افصل دائمًا بين:

1. خطأ النقل: DNS أو اتصال أو timeout أو TLS؛ يظهر من `curl_exec()`.
2. استجابة HTTP فاشلة: مثل 404 أو 500؛ الاتصال نجح لكن حالة الاستجابة ليست نجاحًا.
3. محتوى غير صالح: مثل JSON تالف؛ عالجه عند فك الترميز.

إرسال JSON

```
$payload = json_encode(
    ['email' => 'user@example.com', 'active' => true],
    JSON_THROW_ON_ERROR
);

$handle = curl_init('https://api.example.com/v1/users');
curl_setopt_array($handle, [
    CURLOPT_POST => true,
    CURLOPT_POSTFIELDS => $payload,
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_HTTPHEADER => [
        'Content-Type: application/json',
        'Accept: application/json',
        'Authorization: Bearer ' . $token,
    ],
    CURLOPT_CONNECTTIMEOUT_MS => 1000,
    CURLOPT_TIMEOUT_MS => 5000,
]);
```

لا تسجّل `Authorization` أو Cookies أو أجسامًا تحتوي بيانات شخصية.

TLS وRedirects

لا تعطّل التحقق

لا تستخدم `CURLOPT_SSL_VERIFYPEER => false` أو `CURLOPT_SSL_VERIFYHOST => 0` في الإنتاج. أصلح CA bundle أو إعداد النظام؛ تعطيل التحقق يجعل الاتصال عرضة لـMan-in-the-Middle.

`CURLOPT_FOLLOWLOCATION` قد يرسل الطلب إلى وجهة أخرى. ضع حدًا بـ`CURLOPT_MAXREDIRS`، ولا تسمح للمستخدم بتحديد URL حرًا في خدمة داخلية وإلا قد تنشئ SSRF. تحقّق من scheme وhost، وامنع عناوين الشبكة الداخلية وmetadata endpoints حسب بيئتك.

Guzzle وPSR

Guzzle يقدم API أعلى مستوى ويدعم middleware وpromises وpooling. لكنه ليس "cURL بواجهة جميلة" بشكل مطلق؛ يختار Handler مناسبًا وقد يستخدم cURL أو PHP streams حسب البيئة.

```php
use GuzzleHttp\Client;

$client = new Client([
    'base_uri' => 'https://api.example.com/',
    'connect_timeout' => 3,
    'timeout' => 10,
]);

$response = $client->get('v1/users/42', [
    'headers' => ['Accept' => 'application/json'],
]);

$data = json_decode(
    (string) $response->getBody(),
    true,
    flags: JSON_THROW_ON_ERROR
);
```

عقود PSR-7 تمثل Request/Response، وPSR-18 يعرّف واجهة عميل HTTP. الاعتماد على Interface يجعل اختبار الكود وتبديل العميل أسهل.

المهلات وإعادة المحاولة

- اضبط مهلة اتصال ومدة كلية؛ الافتراضات غير المحدودة قد تحتجز FPM worker.
- أعد المحاولة فقط للأخطاء المؤقتة، مع exponential backoff وjitter.
- لا تُعد POST غير idempotent تلقائيًا إلا بوجود idempotency key يدعمه الخادم.
- احترم `Retry-After` مع 429/503.
- ضع حدًا لحجم الاستجابة واستخدم streaming للملفات الكبيرة.

cURL Multi

`curl_multi_*` يسمح بعدة عمليات نقل متزامنة دون thread لكل طلب. يفيد عندما تكون الاستدعاءات مستقلة، لكن يجب ضبط عدد الاتصالات وحدود API؛ التزامن غير المحدود ينقل الاختناق إلى الطرف الآخر.

مسألة تشغيلية

▼ إيه الحدود الضرورية لأي HTTP client؟

Connect timeout وtotal timeout وحد للحجم وسياسة redirect وTLS verification، مع retry للعمليات الآمنة فقط.

شغّل وتحقق

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

نفّذ نقطة التحقق التالية داخل بيئة الدرس:

```
php http-client-lab.php
```

معيار النجاح: يطبع الاختبار `status=200` للنجاح، و `timeout_handled=yes` لخادم بطيء، ولا يعامل 500 كاستجابة ناجحة.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

اربط النقاط ببعض

افصل connect timeout عن TLS/read/total timeout، وانشر cancellation deadline عبر السلسلة. قبل جلب URL يقدمه مستخدم، امنع SSRF عبر scheme وhost وDNS/IP policy مع حماية من redirects وإعادة resolution. سجّل attempts وlatency/status دون token أو body حساس.

دورة التجربة

قبل التنفيذ اكتب توقعك، ثم شغّل المثال وسجّل الخروج. أحدث فشلًا واحدًا مقصودًا، اجمع الدليل من logs أو metrics، أصلح السبب، وأعد التشغيل لإثبات أن الإصلاح يعالج العطل ولا يخفيه.

جرّب بنفسك

اختبر URL يعيد redirect إلى private IP وخادمًا يفتح الاتصال ولا يرسل body.

3. OPcache وPreloading

تخزين PHP bytecode وضبط OPcache ومراقبته والفرق بينه وبين cache البيانات وحدود Preloading.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **HTTP:** قواعد تبادل الطلبات والردود بين المتصفح والخادم.
- **API:** واجهة محددة تسمح لبرنامج يطلب بيانات أو ينفّذ عملية عند برنامج آخر.
- **Cache:** نسخة مؤقتة من البيانات هدفها تقليل وقت الانتظار والعمل المتكرر.
- **CLI:** واجهة تتعامل معها بكتابة أوامر نصية بدل الضغط على أزرار.
- **Composer:** مدير حزم PHP يثبت المكتبات ويجهز التحميل التلقائي.
- **OPcache:** ذاكرة تحفظ نتيجة ترجمة PHP لتجنب تكرار العمل مع كل طلب.
- **Redis:** مخزن سريع في الذاكرة يُستخدم للكاش والجلسات والطوابير.

لماذا يعيد PHP العمل؟

بدون OPcache يقرأ PHP الملف ويحلله ويحوّله إلى Opcodes عند التنفيذ. OPcache يحتفظ بالـbytecode المترجم في ذاكرة مشتركة لتستفيد منه الطلبات التالية، فيقل وقت parsing وcompilation.

```
PHP source -> tokenize/parse -> compile to opcodes -> execute
                                |
                                +-> OPcache shared memory
```

ما لا يفعله OPcache

OPcache لا يخزّن HTML النهائي أو نتائج SQL أو استجابة API. هذه مسؤولية HTTP cache أو application cache مثل Redis. كما أنه لا يجعل الخوارزمية البطيئة سريعة بعد بدء التنفيذ.

إعداد إنتاجي مبدئي

```
; php.ini / conf.d/opcache.ini
opcache.enable=1
opcache.memory_consumption=256
opcache.interned_strings_buffer=16
opcache.max_accelerated_files=20000
opcache.validate_timestamps=0
```

القيم ليست وصفة ثابتة. راقب عدد الملفات والذاكرة وhit rate وrestarts قبل تعديلها.

- مع `validate_timestamps=1` يفحص OPcache تغيّر الملفات حسب `revalidate_freq`.
- مع `validate_timestamps=0` لا يرى النشر الجديد حتى تعيد تشغيل FPM أو تستدعي reset بطريقة منضبطة.
- CLI له إعداد مستقل عادة عبر `opcache.enable_cli`، وتشغيله لسكربت قصير قد يضيف تكلفة بلا فائدة.

المراقبة

```
$status = opcache_get_status(false);

if ($status !== false) {
    printf(
        "hit-rate=%.2f%% used=%d free=%d\n",
        $status['opcache_statistics']['opcache_hit_rate'],
        $status['memory_usage']['used_memory'],
        $status['memory_usage']['free_memory'],
    );
}
```

لا تعرض endpoint يحوي `opcache_get_status()` للعامة؛ قد يكشف paths ومعلومات تشغيلية. اجعله داخليًا ومحميًا.

مؤشرات تحتاج انتباهًا:

- `cache_full` أو امتلاء الذاكرة.
- `oom_restarts` و `hash_restarts`.
- انخفاض hit rate بعد warm-up.
- عدد scripts قريب من `max_cached_keys`.

Preloading

من PHP 7.4 يمكن تشغيل ملف مرة عند بدء عملية الخادم لتحميل functions/classes/interfaces/traits في ذاكرة دائمة:

```
opcache.preload=/var/www/app/config/preload.php
opcache.preload_user=www-data
```

```php
<?php
declare(strict_types=1);

$files = [
    __DIR__ . '/../src/Domain/Money.php',
    __DIR__ . '/../src/Domain/Order.php',
];

foreach ($files as $file) {
    opcache_compile_file($file);
}
```

التكلفة والمخاطر:

- يرفع الذاكرة الأساسية لكل بيئة تشغيل.
- أي تغيير في الكود المحمّل يحتاج restart؛ لا يكفي invalidation عادي.
- ترتيب الاعتماديات قد يهم إذا استخدمت `require` بدل `opcache_compile_file()`.
- لا يفيد تلقائيًا كل تطبيق؛ قِس قبل وبعد.
- Preloading غير مدعوم على Windows وفق دليل PHP، ومناسب أساسًا للعمليات المستمرة مثل FPM.

النشر الصحيح

1. انشر ملفات release جديدة في directory منفصل.
2. ثبّت الاعتماديات بـComposer وولّد autoloader محسّنًا.
3. شغّل الاختبارات وhealth checks.
4. بدّل symlink/release atomically.
5. أعد تحميل FPM gracefully حتى ترى workers الكود وpreload الجديد.

لا تخلط بين OPcache وComposer class map: الأول يخزّن bytecode، والثاني يسرّع **العثور على الملف** الذي يحتوي الصنف. استخدامهما معًا يعالج مرحلتين مختلفتين.

مسألة تشغيلية

▼ هل OPcache يخزن نتيجة الـrequest؟

لا؛ يخزن bytecode المترجم لتجنب parsing/compilation المتكرر، أما data cache فيخزن بيانات التطبيق.

شغّل وتحقق

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

نفّذ نقطة التحقق التالية داخل بيئة الدرس:

```
php -d opcache.enable_cli=1 -r "var_export(opcache_get_status(false) !== false);"
```

معيار النجاح: الناتج `true`. بعد ذلك قارن hits وmisses قبل وبعد حمل متكرر، ولا تعتبر تفعيل OPcache وحده دليل تحسن.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

اربط النقاط ببعض

قس memory_consumption وinterned strings وhit/miss وwasted percentage لتحديد sizing بدل نسخ إعداد. Invalidation وtimestamp policy يجب أن تتوافق مع atomic deployment. JIT لا يسرع كل workload ويبقى منفصلًا عن فائدة OPcache الأساسية. Preloading يثبت classes داخل عمر العملية ويتطلب restart عند التغيير.

دورة التجربة

قبل التنفيذ اكتب توقعك، ثم شغّل المثال وسجّل الخروج. أحدث فشلًا واحدًا مقصودًا، اجمع الدليل من logs أو metrics، أصلح السبب، وأعد التشغيل لإثبات أن الإصلاح يعالج العطل ولا يخفيه.

جرّب بنفسك

اجمع metrics قبل وبعد حمل مكرر وفسر eviction أو restart.

4. ذاكرة PHP وGarbage Collection

zval وReference Counting وCopy-on-write والمراجع والكائنات والدورات المرجعية وقياس الذاكرة.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Cache:** نسخة مؤقتة من البيانات هدفها تقليل وقت الانتظار والعمل المتكرر.
- **Queue:** طابور مهام تنتظر عاملًا ينفذها في الخلفية.
- **Worker:** برنامج يعمل في الخلفية ويسحب المهام من الطابور وينفذها.
- **Array:** مصفوفة تجمع قيمًا تحت اسم واحد وتسمح بالوصول لها بمفاتيح.
- **OPcache:** ذاكرة تحفظ نتيجة ترجمة PHP لتجنب تكرار العمل مع كل طلب.
- **PHP-FPM:** مدير عمليات يشغّل عمال PHP لصالح خادم الويب.
- **Redis:** مخزن سريع في الذاكرة يُستخدم للكاش والجلسات والطوابير.

كيف تمثل PHP القيم في الذاكرة؟

محرك Zend يمثل قيمة PHP ببنية داخلية تسمى `zval` تحمل نوع القيمة وبياناتها. بعض البيانات المركبة مثل strings وarrays وobjects لها هياكل إضافية محسوبة المراجع.

تصحيح للملاحظات القديمة

وصف `zval` بأنه دائمًا `type + value + is_ref + refcount` مناسب تاريخيًا لـPHP 5، لكنه تبسيط غير دقيق لـPHP 7/8. في المحرك الحديث قد يكون Reference نفسه zval منفصلًا، وتوجد تفاصيل refcount داخل البنية المشار إليها بحسب النوع. تعلّم السلوك المرئي في PHP ولا تعتمد على layout داخلي ثابت.

Reference Counting

عندما لا يعود شيء يشير إلى قيمة محسوبة المراجع، يمكن تحريرها فورًا عادة. لكن `refcount` ليس "عدد المتغيرات" بصورة مطلقة؛ قد تتدخل temporaries وinterning وتحسينات المحرك، لذلك مخرجات debugging قد تبدو أعلى من المتوقع.

```
$first = ['name' => 'Omar'];
$second = $first;

unset($first);
// ما زالت البيانات موجودة لأن $second يستخدمها.
```

Copy-on-write

عند الإسناد لا ينسخ PHP array أو string كبيرًا فورًا عادة؛ يشترك المتغيران في البيانات حتى يحاول أحدهما تعديلها:

```
$original = range(1, 100_000);
$copy = $original;       // مشاركة مؤقتة
$copy[] = 100_001;       // هنا يلزم فصل البيانات المعدلة
```

هذا يوفّر الذاكرة والوقت، لكنه يعني أن تعديل نسخة كبيرة قد يسبب memory spike لحظيًا.

الكائنات تختلف:

```
$a = new stdClass();
$a->count = 1;
$b = $a;
$b->count++;

echo $a->count; // 2
```

`$a` و`$b` يحملان handle للكائن نفسه. للحصول على كائن مستقل استخدم `clone` ونفّذ `__clone()` للنسخ العميق عند الحاجة.

References بعلامة &

```
$value = 10;
$alias =& $value;
$alias = 20;

echo $value; // 20
```

المرجع في PHP alias إلى نفس متغير التخزين، وليس مؤشر C عام يمكن إجراء pointer arithmetic عليه. لا تستخدم `&` كتحسين أداء؛ غالبًا يزيد التعقيد وقد يمنع تحسينات Copy-on-write.

لماذا نحتاج Garbage Collector؟

Reference counting وحده لا يحرر دورة تشير إلى نفسها:

```
final class Node
{
    public ?Node $next = null;
}

$a = new Node();
$b = new Node();
$a->next = $b;
$b->next = $a;

unset($a, $b); // بقيت دورة لا يصل إليها كود المستخدم
```

يسجل GC الحاويات المرشحة ويفحص الدورات دوريًا. يمكن طلب دورة جمع يدويًا:

```
$cycles = gc_collect_cycles();
printf("Collected %d cycles\n", $cycles);
```

لا تستدعها بعد كل عملية. استخدمها في workers طويلة العمر فقط بعد القياس، مثل queue consumer ينشئ graphs كبيرة بين jobs.

الذاكرة في PHP-FPM

في نموذج request/response تُحرر ذاكرة الطلب عادة عند انتهائه، لكن الـWorker نفسه يبقى حيًا وقد تحتفظ Extensions أو caches أو fragmentation بذاكرة. لذلك `pm.max_requests` مفيد لإعادة تدوير workers دوريًا، لكنه لا يصلح تسربًا منطقيًا داخل عملية طويلة.

القياس

```
$before = memory_get_usage(true);

$rows = loadReportRows();

printf(
    "current=%s peak=%s delta=%s\n",
    number_format(memory_get_usage(true)),
    number_format(memory_get_peak_usage(true)),
    number_format(memory_get_usage(true) - $before),
);
```

`true` يعرض الذاكرة المخصصة من النظام لمحرك PHP، وقد تكون أكبر من الحجم الفعلي المستخدم. قِس داخل workload واقعي واستخدم profiler عند الحاجة.

تقليل الاستهلاك

- استخدم Generator أو pagination بدل تحميل ملايين الصفوف.
- حرر المراجع الكبيرة داخل loops طويلة بـ `unset()` عند ثبوت الحاجة.
- تجنب `fetchAll()` للنتائج الضخمة.
- انتبه إلى closures التي تلتقط كائنات كبيرة.
- افصل cache مشتركة مثل OPcache/Redis عن ذاكرة heap الخاصة بالطلب.
- لا ترفع `memory_limit` قبل معرفة سبب الاستهلاك.

مسألة تشغيلية

▼ ليه worker طويل العمر ممكن يزيد استهلاكه؟

مراجع وcaches وcycles قد تبقى بين المهام؛ راقب الذاكرة وحرر الموارد وأعد تشغيل worker بحدود مدروسة.

شغّل وتحقق

الملف يقارن بناء 100000 عنصر بمرور حسابي دون مصفوفة، ويطبع الذاكرة والذروة. الذروة تراكمية داخل العملية وقد لا تنخفض بعد unset؛ الأرقام تختلف حسب البناء والمخصص.

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

نفّذ نقطة التحقق التالية داخل بيئة الدرس:

```
php memory-lab.php
```

هدف تجربة التكامل الموسعة: يطبع السكربت الذاكرة الحالية والذروة لكل مرحلة، وتبقى الذروة تحت الميزانية المحددة للملف الكبير.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

اربط النقاط ببعض

Generator وstreaming يقللان working set عندما لا تحتاج كل البيانات معًا. في worker طويل العمر حرر مراجع resources بين jobs وراقب growth لا لقطة واحدة. `memory_limit` ليس كل RSS لأن extensions وnative libraries وmapped files قد تقع خارجه.

دورة التجربة

قبل التنفيذ اكتب توقعك، ثم شغّل المثال وسجّل الخروج. أحدث فشلًا واحدًا مقصودًا، اجمع الدليل من logs أو metrics، أصلح السبب، وأعد التشغيل لإثبات أن الإصلاح يعالج العطل ولا يخفيه.

جرّب بنفسك

عالج 100 job وراقب RSS بعد كل عشرة لكشف retention.

5. PHP-FPM وإدارة العمليات

FastCGI وMaster وWorkers وPools وأنماط static وdynamic وondemand وحساب السعة والمراقبة.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **HTTP:** قواعد تبادل الطلبات والردود بين المتصفح والخادم.
- **URL:** العنوان الكامل لمورد على الويب، زي صفحة أو صورة أو نقطة API.
- **TCP:** طريقة نقل بتتأكد إن البيانات وصلت كاملة وبالترتيب.
- **Queue:** طابور مهام تنتظر عاملًا ينفذها في الخلفية.
- **Worker:** برنامج يعمل في الخلفية ويسحب المهام من الطابور وينفذها.
- **OPcache:** ذاكرة تحفظ نتيجة ترجمة PHP لتجنب تكرار العمل مع كل طلب.
- **PHP-FPM:** مدير عمليات يشغّل عمال PHP لصالح خادم الويب.
- **FastCGI:** طريقة اتصال يرسل بها خادم الويب طلب التنفيذ إلى PHP-FPM.
- **Nginx:** خادم ويب يقدم الملفات أو يمرر طلبات PHP.
- **Container:** بيئة معزولة تجمع التطبيق واعتماداته.
- **Log:** سجل زمني لحدث حصل داخل التطبيق.

PHP-FPM ليس خادم HTTP

PHP-FPM اختصار **FastCGI Process Manager**، وهو التنفيذ الأساسي لـFastCGI في PHP. يستقبل طلبات FastCGI من خادم موثوق مثل Nginx أو Apache، يدير مجموعة عمليات PHP، ثم يعيد الاستجابة.

```
Nginx/Apache -> FastCGI socket -> FPM master -> worker -> PHP response
```

تحذير

لا تعرّض منفذ PHP-FPM لشبكة غير موثوقة. عميل FastCGI يستطيع تمرير إعدادات تؤثر في تنفيذ الطلب؛ اربطه بـUnix socket أو interface داخلي واضبط الصلاحيات و `listen.allowed_clients` عند TCP.

Workersو Master

- Master process يقرأ إعدادات pools ويدير الإنشاء والإيقاف وإعادة التحميل.
- Worker process يعالج طلب PHP واحدًا في اللحظة في النموذج المعتاد.
- Pool يعزل مجموعة workers بإعدادات مثل user/group وsocket وphp.ini values والحدود.

يمكن أن تخدم تطبيقين بمستخدمين وsockets منفصلة لتقليل أثر الاختراق، بدل وضع كل المواقع في pool واحدة.

أوضاع Process Manager

static

ينشئ workers بالضبط `pm.max_children` ويحتفظ بها. أداء متوقع ولا يوجد spawn عند الضغط، لكنه يحجز الذاكرة حتى في الهدوء.

```
pm = static
pm.max_children = 20
```

dynamic

يبدأ بعدد workers ويحافظ على spare capacity بين حدين:

```
pm = dynamic
pm.max_children = 40
pm.start_servers = 8
pm.min_spare_servers = 4
pm.max_spare_servers = 12
```

غالبًا اختيار متوازن لخدمة نشطة.

ondemand

ينشئ workers عند وصول الطلبات ويقتل الخامل بعد مدة:

```
pm = ondemand
pm.max_children = 20
pm.process_idle_timeout = 10s
```

يوفر ذاكرة للمواقع قليلة المرور مقابل cold-start وتكلفة spawn.

حساب pm.max_children

ابدأ بقياس RSS الفعلي بعد warm-up:

```
available RAM for FPM / realistic high-percentile worker RSS
```

مثال: إذا خصصت 2 GiB لـFPM وكان worker عند الحمل يستهلك 80 MiB، فالحد النظري 25. اترك هامشًا للنظام وخادم الويب وOPcache وقاعدة البيانات؛ قد تبدأ مثلًا بـ18–20 ثم تختبر.

لا تقسم `memory_limit` على RAM مباشرة؛ `memory_limit` حد request تقريبي وليس RSS العامل المعتاد، وقد توجد ذاكرة Extensions أو shared memory خارجه.

إعدادات تشغيل مفيدة

```
pm.max_requests = 500
request_terminate_timeout = 30s
request_terminate_timeout_track_finished = yes
request_slowlog_timeout = 3s
slowlog = /var/log/php-fpm/app-slow.log
catch_workers_output = yes
ping.path = /fpm-ping
pm.status_path = /fpm-status
```

- `pm.max_requests` يعيد تدوير worker بعد عدد طلبات، مفيد للحد من نمو طويل الأمد.
- slowlog يسجل backtrace للطلبات البطيئة، وليس مجرد URL.
- status يعرض active/idle processes وqueue وmax children reached.
- لا تنشر ping/status للعامة؛ اسمح بهما داخليًا فقط.

Little’s Law وqueue

إذا وصل 100 طلب/ثانية ومتوسط خدمة PHP هو 200ms، فالتزامن المتوسط يقارب:

```
concurrency = throughput × latency = 100 × 0.2 = 20 workers
```

هذا متوسط لا يغطي القمم أو tail latency. إذا كان `max children reached` أو listen queue يرتفع، فإما أن السعة قليلة أو التطبيق/قاعدة البيانات بطيئة. زيادة workers بلا حدود قد تغرق قاعدة البيانات.

Graceful reload وfastcgi_finish_request

استخدم reload من مدير الخدمة في النشر حتى تكمل العمليات الحالية حيث أمكن. وتسمح `fastcgi_finish_request()` بإرسال الاستجابة ثم متابعة عمل قصير، لكنها تبقي worker مشغولًا؛ الأعمال الثقيلة أو الموثوقة مكانها queue worker مستقل. افتراضيًا لا يطبق FPM قيمة `request_terminate_timeout` بعد انتهاء الطلب أو بعد `fastcgi_finish_request()` ؛ تفعيل `request_terminate_timeout_track_finished` يجعل الحد يشمل هذه المرحلة أيضًا.

مسألة تشغيلية

▼ إيه خطر رفع عدد FPM workers بلا حساب؟

قد تتجاوز الذاكرة وتزيد التزاحم على CPU أو database؛ احسبها من ذاكرة العملية والحمل وحدود downstream.

شغّل وتحقق

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

شغّل بيئة `examples/php-labs/production` كما في README، ثم نفّذ:

```
curl -sS -o /dev/null -w "status=%{http_code} total=%{time_total}
" http://localhost/health
```

معيار النجاح: يرجع health check حالة 200، وتُظهر `docker compose ps` الخدمتين بحالة healthy. تحت حمل مضبوط يظل عدد workers داخل الحد، واربط التأخير بطابور FPM بدل التخمين. في PowerShell على Windows استبدل `/dev/null` بـ `NUL` .

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

اربط النقاط ببعض

فعّل status page وslowlog على مسار داخلي محمي لتفهم active/idle/listen queue والطلبات البطيئة. احسب memory per child من قياس تحت حمل، واترك هامشًا للنظام والweb server. داخل container لا تجعل autoscaling يخفي max_children غير صحيح، واضبط graceful termination مع مهلة المنصة.

دورة التجربة

قبل التنفيذ اكتب توقعك، ثم شغّل المثال وسجّل الخروج. أحدث فشلًا واحدًا مقصودًا، اجمع الدليل من logs أو metrics، أصلح السبب، وأعد التشغيل لإثبات أن الإصلاح يعالج العطل ولا يخفيه.

جرّب بنفسك

اربط ارتفاع queue بالـslowlog وRSS قبل تغيير عدد العمال.

6. Apache وNginx وFastCGI

مسؤولية خادم الويب وApache MPM وNginx event loop وربط PHP-FPM عبر Unix أو TCP socket.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Runtime:** وقت التشغيل: الفترة اللي البرنامج بيكون شغال فيها فعلًا.
- **HTTP:** قواعد تبادل الطلبات والردود بين المتصفح والخادم.
- **IP:** عنوان رقمي بيميز جهازًا أو واجهة شبكة.
- **TCP:** طريقة نقل بتتأكد إن البيانات وصلت كاملة وبالترتيب.
- **TLS:** طبقة تشفير بتحمي البيانات وهي ماشية بين طرفين.
- **Proxy:** وسيط يستقبل الطلب ويمرره لجهة أخرى حسب قواعد محددة.
- **Worker:** برنامج يعمل في الخلفية ويسحب المهام من الطابور وينفذها.
- **Loop:** حلقة تكرار تعيد تنفيذ مجموعة تعليمات وفق شرط.
- **PHP-FPM:** مدير عمليات يشغّل عمال PHP لصالح خادم الويب.
- **FastCGI:** طريقة اتصال يرسل بها خادم الويب طلب التنفيذ إلى PHP-FPM.
- **Nginx:** خادم ويب يقدم الملفات أو يمرر طلبات PHP.

أين ينتهي خادم الويب وتبدأ PHP؟

خادم الويب يستقبل HTTP/TLS، يقدّم الملفات الثابتة، يطبق routing وحدود الطلب، ويرسل ملفات PHP إلى PHP-FPM. runtime ينفذ PHP؛ لا تجعل كل طلب، بما فيه الصور وCSS، يمر عبر التطبيق بلا داعٍ.

تصحيح مقارنة Apache وNginx

قول "Apache process لكل connection" غير دقيق كقاعدة عامة. Apache يستخدم **MPM** واحدًا:

- `prefork`: عمليات بلا threads؛ مناسب لبعض التوافقات القديمة.
- `worker`: processes تحتوي threads.

- threaded: `event` ويعالج keep-alive بكفاءة أكبر.

Nginx يعتمد workers تقود event loop غير متزامنة، فيخدم اتصالات كثيرة بعدد محدود من العمليات. لكن هذا لا يعني أن PHP نفسها تصبح async؛ كل طلب PHP يشغل FPM worker حتى ينتهي.

Apache

يمكن تشغيل PHP تاريخيًا داخل Apache module، لكن فصل Apache عن PHP-FPM عبر FastCGI يعطي عزلًا وإدارة عمليات أوضح.

```
<VirtualHost *:443>
    ServerName example.com
    DocumentRoot /var/www/app/public

    <Directory /var/www/app/public>
        AllowOverride None
        Require all granted
        FallbackResource /index.php
    </Directory>

    <FilesMatch \.php$>
        SetHandler "proxy:unix:/run/php/app.sock|fcgi://localhost/"
    </FilesMatch>
</VirtualHost>
```

`.htaccess` يسمح بإعدادات موزعة إذا كان `AllowOverride` مفعّلًا، لكنه يسبب filesystem checks ويجعل السياسة موزعة. في خادم تتحكم به، ضع القواعد في VirtualHost واضبط `AllowOverride None`. في shared hosting قد يكون `.htaccess` هو الخيار المتاح.

Nginx مع PHP-FPM

```
server {
    listen 443 ssl;
    server_name example.com;
    root /var/www/app/public;
    index index.php;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        try_files $uri =404;
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        fastcgi_param HTTP_PROXY "";
        fastcgi_pass unix:/run/php/app.sock;
        fastcgi_read_timeout 30s;
    }

    location ~ /\. {
        deny all;
    }
}
```

النمط الآمن لتطبيق Front Controller هو تمرير `/index.php` فقط عندما لا تحتاج تنفيذ ملفات PHP أخرى. يقلل هذا مساحة الخطأ:

```
location = /index.php {
    include fastcgi_params;
    fastcgi_param SCRIPT_FILENAME $document_root/index.php;
    fastcgi_pass unix:/run/php/app.sock;
}
```

لا تضع `.env` أو `vendor/` أو uploads القابلة للتنفيذ داخل public root.

Unix socket أم TCP؟

```
fastcgi_pass unix:/run/php/app.sock;
# أو
fastcgi_pass 127.0.0.1:9000;
```

- Unix socket مناسب عندما يكون Nginx وFPM على الجهاز نفسه؛ راجع owner/group/mode.
- TCP مطلوب عادة عبر containers/hosts منفصلة؛ اربطه بشبكة خاصة وجدار ناري.
- الفرق الأدائي غالبًا أقل أهمية من صحة الإعداد والمراقبة، فلا تختَر بالشعارات.

معاملات FastCGI

`SCRIPT_FILENAME` يخبر PHP بالملف المطلوب. خطأ فيه ينتج "Primary script unknown" أو قد يفتح مسارًا غير مقصود. مرّر كذلك method وquery/content parameters عبر الملف القياسي الملائم لتوزيعتك.

لا تثق في headers يرسلها العميل مثل `X-Forwarded-For` إلا إذا جاءت من reverse proxy موثوق ومحدد. اضبط real-IP/trusted proxies بوضوح حتى لا يزوّر المستخدم IP أو scheme.

حدود ومسؤوليات

- Web server: TLS، static files، request-size limits، timeouts، buffering.
- PHP-FPM: عدد workers، PHP settings، slowlog، status.
- Application: validation، authorization، business logic، response.
- CDN/load balancer: edge caching، health routing، وقد ينهي TLS.

اجعل timeouts متناسقة: مهلة upstream في خادم الويب يجب ألا تخفي عملية PHP عالقة بلا حد، و`request_terminate_timeout` في FPM يجب أن يعكس SLA وطبيعة endpoint.

مسألة تشغيلية

▼ ماذا يعني 502 بين Nginx وPHP-FPM؟

لم يحصل Nginx على FastCGI response صالح؛ افحص socket والخدمة والمهلة والسجلات قبل كود الصفحة.

شغّل وتحقق

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

شغّل `examples/php-labs/production` كما في README، ثم نفّذ:

```
curl -sS -D - http://127.0.0.1:8080/health -o /dev/null
curl -fsS http://127.0.0.1:8080/
```

معيار النجاح: ترى status وheaders من PHP في الطلب الأول، بينما يعرض الطلب الثاني ملفًا ساكنًا من Nginx. طلب `/health.php` أو مسار غير موجود يرجع 404 ولا يكشف مسارًا داخليًا. في PowerShell استبدل `/dev/null` بـ `NUL`.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

اربط النقاط ببعض

حدد من يملك TLS وHTTP/2 والضغط والstatic files. اضبط request buffering وresponse buffering وtimeouts وحدود body حسب endpoint. اربط SCRIPT_FILENAME بمسار موثوق ولا تسمح path info بفتح ملف غير مقصود، وأضف security headers في طبقة واضحة دون تكرار متعارض.

دورة التجربة

قبل التنفيذ اكتب توقعك، ثم شغّل المثال وسجّل الخروج. أحدث فشلًا واحدًا مقصودًا، اجمع الدليل من logs أو metrics، أصلح السبب، وأعد التشغيل لإثبات أن الإصلاح يعالج العطل ولا يخفيه.

جرّب بنفسك

اختبر static وPHP وupload كبيرًا ومسارًا غير موجود واقرأ أي طبقة أجابت.

7. متغيرات البيئة وإدارة الإعدادات

getenv و$_ENV وputenv و.env وتحويل الإعدادات والتحقق منها وحماية الأسرار في PHP-FPM.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Worker:** برنامج يعمل في الخلفية ويسحب المهام من الطابور وينفذها.
- **Boolean:** قيمة منطقية لها حالتان فقط: صح أو خطأ.
- **Function:** دالة: جزء كود له اسم ومهمة محددة ويمكن استدعاؤه أكثر من مرة.
- **Composer:** مدير حزم PHP يثبت المكتبات ويجهز التحميل التلقائي.
- **PHP-FPM:** مدير عمليات يشغّل عمال PHP لصالح خادم الويب.
- **Container:** بيئة معزولة تجمع التطبيق واعتماداته.

افصل الإعداد عن الكود

متغيرات البيئة قيم يوفّرها نظام التشغيل أو container أو مدير الخدمة للعملية. تستخدم لتغيير إعدادات البيئة دون تعديل source code، مثل DSN وقيمة debug واسم البيئة.

```
OS / container / service manager
              -> PHP-FPM master
              -> worker process
              -> getenv() / $_SERVER / $_ENV
```

getenv و$_ENV وputenv

```
$dsn = getenv('DATABASE_DSN');

if ($dsn === false || $dsn === '') {
    throw new RuntimeException('DATABASE_DSN is required');
}
```

افحص `=== false` لأن النص `"0"` قيمة صحيحة لكنه falsy.

- `getenv('NAME')` يقرأ متغيرًا ويعيد `false` عند غيابه.
- `$_ENV` نسخة superglobal من القيم التي استوردتها PHP، وقد تكون فارغة إذا كان `variables_order` لا يحتوي `E`.
- `$_SERVER` قد يحتوي بعض قيم البيئة/CGI حسب SAPI.
- `putenv('NAME=value')` يغيّر بيئة العملية للطلب الحالي، ولا يعني أن `$_ENV` سيتزامن تلقائيًا معه.

> ⚠
>
> **تصحيح مهم**
>
> PHP لا تقرأ ملف `.env` تلقائيًا. تحتاج framework أو مكتبة مثل `vlucas/phpdotenv` أو bootstrap خاص. في الإنتاج قد تُحقن المتغيرات مباشرة من orchestrator أو secret manager دون وجود ملف.

تحميل .env في التطوير

```
composer require vlucas/phpdotenv
```

```
<?php
declare(strict_types=1);

require dirname(__DIR__) . '/vendor/autoload.php';

$dotenv = Dotenv\Dotenv::createImmutable(dirname(__DIR__));
$dotenv->safeLoad();
$dotenv->required(['APP_ENV', 'DATABASE_DSN']);
```

```
# .env.example - أسماء وأمثلة غير سرية
APP_ENV=local
APP_DEBUG=false
DATABASE_DSN=mysql:host=127.0.0.1;dbname=app;charset=utf8mb4
```

- ارفع `.env.example` بلا أسرار.
- ضع `.env` في `.gitignore` وخارج document root.
- لا تضع قيم production الحقيقية في README أو logs أو frontend bundle.
- إذا تسرب سر، احذفه من السجل و**دوّره**؛ حذف commit وحده لا يبطل السر.

طبقة Config واحدة

لا تستدع `getenv()` في كل class. حوّل النصوص وتحقق منها عند بدء التطبيق:

```
final readonly class AppConfig
{
    public function __construct(
        public string $environment,
        public bool $debug,
        public string $databaseDsn,
        public int $httpTimeoutMs,
    ) {}

    public static function fromEnvironment(): self
    {
        $dsn = getenv('DATABASE_DSN');
        if ($dsn === false || $dsn === '') {
            throw new RuntimeException('DATABASE_DSN is required');
        }

        $debug = filter_var(
            getenv('APP_DEBUG') ?: 'false',
            FILTER_VALIDATE_BOOL,
            FILTER_NULL_ON_FAILURE,
        );

        if ($debug === null) {
            throw new RuntimeException('APP_DEBUG must be true or false');
        }

        $timeout = filter_var(
            getenv('HTTP_TIMEOUT_MS') ?: '5000',
            FILTER_VALIDATE_INT,
            ['options' => ['min_range' => 100, 'max_range' => 60_000]],
        );

        if ($timeout === false) {
            throw new RuntimeException('HTTP_TIMEOUT_MS is invalid');
        }

        return new self(
            environment: getenv('APP_ENV') ?: 'production',
            debug: $debug,
            databaseDsn: $dsn,
            httpTimeoutMs: $timeout,
        );
    }
}
```

بهذا يفشل التطبيق مبكرًا بدل اكتشاف إعداد ناقص وسط طلب حقيقي، وتتعامل بقية الطبقات مع أنواع صحيحة.

PHP-FPM والبيئة

FPM قد ينظف البيئة افتراضيًا عبر `clear_env`. مرّر قائمة صريحة في إعداد pool أو عبر مدير الخدمة:

```
; app-pool.conf
clear_env = yes
env[APP_ENV] = $APP_ENV
env[DATABASE_DSN] = $DATABASE_DSN
```

الأفضل عدم نسخ البيئة كاملة بلا حاجة. امنح كل pool الحد الأدنى من الأسرار والصلاحيات.

قواعد الأسرار

- Environment variables ليست تشفيرًا؛ يمكن أن تظهر لأدوات مراقبة أو child processes حسب النظام.
- استخدم secret manager عندما تحتاج rotation وauditing وسياسات وصول.
- لا تعرض `phpinfo()` أو dump كاملًا لـ `$_ENV` في الإنتاج.
- افصل secrets عن الإعدادات العامة، ودوّر كلمات المرور والمفاتيح دوريًا.
- لا ترسل سرًا إلى browser؛ ما يصل إلى JavaScript لم يعد server secret.

ترتيب الأولوية

حدد سياسة واحدة واضحة، مثل:

```
hard-coded safe defaults
  <- committed non-secret config
  <- environment-specific values
  <- secret manager
```

لا تجعل ترتيب الدمج ضمنيًا؛ سجّل **مصدر الإعداد وأسماء المفاتيح** عند التشخيص، دون تسجيل القيم السرية.

مسألة تشغيلية

▼ ليه ما نحطش secret في Git؟

تاريخ Git يظل محتفظًا به وقد يصل لنسخ كثيرة؛ استخدم secret manager ودوّر السر إذا تسرب.

شغّل وتحقق

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

نفّذ نقطة التحقق التالية داخل بيئة الدرس:

```
php config-check.php
```

معيار النجاح: يفشل التشغيل مبكرًا عند غياب متغير مطلوب، ويطبع أسماء الإعدادات المحمّلة لا قيم الأسرار.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

اربط النقاط ببعض

ابن typed configuration schema يحقق القيم مرة عند startup ويحول المدة والحجم وBoolean صراحة. حدد precedence بين defaults وfile وenvironment وimmutable. Configuration secret manager داخل request، وsecret rotation تحتاج نافذة تقبل القديم والجديد دون طباعة أي قيمة.

دورة التجربة

قبل التنفيذ اكتب توقعك، ثم شغّل المثال وسجّل الخروج. أحدث فشلًا واحدًا مقصودًا، اجمع الدليل من logs أو metrics، أصلح السبب، وأعد التشغيل لإثبات أن الإصلاح يعالج العطل ولا يخفيه.

جرّب بنفسك

احذف متغيرًا مطلوبًا وأدخل Boolean ملتبسًا وتأكد من fail-fast.

8. PHP CLI وphp.ini والامتدادات

اختلاف SAPIs واكتشاف الإعدادات وإدارة extensions وضبط التطوير والإنتاج.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Runtime:** وقت التشغيل: الفترة اللي البرنامج بيكون شغال فيها فعلًا.
- **CLI:** واجهة تتعامل معها بكتابة أوامر نصية بدل الضغط على أزرار.

SAPI تحدد بيئة التشغيل

PHP قد تعمل عبر CLI أو FPM/FastCGI أو Apache module. نفس الكود قد يقرأ `php.ini` مختلفة ويملك extensions وإعدادات مختلفة.

```
php -v
php --ini
php -m
php -i
php -r "echo PHP_SAPI, PHP_EOL;"
```

عند ظهور "يعمل في terminal ولا يعمل في الموقع"، قارن executable وSAPI وملف الإعداد والمستخدم والبيئة.

ترتيب الإعداد

`php --ini` يعرض الملف الأساسي ومجلد `conf.d`. الملفات الإضافية قد تعدّل قيمة سابقة. داخل التطبيق:

```
printf(
    "sapi=%s ini=%s memory=%s\n",
    PHP_SAPI,
    php_ini_loaded_file() ?: 'none',
    ini_get('memory_limit'),
);
```

بعض الإعدادات `PHP_INI_SYSTEM` ولا يمكن تغييرها بـ `ini_set()` أثناء الطلب. لا تجعل bootstrap يخفي configuration drift.

Extensions

```
php --ri opcache
php --ri pdo_mysql
composer check-platform-reqs --lock --no-dev
```

`extension_loaded()` يفيد في فحص واضح، لكن declare requirements في Composer أفضل لمنع نشر بيئة ناقصة.

Development مقابل Production

في التطوير: أخطاء ظاهرة، Xdebug عند الحاجة، وOPcache بسياسة مناسبة للتغييرات. في الإنتاج: لا تعرض الأخطاء، فعّل logs وOPcache، اضبط limits وtimeouts، وأزل extensions غير اللازمة.

لا تنسخ `php.ini-development` أو `php.ini-production` بلا مراجعة؛ هما baseline وليسا إعداد تطبيقك النهائي.

CLI scripts

```
#!/usr/bin/env php
<?php
declare(strict_types=1);

set_time_limit(0);
$options = getopt('', ['dry-run', 'limit:']);
```

أضف exit codes صحيحة، signal handling للـlock، logging، workers لمنع نسختين، وtimeouts لأي I/O. `set_time_limit(0)` لا يعني أن النظام أو orchestrator لن يقتل العملية.

دورة حياة PHP والترقية

لا يكفي أن تقول إن التطبيق يدعم PHP 8.x. سجّل أقل وأعلى minor مختبرة، وتاريخ انتهاء الدعم، ونسخ extensions. قبل الترقية:

1. اقرأ Migration Guide والـbackward-incompatible changes والـdeprecations.
2. شغّل الاختبارات والتحليل الساكن على النسختين في CI.
3. ابنِ image جديدة بكل extensions بدل استبدال binary وحدها.
4. راقب startup warnings وOPcache وFPM errors في canary.
5. احتفظ بخطة rollback متوافقة مع schema وqueue payload.

تعامل مع `E_DEPRECATED` كإشارة عمل مبكرًا في CI، لا كسبب لعرض الأخطاء للمستخدم. افصل `display_errors=Off` عن `error_reporting=E_ALL`: يمكن تسجيل الخطأ كاملًا مع إخفائه عن response.

الأعطال المبكرة وExtension ABI

افحص startup errors قبل health check. extension مبنية لـPHP ABI أو thread-safety مختلف قد تفشل قبل تشغيل التطبيق. ثبّت مصدر الحزمة ونسختها داخل image، ثم نفّذ:

```
php -v
php --ini
php -m
php --ri opcache
composer check-platform-reqs --lock --no-dev
```

قارن CLI وFPM من endpoint داخلي محمي، واختبر fatal startup وغياب extension عمدًا. لا تعرض `phpinfo()` للعامة؛ يحتوي paths وإعدادات وبيانات بيئة.

مسألة تشغيلية

▼ ليه السلوك يختلف بين CLI والويب؟

قد يستخدمان php.ini وSAPI وextensions وuser مختلفين؛ افحص `php --ini` و `phpinfo()` لكل بيئة.

شغّل وتحقق

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

نفّذ نقطة التحقق التالية داخل بيئة الدرس:

```
php --ini
```

معيار النجاح: حدّد ملف الإعداد المحمّل للـCLI ثم نفّذ `php -m` وتأكد أن الامتداد المطلوب ظاهر في البيئة نفسها التي تشغّل الأمر.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

اربط النقاط ببعض

INI modes تحدد هل الإعداد يتغير في php.ini أو per-directory أو runtime. قارن `php -ini` وFPM info لتجنب تعديل ملف CLI فقط. PECL extension يجب أن يطابق PHP ABI والـthread safety والمنصة؛ افحص `php -i` وstartup errors بعد الترقية.

دورة التجربة

قبل التنفيذ اكتب توقعك، ثم شغّل المثال وسجّل الخروج. أحدث فشلًا واحدًا مقصودًا، اجمع الدليل من logs أو metrics، أصلح السبب، وأعد التشغيل لإثبات أن الإصلاح يعالج العطل ولا يخفيه.

جرّب بنفسك

اثبت أن CLI وFPM يحملان الإصدار والـini والامتداد المقصود نفسه.

9. Logging وObservability

PSR-3 وstructured logs وrequest IDs وmetrics وtraces والتنقية والمراقبة.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **HTTP:** قواعد تبادل الطلبات والردود بين المتصفح والخادم.
- **API:** واجهة محددة تسمح لبرنامج يطلب بيانات أو ينفّذ عملية عند برنامج آخر.
- **Proxy:** وسيط يستقبل الطلب ويمرره لجهة أخرى حسب قواعد محددة.
- **Cache:** نسخة مؤقتة من البيانات هدفها تقليل وقت الانتظار والعمل المتكرر.
- **Session:** بيانات مؤقتة تساعد الخادم يميّز المستخدم بين أكثر من طلب.
- **Cookie:** قيمة صغيرة يحفظها المتصفح ويرسلها مع الطلبات المناسبة.
- **Queue:** طابور مهام تنتظر عاملًا ينفذها في الخلفية.
- **Function:** دالة: جزء كود له اسم ومهمة محددة ويمكن استدعاؤه أكثر من مرة.

ثلاثة أنواع من الإشارة

- **Logs:** أحداث مفصلة قابلة للبحث.
- **Metrics:** أرقام مجمعة عبر الزمن مثل latency وerror rate.
- **Traces:** رحلة request عبر services وdatabase وqueues.

لا تعالج observability بإضافة `error_log` في كل مكان؛ صمّم schema وسياسة retention وتنبيهات.

Structured Logging وPSR-3

```
use Psr\Log\LoggerInterface;

final class Checkout
{
    public function __construct(private LoggerInterface $logger) {}

    public function run(Order $order, string $requestId): void
    {
        $this->logger->info('checkout.started', [
            'request_id' => $requestId,
            'order_id' => $order->id(),
        ]);
    }
}
```

مرّر البيانات كـcontext بدل تركيب نص يصعب تحليله. Monolog تنفيذ شائع لـPSR-3.

Correlation

أنشئ request ID عند edge أو اقبلها فقط من proxy موثوق، ثم مرّرها إلى logs وHTTP outgoing وTrace ID وqueue metadata. ليست بالضرورة user-visible request ID.

ما لا نسجله

- كلمات المرور وsession IDs وaccess/refresh tokens.
- Authorization/Cookie headers.
- مفاتيح التشفير والأسرار.
- bodies كاملة أو بيانات شخصية بلا حاجة.

طبّق allow-list أو redaction واختبرها. الـlogs نفسها بيانات حساسة وتحتاج access control وintegrity وretention.

Metrics مفيدة

استخدم latency histograms وrequest rate وerror rate وsaturation. راقب PHP-FPM queue و`max children reached`، cache hit ratio، queue lag، external API latency، database pool.

تنبيه عملي

التنبيه يجب أن يعكس أثرًا قابلًا للتصرف، مثل ارتفاع نسبة `5xx` أو p95 latency، لا كل exception منفردة. اربط deployment version بالـlogs والـmetrics لتحديد regression.

أخطاء المحرك والتطبيق

فرّق بين validation متوقعة، exception تطبيق، warning من dependency، وfatal engine error. العام يعطي status response ورسالة آمنة وerror ID، بينما السجل الداخلي يحتفظ بالنوع والـstack والـrequest ID بلا secrets.

```
set_exception_handler(static function (Throwable $error): void {
    $errorId = bin2hex(random_bytes(8));
    error_log(json_encode([
        'event' => 'request.failed',
        'error_id' => $errorId,
        'exception' => $error::class,
    ], JSON_THROW_ON_ERROR));
    http_response_code(500);
    echo json_encode(['error' => 'internal_error', 'error_id' => $errorId]);
});
```

`set_exception_handler` لا يلتقط كل startup أو parse failure. وجّه `log_errors` إلى stderr، راقب FPM/Nginx startup logs، واستخدم shutdown handler بحذر لقراءة `error_get_last()` دون محاولة متابعة request فاسدة. اختبر أن `display_errors=Off` وأن response لا تحتوي path أو stack.

مرجع

- PSR-3 Logger Interface

مسألة تشغيلية

▼ ما الذي يجعل log قابلًا للتتبع؟

حدث منظم مع timestamp وseverity وrequest/trace ID وسياق آمن، من غير كلمات مرور أو tokens.

شغّل وتحقق

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

نفّذ نقطة التحقق التالية داخل بيئة الدرس:

```
php observability-lab.php 2> event.log
```

معيار النجاح: يحتوي كل حدث على timestamp وlevel وrequest_id وmessage، ولا يحتوي كلمة مرور أو Authorization header.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

اربط النقاط ببعض

أضف trace context وspan IDs عبر HTTP والqueue وقاعدة البيانات، وحدد sampling يحفظ الأخطاء والطلبات النادرة دون تكلفة كاملة. اضبط cardinality للlabels ولا تضع user ID الخام في metric. حدد retention وredaction وحق الوصول واربط log وmetric وtrace بـcorrelation واحد.

دورة التجربة

قبل التنفيذ اكتب توقعك، ثم شغّل المثال وسجّل الخروج. أحدث فشلًا واحدًا مقصودًا، اجمع الدليل من logs أو metrics، أصلح السبب، وأعد التشغيل لإثبات أن الإصلاح يعالج العطل ولا يخفيه.

جرّب بنفسك

تتبع request واحدًا عبر ثلاث خدمات وتأكد من عدم ظهور secret.

10. الاختبارات وجودة الكود

PHPUnit وUnit/Integration/Feature tests وData Providers وDoubles وCoverage والتحليل الساكن والتنسيق.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **HTTP:** قواعد تبادل الطلبات والردود بين المتصفح والخادم.
- **Function:** دالة: جزء كود له اسم ومهمة محددة ويمكن استدعاؤه أكثر من مرة.

هرم عملي

- **Unit:** منطق صغير بلا شبكة أو قاعدة بيانات.
- **Integration:** تعاون حقيقي مع database/filesystem/client adapter.
- **Feature/HTTP:** request كاملة عبر التطبيق.
- **End-to-end:** النظام من منظور المستخدم؛ أقل عددًا وأعلى كلفة.

لا تحول كل شيء إلى unit test مع mocks. اختبر العقد عند الحدود والسلوك المهم للمستخدم.

PHPUnit

```
composer require --dev phpunit/phpunit
vendor/bin/phpunit
```

```php
use PHPUnit\Framework\Attributes\DataProvider;
use PHPUnit\Framework\TestCase;

final class DiscountTest extends TestCase
{
    #[DataProvider('cases')]
    public function testDiscount(int $price, int $percent, int $expected): void
    {
        self::assertSame($expected, discount($price, $percent));
    }

    public static function cases(): iterable
    {
        yield 'none' => [1000, 0, 1000];
        yield 'ten percent' => [1000, 10, 900];
    }
}
```

اختبر exception والحدود والـside effects، واجعل اسم الاختبار يشرح السلوك.

Test Doubles

- **Stub:** يعيد بيانات مجهزة.
- **Fake:** تنفيذ خفيف يعمل، مثل repository في الذاكرة.
- **Mock:** يتحقق من interaction متوقع.

فضّل fake/stub عند الإمكان. كثرة mocks تربط الاختبار بتفاصيل التنفيذ وتكسر refactoring.

Coverage

Coverage تكشف السطور غير المنفذة لكنها لا تثبت صحة assertions. راقب الفروع والسلوك الحرج، ولا تجعل الوصول إلى 100% هدفًا مستقلًا.

Style و Static analysis

```bash
vendor/bin/phpstan analyse
vendor/bin/phpunit
vendor/bin/phpcs
```

PHPStan/Psalm تكشف تناقض الأنواع والمسارات المستحيلة. PHPCS أو PHP-CS-Fixer توحّد التنسيق. شغّل الأدوات محليًا وفي CI بنفس الإعداد.

Test data

- اجعل الوقت والعشوائية قابلة للتحكم.
- استخدم transaction/reset لعزل database tests.
- لا تعتمد على ترتيب الاختبارات.
- لا تستخدم بيانات production حقيقية.
- اختبر migration وrollback/forward path حيث يلزم.

Integration tests وContract

اختبر العقد عند الحدود الفعلية: status وheaders وschema لعميل HTTP، constraints وisolation لقاعدة البيانات، ودلالات ack/retry للـ queue. Fake مفيدة للسرعة، لكن لا تثبت توافق driver أو broker.

قسّم suite إلى طبقات لها أوامر ووقت واضحان:

```
unit -> contract -> integration -> HTTP smoke -> end-to-end
```

شغّل integration tests على خدمة مؤقتة وبيانات مستقلة لكل عملية. لا تجعل parallel tests تشترك في database name أو queue أو clock. استخدم unique namespace واحذف الموارد في `finally`.

Flaky tests وMutation

Mutation testing يغير operator أو شرطًا ويتوقع فشل الاختبار. إذا بقي أخضر فالتغطية وصلت للسطر دون assertion قوية. سجّل flaky test كعطل له مالك وموعد؛ إعادة CI حتى ينجح تخفي السباق.

اختبر failure paths: timeout، response تالفة، duplicate message، deadlock، امتلاء القرص، وSIGTERM. اجعل الوقت والعشوائية injectable حتى تكون النتيجة قابلة للتكرار.

مرجع

- PHPUnit Manual

مسألة تشغيلية

▼ إمتى تختار integration test؟

عندما تريد التحقق من تعاون حدود حقيقية مثل database أو HTTP، لا من منطق دالة معزولة فقط.

شغّل وتحقق

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

نفّذ نقطة التحقق التالية داخل بيئة الدرس:

```
php tests.php
```

معيار النجاح: تنجح حالات المسار الطبيعي والحدود والفشل؛ أفسد شرطًا عمدًا مرة واحدة وتأكد أن اختبارًا مناسبًا يفشل قبل إعادة الإصلاح.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

اربط النقاط ببعض

افصل unit عن integration وcontract وend-to-end حسب boundary. اجعل الوقت والعشوائية والخدمات الخارجية قابلة للضبط، وشغل الاختبارات بالتوازي دون state مشتركة. Coverage لا يثبت جودة assertions؛ mutation testing يكشف اختبارًا يمر رغم تغيير المنطق، وCI يبدأ من بيئة نظيفة.

دورة التجربة

قبل التنفيذ اكتب توقعك، ثم شغّل المثال وسجّل الخروج. أحدث فشلًا واحدًا مقصودًا، اجمع الدليل من logs أو metrics، أصلح السبب، وأعد التشغيل لإثبات أن الإصلاح يعالج العطل ولا يخفيه.

جرّب بنفسك

غيّر operator عمدًا وتأكد أن اختبارًا يفشل، ثم شغل suite بترتيب عشوائي.

11. Data Caching وRedis

Cache-aside وTTL وinvalidation وstampede وdistributed locks والفرق عن OPcache وHTTP cache.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **HTTP:** قواعد تبادل الطلبات والردود بين المتصفح والخادم.
- **Cache:** نسخة مؤقتة من البيانات هدفها تقليل وقت الانتظار والعمل المتكرر.
- **Session:** بيانات مؤقتة تساعد الخادم يميّز المستخدم بين أكثر من طلب.
- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.

أي Cache؟

- **OPcache:** PHP bytecode.
- **HTTP cache:** responses حسب HTTP semantics.
- **Application/Data cache:** نتائج queries أو حسابات.
- **Local in-process:** سريع لكنه غير مشترك وقد يختلف بين workers.

Redis أداة شائعة للبيانات المشتركة، لكنه ليس مصدر الحقيقة تلقائيًا.

Cache-aside

```
$key = "product:{$id}:v1";
$json = $redis->get($key);

if ($json === false) {
    $product = $repository->find($id);
    $json = json_encode($product, JSON_THROW_ON_ERROR);
    $redis->setex($key, 300, $json);
}

return json_decode($json, true, flags: JSON_THROW_ON_ERROR);
```

عند الكتابة حدّث database أولًا ثم احذف/حدّث cache وفق استراتيجية واضحة. توقع stale data خلال نافذة محددة.

المفاتيح وTTL

- ضع namespace/version في المفتاح.
- أضف jitter للـTTL حتى لا تنتهي آلاف المفاتيح معًا.
- لا تجعل `KEYS *` جزءًا من request path.
- حدّد serialization format وحجمه.
- لا تخزن secret لمجرد أن Redis "داخلية".

Cache stampede

عند انتهاء key مشهورة قد تعيد عدة requests بناءها معًا. حلول:

- lock قصير مع timeout.
- stale-while-revalidate.
- probabilistic early refresh.
- single-flight داخل العملية.

الـlock يجب أن تملك token فريدة وتحررها فقط إن كنت ما زلت المالك. لا تعتبر distributed lock حلًا بسيطًا لكل consistency.

الفشل

حدد هل cache **optimization** يمكن تجاوزها أم dependency أساسية مثل session store. ضع timeouts قصيرة وراقب hit ratio وevictions وmemory وlatency. لا تجعل سقوط Redis يحول كل traffic فجأة إلى database بلا load protection.

Invalidation

"هناك شيئان صعبان" ليست خطة. اكتب لكل key:

- من ينشئها؟
- ما source of truth؟
- متى تحذف أو تتغير؟
- ما أقصى stale time؟
- ماذا يحدث عند failure؟

مسألة تشغيلية

▼ إيه أصعب مشكلة في cache؟

الإبطال والاتساق: لازم تحدد المفتاح والـTTL ومتى تحذف أو تحدث النسخة بعد تغيير المصدر.

شغّل وتحقق

الناتج المحلي stale_cache=10 database=11 ثم obsolete_refill=rejected يوضح السباق. ده نموذج ترتيب أحداث؛ فحص TTL والانقطاع والحمل يحتاج Redis حقيقيًا.

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

نفّذ نقطة التحقق التالية داخل بيئة الدرس:

```
php cache-lab.php
```

هدف تجربة التكامل الموسعة: القراءة الأولى `source=db` والثانية `source=cache` ، وفي الاختبار التسلسلي بعد التحديث تقرأ القيمة الجديدة وتتأكد من TTL؛ ده لا يثبت غياب القراءات القديمة أثناء التزامن.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

تجربة Redis حقيقية

```
docker compose -f production/compose.yaml -f production/compose.full.yaml up --build -d
curl -c cookies.txt -b cookies.txt http://127.0.0.1:8080/session
docker compose -f production/compose.yaml -f production/compose.full.yaml exec redis redis-cli info memory
docker compose -f production/compose.yaml -f production/compose.full.yaml stop redis
curl -i -c cookies.txt -b cookies.txt http://127.0.0.1:8080/session
```

سجّل زمن الاستجابة وكود HTTP قبل الانقطاع وبعده. أعد Redis وتأكد أن الخدمة لا تستخدم قيمة محلية قديمة بصمت. هذه التجربة تكمل نموذج السباق المحلي ولا تستبدله.

اربط النقاط ببعض

اختر Redis structure حسب العقد: string وhash وset وsorted set وstream ليست متبادلة. افهم eviction policy والحد الأقصى للذاكرة، وحدد أثر replication lag وfailover. Cache ليست مصدر الحقيقة؛ صمم stale tolerance وinvalidations وstampede lock وfallback عند غياب Redis.

دورة التجربة

قبل التنفيذ اكتب توقعك، ثم شغّل المثال وسجّل الخروج. أحدث فشلًا واحدًا مقصودًا، اجمع الدليل من logs أو metrics، أصلح السبب، وأعد التشغيل لإثبات أن الإصلاح يعالج العطل ولا يخفيه.

جرّب بنفسك

اقطع Redis أثناء الحمل وأثبت صحة البيانات حتى لو انخفض الأداء.

إعادة ملء الـcache ممكن تتسابق مع الحذف

تتبّع الترتيب ده: القارئ A مايلقيش المفتاح ويقرأ 10 من قاعدة البيانات؛ الكاتب B يحفظ 11 ويحذف المفتاح؛ A يكمل ويحفظ 10 في cache. حذف المفتاح بعد commit إذن مش ضمان لقراءة حديثة. TTL بيحد بقاء القيمة بعد إدخالها، لكن مش بالضرورة عمرها كله لو قراءة قديمة اتأخرت قبل التخزين.

لقرارات المخزون والدفع، اقرأ وطبّق القيود في قاعدة البيانات. للعرض، حدد مدة قدم مقبولة وحدًا لعمر القارئ؛ ممكن تستخدم مفاتيح بإصدار ومصدر موثوق للإصدار الحالي، أو فحص إصدارات منسق قبل نشر القيمة. القفل اللي انتهت مهلة ملكيته محتاج حماية بإصدار كمان. تدريب: أعد ترتيب الأحداث بعميلين توقفهما مؤقتًا؛ المتوقع إن النسخة الساذجة تعرض 10، وبعد الإصلاح تثبت منع نشر الإصدار القديم.

12. Queues وWorkers وScheduling

Jobs وat-least-once delivery وidempotency وretries وbackoff وDLQ وgraceful shutdown وCron.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **HTTP:** قواعد تبادل الطلبات والردود بين المتصفح والخادم.
- **Queue:** طابور مهام تنتظر عاملًا ينفذها في الخلفية.
- **Worker:** برنامج يعمل في الخلفية ويسحب المهام من الطابور وينفذها.

لماذا Queue؟

انقل العمل البطيء أو القابل لإعادة المحاولة خارج HTTP request: البريد، الصور، التقارير، مزامنة APIs. أعد الاستجابة بعد حفظ نية العمل بصورة موثوقة، لا بعد تشغيل background process عشوائي.

```
{"type":"SendReceipt","job_id":"job_01J...","order_id":42,"attempt":1}
```

أرسل IDs لا object graphs كاملة، وضع version للـpayload.

Delivery وIdempotency

أنظمة كثيرة تقدم **at-least-once**؛ قد تصل الرسالة مرتين. صمّم handler آمنة:

```
<?php
require __DIR__ . '/bootstrap.php';

$db = Lessons\connectInventory(':memory:');
Lessons\initializeInventory($db);
$first = Lessons\purchase($db, 'job-42', 1, 1, 2);
$again = Lessons\purchase($db, 'job-42', 1, 1, 2);
echo json_encode([$first, $again], JSON_THROW_ON_ERROR), PHP_EOL;
```

يجب أن يكون تسجيل النتيجة وidempotency داخل معاملة واحدة وقيد فريد. Transactional Outbox تربط تغيير database بإنشاء الحدث.

Retry

أعد المحاولة للأخطاء المؤقتة فقط:

```
delay = min(cap, base * 2^attempt) + random_jitter
```

لا تعِد ValidationError أو credential مرفوضة بلا تغيير. بعد حد معين انقل الرسالة إلى Dead-Letter Queue مع سبب وتحقيق وتنبيه.

Worker lifecycle

- timeout لكل job وI/O.
- memory limit وإعادة تدوير العملية.
- SIGTERM يوقف استقبال الجديد وينهي الحالي ضمن grace period.
- acknowledge بعد النجاح لا قبله.
- heartbeat وvisibility timeout أطول من job مع renewal عند الحاجة.

راقب queue depth وage of oldest وprocessing latency وretry/DLQ rate.

Scheduler وCron

اجعل المهمة المجدولة idempotent، امنع overlap بقفل له expiry، واستخدم timezone واضحة. الـscheduler يمكن أن ينشر jobs إلى queue بدل تنفيذ كل العمل في عملية واحدة.

مسألة تشغيلية

▼ ليه job لازم تكون idempotent؟

لأن التسليم غالبًا at-least-once وقد يعيد worker المحاولة؛ تكرار الرسالة لا يجب أن يكرر الأثر التجاري.

شغّل وتحقق

المثال يطبع طلبًا جديدًا ثم نفس الطلب مع duplicate=true. محاولات queue وdead-letter تحتاج وسيط رسائل وعاملًا في بيئة اختبار، وليست جزءًا من هذا البرنامج.

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

نفّذ نقطة التحقق التالية داخل بيئة الدرس:

```
php queue-demo.php
```

هدف تجربة التكامل الموسعة: تُعالج الرسالة الناجحة مرة، والفشل المؤقت يُعاد بحد أقصى، والرسالة الدائمة تصل إلى dead-letter مع correlation ID.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

Worker حقيقي على Redis

```
docker compose -f production/compose.yaml -f production/compose.full.yaml up --build -d
curl -fsS -X POST http://127.0.0.1:8080/queue
docker compose -f production/compose.yaml -f production/compose.full.yaml logs queue-worker
docker compose -f production/compose.yaml -f production/compose.full.yaml exec redis redis-cli llen lesson:queue
```

يجب أن يظهر `processed=<job_id>` ويعود طول القائمة إلى صفر. أوقف worker، أرسل job وتأكد أنها تبقى، ثم أعده وأثبت المعالجة. أرسل نفس `job_id` يدويًا مرتين واختبر سجل duplicate.

اربط النقاط ببعض

Visibility timeout يجب أن يتجاوز المعالجة أو يتجدد حتى لا تظهر الرسالة لعامل ثانٍ. Ordering غالبًا مضمون داخل partition فقط. Poison message تحتاج retry محدودًا وdead-letter وتحقيقًا. Transactional outbox يربط تغيير قاعدة البيانات بنشر الحدث دون dual-write gap.

دورة التجربة

قبل التنفيذ اكتب توقعك، ثم شغّل المثال وسجّل الخروج. أحدث فشلًا واحدًا مقصودًا، اجمع الدليل من logs أو metrics، أصلح السبب، وأعد التشغيل لإثبات أن الإصلاح يعالج العطل ولا يخفيه.

جرّب بنفسك

حاكِ crash بعد commit وقبل ack وتأكد أن idempotency تمنع الأثر المكرر.

ليه إثبات التنفيذ داخل المعاملة؟

احفظ المثال باسم `queue-demo.php` في مجلد المختبر. الاستدعاء الأول ينشئ الطلب 1، والثاني يرجع نفس الطلب مع `duplicate:true`. افتح `src/Inventory.php`: إثبات التنفيذ بقيد فريد `(user_id, job_key)` وتغيير المخزون والطلب وحدث outbox بيتحفظوا في معاملة واحدة. نفس المفتاح بمدخلات مختلفة يُرفض. الفشل قبل commit يلغي الأربعة، وما تبعتش إقرار نجاح للـqueue إلا بعد commit. فحص «اتنفذ قبل كده؟» ثم تنفيذ الأثر منفصلين خطر: عاملان ممكن يعدّوا الفحص معًا.

الـoutbox جدول بيحفظ أحداث هتتبعت بعد commit. الناشر ممكن يرسل ثم يتعطل قبل تسجيل الإرسال، فالتسليم لسه ممكن يتكرر. الدفع أو البريد خارج قاعدة البيانات محتاج مفتاح منع تكرار عند المزود أو إزالة التكرار عند المستقبِل؛ معاملة قاعدة البيانات مش هتلغي طلبًا لخدمة خارجية. شغّل `php tests.php inventory` لاختبار التكرار والفشل المتعمد. الاختبارات المحلية دي مش محاكاة لوسيط رسائل حقيقي أو عدة عمليات workers.

13. Deployment وCI/CD وContainers

Build artifacts وpipeline وhealth checks وmigrations وzero-downtime وrollback وإعداد containers.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Runtime:** وقت التشغيل: الفترة اللي البرنامج بيكون شغال فيها فعلًا.
- **Queue:** طابور مهام تنتظر عاملًا ينفذها في الخلفية.
- **Worker:** برنامج يعمل في الخلفية ويسحب المهام من الطابور وينفذها.

Artifact قابلة للتكرار

ابنِ نسخة واحدة ثم رقّها بين البيئات بدل `composer install` مختلف على كل خادم:

```
composer install --no-dev --prefer-dist --no-interaction \
  --optimize-autoloader --classmap-authoritative
composer check-platform-reqs --lock --no-dev
```

احفظ commit SHA وbuild ID داخل metadata قابلة للمراقبة، ولا تضع secrets داخل image.

Pipeline

```
lint -> unit tests -> static analysis -> integration tests
-> dependency audit -> build artifact/image -> scan
-> deploy staging -> smoke test -> production -> verify
```

أوقف النشر عند فشل check. لا تجعل approval يدويًا بديلًا عن اختبارات قابلة للتكرار.

Pipeline قابلة للمراجعة

احتفظ بالـpipeline في المستودع، وثبّت الأدوات والصور، واربط كل artifact بـcommit. مثال المراحل ليس رسمًا فقط؛ كل مرحلة تنتج دليلًا محفوظًا: image digest، scan result، test report، وdeployment record.

```
jobs:
  verify:
    steps:
      - run: composer install --no-interaction --no-progress
      - run: composer validate --strict
      - run: composer audit --locked
      - run: composer test
      - run: docker build --pull --tag app:$GIT_SHA .
```

لا تستخدم secret في build argument أو layer. افصل build عن release، وروّج digest نفسها من staging إلى production.

Container

- image صغيرة بإصدار PHP وextensions مثبت.
- process رئيسية واضحة وnon-root user.
- filesystem read-only حيث يمكن.
- config/secrets وقت التشغيل.
- logs إلى stdout/stderr.
- limits وgraceful shutdown.

افصل web وworker لأن لهما lifecycle وتوسّعًا مختلفين.

حدود النظام واختبار الحمل

اضبط CPU وmemory وprocess/file-descriptor limits، ثم راقب throttling وOOM والـlimit. يجعل بلا request listen queue scheduling عشوائيًا، وmemory limit للحاوية يجب أن يتسع لـFPM workers وOPcache وnative allocations.

اختبار الحمل يثبت نقطة saturation، لا رقمًا تسويقيًا. ارفع الحمل تدريجيًا، وسجل throughput وp95 والأخطاء وRSS واتصالات قاعدة البيانات. أوقف التجربة عندما تكسر SLO، ثم غيّر متغيرًا واحدًا وأعدها.

Health

- **Liveness:** هل العملية عالقة وتحتاج restart؟
- **Readiness:** هل تستطيع استقبال traffic الآن؟
- **Startup:** هل تحتاج وقت warm-up؟

لا تجعل liveness تعتمد على كل external service فتسبب restart storm. يمكن أن تمنع readiness traffic عند dependency أساسية.

Migrations وZero-downtime

استخدم rolling/blue-green حسب المنصة. اجعل schema changes backward-compatible بنمط Expand/Contract:

1. أضف البنية الجديدة.
2. انشر كودًا يدعم القديم والجديد.
3. نفّذ backfill.
4. انقل القراءة.
5. احذف القديم لاحقًا.

أعد تحميل FPM gracefully، ونسق queue workers مع نسخة payload.

Rollback

Rollback ليست فقط إعادة image؛ migration قد تكون غير قابلة للعكس. جهز feature flags، backups، roll-forward، ومقاييس نجاح تلقائية. اختبر الإجراء قبل incident.

مسألة تشغيلية

▼ ليه image ناجحة مش ضمان نشر ناجح؟

النشر يحتاج config وsecrets وmigrations وhealth checks وrollback متوافقة مع البيئة الفعلية.

شغّل وتحقق

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

من داخل `examples/php-labs` أو الحزمة المستخرجة نفّذ:

```
docker compose -f production/compose.yaml config --quiet
docker compose -f production/compose.yaml up --build -d
curl -fsS http://127.0.0.1:8080/health
php load-probe.php http://127.0.0.1:8080/health 50 10
docker compose -f production/compose.yaml down
```

معيار النجاح: تنجح بنية Compose، وتبدأ الخدمات بحسابات غير root وfilesystems للقراءة فقط، ولا يستقبل Nginx traffic قبل صحة FPM. يرجع health JSON بحالة ok، ويظل ملف HTML الساكن خارج PHP. أمر `down` ينظف موارد التجربة.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

اربط النقاط ببعض

ابن image بطبقات قابلة للتكرار وminimal runtime، وشغلها كمستخدم غير root. أنشئ SBOM ووقّع artifact واربط deployment بالdigest. فرّق readiness عن liveness، واستخدم canary أو blue/green مع rollback، وانشر migrations بتوافق أمامي وخلفي.

دورة التجربة

قبل التنفيذ اكتب توقعك، ثم شغّل المثال وسجّل الخروج. أحدث فشلًا واحدًا مقصودًا، اجمع الدليل من logs أو metrics، أصلح السبب، وأعد التشغيل لإثبات أن الإصلاح يعالج العطل ولا يخفيه.

جرّب بنفسك

انشر نسخة canary تفشل readiness وأثبت أنها لا تستقبل traffic.

14. أمان الاعتماديات وSupply Chain

Composer audit والسياسات وplatform checks وlock files وpluginsوscripts والتحقق في CI.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.
- **CLI:** واجهة تتعامل معها بكتابة أوامر نصية بدل الضغط على أزرار.

composer.lock عقد البناء

ارفع `composer.lock` للتطبيقات، واستخدم `composer install` في CI والإنتاج. `update` يقرر نسخًا جديدة ويجب أن يحدث في تغيير مراجع ومختبر، لا أثناء النشر.

Audit وسياسة الحزم

```
composer validate --strict
composer audit --locked
composer check-platform-reqs --lock --no-dev
composer outdated --direct
```

`composer audit` يفحص advisories والحزم المهجورة والسياسات المدعومة. لا تتجاهل advisory بلا سبب موثق ومدة ومراجعة للـreachability.

Scriptsو Plugins

Composer plugins وscripts تنفذ code بصلاحية المستخدم الذي يشغّل Composer:

```
{
  "config": {
    "allow-plugins": {
      "trusted/package": true,
      "*": false
    }
  }
}
```

لا تشغّل Composer كـroot على حزم غير موثوقة. `--no-plugins --no-scripts` يقلل التنفيذ لكنه قد يمنع build مطلوبة؛ افهم المشروع.

Version policy

- ضع SemVer constraints مقصودة.
- اختبر أقل وأعلى نسخة مدعومة عندما تكون مكتبة.
- راجع transitive dependencies لا المباشرة فقط.
- حدّث دوريًا بدل قفزة سنوية ضخمة.
- احذف الحزم غير المستخدمة.

CI

احمِ tokens، ثبّت permissions للـworkflow، لا تطبع environment، وثبّت actions/images حيث يمكن. أنشئ SBOM إذا تتطلب البيئة، واحتفظ بسجل artifact ومن بناها.

SBOM وProvenance والتوقيع

أنشئ SBOM من artifact النهائية، لا من `composer.json` وحده، لأن الصورة تضم نظام تشغيل وextensions وملفات أخرى. اربط provenance بالـcommit والـbuilder والـworkflow والـdigest. التوقيع يثبت مصدر artifact وسلامتها ولا يثبت خلوها من الثغرات.

سياسة القبول يجب أن تجيب: من يوافق على exception؟ ما مدته؟ هل الحزمة المتأثرة reachable؟ وأي بيئة تحتوي digest المصابة؟ اختبر البحث من CVE إلى lock ثم image ثم deployment.

CI والـSecrets

قلّل permissions الافتراضية للـworkflow، ثبّت third-party actions إلى commit موثوق، وامنع pull requests غير الموثوقة من الوصول إلى secrets. نفّذ secret scanning على التاريخ، ودوّر السر إذا ظهر؛ حذفه من آخر commit لا يلغيه.

```
source commit -> locked dependencies -> SBOM -> signed image digest -> deployment record
```

استجابة لثغرة

حدد هل الكود المتأثر reachable، طبّق update أو mitigation، شغّل suite، انشر، وراقب. إذا تسرب secret عبر package/script فدوّره؛ إزالة الحزمة لا تبطل السر.

مراجع

- Composer CLI: audit
- Composer plugins and scripts safety

مسألة تشغيلية

▼ ماذا تفعل عند advisory في dependency؟

حدد هل النسخة والمسار المتأثران مستخدمان، حدّث واختبر، وطبّق mitigation مؤقتًا بدل تجاهل أو ترقية عمياء.

شغّل وتحقق

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

نفّذ نقطة التحقق التالية داخل `examples/php-labs` أو النسخة المستخرجة من الحزمة:

```
composer audit --locked
composer check-platform-reqs --lock --no-dev
```

معيار النجاح: لا توجد ثغرة معروفة غير مقبولة في الـlock الحالي؛ أي استثناء موثق بمالك وسبب وموعد انتهاء.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

اربط النقاط ببعض

أضف SBOM وprovenance وتوقيع artifact، وافحص typosquatting وmaintainer change لا CVE فقط. Secret scanning يمنع token في source/history. استخدم VEX أو توثيقًا مماثلًا لبيان قابلية الاستغلال، وكل استثناء له مالك وسبب وموعد انتهاء.

دورة التجربة

قبل التنفيذ اكتب توقعك، ثم شغّل المثال وسجّل الخروج. أحدث فشلًا واحدًا مقصودًا، اجمع الدليل من logs أو metrics، أصلح السبب، وأعد التشغيل لإثبات أن الإصلاح يعالج العطل ولا يخفيه.

جرّب بنفسك

تتبع package من commit إلى artifact وdeployment واثبت مصدرها.

15. Profiling وSLI/SLO وOpenTelemetry

تحليل الأداء وFlame Graphs وبناء مؤشرات الخدمة وانتقال Trace Context بين الخدمات.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **HTTP:** قواعد تبادل الطلبات والردود بين المتصفح والخادم.
- **API:** واجهة محددة تسمح لبرنامج يطلب بيانات أو ينفّذ عملية عند برنامج آخر.
- **Queue:** طابور مهام تنتظر عاملًا ينفذها في الخلفية.

القياس قبل التحسين

ابدأ بأثر قابل للقياس: Endpoint بطيء، استهلاك CPU مرتفع، Memory Growth أو نسبة أخطاء زادت. لا تستنتج السبب من المتوسط وحده؛ افحص `p50` و`p95` و`p99` وحجم الطلب ومعدل المرور. التحسين بلا Baseline قد ينقل الكلفة إلى قاعدة البيانات أو الذاكرة بدل حلها.

Sampling Profiler وInstrumentation

Sampling Profiler يأخذ عينات دورية من Call Stack بتكلفة أقل ويكشف أين يقضي البرنامج وقته. Instrumentation تسجل مدة عمليات محددة بدقة أكبر لكنها تضيف كودًا وكلفة. استخدم بيئة تشبه الإنتاج، Warm-up مناسبًا، وDataset واقعية.

```
$started = hrtime(true);
try {
    $result = $service->buildReport($request);
} finally {
    $durationMs = (hrtime(true) - $started) / 1_000_000;
    $logger->info('report.completed', [
        'duration_ms' => $durationMs,
        'memory_peak_bytes' => memory_get_peak_usage(true),
    ]);
}
```

هذا قياس لعملية واحدة، وليس Profiler كاملًا. لا تضف Timer يدويًا حول كل دالة؛ استخدم Profiler وAPM وInstrumentation عند الحدود المهمة.

قراءة Flame Graph

عرض الإطار يعبّر عن نسبة العينات، لا عن زمن استدعاء واحد. المحور الرأسي يعرض سلسلة الاستدعاءات. ابحث عن Stack عريض متكرر، ثم اسأل هل السبب Query، Autoloading، Regex، Serialization، CPU متكررة أم انتظار I/O لا يظهر جيدًا في CPU Profile. قارن قبل وبعد بنفس الحمل.

SLI وSLO وError Budget

- **SLI:** قياس فعلي مثل نسبة الطلبات الناجحة تحت 300ms.
- **SLO:** هدف مثل 99.9% من الطلبات الناجحة خلال 30 يومًا.
- **Error Budget:** الجزء المسموح بعدم تحقيق الهدف؛ عند استهلاكه نبطئ التغييرات ونركز على الاعتمادية.

إذا كان SLO هو 99.9% فميزانية الفشل 0.1%، لكنها لا تعني تجاهل تأثير فشل نادر لكنه كارثي. عرّف ما الطلب الصالح وما النجاح، واستبعد Health Checks وحركة الاختبار من الحساب إن كان ذلك موثقًا.

OpenTelemetry وTrace Context

OpenTelemetry يجمع Traces وMetrics وLogs وفق Context مشترك. يبدأ الطلب بـSpan، وتصبح عمليات قاعدة البيانات وHTTP أبناء له. عند استدعاء خدمة أخرى ينتقل Context عادة عبر `traceparent` W3C؛ لا تنشئ Trace جديدة في كل خدمة.

```
Browser -> API span
          -> DB span
          -> HTTP client span -- traceparent --> Payment service span
```

استخدم Auto-instrumentation أو مكتبة معروفة أولًا. إذا حقنت Headers يدويًا، لا تثق بـBaggage قادم من العميل ولا تضع فيه أسرارًا أو بيانات شخصية؛ ينتقل عبر حدود الخدمة وقد يظهر في Telemetry.

تحليل طلب بطيء

1. ابدأ من Trace بطيئة وحدد أطول Span.
2. اربطها بـLogs باستخدام Trace ID.
3. افحص Query Plan أو خدمة الطرف الثالث أو Queue time.
4. استخدم Profile إذا كان الوقت داخل CPU التطبيق.
5. أصلح السبب واختبر Load وRegression.
6. راقب SLI بعد النشر، لا زمن المثال المحلي فقط.

SDK وOTLP عمليًا

المختبر يعتمد `open-telemetry/sdk` و`open-telemetry/exporter-otlp`. يحمّل التطبيق الإعداد من environment ويرسل OTLP/HTTP إلى Collector، ثم يطبع Collector الـspan عبر debug exporter.

```
docker compose -f production/compose.yaml -f production/compose.full.yaml up --build -d
curl -fsS http://127.0.0.1:8080/telemetry
docker compose -f production/compose.yaml -f production/compose.full.yaml logs otel-collector
```

تحقق أن response تحتوي Trace ID غير صفري وأن logs تعرض `lesson.telemetry` و `service.name=php-runtime-lab`. أضف attributes منخفضة cardinality مثل route وmethod وstatus. لا تستخدم user ID أو URL كامل كـmetric label.

Batch processor مناسب للإنتاج، لكن العملية قصيرة العمر تحتاج flush قبل الخروج. ضع sampling policy واعية بالأخطاء والـlatency، وراقب dropped spans وexport failures؛ نجاح request لا يعني نجاح تصدير telemetry.

تأكد من فهمك

01 **لماذا المتوسط لا يكفي لزمن الاستجابة؟**

> **الإجابة:** قد يخفي Tail بطيئًا يؤثر في عدد مهم من المستخدمين؛ راقب Percentiles والحجم والأخطاء معًا.

02 **ماذا يعني عرض إطار كبير في Flame Graph؟**

> **الإجابة:** ظهر هذا المسار في نسبة كبيرة من العينات؛ يحتاج تحقيقًا لكنه لا يثبت وحده أن الدالة بطيئة في كل استدعاء.

03 **ما الفرق بين SLI وSLO؟**

> **الإجابة:** SLI هو القياس، وSLO هو الهدف المحدد لهذا القياس خلال نافذة زمنية.

04 **كيف تحافظ على Trace واحدة بين خدمتين؟**

> **الإجابة:** حقن Trace Context في الطلب الصادر واستخراجه في الخدمة المستقبلة، ويفضل عبر Instrumentation موثوقة.

شغّل وتحقق

العينة الثابتة تنتج p50=10ms وp95=100ms وp99=100ms؛ المتوسط 28ms. القياس الفعلي وتصدير trace تدريب منفصل على الخدمة.

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

نفّذ نقطة التحقق التالية داخل بيئة الدرس:

```
php profiling-lab.php
```

هدف تجربة التكامل الموسعة: قارن p50 وp95 وp99 قبل التغيير وبعده، واربط trace بعملية بطيئة؛ المتوسط وحده لا يحقق شرط القبول.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

اربط النقاط ببعض

أضف instrumentation حقيقيًا لـHTTP وdatabase وqueue وحدد exporter وHistogram. resource attributes يحتاج buckets مناسبة وتفسير quantiles من backend؛ لا تجمع percentiles حسابيًا. اربط تحسين flame graph بتغير SLI لا بمجرد انخفاض دالة واحدة.

دورة التجربة

قبل التنفيذ اكتب توقعك، ثم شغّل المثال وسجّل الخروج. أحدث فشلًا واحدًا مقصودًا، اجمع الدليل من logs أو metrics، أصلح السبب، وأعد التشغيل لإثبات أن الإصلاح يعالج العطل ولا يخفيه.

جرّب بنفسك

صدّر trace وmetric لخدمة تجريبية واربط span بارتفاع p95.

16. المرونة وإدارة Workers

Circuit Breaker وBulkhead وBackpressure وإشارات العمليات وإدارة Workers عبر Supervisor وsystemd.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.
- **Queue:** طابور مهام تنتظر عاملًا ينفذها في الخلفية.
- **Worker:** برنامج يعمل في الخلفية ويسحب المهام من الطابور وينفذها.
- **CLI:** واجهة تتعامل معها بكتابة أوامر نصية بدل الضغط على أزرار.
- **Loop:** حلقة تكرار تعيد تنفيذ مجموعة تعليمات وفق شرط.
- **Function:** دالة: جزء كود له اسم ومهمة محددة ويمكن استدعاؤه أكثر من مرة.

المهلة أول دفاع

كل اتصال خارجي يحتاج Connect Timeout وTotal Timeout. من دونها قد تتراكم PHP Workers وهي تنتظر خدمة بطيئة حتى ينفد الـPool. Retry ليست بديلًا عن المهلة؛ استخدمها فقط لخطأ مؤقت وعملية آمنة أو Idempotent، مع Backoff وJitter وحد أقصى.

Circuit Breaker

يتابع الفشل في نافذة زمنية:

- **Closed:** الطلبات تمر ويُسجل النجاح والفشل.
- **Open:** يفشل سريعًا بدل الضغط على خدمة متعطلة.
- **Half-open:** يسمح بعدد محدود من الطلبات لاختبار التعافي.

لا تفتح الدائرة بسبب خطأ مجال مثل `422`. صنف Timeouts و`5xx` وأخطاء الاتصال، وحدد سياسة منفصلة لكل Dependency. شارك الحالة عبر مخزن مناسب إذا كان القرار يجب أن يشمل عدة Workers، أو اقبل أن لكل Process رؤية محلية موثقة.

Backpressure وBulkhead

Bulkhead يعزل الموارد: Pool منفصل أو حد تزامن لخدمة بطيئة حتى لا تستهلك كل Workers. Backpressure تجعل المنتج يبطئ أو يرفض عملًا جديدًا عندما لا يستطيع المستهلك اللحاق به. الخيارات تشمل Queue bounded، `429` / `503` مع `Retry-After`، أو تقليل Concurrency.

Queue غير محدودة لا تحل الحمل؛ تؤجل الانهيار وتستهلك الذاكرة. راقب Queue Depth وOldest Message Age وProcessing Rate، وليس طولها وحده.

Worker lifecycle والإشارات

عامل CLI طويل العمر يجب أن يلتقط `SIGTERM` ليتوقف بأمان بعد المهمة الحالية، ويستخدم `SIGINT` للتشغيل المحلي. يحتاج امتداد `pcntl` في بيئات Unix-like؛ Windows يتطلب آلية إدارة مختلفة.

```
<?php

declare(strict_types=1);

$running = true;
pcntl_async_signals(true);
pcntl_signal(SIGTERM, static function () use (&$running): void {
    $running = false;
});

while ($running) {
    $job = reserveJob(timeoutSeconds: 5);
    if ($job === null) {
        continue;
    }
    try {
        handle($job);
        acknowledge($job);
    } catch (Throwable $error) {
        releaseOrDeadLetter($job, $error);
    }
}
```

ضع حدًا لعدد Jobs أو الذاكرة ثم أعد تشغيل Worker دوريًا، وأغلق الاتصالات وحرر Locks عند الخروج.

systemd وSupervisor

مدير العمليات يبدأ Worker بعد الإقلاع، يعيده بعد Crash، يمرر Environment صحيحة، ويجمع stdout/stderr. لا تجعل Restart Loop سريعًا يخفي خطأ دائمًا؛ أضف Backoff وحدًا للمحاولات وتنبيهًا.

```
[Service]
ExecStart=/usr/bin/php /srv/app/bin/worker.php
WorkingDirectory=/srv/app
Restart=on-failure
RestartSec=5
TimeoutStopSec=30
KillSignal=SIGTERM
```

Health لا تعني أن Process موجودة فقط: راقب قدرتها على سحب Jobs، آخر نجاح، عمر أقدم رسالة، ومعدل الفشل.

وحدة systemd مكتملة

```
[Unit]
Description=PHP queue worker
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=app
Group=app
WorkingDirectory=/srv/app/current
EnvironmentFile=/etc/app/worker.env
ExecStart=/usr/bin/php bin/worker.php
Restart=on-failure
RestartSec=5s
TimeoutStopSec=30s
KillSignal=SIGTERM
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ProtectHome=true
ReadWritePaths=/srv/app/shared
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

اختبر `systemd-analyze verify`، ثم stop أثناء job طويلة وتأكد أن العامل يصرف العمل ضمن `TimeoutStopSec`. لا تضع secret في unit أو command line؛ اقرأها من ملف محدود الصلاحية أو secret manager.

تأكد من فهمك

01 **لماذا Retry بلا Timeout يزيد العطل؟**

> **الإجابة:** كل محاولة قد تنتظر بلا حد، فتتراكم Workers ويزداد الضغط على الخدمة المتعثرة.

02 **ما وظيفة Half-open؟**

الإجابة: اختبار التعافي بعدد محدود من الطلبات دون إعادة الحمل كاملًا فورًا.

03 **كيف يختلف Bulkhead عن Circuit Breaker؟**

الإجابة: Bulkhead يعزل السعة، أما Circuit Breaker فيوقف الاستدعاء مؤقتًا عند نمط فشل.

04 **ماذا يجب أن يفعل Worker عند SIGTERM؟**

الإجابة: يتوقف عن حجز عمل جديد، ينهي المهمة الحالية وفق المهلة، يغلق الموارد، ثم يخرج بكود واضح.

شغّل وتحقق

الملف يشغّل اختبارات تكرار الأثر وrollback. اختبار circuit breaker يحتاج تنفيذ حالات closed/open/half-open وساعة قابلة للتحكم؛ ليس ضمن ادعاء نجاح الاختبار المحلي.

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

نفّذ نقطة التحقق التالية داخل بيئة الدرس:

```
php resilience-lab.php
```

هدف تجربة التكامل الموسعة: طلبان بالمفتاح idempotency نفسه ينتجان أثر عمل واحدًا، ويفتح circuit breaker بعد الحد ثم يختبر التعافي تدريجيًا.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

اربط النقاط ببعض

Retry يحتاج exponential backoff مع jitter وretry budget حتى لا يصنع عاصفة. Shutdown يوقف جلب jobs جديدة ثم يمنح الحالية deadline ويحرر lease. Distributed worker يحتاج ownership token أو fencing لمنع عامل قديم من الكتابة بعد فقد القفل، وbackpressure يجب أن تصل إلى المنتج.

دورة التجربة

قبل التنفيذ اكتب توقعك، ثم شغّل المثال وسجّل الخروج. أحدث فشلًا واحدًا مقصودًا، اجمع الدليل من logs أو metrics، أصلح السبب، وأعد التشغيل لإثبات أن الإصلاح يعالج العطل ولا يخفيه.

جرّب بنفسك

حاكِ عاملًا يفقد lease ثم يعود، وتأكد أن fencing يمنع الكتابة القديمة.

17. Disaster Recovery واختبار الاستعادة

RPO وRTO والنسخ الاحتياطي والاسترجاع وRunbooks وتمارين التعافي من الكوارث.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **DNS:** خدمة بتحوّل اسم الموقع إلى عنوان رقمي يقدر الجهاز يوصل له.
- **Worker:** برنامج يعمل في الخلفية ويسحب المهام من الطابور وينفذها.

Backup ليست خطة تعافٍ

النسخة الاحتياطية ملف؛ خطة التعافي تحدد ماذا نستعيد وبأي ترتيب ومن يقرر وكيف نثبت صحة النظام. نسخة لم تُختبر استعادتها مجرد أمل. احمِ قواعد البيانات والملفات المرفوعة والإعدادات الحرجة ومفاتيح التشفير وفق سياسة منفصلة.

RPO وRTO

- **RPO:** أكبر فقد بيانات مقبول زمنيًا. RPO ساعة يعني احتمال فقد آخر ساعة.
- **RTO:** أقصى وقت مقبول لإعادة الخدمة.

القيمتان قرارا عمل يحددان تكرار النسخ، Replication، الأتمتة، والكلفة. Replication لا تستبدل Backup؛ حذف أو فساد منطقي قد ينتقل إلى النسخة التابعة.

سياسة النسخ

استخدم نسخًا مشفرة، صلاحيات منفصلة، Retention واضحة، ونسخة خارج الحساب أو المنطقة الأساسية. تحقق من Checksums، راقب نجاح Jobs، واختبر أن مفاتيح فك التشفير متاحة أثناء الكارثة دون تخزينها بجوار النسخة نفسها.

ترتيب الاستعادة

1. أعلن Incident وحدد القائد وقناة الاتصال.
2. أوقف الكتابة أو اعزل النظام إن كان استمرارها يزيد الفساد.
3. اختر Recovery Point موثقًا.

4. أنشئ بيئة نظيفة واستعد قاعدة البيانات والملفات والإعدادات.
5. شغّل Migrations المتوافقة فقط.
6. تحقق من Counts وConstraints وChecksums وسيناريوهات العمل.
7. أعد المرور تدريجيًا وراقب الأخطاء والـLatency.
8. وثّق الفجوة بين RPO/RTO الفعلي والهدف.

تمرين Restore فعلي

نفذ الاستعادة دوريًا إلى بيئة معزولة. استخدم Credentials غير إنتاجية، امنع إرسال Emails/Webhooks، ثم شغّل اختبارات Smoke وقراءات اتساق. سجّل زمن كل خطوة والعوائق اليدوية. Game Day جيد يجب أن يكشف نقص Runbook أو صلاحية أو مفتاح قبل الكارثة الحقيقية.

تطبيق PHP أثناء التعافي

اجعل التطبيق يقبل Read-only Mode عند تعذر الكتابة، ويعرض رسالة واضحة بدل إعادة المحاولة بلا حد. لا تشغّل Worker قديمًا على Schema أحدث. احتفظ بـArtifact قابل لإعادة النشر، وConfig versioned بلا أسرار، وقائمة توافق بين إصدار التطبيق وSchema.

تأكد من فهمك

01 **لماذا Replication ليست Backup؟**

الإجابة: الحذف والفساد المنطقي وبيانات Ransomware قد تتكرر فورًا إلى النسخة التابعة.

02 **ما الفرق بين RPO وRTO؟**

الإجابة: RPO يحدد فقد البيانات المقبول، وRTO يحدد زمن عودة الخدمة.

03 **كيف تثبت أن Restore نجح؟**

الإجابة: بفحوص اتساق وConstraints وCounts وChecksums وSmoke Tests وسيناريوهات عمل، لا بمجرد بدء قاعدة البيانات.

04 **ما هدف Game Day؟**

الإجابة: قياس الخطة فعليًا واكتشاف الاعتماديات والصلاحيات والخطوات اليدوية الناقصة قبل حادث حقيقي.

شغّل وتحقق

الملف ينشئ لقطة SQLite تجريبية ويطبع snapshot=ok وزمن التحقق. هذا الزمن ليس RTO للإنتاج ولا يقيس RPO؛ قِس الفقد وزمن استعادة الخدمة كاملة في تدريب التعافي.

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

نفّذ نقطة التحقق التالية داخل بيئة الدرس:

```
php restore-verify.php
```

هدف تجربة التكامل الموسعة: تُستعاد نسخة معروفة إلى بيئة معزولة، تنجح فحوص السلامة، ويُقاس RPO وRTO الفعليان بدل كتابة هدفين نظريين.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

اربط النقاط ببعض

أضف point-in-time recovery لسجل المعاملات واختبر الحد الذي يمكن العودة إليه. Backup مشفر يحتاج خطة استعادة للمفاتيح لا تخزن في المكان نفسه. رتب استعادة DNS وidentity وsecrets وqueues والطرف الثالث، ونفذ game day يتضمن اتصالات القرار وتوثيق الزمن الفعلي.

دورة التجربة

قبل التنفيذ اكتب توقعك، ثم شغّل المثال وسجّل الخروج. أحدث فشلًا واحدًا مقصودًا، اجمع الدليل من logs أو metrics، أصلح السبب، وأعد التشغيل لإثبات أن الإصلاح يعالج العطل ولا يخفيه.

جرّب بنفسك

نفذ restore بلا اعتماد على الحساب الإنتاجي الرئيسي وقس RPO/RTO.

18. Sessions وShared State عند التوسع

تخزين الجلسات وقفلها وتوزيعها بين النسخ وإدارة TTL وGC والفشل.

قبل ما تبدأ

الجلسة تربط طلبات المستخدم ببعضها. في خادم واحد قد يبدو التخزين في الملفات كافيًا، لكن التوسع إلى أكثر من نسخة يحول مكان التخزين والقفل ومدة الصلاحية إلى قرارات تشغيلية.

كلمات جديدة

- **Session handler:** المكوّن الذي يقرأ بيانات الجلسة ويكتبها.
- **Horizontal scaling:** تشغيل أكثر من نسخة من التطبيق.
- **Sticky session:** توجيه المستخدم إلى النسخة نفسها اعتمادًا على Cookie أو عنوان.
- **Lock contention:** انتظار طلب لأن طلبًا آخر يمسك القفل نفسه.

لماذا تفشل file sessions مع أكثر من نسخة؟

المعالج الافتراضي `files` يحفظ بيانات الجلسة على القرص المحلي. إذا وصل الطلب التالي إلى نسخة أخرى فلن تجد الملف، إلا إذا استخدمت قرصًا مشتركًا أو توجيهًا لاصقًا. القرص المشترك يضيف latency ونقطة فشل، والتوجيه اللاصق يقلل حرية إعادة التوزيع ولا يحل فقد النسخة.

المخزن المشترك مثل Redis يجعل كل النسخ ترى الجلسة نفسها. لكنه يصبح dependency أساسية: سقوطه قد يمنع تسجيل الدخول والشراء، لذلك يحتاج timeout قصيرًا ومراقبة وخطة فشل واضحة.

القفل والطلبات المتوازية

المعالج الملفي يقفل جلسة المستخدم من `session_start()` حتى نهاية الطلب أو `session_write_close()`. طلب AJAX بطيء يمكن أن يحجز طلبات المستخدم التالية رغم وجود عمال FPM متاحين.

```
session_start();
$userId = $_SESSION['user_id'] ?? null;
session_write_close();

$report = buildSlowReport($userId);
```

أغلق الجلسة بعد آخر تعديل مطلوب، ولا تعاود فتحها بلا حاجة. افحص دلالات القفل في Redis handler المستخدم؛ تعطيل القفل بالكامل قد يسبب lost updates.

إعداد آمن

```
session.use_strict_mode=1
session.use_only_cookies=1
session.cookie_httponly=1
session.cookie_secure=1
session.cookie_samesite=Lax
session.gc_maxlifetime=1800
```

اجعل TTL في المخزن متوافقًا مع مدة Cookie وسياسة تسجيل الخروج. لا تعتمد على probabilistic GC داخل كل request لتنظيف مخزن مركزي؛ استخدم expiry أصلية للمخزن. غيّر Session ID بعد تسجيل الدخول أو تغيير الصلاحية، ولا تخزن object graph يعتمد على نسخة كود محددة.

النشر والتوافق

أثناء rolling deployment قد تقرأ النسخة الجديدة جلسة كتبتها القديمة. استخدم schema بسيطة ذات version، واقبل الإصدارين خلال الانتقال. اختبر logout، انتهاء الصلاحية، وتغيير المفتاح، ولا تجعل deploy يطرد الجميع دون قرار.

تجربة المختبر

شغّل البيئة الكاملة:

```
docker compose -f production/compose.yaml -f production/compose.full.yaml up --build -d
curl -c cookies.txt -b cookies.txt http://127.0.0.1:8080/session
curl -c cookies.txt -b cookies.txt http://127.0.0.1:8080/session
docker compose -f production/compose.yaml -f production/compose.full.yaml exec redis redis-cli -n 1 scan 0
```

يجب أن ترتفع `visits` من 1 إلى 2 وأن يظهر مفتاح الجلسة في Redis. نفّذ طلبين متوازيين بعد إضافة تأخير مؤقت، ثم أغلق الجلسة مبكرًا وقارن زمن الانتظار.

تأكد من فهمك

▼ لماذا لا تكفي file sessions مع نسختين؟

لأن الملف محلي للنسخة، وقد يصل الطلب التالي إلى نسخة لا تملكه.

▼ متى تستدعي session_write_close؟

بعد آخر قراءة أو كتابة للجلسة وقبل العمل البطيء الذي لا يحتاجها.

▼ هل Sticky Sessions بديل كامل للمخزن المشترك؟

لا؛ تقلل المرونة ولا تحمي من فقد النسخة أو إعادة الجدولة.

▼ ما الذي يجب مراقبته؟

زمن قراءة وكتابة الجلسة، الأخطاء، القفل، عدد المفاتيح، expirations واستهلاك الذاكرة.

دورة التجربة

قبل التنفيذ اكتب توقعك، ثم شغّل المثال وسجّل الخروج. أحدث فشلًا واحدًا مقصودًا، اجمع الدليل من logs أو metrics، أصلح السبب، وأعد التشغيل لإثبات أن الإصلاح يعالج العطل ولا يخفيه.

جرّب بنفسك

توقع النتيجة، شغّل الطلبين، أوقف Redis ولاحظ كود HTTP، ثم أعده وأثبت أن التطبيق لا يخفي الفشل أو يحوله إلى جلسة محلية صامتة.

19. اتصالات قاعدة البيانات وتشغيلها

ميزانية الاتصالات وPDO والمهلات والاتصالات المستديمة والمعاملات والنسخ والاستعداد للفشل.

قبل ما تبدأ

تصميم الجداول والاستعلامات له مساره، أما هنا فالتركيز على عمر الاتصال من Worker PHP إلى قاعدة البيانات وعلى ما يحدث عند البطء والفشل والنشر.

كلمات جديدة

- **Connection budget:** العدد الذي تستطيع قاعدة البيانات تحمله والمخصص لكل خدمة.
- **Persistent connection:** اتصال يعاد استخدامه داخل عملية PHP.
- **Pooler:** وسيط يدير مجموعة اتصالات قاعدة البيانات.
- **Replica lag:** تأخر نسخة القراءة عن الخادم الأساسي.

احسب الاتصالات قبل زيادة العمال

إذا شغّلت 6 نسخ، وفي كل نسخة 30 عامل FPM واتصال محتمل لكل عامل، فقد تطلب 180 اتصالًا قبل workers والـcron وأدوات الإدارة. لا تجعل `pm.max_children` وautoscaling يتجاوزان حد قاعدة البيانات.

```
database budget
= web replicas × possible connections per replica
+ queue workers
+ scheduled jobs
+ administration reserve
```

اجعل readiness تفشل عندما لا يستطيع التطبيق إنشاء اتصال أساسي، لكن لا تجعل liveness تعيد تشغيل كل النسخ بسبب عطل قاعدة بيانات مشترك.

PDO والمهلات

```
$pdo = new PDO($dsn, $user, $password, [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_TIMEOUT => 2,
    PDO::ATTR_PERSISTENT => false,
]);
```

`ATTR_TIMEOUT` لا يضبط كل أنواع timeout في كل driver. اضبط connect وstatement وlock timeout في مستوى driver أو الخادم، وضع deadline للعملية الكاملة. لا تطبع DSN أو exception trace للمستخدم.

الاتصالات المستديمة

الاتصال المستديم cache داخل كل process وليس pool مشتركة بين كل النسخ. قد يحتفظ بإعداد session أو temporary table أو lock أو transaction من الطلب السابق. استخدمه بعد قياس تكلفة الاتصال وفهم تنظيف driver، وغالبًا استخدم pooler خارجيًا عندما تحتاج تحكمًا مركزيًا.

كل معاملة تنتهي بـcommit أو rollback في مسار مضمون. أعد محاولة deadlock أو serialization failure بعدد محدود وjitter فقط عندما تكون العملية idempotent.

القراءة من Replica

بعد كتابة الطلب إلى primary قد تعيد replica قيمة أقدم. العمليات التي تحتاج read-your-writes تقرأ من primary أو تستخدم consistency token مدعومة. راقب lag، ولا ترسل تقارير حرجة إلى replica متأخرة بلا حد.

Migrations والنشر

استخدم Expand/Contract: أضف nullable column أو table أولًا، انشر كودًا يفهم الشكلين، نفّذ backfill على دفعات، ثم افرض constraint واحذف القديم لاحقًا. راقب مدة lock وحجم transaction ومعدل replication.

تجربة PostgreSQL

```
docker compose -f production/compose.yaml -f production/compose.full.yaml up --build -d
curl -fsS http://127.0.0.1:8080/database
docker compose -f production/compose.yaml -f production/compose.full.yaml exec postgres psql -U app -d app -c
"select * from runtime_probe"
```

يجب أن تتطابق `probe_key` في HTTP وPostgreSQL. أوقف postgres وتأكد أن endpoint يرجع 503 بلا credentials، ثم أعده وقس زمن التعافي.

تأكد من فهمك

▼ لماذا قد تستهلك FPM اتصالات أكثر من المتوقع؟

لأن كل worker وكل replica قد يحتفظ باتصال مستقل، إضافة إلى العمال والمهام الإدارية.

▼ ما خطر persistent connection؟

قد تنتقل حالة الاتصال مثل transaction أو lock أو إعداد session إلى طلب لاحق.

▼ متى تعيد deadlock؟

بعد rollback، بعدد محدود وjitter، وفقط عندما يمكن تكرار العملية بأمان.

▼ لماذا قد لا ترى الكتابة في replica؟

لأن replication غير متزامنة وقد توجد فترة lag.

دورة التجربة

قبل التنفيذ اكتب توقعك، ثم شغّل المثال وسجّل الخروج. أحدث فشلًا واحدًا مقصودًا، اجمع الدليل من logs أو metrics، أصلح السبب، وأعد التشغيل لإثبات أن الإصلاح يعالج العطل ولا يخفيه.

جرّب بنفسك

توقع عدد الاتصالات الأقصى، راقبه أثناء حمل صغير، اخفض حد PostgreSQL حتى يحدث saturation، ثم أصلح ميزانية FPM والـworkers بدل إضافة retries غير محدودة.

20. PHP طويلة العمر وWorker Runtimes

تشغيل FrankenPHP وRoadRunner وSwoole بأمان وإعادة ضبط الحالة والذاكرة والإشارات والنشر.

قبل ما تبدأ

في FPM يبدأ الطلب بحالة PHP شبه نظيفة وتنتهي معظم الذاكرة بانتهاء الطلب. FrankenPHP Worker Mode وRoadRunner وSwoole وOctane يحتفظون بالعملية والتطبيق في الذاكرة لطلبات كثيرة، فيقل bootstrap لكن يتغير عقد البرمجة.

كلمات جديدة

- **Long-running runtime:** عملية PHP تعالج عدة طلبات.
- **State leakage:** وصول بيانات طلب إلى طلب لاحق.
- **Reset hook:** خطوة تنظف خدمة بعد كل طلب.
- **Worker recycling:** إنهاء العامل واستبداله بعد حد محدد.

ما الذي يبقى؟

Static properties وsingletons والـglobals والاتصالات والـin-memory caches قد تبقى. لا تضع Request أو User أو locale داخل service طويلة العمر. اجعل request-scoped state يُنشأ ويُتلف في كل دورة context داخل.

```
final class RequestContext
{
    public function __construct(
        public readonly string $requestId,
        public readonly ?int $userId,
    ) {}
}

function handle(ServerRequestInterface $request): ResponseInterface
{
    $context = new RequestContext(bin2hex(random_bytes(8)), null);
    return dispatch($request, $context);
}
```

اختبر طلب A ببيانات سرية ثم طلب B بلا هوية، وتأكد أن B لا يرى أي قيمة من A. شغّل الاختبار مرات كثيرة وعلى العامل نفسه.

الذاكرة والموارد

تسرب صغير لكل طلب يصبح نموًا دائمًا. راقب RSS بعد كل N طلبات، وحدد max jobs أو max memory لإعادة التدوير كحاجز، ثم أصلح السبب. أغلق streams وcursors، أزل listeners المؤقتة، ونظف context وtracer scope في `finally`.

الإشارات والنشر

عند SIGTERM توقف عن قبول عمل جديد، أكمل الجاري ضمن grace period، صدّر telemetry المتبقية ثم اخرج. يجب أن تتجاوز مهلة المنصة مدة الإنهاء المتوقعة. rollout يحتاج توافق الجلسات والـqueue payload وقاعدة البيانات مع نسختين في الوقت نفسه.

اختيار النموذج

FPM مناسب لعزل بسيط وسلوك request واضح. Runtime طويلة العمر مفيدة عندما يكون bootstrap غاليًا أو تحتاج concurrency وWebSockets، لكنها تتطلب إطارًا يدعم reset واختبارات تسرب. لا تختَرها لمجرد benchmark hello-world.

```
FPM: request -> fresh execution -> response -> cleanup
Worker runtime: boot once -> request -> reset -> request -> reset -> recycle
```

راجع توافق extensions والعملاء مع threads أو coroutines. Blocking I/O داخل event loop قد يجمد عدة طلبات، والاتصال غير الآمن للتزامن لا يصبح آمنًا لأن API تبدو asynchronous.

تمرين state leakage محلي

```
php memory-lab.php
php resilience-lab.php
php long-running-state-lab.php
```

ثم نفّذ التطبيق نفسه على runtime طويلة العمر، أرسل 1000 طلب بهويات مختلفة، وراقب RSS والقيم الثابتة. النجاح يتطلب عدم تسرب الهوية واستقرار الذاكرة بعد warm-up واستجابة صحيحة لـSIGTERM.

تأكد من فهمك

▼ لماذا singleton أخطر هنا؟

لأنه يبقى بين الطلبات وقد يحتفظ ببيانات مستخدم أو إعداد طلب سابق.

▼ هل recycling يصلح memory leak؟

يحد أثره لكنه لا يزيل السبب؛ يجب القياس والإصلاح.

▼ ما وظيفة reset hook؟

تنظيف الحالة الخاصة بالطلب وإرجاع الخدمات إلى حالة آمنة قبل الطلب التالي.

▼ متى تفضّل FPM؟

عندما تكون بساطة العزل أهم من كلفة bootstrap أو عندما لا يدعم التطبيق دورة reset موثوقة.

دورة التجربة

قبل التنفيذ اكتب توقعك، ثم شغّل المثال وسجّل الخروج. أحدث فشلًا واحدًا مقصودًا، اجمع الدليل من logs أو metrics، أصلح السبب، وأعد التشغيل لإثبات أن الإصلاح يعالج العطل ولا يخفيه.

جرّب بنفسك

أنشئ تسربًا عمديًا في static array، أثبت نمو RSS أو انتقال قيمة بين طلبين، ثم أضف reset واختبار regression وحدًا لإعادة تدوير العامل.

05

المسار · 22 درسًا

الأمان والهوية والصلاحيات

شرح متدرج، أمثلة عملية، وتصحيحات تساعدك على الانتقال من الفكرة إلى التطبيق.

الأمان والمصادقة والصلاحيات

مسار عملي لأمان PHP والجلسات وكلمات المرور والصلاحيات وJWT وOAuth 2.0 وOpenID Connect وSSO.

المسار ده يبدأ من حماية جلسة المستخدم، وبعدها يتدرج إلى تسجيل الدخول والصلاحيات والبروتوكولات الأمنية المتقدمة. كل جزء يوضح الخطر الأول، ثم طريقة الحماية، ثم إزاي تتأكد إن الحماية شغالة.

خريطة الدروس

1. حماية الجلسات — Fixation وHijacking وتغيير المعرّف والكوكي الآمنة.
2. التحقق من المدخلات والفلترة — Validation مقابل Sanitization ودوال Filter.
3. الحماية من CSRF — Tokens وSameSite وطرق التحقق.
4. الحماية من XSS — Output Encoding حسب السياق وHTML Sanitization.
5. كلمات المرور وbcrypt — Hashing وArgon2id والتحقق وإعادة الـhash.
6. Encryption وHTTP Authentication — المفاتيح وAEAD وBasic/Bearer.
7. المصادقة والصلاحيات عمليًا — RBAC والملكية والسياسات والدفاع متعدد الطبقات.
8. JWT والتحقق الآمن — البنية والتوقيع والـClaims والتحقق وإدارة المفاتيح.
9. Federated Identity وSSO وSAML وOIDC — الفصل بين البروتوكولات واختيار المناسب.
10. مفاهيم OAuth 2.0 والتوكينات — الأدوار والـClients والـScopes وAccess/Refresh Tokens.
11. OAuth Flows الحديثة — Authorization Code + PKCE وClient Credentials وDevice Flow.
12. أمان OAuth وState — OIDC وNonce وRedirect URIs وتخزين التوكينات والإبطال.
13. دورة حياة الحساب — التسجيل والتأكيد والاسترجاع وإلغاء الجلسات.
14. MFA وTOTP وPasskeys — العوامل وWebAuthn وRecovery وStep-up.
15. API Keys وهوية الأنظمة — التخزين والـScopes والدوران وService Accounts.
16. ABAC وReBAC وعزل الـTenants — السياسات المتقدمة ومنع IDOR.
17. Rate Limiting ومقاومة الإساءة — الخوارزميات وحماية login وسياسة الفشل.
18. Security Logging وAudit Trail — الأحداث والتنقية والسلامة والتنبيهات.
19. إدارة الأسرار والمفاتيح — Secret managers وrotation وenvelope encryption.
20. Threat Modeling واختبار الصلاحيات — أصول وحدود ثقة ومصفوفة أدوار وملكية وTenants.
21. الأجهزة والجلسات وكلمات المرور المسربة وSCIM — إبطال الجلسات وBreached Passwords وProvisioning مؤسسي.
22. PAR وJAR وRAR وحوادث التوكينات — طلبات OAuth المتقدمة وRunbook لتسريب Tokens وKeys.

قاعدة أساسية

كل ما يأتي من المتصفح أو API أو Cookie أو Header أو قاعدة بيانات قديمة يُعامل كبيانات غير موثوقة حتى يُتحقق منه، ثم يُرمّز عند موضع الإخراج المناسب.

مراجع المعايير

- JWT - RFC 7519 وأفضل ممارساته - RFC 8725
- OAuth 2.0 - RFC 6749 وSecurity BCP - RFC 9700
- PKCE - RFC 7636 وDevice Authorization - RFC 8628
- OpenID Connect Core
- SAML 2.0 Technical Overview
- SCIM Protocol - RFC 7644 وPAR - RFC 9126
- JAR - RFC 9101 وRAR - RFC 9396

تابع حسابًا واحدًا عبر حدود النظام

شغّل `php auth-journey.php` من المختبر المحلي. اقرأه بعد الجلسات والـhash والصلاحيات واستعادة الحساب. يحتاج SQLite 3.35+ لاستخدام `DELETE … RETURNING`، ويستخدم نفس مخطط المخزون. البرنامج ينشئ حسابًا مؤقتًا، يرفض كلمة خاطئة، يصدر جلسة عشوائية، يفحص انتهاءها، ينشئ طلبًا بهوية المستخدم المتحقق منها، يمنع مستخدمًا آخر من قراءته، يعيد نفس الطلب، يغير كلمة السر مرة واحدة ويلغي الجلسات القديمة. يفضل في outbox حدث واحد. حدث التدقيق يذهب إلى stderr بلا كلمة سر أو توكين جلسة أو استرجاع.

توكين الجلسة والاسترجاع سر عشوائي من 32 بايت ممثل بالنظام الست عشري؛ قاعدة البيانات تحفظ SHA-256 له فقط. بعكس كلمة السر البشرية، التوكين عالي العشوائية مش محتاج password hash بطيئًا عمدًا. استهلاك توكين الاسترجاع وتغيير كلمة السر وإلغاء الجلسات في معاملة واحدة؛ فشل كتابة يلغي الجميع. DELETE ذرية ترجع user_id تمنع مستهلكين من أخذ نفس التوكين.

تتبّع الحد: تسجيل الدخول يثبت الهوية، و`WHERE id=? AND user_id=?` يفرض الملكية، ومفتاح idempotency يمنع تكرار الأثر التجاري، وسجل التدقيق يثبت النتيجة بلا بيانات اعتماد. الجلسة تصدر عشوائيًا بعد التحقق بدل قبول معرّف جلسة مصادق عليها اختاره المستدعي.

ده مثال محلي كامل لتدفق الحالة، مش خادم مصادقة جاهز للنشر. تسليم التوكين محاكاة في الذاكرة ولا يُرسل بريد. Cookies وCSRF وتحديد المعدل ومنع كشف وجود الحساب وسياسة كلمات المرور وتكامل المزود تتطبق حسب دروسها قبل فتح endpoints. حد الطول هنا سياسة تجريبية صغيرة معلنة، مش سياسة كلمات مرور كاملة. تدريب: زوّد `$now` بعد انتهاء توكين الاسترجاع، وتأكد أن الرفض لا يغير كلمة السر أو الجلسات. ثم ارم استثناء قبل commit وتحقق من بقاء كل الحالة السابقة بسبب rollback.

1. حماية الجلسات

دورة الجلسة الآمنة ومنع Session Fixation وHijacking وإدارة المعرف والكوكي.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **IP:** عنوان رقمي بيميز جهازًا أو واجهة شبكة.
- **URL:** العنوان الكامل لمورد على الويب، زي صفحة أو صورة أو نقطة API.
- **Session:** بيانات مؤقتة تساعد الخادم يميّز المستخدم بين أكثر من طلب.
- **Cookie:** قيمة صغيرة يحفظها المتصفح ويرسلها مع الطلبات المناسبة.
- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.
- **Worker:** برنامج يعمل في الخلفية ويسحب المهام من الطابور وينفذها.

كيف تعمل الجلسة؟

يحفظ الخادم البيانات، ويحمل المتصفح عادة Cookie تحتوي Session ID عشوائيًا. المعرّف **بيانات اعتماد**؛ من يسرقه قد ينتحل الجلسة حتى لو لم يعرف كلمة المرور.

أهم المخاطر:

- **Session Fixation:** يجبر المهاجم الضحية على استخدام ID يعرفه، ثم ينتظر تسجيل الدخول.
- **Session Hijacking:** يسرق ID فعّالًا عبر XSS أو شبكة غير محمية أو Logs أو جهاز مخترق.
- **Brute force:** محاولة تخمين IDs ضعيفة؛ يجب أن يولدها PHP عشوائيًا ولا تنشئها بـ `uniqid()`.
- **Race conditions:** حذف الجلسة القديمة فورًا أثناء طلبات متزامنة قد يفقد الحالة.

إعداد البداية

```php
<?php
declare(strict_types=1);

session_start([
    'use_strict_mode' => true,
    'use_only_cookies' => true,
    'cookie_secure' => true,
    'cookie_httponly' => true,
    'cookie_samesite' => 'Lax',
]);
```

- `Secure` : لا ترسل الكوكي إلا عبر HTTPS.
- `HttpOnly` : يمنع JavaScript من قراءتها، لكنه لا يمنع XSS من تنفيذ أفعال باسم المستخدم.
- `SameSite` : يقلل الإرسال cross-site، وليس بديلًا مطلقًا عن CSRF token.
- `use_strict_mode` : يرفض Session ID غير مهيأ من الخادم.

استدعِ `session_start()` قبل الإخراج. لا تمرر Session ID في URL لأنه يتسرب في History وLogs وReferer.

تسجيل الدخول وتغيير الصلاحية

بعد إثبات كلمة المرور غيّر المعرّف **قبل** تثبيت حالة المصادقة الجديدة:

```php
if (password_verify($password, $user['password_hash'])) {
    session_regenerate_id();
    $_SESSION['user_id'] = $user['id'];
    $_SESSION['auth_time'] = time();
    $_SESSION['last_seen'] = time();
}
```

غيّر المعرف عند login، ورفع الصلاحية، وتغيير كلمة المرور أو حدث أمني مهم. لا تغيّره في كل request دون تصميم؛ الطلبات المتوازية قد تتسابق.

> ⚠
>
> **تنبيه**
>
> استخدام `session_regenerate_id(true)` لحذف التخزين القديم فورًا قد يسبب فقدًا على شبكات غير مستقرة أو مع AJAX المتزامن. الأنظمة الحساسة تستخدم انتقالًا زمنيًا وتمنع استخدام القديم بعد مهلة قصيرة ثم تترك GC يزيله.

انتهاء خمول وعمر كلي

```
$now = time();
$idleLimit = 15 * 60;
$absoluteLimit = 8 * 60 * 60;

if (
    isset($_SESSION['last_seen'], $_SESSION['auth_time'])
    && ($now - $_SESSION['last_seen'] > $idleLimit
        || $now - $_SESSION['auth_time'] > $absoluteLimit)
) {
    logout();
}

$_SESSION['last_seen'] = $now;
```

لا تعتمد على `session.gc_maxlifetime` وحده؛ GC تنظيف تخزين وليس سياسة وصول دقيقة.

Fingerprinting بحذر

يمكن تسجيل User-Agent وتغيرات IP كإشارة مخاطرة، لكن لا تربط الجلسة تطابقًا كاملًا بـIP؛ شبكات الهاتف وVPN وNAT تتغير. استخدم الإشارات لطلب إعادة المصادقة أو تسجيل تنبيه، لا لطرد المستخدم عشوائيًا.

إغلاق القفل والخروج

```
// بعد تعديل الجلسة، احفظ وأغلق القفل مبكرًا
session_write_close();
```

لتسجيل الخروج: ابدأ الجلسة، امسح `$_SESSION`، احذف Cookie بنفس path/domain/options، ثم `session_destroy()`. أبطل كذلك refresh tokens والجلسات المسجلة في قاعدة البيانات إن وجدت.

سيناريو أمني

▼ ليه نغيّر session ID بعد login؟

لمنع session fixation؛ أنشئ هوية جلسة جديدة ولا تقبل المعرّف الذي عرفه المهاجم.

تدريب تهديد

السيناريو: مهاجم حصل على معرّف جلسة قديم قبل تسجيل الدخول ويحاول إعادة استخدامه بعد نجاح تسجيل الدخول.

اختبار المنع: سجّل الدخول، احفظ معرّف الجلسة الجديد، ثم أرسل طلبًا حساسًا بالمعرّف القديم من عميل منفصل.

النتيجة المتوقعة: يُرفض المعرّف القديم، ولا تتغير بيانات الحساب، ويظهر معرّف جديد بعد المصادقة.

مرجع التحقق

- PHP session security

اربط النقاط ببعض

في بيئة موزعة اجعل session store مركزيًا أو استخدم آلية إلغاء تصل لكل nodes، ولا تعتمد على ذاكرة worker. استخدم cookie prefixes المناسبة مثل `__Host-` عندما ينطبق العقد، واضبط Secure وHttpOnly وSameSite والمسار. الجلسة المعتمدة على Cookie ما زالت تحتاج CSRF defense للعمليات المتغيرة.

جرّب بنفسك

ألغ جلسة من node وحاول استخدامها عبر node آخر ثم نفذ CSRF test.

2. التحقق من المدخلات والفلترة

الفرق بين Validation وSanitization واستخدام filter_var وfilter_input وقواعد النطاق.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **IP:** عنوان رقمي بيميز جهازًا أو واجهة شبكة.
- **URL:** العنوان الكامل لمورد على الويب، زي صفحة أو صورة أو نقطة API.

Validation ليست Sanitization

- **Validation:** هل القيمة تطابق شروطنا؟ لا يفترض أن يغيّرها.
- **Normalization:** توحيد تمثيل مشروع، مثل trim أو تحويل case عند الحاجة.
- **Sanitization:** حذف/تغيير محارف لصنع قيمة أخرى؛ قد يخفي الخطأ ويفسد البيانات.
- **Output Encoding:** تحويل القيمة عند إخراجها لسياق HTML/URL/JS، وليس عند الإدخال.

المسار المقترح:

```
Parse → Validate shape/type/range → Authorize action → Use safely
                                                  ↓
                                        Encode at output sink
```

لا تحاول «تنظيف كل المدخلات» مرة واحدة ثم تعتبرها آمنة لكل مكان.

filter_var

```
$email = filter_var($rawEmail, FILTER_VALIDATE_EMAIL);
if ($email === false) {
    throw new InvalidArgumentException('Invalid email');
}

$age = filter_var($rawAge, FILTER_VALIDATE_INT, [
    'options' => ['min_range' => 18, 'max_range' => 120],
]);
if ($age === false) {
    throw new InvalidArgumentException('Age must be 18–120');
}

$ip = filter_var($rawIp, FILTER_VALIDATE_IP);
$url = filter_var($rawUrl, FILTER_VALIDATE_URL);
```

تحقق باستخدام `=== false`؛ القيمة الصحيحة قد تكون `0`. و`FILTER_DEFAULT` يعني `FILTER_UNSAFE_RAW`، فلا يفعل تنظيفًا سحريًا.

filter_input وfilter_input_array

```
$page = filter_input(INPUT_GET, 'page', FILTER_VALIDATE_INT, [
    'options' => ['default' => 1, 'min_range' => 1],
]);

$input = filter_input_array(INPUT_POST, [
    'email' => FILTER_VALIDATE_EMAIL,
    'age' => [
        'filter' => FILTER_VALIDATE_INT,
        'options' => ['min_range' => 18, 'max_range' => 120],
    ],
]);
```

`filter_input()` يقرأ المصدر الخارجي الأصلي حسب SAPI، وقد يختلف عن قيمة عدّلتها داخل `$_GET` أو `$_POST`. اختبر بيئة التشغيل.

قواعد المجال أهم من شكل النوع

بريد صالح نحويًا لا يعني أنه يملك الحساب. وOrder ID صحيح كعدد لا يعني أن المستخدم يملك الطلب.

```
$orderId = filter_input(INPUT_POST, 'order_id', FILTER_VALIDATE_INT);
if ($orderId === false || $orderId < 1) {
    throw new InvalidArgumentException('Invalid order');
}

$order = $orders->findForUser($orderId, $currentUserId);
if ($order === null) {
    throw new RuntimeException('Not found or forbidden');
}
```

طبّق allow-list للـenums والفرز وأسماء الحقول. لا يمكن حماية SQL أو HTML بمجرد Sanitization عام؛ استخدم Prepared Statements وOutput Encoding.

سيناريو أمني

▼ هل validation يغني عن output encoding؟

لا؛ validation يفرض عقد الإدخال، وencoding يعتمد على سياق الإخراج لمنع تفسير البيانات ككود.

تدريب تهديد

السيناريو: يرسل العميل قيمة تبدو رقمية لكنها خارج النطاق أو تحتوي محارف إضافية كي تعبر فحصًا سطحيًا.

اختبار المنع: جرّب قيمة سالبة وقيمة أكبر من الحد وقيمة مثل `12x` في الحقل نفسه.

النتيجة المتوقعة: تُرفض الحالات الثلاث برسالة عامة وموحّدة، ولا يصل أي منها إلى منطق العمل أو قاعدة البيانات.

مرجع التحقق

- OWASP Input Validation Cheat Sheet

اربط النقاط ببعض

Canonicalization تسبق validation عندما توجد تمثيلات متعددة، لكن لا تفك أو تطبع القيمة مرات غير محدودة. تحقق من nested arrays والعمق والعدد والحجم، لا الحقول المفردة فقط. في الملفات افحص upload status والحجم والتوقيع/MIME والمحتوى، واستخدم allowlist للقيم المعروفة.

جرّب بنفسك

اختبر encoding مزدوجًا وarray عميقة وملفًا باسم وصيغة متعارضين.

3. الحماية من CSRF

كيف يستغل CSRF جلسة المستخدم وكيف نمنعه بالـtokens وSameSite والتحقق من المصدر.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **URL:** العنوان الكامل لمورد على الويب، زي صفحة أو صورة أو نقطة API.
- **API:** واجهة محددة تسمح لبرنامج يطلب بيانات أو ينفّذ عملية عند برنامج آخر.
- **Session:** بيانات مؤقتة تساعد الخادم يميّز المستخدم بين أكثر من طلب.
- **Cookie:** قيمة صغيرة يحفظها المتصفح ويرسلها مع الطلبات المناسبة.
- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.
- **UTF-8:** طريقة شائعة لتحويل أرقام Unicode إلى بايتات تُحفظ وتُنقل.

ما الهجوم؟

في CSRF يخدع موقع خبيث متصفح مستخدم مسجل الدخول ليرسل request غير مرغوب إلى موقع موثوق. المتصفح قد يرفق Cookies تلقائيًا، فيرى الخادم جلسة صحيحة لكنه لا يعرف أن المستخدم لم يقصد الفعل.

مثال خطر: Endpoint يغيّر البريد عبر GET أو يقبل POST بلا إثبات لنية المستخدم.

Synchronizer Token Pattern

أنشئ token عشوائيًا server-side، خزنه في Session، وضعه داخل النموذج:

```
session_start();

if (!isset($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}
```

```
<form method="post" action="/account/email">
    <input type="hidden" name="csrf_token"
           value="<?= htmlspecialchars($_SESSION['csrf_token'], ENT_QUOTES, 'UTF-8') ?>">
    <input type="email" name="email" required>
    <button>تحديث</button>
</form>
```

تحقق قبل تنفيذ أي تغيير:

```
$sent = $_POST['csrf_token'] ?? '';
$origin = $_SERVER['HTTP_ORIGIN'] ?? null;
$allowedOrigins = ['https://app.example.com'];

if (
    !is_string($sent)
    || $sent === ''
    || !is_string($origin)
    || !in_array($origin, $allowedOrigins, true)
    || !isset($_SESSION['csrf_token'])
    || !hash_equals($_SESSION['csrf_token'], $sent)
) {
    http_response_code(403);
    exit('Invalid CSRF token');
}
```

`hash_equals()` مقارنة ثابتة الزمن نسبيًا. لا تضع token في URL أو Logs، واجعله مرتبطًا بالجلسة أو الفعل وفق نموذجك.

دفاع متعدد الطبقات

- لا تستخدم GET لتغيير الحالة.
- استخدم `SameSite=Lax` أو `Strict` حيث يناسب.
- تحقق من `Origin` للطلبات الحساسة، واستعمل `Referer` كخيار ثانٍ مدروس.
- في APIs التي لا تعتمد على Cookies، Header مخصص مع Token قد يمنع simple cross-site forms.
- أعد طلب كلمة المرور أو MFA للفعل شديد الحساسية.
- حدّد Content-Type المتوقع ولا تقبل أشكالًا أكثر من الحاجة.

> ⚠
>
> **تنبيه**
>
> CORS ليست بديلًا عن CSRF protection؛ وبعض الطلبات «البسيطة» يمكن إرسالها دون preflight. كذلك XSS في موقعك يستطيع غالبًا قراءة token أو تنفيذ الطلب نفسه، لذلك إصلاح XSS أساسي.

Double-submit Cookie

يمكن إرسال قيمة في Cookie وقيمة في header/body ثم مقارنتهما، لكن الأفضل أن تكون القيمة موقعة ومرتبطة بالجلسة لمنع cookie injection. لا تخترع البروتوكول إن كان framework يوفر حماية مدققة.

Token lifecycle

لا يلزم دائمًا تدوير token بعد كل request؛ ذلك قد يكسر tabs والطلبات المتزامنة. Token لكل جلسة أو لكل نموذج كلاهما صحيح حسب الخطر وتجربة الاستخدام. دوّره عند تغيير الجلسة أو تسجيل الدخول، وحدد فشلًا واضحًا دون تنفيذ جزئي.

سيناريو أمني

▼ ليه SameSite وحده مش كفاية دائمًا؟

له حدود وتوافق وسيناريوهات bypass؛ استخدم token وربط origin مع cookies مناسبة حسب التهديد.

تدريب تهديد

السيناريو: صفحة خارجية تحاول إرسال طلب تغيير بريد إلكتروني بملفات تعريف ارتباط الضحية.

اختبار المنع: أرسل الطلب بلا CSRF token، ثم بتوكين مستخدم آخر، ثم بتوكين صحيح لكن من Origin غير مسموح.

النتيجة المتوقعة: تُرفض المحاولات الثلاث ولا يتغير البريد؛ لا يكفي وجود Cookie صالح لقبول العملية.

مرجع التحقق

- OWASP CSRF Prevention Cheat Sheet

اربط النقاط ببعض

SameSite يقلل بعض المسارات لكنه لا يستبدل token وOrigin checks، وتوجد فروق بين Lax وStrict وLogin CSRF. None قد يربط الضحية بحساب المهاجم قبل وجود جلسة موثوقة. API لا يستخدم ambient cookies غالبًا لا يحتاج CSRF بنفس الصورة، لكنه يحتاج منع token leakage وCORS صحيح.

جرّب بنفسك

اختبر login CSRF وطلب cross-site مع SameSite modes مختلفة.

سياسة المصدر

اضبط الـOrigin الموثوق كاملًا: البروتوكول والدومين والبورت لو مش الافتراضي. ما تبنيش القائمة من Host اللي جاي في الطلب. المثال بيرفض المصدر الغريب وكمان غياب Origin؛ بعض العملاء مش بيبعتوه، فاختبر العملاء الحقيقيين قبل اعتماد fallback مدروس بـReferer. المصدر طبقة إضافية مش إثبات هوية: عميل خارج المتصفح يقدر يكتب الهيدر. احتفظ بفحص توكين الجلسة واستخدم POST للتغييرات.

4. الحماية من XSS

Stored وReflected وDOM XSS والترميز حسب سياق HTML وAttribute وURL وJavaScript.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Sanitizer:** أداة بتحلل HTML غير موثوق وبتشيل الوسوم والخصائص غير المسموحة، مع الحفاظ على التنسيق المسموح؛ ودي غير أدوات فحص ذاكرة C++.
- **URL:** العنوان الكامل لمورد على الويب، زي صفحة أو صورة أو نقطة API.
- **Cookie:** قيمة صغيرة يحفظها المتصفح ويرسلها مع الطلبات المناسبة.
- **UTF-8:** طريقة شائعة لتحويل أرقام Unicode إلى بايتات تُحفظ وتُنقل.
- **Function:** دالة: جزء كود له اسم ومهمة محددة ويمكن استدعاؤه أكثر من مرة.

ما XSS؟

تحدث Cross-Site Scripting عندما تُفسر بيانات غير موثوقة ككود داخل المتصفح.

- **Reflected:** ترجع payload مباشرة في نفس الاستجابة، مثل قيمة بحث.
- **Stored:** تُحفظ في قاعدة البيانات ثم تُعرض لضحايا آخرين.
- **DOM-based:** JavaScript في الصفحة ينقل بيانات إلى sink خطير مثل `innerHTML`.

النتائج قد تشمل سرقة بيانات متاحة للصفحة، تنفيذ أفعال بحساب المستخدم، تعديل الواجهة أو التصيد. `HttpOnly` يحمي قراءة Cookie فقط، ولا يمنع تنفيذ request من الصفحة المصابة.

HTML text وattributes

```
function e(string $value): string
{
    return htmlspecialchars($value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}
```

```
<h1><?= e($title) ?></h1>
<input value="<?= e($displayName) ?>">
```

ضع attribute بين quotes. لا تسمح للمستخدم بتحديد اسم attribute أو event handler.

URL context

رمّز **قيمة المعامل** بـ `rawurlencode` ثم رمّز الرابط كـHTML عند إدخاله في attribute:

```
$url = '/search?q=' . rawurlencode($query);
?>
<a href="<?= e($url) ?>">بحث</a>
```

لا يكفي encoding لمنع scheme `javascript:`. حلّل URL وطبّق allow-list للبروتوكول والمضيف عند الروابط الخارجية.

JavaScript وJSON

الأفضل عدم إدخال نصوص مباشرة في inline JavaScript. إذا لزم نقل بيانات، استخدم JSON بخصائص hex:

```
<script>
const profile = <?= json_encode(
    $profile,
    JSON_HEX_TAG | JSON_HEX_AMP | JSON_HEX_APOS | JSON_HEX_QUOT
) ?>;
</script>
```

في DOM استخدم `textContent` بدل `innerHTML` للنص:

```
message.textContent = untrustedValue;
```

عندما تريد السماح بـHTML

`htmlspecialchars` سيعرض HTML كنص. إذا كان المنتج يسمح بمحتوى غني، استخدم HTML sanitizer موثوقًا بسياسة allow-list، وحدّثه باستمرار. Regex ليست HTML parser.

أماكن خطرة

تجنب وضع غير الموثوق داخل `<script>` أو `<style>` أو HTML comments أو أسماء tags/attributes أو event handlers، وتجنب `eval` و `document.write`.

طبقات إضافية

- Auto-escaping في template engine، مع مراجعة raw/unsafe escape hatches.
- Content Security Policy قوية، ويفضل nonce/hash بدل `unsafe-inline`.
- Trusted Types لتطبيقات DOM الكبيرة حيث يناسب.
- Cookies بـHttpOnly/Secure/SameSite.
- MIME صحيح؛ JSON يجب أن يخرج `application/json`.

CSP طبقة تقليل ضرر وليست بديلًا عن encoding وsanitization.

سيناريو أمني

▼ أين نعمل encoding؟

وقت الإخراج وبحسب السياق: HTML أو attribute أو URL أو JavaScript؛ encoding واحد لا يصلح لكل مكان.

تدريب تهديد

السيناريو: يحفظ المهاجم تعليقًا يحتوي وسمًا أو معالج حدث كي ينفّذ JavaScript عند فتح الصفحة لاحقًا.

اختبار المنع: أدخل نصًا يحوي `<script>` ومحاولة داخل سمة HTML، ثم اعرضه في كل سياق تستخدمه الصفحة.

النتيجة المتوقعة: يظهر النص كنص فقط، ولا ينفذ أي حدث، وتظل سياسة CSP طبقة دفاع إضافية لا بديلًا للترميز حسب السياق.

مرجع التحقق

- OWASP XSS Prevention Cheat Sheet

اربط النقاط ببعض

DOM XSS يبدأ من source غير موثوق يصل إلى sink مثل innerHTML أو eval؛ output encoding على الخادم وحده لا يكفي. Trusted Types يقلل sinks الخطرة في المتصفحات الداعمة، وCSP قوية تستخدم nonces أو hashes بدل allowlist واسعة. لا تخلط sanitization للسماح بـHTML مع encoding لعرض النص.

جرّب بنفسك

تتبع قيمة من location إلى DOM sink ثم استبدل sink أو طبق policy.

5. كلمات المرور وHashing

تخزين كلمات المرور باستخدام password_hash وpassword_verify وArgon2id وbcrypt وإعادة الـhash.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **URL:** العنوان الكامل لمورد على الويب، زي صفحة أو صورة أو نقطة API.
- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.

Hashing ليس Encryption

كلمة المرور لا نحتاج استعادتها؛ نحتاج التحقق منها. لذلك نخزن **password hash بطيئًا ومملحًا**، لا تشفيرًا قابلًا للفك ولا SHA-256 سريعًا وحده.

دوال PHP تدير salt والصيغة والمعاملات داخل hash:

```php
<?php
declare(strict_types=1);

$password = 'a long example passphrase';
// One policy for registration AND rehashing after successful login.
$algorithm = defined('PASSWORD_ARGON2ID') ? PASSWORD_ARGON2ID : PASSWORD_DEFAULT;
$options = $algorithm === PASSWORD_DEFAULT ? [] : [
    'memory_cost' => 64 * 1024, 'time_cost' => 3, 'threads' => 2,
];
$storedHash = password_hash($password, $algorithm, $options);
echo password_verify($password, $storedHash) ? "valid\n" : "invalid\n";
echo password_verify('wrong password', $storedHash) ? "valid\n" : "invalid\n";
if (password_verify($password, $storedHash)
    && password_needs_rehash($storedHash, $algorithm, $options)) {
    $storedHash = password_hash($password, $algorithm, $options);
    // Persist using a conditional update against the previous hash.
}
```

اجعل عمود قاعدة البيانات `VARCHAR(255)` لأن `PASSWORD_DEFAULT` قد تتغير خوارزميته وطول ناتجه.

المثال بيختار Argon2id لو موجود، وبيحتفظ بنفس الخوارزمية ونفس الخيارات وقت إعادة الـhash.

القيم مثال وليست وصفة عالمية؛ benchmark على خادم الإنتاج واختر تكلفة تؤخر المهاجم دون تعطيل المستخدمين.

التحقق وإعادة الـhash

`password_verify()` يستخرج salt/options من hash. لا تقارن hashes يدويًا، ولا تنشئ salt بنفسك.

سياسة صحيحة

- اسمح بعبارات مرور طويلة، ولا تفرض قواعد تركيب مزعجة بلا سبب.
- لا تقص كلمة المرور بصمت. راعِ حد الخوارزمية؛ bcrypt يتعامل تاريخيًا مع أول 72 bytes.
- لا تغيّر case أو trim لكلمة مرور المستخدم دون سياسة معلنة.
- افحص كلمات المرور المسربة إن كانت لديك خدمة مناسبة تحفظ الخصوصية.
- أضف Rate Limiting وتأخيرًا تدريجيًا ومراقبة، وMFA للحسابات الحساسة.
- لا تسجل كلمة المرور أو تضعها في URL.

Pepper اختياري

يمكن إضافة secret server-side منفصل عن قاعدة البيانات باستخدام HMAC قبل `password_hash`، لكنه يزيد تعقيد التدوير والاسترجاع. احفظه في secret manager لا في Git، وخطط لتغيير المفاتيح. Salt ليس سرًا وموجود داخل hash؛ Pepper سر مختلف.

تغيير واسترجاع كلمة المرور

- اطلب إعادة المصادقة للتغيير الحساس.
- Reset token عشوائي، قصير العمر، single-use، وخزّن hash له لا قيمته الخام.
- بعد التغيير ألغِ الجلسات/refresh tokens الأخرى حسب سياسة المنتج.
- لا ترسل كلمة المرور القديمة أو الجديدة عبر البريد.

سيناريو أمني

▼ ليه نستخدم Argon2id أو bcrypt بدل SHA-256؟

password hashes بطيئة ومملحة عمدًا لمقاومة التخمين؛ hash سريع يساعد المهاجم على تجربة كلمات أكثر.

تدريب تهديد

السيناريو: تتسرب قاعدة المستخدمين؛ الهدف أن تبقى كلمات المرور مكلفة للكسر وألا تكشف السجلات أو الاستجابات قيمة حساسة.

اختبار المنع: تحقق أن نفس كلمة المرور تنتج hash مختلفًا مرتين، وأن كلمة خاطئة تفشل، وأن `password_needs_rehash` يكتشف الإعداد القديم.

النتيجة المتوقعة: تنجح المطابقة الصحيحة فقط، ولا تُخزن كلمة المرور الخام، ويُحدّث الـhash بعد تسجيل دخول ناجح عند الحاجة.

مرجع التحقق

- PHP password hashing functions

اربط النقاط ببعض

اختر Argon2 أو bcrypt parameters بقياس latency وذاكرة على عتاد الإنتاج، ثم version policy عبر needs_rehash. افحص كلمات المرور المسربة بطريقة تحمي الخصوصية مع rate limits، واربط reset token بعمر واستخدام واحد وتخزين hash وإلغاء الجلسات حسب السياسة.

جرّب بنفسك

قِس إعدادين للhash وحدد budget ثم اختبر reset token مستخدمًا مرتين.

افهم الناتج والفشل

البرنامج الكامل بيطبع `valid` وبعدها `invalid`. `PASSWORD_DEFAULT` سياسة ممكن PHP يغيرها؛ استخدامها مش معناه تثبيت Argon2id. اختار إعداداتك بالقياس. في PHP 8 فشل التوليد بيرمي استثناء/خطأ، زي `ValueError` للإعداد غير الصحيح أو `Error` لبعض حالات الفشل؛ فحص `=== false` يخص سلوكًا أقدم. خلي معالج الأخطاء المركزي يعيد رسالة عامة من غير تسجيل كلمة السر. تحديث قاعدة البيانات مسؤولية التطبيق: استخدم تحديثًا مشروطًا بالقيمة السابقة حتى لا تكتب فوق تغيير كلمة سر حصل بالتزامن.

تدريب: غيّر `time_cost` فقط بعد إنشاء الـhash. التحقق يفضل ناجح لأن الإعداد القديم محفوظ داخل الـhash، لكن `password_needs_rehash` يرجع true مع السياسة الجديدة.

6. HTTP Authentication وEncryption

التشفير الآمن وإدارة المفاتيح وHTTP Basic وBearer Tokens والفرق عن Hashing.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **HTTP:** قواعد تبادل الطلبات والردود بين المتصفح والخادم.
- **URL:** العنوان الكامل لمورد على الويب، زي صفحة أو صورة أو نقطة API.
- **TLS:** طبقة تشفير بتحمي البيانات وهي ماشية بين طرفين.
- **API:** واجهة محددة تسمح لبرنامج يطلب بيانات أو ينفّذ عملية عند برنامج آخر.
- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.
- **Scope:** نطاق التفويض: اسم صلاحية يطلبها التطبيق أو يحصل عليها، زي orders:read لقراءة الطلبات؛ ولسه لازم نتحقق إن المستخدم يملك الطلب.
- **Function:** دالة: جزء كود له اسم ومهمة محددة ويمكن استدعاؤه أكثر من مرة.

متى نستخدم Encryption؟

نستخدمه عندما يجب استعادة النص الأصلي، مثل سر تكامل خارجي مخزن. كلمة المرور لا تحتاج فكًا، لذلك تستخدم Hashing.

- **Symmetric:** المفتاح نفسه للتشفير وفك التشفير؛ سريع للبيانات.
- **Asymmetric:** Public/Private Key؛ للتوقيع وتبادل الأسرار وحالات محددة.
- **Authenticated Encryption (AEAD):** يحمي السرية ويكشف التعديل؛ فضّله على «تشفير فقط».

مثال Sodium

```
function encrypt(string $plainText, string $key): string
{
    $nonce = random_bytes(SODIUM_CRYPTO_SECRETBOX_NONCEBYTES);
    $cipher = sodium_crypto_secretbox($plainText, $nonce, $key);
    return base64_encode($nonce . $cipher);
}

function decrypt(string $encoded, string $key): string
{
    $payload = base64_decode($encoded, true);
    if ($payload === false || strlen($payload) < SODIUM_CRYPTO_SECRETBOX_NONCEBYTES +
SODIUM_CRYPTO_SECRETBOX_MACBYTES) {
        throw new RuntimeException('Invalid encoding');
    }

    $nonceLength = SODIUM_CRYPTO_SECRETBOX_NONCEBYTES;
    $nonce = substr($payload, 0, $nonceLength);
    $cipher = substr($payload, $nonceLength);
    $plain = sodium_crypto_secretbox_open($cipher, $nonce, $key);

    if ($plain === false) {
        throw new RuntimeException('Authentication failed');
    }

    return $plain;
}
```

المفتاح يجب أن يكون بطول `SODIUM_CRYPTO_SECRETBOX_KEYBYTES` ومن مصدر عشوائي آمن، ويُحفظ في secret manager.
الـNonce ليس سرًا لكنه يجب ألا يعاد استخدامه مع المفتاح وفق متطلبات الخوارزمية.

> ⚠
>
> **تنبيه**
>
> Base64 encoding وليس encryption. ولا تخترع خوارزمية أو تجمع AES/CBC وMAC يدويًا؛ استخدم API عالي المستوى يدعم authentication.

إدارة المفاتيح

- افصل المفتاح عن ciphertext وقاعدة البيانات قدر الإمكان.
- ضع Key ID/version مع البيانات لتسهيل rotation.
- حدّد من يستطيع decrypt وراقب العمليات.
- خطط للنسخ الاحتياطي والتدوير والإلغاء.
- لا تطبع المفتاح في logs أو exceptions.

HTTP Basic

يرسل العميل:

```
Authorization: Basic dXNlcjpwYXNz
```

القيمة Base64 لـ `username:password` ويمكن فكها بسهولة؛ الحماية تأتي من **HTTPS**.

في PHP تحت بعض إعدادات الخادم:

```
$user = $_SERVER['PHP_AUTH_USER'] ?? null;
$pass = $_SERVER['PHP_AUTH_PW'] ?? null;

if ($user === null || !verifyCredentials($user, $pass ?? '')) {
    header('WWW-Authenticate: Basic realm="Admin"');
    http_response_code(401);
    exit('Authentication required');
}
```

Basic مناسب لأدوات داخلية بسيطة مع TLS وrate limiting، لكنه يرسل credential مع كل request ولا يملك logout/token lifecycle متقدمًا.

Bearer Tokens

```
Authorization: Bearer opaque-random-token
```

من يحمل Bearer token يستطيع استخدامه؛ خزنه وانقله كسر. استخدم expiry وscope وrotation وrevocation حيث يلزم. ليس كل token JWT، وJWT لا يعني أنه مشفر أو قابل للإلغاء تلقائيًا.

سيناريو أمني

▼ هل التشفير بديل للتوقيع؟

لا؛ التشفير يخفي المحتوى، بينما التوقيع يثبت السلامة والمنشأ. قد تحتاج الاثنين.

تدريب تهديد

السيناريو: مهاجم على الشبكة يحاول قراءة بيانات اعتماد Basic Authentication أو إجبار العميل على اتصال غير مشفر.

اختبار المنع: اختبر طلب HTTP بلا أي بيانات اعتماد، ثم طلب HTTPS ببيانات اعتماد تجريبية خاطئة. راقب إن العميل لا يرسل الأسرار قبل إنشاء اتصال TLS.

النتيجة المتوقعة: يُعاد توجيه HTTP أو يُرفض، ولا تُقبل البيانات الخاطئة، ولا تظهر الأسرار في URL أو السجلات.

مرجع التحقق

- OWASP Transport Layer Security Cheat Sheet

اربط النقاط ببعض

استخدم AEAD مع nonce فريد لكل مفتاح وassociated data للسياق، ولا تخترع key derivation أو format. افصل encryption key عن data ومكان النسخ الاحتياطي. Basic يرسل credential مع كل طلب ويحتاج TLS دائمًا، وBearer يمنح حامله الصلاحية؛ scopes وaudience والعمر والتخزين أجزاء من العقد.

جرّب بنفسك

غيّر السياق في مثال AEAD التالي وتأكد من فشل فك التشفير. تكرار nonce خطأ استخدام خطير، لكن المكتبة لا تحتفظ بسجل يمنعه تلقائيًا.

اربط النص المشفّر بسياقه باستخدام AEAD

`secretbox` بيتحقق من سلامة النص المشفّر، لكن مالوش معامل للبيانات المصاحبة. الـAEAD تشفير موثّق بيربط كمان سياقًا غير مشفّر بالنص: ملاحظة المستخدم 42 ماينفعش تُقرأ باعتبارها تخص المستخدم 99. السياق مش سر، وابنيه من معرّفات موثوقة وبصيغة ثابتة. مثال Sodium الكامل ده بيطبع `private note` وبعدها `wrong context rejected`.

```php
<?php
declare(strict_types=1);



function seal(string $plain, string $context, string $key): string
{
    $nonce = random_bytes(SODIUM_CRYPTO_AEAD_XCHACHA20POLY1305_IETF_NPUBBYTES);
    $cipher = sodium_crypto_aead_xchacha20poly1305_ietf_encrypt($plain, $context, $nonce, $key);
    return base64_encode($nonce . $cipher);
}

function openSealed(string $encoded, string $context, string $key): string
{
    $payload = base64_decode($encoded, true);
    $nonceBytes = SODIUM_CRYPTO_AEAD_XCHACHA20POLY1305_IETF_NPUBBYTES;
    if ($payload === false || strlen($payload) < $nonceBytes + SODIUM_CRYPTO_AEAD_XCHACHA20POLY1305_IETF_ABYTES) {
        throw new RuntimeException('Invalid encrypted payload');
    }
    $plain = sodium_crypto_aead_xchacha20poly1305_ietf_decrypt(
        substr($payload, $nonceBytes), $context, substr($payload, 0, $nonceBytes), $key,
    );
    if ($plain === false) {
        throw new RuntimeException('Authentication failed');
    }
    return $plain;
}

$key = sodium_crypto_aead_xchacha20poly1305_ietf_keygen();
$encoded = seal("private note", "user:42:note:v1", $key);
echo openSealed($encoded, "user:42:note:v1", $key), PHP_EOL;
try {
    openSealed($encoded, "user:99:note:v1", $key);
} catch (RuntimeException) {
    echo "wrong context rejected", PHP_EOL;
}
```

ولّد nonce عشوائيًا جديدًا من 24 بايت لكل تشفير بالخوارزمية دي. ضمان عدم تكراره مسؤولية المستدعي؛ التكرار مش لازم يرمي خطأ. حاوية المثال التعليمية مافيهاش بروتوكول تدوير مفاتيح أو Key ID؛ أضف صيغة محددة الإصدار ومتحققًا منها قبل التخزين الدائم في الإنتاج.

تحويل HTTP إلى HTTPS مش هيحمي بيانات Basic اللي اتبعتت بالفعل في أول طلب HTTP. اضبط HTTPS عند العميل قبل إرفاق بيانات الاعتماد، وافرض TLS عند مدخل الخدمة، وما تمررش Authorization لدومين آخر عبر التحويلات. HSTS بيساعد المتصفحات اللي تعرف السياسة أو عندها preload؛ مش بيعالج تسريب حصل بالفعل.

PHP XChaCha20-Poly1305

7. المصادقة والصلاحيات عمليًا

Login آمن وRBAC والسياسات وملكية الموارد ومنع IDOR وأقل قدر من الصلاحيات.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **HTTP:** قواعد تبادل الطلبات والردود بين المتصفح والخادم.
- **URL:** العنوان الكامل لمورد على الويب، زي صفحة أو صورة أو نقطة API.
- **Session:** بيانات مؤقتة تساعد الخادم يميّز المستخدم بين أكثر من طلب.
- **Queue:** طابور مهام تنتظر عاملًا ينفذها في الخلفية.
- **CLI:** واجهة تتعامل معها بكتابة أوامر نصية بدل الضغط على أزرار.
- **Function:** دالة: جزء كود له اسم ومهمة محددة ويمكن استدعاؤه أكثر من مرة.

ثلاثة أسئلة مختلفة

1. **Authentication:** من المستخدم؟
2. **Authorization:** هل يُسمح له بهذا الفعل على هذا المورد؟
3. **Accounting/Auditing:** ماذا حدث، ومتى، ومن أي جلسة؟

نجاح تسجيل الدخول لا يعني السماح بكل شيء. وإخفاء الزر في الواجهة ليس Authorization.

Login flow

```
Validate input
→ Load account by normalized identifier
→ Verify password
→ Apply rate/risk controls
→ Regenerate session ID
→ Store minimal identity state
→ Redirect
```

أعد رسالة عامة مثل «بيانات الدخول غير صحيحة» لتقليل كشف وجود الحساب. لا تجعل كل الردود متطابقة زمنيًا بصورة مثالية على حساب DoS، لكن تجنب الفروق الفاضحة. سجّل الأحداث دون passwords أو tokens.

RBAC

```
enum Role: string
{
    case Admin = 'admin';
    case Editor = 'editor';
    case Viewer = 'viewer';
}

function canPublish(array $user): bool
{
    return in_array($user['role'], [
        Role::Admin->value,
        Role::Editor->value,
    ], true);
}
```

RBAC مناسب للصلاحيات العامة، لكنه لا يحل ملكية الموارد.

Ownership وPolicies

```
function canUpdatePost(array $user, array $post): bool
{
    return $user['role'] === 'admin'
        || (
            $user['role'] === 'editor'
            && $post['author_id'] === $user['id']
        );
}
```

الأفضل تحميل المورد مقيدًا بالمستخدم عندما يناسب:

```
SELECT * FROM posts
WHERE id = :post_id AND author_id = :user_id
```

هذا يقلل IDOR/BOLA حيث يغيّر المستخدم ID في URL للوصول إلى مورد غيره.

Middleware لا يلغي الفحص داخل المجال

Middleware يتحقق من وجود مستخدم أو صلاحية عامة. أما قاعدة مثل «لا يمكن رد الطلب بعد الشحن» فتنتمي لخدمة المجال/Policy ويجب تطبيقها مهما كان المدخل HTTP أو CLI أو Queue.

حالات HTTP

- `401 Unauthorized`: المصادقة مفقودة/غير صالحة، رغم أن الاسم التاريخي مربك.
- `403 Forbidden`: الهوية معروفة لكن الفعل غير مسموح.

- `404 Not Found`: قد يُستخدم لإخفاء وجود مورد لا يحق للمستخدم معرفته.

Defense in depth

- Least privilege لحسابات DB والخدمات.
- Deny by default.
- تحقق server-side في كل request.
- MFA وstep-up auth للعمليات الحساسة.
- Audit logs محمية من التعديل.
- إبطال الجلسات عند تغيير كلمة المرور أو الدور.
- اختبارات Authorization: مستخدم صحيح، دور خاطئ، مورد يملكه غيره، ID غير موجود.

سيناريو أمني

▼ ما الفرق بين authentication وauthorization؟

الأول يثبت من أنت، والثاني يقرر ماذا يسمح لك فعله على المورد المحدد.

تدريب تهديد

السيناريو: مستخدم مسجّل الدخول يغيّر معرّف مورد في الطلب ليقرأ فاتورة مستخدم آخر.

اختبار المنع: نفّذ الطلب بحساب يملك المورد، ثم بحساب آخر صحيح المصادقة لكنه لا يملكه.

النتيجة المتوقعة: ينجح طلب المالك ويُرفض الآخر بـ403 أو 404 وفق السياسة، ولا تعتمد الصلاحية على إخفاء الزر في الواجهة.

مرجع التحقق

- OWASP Authorization Cheat Sheet

اربط النقاط ببعض

ابن authorization matrix فيها actor وresource وaction وcontext وexpected decision، ثم حولها لاختبارات allow وdeny. لا يكفي middleware عام إذا تغيرت ملكية المورد داخل domain operation. سجّل القرار وpolicy version دون تسريب بيانات المورد، واختبر direct object reference.

جرّب بنفسك

نفذ المصفوفة على HTTP وjob خلفية وتأكد من القرار نفسه.

8. JWT والتحقق الآمن

بنية JWT والتوقيع والـClaims والتحقق من issuer وaudience والوقت وإدارة المفاتيح.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **URL:** العنوان الكامل لمورد على الويب، زي صفحة أو صورة أو نقطة API.
- **API:** واجهة محددة تسمح لبرنامج يطلب بيانات أو ينفّذ عملية عند برنامج آخر.
- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.
- **Scope:** نطاق التفويض: اسم صلاحية يطلبها التطبيق أو يحصل عليها، زي orders:read لقراءة الطلبات؛ ولسه لازم نتحقق إن المستخدم يملك الطلب.
- **Authorization:** التحقق من الصلاحية: تحديد العمليات المسموح للهوية تنفذها.
- **XSS:** هجوم يحاول تشغيل JavaScript غير موثوق داخل صفحة المستخدم.
- **CSRF:** هجوم يدفع متصفح مستخدم مسجل الدخول لإرسال طلب لم يقصده.
- **JWT:** صيغة Token موقعة؛ التوقيع يكشف التعديل لكنه لا يشفر المحتوى تلقائيًا.
- **Secret:** قيمة حساسة مثل مفتاح API أو كلمة مرور خدمة ولازم تبقى خارج الكود.

JWT ليست جلسة سحرية

JSON Web Token صيغة compact لنقل Claims بين أطراف. الشكل الشائع JWS مكوّن من ثلاثة أجزاء Base64url مفصولة بنقطة:

```
BASE64URL(header).BASE64URL(payload).BASE64URL(signature)
```

- Header: نوع التوكين والخوارزمية وKey ID اختياري.
- Payload: Claims مثل المستخدم والمصدر والجمهور ووقت الانتهاء.
- Signature: تكشف تعديل header أوpayload إذا تم التحقق منها بالمفتاح الصحيح.

Base64url لا يخفي البيانات

يمكن لأي شخص يحمل JWT قراءة header وpayload. التوقيع يوفر السلامة والأصالة، وليس السرية. لا تضع كلمة مرور أو Secret أو بيانات شخصية غير لازمة. التشفير يحتاج JWE أو تصميمًا آخر، وليس JWT موقّعة فقط.

Claims قياسية مهمة

Claim	المعنى	التحقق
`iss`	الجهة المصدرة	تطابق issuer موثوقًا بالضبط
`sub`	موضوع/هوية التوكين	لا تفترض أنه email
`aud`	الجمهور المقصود	يجب أن تتضمن API الحالية
`exp`	انتهاء الصلاحية	ارفض المنتهي
`nbf`	غير صالح قبل	ارفض الاستخدام المبكر
`iat`	وقت الإصدار	افحص المعقولية حسب السياسة
`jti`	معرف فريد	يفيد في التتبع/منع إعادة الاستخدام

الـPayload قد يحتوي أدوارًا أو scopes، لكن تغيّر صلاحية المستخدم بعد إصدار token لن يظهر حتى انتهاء التوكين أو فحص مصدر مركزي. لذلك اجعل access tokens قصيرة العمر ولا تعتبر JWT بديلًا دائمًا لسياسة Authorization.

التوقيع المتناظر وغير المتناظر

- HMAC : `HS256` بمفتاح سري مشترك؛ كل جهة تتحقق تستطيع أيضًا إصدار token.
- `RS256`/أمثاله: Private key للتوقيع وPublic key للتحقق؛ أنسب عندما تتحقق خدمات كثيرة.
- `EdDSA` بخوارزمية مدعومة ومكتبة موثوقة خيار حديث في البيئات المتوافقة.

لا تختَر الخوارزمية من قيمة `alg` داخل token وحدها. ثبّت allowlist في verifier ولا تقبل `none` . لا تستخدم كلمة مرور بشرية كمفتاح HMAC.

مثال PHP بمكتبة

```
composer require firebase/php-jwt
```

```
<?php
declare(strict_types=1);

use Firebase\JWT\JWT;
use Firebase\JWT\Key;

$payload = [
    'iss' => 'https://id.example.com',
    'sub' => 'user_123',
    'aud' => 'https://api.example.com',
    'iat' => time(),
    'nbf' => time(),
    'exp' => time() + 300,
    'scope' => 'orders:read',
];

$token = JWT::encode($payload, $privateKey, 'RS256', 'key-2026-09');
$claims = (array) JWT::decode($token, new Key($publicKey, 'RS256'));

validateAccessClaims($claims, 'https://id.example.com', 'https://api.example.com', time());

if (($claims['iss'] ?? null) !== 'https://id.example.com') {
    throw new RuntimeException('Invalid issuer');
}

$audiences = (array) ($claims['aud'] ?? []);
if (!in_array('https://api.example.com', $audiences, true)) {
    throw new RuntimeException('Invalid audience');
}
```

المكتبة تتعامل مع التوقيع وبعض claims الزمنية، لكن التطبيق ما زال مسؤولًا عن issuer وaudience ونوع token والسياسة. استخدم مكتبة ناضجة بدل بناء Base64/signature يدويًا.

JWKS وKey ID

`kid` يسهّل rotation، لكنه مدخل غير موثوق:

1. اربط issuer بقائمة مفاتيح/JWKS URL موثوقة مسبقًا.
2. لا تستخدم `kid` كاسم ملف أو SQL بلا تحقق.
3. لا تتبع `jku` أو URL يرسله التوكين عشوائيًا؛ هذا قد يخلق SSRF أو مفتاح مهاجم.
4. خزّن JWKS مؤقتًا مع refresh محدود، واحتفظ بالمفتاح القديم حتى تنتهي التوكينات الموقعة به.

قائمة تحقق

- HTTPS دائمًا.
- حدد allowed algorithms و `typ` token.
- تحقق من signature و `iss` و `aud` و `exp` و `nbf` .
- امنح clock skew صغيرًا ومحددًا، لا ساعات.

- افصل مفاتيح وأنواع ID tokens عن access tokens لمنع substitution.
- لا تسجل التوكين كاملًا.
- خطط للإلغاء: مدة قصيرة، denylist لحالات محددة، أو opaque/reference token عندما تحتاج revocation فوريًا.

سيناريو أمني

▼ ليه ما نحطش بيانات سرية في JWT؟

الـpayload غالبًا encoded لا encrypted ويمكن لحامل token قراءته؛ قلل البيانات وحدد العمر والجمهور.

تدريب تهديد

السيناريو: يعدّل المهاجم header أو claim داخل JWT أو يعيد استخدام توكين من issuer أو audience مختلف.

اختبار المنع: اختبر توقيعًا معدلًا و`alg` غير متوقع و`aud` خاطئًا وتوكينًا منتهيًا.

النتيجة المتوقعة: تُرفض كل الحالات قبل تنفيذ منطق العمل؛ الخوارزمية والمصدر والجمهور والعمر قيود من إعداد الخادم لا من التوكين وحده.

مرجع التحقق

- OWASP JSON Web Token Cheat Sheet

اربط النقاط ببعض

JWT access token لا يملك revocation لحظية بذاته؛ استخدم عمرًا قصيرًا وrotation أو introspection/denylist حسب الخطر. حدد مكان التخزين في browser ونموذج XSS/CSRF. ثبّت algorithm وissuer وaudience من config، واضبط JWKS caching والrotation، ولا تقبل nested/encrypted token دون حاجة ومكتبة تدعمها.

جرّب بنفسك

دوّر signing key وتأكد من قبول القديم في النافذة ورفضه بعدها.

وجود الـClaims نفسه جزء من العقد

المثال السابق مقتطف دمج: حمّل Composer autoload ووفر مفاتيح RSA مدارة. أضف الدالة التالية لنفس الملف واستدعها بعد تحقق المكتبة من التوقيع بمفتاح موثوق وخوارزمية ثابتة. سياسة access token هنا بتشترط `iss` و`sub` و`aud` و`exp` كعدد صحيح؛ معيار JWT نفسه مش بيشترط كل claim مسجلة. المكتبة ممكن تتحقق من الانتهاء فقط لو الحقل موجود. `nbf` اختياري هنا لكن لازم يكون عددًا صحيحًا لو موجود. المثال لا يسمح بفارق توقيت؛ لو بيئتك محتاجاه حدده بوضوح وبحد صغير.

```
function validateAccessClaims(array $claims, string $issuer, string $audience, int $now): void
{
    // Only call AFTER a JOSE library has verified the signature and fixed algorithm.
    if (($claims['iss'] ?? null) !== $issuer
        || !is_string($claims['sub'] ?? null) || $claims['sub'] === ''
        || !is_int($claims['exp'] ?? null) || $claims['exp'] <= $now) {
        throw new InvalidArgumentException('Missing or invalid required claim');
    }
    $aud = $claims['aud'] ?? null;
    if (is_string($aud)) {
        $aud = [$aud];
    }
    if (!is_array($aud) || !array_is_list($aud) || $aud === []
        || count(array_filter($aud, 'is_string')) !== count($aud)
        || !in_array($audience, $aud, true)) {
        throw new InvalidArgumentException('Invalid audience');
    }
    if (array_key_exists('nbf', $claims)
        && (!is_int($claims['nbf']) || $claims['nbf'] > $now)) {
        throw new InvalidArgumentException('Invalid not-before claim');
    }
}
```

اختبر توكينًا موقّعًا فعلًا لكن المصدر أنشأه من غير `exp`: لازم يُرفض بسبب السياسة. حذف الحقل من توكين موقّع موجود بيختبر العبث بالتوقيع، وده اختبار مختلف. اختبارات `security` المحلية بتفحص السياسة، مش تنفيذ التوقيع في المكتبة الخارجية.

9. Federated Identity وSSO وSAML وOIDC

الهوية الاتحادية وتسجيل الدخول الموحد وأدوار IdP وSP ومقارنة SAML مع OpenID Connect.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **API:** واجهة محددة تسمح لبرنامج يطلب بيانات أو ينفّذ عملية عند برنامج آخر.
- **Session:** بيانات مؤقتة تساعد الخادم يميّز المستخدم بين أكثر من طلب.
- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.
- **Scope:** نطاق التفويض: اسم صلاحية يطلبها التطبيق أو يحصل عليها، زي orders:read لقراءة الطلبات؛ ولسه لازم نتحقق إن المستخدم يملك الطلب.
- **OAuth:** بروتوكول تفويض يمنح تطبيقًا صلاحية محددة من غير تسليمه كلمة سر المستخدم.
- **OIDC:** طبقة هوية فوق OAuth تضيف طريقة موحدة لمعرفة من سجل الدخول.
- **SAML:** معيار لتبادل بيانات تسجيل الدخول بين جهة هوية وخدمة.

الهوية الاتحادية ليست بروتوكولًا واحدًا

Federated Identity علاقة ثقة تسمح لتطبيق بالاعتماد على جهة خارجية لإثبات هوية المستخدم. **Single Sign-On (SSO)** تجربة يستطيع فيها المستخدم الوصول إلى عدة تطبيقات بعد عملية دخول مركزية، لكنه قد يُنفذ داخل مؤسسة أو عبر اتحاد هويات.

```
User -> Application / Service Provider
        -> Identity Provider
        <- signed identity result
     <- local session
```

التطبيق لا يشارك كلمة مرور المستخدم مع كل خدمة؛ يثق في نتيجة موقعة ومقيدة بجمهور ووقت محددين.

OAuth وOIDC وSAML

التقنية	الهدف الأساسي	النتيجة المعتادة
OAuth 2.0	تفويض وصول عميل إلى Resource Server	Access token
OpenID Connect	مصادقة مبنية فوق OAuth 2.0	ID token + معلومات هوية
SAML 2.0	تبادل Assertions للهوية/الصفات، شائع في Enterprise SSO	SAML Response/Assertion XML

التصحيح الأهم

OAuth وحده ليس بروتوكول Login. استخدام access token وendpoint عشوائي لاستنتاج هوية المستخدم يسمى أحيانًا pseudo-authentication وقد يؤدي لخلط المستخدم أو الجمهور. استخدم OIDC للمصادقة.

SAML Web Browser SSO

الأطراف الرئيسية:

- **Identity Provider (IdP):** يصادق المستخدم ويصدر assertion.
- **Service Provider (SP):** التطبيق الذي يستهلك النتيجة وينشئ جلسة محلية.
- **Assertion:** XML موقّع يحوي subject وconditions وattributes.

تدفق SP-initiated مبسط:

```
Browser -> SP: protected page
SP -> Browser -> IdP: AuthnRequest
IdP authenticates user
IdP -> Browser -> SP ACS: SAMLResponse
SP validates response/assertion
SP creates local session
```

التحقق يجب أن يشمل XML signature من شهادة IdP الموثوقة، `Recipient/Destination`، `AudienceRestriction`، `Issuer`، حدود الزمن، و`InResponseTo` عند استخدام request. امنع replay وسجل assertion ID. استخدم مكتبة SAML ناضجة وعطّل external entities؛ XML Signature معقدة ولا تُنفذ يدويًا.

OpenID Connect

OIDC يضيف طبقة هوية إلى OAuth. وجود `openid` داخل `scope` يحول الطلب إلى طلب OIDC:

```
scope=openid profile email
```

النواتج:

- **ID token:** موجه إلى الـClient ويخبره بنتيجة المصادقة.
- **Access token:** موجه إلى API/Resource Server؛ لا تستخدم ID token بدلًا منه.
- **UserInfo endpoint:** معلومات إضافية اختيارية باستخدام access token.

Claims شائعة: `sub` و`iss` و`aud` و`exp` و`auth_time`، ومع scopes مناسبة قد تظهر `name` و`email`. الـscope طلب لمجموعة claims/صلاحيات وليس ضمانًا أن كل قيمة موجودة.

تحقق ID token

1. اكتشف metadata من issuer موثوق مسبقًا.
2. تحقق من التوقيع عبر JWKS لهذا issuer.
3. طابق `iss` حرفيًا و`aud` مع `client_id`.
4. افحص `exp` و`iat` و`nbf` إن وُجد.
5. طابق `nonce` مع المعاملة التي بدأت في المتصفح.
6. طبّق قواعد `azp` عندما توجد جماهير متعددة.
7. إذا استخدمت UserInfo، يجب أن يطابق `sub` قيمة ID token.

`email` قابل للتغيير وقد لا يكون verified؛ استخدم `iss + sub` كمفتاح هوية خارجي ثابت.

متى أختار ماذا؟

- تكامل حديث لتطبيقات Web/Mobile وAPIs: OIDC غالبًا.
- Enterprise قديم/مؤسسات تعتمد XML وSAML 2.0: SAML metadata.
- منح تطبيق حق الوصول إلى API دون الحاجة لهوية مستخدم: OAuth 2.0.

ليس SAML "غير آمن لأنه قديم" ولا OIDC آمنًا تلقائيًا. الخطر في validation ناقص، trust غير مضبوط، redirect غير دقيق، أو مكتبة غير محدثة.

سيناريو أمني

▼ مين يثبت هوية المستخدم في federation؟

Identity Provider يصادق المستخدم، وRelying Party يتحقق من assertion أو ID token ثم ينشئ جلسته.

تدريب تهديد

السيناريو: تصل استجابة هوية صحيحة التوقيع لكنها تخص Client أو جلسة تسجيل دخول مختلفة.

اختبار المنع: اختبر `audience` مختلفًا و `nonce` قديمًا وredirect URI غير مسجل.

النتيجة المتوقعة: تُرفض الحالات كلها، وتُربط الاستجابة بالطلب المحلي قبل إنشاء الجلسة.

مرجع التحقق

- OpenID Connect Core 1.0

اربط النقاط ببعض

في SAML تحقق من metadata وentity ID وDestination وAudience وInResponseTo والتوقيت والتوقيع في الموضع المتوقع لتجنب XML signature wrapping؛ لا تكتب XML security يدويًا. في OIDC استخدم discovery/JWKS الموثوقين وثبت issuer وclient ID وredirect URI واضبط clock skew بحد صغير.

جرّب بنفسك

اختبر assertion موقعة لكن بعنصر مختلف وID token من issuer آخر.

10. مفاهيم OAuth 2.0 والتوكينات

التفويض المفوض والأدوار والـScopes والـConsent وأنواع Clients وAccess وRefresh Tokens.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **HTTP:** قواعد تبادل الطلبات والردود بين المتصفح والخادم.
- **TLS:** طبقة تشفير بتحمي البيانات وهي ماشية بين طرفين.
- **API:** واجهة محددة تسمح لبرنامج يطلب بيانات أو ينفّذ عملية عند برنامج آخر.
- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.
- **Scope:** نطاق التفويض: اسم صلاحية يطلبها التطبيق أو يحصل عليها، زي orders:read لقراءة الطلبات؛ ولسه لازم نتحقق إن المستخدم يملك الطلب.
- **Authentication:** التحقق من الهوية: التأكد إن المستخدم هو فعلًا صاحب الحساب.
- **Authorization:** التحقق من الصلاحية: تحديد العمليات المسموح للهوية تنفذها.
- **JWT:** صيغة Token موقعة؛ التوقيع يكشف التعديل لكنه لا يشفر المحتوى تلقائيًا.
- **OAuth:** بروتوكول تفويض يمنح تطبيقًا صلاحية محددة من غير تسليمه كلمة سر المستخدم.
- **OIDC:** طبقة هوية فوق OAuth تضيف طريقة موحدة لمعرفة من سجل الدخول.
- **Secret:** قيمة حساسة مثل مفتاح API أو كلمة مرور خدمة ولازم تبقى خارج الكود.

ما المشكلة التي يحلها OAuth؟

OAuth 2.0 إطار **تفويض مفوض**: يمنح المستخدم أو النظام Client وصولًا محدودًا إلى Resource Server دون تسليم كلمة مرور المستخدم للـClient.

مثال: تطبيق صور يطلب قراءة ملفات محددة من خدمة تخزين. المستخدم يصادق نفسه لدى خدمة التخزين، يوافق على النطاق، ويحصل التطبيق على access token محدود بدل كلمة المرور.

الأدوار الأربعة

1. **Resource Owner:** الجهة القادرة على منح الوصول، وغالبًا المستخدم.
2. **Client:** التطبيق الذي يطلب الوصول؛ ليس "المستخدم".
3. **Authorization Server:** يصادق الأطراف حسب الحاجة ويصدر التوكينات بعد التفويض.
4. **Resource Server:** الـAPI التي تستقبل access token وتحمي الموارد.

```
Resource Owner -> Authorization Server: approve
Client -> Authorization Server: grant/code
Authorization Server -> Client: access token
Client -> Resource Server: token + API request
```

قد تكون Authorization Server وResource Server جزءًا من منتج واحد أو خدمتين مختلفتين.

Client registration

يسجل العميل عادة:

- `client_id`: معرف عام، **ليس سرًا**.
- Redirect URIs دقيقة.
- نوع العميل وطرق authentication المسموحة.
- scopes/grants المسموحة وسياسات consent.

Public وConfidential

- Public client: لا يستطيع حفظ secret، مثل native app أو JavaScript يعمل في المتصفح. تضمين `client_secret` في bundle لا يجعله سريًا.
- Confidential client: backend قادر على حماية credentials ويصادق نفسه عند token endpoint.

اسم النوع يصف قدرة حفظ السر، وليس ثقة أخلاقية أو مستوى صلاحية تلقائيًا.

Consent وScope

`scope` اسم صلاحية يطلبها Client:

```
orders:read orders:write profile
```

صمم scopes حول capabilities واضحة وبأقل صلاحية. لا تجعل scope `admin` عامًا يختصر عشرات العمليات الحساسة بلا سبب.

Consent واجهة/قرار يمنح فيه Resource Owner وصولًا. لكنه ليس بديلًا عن سياسة Authorization Server؛ قد ترفض المؤسسة scope حتى لو وافق المستخدم، وقد يكون consent سابقًا أو إداريًا في machine-to-machine.

Access token

Credential يقدمها Client إلى Resource Server:

```
Authorization: Bearer eyJ...
```

- قد يكون opaque string أو JWT؛ OAuth لا يفرض صيغة واحدة.
- يجب تقييده بالمدة والجمهور والموارد والإجراءات.
- Bearer يعني أن من يحمله يمكنه استخدامه، لذلك خزنه وانقله كسر.
- API يجب أن تتحقق أنه موجه إليها وأن scope يسمح بالفعل، لا أن signature صحيحة فقط.

Refresh token

Credential طويل الحساسية يستخدم عند token endpoint للحصول على access token جديد. لا يُرسل إلى Resource Server.

```
Client --refresh token--> Authorization Server
Client <--new access token (+ rotated refresh token)--
```

لـpublic clients استخدم refresh-token rotation مع reuse detection أو sender-constraining حسب النظام. عند اكتشاف إعادة استخدام token قديم، ألغِ family كاملة واطلب تسجيل دخول جديدًا.

OAuth token ليست API key

API key غالبًا تعرّف project/client وتستخدم للحصة أو خدمة بسيطة، ولا تمثل بالضرورة مستخدمًا أو consent أو scopes ديناميكية. تعامل معها كسر، لكن لا تسمها access token ولا تستخدمها لتسجيل دخول مستخدم.

Token endpoint response

```
{
  "access_token": "opaque-or-structured-value",
  "token_type": "Bearer",
  "expires_in": 300,
  "refresh_token": "long-lived-secret",
  "scope": "orders:read"
}
```

الاستجابة حساسة: استخدم TLS، أرسل `Cache-Control: no-store`، ولا تسجل الجسم. `expires_in` مدة من لحظة الاستلام؛ لا تفترض أن access token JWT أو تحاول parse opaque token.

أخطاء مفاهيمية

- OAuth لا يثبت الهوية للتطبيق؛ OIDC يفعل.
- JWT صيغة token وليست بديلًا عن OAuth.
- Client ID ليس secret.
- Scope ليس Role داخليًا بالضرورة.
- Access token لا يذهب إلى صفحة login، وRefresh token لا يذهب إلى API.
- امتلاك token صالح لا يلغي فحص ملكية المورد وقواعد المجال.

سيناريو أمني

▼ هل access token دليل login للتطبيق؟

ليس بالضرورة؛ هو تفويض للوصول إلى API. استخدم OIDC وID token محققًا لإثبات هوية المستخدم.

تدريب تهديد

السيناريو: يحاول عميل استخدام access token لخدمة أخرى أو بصلاحية أوسع مما مُنح له.

اختبار المنع: أرسل توكينًا بجمهور خاطئ، ثم توكينًا صحيحًا يفتقد الـscope المطلوب.

النتيجة المتوقعة: تُرفض الأولى كتوكين غير صالح والثانية كصلاحية غير كافية، ولا يُعامل access token كدليل تسجيل دخول للمستخدم.

مرجع التحقق

- RFC 9700: OAuth 2.0 Security Best Current Practice

اربط النقاط ببعض

Resource indicator أو audience يحدد API المقصودة ويمنع توكين خدمة من العمل في أخرى. Introspection مناسب للتوكين opaque أو القرار المركزي، وrevocation endpoint يلغي grant حسب السياسة مع توقع propagation delay. Consent ليس بديلًا لسياسة أقل صلاحية، وscope ليس role داخليًا تلقائيًا.

جرّب بنفسك

أرسل توكينًا لنفس issuer لكن resource مختلفًا وتأكد من الرفض.

11. OAuth Flows الحديثة

Authorization Code مع PKCE وClient Credentials وDevice Authorization وRefresh Tokens والتدفقات القديمة.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **HTTP:** قواعد تبادل الطلبات والردود بين المتصفح والخادم.
- **Session:** بيانات مؤقتة تساعد الخادم يميّز المستخدم بين أكثر من طلب.
- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.
- **CLI:** واجهة تتعامل معها بكتابة أوامر نصية بدل الضغط على أزرار.
- **Scope:** نطاق التفويض: اسم صلاحية يطلبها التطبيق أو يحصل عليها، زي orders:read لقراءة الطلبات؛ ولسه لازم نتحقق إن المستخدم يملك الطلب.
- **Function:** دالة: جزء كود له اسم ومهمة محددة ويمكن استدعاؤه أكثر من مرة.
- **Authorization:** التحقق من الصلاحية: تحديد العمليات المسموح للهوية تنفذها.
- **OAuth:** بروتوكول تفويض يمنح تطبيقًا صلاحية محددة من غير تسليمه كلمة سر المستخدم.
- **OIDC:** طبقة هوية فوق OAuth تضيف طريقة موحدة لمعرفة من سجل الدخول.
- **SAML:** معيار لتبادل بيانات تسجيل الدخول بين جهة هوية وخدمة.
- **MFA:** تحقق بأكثر من عامل مستقل، مثل كلمة سر وجهاز يملكه المستخدم.
- **Secret:** قيمة حساسة مثل مفتاح API أو كلمة مرور خدمة ولازم تبقى خارج الكود.

اختر التدفق حسب نوع العميل

```
مستخدم + Browser/Web/Mobile  -> Authorization Code + PKCE
خدمة تتحدث مع خدمة           -> Client Credentials
بلا متصفح مناسب TV/CLI       -> Device Authorization
تجديد وصول سابق              -> Refresh Token
```

Authorization Code + PKCE

هذا هو الاختيار الأساسي لتطبيقات الويب وSPA وNative Apps. الكود يمر عبر الـFront Channel لكنه قصير العمر وأحادي الاستخدام، أما التوكينات فتعود من Token Endpoint عبر اتصال مباشر.

```
Browser        Client            Authorization Server
  |              |                          |
  |--- start --->|                          |
  |              |-- authorize + challenge|
  |<------------- redirect to login -------|
  |------------- login/consent ----------->|
  |<---- redirect: code + state ------------|
  |-- callback -->|                          |
  |              |-- code + verifier ------>|
  |              |<-- access/refresh token--|
```

إنشاء PKCE

```php
<?php
declare(strict_types=1);

function base64Url(string $bytes): string
{
    return rtrim(strtr(base64_encode($bytes), '+/', '-_'), '=');
}

$codeVerifier = base64Url(random_bytes(32)); // 43 chars
$codeChallenge = base64Url(hash('sha256', $codeVerifier, true));
$state = base64Url(random_bytes(32));
$nonce = base64Url(random_bytes(32)); // عند استخدام OIDC

$_SESSION['oauth'] = [
    'state' => hash('sha256', $state),
    'verifier' => $codeVerifier,
    'nonce' => $nonce,
    'created_at' => time(),
];
```

طلب التفويض:

```
response_type=code
client_id=...
redirect_uri=https://client.example.com/callback
scope=openid profile orders:read
state=...
nonce=...
code_challenge=...
code_challenge_method=S256
```

عند callback طابق state بقيمة session مقارنة ثابتة الزمن واحذف المعاملة بعد الاستخدام. ثم أرسل code و `code_verifier` وredirect URI نفسها إلى token endpoint. الـAuthorization Server يحسب:

```
BASE64URL(SHA256(code_verifier)) == code_challenge
```

PKCE ليس تشفيرًا للكود؛ يربط الكود بعميل بدأ المعاملة. استخدم قيمة جديدة عالية entropy لكل محاولة و `S256` فقط.

Client Credentials

لـmachine-to-machine عندما يعمل Client بصفته، لا بصفة مستخدم:

```
POST /oauth/token HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Authorization: Basic BASE64(client_id:client_secret)

grant_type=client_credentials&scope=reports:write
```

- للـConfidential clients فقط.
- لا يوجد user أو consent تفاعلي؛ لا تخترع `user_id` لهذا token.
- فضّل مصادقة أقوى مثل mTLS أو `private_key_jwt` في الأنظمة الحساسة.
- اجعل كل workload بهوية وصلاحيات مستقلة بدل secret مشترك بين الخدمات كلها.

Device Authorization

مناسب لـSmart TV أو CLI أو جهاز إدخاله محدود:

1. الجهاز يطلب `device_code` و `user_code` و `verification_uri`.
2. يعرض للمستخدم الرابط والكود، ويفضل QR لا يحتوي أسرارًا غير مطلوبة.
3. المستخدم يفتح الرابط على جهاز آخر ويسجل الدخول ويوافق.
4. الجهاز يعمل polling للـtoken endpoint.

```
{
  "device_code": "secret-device-value",
  "user_code": "WDJB-MJHT",
  "verification_uri": "https://id.example.com/device",
  "expires_in": 600,
  "interval": 5
}
```

احترم `interval`، ومع `slow_down` زد فترة polling خمس ثوانٍ، وتوقف عند `access_denied` أو `expired_token`. اعرض اسم الجهاز والعميل للمستخدم لتقليل هجمات خداع الأكواد.

Refresh Token Grant

```
POST /oauth/token HTTP/1.1
Content-Type: application/x-www-form-urlencoded

grant_type=refresh_token&
refresh_token=...&
client_id=...
```

لا تطلب scope أوسع من الأصلي. استخدم rotation: كل استبدال يصدر refresh token جديدًا ويلغي السابق، مع كشف reuse.

Assertion Grants

يمكن استخدام SAML assertion كAuthorization Grant أو لمصادقة Client وفق RFC 7522. هذا **امتداد تكاملي** بين اتحاد SAML وOAuth وليس التدفق الافتراضي لكل SAML SSO. تحقق من issuer وaudience والوقت والتوقيع ومنع replay.

تدفقات لا تبدأ بها مشروعًا جديدًا

تحذير

- **Implicit Grant:** يعيد access token عبر front channel حيث يزداد التسريب وإعادة الاستخدام. استخدم Authorization Code + PKCE.
- **Resource Owner Password Credentials:** يعلّم المستخدم إدخال كلمة مروره في Client ويصعّب MFA وWebAuthn؛ RFC 9700 يقول إنه يجب عدم استخدامه.

سيناريو أمني

▼ ماذا يمنع PKCE؟

يربط authorization code بعميل بدأ التدفق، فيصعب على معترض الكود استبداله من غير verifier.

تدريب تهديد

السيناريو: يعترض المهاجم authorization code لتطبيق عام ويحاول استبداله قبل العميل الأصلي.

اختبار المنع: ابدأ تدفق PKCE ثم حاول استبدال الكود بلا `code_verifier` أو بقيمة لا تطابق challenge.

النتيجة المتوقعة: يفشل الاستبدال في الحالتين، ويُقبل فقط verifier الأصلي مع redirect URI مطابق تمامًا.

مرجع التحقق

- RFC 7636: Proof Key for Code Exchange

اربط النقاط ببعض

استخدم PKCE S256 ولا تسمح plain عند وجود اختيار آمن، واربط code بـclient وredirect URI وDevice flow معرض verifier. في user code؛ اعرض الخدمة والعميل بوضوح وطبق interval وexpiry. دوّر refresh token واكشف reuse لإلغاء العائلة، ولا تستخدم implicit أوpassword grant جديدًا. phishing

جرّب بنفسك

اختبر verifier خاطئًا وdevice code منتهيًا وrefresh token معاد الاستخدام.

12. أمان OAuth وOIDC

State وNonce وPKCE وRedirect URI وتخزين التوكينات وBFF والإبطال ومراقبة التكامل.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **HTTP:** قواعد تبادل الطلبات والردود بين المتصفح والخادم.
- **URL:** العنوان الكامل لمورد على الويب، زي صفحة أو صورة أو نقطة API.
- **Cache:** نسخة مؤقتة من البيانات هدفها تقليل وقت الانتظار والعمل المتكرر.
- **Session:** بيانات مؤقتة تساعد الخادم يميّز المستخدم بين أكثر من طلب.
- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.
- **Scope:** نطاق التفويض: اسم صلاحية يطلبها التطبيق أو يحصل عليها، زي orders:read لقراءة الطلبات؛ ولسه لازم نتحقق إن المستخدم يملك الطلب.
- **Authorization:** التحقق من الصلاحية: تحديد العمليات المسموح للهوية تنفذها.
- **XSS:** هجوم يحاول تشغيل JavaScript غير موثوق داخل صفحة المستخدم.
- **CSRF:** هجوم يدفع متصفح مستخدم مسجل الدخول لإرسال طلب لم يقصده.
- **JWT:** صيغة Token موقعة؛ التوقيع يكشف التعديل لكنه لا يشفر المحتوى تلقائيًا.
- **OAuth:** بروتوكول تفويض يمنح تطبيقًا صلاحية محددة من غير تسليمه كلمة سر المستخدم.
- **OIDC:** طبقة هوية فوق OAuth تضيف طريقة موحدة لمعرفة من سجل الدخول.
- **Secret:** قيمة حساسة مثل مفتاح API أو كلمة مرور خدمة ولازم تبقى خارج الكود.

طبقات حماية مختلفة

لا تستبدل `state` و`nonce` وPKCE ببعضها عشوائيًا:

الآلية	تحمي أساسًا من
`state` مرتبط بالجلسة	CSRF وتبديل معاملات callback
OIDC `nonce`	replay/substitution لنتيجة المصادقة
PKCE	سرقة أو حقن Authorization Code
Redirect URI exact match	إرسال النتيجة إلى وجهة مهاجم

وفق OAuth Security BCP، PKCE مطلوب للـpublic clients وموصى به أيضًا للـconfidential clients. استخدم قيمًا فريدة لكل معاملة واحذفها عند النجاح أو انتهاء مهلة قصيرة.

Callback آمن

```php
<?php
declare(strict_types=1);

session_start();

$tx = $_SESSION['oauth'] ?? null;
unset($_SESSION['oauth']); // one-time transaction

if (!is_array($tx) || time() - $tx['created_at'] > 600) {
    throw new RuntimeException('OAuth transaction expired');
}

$returnedState = $_GET['state'] ?? '';
if (
    !is_string($returnedState)
    || !hash_equals($tx['state'], hash('sha256', $returnedState))
) {
    throw new RuntimeException('Invalid state');
}

if (isset($_GET['error'])) {
    // اعرض رسالة آمنة، وسجّل code فقط دون التوكينات.
    throw new RuntimeException('Authorization was not completed');
}

$code = $_GET['code'] ?? null;
if (!is_string($code) || $code === '') {
    throw new RuntimeException('Missing authorization code');
}

// Exchange code once using tx['verifier'] and the exact redirect URI.
```

لا تسجل query كاملة لأن callback قد يحتوي code. ضع `Referrer-Policy: no-referrer` وتجنب third-party scripts في صفحة callback.

Redirect URIs

- سجل HTTPS URI كاملة وطابقها حرفيًا؛ لا تستخدم wildcard للإنتاج.
- اسمح loopback/custom URI وفق قواعد native apps الرسمية فقط.
- لا تجعل callback open redirect يأخذ `next=https://evil.example`.
- بعد callback استخدم destination مخزنة server-side أو allowlist لمسارات داخلية.
- اربط المعاملة بالـissuer لتجنب mix-up عندما يدعم Client أكثر من Authorization Server.

مكان التوكينات في تطبيق Browser

Backend for Frontend

```
Browser --Secure HttpOnly session cookie--> BFF
BFF --access token--> APIs
```

هذا الاختيار يقلل وصول JavaScript إلى التوكينات. طبّق CSRF protection وSameSite مناسبًا وsession rotation.

Browser-only client

إذا كان التوكين داخل المتصفح، فضّل memory على storage دائم طويل العمر. `localStorage` متاح لأي JavaScript داخل origin؛ XSS يستطيع سرقته. CSP وTrusted Types وتقليل third-party scripts دفاعات مهمة لكنها لا تجعل التخزين آمنًا مطلقًا.

لا تضع token في URL أو history أو logs أو analytics. ولا تنقل client secret إلى SPA.

Bearer مقابل Sender-constrained

Bearer token قابل لإعادة الاستخدام من أي حامل. للأنظمة الأعلى حساسية يمكن ربط التوكين بعميل عبر mTLS أو DPoP، لكن ذلك يزيد التعقيد ولا يلغي التحقق من audience/scope أو حماية الجهاز.

Access وRefresh lifecycle

- Access token قصيرة العمر ومقيّدة بالـresource/audience.
- Refresh token أطول عمرًا وتخزن بأقوى حماية متاحة.
- Rotate refresh tokens واكشف reuse.
- ألغِ العائلة عند logout أو سرقة أو تغيير أمني.
- وفر revocation endpoint أو introspection للـopaque tokens حيث يلزم.
- لا تفترض أن "JWT لا يمكن إلغاؤها" حقيقة مطلقة؛ يمكن denylist أو version/session checks، لكن لها تكلفة state.

OIDC login checklist

1. Discovery من issuer موثوق، لا من قيمة يرسلها المستخدم.
2. Authorization Code + PKCE S256.
3. PKCE/`state` و`nonce` مرتبطان بالجلسة والمعاملة.
4. Redirect URI مسجلة بدقة.
5. تحقق signature و`iss` و`aud` و`exp` و`nonce`.
6. استخدم `iss + sub` لربط الحساب.
7. لا تمنح صلاحيات محلية من email/domain claim دون سياسة واضحة والتحقق المطلوب.
8. أنشئ جلسة تطبيق جديدة ثم غيّر session ID.

المراقبة دون تسريب

سجّل: flow، client ID، issuer، نتيجة عامة، correlation ID، error code، ووقت الاستجابة. لا تسجل token أو code أو verifier أو secret أو assertion. راقب:

- ارتفاع `invalid_grant`.
- reuse لـRefresh Token.
- redirect/issuer mismatch.
- فشل signature أو audience.
- طلب scopes غير معتادة.
- معدل Device Flow polling وانتهاء الأكواد.

الخلاصة

استخدم مكتبة OIDC/OAuth مجرّبة وموفر هوية ناضج. البروتوكول يبدو redirects وHTTP parameters، لكن صحة الربط بين browser session وissuer وclient وredirect والتوكين هي الجزء الأمني الصعب.

سيناريو أمني

▼ ليه لازم نتحقق من state وnonce؟

state يربط response بالطلب ويقاوم CSRF، وnonce يربط ID token بالتدفق ويقاوم replay.

تدريب تهديد

السيناريو: يغيّر المهاجم redirect URI أو يعيد `state` من معاملة قديمة لخلط جلسات الدخول.

اختبار المنع: جرّب redirect URI قريبًا لكنه غير مطابق، وstate غير موجود، وstate سبق استهلاكه.

النتيجة المتوقعة: تُرفض الثلاثة، وتكون المطابقة حرفية ويُستهلك state مرة واحدة ويرتبط بمتصفح الطلب.

مرجع التحقق

- RFC 9700: OAuth 2.0 Security Best Current Practice

اربط النقاط ببعض

دافع عن mix-up بتثبيت issuer وربط response بالـauthorization server المتوقع، وعن code injection بربط code بـstate/nonce/PKCE/session. Sender-constrained tokens مثل DPoP أو mTLS تقلل قيمة التوكين المسروق لكنها تحتاج replay cache وkey lifecycle. لا تسجل authorization code أو token.

جرّب بنفسك

بدّل issuer داخل callback وأعد DPoP proof وتأكد من فشل الحالتين.

13. دورة حياة الحساب

التسجيل وتأكيد البريد واسترجاع كلمة المرور وتغيير الهوية وإعادة المصادقة وإلغاء الجلسات.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Session:** بيانات مؤقتة تساعد الخادم يميّز المستخدم بين أكثر من طلب.
- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.

التسجيل

عامل التسجيل كعملية هوية لا مجرد `INSERT` :

1. طبّع البريد وفق سياسة موثقة دون تغيير عشوائي.
2. تحقق من الشكل والنطاق وقواعد المنتج.
3. طبّق rate limit ومراقبة الإساءة.
4. خزّن password بـ `password_hash()` .
5. لا تمنح صلاحيات حساسة قبل إثبات البريد/الهاتف المطلوب.

لتقليل account enumeration استخدم رسالة متقاربة مثل "إن كان العنوان صالحًا فستصل رسالة"، مع logging داخلي.

Token التأكيد

```
$token = bin2hex(random_bytes(32));
$tokenHash = hash('sha256', $token);
// خزّن hash + user_id + expires_at + used_at
// أرسل token الخام في رابط HTTPS مرة واحدة
```

عند الاستخدام: hash القيمة، افحص expiry وused_at، ثم علّمها مستخدمة داخل transaction. لا تخزن token الخام.

Password reset

- رسالة عامة سواء الحساب موجودًا أم لا.

- token عشوائية قصيرة العمر وأحادية الاستخدام.
- لا تغير password قبل إثبات token.
- بعد النجاح دوّر session ID وألغِ جلسات/refresh tokens حسب السياسة.
- أرسل إشعارًا أمنيًا.

لا تستخدم أسئلة أمان قابلة للتخمين.

العمليات الحساسة

اطلب re-authentication أو MFA قبل تغيير البريد أو password أو وسائل MFA أو تنفيذ تحويل مالي. لا تعتبر session قديمة دليلًا كافيًا؛ سجل `auth_time` وحدد max age.

تغيير البريد والحذف

تحقق من البريد الجديد، وأبلغ القديم، وتعامل مع uniqueness race داخل database. عرّف الفرق بين suspension وdeactivation وdeletion، وألغِ credentials والجلسات وسجل actor/reason/time.

حالات السباق

استخدم transaction وunique constraints، وامنَع استخدام token نفسها مرتين. اختبر click مزدوج وطلب reset متزامن وتغيير البريد أثناء وجود جلسات أخرى.

سيناريو أمني

▼ ماذا يحدث عند تعطيل موظف؟

أوقف الدخول وأبطل sessions وtokens والمفاتيح وراجع الملكية والمهام؛ حذف صف المستخدم وحده غير كافٍ.

تدريب تهديد

السيناريو: موظف غادر المؤسسة لكن جلسة قديمة أو refresh token ما زال صالحًا.

اختبار المنع: عطّل الحساب ثم حاول تسجيل دخول جديد وتجديد توكين واستخدام جلسة قائمة.

النتيجة المتوقعة: تفشل المسارات الثلاثة ضمن زمن الإلغاء المتفق عليه، ويُسجّل سبب التعطيل والفاعل دون بيانات حساسة.

مرجع التحقق

- OWASP Authentication Cheat Sheet

اربط النقاط ببعض

طبّع البريد وفق سياسة معلنة دون افتراض أن كل providers تتعامل معه بالطريقة نفسها، وافرض uniqueness داخل database. اجعل رسائل التسجيل والاسترجاع مقاومة لتعداد الحسابات. عالج الحسابات الخاملة والاحتفاظ والحذف والـlegal hold، وسجل transitions ومن نفذها.

جرّب بنفسك

اختبر طلب reset لبريد موجود وغير موجود بنفس الشكل والزمن المقبول.

14. MFA وTOTP وPasskeys

عوامل المصادقة وTOTP وRecovery Codes وWebAuthn/Passkeys وStep-up والاسترجاع الآمن.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

الدرس ده مفيهوش اسم تقني جديد محتاج تحفظه لوحده؛ أي فكرة جديدة هتتشرح وقت ظهورها في المثال.

العامل ليس مجرد خطوة ثانية

- شيء تعرفه: password/PIN.
- شيء تملكه: authenticator أو security key.
- شيء تكونه: biometric محلي يفتح credential.

عاملان من الفئة نفسها ليسا MFA قوية. SMS أفضل أحيانًا من password وحدها لكنه معرض لـSIM swap وphishing.

TOTP

عند التسجيل أنشئ secret عشوائية، اعرض QR عبر قناة authenticated، واطلب code لإثبات الإعداد قبل التفعيل. خزّن السر مشفرًا بمفتاح مُدار، لا hashed فقط لأن الخادم يحتاجه للتحقق.

- نافذة زمنية صغيرة مع clock متزامن.
- بعد نجاح رمز TOTP لازم ترفض استخدامه مرة تانية. خزّن وقت الخطوة المقبولة لكل جهاز بتحديث ذري: طلبان متزامنان ما ينفعش ينجحوا بنفس الرمز.
- rate limit للتحقق.
- لا تسجل secret أو code.

Recovery Codes

أنشئ codes عشوائية أحادية الاستخدام، اعرضها مرة، وخزّن hashes قوية لها. عند استخدام code علّمها مستخدمة وأخبر المستخدم، واسمح بتجديد المجموعة بعد re-authentication.

Passkeys وWebAuthn

```
server challenge -> browser/authenticator -> public-key credential
server stores credential ID + public key + metadata
```

في الدخول يتحقق الخادم من signature وchallenge وorigin وRP ID وuser verification وسياسة counter. الخادم لا يخزن private key. Passkeys مقاومة للتصيد لأنها مرتبطة بالـRP/origin.

Step-up

اطلب assurance أعلى عند تغيير وسائل الاسترجاع، دفع كبير، عرض سر، أو سلوك عالي المخاطر. اربط النتيجة بالعملية ومدة قصيرة.

Recovery هي أضعف حلقة

- عدة قنوات/إثباتات حسب المخاطر.
- تأخير وإشعارات للتغيير الحساس.
- مراجعة دعم بشري بإجراءات وتدقيق.
- إلغاء credentials المفقودة وتسجيل الحدث.

مرجع

- OWASP Authentication Cheat Sheet

سيناريو أمني

▼ هل SMS أقوى MFA؟

لا؛ معرض لنقل الشريحة والتصيد. passkeys مقاومة للتصيد عادة، وTOTP أفضل من كلمة مرور وحدها.

تدريب تهديد

السيناريو: يمتلك المهاجم كلمة المرور ويحاول تجاوز العامل الثاني أو خداع المستخدم بتحدٍ صادر لنطاق آخر.

اختبار المنع: اختبر OTP خاطئًا ومعاد الاستخدام، ثم WebAuthn assertion بـchallenge أو origin مختلف.

النتيجة المتوقعة: تُرفض كل حالة، ويُستهلك التحدي مرة واحدة، ولا يسمح مسار الاسترداد بتجاوز حماية أقوى بصمت.

مرجع التحقق

- W3C Web Authentication Level 3

اربط النقاط ببعض

Enrollment عملية حساسة تحتاج reauthentication وإشعارًا وسجل أجهزة. خطط لفقد الجهاز وrecovery دون إسقاط الحماية إلى سؤال ضعيف. افهم synced passkeys مقابل device-bound credentials وسياسة attestation؛ لا تجعل attestation شرطًا بلا سبب. Step-up يرتبط بخطر العملية وعمر المصادقة.

جرّب بنفسك

سجل passkey جديدة ثم ألغها واختبر recovery بلا جهاز أصلي.

ارفض الـOTP اللي نجح قبل كده

صلاحية TOTP خلال النافذة الزمنية مش معناها السماح بإعادة استخدامه. بعد التحقق، استهلك الخطوة الزمنية المطابقة للعامل بشكل ذري قبل منح الدخول. قيد فريد على `(factor_id, time_step)` في قاعدة البيانات يسمح لطلب واحد فقط بالنجاح حتى مع التزامن. سجل الخطوة اللي طابقت فعلًا، مش وقت الخادم الحالي فقط لو بتسمح بنافذة فرق توقيت. احتفظ بالسجل لحد ما الخطوة دي ماينفعش تتقبل تاني، وتغيير العامل يغير معرّفه. اختبر إرسالين متزامنين: واحد فقط ينجح. أكواد الاسترجاع محتاجة نفس ضمان الاستخدام الواحد.

RFC 6238, section 5.2

15. API Keys وهوية الأنظمة

إصدار مفاتيح API وتخزينها وتدويرها وScopes وService Accounts وmTLS والفرق عن جلسة المستخدم.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **IP:** عنوان رقمي بيميز جهازًا أو واجهة شبكة.
- **TLS:** طبقة تشفير بتحمي البيانات وهي ماشية بين طرفين.
- **API:** واجهة محددة تسمح لبرنامج يطلب بيانات أو ينفّذ عملية عند برنامج آخر.
- **Scope:** نطاق التفويض: اسم صلاحية يطلبها التطبيق أو يحصل عليها، زي orders:read لقراءة الطلبات؛ ولسه لازم نتحقق إن المستخدم يملك الطلب.

API key ليست هوية مستخدم

المفتاح يعرّف application أوintegration غالبًا، ولا يثبت وحده المستخدم النهائي أو التفويض المفوض مثل OAuth. اربطه بـprincipal واضح مع owner وpurpose وبيئة وصلاحيات.

شكل وتخزين

استخدم prefix يوضح النوع ومعرفًا عامًا وسرًا عشوائيًا:

```
live_ak_7F2K.<random-secret>
```

ابحث بالمعرف، وخزّن hash للسر:

```
$secret = bin2hex(random_bytes(32));
$hash = hash('sha256', $secret);
// اعرض السر مرة واحدة ثم خزّن hash فقط.
```

الـprefix يساعد secret scanners والدعم، لكنه ليس سرًا.

صلاحيات ودورة حياة

- scopes صغيرة وdeny-by-default.
- expiry عند الإمكان.
- last-used metadata بلا body حساس.
- rotation بفترة overlap قصيرة.
- revoke فوري.
- مفاتيح منفصلة لكل نظام وبيئة.

أرسل key في header `Authorization` أو header مخصص عبر TLS، لا query string.

Service Accounts وRate limits

طبّق quota لكل key/tenant. IP allow-list طبقة إضافية وليست هوية وحدها. امنح الحساب الآلي أقل صلاحية واربط actions به في audit trail. فضّل credentials قصيرة العمر بدل ملفات keys ثابتة عند توفر workload identity.

mTLS

Mutual TLS يسمح للطرفين بتقديم شهادات. يحتاج إصدار وتجديد وإلغاء وربط subject بصلاحيات؛ التشفير المتبادل لا يلغي authorization على العملية.

سيناريو أمني

▼ كيف نخزن API key؟

اعرض السر مرة، خزّن hash أو استخدم secret manager، أضف scope وexpiry وrotation وسجل الاستخدام.

تدريب تهديد

السيناريو: يتسرب API key لخدمة ذات صلاحيات واسعة ويُستخدم من بيئة غير متوقعة.

اختبار المنع: جرّب المفتاح بعد إلغائه، ومن مصدر غير مسموح، وعلى عملية خارج نطاقه.

النتيجة المتوقعة: تُرفض المحاولات الثلاث، ويكون الدوران ممكنًا بلا توقف الخدمة، وتعرض السجلات معرّف المفتاح لا قيمته السرية.

مرجع التحقق

- OWASP Secrets Management Cheat Sheet

اربط النقاط ببعض

للخدمات الحديثة فضل workload identity federation أو short-lived credentials بدل secret ثابت عندما تتوفر البنية. Client assertion الموقعة تحتاج audience ووقت jti ومنع replay. HSM/KMS يحمي private key غير القابل للتصدير، ودوران المفتاح يحتاج overlap ومعرفًا يحدد الإصدار دون توقف.

جرّب بنفسك

دوّر credential لخدمة حية وتأكد أن القديم ينتهي بعد نافذة محددة.

16. ABAC وReBAC وعزل الـTenants

نماذج صلاحيات تتجاوز RBAC مع السياسات والعلاقات والعزل متعدد المستأجرين والاختبار.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **HTTP:** قواعد تبادل الطلبات والردود بين المتصفح والخادم.
- **Cache:** نسخة مؤقتة من البيانات هدفها تقليل وقت الانتظار والعمل المتكرر.
- **Queue:** طابور مهام تنتظر عاملًا ينفذها في الخلفية.
- **CLI:** واجهة تتعامل معها بكتابة أوامر نصية بدل الضغط على أزرار.
- **Function:** دالة: جزء كود له اسم ومهمة محددة ويمكن استدعاؤه أكثر من مرة.

RBAC ليست كل شيء

- **RBAC:** القرار مبني على roles.
- **ABAC:** attributes للمستخدم والمورد والسياق.
- **ReBAC:** علاقات مثل owner/member/manager.

```
allow if
  subject.tenant_id == resource.tenant_id
  AND subject.department == resource.department
  AND action == "read"
```

ابدأ بسياسة بسيطة، واستخدم نموذجًا أعقد فقط عندما يعبر عن قواعد حقيقية.

Policy decision

```
final class InvoicePolicy
{
    public function view(User $user, Invoice $invoice): bool
    {
        return $user->tenantId() === $invoice->tenantId()
            && ($user->id() === $invoice->ownerId() || $user->hasRole('auditor'));
    }
}
```

الـUI قد تخفي الزر، لكن الخادم يعيد القرار لكل request وpath queue/CLI.

Multi-tenancy

مرّر tenant من الهوية الموثقة لا من body فقط. طبّق العزل في queries وcache keys وobject storage وqueue messages وsearch indexes وexports.

اختر database مشتركة أوschema منفصلة أوdatabase لكل tenant بناءً على العزل والتكلفة والتشغيل، ولا تفترض أن نمطًا واحدًا آمن تلقائيًا.

Deny by default والاختبار

أي action غير معروفة تُرفض. راجع inheritance والتعارض بين allow/deny، وسجل نسخة policy والسبب العام داخليًا.

اختبر matrix تشمل owner في tenant نفسه، مستخدمًا بلا role، ID مطابقًا في tenant مختلف، auditor، وresource غير موجود.
اختبر IDOR بتبديل IDs وكل HTTP/CLI/queue path.

مرجع

- OWASP Authorization Cheat Sheet

سيناريو أمني

▼ فين لازم نطبّق tenant boundary؟

في كل query وcache key وjob وobject storage path، ويفضل فرضها بطبقة بيانات لا تنسى.

تدريب تهديد

السيناريو: مستخدم من Tenant A يضع معرّف Tenant B في المسار أو جسم الطلب.

اختبار المنع: كرر طلب القراءة والتعديل بمعرّف مورد صحيح لكنه تابع لمستأجر آخر.

النتيجة المتوقعة: لا تُرجع القراءة أي بيانات ولا ينفذ التعديل؛ يُشتق سياق المستأجر من الهوية الموثوقة ويُفرض داخل الاستعلام.

مرجع التحقق

- OWASP Authorization Cheat Sheet

اربط النقاط ببعض

Policy engine يحتاج versioned policy وقرارًا قابلًا للتفسير وcache invalidation عند تغير العلاقة أو الدور. مثل hierarchy والعلاقات صراحة بدل شروط متناثرة. استخرج tenant من identity موثوقة وطبقه في query وstorage key وcache key وlogs، واختبر side channels في counts وtiming و404.

جرّب بنفسك

غيّر علاقة أثناء وجود cache وتأكد أن القرار القديم لا يبقى.

17. Rate Limiting ومقاومة الإساءة

Fixed وSliding Window وToken Bucket وحدود تسجيل الدخول وCredential Stuffing وCAPTCHA وسياسة الفشل.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **HTTP:** قواعد تبادل الطلبات والردود بين المتصفح والخادم.
- **IP:** عنوان رقمي بيميز جهازًا أو واجهة شبكة.
- **API:** واجهة محددة تسمح لبرنامج يطلب بيانات أو ينفّذ عملية عند برنامج آخر.
- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.

ما الذي نحده؟

لا تعتمد على IP فقط؛ قد يشترك مستخدمون في NAT وقد يوزع المهاجم الطلبات. كوّن مفاتيح حسب العملية:

- login: account + IP/network + device signals.
- password reset: account/contact + IP.
- API: key + tenant + endpoint.
- expensive search: user + query cost.

ضع حدودًا أقسى للفشل والعمليات المكلفة، مع حد عالمي يحمي السعة.

الخوارزميات

الأسلوب	الفكرة
Fixed window	عداد داخل فترة؛ بسيط وله burst عند الحدود
Sliding log/window	أدق وأعلى تكلفة
Token bucket	tokens تتجدد وتسمح burst مضبوط
Leaky bucket	يصقل معدل الخروج

يجب أن تكون العملية atomic في التخزين المشترك عند تعدد الخوادم.

استجابة HTTP

```
HTTP/1.1 429 Too Many Requests
Retry-After: 30
```

لا تكشف هل username موجود. أضف delay/backoff بحذر دون حجز workers طويلًا.

Login وCredential Stuffing

- لا تستخدم lockout دائمًا يسمح للمهاجم بقفل حساب الضحية.
- استخدم progressive delay وحدودًا متعددة.
- راقب passwords مسربة وفق سياسة الخصوصية.
- MFA/Passkeys تقللان أثر password المسروقة.
- أخطر المستخدم عند نشاط غير معتاد.

CAPTCHA وRisk

CAPTCHA friction وليست proof of humanity كاملة، ويمكن تجاوزها. استخدمها بعد إشارة خطر لا لكل المستخدمين، وراع accessibility والخصوصية.

الفشل والتشغيل

حدد fail-open أوfail-closed لكل عملية إذا تعطل مخزن limits. login إداري حساس يختلف عن endpoint عامة. راقب allow/deny وlatency وtop keys دون تخزين credentials.

سيناريو أمني

▼ هل rate limit بالـIP كفاية؟

لا؛ عناوين مشتركة وbotnets تجعلها إشارة واحدة. ادمج identity وdevice وroute وتكلفة العملية.

تدريب تهديد

السيناريو: يوزع المهاجم محاولات تسجيل الدخول على عناوين IP متعددة لتجاوز عدّاد بسيط لكل عنوان.

اختبار المنع: أرسل دفعة محاولات لحساب واحد من مصادر مختلفة، ثم طلبًا شرعيًا لحساب آخر.

النتيجة المتوقعة: يتباطأ أو يُحظر الهجوم حسب الحساب والسياق، بينما يبقى المستخدم الآخر قادرًا على العمل؛ لا يتحول الدفاع إلى DoS شامل.

مرجع التحقق

- OWASP Denial of Service Cheat Sheet

اربط النقاط ببعض

Distributed limiter يحتاج atomic shared state أو خوارزمية تتحمل التقريب، مع فهم clock skew وfailover. لا تثق في X-Forwarded-For إلا من proxies معروفة. أضف cost-based limits للعمليات الثقيلة وميزانية حسب الحساب والجهاز وIP، وراقب أن الدفاع لا يتحول إلى DoS على المستخدمين الشرعيين.

جرّب بنفسك

شغّل limiter على عقدتين وحاكِ فقد التخزين المشترك وتبديل IP.

18. Security Logging وAudit Trail

أحداث الأمان وبنية السجل والتنقية والـaudit غير القابل للعبث والتنبيهات والتحقيق.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **API:** واجهة محددة تسمح لبرنامج يطلب بيانات أو ينفّذ عملية عند برنامج آخر.
- **Session:** بيانات مؤقتة تساعد الخادم يميّز المستخدم بين أكثر من طلب.
- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.

Operational log أم Audit؟

- operational log للتشخيص والأداء وقد يتغير retention.
- security event للكشف والتنبيه.
- audit trail يسجل من فعل ماذا ومتى وعلى أي مورد، مع متطلبات سلامة واحتفاظ أعلى.

لا تستخدم نصوصًا غير منظمة لسجل تدقيق قانوني.

Schema مقترحة

```
{
  "event": "authorization.denied",
  "time": "2026-09-21T10:15:00Z",
  "request_id": "req_01J...",
  "actor_id": "usr_42",
  "tenant_id": "tn_7",
  "action": "invoice.delete",
  "resource_id": "inv_99",
  "result": "deny",
  "reason_code": "not_owner"
}
```

استخدم IDs لا أسماء/بيانات شخصية إن لم تلزم. وحّد UTC ونسخة schema ومصدر الخدمة.

أحداث مهمة

- نجاح/فشل login وMFA وrecovery.
- تغيير password/email/MFA/roles.
- إنشاء أو تدوير أو إلغاء API key.
- authorization deny والعمليات الإدارية.
- export/delete للبيانات.
- secret access وpolicy changes.

لا تسجل password أوtoken أوsession ID أوkey خام أوrecovery code.

سلامة ووصول

أرسل logs إلى مخزن مركزي محدود الوصول، افصل صلاحية الكتابة عن الحذف، واستخدم retention وbackup وتزامن وقت. راقب توقف ingestion ومحاولات العبث.

التنبيه والتحقيق

التنبيه على نمط: failures موزعة، impossible travel، رفع صلاحيات، key جديدة ثم export ضخم. ضع runbook وowner وseverity، واحفظ request/trace IDs لربط الأدلة.

اختبر أن logging لا تفشل العملية الأساسية بلا سبب ولا تسرب أسرارًا عند exception.

مرجع

- OWASP Logging Cheat Sheet

سيناريو أمني

▼ إيه اللي نسجله من حدث أمني؟

الفاعل والفعل والمورد والنتيجة والوقت وcorrelation ID من غير password أو token أو بيانات حساسة زائدة.

تدريب تهديد

السيناريو: يحاول مهاجم إخفاء أثره بحقن سطر جديد في حقل يُسجل أو بإغراق السجلات بأسرار.

اختبار المنع: أرسل قيمة تحوي محرف سطر جديد وتوكينًا شكليًا، ثم افحص الحدث المخزن وسلسلة الارتباط.

النتيجة المتوقعة: يبقى الحدث سجلًا واحدًا منظمًا، تُحجب الأسرار، وتوجد هوية فاعل ووقت ونتيجة وcorrelation ID قابلة للبحث.

مرجع التحقق

- OWASP Logging Cheat Sheet

اربط النقاط ببعض

Audit trail يحتاج append-only أو tamper-evident storage مع فصل صلاحية الكتابة عن الحذف، ووقتًا متزامنًا ومصدر timestamp واضحًا. حدد retention وlegal access وexport إلى SIEM، واختبر log injection وredaction. التنبيه يملك owner وrunbook وحدًا للضوضاء.

جرّب بنفسك

عدّل event مخزنًا أو أدخل newline وتأكد أن التلاعب يُكشف.

19. إدارة الأسرار ومفاتيح التشفير

Secret managers وenvelope encryption وrotation وrevocation وaccess policy وbreak-glass والاستجابة للتسريب.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **Runtime:** وقت التشغيل: الفترة اللي البرنامج بيكون شغال فيها فعلًا.
- **URL:** العنوان الكامل لمورد على الويب، زي صفحة أو صورة أو نقطة API.
- **API:** واجهة محددة تسمح لبرنامج يطلب بيانات أو ينفّذ عملية عند برنامج آخر.
- **Cache:** نسخة مؤقتة من البيانات هدفها تقليل وقت الانتظار والعمل المتكرر.
- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.

ما السر؟

كلمات مرور قواعد البيانات وAPI keys وprivate keys وencryption keys وsigning secrets أسرار. اسم البيئة وURL عامة ليست أسرارًا. التصنيف يحدد التخزين والوصول والدوران.

ملف `.env` وسيلة تحميل في التطوير وليس secret manager، وBase64 ليست تشفيرًا.

دورة الحياة

```
generate -> store -> distribute -> use -> rotate -> revoke -> destroy
```

لكل سر owner وpurpose وconsumers وcreated/expiry وrotation policy. لا تشارك سرًا واحدًا بين خدمات كثيرة؛ يصعب معرفة المتسبب وإلغاؤه.

Secret Manager

التطبيق يحصل على السر بهوية workload وبأقل صلاحية. فضّل secrets قصيرة العمر أو dynamic credentials. Cache في الذاكرة مدة محدودة وتوقع renewal/failure دون كتابتها إلى disk أو logs.

Envelope Encryption

```
KMS/HSM master key encrypts a data-encryption key (DEK)
DEK encrypts application data
store ciphertext + encrypted DEK + algorithm/version metadata
```

لا تستخدم master key مباشرة لكل record. استخدم authenticated encryption مثل Sodium AEAD، مع nonce صحيحة وassociated data تربط ciphertext بالسياق.

Rotation

احفظ key ID/version مع البيانات أوtoken. أثناء الدوران:

1. أنشئ key جديدة.
2. ابدأ الكتابة بها.
3. استمر في قراءة النسخ القديمة.
4. أعد التشفير/التوقيع حسب الحاجة.
5. ألغِ القديمة بعد نافذة آمنة.

تدوير signing key يختلف عن encryption key؛ حذف مفتاح تشفير قد يجعل البيانات غير قابلة للقراءة.

الوصول والتدقيق

- least privilege وفصل admin عن runtime.
- audit لكل قراءة/تغيير وإلغاء.
- approval أوbreak-glass للعمليات شديدة الحساسية.
- تنبيه على access غير معتاد.
- backups مشفرة واختبار recovery للمفاتيح المطلوبة.

عند التسريب

أوقف/قيّد الاستخدام، دوّر السر، حدّد النطاق من audit logs، أصلح المصدر، وافحص history وartifacts وlogs. حذف السر من Git لا يلغيه. لا تطبع قيم secrets في رسائل CI.

مرجع

- OWASP Secrets Management Cheat Sheet

سيناريو أمني

▼ إيه أول خطوة بعد تسريب secret؟

أبطل أو دوّر السر فورًا، حدد نطاق الاستخدام من السجلات، أصلح المصدر ثم راقب إساءة الاستخدام.

تدريب تهديد

السيناريو: تسرّب إصدار قديم من مفتاح تشفير بينما ما زالت بعض البيانات مشفرة به.

اختبار المنع: دوّر المفتاح، اختبر تشفير بيانات جديدة، فك بيانات قديمة خلال نافذة الهجرة، ثم عطّل الإصدار القديم.

النتيجة المتوقعة: تستخدم الكتابات المفتاح الجديد فورًا، تنجح الهجرة المراقبة، وبعد الإلغاء لا يستطيع المفتاح القديم تنفيذ عمليات جديدة.

مرجع التحقق

- OWASP Secrets Management Cheat Sheet

اربط النقاط ببعض

حدد cryptoperiod لكل key واستخدم KMS/HSM عندما تحتاج non-exportable keys أو فصل الصلاحيات. Backup المشفر يحتاج key backup مستقلًا واختبار restore. Key destruction قرار موثق يمنع فك بيانات مستقبلًا، وrotation يختلف عن re-encryption؛ نفذ dual-read/single-write أثناء الهجرة.

جرّب بنفسك

دوّر key ثم استعد backup قديمًا وأثبت وجود أو غياب المفتاح المطلوب وفق السياسة.

20. Threat Modeling واختبار الصلاحيات

نمذجة تهديد تدفق الدخول وبناء Authorization Test Matrix للأدوار والملكية وعزل المستأجرين.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **API:** واجهة محددة تسمح لبرنامج يطلب بيانات أو ينفّذ عملية عند برنامج آخر.
- **Session:** بيانات مؤقتة تساعد الخادم يميّز المستخدم بين أكثر من طلب.
- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.

ابدأ بالأصول وحدود الثقة

Threat Model ليست قائمة ثغرات عامة. ارسم النظام وحدد الأصول: Credentials وSessions وTokens وبيانات المستخدم وسجلات التدقيق. ثم حدد الجهات: مستخدم، Admin، خدمة داخلية، مزود هوية ومهاجم. كل انتقال بين Browser وAPI وIdentity Provider وDatabase هو Trust Boundary يحتاج تحققًا مستقلًا.

```
Browser -- credentials --> Login endpoint
Browser <-- session ---- Application
Application -- code ----> Identity Provider
Application -- query ---> User / tenant database
```

حلل تدفق تسجيل الدخول

لكل خطوة اسأل:

- هل يمكن Spoofing للهوية أو إعادة استخدام الطلب؟
- هل يمكن تعديل `return_to` أو `tenant_id` أو Role في الطريق؟
- هل Logs تحفظ Password أوCode أوToken؟
- هل المهاجم يستطيع إنكار عملية حساسة لغياب Audit trail؟
- هل Error Message أوTiming يكشف وجود الحساب؟
- هل Rate Limit وMFA وRecovery تقاوم Credential Stuffing؟

حول كل تهديد إلى Control واختبار وTelemetry. مثال: Session Fixation يقابله `session_regenerate_id(true)` بعد نجاح الدخول، واختبار يثبت تغير المعرّف، وLog يسجل حدث التجديد دون قيمة Session ID.

Authorization Test Matrix

لا تختبر "Admin ينجح" فقط. المحاور الأساسية:

Subject	Resource	Tenant	Action	المتوقع
Owner	Own invoice	Same	read	Allow
User	Other invoice	Same	read	Deny
Admin	Invoice	Same	refund	Allow حسب Policy
Admin	Invoice	Other	read	Deny
Suspended user	Own invoice	Same	read	Deny أوRead-only وفق السياسة

أضف صفوفًا للهوية الغائبة والـRole القديمة والـResource المحذوفة وID عشوائي. اختبر القرار في Service/Policy، ثم Integration Test عبر Endpoint حتى تتأكد أن Routing وMiddleware لا تتجاوزانه.

Deny by default وعدم تسريب الوجود

أي Action بلا Policy صريحة مرفوضة. عند محاولة الوصول إلى Resource خارج Tenant قد تستخدم `404` بدل `403` حتى لا تكشف وجوده، مع تسجيل السبب داخليًا. لا تعتمد على إخفاء زر في الواجهة؛ الخادم يعيد القرار من Subject وAction وResource وContext موثوق.

مراجعة التهديد عند التغيير

أعد المراجعة عند إضافة Login Provider أوWebhook أوRole أوTenant Boundary أوطريقة Recovery. اربط كل تهديد بمالك وموعد ومخاطرة متبقية؛ الرسم الذي لا ينتج قرارًا أو اختبارًا لا يحمي النظام.

تأكد من فهمك

01 **ما الفرق بين Asset وTrust Boundary؟**

الإجابة: الأصل شيء نحميه، أما حد الثقة فهو نقطة انتقال تتغير عندها افتراضات الثقة ويجب إعادة التحقق.

02 **لماذا اختبار Admin وحده غير كافٍ؟**

الإجابة: أخطر الأخطاء تظهر في الملكية وعبور Tenant والحالات الموقوفة أو القديمة، لا في المسار المسموح الواضح.

03 **متى قد نعيد 404 بدل 403؟**

الإجابة: عندما كشف وجود المورد لمستخدم غير مصرح له يعد تسريبًا، مع الاحتفاظ بسبب الرفض في السجل الداخلي.

04 **حوّل Session Fixation إلى Control واختبار.**

الإجابة: جدد Session ID بعد الدخول وأبطل القديم، ثم اختبر أن القديم لا يعمل والجديد يحمل الصلاحية الصحيحة.

تدريب تهديد

السيناريو: تغطي الاختبارات المسار الطبيعي فقط، بينما يسمح مسار فرعي بتنفيذ العملية نفسها دون فحص الصلاحية.

اختبار المنع: حوّل كل أصل وحدّ ثقة في النموذج إلى اختبار سماح واختبار منع، ثم كررها على REST والمهام الخلفية.

النتيجة المتوقعة: كل عملية حساسة لها دليل نجاح للفاعل المسموح ودليل رفض للفاعل غير المسموح عبر جميع نقاط الدخول.

مرجع التحقق

- OWASP Threat Modeling

اربط النقاط ببعض

طبق STRIDE كعدسة لا checklist: spoofing وtampering وrepudiation وinformation disclosure وdenial of service وelevation ترتبط بأصل وحد ثقة. حوّل كل abuse case إلى test أو control ومالك، وأعد المراجعة عند تغير data flow أو هوية أو طرف خارجي.

جرّب بنفسك

اختر flow واحدًا واكتب تهديدًا واختبارًا ومالكًا لكل فئة تنطبق.

21. الأجهزة والجلسات وكلمات المرور المسربة وSCIM

إدارة جلسات الأجهزة وفحص كلمات المرور المسربة وProvisioning مؤسسي آمن عبر SCIM.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **HTTP:** قواعد تبادل الطلبات والردود بين المتصفح والخادم.
- **IP:** عنوان رقمي بيميز جهازًا أو واجهة شبكة.
- **API:** واجهة محددة تسمح لبرنامج يطلب بيانات أو ينفّذ عملية عند برنامج آخر.
- **Session:** بيانات مؤقتة تساعد الخادم يميّز المستخدم بين أكثر من طلب.
- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.
- **Scope:** نطاق التفويض: اسم صلاحية يطلبها التطبيق أو يحصل عليها، زي orders:read لقراءة الطلبات؛ ولسه لازم نتحقق إن المستخدم يملك الطلب.
- **Function:** دالة: جزء كود له اسم ومهمة محددة ويمكن استدعاؤه أكثر من مرة.

سجل جلسات قابل للإدارة

لكل Session خزّن Server-side معرفًا Hash، ووقت الإنشاء وآخر استخدام ووقت الانتهاء وDevice label تقريبيًا وعنوان IP مختصرًا عند الحاجة القانونية والتشغيلية. لا تدّع أن User-Agent يحدد جهازًا بهوية مؤكدة؛ هو Metadata قابلة للتغيير.

صفحة "أجهزتك" تعرض الجلسات النشطة وتسمح بإبطال واحدة أو الجميع. عند "تسجيل الخروج من كل الأجهزة" غيّر Session version أو أبطل كل Refresh Tokens والجلسات ما عدا الحالية إذا اختار المستخدم ذلك. العمليات الحساسة تحتاج Re-authentication، لا مجرد Session قديمة.

```
function revokeAllSessions(PDO $pdo, int $userId): void
{
    $statement = $pdo->prepare(
        'UPDATE sessions SET revoked_at = CURRENT_TIMESTAMP
         WHERE user_id = :user_id AND revoked_at IS NULL'
    );
    $statement->execute(['user_id' => $userId]);
}
```

أرسل Notification بعد Password change أوجلسة جديدة غير معتادة، لكن لا تضع Token الإبطال نفسه في Email أوLogs.

فحص كلمات المرور المسربة

سياسة الطول و`password_hash` لا تكشف أن المستخدم اختار Password ظهرت في تسريب. افحصها عند التسجيل والتغيير ضد خدمة أوDataset موثوقة. لا ترسل Password الخام؛ استخدم بروتوكول خصوصية مثل k-anonymity إن توفر، مع Timeout وفشل آمن حسب سياسة المنتج.

لا تحفظ نتيجة الفحص بجانب Password، ولا تمنع المستخدم بسبب Service outage بلا قرار موثق. اسمح Password Manager وPaste، ولا تفرض تغييرات دورية بلا دليل اختراق.

SCIM: Provisioning وليس Login

SCIM بروتوكول HTTP لإدارة Users وGroups بين Identity System وApplication. لا يستبدل SAML أوOIDC لتسجيل الدخول. أهم العمليات: إنشاء مستخدم، تعديل Attributes، تعطيل `active` , وإدارة Group membership.

```
{
  "schemas": ["urn:ietf:params:scim:schemas:core:2.0:User"],
  "userName": "omar@example.com",
  "active": true,
  "name": {"givenName": "Omar", "familyName": "Ahmed"}
}
```

اربط كل SCIM Client بـTenant واحد، طبق OAuth scope أوCredential مخصصة، تحقق من `Content-Type` وSchema، واستخدم ETag/Version لتجنب الكتابة فوق تعديل متزامن.

Deprovisioning أهم من Provisioning

عندما يصبح `active=false` :

1. امنع Login جديدًا.
2. أبطل Sessions وRefresh Tokens وAPI Keys المرتبطة حسب السياسة.
3. أزل Group-derived privileges.
4. احتفظ بالبيانات أوانقل الملكية وفق Retention policy.
5. سجل Actor وSource وCorrelation ID.

اختبر Replay وDuplicate events وOut-of-order updates. اجعل العمليات Idempotent، ولا تحذف بيانات المستخدم فورًا لمجرد وصول طلب Disable.

تأكد من فهمك

01 **لماذا اسم الجهاز ليس إثبات هوية؟**

الإجابة: غالبًا مشتق من User-Agent وIP وهما قابلان للتغيير والتشارك؛ استخدمهما للعرض والمخاطر لا كعامل مصادقة.

02 **ماذا يجب أن يحدث عند Logout all devices؟**

الإجابة: إبطال الجلسات وRefresh Tokens المناسبة، تسجيل الحدث، وإعلام المستخدم دون تسريب قيم الاعتماد.

03 **لماذا SCIM لا يستبدل OIDC؟**

الإجابة: SCIM يدير دورة حياة الحسابات والمجموعات؛ OIDC يثبت هوية المستخدم عند تسجيل الدخول.

04 **لماذا يجب أن يكون Deprovision idempotent؟**

الإجابة: لأن المزود قد يعيد الطلب؛ التكرار يجب ألا يعيد حذفًا أوTransfer أوSide effect خطيرًا.

تدريب تهديد

السيناريو: يعطّل المسؤول مستخدمًا عبر SCIM لكن جلساته على أجهزة متعددة تبقى نشطة.

اختبار المنع: أرسل حدث تعطيل مكررًا، ثم حاول استخدام كل جلسة وتجديد توكين وتسجيل دخول بكلمة مرور مسربة.

النتيجة المتوقعة: المعالجة idempotent، تُلغى الجلسات والتوكينات ضمن المهلة، وتُرفض كلمة المرور وفق سياسة الخصوصية ومقاومة الإساءة.

مرجع التحقق

- RFC 7644: SCIM Protocol

اربط النقاط ببعض

Device binding إشارة مخاطرة لا حقيقة مطلقة لأن المتصفح والجهاز يتغيران؛ اشرح للمستخدم الجلسات ووقت آخر نشاط ومكان تقريبي مع إلغاء فردي وكلي. في SCIM اختبر PATCH وgroups وidempotency وout-of-order events، واجعل deprovisioning يلغي sessions وtokens ضمن SLO مراقب.

جرّب بنفسك

أرسل SCIM disable مكررًا ومتأخرًا وتأكد من الحالة النهائية والإلغاء.

22. PAR وJAR وRAR وحوادث التوكينات

حماية طلبات OAuth المتقدمة والاستجابة العملية لتسريب Access Token أوRefresh Token أوSecret.

قبل ما تبدأ

ذاكر الدرس على 3 خطوات: افهم المشكلة الأول، تابع المثال، وبعدها جرّب الجزء العملي بنفسك. المصطلحات الجديدة الموجودة تحت متشرحة قبل ما ندخل في التفاصيل.

كلمات جديدة في الدرس

- **URL:** العنوان الكامل لمورد على الويب، زي صفحة أو صورة أو نقطة API.
- **Session:** بيانات مؤقتة تساعد الخادم يميّز المستخدم بين أكثر من طلب.
- **Token:** قيمة تمثل هوية أو صلاحية محددة بدل إرسال كلمة السر كل مرة.
- **Scope:** نطاق التفويض: اسم صلاحية يطلبها التطبيق أو يحصل عليها، زي orders:read لقراءة الطلبات؛ ولسه لازم نتحقق إن المستخدم يملك الطلب.

المشكلة التي تعالجها الإضافات

Authorization Request عادي يمر عبر Browser ويحتوي `client_id` و`redirect_uri` و`scope` و`state` وربما تفاصيل حساسة. التوقيع أوالدفع المباشر إلى Authorization Server يقلل العبث والتسريب، لكنه لا يلغي PKCE وState وNonce والتحقق من Redirect URI.

PAR — Pushed Authorization Requests

يرسل Client تفاصيل الطلب مباشرة إلى Authorization Server عبر Back Channel موثق. يحصل على `request_uri` قصير العمر، ثم يرسل Browser إلى Authorization Endpoint بهذا المرجع. هذا يقلل طول URL ويمنع المتصفح من تعديل المعاملات المدفوعة.

```
Client -- authenticated POST --> PAR endpoint
Client <-- request_uri ---------- Authorization Server
Browser --> /authorize?client_id=...&request_uri=...
```

اجعل `request_uri` أحادي الاستخدام أو قصير العمر حسب الخادم، واربطه بالـClient الذي أنشأه.

JAR — JWT-Secured Authorization Request

يضع Parameters داخل JWT موقع يسمى Request Object. التوقيع يثبت سلامة المعاملات ومصدرها وفق سياسة المفاتيح. تحقق من `iss` و`aud` و`exp` والتوقيع وعدم التعارض بين قيم JWT وQuery. التشفير اختياري ومختلف عن التوقيع؛ التوقيع لا يخفي المحتوى.

RAR — Rich Authorization Requests

بدل Scope عام مثل `payments`، يرسل Client تفاصيل منظمة في `authorization_details`: نوع العملية والمبلغ والعملة والمستفيد مثلًا. هذا يسمح Consent أدق، لكنه يتطلب Schema وPolicy ومنع أن يطلب Client صلاحية أوسع مما هو مسجل له.

```
{
  "type": "payment_initiation",
  "amount": {"currency": "EGP", "value": "250.00"},
  "creditor": {"name": "Example Store"}
}
```

PAR وJAR وRAR أدوات مختلفة ويمكن جمعها: RAR يصف الصلاحية الغنية، JAR يحمي Request Object، وPAR يدفع الطلب عبر قناة خلفية.

سيناريو Incident: Access Token مسرب

1. فعّل Incident Command وحدد النطاق: Token واحد أمClient أمSigning Key.
2. ابطل Token عند دعم Revocation أوعطل Session/Grant المرتبط.
3. أوقف Logs أوTelemetry التي تواصل تسريب القيمة، واحفظ Evidence بصلاحيات محدودة.
4. دوّر Secret أوKey إذا كان مصدر التسريب أوسع من Token واحد.
5. ابحث عن استخدام غير طبيعي حسب `jti` أوClient أوSubject دون تسجيل Token الخام.
6. أخطر الأطراف المتأثرة وفق السياسة والقانون.
7. أصلح المصدر وأضف Regression Test وDetection.

إذا تسرب Refresh Token فالأثر أطول؛ أبطل Token family واستخدم Rotation مع Reuse Detection. إذا تسرب Signing Key فقد يلزم رفض كل Tokens الموقعة بالمفتاح، نشر JWKS جديد، وتسريع انتهاء القديم بتنسيق حذر.

Runbook يمنع القرارات المرتجلة

وثق Contacts، صلاحيات الإبطال، أوامر آمنة، مصادر Logs، قوالب الاتصال، ومعايير إغلاق الحادث. اختبر Tabletop Exercise: ماذا يحدث لو ظهر Token في Git commit أوError trace أوFrontend bundle؟ قيّم زمن الاكتشاف والاحتواء لا زمن كتابة التقرير فقط.

تأكد من فهمك

01 **ما الفرق بين PAR وJAR؟**

الإجابة: PAR يدفع الطلب عبر Back Channel ويعيد مرجعًا؛ JAR يضع المعاملات في Request Object موقع.

02 **هل توقيع JAR يخفي البيانات؟**

الإجابة: لا؛ التوقيع يحمي السلامة والأصالة. السرية تحتاج تشفيرًا مناسبًا وسياسة مفاتيح منفصلة.

03 **ماذا يضيف RAR فوق Scope عام؟**

الإجابة: تفاصيل منظمة للعملية تسمح بقرار وConsent أدق من اسم Scope واسع.

04 **ماذا تفعل إذا تسرب Refresh Token؟**

الإجابة: ابطل العائلة أوGrant، افحص Reuse، احتوِ مصدر التسريب، دوّر الأسرار عند الحاجة، وحقق في الاستخدام غير الطبيعي.

تدريب تهديد

السيناريو: يعدّل المهاجم معاملات authorization في المتصفح أو يعيد توكينًا مسروقًا أثناء الاستجابة للحادث.

اختبار المنع: غيّر معاملة خارج الطلب الموقّع، ثم ألغِ عائلة refresh token وحاول استخدامها مرة أخرى.

النتيجة المتوقعة: يرفض الخادم المعلمات غير المرتبطة بالطلب المحمي، ويكشف إعادة الاستخدام ويلغي العائلة دون قبول توكين جديد.

مرجع التحقق

- RFC 9126: Pushed Authorization Requests

اربط النقاط ببعض

ارسم sequence يوضح browser وclient وauthorization server وresource server، وحدد أي parameters محمية بـPAR/JAR وأيها تتحقق عند كل طرف. تحقق من request object issuer/audience/time/jti/signature ومنع replay. نفذ tabletop لحادث token يحدد detection وcontainment وrevocation وevidence وpostmortem.

جرّب بنفسك

أعد request object نفسه وغيّر parameter خارجه وتأكد من رفض replay والتعارض.

06

المسار · 6 درسًا

قواعد البيانات وPDO

شرح متدرج، أمثلة عملية، وتصحيحات تساعدك على الانتقال من الفكرة إلى التطبيق.

قواعد البيانات وPDO

اتصال آمن، ACID، Transactions، ORM، Prepared Statements، العزل، الأقفال والفهارس.

المسار ده يبدأ من أول اتصال بقاعدة البيانات، وبعدها يشرح القراءة والكتابة الآمنة وتنظيم المعاملات والفهارس والترحيلات خطوة بخطوة.

0. أساسيات SQL والجداول والربط.
1. PDO والاتصال الآمن — DSN والإعدادات والاستعلامات المحضّرة.
2. ORM وأنماط الوصول للبيانات — Active Record وData Mapper وN+1.
3. Transactions وAuto-commit — ACID وCommit وRollback.
4. Isolation والأقفال وDeadlocks — مشاكل التزامن وإعادة المحاولة وSavepoints.
5. تصميم المخطط والفهارس وMigrations — القيود والفهارس وقياس الأداء.

نصيحة

ابدأ بـPDO وSQL قبل ORM. الأداة لا تلغي ضرورة فهم الاستعلامات والفهارس والمعاملات.

أساسيات SQL قبل PDO وORM

.Tables, constraints, joins, grouping and safe changes

قبل ما نتعلم PDO أو ORM، محتاجين نفهم السؤال اللي بنوجهه لقاعدة البيانات. في الدرس ده هنبني جدول طلاب ومحاولات، ونستخرج عدد المحاولات ومتوسط درجات كل طالب، حتى الطالب اللي ماحاولش.

من البيانات إلى جدولين

الجدول مجموعة صفوف لها أعمدة محددة. الصف هنا طالب واحد أو محاولة واحدة. المفتاح الأساسي `PRIMARY KEY` بيميز كل صف؛ الاسم وحده مش كفاية لأن طالبين ممكن يكون لهم نفس الاسم. المفتاح الخارجي `FOREIGN KEY` بيربط محاولة بطالب موجود. العلاقة هنا واحد إلى متعدد: الطالب له محاولات كثيرة، والمحاولة تخص طالبًا واحدًا.

`NOT NULL` يمنع غياب قيمة لازمة، و`CHECK` يفرض قاعدة زي درجة بين صفر و100. التحقق في PHP يعطي رسالة مفهومة، والقيود في قاعدة البيانات تحمي القاعدة حتى لو برنامج آخر كتب فيها.

اكتب السؤال قبل SQL

عايزين كل الطلاب، ونضم محاولاتهم، ثم نلخص كل مجموعة محاولات. `SELECT` يحدد الأعمدة الناتجة، و`FROM` الجدول الذي نبدأ منه. `LEFT JOIN` يحافظ على طالب بلا محاولات؛ `INNER JOIN` هيستبعده. شرط `ON` يحدد أي محاولة تخص أي طالب، و`AS` يعطي اسمًا مختصرًا للجدول أو اسمًا واضحًا للعمود الناتج.

```
CREATE TABLE learners (id INTEGER PRIMARY KEY, name TEXT NOT NULL);
CREATE TABLE attempts (
    id INTEGER PRIMARY KEY,
    learner_id INTEGER NOT NULL REFERENCES learners(id),
    score INTEGER NOT NULL CHECK(score BETWEEN 0 AND 100)
);
INSERT INTO learners VALUES (1, 'Omar'), (2, 'Mona'), (3, 'Ali');
INSERT INTO attempts VALUES (1, 1, 80), (2, 1, 100), (3, 2, 60);
SELECT l.id, l.name, COUNT(a.id) AS attempts, AVG(a.score) AS average
FROM learners AS l
LEFT JOIN attempts AS a ON a.learner_id = l.id
GROUP BY l.id, l.name
ORDER BY l.id;
```

نفّذ المثال الكامل الجاهز بالأمر `php sql-foundations.php` من المختبر. الملف ينشئ قاعدة SQLite مؤقتة في الذاكرة ويفعّل قيود العلاقات، ثم ينفذ نفس SQL ويطبع JSON. كائن PDO هو اتصال PHP بقاعدة البيانات؛ الدرس التالي هيفكك الاتصال والاستعلام المحضّر خطوة خطوة.

الطالب	عدد المحاولات	المتوسط
Omar	2	90
Mona	1	60
Ali	0	NULL

`GROUP BY` يجمع صفوف الطالب قبل `COUNT` و`AVG`. استخدمنا `COUNT(a.id)` علشان الصف الناتج من LEFT JOIN بلا محاولة مايتحسبش كمحاولة؛ `COUNT(*)` كان هيحسب صف Ali ويطلع 1. `AVG` يتجاهل NULL، ولعدم وجود درجات يرجع NULL، مش صفرًا. `ORDER BY` يضمن ترتيبًا محددًا؛ ترتيب الإدخال مش ضمان لترتيب النتيجة.

الفلترة والتعديل

`WHERE` يفلتر الصفوف قبل التجميع، و`HAVING` يفلتر المجموعات بعده. لإظهار متوسط أعلى من 70 أضف `HAVING AVG(a.score) > 70` بعد GROUP BY وقبل ORDER BY: النتيجة Omar فقط. لو نقلت شرطًا على جدول المحاولات إلى WHERE بعد LEFT JOIN، ممكن تستبعد الطالب بلا محاولات من غير قصد.

`NULL` معناها قيمة مجهولة أو غير موجودة؛ افحصها بـ `IS NULL`، مش `= NULL`. المقارنة مع NULL لا تنتج true، وWHERE يحتفظ فقط بالصفوف اللي شرطها true.

```
UPDATE attempts SET score = 85 WHERE id = 1;
DELETE FROM attempts WHERE id = 3;
```

دول مثالان للتنفيذ بعد الإنشاء: الأول يعدّل محاولة واحدة والثاني يحذف واحدة. قبل تعديل مهم، جرّب SELECT بنفس WHERE وراجع الصفوف، واستخدم معاملة عندما تتعاون تغييرات متعددة. نسيان WHERE قد يغير كل الصفوف. في التطبيق مرّر القيم عبر placeholders، ولا تلصق مدخل المستخدم في SQL.

الفهرس مش قاعدة صحة

الفهرس بنية إضافية تساعد المحرك يلاقي الصفوف أسرع، لكنه يستهلك مساحة ويزيد كلفة الكتابة. فهرس على `attempts(learner_id)` قد يساعد الربط. الفهرس العادي لا يمنع التكرار؛ `UNIQUE` يفرض عدم تكرار القيم حسب سلوك المحرك. استخدم خطة التنفيذ وبيانات واقعية قبل الحكم على الأداء.

جرّب وفسر

أضف طالبًا رابعًا بلا محاولات: المفروض يظهر بعداد صفر ومتوسط NULL. حاول إدخال درجة 101 أو `learner_id=999`: القيود ترفض الحالتين. غيّر LEFT JOIN إلى INNER JOIN: يختفي Ali. الحل مش حفظ الأوامر، بل تتبع الصفوف قبل الربط وبعده وبعد التجميع.

SQLite هنا وسيلة تشغيل سهلة. أنواع الأعمدة، تفعيل العلاقات، أوامر الهوية، الأقفال ومستويات العزل تختلف بين المحركات؛ ارجع لدروس PDO والمعاملات قبل نقل المثال إلى MySQL أو PostgreSQL.

1. PDO والاتصال الآمن

إنشاء اتصال PDO وضبط الأخطاء وتنفيذ Prepared Statements ومنع SQL Injection.

لماذا PDO؟

PHP Data Objects واجهة موحدة للتعامل مع عدة drivers مثل MySQL وPostgreSQL وSQLite. التوحيد لا يعني أن SQL وخصائص قواعد البيانات متطابقة؛ ارجع دائمًا لتوثيق driver.

الاتصال

```php
<?php
declare(strict_types=1);

$dsn = 'mysql:host=127.0.0.1;port=3306;dbname=shop;charset=utf8mb4';

$pdo = new PDO(
    $dsn,
    getenv('DB_USER') ?: throw new RuntimeException('DB_USER missing'),
    getenv('DB_PASSWORD') ?: throw new RuntimeException('DB_PASSWORD missing'),
    [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false,
    ]
);
```

- لا تحفظ credentials داخل Git.
- استخدم charset في DSN، وليس query يدويًا بعد الاتصال.
- الاستثناءات تجعل الفشل واضحًا، لكن لا تعرض رسالتها للمستخدم.
- `ATTR_EMULATE_PREPARES=false` يطلب native prepares عندما يدعمها driver؛ اختبر SQL الفعلي وخصائص driver بدل اعتباره قانونًا عامًا.

```
$stmt = $pdo->prepare(
    'SELECT id, name, email
     FROM users
     WHERE email = :email AND status = :status'
);

$stmt->execute([
    'email' => $email,
    'status' => 'active',
]);

$user = $stmt->fetch(); // array|false
```

الـplaceholder يفصل **قيمة البيانات** عن تركيب SQL، فيمنع تحويل quote داخل القيمة إلى أمر SQL.

```
$stmt = $pdo->prepare(
    'INSERT INTO products (name, price_cents)
     VALUES (:name, :price)'
);

$stmt->execute([
    'name' => $name,
    'price' => $priceCents,
]);

$id = (int) $pdo->lastInsertId();
```

ما لا يمكن ربطه؟

الـparameter يمثل data literal كاملًا؛ لا يمثل اسم جدول/عمود أو keyword أو اتجاه ترتيب:

```
$allowedSort = ['name', 'created_at', 'price_cents'];
$sort = $_GET['sort'] ?? 'created_at';

if (!in_array($sort, $allowedSort, true)) {
    $sort = 'created_at';
}

$sql = "SELECT * FROM products ORDER BY {$sort} DESC";
$rows = $pdo->query($sql)->fetchAll();
```

استخدم allow-list للأسماء. ولا تربط قائمة `IN` كلها بـplaceholder واحد:

```
$ids = [4, 7, 9];
$marks = implode(',', array_fill(0, count($ids), '?'));
$stmt = $pdo->prepare("SELECT * FROM products WHERE id IN ({$marks})");
$stmt->execute($ids);
```

تعامل مع القائمة الفارغة منفصلًا، وقيّد عدد العناصر.

bindValue أم execute array؟

```
$stmt = $pdo->prepare('SELECT * FROM users WHERE id = :id');
$stmt->bindValue('id', $id, PDO::PARAM_INT);
$stmt->execute();
```

`bindValue` يربط القيمة الآن. `bindParam` يربط المتغير بالمرجع وقد يفاجئك إذا تغير. تمرير array إلى `execute` أبسط غالبًا، واستخدم النوع الصريح عندما يؤثر في driver/query plan.

Fetching والذاكرة

- `fetch()`: صف واحد أو `false`.
- `fetchAll()`: كل الصفوف؛ قد تستهلك ذاكرة كبيرة.
- `fetchColumn()`: عمود واحد.
- `rowCount()`: موثوق عادة لعمليات التعديل، وسلوكه مع SELECT يختلف بين drivers.

مرّ على النتائج الكبيرة صفًا صفًا، وحدد الأعمدة بدل `SELECT *` في الكود الإنتاجي.

معالجة الخطأ

```
try {
    $user = $stmt->fetch();
} catch (PDOException $e) {
    error_log($e);
    throw new RuntimeException('Database operation failed', previous: $e);
}
```

في المنتج أظهر رسالة عامة وRequest ID، وسجل التفاصيل في مكان محمي دون credentials أو بيانات شخصية غير لازمة.

مسائل مرتبطة بالدرس

▼ لماذا لا يكفي escaping بدل prepared statements؟

الـprepared statement يفصل بنية SQL عن القيم؛ escaping يدوي سهل الخطأ ويتأثر بالترميز والسياق.

▼ هل parameter يصلح لاسم جدول؟

لا عادة؛ placeholders للقيم. اختر identifiers من allowlist ثم ضمّن الاسم الموثوق.

شغّل وتحقق

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

نفّذ نقطة التحقق التالية داخل بيئة الدرس:

```
php pdo-lab.php
```

معيار النجاح: ينجح الإدخال الصحيح، وتبقى محاولة مثل `1 OR 1=1` قيمة لا جزءًا من SQL، ويظهر عدد الصفوف المتوقع فقط.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

2. ORM وأنماط الوصول للبيانات

معنى ORM وActive Record وData Mapper والعلاقات ومشكلة N+1 وحدود التجريد.

ما ORM؟

Object-Relational Mapping يربط صفوف الجداول بكائنات وخصائص وعلاقات. يقلل SQL المتكرر لكنه لا يلغي قاعدة البيانات ولا تكلفة الاستعلام.

```
Object/Entity ⇔ Mapper/Model ⇔ Table Row
User          ⇔ ORM          ⇔ users
```

Active Record

الكائن يمثل صفًا ويحتوي عمليات الحفظ والاستعلام. Laravel Eloquent مثال مشهور:

```
$user = User::findOrFail($id);
$user->name = 'Omar';
$user->save();

$orders = Order::query()
    ->where('status', 'paid')
    ->latest()
    ->get();
```

مميزاته: مباشر وسريع في CRUD. عيوبه: قد يخلط منطق المجال بالتخزين، ويصعب فصل الاختبارات في المجالات المعقدة.

Data Mapper

Entity تبقى أقرب إلى منطق المجال، وMapper/Repository يدير التخزين. Doctrine ORM مثال:

```
$user = $entityManager->find(User::class, $id);
$user->rename('Omar');
$entityManager->flush();
```

مميزاته: فصل أقوى ونمذجة مجال أغنى. عيوبه: مفاهيم وحالة Unit of Work وIdentity Map وتعقيد أكبر.

ملاحظة

Django Active Record-like ORM مثال من Python، وليس مكتبة PHP. ذكره مفيد للمقارنة فقط؛ في PHP قارن Eloquent وDoctrine.

العلاقات

- One-to-one.
- One-to-many.
- Many-to-many عبر join table.
- Foreign keys تحمي التكامل في قاعدة البيانات، لا تعتمد على ORM وحده.

حدد cascade بعناية؛ حذف parent لا يجب أن يمسح بيانات كثيرة بالمفاجأة.

N+1 Queries

```
$orders = Order::all();

foreach ($orders as $order) {
    echo $order->customer->name; // قد ينفذ query لكل order
}
```

الحل يكون eager loading أو join/select مصممًا:

```
$orders = Order::with('customer')->get();
```

قِس عدد الاستعلامات وزمنها، ولا تجعل eager load كل العلاقات افتراضيًا لأنه قد ينقل بيانات ضخمة.

Serialization وMass Assignment

لا تمرر `$_POST` كاملًا إلى model:

```
$user->fill([
    'name' => $validated['name'],
    'email' => $validated['email'],
]);
```

وإلا قد يمر حقل مثل `is_admin`. استخدم allow-list وDTO/validated input. كذلك لا تحول entity كاملة إلى JSON إن كانت تحمل password hash أو secrets.

متى نستخدم SQL مباشرًا؟

- تقارير وتجميعات معقدة.
- Bulk operations.
- استعلامات تعتمد على خصائص قوية في قاعدة محددة.
- مسار أداء حرج بعد القياس.

يمكن الجمع بين ORM وQuery Builder وPDO. الهدف وضوح وصحة وأداء، لا الالتزام العقائدي بأداة واحدة.

مسائل مرتبطة بالدرس

▼ ما مشكلة N+1؟

يجلب التطبيق قائمة باستعلام ثم استعلامًا لكل عنصر؛ استخدم eager loading أو join مناسبًا وراقب عدد الاستعلامات.

▼ هل ORM يلغي الحاجة لفهم SQL؟

لا؛ ما زلت تحتاج فهم الخطة والفهارس والمعاملات والحدود لتجنب أداء وسلوك خاطئ.

شغّل وتحقق

المثال يستخدم PDO لتوضيح N+1 ويطبع naive_queries=21 وbatch_queries=2؛ لا يختبر lazy loading في ORM بعينه.

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

نفّذ نقطة التحقق التالية داخل بيئة الدرس:

```
php orm-query-lab.php
```

هدف تجربة التكامل الموسعة: يثبت عدّاد الاستعلامات أن تحميل 20 عنصرًا لا ينتج 21 query، ولا يحدث lazy loading بعد إغلاق وحدة العمل.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

3. Transactions وACID

Auto-commit وbeginTransaction وcommit وrollBack وتطبيق ACID في عمليات متعددة الخطوات.

لماذا Transaction؟

بدون Transaction قد تنجح خطوة وتفشل أخرى، فتترك بيانات نصف مكتملة. في وضع Auto-commit المعتاد يصبح كل statement مستقلًا ما لم تبدأ معاملة صريحة.

مثال تحويل رصيد:

1. خصم من الحساب A.
2. إضافة للحساب B.
3. تسجيل الحركة.

يجب أن تنجح كلها أو لا يحدث شيء.

ACID

- **Atomicity:** المعاملة وحدة واحدة؛ Commit كامل أو Rollback.
- **Consistency:** تنتقل البيانات بين حالات تحترم القيود والقواعد. التطبيق والـconstraints مسؤولان أيضًا.
- **Isolation:** نتائج المعاملات المتزامنة لا تتداخل بطريقة تكسر المستوى المختار.
- **Durability:** بعد Commit الناجح تبقى النتيجة رغم الأعطال ضمن ضمانات النظام.

ACID ليس معناه «لا تحدث أخطاء» ولا أن كل قواعد البيانات والإعدادات تعطي المستوى نفسه.

PDO transaction

```php
function transfer(PDO $pdo, int $from, int $to, int $amount): void
{
    if ($amount <= 0 || $from === $to) {
        throw new InvalidArgumentException('Invalid transfer');
    }

    $pdo->beginTransaction();

    try {
        $lock = $pdo->prepare(
            'SELECT id, balance_cents
             FROM accounts
             WHERE id IN (?, ?)
             ORDER BY id
             FOR UPDATE'
        );
        $lock->execute([$from, $to]);
        $accounts = $lock->fetchAll();

        if (count($accounts) !== 2) {
            throw new RuntimeException('Account not found');
        }

        $balances = array_column($accounts, 'balance_cents', 'id');
        if ((int) $balances[$from] < $amount) {
            throw new RuntimeException('Insufficient balance');
        }

        $debit = $pdo->prepare(
            'UPDATE accounts
             SET balance_cents = balance_cents - :amount
             WHERE id = :id'
        );
        $credit = $pdo->prepare(
            'UPDATE accounts
             SET balance_cents = balance_cents + :amount
             WHERE id = :id'
        );

        $debit->execute(['amount' => $amount, 'id' => $from]);
        $credit->execute(['amount' => $amount, 'id' => $to]);

        $pdo->commit();
    } catch (Throwable $e) {
        if ($pdo->inTransaction()) {
            $pdo->rollBack();
        }
        throw $e;
    }
}
```

ترتيب قفل الحسابات حسب ID يقلل احتمال Deadlock لكنه لا يلغيه. أسماء الصياغة مثل `FOR UPDATE` ودعمها تختلف حسب قاعدة البيانات.

حدود المعاملة

اجعلها قصيرة:

- لا تنتظر API أو بريدًا داخل lock إن أمكن.
- تحقق من المدخلات قبل البداية.
- ضع كل تغييرات قاعدة البيانات اللازمة داخلها.
- لا تُرجع نجاحًا قبل Commit.
- لا تمسك Transaction أثناء عرض صفحة للمستخدم.

لإرسال حدث/رسالة بعد نجاح البيانات استخدم **Transactional Outbox**: اكتب الحدث في جدول داخل المعاملة، ثم ينشره worker لاحقًا.

DDL وImplicit Commit

بعض الأنظمة، ومنها حالات في MySQL، تنفذ implicit commit عند أوامر مثل `CREATE TABLE` و`DROP TABLE`. لا تفترض أن كل DDL يمكن Rollback؛ افحص محرك قاعدة البيانات.

ينطبق التنبيه كذلك على Oracle: معاملات DML تدعم ACID، بينما أوامر DDL تنفذ عادة implicit commit قبل الأمر وبعده. لا تخلط تغيير schema مع business transaction وتتوقع التراجع عن الاثنين كوحدة واحدة.

أخطاء شائعة

- بدء المعاملة بعد أول UPDATE.
- نسيان Rollback في catch.
- ابتلاع exception وإكمال الطلب.
- الاعتماد على Transaction دون constraints.
- إجراء external side effect ثم Rollback؛ قاعدة البيانات لا تستطيع التراجع عن بريد أُرسل.

مسائل مرتبطة بالدرس

▼ متى تعمل rollback؟

عند فشل أي خطوة ضمن invariant واحد حتى لا تبقى كتابة جزئية.

▼ هل transaction الخارجية تجعل API call جزءًا ذريًا؟

لا؛ قاعدة البيانات لا تستطيع عادة rollback لخدمة خارجية. استخدم outbox أو workflow يعالج الفشل والتكرار.

شغّل وتحقق

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

نفّذ نقطة التحقق التالية داخل بيئة الدرس:

```
php transaction-lab.php
```

معيار النجاح: في النجاح تتغير كل الجداول، وفي exception وسط العملية لا يتغير أي منها. أعد الاختبار بعد قطع اتصال اصطناعي.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

4. Isolation والأقفال وDeadlocks

مشكلات التزامن ومستويات العزل والأقفال وDeadlocks وSavepoints وسياسة إعادة المحاولة.

لماذا التزامن صعب؟

قد يقرأ طلبان القيمة نفسها ثم يكتبان نتائج متعارضة. Transaction وحدها لا تعني أن كل المعاملات تعمل واحدًا بعد الآخر؛ **Isolation Level** يحدد ما يمكن لكل معاملة رؤيته.

مشكلات معروفة:

- **Dirty Read:** قراءة تغيير لم يُعمل له Commit.
- **Non-repeatable Read:** قراءة الصف مرتين والحصول على قيمتين بسبب Commit آخر.
- **Phantom Read:** إعادة query شرطية وظهور/اختفاء صفوف.
- **Lost Update:** تحديث يطغى على تحديث متزامن.
- **Write Skew:** معاملتان تقرآن حالة مشتركة وتكتبان صفوفًا مختلفة فتنكسر قاعدة.

مستويات العزل

المستوى	الفكرة
Read Uncommitted	أقل عزل؛ يسمح برؤية أوسع لتغييرات غير مثبتة حسب النظام
Read Committed	كل statement يرى بيانات committed
Repeatable Read	القراءات داخل المعاملة أكثر ثباتًا
Serializable	أقرب لتنفيذ متسلسل، وقد يزيد الانتظار/الفشل القابل لإعادة المحاولة

التفاصيل تختلف بشدة حسب MySQL/PostgreSQL/SQL Server وآلية MVCC. لا تحفظ جدولًا عامًا فقط؛ اختبر قاعدة بياناتك.

Pessimistic locking

```
SELECT id, stock
FROM products
WHERE id = :id
FOR UPDATE
```

يقفل الصفوف المختارة عادة حتى نهاية المعاملة. استخدم index ليصل إلى الصفوف المقصودة؛ query سيئة قد تقفل نطاقًا أوسع.

Optimistic locking

أضف `version` :

```
UPDATE products
SET stock = :new_stock, version = version + 1
WHERE id = :id AND version = :old_version
```

إذا كان affected rows = 0 فقد عدّل طرف آخر الصف؛ أعد القراءة وقرر retry أو `409` conflict.

Deadlock

يحدث عندما تمسك معاملة A قفلًا تحتاجه B، وتمسك B قفلًا تحتاجه A. قاعدة البيانات تقتل إحدى المعاملات لحل الحلقة.

قلل الاحتمال عبر:

- ترتيب الوصول للموارد دائمًا بنفس الطريقة.
- معاملات قصيرة.
- فهارس مناسبة.
- عدم طلب تفاعل المستخدم داخل المعاملة.
- Retry محدود مع backoff وjitter للأخطاء القابلة للإعادة.

يجب أن تكون العملية idempotent أو تستخدم idempotency key حتى لا يتكرر side effect عند retry.

Savepoints والمعاملات المتداخلة

PDO لا يوفر nested transactions حقيقية بطريقة عامة. بعض قواعد البيانات تدعم Savepoints:

```
$pdo->beginTransaction();

try {
    $pdo->exec('SAVEPOINT before_optional_step');
    // optional database work
    $pdo->exec('RELEASE SAVEPOINT before_optional_step');
    $pdo->commit();
} catch (Throwable $e) {
    if ($pdo->inTransaction()) {
        $pdo->rollBack();
    }
    throw $e;
}
```

للرجوع الجزئي استخدم `ROLLBACK TO SAVEPOINT ...` وفق dialect. انتبه: rollback إلى savepoint لا يلغي ضرورة commit/rollback للمعاملة الخارجية.

لا تجعل isolation دواءً سحريًا

رفع المستوى إلى Serializable قد يحسن الصحة لكنه يزيد conflicts والـretries. ابدأ من invariants: ما القاعدة التي لا يجوز كسرها؟ ثم استخدم constraints وatomic updates وlocks أو optimistic version بما يناسب.

مسائل مرتبطة بالدرس

▼ لماذا تحدث deadlock؟

معاملتان تمسكان موارد مختلفة وتنتظر كل منهما ما لدى الأخرى؛ وحّد ترتيب الأقفال وأعد المحاولة بأمان.

▼ هل أعلى isolation أفضل دائمًا؟

لا؛ يقلل anomalies لكنه قد يزيد الانتظار والتعارض. اختر المستوى حسب invariant والحمل.

شغّل وتحقق

الاختبار يشغّل عمليتين حقيقيتين على SQLite؛ إحداهما تشتري والأخرى ترفض نقص المخزون، والنتيجة stock=2 orders=1. اختبار deadlock ومستوى العزل يجب تكراره على محرك الإنتاج.

استخدم المختبر القابل للتنزيل للسكربتات المرفقة. أوامر Composer وFPM وDocker والخادم الحقيقي تُنفذ داخل المشروع المُجهز للخدمة، مش مجلد فاضي.

نفّذ نقطة التحقق التالية داخل بيئة الدرس:

```
php locking-lab.php
```

هدف تجربة التكامل الموسعة: تسجّل عمليتان متزامنتان النتيجة النهائية المتوقعة بلا lost update، وتنتهي إعادة محاولة deadlock ضمن حد واضح.

دوّن كود الخروج والدليل الفعلي. إذا اختلف الناتج، فسر البيئة أو الفرضية التي اختلفت بدل تعديل «المتوقع» حتى يطابق الخطأ.

5. تصميم المخطط والفهارس وMigrations

أنواع البيانات والقيود والعلاقات والفهارس المركبة وEXPLAIN وإدارة تغييرات المخطط.

اجعل قاعدة البيانات تحمي قواعدها

التطبيق قد يحتوي bug أو توجد أداة إدارية أو worker آخر. ضع القيود الأساسية في Schema أيضًا:

```
CREATE TABLE orders (
    id BIGINT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    status VARCHAR(30) NOT NULL,
    total_cents BIGINT NOT NULL CHECK (total_cents >= 0),
    created_at TIMESTAMP NOT NULL,
    CONSTRAINT fk_orders_user
        FOREIGN KEY (user_id) REFERENCES users(id)
);
```

استخدم:

- `NOT NULL` للقيم الإلزامية.
- `UNIQUE` للهوية الفريدة.
- `CHECK` للقواعد المحلية المدعومة.
- Foreign keys للتكامل المرجعي.
- أنواعًا مناسبة؛ المال غالبًا integer minor units أو DECIMAL، لا float.
- timestamps مع سياسة timezone واضحة.

الفهرس

الفهرس بنية إضافية تسرّع lookup/sort/join مقابل مساحة وكلفة كتابة.

```
CREATE INDEX idx_orders_user_status_created
ON orders (user_id, status, created_at);
```

ترتيب الأعمدة مهم. يفيد هذا غالبًا queries تبدأ بـ `user_id` ثم `status` وتستفيد من `created_at`. ليس ضمانًا أن index لكل عمود أو كل query أفضل.

EXPLAIN والقياس

```
EXPLAIN
SELECT id, total_cents
FROM orders
WHERE user_id = 42 AND status = 'paid'
ORDER BY created_at DESC
LIMIT 20;
```

افحص plan والصفوف المقدرة والفعلية إن كانت الأداة تدعم analysis. اختبر بيانات بحجم قريب من الإنتاج، وراقب slow query log وp95/p99.

أسباب بطء شائعة:

- N+1.
- `SELECT *` ونقل بيانات غير لازمة.
- Pagination عميقة بـOFFSET.
- Functions/casts تمنع استخدام index.
- Missing أو low-selectivity index.
- Query كثيرة بدل batch.

Pagination

Offset سهل:

```
SELECT * FROM orders
ORDER BY id DESC
LIMIT 20 OFFSET 10000
```

لكنه يزداد كلفة وقد يتغير مع إدخالات متزامنة. Keyset pagination:

```
SELECT *
FROM orders
WHERE id < :last_seen_id
ORDER BY id DESC
LIMIT 20
```

أفضل غالبًا للfeeds الكبيرة مع ترتيب ثابت.

Migrations

Migration تغيير versioned قابل للمراجعة والتطبيق آليًا:

```
20260921_001_create_orders
20260921_002_add_orders_status_index
```

قواعد مهمة:

1. خذ backup واختبر الاسترجاع.
2. جرّب على نسخة وحجم بيانات واقعي.
3. اجعل التغيير متوافقًا أثناء النشر المرحلي.
4. افصل schema change المكلف عن backfill الضخم.
5. راقب locks ووقت التنفيذ.
6. لا تفترض أن rollback آمن لكل DDL.

نمط Expand/Contract: أضف العمود/البنية الجديدة أولًا، انشر كودًا يدعم القديم والجديد، انقل البيانات، ثم احذف القديم في نشر لاحق.

Seed وبيانات الاختبار

Migrations للبنية والتغييرات الحتمية. Seed ينشئ بيانات مرجعية أو تطويرية. لا تضع بيانات شخصية إنتاجية في fixtures، واجعل seed قابلًا للتكرار دون duplication.

Checklist

- هل كل query تستخدم prepared values؟
- هل القيود تمنع الحالة المستحيلة؟
- هل transaction تغطي كل التغييرات المطلوبة فقط؟
- هل الفهرس يخدم query مقاسة؟
- هل migration آمنة على بيانات كبيرة؟
- هل النسخ الاحتياطي والاسترجاع مجرّبان؟

مسائل مرتبطة بالدرس

▼ لماذا لا نضيف index لكل عمود؟

الفهارس تستهلك مساحة وتبطئ الكتابة؛ صممها للاستعلامات الفعلية وافحص execution plan.

▼ كيف تنشر migration خطرة بأمان؟

استخدم expand/migrate/contract: أضف المتوافق أولًا، انقل البيانات، حدّث التطبيق، ثم احذف القديم لاحقًا.

مشروع تراكمي: مخزون لا يسمح بالبيع الزائد

صمّم جداول `products` و`orders` و`order_items`، ثم نفّذ عملية شراء واحدة عبر PDO. يجب أن تحمي قاعدة البيانات القواعد التي لا يجوز كسرها حتى لو أخطأ التطبيق: كمية موجبة، مفاتيح أجنبية صحيحة، ومعرّف طلب فريد.

السيناريو المتزامن

1. ابدأ Transaction واقرأ صف المنتج بالقفل المناسب لمحركك.
2. ارفض العملية إذا كان المخزون أقل من الكمية المطلوبة.
3. أنشئ الطلب وبنوده، ثم حدّث المخزون والتزم.
4. عند أي خطأ نفّذ rollback كاملًا؛ لا تترك طلبًا بلا بنود أو خصمًا بلا طلب.

بيانات تحقق ثابتة

```
initial_stock=5
buyer_A_requests=4
buyer_B_requests=4
expected_successes=1
expected_final_stock=1
expected_negative_stock_rows=0
```

شغّل العمليتين بالتوازي عدة مرات. وجود نجاحين أو مخزون سالب يعني أن الاختبار كشف Race Condition حقيقية، لا مشكلة تجميلية.

قياس الفهرس والهجرة

احفظ ناتج `EXPLAIN` لاستعلام الطلبات حسب `user_id, created_at` قبل الفهرس وبعده، وسجّل الصفوف المقدّرة أو المقروءة بدل الاكتفاء بعبارة «أصبح أسرع». طبّق migration بأسلوب expand/migrate/contract: أضف البنية المتوافقة، انقل البيانات على دفعات قابلة للاستئناف، حوّل القراءة والكتابة، ثم اترك إزالة القديم لإصدار لاحق بعد إثبات عدم استخدامه.

شروط القبول

- كل قيم العميل تُمرر كـprepared parameters.
- اختبار التكامل يثبت commit وrollback والتنافس بين مشتريين.
- القيود تمنع الحالة المستحيلة مباشرة داخل القاعدة.
- خطة rollback أو roll-forward مكتوبة، واسترجاع النسخة الاحتياطية مُجرّب لا مفترض.

OMAR DEV NOTES

English edition

A print-ready edition containing every published track and lesson in the documentation.

01

TRACK · 60 LESSONS

Computer, programming, and networking fundamentals

Progressive explanations, practical examples, and corrections that take each idea into real code.

Computer, programming, and networking fundamentals

Programmer foundations: computers, software, and networks

Hardware, operating systems, code execution, memory, and files before networks, the web, and problem solving.

Learn what happens when code runs: which component executes instructions, where data lives, why programs wait, and how one program communicates with another.

Start with hardware only as far as it serves programmers. Then concentrate on software and operating-system services. General examples use PHP, with Laravel when a framework helps; a language-specific track uses its own language.

1. Computers and operating systems

Begin with computers and operating systems.

Connect CPU, RAM, and storage, then distinguish source code, programs, and processes. Follow script execution, file access, CPU scheduling, memory use, and permissions.

The sequence is **hardware overview → software and runtime → operating system → execution → processors and data representation → memory and files → execution and debugging tools**.

2. Networks and the web

Continue with networks and the web after understanding processes, files, and communication ports.

Trace connectivity, name resolution, transport, web requests, encryption, and servers. Explain observations and locate failures.

3. Thinking and mathematics for code

Use mathematics and problem solving as needed for conditions, remainders, numeric precision, solution steps, and cost comparisons. Completing every mathematical topic is not a prerequisite for starting a language.

How to study

Read the problem and follow an example from input to result. Predict the result before running it, then change one condition and test your explanation. Explain new terms in context and briefly recall prerequisites with links.

After execution fundamentals, choose PHP for web applications or C++ if that is the language you are studying.

Computer, programming, and networking fundamentals

Computers and operating systems for programmers

A short hardware introduction followed by software, operating systems, execution, memory, files, and developer tools.

When a program becomes slow, consumes memory, or cannot open a file, syntax knowledge alone cannot explain why. Follow what resources the program requests and how the operating system supplies them.

Start with hardware only as far as it serves programming, then follow software and execution. The introductory lesson explains source code, processes, runtimes, libraries, and frameworks before operating-system details.

Learning order

- Hardware components and motherboard architecture
- Software: from source code to a running program
- Operating-system architecture and types
- From solution steps to running code
- How a processor executes instructions
- Representing numbers and characters in memory
- Images, sound, and size units
- Addresses and virtual memory
- Allocation and the lifetime of data
- Moving data and making writes durable
- When does parallel work help?
- Read and write files from a terminal
- Command channels, permissions, and environment
- Test behavior and locate a defect

Connect observations to explanations

Predict an example's output, run it, and change one input. For a file operation, identify the process, its identity, the path, and the expected result. For a delay, distinguish active computation from waiting.

After processes, memory, and resources, continue with networks and the web.

Hardware components and motherboard architecture

Motherboards, buses, storage, I/O devices, power, cooling, and the path data follows through a computer.

Give each connection a purpose

Part	Meaning	Example
Processor socket	Mount and electrical connection on the board	CPU compatibility requires the correct socket, not merely a similar size
Bus	Communication path for data and signals	USB connects a peripheral to its controller
Seek	Moving an HDD head to a data location	Scattered reads require movement; an SSD has no moving head
PSU — Power Supply Unit	Converts input power to voltages components need	Capacity, quality, and connectors affect stability

For a file read: storage reads, a bus transfers, memory holds, and a processor uses data. Power and cooling enable operation; they do not store the file.

Hardware is a connected system

A computer is not an isolated CPU (Central Processing Unit, the main instruction-executing processor) surrounded by unrelated parts. The motherboard connects the processor, memory, storage, and expansion devices through electrical links and protocols. Applications request operating-system services; a driver communicates with each device controller.

Motherboard and chipset

The board determines the CPU socket, RAM (Random Access Memory, working memory for active program data and instructions) technology and channels, storage connectors, PCIe (Peripheral Component Interconnect Express, a component interconnect using data lanes) lanes, and external I/O (Input/Output, reading, writing, and device communication). A chipset supplies additional controllers and connectivity. Matching connector shape is insufficient: firmware, electrical power, protocol generation, and available lanes also determine compatibility.

Buses, PCIe, and USB

A bus carries data, addressing, and control information. PCIe is a point-to-point serial interconnect built from lanes. An `x16` link exposes more lanes than `x1`, but throughput also depends on PCIe generation and the device. USB (Universal Serial Bus, a device connection standard supporting data and power according to capabilities) carries data and often power; a USB-C connector alone does not promise a particular speed or feature set.

HDD, SATA SSD, and NVMe

An HDD (Hard Disk Drive, storage using rotating magnetic disks) is mechanical magnetic storage with inexpensive capacity and relatively high seek latency. A SATA (Serial ATA, a storage connection standard) SSD (Solid-State Drive, electronic storage without moving mechanical parts) removes mechanical motion. NVMe (Non-Volatile Memory Express, a storage protocol commonly used over PCIe) normally uses PCIe and multiple queues designed for solid-state storage. Storage is persistent; RAM is faster and volatile. Opening a file does not copy the entire disk into RAM—systems fetch blocks or pages as needed and cache them.

I/O, controllers, drivers, and interrupts

Input and network devices produce data or events; displays and printers consume results; many devices do both. Hardware controllers operate devices, and kernel drivers expose a stable software interface. Interrupts report events without permanent polling. DMA (Direct Memory Access, device-memory transfer without the CPU copying every byte) can transfer blocks between a device and RAM without making the CPU copy every byte.

Power and cooling

The PSU (Power Supply Unit, hardware supplying appropriate electrical power) converts incoming power into regulated rails and needs suitable capacity, quality, and protection. Heat can cause throttling and instability, so heatsinks, airflow, fans, and thermal interfaces are functional components.

Data path

```
SSD/HDD -> controller -> RAM -> CPU cache/registers -> execution
                                  |
                                  +-> GPU/NIC/display/storage
```

Caches, DMA, memory mapping, and buffers can reduce or postpone copying. Overall performance is constrained by the bottleneck on the actual path, not the largest number printed on one component.

Startup and bottleneck evidence

Firmware initializes hardware. **UEFI (Unified Extensible Firmware Interface, an interface for platform startup firmware)** manages startup entries and launches a **bootloader**, which loads the operating system; neither is the full OS.

A **bottleneck** is the stage limiting the actual workload. **Queue depth** measures outstanding storage requests according to the tool; **utilization** measures resource activity. **Thermal throttling** reduces processor rate at thermal limits. A busy disk with a long queue is different evidence from a hot processor reducing its clock.

RAID (Redundant Array of Independent Disks, combining drives for properties such as availability or performance depending on level) organizes multiple drives for properties such as performance or fault tolerance, depending on level. It is not a backup; mistaken deletion or ransomware can affect the whole array.

Worked check: Slow file opening with little CPU activity calls for storage latency, queue, and health evidence. Slowness correlated with temperature and falling CPU frequency calls for cooling and controlled-load checks, not an immediate purchase recommendation.

Check your understanding

01 **Why does USB-C shape not identify speed?**

> **Answer:** Connector shape is only one layer; protocol version, cable, host, and device determine speed, power, and features.

02 **How do a controller and driver differ?**

> **Answer:** A controller is hardware operating a device; a driver is operating-system software that communicates with it.

03 **Why is NVMe not simply another word for SSD?**

> **Answer:** SSD describes solid-state media; NVMe is a protocol designed for nonvolatile storage over PCIe.

04 **What does DMA improve?**

> **Answer:** It lets devices transfer blocks to or from RAM without CPU instructions copying every byte.

Software: from source code to a running program

Distinguish source code, programs, processes, runtimes, libraries, and the operating system by following a small PHP script.

You saved `echo 'Hello';` in a file. The text exists on disk, but nothing has executed yet. This lesson follows the parts that turn that file into work on a computer.

You need only a basic hardware model: a processor executes instructions, RAM holds active work, and storage retains files. Previous programming experience is unnecessary.

Hardware needs instructions

Hardware means physical components such as processors, memory, and disks. **Software** includes programs, instructions, data, and configuration that define what the computer does. The same hardware can run a browser or an accounting application because different instructions process different data.

Software has different roles. Applications serve users; operating systems manage resources; developer tools include PHP, compilers, and editors. **Firmware** is software associated with a device, used for initialization or control, such as UEFI during startup. Software does not need a window to perform useful work.

Source code, program, and process

Source code is the text a programmer writes for language tools to understand. `hello.php` is source code. A **program** is the instructions and components that accomplish a task, potentially spread across many files. A **process** is a running instance of a program with an identity, memory, and resources managed by the operating system.

Running the same PHP script in two terminals normally creates two processes. A variable in one does not automatically change a variable in the other. Communication needs an explicit mechanism such as a file, database, or connection.

A program file can exist without running. A running program can wait for a user, storage, or a network rather than continually calculating.

Why the processor does not read PHP directly

A processor executes machine instructions appropriate for its architecture. Language tools translate or execute source code. A **compiler** translates code into another representation, which may be machine code or an intermediate format. An **interpreter** executes a program or representation under another program's control.

The distinction is not a strict classification of every language into one category. A typical PHP implementation parses source and compiles it into intermediate instructions called **opcodes**, then its execution engine runs them. A **runtime** supplies services used while a program executes, such as managing values and memory, invoking operations, and handling errors.

PHP is itself a program built to run on the operating system. When a PHP script requests a file, the runtime uses operating-system services. The script does not directly control the disk.

```
hello.php on storage
        ↓ read by PHP
Parsing and preparation
        ↓
PHP execution engine
        ↓ as needed
Operating-system memory, file, and device services
        ↓
Processor executes program and operating-system instructions
```

The diagram shows responsibilities, not isolated time periods. The operating system participates throughout; the processor executes the engine and kernel rather than waiting for them to finish.

Run a small program

Install PHP CLI, the command-line interface for PHP, and confirm it is available:

```
php -v
```

If the command is not recognized, the terminal cannot find PHP in its search paths or PHP is not installed. The script has not started, so editing `echo` cannot repair that failure.

Save this as `hello.php`:

```
<?php
declare(strict_types=1);

$name = 'Omar';
echo 'Hello ' . $name . PHP_EOL;
```

Open a terminal in that file's directory:

```
php hello.php
```

Expected output:

```
Hello Omar
```

`<?php` introduces PHP code. `$name` names a value during execution. The `.` operator joins strings. `echo` writes the result, and `PHP_EOL` supplies a platform-appropriate line ending. `strict_types` affects certain scalar type checks at function calls; it does not declare every variable's type.

The operating system starts a PHP process. PHP reads the script, executes it, and writes to **standard output**, the normal output channel displayed by the terminal. The process then ends. The file remains on disk, but `$name` does not become persistent database state.

Change the name to `Mona` and run it again. The new invocation reads the changed source. Saving and executing are different actions.

Libraries, frameworks, and extensions

A **library** supplies reusable code your program calls for a particular task. A **framework** supplies structure and parts of an application's execution lifecycle, invoking your code at designated points. Laravel is a PHP framework that routes requests and provides tools for application data and services.

A **PHP extension** adds functionality to the PHP engine, such as PDO for database access. Installing a project library is different from enabling an engine extension. A **dependency** is a component your application needs. Composer manages PHP packages; it does not replace PHP installation or extension configuration.

Learn variables, files, and functions in plain PHP first. Laravel can then demonstrate their use in a larger application. Printing a line does not require a framework.

Installed does not mean running

Installation prepares program files and components. **Configuration** selects settings such as a data location or database address. **Execution** actually runs instructions.

PHP can be installed while a required extension is missing. A script can start but fail to open a file because of permissions. A database can be installed while its service is stopped. Each situation requires different evidence.

A **service** is a process or set of processes kept ready for new work, such as a web server. A short CLI script can start and end per invocation; a worker can remain alive across many jobs. A long-lived process may retain values that a short-lived process would discard on exit if application code keeps references to them.

Diagnose the stage that failed

Observation	Likely category	First check
`php` is not recognized	Installation or search path	PHP installation and PATH
`Could not open input file`	Script location	Filename and working directory
`Parse error`	Language syntax	Reported location and preceding line
Old output	Different file or invocation	Saved file, command, and path
`Permission denied`	Resource access denied	Process identity and resource permissions

These are starting points, not proof of a single cause. Read the complete message and locate the failed stage before changing code.

Try it yourself

Run the script in two terminals after changing its name value. Does the first invocation retain the variable after it ends? Does an identical filename guarantee identical source in two directories?

▼ Reasoning and answer

Each invocation has its own process-local state. Its variables do not persist after the process ends merely because the source file remains. Two directories can contain different files with the same name; inspect the working directory and path passed to PHP.

The model now connects saved source, execution tools, process resources, and an operating system managing those resources. Continue with operating systems to understand resource sharing and protected operations.

References

- PHP CLI
- PHP type declarations
- Composer introduction

Operating-system architecture and types

Kernels, user mode, system calls, processes, threads, scheduling, memory, filesystems, drivers, and operating-system categories.

Observe a running program without changing it

On Windows, open Notepad and Task Manager with Ctrl+Shift+Esc. In Details, find `notepad.exe` (names may vary). A **process** is a running program instance with resources; a **thread** is an execution path inside it.

Enable PID and Threads columns where available. PID identifies the current process; Threads counts its threads. Record the count, type in Notepad, and observe CPU usage. Near-zero usage can mean waiting for input, not termination. A UI Status value such as Running does not expose every thread's scheduler state.

Do not end processes. **Check:** more threads do not prove higher speed; measure the workload, and remember threads can wait.

Kernel, operating system, and distribution

The kernel manages processors, memory, devices, isolation, and privileged operations. An operating system includes the kernel plus system services, libraries, tools, and user interfaces. Linux is a kernel; Ubuntu is a distribution combining it with packages and policy. Windows and macOS are complete systems with different kernels and surrounding components.

User mode and kernel mode

Applications normally run with restricted privilege. A system call requests a file, socket, memory mapping, or another protected operation. The kernel validates arguments and permission before operating on resources. This boundary prevents ordinary code from directly reading another process or executing privileged instructions.

Processes, threads, and scheduling

A process owns an address space, identity, and resource handles. Threads share process memory and files while retaining separate stacks and execution state. The scheduler assigns CPU (Central Processing Unit, the main instruction-executing processor) time and performs context switches. Too many threads can increase switching and lock contention rather than speed.

Common states include ready, running, blocked, and terminated. A thread waiting for disk or network becomes blocked so another can use the CPU.

Memory management

The kernel creates address spaces, maintains page tables, resolves page faults, manages shared mappings and swap, and applies protection bits. Out-of-memory policy determines what happens when allocation cannot be satisfied.

Filesystems, I/O, and drivers

A filesystem maps names and metadata to stored blocks. Page cache accelerates access. Drivers operate hardware controllers, while interrupts or polling report events. Open resources are represented by descriptors or handles and must be closed through a clear ownership contract.

Users, groups, and permissions

Processes run as identities with restricted privileges. A web service should not run as root or administrator, and a service account should access only required files and ports. Permissions remain one layer alongside secret protection and network isolation.

Operating-system categories

- Desktop systems prioritize interactive applications.
- Server systems emphasize long-lived services, remote administration, and reliability.
- Mobile systems manage power, sensors, sandboxes, and controlled distribution.
- Embedded systems serve a focused device with constrained resources.
- Real-time systems provide predictable timing; missing a hard deadline is a failure.
- Hypervisors manage virtual machines directly on hardware or above a host OS.

Windows, Linux, and macOS share processes, virtual memory, files, and drivers but differ in APIs, architecture, tooling, filesystem conventions, packaging, and licensing. Selection follows workload and operations, not a universal winner.

Starting a program and reading a file

```
Shell/GUI -> create process -> loader maps executable and libraries
          -> virtual memory, stack, and handles
          -> scheduler runs the first thread
          -> system calls access files, networks, and devices
```

Follow this with a small PHP CLI program in a writable training directory. Save `os-file-lab.php`:

```php
<?php
declare(strict_types=1);

$path = __DIR__ . '/os-lab-note.txt';
$data = "Hello from PHP\n";
$written = file_put_contents($path, $data);
if ($written !== strlen($data)) {
    throw new RuntimeException('Could not write the complete file');
}
$loaded = file_get_contents($path);
if ($loaded === false) {
    throw new RuntimeException('Could not read the file');
}
echo $loaded;
```

Run:

```
php os-file-lab.php
```

Expect `Hello from PHP` and a file named `os-lab-note.txt` beside the script. Repeated execution replaces that training file's contents with the same text. Use a dedicated lab directory.

`__DIR__` makes the location relative to the script rather than the terminal's working directory. `$path` identifies the file. `file_put_contents` returns bytes written or `false`; comparing the count also detects an incomplete write. `file_get_contents` returns contents or `false`. `echo` writes to the terminal's output channel.

Below these functions, the PHP runtime invokes operating-system services. The system resolves the path, checks the process identity's permissions, and uses filesystem, cache, and device services as needed. A PHP call does not necessarily equal one system call: libraries and buffering can combine or split operations.

Cached data can return quickly; slower I/O can block a thread while another ready thread runs. Waiting and using CPU are different states. A successful write here establishes API acceptance, not survival after power failure. See buffers and durable writes for the additional guarantees.

Locate the failed layer

Observation	Hypothesis to check	Why administrator privileges do not solve it automatically
Missing file	Path, name, or working directory	Permissions do not create the intended file
Access denied	Process identity and resource permissions	Elevation can hide the cause and increase damage
Waiting with low CPU	Storage, network, lock, or timer	Waiting does not inherently require privilege
Growing memory	Retained data or excess allocation	Privilege does not repair lifetime management

The browser's user is not PHP's server execution identity. A web service may run under a different account from your terminal, explaining why the same path works in CLI and fails through the service. Compare identity, path, and permissions in both cases.

Inter-process communication and waiting

IPC here means **Inter-Process Communication**, not CPU instructions per cycle. A **pipe** connects process data streams; a **socket** provides local or network communication; **shared memory** exposes a common region requiring synchronization; a **signal** notifies a process of an event.

A **deadlock** can occur when A holds resource 1 while waiting for 2, and B holds 2 while waiting for 1. The usual necessary conditions are mutual exclusion, holding while waiting, no forced preemption, and circular waiting. Breaking one, such as imposing one acquisition order, prevents that form. It differs from ordinary delay.

Worked check: A file-read system call may return from a memory cache or require storage work, blocking the thread while another runs. Paging manages pages, not necessarily whole-process swapping. A **loader** prepares executables and libraries, **GUI** means graphical user interface, and a **handle** identifies an open resource that needs a clear closing responsibility.

Check your understanding

01 **Are Linux and Ubuntu the same thing?**

> **Answer:** Linux is a kernel; Ubuntu is a distribution containing it with tools, packages, services, and policy.

02 **Why does an application need a system call?**

> **Answer:** User-mode code cannot directly perform privileged resource and device operations; the kernel validates and performs them.

03 **What do threads in one process share?**

> **Answer:** They share address space and resources such as files, while each keeps a stack and execution state.

04 **How does a server OS goal differ from a real-time OS goal?**

> **Answer:** A server emphasizes services, capacity, and reliability; real-time design emphasizes predictable timing bounds.

From solution steps to running code

From solution steps to running code

After representing values, follow instructions: what translates them, what executes them, and why a program may need an external library.

A **compiler** translates source to machine code or another executable representation. An **interpreter** executes code or its representation. A **runtime** supplies execution services. **JIT**, just-in-time compilation, translates selected code while a program runs.

From an algorithm to a program

1. Define inputs, outputs, and constraints.
2. Write language-independent steps as pseudocode or a flowchart.
3. Dry-run a normal, boundary, and invalid case.
4. Translate the design into code and compare behavior with expectations.
5. Optimize only after correctness and measurement show a real need.

From a problem to machine-executable instructions

A compiler translates source before execution, an interpreter executes a representation at runtime, and a JIT (Just-In-Time compilation, translating code during execution) compiles selected code while the program runs. A runtime supplies services required by the language or platform.

Libraries may link statically or load dynamically. Projects record dependency versions because API (Application Programming Interface, a defined contract for requesting data or actions from another component) and behavior can change between releases.

A **library** is reusable code; a **dependency** is something a project requires. **Static linking** incorporates library code into a build; **dynamic linking** loads it for execution. A **lock file** records selected dependency versions to help reproduce a build.

How a processor executes instructions

How a processor executes instructions

Two instructions, then data location

Imagine two simplified instructions: put 5 in a processor working slot, then add 3 to it. The slot is a **register**. The processor fetches an instruction, decodes its required operation, and executes it. Results are 5, then 8. This is a teaching model, not a specific machine syntax.

```
CPU registers ⇔ L1 cache ⇔ L2 cache ⇔ L3 cache ⇔ RAM
```

A **cache** keeps nearby copies to reduce repeated data waits. L1/L2/L3 are levels in a common simplified hierarchy; organization and sharing vary. Arrows represent requests/transfers, not a requirement that every read visits every level. Learn this picture before parallelism and prediction details.

Inside a CPU

The processor fetches, decodes, and executes instructions. A program counter identifies the next instruction, the control unit coordinates it, the ALU (Arithmetic Logic Unit, the processor's arithmetic and logical operation unit) performs arithmetic and logic, and registers hold values closest to execution. Other units handle floating point, vectors, branches, and memory access.

Modern CPUs pipeline instructions, predict branches, and execute independent work out of order. "One instruction per clock" is therefore an oversimplification, and GHz (gigahertz, a billion cycles per second, not a count of completed instructions) alone cannot predict performance.

Cores, threads, clock, and IPC

A core can execute an independent instruction stream. A hardware thread shares some core resources; it is not a complete additional core. Clock rate counts cycles, while IPC (Instructions Per Cycle, average instructions completed per processor cycle) measures useful instructions completed per cycle. Memory behavior, architecture, heat, and power limits matter as well.

More cores help only when work can be divided. Serial sections and shared locks limit speedup.

Registers and L1/L2/L3 caches

Registers are the smallest and fastest storage. L1 is tiny and very fast, L2 is larger, and L3 is larger again and often shared. Caches move cache lines, so contiguous access benefits from locality while random access causes more misses and RAM (Random Access Memory, working memory for active program data and instructions) waits.

Cache coherence does not replace locks or atomic operations when threads mutate shared state.

Execution terms and a measurement exercise

Fetch–Decode–Execute means fetching an instruction, identifying its operation and operands, and performing it. **Floating point** represents fractional values with limited precision; a **vector** groups values; a **branch** selects an instruction path; **serial** work must proceed sequentially.

SIMD (Single Instruction Multiple Data, applying an instruction to multiple data elements) applies one instruction to several data elements, unlike multiple cores running independent instruction streams. A GPU **kernel** is a compute program, not an OS kernel. **Branch divergence** makes grouped work take differing paths and can reduce efficiency. **Unified memory** simplifies the programming view but does not always eliminate physical transfers.

If 50% of a program's time cannot be parallelized, infinitely accelerating the other half leaves at least 50% of the original time: maximum speedup is two. This illustrates **Amdahl's law**.

Worked measurement: After warm-up, record 9,10,10,11,30 milliseconds under the same conditions. The median is 10; the mean is 14. Do not discard 30 without investigating. Record input size and transfer costs as well.

Check your understanding

01 **Why can a 3.5GHz CPU outperform a 4GHz CPU?**

> **Answer:** IPC, architecture, caches, core count, power limits, and workload all affect completed work.

02 **How does a hardware thread differ from a core?**

> **Answer:** A thread shares execution resources within a core; it is not another complete physical core.

03 **When is a GPU a poor choice?**

> **Answer:** For small, serial, heavily branching work or when data-transfer cost exceeds computation.

04 **How does locality improve speed?**

> **Answer:** Nearby access reuses cache lines; scattered access creates misses and waits for RAM.

Next step

After completing this practice, continue with When does parallel work help?.

Representing numbers and characters in memory

Representing numbers and characters in memory

Overview

All data and instructions become bits inside the machine. Programming languages let us express algorithms clearly before a compiler or interpreter turns them into executable work.

Core concepts

- A bit is 0 or 1; eight bits form a byte.
- Binary is a positional base-2 representation.
- Text is stored using code points defined by Unicode and bytes defined by an encoding form such as UTF-8 (Unicode Transformation Format with 8-bit units, encoding Unicode character numbers as bytes), then as bits.
- An algorithm is a finite, precise solution independent of any language.
- Language choice depends on domain, ecosystem, performance, and team constraints.

Worked example

To classify a grade: read it, validate that it is between 0 and 100, then report pass for 50 or more and fail otherwise. Validation must precede the decision.

Corrections and common mistakes

- Code is one language-specific implementation of an algorithm.
- One successful sample is insufficient; test boundaries and invalid input.

From binary to hexadecimal

Hexadecimal makes bit patterns shorter because one hex digit represents four bits. For example, `1111 1010₂` equals `FA₁₆`. Hex appears in memory addresses, colors such as `#22C55E`, binary-file viewers, and bit masks.

Unicode and UTF-8 in practice

Unicode assigns a code point to each character, such as `U+0639` for the Arabic letter ع. UTF-8 defines the bytes used to encode that value. Basic English characters use one byte, Arabic letters usually use two, and other symbols may use up to four.

Do not assume character count equals byte count or slice text at arbitrary byte positions. Use library operations that understand Unicode when measuring or splitting human text.

Signed values, floating point, endianness, and runtimes

The same bits have different meanings under different types. Unsigned values wrap within their range, while common signed integers use two's complement. Floating point stores an approximation using a sign, exponent, and significand, so many decimal fractions cannot be represented exactly.

Endianness defines byte order for multi-byte values. It matters in binary files and network protocols.

Work through actual representations

First review 5 to 101 and text/image/audio representation. **Hexadecimal** uses 0–9 and A–F for 10–15. FA is 15×16+10=250, or binary 11111010. A **bit mask** selects particular bits, such as the low bit for odd/even checks.

In 8-bit **two's complement**, 5 is 00000101; invert bits to 11111010 and add one to get 11111011, interpreted as -5. The same pattern is unsigned 251. The signed range is -128 (10000000) through 127 (01111111). Do not generalize unsigned wrapping to every signed type: signed overflow in C++ is not guaranteed wrapping.

Floating point uses sign, exponent, and significand; the exponent scales the value and the significand supplies precision. Just as decimal 1/3 repeats forever, binary 0.1 does not terminate. JavaScript commonly displays 0.1+0.2 as 0.30000000000000004. Choose domain-appropriate accuracy and rounding rules rather than an arbitrary tolerance. Integer minor units can suit money with explicit conversion and rounding policies.

Endianness orders bytes, not bits within each byte. For two-byte hexadecimal 0x1234, big-endian stores 12 then 34 from the lower address; little-endian stores 34 then 12. The 0x prefix denotes hexadecimal. **Worked check:** bytes 35 and 00 represent 53 as a little-endian 16-bit integer, while the first byte alone encodes the UTF-8 character 5. State the type, byte order, and encoding.

Check your understanding

01 **Represent 45 in binary and explain why its value does not change.**

> **Explained answer:** 45 is 32+8+4+1, so it is 101101. Only the representation and base changed; the mathematical value did not.

02 **Why is UTF-8 not a programming language even though it maps text to bytes?**

> **Explained answer:** UTF-8 is an encoding for character representation. It has no executable instructions, control flow, or algorithm semantics.

03 **Is an algorithm correct because one sample passes? Give a counterexample.**

> **Explained answer:** No. Division may work for 10/2 but fail for a zero denominator; boundaries and invalid input must also be tested.

04 **When might ahead-of-time compilation matter compared with interpretation or JIT?**

> **Explained answer:** It depends on platform, performance, and workflow. AOT supports native distribution and predictable startup; interpretation/JIT can favor dynamic execution and iteration.

Next step

After completing this practice, continue with From solution steps to running code.

Images, sound, and size units

Images, sound, and size units

After representing numbers and characters, use bytes to represent images and sound. Calculate each example before moving on; only simple multiplication and division are needed.

Images and sound

A digital image can be represented as a grid of **pixels**, small colored points. **RGB**, Red Green Blue, describes amounts of three color channels. A common representation uses one byte per channel: 255,0,0 is red, and 0,0,0 is black. A 2×2 image has four pixels; at three bytes per pixel, raw color data needs 12 bytes. The actual file size may differ because of file metadata, compression, or another pixel representation.

Sound can be represented by **samples**, measurements of a signal at successive times. The **sample rate** is the number of measurements per second. One channel at 8000 samples per second and 16 bits per sample needs `8000×16÷8 = 16000` raw bytes per second. These are example settings, not a rule for every recording.

Compression reduces representation size. Lossless compression can recover the exact original data; lossy compression discards some detail. Text and executable programs need exact recovery, while some image and audio uses tolerate selected losses.

Size and transfer-rate units

Lowercase b means bit; uppercase B means byte. `kB = 1000 B` and `MB = 1,000,000 B`. Binary units have distinct names: `KiB = 1024 B` and `MiB = 1,048,576 B`. Some interfaces use ambiguous labels, so check the intended unit.

A rate of `8 Mb/s` is eight million bits per second, theoretically `1 MB/s` before protocol overhead and other limits. File size measures data; transfer rate measures data per unit of time.

Keep the calculations separate

1. Mono audio,4000 samples/second,16bits/sample,one second: `4000×16÷8=8000` raw bytes. Doubling only duration gives16000.
2. Program files require lossless compression to recover identical bytes. Display images may accept lossy compression if retained detail meets the purpose. 3.16Mb/s theoretically equals2MB/s before overhead.10MB needs at least five seconds at that constant theoretical rate, potentially longer in practice.

Write units beside values; confusing bits with bytes changes the result by eight.

Addresses and virtual memory

Addresses and virtual memory

Two addresses for the same data

A **virtual address** belongs to a program's address space; a **physical address** identifies its corresponding memory location. Hardware and the system translate between them. With illustrative4096-byte pages, address5000 is page1, offset904. Mapping to frame7 gives `7×4096+904=29576`.

```
Virtual: page 1 + offset 904 → page table: 1 maps to 7 → Physical: frame 7 + offset 904
```

A **page table** stores mappings; an **offset** locates a byte within a page. Real page sizes vary. **Try:**4096 has offset0;4095 belongs to page0 at offset4095.

RAM and addresses

RAM (Random Access Memory, working memory for active program data and instructions) is fast volatile storage for active code and data. The CPU (Central Processing Unit, the main instruction-executing processor) reads addresses, usually through cache lines. The memory controller, channels, frequency, bandwidth, and latency all influence performance.

Physical and virtual memory

Each process sees a private virtual address space. Memory is divided into pages; the MMU (Memory Management Unit, hardware translating memory addresses and enforcing access) translates virtual addresses to physical frames through page tables managed by the kernel. This provides isolation, demand loading, and controlled sharing.

A page fault can be a normal request to load a valid page. Swapping cold pages to storage is much slower than RAM, and thrashing occurs when the system spends more time moving pages than performing work.

Trace an address and a growing queue

Assume 4096-byte pages and zero-based indexing. Virtual address 5000 is on page 1 at offset 904. If that page maps to physical frame 7, the illustrative physical address is 7×4096+904=29576. A **page** is a virtual-memory unit; a **frame** is its physical counterpart. Actual page sizes vary.

A **TLB (Translation Lookaside Buffer, a cache of memory-address translations)** caches translations. A TLB miss need not be a page fault because a valid page-table entry may exist. **Demand paging** prepares pages when needed. **Overcommit** can allow virtual reservations larger than immediately available backing; it does not promise every later write succeeds.

Check your understanding

01 **Is every page fault fatal?**

> **Answer:** No. A valid page may simply need loading; an invalid access fails when the kernel cannot resolve it.

03 **Why is swap not equivalent to more RAM?**

> **Answer:** Storage is far slower; swap may avoid immediate failure but can create severe latency and thrashing.

Next step

After completing this practice, continue with Allocation and the lifetime of data.

After completing this practice, continue with Moving data and making writes durable.

Allocation and the lifetime of data

Allocation and the lifetime of data

Trace a local value, then data needed for longer. First understand memory addresses in the preceding lesson.

Short-lived values and retained data

A price function needs its inputs and intermediate result during a call. Local data lifetime can end afterward according to language rules, while the shopping list must survive the add function. **Allocation** obtains storage; **lifetime** describes when data remains valid. These are different questions.

The **stack** commonly organizes call data. The **heap** supports flexible allocation lifetimes. Language implementations may optimize placement; not every local value physically resides on a stack.

Check: returning from add does not erase a list still needed by the program. Language rules, memory management, and references govern its lifetime.

Stack and heap

Stack frames normally hold call state and local values and disappear as calls return. The heap supports dynamically lived objects. Deep recursion can overflow a stack; heaps can suffer retention leaks, fragmentation, or out-of-memory failure. These are virtual address regions, not separate RAM chips.

Allocation, leaks, fragmentation, and OOM

An allocator manages blocks and reuses freed space. A leak keeps memory reachable or allocated after it is no longer useful. Fragmentation leaves unusable gaps or overhead. Observe resident and peak memory, not only theoretical object size. Garbage collection cannot correct an intentionally retained unbounded cache.

Managed memory is administered by the language; **native memory** may lie outside its usual object-heap counters. **Retention** keeps unneeded objects reachable, so garbage collection cannot reclaim them. **RSS (Resident Set Size, process memory currently resident in physical RAM under the system's accounting)** can rise because of thread stacks, native library allocations, or mapped pages while a heap counter stays flat.

Moving data and making writes durable

Moving data and making writes durable

A program can produce data faster than a device consumes it. Understand waiting storage, then distinguish handing off a write from making it durable.

Waiting data or a reusable copy?

A producer emits100 messages/second while a consumer handles60:40 accumulate per second. A **buffer** holds data in transit; a **queue** organizes waiting order. A **cache** retains a reusable, usually reproducible copy.

Ten seconds adds400 messages. **Backpressure** slows production in response to a slower consumer, or a bounded system rejects new work. After a write reaches a device, ask whether it survives power loss: **durability** is persistence under a specified failure guarantee.

Buffer, cache, queue, stream, and pool

Concept	Purpose
Buffer	Absorb rate differences or batch transfer
Cache	Keep a reproducible copy to avoid repeated work
Queue	Order work waiting for consumption
Stream	Sequential flow whose final size may be unknown
Pool	Reusable prepared resources

A cache can normally be dropped and rebuilt; unsent buffered data may be unique and must not be discarded.

Common buffers

Input buffers collect events. File buffers reduce small system calls. Socket send and receive buffers decouple network and application rates. `stdout` may be line-buffered on a terminal and fully buffered when redirected. Ring buffers reuse fixed storage; double buffering prepares one frame while another is displayed.

A buffer overflow writes beyond a boundary in an unsafe environment. Media underflow means consumption outran production. Backpressure prevents an unbounded producer from filling memory.

Flush, DMA, and zero-copy

A library `flush` may move bytes only into the kernel, which can retain them in page cache, while the device may have another cache. Power-loss durability requires an explicit filesystem or database contract. DMA (Direct Memory Access, device-memory transfer without the CPU copying every byte) transfers blocks without CPU copying each byte. Zero-copy techniques reduce intermediate copies but do not mean data never moves.

Worked check: A producer adds 100 messages per second and a consumer processes 60. The queue grows by 40 per second, or 400 after ten seconds. A size limit plus rejection or backpressure bounds growth; adding RAM only delays exhaustion. A flush can move data between buffers without guaranteeing power-loss durability.

02 **What is the essential buffer/cache difference?**

> **Answer:** A buffer holds data in transit; a cache retains a usually reproducible copy for speed.

04 **Why may flush not guarantee durability?**

> **Answer:** Data may remain in kernel or device caches; durable persistence needs an explicit sync and storage contract.

When does parallel work help?

When does parallel work help?

Compare dependent steps with thousands of independent operations. First understand the processor and memory.

Why a GPU is different

A CPU (Central Processing Unit, the main instruction-executing processor) has fewer powerful cores optimized for low latency, branching, and general-purpose work. A GPU (Graphics Processing Unit, a processor suited to many similar operations across data) contains many simpler execution units designed to apply similar operations to large data sets. It excels at rendering, matrix work, machine learning, and scientific computation.

Irregular branching and repeated small transfers can erase the advantage. A discrete GPU normally has separate VRAM (Video RAM, memory for graphics data, often on a discrete card); copying data, launching a kernel, and returning results all cost time.

Integrated and discrete GPUs

An integrated GPU usually shares system memory and power. A discrete GPU has its own VRAM, power budget, and cooling. Unified-memory APIs simplify programming but do not guarantee that physical data movement disappears.

Workload comparison

Workload	Typical choice	Reason
Web request and business rules	CPU	Branching, I/O (Input/Output, reading, writing, and device communication), and latency
Small compression task	CPU	GPU transfer cost dominates
Large matrix multiplication	GPU	High data parallelism
Rendering millions of pixels	GPU	Similar operation over many values
Database transaction	CPU	Control, memory, and I/O heavy

Read and write files from a terminal

Read and write files from a terminal

Terminal and shell

A terminal displays a text session; a shell parses commands, quoting, expansions, and pipes before starting processes. PowerShell, Bash, zsh, and `cmd.exe` use different syntax, so escaping rules should not be copied blindly between them.

Paths and working directory

An absolute path starts at a root or drive. A relative path is resolved from the current working directory; `.` means current and `..` means parent. A filename is not a complete path, and an extension does not prove content type.

Use tab completion and literal-path options for special characters. Before recursive deletion or movement, resolve and display the exact target.

A reproducible terminal workspace

Open PowerShell on Windows and create a new training directory under your user directory. Choose another name if it already exists:

```
Set-Location $HOME
New-Item -ItemType Directory -Name terminal-practice
Set-Location .\terminal-practice
Set-Content -LiteralPath '.\note.txt' -Value 'Hello'
Get-Location
Get-ChildItem
Get-Content -LiteralPath '.\note.txt'
```

Set-Location changes the working directory; $HOME is your user directory. New-Item creates a directory. Set-Content writes text and would replace an existing file, so use the new training folder. The Get commands display location, children, and contents. Expect note.txt and Hello. Tab completes names.

Quoting preserves paths containing spaces. A **wildcard**, such as *, can match many items; LiteralPath treats the path literally. **Recursive** includes child directories and therefore broadens a move or deletion.

Check your understanding

01 **How do a terminal and shell differ?**

> **Answer:** The terminal presents the session; the shell interprets commands and connects processes and streams.

02 **Why can a valid relative path fail?**

> **Answer:** It is resolved from the current working directory, which may differ from the script or project directory.

03 **Why separate stdout and stderr?**

> **Answer:** Machine-readable output remains clean while diagnostics can be displayed or logged independently.

04 **Is an environment variable a secret vault?**

> **Answer:** No. It is a configuration channel that can leak; use secret management, permissions, rotation, and log redaction.

Next step

After completing this practice, continue with Command channels, permissions, and environment.

Command channels, permissions, and environment

Command channels, permissions, and environment

Begin after creating and reading a file in the terminal. Separate normal output from diagnostics, then examine inherited settings and permission boundaries.

stdin, stdout, stderr, and exit status

Programs read standard input, emit normal data on standard output, diagnostics on standard error, and return a status code. Zero conventionally means success. Pipes connect one process's stdout to another's stdin; backpressure can make a fast producer wait for a slower consumer.

Keep progress messages away from machine-readable stdout, quote paths with spaces, inspect help before bulk operations, and never expose secrets in command history or process arguments.

Files, directories, and metadata

A file is bytes plus metadata such as size, timestamps, owner, and permissions. A directory maps names to entries. A same-filesystem rename may update metadata quickly, while a cross-device move may copy and delete. A symbolic link refers to another path rather than copying its data.

Users, groups, and permissions

Permissions determine read, write, and execution access. Unix commonly expresses owner/group/others bits; Windows uses detailed ACLs (Access Control Lists, rules granting operations on resources to identities). Services should not run as root or administrator unnecessarily, and secret files should not be globally readable.

Environment variables

Environment variables are key-value configuration inherited by child processes. They are not a secret vault and may appear in process inspection, crash reports, or logs. Validate required values at startup and use a production secret manager.

Try the channels after defining them

PowerShell cmdlet pipes normally pass **objects**, values with properties, rather than always plain text:

```
Get-ChildItem -File | Select-Object Name, Length
```

Select-Object chooses the Name and Length properties. External programs expose standard input, output, error, and an exit status. This Windows-only example deliberately returns status 3:

```
cmd /c 'echo result & echo diagnostic 1>&2 & exit /b 3' 1> output.txt 2> error.txt
$LASTEXITCODE
```

cmd writes result to standard output and diagnostic to standard error. 1> and 2> save them separately. Expect LASTEXITCODE=3; that is intentional. cmd, PowerShell, and Bash quoting are not interchangeable.

In the cmd example, `/c` executes the supplied text and exits; `&` inside it separates cmd commands; `echo` writes a message; `1>&2` routes stdout to stderr; `exit /b 3` selects exit status3. Those quoted operators belong to cmd; redirection outside the quotes belongs to PowerShell.

Test behavior and locate a defect

Test behavior and locate a defect

Testing compares behavior with its promise; Git records file history. Separate the jobs: write the expected result independently, run the case, and locate the first differing step.

Evidence-based debugging

Reproduce with fixed input, state expected and actual behavior, find the last known-correct point, gather logs and debugger evidence, test one hypothesis at a time, fix the cause, and add a regression test. `git bisect` can binary-search history when a reliable pass/fail test exists.

Test levels

Unit tests cover small logic, integration tests cover component boundaries, end-to-end tests cover user journeys, and regression tests preserve a previously failing case. A good test has controlled dependencies, a clear failure reason, and explicit arrange–act–assert structure.

Computer, programming, and networking fundamentals

Networking and the web

Follow a request from the local network through addressing and transport to a protected web connection. Study server application topics after the core route.

Follow two local devices, a site address, and a page request, then run a local server and investigate caching, updates, and request observation. Start the Node.js lab before server exercises; optional PHP examples link to their language setup.

Connections and data transport

- The internet, the web, and a request's journey
- How devices communicate on a local network
- Segment networks and prevent frame loops
- Connect to Wi-Fi and diagnose problems
- Device addresses and connection settings
- Calculate a subnet from its address
- Choose a route and read IPv6 addresses
- How a domain name becomes an address
- DNS failures and protection
- TCP: from establishment to closure
- UDP: independent messages
- MTU: packet size and path diagnosis

The web and protected connections

- Web addresses, ports, and HTTP
- Characters and encoding in web addresses
- Read an HTTP request and response
- Remembering users and protecting updates
- Connection encryption and website certificates
- How peers establish an encrypted connection
- How the browser displays a page
- Load scripts and schedule browser tasks

Web applications and servers

- Set up a local request-and-response lab
- A request's journey inside the server
- Proxies and request distribution
- Load distribution and proxy policies
- Web caching and compression
- Controlling cross-origin data access
- Resource loading and window isolation
- Poll or receive a server stream
- Receive a webhook and handle repeated delivery
- Designing data interfaces
- How HTTP/2 and HTTP/3 differ
- Deliver content and understand the origin server
- Protecting requests and limiting their rate
- Trace requests and interpret measurements

The internet, the web, and a request's journey

Understand the Internet, the Web, client/server roles, and the request-response journey from URL to page.

When you enter a URL (Uniform Resource Locator, an address identifying a resource and how to access it), a page seems to appear in one step. It actually takes a chain of steps. This lesson gives you the map that connects DNS (Domain Name System, a distributed system answering queries about domain names), TCP (Transmission Control Protocol, ordered byte transport with loss recovery; connections can still fail), TLS (Transport Layer Security, rules for establishing an authenticated protected connection), HTTP (Hypertext Transfer Protocol, the rules for web requests and responses), and PHP (a programming language commonly used for server-side web processing) in the lessons ahead.

Four steps before details

Enter a site name; **DNS** answers questions about the name, including its address. The browser establishes an appropriate connection, sends a page **request**, and displays a **response**.

```
Browser ── name question ──> DNS
Browser <─ address answer ── DNS
Browser ── connection + HTTP request ──> Server
Browser <──────── HTTP response ──────── Server
```

DNS supplies an address; page content does not flow through it. Proxy and encryption details come in their lessons. **Predict:** a nonexistent name fails before a page request; a reachable server can return404 for a missing resource.

Begin with the mental model

Think of the Internet as a worldwide road system and the Web as one delivery service that uses those roads. The roads can carry other services too, so **the Internet is not the Web**:

- The **Internet** is the infrastructure that connects devices and networks with protocols such as IP (Internet Protocol, the addressing and routing protocol; an IP address identifies a network interface in context).
- The **Web** is a service that uses that infrastructure to retrieve URL-addressed resources over HTTP or HTTPS (HTTP carried over a TLS-protected connection).
- Email, Internet calls, and online games also use the Internet without being Web pages.

This distinction helps with debugging: having an Internet connection does not prove that a particular site or its HTTP service works.

Client and server are roles

A **client** starts a request. It may be a browser, a mobile app, or a tool such as `curl`. A **server** program waits for requests and returns responses. Both can run on your computer during development, while a production request may pass through several servers.

```
User → Client → HTTP Request → Server
                                 ↓
                          Application ↔ its data
                                 ↓
User ← Rendered page ← HTTP Response
```

“Server” can mean the machine or the program, depending on context. Here, the useful meaning is the role: a program listening for requests at an address and port.

Read the URL before following it

```
https://shop.example/products/42?currency=EGP#reviews
```

- `https` is the scheme or requested protocol.
- `shop.example` is the host name that DNS must resolve.
- `/products/42` is the path sent to the server.
- `currency=EGP` is the query string and is normally sent to the server.
- `#reviews` is a fragment used by the browser and is normally absent from the HTTP request.

Make a real request

```
curl -i "https://example.com/"
```

You should see output resembling this; individual headers may differ:

```
HTTP/2 200
content-type: text/html

<!doctype html>
...
```

- `curl` is the client in place of a browser.
- `-i` includes response headers with the body.
- `HTTP/2 200` says the HTTP request succeeded; it does not prove every business rule is correct.

- `content-type` tells the client how to interpret the body.
- Content after the blank line is the response body.

To inspect more of the connection:

```
curl -v "https://example.com/" -o NUL
```

Use `/dev/null` instead of `NUL` on Linux and macOS. `-v` prints connection, TLS, and request details; `-o` hides the page body so you can focus on the journey.

Common mistakes

- **"DNS opens the page."** DNS returns an address; HTTP requests the resource.
- **"Nginx executes PHP itself."** It commonly forwards the request to PHP-FPM.
- **"A `200` response means everything is correct."** A successful response may still contain incorrect business output.
- **"Every visit reaches the database."** A cache or static file may answer earlier.
- **"A server must be a large machine."** It is a role that a small program on your computer can perform.

Apply the model

▼ A site shows a DNS error. Which later stages have not started?

The client has no suitable IP, so it normally cannot begin TCP or QUIC, TLS, or HTTP with that server. Inspect the name and DNS before PHP code.

▼ `curl` returns correct HTML (Hypertext Markup Language, a language describing page structure), but the browser page is broken. What does that tell you?

The basic DNS, connection, and HTTP path works from the `curl` environment. Investigate JavaScript, dependent resources, CORS (Cross-Origin Resource Sharing, browser rules for permitted cross-origin response access), cookies, and cache. This narrows the search but does not prove both environments are identical.

▼ Does `#reviews` reach PHP?

Normally no. The browser uses the fragment locally. The path and query string, such as `/products/42?currency=EGP`, do reach the server.

Check your understanding

1. What is the practical difference between the Internet and the Web?

- **Answer:** The Internet is general transport infrastructure; the Web is one service using it through HTTP/HTTPS. Internet access can work while a particular Web service fails.

2. How can one computer be both client and server?
    - **Answer:** They are program roles. A browser can start the request while a local development server receives it on the same machine.
3. Where should you look if DNS and TLS succeed but the server returns `403`?
    - **Answer:** The request reached HTTP, and `403` says the server understood it but denied access. Inspect identity, permissions, and web-server or WAF rules.

How devices communicate on a local network

How application data crosses network layers and how devices communicate inside a LAN.

Two computers and a router

```
Laptop A ── Switch ── Laptop B
              |
           Router ── other networks
```

A **switch** connects local devices; a **router** connects networks. Assume A=192.168.1.20 and B=192.168.1.30, both /24: the first24bits identify the subnet. A recognizes B as local, uses ARP to ask for B's interface address, and sends a **frame** through the switch. **MAC** addresses identify interfaces on the link.

For a remote destination, the local frame targets the router's interface while the IP destination remains the remote host in this no-translation example. **Trace:** B→A targets A locally; B→another network targets the router locally.

Start with one trip across a local network

Suppose a laptop requests a page. A network card does not transmit the URL (Uniform Resource Locator, an address identifying a resource and how to access it) as-is. Data moves down layers, and each layer adds information for its responsibility:

```
HTTP data → TCP segment / UDP datagram → IP packet → Ethernet frame → signals
```

Layers are responsibility boundaries, not necessarily separate devices. HTTP (Hypertext Transfer Protocol, the rules for web requests and responses) defines request meaning, TCP (Transmission Control Protocol, ordered byte transport with loss recovery; connections can still fail) manages ordered delivery, IP (Internet Protocol, the addressing and routing protocol; an IP address identifies a network interface in context) routes between networks, and Ethernet/Wi-Fi move one frame over the current link.

For a remote destination, the laptop needs the local default gateway's MAC (Media Access Control, an interface address used for local-link communication), not the remote server's MAC. Every router removes the current link frame and creates another for the next hop. IP identifies the routed destination; MAC addresses change hop by hop.

ARP (Address Resolution Protocol, finding a local-link MAC address for an IPv4 address) asks a local IPv4 question: "Who has this IP, and what is your MAC?" The cached answer is not authentication; ARP spoofing can advertise a malicious mapping on the LAN (Local Area Network, a network within an area such as a home or office).

TCP/IP layers

Layers separate responsibilities. HTTP defines application messages, TCP or UDP (User Datagram Protocol, independent messages without built-in delivery or ordering guarantees) transports them, IP routes packets between networks, and Ethernet or Wi-Fi moves frames across the current link. Each layer adds a header during encapsulation and the receiving side removes it.

Layer	Examples	Data unit
Application	HTTP, DNS (Domain Name System, a distributed system answering queries about domain names), TLS (Transport Layer Security, rules for establishing an authenticated protected connection)	Message/Data
Transport	TCP, UDP	Segment/Datagram
Internet	IPv4, IPv6	Packet
Link	Ethernet, Wi-Fi	Frame

LAN, Ethernet, and switches

A LAN connects devices within a local area. An Ethernet frame contains source and destination MAC addresses, a payload type, data, and an error-detection value. A switch learns which MAC address appears on each port and forwards frames to the appropriate port.

MAC addresses serve the current local link; IP addresses identify endpoints across routed networks. Neither replaces the other.

ARP and the default gateway

ARP discovers the MAC address associated with an IPv4 address on the LAN. A host broadcasts a request and caches the reply. For a remote IP, the host sends the frame to the default gateway's MAC address while the packet keeps the remote server's IP as its destination.

Switches and routers

A switch forwards local frames using MAC addresses. A router connects IP networks and selects a next hop using its routing table. A home device may combine routing, switching, Wi-Fi, DHCP (Dynamic Host Configuration Protocol, automatic network configuration for hosts), and NAT (Network Address Translation, rewriting IP addresses at a network boundary), but these remain distinct functions.

Practical exercises

▼ Why is the remote server's MAC absent from the outgoing frame?

Ethernet covers the local link. The laptop addresses its default gateway, and each router creates a new frame for the next link.

▼ Order segment, packet, and frame

Application data enters a transport segment, then an IP packet, then a link frame. The receiver decapsulates in reverse.

Check your understanding

Why are both IP and MAC addresses needed? IP identifies an endpoint across networks; MAC delivers the frame on the current local link.

Next step

After completing this practice, continue with Segment networks and prevent frame loops.

Segment networks and prevent frame loops

Segment networks and prevent frame loops

After tracing a frame between two devices, consider an office needing separated groups and redundant links. A **frame** carries data on a local link. Understand the need for separation before its mechanisms.

Broadcast boundaries and loops

ICMP, Internet Control Message Protocol, carries diagnostics and errors, not just ping. **VLAN, Virtual Local Area Network**, defines a logical local broadcast domain. **Broadcast** reaches devices in that domain, not the entire internet. **STP, Spanning Tree Protocol**, disables selected redundant link paths to prevent Ethernet frames from circulating indefinitely.

ARP spoofing advertises a false IP-to-MAC mapping because ARP alone does not authenticate answers. IPv6 uses **NDP, Neighbor Discovery Protocol**, for neighbor discovery and other local functions, with related trust concerns.

Ethernet and Wi-Fi have different link-frame formats: use Ethernet framing on an Ethernet link and Wi-Fi framing on Wi-Fi. At transport level, TCP uses segments and UDP uses datagrams.

Worked check: A host in VLAN 10 sends to VLAN 20. Even on the same switch chassis, this normally needs routing functionality and an allowing policy. The incoming link frame ends at routing, and the IP packet leaves in a new frame. A switch can flood unknown-destination or broadcast traffic within its domain; it does not always select just one output port.

Connect to Wi-Fi and diagnose problems

Distinguish local connection from internet access and understand network names, access points, signal, security, and diagnosis.

A phone shows Wi-Fi, yet a website will not open. Is the failure in the phone, the home network, or its internet connection? Separate the stages before changing settings. First read about the local network, the nearby devices communicating within a place such as a home.

Roles inside the home

Wi-Fi provides a wireless local-network connection using radio waves; it is not a technical acronym you need to expand. An **access point** is the component a wireless device joins to enter the local network. A **router** forwards data between your network and other networks. One box may perform both roles.

A **modem** connects to a provider's particular access line, such as telephone or cable, converting the relevant signals. Fiber connections may use an **ONT, Optical Network Terminal**. Homes do not all use identical equipment or role boundaries; one device can combine several functions.

```
Phone --Wi-Fi--> Access point --local network--> Router --> Provider --> Internet
```

Each arrow is part of the path. A strong first connection does not establish that the rest work. Local file sharing can work while the internet link is down. Likewise, a wired **Ethernet** connection alone does not guarantee internet access.

Network name and connection

The **SSID, Service Set Identifier**, is the network name shown in the Wi-Fi list, such as `Home-Learning`. A name does not prove ownership: different networks can advertise the same one. Use an authorized network and obtain its name and password from its owner.

On Windows, open the network list from the taskbar or network settings, choose the name, connect, and enter the password. Labels vary by release. Do not include passwords in diagnostic reports or screenshots.

Hotel and airport networks may use a **captive portal**, a page requiring acceptance or sign-in before providing internet access. Joining Wi-Fi does not complete that step. Use the system's network sign-in notification if shown, and verify that you joined the venue's actual network.

Signal strength and speed differ

Wi-Fi uses frequency bands including 2.4, 5, and 6 **GHz**, gigahertz; one gigahertz is one billion cycles per second. This describes the radio wave, not a file-download rate, and 5 GHz is not the same thing as cellular 5G.

The 2.4 GHz band often reaches farther through obstacles. With suitable equipment and conditions, 5 and 6 GHz may offer more capacity nearby. Walls, placement, interference, and device capabilities affect the result; the frequency number alone does not decide performance.

A **channel** is a portion of a frequency band used by a network. Nearby networks and other devices can share airtime or interfere, increasing waiting and retries. Signal bars estimate reception strength; they do not independently measure congestion or your internet subscription's speed.

A range extender relays the connection. If its link to the main access point is weak, standing close to the extender does not repair that weak segment. Compare placements before changing or buying equipment.

Protecting the connection

WPA, Wi-Fi Protected Access, names a family of wireless security mechanisms. WPA2 and WPA3 are versions. Use a modern mode supported by your devices, such as WPA3-Personal or WPA2/WPA3 compatibility when needed. Do not treat an unprotected open network as suitable protection for sensitive data.

The Wi-Fi password controls network admission; the router's administration password permits configuration changes. They serve different purposes. Change default administration credentials and follow manufacturer updates. A **guest network** may isolate visitors from your devices depending on its settings; verify isolation rather than assuming the name establishes it.

Wi-Fi protection concerns the local wireless link. **HTTPS** protects the web connection between browser and site; a later lesson explains it. Neither replaces the other or guarantees that a site's owner is honest.

Diagnose by comparison

Observation	Next check and reason
Network name is missing	Check Wi-Fi, airplane mode, distance, and device band support
Name is visible but password is rejected	Verify the selected network, password, and letter case
Works nearby but fails farther away	Compare the same device and test near the access point to investigate coverage
All devices join Wi-Fi but cannot reach the internet	Check the provider link; this is not limited to one client
Only one device fails in the same location	Investigate that device; comparison narrows possibilities without proving a cause
One site fails while others work	Investigate the site or its name resolution; general connectivity may be healthy

Worked experiment: open the same page on the same phone near the router and then in a distant room. If only the distant case fails, investigate coverage first. If two nearby devices cannot reach any site, investigate the internet path. Do not begin with a factory reset: it erases configuration and may require provider credentials to restore service.

The next addressing lesson explains addresses and gateways, followed by tools for distinguishing connection stages. The goal here is to choose the next test and avoid confusing a Wi-Fi icon with working end-to-end internet access.

References: Windows Wi-Fi diagnosis, Router and access-point settings.

Record a comparison

Use the same page at similar times and change one factor at a time. These are blank observations, not invented results:

Trial	Device	Position	Network name	Shown signal	New page load	Notes
1	First	Near	…	…	Success/failure and duration	…
2	First	Far	…	…	…	…
3	Second	Near	…	…	…	…

Near success/far failure on one device suggests investigating coverage/interference. Both failing nearby with strong signals suggests checking beyond the access point. Repeat before treating one observation as proof.

Device addresses and connection settings

Device addresses and connection settings

Read settings before calculating

On Windows run `ipconfig /all` in PowerShell. Find the connected adapter and read four fields: IPv4 Address, Subnet Mask identifying network bits, Default Gateway used without a more specific route, and DNS Servers answering name queries. Do not edit settings.

DHCP, Dynamic Host Configuration Protocol, supplies settings for a renewable lease. Example:192.168.1.20, mask255.255.255.0, gateway192.168.1.1. Your values may differ. Detailed subnet calculation, routing, and IPv6 follow separately.

DHCP

DHCP supplies an IP (Internet Protocol, the addressing and routing protocol; an IP address identifies a network interface in context) address, subnet mask, default gateway, DNS servers, and a lease time. The common exchange is Discover, Offer, Request, and Acknowledgement. A client address may change when the lease changes.

Private addresses and NAT

Private IPv4 ranges include `10.0.0.0/8`, `172.16.0.0/12`, and `192.168.0.0/16`. NAT rewrites addresses at a boundary, while PAT uses ports to distinguish many connections behind one public IP. Port forwarding maps selected inbound traffic to an internal host. Carrier-grade NAT may prevent direct inbound connections even when a home router is configured.

Practical exercises

▼ A device has an IP but no default gateway. What can work?

It can usually reach its own subnet but lacks a route to external networks unless another route exists.

▼ Why is NAT not a firewall?

NAT translates addresses and ports; a firewall applies allow/deny policy. NAT can coexist with weak rules or exposed forwarding.

Check your understanding

NAT translates traffic at a boundary; port forwarding adds an inbound mapping to one internal service.

Next step

After completing this practice, continue with Calculate a subnet from its address.

After completing this practice, continue with Choose a route and read IPv6 addresses.

Calculate a subnet from its address

Calculate a subnet from its address

Two device addresses end in .20 and .70. Are they in one subnet? The address alone is insufficient: the mask identifies network bits. Calculate with /24 and /26 after reviewing binary representation.

CIDR and subnetting

A **subnet** shares an address prefix. A **subnet mask** identifies those network bits. **CIDR, Classless Inter-Domain Routing**, writes the prefix length after a slash. Review binary representation before calculating.

IPv4 is 32 bits, displayed as four 8-bit groups from 0 to 255. A /24 leaves eight host bits: 256 total addresses. In an ordinary IPv4 subnet of this size, reserve the network and broadcast addresses, leaving 254 hosts. Do not subtract two indiscriminately: /31 point-to-point links and /32 host routes have different uses.

Calculate /24 and /26

192.168.1.0/24 has mask 255.255.255.0, network .0, broadcast .255, and ordinary host addresses .1–.254. A device at 192.168.1.20/24 treats .70 as local.

192.168.1.0/26 leaves six bits: 64 addresses. Its mask is 255.255.255.192; the final byte is 11000000. The range is .0–.63, hosts .1–.62, broadcast .63. Address .70 belongs to .64/26 and needs a route from .20/26.

Bitwise **AND** outputs 1 only when both input bits are 1:

```
20:   00010100
192:  11000000
AND:  00000000 -> network .0
```

70 is 01000110; AND with the same mask gives 01000000, or .64. **Worked check:** .62 is local to .20/26; .65 is not. Check routing and access policy instead of trying to send to a remote server's MAC (Media Access Control, an interface address used for local-link communication).

In `/24`, the first 24 bits identify the network and the remaining bits identify hosts. A device applies the mask to decide whether a destination is local or must go through the default gateway. A longer prefix creates smaller networks with fewer host addresses.

Choose a route and read IPv6 addresses

Choose a route and read IPv6 addresses

After deciding whether a destination is local, choose the next device along the route. **Routing** selects a path using a table. A **next hop** is the device receiving the packet next. Start with a small table before extending addresses to IPv6.

Routing and IPv6

A routing table lists prefixes, next hops, interfaces, and metrics. Routers prefer the most specific matching prefix, then use a default route when no specific entry matches.

IPv6 uses 128-bit addresses such as `2001:db8::1`. It uses Neighbor Discovery instead of ARP. `::1` is loopback, and `fe80::/10` contains link-local addresses. IPv6 still requires firewall and access-control policies.

Choosing routes and reading IPv6

A **prefix** is the shared network bit sequence; a **next hop** is the next forwarding device. A **metric** is a route cost used under the router's policy. In an ordinary selected routing table, prefer the longest matching prefix before comparing suitable equal-prefix alternatives.

Worked check: With 0.0.0.0/0 via A, 192.168.0.0/16 via B, and 192.168.1.0/24 via C, address 192.168.1.70 selects C, 192.168.2.8 selects B, and 203.0.113.8 selects A.

IPv6 usually writes 128 bits as eight 16-bit hexadecimal groups. Omit leading zeroes in a group; use :: once to compress consecutive zero groups. 2001:db8::1 is for examples, ::1 is loopback, and fe80::/10 is the link-local range.

SLAAC, Stateless Address Autoconfiguration, forms addresses using router information. **Router Advertisements** also identify default routers. **DHCPv6** can supply addresses or other settings depending on the network; the default route is normally learned from router advertisements rather than DHCPv6. **NDP (Neighbor Discovery Protocol, IPv6 discovery of neighbors and routers)** discovers neighbors and local-link information. A **firewall** applies allow/deny policy; NAT does not replace it.

A device needs more than an IP address

Joining Wi-Fi normally requires an address, local prefix, default gateway, and DNS (Domain Name System, a distributed system answering queries about domain names) resolver. DHCP (Dynamic Host Configuration Protocol, automatic network configuration for hosts) commonly supplies them:

```
Discover → Offer → Request → Acknowledge
```

The subnet/CIDR (Classless Inter-Domain Routing, specifying a network prefix length after a slash) prefix decides which destinations are local. Local traffic stays on the link; non-local traffic goes to a gateway. A router selects the most specific matching route and next hop.

NAT (Network Address Translation, rewriting IP addresses at a network boundary) is not routing. Routing selects a path; NAT changes addresses or ports. Home PAT (Port Address Translation, distinguishing connections behind one address through port mappings) lets many devices share one public IPv4 address through port mappings. That is not a firewall and does not automatically secure the internal network.

IPv6 expands address space and uses Neighbor Discovery rather than ARP (Address Resolution Protocol, finding a local-link MAC address for an IPv4 address). Notation changes, while prefixes, routes, and next-hop decisions remain fundamental.

How a domain name becomes an address

Understand why a service needs a name and an address, and how DNS reaches an authoritative answer through caches.

The client in the previous lesson needed the server's address before it could connect. Keep two jobs separate: **an IP (Internet Protocol, the addressing and routing protocol; an IP address identifies a network interface in context) address is a routable destination, while DNS (Domain Name System, a distributed system answering queries about domain names) finds suitable addresses for a name**.

Read a lookup result

Run `nslookup example.com`. This is an **illustration of its shape**, not a captured live result:

```
Server:  router.home
Address: 192.168.1.1
Non-authoritative answer:
Name:    example.com
Address: 203.0.113.10
```

The first Server/Address identify the answering resolver, not the website. The later Name/Address describe the queried name. 203.0.113.10 is a documentation address, not a destination to try. **Non-authoritative** means the answer was not obtained directly from the domain's authoritative server; it does not by itself mean incorrect. Real output may contain multiple or IPv6 addresses.

The mental model: a distributed address book

People remember `example.com` more easily than a number, and a service can move without changing its name. DNS is not one global file. It is a distributed system in which different authorities answer for different parts of the name space.

An IP address identifies a network interface for routing:

- `192.168.1.20`: a private IPv4 example inside a LAN (Local Area Network, a network within an area such as a home or office).
- `203.0.113.10`: an IPv4 address reserved for documentation.
- `2001:db8::10`: an IPv6 documentation address.
- `127.0.0.1` and `::1`: loopback addresses for this machine.
- `localhost`: a name that normally resolves to loopback.

A private home address is not necessarily your public address; a router commonly performs NAT (Network Address Translation, rewriting IP addresses at a network boundary). An IP also does not permanently identify a person or device: addresses can change or be shared.

What does DNS return?

DNS returns typed records:

Record	Meaning
`A`	A name mapped to IPv4
`AAAA`	A name mapped to IPv6
`CNAME`	An alias pointing to another name
`MX`	Mail receivers for a domain
`TXT`	Verification text and policies
`NS`	Name servers responsible for a zone

One name may return several addresses for load distribution or locality. DNS locates a possible destination; it does not carry HTML (Hypertext Markup Language, a language describing page structure) or execute PHP (a programming language commonly used for server-side web processing).

A cache-miss journey

1. The browser checks its cached answers.
2. The operating system checks its hosts file and cache.
3. It asks a **recursive resolver** supplied by an ISP (Internet Service Provider, an organization providing internet connectivity), organization, or public service.
4. On a cache miss, the resolver asks a **root** server who handles `.com`.
5. Root refers it to the `.com` **TLD (Top-Level Domain, a top-level name component such as com)** servers.
6. The TLD refers it to the domain's **authoritative name server**.
7. The authoritative server returns the final record.
8. The resolver returns and caches the answer according to its TTL (Time To Live, a DNS cache lifetime or an IPv4 forwarding limit depending on context).

⚠

Important correction

The browser does not normally query root and TLD servers itself, and the full journey is not repeated for every request. The recursive resolver performs it when no valid cached answer exists.

TTL: speed versus change

TTL is the number of seconds an answer may be cached. A long TTL reduces queries but can make a record change take longer to appear to some clients. A short TTL allows clients to refresh sooner after cached values expire but produces more queries.

“DNS propagation” is not one global timer. Each cache keeps its old answer until expiry and may apply additional local policies.

Run a query and read it

```
nslookup -type=A example.com
nslookup -type=AAAA example.com
nslookup -type=MX example.com
```

An `A` response should resemble this, although the address and resolver can differ:

```
Name:    example.com
Address: 93.184.216.34
```

- `-type=A` specifically requests IPv4.
- `Name` is the resolved name.
- `Address` is one returned address, not proof that HTTP (Hypertext Transfer Protocol, the rules for web requests and responses) works.
- An `AAAA` query may return IPv6 or report no record.
- An `MX` query asks about mail servers, not the website.

Common mistakes

- Treating DNS as a connection to the server; it only discovers an address.

- Assuming `localhost` means a remote machine; it means the machine running the command.
- Expecting a changed record to appear immediately despite caches and TTL.
- Hard-coding an old IP and then breaking TLS (Transport Layer Security, rules for establishing an authenticated protected connection) or name-based hosting.
- Assuming a missing `AAAA` record means the independent `A` record cannot work.

Practical problems

▼ You changed an A record, but a colleague still reaches the old server. Did the change fail?

Not necessarily. Check the authoritative answer, the TTL, and the resolver your colleague uses. A previously cached answer may still be valid.

▼ `nslookup` succeeds but `curl` fails. What did the test prove?

It proved only that DNS returned an address. Routing, TCP (Transmission Control Protocol, ordered byte transport with loss recovery; connections can still fail), TLS, HTTP, or the service itself may fail later.

▼ Why not put an IP address directly in every application?

Addresses can change or multiply, and TLS plus virtual hosting rely on the host name. A name separates the service's logical identity from its current location.

Check your understanding

1. Who normally asks root and TLD servers? **The recursive resolver on a cache miss, not the browser directly.**
2. What happens when TTL expires? **The cached answer is no longer valid, so the client or resolver must query again.**
3. Does successful DNS mean the site will open? **No. DNS supplies an address; connection, TLS, HTTP, and application stages remain.**

Next step

After completing this practice, continue with DNS failures and protection.

DNS failures and protection

DNS failures and protection

After reading a successful lookup, distinguish a nonexistent name from lookup failure, then privacy from data authenticity.

Missing answers and DNS protection

A and AAAA return IPv4 and IPv6 addresses; CNAME aliases another name; MX identifies mail receivers; TXT carries text; NS identifies authoritative servers. **Negative caching** saves an answer that a name or record type does not exist, so a newly added record need not appear immediately to every client.

DNSSEC, Domain Name System Security Extensions, adds signatures that supporting resolvers validate through a trust chain; it does not encrypt queries. **DoH (DNS over HTTPS (HTTP carried over a TLS-protected connection))** and **DoT (DNS over TLS)** protect transport to a resolver, not the honesty of the returned website.

Worked check: An A query succeeds and an AAAA query has no address. That does not invalidate A or prove that the website is down. To inspect details including TTL where shown, try `nslookup -debug -type=A example.com`, then AAAA, MX, and NS. Separate answer records from other sections. The resolver, values, and remaining lifetimes vary.

TCP: from the first SYN to connection closure

Connections, byte streams, ordering, retransmission, flow and congestion control, closure, and diagnosis with a local PHP lab.

A network can lose, delay, duplicate, or reorder data. When sending a file or request, your application needs an ordered sequence of bytes and must handle failure if delivery cannot complete. TCP provides this transport service.

Prerequisites: **IP** addresses and routes packets between devices. A **port** is a number used to identify a transport endpoint within a device. DNS can discover an address; TCP can also connect directly to an address without a DNS lookup.

The service TCP provides

Transmission Control Protocol supplies an ordered **byte stream** between endpoints. A byte contains eight bits. TCP handles loss, ordering, and duplicates within a connection, but does not preserve application-message boundaries.

Two writes containing `Hello` and `World` might be read as `HelloWorld` or as smaller pieces. The number of writes need not match the number of reads. The application needs framing, such as a newline terminator or a length prefix.

Reliability does not mean the network cannot fail. Timeouts, resources, and recovery limits still exist. A connection is transport state maintained by endpoints, not an exclusively reserved physical circuit.

Identify the endpoints

A typical TCP connection is identified by each endpoint's IP address and port:

```
Client 127.0.0.1:51024  <-->  Server 127.0.0.1:9099
```

`127.0.0.1` refers to the local computer. The operating system normally assigns the client a temporary source port. The server listens on a known port. Another client can reach the same server using a different source endpoint.

A **socket** is the application's communication interface. A server **listens** for connections and **accepts** a connection to obtain a connected resource. Accepting a connection is separate from receiving an application message.

Establish a connection

The usual three-way handshake is:

```
Client                              Server
SYN, sequence=x  ------------------->
                 <------------------- SYN+ACK, sequence=y, ack=x+1
ACK, ack=y+1   ---------------------->
                 Application data can follow
```

SYN synchronizes sequence numbers. **ACK** acknowledges progress expected from the peer. Each direction has an independent sequence space. SYN consumes one sequence number, explaining the increment shown.

Completing this handshake means a TCP endpoint accepted the connection. It does not prove user authorization, application health, or database availability.

Sequence numbers and acknowledgements

For a simplified data range starting at 1000, bytes `ABC` occupy 1000–1002 and `DEF` start at 1003. Sequence numbers locate bytes rather than numbering purchase orders or merely counting packets.

An ordinary **cumulative ACK** identifies the next expected byte. An ACK of 1003 acknowledges the contiguous preceding bytes. TCP can retain later data while recovering a missing range, then deliver an ordered stream.

```
Received:      ABC     [gap]     GHI
Application:   ABC
After recovery: ABCDEF GHI in order
```

Retransmission resends data inferred to be lost using timers and acknowledgement information. Delay or reordering can make that inference imperfect. **Selective acknowledgements, SACK**, let the receiver report additional received ranges to help recovery.

Later bytes may wait behind a gap. This is transport **head-of-line blocking**: data exists but cannot yet be exposed as the next ordered bytes.

Segments and path size

Application bytes travel through layers:

```
Application bytes → TCP segment → IP packet → Ethernet/Wi-Fi frame
```

Each layer adds a **header** containing its control information. TCP includes ports, sequence numbers, and flags. IP includes addresses and routing-related information. MAC addresses concern the local link; routers select an IP packet's next hop.

MTU limits the higher-layer payload a link can carry. **MSS** is the maximum amount of TCP data a peer advertises it can receive per segment, excluding TCP/IP headers. MSS and MTU are not interchangeable; options and path information affect actual segmentation. A large application message is not necessarily one equally large packet.

Path MTU Discovery tries to find a usable path size. Filtering relevant control messages can make small exchanges succeed while larger transfers stall. This is one diagnostic hypothesis, not the explanation for every failed upload.

A **checksum** detects some accidental corruption. It is not protection against an attacker forging traffic. TCP alone does not encrypt application data; a protocol such as TLS supplies protected communication.

Receiver capacity versus network congestion

Flow control protects the receiver through its advertised receive window, `rwnd`. **Congestion control** adapts to conditions along the network path through the sender's congestion window, `cwnd`.

The smaller window helps bound unacknowledged data in flight, accounting for data already outstanding. A faster CPU does not automatically repair a congested network or an application that is not reading.

Different congestion-control algorithms use different policies. Do not assume every implementation has an identical reaction to loss.

Close the connection

Each direction can close independently. **FIN** means no more data will be sent in that direction. The peer acknowledges it and can finish its remaining output. The usual bidirectional close involves FIN and ACK for both directions, but messages can combine; four distinct packets are not compulsory.

A **half-close** ends sending while reception continues. **RST** resets a connection abruptly. Applications must distinguish a complete application message from EOF or an error.

TIME_WAIT is a normal state following orderly closure at the relevant endpoint. It helps handle delayed segments and retransmitted closing messages. Reducing it indiscriminately changes protocol behavior.

TCP **keepalive** can detect some dead idle peers according to its configuration. It differs from HTTP connection reuse and does not replace an application deadline.

Local PHP lab: a message is not a connection

Use PHP CLI in a training directory. Start a server and client in separate terminals. Bind only to `127.0.0.1:9099`; no internet exposure is required.

Save `tcp-server.php`:

```php
<?php
declare(strict_types=1);

$server = stream_socket_server('tcp://127.0.0.1:9099', $code, $message);
if ($server === false) {
    throw new RuntimeException($message, $code);
}
echo 'Listening', PHP_EOL;
$client = stream_socket_accept($server, 15);
if ($client === false) {
    fclose($server);
    throw new RuntimeException('No client connected');
}
stream_set_timeout($client, 5);
$line = fgets($client, 128);
if ($line !== "Hello\n") {
    fclose($client);
    fclose($server);
    throw new RuntimeException('Expected one short newline-terminated message');
}
$response = "Received: Hello\n";
while ($response !== '') {
    $written = fwrite($client, $response);
    if ($written === false || $written === 0) {
        throw new RuntimeException('Write failed');
    }
    $response = substr($response, $written);
}
fclose($client);
fclose($server);
```

Run:

```
php tcp-server.php
```

The server prints `Listening` and accepts one client within 15 seconds. After acceptance it applies a five-second read timeout. Save `tcp-client.php` and start it in a second terminal while the server waits:

```php
<?php
declare(strict_types=1);

$socket = stream_socket_client('tcp://127.0.0.1:9099', $code, $message, 5);
if ($socket === false) {
    throw new RuntimeException($message, $code);
}
stream_set_timeout($socket, 5);
$request = "Hello\n";
while ($request !== '') {
    $written = fwrite($socket, $request);
    if ($written === false || $written === 0) {
        throw new RuntimeException('Write failed');
    }
    $request = substr($request, $written);
}
$response = fgets($socket, 128);
fclose($socket);
if ($response !== "Received: Hello\n") {
    throw new RuntimeException('Missing or incomplete response');
}
echo $response;
```

```
php tcp-client.php
```

Expected client output:

```
Received: Hello
```

Here, `fgets` reads until a newline, a length limit, or end of the resource. The newline is **application framing**. The write loops are necessary because `fwrite` can accept fewer bytes than requested. This bounded teaching protocol rejects missing, incomplete, or oversized messages; it does not accept arbitrary-length input.

The server accepts one client and exits. The client first establishes TCP, then sends its message, then verifies an application response. These are different success conditions. This example does not encrypt the connection.

Diagnose observed failures

For `Connection refused`, check whether a service is listening on the intended endpoint. A rejecting network policy can also produce refusal. A timeout can reflect dropped traffic, routing trouble, or an unresponsive peer; the error alone does not identify the cause.

On Windows PowerShell:

```
Get-NetTCPConnection -LocalPort 9099 -ErrorAction SilentlyContinue
```

On Linux:

```
ss -tn
```

A short connection may end before inspection. A **packet capture** records traffic and can reveal handshakes, acknowledgements, retransmissions, and closure. It may expose sensitive content, so use a local lab.

`ping` uses ICMP rather than TCP. Success does not prove that a TCP server accepts connections on a particular port.

Delivery is not business completion

A **TCP ACK** concerns transport reception, not an order committed to a database. A server might execute a payment, then lose the connection before its response reaches the client. A disconnect alone cannot tell the client whether the payment happened.

Retrying can repeat the business operation. **Idempotency** is application-level protection against duplicating the intended effect, often using a unique request key and recorded outcome. TCP suppresses duplicate stream bytes within a connection; applications must handle repeated requests across connections.

Check your understanding

▼ Two writes were read together. Is TCP broken?
No. It supplies a byte stream. Define application framing with a delimiter, length, or protocol format.
▼ An ACK arrived but payment processing failed. Is that contradictory?
No. The ACK concerns transport. Payment validation, transactions, and application responses are separate.
▼ A handshake succeeds but a large upload stalls. What should you inspect?
Check whether the application reads, whether deadlines expire, whether retransmissions occur, and whether path size causes trouble. Gather evidence instead of assuming MTU is the cause.
▼ How do FIN and RST differ?
FIN orderly ends one sending direction while the opposite direction can continue. RST abruptly resets the connection.

References

- TCP specification — RFC 9293
- PHP stream_socket_server
- PHP stream_socket_client
- PHP fwrite and partial writes

UDP: independent messages without a TCP connection

Datagram boundaries, loss, ordering, duplication, message size, application responsibilities, and transport choices.

A game sends a player's position repeatedly. If an old update is delayed, displaying the newest position may be more useful than waiting for every old update. Choosing transport requires understanding both its service and the responsibilities left to your application.

UDP, User Datagram Protocol, transports independent **datagrams** over IP. IP addresses and routes packets; a port identifies a transport endpoint inside a device.

Message boundaries

Two sends create separate datagrams, although either can be lost. UDP does not combine messages into a byte stream like TCP. The receiver needs adequate message-buffer capacity. Some APIs truncate an oversized message and discard the excess instead of returning it on the next read; check the API's contract.

The basic UDP header occupies eight bytes: source port, destination port, length, and checksum. A **checksum** detects accidental corruption; it does not authenticate or encrypt messages. Checksum requirements vary with IP and extensions, so its presence is not an application security mechanism.

Responsibilities UDP does not provide automatically

UDP has no TCP connection-establishment handshake and supplies no built-in retransmission, delivery guarantee, ordering, duplicate suppression, or TCP-style congestion control. Applications must consider lost, reordered, and duplicated messages.

An API can provide UDP `connect` to select a default peer, filter reception, or improve error reporting. That call does not create a TCP handshake or reliable delivery.

Follow a message sequence

```
sent:     seq=10 x=40 | seq=11 x=42 | seq=12 x=45
received: seq=10 x=40 | seq=12 x=45 | seq=11 x=42
```

The application adds `seq`; it is not a UDP header field. Applying arrival order would move the player from position 45 back to 42 using stale state.

For some real-time state updates, retaining the highest applied sequence and ignoring older updates is appropriate. The final position is 45. This is not a complete solution for debiting money or creating an order: those operations require a known business outcome and duplicate protection.

Message size and the path

A link has an **MTU**, its maximum supported higher-layer payload size. A large datagram may require lower-layer fragmentation or fail, depending on protocols, configuration, and path. Losing one fragment can prevent reconstruction of the complete message.

A small header does not justify sending an entire large file as one datagram. Define message limits and recovery policies for the application. There is no single universally safe payload size independent of the path and protocol.

Suitable uses

DNS uses UDP for many exchanges but also supports TCP. Games and real-time media may use UDP with higher protocols supplying timing, ordering, or loss recovery appropriate to their data.

HTTP/3 uses **QUIC** over UDP. QUIC supplies reliable streams, encryption, and congestion control. These properties belong to QUIC, not to UDP itself.

Misleading assumptions

- “UDP is always faster”: measure useful delivery, latency, and load rather than header size alone.
- “The send succeeded, so the message arrived”: local API success is not acknowledgement from the remote application.
- “Retry forever”: uncontrolled retries can increase congestion and duplicate application effects.
- “The source address proves identity”: UDP does not authenticate senders; source addresses can be spoofed in some circumstances.

Public services must also consider **amplification**: an attacker sends a small request with a spoofed victim address, causing a larger reply to reach that victim. Address validation, response-size policy, and rate limits depend on the application’s protocol.

Exercise

A receiver has applied `seq=12`, then receives another `seq=12`, then `seq=11`. What should happen for a complete player-position update? How does the requirement change for “debit 10 pounds”?

▼ Reasoning and answer

A numbered latest-state policy can ignore duplicates and older updates. A financial operation requires authenticated identity, a unique operation identifier, durable recorded outcomes, and transaction guarantees. The client also needs a policy for an unknown outcome after a missing response. Transport choice follows those requirements rather than a blanket speed claim.

References

- UDP — RFC 768
- UDP usage guidelines — RFC 8085
- QUIC — RFC 9000

MTU: why small exchanges work while large transfers stall

Packet size, MTU, MSS, path discovery, and evidence that distinguishes network-size problems from application delays.

A connection opens and a small response arrives, but a large upload stalls. Possible causes include application waiting, slow storage, deadlines, and packet-size restrictions along the path. This lesson investigates the last hypothesis.

Prerequisites: IP carries packets; TCP divides a byte stream into segments and recovers missing bytes. FIN/RST and connection closure belong in the TCP lesson.

A file is not one packet

A 10 MB file need not be sent as one packet. Applications write bytes; networking layers divide them and add control information.

MTU, Maximum Transmission Unit, limits the higher-layer payload a link can carry. With an IP MTU of 1500 bytes, IP and transport headers consume part of that capacity.

MSS, Maximum Segment Size, concerns TCP data excluding TCP/IP headers. As a simplified calculation, MTU 1500 minus a 20-byte IPv4 header and 20-byte TCP header leaves 1460 bytes. Options or additional encapsulation change this; 1460 is not a universal setting.

A path contains multiple links

Suppose one link accepts 1500 bytes but a tunnel along the route permits only 1400. The **path MTU** follows the tightest size restriction along that path. Routes can change; your interface's value does not describe the entire route.

Oversized-packet handling depends on the IP version and flags. IPv4 can permit fragmentation, while **Don't Fragment** prohibits it. IPv6 routers do not fragment on the sender's behalf; the sending endpoint must respect path limits.

Path MTU Discovery comprises mechanisms to find a usable path size. Some rely on size-related ICMP messages; other packet-layer mechanisms probe and validate size without relying solely on those messages. Indiscriminately filtering control traffic can break discovery and create a connection that appears established while data stalls.

Gather evidence

Use controlled small and large transfers between systems you own. Record application timing, received-byte evidence, and packet observations. A **packet capture** records traffic and can expose sensitive contents, so scope it to your lab.

On Windows, an IPv4 ICMP test with fragmentation prohibited is:

```
ping -4 -f -l 1472 example.com
```

1472 is the ICMP payload in this test. Adding a basic 20-byte IPv4 header and 8-byte ICMP header gives 1500 bytes before link-layer framing. Actual headers, destination, and path determine whether that model applies. An explicit fragmentation-needed response provides size evidence; a timeout alone does not, because ICMP may be filtered.

This does not directly test TCP or justify a production MTU change. Inspect TCP progress, retransmissions, size-related messages, and application state for a controlled transfer.

Network transfer versus application processing

If logs show all upload bytes arrived before compression or video conversion began, the apparent stall can be computation or storage after network delivery. If transfer stops early with repeated retransmissions and size-related evidence, investigate the path.

A **checksum** detects some accidental corruption. A **retransmission** repeats data inferred lost. Neither encrypts the connection nor explains the cause of loss by itself.

Diagnostic exercise

A small request succeeds but a large upload fails. Before accepting a proposal to lower MTU, identify evidence distinguishing a path problem from a server that has already received the upload and is still processing it.

▼ Reasoning and answer

Compare application received-byte counts and timing with transport progress. A complete upload followed by high CPU or storage activity points toward processing. Early stalled transfer with size messages or retransmissions supports path investigation. Controlled size comparisons help; ping success or failure alone does not establish TCP behavior. Change one variable in a lab and measure again.

References

- IPv4 Path MTU Discovery — RFC 1191
- IPv6 Path MTU Discovery — RFC 8201
- Microsoft ping

Web addresses, ports, and HTTP

URL anatomy, ports, sockets, ranges, and the role of HTTP.

Beginner bridge

A URL (Uniform Resource Locator, an address identifying a resource and how to access it) identifies a resource and tells a client how to reach it. The scheme selects a protocol policy, the host selects a destination name, the port selects a listening service, and the path plus query identify the requested resource.

Keep URL parsing separate from DNS (Domain Name System, a distributed system answering queries about domain names) and connection establishment. A syntactically valid URL may resolve to no address, connect to no service, or receive an HTTP (Hypertext Transfer Protocol, the rules for web requests and responses) error. Each stage has its own evidence and debugging tools.

Before the details

Before the details: a URL says **what** you want and **where** to request it, a port selects the receiving program on that host, and HTTP defines the conversation.

HTTP is an application protocol defining request/response semantics for resources. It is not the network connection itself; it runs over transports such as TCP (Transmission Control Protocol, ordered byte transport with loss recovery; connections can still fail) or QUIC (a UDP-based transport protocol adding reliable streams and protected communication). HTTP is stateless by default, so applications add cookies, sessions, or tokens for continuity.

```
https://example.com:443/products/42?currency=EGP#reviews
scheme  host        port path         query        fragment
```

The fragment is normally browser-local and is not sent in the HTTP request. Never place passwords, tokens, or other secrets in URLs because history, logs, analytics, and referrers may expose them.

A TCP or UDP (User Datagram Protocol, independent messages without built-in delivery or ordering guarantees) port is a 16-bit number from 0 to 65535 identifying a service in the operating system. Common values include FTP (File Transfer Protocol) 21, SSH (Secure Shell, a protocol for secure remote access) 22, SMTP (Simple Mail Transfer Protocol, used for sending email between systems) 25, DNS 53, HTTP 80, HTTPS (HTTP carried over a TLS-protected connection) 443, MySQL 3306, and PostgreSQL 5432. A conventional port is configuration, not security.

A socket endpoint can be simplified as protocol + IP (Internet Protocol, the addressing and routing protocol; an IP address identifies a network interface in context) + port. A TCP connection is distinguished by client IP/port and server IP/port, allowing many clients to share server port 443.

For this optional practice, install PHP, create a new practice folder, and save hello.php containing `<?php echo 'Hello';`. The opening tag starts PHP and echo displays text. Open a terminal in that folder and run:

```
php -S localhost:8000
```

In `http://localhost:8000/hello.php`, the scheme is HTTP, host is localhost, port is 8000, and path is `/hello.php`. Omitting a port means the scheme's default, not "no port".

The browser should display Hello. localhost refers to your own computer; -S starts a development server. Keep its terminal open while testing and stop it with Ctrl+C afterward.

Practical problems

▼ Does the browser send `#section` to the server?

No. The fragment is normally browser-local, while the path and query string are sent.

▼ A service works at `localhost:8080`, but a URL without a port fails. Why?

HTTP defaults to 80 and HTTPS to 443. The URL must include `:8080` while the service listens there.

▼ Does port 443 guarantee encryption?

No. It is a convention; an actual TLS (Transport Layer Security, rules for establishing an authenticated protected connection) handshake creates encryption. A misconfigured plain service can listen on 443.

Next step

After completing this practice, continue with Characters and encoding in web addresses.

Characters and encoding in web addresses

Characters and encoding in web addresses

What happens to an address containing Arabic letters or a space? First learn the parts of a URL.

Encoded characters and address details

Percent-encoding writes a byte as two hexadecimal digits after %. For example, %20 represents a space. Hexadecimal compactly represents bits, explained in data representation. Repeated decoding can change meaning: %252F becomes %2F and then a slash.

URI, Uniform Resource Identifier, is the general resource-identifier term. A URL describes how to locate a resource. **IRI, Internationalized Resource Identifier**, supports international characters. **IDNA, Internationalized Domain Names in Applications**, governs international domain-name handling; it differs from percent-encoding a path or query. Some URI syntax permits user information, but passwords must not be embedded in web URLs.

Worked check: For `http://[::1]:8080/search?q=hello%20world&tag=a&tag=b#results`, the host is bracketed IPv6 loopback and the port is 8080. The HTTP/1.1 request target is `/search?q=hello%20world&tag=a&tag=b`. The fragment is not sent. Repeated tag parameters may become a list or a selected value depending on the application's parser.

TCP/UDP port ranges are 0–1023 system ports, 1024–49151 user/registered ports, and 49152–65535 dynamic/private ports in the IANA registry. **IANA, Internet Assigned Numbers Authority**, coordinates these assignments. Actual operating-system ephemeral-port ranges can differ. FTP control commonly uses 21, SSH secure remote access 22, SMTP mail transfer 25, DNS 53, HTTP 80, HTTPS 443, and the MySQL/PostgreSQL database services 3306/5432. A number is an agreed configuration, not proof of the running protocol.

Read an HTTP request and response

Read an HTTP request and response

Read one message before tracing a session

HTTP (Hypertext Transfer Protocol, the rules for web requests and responses) exchanges discrete request and response messages. A request method expresses intent, headers carry metadata, and an optional body carries a representation. The response status describes the protocol result; it should not be guessed from body text.

HTTP being stateless means the protocol does not automatically remember an earlier request. Applications build continuity with cookies, server-side sessions, or tokens. That state needs independent expiry, integrity, privacy, and authorization rules.

Before the details

Before sessions or tokens, separate **one HTTP message** from user state across messages. A request carries client intent; a response carries the server result.

An HTTP/1.1 request contains a request line, header fields, a blank line, and optional content:

```
POST /api/orders?notify=1 HTTP/1.1
Host: shop.example
Accept: application/json
Content-Type: application/json
Authorization: Bearer ey...
Cookie: session=abc123
Content-Length: 27

{"product_id":42,"count":2}
```

The example body is **27 UTF-8 (Unicode Transformation Format with 8-bit units, encoding Unicode character numbers as bytes) bytes**, with no trailing newline. Content-Length counts bytes, not visible characters. Bearer ey... is a placeholder credential, not a usable token. **JSON, JavaScript Object Notation**, is structured text mapping names such as count to values such as 2.

A **safe** method requests no change to resource state; incidental logging is allowed. An **idempotent** method has the same intended state effect when repeated, although response codes may differ. DELETE can be idempotent without being safe. A **representation** is returned data describing a resource.

GET retrieves, HEAD is GET without response content, POST performs resource-specific processing, PUT replaces, PATCH partially updates, DELETE removes, and OPTIONS describes communication options. Safe/idempotent semantics are contracts, not authorization. Never implement destructive work through GET.

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/orders/901

{"id":901,"status":"pending"}
```

Response classes are 1xx informational, 2xx success, 3xx redirection/cache, 4xx request/client error, and 5xx server error. 401 generally indicates missing/invalid authentication; 403 indicates understood but unauthorized.

Practical problems

▼ How do `401` and `403` differ?

401 means valid authentication is required; 403 means permission was refused; it does not prove the identity was known.

▼ How can HTTP be stateless while login persists?

Each protocol request is independent, but a cookie or token supplies an identifier that lets the application recover state.

▼ Should GET transfer account credit?

No. GET is expected to be safe and may be repeated by browsers or crawlers. Use a state-changing method with protection and safe retry design.

Next step

After completing this practice, continue with Remembering users and protecting updates.

Remembering users and protecting updates

Remembering users and protecting updates

After reading one request and response, connect a user's requests and prevent an older edit from overwriting a newer one.

Methods, fields, and concurrent changes

Method	Intended use	Safe?	Idempotent?
GET	Retrieve a representation	Yes	Yes
HEAD	GET metadata without response content	Yes	Yes
POST	Resource-specific processing, often creation	No	Not inherently
PUT	Replace resource state	No	Yes
PATCH	Partial modification	No	Depends on operation
DELETE	Remove the resource	No	Yes
OPTIONS	Describe communication options	Yes	Yes

Host selects the website name; Accept states response formats; Content-Type identifies the sent body; Authorization supplies credentials; Cookie sends matching stored values; User-Agent describes the client; Cache-Control sets caching policy. **Virtual hosting** serves several website names on one server. **JWT (JSON Web Token)** is one token format; a token need not use it or a server-side session store.

HTTP/2 and HTTP/3 use binary message representations rather than HTTP/1.1 text lines. Binary representation itself is not encryption. A 401 response requires valid authentication and its appropriate challenge; a 403 refusal does not prove that the user's identity is known.

Content negotiation selects a representation using fields such as Accept-Language and Accept-Encoding. **Vary** tells caches which request fields distinguish variants. An **ETag (Entity Tag)** identifies a representation version. **If-None-Match** supports checking a saved copy; **If-Match** permits modification only if the version still matches, preventing a **lost update**.

Worked check: A client read "v7", but another client saved "v8". An update with If-Match: "v7" must fail with **412 Precondition Failed** when the condition does not match. Checking the condition and updating must be atomic with respect to competing writes. The client should reread the latest version and handle the conflict.

Follow state and run a response

HTTP does not automatically remember an earlier request. Cookies are browser-stored values sent by matching rules; sessions are server-side state commonly linked by a cookie ID; tokens are client-sent credentials. Not every token is a JWT (JSON Web Token, a token format carrying claims; not all tokens use it), and JWT does not automatically replace session design.

This optional example requires PHP lab setup. Save it as hello.php in a new practice folder and run `php -S localhost:8000` there. Open `http://localhost:8000/hello.php?name=Omar`; the expected body is `{"message":"Hello Omar"}`.

```
<?php
header('Content-Type: application/json; charset=utf-8');

if (($_SERVER['REQUEST_METHOD'] ?? '') !== 'GET') {
    http_response_code(405);
    header('Allow: GET');
    echo json_encode(['error' => 'Method not allowed']);
    exit;
}

$rawName = $_GET['name'] ?? 'Guest';
if (!is_string($rawName)) {
    http_response_code(400);
    echo json_encode(['error' => 'name must be text']);
    exit;
}
$name = trim($rawName);
echo json_encode(['message' => "Hello {$name}"], JSON_INVALID_UTF8_SUBSTITUTE | JSON_THROW_ON_ERROR);
```

The request method is read from `$_SERVER`; `$_GET` supplies URL parameters. The `??` operator supplies a fallback, `is_string` checks that the name is text, and `trim` removes surrounding whitespace. `header` sets a response field, `http_response_code` sets the status, and `json_encode` builds JSON. Its flags replace invalid UTF-8 bytes and raise an exception for other encoding failures.

Test `?name[]=Omar`: the array input is rejected with400. POST is rejected with405 and Allow: GET. No name returns Guest. Stop the local server with Ctrl+C.

Connection encryption and website certificates

Understand TLS, certificate authorities, DV, OV, and EV through an explained connection and practical checks.

A password travels through networks before reaching a website. We need to prevent an observer from reading or changing it and verify the destination. **TLS, Transport Layer Security**, defines how two programs establish a protected connection. This lesson explains that connection and then the certificate labels.

Review HTTP messages first: the browser is a client sending a request, and the server is the program responding.

HTTP and HTTPS

HTTP, Hypertext Transfer Protocol, defines web requests and responses. **HTTPS (HTTP carried over a TLS-protected connection)** carries HTTP over TLS. GET still requests a resource, and 200 still indicates HTTP success; transport protection changes.

TLS provides **confidentiality**, preventing observers from reading protected content; **integrity**, detecting unauthorized changes; and **authentication**, checking evidence that binds the destination name to a key controlled by the server.

Protection ends at the connection endpoints. A compromised server can read decrypted data. HTTPS does not fix **SQL injection**, input that changes a database command, **XSS (Cross-Site Scripting)**, hostile code running in a user's page, or broken access permissions. The destination **IP (Internet Protocol, the addressing and routing protocol; an IP address identifies a network interface in context) address**, its network address, can remain visible, and the hostname may be exposed by parts of the connection process.

What a certificate says

A **domain name** is a name such as example.com. A **digital certificate** binds website names to a public key and validity period, with an issuer's signature.

A **CA, Certificate Authority**, checks a certificate applicant and signs the certificate. A browser or operating system maintains a **trust store**, a collection of trusted authorities. A certificate chain commonly runs from the website through an intermediate authority to a trusted **root CA**.

The client checks:

1. Whether the requested name appears in **SAN, Subject Alternative Name**, the certificate field listing covered names.
2. Whether the current time falls within the validity period; an incorrect device clock can cause errors.
3. Whether the signature chain reaches a trusted authority.
4. Whether **Key Usage and Extended Key Usage** permit the intended use.

5. Whether the server proves private-key possession and whether applicable revocation checks accept the certificate.

A valid certificate for shop.example does not validate bank.example. Encryption without checking the intended name is insufficient.

DV, OV, and EV

These labels describe **validation before certificate issuance**. Domain control means proving the ability to perform a required challenge in domain settings or at an approved location on the website. It is not a guarantee that every page is secure.

Label and full name	Plain meaning	Example of what it establishes
DV — Domain Validation	The applicant controls the domain name	Ability to publish an approved domain-verification record; no verified legal company identity
OV — Organization Validation	Domain control plus checks on the applicant organization	Organization identity and existence checked using accepted records and sources
EV — Extended Validation	Organizational validation with additional specified checks	Legal and operational existence and the authority of the certificate requester

OV and EV do not provide stronger encryption than DV. They can use the same TLS settings. The difference concerns identity checks, not guaranteed honesty. Browser certificate displays vary; do not identify certificate type from an obsolete color or interface icon.

Inspect a certificate

Open https://example.com and find certificate information through the browser's connection controls; exact menus vary. Record one SAN name, the **issuer**, and validity dates. These are public information, not the private key.

On Windows, open PowerShell from Start:

```
curl.exe -v https://example.com/ -o NUL
```

curl.exe is an HTTP client; -v shows connection details, and -o NUL discards page content. On Linux/macOS, use curl and /dev/null instead of NUL. Depending on the build, details may show TLS and ALPN. Do not bypass verification errors or print passwords and access tokens while diagnosing.

Worked exercise: A certificate-name mismatch appears. Would buying EV fix it? No: the certificate must cover the requested name and be configured correctly. If verification succeeds but HTTP returns 500, investigate request processing rather than replacing a valid certificate.

Mistakes and understanding checks

- Sending a password over http:// does not gain TLS protection; use the correct HTTPS address.
- Committing a private key to **Git**, a file-history tool, exposes it to history readers. Exposure requires key replacement and appropriate certificate revocation, not merely deleting a line.
- **Base64** is a reversible representation using text characters. It is **encoding**, not secret-key encryption.

▼ Does working HTTPS prove the site owner is honest?

No. It establishes a protected connection to the checked name. A fraudulent site can have a valid certificate for its own name.

▼ Why separate certificates from traffic keys?

A certificate binds identity to a public key. Setup verifies identity and establishes secrets; efficient symmetric traffic keys protect application data.

▼ Explain DV, OV, and EV in one sentence.

DV checks domain control, OV adds organization checks, and EV applies more extensive organizational requirements; this is identity validation, not an encryption-strength ranking.

You can now distinguish network reachability, certificate verification, and HTTP errors after connection setup. Next: how the browser renders a page.

References

- TLS 1.3 handshake messages.
- Website certificate requirements.
- Extended Validation requirements.

Next step

After completing this practice, continue with How peers establish an encrypted connection.

How peers establish an encrypted connection

How peers establish an encrypted connection

Study the details behind an encrypted connection after learning encryption, certificates, and issuers in the HTTPS lesson. Follow each message in order.

Keys are not the password you type

An **encryption key** is a value used by a cryptographic algorithm, a defined computational procedure, to protect or recover data.

- **Symmetric cryptography** uses shared secret material to protect messages efficiently. **AES-GCM** and **ChaCha20-Poly1305** are algorithms that combine encryption with detection of modification.
- **Asymmetric cryptography** uses a shareable **public key** and a secret **private key**. A **digital signature** is evidence created with the private key and checked with the public key.

We do not encrypt the entire page with the public key. Connection setup establishes identity and secrets; **traffic keys** then protect application data. The phrase **session keys** refers here to connection keys, not an application login session.

Older **RSA key exchange** encrypted a preliminary secret with the server public key; connection keys were derived from it. TLS 1.3 does not use that mechanism. A typical certificate-based TLS 1.3 handshake can use **ECDHE, Elliptic Curve Diffie–Hellman Ephemeral**, a temporary key-agreement method: each peer sends a public contribution and retains a secret contribution, and both derive the same secret without sending the final secret.

Correct ephemeral agreement and erasure of temporary secrets provide **forward secrecy**: later theft of the certificate private key alone cannot decrypt recorded earlier traffic. Resuming a previous connection has additional rules; one diagram does not cover every handshake mode.

Starting the connection

A **handshake** is the setup exchange. This simplified diagram shows a full certificate-based TLS 1.3 handshake:

```
Client -> ClientHello: supported versions, key share, hostname
Server -> ServerHello: chosen settings, server key share
Both derive keys protecting the rest of the handshake
Server -> EncryptedExtensions, Certificate, CertificateVerify, Finished
Client checks the certificate, signature, and handshake completion
Client -> Finished
Both exchange HTTP using application traffic keys
```

Hello messages begin negotiation. EncryptedExtensions carries protected settings. Certificate carries the certificate, CertificateVerify proves private-key possession and binds it to this handshake, and Finished checks the handshake agreement. **The certificate is not part of ServerHello itself.**

Cipher suites identify supported protection algorithms. In TLS 1.3, the chosen suite alone does not determine the key-agreement method or certificate type. Resumption can reduce setup messages. HTTP/3 uses **QUIC (a UDP-based transport protocol adding reliable streams and protected communication)**, a transport protocol integrating TLS 1.3, covered in the HTTP versions lesson.

Expiry and revocation

Expiry means reaching the end of the stated validity period. **Revocation** withdraws trust before expiry, for example after private-key exposure.

- **CRL, Certificate Revocation List:** a list of certificates revoked by an issuer.
- **OCSP, Online Certificate Status Protocol:** a protocol for querying certificate status.
- **OCSP stapling:** the server supplies an issuer-signed status response instead of requiring a separate client retrieval.

Support and behavior when status cannot be checked vary by client and configuration.

Related connection features

HSTS, HTTP Strict Transport Security, is a policy learned over valid HTTPS that asks the browser to use HTTPS for future connections during a specified period. Some sites are also covered by preload lists. HSTS does not repair an expired or mismatched certificate.

ALPN, Application-Layer Protocol Negotiation, negotiates an application protocol such as HTTP/2 within TLS. It is not another certificate type.

mTLS, Mutual TLS, also authenticates the client with a certificate. It can identify communicating services, but the application still decides which operations that identity may perform.

How the browser displays a page

Navigation, DOM, CSSOM, render trees, JavaScript execution, and practical DevTools analysis.

Change text size and measure width

Save as `render.html` or run it. **CSS** specifies presentation; `font-size` sets size and `px` is a CSS pixel, not necessarily one physical screen dot. `span` groups text; `style` sets presentation. `getBoundingClientRect()` reads the laid-out element box, whose `width` we display.

```
<!doctype html>
<html lang="en">
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>Measure a visible change</title>
<p><span id="sample" style="font-size:16px;display:inline-block">Hello</span></p>
<button id="grow">Grow text</button>
<p id="output"></p>
<script>
function measure() {
  const width = document.querySelector("#sample").getBoundingClientRect().width;
  document.querySelector("#output").textContent = "Width: " + width + " CSS pixels";
}
function grow() {
  document.querySelector("#sample").style.fontSize = "32px";
  measure();
}
document.querySelector("#grow").addEventListener("click", grow);
measure();
</script>
</html>
```

Record width before clicking. Text grows from16 to32 and measured width increases; exact numbers depend on font/browser. **DOM** represents elements; **layout** computes their geometry; **paint** prepares appearance. Resizing needs updated geometry, unlike merely changing color.

Before the details

Receiving HTML (Hypertext Markup Language, a language describing page structure) is not the end. The browser builds a DOM (Document Object Model, the page's element tree exposed to programs), resolves CSS (Cascading Style Sheets, rules describing element appearance), constructs what to paint, and runs JavaScript; network and script work can block parts of that path.

From a response to pixels

```
HTML bytes -> decode -> tokens -> DOM
CSS bytes  -> parse  -> CSSOM
DOM + CSSOM -> Render Tree -> Layout -> Paint -> Composite
```

- **DOM (Document Object Model)** represents elements and content as a tree, such as a page containing a heading and paragraphs.
- **CSSOM (CSS Object Model)** contains parsed style rules. Parsing reads structured text into a usable representation.
- **Layout** calculates geometry.
- **Paint** draws text, color, and borders.
- **Composite** assembles layers into the final frame.

Render-critical CSS can delay paint. A classic script can pause HTML parsing while it downloads and executes.

DevTools workflow

1. Use **Network** with cache disabled for controlled tests.
2. Inspect DNS (Domain Name System, a distributed system answering queries about domain names), connection, TLS (Transport Layer Security, rules for establishing an authenticated protected connection), TTFB (Time To First Byte, time until the first response byte according to the measurement tool), and download phases.
3. Check headers, transferred size, and protocol.
4. Use **Performance** for tasks, layout, and paint.
5. Use **Elements** for DOM and computed styles.

Tip

A high TTFB may come from the network, CDN (Content Delivery Network, distributed servers delivering content closer to users), PHP (a programming language commonly used for server-side web processing), or the database. Correlate browser traces with server logs and profiling.

Practical problems

▼ Why might a page briefly appear unstyled?

HTML may arrive before CSS. The browser can build the DOM but needs the CSSOM to apply styles during rendering.

▼ Where do you investigate a JavaScript file returning 404?

Start in Network for URL (Uniform Resource Locator, an address identifying a resource and how to access it), status, and initiator, then use Console to inspect the effect of the failed load.

▼ Does `DOMContentLoaded` mean every image finished?

No. It means HTML parsing completed and the DOM is ready; images and other resources may still be loading.

Measure visible behavior

Decoding reads bytes according to their text encoding; **tokens** are meaningful pieces such as tag boundaries. A **render tree** contains information used to display eligible elements. **Pixels** are small displayed color points. The arrows show responsibilities, not a promise that each happens only once.

LCP, Largest Contentful Paint, measures the display time of the largest eligible visible content element. **INP, Interaction to Next Paint**, measures interaction responsiveness over a visit. **CLS, Cumulative Layout Shift**, measures unexpected layout movement. They answer different questions.

The **main thread** handles much page execution. The **event loop** schedules work; **microtasks** run at specified checkpoints after current execution and before a subsequent rendering opportunity. A never-ending microtask chain can delay rendering. Painting need not happen after every task. **Layout thrashing** is repeated geometry reads and writes forcing repeated layout computation. Group reads and writes where appropriate. **Accessibility** includes keyboard and assistive-technology use.

The script snippets reference example files; they demonstrate loading attributes, not standalone programs. Modules form a **dependency graph**, a relationship map showing which files import others. DOMContentLoaded follows HTML parsing and relevant deferred/module execution, not completion of every image.

Worked check: A large image appears late although HTML arrived quickly. Inspect when its request starts, its size, and code delaying its display. Change one factor and compare several loads under the same conditions. Faster server response alone does not prove faster image display.

Next step

After completing this practice, continue with Load scripts and schedule browser tasks.

Load scripts and schedule browser tasks

Load scripts and schedule browser tasks

First change a page and measure its size. A **module** declares provided code and imports. A **dependency graph** connects files to what they need; if A imports B, B must be prepared according to loading rules before dependent execution. Relate scheduling to responsiveness.

JavaScript loading

```
<script src="/app.js" defer></script>
<script type="module" src="/main.js"></script>
```

`defer` preserves order and runs after DOM parsing. Modules are deferred by default. `async` runs as soon as it is ready and does not preserve ordering between independent scripts.

Repeated layout reads and writes can cause layout thrashing. Batch reads and writes, and prefer compositor-friendly properties such as `transform` where appropriate.

Event loop and responsiveness

Long JavaScript tasks block input and rendering. Inspect Largest Contentful Paint, Interaction to Next Paint, Cumulative Layout Shift, and main-thread long tasks.

Why the second load can differ

A **redirect** asks the browser to request another address. **Connection reuse** sends another request over an existing connection instead of repeating setup. A **Service Worker** is a website-registered browser program that can handle selected requests, including serving saved local responses.

Open browser **DevTools**, the developer inspection tools, and choose **Network** before loading. The **waterfall** displays each request's timing. A **cookie** is a browser-stored value sent with matching requests; **CORS** controls whether page scripts may read selected cross-origin responses, covered later.

Worked check: The second request has no new connection phase. Does that prove a cache hit? No. It may reuse an existing connection and still reach the server. Inspect explicit cache indicators and response headers rather than inferring the source from speed alone.

Set up a local request-and-response lab

Set up a local request-and-response lab

Run a browser client and a data server locally. A **client** requests; a **server** responds. Different **ports** identify receiving programs. Use complete provided files and investigate one behavior at a time.

Setup

1. Install Node.js and verify `node --version` using local server setup. No extra packages are required.
2. Create `network-practice`. Save the complete lab.mjs using Save link as; check it is not `lab.mjs.txt`.
3. Open PowerShell in that folder from File Explorer's address bar and run:

```
node lab.mjs
```

4. Visit `http://127.0.0.1:8765/` for the buttons. The API runs at `http://127.0.0.1:8766`. Keep the terminal open: requests log path, status, duration, and ID.
5. Stop with Ctrl+C. If a port is occupied, stop the earlier lab. Restarting clears event data and rate counters.

Start with one response

Visit `http://127.0.0.1:8766/api/products/1`: expect Notebook, price20, currencyEGP. Open developer tools with F12, select Network, and reload. The request has status200. Change the final1 to2: expect404, a missing product in this lab.

In another **Windows PowerShell** window:

```
curl.exe -i http://127.0.0.1:8766/api/products/1
```

`-i` displays response headers. Use `curl.exe` to select the executable; some PowerShell versions alias `curl` to another tool. On macOS/Linux use `curl` instead.

Experiment map

Experiment	Route or button	Evidence
One server	`/api/products/1` on8766	200 and JSON
One proxy	Through one proxy	Same product; two headers carry the same request ID
Cross-origin reading	CORS buttons	Both responses arrive, but JavaScript reads only the allowed one
Cached copy	`/cache` on8766	`max-age=2`, version tag, conditional304
Polling	Poll three times	Three requests and times
Server stream	Receive three events	One extended request, three events
Missing route	`/missing` on8766	404 with request ID
Intentional failure	`/failure` on8766	500 with request ID
Rate limit	`/limited` on8766 three times quickly	First two200, third429 within a ten-second window

The loopback-only lab uses no personal data or accounts. Its settings teach behavior, not deployment or payments. The complete source is available; each lesson explains its relevant experiment.

A request's journey inside the server

Load balancers, web servers, virtual hosts, PHP-FPM, applications, and the response path.

One server makes the flow concrete

Set up the local lab. Visit `/api/products/1` on8766: read path, choose product, construct JSON, return200 and body. Product2 returns404. A **route** maps an address to application work.

```
Browser → one server → choose route → build response → Browser
```

Match the terminal log to Network status. The next lesson adds a proxy in front; it is not required for this first request.

Beginner bridge

Once traffic reaches the backend, several programs may cooperate: a load balancer chooses a target, a web server handles HTTP (Hypertext Transfer Protocol, the rules for web requests and responses) and static files, PHP-FPM (PHP FastCGI Process Manager, managing PHP workers behind a web server) manages PHP (a programming language commonly used for server-side web processing) workers, and the application talks to caches or databases. These are responsibility boundaries, even when one machine runs several of them.

Debug from the outside inward. First identify which component produced the status, then correlate request IDs and timestamps. A gateway timeout, an application exception, and a database lock can all look like a slow page but require different evidence.

Before the details

This lesson follows a request after it reaches the backend. The goal is not memorizing products; it is learning responsibility boundaries so you inspect the right failure layer.

What is a server?

Server describes a role rather than one physical shape. It may mean physical hardware or a virtual machine supplying compute and networking, a program listening on an IP (Internet Protocol, the addressing and routing protocol; an IP address identifies a network interface in context) address and port such as Nginx or PostgreSQL, or the provider side of one interaction. The same application is a server to a browser and a client of a database.

A server process normally opens and binds a socket, listens, reads and validates requests, performs work, and returns responses. Running `localhost:8000` means a process on your machine listens locally; it does not imply a separate computer. Development may combine web, application, and database roles, while production separates them for scaling and isolation.

“The server is down” is incomplete until you identify whether the host, process, network, dependency, or capacity failed.

```
Network → Load Balancer → Web Server
                          ├─ Static file → Response
                          └─ PHP-FPM → App → Database/Cache/API → Response
```

A load balancer distributes traffic across healthy backends. Layer 4 works with transport connections; Layer 7 understands HTTP. It may terminate TLS (Transport Layer Security, rules for establishing an authenticated protected connection), perform health checks, and add forwarding headers. Only trust forwarded client IP headers from configured trusted proxies.

Nginx or Apache selects a virtual host, serves static assets directly, reverse-proxies services, or forwards dynamic PHP through FastCGI. Nginx does not execute PHP inside its own process; PHP-FPM workers run the entry point and return headers/content.

The application routes the request, runs middleware, validates input/authentication/authorization, executes business logic, and may use a database, cache, queue, or external API (Application Programming Interface, a defined contract for requesting data or actions from another component).

```php
<?php
header('Content-Type: application/json; charset=utf-8');

try {
    echo json_encode(['id' => 42, 'name' => 'Keyboard'], JSON_THROW_ON_ERROR);
} catch (Throwable $e) {
    error_log($e);
    http_response_code(500);
    echo json_encode(['error' => 'Internal server error']);
}
```

Do not expose stack traces or secrets. On the return path, layers may add compression, security headers, and caching. A browser/CDN (Content Delivery Network, distributed servers delivering content closer to users) cache hit skips the app; a static asset can stop at Nginx; a dynamic request reaches PHP-FPM. A proxy commonly returns 502 for an invalid/unavailable upstream and 504 for an upstream timeout, while 500 generally comes from the processing server/application.

Use request IDs and timing to trace the whole path rather than measuring PHP alone.

Practical problems

▼ Nginx returns `502`. Did PHP necessarily run and fail?

No. The gateway received no valid upstream response. Inspect PHP-FPM, connectivity, and timeouts before application logic.

▼ When can one user reach different application servers?

A load balancer may distribute requests without sticky sessions. Important state should therefore not live only in one server process.

▼ Who normally serves a static CSS (Cascading Style Sheets, rules describing element appearance) file?

A web server or CDN can serve it directly, so a typical setup does not send it through PHP.

Understand the backend terms

A **backend/upstream** is the next service selected by an intermediary. **Layer 4** refers to transport handling; **Layer 7** to application handling in the seven-layer **OSI (Open Systems Interconnection)** teaching model. A **health check** tests whether a service can respond. **Round robin** takes turns; **least connections** prefers fewer active connections. **Sticky sessions** try to keep a client on one instance, while a **shared session store** makes session data available across instances.

TLS termination decrypts the incoming connection at an intermediary. The next connection requires its own protection decision. **PHP-FPM, PHP FastCGI Process Manager**, manages ready PHP worker processes. **FastCGI** carries requests between a web server and an application execution environment. An **entry point** is the initial application file, a **route** maps a request to a handler, **middleware** performs shared processing around it, and **business logic** defines the application rules. A **stack trace** lists execution locations associated with a failure; do not disclose it or secrets in public responses.

Budget time and memory

A **timeout** limits work or waiting. For a two-second client budget, example limits are 0.6 seconds for a database query, 1.3 for application work, and 1.6 for proxy waiting, leaving response-transfer margin. These partly nested deadlines are not simply durations to add. Client disconnects do not always cancel server work automatically.

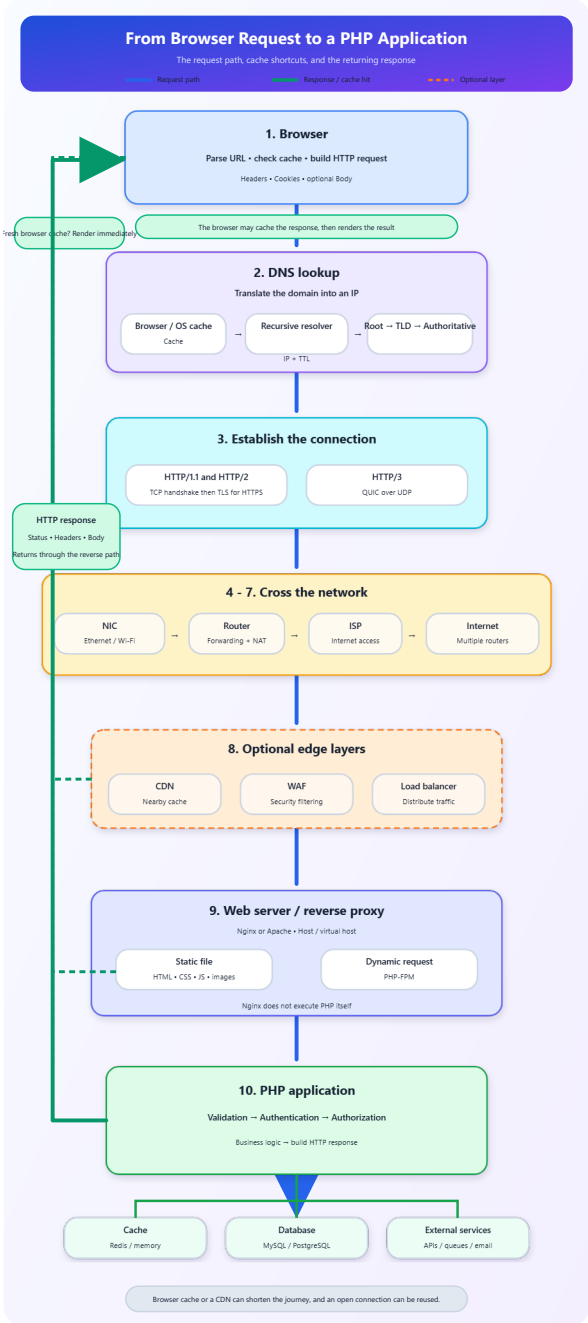
Saturated workers create a queue. More workers require a RAM (Random Access Memory, working memory for active program data and instructions) budget and can otherwise cause **OOM, Out Of Memory**. A **Unix socket** provides local process communication; TCP (Transmission Control Protocol, ordered byte transport with loss recovery; connections can still fail) works locally or across hosts and **containers**, isolated application execution environments.

Worked check: A proxy times out after one second while processing takes 1.5 seconds. Check whether work is slow relative to the intended contract or the deadline is too short. Increasing every limit blindly can hide faults and increase queued work.

For practice, inspect request method, status, content type, and cache indicators in Network. Compare a static CSS file with a dynamic endpoint, propagate a request ID in a test service, and draw the actual path rather than assuming every optional layer exists.

What happens after Enter?

1. The browser parses the URL and checks local cache and policies.
2. DNS turns the host name into an IP address if no valid answer is cached.
3. The operating system chooses a network interface and route; data crosses Wi-Fi or Ethernet, a router, an ISP (Internet Service Provider, an organization providing internet connectivity), and other networks.
4. The client establishes TCP for HTTP/1.1 or HTTP/2, or QUIC (a UDP-based transport protocol adding reliable streams and protected communication) over UDP (User Datagram Protocol, independent messages without built-in delivery or ordering guarantees) for HTTP/3.
5. With HTTPS, a TLS handshake authenticates the server and creates an encrypted channel.
6. The browser sends an HTTP request containing a method, path, headers, and sometimes a body.
7. A CDN (Content Delivery Network, distributed servers delivering content closer to users), WAF (Web Application Firewall, a system applying security rules to web requests), or load balancer may receive it before the origin server.
8. A web server returns a static file or forwards dynamic work to an application, such as PHP through PHP-FPM (PHP FastCGI Process Manager, managing PHP workers behind a web server).
9. The application validates input, identity, and permissions and may read a cache or database.
10. An HTTP response returns with a status code, headers, and body. The browser interprets and renders it.



Open the diagram full size

Note

This is a comprehensive map, not a mandatory fixed route. Browser or CDN cache may answer without reaching PHP, and a small application may have no load balancer or database.

Responsibility boundaries

Component	Main responsibility
Browser or client	Build the request, manage cache/cookies, interpret the response
DNS	Resolve a name to suitable addresses
TCP or QUIC	Carry data and provide connection behavior
TLS	Encryption, integrity, and server authentication
HTTP	Define request and response meaning
Web server	Receive HTTP, serve static files, forward dynamic work
PHP	Run application logic and build a response
Database	Store and retrieve data

Proxies and request distribution

Practical differences among servers, forward and reverse proxies, load balancers, and API gateways, including trust and failure boundaries.

Try one proxy

In the local lab, click Through one proxy. The browser requests `/proxy` on8765; that server requests the product on8766 and returns the response. The **upstream** is8766.

Inspect `/proxy` in Network: `X-Lab-Proxy: forwarded` identifies forwarding; `X-Request-Id` and `X-Upstream-Request-Id` match. Compare the terminal's two log entries. Understand this single path before distributing requests among copies.

Before the details

Every layer in front of an application should solve a named problem. Unneeded proxies add failure points; clear boundaries can improve security and scaling.

Why place layers in front of an application?

Production systems often need TLS (Transport Layer Security, rules for establishing an authenticated protected connection), protection, traffic distribution, routing, observability, and shared policies. One product may perform multiple roles, yet their responsibilities remain different.

```
Client ⇄ DNS (name lookup only)
Client → CDN/WAF → Load Balancer → Reverse Proxy → API Gateway
                                                    ├─ Identity
                                                    ├─ Orders
                                                    └─ Catalog → Cache/DB/Queue
```

Not every system needs every layer. A small app may use Nginx for TLS and reverse proxying; a multi-service platform may need a gateway.

Forward and reverse proxies

A **forward proxy** acts for clients. Corporate devices send outbound traffic through it for policy, destination controls, caching, or auditing. It may hide an address from the destination but is not automatically anonymous; the proxy can identify and log the client.

A **reverse proxy** acts for servers. Clients request the app normally and the proxy selects an upstream. It may terminate TLS, route hosts and paths, compress or cache responses, enforce body limits, add request IDs, and hide internal addresses. Nginx, HAProxy, and Envoy can perform these duties. A proxy cannot replace application validation and authorization.

Role comparison

Component	Acts for	Primary decision	Typical use
Server	Service	How to process a request	App or database
Forward proxy	Client	Whether/where traffic exits	Corporate network
Reverse proxy	Servers	Which upstream receives it	TLS and host routing
Load balancer	Replicas	Which healthy replica gets load	Worker distribution
API gateway	API platform	Which API, identity, and policy apply	Microservice entry

One product can combine roles. Review the responsibility, trust boundary, and failure mode rather than relying on a product label.

Practical problems

▼ What is the central difference between forward and reverse proxies?

A forward proxy represents clients toward the Internet; a reverse proxy receives client traffic on behalf of servers.

▼ Why not trust every `X-Forwarded-For` value?

A client can forge it. Trust only a header rebuilt by a trusted proxy and configure the trusted-proxy list.

▼ Does an API gateway replace authorization inside a service?

No. It can apply shared policy, but each service remains responsible for access rules specific to its resources.

Next step

After completing this practice, continue with Load distribution and proxy policies.

Load distribution and proxy policies

Load distribution and proxy policies

Start after trying one proxy. Multiple service copies require selection rules, waiting and retry limits, and a clear trust boundary for incoming proxy metadata.

Connect each term to a decision

With two application copies, **load balancing** selects a recipient. A **timeout** bounds waiting; a **retry** makes another attempt; a **retry budget** bounds attempts. A **health check** evaluates a copy's ability to serve work.

A **token** carries a credential that the server verifies; a **quota** bounds total use over a period. **Business logic** includes ownership and permitted edits, finally enforced by the data-owning service. **Rollback** restores a known earlier configuration after trouble. These terms describe decisions, not a tool installation list.

Load balancers

A **load balancer** distributes work across healthy instances. Layer 4 operates on TCP (Transmission Control Protocol, ordered byte transport with loss recovery; connections can still fail)/UDP (User Datagram Protocol, independent messages without built-in delivery or ordering guarantees) connections; Layer 7 understands HTTP (Hypertext Transfer Protocol, the rules for web requests and responses) hosts, paths, and headers. Common methods include round robin, least connections, and consistent hashing.

Separate liveness from readiness: a process can be alive but unable to accept work. Unlimited retries multiply load, so define timeouts, retry budgets, and idempotency for repeatable operations.

API gateways

An **API (Application Programming Interface, a defined contract for requesting data or actions from another component) gateway** is an organized API entry point. It may route versions, validate a token, enforce quotas, transform protocols, aggregate responses, and emit shared metrics and traces. The service that owns a resource must still enforce its authorization.

Do not move all domain logic into the gateway. That creates a central monolith and release bottleneck. Keep shared edge policy in the gateway and domain rules in their owning services.

Trust, security, and failure

Never trust `Forwarded` or `X-Forwarded-For` from arbitrary clients. Accept them only from configured trusted proxies that sanitize the chain. Do not forward credentials to unintended upstreams; cap header/body sizes and timeouts; protect management APIs; use internal TLS when the threat model requires it.

Every layer adds latency and a failure point. Replicate critical layers, use meaningful health checks, roll configuration out gradually, and propagate a correlation/trace ID. A `502` usually means an invalid or unavailable upstream response; `504` means the upstream deadline expired. Confirm with both sides' logs and a trace.

Choosing distribution and bounding retries

Round robin takes turns among instances, a reasonable starting point for similarly costly requests. **Least connections** prefers fewer active connections, which can help with long-lived connections but does not directly measure their work. **Consistent hashing** maps a key to an instance while limiting remapping when membership changes.

Session affinity tries to keep a client on one instance; it does not make local state survive instance failure. **Liveness** asks whether a process is alive; **readiness** asks whether it can accept work now. A **retry budget** limits retry counts or time. Three layers each making three attempts can produce 27 downstream attempts.

An **idempotency key** identifies an operation whose recorded result prevents repeated effects. **Rate limits** cap request rates; **quotas** cap total consumption. **Metrics** aggregate measurements, **logs** record events, and **traces** link a request across stages.

Worked check: Short independent reads can use round robin. A **WebSocket**, a persistent bidirectional channel, normally stays on the instance that accepted it for that connection's lifetime. Least-connections selection can help new connections. A reconnect must not assume that messages or local state have automatically been restored.

Web caching and compression

Freshness, validation, Cache-Control, ETag, Vary, CDNs, gzip, and Brotli.

Three moments for one representation

A **cache** saves a reusable copy. The lab's `/cache` sends `ETag: "lesson-v1"`, a version identifier, and `Cache-Control: public, max-age=2`, a two-second freshness lifetime.

Moment	Action	Result
First request	No copy exists	200 with content
Ordinary request while fresh	Browser may reuse its copy if policy allows	No origin body download needed
After expiry	Ask whether the version changed	304 if unchanged; reuse saved body

Start the lab and run:

```
curl.exe -i http://127.0.0.1:8766/cache
Set-Content -LiteralPath cache-headers.txt -Value 'If-None-Match: "lesson-v1"' -Encoding ascii
curl.exe -i -H "@cache-headers.txt" http://127.0.0.1:8766/cache
```

`Set-Content` writes the request header to a practice file; `-H "@cache-headers.txt"` reads it unchanged, preserving quotation marks across PowerShell versions.

Expect200 then304 without a body. `If-None-Match` supplies the stored version tag. curl does not implement a browser cache here: the second command explicitly simulates revalidation. In Network leave Disable cache unchecked; reload can force revalidation and is not always an ordinary request. Understand this sequence before policy and compression details.

Before the details

A cache reuses an earlier result to reduce latency and work. The safe questions are who may store it, for how long, and how a client validates change.

Why cache a response?

Caching reduces latency, bandwidth, PHP (a programming language commonly used for server-side web processing) work, and database load. Copies may exist in a browser, proxy/CDN (Content Delivery Network, distributed servers delivering content closer to users), or application, and each layer needs an explicit policy.

Freshness and validation

```
Cache-Control: public, max-age=60, s-maxage=300
ETag: "product-42-v7"
Vary: Accept-Encoding, Accept-Language
```

- `max-age` sets freshness lifetime in seconds for private and shared caches unless a more specific policy overrides it.
- `s-maxage` overrides that lifetime for shared caches.
- `private` prevents shared reuse.
- `no-store` asks caches not to store.
- `no-cache` permits storage but requires validation before reuse.
- `Vary` adds selected request fields to the cache key.

An expired response can be revalidated with `If-None-Match`. If unchanged, the server returns `304 Not Modified` without a body. `Last-Modified` and `If-Modified-Since` are another validator pair.

PHP example

```
<?php
// Save inside the downloadable php-labs folder.
require __DIR__ . '/http-router.php';
```

Never mark personalized data `public`.

Compression

gzip and Brotli reduce text assets such as HTML (Hypertext Markup Language, a language describing page structure), CSS (Cascading Style Sheets, rules describing element appearance), and JSON (JavaScript Object Notation, a text format for structured values and lists). Configure them in the web server or CDN, avoid recompressing already compressed images without evidence, and vary cached representations by `Accept-Encoding`.

Checklist

- Define who can store the response and for how long.
- Use fingerprinted filenames for immutable static assets.
- Test invalidation before choosing a long lifetime.
- Do not publicly cache personal responses or unsafe `Set-Cookie` responses.
- Monitor hit ratio, transferred size, and latency.

Reference

- RFC 9111: HTTP Caching

Practical problems

▼ Why not put an account page in a shared cache?

It is user-specific and could leak. Apply an appropriate private or no-store policy and prevent CDN sharing.

▼ What does `304` mean?

The cached representation is still valid according to a validator. The server omits the full body and the client uses its copy.

▼ When is `Vary: Accept-Encoding` needed?

When content differs for gzip or br so a cache does not serve an encoding that the client cannot decode.

Stale responses and safe cache keys

A **stale** response has passed its direct-reuse lifetime; that does not prove the content changed. **stale-while-revalidate** permits bounded stale reuse during validation. **stale-if-error** can permit a bounded old response when fetching fails. Sensitive decisions need carefully limited policies.

A **cache key** determines which requests share a saved response. Omitting a response-changing input can cause **cache poisoning**, storing inappropriate content for others. A **side channel** reveals information indirectly; compressing secrets beside attacker-controlled text can expose clues through response size.

Worked check: A public price page varies by language and supported compression. Distinguish variants using appropriate Vary fields and cache configuration. Do not share a personalized account response. Set-Cookie alone does not automatically forbid every shared cache from storing; choose an explicit policy.

Freshness allows direct reuse; **validation** asks whether a stored version remains valid. **ETag** identifies a version, Last-Modified states its modification time, and the corresponding conditional fields request a check. **gzip** and **Brotli** are compression formats. Fingerprinted filenames contain a content identifier so a changed asset can use a different URL

(Uniform Resource Locator, an address identifying a resource and how to access it). **Hit ratio** is the fraction of lookups served from the cache.

In the lab below, a **weak entity tag** begins W/ and can indicate equivalent content without promising identical bytes. GET/HEAD If-None-Match uses weak comparison. The downloadable router, rather than the short include snippet alone, supplies the complete runnable example.

Conditional requests, step by step

Use the local lab, start `php -S 127.0.0.1:8097 http-router.php`, then run `php http-client-lab.php` in a second terminal. Inspect `http-router.php` and `src/Http.php`: the route accepts GET/HEAD only, computes the current representation's ETag, parses a list of entity tags, and uses weak comparison for If-None-Match. `W/"abc"` can match `"abc"`; `*` matches an existing representation. Splitting on every comma is wrong because a quoted tag may contain a comma. Matching GET/HEAD returns 304 with no body; HEAD without a match returns 200 with headers and no body. This endpoint rejects other methods with 405; a general write endpoint would use 412 for a failed If-None-Match precondition. Malformed fields receive 400 under this application policy.

Exercise: send a nonmatching tag, a matching weak tag in a list, and `*`. Expect 200, 304, and 304. These tags describe the actual selected representation: if compression or language changes its bytes, use an appropriate variant strategy and Vary headers.

RFC 9110, If-None-Match

Controlling cross-origin data access

Origins, browser boundaries, simple requests, preflight, credentials, and safe configuration.

Same response, blocked then readable

Start the local lab on8765. An origin consists of scheme, host, and port;8766 differs.

1. Open Network and click CORS: blocked read. `/cors/no` can show200 while JavaScript reports Blocked: it lacks read permission.
2. Click CORS: allowed read. `/cors/yes` adds `Access-Control-Allow-Origin: http://127.0.0.1:8765`, so the page reads the same data.
3. Compare response headers, not status alone. Direct navigation to the URL is different from a script reading another origin.

Explain: this permission proves neither login nor product ownership. This simple GET differs from requests needing preflight, covered below. CORS guide.

Before the details

The Same-Origin Policy stops a malicious page from reading another site's data with the user's browser privileges. CORS (Cross-Origin Resource Sharing, browser rules for permitted cross-origin response access) is a server-declared exception, not a firewall against every client.

What is an origin?

An origin is **scheme + host + port**. `https://app.example` therefore differs from `http://app.example`, `https://api.example`, or a different port.

The same-origin policy stops JavaScript from reading many cross-origin resources without permission. It does not prevent every request and does not control server-to-server clients.

CORS response fields

```
Access-Control-Allow-Origin: https://app.example
Access-Control-Allow-Credentials: true
Vary: Origin
```

Credentials cannot be combined with a wildcard origin. Match a strict allow-list and never blindly reflect an arbitrary `Origin`.

CORS, Cross-Origin Resource Sharing, is a server-declared permission for browser scripts to read selected cross-origin responses. **Credentials** include browser-managed authentication data such as cookies. An **allow-list** explicitly enumerates accepted origins.

A so-called **simple request** meets the method, field, and content-type restrictions for sending without a preflight. Not every GET qualifies, and not every POST needs preflight. JSON (JavaScript Object Notation, a text format for structured values and lists) POST normally requires it; an eligible form submission may not. Response reading remains subject to CORS.

Preflight

```
OPTIONS /api/orders HTTP/1.1
Origin: https://app.example
Access-Control-Request-Method: POST
Access-Control-Request-Headers: Content-Type, Authorization
```

The server answers with allowed methods, fields, and an optional cache duration. Handle `OPTIONS` before middleware that requires credentials which preflight does not carry.

A **preflight** is the preliminary OPTIONS permission request. A 204 means success without response content. **Authentication** checks identity, **authorization** checks allowed actions, and **CSRF, Cross-Site Request Forgery**, tricks a browser into an unintended authenticated operation.

Minimal PHP handling

This is a fragment inside a PHP (a programming language commonly used for server-side web processing) request handler, after its opening PHP tag and before output, not a complete login service. It illustrates CORS fields; the real request still needs authentication and authorization. See PHP setup for the runtime and local server.

```
$allowed = ['https://app.example', 'https://admin.example'];
$origin = $_SERVER['HTTP_ORIGIN'] ?? '';

if (in_array($origin, $allowed, true)) {
    header("Access-Control-Allow-Origin: {$origin}");
    header('Vary: Origin');
    header('Access-Control-Allow-Credentials: true');
}

if (($_SERVER['REQUEST_METHOD'] ?? '') === 'OPTIONS') {
    header('Access-Control-Allow-Methods: GET, POST');
    header('Access-Control-Allow-Headers: Content-Type, Authorization');
    http_response_code(204);
    exit;
}
```

Danger

CORS is not authentication, authorization, or a replacement for CSRF protection. Non-browser clients can ignore it, so the server must validate every request.

Practical problems

▼ Does CORS stop `curl` from sending a request?

No. Browsers enforce CORS for JavaScript. The server still needs authentication and authorization.

▼ Why can credentials not use a wildcard origin?

Cookie-bearing responses must allow a specific origin, not every site, together with `Access-Control-Allow-Credentials`.

▼ When does a browser send a preflight?

Before a non-simple cross-origin request, using OPTIONS to ask whether the intended method and headers are allowed.

Inspect both responses

Preflight permissions have a cache distinct from ordinary response content caching. Check whether OPTIONS failed or whether it succeeded but the actual response omitted the correct fields. curl does not enforce browser JavaScript response-reading policy.

no-cors is not a way to bypass restrictions and read data. An **opaque response** hides its status and body from the page script.

Worked check: OPTIONS succeeds but the actual reply omits Access-Control-Allow-Origin. Fix the actual response fields, including appropriate error responses, using the allow-list. Disabling browser protection or adding no-cors does not produce a readable authorized response.

Resource loading and window isolation

Resource loading and window isolation

Read the CORS experiment first: it governs a script's access to a cross-origin response. These policies answer different questions:

Policy	Full name	Question
CORP	Cross-Origin Resource Policy	May a resource be used in relevant cross-origin requests, including no-cors loads?
COEP	Cross-Origin Embedder Policy	Does the document require appropriate permissions for embedded resources?
COOP	Cross-Origin Opener Policy	Should browsing contexts be separated when opener policies differ?

no-cors gives a script an opaque response rather than disabling protection. Appropriate combinations can provide isolation required by certain browser capabilities; careless headers can block loads or change window communication.

Reason: displaying a cross-origin image does not prove that script can read its response. Distinguish embedding and response access before changing policy.

Other policies solve different isolation problems: **CORP, Cross-Origin Resource Policy**, restricts selected resource loading; **COEP, Cross-Origin Embedder Policy**, sets requirements for resources embedded by a document; **COOP, Cross-Origin Opener Policy**, separates browsing contexts according to policy.

Poll or receive a server stream

Poll or receive a server stream

Run polling, then streaming

In the lab, open Network:

1. Click Poll three times: request, read time, wait briefly, repeat. Expect three times and three `/proxy-clock` requests.
2. Clear the Network display and click Receive three events. **SSE, Server-Sent Events**, carries text events from server to browser: one `/proxy-events` request contains events1,2,3 before the example closes it.
3. Browser `EventSource` opens the channel; `onmessage` receives events and `close()` stops it. Without explicit closure it may reconnect after interruption. The complete client and server are in the downloadable lab file.

Compare: polling asks repeatedly even without new data; streaming keeps a connection. WebSocket, not implemented in this exercise, permits both peers to send. SSE explanation.

Before the details

Choose an update channel by data direction, frequency, and tolerated delay. Polling, SSE (Server-Sent Events, an HTTP-based server-to-browser text-event stream), WebSocket, and webhooks are not a ranking; each fits a different communication shape.

Polling asks periodically; **long polling** holds a request until an event or deadline and then repeats. **SSE, Server-Sent Events**, streams text events from server to browser. **WebSocket** is a persistent bidirectional message channel. A **webhook** is a server request notifying another server about an event.

Heartbeat messages test connection liveness. **Backpressure** slows production when consumption cannot keep up. **Horizontal scaling** adds service instances to share work.

Communication models

Pattern	Direction	Good fit
Polling	Client asks periodically	Infrequent updates and simplicity
Long polling	A request waits for an event	Broad compatibility and lower latency
SSE	Server → browser	Text notifications and feeds
WebSocket	Persistent two-way	Chat, games, collaboration
Webhook	Server → server	Notify another system

WebSockets add connection state, heartbeat, backpressure, and scaling concerns. Choose the simplest pattern that meets the requirement.

Server-Sent Events

The following fragment sends one event from a PHP (a programming language commonly used for server-side web processing) handler after its opening tag; it is not a complete streaming server. event names the event, data supplies content, and a blank line ends the event. X-Accel-Buffering asks Nginx not to buffer this response. flush attempts to push output, but other buffers and proxies also matter. The browser's **EventSource** interface normally implements reconnection; the server must retain data needed to honor Last-Event-ID.

```
header('Content-Type: text/event-stream');
header('Cache-Control: no-cache');
header('X-Accel-Buffering: no');

echo "event: order.updated\n";
echo 'data: ' . json_encode(['id' => 42, 'status' => 'paid']) . "\n\n";
flush();
```

Browsers reconnect automatically. Event IDs and `Last-Event-ID` can support resume. Long streams can occupy FPM (FastCGI Process Manager, managing PHP request workers) workers, so validate the server architecture and timeouts.

Operations

- Limit connection count, message rate, and size.
- Use heartbeat and detect dead peers.
- Authenticate connections and authorize every channel or event.
- Monitor reconnect rate, queue lag, and delivery failures.

Practical problems

▼ When is SSE simpler than WebSocket?

When a browser only needs a continuing server-to-client stream. SSE is one-way HTTP (Hypertext Transfer Protocol, the rules for web requests and responses) and includes reconnection behavior.

▼ How should a webhook receiver authenticate the sender?

Verify a signature over the raw body with a secret, check the timestamp window, and deduplicate by event ID.

▼ Why must webhook handling be idempotent?

A sender may retry after a timeout even if processing succeeded. Repeated delivery must not repeat the business effect.

Next step

After completing this practice, continue with Receive a webhook and handle repeated delivery.

Receive a webhook and handle repeated delivery

Receive a webhook and handle repeated delivery

A **webhook** is a server-to-server request sent for an agreed event. Its client is a server program, not the browser tab. Send one local event, then repeat it to observe duplicate-effect prevention.

Send one event twice

Start the lab. In PowerShell:

```
Set-Content -LiteralPath event.json -Value '{"id":"order-1"}' -Encoding ascii
curl.exe -i -H "Content-Type: application/json" --data-binary "@event.json" http://127.0.0.1:8766/webhook
```

The first command writes a practice event ID. `--data-binary` sends the file as a POST body. Expect `duplicate:false`, `processedCount:1`. Repeat curl: `duplicate:true`, count still1. Change the ID to order-2 and send: count2.

This local receiver loses memory on restart and does not verify an external sender signature. A deployed service needs source verification and durable deduplication; the following sections explain why after this first complete experiment.

Webhook receiver

The **raw body** is the received bytes before parsing or changing whitespace. **HMAC, Hash-based Message Authentication Code**, computes a verification value using a shared secret and message. A **timestamp** records time; **replay** resends a previously valid message.

The code below is a fragment: timestamp, rawBody, secret, and signature must already be read and validated according to the provider's exact signing contract. SHA-256 is the hash used in this HMAC example; hash_equals avoids comparison-time differences revealing the matching prefix. This fragment alone neither checks message age nor deduplicates events.

1. Read the raw body.
2. Verify an HMAC signature and timestamp.
3. Reject replay outside a time window.
4. persist the event ID to deduplicate.
5. Acknowledge quickly and enqueue heavy work.

```
$expected = hash_hmac('sha256', $timestamp . '.' . $rawBody, $secret);
if (!hash_equals($expected, $signature)) {
    http_response_code(401);
    exit;
}
```

Senders retry failed deliveries, so idempotency is required.

Repeated delivery without repeated effects

A reconnect does not recover messages automatically. A **sequence** orders them, a **cursor** records a read position, and a **snapshot** supplies current state. Choose a recovery mechanism appropriate to the service.

Idempotent processing avoids repeating the intended effect of the same operation. After signature and time checks, store the event identifier and business change atomically, meaning all succeed or all fail together, or use an equivalent reliable design. A separate “already seen?” check followed by a write can race with another request.

Worked check: Payment event pay-42 arrives twice. A unique processing record lets the first application record payment and the second recognize completion without adding credit again. When enqueuing, acknowledge after durable acceptance rather than before storing. Late or reordered events also require checks against the order’s current state and transition rules.

Designing data interfaces

REST, RPC, GraphQL, resources, status codes, errors, pagination, versioning, and contracts.

One product contract before interface styles

An **API, Application Programming Interface**, defines how programs request data or actions. Its **contract** specifies accepted requests and responses. Test this contract on8766 in the lab:

Request	Status	Content
GET `/api/products/1`	200	numeric id, text name, numeric price20, currencyEGP
GET `/api/products/2`	404	error.code NOT_FOUND
GET `/api/products/abc`	400	error.code INVALID_ID
POST `/api/products/1`	405	Allow lists GET and HEAD

GET requests reading; this route rejects POST processing. JSON fields have names and values like the earlier product object. Open the first URL, then run `curl.exe -i -X POST http://127.0.0.1:8766/api/products/1`; `-X` selects the method.

Write expectations before testing. Success includes field types and meaning, not only200. Creating products needs another input, validation, and authorization contract before comparing REST, RPC, and GraphQL.

Before the details

A good API (Application Programming Interface, a defined contract for requesting data or actions from another component) is a contract clients can predict, not merely endpoints that work today. Design resources, statuses, errors, pagination, and compatibility before dependencies multiply.

An **API, Application Programming Interface**, is a contract another program can use. An **endpoint** combines an address and operation. A **contract** specifies inputs, types, results, and failures.

CRUD (Create Read Update Delete, the four common data operations) means Create, Read, Update, Delete. **REST, Representational State Transfer**, organizes around resources and constraints such as client/server separation and stateless requests; a JSON (JavaScript Object Notation, a text format for structured values and lists) endpoint alone does not establish all REST constraints. **RPC, Remote Procedure Call**, requests a named remote operation. **GraphQL** selects fields against a typed **schema**; a **resolver** retrieves a field, and complexity limits bound expensive queries.

Choose a style for the problem

- **REST** models resources with HTTP (Hypertext Transfer Protocol, the rules for web requests and responses) semantics.
- **RPC** exposes explicit operations such as `calculateShipping`.
- **GraphQL** lets clients select fields through a schema, but needs complexity controls and resolver-level authorization.

A label does not guarantee quality. Clear contracts, compatibility, security, and observability matter more.

Resources and HTTP

```
GET    /api/orders/42
POST   /api/orders
PATCH  /api/orders/42
DELETE /api/orders/42
```

Use consistent nouns. Return `201` and `Location` for creation, `204` when no body is needed, and distinguish authentication, permission, absence, conflict, and validation failures instead of returning `200` for everything.

Validation and Problem Details

```
{
  "type": "https://docs.example/errors/validation",
  "title": "Validation failed",
  "status": 422,
  "errors": {"email": ["Invalid format"]},
  "request_id": "req_01J..."
}
```

Keep a stable error shape and never expose stack traces or SQL.

Problem Details standardizes HTTP error descriptions. In the example, type links to the error category, title summarizes it, and status matches the HTTP status. errors and request_id are extension fields for validation failures and request tracing. A stack trace lists execution locations; SQL is a database query language, not a suitable public error explanation.

Pagination and filtering

Pagination divides results into pages; filtering selects records matching conditions. A cursor identifies a position under a stable order, such as date plus unique identifier. An offset counts skipped records. Validate cursor input and authorization; Base64 encoding does not make a cursor trustworthy.

Common statuses include 400 for malformed input, 401 for an authentication requirement, 403 for refusal, 404 for absence or deliberate concealment, 409 for conflict, and 422 for input that cannot be processed under validation rules. Define the precise contract.

```
GET /api/orders?status=paid&limit=20&cursor=eyJpZCI6OTAwfQ
```

Set a maximum page size. Cursor pagination usually behaves better for large changing datasets; offset pagination is simpler for limited navigation.

Compatibility and versions

- Prefer backward-compatible additions.
- Never silently change an existing field's meaning.
- Announce deprecation and removal dates.
- Contract-test consumers.
- Path or header versioning can work; consistency matters most.

Contract and operations

Publish a testable OpenAPI/schema contract. Include authentication, rate limits, idempotency, request IDs, timeouts, and observability. Design for partial failure and retries, not only the happy path.

References

- RFC 9110: HTTP Semantics
- RFC 9457: Problem Details for HTTP APIs

Practical problems

▼ When should an API return `404` versus `403`?

404 means no resource; 403 means it exists but access is denied. A security policy may deliberately use 404 to hide a sensitive resource.

▼ Why can cursor pagination be better than offset?

It handles changing data and large skips better, but requires stable ordering and a tamper-resistant cursor.

▼ Does every change require a new API version?

No. Compatible additions often do not; removing a field or changing its meaning requires an explicit version and migration plan.

Test retries and compatibility

Backward compatible means old clients continue working under their contract. **Deprecation** announces a transition before removal. **OpenAPI** describes API inputs and outputs in a machine-readable form.

An **Idempotency-Key** identifies a retryable operation. Bind it to the actor, operation, validated input, and saved result, and prevent concurrent duplicate execution. Retention time is part of the contract. **Rate limits** cap requests per interval; **quotas** cap total consumption. A 429 can include **Retry-After**, when to retry. **Observability** uses evidence such as logs and measurements to explain operation.

Worked check: Reusing the same key with a different body should produce a documented conflict, for example 409, rather than silently executing a new action. Also test that old clients tolerate an optional added field and do not rely on JSON field order.

How HTTP/2 and HTTP/3 differ

Binary framing, multiplexed streams, and QUIC's transport over UDP.

The same three requests under loss

A **stream** is a logical message flow within a shared connection. A **protocol frame** carries a type, stream identifier, and data according to its protocol; it need not mean an Ethernet frame.

Assume resources A,B,C and loss affecting A. This compares ordering, not measured speed:

Version and assumption	Before loss	Effect
HTTP/1.1, one connection, sequential requests	A thenB thenC	Waiting for A delays starting the others in this model; browsers can open more connections
HTTP/2 over one TCP connection	A,B,C pieces interleave	Missing TCP data delays later byte delivery for all its streams until recovery
HTTP/3 over QUIC streams	A,B,C pieces interleave	Recovering A does not impose its byte ordering on B/C; congestion and shared resources still matter

Check: HTTP/3 does not remove every delay. Separate cross-stream transport ordering from capacity and application work.

Why did HTTP need more versions?

The meaning of `GET`, `404`, and headers remained. Message transport changed. HTTP/1.1 (Hypertext Transfer Protocol, the rules for web requests and responses) browsers often used several connections; HTTP/2 multiplexes framed streams over one TCP (Transmission Control Protocol, ordered byte transport with loss recovery; connections can still fail) connection.

```
One TCP connection
  ├─ Stream 1: HTML
  ├─ Stream 3: CSS
  └─ Stream 5: image
```

TCP still exposes one ordered byte stream. Packet loss can delay all HTTP/2 streams until missing bytes are recovered—transport head-of-line blocking.

QUIC (a UDP-based transport protocol adding reliable streams and protected communication) runs over UDP (User Datagram Protocol, independent messages without built-in delivery or ordering guarantees) but implements reliability, encryption, retransmission, stream ordering, and congestion control itself. HTTP/3 uses QUIC, so UDP does not imply

unreliable application data.

Client and server negotiate and can fall back. A newer protocol alone does not guarantee speed; distance, loss, configuration, resource design, and caching matter.

HTTP semantics and transport

Methods, status codes, and header meanings remain broadly consistent across HTTP versions, while framing and transport change. HTTP/1.1 uses its familiar textual message format, HTTP/2 uses binary frames over TCP, and HTTP/3 runs over QUIC.

HTTP/2 multiplexing

HTTP/2 splits one connection into streams, allowing several requests and responses to progress concurrently. HPACK compresses headers. TCP still exposes every stream to transport head-of-line blocking when a missing packet delays the ordered byte stream.

QUIC and HTTP/3

QUIC implements reliable streams, retransmission, and congestion control over UDP and integrates TLS (Transport Layer Security, rules for establishing an authenticated protected connection) 1.3. Loss in one stream does not block delivery in unrelated streams. Connection IDs also help a connection survive a change between Wi-Fi and mobile data.

Using UDP does not make HTTP/3 unreliable; QUIC supplies the reliability above UDP. Clients negotiate protocol support and may fall back to HTTP/2 or HTTP/1.1. Measure real behavior because distance, loss, server configuration, and resource sizes affect performance.

Practical exercises

▼ Did HTTP/2 change semantics or transport?

Core semantics remained while transport gained binary frames, multiplexing, and header compression.

▼ Why is HTTP/3 reliable over UDP?

QUIC supplies acknowledgements, retransmission, ordering, congestion control, and encryption above UDP.

▼ How do you verify the negotiated version?

Inspect the browser Network Protocol column or another measurement tool instead of assuming from server support.

Check your understanding

Multiplexing allows multiple logical streams to share a connection. QUIC reduces cross-stream blocking by maintaining ordering independently per stream.

Header compression and early data

Multiplexing interleaves several logical streams on one connection. A **stream** is an ordered exchange, and a **frame** is a protocol-defined unit within that exchange; HTTP frames are distinct from link-layer frames. **Head-of-line blocking** makes otherwise-ready work wait for an earlier missing item.

HPACK compresses HTTP/2 header fields; **QPACK** compresses HTTP/3 fields. QPACK reduces cross-stream dependency problems but does not eliminate all waiting: a stream can depend on a compression-table update.

RTT, Round-Trip Time, measures a round trip. **0-RTT** sends early application data when resuming an eligible connection. Such data can be replayed; state-changing operations need explicit protections. **Connection IDs** can support **migration**, changing the network path while retaining a connection, subject to endpoint support, validation, and network policy.

Worked check: Data for an image stream is lost in HTTP/3. Another independent stream whose required data arrived can progress, but shared congestion control or header dependencies may still affect it. HTTP/2 over TCP can withhold later bytes for all streams on that connection until the gap is filled. Compare under matching conditions rather than assuming the newer protocol wins.

Deliver content and understand the origin server

Deliver content and understand the origin server

CDN, origin, and points of presence

A **CDN (Content Delivery Network, distributed servers delivering content closer to users)** runs points of presence near users. An edge returns a cached representation when its key is valid; otherwise it fetches the **origin** and may store the result. This reduces latency and origin load.

The key usually includes host and path plus selected query or `Vary` dimensions. Including every cookie destroys cache efficiency, while ignoring identity dimensions for personalized content can expose one user's response to another.

Use `public` only for shareable content, `private` for browser-only caches, `s-maxage` for shared caches, and content-hashed asset names for long lifetimes. Plan invalidation and protect the origin from direct bypass.

Next step

After completing this practice, continue with Protecting requests and limiting their rate.

After completing this practice, continue with Trace requests and interpret measurements.

Protecting requests and limiting their rate

Protecting requests and limiting their rate

A server can receive harmful requests or more requests than it can handle. Distinguish inspecting requests from controlling their rate.

Three counters with small numbers

A **rate limit** decides how many requests to accept over time. For two per ten seconds:

Method	Rule	Effect
Fixed window	Reset a counter every ten seconds	Two at one window's end plus two at the next start can cluster
Sliding window	Consider the preceding ten seconds	Avoids that boundary jump; implementation methods vary
Token bucket	Capacity2, replenish one token every five seconds up to capacity	Two immediate requests when full; each consumes a token

In the lab, request `/limited` on8766 three times quickly:200,200,429. `Retry-After` suggests a wait. This is one shared teaching window, not a fair per-user policy. Content inspection is a different concern.

A WAF does not replace application security

A **WAF (Web Application Firewall, a system applying security rules to web requests)** applies managed or custom rules to request patterns. It can block, challenge, or log known attacks, but it does not know every domain rule. A well-formed invoice update may still be unauthorized. Applications must validate input, authenticate, authorize resources, and use safe database APIs.

Test broad rules in monitor mode when possible. False positives can block real customers. Give each exception a reason, owner, and expiry.

Rate limiting and bursts

Limits may use IP (Internet Protocol, the addressing and routing protocol; an IP address identifies a network interface in context), user, API (Application Programming Interface, a defined contract for requesting data or actions from another component) key, tenant, or route. Fixed window, sliding window, and token bucket have different burst behavior. Return `429 Too Many Requests` and a truthful `Retry-After` when available.

Rate limits control arrival rate, concurrency limits protect worker capacity, and quotas cap total use over a period. Under overload, apply backpressure or load shedding rather than accepting work that cannot complete.

Trace requests and interpret measurements

Trace requests and interpret measurements

When a request is slow, collect evidence of where the delay happened. Read a log, duration, and request ID before connecting measurements across services.

Read evidence from your own environment

In the lab, request the product, `/missing`, and `/failure`. Each response has `X-Request-Id`, matched by a terminal entry with200/404/500. A **log** records an event; the ID connects response and log. `/proxy` preserves the ID at both servers.

A **span** is a timed part of work, such as a database query. Related spans form a **trace**. A request ID alone is not a complete trace: this lab proves correlation across two requests, not nonexistent database timing.

A **percentile** is a value at or below which a portion of measurements falls. For twenty sorted durations10,20,...,200ms, nearest-rank p50 uses rank10=100ms; p95 uses19=190ms; p99 uses20=200ms. Tools may interpolate differently, and a small sample poorly estimates a large population's tail. State method and sample size; p95 is not a mean.

Deliver: three status/ID/log pairs, one proxy request, and explanations of200/404/500. Then use the public-site project below as an extension, accepting that public sites may hide fields.

Logs, metrics, and traces

- A **log** records a detailed structured event.
- A **metric** aggregates rates, errors, latency percentiles, and saturation.
- A **trace** connects spans across edge, gateway, app, database, and dependencies.

Propagate a trace/correlation ID without logging passwords, tokens, or unnecessary personal data. Watch `p50`, `p95`, and `p99`; an average hides a slow minority.

```
Client: DNS → Connect → TLS → TTFB → Download
Server: Edge → Gateway → App → DB → External API
```

High TTFB (Time To First Byte, time until the first response byte according to the measurement tool) with normal application time points toward queueing, edge, or network work. A slow database span directs investigation toward queries, indexes, and connection capacity.

Check your understanding

01 **How can a bad cache key leak data?**

> **Answer:** If it ignores identity or another response-varying dimension, the CDN can serve one user's representation to another.

02 **Why can a WAF not secure an invoice-update endpoint alone?**

> **Answer:** It can detect patterns but does not own invoice authorization and business rules.

03 **Differentiate rate limit, concurrency limit, and quota.**

> **Answer:** They cap arrival rate, simultaneous work, and total use over a period respectively.

04 **Why use logs, metrics, and traces together?**

> **Answer:** A metric reveals a trend, a trace locates the failing stage, and a log provides event details needed to explain it.

Cumulative project: trace one request from name to origin

Choose a non-sensitive public URL (Uniform Resource Locator, an address identifying a resource and how to access it) and build a journey card for one request. Do not stop at a DevTools screenshot; connect every observation to a layer and reproducible evidence.

1. Name and address

```
nslookup example.com
```

Record the resolver, returned addresses, and TTL (Time To Live, a DNS cache lifetime or an IPv4 forwarding limit depending on context) when your tool exposes it. An address change between runs can be CDN or DNS (Domain Name System, a distributed system answering queries about domain names) load balancing rather than a fault.

2. Connection and HTTP

```
curl.exe -sS -o NUL -D headers.txt -w "status=%{http_code} remote=%{remote_ip} connect=%{time_connect} ttfb=%
{time_starttransfer} total=%{time_total}\n" https://example.com/
```

This is an output shape, not a value to copy:

```
status=200 remote=203.0.113.10 connect=0.042 ttfb=0.118 total=0.121
```

On macOS or Linux replace `NUL` with `/dev/null`. Inspect `headers.txt` for `cache-control`, `age`, `via`, and a request ID; a missing header does not prove that no CDN exists.

3. Repeat and compare

Run the request five times and save status, TTFB, and Age. Add a safe query parameter and observe whether it changes the cache key. Never infer a cache hit from speed alone; require header or provider evidence.

4. Failure path

Request a missing path and verify that the 404 still carries a request ID and is not cached for an unintended duration. In a staging environment you control, trigger a monitored 500 and verify trace continuity from edge to application while logs exclude Cookie and Authorization values.

Deliverable

Provide a five-attempt table, a Network waterfall capture, an explanation of the TTFB difference, and one falsifiable hypothesis about where delay occurs. Label measured facts separately from inferences.

Interpret the operational terms

The **origin** is the source server; **edge/PoP (Point of Presence)** is a distributed serving location. A **cache key** groups requests sharing a representation. A content **hash** fingerprints data; it is not encryption. **Purge/invalidation** removes or expires a stored copy and may propagate gradually.

A **fixed window** counts requests in set intervals; a **sliding window** tracks a moving interval. A **token bucket** replenishes permission tokens over time and permits bursts up to its capacity. A **tenant** is one customer organization sharing a service. **Backpressure** slows incoming production; **load shedding** rejects work the system cannot complete. IP-only limits can penalize unrelated users sharing NAT (Network Address Translation, rewriting IP addresses at a network boundary).

A **span** is one timed operation within a trace. **p95** means 95 percent of measured values are at or below that value; p50 is the median and p99 describes a slower tail. A **false positive** is a legitimate request flagged as an attack. **Rollback** restores a known configuration. **Trace context** carries the identifiers needed to connect related operations across services and queued work.

Worked check: A second request is faster without a cache indicator. Do not declare a cache hit; connection reuse or load changes can explain it. In your own service, combine Age, cache-status fields, and logs. Comparing purge completion across locations requires actual separate measurement locations; two commands on one device do not establish global propagation. Public sites need not expose request IDs on their 404 or 500 responses.

Math and problem solving

Start with the arithmetic and logic a problem needs, then describe and trace a solution. Compare algorithms and explore applied mathematics after that.

Start with the arithmetic and logic a problem needs, then describe and trace a solution. Compare algorithms and explore applied mathematics after that.

Compare and explore

The later sections on growth rates, graph traversal, and dynamic programming build on lists, loops, and functions. Work through their traces after programming practice; they are not prerequisites for writing your first program.

Follow the sequence and practice each step

- Ratios, averages, and powers
- Division, remainders, and numerical precision
- Variables, equations, and Boolean logic
- Sets, relations, and function inputs
- Computational thinking and requirements analysis
- Algorithms, pseudocode, and decision trees
- Flowcharts, loops, and debugging
- Comparing algorithms and data structures
- Why does a loop work and stop?
- Turn solution patterns into steps
- Choose between greedy steps and saved results
- Measurement, units, and prime numbers
- Sequences, counting, and probability
- Draw relationships and search for a path

Ratios, averages, and powers

Ratios, averages, and powers

Use this section after values, decisions, and loops. Review arithmetic when needed; graphs, complexity, and dynamic programming are later extensions, not first-program prerequisites.

One calculation with meaning

Three notebooks at 20 cost `3×20=60`. A 10% discount is `60×10÷100=6`, leaving 54. **Try:** price 30 and quantity 2 still total 60. The power `2³` is `2×2×2=8`; `√25=5` because `5×5=25`. The percent sign in prose differs from a language's remainder operator.

Overview

Core arithmetic appears directly in programs: remainder, percentages, rates, averages, powers, roots, and precedence. Connect every formula to the problem meaning before calculating.

Core concepts

- Remainder supports parity checks, cycles, and time conversion.
- A percentage is a part per hundred; discount = price × rate ÷ 100.
- A rate compares quantities with different units, such as km/hour.
- Arithmetic mean is sum divided by count and is sensitive to outliers.
- Use parentheses, powers, multiplication/division, then addition/subtraction; multiplication/division and addition/subtraction associate left to right. Powers and programming operators may associate differently; use parentheses to state intent.

Worked example

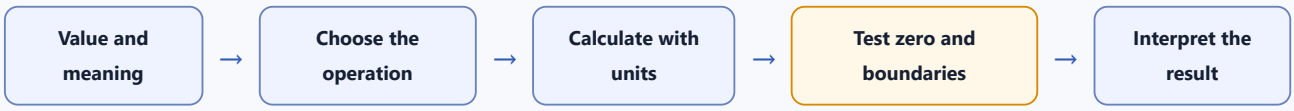
For a price of 800 with a 15% discount, the discount is 120 and the final price is 680. Subtracting 15 directly confuses a percentage with an amount.

Corrections and common mistakes

- Avoid integer division when the result may contain a fraction.
- Carry units through calculations to catch invalid formulas.

Lesson map

Concept map: Math foundations: remainder, ratios, averages, and powers



Check your understanding

01 **Why can 9/5 break a temperature conversion in some languages?**

> **Explained answer:** With two integer operands, integer division produces 1. Use 9.0/5.0 or convert an operand to a floating type.

02 **An 800 item gets 15% off, then 15% tax. Does it return to 800?**

> **Explained answer:** No. Discount gives 680; tax is then 102, producing 782. The two percentages use different bases.

03 **When can the arithmetic mean be misleading?**

> **Explained answer:** Outliers or skewed data can dominate it; the median and the distribution may communicate the situation better.

04 **Convert 10,000 seconds into hours, minutes, and seconds using division and remainder.**

> **Explained answer:** Hours are 2 with 2800 left; minutes are 46 with 40 left, so the result is 2:46:40.

Next step

After completing this practice, continue with Division, remainders, and numerical precision.

Division, remainders, and numerical precision

Division, remainders, and numerical precision

You have 10 items and boxes holding 3. Three boxes are insufficient: all items need four. If you ask only for full boxes, the answer is three with one remaining. The question determines the division operation.

Calculate before naming

`floor(10/3)=3` chooses the greatest integer no larger than the value; `ceil(10/3)=4` chooses the least integer no smaller. For −2.3, floor is −3 and ceiling −2: locate them on a number line.

Grouping 12 and 18 items equally without leftovers allows a largest group size of 6: **GCD, Greatest Common Divisor**. Bells ringing every 4 and 6 minutes first coincide again at 12: **LCM, Least Common Multiple**, for these positive integers.

Try: 17 items in size-5 boxes: full boxes, remainder, and boxes for everything are 3, 2, and 4. Explain each before studying the rules.

Integer division, number theory, and precision

Integer division returns an integer quotient; negative-value rounding depends on the language, as the worked examples below show. For positive integers, `(count + size - 1) / size` computes ceiling division. Euclid's algorithm repeatedly replaces `(a, b)` with `(b, a % b)` to find the GCD (Greatest Common Divisor, the largest common divisor); the LCM (Least Common Multiple, the least common multiple under the stated zero convention) can then be derived carefully without overflowing intermediate values.

Modular arithmetic models cycles such as clocks and parity. Floating-point results are approximate, so comparisons may require a tolerance chosen for the problem's scale.

Calculate instead of memorizing names

A **power** is repeated multiplication: $2^3=8$. The nonnegative **square root** of 25 is 5. For 2,4,9, the mean is 15/3=5 and the median is 4. An empty-list mean needs a policy instead of division by zero.

17 divided by 5 has remainder 2 because 17=3×5+2. Many languages use %. Integer division of negatives differs: C++ -7/3 truncates toward zero to -2; Python -7//3 floors to -3. JavaScript 7/3 is ordinary division. Specify the language.

Ten items in groups of three require four groups. The ceiling-division formula (count+size−1)/size requires integer division, nonnegative count, positive size, and no intermediate overflow. Zero items need zero groups; zero group size is invalid.

For **GCD**, 48%18=12, 18%12=6, 12%6=0, so the greatest common divisor is 6. **LCM** is 48/6×18=144. Under the common programming convention, return zero if either argument is zero, handling (0,0) before division. abs means absolute value.

Worked check: Five hours after hour 23 is (23+5)%24=4. This is modular arithmetic. **Tolerance** is a domain-appropriate permitted numerical difference; **overflow** exceeds a type's range and is not fixed by an approximation tolerance.

Variables, equations, and Boolean logic

A variable names a value that may change, while an equation describes a relationship. Boolean logic turns problem rules into decisions a program can evaluate.

Use this section after values, decisions, and loops. Review arithmetic when needed; graphs, complexity, and dynamic programming are later extensions, not first-program prerequisites.

Test one logical statement

Entry requires a ticket **and not being blocked**. Boolean values are `true` and `false`. `AND` requires both conditions; `NOT` reverses an answer.

Ticket?	Blocked?	Allowed?
No	No	No
No	Yes	No
Yes	No	Yes
Yes	Yes	No

This **truth table** lists every input combination and its outcome. Explain the last row before learning symbols. Sets and relations follow in their own lesson.

Overview

A variable names a value that may change, while an equation describes a relationship. Boolean logic turns problem rules into decisions a program can evaluate.

Core concepts

- An expression produces a value; an equation or comparison describes a relation.
- Assignment changes state, so statement order can change the result.
- AND requires every condition, OR requires at least one, and NOT negates.
- XOR is true when its operands differ, but is less common in business rules.
- Name simple conditions separately before combining them.

Worked example

Loan eligibility might require an acceptable age AND sufficient income AND no risky debt. Represent each fact as a clearly named Boolean, then combine them.

Corrections and common mistakes

- Do not confuse equality with assignment.
- Use parentheses when mixing AND and OR to make intent explicit.

Lesson map

Concept map: Variables, equations, and Boolean logic

Check your understanding

01 **How does x = x + 1 differ in programming and algebra?**

Explained answer: In code it is assignment using the old value. As an algebraic equality over ordinary numbers, x=x+1 has no solution.

02 **Translate: accepted if employed or sufficiently funded, provided the applicant is not blocked.**

Explained answer: eligible = (isEmployee OR hasEnoughIncome) AND NOT isBlocked. Parentheses ensure blocking rejects either route.

03 **When do separate if statements differ from an else-if chain?**

Explained answer: Independent if statements can execute several branches; an else-if chain selects only the first true branch.

04 **How do you systematically test a compound Boolean rule?**

> **Explained answer:** Build a truth or decision table covering each predicate, important combinations, and comparison boundaries with expected outcomes defined first.

Next step

After completing this practice, continue with Sets, relations, and function inputs.

Sets, relations, and function inputs

Sets, relations, and function inputs

Start with squaring, allowing only inputs 1 and 2. Outputs are 1 and 4. The **domain** is allowed inputs {1,2}. If outputs are declared to be nonnegative integers, that is the **codomain**. The **range** actually produced is {1,4}. Allowed versus achieved results differ.

Try: Add input 3: the range gains 9, but still does not contain every nonnegative integer. Next use sets to understand relations and “every” versus “exists.”

Sets, functions, truth tables, and predicates

A set contains unique elements. Union combines membership, intersection keeps shared elements, and difference removes one set from another. A function maps every element in its domain to one result; its range contains the results actually produced.

Truth tables enumerate Boolean inputs. De Morgan’s laws transform `!(A && B)` into `!A || !B` and `!(A || B)` into `!A && !B`. A predicate is a Boolean statement about a value. Universal and existential conditions correspond to operations such as “all elements” and “at least one element.”

Truth tables and concrete relations

A **Boolean** is true or false. Many languages use &&, ||, and ! for AND, OR, and NOT; actual syntax is language-specific.

A	B	A AND B	A OR B	A XOR B	A implies B
false	false	false	false	false	true
false	true	false	true	true	true
true	false	false	true	true	false
true	true	true	true	false	true

Implication fails only for a true premise and false conclusion; it does not prove causation. **De Morgan’s laws** state NOT(A AND B)=(NOT A OR NOT B), and NOT(A OR B)=(NOT A AND NOT B). Verify all four rows.

For A={1,2}, B={2,3}, union is {1,2,3}, intersection {2}, and difference A−B={1}. {1} is a subset of A. A relation is **reflexive** if each element relates to itself, **symmetric** if aRb implies bRa, and **transitive** if aRb and bRc imply aRc. Equality satisfies all three.

For $f(x)=x\times x$ on $\{1,2\}$, the **domain** is $\{1,2\}$. If the **codomain** is nonnegative integers, the actual **range** is $\{1,4\}$. Each input has exactly one output.

A **predicate** is a value-dependent condition, such as x is even. A **quantifier** specifies every element or at least one. In $\{2,3\}$, an even element exists, but not every element is even: changing the quantifier changes the claim.

Computational thinking and requirements analysis

Computational thinking is a disciplined way to turn an unclear problem into components, rules, and steps that can be implemented and tested.

Use this section after values, decisions, and loops. Review arithmetic when needed; graphs, complexity, and dynamic programming are later extensions, not first-program prerequisites.

Turn one requirement into tests

ATM rule: “Allow a positive multiple of 50 no greater than the balance.” Assume a balance of 200 and no fees. **Constraints** are input limits.

Input	Expected	Rule exercised
150	Accept; 50 left	Normal case
200	Accept; 0 left	Exact upper boundary
250	Reject	Exceeds balance
0 or −50	Reject	Must be positive
75	Reject	Not a multiple of 50
Nonnumeric text	Reject before arithmetic	Input type

Separate reading, validation, then deduction/display. Rejection preserves balance. **Try:** adding a fixed fee of 10 makes withdrawal 200 invalid because total cost is 210. Update tests when the requirement changes.

Overview

Computational thinking is a disciplined way to turn an unclear problem into components, rules, and steps that can be implemented and tested.

Core concepts

- Decomposition splits a problem into manageable tasks.
- Pattern recognition reuses known structures instead of restarting.

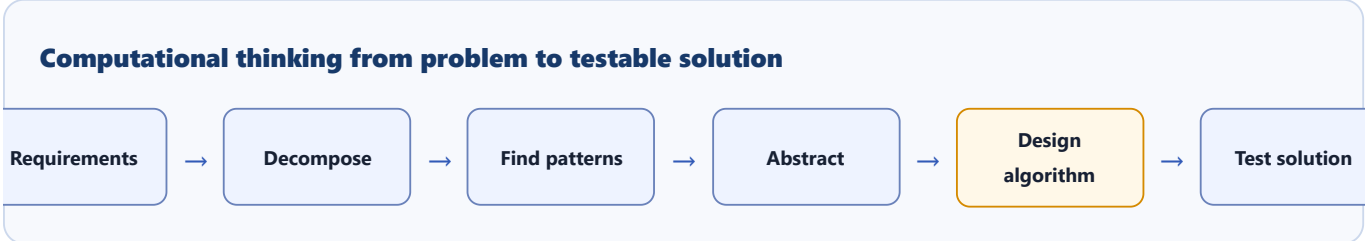
- Abstraction keeps relevant details and postpones irrelevant ones.
- Algorithm design orders actions, decisions, and repetition.
- Requirements analysis defines inputs, rules, edge cases, outputs, and success criteria.

Worked example

For an ATM (Automated Teller Machine): separate card validation, PIN (Personal Identification Number, a secret numeric authentication code) checking, operation selection, balance and daily-limit checks, withdrawal, balance update, and receipt generation.

Corrections and common mistakes

- Do not code before identifying rejection paths and boundaries.
- A vague requirement such as “fast” needs a measurable criterion.



Make a small requirement testable

For our shopping list, “accept a quantity” is vague. Specify a whole number from 1 through 100. Split the work into reading text, rejecting an empty field, converting to a number, checking the bounds, and only then adding. Test 0, 1, 100, 101, and 2.5: only 1 and 100 pass. Separating these responsibilities makes the first failing step visible.

For comparisons of running cost and reusable solution patterns, continue later to algorithm comparison. Here the outcome is a clear problem statement and independently predicted tests.

Check your understanding

01 **How do decomposition and abstraction differ?**

> **Explained answer:** Decomposition splits the system into smaller problems; abstraction keeps relevant details and hides those not needed by the current solution.

02 **Name a vague booking-system requirement and make it testable.**

> **Explained answer:** “The system is fast” is vague. A testable form is: 95% of searches finish under 500 ms at a defined load.

03 **How does pattern recognition prevent duplicated solutions?**

> **Explained answer:** It exposes a shared structure such as validate-decide-record, allowing one reusable rule or function instead of copied logic.

04 **Decompose an ATM withdrawal and name a critical boundary case.**

> **Explained answer:** Validate card/PIN, select amount, check balance and limit, dispense, update, and receipt. Test an amount exactly equal to balance or daily limit.

Algorithms, pseudocode, and decision trees

Pseudocode and decision trees model thinking before code: pseudocode emphasizes sequence, while a tree emphasizes branching decision paths.

Use this section after values, decisions, and loops. Review arithmetic when needed; graphs, complexity, and dynamic programming are later extensions, not first-program prerequisites.

Overview

Pseudocode and decision trees model thinking before code: pseudocode emphasizes sequence, while a tree emphasizes branching decision paths.

Core concepts

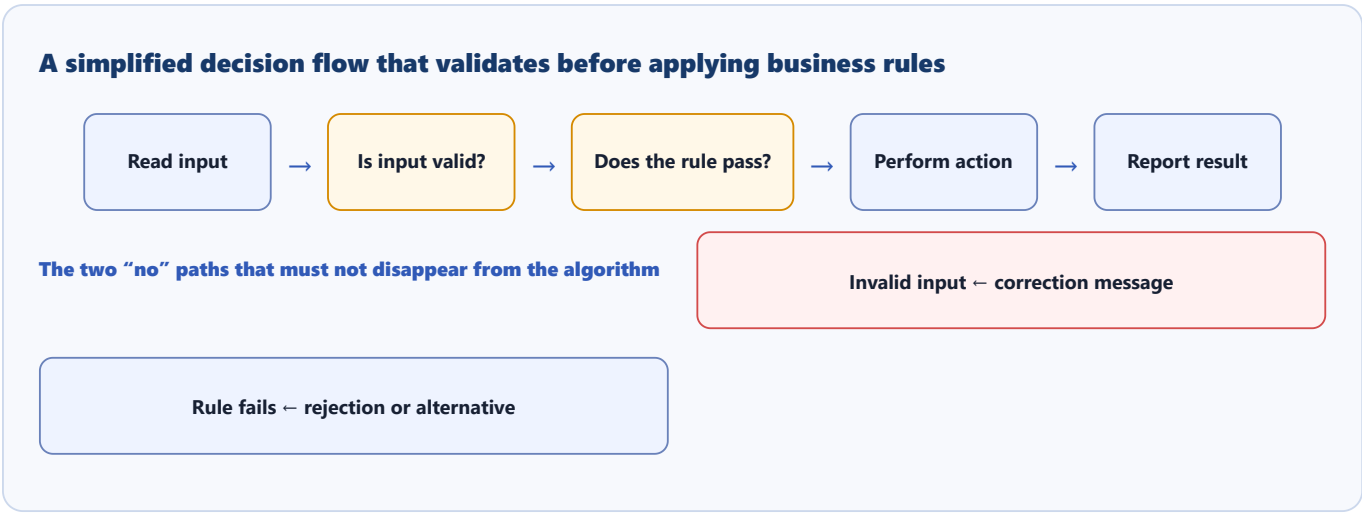
- A sound algorithm has defined inputs, unambiguous steps, termination, and correct outputs.
- Use domain names instead of x and y in business problems.
- Pseudocode is language-independent but must remain precise.
- Decision trees work well when answers lead to several terminal outcomes.
- Validate invalid input first, then apply business rules from general to specific.

Worked example

To find the largest of three values, initialize the result with the first, compare and update with the second, then repeat for the third. This avoids enumerating every ordering.

Corrections and common mistakes

- Large trees become difficult to maintain; group related rules and name conditions.
- Every possible path must end with a defined outcome.



Write and trace the algorithm

Pseudocode describes precise steps without requiring a runnable programming language. This example accepts three valid numbers and returns the greatest:

```
read a, b, c
largest = a
if b > largest:
    largest = b
if c > largest:
    largest = c
print largest
```

Here read obtains input, print displays output, if tests a condition, and = assigns a value. For a=−7, b=−2, c=−5, largest starts at−7, changes to−2, then remains there because−5 is smaller. Starting at zero would incorrectly return a value absent from the inputs.

A **precondition** states what must hold before starting: valid numbers. A **postcondition** states the required result: one input value, with no input greater than it. For a list, the **loop invariant** is that largest is the greatest value examined so far. **Termination** follows because each iteration examines another element of a finite list. Empty input needs an explicit result, such as “no value”, instead of reading a nonexistent first element.

The same maximum example as a tree

Assume valid numbers a, b, and c. Each yes/no answer chooses a branch; every leaf returns a value:

```
a >= b?
├─ Yes: a >= c?
│        ├─ Yes → a
│        └─ No  → c
└─ No:  b >= c?
         ├─ Yes → b
         └─ No  → c
```

`>=` means greater than or equal. For −7, −2, −5 take No then Yes: b is −2. Equal values produce the same maximum whichever equal value is selected. Compare with updating `largest`: repeated updates extend to a list more easily.

Check your understanding

01 **Why is “initialize with the first value, then update the maximum” better than enumerating three-value orderings?**

> **Explained answer:** It needs only two comparisons and scales to any number of values; enumerating permutations grows quickly and is error-prone.

02 **When does a decision tree become a poor representation?**

> **Explained answer:** When branches and shared rules explode and repeat. A decision table, named predicates, or a state machine may then be clearer.

03 **Which properties should a sound algorithm establish?**

> **Explained answer:** Defined inputs and outputs, unambiguous steps, finite termination, and correctness for every case inside the requirements.

04 **Order checks for a discount based on valid price, membership, and total.**

> **Explained answer:** Validate price first, then determine eligibility, then apply membership and threshold rules. Invalid data must never reach discount calculation.

Greedy versus dynamic programming: choose with evidence

The worked coin example follows the algorithm comparison lesson, after data structures and growth rates have been introduced. First finish describing and tracing an ordinary algorithm here.

When a decision depends on earlier state

Each **leaf** of a decision tree ends a path. For a discount requiring membership and a price of at least100, test members at99 and100, a nonmember at100, and a rejected negative price. A **state machine** models states and allowed transitions, such as new → paid → shipped; this contract disallows new → shipped.

Flowcharts, loops, and debugging

A flowchart visualizes execution, loops represent repetition, and debugging compares actual behavior with intended behavior.

Use this section after values, decisions, and loops. Review arithmetic when needed; graphs, complexity, and dynamic programming are later extensions, not first-program prerequisites.

Overview

A flowchart visualizes execution, loops represent repetition, and debugging compares actual behavior with intended behavior.

Core concepts

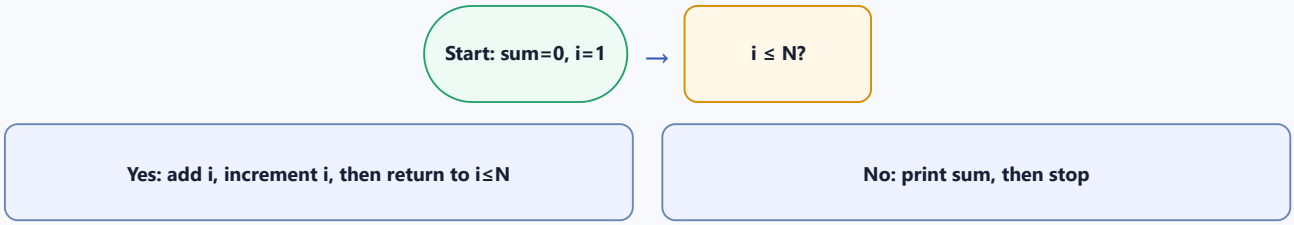
- Start/end uses an oval, processing a rectangle, decisions a diamond, and input/output a parallelogram.
- Every decision should label its outgoing branches, commonly yes and no.
- A loop needs initialization, a continuation condition, and an update; missing the update often causes an infinite loop.
- A counter counts items, an accumulator combines values, and a sentinel ends input.
- A trace table records variable values each iteration and exposes logic errors.

Worked example

To sum `1..N`, initialize `sum = 0` and `i = 1`; while `i <= N`, add `i` to `sum` and increment `i`; print `sum` after the loop.

Corrections and common mistakes

- A flowchart communicates logic but does not replace testing.
- A program can run without errors and still contain a logic defect.



Trace the loop one iteration at a time

A **counter** counts events, such as attempts. An **accumulator** combines values, such as sum for prices. A **sentinel** is an agreed stopping marker, such as “stop” to end input; it must not conflict with valid data.

To sum1 throughN, require a nonnegative integer N:

Before testing	i	sum	Is i≤3?	After the iteration
Initial	1	0	yes	sum=1, i=2
Second	2	1	yes	sum=3, i=3
Third	3	3	yes	sum=6, i=4
Fourth	4	6	no	exit and print6

The update returns to the condition; exit is the condition’s “no” branch. N=0 runs zero iterations and returns0; N=1 returns1. An **off-by-one** error adds or omits one iteration. Using i<N instead of i≤N returns3 for N=3 because it omits the final3.

Worked check: omitting i=i+1 leaves the condition true for N≥1. Watch i in a trace table or a debugger that pauses execution one step at a time; after fixing it, retest0,1,3.

Check your understanding

01 **Which three loop elements must be checked to prevent an infinite loop?**

> **Explained answer:** Initialization, continuation condition, and an update on every path. The update must move state toward making the condition false.

02 **How do a counter, accumulator, and sentinel differ?**

> **Explained answer:** A counter counts events, an accumulator combines values, and a sentinel is a special value that terminates input and is usually excluded from calculation.

03 **A 1..N sum is too large by N. What should you inspect first?**

> **Explained answer:** Check initialization and bounds: sum may start at N or count N twice. Including N+1 adds N+1 rather than N: for N=3 the correct sum is 6, and including 4 gives 10. A trace table reveals the first wrong iteration.

04 **Design a three-attempt login without an off-by-one bug.**

> **Explained answer:** Start attempts at zero, increment after each failure, and continue while not successful and attempts<3. Test first/third success and three failures.

Comparing algorithms and data structures

Turn a problem into steps, write pseudocode, choose data structures, and reason about complexity.

Start after practicing variables, loops, and lists in your chosen language.

Before the details

Programming begins before code: define inputs, outputs, and constraints; write testable steps; then choose data structures and complexity that fit the expected size.

From a problem to a program

Before coding, define inputs, outputs, rules, small testable steps, and boundary or invalid cases.

```
INPUT items, discountPercent
IF discountPercent < 0 OR discountPercent > 100
    RETURN error
total = 0
FOR EACH item IN items
    IF item.price < 0 OR item.quantity < 1
        RETURN error
    total = total + item.price * item.quantity
RETURN total * (1 - discountPercent / 100)
```

Pseudocode communicates logic without binding it to PHP (a programming language commonly used for server-side web processing), JavaScript, or another language.

Choose a data structure

Need	Useful structure
Ordered values with duplicates	List/array
Lookup by key	Map/associative array
Unique membership	Set
First in, first out	Queue
Last in, first out	Stack

A good structure improves clarity and cost. A map can replace a repeated nested search with direct lookup.

Big O

Big O describes how time or memory grows with input size, not exact seconds.

Complexity	Example
`O(1)`	Read a known position in a random-access array
`O(log n)`	Binary search in sorted data
`O(n)`	Scan a list once
`O(n log n)`	Common efficient sorts
`O(n²)`	Compare every item with every item

Correctness and clarity come first. Measure before optimizing because constants and real workloads still matter.

Exercise

Given orders containing `customer_id` and `total`, design an algorithm that returns a total per customer. Define edge cases, use a map for `O(n)` traversal, and write three test cases before the implementation.

Practical problems

▼ When is a list better than a set?

When order, duplicates, or sequential traversal matter and fast membership tests do not justify a set.

▼ Why is success on three items insufficient?

An algorithm may become impractical at one million items. Verify correctness, then estimate time and memory growth.

▼ What is pseudocode for?

It separates solution logic from language syntax, exposing missing cases before implementation details.

Solve the customer exercise and explain the structures

A **list/array** stores elements accessible by position, such as [8,3,8]. A **map** associates a key with a value, such as a customer ID and total. A **set** keeps distinct values. A **queue** removes the oldest item first, like a waiting line; a **stack** removes the newest first, like stacked plates. Ordering and performance depend on the language and implementation.

A **lookup** retrieves a value. Accessing a known array position is typically O(1). A **hash map** uses a function to locate a key's search area; lookup is expected O(1) under suitable distribution and capacity management, but a conventional implementation may take O(n) in the worst case when many keys collide. A tree-based ordered map may provide O(log n). Not every map operation is constant time.

Binary search discards half a sorted range at each step. To find7 in [1,3,5,7,9], inspect5 and continue in the greater half. $\log_2 n$ counts repeated divisions by2 until1;1024 needs10 divisions. Sorting rearranges elements under a rule, such as ascending value.

The customer contract accepts a positive integer customer ID and a nonnegative total, rejecting other values. For this example amounts use small integer currency units, such as cents, to avoid fractional rounding. Empty input returns an empty map.

```
totals = empty map
for each order in orders:
    validate order.customer_id and order.total
    old = totals.get(order.customer_id, default=0)
    totals[order.customer_id] = old + order.total
return totals
```

get retrieves the value or0 for a new customer; validate applies the rules above and is not a supplied language function. Inputs (customer1,100), (customer2,50), (customer1,20) produce {1:120,2:50}; a total of−1 is rejected. With a suitable hash map, expected time is O(n) and extra space O(k) for k distinct customers.

A **precondition** specifies valid starting input; a **postcondition** specifies the required result. The **invariant** is that after each order, the map totals all examined orders for each customer. **Termination** follows from processing a finite list. **Recursion** calls the same function on a smaller problem and needs a stopping case; summing a list can add the first item to the sum of the rest (Representational State Transfer, an architectural style with resources, a uniform interface, and stateless interactions), stopping at an empty list. Waiting calls consume call-stack space, so a loop may be simpler for a long sequence.

Worked check: the maximum of [−4] is−4; empty input returns "no value" under this contract. After each examined item, largest represents the greatest value seen; initializing it to zero fails on negative-only lists.

Complexity, solution patterns, and test design

Best, average, and worst cases may perform different amounts of work. Time complexity tracks operation growth; space complexity tracks additional memory. A hash set may spend memory to reduce repeated searches.

Common patterns include brute force, simulation, frequency maps, prefix sums, two pointers, and greedy choice. Each depends on constraints and correctness reasoning rather than its name alone.

Tests should cover equivalence classes, boundaries, empty and single-element input, duplicates, large values, invalid data, and overflow. Write the expected result independently before comparing program output.

Next step

After completing this practice, continue with Choose between greedy steps and saved results.

After completing this practice, continue with Turn solution patterns into steps.

Why does a loop work and stop?

Why does a loop work and stop?

Return to the trace for summing 1 through 3. Before each iteration ask what has already been computed. A **loop invariant** remains true at a specified point in every iteration, not just one sample run.

Trace, then justify

Before i=1 the sum is 0; before i=2 it is 1; before i=3 it is 3. Each time sum contains all integers before i. Adding i preserves the claim for the next iteration. Remaining values decrease to zero, so the loop stops. This distinguishes sample testing from reasoning about correctness.

The **loop invariant** is that before each test, sum contains integers1 through i−1. It starts true because an empty sum is0, and adding i preserves it. A **variant**, a quantity moving toward termination, is the count of remaining values N−i+1 while the loop continues; it decreases by one each iteration.

Nested loops are not automatically quadratic. If the outer loop runs n times and the inner runs3 times, total work is3n, or linear O(n). If the inner loop runs n times for every outer iteration, work is n^2. Count the steps before labeling growth.

Try: four outer iterations and three inner iterations execute the body 12 times. If their counts vary independently, call them n and m: n×m, not necessarily n^2.

Turn solution patterns into steps

Turn solution patterns into steps

A pattern name is not a solution. Use a small list to count frequencies and sum a range, then learn when sorted order justifies moving two pointers. Calculate the answer manually before choosing the method.

Turn pattern names into steps

For [2,1,2,3], a **frequency map** associates each value with its count: {2:2,1:1,3:1}. A **set** stores only distinct values, {1,2,3}, answering whether a value appeared before.

A **prefix sum** array for [2,1,2,3] is P=[0,2,3,5,8]. The initial zero represents no elements. With element positions starting at1, positions2 through4 sum to P[4]−P[1]=8−2=6. Building P visits the list once; each range query then needs two lookups and a subtraction.

Two pointers can find two values summing to9 in the sorted list [1,3,5,8]. Start at both ends:1+8=9. If the sum is too small, move the lower pointer right; if too large, move the upper pointer left. Sorted order justifies discarding those candidates; the rule does not work unchanged on an unsorted list.

Brute force tries every pair: n(n−1)/2 pairs for n elements. Growing from10 to100 elements raises the count from45 to4950. This quadratic growth is written $O(n^2)$. Visiting n elements once is linear, O(n). Constant work, O(1), does not grow with n; it does not mean one second.

Greedy chooses the best immediate step and needs a correctness argument; see the coin counterexample in the next lesson's worked coin example. **Dynamic programming** saves results of repeated smaller problems; optimization problems also need an appropriate relationship between optimal smaller solutions and the whole solution. **Recursion** means a function calls itself on a smaller input with a stopping case; not every recursive solution is dynamic programming. **Divide and conquer** solves smaller parts and combines them, as when sorting two halves and merging them.

Worked check: [4,4,2] contains two4s, two distinct values, and sums to10. Under this contract an empty list sums to0; "hello" is rejected because inputs must be numbers. **Overflow** occurs when a result cannot fit its numeric type; check bounds before trusting the result.

Choose between greedy steps and saved results

Choose between greedy steps and saved results

Find the fewest coins for an amount. Before naming an algorithm, make 6 from 1, 3, and 4: largest-first gives 4+1+1, while 3+3 is better. Learn why a quick choice can fail and how a table of smaller results helps. First study complexity and solution patterns.

Greedy versus dynamic programming: choose with evidence

In the coin example below, dp names the results table, dp[x] is the entry for amount x, min selects the smaller value, and != means “not equal”. O(Tk) describes work growing with target amount T times the number of coin types k; O(T) describes table space growing with the target.

Form 6 with the fewest coins from {1,3,4}, allowing reuse. Greedy chooses the largest usable coin now: 4+1+1 uses three, but 3+3 uses two. A locally best choice does not guarantee a globally best solution; greedy needs a problem-specific proof.

Dynamic programming stores smaller solutions. Let dp[x] be the minimum coins forming x. Start with dp[0]=0 and other entries unreachable. Try each last coin c: remainder x−c needs dp[x−c], then add one. Denominations must be positive integers and the target a nonnegative integer.

```
coins = [1, 3, 4]
dp[0] = 0
for x from 1 through target:
    dp[x] = infinity
    for c in coins:
        if c <= x and dp[x-c] != infinity:
            dp[x] = min(dp[x], dp[x-c] + 1)
```

x	0	1	2	3	4	5	6
dp[x]	0	1	2	1	1	2	2

For 6 compare dp[5]+1=3, dp[3]+1=2, and dp[2]+1=3: choose two. Every solution ends with a tried coin, and its remainder is a smaller solved problem; this justifies the recurrence. Infinity is a pseudocode marker: use a special value or a large sentinel without adding one to it. For k denominations and target T, time is O(Tk), memory O(T); a billion-sized target is unsuitable even if k is small.

Worked exercise: With {3,4}, target 2 is unreachable, not zero coins. Target zero takes zero coins. If each coin is available once, this algorithm violates the contract by reusing it; change the state or update order to represent limited inventory.

Measurement, units, and prime numbers

Measurement, units, and prime numbers

Use this section after values, decisions, and loops. Review arithmetic when needed; graphs, complexity, and dynamic programming are later extensions, not first-program prerequisites.

Overview

Applied mathematics connects numbers to units and representations. Conversion, rounding, coordinates, graphs, and primes are practical tools rather than isolated formulas.

Core concepts

- Choose units before calculating and convert values to a common unit.
- floor moves downward and ceil upward, including for negatives; round chooses the nearest value.
- A coordinate (x,y) identifies a point, and a graph shows how variables change together.
- For y=2n+3, slope 2 means y rises by 2 when n rises by 1.
- To test n for primality, checking divisors through √n is sufficient.

Worked example

To test 29, try prime divisors up to √29≈5.38: 2, 3, and 5. None divides it, so 29 is prime.

Corrections and common mistakes

- ceil(-2.3) is -2, not -3.
- One is not prime, and every even number greater than two is composite.

Lesson map

Concept map: Applied math, measurement, graphs, and prime numbers

Check your understanding

01 **Why is ceil(-2.3) -2 while floor(-2.3) is -3?**

> **Explained answer:** Ceil is the smallest integer not below the value; floor is the greatest integer not above it.

02 **Prove why primality testing only needs divisors through √n.**

> **Explained answer:** If n=a×b and both factors exceeded √n, their product would exceed n. Every composite factor pair therefore has at least one factor at or below √n.

03 **What does slope 2 mean in y=2n+3?**

> **Explained answer:** Each one-unit increase in n increases y by two; 3 is the intercept, or the value at n=0.

04 **Why is testing that 1 is not divisible by 2 insufficient to call it prime?**

> **Explained answer:** A prime has exactly two positive divisors: 1 and itself. One has only one positive divisor.

Next step

After completing this practice, continue with Sequences, counting, and probability.

After completing this practice, continue with Draw relationships and search for a path.

Sequences, counting, and probability

Sequences, counting, and probability

Two shirts and three trousers give six combinations: list them first. From counting, distinguish ordering people from choosing a group, then calculate probability with explicit assumptions. First understand sets from the preceding lesson.

Sequences, counting, probability, and graphs

Arithmetic sequences add a fixed difference, while geometric sequences multiply by a fixed ratio. A recurrence defines terms from earlier terms and must include base cases.

The multiplication principle multiplies the number of available choices at each stage when that count is the same for every preceding choice; it does not require probabilistic independence. Permutations consider order; combinations do not. Probability begins with a sample space and events. Pseudorandom generators are deterministic and ordinary generators must not protect secrets.

Work through sequences, counting, and randomness

An arithmetic sequence such as 3,5,7,9 adds2 each time; term n, starting at n=1, is 3+2(n−1). A geometric sequence such as 3,6,12,24 multiplies by2; term n is 3×2^(n−1). The caret here denotes a power in plain mathematical text, not an operator shared by every programming language.

A **recurrence** defines a value using earlier values. Fibonacci starts with F(0)=0 and F(1)=1, then F(n)=F(n−1)+F(n−2) for n≥2. Those **base cases** let us calculate 0,1,1,2,3,5; the recurrence alone does not supply a starting value.

The **factorial**, n!, multiplies integers from1 through n; 0!=1. Thus4!=24 arrangements of four different people. Values grow quickly:20! exceeds a32-bit integer. Check the maximum before choosing a bounded numeric type.

A **pseudorandom** sequence is generated by an algorithm from an initial state called a **seed**. The same algorithm and seed reproduce the sequence, which helps repeat a simulation test. Passwords and keys require a generator designed for security; using the current time as a seed is not enough.

Worked check: the fourth sequence terms are9 and24. Rounding −2.3 downward gives−3 and upward gives−2. A halfway value such as2.5 depends on the language's rounding rule; do not assume all round functions resolve ties alike.

Worked sets, counting, and probability

A set contains distinct elements. Let A={1,2,3} represent learners passing test one and B={3,4} test two. Union A∪B={1,2,3,4} means passing at least one, intersection A∩B={3} means both, and difference A−B={1,2} means only the first. The union size is 3+2−1=4: subtract the overlap counted twice.

Choosing a leader and deputy from four people without repetition gives $4\times3=12$ because roles are ordered. Choosing two members without roles gives $12\div2=6$ because each pair was counted twice. The multiplication principle tracks available choices at each step; probabilistic independence is not required.

For a fair die, even has probability $3/6=1/2$. Given a result greater than 3, the remaining outcomes are $\{4,5,6\}$, so even has probability $2/3$. Favorable outcomes divided by all outcomes assumes equal likelihood.

Worked exercise: A bag has two red balls and one blue. Drawing two without replacement gives probability $(2/3)\times(1/2)=1/3$ for two reds: after a first red, one of two remaining balls is red. With replacement it becomes $(2/3)\times(2/3)=4/9$. State the assumptions before calculating.

Draw relationships and search for a path

Draw relationships and search for a path

A **graph** has vertices for things and edges for relationships. Every edge here is undirected and costs one step. F is isolated:

Edges are A–B, A–C, B–D, C–E, D–E. A **queue** removes the earliest inserted item. Start with A and mark vertices when enqueuing them so they cannot be queued repeatedly.

Removed vertex	Queue after adding undiscovered neighbors
Start	A
A	B, C
B	C, D
C	D, E
D	E
E	empty

E was discovered through C: A→C→E takes two edges. **Try:** from B, a shortest path is B→D→E. F remains unreachable. Next name this BFS and measure its work.

Trace a graph with a queue

A vertex represents a place and an edge connects places. Use an undirected graph with unit-cost edges: A connects to B,C; B to D; C to E; D to E. F is isolated.

Breadth-first search (BFS) visits levels. A queue removes the earliest inserted element first. Start with [A], remove A and add B,C; remove B and add D; remove C and add E. A shortest path from A to E is A→C→E with two edges. Mark vertices discovered when enqueuing to avoid repeated insertion around cycles. Record parent[E]=C and parent[C]=A to reconstruct the path backward.

F is unreachable. BFS minimizes edge count, not general weighted cost. With V vertices, E edges, and adjacency lists, time is O(V+E) and extra memory O(V). This graph is not a tree: A→B→D→E→C→A is a cycle, and F disconnects it. A simple undirected tree is connected, has no cycles, and has V−1 edges.

A graph contains vertices and edges. Edges may be directed or weighted. Paths connect vertices, cycles revisit a vertex, and a tree is connected and acyclic. Graphs model networks, routes, dependencies, and social relationships.

In the graph example below, V counts vertices and E counts edges. **O(V+E)** describes work growing with their sum, assuming an adjacency list stores each vertex's neighbors. **O(V)** additional space grows with the number of vertices. These expressions describe growth, not seconds. See comparing solutions.

02

TRACK · 14 LESSONS

The C++ language

Progressive explanations, practical examples, and corrections that take each idea into real code.

The C++ language

C++

An ordered path from problem solving and computer fundamentals through C++ builds, data, decisions, loops, and projects.

This section begins with problem analysis and number representation, then explains C++ tools, program structure, and the compiler pipeline in detail. The lessons continue through types, variables, text encoding, operators, input, conditions, `switch`, and loops. Complete examples, projects, common mistakes, and review questions accompany the explanations.

Lesson map

1. **Problem solving, diagrams, and debugging** — Decision trees, flowcharts, pseudocode, tests, and logic-error tracing.
2. **Number systems, computers, and memory** — Binary, octal, hex, CPU, memory, storage, and operating systems.
3. **C++ introduction, tools, and the first program** — Language goals, standards, editors, IDEs, program structure, and output.
4. **Compiler, build pipeline, and diagnostics** — Preprocessing, tokens, AST, IR, optimization, assembly, objects, linking, and errors.
5. **Types, variables, scope, and memory** — Built-in types, sizes, ranges, initialization, constants, scope, and lifetime.
6. **Text encoding, characters, and casts** — ASCII, Unicode, UTF-8, `char`, and explicit conversions.
7. **Operators, expressions, and bitwise operations** — Precedence, assignment, comparison, logic, short circuiting, and bit operations.
8. **Input, conversions, limits, and math** — Stream state, casts, `numeric_limits`, `cmath`, and formulas.
9. **if, else, validation, and nesting** — Exclusive and independent branches, ranges, ordering, and nested rules.
10. **switch, control flow, and a calculator** — `case`, `break`, `default`, fall-through, and local scope.
11. **Loops and repetition patterns** — `while`, `for`, `do-while`, counters, accumulators, sentinels, and nested loops.
12. **Input and business rules** — Turning requirements into variables, Boolean rules, and tests. The **problem-practice** material is now inside the problem-solving lesson, while condition and formula exercises are inside the `if` and `else` lesson. The old files remain in the project as archives, but they are no longer part of the study order.

> **Study method**
>
> Run every example, change its inputs, and predict the result before execution. Draw the build pipeline during the compiler lesson and write test cases before conditions and loops.

13. Functions, containers and references
14. Objects, STL, RAII and testing

1. Problem solving, diagrams, and debugging

Before writing C++, stabilize the solution logic: define inputs and output, write the steps, model decisions, and dry-run the algorithm. The language implements the reasoning; it does not replace it.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Runtime:** The period when a program is actually running.
- **Loop:** A structure that repeats instructions according to a condition.

Problem analysis before code

Before writing C++, stabilize the solution logic: define inputs and output, write the steps, model decisions, and dry-run the algorithm. The language implements the reasoning; it does not replace it.

Inputs, outputs, constraints, and tests

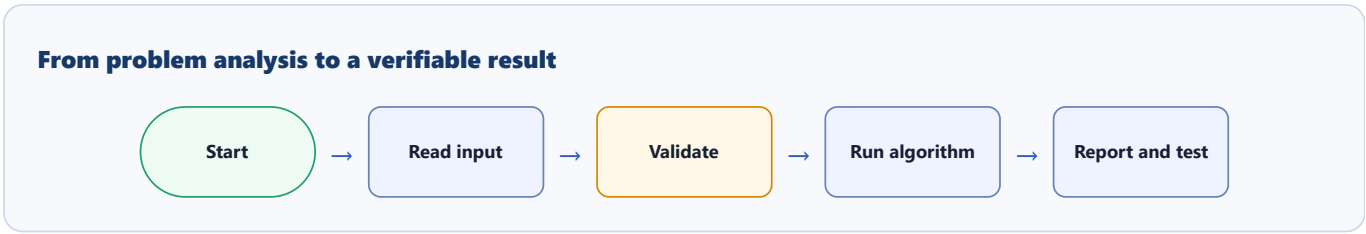
- Use pseudocode for sequence and a flowchart for visible branches.
- Design the valid path, then add boundaries and invalid inputs.
- Syntax errors stop the build; logic errors produce wrong answers even when the program runs.
- A trace table records variable values after each step and exposes divergence.

Example: safe division

```
READ a, b
IF b = 0 THEN
  PRINT "Division is undefined"
ELSE
  PRINT a / b
END IF
```

Syntax, runtime, and logic errors

- Do not test only easy values; include zero, boundaries, and negatives.
- A diagram must reflect real execution paths, not act as decoration.



Decision trees and business rules

A decision tree places a question at each node, a branch for each answer, and a complete outcome at each leaf. It exposes missing paths and contradictory rules before they become nested `if` statements. Test exact boundaries such as age 21 or subtotal 500.

Flowcharts and pseudocode

Flowcharts use terminators, process boxes, decision diamonds, input/output shapes, connectors, and directed lines. Pseudocode expresses the same logic in structured, language-independent text. A calculator flowchart should branch by operator, validate a zero divisor on the division branch, and rejoin successful paths at one result output.

Tests and logic-error tracing

Prepare normal, boundary, and invalid cases before code. Reproduce a wrong result, record state after each step, find the first divergence, correct one cause, and rerun the failing case plus its neighbors. In a running-total bug, the first incorrect iteration is more informative than later totals that inherited the error.

Problem contract and algorithm correctness

Write the input domain, required output, constraints, and invalid cases before choosing syntax. An algorithm is correct only when it terminates and produces the required result for every valid input, not only for the sample. A trace table records the state after each step and exposes missing updates or incorrect branches.

Flowchart symbols and equivalent test classes

Use an oval for start/end, a parallelogram for input/output, a rectangle for processing, and a diamond for a decision. Label both decision exits. Group similar inputs into equivalence classes, then test boundaries on both sides: for an allowed score `0..100`, test `-1, 0, 1, 99, 100, 101`.

Logic-error investigation

When code runs but returns a wrong answer, reproduce the failure with the smallest input, write the expected trace, compare it with the actual state, and locate the first divergence. Keep that case as a regression test after fixing the defect.

Pseudocode, flowcharts, and trace tables

Pseudocode records the order of decisions without committing to C++ syntax. A flowchart makes branching and repetition visible. A trace table then verifies the algorithm by listing each important variable after every step. These tools answer different questions: pseudocode explains the procedure, the flowchart exposes paths, and the trace table tests state changes.

For safe division, write the contract first:

1. Read a numerator and denominator.
2. If the denominator is zero, produce an error and stop that path.
3. Otherwise divide and display the result.

numerator	denominator	selected path	result
8	2	valid division	4
8	0	rejection	—
-9	3	valid division	-3

Test cases before implementation

Partition inputs into equivalent classes instead of choosing only comfortable samples: a normal positive value, zero, a negative value, a minimum or maximum boundary, and malformed input when the program reads text. A single representative from each class is useful, but values immediately below, at, and above every boundary are essential for discovering off-by-one errors.

When investigating a logic error, preserve the failing input, state the expected result, record the actual result, and trace only the variables that influence the wrong decision. Change one hypothesis at a time; random edits destroy evidence.

Apply the problem-solving method on practice platforms

Competitive programming

Competitive programming trains constraint analysis, algorithm choice, and fast correct implementation. The judge is a measurement tool, not a substitute for understanding.

Statements, constraints, and complexity

- Read input, output, and constraints before samples.
- Translate constraints into acceptable complexity; n=10^5 usually rules out $O(n^2)$.
- Solve samples by hand, then add custom cases.
- After Wrong Answer, inspect boundaries, types, rounding, and interpretation.
- State time and memory complexity after solving.

Example: Elephant and ceiling division

```
// Elephant: minimum moves of length at most 5
int moves = (distance + 4) / 5; // integer ceiling
```

Complete runnable submission

```
#include <iostream>

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);

    int distance{};
    if (!(std::cin >> distance) || distance < 0) {
        return 1;
    }

    const int moves = (distance + 4) / 5;
    std::cout << moves << '\n';
}
```

For input `12`, the output is `3`: two moves of length 5 and a final move of length 2. Time and auxiliary memory are both `O(1)`. The program prints no prompt such as `Enter distance` because an online judge compares output exactly. The validation helps during local use, although the original problem normally guarantees valid input.

The Online Judge workflow

The judge compiles submitted source code with a documented compiler and options, runs the executable against hidden tests under time and memory limits, and compares actual output with expected output through a checker. `Compilation Error` means the source did not build. `Wrong Answer` means it ran but produced incorrect output. `Runtime Error` covers failures such as invalid access or division by zero. `Time Limit Exceeded` and `Memory Limit Exceeded` identify exhausted resource limits.

Samples, submissions, and random changes

- Passing samples does not prove correctness.
- Do not start with maximum difficulty; build a consistent practice ladder.

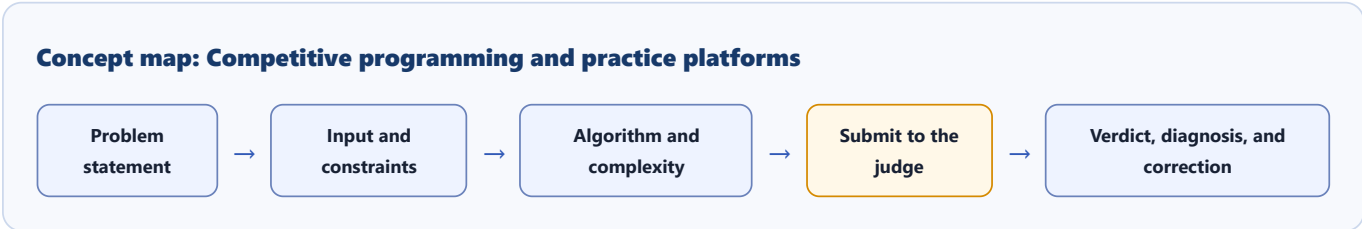
Platforms and problem patterns

Codeforces emphasizes contests, ratings, and broad problem sets. LeetCode often emphasizes interview-oriented data-structure patterns. Begin with implementation, arithmetic, and conditions, then add loops and containers. After each problem record the idea, the bug, complexity, and a missed test case.

The practice set covers: ceiling division in 617A and 1A, `min` and remainder in 581A, character handling in 281A, parity in 4A and 959A, formula derivation in 486A, three-way comparison in 835A, guaranteed outcomes in 1173A, odd/even block mapping in 318A, and geometric case analysis in 459A.

Read an editorial only after a genuine attempt, then close it and derive the method again on a different example.

Lesson map



Check your understanding

01 **Why must a denominator be checked before division?**

> **Explained answer:** After division the invalid operation has already happened. Early validation prevents it and defines a clear rejection path.

02 **A program runs but mishandles equal maximum values. What kind of defect is this?**

> **Explained answer:** A logic error in comparisons or branch order; successful compilation and execution do not prove algorithm correctness.

03 **When is a flowchart better than pseudocode?**

> **Explained answer:** When branches and returns need visual communication; pseudocode is better for detailed sequential logic close to code.

04 **How would you prove a largest-of-three algorithm covers its domain?**

> **Explained answer:** Test each position as largest, ties, negatives, and type boundaries, with expected results written first.

Connect the ideas

After drawing the solution, state a loop invariant and a termination argument instead of relying on examples. Convert the division example into a complete C++ program with zero, negative, and boundary tests, then compare its trace with the prediction.

Try it yourself

Introduce a compiling logic defect and prove a boundary test catches it.

2. Number systems, computers, and memory

Number systems explain how values are represented; computer architecture explains where a program lives and how it moves from storage to execution.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Cache:** A temporary copy that reduces waiting and repeated work.

Values and numeral systems

Number systems explain how values are represented; computer architecture explains where a program lives and how it moves from storage to execution.

Binary, octal, decimal, and hexadecimal

- Decimal uses base 10, binary 2, octal 8, and hexadecimal 16.
- A positional digit contributes digit × base^position.
- One hexadecimal digit represents four bits.
- The CPU executes instructions, RAM holds active state, and cache reduces repeated-access latency.
- The OS loads a program from storage into memory and creates a process.

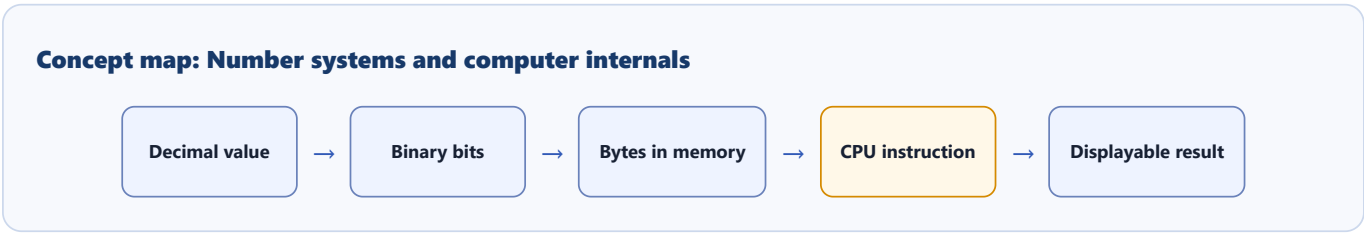
Example: one value in three bases

$$45_{10} = 32 + 8 + 4 + 1 = 101101_{2} = 2D_{16}$$

RAM, storage, and representation mistakes

- RAM is not persistent storage.
- Conversion changes representation, not the value.

Lesson map



Bases and conversion

Decimal, binary, octal, and hexadecimal use positional powers of 10, 2, 8, and 16. Convert binary to decimal by summing active place values. Convert decimal to binary by repeated division or powers-of-two decomposition. One hex digit represents four bits, so grouping binary from the right gives a compact exact representation.

Hex appears in addresses, debuggers, byte viewers, colors, and bit masks. Octal still appears in areas such as Unix permissions.

Hardware and memory hierarchy

Storage holds program files, RAM holds active pages, and the CPU fetches and executes instructions. Registers are smallest and fastest, followed by caches, RAM, and persistent storage. The motherboard connects components; the GPU handles graphics and suitable parallel work; power, cooling, network, input, and output devices support execution.

The operating system creates a process, maps memory, schedules CPU time, manages files and devices, and displays results. A stored executable is not a running process until the system loads and starts it.

Positional value and manual conversion

In base `b`, each digit is multiplied by a power of `b`. Convert binary to decimal by summing set-bit positions. Convert a nonnegative decimal integer to another base by repeated division and reading remainders in reverse. Hexadecimal maps each digit to exactly four bits, which makes it useful for addresses and masks.

Signed values and two’s complement

For a fixed width, two’s complement represents negative values so addition hardware can use the same circuit. Negating a bit pattern means invert the bits and add one. Width matters: an 8-bit demonstration does not describe the range of every C++ `int`.

CPU, memory, storage, and process execution

Storage keeps files without power. RAM holds active program data. CPU registers are smaller and faster; the ALU performs arithmetic and logic, while the control unit coordinates instructions. The operating system loads an executable, creates a process and virtual address space, schedules threads, and provides controlled access to files and devices through system calls.

Memory hierarchy

Registers, caches, RAM, and persistent storage trade capacity for latency. A value may exist at several levels during execution. Virtual memory maps process addresses to physical pages; a page fault asks the operating system to make a page available and is much slower than a cache miss.

Hexadecimal and the nibble

One hexadecimal digit represents exactly four binary bits, called a nibble. That is why `0xAF` maps directly to `1010 1111`: `A` is 10 or `1010`, and `F` is 15 or `1111`. Hexadecimal is a compact notation for bit patterns; it does not change the stored value.

Bits, bytes, words, and signed values

A bit has two states. A byte is the smallest addressable unit on typical systems and is normally eight bits. A machine word is the processor's natural working width and depends on the architecture. Do not infer a C++ type's exact range only from the machine being "64-bit"; use `sizeof` and `<limits>`, or fixed-width types when the exact width is part of the contract.

Signed integers are commonly represented with two's complement. For an `n`-bit signed value, the usual range is `-2^(n-1)` through `2^(n-1)-1`; the negative side has one extra value. This asymmetry matters when taking the absolute value of the minimum representable integer.

From storage to a running process

An executable rests on persistent storage. The operating system loader maps its code and data into memory, prepares the process, and schedules its threads. The CPU fetches, decodes, and executes instructions; registers and caches hold nearby working data, RAM holds the active program state, and storage preserves files after power is removed. Faster levels are smaller and closer to the CPU, while slower levels are larger.

Input devices, displays, network adapters, and storage controllers communicate through operating-system drivers. They are not "inside the CPU"; the OS coordinates access and presents abstractions such as files, sockets, and processes.

Check your understanding

01 **Convert 0x2D to decimal and binary.**

> **Explained answer:** 2D is 2×16+13 = 45; each hex digit is four bits, so it becomes 0010 1101.

02 **Why is hexadecimal common for addresses and colors?**

> **Explained answer:** It is much shorter than binary while mapping exactly to groups of four bits.

03 **What roughly happens when a program is opened from an SSD?**

> **Explained answer:** The OS loads needed code/data into RAM, creates a process, and the CPU fetches instructions with cache assistance.

04 **Does adding RAM make the CPU itself faster?**

> **Explained answer:** No, but it can reduce slow storage paging under memory pressure and improve overall throughput.

Connect the ideas

Endianness controls byte order for multi-byte values and does not reverse written bit order inside a byte. IEEE-754 splits floating values into sign, exponent, and fraction with special values. A pointer is a virtual address, not a physical-slot number.

Try it yourself

Inspect bytes for an integer and float on your machine and interpret them without assuming portability.

3. C++ introduction, tools, and the first program

C++ is a compiled language with strong performance and memory control. Understanding program structure and the build path matters more than depending on one IDE.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Compiler:** A program that turns source code into a form the computer can run.
- **Runtime:** The period when a program is actually running.
- **Debugger:** A tool that pauses a program so you can inspect values and execution step by step.
- **Token:** A value representing identity or permission without resending a password.
- **Scope:** The region of code in which a name or variable is visible.
- **Function:** A named, reusable block of code with one defined job.

The C++ language

C++ is a compiled language with strong performance and memory control. Understanding program structure and the build path matters more than depending on one IDE.

Editors, compilers, IDEs, and source code

- An editor writes files, a compiler translates them, and an IDE integrates several tools.
- A statement performs an action, an expression produces a value, and a token is a smallest meaningful unit.
- Execution starts at main; returning zero conventionally signals success.
- iostream provides streams, and std is the standard-library namespace.
- The build pipeline covers preprocessing, compilation, assembly, and linking.

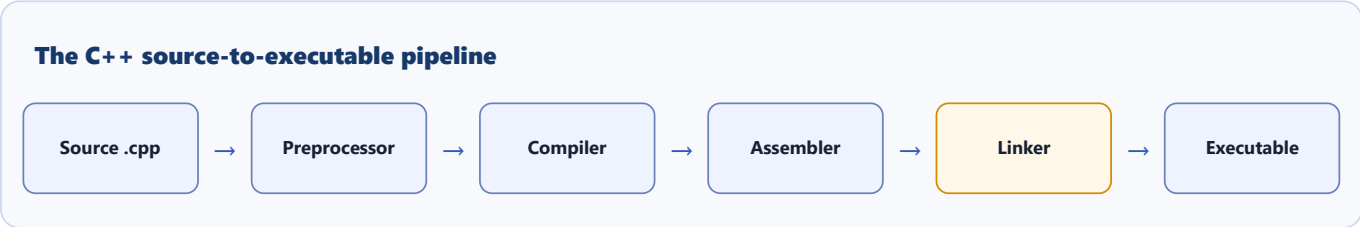
The first C++ program

```
#include <iostream>

int main() {
    std::cout << "Hello, C++!\n";
    return 0;
}
```

Build warnings and namespace mistakes

- Enable warnings and start with the first diagnostic related to your code.
- Avoid global using namespace std in headers or large projects.



Toolchains, standards, and diagnostics

An editor changes text; an IDE also manages projects, builds, and debugging. MSVC, GCC, and Clang are the toolchains that compile the program. Select a project standard such as C++20 or C++23 and verify compiler support instead of assuming every feature is available.

The preprocessor expands includes and directives. The compiler analyzes tokens, syntax, and types and generates code. The assembler creates object files, and the linker resolves symbols across object files and libraries. A missing semicolon is a compile error; a declared but undefined function usually becomes a link error; a crash is a runtime error; a wrong result is a logic error.

Read the first relevant diagnostic, reduce the problem to a small reproduction, change one cause, and rebuild.

Language, standards, and tools

C++ combines high-level structure with explicit resource and memory control. It appears in engines, browsers, embedded systems, and performance-sensitive software. Standards from C++98 through C++11, 17, 20, and 23 evolve the language; the selected compiler mode determines which standard rules are enabled.

An editor changes files, an IDE also integrates projects and debugging, and a toolchain performs compilation. Visual Studio provides MSVC through its C++ workload. VS Code needs an external compiler. Code::Blocks may bundle one. Tool choice does not change language rules.

Program structure and output

Tokens form expressions and statements. `price + tax` produces a value; `total = price + tax;` performs an assignment. `main` is the entry point, `<iostream>` provides stream declarations, and `std::cout` writes standard output. Escape sequences include `\n`, `\t`, `\"`, and `\\`. A character literal such as `'A'` differs from the string literal `"A"`.

Namespaces prevent collisions. Prefer qualified names such as `std::cout`; broad `using namespace` directives are especially harmful in headers.

Tokens, expressions, statements, and blocks

The compiler first sees tokens such as identifiers, keywords, literals, and operators. An expression produces a value or effect; a statement forms a complete action; braces create a block and often a scope. A semicolon ends many statements but must not be added blindly after `if` or `while`.

Source files, headers, and commands

Place declarations shared by files in guarded headers and definitions in `.cpp` files. A command such as `g++ -std=c++20 -Wall -Wextra -Wpedantic main.cpp -o app` selects a standard and useful warnings. Compilation success does not prove the formula is correct.

main, exit status, stdout, and stderr

`main` returns an integer status to the environment: zero normally means success. Command-line arguments arrive through `argc` and `argv` when that signature is used. Normal results go to `std::cout`; diagnostics go to `std::cerr`. Newline and flushing are different operations, so use `\n` unless an immediate flush is required.

Escape sequences and namespaces

`\n`, `\t`, `\\`, and `\"` represent characters that would otherwise be awkward inside a literal. `std::` identifies names from the standard library. Avoid `using namespace std;` in headers because it injects many names into every including translation unit.

Why C++ and what the standard controls

C++ combines high-level abstractions with explicit control over object lifetime, memory layout, and performance. The ISO language standard defines source-level behavior; an implementation supplies a compiler, standard library, linker, and platform-specific ABI. Selecting `-std=c++20` or another required standard prevents accidental dependence on whatever default a compiler happens to use.

Build stages and diagnostics

1. Preprocessing expands includes and macros into translation units.
2. Compilation parses and type-checks each translation unit and emits object code.
3. Linking resolves referenced symbols and combines object files and libraries into an executable.
4. Loading is performed by the operating system when the executable starts.

A missing semicolon is a compile error. Declaring a function without supplying its definition can become a linker error. Accessing invalid memory is a runtime failure. Producing the wrong total without crashing is a logic error. Read the first meaningful diagnostic before secondary messages caused by it.

math.hpp

```
#pragma once
int add(int left, int right);
```

math.cpp

```
#include "math.hpp"
int add(int left, int right) { return left + right; }
```

Build both source files, not the header by itself: `g++ -std=c++20 -Wall -Wextra -Wpedantic main.cpp math.cpp -o app`.

Strings, escape sequences, and buffered output

String literals use double quotes; character literals use single quotes. `\n` inserts a newline, `\t` a tab, `\\` a backslash, and `\"` a quote inside a string. Prefer `\n` for ordinary line endings. `std::endl` also flushes the stream, which is useful only when an immediate flush is required.

`std::cout` is standard output and `std::cerr` is the diagnostic stream. A command-line program communicates success or failure through the integer returned by `main`; returning zero conventionally means success. `int main(int argc, char* argv[])` exposes command-line arguments when they are part of the program contract.

Check your understanding

01 **How do an IDE and a compiler differ?**

> **Explained answer:** An IDE integrates editing, building, and debugging; a compiler translates and checks source. Either can be used without the other.

02 **Why can compilation succeed while linking fails?**

> **Explained answer:** Each unit can be valid alone while the linker cannot find a referenced definition or finds duplicate definitions.

03 **What is main and what does return 0 mean?**

> **Explained answer:** main is the hosted program entry point; zero conventionally reports successful completion to the operating system.

04 **How should you handle ten diagnostics caused by one mistake?**

> **Explained answer:** Fix the earliest relevant diagnostic and rebuild; later messages may only be cascading consequences.

Connect the ideas

Use a debugger with breakpoints and stack inspection instead of relying only on prints. Enable AddressSanitizer and UndefinedBehaviorSanitizer in development, and use CMake or a build script to apply consistent flags. Package managers resolve dependencies but do not replace version pinning and review.

Try it yourself

Build with warnings and sanitizers, then diagnose an intentional out-of-bounds access.

4. Compiler, build pipeline, and diagnostics

Follow C++ from source and translation units through tokens, AST, IR, optimization, assembly, object files, linking, loading, warnings, and failures.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Compiler:** A program that turns source code into a form the computer can run.
- **Runtime:** The period when a program is actually running.
- **Token:** A value representing identity or permission without resending a password.
- **Function:** A named, reusable block of code with one defined job.

Why C++ needs translation

Processors execute machine instructions for a target architecture. C++ source uses a human-readable language, so a toolchain transforms it into a binary understood by the operating system and CPU. “Compiler” often names the driver that coordinates several distinct stages.

```
Source files
  ↓ Preprocessor
Translation units
  ↓ Frontend + optimizer + backend
Assembly or machine code
  ↓ Assembler
Object files
  ↓ Linker + libraries
Executable
  ↓ Operating-system loader
Running process
```

Real toolchains may integrate stages and never save a separate assembly file, but this model explains where each diagnostic originates.

Compilation, interpretation, and JIT

Ahead-of-time compilation produces objects and a platform binary before execution. An interpreter manages translation and execution at runtime. Hybrid systems may compile to bytecode, run it in a virtual machine, and JIT-compile frequently executed paths. C++ normally compiles to native code, while Java, C#, and modern JavaScript runtimes illustrate hybrid models.

GCC, Clang, and MSVC

The compiler, standard library, target platform, architecture, and build options jointly affect available features, diagnostics, ABI, and binary format.

```
g++ -std=c++20 -Wall -Wextra -Wpedantic main.cpp -o app
clang++ -std=c++20 -Wall -Wextra -Wpedantic main.cpp -o app
cl /std:c++20 /W4 /EHsc main.cpp
```

Debug builds retain useful symbols and reduce optimization surprises while investigating. Release builds enable stronger optimization. Neither mode makes undefined behavior safe.

A program through the pipeline

```
#include <iostream>
#define TAX_RATE 0.14

int add(int a, int b) { return a + b; }

int main()
{
    int price{100};
    double total = price + price * TAX_RATE;
    std::cout << add(10, 20) << '\n' << total << '\n';
}
```

The expected output is `30` and `114`. The macro demonstrates preprocessing, although a typed `constexpr` is preferable for an ordinary numeric constant.

1. Preprocessing and translation units

The preprocessor handles `#include`, token substitution with `#define`, and conditional source selection through `#if`, `#ifdef`, and related directives. Header guards or `#pragma once` prevent repeated declarations inside one translation unit.

Each `.cpp` file after preprocessing, together with included header content, forms a **translation unit**. A widely included header can force many units to recompile when it changes. Headers normally provide shared declarations and types; ordinary non-inline definitions belong in source files.

2. Lexical analysis

The frontend recognizes tokens. In `int x = 10 + 20;`, tokens include a keyword, identifier, assignment operator, literals, addition operator, and punctuation. Tokenization identifies pieces but does not prove that their arrangement or meaning is valid. Comments normally disappear during early translation and do not affect runtime speed.

3. Parsing and AST

The parser validates token order against C++ grammar and builds an **abstract syntax tree**. The tree for `10 + 20 * 3` places multiplication below addition, representing operator precedence. `int x = ;` is a syntax error because an expression is missing. A diagnostic may point after the true mistake because the parser detects the inconsistency there.

4. Semantic analysis

The compiler then checks names, scopes, types, conversions, overload resolution, access rules, and `const` requirements.

```
int count = "Hello"; // incompatible type
unknown = 10;        // undeclared name
```

Both lines have recognizable structure, but their meanings violate C++ rules. One root mistake may produce many follow-up diagnostics, so fix the first relevant cause and rebuild.

5. Intermediate representation

An IR gives the optimizer a simpler, compiler-specific representation:

```
temp1 = b * c
temp2 = a + temp1
x = temp2
```

This is teaching pseudocode rather than real LLVM IR. A compiler may use several representations for different analyses.

6. Optimization

Optimization can fold constant expressions, remove unreachable work, inline functions, and transform loops while preserving required observable behavior for a well-defined program. Undefined behavior, such as signed overflow or using an object after its lifetime, removes important guarantees and may produce surprising optimized results.

7. Code generation and assembly

The backend selects instructions for the target. An addition may resemble:

```
mov eax, edi
add eax, esi
ret
```

This illustrative x86-64 sequence depends on the calling convention and ABI. Windows x64, System V x86-64, and ARM64 differ. Assembly is the textual instruction language; the assembler is the tool that encodes it as machine bytes.

8. Object files

An `.o` or `.obj` file commonly contains machine-code and data sections, provided and required symbols, relocation entries for unresolved addresses, optional debug information, and format metadata such as ELF or COFF. It is usually not runnable because references and final addresses remain unresolved.

9. Separate compilation

math.cpp

```
int add(int a, int b) { return a + b; }
```

main.cpp

```
int add(int, int);
int main() { return add(10, 20); }
```

Each translation unit becomes its own object. `main.obj` requires the function symbol; `math.obj` provides it. This allows incremental builds, though a changed header requires dependent translation units to rebuild.

10. Linking

The linker combines objects and required library components, resolves symbols, assigns final addresses, and writes the executable.

- **Undefined reference / unresolved external:** a declaration was visible, but no matching definition was linked.
- **Multiple definition:** the same non-inline definition appeared in more than one unit.
- **Architecture or library mismatch:** a dependency does not match the target or link configuration.

Static linking copies required library parts into the result. Dynamic linking leaves dependencies that the loader resolves at execution time.

11. Loading and execution

The operating-system loader reads the executable format, creates a process and virtual address space, maps program sections and shared libraries, prepares runtime state, and transfers control through startup code to `main`.

A successful build does not guarantee successful loading, safe execution, or correct results.

Diagnostic categories

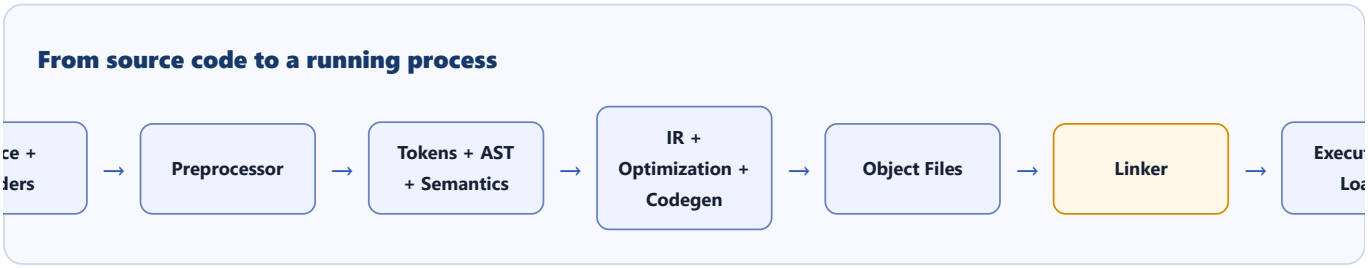
Category	Example	Stage
Preprocessor error	Missing header	Preprocessing
Syntax error	Missing semicolon	Parsing
Semantic error	Incompatible type	Frontend analysis
Link error	Missing function definition	Linking
Load error	Missing dynamic library	Loading
Runtime failure	Invalid memory access	Execution
Logic error	`area = width + height`	Testing reveals a wrong result
Warning	Narrowing `double` to `int`	Build may still succeed

Treat warnings as findings. Enable strong warnings and prevent new ones from accumulating.

A diagnostic workflow

1. Reproduce the issue with fixed input.
2. Identify whether it occurs during preprocessing, compilation, linking, loading, execution, or result verification.
3. Read the first diagnostic related to your code.
4. Reduce the program to the smallest reproduction.
5. Change one cause and rebuild.
6. Add a test that would catch the regression.

Lesson map



Check your understanding

01 **How do a source file, translation unit, and object file differ?**

> **Answer:** The source file is written code. The translation unit is its preprocessed form with included declarations. The object file contains compiled code, symbols, and relocations but may not run independently.

02 **Why can valid syntax still produce a semantic error?**

> **Answer:** Grammar may accept the statement shape while name lookup, type checking, or conversion rules reject its meaning.

03 **Why can compilation succeed before an undefined-reference failure?**

> **Answer:** The compiler only needed a declaration. The linker later failed to find a matching definition in linked objects or libraries.

04 **What does IR add between an AST and target code?**

> **Answer:** A compiler-oriented representation that simplifies analysis, optimization, and retargeting to different backends.

05 **Classify a missing header, undeclared name, missing function definition, missing DLL, and wrong area formula.**

> **Answer:** Preprocessing, semantic analysis, linking, loading, and logic testing respectively.

06 **Must optimization preserve a valid program's observable behavior, and what changes when the program has undefined behavior?**

> **Answer:** Optimization must preserve the required observable behavior of a program whose behavior is defined. Once undefined behavior occurs, the language no longer supplies that guarantee, so optimized results may appear surprising.

Connect the ideas

Study the lesson in three passes: preprocessing/compilation, object files/linking, then loading/runtime. Save preprocessed source, inspect symbols with `nm` or the platform equivalent, and inspect sections with `objdump`. Include ODR and template instantiation as causes of difficult linker failures.

Try it yourself

Keep three artifacts from one program and map each to a stage and possible failure.

5. Types, variables, scope, and memory

A type defines representation, valid operations, and range. A variable combines a name, type, value, lifetime, and visibility.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **API:** A defined interface through which one program requests data or actions from another.
- **Unicode:** A standard that assigns consistent numbers to characters and symbols.
- **UTF-8:** A common encoding that stores Unicode numbers as bytes.
- **Scope:** The region of code in which a name or variable is visible.
- **Boolean:** A logical value with only two states: true or false.

Types and variables

A type defines representation, valid operations, and range. A variable combines a name, type, value, lifetime, and visibility.

Type selection, initialization, scope, and lifetime

- Choose a type by meaning and range, not size alone.
- Brace initialization rejects many accidental narrowing conversions.
- const prevents later mutation; constexpr can represent compile-time values.
- Local scope reduces coupling; a static local retains value across calls.
- Unicode defines characters; UTF-8 and UTF-16 encode them into storage units.

Declaration and initialization examples

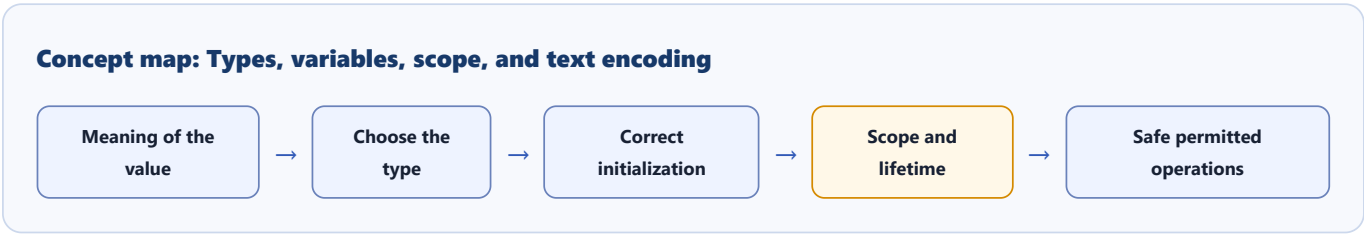
```
#include <limits>
#include <string>

int age{20};
double price{49.95};
const std::string country{"Egypt"};
constexpr int daysPerWeek{7};
```

Range, precision, and platform mistakes

- Type sizes can vary; use sizeof and numeric_limits when it matters.
- float and double are approximate, so direct decimal equality can be unsafe.

Lesson map



Type categories and ranges

Built-in types include Boolean, character, integer, floating-point, and `void`. Compound and user-defined types include arrays, pointers, references, classes, and enums, while `std::string` comes from the standard library. Sizes vary by implementation, so inspect `sizeof` and `numeric_limits` rather than assuming one platform.

Unsigned arithmetic wraps modulo its width, while signed overflow is undefined. Mixing signed and unsigned values can turn a negative number into a large positive value. Floating-point values have limited binary precision, so exact decimal equality is often inappropriate.

Initialization, constants, scope, and lifetime

Brace initialization catches many narrowing conversions. Value initialization with `{}` gives a defined zero-like value for arithmetic types. `auto` still has a concrete inferred type. `const` prevents later modification through that name; `constexpr` supports compile-time values when requirements are met.

Scope determines where a name is visible. Lifetime determines when the object exists. Shadowing hides an outer name and can confuse review. Optimizers may keep values in registers or remove storage entirely, so a source-level variable does not guarantee a permanent memory slot.

Signed, unsigned, floating point, and literals

Signed integer overflow is undefined behavior; unsigned arithmetic wraps modulo its range but can still be a logic bug. Floating-point values have limited precision, so many decimal fractions are approximations. `10` is an integer literal, `10.0` is `double`, `10LL` requests `long long`, and `10.0f` requests `float`.

Initialization, assignment, auto, and undefined values

Initialization creates an object with its first value; assignment changes an existing object. `auto` performs compile-time type deduction and does not create a dynamically changing type. Reading an uninitialized automatic variable can produce undefined behavior. Use the narrowest useful scope and distinguish name visibility from object lifetime.

Complete program: types and range validation

```
#include <iostream>
#include <string>
int main() {
    std::string name; int age{}; long long balancePiasters{};
    std::cout << "Name age balance-in-piasters: ";
    if (!(std::cin >> name >> age >> balancePiasters)) {
        std::cerr << "Invalid input\n"; return 1;
    }
    if (age < 0 || age > 130 || balancePiasters < 0) {
        std::cerr << "Value outside the accepted range\n"; return 1;
    }
    const bool adult = age >= 18;
    std::cout << name << " | age=" << age
              << " | adult=" << std::boolalpha << adult
              << " | balance=" << balancePiasters << " piasters\n";
}
```

`Omar 24 12550` is valid, while `Omar -2 100` has valid C++ types but violates the domain. A type does not replace validation.

Signed, unsigned, and range

Unsigned arithmetic wraps modulo `2^n`, but that does not make it a general cure for negative values or overflow. Mixing signed and unsigned operands may convert the signed value to unsigned and produce a surprising comparison. Choose a signed type for ordinary quantities that may participate in subtraction, and validate that domain values are nonnegative.

Use `std::numeric_limits<T>::min()` and `max()` when range matters. `<cstdint>` types such as `std::int64_t` communicate width when the implementation provides them.

Floating point and precision

`float` and `double` approximate most decimal fractions in binary. Equality after arithmetic can therefore be unreliable; compare a difference against a tolerance chosen for the problem scale. Floating point is appropriate for measurements, but exact money calculations usually use integer minor units.

Constants, storage duration, scope, and shadowing

`const` prevents mutation after initialization. `constexpr` additionally allows compile-time evaluation when its initializer is a constant expression. Local automatic variables live until their block ends; static-storage objects live for the program duration. Scope controls where a name is visible, while lifetime controls whether the object still exists.

Declaring an inner variable with the same name shadows the outer one:

```
int count{10};
{
    int count{2};  // a different object
    std::cout << count << '\n';
}
std::cout << count << '\n';
```

The output is `2` then `10`. Avoid unnecessary shadowing because it makes reviews and debugging harder.

Check your understanding

01 **Why is int not automatically a good age model even if values are small?**

> **Explained answer:** It may fit numerically, but type alone does not reject negative or unrealistic ages; domain validation is still required.

02 **What does brace initialization improve?**

> **Explained answer:** It makes initialization explicit and rejects many narrowing conversions such as directly placing 3.7 into int.

03 **How do const and constexpr differ?**

> **Explained answer:** const prevents later mutation; constexpr represents a value that can be evaluated at compile time in the required context.

04 **Why does one char usually not hold an Arabic UTF-8 character?**

> **Explained answer:** That character is commonly encoded as multiple bytes, while char stores one byte-sized code unit.

05 **Predict the output of `int x{3}; double y{x / 2};` when `y` is printed. Why?**

> **Explained answer:** It prints 1 because integer division happens before conversion to double. Use `double y = x / 2.0;` to obtain 1.5.

06 **Correct a declaration that stores a visitor count above two billion and must not change after initialization.**

> **Explained answer:** After input validation, use a documented-width type such as `const std::int64_t visitors{value};` instead of assuming `int` is wide enough.

Connect the ideas

A reference aliases an existing object and is not normally null, while a pointer is an address value that can be null or reassigned. enum class avoids name leakage and implicit integer conversion. Prefer a sized view such as `std::span` when passing contiguous data.

Try it yourself

Write one API with pointer, reference, and span variants and state each contract.

6. Text encoding, characters, and casts

Text encoding, conversions, and math functions often meet in real problems. Know what a value represents before casting or calculating.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Runtime:** The period when a program is actually running.
- **Unicode:** A standard that assigns consistent numbers to characters and symbols.
- **UTF-8:** A common encoding that stores Unicode numbers as bytes.

Characters, text, and encodings

Text encoding, conversions, and math functions often meet in real problems. Know what a value represents before casting or calculating.

ASCII, Unicode, UTF-8, and UTF-16

- ASCII covers a limited legacy set; Unicode defines code points across scripts.
- UTF-8 is variable length and ASCII-compatible; UTF-16 uses one or two code units.
- A char does not necessarily hold a full Unicode character.
- Widening usually preserves value; narrowing may lose data or exceed range.
- Math functions return specific types and may require domain checks.

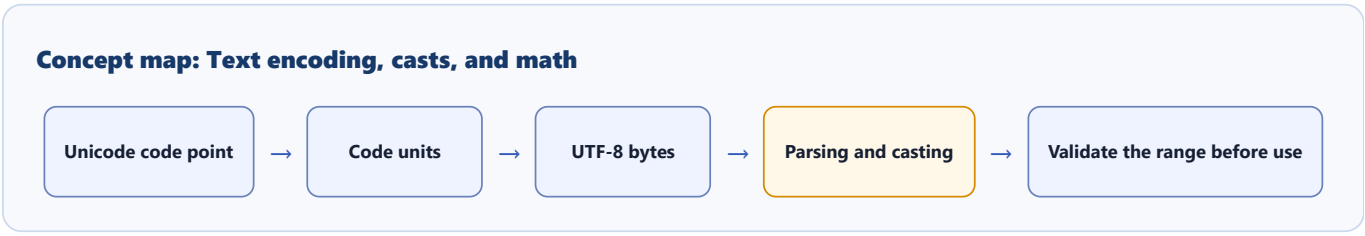
Example: widening and domain checks

```
unsigned char raw{255};
int widened = raw;
double root = value >= 0 ? std::sqrt(value) : 0.0;
```

Byte counts, characters, and unsafe narrowing

- Byte count is not always visible-character count.
- static_cast does not prove a value is in range; validate first.

Lesson map



Characters, code points, and bytes

Unicode assigns code points; UTF-8 encodes them into one to four bytes. A C++ `char` stores one byte-sized code unit, not necessarily one complete user-visible character. `std::string::size()` reports bytes, and a visible grapheme can contain several code points.

Conversion rules

Integral promotions and usual arithmetic conversions choose evaluation types. `static_cast` documents supported conversions but does not guarantee that a value fits. `const_cast`, `dynamic_cast`, and `reinterpret_cast` solve specialized problems and do not replace validation. Narrowing a floating value to an integer discards its fraction.

Parsing text such as `"123"` is not a cast. Use a parser such as `from_chars` or a checked stream, and verify complete consumption and range.

Code points, code units, and visible characters

A Unicode code point is an abstract numbered value such as `U+0627`. An encoding represents it with code units: UTF-8 uses one to four bytes, while UTF-16 uses one or two 16-bit code units. A visible grapheme may combine several code points, so byte count, code-point count, and what a user sees are three different measurements.

UTF-8 preserves ASCII as single-byte values. Leading-bit patterns identify the length of a multibyte sequence, and continuation bytes begin with `10`. Indexing a `std::string` visits bytes, not Unicode characters. Real normalization, case mapping, and grapheme iteration require a Unicode-aware library rather than hand-written byte slicing.

Parsing is not casting

Casting converts an already existing value between C++ types. Parsing interprets text according to a grammar. `static_cast<int>('7')` returns the character code, not the number seven; parsing must subtract `'0'` after validation or use a numeric parser for longer text.

Promotions and the four named casts

Small integer types normally promote to `int` before arithmetic. Mixed arithmetic then selects a common type, so inspect intermediate expressions as well as destination variables.

- `static_cast` expresses ordinary checked-at-compile-time conversions, but it does not perform runtime range validation.
- `dynamic_cast` performs runtime-checked navigation in a polymorphic class hierarchy.
- `const_cast` changes cv-qualification; modifying an object that was originally const is undefined behavior.
- `reinterpret_cast` reinterprets low-level representation and should be isolated behind a carefully reviewed systems-level requirement.

Prefer redesigning the types over scattering casts. Every narrowing conversion needs an explicit domain check.

Check your understanding

01 **How do a Unicode code point and UTF-8 bytes differ?**

> **Explained answer:** A code point is an abstract character identity; UTF-8 encodes it into one or more bytes.

02 **Why may text.size() not equal visible character count?**

> **Explained answer:** Many UTF-8 strings report bytes, and one grapheme can itself contain several code points.

03 **When is narrowing acceptable?**

> **Explained answer:** When range and required precision are checked and the intended loss is documented; a cast alone is not enough.

04 **How should negative user input to sqrt be handled?**

> **Explained answer:** Reject it or use complex arithmetic when required; do not disguise NaN as a valid result.

Connect the ideas

The C++ standard library does not provide complete grapheme processing; use a Unicode library for case folding, normalization, or segmentation. `dynamic_cast` needs a polymorphic base and RTTI, while C-style casts hide several conversion kinds. Inspect integer promotions before arithmetic on small types.

Try it yourself

Test Arabic and emoji text and compare bytes, code points, and graphemes with a suitable library.

7. Operators, expressions, and bitwise operations

Operators build expressions, and conversions determine the type used for evaluation. Learn the rules rather than relying on trial and error.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Boolean:** A logical value with only two states: true or false.
- **Function:** A named, reusable block of code with one defined job.

Expressions and operators

Operators build expressions, and conversions determine the type used for evaluation. Learn the rules rather than relying on trial and error.

Arithmetic, comparisons, logic, and bits

- Integer division discards the fraction; convert an operand to double when needed.
- % computes an integer remainder.
- Relational and logical operators produce bool.
- static_cast documents intent but cannot make a lossy conversion safe.
- Signed overflow is dangerous; inspect limits with numeric_limits.
- Bitwise operators manipulate bits and do not replace && or ||.

Example: an average without integer truncation

```
int total{7};
int count{2};
double average = static_cast<double>(total) / count; // 3.5
bool valid = count > 0 && average >= 0.0;
```

Assignment, precedence, and overflow mistakes

- Do not confuse `=` with `==`.

- Add parentheses when precedence makes intent unclear.

Input, operator groups, and math

Check the stream before using input. Use `getline` for text containing spaces and handle the leftover newline when mixing it with `>>`.

```
double price{};
int quantity{};
if (!(std::cin >> price >> quantity) || price < 0 || quantity < 0) {
    std::cerr << "Invalid input\n";
    return 1;
}
```

Arithmetic, comparison, logical, assignment, increment, bitwise, and conditional operators solve different problems. Short circuiting can guard an unsafe operation. Prefer `static_cast` to document conversions, use `numeric_limits` for type bounds, and use `<cmath>` for `sqrt`, `round`, `ceil`, and `floor`. Name intermediate formula results and normalize units before calculating.

Expressions, precedence, and side effects

Expressions may produce values and change state. Keep multiple mutations out of one dense expression. Precedence groups multiplication before addition, `&&` before `||`, and assignment from right to left. Use parentheses when several operator families make the business rule hard to read.

Prefix increment produces the new value; postfix produces the old value before the increment. Bitwise `&`, `|`, `^`, `~`, `<<`, and `>>` manipulate representations and masks. They differ from logical `&&` and `||`, which produce Boolean results and short-circuit.

Use the conditional operator for one short value choice. Replace deeply nested ternaries with readable branches.

Lesson map

Compound assignment, increments, and short circuiting

`x += y` means update `x` with the sum, while prefix and postfix increment differ in the value produced by the expression. Prefer prefix when the old value is not needed. `&&` and `||` short circuit, so a safety check such as `denominator != 0 && numerator / denominator > 2` prevents invalid division.

Overflow and bit masks

Signed overflow is undefined; unsigned values wrap but may still violate the program's contract. Widen an operand before multiplication. A named unsigned mask can enable a flag with `|`, test it with `&`, remove it with `& ~flag`, and toggle it with `^`. Bitwise operations are not substitutes for Boolean logic.

Complete program: conversion and safe division

```
#include <iostream>
int main() {
    long long total{}; int count{};
    std::cout << "Total and count: ";
    if (!(std::cin >> total >> count) || count <= 0) {
        std::cerr << "Count must be positive\n"; return 1;
    }
    const double average = static_cast<double>(total) / count;
    std::cout << "Average: " << average << '\n'
              << "High average: " << std::boolalpha << (average >= 85.0) << '\n';
}
```

`double average = total / count;` performs integer division before conversion. Test `7 2` and a zero count.

Conversions and narrowing

Integral promotion occurs before many arithmetic operations. Usual arithmetic conversions then choose a common type for both operands. Assignment converts the result afterward, so a wide destination cannot repair overflow that already happened in a narrow intermediate expression.

Brace initialization rejects many narrowing conversions. `static_cast<int>(3.9)` makes truncation explicit but does not prove that an arbitrary value lies inside the target range; check the range first.

Prefix, postfix, and the conditional operator

Prefix increment changes the value and yields the updated value. Postfix yields the old value and then increments. Do not combine several mutations of the same variable in a dense expression; separate statements preserve intent.

The conditional operator selects one of two expressions:

```
const int absolute = value >= 0 ? value : -value;
```

Use it for a small value choice, not as a replacement for a multi-step `if` block.

Bitwise operators and masks

`&`, `|`, `^`, `~`, `<<`, and `>>` operate on bit patterns. A mask can test, set, clear, or toggle one flag. Keep bitwise expressions on unsigned values when shifts are involved, document each bit's meaning, and do not confuse `&` with logical `&&`.

```
constexpr unsigned canRead = 1u << 0;
constexpr unsigned canWrite = 1u << 1;
unsigned permissions = canRead | canWrite;
const bool writable = (permissions & canWrite) != 0;
```

Math formulas and domain checks

`<cmath>` provides `std::sqrt`, `std::pow`, `std::floor`, `std::ceil`, and rounding functions. Translate the mathematical formula with parentheses, verify the function domain, and test intermediate range. For example, square root requires a nonnegative real argument, and an integer formula such as `n * (n + 1) / 2` must widen before multiplication.

Check your understanding

01 **What are 7/2 and 7.0/2, and why?**

> **Explained answer:** The first is integer 3; the second is 3.5 because a double operand promotes floating-point division.

02 **Why is static_cast<int>(largeDouble) not a safety guarantee?**

> **Explained answer:** It documents intent but can discard fractions or receive an out-of-range value; validate before casting.

03 **How do && and & differ for Boolean-looking values?**

> **Explained answer:** && is logical and short-circuits; & is bitwise and evaluates both operands. Their apparent Boolean results can hide different behavior.

04 **How can n*(n+1)/2 avoid intermediate overflow?**

> **Explained answer:** Promote before multiplication, for example 1LL*n, and confirm the wider type supports the maximum input.

05 **Predict `7 / 2` and `7 / 2.0`, then explain the difference.**

> **Explained answer:** The first is 3 because both operands are integers; the second is 3.5 because 2.0 makes the operation floating point.

06 **What is wrong with `long long total = a * b;` when `a` and `b` are int?**

> **Explained answer:** The int multiplication can overflow before assignment. Use `1LL * a * b` and still confirm that long long covers the required range.

Connect the ideas

Operand evaluation order is not always textual order, so do not modify one value several times in an expression. Signed overflow is undefined behavior while unsigned wraps modulo. Shifting negatives or by an out-of-range count is dangerous; use unsigned types and validate the count.

Try it yourself

Test a side-effect-heavy expression and rewrite it as explicit statements.

8. Input, conversions, limits, and math

Input and output are part of program design. Validate extraction, understand cin versus getline, and distinguish build, runtime, and logic failures.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Compiler:** A program that turns source code into a form the computer can run.
- **Runtime:** The period when a program is actually running.
- **Token:** A value representing identity or permission without resending a password.
- **Loop:** A structure that repeats instructions according to a condition.
- **Function:** A named, reusable block of code with one defined job.

Reading input and writing output

Input and output are part of program design. Validate extraction, understand cin versus getline, and distinguish build, runtime, and logic failures.

cin, getline, stream state, and cmath

- cin extracts whitespace-delimited values; getline reads a full line.
- After cin, a newline may remain; std::ws is useful before getline.
- Comments should explain why or constraints, not restate syntax.
- Includes are preprocessed; source becomes objects and then an executable through linking.
- cmath provides sqrt, pow, abs, floor, ceil, and round; respect domain and types.

Example: reading a number and a full line

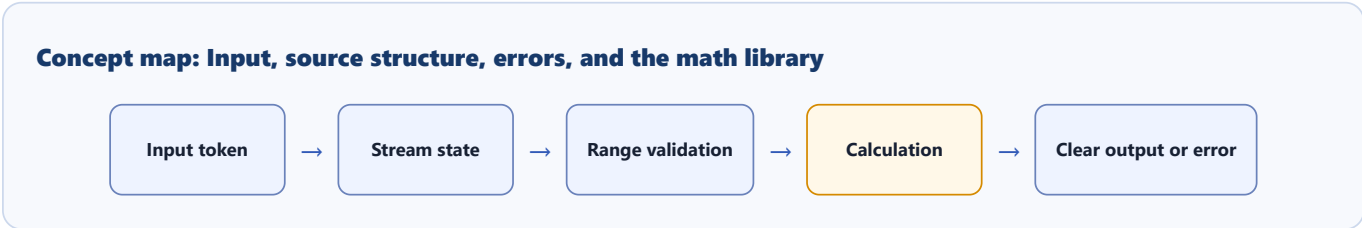
```
#include <iostream>
#include <string>

int main() {
    int age{};
    std::string name;
    if (!(std::cin >> age)) return 1;
    std::getline(std::cin >> std::ws, name);
    std::cout << name << " is " << age << "\n";
}
```

Input, linker, and formula mistakes

- A linker error differs from a syntax error; look for missing definitions or duplicate symbols.
- Real-number sqrt is invalid for negative input.

Lesson map



Stream state and line input

Extraction returns the stream, which becomes false on failure. Call `clear()` before recovery and discard the invalid remainder with `ignore()`. Mixing `>>` with `getline` can leave a newline that produces an empty line, so consume the delimiter deliberately.

Limits, math, and formulas

Integer division drops the fractional part; convert before division. `static_cast` does not prevent overflow. Use `numeric_limits` for implementation-specific bounds. `<cmath>` supplies `sqrt`, `pow`, `abs`, `round`, `ceil`, and `floor`; note that ceiling and floor behave differently for negative values.

Translate formulas by naming quantities and normalizing units. Test zero, invalid negative input, exact boundaries, and values large enough to expose overflow. Keep shared declarations in headers and definitions in source files, and include every direct dependency explicitly.

Recovering a failed stream

```
#include <iostream>
#include <limits>

int main() {
    int value{};
    while (true) {
        std::cout << "Enter an integer: ";
        if (std::cin >> value) break;
        std::cout << "Invalid token; try again.\n";
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }
    std::cout << "Accepted: " << value << '\n';
}
```

`clear()` removes the failure flags; `ignore()` discards the bad line. Omitting either operation can leave the loop reading the same failed state forever.

Output formatting and mathematical domains

Use `<iomanip>` tools such as `fixed`, `setprecision`, and `setw` when the output contract requires them. `sqrt` requires a nonnegative real argument, division requires a nonzero denominator, and floating calculations may produce infinity or NaN. `pow(x, 2)` is often less clear than `x * x`.

Compile, link, runtime, and logic errors

A syntax or type error belongs to compilation. A missing definition appears during linking. Invalid runtime data or resource failure happens while executing. A program that runs but implements the wrong formula has a logic error; the compiler cannot infer the intended business rule.

Converting input and checking type limits

Formatted extraction reports failure through the stream state; it does not guarantee that a successfully parsed value belongs to the business domain. First verify extraction, then compare against `std::numeric_limits<T>` or narrower application limits. Converting text manually with functions such as `std::stoi` also requires handling invalid text and out-of-range values.

Source structure and error categories

Declarations in headers describe interfaces, while definitions usually belong in source files. Include guards or `#pragma once` prevent repeated declarations in one translation unit. Compilers process source files separately, then the linker resolves cross-file definitions. This distinction explains why “declared but undefined” may compile and fail only during linking.

Compile errors violate language rules; linker errors leave unresolved or duplicate symbols; runtime errors occur during execution; logic errors finish but produce incorrect behavior. A useful bug report includes input, expected output, actual output, compiler command, and the first diagnostic.

Rounding and mathematical domains

`std::floor` rounds toward negative infinity, `std::ceil` toward positive infinity, `std::trunc` toward zero, and `std::round` to the nearest integer with halfway cases away from zero. These are not interchangeable for negative inputs. Before `std::sqrt`, check that the value is nonnegative; before division, check the denominator; before converting a floating result to an integer, define the intended rounding policy.

Check your understanding

01 **Why can getline read an empty line after cin >> age?**

> **Explained answer:** The newline remains in the stream; use getline(cin >> std::ws, name) or consume it deliberately.

02 **How do you detect failed numeric extraction?**

> **Explained answer:** Test stream state with if (!(cin >> value)), then handle failure and repair input before continuing.

03 **Classify a missing header, zero division, and a wrong discount formula.**

> **Explained answer:** They are respectively a build error, runtime/invalid operation, and logic error.

04 **Why does sqrt need validation even when code compiles?**

> **Explained answer:** The compiler checks types, not runtime values; a negative real input is outside the function domain and produces NaN.

Connect the ideas

`std::from_chars` separates parsing from locale and reports stop position and error without exceptions. Stream exceptions are optional and need a policy. File input adds open failure, EOF, and partial data, while locale can alter decimal parsing.

Try it yourself

Read the same value through cin, from_chars, and a file and compare failure contracts.

9. if, else, validation, and nesting

if and else direct execution. Write conditions as readable rules and reduce nesting through early validation and grouped logic.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Compiler:** A program that turns source code into a form the computer can run.
- **Boolean:** A logical value with only two states: true or false.

How if chooses an execution path

if and else direct execution. Write conditions as readable rules and reduce nesting through early validation and grouped logic.

if, else, else-if, and short circuiting

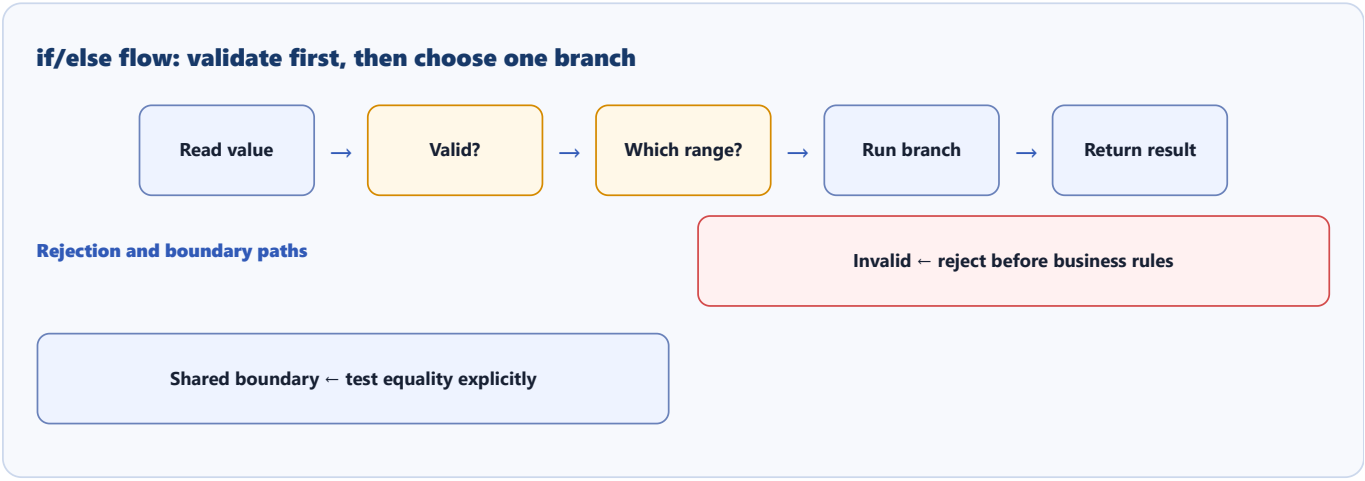
- if accepts a contextually Boolean expression, but explicit predicates are often clearer.
- Braces prevent maintenance bugs when a one-line body grows.
- else binds to the nearest unmatched if; braces remove ambiguity.
- An else-if chain is exclusive; separate if statements may all run.
- Place specific boundary cases before broad cases.
- Short-circuiting can validate a denominator or index before use.

Example: validate and classify a score

```
if (score < 0 || score > 100) {
    std::cout << "Invalid score\n";
} else if (score >= 85) {
    std::cout << "Excellent\n";
} else if (score >= 50) {
    std::cout << "Pass\n";
} else {
    std::cout << "Fail\n";
}
```

Assignment, empty-body, and nesting mistakes

- if (x = 5) assigns instead of comparing.
- A semicolon immediately after if creates an empty body.



From business rule to condition

Write the rule in plain language, identify variables and boundaries, and then translate it into a Boolean expression. “Accept when quantity is positive and stock is sufficient” becomes `quantity > 0 && quantity <= stock`. Test zero, one, exactly the stock, and one above it.

Validate invalid input first, then special cases, then the general rule. Use separate `if` statements when several labels may apply, and an `else if` chain when exactly one outcome must win. Parenthesize mixed `&&` and `||` rules even when precedence would produce the same result.

Decision ordering and short circuiting

Reject invalid data first, then handle special cases, then the general rule. `&&` and `||` short circuit, but parentheses still document grouped business rules. Several independent `if` statements may all run; an `if/else if/else` chain selects only the first true branch.

Nested decisions, guard clauses, and strings

Nesting is useful when the second question has no meaning until the first succeeds. Guard clauses can reject invalid states early and keep the normal path shallow. `std::string` supports `==`, but case and surrounding whitespace are part of the value unless the input contract defines normalization.

De Morgan’s laws

The negation of `(age >= 18 && hasId)` is `(age < 18 || !hasId)`. Use the equivalent form that states the rule most clearly, and name complex Boolean parts instead of building one unreadable expression.

Complete program: validate, then classify

```
#include <iostream>
int main() {
    int score{};
    std::cout << "Score from 0 to 100: ";
    if (!(std::cin >> score)) { std::cerr << "Score must be an integer\n"; return 1; }
    if (score < 0 || score > 100) std::cout << "Invalid score\n";
    else if (score >= 85) std::cout << "Excellent\n";
    else if (score >= 50) std::cout << "Pass\n";
    else std::cout << "Fail\n";
}
```

Test `-1`, `0`, `49`, `50`, `84`, `85`, `100`, `101`, and malformed text. Putting `score >= 50` first makes the excellent branch unreachable.

Boolean context, independent conditions, and exclusive chains

An `if` converts its condition to Boolean. Write comparisons that communicate intent instead of depending on accidental nonzero values. Use separate `if` statements when several outcomes may all apply, such as awarding multiple badges. Use one `if` / `else if` chain when exactly one classification must win.

Order overlapping thresholds from the most specific to the least specific. If `score >= 50` appears before `score >= 85`, an excellent score is captured by the earlier branch.

Compound text input and early validation

When a decision uses a full name or sentence, read it with `std::getline`. If formatted extraction preceded it, consume the leftover newline deliberately. Validate syntax and range before applying business rules, and use guard clauses to keep the valid path shallow.

```
if (!(std::cin >> age)) {
    std::cerr << "Invalid number\n";
    return 1;
}
if (age < 0 || age > 120) {
    std::cerr << "Age outside the accepted range\n";
    return 1;
}
```

Problems that apply conditions and formulas

Complete problem solutions

Each solution below includes a complete program, the governing formula, boundary tests, and complexity. The goal is to derive the program rather than memorize it.

959A Mahmoud and Ehab

Only parity matters. An even `n` prints Mahmoud; an odd `n` prints Ehab.

```
#include <iostream>
int main() { int n{}; std::cin >> n; std::cout << (n % 2 == 0 ? "Mahmoud" : "Ehab") << '\n'; }
```

Trace `1` and `2`. Time and memory are `O(1)`. Simulating turns adds work without information.

486A Calculating Function

Every pair `-1 + 2`, `-3 + 4`, and so on contributes `1`. An odd `n` leaves one final negative term.

```
#include <iostream>
int main() {
    long long n{}; std::cin >> n;
    std::cout << (n % 2 == 0 ? n / 2 : -(n + 1) / 2) << '\n';
}
```

`n=4` gives `2`; `n=5` gives `-3`. Use `long long`. The direct solution is `O(1)`.

4A Watermelon

Two positive even parts require an even weight greater than 2.

```
#include <iostream>
int main() {
    int w{}; std::cin >> w;
    std::cout << (w > 2 && w % 2 == 0 ? "YES" : "NO") << '\n';
}
```

Test `2`, `3`, and `4`. Checking parity alone incorrectly accepts `2`. Complexity is `O(1)`.

835A Key Races

Compute each total once as `s * v + 2 * t`, then compare the two values.

```
#include <iostream>
int main() {
    long long s{}, v1{}, v2{}, t1{}, t2{};
    std::cin >> s >> v1 >> v2 >> t1 >> t2;
    const long long first = s * v1 + 2 * t1;
    const long long second = s * v2 + 2 * t2;
    if (first < second) std::cout << "First\n";
    else if (second < first) std::cout << "Second\n";
    else std::cout << "Friendship\n";
}
```

Test one win for each player and a tie. Complexity is `O(1)`. Do not repeat the formula in every branch.

1173A Nauuo and Votes

`+` is guaranteed only when `x > y + z`; the negative case is symmetric. A guaranteed tie requires `z == 0 && x == y`.

```
#include <iostream>
int main() {
    long long x{}, y{}, z{}; std::cin >> x >> y >> z;
    if (x > y + z) std::cout << "+\n";
    else if (y > x + z) std::cout << "-\n";
    else if (z == 0 && x == y) std::cout << "0\n";
    else std::cout << "?\n";
}
```

Test all four outputs. Complexity is `O(1)`. Comparing only `x` and `y` ignores uncertainty.

318A Even Odds

The odd half contains `(n + 1) / 2` values. Map `k` into that half or subtract its size and map into the even half.

```
#include <iostream>
int main() {
    long long n{}, k{}; std::cin >> n >> k;
    const long long oddCount = (n + 1) / 2;
    std::cout << (k <= oddCount ? 2 * k - 1 : 2 * (k - oddCount)) << '\n';
}
```

For `n=10`, test `k=5` and `k=6`. The formula is `O(1)` and avoids building the sequence.

459A Pashmak and Garden

Handle a vertical side, a horizontal side, a 45-degree diagonal, and the impossible case separately.

```
#include <cstdlib>
#include <iostream>
int main() {
    int x1{}, y1{}, x2{}, y2{}; std::cin >> x1 >> y1 >> x2 >> y2;
    if (x1 == x2 && y1 != y2) {
        const int d = std::abs(y1 - y2);
        std::cout << x1 + d << ' ' << y1 << ' ' << x2 + d << ' ' << y2 << '\n';
    } else if (y1 == y2 && x1 != x2) {
        const int d = std::abs(x1 - x2);
        std::cout << x1 << ' ' << y1 + d << ' ' << x2 << ' ' << y2 + d << '\n';
    } else if (std::abs(x1 - x2) == std::abs(y1 - y2)) {
        std::cout << x1 << ' ' << y2 << ' ' << x2 << ' ' << y1 << '\n';
    } else std::cout << "-1\n";
}
```

Test all four cases, including identical points. Complexity is `O(1)`.

617A Elephant

Ceiling division by 5 gives the minimum number of steps.

```
#include <iostream>
int main() { int x{}; std::cin >> x; std::cout << (x + 4) / 5 << '\n'; }
```

Test `1`, `5`, and `6`. Complexity is `O(1)`.

581A Vasya the Hipster

Different-color days equal `min(a,b)`; the remaining difference supplies same-color pairs.

```
#include <algorithm>
#include <cstdlib>
#include <iostream>
int main() {
    int a{}, b{}; std::cin >> a >> b;
    std::cout << std::min(a, b) << ' ' << std::abs(a - b) / 2 << '\n';
}
```

For `3 7`, the result is `3 2`. Complexity is `O(1)`.

281A Word Capitalization

Change only the first character. Cast through `unsigned char` before calling `<cctype>`.

```
#include <cctype>
#include <iostream>
#include <string>
int main() {
    std::string word; std::cin >> word;
    if (!word.empty()) {
        const auto first = static_cast<unsigned char>(word.front());
        word.front() = static_cast<char>(std::toupper(first));
    }
    std::cout << word << '\n';
}
```

This solves the problem's Latin-character input; it is not general Unicode case conversion. Work after input is `O(1)`.

1A Theatre Square

Apply ceiling division to both dimensions and multiply with `long long`.

```
#include <iostream>
int main() {
    long long n{}, m{}, a{}; std::cin >> n >> m >> a;
    const long long rows = (n + a - 1) / a;
    const long long columns = (m + a - 1) / a;
    std::cout << rows * columns << '\n';
}
```

For `6 6 4`, the answer is `4`. Plain integer division undercounts whenever a remainder exists. Complexity is `O(1)`.

Check your understanding

01 **Why must score>=85 come before score>=50?**

Explained answer: An else-if chain stops at the first true branch; placing >=50 first would swallow excellent scores.

02 **What does if(x=5) do?**

Explained answer: It assigns 5 to x and then treats the nonzero result as true; use `==` and enable compiler warnings.

03 **When are independent if statements better than else-if?**

Explained answer: When multiple outcomes may coexist, such as awarding several badges; else-if is for exclusive results.

04 **How can deeply nested validation be reduced?**

Explained answer: Reject invalid states early with guard clauses so the main valid path remains shallow.

05 **A chain tests `score >= 50` before `score >= 85`. Why can it never award excellent?**

Explained answer: A score of 85 satisfies the first branch and the else-if chain stops. Order thresholds from most specific and highest to lowest.

06 **Correct `if (age = 18)` and name one way to prevent the bug from passing silently.**

Explained answer: Write `if (age == 18)`, enable warnings such as `-Wall -Wextra -Werror`, and test ages 17 and 18.

Connect the ideas

After hand-written cases, add property-style checks for boundaries: output stays in range and a larger input cannot move to a lower class without an explicit rule. Use compiler warnings to expose constant conditions and unreachable branches.

Try it yourself

Generate values around every boundary and prove no unintended gap or overlap exists.

10. switch, control flow, and a calculator

switch selects a branch from one discrete value. Use it for clear constant cases; prefer if for ranges and compound predicates.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Runtime:** The period when a program is actually running.
- **Scope:** The region of code in which a name or variable is visible.

Selection by one discrete value

switch selects a branch from one discrete value. Use it for clear constant cases; prefer if for ranges and compound predicates.

selector, case, break, and default

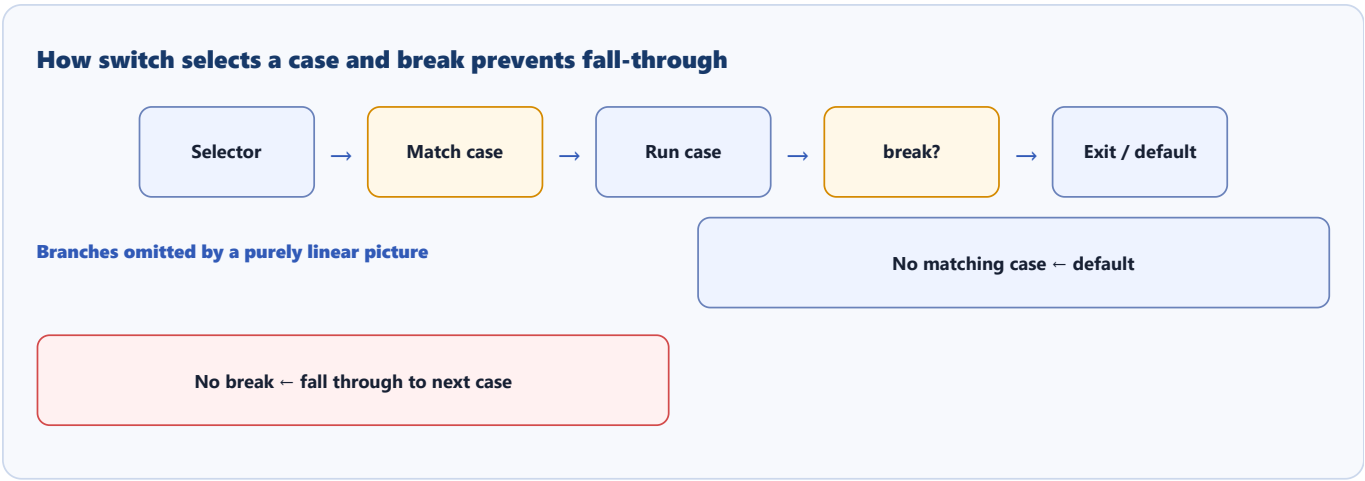
- Case labels must be unique compile-time constants compatible with the selector.
- break prevents execution from falling into the next case.
- default handles unmatched values.
- Grouped cases are useful when several values share behavior.
- Intentional fall-through should be explicit, optionally with [[fallthrough]].
- Use braces around a case body when local declarations need scope.

Example: a command menu

```
switch (choice) {
case 1: runReport(); break;
case 2: saveFile(); break;
case 0: std::cout << "Bye\n"; break;
default: std::cout << "Unknown option\n";
}
```

Range and fall-through mistakes

- `switch` cannot directly express `score >= 50`.
- A missing break may execute multiple cases.



Selector limits and a safe calculator

The selector may use integral types, `char`, or an enumeration. It cannot directly use `double` or `std::string`, and each `case` must be a distinct constant expression. Group labels that share behavior, mark intentional continuation with `[[fallthrough]]`, and use braces around case-local variables.

In a calculator, `switch` selects `+`, `-`, `*`, or `/`, while a nested `if` rejects a zero divisor. This illustrates the rule: use `switch` for one discrete selector and `if` for ranges or compound predicates.

How switch executes a decision

The program evaluates the `switch` expression once, converts the result when required, and jumps to the matching label. A `case` is an entry point, not a predicate that gets checked again after execution has entered the statement.

```
char action{'P'};

switch (action) {
case 'R':
    std::cout << "Registration\n";
    break;
case 'P':
    std::cout << "Payment\n";
    break;
case 'S':
    std::cout << "Status\n";
    break;
default:
    std::cout << "Unknown action\n";
}
```

The selector is `'P'`, so execution starts in the payment case and `break` transfers control past the `switch`.

Selector types

Type	Directly valid?	Notes
`char` and integral types	Yes	Common for command characters and menu numbers
`bool`	Technically yes	An `if` is usually clearer for two outcomes
`enum` and `enum class`	Yes	Useful for named application states after input parsing
`float` and `double`	No	Floating-point values are not switch selectors
`std::string`	No	Use `if`, a lookup table, or parse the text into an enum

A class with a suitable conversion may also qualify, but an implicit conversion can hide the meaning of the decision. Prefer an explicit selector type.

Case labels are unique constant expressions

Each label must be an integral constant expression available at compile time and must remain unique after conversion to the selector type.

```
constexpr int registerCode = 1;
constexpr int reportCode = 1 + 1;

int choice{};
std::cin >> choice;

switch (choice) {
case registerCode:
    std::cout << "Register\n";
    break;
case reportCode:
    std::cout << "Report\n";
    break;
}
```

These examples are invalid:

```
// case choice:       // runtime value
// case choice > 2:   // predicate, not a discrete value
// case 2:            // duplicate when reportCode is already 2
```

`constexpr` states the compile-time intention directly. A `const int` may satisfy the constant-expression rules in some contexts, but `constexpr` is clearer here.

Trace fall-through precisely

```
int choice{1};

switch (choice) {
case 1:
    std::cout << "One\n";
case 2:
    std::cout << "Two\n";
    break;
default:
    std::cout << "Other\n";
}
```

The program prints `One`, continues forward because no `break` follows it, then prints `Two`. It does not compare the selector with `2` again. Use `[[fallthrough]];` in C++17 or later when continuation between nonempty case bodies is intentional. Consecutive empty labels such as `'Y'` and `'y'` simply group several values onto one body.

default and unmatched input

`default` is optional. Without it, an unmatched selector executes no case body. It may appear in any position, but placing it last is easier to read. A `default` in the middle can fall through into a later case if control reaches it and no statement stops execution.

Complete calculator with input validation

```
#include <iostream>

int main() {
    double left{}, right{};
    char operation{};

    std::cout << "Enter: number operator number: ";
    if (!(std::cin >> left >> operation >> right)) {
        std::cerr << "Invalid input\n";
        return 1;
    }

    switch (operation) {
    case '+':
        std::cout << "Result: " << left + right << '\n';
        break;
    case '-':
        std::cout << "Result: " << left - right << '\n';
        break;
    case '*':
        std::cout << "Result: " << left * right << '\n';
        break;
    case '/':
        if (right == 0.0) {
            std::cerr << "Cannot divide by zero\n";
            return 1;
        }
        std::cout << "Result: " << left / right << '\n';
        break;
    default:
        std::cerr << "Unsupported operator\n";
        return 1;
    }
}
```

Test `8 / 2`, `8 / 0`, `5 * 3`, and `5 ? 3`. These cover successful division, the zero-divisor rule, a normal operation, and `default`. Stream validation must happen before the `switch`; control flow cannot repair a failed extraction.

Check your understanding

01 **Why can switch not directly express score>=50?**

> **Explained answer:** Case labels match discrete constants, not ranges or comparison expressions; use if/else for ranges.

02 **What happens when break is omitted?**

> **Explained answer:** Execution continues through later cases until a break or the switch ends; this may be a bug or documented fall-through.

03 **When should several cases be grouped?**

> **Explained answer:** When multiple selector values intentionally share the same behavior.

04 **Why add braces around a case with local variables?**

> **Explained answer:** They create a clear scope and avoid jumping across a variable initialization between labels.

05 **After entering case 1 without a break, does the program compare the selector with case 2 again?**

> **Explained answer:** No. Once execution jumps to a matching label, it continues forward until a break, return, or the end of the switch.

06 **Why does the calculator use switch for the operation but if inside division?**

> **Explained answer:** The operator is one discrete character, while rejecting a zero divisor is a runtime predicate, so each construct matches a different kind of decision.

Connect the ideas

C++17 permits an init-statement before the selector to narrow scope. With enum class, enable warnings for unhandled cases and decide whether default is appropriate: omitting it can expose new enum values, while external input still needs validation and fallback.

Try it yourself

Add an enum value and confirm the build or a test exposes an incomplete switch.

11. Loops and repetition patterns

A loop has initial state, a continuation condition, and an update. Choose while, for, or do-while according to the repetition model.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Compiler:** A program that turns source code into a form the computer can run.
- **Token:** A value representing identity or permission without resending a password.
- **Loop:** A structure that repeats instructions according to a condition.

The parts of a loop

A loop has initial state, a continuation condition, and an update. Choose while, for, or do-while according to the repetition model.

for, while, do-while, counters, and sentinels

- for fits a known counter progression.
- while fits termination driven by evolving state.
- do-while runs at least once and can support interactive validation.
- A counter counts, an accumulator combines, and a sentinel ends unknown-length input.
- break exits the nearest loop; continue skips the rest of one iteration.
- Nested loops often multiply iteration counts; analyze complexity.

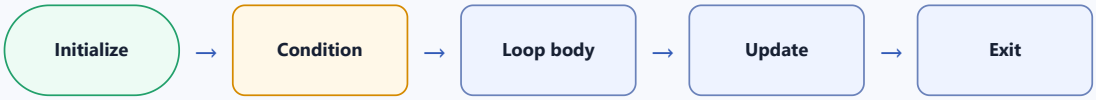
Example: summing even numbers

```
long long sum{};
for (int i = 1; i <= n; ++i) {
    if (i % 2 != 0) continue;
    sum += i;
}
```

Infinite loops and off-by-one errors

- Every path should move toward termination.
- Updating the counter in the wrong place causes off-by-one errors.

The loop cycle and its initialization, bound, and update points



Trace and repetition patterns

Before coding, define the start, continuation condition, and step. Trace the control variable and state after every iteration. The update must move the state toward termination.

A counter starts at zero and increments on a match. An accumulator starts with a neutral value. A sentinel ends an unknown-length sequence but is not part of the data. A validation loop repeats until input is valid. Nested loops often multiply iteration counts. You do not need an array when each value can update the final summary immediately.

Choose the loop from the repetition model

Use `for` when initialization, condition, and update form one visible counter progression. Use `while` when the next iteration depends on input or evolving state rather than a fixed count. Use `do-while` only when one execution is genuinely required before testing the condition, such as displaying a menu for the first time.

```
int value{};
long long sum{};
int count{};

while (std::cin >> value && value != -1) {
    sum += value;
    ++count;
}

if (count != 0) {
    std::cout << static_cast<double>(sum) / count << '\n';
}
```

The sentinel `-1` ends the sequence and is not included. The input operation is part of the condition, so an end-of-file or malformed token also stops safely.

Common patterns

- **Counting:** initialize to zero and increment only when a predicate is true.
- **Summation:** start from zero; for a product, start from one.
- **Minimum or maximum:** initialize from the first real value or use an explicit “no value yet” state.

- **Validation:** repeat input until both extraction and the domain rule succeed.
- **Digit processing:** repeatedly use `% 10` to read the last digit and `/= 10` to remove it.
- **Nested patterns:** let the outer loop represent rows and the inner loop represent columns.

`break` exits the nearest loop; it should represent a clear event such as finding the requested value. `continue` skips the remaining statements in the current iteration. In a `while` loop, be careful that `continue` does not bypass the state update and accidentally create an infinite loop.

Trace invariants and complexity

An invariant is a fact that remains true before and after each iteration. For a running sum, after processing `i` values, `sum` must equal the total of exactly those `i` values. Stating this fact makes initialization, updates, and final output easier to verify.

One loop from `0` to `n-1` is usually `O(n)`. Two independent nested loops of size `n` are usually `O(n²)`, but count the actual work rather than guessing from indentation. If an inner pointer only moves forward across the entire program, total work may still be linear.

The loop model: start, condition, body, and update

Before choosing `while` or `for`, answer four questions:

1. What is the initial state?
2. What condition permits another iteration?
3. What work belongs in the repeated body?
4. What update moves the state toward termination?

One execution of the body is an iteration. A pre-test loop checks its condition before every possible iteration, so a false initial condition produces zero executions.

Trace a while loop by hand

```
int i = 1;

while (i <= 5) {
    std::cout << i << '\n';
    ++i;
}
```

Value before iteration	Result of `i <= 5`	Output	Value after `++i`
1	true	1	2
2	true	2	3
3	true	3	4
4	true	4	5
5	true	5	6
6	false	none	no update

This table exposes boundary errors before execution. With `i < 5`, the last output is 4. Without `++i`, the state never changes and the program prints 1 indefinitely.

Counting up, counting down, and changing the step

```
// 0 through 9: exactly ten iterations
int i = 0;
while (i < 10) {
    std::cout << i << ' ';
    ++i;
}

// 10 through 1
int countdown = 10;
while (countdown >= 1) {
    std::cout << countdown << ' ';
    --countdown;
}

// Even numbers from 0 through 20
int even = 0;
while (even <= 20) {
    std::cout << even << ' ';
    even += 2;
}
```

The update must move toward the stopping boundary. Incrementing `countdown` while the condition is `countdown >= 1` moves away from termination and may eventually cause signed overflow.

Multiplicative updates

```
int value = 1;
while (value <= 64) {
    std::cout << value << ' ';
    value *= 2;
}
// 1 2 4 8 16 32 64
```

```
int value{};
std::cin >> value;

while (value > 0) {
    std::cout << value << ' ';
    value /= 2;
}
```

For input `100`, the second trace is `100, 50, 25, 12, 6, 3, 1`. Integer division discards fractions, and the next update produces zero. Repeated halving takes roughly $\log_2(value)$ iterations. Repeated multiplication requires overflow analysis because wrapping or undefined behavior may prevent the expected termination.

The stray semicolon after while

```
int i = 1;
while (i <= 5);  // empty loop body
{
    std::cout << i;
    ++i;
}
```

The semicolon is the complete empty body. The following block is not part of the loop, and `i` cannot change while the condition remains true. Compiler warnings help catch this mistake.

Sentinel-controlled input

```
int number{};
std::cin >> number;

while (number != 0) {
    std::cout << "Accepted: " << number << '\n';
    std::cin >> number;
}
```

Zero ends the sequence and is not data. The loop needs a priming read before the condition and another read in the body. Omitting the second read leaves the condition unchanged. A safer form also accounts for stream failure:

```
while (std::cin >> number && number != 0) {
    std::cout << "Accepted: " << number << '\n';
}
```

do-while for required first execution

```
int number{};

do {
    std::cout << "Enter a positive number: ";
    std::cin >> number;
} while (number <= 0);
```

The first input happens before any condition check. The semicolon after `while (condition);` is required in this syntax, unlike the accidental semicolon after a normal `while` header.

A menu that must appear once

```
int choice{};

do {
    std::cout << "1. Play\n2. Settings\n0. Exit\n";
    std::cin >> choice;

    if (choice == 1) {
        std::cout << "Playing\n";
    } else if (choice == 2) {
        std::cout << "Settings\n";
    } else if (choice != 0) {
        std::cout << "Unknown choice\n";
    }
} while (choice != 0);
```

A real program must also handle stream failure so that invalid text does not leave the menu using stale state forever.

The exact execution order of for

```
for (int i = 1; i <= 5; ++i) {
    std::cout << i << '\n';
}
```

Initialization runs once. The condition runs before every possible iteration. When it is true, the body runs, followed by the update. Control then returns to the condition; initialization does not run again.

The equivalent `while` form is:

```
int i = 1;
while (i <= 5) {
    std::cout << i << '\n';
    ++i;
}
```

For `n` zero-based elements, `i < n` visits `0` through `n - 1`. With `n = 5`, these are `0, 1, 2, 3, 4`. Using `i <= n` creates six iterations and later causes an out-of-bounds access when used as an index.

Counters, accumulators, and averages

A counter answers “how many?” while an accumulator combines values.

```
int evenCount = 0;

for (int i = 1; i <= 10; ++i) {
    if (i % 2 == 0) {
        ++evenCount;
    }
}

std::cout << evenCount; // 5
```

```
int n{};
std::cin >> n;

long long sum = 0;
for (int i = 0; i < n; ++i) {
    int number{};
    std::cin >> number;
    sum += number;
}

if (n > 0) {
    double average = static_cast<double>(sum) / n;
    std::cout << "Sum: " << sum << "\nAverage: " << average;
}
```

Zero is the additive identity. The cast happens before division so the fractional part survives. A `long long` sum can be necessary even when each individual input fits in `int`.

Product accumulation

```
long long product = 1;

for (int i = 0; i < 5; ++i) {
    int number{};
    std::cin >> number;
    product *= number;
}

std::cout << product;
```

One is the multiplicative identity. Starting from zero makes every product zero. Product growth is fast, so the wider type still needs a justified input range.

Classify values without storing them

```
int positive = 0;
int negative = 0;
int even = 0;
int odd = 0;

for (int i = 0; i < n; ++i) {
    int number{};
    std::cin >> number;

    if (number > 0) {
        ++positive;
    } else if (number < 0) {
        ++negative;
    }

    if (number % 2 == 0) {
        ++even;
    } else {
        ++odd;
    }
}
```

Zero is neither positive nor negative, but it is even. No array is needed because each summary updates as its value arrives. Store the sequence only if later work needs the original elements.

Maximum without assuming zero

```
int maximum{};

if (!(std::cin >> maximum)) {
    return 1;
}

for (int i = 1; i < 5; ++i) {
    int number{};
    std::cin >> number;

    if (number > maximum) {
        maximum = number;
    }
}
```

Initializing from the first real input handles an all-negative sequence. `maximum = 0` would incorrectly report a value that never appeared. If an empty sequence is possible, represent the “no value yet” state explicitly.

Search flags and early exit

```
bool found = false;

for (int i = 0; i < 10; ++i) {
    int number{};
    std::cin >> number;

    if (number == 7) {
        found = true;
    }
}
```

The flag remains true after the first match. Add `break` when the first match is sufficient and leaving unread input does not damage later parsing.

break and continue traces

```
for (int i = 1; i <= 10; ++i) {
    if (i == 5) break;
    std::cout << i << ' ';
}
// 1 2 3 4
```

`break` exits the nearest loop immediately. By contrast:

```
for (int i = 1; i <= 10; ++i) {
    if (i == 5) continue;
    std::cout << i << ' ';
}
// 1 2 3 4 6 7 8 9 10
```

`continue` skips only the rest of the current iteration. In a `for` loop control goes to the update. In a `while` loop it returns directly to the condition, which can accidentally bypass a state update placed at the end of the body.

Nested loops as rows and columns

```
for (int row = 1; row <= 3; ++row) {
    for (int column = 1; column <= 4; ++column) {
        std::cout << "* ";
    }
    std::cout << '\n';
}
```

The inner loop completes four columns for each of three rows, so the print statement runs 12 times. The `while` form must reset its column inside every outer iteration:

```
int row = 1;

while (row <= 3) {
    int column = 1;

    while (column <= 4) {
        std::cout << "* ";
        ++column;
    }

    std::cout << '\n';
    ++row;
}
```

If `column` is initialized only once outside the outer loop, it reaches 5 after the first row and later rows print no stars. Different loop types may be nested when each one models a different role.

Interactive menu with stream recovery

```
#include <iostream>
#include <sstream>
#include <string>

int main() {
    std::string line;
    while (true) {
        std::cout << "1. Continue\n0. Exit\nChoice: ";
        if (!std::getline(std::cin, line)) {
            if (std::cin.eof()) return 0;
            std::cerr << "Input read failed\n";
            return 1;
        }
        std::istringstream input(line);
        int choice{};
        char extra{};
        if (!(input >> choice) || (input >> extra)) {
            std::cout << "Enter one integer only.\n";
            continue;
        }
        if (choice == 0) return 0;
        if (choice == 1) std::cout << "Continuing\n";
        else std::cout << "Unknown choice\n";
    }
}
```

`clear()` restores the stream state and `ignore()` removes the bad line. Omitting either step can trap the menu in a failed-input loop.

Check your understanding

01 **When is do-while preferable to while?**

> **Explained answer:** When the body must execute once before continuation is tested, such as showing a menu or reading a first attempt.

02 **What commonly causes an off-by-one bug?**

Explained answer: Ambiguous inclusive bounds, starting value, or update position; write the intended first and last iterations before coding.

03 **How do you average values terminated by -1?**

Explained answer: Accumulate and count values without including -1, then divide only after confirming count is nonzero.

04 **What is the complexity of two n-sized nested loops?**

Explained answer: Usually $O(n^2)$, but actual bounds matter; shrinking or globally advancing inner work can change the result.

05 **What does repeated `i /= 2` print from 100, and why do fractions disappear?**

Explained answer: It prints 100, 50, 25, 12, 6, 3, 1. Integer division discards the fractional part; the update after 1 produces 0 and stops the loop.

06 **Why does a product start at 1, and why should a maximum not always start at 0?**

Explained answer: One is the multiplicative identity, whereas zero destroys a product. A maximum should start from real data because every input may be negative.

07 **How do break and continue differ, and what is continue's special risk in while?**

Explained answer: break exits the nearest loop; continue skips one iteration's remaining body. In while, continue may bypass the state update and create an infinite loop.

08 **Why must column be reset inside each outer row iteration?**

Explained answer: The first row advances column past its limit. Reinitializing inside the outer loop gives every row a fresh complete column traversal.

Connect the ideas

This lesson is extremely dense; study it in three units: loop construction/tracing, counters/accumulators/search, then nested loops/complexity. A range-based for visits elements directly. Resizing a container while iterating may invalidate the iterator, the position used to traverse it; see the container lesson. Test each pattern before advancing.

Try it yourself

Create three independent practice files instead of combining all examples into one program.

Test the menu's input contract

The full program uses a line parser so `abc`, an empty line, `1x`, and an out-of-range integer are rejected as whole inputs. Whitespace around one integer is accepted. Test the lines `abc`, `1`, `0`: an error is followed by `Continuing`, then exit. End input immediately, and again after `1`; both must exit without spinning. In the earlier do-while fragment, input is assumed valid. `continue` in do-while still checks the condition, so a failed extraction that leaves zero can accidentally exit. Use the complete loop when recovering from invalid input.

Save as `menu.cpp` and compile with `g++ -std=c++17 -Wall -Wextra -pedantic menu.cpp -o menu`. Run `./menu` on Linux/macOS or `.\menu.exe` in PowerShell.

12. Input and business rules

Business rules become clear conditions when each fact is named and inputs are validated. Separate data validity from the acceptance decision.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Boolean:** A logical value with only two states: true or false.
- **Function:** A named, reusable block of code with one defined job.

From requirements to conditions

Business rules become clear conditions when each fact is named and inputs are validated. Separate data validity from the acceptance decision.

Data validity and business decisions

- Start with a table of inputs, types, and valid ranges.
- Name predicates: hasPassingGrade is clearer than a repeated expression.
- Mandatory rules combine with &&; acceptable alternatives use ||.
- Report the specific rejection reason.
- Test just below, exactly at, and just above every threshold.

Example: student eligibility

```
bool validScore = score >= 0 && score <= 100;
bool eligible = validScore && score >= 60 && attendance >= 75;
```

Threshold and invalid-data mistakes

- Do not allow invalid data into business decisions.
- Confirm whether each threshold is inclusive.

Turn prose into an executable rule

Do not start by typing a long `if`. First rewrite the requirement as facts that can be checked independently. For a course application, those facts might be: the score is within `0..100`, attendance is within `0..100`, the student passed the score threshold, and attendance satisfies the policy. This creates two layers:

1. **Data validation:** can these values represent a valid score and attendance percentage?
2. **Business decision:** does a valid applicant satisfy the current acceptance policy?

```
bool scoreIsValid = score >= 0 && score <= 100;
bool attendanceIsValid = attendance >= 0 && attendance <= 100;

if (!scoreIsValid || !attendanceIsValid) {
    std::cout << "Invalid input\n";
} else {
    bool passedScore = score >= 60;
    bool passedAttendance = attendance >= 75;
    bool accepted = passedScore && passedAttendance;
    std::cout << (accepted ? "Accepted\n" : "Rejected\n");
}
```

Named Boolean variables make the program read like the original requirement. They also make later changes safer: if attendance becomes optional for a special track, the policy can change without weakening the input validation.

Truth tables and decision tables

When two or more facts interact, list the combinations before writing code. For `passedScore && passedAttendance`, only the `true/true` row accepts. If a scholarship instead requires a high score **or** a competition award, `||` is appropriate. A decision table becomes even more valuable when there are exceptions, because it exposes overlapping or missing cases before they turn into deeply nested conditions.

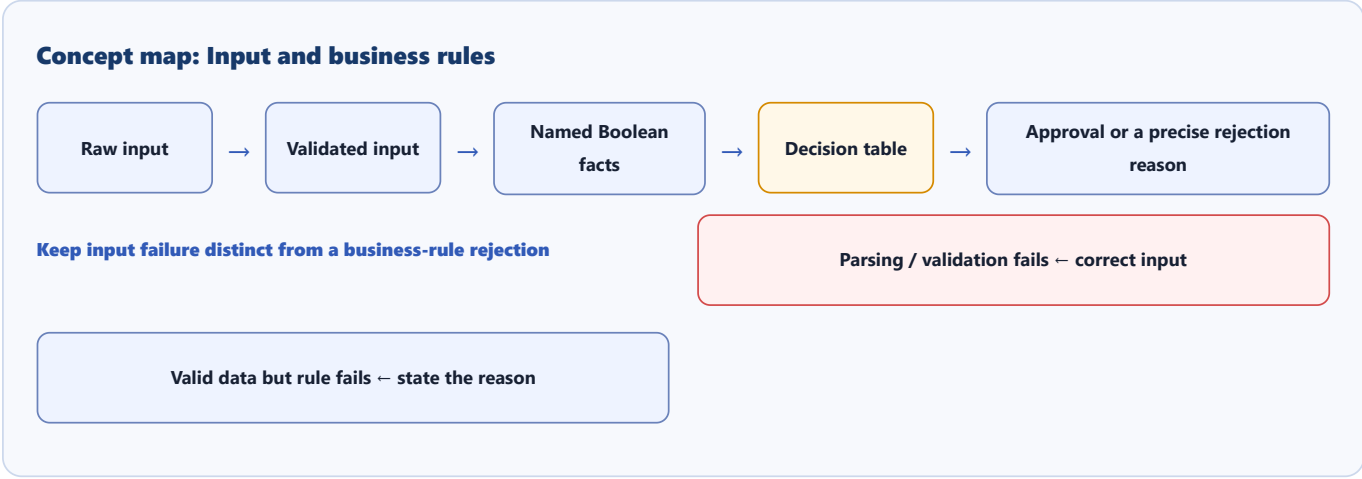
Keep precedence visible when operators are mixed:

```
bool accepted = validData &&
                (regularRequirements || hasApprovedException);
```

Boundary-focused testing

For each rule, identify the first valid value and test one value below it, the boundary itself, and one value above it. For `score >= 60`, use `59`, `60`, and `61`. Also test impossible values such as `-1` and `101`. This method catches the most common mistakes: choosing `>` instead of `>=`, applying a decision to invalid input, or combining rules with the wrong logical operator.

Lesson map



Complete program: valid data before eligibility

```
#include <iostream>
int main() {
    int score{}, attendance{}; char paidAnswer{};
    std::cout << "Score attendance paid(y/n): ";
    if (!(std::cin >> score >> attendance >> paidAnswer)) {
        std::cerr << "Invalid input format\n"; return 1;
    }
    if (score < 0 || score > 100 || attendance < 0 || attendance > 100 ||
        (paidAnswer != 'y' && paidAnswer != 'Y' && paidAnswer != 'n' && paidAnswer != 'N')) {
        std::cerr << "Input outside the accepted domain\n"; return 1;
    }
    const bool paid = paidAnswer == 'y' || paidAnswer == 'Y';
    const bool eligible = score >= 60 && attendance >= 75 && paid;
    std::cout << (eligible ? "Eligible\n" : "Not eligible\n");
}
```

Test `59/60`, `74/75`, both payment answers, and out-of-range input. Validation must precede the eligibility decision.

Check your understanding

01 **Why separate validScore from eligible?**

> **Explained answer:** The first validates data; the second applies policy. Separation distinguishes malformed input from a normal rejection.

02 **How should a 75% attendance threshold be tested?**

> **Explained answer:** Test just below, exactly 75, above, and invalid values below zero and above 100.

03 **When do eligibility rules use && versus ||?**

> **Explained answer:** Use && for jointly required facts and || for alternative routes; add parentheses when mixing them.

04 **Why is “not eligible” alone a weak message?**

> **Explained answer:** It does not identify the failed rule for users, tests, or support; return a specific but non-sensitive reason.

05 **Design the smallest useful test set for age 18–60 inclusive and attendance at least 75%.**

> **Explained answer:** Test ages 17, 18, 60, and 61 with valid attendance; then 74.99 and 75 with a valid age; finally include non-numeric input.

06 **How can a decision table expose conflicting rules before coding?**

> **Explained answer:** It lists combinations of facts and their outcomes. A row with two outcomes, or no outcome, reveals ambiguity before it becomes an if chain.

Connect the ideas

In a larger application, parsing converts text into trusted types, validation enforces shape, and a domain service decides eligibility. Do not duplicate Boolean facts across controllers, UI, and storage; centralize the decision and test it with a case table.

Try it yourself

Extract the decision into a pure function and prove two entry points use the same contract.

Archived reference: problem-solving practice

An archived copy of the practice material now integrated into the problem-solving lesson.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Compiler:** A program that turns source code into a form the computer can run.
- **Runtime:** The period when a program is actually running.

Competitive programming

Competitive programming trains constraint analysis, algorithm choice, and fast correct implementation. The judge is a measurement tool, not a substitute for understanding.

Statements, constraints, and complexity

- Read input, output, and constraints before samples.
- Translate constraints into acceptable complexity; n=10^5 usually rules out $O(n^2)$.
- Solve samples by hand, then add custom cases.
- After Wrong Answer, inspect boundaries, types, rounding, and interpretation.
- State time and memory complexity after solving.

Example: Elephant and ceiling division

```
// Elephant: minimum moves of length at most 5
int moves = (distance + 4) / 5; // integer ceiling
```

Complete runnable submission

```cpp
#include <iostream>

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);

    int distance{};
    if (!(std::cin >> distance) || distance < 0) {
        return 1;
    }

    const int moves = (distance + 4) / 5;
    std::cout << moves << '\n';
}
```

For input `12`, the output is `3`: two moves of length 5 and a final move of length 2. Time and auxiliary memory are both `O(1)`. The program prints no prompt such as `Enter distance` because an online judge compares output exactly. The validation helps during local use, although the original problem normally guarantees valid input.

The Online Judge workflow

The judge compiles submitted source code with a documented compiler and options, runs the executable against hidden tests under time and memory limits, and compares actual output with expected output through a checker. `Compilation Error` means the source did not build. `Wrong Answer` means it ran but produced incorrect output. `Runtime Error` covers failures such as invalid access or division by zero. `Time Limit Exceeded` and `Memory Limit Exceeded` identify exhausted resource limits.

Samples, submissions, and random changes

- Passing samples does not prove correctness.
- Do not start with maximum difficulty; build a consistent practice ladder.

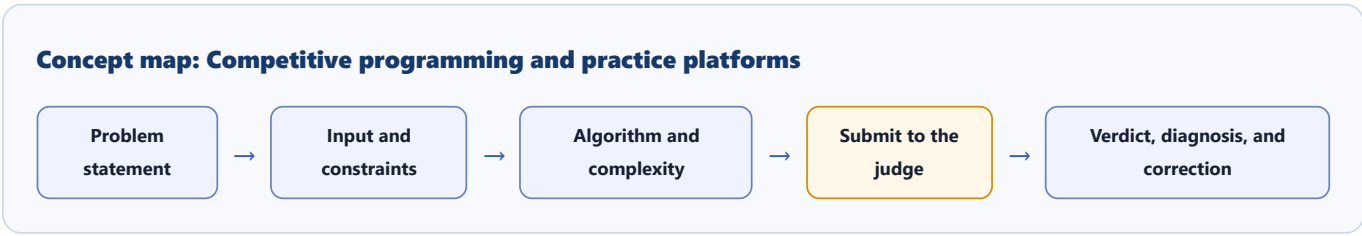
Platforms and problem patterns

Codeforces emphasizes contests, ratings, and broad problem sets. LeetCode often emphasizes interview-oriented data-structure patterns. Begin with implementation, arithmetic, and conditions, then add loops and containers. After each problem record the idea, the bug, complexity, and a missed test case.

The practice set covers: ceiling division in 617A and 1A, `min` and remainder in 581A, character handling in 281A, parity in 4A and 959A, formula derivation in 486A, three-way comparison in 835A, guaranteed outcomes in 1173A, odd/even block mapping in 318A, and geometric case analysis in 459A.

Read an editorial only after a genuine attempt, then close it and derive the method again on a different example.

Lesson map



Check your understanding

01 **Why do constraints select the algorithm before coding?**

Explained answer: For n=10^5, $O(n^2)$ may be impossible regardless of code quality, while O(n log n) or O(n) can fit.

02 **What does Wrong Answer after passing samples imply?**

Explained answer: Samples are not proof; revisit interpretation, boundaries, types, rounding, and construct counterexamples.

03 **Why does (d+4)/5 solve Elephant?**

Explained answer: It performs integer ceiling by five: adding four turns any nonzero remainder into one additional move.

04 **How do you learn after an accepted solution?**

Explained answer: Record complexity, study an alternative, reimplement later, and document the misconception that blocked the first attempt.

Connect the ideas

Use a small template that does not hide types or errors, and enable fast I/O only when constraints justify it. Separate local diagnostics from submitted output and generate small cases comparing the fast solution with brute force. Track defect types and time-to-solution, not only problem count.

Try it yourself

Write a stress test that generates small inputs and stops at the first mismatch.

Functions, containers, and references

Split programs into clear functions, choose array, vector, or string for the data shape, and pass values safely by value or reference.

This lesson follows conditions and loops. Short snippets demonstrate syntax and belong inside a program with the needed headers; the final example below is a complete C++17 program. A reference is another name for an existing object; a container holds a collection of values.

Functions give work a name

As a program grows, its steps should not remain inside `main`. A function names one task, defines its inputs and result, and can be tested and reused independently.

```
double finalPrice(double subtotal, double discountRate)
{
    return subtotal * (1.0 - discountRate);
}
```

A declaration introduces the name and types. A definition supplies the body. The compiler needs a declaration before the first call, while the linker finds the final definition across object files.

Value, reference, and const

Passing by value copies an argument. `T&` permits modification of the original. `const T&` avoids a potentially expensive copy while preventing modification.

```
void addTax(double& total, double rate) { total *= 1.0 + rate; }
double average(const std::vector<int>& values);
```

Use values for small types and independent inputs, `const&` for read-only access to a large object, and `&` only when mutation is part of the function's contract. Never return a reference to a local variable whose lifetime ends at return.

Scope and lifetime

A local variable lives until its block ends. A local `static` retains its value across calls. Scope answers where a name is visible; lifetime answers when the object exists.

array, vector, and string

Type	Size	Use
`std::array<T, N>`	Fixed at compile time	A fixed element count with a container interface
`std::vector<T>`	Changes at runtime	The default general-purpose sequence
`std::string`	Dynamic text	Text modification, search, and composition

```
std::vector<int> scores{75, 91, 60};
scores.push_back(84);

int total = 0;
for (int score : scores) total += score;
double avg = scores.empty() ? 0.0
                            : static_cast<double>(total) / scores.size();
```

Indexes begin at zero. `operator[]` does not check bounds; `at()` reports an invalid index. Vector growth may reallocate storage and invalidate older pointers and references to its elements.

Files and interfaces

Put shared declarations in a header protected by `#pragma once` or header guards, and definitions in `.cpp` files. Avoid `using namespace std;` in a header because it affects every file that includes it.

Check your understanding

01 **When is const T& preferable to T?**

> **Answer:** When reading a large object without copying or modifying it. Small types such as int are usually clearer by value.

02 **Why can push_back invalidate a reference to a vector element?**

> **Answer:** The vector may move all elements to a larger allocation, leaving the reference pointing at old storage.

03 **How do you make average testable without cin or cout?**

> **Answer:** Accept the container and return the result. Keep input and presentation in separate functions.

Complete program and boundary checks

```
#include <iostream>
#include <stdexcept>
#include <vector>

double average(const std::vector<int>& scores) {
    if (scores.empty()) throw std::invalid_argument("No scores");
    double total = 0;
    for (int score : scores) {
        if (score < 0 || score > 100) throw std::invalid_argument("Score outside 0..100");
        total += score;
    }
    return total / static_cast<double>(scores.size());
}
int main() {
    const std::vector<int> scores{75, 90, 60};
    std::cout << average(scores) << '\n';
    try { std::cout << average({}) << '\n'; }
    catch (const std::invalid_argument& error) { std::cout << error.what() << '\n'; }
}
```

Save as `scores.cpp`, compile with `g++ -std=c++17 -Wall -Wextra -pedantic scores.cpp -o scores`, and run it. The output is `75` then `No scores`. `const&` avoids copying the vector and prevents this function from changing it. The double accumulator is acceptable for bounded approximate grades; it is not a money representation. Empty input is rejected rather than silently inventing an average of zero. Try `{0}`, `{100}`, and `{101}`: the first two return their sole score; the last throws. `average({})` creates a temporary vector that lives through the call, but returning a reference to a local vector would dangle after return.

Archived reference: condition and formula problems

An archived copy of the problems now integrated into the if and else lesson.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Unicode:** A standard that assigns consistent numbers to characters and symbols.
- **Loop:** A structure that repeats instructions according to a condition.
- **Function:** A named, reusable block of code with one defined job.

From a problem statement to cases and formulas

Conditional problems train translation from story text into equations and exclusive cases. The main difficulty is usually boundaries, not typing if.

Boundaries, exclusive cases, and direct formulas

- Define symbols and equations before coding.
- Look for a mathematical shortcut instead of unnecessary simulation.
- Make cases mutually exclusive or define a clear priority.
- Use a wide type when large constraints are multiplied.
- For Even Odds, split the odd and even halves using the 1-based position.

Example: deriving Even Odds

```
long long oddCount = (n + 1) / 2;
long long answer = (k <= oddCount)
    ? 2 * k - 1
    : 2 * (k - oddCount);
```

Indexing, overflow, and precision mistakes

- Confirm whether k is 1-based or 0-based.

- Do not use double for exact integer arithmetic.

Complete problem solutions

Each solution below includes a complete program, the governing formula, boundary tests, and complexity. The goal is to derive the program rather than memorize it.

959A Mahmoud and Ehab

Only parity matters. An even `n` prints Mahmoud; an odd `n` prints Ehab.

```
#include <iostream>
int main() { int n{}; std::cin >> n; std::cout << (n % 2 == 0 ? "Mahmoud" : "Ehab") << '\n'; }
```

Trace `1` and `2`. Time and memory are `O(1)`. Simulating turns adds work without information.

486A Calculating Function

Every pair `-1 + 2`, `-3 + 4`, and so on contributes `1`. An odd `n` leaves one final negative term.

```
#include <iostream>
int main() {
    long long n{}; std::cin >> n;
    std::cout << (n % 2 == 0 ? n / 2 : -(n + 1) / 2) << '\n';
}
```

`n=4` gives `2`; `n=5` gives `-3`. Use `long long`. The direct solution is `O(1)`.

4A Watermelon

Two positive even parts require an even weight greater than 2.

```
#include <iostream>
int main() {
    int w{}; std::cin >> w;
    std::cout << (w > 2 && w % 2 == 0 ? "YES" : "NO") << '\n';
}
```

Test `2`, `3`, and `4`. Checking parity alone incorrectly accepts `2`. Complexity is `O(1)`.

835A Key Races

Compute each total once as `s * v + 2 * t`, then compare the two values.

```
#include <iostream>
int main() {
    long long s{}, v1{}, v2{}, t1{}, t2{};
    std::cin >> s >> v1 >> v2 >> t1 >> t2;
    const long long first = s * v1 + 2 * t1;
    const long long second = s * v2 + 2 * t2;
    if (first < second) std::cout << "First\n";
    else if (second < first) std::cout << "Second\n";
    else std::cout << "Friendship\n";
}
```

Test one win for each player and a tie. Complexity is `O(1)`. Do not repeat the formula in every branch.

1173A Nauuo and Votes

`+` is guaranteed only when `x > y + z`; the negative case is symmetric. A guaranteed tie requires `z == 0 && x == y`.

```
#include <iostream>
int main() {
    long long x{}, y{}, z{}; std::cin >> x >> y >> z;
    if (x > y + z) std::cout << "+\n";
    else if (y > x + z) std::cout << "-\n";
    else if (z == 0 && x == y) std::cout << "0\n";
    else std::cout << "?\n";
}
```

Test all four outputs. Complexity is `O(1)`. Comparing only `x` and `y` ignores uncertainty.

318A Even Odds

The odd half contains `(n + 1) / 2` values. Map `k` into that half or subtract its size and map into the even half.

```
#include <iostream>
int main() {
    long long n{}, k{}; std::cin >> n >> k;
    const long long oddCount = (n + 1) / 2;
    std::cout << (k <= oddCount ? 2 * k - 1 : 2 * (k - oddCount)) << '\n';
}
```

For `n=10`, test `k=5` and `k=6`. The formula is `O(1)` and avoids building the sequence.

459A Pashmak and Garden

Handle a vertical side, a horizontal side, a 45-degree diagonal, and the impossible case separately.

```cpp
#include <cstdlib>
#include <iostream>
int main() {
    int x1{}, y1{}, x2{}, y2{}; std::cin >> x1 >> y1 >> x2 >> y2;
    if (x1 == x2 && y1 != y2) {
        const int d = std::abs(y1 - y2);
        std::cout << x1 + d << ' ' << y1 << ' ' << x2 + d << ' ' << y2 << '\n';
    } else if (y1 == y2 && x1 != x2) {
        const int d = std::abs(x1 - x2);
        std::cout << x1 << ' ' << y1 + d << ' ' << x2 << ' ' << y2 + d << '\n';
    } else if (std::abs(x1 - x2) == std::abs(y1 - y2)) {
        std::cout << x1 << ' ' << y2 << ' ' << x2 << ' ' << y1 << '\n';
    } else std::cout << "-1\n";
}
```

Test all four cases, including identical points. Complexity is `O(1)`.

617A Elephant

Ceiling division by 5 gives the minimum number of steps.

```cpp
#include <iostream>
int main() { int x{}; std::cin >> x; std::cout << (x + 4) / 5 << '\n'; }
```

Test `1`, `5`, and `6`. Complexity is `O(1)`.

581A Vasya the Hipster

Different-color days equal `min(a,b)`; the remaining difference supplies same-color pairs.

```cpp
#include <algorithm>
#include <cstdlib>
#include <iostream>
int main() {
    int a{}, b{}; std::cin >> a >> b;
    std::cout << std::min(a, b) << ' ' << std::abs(a - b) / 2 << '\n';
}
```

For `3 7`, the result is `3 2`. Complexity is `O(1)`.

281A Word Capitalization

Change only the first character. Cast through `unsigned char` before calling `<cctype>`.

```cpp
#include <cctype>
#include <iostream>
#include <string>
int main() {
    std::string word; std::cin >> word;
    if (!word.empty()) {
        const auto first = static_cast<unsigned char>(word.front());
        word.front() = static_cast<char>(std::toupper(first));
    }
    std::cout << word << '\n';
}
```

This solves the problem's Latin-character input; it is not general Unicode case conversion. Work after input is `O(1)`.

1A Theatre Square

Apply ceiling division to both dimensions and multiply with `long long`.

```
#include <iostream>
int main() {
    long long n{}, m{}, a{}; std::cin >> n >> m >> a;
    const long long rows = (n + a - 1) / a;
    const long long columns = (m + a - 1) / a;
    std::cout << rows * columns << '\n';
}
```

For `6 6 4`, the answer is `4`. Plain integer division undercounts whenever a remainder exists. Complexity is `O(1)`.

Derive instead of simulate

Constraints decide whether simulation is possible. If `n` may reach `10^18`, an `O(n)` loop cannot finish, so a direct or logarithmic solution is required. Test every transition boundary, such as the last odd position and first even position in Even Odds.

A disciplined contest workflow

Extract the input and output, write the cases and formulas, choose types from the largest intermediate value, and compute small expected answers by hand. Then test the first and last valid inputs and every point where the selected branch changes.

Lesson map

Check your understanding

01 **Derive the odd half of Even Odds.**

> **Explained answer:** There are (n+1)/2 odds through n; position k in that half maps to 2k-1.

02 **Why use long long when each input fits in int?**

Explained answer: Products and sums can overflow even when individual inputs fit; intermediate expression range matters.

03 **When is a formula better than simulation?**

Explained answer: If position or total can be computed directly, O(1) avoids a loop that may exceed time limits.

04 **Which boundaries expose a 0-based/1-based mistake?**

Explained answer: Test k=1, the last position of the first half, the first of the second half, and k=n.

05 **Why is `w % 2 == 0` alone insufficient for Watermelon?**

Explained answer: Two is even but cannot be split into two positive even parts. The full condition is `w > 2 && w % 2 == 0`.

06 **For Even Odds with `n = 7`, what are the final odd position and first even position?**

Explained answer: There are `(7 + 1) / 2 = 4` odd values. Position 4 contains 7; position 5 begins the even block with 2.

07 **Why does Theatre Square need `long long` even when each dimension fits in int?**

Explained answer: Multiplying tile counts for both axes may exceed int. Intermediate values and multiplication must be wide, not only the destination variable.

08 **Which three geometric cases must Pashmak and Garden distinguish?**

Explained answer: A vertical side, a horizontal side, or a 45-degree diagonal where `abs(dx) == abs(dy)`. Any other slanted case is impossible.

Connect the ideas

Turn samples into automated tests, then add boundaries, case transitions, and random inputs checked against a trusted slow solution. Record problem source and constraints because formula correctness depends on them; passing samples is not a proof.

Try it yourself

Choose one problem and test every point where a branch or formula changes.

Objects, the STL, RAII, and testing

Model data and rules with clear types, then use the standard library, automatic resource management, and focused tests to build maintainable programs.

First read functions and containers. The small class/algorithm fragments need their surrounding program and headers. The final example is complete. An iterator identifies a position in a container; `[begin, end)` includes the first position and excludes the position after the last element.

Types that represent the domain

A class keeps state together with operations that preserve its validity. Hide implementation details and expose an interface that cannot create an invalid object.

```
class Order {
public:
    explicit Order(double subtotal) : subtotal_{subtotal} {
        if (!std::isfinite(subtotal) || subtotal < 0) throw std::invalid_argument{"negative subtotal"};
    }

    double total(double discount) const {
        if (!std::isfinite(discount) || discount < 0 || discount > 1)
            throw std::invalid_argument{"discount outside 0..1"};
        return subtotal_ * (1.0 - discount);
    }
private:
    double subtotal_{};
};
```

The constructor establishes a valid object. The trailing `const` promises that the method does not change observable state. Prefer composition before inheritance when a type is assembled from collaborating parts rather than forming a true substitutable is-a relationship.

RAII and resource ownership

RAII ties a resource to an object's lifetime: acquisition happens during construction and release during destruction. `vector`, `string`, and file streams use this pattern.

Avoid direct `new` and `delete` in application code. Use `std::unique_ptr` when dynamic ownership is necessary. Use `std::shared_ptr` only for genuine shared ownership because reference cycles can prevent cleanup.

STL algorithms

Algorithms express intent across iterator ranges.

```
std::vector<int> scores{75, 42, 91, 60};
std::sort(scores.begin(), scores.end());
auto passed = std::count_if(scores.begin(), scores.end(),
    [](int score) { return score >= 50; });
```

Learn `find`, `transform`, and `accumulate` as well. Use an algorithm when it makes the operation and boundaries clearer, not only to shorten code.

Errors and tests

Represent expected invalid user input as a validation result. Use an exception when an operation cannot fulfill its contract. Catch it where the program can recover or create a useful message.

Test normal, boundary, and invalid cases. Compiler warnings, a debugger, and sanitizers help expose out-of-bounds access and lifetime errors. Every fixed bug should gain a regression test that failed before the fix.

Capstone refactor

Rebuild the online order project with `Product`, `Cart`, and a pricing policy. Store items in a vector, use algorithms for totals, and test an empty cart, the discount boundary, and an invalid quantity.

Check your understanding

01 **Why is RAII safer than cleanup copied into every control-flow path?**

> **Answer:** Destruction runs when scope ends, including early returns and exceptions, so cleanup is centralized and leaks are less likely.

02 **When is composition clearer than inheritance?**

> **Answer:** When a type contains collaborating parts and there is no genuine substitutable is-a relationship.

03 **Which regression test follows a discount bug at subtotal 500?**

> **Answer:** Test 500 together with 499 and 501 so the exact boundary and both neighboring behaviors remain defined.

Complete program and boundary checks

```
#include <algorithm>
#include <fstream>
#include <iostream>
#include <stdexcept>
#include <vector>

void savePassingScores(const std::vector<int>& scores) {
    std::ofstream file("passing-scores.txt");
    if (!file) throw std::runtime_error("Cannot open output");
    for (int score : scores) {
        if (score < 0 || score > 100) throw std::invalid_argument("Invalid score");
        if (score >= 50) file << score << '\n';
    }
    file.flush();
    if (!file) throw std::runtime_error("Write failed");
    file.close();
    if (!file) throw std::runtime_error("Close failed");
}
int main() {
    std::vector<int> scores{75, 42, 91, 60};
    std::sort(scores.begin(), scores.end());
    const auto passed = std::count_if(scores.begin(), scores.end(),
        [](int score) { return score >= 50; });
    try {
        savePassingScores(scores);
        std::cout << "passed=" << passed << '\n';
    } catch (const std::exception& error) {
        std::cerr << error.what() << '\n';
        return 1;
    }
}
```

Save as `raii.cpp`, compile with `g++ -std=c++17 -Wall -Wextra -pedantic raii.cpp -o raii`, then run in a writable scratch folder. It overwrites `passing-scores.txt` with 60,75,91 on separate lines and prints `passed=3`. `sort` changes the vector; `count_if` reads it using a lambda, a small callable defined where it is needed. The stream owns its file handle and releases it on scope exit, including exception unwinding. Explicit flush/close checks make write failures observable; a destructor cannot report them through this normal result path. RAII releases resources, but does not undo already written bytes: inserting an invalid score can leave a partial file. Validate first and use a temporary-file replacement protocol when all-or-nothing output is required. Test an empty list and an unwritable directory.

The Order fragment needs `<cmath>` and `<stdexcept>`. It rejects non-finite values such as NaN as well as negative subtotals and discounts outside 0..1. Approximate double values illustrate class structure here; represent financial amounts with bounded integer minor units or a decimal type in a real pricing model.

1. C++ & Computer Foundations

Problem-solving review, number systems, tools, translation, types, variables, operators, and input.

This group explains problem solving, number representation, C++ tools, types, variables, operators, and input. Follow the lessons in order because each one uses terms introduced earlier.

2. Conditions, Validation & Projects

Business rules, the build pipeline, operators, conditions, switch, and two complete projects.

This group connects input, business rules, the build pipeline, conditions, and `switch`, then applies them in an online order and training center project.

3. Encoding and loops

Text encoding, conversions, loops, counters, accumulators, and repetition patterns.

This group explains text encoding and conversions, then covers loops, counters, accumulators, and nested repetition. Problem-statement and constraint analysis now belongs to the problem-solving lesson.

03

TRACK · 18 LESSONS

The PHP language

Progressive explanations, practical examples, and corrections that take each idea into real code.

The PHP language

PHP

A complete PHP path that moves from language fundamentals to understandable, testable web programs.

This path starts with PHP fundamentals and builds each idea on the previous one until you can read a web program and understand its parts. Lessons include runnable examples, common cases, and mistakes with their causes and fixes.

Learning path

1. Introduction and syntax
2. Variables, scope, and superglobals
3. Data types and the type system
4. Output, debugging, and constants
5. Conditions and loops — Trace branches, loop progress, boundaries, break, continue, and references.
6. Expressions and operators
7. Functions, callbacks, and includes — Call stacks, recursion, callbacks, and a complete two-file program.
8. Files, streams, JSON, and CSV
9. Uploads, cookies, and sessions
10. Namespaces, Composer, and quality tools — A PSR-4 project, lock files, SemVer, scripts, audit, PHPUnit, PHPStan, and PHP-CS-Fixer.
11. Errors and exceptions
12. Arrays and transformation tools
13. Strings, Unicode, and regular expressions
14. Date, time, and timezones
15. From HTTP request to router and response
16. Modern PHP 8.0–8.5 roadmap — WeakMap, Fibers, DNF types, readonly classes, hooks, and later features.
17. Integrated programs and practical debugging — A complete notebook connecting forms, validation, sessions, files, and routing, with Xdebug and regression tests.

How to study

Run every example, change its inputs, and predict the result before running it again. The security notes are part of correct PHP usage, not optional extras.

```php
<?php

declare(strict_types=1);

function greet(string $name): string
{
    return "Hello {$name}";
}

echo greet('Omar');
```

> Examples target PHP 8.x. Features requiring a newer minor release are labelled explicitly.

Set up the downloadable labs and runnable examples.

A project that grows with you

Start with the name form in lesson 9, understand the Composer project in 10, trace the API in 15, then build the notebook in 17. Download the complete files: lesson programs, session/upload forms, a locked Composer project with tests, and the notebook. The course keeps 17 numbered lessons plus the lab setup page; navigation and homepage counts are generated from the files.

Set up the labs and run examples

PHP labs, fixtures, reproducible tests and expected results.

Prepare files you can actually run, and distinguish a local function test from a whole-service test. If variables and functions are new, begin the PHP lessons and return to each lab as you learn its prerequisites.

Setup

Download the lab files, extract them, and open a terminal inside `php-labs`. In a project checkout, use `examples/php-labs`. Install PHP 8.4 or later, then run:

```
php -v
php -m
php tests.php
```

Core tests require `pdo_sqlite`, `sodium`, and `mbstring`; HTTP tests additionally require `curl`. No third-party library is required for core tests. Examples explicitly marked PHP 8.5 in the modern-features lesson require 8.5. For “could not find driver,” enable `pdo_sqlite` in the configuration reported by `php --ini` for the same executable you run.

Expect `PASS` lines and a final `failures=0`. Exit code 0 means success; a failed test returns 1. Tests throw explicit exceptions rather than relying on PHP `assert`, which configuration may disable. Read the exit code with `$LASTEXITCODE` in PowerShell or `echo $?` in Bash.

Select a test

Command	Coverage
`php tests.php values`	Amount grammar, addition/conversion bounds, ID lists
`php tests.php php`	Attributes, CSV, references, Unicode, dates
`php tests.php security`	Post-signature claims policy, CSRF, encrypted context, hashing policy
`php tests.php inventory`	Purchase transaction, deduplication, rollback, constraints
`php locking-lab.php`	Two processes competing for SQLite stock
`php total.php 12.50 3.25`	Addition printing `15.75`
`php orders.php fixtures/orders.json`	Bounded JSON order import
`php stream-lab.php fixtures/large.csv`	Incremental reading of 10000 CSV records
`php orm-query-lab.php`	21 queries versus two for relationship loading
`php restore-verify.php`	Create, open, and validate a SQLite snapshot

In-memory SQLite starts fresh on each run. Concurrency and restore tests create and clean up their own temporary files; they do not connect to your application database.

Real local HTTP

In one terminal inside the lab folder:

```
php -S 127.0.0.1:8097 http-router.php
```

In a second terminal in the same folder:

```
php http-client-lab.php
```

Expect `status=200`, `server_error=500`, `conditional=304`, and `timeout_handled=yes`. Read `http-router.php` for each endpoint and `src/Http.php` for ETag parsing. Stop the PHP server with Ctrl+C. This local development server does not test FPM, HTTPS, or production load.

Read the implementation

`bootstrap.php` loads functions relative to `__DIR__`, the file's own directory, so inclusion does not depend on the terminal directory. Read `src/Values.php` for value rules, `schema.sql` for tables and seed data, `src/Inventory.php` for the transaction, and `tests.php` for successful and failing examples.

A fixture is a known, fixed input used to reproduce a test. `fixtures/broken.json` is deliberately malformed: importing it must reject JSON syntax, not silently succeed with empty data. Change a copy of the input, not the expected result, and identify the violated rule.

What these tests establish

`cache-lab.php` models a race schedule locally; it does not run Redis. `profiling-lab.php` calculates percentiles from fixed data; it does not export traces. JWT tests check claims after verification and do not implement an identity protocol. SQLite success does not establish PostgreSQL/MySQL isolation or deadlock behavior. Complete service-specific exercises in an isolated environment with the actual engine and configuration.

1. PHP introduction and syntax

A calm start from zero—where PHP runs, how a request reaches it, and how to write, run, and safely embed your first PHP program.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Runtime:** The period when a program is actually running.
- **HTTP:** The rules used to exchange requests and responses on the web.
- **API:** A defined interface through which one program requests data or actions from another.
- **Queue:** A line of background jobs waiting to be processed.
- **CLI:** A text-based interface controlled by typed commands.
- **UTF-8:** A common encoding that stores Unicode numbers as bytes.
- **Scope:** The region of code in which a name or variable is visible.

What will you learn?

Before memorizing syntax, build the mental model. When someone submits a login form, the browser sends a request to a server. A program must read the data, validate it, talk to storage when necessary, and return a result. **PHP is one language you can use to write that server-side program.**

By the end of this lesson, you should understand:

- where PHP runs and how it differs from HTML and browser JavaScript;
- how to run a PHP file from a terminal and a browser;
- how PHP statements, blocks, and semicolons work;
- how PHP and HTML fit together safely;
- how to read a basic error instead of guessing.

What is PHP?

PHP is a general-purpose programming language best known for backend web development. Its name originally referred to “Personal Home Page”; the recursive name used today is **PHP: Hypertext Preprocessor**.

Keep these three roles separate:

- **HTML** describes page content and structure sent to the browser.
- **Browser JavaScript** can add behavior after the page reaches the user.
- **PHP** commonly runs on the server before the response reaches the browser.

If the server contains this code:

```
<?php
$name = 'Omar';
echo "Hello {$name}";
```

the browser does not receive `$name` or `echo`. It receives the output:

```
Hello Omar
```

PHP source should remain on the server. If source code or database credentials are downloaded by a browser, the server is misconfigured; that is not normal PHP execution.

How does a request reach PHP?

Imagine opening `/products.php`:

1. The browser sends an HTTP request.
2. A web server such as Nginx or Apache receives it.
3. For dynamic PHP work, the web server passes the request to the PHP runtime.
4. PHP executes the code and may read files, query a database, or call another API.
5. PHP produces a body, headers, and a status code.
6. The web server sends the HTTP response.
7. The browser renders HTML or processes JSON, depending on the response.

```
Browser → HTTP Request → Web Server → PHP → Database/File/API
Browser ← HTTP Response ← Web Server ← Result
```

Not every request needs a database, and PHP does not have to return HTML. It can return JSON to a mobile application, generate a file, or run from the command line without a browser.

Is PHP interpreted line by line?

That phrase is a useful first approximation but technically incomplete. The engine compiles source into opcodes and executes them; OPcache can retain compiled opcodes, and modern PHP also includes a JIT. The practical beginner takeaway is that normal PHP development does not require a traditional manual build after every edit.

What can PHP build?

PHP can:

- receive forms and validate HTTP input;
- generate dynamic pages and web APIs;
- read and write databases and files;
- manage sessions, authentication, and authorization;
- run CLI commands, scheduled jobs, and queue workers;
- send email and communicate with external services;
- create images or PDFs with suitable libraries;
- support procedural and object-oriented code.

External tools can target desktop applications, but that is not PHP's most common use or its natural first use case.

Prepare the environment

You may install PHP separately or use a local bundle:

- **XAMPP** on several operating systems;
- **WAMP** on Windows;
- **MAMP** on macOS;
- **LAMP**, commonly meaning Linux, Apache, MySQL/MariaDB, and PHP.

Open a terminal and run:

```
php --version
```

A working installation prints a version. If the command is unknown, PHP is probably not installed or its executable directory is missing from `PATH`.

Your first CLI program

Create `hello.php`:

```
<?php

$name = 'Omar';
$lessonCount = 1;

echo "Hello {$name}!", PHP_EOL;
echo "You finished lesson {$lessonCount}.", PHP_EOL;
```

Run it:

```
php hello.php
```

Expected output:

```
Hello Omar!
You finished lesson 1.
```

Line by line:

- `<?php` begins PHP mode.
- `$name` stores a string in a variable.
- `$lessonCount` stores an integer.
- `echo` outputs values.
- braces make a variable's boundary clear inside a string.
- `PHP_EOL` emits the platform-appropriate line ending.
- `;` terminates a simple statement.

In a PHP-only file, normally omit the closing `?>`. This avoids accidental whitespace output after the closing tag.

Run PHP in a browser

Create a `public` directory with `index.php`:

```
<?php

echo '<h1>My first PHP page</h1>';
echo '<p>The server generated this HTML.</p>';
```

From the directory that contains `public`, run:

```
php -S localhost:8000 -t public
```

Then open `http://localhost:8000`. The built-in server is designed for learning and local development, not direct production exposure.

Statements and blocks

Simple statements normally end in a semicolon:

```
$price = 150;
$quantity = 2;
$total = $price * $quantity;
echo $total;
```

A control block does not take a semicolon after its closing brace:

```
if ($total >= 300) {
    echo 'Free shipping';
} else {
    echo 'Shipping fee applies';
}
```

When a semicolon or quote is missing, the parser may report the following line because that is where the statement finally became impossible to parse. Always inspect the previous line too.

Embed PHP in HTML

PHP can enter and leave PHP mode inside an HTML template:

```php
<?php
$title = 'My shop';
$products = ['Keyboard', 'Mouse', 'Monitor'];
?>
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title><?= htmlspecialchars($title, ENT_QUOTES, 'UTF-8') ?></title>
</head>
<body>
    <h1><?= htmlspecialchars($title, ENT_QUOTES, 'UTF-8') ?></h1>

    <?php if ($products === []): ?>
        <p>No products are available.</p>
    <?php else: ?>
        <ul>
            <?php foreach ($products as $product): ?>
                <li><?= htmlspecialchars($product, ENT_QUOTES, 'UTF-8') ?></li>
            <?php endforeach; ?>
        </ul>
    <?php endif; ?>
</body>
</html>
```

`<?= $value ?>` means “output this value.” The alternative `if: ... endif;` and `foreach: ... endforeach;` syntax is easier to follow inside HTML than many nested braces.

We used `htmlspecialchars()` because untrusted text must be encoded before insertion into HTML. If a product name contains `<script>`, it should appear as text rather than execute as browser code.

> ⚠
>
> **Encoding depends on context**
>
> `htmlspecialchars()` is suitable for ordinary HTML text. URLs, JavaScript, CSS, and attributes can have additional rules. Encode at output time for the exact destination context.

Headers must precede the body

An HTTP response contains headers followed by a body. Call `header()`, `setcookie()`, and `session_start()` before outputting HTML or text:

```php
<?php

header('Content-Type: application/json; charset=utf-8');

$response = ['status' => 'ok'];
echo json_encode($response, JSON_UNESCAPED_UNICODE | JSON_THROW_ON_ERROR);
```

Output:

```
{"status":"ok"}
```

If output happens first, PHP may report `headers already sent`. Look for earlier HTML, `echo`, or even whitespace before `<?php`.

Comments and letter case

The syntax example below uses comments: text ignored by PHP. Composer manages PHP packages; OPcache caches compiled instructions; Nginx receives web requests and can forward PHP work. A variable names a stored value, and syntax defines how code must be written.

```
// One-line comment

# Another one-line form, used less often
/* A comment
   across lines */

$userName = 'Omar';
echo $userName;
// echo $username; // a different variable
```

Variable names are case-sensitive. Keywords such as `if` are effectively case-insensitive, and functions/classes are resolved case-insensitively, but do not rely on that behavior. Use the declared spelling and follow PSR conventions.

A useful comment explains a decision rather than narrating syntax:

```
// Keep money in cents to avoid binary floating-point rounding.
$priceCents = 1999;
```

A complete small example

This program reads a name from the query string and renders it safely. Start the built-in server and open `http://localhost:8000/?name=Omar`:

```php
<?php

$rawName = $_GET['name'] ?? 'Guest';
$name = is_string($rawName) ? trim($rawName) : 'Guest';

if ($name === '') {
    $name = 'Guest';
}

$safeName = htmlspecialchars($name, ENT_QUOTES, 'UTF-8');
?>
<!doctype html>
<html lang="en">
<head><meta charset="utf-8"><title>Welcome</title></head>
<body><h1>Hello <?= $safeName ?></h1></body>
</html>
```

Try `?name=<b>Omar</b>`. The markup should appear as text rather than bold output, proving that the value was encoded.

Common first-day mistakes

- Opening a PHP file directly: run it with PHP CLI or a web server.
- `php` is unknown: check installation and `PATH`.
- A parse error points at innocent code: inspect the previous line for a missing quote, bracket, or semicolon.
- An undefined variable appears: check spelling and whether assignment ran first.
- `headers already sent`: body output happened before header changes.
- User input is rendered directly: validate it and encode for the output context.

The first-program journey

```
Write hello.php → Run PHP → Read output or error → Change one thing → Run again
```

Check your understanding

01 **Does a browser receive PHP source? Describe the request path.**

> **Answer:** No. The web server hands work to PHP; PHP executes source and returns output. The browser receives an HTTP response such as HTML or JSON.

02 **Predict the output of `echo 'PHP', ' ', 8;`.**

> **Answer:** `PHP 8`. `echo` can output several comma-separated values in order.

03 **An error points to line 8, but line 8 looks valid. Where should you inspect next?**

> **Answer:** Inspect the preceding line for an unclosed string or bracket or a missing semicolon.

04 **Why is `echo $_GET['name'];` unsafe in HTML?**

> **Answer:** A user can submit markup or script. Validate according to business rules and encode the value for its HTML context at output time.

05 **Fix `$name = 'Omar' echo $name;`.**

> **Answer:** Add the missing statement terminator: `$name = 'Omar'; echo $name;`.

06 **Build a page that accepts `product` and `price` and displays them safely.**

> **Solution outline:** Read defaults with `??`, validate the price as a number in an allowed range, and pass the product through `htmlspecialchars()` when rendering. Test missing, invalid, and markup-containing values.

Summary

PHP usually runs on the server and the browser receives its output. You can run PHP through the CLI or a local web server. Begin PHP mode with `<?php`, terminate simple statements with semicolons, and encode untrusted values for their output context. The next lesson explains how variables store values, how scope controls visibility, and how request data enters PHP.

Run and verify

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Execute this checkpoint inside the lesson environment:

```
php hello.php
```

Success criterion: The program prints the expected line and exits 0; then introduce one syntax error and confirm the diagnostic identifies file and line.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

Connect the ideas

To reduce overload, study this lesson in two passes: CLI for syntax and exit codes, then Web for headers and body. Composer is not required for a first file but becomes the project entry point for autoloading and locked dependencies. The built-in server is not production.

Try it yourself

Run the example in CLI and Web modes and identify differences in input, output, and hosting process.

2. Variables, scope, and superglobals

Variables from first principles, value and reference assignment, scope, and how request data reaches a PHP program through superglobals.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **HTTP:** The rules used to exchange requests and responses on the web.
- **URL:** The complete address of a resource such as a page or API endpoint.
- **Cache:** A temporary copy that reduces waiting and repeated work.
- **Session:** Temporary server-side state used to recognize a user across requests.
- **UTF-8:** A common encoding that stores Unicode numbers as bytes.
- **Scope:** The region of code in which a name or variable is visible.
- **Function:** A named, reusable block of code with one defined job.

Why do programs need variables?

Programs work with values that change: a user's name, a product price, a quantity, or a calculated result. A **variable** gives a value a meaningful name so later instructions can use it.

The "labeled box" analogy is useful at first, although the engine's real storage strategy depends on the value type and memory management.

```
<?php

$productName = 'Keyboard';
$priceCents = 150000;
$inStock = true;

echo $productName, PHP_EOL;
echo $priceCents, PHP_EOL;
```

The `$` introduces a variable name. Here `=` is assignment: evaluate the right side and assign its value to the name on the left.

Naming rules and useful names

After `$`, a variable name starts with a letter or underscore and may then contain digits:

```
$name = 'Omar';       // valid
$_count = 3;          // valid
$price2 = 19.5;       // valid
// $2price = 19.5;    // invalid
```

Variable names are case-sensitive:

```
$userName = 'Omar';
$username = 'Ali';

echo $userName; // Omar
echo $username; // Ali
```

Choose a name that carries meaning. `$priceCents` is clearer than `$p`; `$isEmailVerified` is clearer than `$flag`.

The value determines the current type

PHP is dynamically typed, so a variable declaration does not need a type:

```
$value = 10;      // int
$value = 'ten';   // now string
```

This does not mean PHP has no types. Every value has a type, and operations, conversions, and comparisons follow type rules. Inspect a value while learning or debugging:

```
$price = 19.5;

var_dump($price);            // float(19.5)
echo get_debug_type($price); // float
```

Variables inside strings

Double-quoted strings interpolate variables; single-quoted strings usually preserve the text:

```
$name = 'Omar';
$price = 20;

echo "Hello {$name}, the price is {$price} pounds.", PHP_EOL;
echo 'Hello $name', PHP_EOL;
```

Output:

```
Hello Omar, the price is 20 pounds.
Hello $name
```

Braces clearly mark the variable boundary. The older `${name}` interpolation syntax is deprecated.

Assignment by value

Predict this result before running it:

```
$a = 10;
$b = $a;
$b++;

echo $a, PHP_EOL;
echo $b, PHP_EOL;
```

Output:

```
10
11
```

`$b = $a` gives `$b` a logically independent value. Changing `$b` does not change `$a`.

For arrays and strings, PHP can optimize copying internally through **copy-on-write**. It need not copy every byte immediately, but the values behave independently and separation occurs when one is changed. Learn the visible semantics before the storage optimization.

Assignment by reference

`&` links two variable names to the same logical container:

```
$score = 10;
$alias =& $score;

$alias++;
echo $score; // 11
```

`unset($alias)` removes that name's link; it does not delete `$score`.

A PHP reference is not manual pointer arithmetic like C. Most application code does not need references. Unnecessary aliases make it difficult to answer "which code changed this value?"

Objects behave differently

Assigning an object to another variable normally makes both variables refer to the same object:

```
$first = new stdClass();
$first->name = 'Omar';

$second = $first;
$second->name = 'Ali';

echo $first->name; // Ali
```

Use `clone` when you need a distinct object. The OOP track explains shallow and deep copying later.

Variable variables

PHP can use one variable's value as another variable's name:

```
$field = 'email';
$$field = 'omar@example.com';

echo $email;
```

The feature exists, but an array or object is usually clearer:

```
$user = ['email' => 'omar@example.com'];
echo $user['email'];
```

Never turn arbitrary user input directly into variable names. It hides allowed fields and makes validation and reasoning difficult.

What is scope?

Scope is the region where a variable name is visible. A variable created inside a function is local to that function; a file-level variable is not automatically visible inside it.

```php
$taxRate = 0.14;

function showLocalScope(): void
{
    $message = 'I exist inside this function';
    echo $message;

    // echo $taxRate; // not automatically visible
}

showLocalScope();
// echo $message; // not visible here
```

Prefer explicit parameters:

```php
function priceWithTax(int $priceCents, float $taxRate): int
{
    return (int) round($priceCents * (1 + $taxRate));
}

echo priceWithTax(10000, 0.14); // 11400
```

The dependency is visible, and the function is easy to test with chosen inputs.

`global` and `$GLOBALS`

PHP allows access to file-level state:

```php
$total = 100;

function addTaxUsingGlobal(float $rate): float
{
    global $total;
    return $total * (1 + $rate);
}
```

The same state is available through `$GLOBALS['total']`. Both forms hide a dependency. If distant code changes `$total`, this function's result changes. Passing the value explicitly is normally easier to understand and test.

Local `static` variables

A local `static` variable is initialized once and keeps its value between calls in the current process:

```php
function nextId(): int
{
    static $id = 0;
    return ++$id;
}

echo nextId(), PHP_EOL; // 1
echo nextId(), PHP_EOL; // 2
echo nextId(), PHP_EOL; // 3
```

This is not permanent storage. A new process starts over. Hidden state can also complicate tests, so do not use a local static variable as a database or shared cache.

Scope of included files

An `include` or `require` inherits the scope at the inclusion point:

```php
function loadConfig(): array
{
    $environment = 'development';
    return require __DIR__ . '/config.php';
}
```

`config.php` can see `$environment` because inclusion happened inside the function. Prefer configuration files that return a clear value instead of silently modifying many surrounding variables.

Superglobals bring request data into PHP

PHP places request and environment data in special arrays available in every scope:

Variable	Typical content
`$_GET`	Query-string values after `?`
`$_POST`	Fields from form-encoded POST bodies
`$_SERVER`	Request, server, and available header information
`$_FILES`	Uploaded-file metadata
`$_COOKIE`	Cookies sent by the client
`$_SESSION`	Session data after `session_start()`
`$GLOBALS`	Global symbol table

For this URL:

```
http://localhost:8000/?page=2&category=books
```

you can read:

```
$page = $_GET['page'] ?? '1';
$category = $_GET['category'] ?? 'all';
```

All client-controlled data is untrusted, even when it looks normal.

Read, validate, convert, and encode

These are separate steps:

1. Read while handling absence.
2. Validate against application rules.
3. Convert to a useful internal type.
4. Encode when outputting into a specific context.

```
$rawPage = $_GET['page'] ?? null;
$page = filter_var($rawPage, FILTER_VALIDATE_INT, [
    'options' => ['min_range' => 1, 'max_range' => 1000],
]);

if ($page === false) {
    $page = 1;
}
```

A checkbox list may arrive as an array:

```
?order_ids[]=10&order_ids[]=20
```

```
function positiveIds(mixed $input, int $maximumCount = 100): array
{
    if (!is_array($input) || !array_is_list($input) || count($input) > $maximumCount) {
        throw new InvalidArgumentException('Expected a bounded list of IDs');
    }
    $ids = [];
    foreach ($input as $raw) {
        if ((!is_string($raw) && !is_int($raw))
            || preg_match('/\A[1-9][0-9]*\z/', (string) $raw) !== 1) {
            throw new InvalidArgumentException('Invalid ID');
        }
        $id = filter_var($raw, FILTER_VALIDATE_INT, ['options' => ['min_range' => 1]]);
        if ($id === false) {
            throw new InvalidArgumentException('ID exceeds integer range');
        }
        $ids[] = $id;
    }
    return $ids;
}

try {
    $orderIds = positiveIds($_GET['order_ids'] ?? []);
} catch (InvalidArgumentException) {
    http_response_code(422);
    exit('Invalid order IDs');
}
```

Conversion is not validation: `intval("12x")` silently produces 12. Require a list of at most 100 canonical positive integer IDs and reject nested arrays, zero, leading zeros, and overflow. The empty list is allowed. After validation, separately check that the authenticated user may access every selected order.

Validation is not output encoding. A valid name still needs `htmlspecialchars()` in HTML and a value still needs a prepared statement when used in SQL.

Why `eval()` is dangerous

`eval()` executes a string as PHP source:

```
$code = 'echo "hello";';
eval($code);
```

If that string comes from a user, you have allowed them to execute server code. Ordinary applications do not need this. Use functions, callback maps, or `match` instead:

```
$operations = [
    'trim' => static fn (string $value): string => trim($value),
    'upper' => static fn (string $value): string => strtoupper($value),
];

$result = $operations['trim']('  hello  ');
```

Complete example: greeting from a URL

```
<?php
declare(strict_types=1);

$rawName = $_GET['name'] ?? 'Guest';

if (!is_string($rawName)) {
    $rawName = 'Guest';
}

$name = trim($rawName);
if ($name === '' || mb_strlen($name) > 50) {
    $name = 'Guest';
}

$safeName = htmlspecialchars($name, ENT_QUOTES, 'UTF-8');
?>
<!doctype html>
<html lang="en">
<head><meta charset="utf-8"><title>Greeting</title></head>
<body><h1>Hello <?= $safeName ?></h1></body>
</html>
```

Test `?name=Omar`, an empty name, `?name[]=Omar`, and a name containing HTML. Each case checks a different assumption.

Value flow

```
Request value → Read → Check type and rules → Store clearly → Encode for output
```

Check your understanding

01 **Predict `$a = 5; $b = $a; $b++; echo "$a,$b";`.**

> **Answer:** `5,6`; normal assignment produces logically independent values.

02 **What changes if assignment uses `&`?**

> **Answer:** With `$b =& $a`, the result becomes `6,6` because the names are linked.

03 **Why is an explicit parameter easier to test than `global` state?**

> **Answer:** All dependencies appear in the call, so a test can supply them. A global dependency can change elsewhere without being visible in the function signature.

04 **Why is casting `?page=abc` directly to `int` insufficient?**

> **Answer:** Casting can produce zero and hide invalid input. Validate an integer in the allowed range, then deliberately choose a default or error.

05 **How do validation and HTML encoding differ?**

> **Answer:** Validation checks application rules. Encoding makes a value data rather than executable markup in a specific output context.

06 **Why should a local static counter not store permanent visit counts?**

> **Answer:** It lasts only in one process; traffic may use other processes or servers, and the state disappears when that process ends. Use shared persistent storage.

Summary

A variable names a value during execution. Normal assignment, references, and objects do not have identical sharing behavior. Scope controls visibility, and explicit parameters are clearer than global dependencies. Superglobals bring request data into PHP, but data must be read defensively, validated, converted, and encoded for its output context.

Run and verify

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Execute this checkpoint inside the lesson environment:

```
php scope-lab.php
```

Success criterion: Output proves the difference among local, global, and static state, and no superglobal value is read before checking key presence and shape.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

Connect the ideas

Superglobals are trust boundaries, not ready data: check presence, shape, and size, then convert into a request object. Variable variables and `eval` are rarely sound design tools; a map, callable registry, or explicit parser is clearer and safer. Avoid hidden global dependencies.

Try it yourself

Refactor an endpoint reading $_GET everywhere into validated values passed to functions.

3. Data types and the type system

Types from first principles: strings, numbers, booleans, null, arrays, objects, declarations, and advanced callable and iterable types.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Runtime:** The period when a program is actually running.
- **Unicode:** A standard that assigns consistent numbers to characters and symbols.
- **Boolean:** A logical value with only two states: true or false.
- **Loop:** A structure that repeats instructions according to a condition.
- **Function:** A named, reusable block of code with one defined job.

What is a data type?

The integer `10` and the string `'10'` look similar but mean different things. The first is ready for arithmetic; the second is a sequence of characters, perhaps received from a form. A **data type** tells PHP and the programmer what a value represents and which operations make sense.

```
$quantity = 10;       // int
$input = '10';        // string
$price = 19.95;       // float
$isAvailable = true;  // bool
$discount = null;     // no value at present
```

Ignoring types can make a program calculate with invalid text, confuse missing data with zero, or compare values unexpectedly. Do not merely memorize type names; ask what value you have, where it came from, and which operations are safe.

The type map

- **Scalar:** one value— `bool`, `int`, `float`, or `string`.
- **Compound:** values and behavior— `array` and `object`.
- **Special:** `null` and `resource`.

- **Function-contract types:** `callable`, `iterable`, `mixed`, `void`, and `never`.
- **User-defined:** classes, interfaces, and enums.
- **Combinations:** a union such as `int|string` or an intersection such as `Countable&Iterator`.

You do not need every advanced type immediately. Master scalars and arrays first; revisit callables and intersections after functions and OOP.

`bool`: true or false

Booleans represent decisions:

```
$isLoggedIn = true;
$hasPermission = false;

if ($isLoggedIn && $hasPermission) {
    echo 'Allowed';
} else {
    echo 'Denied';
}
```

Names such as `$isActive`, `$hasAccess`, and `$canEdit` reveal that a variable answers a yes/no question.

PHP treats these values as false in a Boolean context: `false`, `0`, `0.0`, `''`, `'0'`, `[]`, and `null`. Therefore do not rely on truthiness when zero and absence have different meanings.

`null`: no value

`null` means no value is currently present. It is different from zero, an empty string, and `false`:

```
$middleName = null;

if ($middleName === null) {
    echo 'No middle name was provided';
}
```

`isset($array['key'])` is false for a missing key and for a key whose value is `null`. Use `array_key_exists()` when key presence matters even if its value is null.

Loose and strict comparison

`==` may convert types; `===` requires the same type and value:

```
var_dump(0 == false);    // true
var_dump(0 === false);   // false
var_dump('10' == 10);    // true
var_dump('10' === 10);   // false
```

Start with `===` and `!==` in application code. Use loose comparison only with a deliberate understanding of its conversion rules.

`int`: whole numbers

```
$decimal = 42;
$octal = 0o52;
$hex = 0x2A;
$binary = 0b101010;
```

All four values equal 42. Binary and hexadecimal forms appear in masks, colors, and protocols.

```
echo 7 / 2;        // 3.5
echo intdiv(7, 2); // 3
echo 7 % 2;        // 1

echo (int) 3.9;    // 3: truncation
echo round(3.9);   // 4: rounding
```

Integer size depends on the platform; inspect `PHP_INT_MAX` and `PHP_INT_MIN`. Overflow can produce a float and lose exact integer precision.

`float`: decimal-looking values

Floats use finite binary fractions, so many decimal fractions are approximate:

```
$result = 0.1 + 0.2;

var_dump($result === 0.3); // usually false
```

For scientific comparisons, use an appropriate tolerance:

```
$epsilon = 0.000001;

if (abs($result - 0.3) < $epsilon) {
    echo 'Close enough';
}
```

For money, a simple safe model is often an integer in the smallest unit:

```php
$priceCents = 1999;
$quantity = 3;
$totalCents = $priceCents * $quantity; // 5997
```

Use an appropriate decimal library when the domain requires exact fractional units or very large numbers.

`string`: text stored as bytes

```php
$name = 'Omar';
$message = "Hello {$name}";
$joined = 'PHP' . ' ' . '8';
```

The string-concatenation operator is `.`, not `+`.

A PHP string is a sequence of bytes. Byte indexing works predictably for basic ASCII but not for splitting arbitrary Unicode text:

```php
$english = 'PHP';
echo $english[0];  // P
echo $english[-1]; // P
```

Use `mb_strlen()` and `mb_substr()` when working with multibyte text and the `mbstring` extension.

Heredoc and nowdoc

```php
$name = 'Omar';

$heredoc = <<<TEXT
Hello $name
This value is interpolated.
TEXT;

$nowdoc = <<<'TEXT'
$name stays exactly as written.
TEXT;
```

Heredoc behaves like a double-quoted string; nowdoc behaves like a single-quoted one.

Numeric text is not validated data

Forms and query strings normally provide strings. Validate before conversion:

```php
$rawAge = $_GET['age'] ?? null;
$age = filter_var($rawAge, FILTER_VALIDATE_INT, [
    'options' => ['min_range' => 1, 'max_range' => 120],
]);

if ($age === false) {
    echo 'Invalid age';
} else {
    echo "Next year you will be ", $age + 1;
}
```

`is_numeric()` recognizes numeric syntax but does not enforce your business rule. `-20` is numeric but not a valid human age.

`array`: list and map

A PHP array is an ordered map and can represent a list:

```php
$colors = ['red', 'blue'];
echo $colors[0];
```

or keyed data:

```php
$user = [
    'id' => 7,
    'name' => 'Omar',
    'active' => true,
];

echo $user['name'];
```

It can also nest:

```php
$orders = [
    ['id' => 101, 'total_cents' => 5000],
    ['id' => 102, 'total_cents' => 7500],
];

echo $orders[1]['total_cents']; // 7500
```

Use `??` when a missing key is expected: `$country = $user['country'] ?? 'Unknown';`.

Objects and classes

An object is an instance of a class that combines state and behavior:

```
final class Product
{
    public function __construct(
        public string $name,
        public int $priceCents,
    ) {}
}

$product = new Product('Keyboard', 150000);
echo $product->name;
```

The important point now is that the value's type is `Product` and can carry meaningful properties and methods. The OOP track develops the full model.

Enums define allowed states

An enum replaces error-prone free text with a closed set:

```
enum OrderStatus: string
{
    case Pending = 'pending';
    case Paid = 'paid';
    case Cancelled = 'cancelled';
}

$status = OrderStatus::Paid;
echo $status->value;
```

An accidental status such as `'paied'` can no longer silently enter typed domain code.

Resources are handles

Some functions return a `resource` handle to an external stream or connection:

```
$handle = fopen(__FILE__, 'rb');

if ($handle === false) {
    throw new RuntimeException('Could not open the file');
}

echo get_debug_type($handle); // resource (stream)
fclose($handle);
```

The handle is not the file contents. Modern extensions increasingly return objects instead, so check current documentation and runtime types.

Type declarations and strict types

Function contracts can declare inputs and output:

```
function calculateTotal(int $priceCents, int $quantity): int
{
    return $priceCents * $quantity;
}
```

At a file's start:

```
<?php
declare(strict_types=1);
```

Without strict mode, PHP may coerce compatible scalar arguments. With strict mode, passing `'3'` to an `int` parameter generally throws `TypeError`. Strictness is chosen by the **calling file**. PHP still accepts an `int` for a `float` parameter.

Nullable and union types

```
function findUser(int $id): ?array
{
    return $id === 7 ? ['id' => 7, 'name' => 'Omar'] : null;
}
```

`?array` means `array|null`. A union supports other explicit alternatives:

```
function normalizeId(int|string $id): int
{
    if (is_string($id) && !ctype_digit($id)) {
        throw new InvalidArgumentException('Invalid ID');
    }

    return (int) $id;
}
```

Do not broaden a type merely for convenience; every alternative creates behavior that must be defined and tested.

`mixed`, `void`, and `never`

- `mixed` accepts any type, including `null`; prefer a narrower contract when possible.
- `void` returns no useful value.
- `never` cannot return normally because it throws, exits, or never terminates.

```
function logMessage(string $message): void
{
    error_log($message);
}

function fail(string $message): never
{
    throw new RuntimeException($message);
}
```

Callables and closures

A callable is a value PHP can invoke. A callback is a callable passed for later invocation:

```
$double = function (int $number): int {
    return $number * 2;
};

$shortDouble = fn (int $number): int => $number * 2;

echo $double(4);      // 8
echo $shortDouble(5); // 10
```

A Closure is an object representing an anonymous function. PHP also supports first-class callable syntax:

```
function clean(string $value): string
{
    return trim($value);
}

$cleaner = clean(...);
```

An object becomes callable by defining `__invoke()`.

Iterables and generators

`iterable` accepts an array or an object implementing `Traversable`:

```
function printValues(iterable $values): void
{
    foreach ($values as $value) {
        echo $value, PHP_EOL;
    }
}
```

A generator yields values one at a time:

```
function numbers(int $maximum): iterable
{
    for ($number = 1; $number <= $maximum; $number++) {
        yield $number;
    }
}

printValues(numbers(3));
```

`yield` suspends execution and preserves function state until the next value is requested. Every generator is an Iterator, but not every Iterator is a generator.

Intersection and DNF types: recognize them for now

`Countable&Iterator` requires an object to satisfy both interfaces. DNF types combine unions and intersections with allowed parentheses, such as `(A&B)|null`.

These contracts matter in advanced library design, but do not force them into beginner code. Return after learning interfaces and OOP.

Complete example: order total

```
<?php
declare(strict_types=1);

function readPositiveInt(mixed $value): ?int
{
    $result = filter_var($value, FILTER_VALIDATE_INT, [
        'options' => ['min_range' => 1],
    ]);

    return $result === false ? null : $result;
}

$priceCents = readPositiveInt($_GET['price_cents'] ?? null);
$quantity = readPositiveInt($_GET['quantity'] ?? null);

if ($priceCents === null || $quantity === null) {
    http_response_code(400);
    echo 'price_cents and quantity must be positive integers';
    exit;
}

$totalCents = $priceCents * $quantity;
echo "Total: {$totalCents} cents";
```

Test a valid request, alphabetic price, zero quantity, and a missing value. The example connects untrusted `mixed` input, validation, nullable results, strict comparison, and integer money.

Common mistakes

- Using `==` and being surprised by coercion.
- Storing money in floats without a precision decision.
- Assuming every `$_GET` value is one string; a client can send an array.
- Byte-indexing Arabic or other multibyte text.
- Accepting `mixed` everywhere instead of narrowing at boundaries.
- Adding a broad union to hide unclear design.
- Building a huge array when values could be streamed.

Check your understanding

01 **How do `0`, `'0'`, `false`, and `null` differ?**

> **Answer:** They are integer, string, Boolean, and null values. Some Boolean contexts or loose comparisons make them look similar, but their types and domain meanings differ.

02 **Predict `var_dump('5' === 5); var_dump('5' == 5);`.**

> **Answer:** The strict comparison is `false`; the loose comparison is `true` after coercion.

03 **Why can `0.1 + 0.2 === 0.3` be false?**

> **Answer:** Finite binary floating-point cannot exactly represent every decimal fraction. Compare with a domain-appropriate tolerance or use exact decimal/integer representation.

04 **When should you use `array_key_exists()` rather than `isset()`?**

> **Answer:** When key presence matters even if the stored value is `null`.

05 **How should an application narrow a `mixed` price from an external request?**

> **Answer:** Accept uncertainty only at the boundary, validate and convert to an integer number of cents, and let internal domain functions require that clear `int` contract.

06 **Write a generator that yields even numbers from 2 to a maximum.**

> **Solution outline:** Start a loop at 2, increment by 2, and `yield` each number while it is below or equal to the maximum.

Summary

A type gives a value meaning and defines useful operations. Prefer strict comparison, validate strings before conversion, and make an explicit money-precision choice. Arrays model lists and maps; objects and enums introduce domain-specific types. Declarations and strict types clarify function boundaries, while advanced types should be used when their problem actually appears.

Run and verify

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Execute this checkpoint inside the lesson environment:

```
php types-lab.php
```

Success criterion: Valid cases pass and strict mode rejects the wrong type rather than silently coercing it; record the error type and exit code.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

Connect the ideas

Study in three passes: scalar/null, arrays/objects/enums, then advanced declarations such as union/intersection/never. Test numeric limits, numeric strings, NaN, and INF, and do not use an advanced type without a contract requiring it. Serialization is a separate storage and trust concern.

Try it yourself

Create a case matrix showing value and type before and after each conversion.

4. Output, debugging, and constants

Produce user output, inspect values during a failure, and understand when a constant is more appropriate than a variable.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Runtime:** The period when a program is actually running.
- **Debugger:** A tool that pauses a program so you can inspect values and execution step by step.
- **HTTP:** The rules used to exchange requests and responses on the web.
- **API:** A defined interface through which one program requests data or actions from another.
- **Session:** Temporary server-side state used to recognize a user across requests.
- **CLI:** A text-based interface controlled by typed commands.
- **UTF-8:** A common encoding that stores Unicode numbers as bytes.
- **Function:** A named, reusable block of code with one defined job.

User output and developer evidence are different

While learning, you need to see both program output and temporary evidence about internal state. `echo` produces output; `var_dump()` and related tools expose types and structure for diagnosis. Mixing debug text into a production response can leak sensitive data or corrupt HTML and JSON.

This lesson also introduces constants: named values that should not change during execution, such as an application name or a known HTTP status.

`echo`

```
$name = 'Omar';
echo 'Hello ', $name, PHP_EOL;
```

`echo` is a language construct, not a normal function. Parentheses are optional and it accepts multiple comma-separated values. Concatenation with `.` creates one string first:

```php
$message = 'Hello ' . $name;
echo $message;
```

Inside HTML, `<?= ... ?>` is the output shorthand:

```php
<h1><?= htmlspecialchars($name, ENT_QUOTES, 'UTF-8') ?></h1>
```

Encode untrusted values for the exact output context rather than printing them directly.

`print`

```php
$result = print 'Printed';
var_dump($result); // int(1)
```

Property	`echo`	`print`
Return value	none	always `1`
Values	several without parentheses	one
In an expression	no	technically yes
Template shorthand	`<?= ... ?>`	none

The performance difference is not a useful design criterion. Prefer `echo` in ordinary output and recognize `print` in existing code.

CLI, HTML, and JSON output

`PHP_EOL` creates a terminal line break. HTML needs semantic elements such as `<p>` or `<br>`. JSON responses must contain valid JSON without debug text around them:

```php
header('Content-Type: application/json; charset=utf-8');
echo json_encode(['status' => 'ok'], JSON_THROW_ON_ERROR);
```

A `var_dump()` before that JSON makes the response invalid even if the correct object follows it.

Why `echo` is not enough for debugging

```
$value = false;

echo $value;      // almost no visible evidence
var_dump($value); // bool(false)
```

Arrays and objects also need structural inspection rather than ordinary output.

```
$user = [
    'id' => 7,
    'active' => true,
    'roles' => ['editor', 'reviewer'],
];

var_dump($user);
print_r($user);
echo get_debug_type($user);
```

- `var_dump()` includes type, value, size, and nested detail.
- `print_r()` is visually simple for arrays and objects but less precise about types.
- `print_r($value, true)` returns its representation as a string.
- `get_debug_type()` returns a useful type name, especially for objects and resources.

Debug with a hypothesis

This calculation subtracts ten cents rather than ten percent:

```
$priceCents = 10000;
$discountPercent = 10;
$finalCents = $priceCents - $discountPercent;
```

Do not change random lines. State the expected result (`9000`), inspect value types, isolate the formula, correct it, and add a test:

```
$discountCents = (int) round($priceCents * ($discountPercent / 100));
$finalCents = $priceCents - $discountCents;

var_dump($priceCents, $discountPercent, $discountCents, $finalCents);
```

Framework dumps and logging

Frameworks often provide `dump()` and `dd()` (“dump and die”). They are development tools; `dd()` stops a request, and either helper can expose tokens, cookies, or customer data.

Use structured logs in production:

```
error_log(json_encode([
    'event' => 'checkout_failed',
    'order_id' => 42,
    'reason' => 'payment_timeout',
], JSON_THROW_ON_ERROR));
```

Never log passwords, session identifiers, or access tokens. Return a safe user message and keep protected technical detail in the log.

What is a constant?

A variable can change; a constant should not change after definition:

```
const APP_NAME = 'Omar Notes';
echo APP_NAME;
```

Constants do not start with `$`. Use one for a truly code-level invariant. A database password varies by environment and is a secret, so it belongs in injected configuration or a secret manager, not a repository constant.

`const` and `define()`

```
const APP_NAME = 'Omar Notes';
define('APP_VERSION', '1.0.0');
```

- `const` is a language declaration.
- `define()` executes at runtime and can appear conditionally.
- `define()` creates a global constant, not a class constant.
- A local `const` cannot be placed in a function or conditional block.

Conditional global constants are legal but can create hidden state; a configuration object is usually clearer in a large application.

Class constants

```
final class HttpStatus
{
    public const OK = 200;
    public const NOT_FOUND = 404;
}

echo HttpStatus::NOT_FOUND;
```

A class constant keeps the name with its concept. An enum may be better when a closed set also needs values and behavior.

Magic and predefined constants

Magic constants are calculated from source context:

```
echo __LINE__, PHP_EOL;
echo __FILE__, PHP_EOL;
echo __DIR__, PHP_EOL;
echo __FUNCTION__, PHP_EOL;
echo __CLASS__, PHP_EOL;
echo __METHOD__, PHP_EOL;
echo __NAMESPACE__, PHP_EOL;
```

`__DIR__` makes include paths independent of the current working directory:

```
$config = require __DIR__ . '/../config/app.php';
```

PHP and extensions also provide constants such as `PHP_VERSION`, `PHP_OS_FAMILY`, `PHP_INT_MAX`, and `PHP_EOL`. If an extension-defined constant is optional, check `defined('NAME')` before using it.

Complete example

```php
<?php
declare(strict_types=1);

const TAX_RATE = 0.14;

$priceCents = 25000;
$taxCents = (int) round($priceCents * TAX_RATE);
$totalCents = $priceCents + $taxCents;

if (getenv('APP_DEBUG') === '1') {
    var_dump([
        'file' => __FILE__,
        'price_cents' => $priceCents,
        'tax_cents' => $taxCents,
    ]);
}

echo "Total: {$totalCents} cents", PHP_EOL;
```

Run it with and without `APP_DEBUG=1`. Core output remains, while internal evidence appears only in the intended development mode.

Common mistakes

- Using `echo` to inspect booleans or arrays.
- Leaving `var_dump()` or `dd()` in a production API.
- Logging secrets while diagnosing a failure.
- Hard-coding environment configuration as constants.
- Building relative paths from the working directory rather than `__DIR__`.
- Defining broad global constant names that can collide.

Check your understanding

01 **What is output and what value is assigned by `$result = print 'Hi';`?**

> **Answer:** It outputs `Hi` and assigns integer `1` to `$result`.

02 **Why is `echo false;` weak debugging evidence?**

> **Answer:** It produces almost no visible output. `var_dump(false)` preserves both type and value.

03 **A JSON endpoint returns invalid JSON. What should you inspect first?**

> **Answer:** Look for dumps, warnings, or HTML around the payload and ensure the endpoint returns one JSON document with the right Content-Type.

04 **Why use `__DIR__` for an include path?**

> **Answer:** It anchors the path to the source file rather than whichever directory started the process.

05 **Should a database password be a source-code constant?**

> **Answer:** No. It is secret, environment-specific, and rotated independently; inject it through protected configuration.

06 **Correct `$final = $priceCents - $percent;`.**

> **Answer:** Calculate the discount amount from the percentage, define rounding policy, subtract it, and test 0%, 100%, and a normal value.

Summary

`echo` and `print` produce output; debugging tools reveal type and structure. Keep user responses separate from developer evidence and never expose secrets. Constants model code-level invariants, while `__DIR__` is a practical way to construct dependable file paths.

Run and verify

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Execute this checkpoint inside the lesson environment:

```
php debug-lab.php 2> debug.log
```

Success criterion: Intended output goes to stdout and diagnostics to stderr or a logger, with no secret value exposed in either stream.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

Connect the ideas

In development, use Xdebug or another debugger for stacks and breakpoints. In production, disable display_errors and send detail to protected logging with a correlation ID. Constants suit structural values, while environment configuration is not a class constant merely because it is currently stable.

Try it yourself

Test a failure in development and production and confirm users never see a stack trace.

5. Conditions and loops

if, switch, match, for, while, do-while, and foreach with important differences.

The problem: choose a path, then repeat

A grade sheet gives 95 an A, 40 an F, and rejects 150. Then we repeat that decision for every student. **Control flow** is execution's route through instructions; a **condition** is a question producing `true` or `false`, and a **loop** repeats instructions. Study branching first, then trace repetition on paper. Every complete program here runs on PHP 8.0+ in its own file.

if, elseif, else: the first matching branch

A **block** groups instructions inside `{ }`. A **branch** is a selected path. Read `if` as "if," `elseif` as "otherwise if," and `else` as "in all remaining cases." PHP checks top to bottom and selects the first true branch in the chain. Save `grade.php` and run `php grade.php`:

```
<?php
$score = 75;
if ($score < 0 || $score > 100) {
    $grade = 'invalid';
} elseif ($score >= 90) {
    $grade = 'A';
} elseif ($score >= 75) {
    $grade = 'B';
} elseif ($score >= 50) {
    $grade = 'C';
} else {
    $grade = 'F';
}
echo "Score {$score}: {$grade}", PHP_EOL;
```

```
Score 75: B
```

`$score` is the input. `||` means "or," rejecting values below 0 or above 100. For 75, the first condition is false, the 90 check is false, and the 75 check is true. We store B and skip later branches. `echo` is outside the chain and runs after any branch.

Input	Result
-1 / 101	invalid
0 / 49	F
50 / 74	C
75 / 89	B
90 / 100	A

Mistake and correction: starting with `$score >= 50` gives 95 a C. Check higher thresholds first. Separate `if` statements can overwrite the result several times; keep alternatives in a chain and reserve independent conditions for independent actions. `>=` includes the boundary; `>` excludes it.

Short forms for choosing a value

`$active ? 'active' : 'paused'` is ternary: condition, true result, false result. Use it for a short choice. `$input['name'] ?? 'guest'` replaces only missing keys or `null`, preserving `0` and `''`. The shorter `?:` uses Boolean conversion and also replaces `'0'`.

`$user?->address?->city` uses nullsafe access: it stops at `null` in an object chain, not at an exception or a missing array key. See OOP for objects. In HTML you may use `if (...): ... else: ... endif;` instead of braces, but do not mix styles within one block.

switch and match

`switch` selects a `case` by a value. `break` prevents continuing into the next case; that continuation is called **fall-through**. PHP 8.0's `match` is an expression returning a value, compares type and value with `===`, and never falls through. Run `status.php`:

```
<?php
$status = '200';
switch ($status) {
    case 200:
        echo "switch: success", PHP_EOL;
        break;
    default:
        echo "switch: other", PHP_EOL;
}
$message = match ($status) {
    200, 201 => 'success',
    default => 'other',
};
echo "match: {$message}", PHP_EOL;
```

```
switch: success
match: other
```

`switch` compares loosely, matching the string `'200'` to integer `200`. `match` does not. An **arm** supplies a matching value and a result after `=>`; multiple values can share one result. No match and no `default` raises `UnhandledMatchError`. `match (true)` can use range conditions such as `$age < 18`, although `if` is often clearer. Validate input before converting its type.

for: initialize, test, run the body, update

A **counter** tracks iterations; an **accumulator** collects the result. Save and run `sum.php`:

```
<?php
$total = 0;
for ($i = 1; $i <= 3; $i++) {
    $total += $i;
    echo "i={$i}, total={$total}", PHP_EOL;
}
echo "after: i={$i}, total={$total}", PHP_EOL;
```

```
i=1, total=1
i=2, total=3
i=3, total=6
after: i=4, total=6
```

`$total = 0` runs once; putting it in the body resets the total on every iteration. `$i = 1` initializes once, then `$i <= 3` is tested before each iteration. The body adds and prints; `$i++` increments afterward. At 4 we exit. Do not put `;` after `for (...)`: it creates an empty body.

Before body	Total before	Total after addition	After increment
i=1	0	1	i=2
i=2	1	3	i=3
i=3	3	6	i=4

A three-element list has keys 0, 1, 2, so use `$i < count($names)`, not `<=`. One extra or missing iteration is an **off-by-one error**. All `for` parts are optional; `for (;;)` cannot stop by itself. Templates may use `for (...): ... endfor;`.

while and do-while: when do we check?

`while` checks before its body and may run zero times. `do-while` checks afterward and runs at least once. `attempts.php` uses fixed inputs so the experiment is repeatable:

```php
<?php
$replies = ['', '', 'Omar'];
$attempt = 0;
do {
    $input = $replies[$attempt] ?? '';
    $attempt++;
    echo "attempt={$attempt}", PHP_EOL;
} while ($input === '' && $attempt < 3);
echo $input === '' ? "No name\n" : "Hello {$input}\n";

$remaining = 0;
while ($remaining > 0) {
    echo "work", PHP_EOL;
    $remaining--;
}
```

```
attempt=1
attempt=2
attempt=3
Hello Omar
```

We read an element, then increment before testing. `&&` requires both conditions: the name is empty and fewer than 3 attempts have occurred. The final loop never runs. With `fgets(STDIN)`, check `false` for end of input. Every loop needs progress and an exit condition; an increment on one path does not guarantee every path reaches it.

foreach, continue, and break

`foreach` visits elements without managing an index. `continue` skips the rest of the current iteration; `break` ends the loop. Run `scores.php`:

```php
<?php
$scores = ['Ali' => 0, 'Mona' => -1, 'Omar' => 100, 'Nour' => 80];
foreach ($scores as $name => $score) {
    if ($score < 0) {
        continue;
    }
    echo "{$name}: {$score}", PHP_EOL;
    if ($score === 100) {
        break;
    }
}
```

```
Ali: 0
Omar: 100
```

`$name => $score` reads key and value. Zero is valid; Mona is skipped and Nour is never visited. For rows you may use `foreach ($users as ['id' => $id, 'name' => $name])` when fields exist. `break 2` exits two levels; draw the nesting first. `continue` inside a nested `switch` can target the wrong level; simplify the structure or specify the level carefully.

`foreach ($prices as &$price)` modifies original elements by reference. Afterward `$price` still points to the last element: call `unset($price)` immediately. Start with iteration by value and use references only for deliberate mutation.

Predict, debug, complete

▼ Predict grades 49, 50, 75, 90, and a loop starting at 4 with i <= 3

F, C, B, A. The loop runs zero times because its condition is checked before the body. Test both sides of each boundary.

▼ Debug: $i = 0; while ($i < 3) { if ($i === 1) continue; $i++; }

The counter gets stuck at 1 because continue skips the increment. Move progress before the skip, or use for with its update in the third part; for still executes that update after continue.

▼ Complete a loop summing even integers from 1 through 6

Start total at zero and iterate 1 through 6 inclusive, then `if ($i % 2 === 0) { $total += $i; }`. Zero remainder means even, so the result is 12.

▼ After foreach by reference, you assign $price = 99 without unset. What changes?

The last element becomes 99. Call `unset($price)` after the loop; removing the reference variable does not delete the array element.

A step toward the project

The notebook in lesson 17 will choose between an error and saving a submitted form, then display elements using a loop. List missing, empty, and valid input cases now. For extra practice run `php control-flow-lab.php` from the existing lab and check zero, one, and several elements.

6. Expressions and operators

Values, precedence, arithmetic, assignment, comparison, logic, arrays, execution, and pipes.

The problem: plausible arithmetic, wrong results

A price is 1500 minor units and quantity is 3. We need a subtotal, a discount, and a free-shipping decision. An **expression** produces a value; an **operator** specifies an operation; an **operand** is a value it acts on. In `$price * $quantity`, `*` is the operator and the variables are its operands. Identify input and result types before writing an expression.

Core examples require PHP 8.0+. Save `checkout.php` and run `php checkout.php`:

```
<?php
declare(strict_types=1);

$price = 1500;
$quantity = 3;
$subtotal = $price * $quantity;
$discount = intdiv($subtotal * 10, 100);
$afterDiscount = $subtotal - $discount;
$shipping = $afterDiscount >= 4000 ? 0 : 500;
$total = $afterDiscount + $shipping;
echo "subtotal={$subtotal}", PHP_EOL;
echo "discount={$discount}", PHP_EOL;
echo "shipping={$shipping}", PHP_EOL;
echo "total={$total}", PHP_EOL;
```

```
subtotal=4500
discount=450
shipping=0
total=4050
```

The first variables are integers in minor units. Multiplication produces 4500. `intdiv` performs integer division: this example explicitly rounds a positive discount down. We subtract 450 and compare 4050 with 4000; true selects zero shipping. Finally we add and print. These amounts are small; large values need overflow checks before multiplication, and other financial policies need explicit rounding or Decimal arithmetic.

Wrong: `$subtotal - 10` subtracts 10 minor units, not 10%. Testing `$subtotal >= 4000` checks eligibility before discount; choose the variable matching the store's rule. Change quantity to 2: the discounted subtotal is 2700, shipping is 500, and the result is 3200.

Precedence is not a guaranteed evaluation order

Precedence controls grouping: `2 + 3 * 4` means `2 + (3 * 4)` and produces 14. `(2 + 3) * 4` produces 20. Do not rely on evaluation order when expressions mutate the same variable, such as adding two increments of one counter; put each mutation on a separate line.

Save `precedence.php`:

```
<?php
$a = true && false;
$b = true and false;
var_dump($a, $b);
$x = 5;
echo $x++, PHP_EOL;
echo ++$x, PHP_EOL;
```

```
bool(false)
bool(true)
5
7
```

`&&` binds more tightly than assignment, so `$a` receives false. `and` binds less tightly: `($b = true) and false` leaves b true because the final result is not assigned. Prefer `&&`, `||`, and clear parentheses. `$x++` returns the old value then increments; `++$x` increments first. Assignment itself returns a value, so `$b = $a = 5` stores 5 in both, although two lines are often clearer.

An operation map

Group	Operators and meaning
Arithmetic	`+ - * /`, remainder `%`, power `**`
Compound assignment	`+= -= *= /= %= **= .= ??=`
Comparison	Loose `== != <>`, strict `=== !==`, ordering `< > <= >=`
Three-way comparison	`<=>` returns -1, 0, or 1 for simple numbers
Logic	`&&` and, `
Strings	`.` concatenates; `.=` concatenates and assigns

`/` may produce a float: `10 / 4` is 2.5, `intdiv(10, 4)` is 2, and `10 % 4` is 2. A zero divisor raises `DivisionByZeroError`. Floats approximate many decimal fractions; repeated decimal calculations need an explicit precision policy rather than blind exact equality.

Type is part of comparison

Run `compare.php`. **Short circuiting** means the second operand is not evaluated when the first already determines the result:

```php
<?php
var_dump('0' == 0);
var_dump('0' === 0);
var_dump('0' != 0);
var_dump('0' !== 0);
$value = '0';
echo $value ?? 'missing', PHP_EOL;
echo $value ?: 'empty', PHP_EOL;
$divisor = 0;
var_dump($divisor !== 0 && 10 / $divisor > 2);
```

```
bool(true)
bool(false)
bool(false)
bool(true)
0
empty
bool(false)
```

The first four comparisons distinguish string and integer value/type. `??` preserves `'0'`, whereas `?:` treats it as falsy. The last line's first operand is false, so division never occurs. Do not put essential saving or counting inside an operand that may be skipped. `$input ??= 'guest'` assigns a fallback only when missing or null.

The same symbol with arrays or objects

Array `+` is a key-based union: the left value wins on duplicate keys. `array_merge` replaces string keys with right-hand values and renumbers numeric keys. Try:

```php
<?php
$left = ['timeout' => 3, 0 => 'A'];
$right = ['timeout' => 5, 0 => 'B'];
echo json_encode($left + $right, JSON_THROW_ON_ERROR), PHP_EOL;
echo json_encode(array_merge($left, $right), JSON_THROW_ON_ERROR), PHP_EOL;
```

```
{"timeout":3,"0":"A"}
{"timeout":5,"0":"A","1":"B"}
```

Array `===` also checks types and key/value order; `==` does not require the same order. `instanceof` checks an object's class/interface. Assigning an object to another variable does not copy it; `clone` is shallow and nested objects may remain shared. See OOP.

Operators do not repair hidden failures

`@` may suppress an expression's diagnostics but cannot fix the failure. Check return values or handle exceptions. PHP backticks execute a shell command like `shell_exec`; they do not display a string. Never insert user input into `exec`, `system`, or `proc_open`. Prefer a direct API, or fixed allowed commands with separate arguments when process execution is necessary.

Pipe requires PHP 8.5

A **callable** refers to something that can be invoked; lesson 7 explains it. `|>` passes its left value as one argument to the right callable. Run `pipe.php` with PHP 8.5; the file cannot be parsed by 8.4:

```php
<?php
$slug = ' Hello PHP '
    |> trim(...)
    |> strtolower(...)
    |> (fn (string $s): string => str_replace(' ', '-', $s));
echo $slug, PHP_EOL;
```

```
hello-php
```

Each stage returns a new value. An 8.0 alternative is `str_replace(' ', '-', strtolower(trim(' Hello PHP ')))`; it does not need first-class callable syntax. See the official pipe reference.

Predict, debug, complete

▼ Predict 2 + 3 * 4, then (2 + 3) * 4

14, then 20. Parentheses change grouping, not the numbers themselves.

▼ Debug: if ($quantity = 0) instead of comparison

Assignment stores zero and the condition becomes false. Use `$quantity === 0` after checking that input is an integer. Do not switch to == just to hide a type mismatch.

▼ Complete a safe division condition with divisor=0

`$divisor !== 0 && $amount / $divisor > 2`. Checking zero first prevents evaluating division; reversing operands loses that protection.

▼ Why does ['timeout' => 3] + ['timeout' => 5] not update it to 5?

Union preserves the left value. For an override, use array_merge or unpacking with the intended key rules.

Run `php operators-lab.php` from the lab. Our notebook project uses `===` for the HTTP method and `??` for missing keys; truthiness cannot replace type validation.

7. Functions, callbacks, and includes

Parameters, references, closures, arrow functions, callables, include, and require.

The problem: the same calculation in five places

When a price rule changes, updating five copies risks missing one. A **function** groups a task under a name, receives input, and returns a result. A **parameter** names input in the definition, an **argument** is a value passed during a call, and a **return value** is the result sent back.

Start with a calculation that returns rather than prints. It can then serve CLI output, HTML, and tests. Save `functions.php` for PHP 8.0+ and run `php functions.php`:

```
<?php
declare(strict_types=1);

function subtotal(int $price, int $quantity = 1): int
{
    if ($price < 0 || $quantity < 1) {
        throw new InvalidArgumentException('Invalid order');
    }
    return $price * $quantity;
}
function receipt(int $price, int $quantity): string
{
    $amount = subtotal($price, $quantity);
    return "Total: {$amount}";
}
echo receipt(1500, 3), PHP_EOL;
echo subtotal(quantity: 2, price: 500), PHP_EOL;
```

```
Total: 4500
1000
```

`declare` makes scalar calls originating in this file strict. `function subtotal` defines a function; its body does not run just because PHP reads the definition. `int` before a parameter specifies input type; after the parentheses it specifies result type. `= 1` supplies quantity when omitted. The initial condition rejects values outside the contract; `throw` aborts that path with a failure, explained in lesson 11. `return` ends the call and sends back the product. The second function builds a message. The final line uses **named arguments**, so renaming a parameter can break callers.

PHP does not overload functions in one namespace by their argument count. Use defaults, union types, or variadics when appropriate. These numbers are small; extending the program requires checking multiplication bounds.

The call stack: where execution goes and returns

The **call stack** holds active calls. Each call adds a **frame** containing its local state and return location. The last function entered is the first to return. Trace `receipt(1500, 3)`:

```
main
main → receipt(price=1500, quantity=3)
main → receipt → subtotal(price=1500, quantity=3)
main → receipt(amount=4500)
main → echo("Total: 4500")
```

`$amount` is local to receipt; matching variable names in different frames are not the same variable. `return` resumes the caller, not the start of the program. **Recursion** is a function calling itself: each call adds a frame, so it needs a stopping case and bounded input. In `countdown.php`:

```
<?php
function countdown(int $n): void
{
    if ($n < 0 || $n > 10) {
        throw new InvalidArgumentException('Use 0..10');
    }
    if ($n === 0) {
        echo "go", PHP_EOL;
        return;
    }
    echo $n, PHP_EOL;
    countdown($n - 1);
}
countdown(3);
```

```
3
2
1
go
```

`void` means no useful returned value. The zero check is the **base case**, and subtracting one moves toward it; removing either can exhaust memory. A loop is often better for simple repetition. Understanding frames also makes an error's stack trace readable.

Values, references, and variadics

Ordinary passing does not let a function reassign the caller's variable. `&` allows that mutation; use it deliberately. `...` in a parameter collects a variable number of arguments into an array. Run `arguments.php`:

```php
<?php
function increment(int $n): int { return $n + 1; }
function incrementInPlace(int &$n): void { $n++; }
function sum(int ...$numbers): int { return array_sum($numbers); }

$n = 4;
echo increment($n), ':', $n, PHP_EOL;
incrementInPlace($n);
echo $n, PHP_EOL;
echo sum(...[2, 3, 4]), PHP_EOL;
```

```
5:4
5
9
```

The first returns 5 while leaving n=4; the second changes n to 5. At the call site `...` unpacks a list into arguments. Mutating an object differs from reassigning a variable: a function may change the shared object's state without `&`; the OOP track covers this distinction.

Callback: pass the operation, not its result

A **callable** is a value that can be invoked. A **callback** is a function passed to other code, which decides when to call it. It does not automatically mean delayed or parallel work: `array_map` invokes callbacks synchronously for each element. In `callbacks.php` we write the caller ourselves:

```php
<?php
declare(strict_types=1);

function transform(array $values, callable $operation): array
{
    $result = [];
    foreach ($values as $value) {
        $result[] = $operation($value);
    }
    return $result;
}
$factor = 2;
$double = function (int $n) use ($factor): int {
    return $n * $factor;
};
$factor = 10;
$values = transform([1, 2, 3], $double);
echo implode(',', $values), PHP_EOL;
$plusOne = fn (int $n): int => $n + 1;
echo implode(',', transform($values, $plusOne)), PHP_EOL;
```

```
2,4,6
3,5,7
```

`transform` receives an array and an operation; it does not know whether that operation multiplies or adds. Each iteration invokes `$operation($value)` and appends its result. `function (...) use ($factor)` is an anonymous function, a **closure**, capturing factor by value when created: 2 even after the variable becomes 10. `use (&$factor)`

instead observes the shared variable; use it only intentionally. `fn` is an arrow function with one expression, implicit return, and by-value capture.

Wrong: passing `$double(3)` to transform passes the integer 6, not a function. Pass `$double` without invoking it. A named function can be referenced by a string such as `'trim'` (a variable function), or by `trim(...)` from PHP 8.1 as a first-class callable. Then invoke that value. Use `is_callable` for uncertain sources and never let users select arbitrary function names. Prefer a local closure over declaring a named function inside a function: the inner declaration occurs only after calling the outer function, shares function-name scope, and may be redeclared.

A two-file project: require executes a file

Create a folder containing `config.php` and `main.php`. This is a complete project without Composer. The first returns data; the second needs it:

config.php

```
<?php
return ['name' => 'Notebook', 'limit' => 3];
```

main.php

```
<?php
$config = require __DIR__ . '/config.php';
echo $config['name'], ': ', $config['limit'], PHP_EOL;
```

```
Notebook: 3
```

Run `php main.php`. `__DIR__` is the current file's directory, independent of the working directory. `require` executes the file and receives its return value. Without an explicit return, successful inclusion normally returns 1. Included code inherits the inclusion scope, so avoid hidden dependencies on outside variables.

`include` emits a warning on failure and usually continues. Modern PHP's `require` raises an Error, interrupting the current path unless handled. The `_once` versions prevent repeated definitions. Do not expect repeated require_once calls for configuration to return the same array; a later call may return true. Never build inclusion paths from user input. Lesson 10 moves class loading to Composer.

`goto` jumps to a label in the same file and scope; it cannot enter a loop or switch from outside. `goto done; echo 'skip'; done: echo 'done';` prints done, but a small function or break is usually easier to follow.

Predict, debug, complete

▼ Predict: factor changes from 2 to 10 after creating the closure

With use ($factor), results remain 2,4,6 because capture was by value. With use (&$factor), they become 10,20,30. Arrow functions also capture by value.

▼ Debug: a function echoes a value and I try to add its result

Printing does not return the value for arithmetic. Return the result and echo it at the presentation boundary; the function becomes testable without capturing output.

▼ Complete a callback that squares an integer

`fn (int $n): int => $n * $n`. Pass it to transform with [1,2,3] to obtain [1,4,9]. Do not invoke it before passing it.

▼ Diagnose recursion calling countdown($n) without changing n

Each call adds a frame with the same n and never reaches zero. Pass n-1 and keep both a base case and an input bound; writing a base case alone does not guarantee progress.

Our notebook will separate validation, storage, and presentation into functions, then pass handlers as callbacks to the router. Also run `php functions-lab.php` from the lab.

8. Files, streams, JSON, and CSV

Stream wrappers and contexts, open modes, permissions, JSON, serialization, and CSV.

The problem: variables disappear when the program stops

Memory is temporary. To keep a note across runs, write it to a file. A file stores bytes; the program opens it, reads or writes, then closes it. A **stream** is an interface for moving data, like a window through which you read one piece at a time. A **handle** returned by `fopen` identifies the opened resource; it is not the file contents.

A **cursor** is the current read/write position, a **chunk** is a bounded piece of bytes, and a **buffer** temporarily collects data. These distinctions help avoid loading an entire large file just to count its lines. Examples need PHP 8.0+. `finally` performs cleanup after an attempt even if it fails; lesson 11 explains it further.

First complete experiment without persistent files

Save `stream.php` and run `php stream.php`. `php://temp` begins in memory and may spill into a temporary file after its memory threshold:

```
<?php
$stream = fopen('php://temp', 'w+b');
if ($stream === false) {
    throw new RuntimeException('Cannot open stream');
}
try {
    $text = "Ali\nMona\n";
    $offset = 0;
    while ($offset < strlen($text)) {
        $written = fwrite($stream, substr($text, $offset));
        if ($written === false || $written === 0) {
            throw new RuntimeException('Write made no progress');
        }
        $offset += $written;
    }
    echo 'position=', ftell($stream), PHP_EOL;
    if (!rewind($stream)) {
        throw new RuntimeException('Cannot rewind');
    }
    $count = 0;
    while (($line = fgets($stream)) !== false) {
        $count++;
        echo $count, ': ', rtrim($line, "\r\n"), PHP_EOL;
    }
    if (!feof($stream)) {
        throw new RuntimeException('Read failed before EOF');
    }
    echo "count={$count}", PHP_EOL;
} finally {
    fclose($stream);
}
```

```
position=9
1: Ali
2: Mona
count=2
```

The first line opens a resource; strict comparison detects failure. The text has 9 bytes: 3 for Ali, a newline, 4 for Mona, and a newline. `fwrite` reports bytes written; the offset loop retries **only the remainder** and stops on zero progress. `ftell` shows the cursor at the end. `rewind` moves it back; reading immediately after writing might otherwise find nothing.

`fgets` reads a line and `!== false` distinguishes valid text such as `'0'` from failure. This `rtrim` removes line endings only, preserving intentional spaces. Checking `feof` afterward separates normal end-of-file from a read error. `fclose` in finally releases the resource. Line-by-line memory depends on the longest line; one enormous line can still be expensive. `fread($stream, 8192)` reads bounded chunks instead.

Choose the opening mode before writing

Mode	Position and effect
`r / r+`	File must exist; read / read and write from the start
`w / w+`	Truncate immediately on opening, or create
`a / a+`	Create if needed; writes always append
`x / x+`	Exclusive creation; fail if already present
`c / c+`	Create if needed without truncating; start at the beginning

`+` enables both reading and writing; `b` avoids platform-specific text translations. **Serious mistake:** opening with `w` before acquiring a lock truncates the file before protection begins. For an existing-data update, use `c+` and lock before reading/modifying. Existence and `is_writable` are preliminary checks; the operation itself may still fail.

JSON is a representation, not semantic validation

Serialization turns data into a storable representation. JSON works for simple values shared across languages. Save `notes-json.php` in a writable practice folder; this example replaces `notes-demo.json` there:

```php
<?php
$path = __DIR__ . '/notes-demo.json';
$notes = [['name' => 'Omar', 'text' => 'Learn streams']];
$json = json_encode($notes, JSON_THROW_ON_ERROR);
$bytes = file_put_contents($path, $json, LOCK_EX);
if ($bytes === false || $bytes !== strlen($json)) {
    throw new RuntimeException('Save failed');
}
$raw = file_get_contents($path);
if ($raw === false) {
    throw new RuntimeException('Read failed');
}
$loaded = json_decode($raw, true, flags: JSON_THROW_ON_ERROR);
if (!is_array($loaded) || !is_string($loaded[0]['text'] ?? null)) {
    throw new RuntimeException('Unexpected data shape');
}
echo $loaded[0]['text'], PHP_EOL;
```

```
Learn streams
```

`__DIR__` anchors the path. `json_encode` converts the array into text; `JSON_THROW_ON_ERROR` prevents silent failure. `file_put_contents` suits small files and returns bytes written. `LOCK_EX` coordinates cooperating writers; it does not protect an entire read→modify→write cycle or a reader that takes no lock.

`json_decode(..., true)` turns JSON objects into arrays, but valid JSON may also be a number or null, so shape validation is separate. PHP 8.3's `json_validate` is useful when syntax validation alone is needed; do not validate and then decode unnecessarily. `JSON_UNESCAPED_UNICODE` changes presentation, not meaning. Never use `unserialize` with untrusted input; `allowed_classes` does not make arbitrary hostile serialized input safe.

CSV: a comma can belong inside a field

CSV is a text table whose comma-containing or quoted fields follow quoting rules. `explode(',')` cannot handle those rules. Save `csv.php`:

```php
<?php
$stream = fopen('php://temp', 'w+b');
if ($stream === false) {
    throw new RuntimeException('Cannot open CSV');
}
try {
    $expected = ['a,b', 'say "hi"', 'back\\slash', 'عمر'];
    if (fputcsv($stream, $expected, escape: '') === false) {
        throw new RuntimeException('CSV write failed');
    }
    if (!rewind($stream)) {
        throw new RuntimeException('Cannot rewind');
    }
    $actual = fgetcsv($stream, escape: '');
    if ($actual !== $expected) {
        throw new RuntimeException('CSV round-trip failed');
    }
    echo "CSV round-trip OK", PHP_EOL;
} finally {
    fclose($stream);
}
```

```
CSV round-trip OK
```

The program writes, reads back, and checks exact types and values. Explicit `escape: ''` avoids the default deprecated in 8.4 and uses conventional doubled quotes. `str_getcsv` parses an already-loaded text row. Agree on encoding and delimiter, then validate column count and types. Spreadsheet exports also need a policy for values that an application might interpret as formulas.

Wrapper, context, and filter solve different problems

A **wrapper** maps an address to a source: `file://` files, `php://memory` memory only, `php://temp` memory then temporary storage, and `php://input` raw HTTP request bodies. Other wrappers include `http://`, `ftp://`, `data://`, and `compress.zlib://` depending on extensions/settings. Custom wrappers can use `stream_wrapper_register`, but beginners do not need to implement one.

A **context** configures an operation, such as a timeout or HTTP headers; it does not make a failed request successful. A **filter** transforms bytes as they pass. Here is complete `filter.php`:

```php
<?php
$stream = fopen('php://temp', 'w+b');
if ($stream === false) {
    throw new RuntimeException('Open failed');
}
try {
    if (fwrite($stream, 'hello') !== 5 || !rewind($stream)) {
        throw new RuntimeException('Prepare failed');
    }
    $filter = stream_filter_append($stream, 'string.toupper', STREAM_FILTER_READ);
    if ($filter === false) {
        throw new RuntimeException('Filter failed');
    }
    $text = stream_get_contents($stream);
    if ($text === false) {
        throw new RuntimeException('Read failed');
    }
    echo $text, PHP_EOL;
} finally {
    fclose($stream);
}
```

```
HELLO
```

The read filter transforms the returned bytes; this ASCII filter is not a Unicode case-conversion tool. An HTTP context could be `stream_context_create(['http' => ['timeout' => 3, 'header' => "Accept: application/json\r\n"]])`, passed to the reading function. Real requests also need status, TLS, and size checks; `ignore_errors` permits reading an error response body, not treating it as success. `allow_url_fopen` controls URL-aware wrappers. Keep `allow_url_include` disabled and do not let arbitrary user URLs access the server's network.

Storage and permissions

On Unix, read=4, write=2, execute=1 for owner/group/others. `0600` is owner-only, `0644` allows others to read, and `0755` is common for directories. `chmod($path, 0640)` does not replace configuring the service user; Windows uses ACLs. Do not solve permission errors with `0777`.

For appending records, acquire `flock(..., LOCK_EX)`, write all bytes, then `fflush` and release in finally. Cooperating readers take `LOCK_SH`. Replacing a complete file can use a temporary file on the same filesystem followed by rename, checking failures and platform behavior. `fflush` alone does not guarantee power-loss durability. A separate stable lock file is needed when coordinating processes that replace the data file itself.

The application chooses storage paths, not user-supplied names. `realpath` helps for existing paths but returns false for new files; check the parent and directory boundary, including symlink policy. Huge JSON needs a streaming parser, not merely fopen before json_decode.

Predict, debug, complete

▼ Predict: remove rewind from stream.php

The cursor remains at byte 9 after writing, so reading starts at the end and count is zero. Handles do not rewind themselves.

▼ Debug: if (!$raw) after file_get_contents

This confuses failure with empty text or '0'. Use `$raw === false`, then separately decide whether empty content is allowed.

▼ Complete a condition that stops a non-progressing write loop

`$written === false || $written === 0`. Checking only false leaves a possible infinite loop on zero progress.

▼ Does LOCK_EX on saving alone prevent two updates based on the same old data?

No. Both may read the old state and the second may overwrite the first change. Lock the complete read-modify-write cycle, or use a database transaction as the application grows.

Also run `php stream-lab.php fixtures/large.csv` from the lab. Lesson 17 stores each note in its own file in a private directory. Treat empty, missing, and malformed JSON files as distinct cases.

References: Streams, CSV, writing.

9. Uploads, cookies, and sessions

Secure file uploads, cookie options, CSRF, session lifecycle, and session security.

The problem: every request starts again

A browser opens a page and then submits a form, but local variables from the first request do not carry into the second. We need to associate requests. A **cookie** is a small value stored by the browser and sent with matching requests. A **session** stores server-side data associated with a random identifier; normally the browser holds only the identifier. An **upload** transfers file bytes to the server, a separate problem requiring input validation too.

This lesson is not a login system: it explains data transfer and persistent state. Identity and permissions belong in the security track. Examples require PHP 8.1+, mbstring, and fileinfo.

The session lifecycle, step by step

1. The browser sends an initial GET without a session cookie.
2. `session_start` opens state through the session handler. Without an accepted identifier, PHP creates one.
3. The server sends `Set-Cookie` in a **response header**, which the browser stores.
4. The next request sends `Cookie` in a **request header**.
5. `session_start` loads data into `$_SESSION`; the default file handler locks the session while it is in use.
6. `session_write_close` or request shutdown saves changes and releases the lock.

`setcookie` does not change `$_COOKIE` in the current request; that array reflects what already arrived. Start sessions before HTML or echo because session handling may send headers.

Complete program: a form remembers your name

Create `public/preferences.php`. Run `php -S 127.0.0.1:8081 -t public` and open the local form. The teaching server is for local use. Secure is explicitly false for local HTTP; use true for deployed HTTPS through trusted configuration, not a user-controlled header.

CSRF means another site causing your browser to submit an unwanted change. Here a **token** is a random value stored in both the session and form, checked before modifying state. It is neither a password nor the session ID.

```php
<?php
declare(strict_types=1);

if (!session_start([
    'use_strict_mode' => true,
    'use_only_cookies' => true,
    'cookie_httponly' => true,
    'cookie_secure' => false,
    'cookie_samesite' => 'Lax',
    'cookie_path' => '/',
])) {
    throw new RuntimeException('Session unavailable');
}
$_SESSION['csrf'] ??= bin2hex(random_bytes(32));
$error = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $token = $_POST['csrf'] ?? null;
    if (!is_string($token) || !hash_equals($_SESSION['csrf'], $token)) {
        http_response_code(403);
        exit('Invalid form token');
    }
    $name = $_POST['name'] ?? null;
    if (!is_string($name) || !mb_check_encoding($name, 'UTF-8')) {
        $error = 'Name must be UTF-8 text';
    } else {
        $name = trim($name);
        if ($name === '' || mb_strlen($name, 'UTF-8') > 40) {
            $error = 'Use 1 to 40 code points';
        } else {
            $_SESSION['name'] = $name;
            session_write_close();
            header('Location: /preferences.php', true, 303);
            exit;
        }
    }
    http_response_code(422);
}
$name = $_SESSION['name'] ?? 'Guest';
$csrf = $_SESSION['csrf'];
session_write_close();
function escape(string $value): string
{
    return htmlspecialchars($value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}
?>
<!doctype html>
<html lang="en"><meta charset="utf-8"><title>Preferences</title>
<p>Hello <?= escape($name) ?></p>
<p><?= escape($error) ?></p>
<form method="post">
  <input type="hidden" name="csrf" value="<?= escape($csrf) ?>">
  <label>Name <input name="name" maxlength="40" required></label>
  <button>Save</button>
</form>
</html>
```

Expect `Hello Guest` on the first visit. Submit Omar: POST returns 303, the browser follows with GET, and `Hello Omar` appears. A private window starts as Guest because it has another session. Submit only spaces: 422 with no name change. Remove the token: 403 before saving.

Read in order: configure/start the session, create a CSRF token once, recognize POST, check type before hash_equals, then validate UTF-8 and length. `maxlength` assists the browser but does not replace server validation; browsers and PHP can count different text units, covered in lesson 13. `303` implements **Post/Redirect/Get**, so refreshing repeats GET, not a general guarantee against duplicate POSTs. Copy needed values before closing the session early. `htmlspecialchars` encodes HTML text and quoted attribute output without changing stored data.

Cookie flags and their limits

Setting	Effect
Secure	Send over HTTPS; false is used only for the local example
HttpOnly	Blocks JavaScript from reading the cookie, not XSS-triggered requests
SameSite=Lax	Restricts cross-site sending; does not replace a CSRF token
SameSite=None	Requires Secure; use only for a concrete need
Path/Domain	Select matching requests; they are not authorization controls
expires / cookie_lifetime	Cookie lifetime, not the server-side validity policy by itself

Users can modify preference cookies such as theme; validate against `['light', 'dark']`. Do not store passwords or raw administrative authority in them. Keep sensitive data in appropriate storage and treat session identifiers themselves as secrets.

Session fixation: the attacker knows the ticket before login

Fixation convinces a victim to use an identifier known to the attacker, which remains valid after login. Theft instead obtains an existing victim's identifier. `use_strict_mode` rejects uninitialized identifiers but cannot solve every fixation scenario involving an existing session.

After credentials have genuinely been verified, rotate the identifier **before** storing the newly authenticated identity. The following is a login-boundary excerpt for an existing authentication system, not a working login system:

```
// After credentials were verified and a session was started:
if (!session_regenerate_id(false)) {
    throw new RuntimeException('Cannot rotate session');
}
$_SESSION['user_id'] = $verifiedUserId;
$_SESSION['authenticated_at'] = time();
```

`false` keeps old session data to avoid abruptly breaking concurrent requests, but it is not a complete revocation design. A real system needs obsolete markers/timestamps on old records, a short transition policy that denies new privileges to old identifiers, and server-enforced idle/absolute expiry. Immediate deletion with `true` can suit a

serialized demonstration but cause lost sessions and races on unreliable connections. Never display or log identifiers. Follow session security management and the session security lesson.

Ending a session is more than clearing a variable

In a CSRF-protected POST after session_start, clear `$_SESSION = []`, expire the cookie using the same Path, Domain, and flags, then call `session_destroy`. Destroy alone clears neither the browser cookie nor the local array. `session_unset` clears session variables only. `session_write_close` saves and unlocks; later `$_SESSION` changes are not automatically persisted. `session_start(['read_and_close' => true])` is for reading only, with attention to storage expiry policy.

Redis/database storage needs `SessionHandlerInterface` or `session_set_save_handler` plus explicit locking and expiry policies. Garbage collection is storage cleanup, not the authorization decision for a request.

Uploading an image into private storage

Create `public/upload.php` in the same folder; it creates `storage/uploads` outside public. **MIME** describes content type; neither browser-supplied type nor filename extension is trustworthy. `multipart/form-data` carries fields and files. This example allows PNG/JPEG up to 2 MiB and requires CSRF:

```
<?php
declare(strict_types=1);
session_start([
    'use_strict_mode' => true,
    'use_only_cookies' => true,
    'cookie_httponly' => true,
    'cookie_secure' => false,
    'cookie_samesite' => 'Lax',
]);
$_SESSION['upload_csrf'] ??= bin2hex(random_bytes(32));
$csrf = $_SESSION['upload_csrf'];
session_write_close();
$message = '';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $token = $_POST['csrf'] ?? null;
    if (!is_string($token) || !hash_equals($csrf, $token)) {
        http_response_code(403);
        exit('Invalid form token');
    }
    $file = $_FILES['avatar'] ?? null;
    if (!is_array($file)
        || ($file['error'] ?? null) !== UPLOAD_ERR_OK
        || !is_string($file['tmp_name'] ?? null)
        || !is_uploaded_file($file['tmp_name'])) {
        http_response_code(422);
        exit('Upload failed');
    }
    $size = filesize($file['tmp_name']);
    $mime = (new finfo(FILEINFO_MIME_TYPE))->file($file['tmp_name']);
    $extensions = ['image/png' => 'png', 'image/jpeg' => 'jpg'];
    if ($size === false || $size < 1 || $size > 2 * 1024 * 1024
        || !is_string($mime) || !isset($extensions[$mime])) {
        http_response_code(422);
        exit('Unsupported file');
    }
    $directory = dirname(__DIR__) . '/storage/uploads';
    if (!is_dir($directory) && !mkdir($directory, 0700, true) && !is_dir($directory)) {
        throw new RuntimeException('Storage unavailable');
    }
    $name = bin2hex(random_bytes(16)) . '.' . $extensions[$mime];
    if (!move_uploaded_file($file['tmp_name'], $directory . '/' . $name)) {
        throw new RuntimeException('Save failed');
    }
    $message = 'Saved privately';
}
?>
<!doctype html>
<html lang="en"><meta charset="utf-8"><title>Upload</title>
<p><?= $message ?></p>
<form method="post" enctype="multipart/form-data">
  <input type="hidden" name="csrf" value="<?= htmlspecialchars($csrf, ENT_QUOTES, 'UTF-8') ?>">
  <input type="file" name="avatar" accept="image/png,image/jpeg" required>
  <button>Upload</button>
</form>
</html>
```

Open `/upload.php`: a permitted image produces `Saved privately` and a generated filename; text renamed .jpg receives 422. Check the upload error and tmp_name shape before using it: an errored upload may have no file. `filesize` and `finfo` inspect the server's temporary file. `move_uploaded_file` moves a genuine HTTP upload. The success message is fixed, and the client filename never enters the path.

`accept` is only a UI hint. MIME detection does not establish that an image is safe for every use; actual display pipelines need decoding/re-encoding, dimension limits, and suitable inspection. Configure `upload_max_filesize` and a larger `post_max_size` to allow multipart overhead, plus a server body limit. Exceeding post_max_size can leave POST and FILES empty; identify 413 in the body-limit layer, as the project will demonstrate. See the upload reference.

Predict, debug, complete

▼ Predict: save a name, then open a private window

The original window retains the name; the private cookie jar starts as Guest. The name is on the server and the identifier connects requests to it.

▼ Debug: session_start after HTML output

Headers may already be sent, preventing the cookie header. Start the session before output and check for whitespace/BOM before the opening tag. Buffering is not a substitute for a clear order.

▼ Complete the defense against name[]=Omar

Check `is_string($name)` before trim or mb_strlen. A field name does not guarantee its type; the request can supply an array.

▼ An attacker knows an ID before login. Is HttpOnly enough?

No. HttpOnly controls JavaScript access. Use strict mode, rotate at privilege transitions, and invalidate old identifiers with a correct policy; this is fixation.

▼ Accept avatar.php.jpg because its submitted type says image/jpeg?

No. Check error, size, and content; generate the filename and store outside public. Rejected content must never reach the move step.

Preferences is the Form→Validation→Session stage. Keep it: lesson 17 adds Files and Router. `php tests.php security` in the lab is a supporting policy test, not a substitute for actual multipart requests.

10. Namespaces, Composer, and quality tools

Names, imports, aliases, resolution rules, and mapping a namespace to directories with Composer PSR-4.

Two separate problems: duplicate names and unloaded files

Your project and an external library both define a User class. A **namespace** gives the name an address, like identical street names in different cities. **Autoloading** solves another problem: who loads the file when PHP needs a class? **Composer** manages packages, resolves compatible libraries, and generates an autoloader.

We use a small class only as a container for a function; class design belongs in OOP. This project needs PHP 8.3+ and Composer 2. Check `php -v` and `composer --version` and run every command inside the project directory, not the documentation site directory.

The complete project

A ready copy lives in `examples/php-course/composer-demo` in the repository and the course examples download. Alternatively, create these four files with the exact casing:

```
composer-demo/
  composer.json
  bin/demo.php
  src/Billing/PriceCalculator.php
  tests/PriceCalculatorTest.php
```

1. composer.json: describe the project

`require` lists runtime requirements; `require-dev` lists development tools. These version ranges are deliberately compatible choices for the example, not a claim to be the latest versions:

```
{
  "name": "learning/price-demo",
  "type": "project",
  "require": {
    "php": "^8.3"
  },
  "require-dev": {
    "phpunit/phpunit": "^11.5",
    "phpstan/phpstan": "^2.1",
    "friendsofphp/php-cs-fixer": "^3.0"
  },
  "autoload": {
    "psr-4": {
      "App\\": "src/"
    }
  },
  "scripts": {
    "demo": "@php bin/demo.php",
    "test": "@php vendor/bin/phpunit --bootstrap vendor/autoload.php tests",
    "analyse": "@php vendor/bin/phpstan analyse src --level=6 --no-progress",
    "style": "@php vendor/bin/php-cs-fixer fix src --dry-run --diff --rules=@PSR12",
    "check": [
      "@test",
      "@analyse",
      "@style"
    ]
  },
  "config": {
    "platform": {
      "php": "8.3.0"
    }
  },
  "description": "A small PSR-4 teaching project with boundary tests and development tools"
}
```

In JSON, `App\\` encodes the string `App\` because backslashes are escaped. **PSR-4** maps a namespace prefix to a base directory. Composer removes `App\` and converts the rest into a path: `App\Billing\PriceCalculator` → `src/Billing/PriceCalculator.php`. PHP itself does not impose this directory structure; PSR-4 does.

2. The class: src/Billing/PriceCalculator.php

```
<?php

declare(strict_types=1);

namespace App\Billing;

final class PriceCalculator
{
    public function subtotal(int $price, int $quantity): int
    {
        if ($price < 0 || $quantity < 1 || $price > intdiv(PHP_INT_MAX, $quantity)) {
            throw new \InvalidArgumentException('Invalid order');
        }

        return $price * $quantity;
    }
}
```

`namespace` follows declare and precedes definitions. The full class name is now App\Billing\PriceCalculator. `public function` exposes a method on the object. The guard rejects negative prices and quantities below 1. `||` short-circuits, so zero quantity never reaches division. Comparing price with `intdiv(PHP_INT_MAX, $quantity)` rejects multiplication overflow **before** it happens. `\InvalidArgumentException` is a fully qualified global name.

3. Entry point: bin/demo.php

```
<?php
declare(strict_types=1);

use App\Billing\PriceCalculator;

require dirname(__DIR__) . '/vendor/autoload.php';

$calculator = new PriceCalculator();
echo $calculator->subtotal(1500, 3), PHP_EOL;
```

`use` is a local name alias; it neither reads a file nor constructs an object. Requiring `vendor/autoload.php` registers the loader; `new` requests the class, which Composer loads. `composer demo` runs `@php` using Composer's PHP executable. The result is `4500`.

4. Test behavior: tests/PriceCalculatorTest.php

A **test** is an automated experiment with input and an expected result. An **assertion** fails with a diagnostic when reality differs. PHPUnit discovers test methods; the chosen 11.5 line supports this project's PHP baseline:

```php
<?php
declare(strict_types=1);

use App\Billing\PriceCalculator;
use PHPUnit\Framework\TestCase;

final class PriceCalculatorTest extends TestCase
{
    public function testValidOrder(): void
    {
        self::assertSame(4500, (new PriceCalculator())->subtotal(1500, 3));
    }

    public function testZeroPriceIsAllowed(): void
    {
        self::assertSame(0, (new PriceCalculator())->subtotal(0, 1));
    }

    public function testZeroQuantityIsRejected(): void
    {
        $this->expectException(InvalidArgumentException::class);
        (new PriceCalculator())->subtotal(100, 0);
    }

    public function testNegativePriceIsRejected(): void
    {
        $this->expectException(InvalidArgumentException::class);
        (new PriceCalculator())->subtotal(-1, 1);
    }

    public function testOverflowIsRejected(): void
    {
        $this->expectException(InvalidArgumentException::class);
        (new PriceCalculator())->subtotal(PHP_INT_MAX, 2);
    }
}
```

`assertSame` checks both type and value. Set expectException before the operation expected to fail. These tests verify pricing and boundaries, not variable names. The first authoring run creates the lock file:

```
composer update
composer demo
composer check
composer audit
```

composer.lock: the same versions on another machine

`composer.json` declares ranges; `composer.lock` records exact resolved versions. **install** with a lock installs those versions. **update** resolves ranges again and may change the lock. Commit both files for this application and do not edit the lock manually. Teammates use `composer install`, not update just to start work. `vendor` is reproducible output and should not be committed.

SemVer uses `major.minor.patch`: breaking changes, compatible additions, compatible fixes according to package policy. `^2.1` allows `>=2.1.0 <3.0.0`. `~2.1.0` allows `>=2.1.0 <2.2.0`. `^0.3.2` stops before `0.4.0` because pre-1.0 compatibility is narrower. Ranges express a publisher's compatibility promise, not a substitute for tests.

`composer require vendor/package` adds a dependency and updates the lock; `composer require --dev ...` adds development tooling. Review origin and diff before adopting a package. `composer audit` checks published advisories using current data; success is not proof of complete security. Update a selected dependency with `composer update vendor/package` and test afterward.

Scripts, analysis, and formatting

A **script** is a named command in composer.json. `composer check` runs test, analyse, then style; a failure stops the sequence with a nonzero exit code. Scripts and plugins can execute code, so inspect what a project supplies before running it.

Static analysis checks types and code paths without executing every input. PHPStan analyzes src at level 6 here. Temporarily return a string from subtotal: analysis should fail; then restore the correct implementation. Analysis cannot replace a test for a mathematically wrong result of the correct type.

PHP-CS-Fixer checks presentation. `--dry-run --diff` reports changes without applying them. To deliberately apply fixes use `composer exec -- php-cs-fixer fix src --rules=@PSR12`. Formatting does not fix logic. Pest, Psalm, and Pint are alternatives; this lab does not install two tools for each role.

Diagnose Class not found

Trace the name through use, the namespace declaration, Composer's prefix, the file path/casing, and the autoloader require. Linux is usually case-sensitive; `billing` working in place of `Billing` on your machine does not prove correctness. After changing autoload mappings, run `composer dump-autoload`. A new correctly placed PSR-4 class usually needs no regeneration with the ordinary loader; optimized/authoritative classmaps have their own regeneration needs.

Names starting with `\` are fully qualified. `namespace\Name` is relative to the current namespace. Short class names resolve through imports and then the current namespace; unqualified function/constant names may fall back to global definitions, so do not generalize class rules to them. Use aliases such as `use X\User as ExternalUser`, `use function X\helper`, or `use const X\LIMIT`. Imports operate at file/compile scope and differ from closure capture use.

Composer supports `classmap` for non-PSR-4 code, `files` for always-loaded functions, and `autoload-dev` for test definitions. Loading a class file should not make network requests.

Development and production setup

Development installs require-dev and runs check. Deploy from the lock using `composer install --no-dev --optimize-autoloader`, then `composer check-platform-reqs --no-dev` on the real production platform. `config.platform` guides dependency resolution for a target but neither installs extensions nor changes the actual PHP

runtime. Do not run an arbitrary update during deployment. Lessons 11 and 17 cover error settings and Xdebug.

Predict, debug, complete

▼ Predict: delete vendor, then composer install with a lock

It restores the locked versions on a compatible platform; install does not search for the newest version. A missing required extension causes a platform failure.

▼ Debug: use is correct but the class cannot be found

use does not load files. Check vendor/autoload.php, mappings, namespace, filename, and casing; regenerate the loader after mapping changes.

▼ Complete PSR-4 for App\Billing\PriceCalculator

The prefix is `App\` mapped to src/, leaving Billing/PriceCalculator.php. JSON requires two backslashes to represent one.

▼ Tests and formatting pass. Does that prove all inputs work?

No. Tests cover selected cases, analysis checks contracts, and formatting checks presentation. Add boundary cases derived from requirements, not tests that merely repeat implementation.

References: Composer, constraints, PHPUnit, PHPStan, PHP-CS-Fixer.

11. Errors and exceptions

Throwable, Error, Exception, try/catch/finally, custom exceptions, boundaries, and safe reporting.

The problem: failure is not an ordinary result

A withdrawal can succeed, reject a negative amount, or reject an amount exceeding the balance. A file read may fail because storage is unavailable. If all these return `null`, the caller cannot distinguish “no data” from a bug.

An **exception** signals that an operation could not produce its normal result. `throw` raises it; `try` marks the guarded region; `catch` handles a type it understands. **Throwable** covers both Exception and Error. Error includes TypeError and DivisionByZeroError. Not every PHP diagnostic is an exception: a warning may report a problem and return false.

This lesson uses PHP 8.1+. Recall lesson 7’s call stack: exceptions travel outward through calls until a suitable catch is found.

A complete three-case program

Save `withdraw.php` and run `php withdraw.php`. The **domain** is our withdrawal rule; DomainException represents a business-rule failure even when types are valid:

```
<?php
declare(strict_types=1);

function withdraw(int $balance, int $amount): int
{
    if ($balance < 0 || $amount <= 0) {
        throw new InvalidArgumentException('Use a nonnegative balance and positive amount');
    }
    if ($amount > $balance) {
        throw new DomainException('Insufficient funds');
    }
    return $balance - $amount;
}
foreach ([200, 1200, -1] as $amount) {
    echo "request={$amount}", PHP_EOL;
    try {
        $remaining = withdraw(1000, $amount);
        echo "remaining={$remaining}", PHP_EOL;
    } catch (InvalidArgumentException $error) {
        echo "invalid input", PHP_EOL;
    } catch (DomainException $error) {
        echo "declined", PHP_EOL;
    } finally {
        echo "finished attempt", PHP_EOL;
    }
}
```

```
request=200
remaining=800
finished attempt
request=1200
declined
finished attempt
request=-1
invalid input
finished attempt
```

Each iteration starts with 1000; these are independent cases, not a running bank statement. 200 returns 800. 1200 throws before return, skipping the remaining-balance echo and reaching the domain catch. -1 triggers input rejection. Finally runs for all three. Put specific catches before general ones so a broad catch does not consume them first.

An exception from a deeper function unwinds that call before searching outward. Catch does not resume at the failed line; execution continues after the handled block. An uncaught failure reaches the application boundary and normally ends the request/script.

finally cleans up; it should not replace the result

An opened resource needs closing on both success and failure. Run `cleanup.php`:

```php
<?php
$stream = fopen('php://temp', 'w+b');
if ($stream === false) {
    throw new RuntimeException('Open failed');
}
try {
    try {
        throw new RuntimeException('Simulated read failure');
    } finally {
        fclose($stream);
        echo "closed", PHP_EOL;
    }
} catch (RuntimeException $error) {
    echo "handled", PHP_EOL;
}
echo is_resource($stream) ? "open\n" : "not open\n";
```

```
closed
handled
not open
```

The inner finally runs while the exception is leaving, then the outer catch handles it. Finally also runs on ordinary return, but do not rely on it after `exit` or forced process termination. **Mistake:** returning from finally can hide a previous result or exception. Keep cleanup from masking the original failure.

A custom exception such as `final class InsufficientStock extends DomainException {}` provides a meaningful type, but not every message needs a new class. When wrapping a failure, preserve previous: `throw new RuntimeException('Could not load notes', 0, $error);`. Program decisions should depend on type, not matching message text.

A warning does not automatically enter catch

`file_get_contents` may emit a warning and return false. Explicitly check it, as in the files lesson. `@` hides diagnostics without repairing the read. If a specific I/O boundary must convert warnings to exceptions, install a temporary handler, restore the previous one in finally, and respect `error_reporting()`. Do not arbitrarily convert every application notice and then report success.

For a long-running worker, the message boundary decides retry versus rejection and resets state. Retry is appropriate only for a temporary failure and an **idempotent** operation, one whose repetition does not duplicate its effect. A TypeError in arithmetic needs repair, not five retries.

Development and production: same error, different audience

Development is your working environment; **production** serves users. Use `php --ini` to identify loaded configuration; CLI settings can differ from FPM/Apache. A development ini excerpt:

```
error_reporting=E_ALL
display_errors=On
display_startup_errors=On
log_errors=On
```

In production, with error_log configured to a protected service-writable path:

```
error_reporting=E_ALL
display_errors=Off
display_startup_errors=Off
log_errors=On
zend.exception_ignore_args=On
```

The goal is protected diagnostics rather than exposing them to users. Never log passwords, session IDs, or complete bodies. Omitting trace arguments reduces exposure but cannot sanitize a message containing manually inserted secrets. A syntax error in the same file may occur before ini_set executes; php.ini is applied earlier.

A complete JSON application boundary

Save `boundary.php` and serve it with a local PHP server. The request returns 500 and JSON with a different request_id each time. Logging here includes only type and location to minimize data; add sanitized context when needed:

```php
<?php
declare(strict_types=1);

set_exception_handler(static function (Throwable $error): void {
    $id = bin2hex(random_bytes(8));
    error_log(json_encode([
        'event' => 'unhandled_failure',
        'request_id' => $id,
        'type' => get_class($error),
        'file' => basename($error->getFile()),
        'line' => $error->getLine(),
    ], JSON_THROW_ON_ERROR));
    http_response_code(500);
    header('Content-Type: application/json; charset=utf-8');
    header('Cache-Control: no-store');
    echo json_encode(['error' => 'Internal error', 'request_id' => $id], JSON_THROW_ON_ERROR);
});
throw new RuntimeException('Demonstration failure');
```

The handler is a safety net for unexpected failures, not a replacement for 422 validation responses. Once a body has already been sent, the response cannot reliably be repaired; emit in one place. Empty catches and `catch (Throwable) { return null; }` hide failures. Log once at the responsible boundary rather than in every layer the failure crosses.

Predict, debug, complete

▼ Predict cleanup.php's order when the operation fails

closed, handled, not open. Cleanup runs while the exception leaves, before the outer catch.

▼ Debug: catch (Exception) does not catch TypeError

TypeError extends Error; both Error and Exception implement Throwable. Handle expected types near an operation and Throwable at the application boundary without converting bugs into success.

▼ Complete wrapping an error while preserving its cause

`throw new RuntimeException('Load failed', 0, $error);`. The third argument retains the previous failure for diagnosis.

▼ A user sees an internal path in production JSON

Check display_errors, stray output, and raw exception messages. Separate a generic response/tracking ID from protected logs, then confirm the response is one valid JSON document.

Run `php error-lab.php` from the lab. In the notebook, invalid form data means 422; unexpected storage failure means 500. They are different failures.

12. Arrays and transformation tools

Lists, maps, destructuring, unpacking, map, filter, reduce, sorting, and memory trade-offs.

The problem: select, transform, and summarize a collection

A collection of prices needs more than one variable. A **list** has keys 0 through n-1 in order. A **map** associates meaningful keys with values, such as name→Omar. PHP uses `array` for both: an ordered map. Do not assume key shape from a variable's name.

Map transforms elements, **filter** selects elements, and **reduce** combines a collection into one result. They describe data movement; using them is not a badge of sophistication. This lesson needs PHP 8.1+ for array_is_list and string-key unpacking.

Complete program: total paid orders

Save `paid.php` and run `php paid.php`. Rows are assumed to have passed type and shape validation before this stage:

```
<?php
declare(strict_types=1);

$orders = [
    ['id' => 'A', 'status' => 'paid', 'total' => 1200],
    ['id' => 'B', 'status' => 'pending', 'total' => 500],
    ['id' => 'C', 'status' => 'paid', 'total' => 800],
];
$paid = array_filter($orders, static fn (array $row): bool => $row['status'] === 'paid');
echo 'keys=', implode(',', array_keys($paid)), PHP_EOL;
$totals = array_map(static fn (array $row): int => $row['total'], $paid);
echo json_encode($totals, JSON_THROW_ON_ERROR), PHP_EOL;
$totals = array_values($totals);
echo json_encode($totals, JSON_THROW_ON_ERROR), PHP_EOL;
$sum = array_reduce($totals, static fn (int $carry, int $n): int => $carry + $n, 0);
echo "sum={$sum}", PHP_EOL;
echo array_is_list($totals) ? "list\n" : "map\n";
```

```
keys=0,2
{"0":1200,"2":800}
[1200,800]
sum=2000
list
```

`array_filter` invokes a status-checking callback and preserves keys 0 and 2; removing B does not renumber. `array_map` with one array preserves those keys while turning each row into an integer. JSON sees the gap and produces an object. `array_values` renumbers, producing a JSON list. Reduce starts with carry=0, then 1200, then 2000. An empty list returns 0; without an initial value it could return null. `array_sum` is simpler for numeric addition; reduce is shown to explain accumulation.

The original array stays unchanged. These operations build new arrays, but object elements may still reference the same objects: a copied container is not a deep copy.

An equivalent loop, and when it is clearer

```
<?php
$orders = [
    ['status' => 'paid', 'total' => 1200],
    ['status' => 'pending', 'total' => 500],
    ['status' => 'paid', 'total' => 800],
];
$sum = 0;
foreach ($orders as $row) {
    if ($row['status'] !== 'paid') {
        continue;
    }
    $sum += $row['total'];
}
echo $sum, PHP_EOL;
```

```
2000
```

The same selection and addition use no intermediate arrays. A loop may be clearer for early exit or multiple statistics. Very large datasets need generators or pagination rather than `fetchAll` or several materialized arrays. A generator supplies an element on demand; lesson 16 explains it. PHP arrays store metadata for keys and values rather than packed numeric storage. Measure `memory_get_peak_usage(true)` on equivalent data before redesigning. `SplQueue` and `SplHeap` provide queue/priority behavior when that is actually required.

Keys have types and conversion rules

Save `keys.php`. **Destructuring** extracts selected positions or named fields; **unpacking** expands elements into an array or argument list:

```
<?php
$keys = [0 => 'first', '0' => 'second', '01' => 'third'];
echo json_encode($keys, JSON_THROW_ON_ERROR), PHP_EOL;
$user = ['id' => 7, 'name' => 'Omar'];
['id' => $id, 'name' => $name] = $user;
[$first, $second] = [10, 20];
echo "{$id}:{$name}:{$first}:{$second}", PHP_EOL;

$defaults = ['timeout' => 3, 'retries' => 1];
$environment = ['timeout' => 5];
echo json_encode([...$defaults, ...$environment], JSON_THROW_ON_ERROR), PHP_EOL;
echo json_encode($defaults + $environment, JSON_THROW_ON_ERROR), PHP_EOL;
```

```
{"0":"second","01":"third"}
7:Omar:10:20
{"timeout":5,"retries":1}
{"timeout":3,"retries":1}
```

`'0'` becomes an integer key and overwrites 0; `'01'` stays a string. Destructuring a missing field emits a warning, so validate input shape first. Later string keys win in unpacking; left keys win with `+`. Numeric keys are renumbered during unpacking. `sum(...$numbers)` expands arguments; `int ...$numbers` in a declaration instead collects them.

`isset($row['x'])` treats missing and null alike; `array_key_exists('x', $row)` detects a present null key. `in_array($needle, $values, true)` checks type and value; omitting true uses loose comparison. `array_filter($values)` without a callback removes all falsy values, including `0` and `'0'`. Write an explicit predicate when zero is valid.

Sorting mutates the array

A **comparator** returns negative when the first item belongs before the second, zero for equality, and positive for after. Do not return a Boolean. Run `sort.php`:

```php
<?php
$orders = [
    ['id' => 2, 'total' => 800],
    ['id' => 3, 'total' => 1200],
    ['id' => 1, 'total' => 800],
];
usort($orders, static fn (array $a, array $b): int =>
    [$a['total'], $a['id']] <=> [$b['total'], $b['id']]
);
echo implode(',', array_column($orders, 'id')), PHP_EOL;
```

```
1,2,3
```

We compare total, then id as a tie-breaker, so 1 precedes 2 at the same price. `usort` renumbers keys and returns a success flag, not the sorted array. **Wrong:** `$sorted = sort($values)` makes sorted a Boolean. Copy values first if the original must be retained, then sort the copy.

Function	Sorts	Preserves keys?
sort / rsort	Values ascending/descending	No
asort / arsort	Values	Yes
ksort / krsort	Keys	Yes
usort	Values with a custom comparator	No

Reverse operands for descending comparison. Compare consistent types; mixing strings and numbers can produce unintended ordering.

Predict, debug, complete

▼ Predict array_filter([0, 5, 0, 8]) followed by JSON

Keys remain 1 and 3, producing an object. If zero is valid, use a predicate that preserves it. If a list is needed after filtering, apply array_values.

▼ Debug a comparator returning $a['total'] > $b['total']

A Boolean cannot distinguish “before” from “equal.” Use `$a['total'] <=> $b['total']` with an int return type.

▼ Complete reduce to multiply [2,3,4], including an empty list

Start with 1 and use `fn (int $carry, int $n): int => $carry * $n`. The result is 24; empty input returns 1 under the product contract, not zero.

▼ Which approach suits count plus sum with early termination?

A clear loop often does: update both accumulators and break at the limit without intermediate arrays or hidden callback side effects.

The notebook displays a note list and validates its shape when loading. Run `php arrays-lab.php` from the lab, including zeros and an empty collection.

13. Strings, Unicode, and regular expressions

Bytes, UTF-8, mbstring, normalization, contextual encoding, and safe regular expressions.

The problem: a three-letter name has strlen six

A PHP `string` is a sequence of **bytes**, each an 8-bit storage unit. **Unicode** assigns numbers called **code points** to characters. **UTF-8** encodes a code point using 1–4 bytes. A **grapheme cluster** combines code points that a user usually perceives as one character. These are different measurements; choose according to the question.

For example, عمر has three code points and six bytes. An e followed by a separate accent appears as one character but has two code points. A family emoji combines several symbols and invisible joiners. This lesson needs PHP 8.1+ with `mbstring` and `intl`; check `php -m`.

Compare all three units in one program

Save `unicode.php` as UTF-8 and run `php unicode.php`:

```
<?php
declare(strict_types=1);

$samples = [
    'ASCII' => 'Omar',
    'Arabic' => 'عمر',
    'accent' => "e\u{0301}",
    'family' => "👨‍👩‍👧",
];
foreach ($samples as $label => $text) {
    if (!mb_check_encoding($text, 'UTF-8')) {
        throw new InvalidArgumentException('Invalid UTF-8');
    }
    printf("%s: bytes=%d points=%d graphemes=%d\n",
        $label, strlen($text), mb_strlen($text, 'UTF-8'), grapheme_strlen($text));
}
$text = 'عمر';
echo 'first byte=', bin2hex($text[0]), PHP_EOL;
echo 'first point=', mb_substr($text, 0, 1, 'UTF-8'), PHP_EOL;
```

```
ASCII: bytes=4 points=4 graphemes=4
Arabic: bytes=6 points=3 graphemes=3
accent: bytes=3 points=2 graphemes=1
family: bytes=25 points=7 graphemes=1
first byte=d8
first point=ع
```

`\u{0301}` inserts the accent code point in a double-quoted string. `mb_check_encoding` verifies that the bytes form valid UTF-8 before interpretation. `strlen` suits file/request sizes, `mb_strlen` counts code points, and `grapheme_strlen` counts clusters according to the installed Unicode/ICU rules. Newer symbols can depend on ICU

version; these examples use established characters.

`$text[0]` is the single byte d8, an incomplete part of ع. `substr` can also cut through a UTF-8 sequence. `mb_substr` cuts by code points but can separate a combining accent; `grapheme_substr` better fits visible-character limits. Grapheme limits alone do not bound request size because one cluster can contain many marks; bound bytes too.

Two representations of one character: normalization

Normalization standardizes equivalent Unicode representations under a chosen rule. NFC favors composed forms where available. Save `normalize.php`:

```
<?php
$composed = "\u{00E9}";
$decomposed = "e\u{0301}";
var_dump($composed === $decomposed);
$normalized = Normalizer::normalize($decomposed, Normalizer::FORM_C);
if ($normalized === false) {
    throw new RuntimeException('Normalization failed');
}
var_dump($composed === $normalized);
echo strlen($decomposed), ' -> ', strlen($normalized), PHP_EOL;
```

```
bool(false)
bool(true)
3 -> 2
```

The first comparison sees different bytes despite matching appearance. NFC makes this pair identical. Normalization does not equate every visually similar character and does not repair XSS. Define identifier/search policy before normalizing and retain original text for display when needed. Never silently normalize or alter passwords. `mb_strtolower` is more suitable than strtolower for multilingual case conversion, but case folding and linguistic ordering have additional rules; intl's `Collator` supports locale-aware sorting.

Regex describes a small pattern, not every language

A **regular expression** describes text shape. Start with a length bound and a simple pattern. `\A` anchors the actual start, `\z` the actual end, `[0-9]` an ASCII digit, and `{4}` four repetitions. `(?<id>...)` names a captured group. A `u` modifier enables UTF-8 mode; it does not automatically implement business rules.

`code.php` accepts ORD followed by exactly four digits:

```php
<?php
function orderId(string $input): ?string
{
    if (strlen($input) > 32) {
        return null;
    }
    $matched = preg_match('/\AORD-(?<id>[0-9]{4})\z/u', $input, $matches);
    if ($matched === false) {
        throw new RuntimeException(preg_last_error_msg());
    }
    return $matched === 1 ? $matches['id'] : null;
}
foreach (['ORD-1234', 'xORD-1234', "ORD-1234\n", 'ORD-0000'] as $input) {
    echo json_encode($input), ' => ', orderId($input) ?? 'invalid', PHP_EOL;
}
```

```
"ORD-1234" => 1234
"xORD-1234" => invalid
"ORD-1234\n" => invalid
"ORD-0000" => 0000
```

The function returns a string, preserving leading zeros. A pre-engine length check bounds work. `preg_match` returns 1 for match, 0 for no match, false for failure; do not merge 0 and false. Read captures only after a match. Accepting 0000 structurally does not prove an order exists; that needs a separate data check.

Mistake: `/^ORD-[0-9]{4}$/` can match before a final newline. `\z` enforces the actual end. Patterns such as `(a+)+` can cause expensive **backtracking**, where the engine tries many partitions near failure. Simplify the pattern, bound input length, and check PCRE errors. Do not freely execute untrusted user patterns. JSON and HTML have parsers; regex is not a universal substitute.

Encode for the destination context

Validation decides whether data is allowed. **Encoding** makes it text in the output context rather than instructions. No universal sanitization function exists. Save `escaping.php`:

```php
<?php
$name = '<Omar & Mona>';
echo htmlspecialchars($name, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8'), PHP_EOL;
echo '/search?q=', rawurlencode($name), PHP_EOL;
echo json_encode(['name' => $name], JSON_THROW_ON_ERROR), PHP_EOL;
```

```
&lt;Omar &amp; Mona&gt;
/search?q=%3COmar%20%26%20Mona%3E
{"name":"<Omar & Mona>"}
```

HTML text/quoted attributes need htmlspecialchars; a URL parameter needs rawurlencode or http_build_query. A URL inside an HTML attribute needs HTML encoding after URL construction. JavaScript is another context: prefer a separate JSON response or safe embedding mechanism, not concatenation. `sprintf`/interpolation serve presentation, not SQL construction. Compare fixed-length secrets such as CSRF tokens with `hash_equals`; that is a different purpose from ordinary user-text comparison.

Predict, debug, complete

▼ Predict measurements for e followed by U+0301

3 bytes, 2 code points, 1 grapheme. ASCII e uses one byte and the accent two; rendering combines them.

▼ Debug cutting Arabic with substr($name, 0, 1)

It takes one byte and may create invalid UTF-8. Use mb_substr for code points or grapheme_substr for perceived characters, according to the contract.

▼ Complete a regex accepting exactly AB-123456

`/\A[A-Z]{2}-[0-9]{6}\z/`. Test extra prefixes and a trailing newline, then separately check whether the country code is allowed.

▼ preg_match returned false. Is that simply no match?

No: zero means no match. False indicates a pattern, encoding, or engine-limit error. Inspect preg_last_error_msg and handle it at the application boundary.

Run `php text-lab.php` in the lab. The notebook bounds bytes and code points, validates UTF-8, and encodes on output. See the grapheme reference.

14. Date, time, and timezones

DateTimeImmutable, UTC, named zones, DST, strict parsing, intervals, and storage policy.

The problem: “nine o’clock” is not a complete appointment

Date and location are missing: 9 in Cairo is not 9 in London. An **instant** is one global point in time. **Local date/time** describes the clock/calendar in a place. A **timezone**, such as Africa/Cairo, holds that place’s rules, including daylight saving time (**DST**). An **offset**, such as +02:00, is the UTC difference at one instant, not a replacement for zone rules. A **duration** is a length of time, not a date.

Past event instants are commonly stored in UTC and displayed in the user’s zone. A recurring “9 every day in Cairo” appointment needs a local time, zone name, and DST policy, not one UTC offset forever. Examples require PHP 8.1+ and use fixed dates so results can be checked.

One instant, two displays

Save `zones.php` and run `php zones.php`. **Immutable** means operations return a new object instead of mutating the original:

```
<?php
$utc = new DateTimeImmutable('2024-01-15 12:00:00', new DateTimeZone('UTC'));
$cairo = $utc->setTimezone(new DateTimeZone('Africa/Cairo'));
echo $utc->format('Y-m-d H:i:s P'), PHP_EOL;
echo $cairo->format('Y-m-d H:i:s P'), PHP_EOL;
var_dump($utc->getTimestamp() === $cairo->getTimestamp());
$tomorrow = $utc->modify('+1 day');
echo $utc->format('Y-m-d'), ' / ', $tomorrow->format('Y-m-d'), PHP_EOL;
```

```
2024-01-15 12:00:00 +00:00
2024-01-15 14:00:00 +02:00
bool(true)
2024-01-15 / 2024-01-16
```

We construct UTC time, then setTimezone changes its representation. `getTimestamp` returns Unix-epoch seconds for the same instant, so they match. `Y-m-d` is year-month-day, `H:i:s` is hour-minute-second: i means minutes, not m. `P` prints the offset. `DateTimeInterface::ATOM` provides a transport-friendly format including offset. The last line proves modify did not mutate utc.

Mistake: calling `$utc->modify('+1 day');` and ignoring its return, then expecting utc to change. Assign the result. DateTimeImmutable prevents surprising mutation when functions share an object.

Strict parsing: automatic correction may be wrong for your form

Parsing turns text into a date value. A flexible parser may turn February 30 into a March date. Save `dates.php`:

```
<?php
declare(strict_types=1);
function parseDate(string $input): DateTimeImmutable
{
    if (preg_match('/\A[0-9]{4}-[0-9]{2}-[0-9]{2}\z/', $input) !== 1) {
        throw new InvalidArgumentException('Use YYYY-MM-DD');
    }
    $date = DateTimeImmutable::createFromFormat('!Y-m-d', $input, new DateTimeZone('UTC'));
    $errors = DateTimeImmutable::getLastErrors();
    if ($date === false
        || ($errors !== false && ($errors['warning_count'] > 0 || $errors['error_count'] > 0))
        || $date->format('Y-m-d') !== $input) {
        throw new InvalidArgumentException('Invalid calendar date');
    }
    return $date;
}
foreach (['2024-02-29', '2023-02-29', '2024-2-9'] as $input) {
    try {
        echo parseDate($input)->format('Y-m-d H:i:s'), PHP_EOL;
    } catch (InvalidArgumentException) {
        echo "invalid: {$input}", PHP_EOL;
    }
}
```

```
2024-02-29 00:00:00
invalid: 2023-02-29
invalid: 2024-2-9
```

Regex enforces shape and component width, not calendar validity. `!` resets unspecified fields instead of inheriting the current clock time. Capture getLastErrors immediately; since PHP 8.2 it may return false when there are no errors, so do not assume an array. Warnings matter because corrected impossible dates may produce warnings. Finally, format back and compare. Short-circuit `||` prevents calling format on false.

A date-only birthday is not necessarily an instant; a local DATE has different semantics from created_at. Do not attach a timezone merely because other time values use one.

A calendar day is not always 24 hours

An **interval** describes a difference: `P1D` is a calendar day and `PT24H` is 24 hours. Run `dst.php` using London's historical March 2024 transition:

```php
<?php
$start = new DateTimeImmutable('2024-03-30 12:00:00', new DateTimeZone('Europe/London'));
$calendar = $start->add(new DateInterval('P1D'));
$elapsed = $start->add(new DateInterval('PT24H'));
foreach (['calendar' => $calendar, 'elapsed' => $elapsed] as $label => $end) {
    $seconds = $end->getTimestamp() - $start->getTimestamp();
    echo $label, ': ', $end->format('Y-m-d H:i P'), " seconds={$seconds}", PHP_EOL;
}
```

```
calendar: 2024-03-31 12:00 +01:00 seconds=82800
elapsed: 2024-03-31 13:00 +01:00 seconds=86400
```

The calendar result preserves local noon but spans 23 hours because the clock moves forward. The elapsed result preserves 24 hours and becomes 13:00 local. A local time may not exist during spring-forward or may occur twice during fall-back. Choose a rejection/offset policy rather than silently accepting parser choices for important appointments.

`P1M` means a month, not 30 days: adding it to January 31 can overflow February. Decide whether the rule means the last day of the following month or ordinary calendar addition. Test leap days and month/year boundaries. Timezone rules can change politically; keep timezone data current and reassess future local schedules under an explicit policy.

Test “now” without waiting

Clock injection means passing the current time as input instead of reading it deep inside logic. A simple version needs no interfaces:

```php
<?php
function expired(DateTimeImmutable $deadline, DateTimeImmutable $now): bool
{
    return $now >= $deadline;
}
$deadline = new DateTimeImmutable('2024-01-01T12:00:00+00:00');
foreach (['11:59:59', '12:00:00', '12:00:01'] as $time) {
    $now = new DateTimeImmutable("2024-01-01T{$time}+00:00");
    echo $time, ': ', expired($deadline, $now) ? 'expired' : 'valid', PHP_EOL;
}
```

```
11:59:59: valid
12:00:00: expired
12:00:01: expired
```

The boundary itself is expired under `>=`, an explicit testable rule. Measure execution duration with a monotonic clock such as `hrtime`, rather than a wall clock that may be corrected during measurement. Store event instants explicitly, retain zones needed for local schedules, and convert at presentation.

Predict, debug, complete

▼ UTC 12:00 and Cairo 14:00 in the example: which is later?

They are the same instant, proven by equal timestamps. Only the local representation differs.

▼ Debug: createFromFormat returned an object for 2023-02-29, so I accepted it

The parser may have corrected it. Inspect warnings/errors and round-trip formatting; an object does not prove literal acceptance.

▼ Complete expiry at the deadline itself

Use `$now >= $deadline`. Test one second before, equality, and one second after. A strict > allows the exact boundary.

▼ A daily local 9:00 appointment: store only +02:00?

No. Retain local time, zone name, and an ambiguity policy because DST or legal changes can alter the offset.

Run `php datetime-lab.php` from the lab. The notebook's created_at is a UTC instant formatted at presentation. See the parsing reference.

15. From HTTP request to router and response

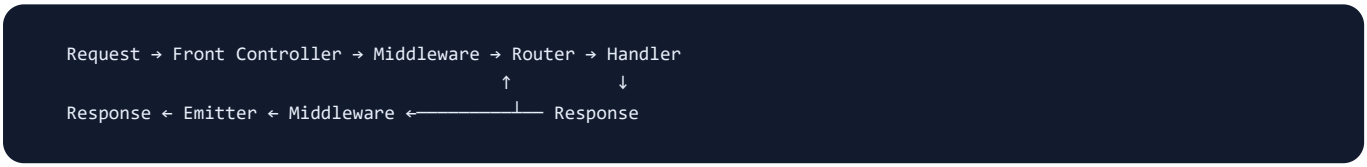
Front controllers, JSON bodies, routing, middleware, and correct HTTP responses without a framework.

The problem: a URL does not have to name a file

Instead of `save.php` and `list.php`, we want `GET /health` and `POST /notes` with one error/output policy. Separate responsibilities before adopting a framework:

Term	Responsibility
Request	Client method, path, headers, and body
Front controller	One entry point, usually public/index.php
Middleware	Shared work around later steps, or early rejection
Router	Select a handler using method and path
Handler	Execute one use case and return a response
Response / emitter	Status/headers/body data; an emitter sends them once

Requests enter in this order. Route-specific middleware may follow routing, but the example's general middleware wraps it:



One complete file with explicit boundaries

Use PHP 8.1+ and mbstring. Create `public/index.php` below, or use `http-demo` in the examples package. From http-demo run `php -S 127.0.0.1:8082 -t public public/index.php`. The final argument routes all requests through the front controller, not only existing files. This teaching API returns a note without persisting it; lesson 17 adds storage.

```php
<?php
declare(strict_types=1);

function response(array $data, int $status = 200, array $headers = []): array
{
    return [
        'status' => $status,
        'headers' => $headers + [
            'Content-Type' => 'application/json; charset=utf-8',
            'Cache-Control' => 'no-store',
            'X-Content-Type-Options' => 'nosniff',
        ],
        'body' => json_encode($data, JSON_THROW_ON_ERROR | JSON_UNESCAPED_UNICODE),
    ];
}
function createNote(array $request): array
{
    if ($request['type'] !== 'application/json') {
        return response(['error' => 'Unsupported media type'], 415);
    }
    try {
        $payload = json_decode($request['body'], false, 32, JSON_THROW_ON_ERROR);
    } catch (JsonException) {
        return response(['error' => 'Invalid JSON'], 400);
    }
    if (!$payload instanceof stdClass
        || !is_string($payload->text ?? null)
        || trim($payload->text) === ''
        || mb_strlen($payload->text, 'UTF-8') > 200) {
        return response(['error' => 'Use a text field with 1 to 200 code points'], 422);
    }
    return response(['text' => trim($payload->text)], 200);
}
function route(array $request): array
{
    $routes = [
        '/health' => ['GET' => static fn (array $r): array => response(['status' => 'ok'])],
        '/notes' => ['POST' => 'createNote'],
    ];
    $methods = $routes[$request['path']] ?? null;
    if ($methods === null) {
        return response(['error' => 'Not found'], 404);
    }
    $handler = $methods[$request['method']] ?? null;
    if ($handler === null) {
        return response(['error' => 'Method not allowed'], 405, ['Allow' => implode(', ', array_keys($methods))]);
    }
    return $handler($request);
}
function middleware(array $request, callable $next): array
{
    $id = bin2hex(random_bytes(8));
    try {
        if (strlen($request['body']) > 4096) {
            $reply = response(['error' => 'Body too large'], 413);
        } else {
            $reply = $next($request);
        }
    } catch (Throwable $error) {
        error_log(json_encode(['request_id' => $id, 'type' => get_class($error)], JSON_THROW_ON_ERROR));
        $reply = response(['error' => 'Internal error'], 500);
    }
    $reply['headers']['X-Request-ID'] = $id;
    return $reply;
}
```

```
$body = file_get_contents('php://input', false, null, 0, 4097);
if ($body === false) {
    $reply = response(['error' => 'Body unavailable'], 500);
} else {
    $path = parse_url($_SERVER['REQUEST_URI'] ?? '/', PHP_URL_PATH);
    $request = [
        'method' => $_SERVER['REQUEST_METHOD'] ?? 'GET',
        'path' => is_string($path) ? $path : '',
        'type' => strtolower(trim(explode(';', $_SERVER['CONTENT_TYPE'] ?? '')[0])),
        'body' => $body,
    ];
    $reply = middleware($request, 'route');
}
http_response_code($reply['status']);
foreach ($reply['headers'] as $name => $value) {
    header("{$name}: {$value}");
}
echo $reply['body'];
```

Read from the entry point, then trace one request

Function definitions do not execute their bodies. Execution starts at the body read near the bottom. Reading at most 4097 bytes detects exceeding 4096 without loading an unbounded body into application memory. The server also needs a body limit because PHP/server infrastructure may already have received or buffered it. `parse_url(..., PHP_URL_PATH)` excludes the query, so `/health?check=1` selects the same route.

We construct a request array and call middleware with `'route'` as a callback. Middleware creates an ID and checks size before parsing. If acceptable it invokes next; the router finds a path, then method, then calls a handler. The response returns outward; middleware adds a header, and the emitter sends status, headers, then body.

`response` builds data without echo or exit. Encoding occurs before headers are sent, so the general boundary can turn encoding failure into 500. Never mix var_dump with the response. The read-failure branch here has its own 500 outside middleware; put request construction inside a broader boundary if every failure needs an ID.

Valid JSON does not imply a valid request

createNote asks three separate questions: is Content-Type supported, is the text valid JSON, and is its root an object containing a nonempty bounded text string? `[]`, `null`, and `42` are valid JSON but violate this contract. Decoding without true produces stdClass for an object, making it distinguishable from a list.

`trim` removes ordinary surrounding whitespace; it does not promise all Unicode whitespace handling. `mb_strlen` measures the agreed code points. This validation/echo example returns 200 because it creates no persistent resource; a real creation endpoint adds storage and returns 201 plus Location when the resource has an address.

Routes here are fixed literal paths. Do not repeatedly decode them or convert paths into include filenames. A production router also needs policies for parameters, URL decoding, HEAD/OPTIONS, and content negotiation; use a framework when appropriate. Trust X-Forwarded-* only from configured trusted proxies.

Exercise the contract from a terminal

On Windows use `curl.exe` instead of PowerShell's alias. To avoid shell quoting differences, save `note.json` containing `{"text":"Learn routing"}` and `broken.json` containing only `{`, then run:

```
curl -i http://127.0.0.1:8082/health
curl -i -X POST -H "Content-Type: application/json" --data-binary @note.json http://127.0.0.1:8082/notes
curl -i -X POST -H "Content-Type: application/json" --data-binary @broken.json http://127.0.0.1:8082/notes
```

Request	Result
GET /health	200 and `{"status":"ok"}`
GET /missing	404
GET /notes	405 and Allow: POST
POST /notes without JSON Content-Type	415
POST /notes with broken JSON	400
POST /notes with [] or empty/array text	422
Body exceeding 4096 bytes	413 before parsing
Valid text	200 and `{"text":"Learn routing"}`

X-Request-ID is random; tests should check its presence/shape, not one fixed value. Assert Content-Type and status as well as body. Expose only public as document root, keeping storage and vendor inaccessible.

Predict, debug, complete

▼ Predict GET /notes versus GET /unknown

The first is a known path with an unsupported method: 405 and Allow. The second is an unknown path: 404. Checking path before method creates that distinction.

▼ Debug: a router echoes before middleware adds a header

Early output can send headers or corrupt JSON. Return response data from handlers and use one emitter after the chain returns.

▼ Complete a body-limit check before json_decode

Read limit+1 bytes, compare length with the limit, and return 413. Add a server limit too; Content-Length alone does not establish actual body size for every transfer mode.

▼ Order authentication, authorization, and validation

After route selection, establish identity and permission before protected effects, and validate input before business logic. Error handling wraps the chain; body limits precede parsing. Detailed ordering depends on each middleware's responsibility.

The cumulative project moves these boundaries into files and adds forms, sessions, and storage. For additional HTTP practice see lab setup.

16. Modern PHP 8.0–8.5 roadmap

A versioned roadmap covering WeakMap, Fibers, DNF types, readonly classes, property hooks, and later PHP features.

The problem: valid code that your server cannot parse

A new syntax feature can make a file fail **before execution** on an older PHP version. An `if (PHP_VERSION_ID >= ...)` in that same file cannot protect syntax its parser does not understand. Declare a minimum in Composer, actually test it, and adopt a feature to solve a concrete problem.

Read in two passes: generators, WeakMap, and fibers explain execution/memory; then recognize newer type/object features. Detailed OOP belongs in its track, without building a framework here.

A timeline of released versions

These are each series' initial release dates, not advice to install the old .0 builds. PHP 8.5 shipped on November 20, 2025; it is not a future RFC. Choose a supported patch release when installing.

Version	Release date	Features
8.0	2020-11-26	Named arguments, attributes, constructor promotion, union types, match, nullsafe, WeakMap, ValueError, mixed, JIT
8.1	2021-11-25	Enums, fibers, first-class callables, intersection types, never, readonly properties, array_is_list
8.2	2022-12-08	DNF types, readonly classes, standalone true/false/null, SensitiveParameter, dynamic-property deprecation
8.3	2023-11-23	Typed class constants, Override, json_validate, readonly reinitialization during cloning, dynamic class-constant access
8.4	2024-11-21	Property hooks, asymmetric visibility, lazy objects, array_find/array_any/array_all, Deprecated attribute
8.5	2025-11-20	Pipe, URI extension, clone with, NoDiscard, array_first/array_last, partitioned cookie option

Dates are recorded in the changelog. The table locates features rather than demanding memorization. **Historical distinction:** generators and `yield` arrived in 5.5; `yield from` in 7.0. Appearing in this lesson does not make them PHP 8 features. **JIT** compiles during execution but does not guarantee a faster website; measure your application.

`require: {"php":"^8.3"}` means `>=8.3.0 <9.0.0`, so do not put 8.4 hooks in a file claiming 8.3 support. `composer check-platform-reqs` checks the actual platform. Review migration/deprecation notes and test your minimum and maximum supported versions; a newer number does not replace tests.

Generators: an item on demand, not a complete array

A **generator** function contains yield. Calling it returns an iterator object; its body advances during foreach and retains its position/local variables between elements. `yield from` delegates element production to another iterable. Save `generator.php`; its syntax is compatible with PHP 7.0+, but run it on a supported PHP installation:

```
<?php
function batches(): Generator
{
    echo "begin", PHP_EOL;
    yield from [10, 20];
    yield from [30];
    echo "end", PHP_EOL;
}
$items = batches();
echo "created", PHP_EOL;
foreach ($items as $key => $value) {
    echo "{$key}:{$value}", PHP_EOL;
}
echo implode(',', iterator_to_array(batches(), false)), PHP_EOL;
```

```
created
begin
0:10
1:20
0:30
end
begin
end
10,20,30
```

created before begin proves that calling did not build the sequence. Key 0 repeats because yield from preserves source keys. `iterator_to_array(..., false)` renumbers and avoids losing 10; its default key-preserving behavior can overwrite duplicates. Materializing an array loads every element, losing the memory advantage. For another pass, create a new generator rather than assuming a consumed one can rewind.

For large files, read lines inside try/finally and close the handle when iteration completes or the generator is destroyed. Retaining a suspended generator retains its resources. `yield from` does not make I/O asynchronous.

WeakMap from 8.0: metadata tied to an object's lifetime

A **strong reference** keeps an object alive. WeakMap uses object keys without itself keeping those keys alive; it suits temporary **metadata**, additional information about an object. Save `weak.php` for PHP 8.0+:

```php
<?php
$map = new WeakMap();
$request = new stdClass();
$map[$request] = 'checked';
$alias = $request;
echo count($map), PHP_EOL;
unset($request);
echo count($map), PHP_EOL;
unset($alias);
echo count($map), PHP_EOL;
```

```
1
1
0
```

Removing request alone leaves alias as a strong reference. Removing both drops the entry in this cycle-free example. A stored value that itself references the key may keep it alive; weak keys do not promise deletion of every object graph. Do not make critical business decisions depend on collection timing or use WeakMap as durable caching. See the reference.

Fibers from 8.1: suspend and resume a call stack

A **fiber** is an execution context that can suspend a call stack and continue it later. **Cooperative** means code explicitly yields control; it is not a thread or CPU parallelism. Save `fiber.php` for PHP 8.1+:

```php
<?php
$fiber = new Fiber(function (): string {
    echo "fiber entered", PHP_EOL;
    $reply = Fiber::suspend('need input');
    echo "fiber resumed", PHP_EOL;
    return strtoupper($reply);
});
echo "main before", PHP_EOL;
echo $fiber->start(), PHP_EOL;
echo "main between", PHP_EOL;
$fiber->resume('done');
echo $fiber->getReturn(), PHP_EOL;
```

```
main before
fiber entered
need input
main between
fiber resumed
DONE
```

Constructing a fiber does not run it. start enters its body until suspend returns need input to the caller. resume sends done back as the suspended expression's value in `$reply`. Once terminated, getReturn reads DONE; the final return is not necessarily resume's result. Do not resume before starting or after termination.
`isStarted/isSuspended/isTerminated` expose state.

An event-loop library decides which operation resumes when I/O becomes ready. Fibers alone provide neither a scheduler nor nonblocking file_get_contents. Use a suitable library for asynchronous work. See the fiber reference.

DNF types and readonly classes from 8.2

A **union** accepts one of several types, A or B. An **intersection** requires the same object to satisfy both A and B. **DNF** combines them as "(A and B) or C," requiring parentheses around the intersection. Complete `dnf.php` needs PHP 8.2+:

```
<?php
function describe((Countable&Stringable)|array $value): string
{
    return is_array($value) ? 'array:' . count($value) : (string) $value;
}
$items = new class implements Countable, Stringable {
    public function count(): int { return 2; }
    public function __toString(): string { return $this->count() . ' items'; }
};
echo describe([10, 20]), PHP_EOL;
echo describe($items), PHP_EOL;

readonly class Amount
{
    public function __construct(public int $minorUnits) {}
}
$amount = new Amount(500);
echo $amount->minorUnits, PHP_EOL;
try {
    $amount->minorUnits = 600;
} catch (Error) {
    echo "readonly blocked reassignment", PHP_EOL;
}
```

```
array:2
2 items
500
readonly blocked reassignment
```

The array uses one union branch; the object satisfies both Countable and Stringable. A readonly class makes typed instance properties readonly and disallows dynamic properties. It is not deep immutability: an object stored in a property may still mutate internally. Constructor promotion, shorthand for defining and receiving properties, arrived in 8.0; readonly classes arrived in 8.2.

PHP 8.1 enums represent a closed set of cases. Backed enums use from, which throws ValueError for unknown values, or tryFrom, which returns null. Readonly properties also arrived in 8.1; never marks functions that do not return normally, such as those always terminating or throwing. See OOP and types.

Attributes: metadata needs a reader

Reflection inspects code definitions at runtime. Run `attribute.php` on PHP 8.0+:

```php
<?php
#[Attribute(Attribute::TARGET_FUNCTION)]
final class Label
{
    public function __construct(public string $text) {}
}
#[Label('Preview')]
function preview(): void {}
$definition = new ReflectionFunction('preview');
$label = $definition->getAttributes(Label::class)[0]->newInstance();
echo $label->text, PHP_EOL;
```

```
Preview
```

An attribute does not automatically enforce a permission or route. Reflection reads it here and we use its text. PHP 8.2's `#[SensitiveParameter]` redacts parameter values in traces, not manually written logs. PHP 8.3's `#[Override]` detects methods claimed to override a nonexistent parent/interface method. Typed class constants such as `public const int LIMIT = 10;` and dynamic `ClassName::{$name}` access arrived in 8.3.

8.4: hooks, asymmetric visibility, and lazy objects

A **hook** runs when a property is read/written. **Asymmetric visibility** gives reading and writing different access rules. `contact.php` requires 8.4+:

```php
<?php
final class Contact
{
    public private(set) string $email {
        set {
            $clean = trim($value);
            if (filter_var($clean, FILTER_VALIDATE_EMAIL) === false) {
                throw new InvalidArgumentException('Invalid email');
            }
            $this->email = strtolower($clean);
        }
    }
    public function __construct(string $email) { $this->email = $email; }
}
$contact = new Contact(' OMAR@EXAMPLE.COM ');
echo $contact->email, PHP_EOL;
```

```
omar@example.com
```

The constructor may write while callers may read; outside writes are rejected. Lowercasing the entire email is this demonstration's policy, not a rule for every mail system. A **lazy object** defers initialization until state is needed; Reflection supports ghost/proxy variants. DI containers and ORMs commonly use them. Lazy does not mean every method immediately triggers initialization. Prefer named methods for complex operations rather than hidden property logic. See lazy objects.

8.5: a program using features that have shipped

Save `features85.php` and run it with PHP 8.5. A **URI** identifies a resource; the new extension offers APIs for RFC 3986 and WHATWG rules. Clone with copies an object and updates properties during cloning; it does not automatically rerun the constructor, so do not rely on constructor validation being repeated:

```
<?php
readonly class Page
{
    public function __construct(public string $title) {}
    public function withTitle(string $title): self
    {
        return clone($this, ['title' => $title]);
    }
}
$draft = new Page('Draft');
$published = $draft->withTitle('Published');
$slug = ' Learn PHP ' |> trim(...) |> strtolower(...);
$uri = new Uri\Rfc3986\Uri('https://example.com/notes?sort=new');
echo $draft->title, ' / ', $published->title, PHP_EOL;
echo $slug, PHP_EOL;
echo $uri->getHost(), PHP_EOL;
echo array_first(['A', 'B']), '/', array_last(['A', 'B']), PHP_EOL;
```

```
Draft / Published
learn php
example.com
A/B
```

The original Draft remains unchanged. Clone with runs inside a method with write access; readonly properties have implicit protected(set) since 8.4, so global-scope updates are not automatically allowed. Pipe passes one argument to each callable; lesson 6 explains it. array_first/last return null for an empty array, but null may itself be an element value, so distinguish cases when required.

`#[NoDiscard]` warns when a marked return value is ignored; `(void)` explicitly discards it in 8.5. Other additions include attributes on constants, callable/static closures in constant expressions, expanded asymmetric visibility, and the `partitioned` cookie option. Consult the official 8.5 release before adopting an API; availability is not a requirement to use it.

Predict, debug, complete

▼ Predict WeakMap count after unset(request) while alias remains

1: alias is a strong reference. Weak keys do not remove other references.

▼ Debug a pipe expression inside an if version check on PHP 8.4

Parsing fails before if runs. Isolate the example in an 8.5-only file, or use older syntax and deliberately plan a minimum-version increase.

▼ Complete a type accepting an array or an object both Countable and Stringable

`(Countable&Stringable)|array` requires 8.2. The same object must satisfy both interfaces; a union between them changes the contract.

▼ When does begin appear if you only call batches?

Not until iteration begins. Generators are deferred; converting to an array consumes all elements and can use substantial memory.

▼ Does a fiber automatically accelerate a CPU-heavy calculation?

No threads or parallel execution are created. Without suspend, the calculation retains control. Fibers organize suspension with a scheduler and suitable I/O.

Run each file on its declared baseline and use `php -l` before execution. `php modern-features-lab.php` from the lab adds enum, readonly, and match failure cases. Check current support before deployment.

17. Notebook project and practical debugging

Complete PHP programs connecting types, functions, files, requests, and exceptions, with output-prediction and debugging exercises.

The problem: individual pieces work, but does the request work?

A function may be correct while a form sends an array instead of text, storage fails, or early output breaks a redirect. We will connect forms, validation, sessions, files, and routing in a **local learning notebook**. Each browser session owns its notebook; there is no login or database. For accounts and larger shared storage, continue to security and databases instead of inventing them here.

Complete files live in `examples/php-course/notebook` or the course examples package. You need PHP 8.1+, mbstring, and writable session storage. Study in three sittings: request/validation, storage/presentation, then tests/debugging.

The stages we have built

Stage	Learned	Practical evidence
Lessons 5–7	Conditions, functions, callbacks	Reject or continue, then invoke a handler
Lesson 8	Files, JSON, I/O failure	Verify written bytes and decoded shape
Lesson 9	Form→Validation→Session	preferences.php remembers names and checks CSRF
Lesson 10	Composer and quality tools	composer-demo has a lock and real tests
Lessons 13–15	Unicode, time, request flow	UTF-8, UTC, router, and emitter
Here	Connect boundaries	Save a note, then display it through GET

Do not paste unrelated fragments together. Create this tree and write each complete file:

```
notebook/
  config.php
  public/index.php
  src/notebook.php
  views/notebook.php
  storage/              (created on first save)
```

1. Explicit configuration

`config.php` reads deployment configuration, never a request field:

```
<?php
$environment = getenv('APP_ENV') ?: 'development';
if (!in_array($environment, ['development', 'production'], true)) {
    throw new RuntimeException('Invalid APP_ENV');
}
return [
    'secure_cookie' => $environment === 'production',
    'storage' => getenv('NOTEBOOK_STORAGE') ?: __DIR__ . '/storage',
];
```

Local HTTP uses a non-Secure cookie. Production enables Secure and requires actual HTTPS. NOTEBOOK_STORAGE is a trusted test/deployment storage path; the default is outside public. Unknown environments fail instead of choosing an accidental policy. Server/php.ini body limits complement the application limit; production errors are logged, not displayed.

2. Validation, storage, presentation: src/notebook.php

```
<?php
declare(strict_types=1);

function validateNote(array $input): array
{
    $data = [];
    $errors = [];
    foreach (['name' => 40, 'text' => 200] as $field => $limit) {
        $value = $input[$field] ?? null;
        if (!is_string($value) || strlen($value) > $limit * 4
            || !mb_check_encoding($value, 'UTF-8')) {
            $errors[$field] = 'Use valid UTF-8 text';
            $data[$field] = '';
            continue;
        }
        $value = trim($value);
        $data[$field] = $value;
        if ($value === '' || mb_strlen($value, 'UTF-8') > $limit) {
            $errors[$field] = "Use 1 to {$limit} code points";
        }
    }
    return ['data' => $data, 'errors' => $errors];
}
function ownerDirectory(string $root, string $owner): string
{
    if (preg_match('/\A[a-f0-9]{32}\z/', $owner) !== 1) {
        throw new RuntimeException('Invalid storage identifier');
    }
    return $root . '/' . $owner;
}
function readNotes(string $root, string $owner): array
{
    $directory = ownerDirectory($root, $owner);
    if (file_exists($root) && (!is_dir($root) || !is_readable($root))) {
        throw new RuntimeException('Storage unavailable');
    }
    if (!is_dir($directory)) {
        return [];
    }
    $paths = glob($directory . '/*.json');
    if ($paths === false || count($paths) > 100) {
        throw new RuntimeException('Cannot list notes');
    }
    $notes = [];
    foreach ($paths as $path) {
        $raw = file_get_contents($path, false, null, 0, 8193);
        if ($raw === false || strlen($raw) > 8192) {
            throw new RuntimeException('Cannot read note');
        }
        $note = json_decode($raw, true, 32, JSON_THROW_ON_ERROR);
        if (!is_array($note)
            || !is_string($note['name'] ?? null)
            || !is_string($note['text'] ?? null)
            || !is_string($note['created_at'] ?? null)
            || !is_string($note['id'] ?? null)) {
            throw new RuntimeException('Invalid stored note');
        }
        $notes[] = $note;
    }
    usort($notes, static fn (array $a, array $b): int =>
        [$a['created_at'], $a['id']] <=> [$b['created_at'], $b['id']]);
```

```
        return $notes;
    }
    function saveNote(string $root, string $owner, array $data, DateTimeImmutable $now): void
    {
        $directory = ownerDirectory($root, $owner);
        if (!is_dir($directory) && !mkdir($directory, 0700, true) && !is_dir($directory)) {
            throw new RuntimeException('Cannot create storage');
        }
        $id = bin2hex(random_bytes(16));
        $note = [
            'id' => $id,
            'name' => $data['name'],
            'text' => $data['text'],
            'created_at' => $now->setTimezone(new DateTimeZone('UTC'))->format(DateTimeInterface::ATOM),
        ];
        $json = json_encode($note, JSON_UNESCAPED_UNICODE | JSON_THROW_ON_ERROR);
        $temporary = tempnam($directory, '.pending-');
        if ($temporary === false) {
            throw new RuntimeException('Cannot create temporary file');
        }
        if (realpath(dirname($temporary)) !== realpath($directory)) {
            unlink($temporary);
            throw new RuntimeException('Temporary file outside storage');
        }
        try {
            if (file_put_contents($temporary, $json) !== strlen($json)) {
                throw new RuntimeException('Cannot write complete note');
            }
            if (!rename($temporary, $directory . '/' . $id . '.json')) {
                throw new RuntimeException('Cannot publish note');
            }
        } finally {
            if (is_file($temporary)) {
                unlink($temporary);
            }
        }
    }
    function escapeHtml(string $value): string
    {
        return htmlspecialchars($value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
    }
    function renderNotes(array $notes, array $errors, array $old, string $csrf, string $flash): string
    {
        ob_start();
        try {
            require dirname(__DIR__) . '/views/notebook.php';
            return (string) ob_get_contents();
        } finally {
            ob_end_clean();
        }
    }
```

Read one function at a time

`validateNote` returns cleaned values and errors. Check is_string before strlen and mb_check_encoding so `name[]=x` cannot crash it. Byte limits precede Unicode processing, followed by trim and code-point bounds. Continue skips remaining checks for rejected values. The string `'0'` is valid; empty would incorrectly reject it.

`ownerDirectory` accepts only a generated 32-character hex identifier. The browser never chooses a filename; owner comes from server-side session state. This isolates session notebooks, not recoverable user accounts.

`readNotes` treats a not-yet-created directory as an empty notebook. Glob selects published json files only. Reads are bounded and JSON failures are explicit. Corrupt storage must not silently become “no notes,” hiding data loss. Validate shape and sort by timestamp then id; random IDs do not express creation order within one second.

`saveNote` creates private storage and an ID, converting the injected now to UTC. Tempnam creates a temporary file; verify all bytes were written, then rename to the final json name on the same filesystem. Readers ignore pending files. Verify rename guarantees on the deployment platform; this example promises no power-loss durability. Finally cleans up the temporary file on failure.

`renderNotes` buffers template output into a string rather than sending it immediately; finally cleans the buffer even when rendering fails. `escapeHtml` encodes at presentation, not storage.

3. The template: views/notebook.php

```
<!doctype html>
<html lang="en">
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>My learning notebook</title>
<h1>My learning notebook</h1>
<p>This browser session owns this notebook. This demo has no login.</p>
<p role="status"><?= escapeHtml($flash) ?></p>
<?php foreach ($errors as $field => $message): ?>
  <p role="alert"><?= escapeHtml($field . ': ' . $message) ?></p>
<?php endforeach; ?>
<form method="post" action="/notes">
  <input type="hidden" name="csrf" value="<?= escapeHtml($csrf) ?>">
  <p><label>Name <input name="name" required value="<?= escapeHtml($old['name'] ?? '') ?>"></label></p>
  <p><label>Note <textarea name="text" required><?= escapeHtml($old['text'] ?? '') ?></textarea></label></p>
  <button>Save note</button>
</form>
<h2>Saved notes</h2>
<?php if ($notes === []): ?><p>No notes yet.</p><?php endif; ?>
<ol>
<?php foreach ($notes as $note): ?>
  <li><strong><?= escapeHtml($note['name']) ?></strong>:
    <span class="note-text"><?= escapeHtml($note['text']) ?></span>
    <time datetime="<?= escapeHtml($note['created_at']) ?>"><?= escapeHtml($note['created_at']) ?></time>
  </li>
<?php endforeach; ?>
</ol>
</html>
```

The hidden csrf field comes from the session, but users can change hidden fields; protection comes from server comparison, not visual hiding. Errors appear beside the form, and valid input is retained after 422. Every name, note, and time passes through escapeHtml even when loaded from our own files. Trusted storage does not turn text into allowed HTML. Enter `<b>hello</b>` and it should appear literally, not bold.

The English UI deliberately keeps the example identical in both lesson languages; names and notes accept Arabic/UTF-8. Translating UI strings does not change the lesson’s logic.

4. Entry point, middleware, and router

`public/index.php` is the single entry point for every route:

```
<?php
declare(strict_types=1);
require dirname(__DIR__) . '/src/notebook.php';

function reply(string $body, int $status = 200, array $headers = []): array
{
    return ['status' => $status, 'headers' => $headers, 'body' => $body];
}
function routeNotebook(string $method, string $path, array $config): array
{
    $routes = ['/' => 'GET', '/notes' => 'POST'];
    if (!isset($routes[$path])) {
        return reply('Not found', 404);
    }
    if ($routes[$path] !== $method) {
        return reply('Method not allowed', 405, ['Allow' => $routes[$path]]);
    }
    $owner = $_SESSION['owner'];
    $csrf = $_SESSION['csrf'];
    if ($method === 'POST') {
        $type = strtolower(trim(explode(';', $_SERVER['CONTENT_TYPE'] ?? '')[0]));
        if ($type !== 'application/x-www-form-urlencoded') {
            return reply('Unsupported media type', 415);
        }
        $token = $_POST['csrf'] ?? null;
        if (!is_string($token) || !hash_equals($csrf, $token)) {
            return reply('Invalid form token', 403);
        }
        $result = validateNote($_POST);
        $notes = readNotes($config['storage'], $owner);
        if ($result['errors'] !== []) {
            return reply(renderNotes($notes, $result['errors'], $result['data'], $csrf, ''), 422);
        }
        if (count($notes) >= 100) {
            return reply('Notebook is full', 409);
        }
        saveNote($config['storage'], $owner, $result['data'], new DateTimeImmutable('now', new
DateTimeZone('UTC')));
        $_SESSION['name'] = $result['data']['name'];
        $_SESSION['flash'] = 'Note saved';
        return reply('', 303, ['Location' => '/']);
    }
    $flash = $_SESSION['flash'] ?? '';
    unset($_SESSION['flash']);
    return reply(renderNotes(
        readNotes($config['storage'], $owner),
        [],
        ['name' => $_SESSION['name'] ?? '', 'text' => ''],
        $csrf,
        $flash,
    ));
}
function sessionMiddleware(callable $next, array $config): array
{
    $raw = file_get_contents('php://input', false, null, 0, 4097);
    if ($raw === false) {
        throw new RuntimeException('Cannot read body');
    }
    if (strlen($raw) > 4096 || (int) ($_SERVER['CONTENT_LENGTH'] ?? 0) > 4096) {
        return reply('Body too large', 413);
    }
    if (!session_start([
        'use_strict_mode' => true,
        'use_only_cookies' => true,
        'cookie_secure' => $config['secure_cookie'],
```

```
            'cookie_httponly' => true,
            'cookie_samesite' => 'Lax',
            'cookie_path' => '/',
        ])) {
            throw new RuntimeException('Session unavailable');
        }
        try {
            $_SESSION['owner'] ??= bin2hex(random_bytes(16));
            $_SESSION['csrf'] ??= bin2hex(random_bytes(32));
            return $next();
        } finally {
            session_write_close();
        }
    }

    ini_set('display_errors', '0');
    ini_set('log_errors', '1');
    $requestId = bin2hex(random_bytes(8));
    try {
        $config = require dirname(__DIR__) . '/config.php';
        $method = $_SERVER['REQUEST_METHOD'] ?? 'GET';
        $path = parse_url($_SERVER['REQUEST_URI'] ?? '/', PHP_URL_PATH);
        $response = sessionMiddleware(
            static fn (): array => routeNotebook($method, is_string($path) ? $path : '', $config),
            $config,
        );
    } catch (Throwable $error) {
        error_log(json_encode(['request_id' => $requestId, 'type' => get_class($error)], JSON_THROW_ON_ERROR));
        $response = reply('Internal error. Reference: ' . $requestId, 500);
    }
    http_response_code($response['status']);
    header('Content-Type: text/html; charset=utf-8');
    header('Cache-Control: no-store');
    header('X-Content-Type-Options: nosniff');
    header('X-Request-ID: ' . $requestId);
    foreach ($response['headers'] as $name => $value) {
        header("{$name}: {$value}");
    }
    echo $response['body'];
```

Trace a save through each decision

Read configuration and method/path. Session middleware bounds the body and opens the session. Owner is a random session-stable value; csrf is another random value. Owner never appears in the form. Calling next reaches the router: path, method, media type, token, then validateNote. **No file is written until those checks succeed.**

Rejected input returns 422 with the form. A notebook containing 100 notes returns 409. On success, read the list, save, store name/flash, and return 303. Finally closes the session before emission. With PHP’s default file session handler, the lock spans count then save; concurrent same-session requests cannot bypass the limit by reading the same count. A different handler needs equivalent locking.

The browser follows Location with GET /. The handler reads then removes flash, making Note saved appear once. Reading never saves again. One emitter sends status, headers, then body. Unexpected errors receive 500 and a log reference; logs contain neither note text nor cookies.

303 does not prevent deliberate repeated POSTs or every double-click scenario; another POST can create another note here. Idempotency keys are a separate extension when needed. Session expiry loses the browser-to-notebook association; old-file cleanup and account recovery are outside this exercise and documented in README.

Run and verify

From the notebook directory, run `php tests.php` first. Expect `PASS: 15 notebook checks` for validation, persistence, isolation, UTC, and output encoding. Then start the server:

```
php -S 127.0.0.1:8083 -t public public/index.php
```

Open the local notebook. Submit Omar and Learn PHP. Expect POST 303 followed by GET 200, Note saved, and a stored note. Refresh again: the message disappears while the note remains. A private window starts empty. Restarting the server preserves files; access also requires the same cookie and a still-valid session store.

Experiment	Required result
Empty name or name[]	422, no new file
Arabic and emoji	Correct storage and display
HTML in note text	Displayed literally, never executed
Missing/wrong CSRF	403, no save
GET /notes	405 with Allow: POST
/missing	404
JSON instead of form-urlencoded	415
Body over 4096 bytes	413
Corrupt stored JSON	500 with reference, never a false empty notebook
GET /storage/...	404; only public is exposed

Test corrupt files in separate test storage using NOTEBOOK_STORAGE, not your real notes. The repository's automated checks exercise sessions, isolation, CSRF, UTF-8, encoding, storage, and failure paths.

Debugging: collect evidence before changing code

1. Capture a small request reproducing the defect and record expected versus actual.
2. Locate the boundary: parsing, validation, storage, or presentation.

3. Inspect a value and type there without printing into the response.
4. Test one hypothesis and change one cause.
5. Add a regression test that fails before the fix and passes afterward.

A **breakpoint** pauses execution before a selected line. Xdebug is a PHP extension; the IDE is a client displaying variables and the call stack. Install a binary matching PHP's version/build using the official installation guide, then verify with `php --ri xdebug`. Local configuration, with zend_extension pointing to your actual binary:

```
xdebug.mode=debug
xdebug.start_with_request=trigger
xdebug.client_host=127.0.0.1
xdebug.client_port=9003
```

Start the IDE listener and configure path mappings if PHP runs in a container. Set a breakpoint on `$result = validateNote($_POST);` and submit with XDEBUG_TRIGGER via a browser helper/cookie. **Step Into** enters a function, **Step Over** executes the line, and **Step Out** returns to its caller. Watch `$_POST['name']`, errors, and the stack: index→middleware→route→validateNote.

For PowerShell CLI, set `$env:XDEBUG_TRIGGER='1'`, run the script, then `Remove-Item Env:XDEBUG_TRIGGER`. If no pause occurs, check PHP binary, ini, listener, trigger, and mapping. CLI loading Xdebug does not mean FPM loads it. See step debugging. Production should use protected logs, request IDs, and display_errors=Off rather than a publicly enabled debugger. Lesson 11 explains configuration. Log sanitized fields, never the entire session.

Use the tools on a real project

Return to lesson 10's composer-demo: `composer install`, `composer check`, then `composer audit`. PHPUnit checks results, PHPStan checks contracts, and PHP-CS-Fixer checks formatting. The lock pins versions; do not delete it to fix a failing test. Deploy using install from the lock and check-platform-reqs against the target platform, with separate development/production PHP settings.

Further practice: the earlier complete programs remain available

The original PHP lab includes complete `total.php`, `orders.php`, and `stream-lab.php` programs:

- `php total.php 12.50 3.25` → `15.75`. Amounts are converted to integer minor units as text: enforce `\A` / `\z`, pad the fraction to two digits, compare digit length and lexicographic order against PHP_INT_MAX **before conversion**. Addition checks `left <= PHP_INT_MAX - right`. Test 0, 12, 12.5, 12.50; reject -1, 1.234, trailing newlines, and oversized amounts. `php tests.php values` checks these boundaries.
- `php orders.php fixtures/orders.json` → `orders=2`, `items=3`, `total_minor=2999` for 1250×2 and 499×1. Broken JSON returns `ERROR invalid JSON` and exit code 2. Separate reading, validation, calculation, and output; test zero quantity, negative price, and overflow.
- `php stream-lab.php fixtures/large.csv` processes sequentially. Test empty/missing files and long lines. Generators or SplFileObject cannot make an unbounded single line safe. Observe count and memory; distinguish read failure from empty input.

Lesson 15's http-demo contains the complete JSON endpoint and 400/422/415/413 distinctions. Each program needs a README, fixtures, and checks; running from a clean copy reveals forgotten local dependencies.

Predict, debug, complete

▼ Predict a successful save followed by two refreshes

POST saves once and returns 303. The first GET shows and removes flash; the next displays the note without flash or another save. Repeating POST itself is different and may add a note.

▼ Debug name[]=Omar causing TypeError

An array reached a string function before validation. Check is_string in validateNote before trim/mb_strlen, then assert 422 and an unchanged file count.

▼ Complete a stored-XSS regression test

Save text such as <img src=x onerror=alert(1)>, then inspect the actual response: encoded text appears in note-text and no corresponding img element is created. Testing escapeHtml alone misses integration failures.

▼ Corrupt JSON produces an empty notebook and success. What is wrong?

The failure was swallowed. Throw on decode/shape failure, log a reference, and return 500 without sensitive details. Do not change expectations to accommodate data loss.

▼ Money totals work normally but fail at the integer limit. First step?

Capture boundary and overflow cases in tests; check before multiplication/addition. Overflow may already convert a result to float by the time you inspect it.

Completing this lesson means tracing one request from form through storage and response, proving a rejected request has no effect, and explaining each HTTP result.

04

TRACK · 20 LESSONS

PHP runtime and production

Progressive explanations, practical examples, and corrections that take each idea into real code.

PHP runtime and production

PHP Runtime & Production

From Composer, OPcache, and PHP-FPM to testing, Redis, queues, deployment, observability, resilience, and recovery.

This path explains the tools and layers around PHP after the application code is written. It gradually separates dependency management, HTTP clients, web serving, and PHP-FPM so the role of each part stays clear.

Why a separate section?

These subjects are neither OOP nor PHP syntax. They form the **runtime and production** layer: reproducible dependencies, autoloading, efficient execution, outbound HTTP, process capacity, and safe configuration.

Learning path

1. Composer and dependency management
2. cURL and HTTP clients
3. OPcache and preloading
4. PHP memory and garbage collection
5. PHP-FPM and process management
6. Apache, Nginx, and FastCGI
7. Environment variables and configuration
8. PHP CLI, php.ini, and extensions
9. Logging and observability
10. Testing and code quality
11. Data caching and Redis
12. Queues, workers, and scheduling
13. Deployment, CI/CD, and containers
14. Dependency and supply-chain security
15. Profiling, SLI/SLO, and OpenTelemetry — Flame graphs, percentiles, objectives, and trace propagation.
16. Resilience and worker management — Circuit breakers, bulkheads, backpressure, signals, and systemd.
17. Disaster recovery and restore testing — RPO, RTO, backups, restores, and game days.
18. Sessions and shared state at scale — shared storage, locking, TTL, and release compatibility.
19. Database connections and runtime operations — connection budgets, timeouts, pooling, and replicas.
20. Long-running PHP and worker runtimes — FrankenPHP, RoadRunner, Swoole, state leakage, and reset.

The practice protocol

Every exercise follows five steps: predict the result, run it, introduce a controlled failure, collect evidence from output, logs, or metrics, then repair the cause and repeat. A successful command without reviewable acceptance evidence is insufficient.

```
Client
  -> Nginx / Apache (TLS, static files, routing)
  -> FastCGI
  -> PHP-FPM pool
  -> Composer autoloader + OPcache
  -> Application / Database / External API
  <- HTTP response
```

> Examples target PHP 8.x. Server snippets are educational baselines; adapt users, paths, and capacity to the actual operating system and workload.

Official references

- Composer basic usage and autoloader optimization
- cURL and libcurl documentation
- OPcache and preloading
- PHP garbage collection
- PHP-FPM
- Session configuration and PDO connection management
- FrankenPHP worker mode
- Nginx FastCGI and Apache MPMs
- getenv() and `$_ENV`

1. Composer and dependency management

composer.json, composer.lock, install, update, require-dev, PSR-4, and production installs.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

- **Autoloading:** Loading a class file when the class is first used.
- **Composer:** PHP dependency manager that installs libraries and prepares autoloading.

New terms in this lesson

This lesson introduces no extra technical label that needs memorizing; its new ideas are explained where they first appear.

Composer: PHP dependency management

Composer resolves project libraries and versions and generates an autoloader. It does not install PHP itself; it resolves constraints in `composer.json` and normally installs packages under `vendor/`.

The two core files

- `composer.json` declares requirements, allowed version ranges, autoloading, and scripts.
- `composer.lock` records the exact resolved versions and metadata for reproducible installs.

```
{
  "require": {
    "php": "^8.3",
    "guzzlehttp/guzzle": "^7.9"
  },
  "require-dev": {
    "phpunit/phpunit": "^11.0"
  },
  "autoload": {
    "psr-4": {
      "App\\": "src/"
    }
  }
}
```

Commit the lock file for applications. A published library's consumers still resolve that library inside their own application dependency graph.

install versus update

Command	Purpose
`composer install`	installs exact locked versions; use in CI and production
`composer update`	resolves allowed constraints again and rewrites the lock
`composer update vendor/package`	narrows the requested update, with related dependency changes possible

```
composer install
composer require monolog/monolog
composer require --dev phpunit/phpunit
composer update guzzlehttp/guzzle --with-all-dependencies
```

`require` normally edits the manifest, resolves the lock, and installs. `--dev` writes the package under `require-dev`.

PSR-4 autoloading

```
composer dump-autoload
```

```
<?php
declare(strict_types=1);

require dirname(__DIR__) . '/vendor/autoload.php';

$service = new App\Billing\InvoiceService();
```

`App\Billing\InvoiceService` conventionally maps to `src/Billing/InvoiceService.php`. Linux filesystems are commonly case-sensitive, so incorrect casing that survives on Windows can fail after deployment.

```
src/Billing/InvoiceService.php
```

A production install

```
composer validate --strict
composer install --no-dev --prefer-dist --optimize-autoloader --no-interaction
composer audit --locked
composer check-platform-reqs --lock --no-dev
```

- `--no-dev` excludes development packages at deployment time.
- `--optimize-autoloader` builds a faster class map and is suited to production.
- Test `--classmap-authoritative` carefully because runtime-generated classes may be absent.
- Do not hide incompatible PHP or extension requirements with `--ignore-platform-reqs`.
- Composer scripts and plugins can execute code during installation; review untrusted projects before running them.

Recommended workflow

1. Resolve dependency changes during development.
2. Review both manifest and lock diffs.
3. Run tests and `composer audit`.
4. Commit both files; normally exclude `vendor/`.
5. Deploy with `composer install`, never an uncontrolled `update`.

Operational problem

▼ Why commit composer.lock for an application?

It reproduces tested versions across environments; composer.json states constraints while the lock records the resolved set.

Run and verify

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Run this checkpoint inside `examples/php-labs` or the extracted lab package:

```
composer validate --strict
composer audit --locked
composer check-platform-reqs --lock --no-dev
```

Success criterion: All commands exit 0; `composer.lock` is valid, its contents have no known advisory, and the real PHP version and extensions satisfy production requirements. The lock makes `-locked` reproducible; `validate` alone neither creates nor guarantees a lock.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

Connect the ideas

Treat SemVer as a compatibility contract, not a quality guarantee: `^` and `~` open different ranges while the lock records actual choices. Validate platform requirements and extensions, configure private repositories without committed tokens, and review scripts because Composer executes them.

Practice cycle

Write your prediction before running the example and record the output. Introduce one controlled failure, collect evidence from logs or metrics, repair the cause, and rerun the check to prove the fix handles the fault instead of hiding it.

Try it yourself

Explain why install from the lock differs from update in production.

2. cURL and HTTP clients

libcurl, PHP's extension, Guzzle, timeouts, TLS, JSON requests, and error handling.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **HTTP:** The rules used to exchange requests and responses on the web.
- **DNS:** The service that translates a site name into a numeric network address.
- **IP:** A numeric address that identifies a device or network interface.
- **URL:** The complete address of a resource such as a page or API endpoint.
- **TLS:** An encryption layer that protects data while it moves between two parties.
- **API:** A defined interface through which one program requests data or actions from another.
- **Token:** A value representing identity or permission without resending a password.
- **Worker:** A background process that takes jobs from a queue and runs them.
- **Log:** A timestamped record of an application event.

From cURL to a reliable HTTP client

The cURL project produces the `curl` command-line tool and the `libcurl` transfer library. PHP's `ext-curl` binds PHP to libcurl. It is an outbound client, not a web server or a socket server.

A complete request

```
<?php
declare(strict_types=1);

$handle = curl_init('https://api.example.com/v1/users/42');
curl_setopt_array($handle, [
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_HTTPHEADER => [
        'Accept: application/json',
        'User-Agent: OmarDevNotes/1.0',
    ],
    CURLOPT_CONNECTTIMEOUT => 3,
    CURLOPT_TIMEOUT => 10,
    CURLOPT_FOLLOWLOCATION => false,
]);

$body = curl_exec($handle);
if ($body === false) {
    $message = curl_error($handle);
    $code = curl_errno($handle);
    curl_close($handle);
    throw new RuntimeException("Network error {$code}: {$message}");
}

$status = curl_getinfo($handle, CURLINFO_RESPONSE_CODE);
curl_close($handle);

if ($status < 200 || $status >= 300) {
    throw new RuntimeException("Unexpected HTTP status: {$status}");
}

$data = json_decode($body, true, flags: JSON_THROW_ON_ERROR);
```

Keep three failure classes separate: transport failure, non-successful HTTP response, and invalid response content.

Sending JSON

```
$payload = json_encode(
    ['email' => 'user@example.com', 'active' => true],
    JSON_THROW_ON_ERROR
);

$handle = curl_init('https://api.example.com/v1/users');
curl_setopt_array($handle, [
    CURLOPT_POST => true,
    CURLOPT_POSTFIELDS => $payload,
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_HTTPHEADER => [
        'Content-Type: application/json',
        'Accept: application/json',
        'Authorization: Bearer ' . $token,
    ],
    CURLOPT_CONNECTTIMEOUT_MS => 1000,
    CURLOPT_TIMEOUT_MS => 5000,
]);
```

Never log authorization headers, cookies, or sensitive bodies.

TLS, redirects, and SSRF

Never disable peer or hostname verification in production. Repair the CA trust configuration instead. Redirects can move a request to a different destination, so set a maximum and validate destinations.

Do not accept an arbitrary user-controlled URL in server-side fetching. Validate scheme and host and block internal, loopback, and metadata endpoints as required by the environment.

Guzzle and standards

Guzzle offers middleware, promises, and pooling. It is not permanently tied to cURL; its handler can use cURL or PHP streams.

```
use GuzzleHttp\Client;

$client = new Client([
    'base_uri' => 'https://api.example.com/',
    'connect_timeout' => 3,
    'timeout' => 10,
]);

$response = $client->get('v1/users/42', [
    'headers' => ['Accept' => 'application/json'],
]);
```

PSR-7 models HTTP messages and PSR-18 defines a client interface. Depending on an interface improves replacement and testing.

Production rules

- Set both connection and total timeouts so a slow dependency does not occupy an FPM worker indefinitely.
- Retry only transient failures, with exponential backoff and jitter.
- Retry non-idempotent writes only with a server-supported idempotency key.
- Respect `Retry-After` for 429/503.
- Limit response sizes and stream large bodies.
- Use `curl_multi_*` or a client pool for bounded concurrency, never unbounded fan-out.

Operational problem

▼ What boundaries does an HTTP client need?

Connection and total timeouts, size and redirect limits, TLS verification, and retries only for safe operations.

Run and verify

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Execute this checkpoint inside the lesson environment:

```
php http-client-lab.php
```

Success criterion: The lab prints `status=200` for success and `timeout_handled=yes` for a slow server, and it never treats 500 as success.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

Connect the ideas

Separate connect, TLS/read, and total timeouts and propagate cancellation deadlines. Before fetching a user-supplied URL, prevent SSRF with scheme, host, DNS/IP policy, redirect controls, and re-resolution. Record attempts, latency, and status without tokens or sensitive bodies.

Practice cycle

Write your prediction before running the example and record the output. Introduce one controlled failure, collect evidence from logs or metrics, repair the cause, and rerun the check to prove the fix handles the fault instead of hiding it.

Try it yourself

Test a URL redirecting to a private IP and a server that connects but never sends a body.

3. OPcache and preloading

PHP bytecode caching, OPcache configuration and monitoring, data-cache differences, and preloading limits.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **HTTP:** The rules used to exchange requests and responses on the web.
- **API:** A defined interface through which one program requests data or actions from another.
- **Cache:** A temporary copy that reduces waiting and repeated work.
- **CLI:** A text-based interface controlled by typed commands.
- **Composer:** PHP dependency manager that installs libraries and prepares autoloading.
- **OPcache:** Memory that keeps compiled PHP instructions for later requests.
- **Redis:** A fast in-memory store used for caches, sessions, and queues.

Avoid repeated parsing and compilation

PHP normally parses source and compiles it to opcodes before execution. OPcache keeps compiled bytecode in shared memory so later requests can skip much of that work.

```
PHP source -> parse -> compile to opcodes -> execute
                       |
                       +-> OPcache shared memory
```

OPcache does not cache final HTML, SQL results, or API responses. Those belong to HTTP or application caches such as Redis.

Production baseline

```
opcache.enable=1
opcache.memory_consumption=256
opcache.interned_strings_buffer=16
opcache.max_accelerated_files=20000
opcache.validate_timestamps=0
```

Treat the numbers as a starting point, not universal truth.

- Timestamp validation detects changed scripts according to `revalidate_freq`.
- With validation disabled, restart or gracefully reload FPM on every release.
- CLI OPcache is configured separately through `opcache.enable_cli` and may not help short commands.

Observe before tuning

```
$status = opcache_get_status(false);
if ($status !== false) {
    printf(
        "hit-rate=%.2f%% used=%d free=%d\n",
        $status['opcache_statistics']['opcache_hit_rate'],
        $status['memory_usage']['used_memory'],
        $status['memory_usage']['free_memory'],
    );
}
```

Keep status data behind internal authorization because it can expose file paths. Watch `cache_full`, OOM/hash restarts, hit rate after warm-up, and the number of cached keys.

Preloading

PHP 7.4+ can execute a preload file once at server startup and keep loaded functions, classes, interfaces, and traits available:

```
opcache.preload=/var/www/app/config/preload.php
opcache.preload_user=www-data
```

```
foreach ([
    __DIR__ . '/../src/Domain/Money.php',
    __DIR__ . '/../src/Domain/Order.php',
] as $file) {
    opcache_compile_file($file);
}
```

Preloading increases baseline memory, needs a process restart for code changes, and is useful only for persistent processes. Measure it rather than preloading everything. The PHP manual also notes that preloading is not supported on Windows.

Deployment sequence

Install exact Composer dependencies, optimize the autoloader, test a separate release directory, switch releases atomically, then gracefully reload FPM. Composer's class map speeds file discovery; OPcache caches compiled bytecode. They optimize different stages.

Operational problem

▼ Does OPcache cache request results?

No. It caches compiled bytecode; an application data cache stores computed or retrieved data.

Run and verify

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Execute this checkpoint inside the lesson environment:

```
php -d opcache.enable_cli=1 -r "var_export(opcache_get_status(false) !== false);"
```

Success criterion: The output is `true`. Then compare hits and misses before and after repeat load; enabling OPcache alone is not proof of improvement.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

Connect the ideas

Measure memory consumption, interned strings, hit/miss, and wasted percentage instead of copying a size. Invalidation and timestamp policy must match atomic deployment. JIT does not speed every workload and is separate from the core OPcache benefit. Preloading persists for process lifetime and needs restart.

Practice cycle

Write your prediction before running the example and record the output. Introduce one controlled failure, collect evidence from logs or metrics, repair the cause, and rerun the check to prove the fix handles the fault instead of hiding it.

Try it yourself

Collect metrics before and after repeated load and explain eviction or restart behavior.

4. PHP memory and garbage collection

zvals, reference counting, copy-on-write, references, objects, cycles, and memory measurement.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Worker:** A background process that takes jobs from a queue and runs them.
- **OPcache:** Memory that keeps compiled PHP instructions for later requests.

How PHP represents values

The Zend Engine represents a PHP value with an internal `zval`. Compound data such as strings, arrays, and objects can reference additional refcounted structures.

Version-aware correction

The simple `type + value + is_ref + refcount` description reflects PHP 5-era internals. PHP 7/8 reorganized zvals and references. Learn observable semantics instead of depending on a fixed internal memory layout.

Reference counting and copy-on-write

```
$first = ['name' => 'Omar'];
$second = $first;

unset($first);
// The data remains alive because $second still uses it.
```

```
$original = range(1, 100_000);
$copy = $original;       // shared for now
$copy[] = 100_001;       // modified data must now separate
```

PHP generally delays copying large arrays and strings until one copy changes. This saves work, although the eventual write can produce a temporary memory spike.

Do not interpret a debug refcount as a perfect number of source-level variables. Temporaries, interning, and engine details affect it.

Objects and references

```
$a = new stdClass();
$a->count = 1;
$b = $a;
$b->count++;

echo $a->count; // 2
```

Both variables hold a handle to the same object. Use `clone` for a separate object and implement `__clone()` where nested mutable state needs copying.

```
$value = 10;
$alias =& $value;
$alias = 20;
echo $value; // 20
```

A PHP reference is an alias, not a general C pointer. Do not use `&` as a performance trick.

Cyclic garbage

Refcounting alone cannot free an unreachable cycle:

```
final class Node
{
    public ?Node $next = null;
}

$a = new Node();
$b = new Node();
$a->next = $b;
$b->next = $a;
unset($a, $b);
```

The cyclic garbage collector records candidates and periodically detects unreachable cycles. `gc_collect_cycles()` can force a pass, but it should be a measured tool for long-running workers, not a call after every operation.

```
$cycles = gc_collect_cycles();
printf("collected=%d\n", $cycles);
```

Requests, workers, and measurement

Request-owned memory is normally released at request shutdown, while an FPM worker remains alive. Extensions, caches, or allocator fragmentation can keep its resident size high. `pm.max_requests` recycles workers, but does not replace fixing leaks in long-running code.

```
$before = memory_get_usage(true);
$rows = loadReportRows();

printf(
    "current=%s peak=%s delta=%s\n",
    number_format(memory_get_usage(true)),
    number_format(memory_get_peak_usage(true)),
    number_format(memory_get_usage(true) - $before),
);
```

The `true` form reports memory obtained from the system and may exceed currently used payload memory.

Practical reductions

- Stream or paginate instead of loading entire datasets.
- Avoid `fetchAll()` for huge results.
- Watch closures that capture large object graphs.
- Release proven large references inside long loops.
- Keep OPcache/shared caches conceptually separate from the request heap.
- Profile before merely increasing `memory_limit`.

Operational problem

▼ Why can a long-lived worker grow in memory?

References, caches, and cycles may survive jobs; monitor usage, release resources, and recycle workers by policy.

Run and verify

The file compares a 100000-element array with a calculation without an array and prints memory/peak. Peak is cumulative within a process and need not fall after unset; values depend on the build and allocator.

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Execute this checkpoint inside the lesson environment:

```
php memory-lab.php
```

Extended integration exercise target: The script prints current and peak memory for every stage, and peak memory stays below the budget set for the large fixture.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

Connect the ideas

Generators and streaming reduce the working set when all data is not needed at once. In long-running workers, release references and resources between jobs and observe growth rather than one snapshot. `memory_limit` does not equal total RSS because extensions, native libraries, and mapped files may sit outside it.

Practice cycle

Write your prediction before running the example and record the output. Introduce one controlled failure, collect evidence from logs or metrics, repair the cause, and rerun the check to prove the fix handles the fault instead of hiding it.

Try it yourself

Process 100 jobs and record RSS every ten to expose retention.

PHP runtime and production

5. PHP-FPM and process management

FastCGI, master and workers, pools, static/dynamic/ondemand modes, capacity, and monitoring.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **HTTP:** The rules used to exchange requests and responses on the web.
- **Queue:** A line of background jobs waiting to be processed.
- **Worker:** A background process that takes jobs from a queue and runs them.
- **OPcache:** Memory that keeps compiled PHP instructions for later requests.
- **PHP-FPM:** A process manager that runs PHP workers for a web server.
- **FastCGI:** A protocol used to send execution work to PHP-FPM.
- **Nginx:** A web server that serves files or forwards PHP requests.
- **Log:** A timestamped record of an application event.

PHP-FPM is not an HTTP server

PHP-FPM is PHP's primary **FastCGI Process Manager**. A trusted web server sends it FastCGI requests; FPM manages PHP processes and returns responses.

```
Nginx/Apache -> FastCGI socket -> FPM master -> worker -> PHP response
```

Never expose FPM to an untrusted network. Bind a Unix socket or an internal interface and configure socket permissions and `listen.allowed_clients` where applicable.

Master, workers, and pools

- The master reads pool configuration and controls lifecycle.
- A worker normally executes one PHP request at a time.
- A pool groups workers under a user/group, listener, limits, and PHP settings.

Separate applications into pools and OS identities where isolation matters.

Process-manager modes

```
pm = static
pm.max_children = 20
```

```
pm = dynamic
pm.max_children = 40
pm.start_servers = 8
pm.min_spare_servers = 4
pm.max_spare_servers = 12
```

```
pm = ondemand
pm.max_children = 20
pm.process_idle_timeout = 10s
```

`static` avoids spawn latency but reserves memory. `dynamic` is a common balance. `ondemand` helps low-traffic pools at the cost of cold starts.

Capacity from measurements

Estimate:

```
RAM budget for FPM / realistic high-percentile worker RSS
```

Leave headroom for the operating system, web server, OPcache, and other services. Do not divide RAM by `memory_limit`; that limit is not the worker's typical resident set.

Arrival rate also matters. At 100 requests/s and a 200 ms average service time, average concurrency is roughly 20 workers. Peaks and tail latency need additional headroom, while excessive workers can overload the database.

Operational settings

```
pm.max_requests = 500
request_terminate_timeout = 30s
request_terminate_timeout_track_finished = yes
request_slowlog_timeout = 3s
slowlog = /var/log/php-fpm/app-slow.log
catch_workers_output = yes
ping.path = /fpm-ping
pm.status_path = /fpm-status
```

```
concurrency = throughput × latency = 100 × 0.2 = 20 workers
```

Monitor active/idle processes, listen queue, slow traces, and `max children reached`. Keep status endpoints private.

Use graceful reloads during deployment. `fastcgi_finish_request()` can flush the response before brief follow-up work, but it still occupies a worker; durable or heavy tasks belong in a queue worker. By default FPM does not apply `request_terminate_timeout` after request completion or `fastcgi_finish_request()`; `request_terminate_timeout_track_finished` extends the limit to that phase.

Operational problem

▼ Why not raise FPM workers without calculation?

They can exhaust memory and overload CPU or databases; size from process memory, workload, and downstream limits.

Run and verify

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Start the `examples/php-labs/production` environment as described in its README, then run:

```
curl -sS -o /dev/null -w "status=%{http_code} total=%{time_total}
" http://localhost/health
```

Success criterion: The health check returns 200 and `docker compose ps` reports both services healthy. Under controlled load the worker count stays bounded; correlate delay with the FPM queue rather than guessing. In Windows PowerShell, replace `/dev/null` with `NUL`.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

Connect the ideas

Expose the FPM status page and slowlog only on a protected internal path to inspect active/idle workers, listen queue, and slow requests. Measure memory per child under load and reserve headroom. In containers, do not let autoscaling hide bad max_children, and align graceful termination with platform deadlines.

Practice cycle

Write your prediction before running the example and record the output. Introduce one controlled failure, collect evidence from logs or metrics, repair the cause, and rerun the check to prove the fix handles the fault instead of hiding it.

Try it yourself

Correlate queue growth with slowlog and RSS before changing worker count.

PHP runtime and production

6. Apache, Nginx, and FastCGI

Web-server responsibilities, Apache MPMs, Nginx's event model, and PHP-FPM over Unix or TCP sockets.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **HTTP:** The rules used to exchange requests and responses on the web.
- **IP:** A numeric address that identifies a device or network interface.
- **TCP:** A transport method that checks that data arrives completely and in order.
- **TLS:** An encryption layer that protects data while it moves between two parties.
- **Proxy:** An intermediary that receives a request and forwards it according to rules.
- **Worker:** A background process that takes jobs from a queue and runs them.
- **PHP-FPM:** A process manager that runs PHP workers for a web server.
- **FastCGI:** A protocol used to send execution work to PHP-FPM.
- **Nginx:** A web server that serves files or forwards PHP requests.
- **Observability:** Understanding system state from logs, metrics, and traces.

Where the web server ends and PHP begins

The web server terminates HTTP/TLS, serves static files, applies request limits and routing, and forwards PHP execution. PHP-FPM executes PHP.

Correcting the Apache/Nginx comparison

Apache is not universally “one process per connection.” Its active MPM determines the model:

- `prefork`: multiple non-threaded processes.
- `worker`: processes containing threads.
- `event`: threaded, with improved keep-alive handling.

Nginx workers drive event loops and can manage many connections with few processes. That does not make application execution asynchronous: an active PHP request still occupies an FPM worker.

Apache with FPM

```
<VirtualHost *:443>
    ServerName example.com
    DocumentRoot /var/www/app/public

    <Directory /var/www/app/public>
        AllowOverride None
        Require all granted
        FallbackResource /index.php
    </Directory>

    <FilesMatch \.php$>
        SetHandler "proxy:unix:/run/php/app.sock|fcgi://localhost/"
    </FilesMatch>
</VirtualHost>
```

`.htaccess` is useful when hosting does not grant main configuration access. On a server you control, central VirtualHost configuration with `AllowOverride None` avoids per-directory checks and scattered policy.

Nginx with FPM

```
server {
    listen 443 ssl;
    server_name example.com;
    root /var/www/app/public;
    index index.php;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        try_files $uri =404;
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        fastcgi_param HTTP_PROXY "";
        fastcgi_pass unix:/run/php/app.sock;
        fastcgi_read_timeout 30s;
    }

    location ~ /\. {
        deny all;
    }
}
```

Where possible, forwarding only the front controller `/index.php` reduces the execution surface:

```
location = /index.php {
    include fastcgi_params;
    fastcgi_param SCRIPT_FILENAME $document_root/index.php;
    fastcgi_pass unix:/run/php/app.sock;
}
```

Keep `.env`, `vendor/`, and executable uploads outside the public document root.

Unix versus TCP

```
fastcgi_pass unix:/run/php/app.sock;
# or
fastcgi_pass 127.0.0.1:9000;
```

Unix sockets suit services on one host and need correct filesystem ownership. TCP is common across containers or hosts and must remain on a private, firewalled network. Correctness and observability matter more than simplistic performance claims.

`SCRIPT_FILENAME` identifies the script to PHP. A wrong value can yield “Primary script unknown” or expose unintended paths.

Trust forwarded IP/protocol headers only from explicitly trusted proxies. Align web-server upstream timeouts with FPM request limits and the endpoint’s service objective.

Operational problem

▼ What does a 502 between Nginx and PHP-FPM mean?

Nginx received no valid FastCGI response; inspect the socket, service, timeout, and logs before page logic.

Run and verify

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Start `examples/php-labs/production` as described in its README, then run:

```
curl -sS -D - http://127.0.0.1:8080/health -o /dev/null
curl -fsS http://127.0.0.1:8080/
```

Success criterion: The first request shows status and headers from PHP, while the second returns a static file from Nginx. A request for `/health.php` or a missing path returns 404 and exposes no internal filesystem path. In Windows PowerShell, replace `/dev/null` with `NUL`.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

Connect the ideas

Define ownership of TLS, HTTP/2, compression, and static files. Tune request/response buffering, timeouts, and body limits by endpoint. Bind SCRIPT_FILENAME to trusted paths, prevent path-info confusion, and add security headers in one clear layer without conflicting duplication.

Practice cycle

Write your prediction before running the example and record the output. Introduce one controlled failure, collect evidence from logs or metrics, repair the cause, and rerun the check to prove the fix handles the fault instead of hiding it.

Try it yourself

Test static, PHP, a large upload, and a missing path and identify the responding layer.

7. Environment variables and configuration

getenv, $_ENV, putenv, .env files, typed validation, and secret handling with PHP-FPM.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Worker:** A background process that takes jobs from a queue and runs them.
- **Boolean:** A logical value with only two states: true or false.
- **Function:** A named, reusable block of code with one defined job.
- **Variable:** A name used to access a value that may change at runtime.
- **PHP-FPM:** A process manager that runs PHP workers for a web server.
- **Container:** An isolated environment packaging an application and its dependencies.

Separate configuration from code

Environment variables are values supplied to a process by the operating system, container, or service manager.

```
OS / container / service manager
              -> PHP-FPM master
              -> worker process
              -> getenv() / $_SERVER / $_ENV
```

PHP access paths

```
$dsn = getenv('DATABASE_DSN');
if ($dsn === false || $dsn === '') {
    throw new RuntimeException('DATABASE_DSN is required');
}
```

Use `=== false`; the string `"0"` is a valid value even though it is falsy.

- `getenv()` reads an environment variable.

- `$_ENV` contains values imported by PHP and may be empty when `variables_order` excludes `E`.
- `$_SERVER` can contain environment/CGI values depending on the SAPI.
- `putenv()` changes the current process environment for the request and does not automatically synchronize `$_ENV`.

PHP does not load `.env` files by itself. A framework, a package such as `vlucas/phpdotenv`, or custom bootstrap code does so. Production can inject values directly with no file.

Development files

```
composer require vlucas/phpdotenv
```

```
require dirname(__DIR__) . '/vendor/autoload.php';

$dotenv = Dotenv\Dotenv::createImmutable(dirname(__DIR__));
$dotenv->safeLoad();
$dotenv->required(['APP_ENV', 'DATABASE_DSN']);
```

```
# .env.example - names and non-secret examples
APP_ENV=local
APP_DEBUG=false
DATABASE_DSN=mysql:host=127.0.0.1;dbname=app;charset=utf8mb4
```

Commit a non-secret `.env.example`, ignore the real `.env`, and keep it outside the document root. If a secret leaks, revoke or rotate it; deleting one commit does not invalidate the credential.

Parse once into typed configuration

```
final readonly class AppConfig
{
    public function __construct(
        public string $environment,
        public bool $debug,
        public string $databaseDsn,
        public int $httpTimeoutMs,
    ) {}

    public static function fromEnvironment(): self
    {
        $dsn = getenv('DATABASE_DSN');
        if ($dsn === false || $dsn === '') {
            throw new RuntimeException('DATABASE_DSN is required');
        }

        $debug = filter_var(
            getenv('APP_DEBUG') ?: 'false',
            FILTER_VALIDATE_BOOL,
            FILTER_NULL_ON_FAILURE,
        );
        if ($debug === null) {
            throw new RuntimeException('APP_DEBUG must be true or false');
        }

        $timeout = filter_var(
            getenv('HTTP_TIMEOUT_MS') ?: '5000',
            FILTER_VALIDATE_INT,
            ['options' => ['min_range' => 100, 'max_range' => 60_000]],
        );
        if ($timeout === false) {
            throw new RuntimeException('HTTP_TIMEOUT_MS is invalid');
        }

        return new self(
            getenv('APP_ENV') ?: 'production',
            $debug,
            $dsn,
            $timeout,
        );
    }
}
```

Validate once at startup and inject configuration instead of calling `getenv()` throughout domain classes.

FPM and secrets

FPM commonly clears worker environments. Pass an explicit allowlist through a pool or service manager:

```
clear_env = yes
env[APP_ENV] = $APP_ENV
env[DATABASE_DSN] = $DATABASE_DSN
```

Environment variables are configuration transport, not encryption. They may be visible to diagnostics or child processes. Use a secret manager for rotation, auditing, and policy; never expose `phpinfo()`, full environment dumps, or server secrets to a browser.

```
hard-coded safe defaults
  <- committed non-secret config
  <- environment-specific values
  <- secret manager
```

Operational problem

▼ Why must secrets stay out of Git?

History retains them and clones spread them; use a secret manager and rotate any exposed secret.

Run and verify

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Execute this checkpoint inside the lesson environment:

```
php config-check.php
```

Success criterion: Startup fails early when a required variable is absent and prints loaded configuration names, never secret values.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

Connect the ideas

Build a typed configuration schema that validates once at startup and explicitly parses durations, sizes, and booleans. Define precedence among defaults, files, environment, and secret managers. Keep configuration immutable during a request and rotate secrets with an overlap window without printing values.

Practice cycle

Write your prediction before running the example and record the output. Introduce one controlled failure, collect evidence from logs or metrics, repair the cause, and rerun the check to prove the fix handles the fault instead of hiding it.

Try it yourself

Remove a required variable, provide an ambiguous boolean, and verify fail-fast behavior.

PHP runtime and production

8. PHP CLI, php.ini, and extensions

SAPIs, configuration discovery, extensions, and separate development and production policy.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Runtime:** The period when a program is actually running.
- **HTTP:** The rules used to exchange requests and responses on the web.
- **CLI:** A text-based interface controlled by typed commands.

The SAPI changes the environment

PHP may run through CLI, FPM/FastCGI, or an Apache module. The same code can load different `php.ini` files, extensions, users, and environment values.

```
php -v
php --ini
php -m
php -i
php -r "echo PHP_SAPI, PHP_EOL;"
```

When code works in a terminal but not through HTTP, compare the executable, SAPI, loaded configuration, process user, and environment.

Configuration and extensions

`php --ini` shows the main file and scanned `conf.d` directory. Later files may override earlier values.

```
printf(
    "sapi=%s ini=%s memory=%s\n",
    PHP_SAPI,
    php_ini_loaded_file() ?: 'none',
    ini_get('memory_limit'),
);
```

Some directives cannot change at runtime. Declare extension requirements in Composer and run:

```
php --ri opcache
php --ri pdo_mysql
composer check-platform-reqs --lock --no-dev
```

Environment policy

Development can show detailed errors and load Xdebug. Production should hide errors, log them safely, enable OPcache, set resource limits, and remove unnecessary extensions. The distributed development/production ini files are starting points, not complete application policy.

CLI programs

```
#!/usr/bin/env php
<?php
declare(strict_types=1);

set_time_limit(0);
$options = getopt('', ['dry-run', 'limit:']);
```

Use meaningful exit codes, timeouts, logging, signal handling for workers, and a lock when duplicate execution is unsafe. `set_time_limit(0)` does not stop the operating system or orchestrator from terminating a process.

PHP lifecycle and upgrades

Saying an application supports PHP 8.x is insufficient. Record the minimum and maximum tested minor versions, support deadlines, and extension versions. Before upgrading:

1. Read the migration guide, backward-incompatible changes, and deprecations.
2. Run tests and static analysis on both versions in CI.
3. Build a new image with every extension instead of replacing only the binary.
4. Watch startup warnings, OPcache, and FPM errors in a canary.
5. Keep a rollback path compatible with schema and queue payloads.

Treat `E_DEPRECATED` as early CI work rather than output for users. `display_errors=Off` and `error_reporting=E_ALL` can coexist: record complete errors without exposing them in a response.

Startup failures and extension ABI

Check startup errors before declaring the service healthy. An extension compiled for another PHP ABI or thread-safety mode can fail before application code runs. Pin its source and version in the image, then run:

```
php -v
php --ini
php -m
php --ri opcache
composer check-platform-reqs --lock --no-dev
```

Compare CLI and FPM through a protected internal endpoint and deliberately test a missing extension and startup failure. Never expose `phpinfo()` publicly; it reveals paths, configuration, and environment data.

Operational problem

▼ Why can CLI and Web PHP behave differently?

They may use different SAPIs, ini files, extensions, and users; inspect each environment directly.

Run and verify

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Execute this checkpoint inside the lesson environment:

```
php --ini
```

Success criterion: Identify the CLI configuration file, then run `php -m` and confirm the required extension exists in that same execution environment.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

Connect the ideas

INI modes determine whether a setting changes in php.ini, per-directory configuration, or runtime. Compare `php -ini` with FPM configuration to avoid editing CLI only. A PECL extension must match PHP ABI, thread-safety mode, and platform; inspect startup errors after upgrades.

Practice cycle

Write your prediction before running the example and record the output. Introduce one controlled failure, collect evidence from logs or metrics, repair the cause, and rerun the check to prove the fix handles the fault instead of hiding it.

Try it yourself

Prove CLI and FPM load the intended version, ini file, and extension.

PHP runtime and production

9. Logging and observability

PSR-3, structured events, request IDs, metrics, traces, redaction, and actionable alerts.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **HTTP:** The rules used to exchange requests and responses on the web.
- **API:** A defined interface through which one program requests data or actions from another.
- **Cache:** A temporary copy that reduces waiting and repeated work.
- **Session:** Temporary server-side state used to recognize a user across requests.
- **Cookie:** A small value stored by the browser and sent with matching requests.
- **Queue:** A line of background jobs waiting to be processed.
- **Function:** A named, reusable block of code with one defined job.

Three signal types

- **Logs** are detailed searchable events.
- **Metrics** aggregate numbers over time.
- **Traces** follow work through services, databases, and queues.

Observability needs schemas, retention, access policy, and alerts—not random `error_log` calls.

PSR-3 structured logs

```
use Psr\Log\LoggerInterface;

final class Checkout
{
    public function __construct(private LoggerInterface $logger) {}

    public function run(Order $order, string $requestId): void
    {
        $this->logger->info('checkout.started', [
            'request_id' => $requestId,
            'order_id' => $order->id(),
        ]);
    }
}
```

Pass machine-readable context instead of concatenating text. Monolog is a common PSR-3 implementation.

Correlation and redaction

Create a request ID at the trusted edge and carry it into logs, outgoing HTTP, and queue metadata. Never log passwords, session IDs, tokens, authorization or cookie fields, encryption keys, or unreviewed personal bodies. Use allow-lists or tested redaction.

Metrics and alerts

Track request rate, error rate, latency distributions, and saturation. Useful PHP signals include FPM queue depth, max-children events, database pool pressure, API latency, queue lag, and cache hit ratio.

Alert on actionable user impact such as sustained `5xx` rate or p95 latency. Attach deployment version to signals to identify regressions.

Engine and application failures

Separate expected validation failures, application exceptions, dependency warnings, and fatal engine errors. A public response contains a safe status and error ID while the internal record keeps type, stack, and request ID without secrets.

```
set_exception_handler(static function (Throwable $error): void {
    $errorId = bin2hex(random_bytes(8));
    error_log(json_encode([
        'event' => 'request.failed',
        'error_id' => $errorId,
        'exception' => $error::class,
    ], JSON_THROW_ON_ERROR));
    http_response_code(500);
    echo json_encode(['error' => 'internal_error', 'error_id' => $errorId]);
});
```

`set_exception_handler` cannot catch every startup or parse failure. Send `log_errors` to stderr, monitor FPM and Nginx startup logs, and use a shutdown handler carefully to inspect `error_get_last()` without continuing a corrupted request. Test that `display_errors=Off` and that responses contain no path or stack.

Reference

- PSR-3 Logger Interface

Operational problem

▼ What makes a log traceable?

A structured event with time, severity, request or trace ID, and safe context without passwords or tokens.

Run and verify

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Execute this checkpoint inside the lesson environment:

```
php observability-lab.php 2> event.log
```

Success criterion: Every event contains timestamp, level, request_id, and message, and contains neither a password nor an Authorization header.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

Connect the ideas

Propagate trace context and span IDs through HTTP, queues, and databases, with sampling that preserves errors and rare requests without full cost. Control label cardinality and keep raw user IDs out of metrics. Define retention, redaction, and access while linking logs, metrics, and traces.

Practice cycle

Write your prediction before running the example and record the output. Introduce one controlled failure, collect evidence from logs or metrics, repair the cause, and rerun the check to prove the fix handles the fault instead of hiding it.

Try it yourself

Trace one request across three services and confirm no secret is exposed.

10. Testing and code quality

PHPUnit, unit/integration/feature tests, data providers, doubles, coverage, static analysis, and style.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **HTTP:** The rules used to exchange requests and responses on the web.
- **Function:** A named, reusable block of code with one defined job.

A practical test pyramid

- **Unit:** small logic without network or database.
- **Integration:** real collaboration with a database, filesystem, or adapter.
- **Feature/HTTP:** a complete request through the application.
- **End-to-end:** user-visible system behavior; fewer and more expensive.

Do not replace every collaborator with a mock. Test contracts at boundaries and behavior that matters to users.

PHPUnit

```
composer require --dev phpunit/phpunit
vendor/bin/phpunit
```

```
use PHPUnit\Framework\Attributes\DataProvider;
use PHPUnit\Framework\TestCase;

final class DiscountTest extends TestCase
{
    #[DataProvider('cases')]
    public function testDiscount(int $price, int $percent, int $expected): void
    {
        self::assertSame($expected, discount($price, $percent));
    }

    public static function cases(): iterable
    {
        yield 'none' => [1000, 0, 1000];
        yield 'ten percent' => [1000, 10, 900];
    }
}
```

Test boundaries, exceptions, and side effects. Make test names describe behavior.

Doubles and coverage

A stub returns prepared data, a fake is a lightweight working implementation, and a mock verifies interaction. Prefer fakes or stubs where possible; excessive mocks couple tests to implementation.

Coverage finds unexecuted code but cannot prove useful assertions. Focus on branches and critical behavior rather than an isolated 100% target.

Automated quality

Run PHPUnit, PHPStan/Psalm, and PHPCS/PHP-CS-Fixer locally and in CI with the same configuration. Control clocks and randomness, isolate database tests, avoid ordering dependencies, and never copy real production personal data into fixtures.

```
vendor/bin/phpstan analyse
vendor/bin/phpunit
vendor/bin/phpcs
```

Contract and integration tests

Test real boundary contracts: status, headers, and schema for HTTP; constraints and isolation for databases; and ack/retry semantics for queues. A fake improves speed but cannot prove driver or broker compatibility.

Give each suite layer a clear command and time budget:

```
unit -> contract -> integration -> HTTP smoke -> end-to-end
```

Run integration tests against temporary services and isolate data for every process. Parallel tests must not share a database name, queue, or clock. Use unique namespaces and clean resources in `finally`.

Mutation and flaky tests

Mutation testing changes an operator or condition and expects a test to fail. A surviving mutation means coverage reached the line without a strong assertion. Track a flaky test as a defect with an owner and deadline; rerunning CI until green hides the race.

Exercise failure paths: timeout, malformed response, deadlock, duplicate delivery, full disk, and SIGTERM. Inject time and randomness so the result is repeatable.

Reference

- PHPUnit Manual

Operational problem

▼ When should you choose an integration test?

When validating real boundaries such as a database or HTTP collaboration rather than isolated function logic.

Run and verify

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Execute this checkpoint inside the lesson environment:

```
php tests.php
```

Success criterion: Happy-path, boundary, and failure tests pass; deliberately break one condition once and confirm the relevant test fails before restoring it.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

Connect the ideas

Separate unit, integration, contract, and end-to-end tests by boundary. Control time, randomness, and external services, and run tests in parallel without shared state. Coverage does not prove assertion quality; mutation testing exposes tests that survive logic changes, and CI starts clean.

Practice cycle

Write your prediction before running the example and record the output. Introduce one controlled failure, collect evidence from logs or metrics, repair the cause, and rerun the check to prove the fix handles the fault instead of hiding it.

Try it yourself

Mutate an operator and prove a test fails, then run the suite in random order.

PHP runtime and production

11. Data caching and Redis

Cache-aside, TTL, invalidation, stampede protection, distributed locks, and failure policy.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **HTTP:** The rules used to exchange requests and responses on the web.
- **Cache:** A temporary copy that reduces waiting and repeated work.
- **Session:** Temporary server-side state used to recognize a user across requests.
- **Token:** A value representing identity or permission without resending a password.

Know the cache layer

OPcache stores PHP bytecode, HTTP caches store responses, application caches store computed/query data, and local in-process caches are fast but not shared. Redis is a common shared data cache, not automatically the source of truth.

Cache-aside

```
$key = "product:{$id}:v1";
$json = $redis->get($key);

if ($json === false) {
    $product = $repository->find($id);
    $json = json_encode($product, JSON_THROW_ON_ERROR);
    $redis->setex($key, 300, $json);
}

return json_decode($json, true, flags: JSON_THROW_ON_ERROR);
```

After a write, update the database first and then invalidate or refresh the cache according to an explicit consistency policy.

Keys, TTL, and stampedes

Version key names, add TTL jitter, avoid `KEYS *` in request paths, define serialization, and never treat an internal cache as a safe secret store. Popular expired keys can trigger a rebuild stampede; consider a short lock, stale-while-revalidate, early refresh, or single-flight.

A lock needs a unique ownership token and safe release. Distributed locking is not a universal consistency solution.

Failure policy

Decide whether the cache is an optional optimization or a required dependency such as a session store. Use short timeouts and monitor hit ratio, evictions, memory, and latency. Prevent cache failure from instantly flooding the database.

For each key document its writer, source of truth, invalidation event, maximum stale time, and failure behavior.

Operational problem

▼ What is the hardest cache problem?

Invalidation and consistency: define keys, TTL, and how cached values change when the source changes.

Run and verify

Local output database=11 stale_cache=10 followed by obsolete_refill=rejected demonstrates the race. This is an event-order model; TTL, outages, and load need real Redis.

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Execute this checkpoint inside the lesson environment:

```
php cache-lab.php
```

Extended integration exercise target: The first read reports `source=db`, the second `source=cache`, and a sequential read after update returns the new value; assert TTL separately. This does not establish freshness under concurrent refill.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

A real Redis exercise

```
docker compose -f production/compose.yaml -f production/compose.full.yaml up --build -d
curl -c cookies.txt -b cookies.txt http://127.0.0.1:8080/session
docker compose -f production/compose.yaml -f production/compose.full.yaml exec redis redis-cli info memory
docker compose -f production/compose.yaml -f production/compose.full.yaml stop redis
curl -i -c cookies.txt -b cookies.txt http://127.0.0.1:8080/session
```

Record latency and HTTP status before and after the outage. Restart Redis and prove that the service does not silently use stale local state. This complements rather than replaces the local race model.

Connect the ideas

Choose Redis structures by contract: strings, hashes, sets, sorted sets, and streams are not interchangeable. Understand eviction policy, memory limits, replication lag, and failover. Cache is not the source of truth; define stale tolerance, invalidation, stampede control, and fallback.

Practice cycle

Write your prediction before running the example and record the output. Introduce one controlled failure, collect evidence from logs or metrics, repair the cause, and rerun the check to prove the fix handles the fault instead of hiding it.

Try it yourself

Disable Redis under load and prove data remains correct even if performance drops.

Refill can race with invalidation

Trace this schedule: reader A misses cache and reads database value 10; writer B commits 11 and deletes the cache key; A resumes and caches 10. Deleting after commit therefore does not guarantee fresh reads. TTL bounds how long a stored entry survives after insertion; it does not automatically bound total age if an old read is delayed before insertion.

For stock/payment decisions, read and enforce constraints in the database. For a display cache, declare an acceptable stale window and bound reader lifetime; consider versioned keys with a trustworthy current-version source or coordinated version checks before publishing a refill. A lock with an expired lease needs fencing/version checks too. Exercise: reproduce the schedule with two paused clients; expect the naive cache to show 10, then verify the chosen fix prevents publishing an obsolete version.

12. Queues, workers, and scheduling

Jobs, at-least-once delivery, idempotency, retry, backoff, DLQs, graceful shutdown, and cron.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Runtime:** The period when a program is actually running.
- **HTTP:** The rules used to exchange requests and responses on the web.
- **API:** A defined interface through which one program requests data or actions from another.
- **Queue:** A line of background jobs waiting to be processed.
- **Worker:** A background process that takes jobs from a queue and runs them.

Why a queue?

Move slow or retryable work—mail, images, reports, API synchronization—outside the HTTP request. Respond after durably recording the intent, not after spawning an unmanaged background process. Version small payloads and send identifiers instead of serialized object graphs.

```
{"type":"SendReceipt","job_id":"job_01J...","order_id":42,"attempt":1}
```

Delivery and idempotency

Many systems provide **at-least-once** delivery, so a message can repeat. Deduplicate by job or business idempotency key and make result storage atomic within one transaction and a unique constraint. A transactional outbox ties a database change to publishing intent.

```
<?php
require __DIR__ . '/bootstrap.php';

$db = Lessons\connectInventory(':memory:');
Lessons\initializeInventory($db);
$first = Lessons\purchase($db, 'job-42', 1, 1, 2);
$again = Lessons\purchase($db, 'job-42', 1, 1, 2);
echo json_encode([$first, $again], JSON_THROW_ON_ERROR), PHP_EOL;
```

Retry policy

Retry transient failures with capped exponential backoff and jitter. Do not retry permanent validation or credential errors unchanged. After a limit, move the message to a dead-letter queue with diagnosis and alerting.

```
delay = min(cap, base * 2^attempt) + random_jitter
```

Worker lifecycle

- Set per-job and I/O timeouts.
- Enforce memory limits and recycle long-lived processes.
- On SIGTERM, stop accepting work and finish current work within a grace period.
- Acknowledge after success.
- Align visibility timeout with runtime and renew it when supported.

Monitor queue depth, oldest-message age, processing latency, retries, and DLQ rate.

Scheduler

Make scheduled work idempotent, prevent unsafe overlap with an expiring lock, and define the timezone. A scheduler can enqueue work instead of performing everything in one process.

Operational problem

▼ Why must a queued job be idempotent?

Delivery is often at least once and workers retry; duplicate delivery must not duplicate the business effect.

Run and verify

The program returns a new order and then the same order with duplicate=true. Queue retries and dead-letter behavior require a broker and worker in a test environment, beyond this program.

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Execute this checkpoint inside the lesson environment:

```
php queue-demo.php
```

Extended integration exercise target: A successful message is processed once, a transient failure retries within a bound, and a permanent failure reaches dead-letter storage with a correlation ID.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

A real Redis-backed worker

```
docker compose -f production/compose.yaml -f production/compose.full.yaml up --build -d
curl -fsS -X POST http://127.0.0.1:8080/queue
docker compose -f production/compose.yaml -f production/compose.full.yaml logs queue-worker
docker compose -f production/compose.yaml -f production/compose.full.yaml exec redis redis-cli llen lesson:queue
```

The log must contain `processed=<job_id>` and the list length must return to zero. Stop the worker, submit a job and prove it remains queued, then restart the worker. Submit the same `job_id` twice manually and inspect the duplicate record.

Connect the ideas

Visibility timeout must exceed processing time or be renewed so another worker does not see the message. Ordering is often guaranteed only within a partition. Poison messages need bounded retries, dead-letter handling, and investigation. A transactional outbox closes the database/message dual-write gap.

Practice cycle

Write your prediction before running the example and record the output. Introduce one controlled failure, collect evidence from logs or metrics, repair the cause, and rerun the check to prove the fix handles the fault instead of hiding it.

Try it yourself

Simulate a crash after commit but before ack and prove idempotency prevents duplicate effects.

Why the claim belongs in the transaction

Save the example as `queue-demo.php` in the lab folder. The first call creates order 1; the second returns the same order with `duplicate: true`. Read `src/Inventory.php`: a unique `(user_id, job_key)` claim, stock update, order, and outbox event commit together. A different payload with the same key is rejected. Failure before commit rolls all four

back; acknowledge the queue message only after commit. A separate “already processed?” read followed by an effect is unsafe because two workers can both pass the read.

An outbox is a table holding events to publish after commit. A publisher may crash after sending and before marking an event sent, so delivery can still repeat. External payments/email need a provider idempotency key or receiver deduplication; a database transaction cannot roll back a remote service. Run `php tests.php inventory` to test duplicate calls and injected failure. These local tests do not simulate a real queue broker or multiple worker processes.

PHP runtime and production

13. Deployment, CI/CD, and containers

Reproducible artifacts, pipelines, health checks, migrations, zero-downtime rollout, and rollback.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Runtime:** The period when a program is actually running.
- **Queue:** A line of background jobs waiting to be processed.
- **Worker:** A background process that takes jobs from a queue and runs them.

Reproducible artifact

Build once and promote the same artifact instead of resolving dependencies differently on every server:

```
composer install --no-dev --prefer-dist --no-interaction \
  --optimize-autoloader --classmap-authoritative
composer check-platform-reqs --lock --no-dev
```

Record commit and build IDs. Inject secrets at runtime, never into the image.

Pipeline

```
lint -> unit tests -> static analysis -> integration tests
-> dependency audit -> build artifact/image -> scan
-> deploy staging -> smoke test -> production -> verify
```

Fail closed when checks fail. Manual approval is not a substitute for repeatable verification.

A reviewable pipeline

Keep the pipeline in the repository, pin tools and images, and connect each artifact to a commit. Every stage should produce retained evidence: test reports, scan results, an image digest, and a deployment record.

```
jobs:
  verify:
    steps:
      - run: composer install --no-interaction --no-progress
      - run: composer validate --strict
      - run: composer audit --locked
      - run: composer test
      - run: docker build --pull --tag app:$GIT_SHA .
```

Do not place a secret in a build argument or image layer. Separate build from release and promote the same digest from staging to production.

Operating-system limits and load

Configure CPU, memory, process, and file-descriptor limits, then monitor throttling, OOM events, and the FPM listen queue. Container memory must cover FPM workers, OPcache, and native allocations.

A load test identifies saturation rather than producing a marketing number. Increase load gradually and record throughput, p95, errors, RSS, and database connections. Stop when the SLO fails, change one variable, and repeat.

Containers and health

Use a small pinned image, explicit main process, non-root user, runtime config, stdout/stderr logs, resource limits, and graceful shutdown. Separate web and worker lifecycles.

Liveness asks whether a process should restart, readiness whether it can receive traffic, and startup whether warm-up is complete. Do not tie liveness to every external dependency and cause a restart storm.

Safe rollout and schema changes

Use rolling or blue-green deployment. Follow Expand/Contract: add the new shape, deploy compatible code, backfill, switch reads, then remove the old shape later. Gracefully reload FPM and version queue payloads.

Rollback may not undo a destructive migration. Prepare roll-forward, backups, feature flags, and automatic success metrics, and rehearse the procedure.

Operational problem

▼ Why does a successful image not guarantee a successful deployment?

Runtime config, secrets, migrations, health checks, and rollback must also fit the real environment.

Run and verify

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

From `examples/php-labs` or the extracted package, run:

```
docker compose -f production/compose.yaml config --quiet
docker compose -f production/compose.yaml up --build -d
curl -fsS http://127.0.0.1:8080/health
php load-probe.php http://127.0.0.1:8080/health 50 10
docker compose -f production/compose.yaml down
```

Success criterion: Compose validation passes; the services start as non-root with read-only filesystems, and Nginx receives traffic only after FPM is healthy. Health returns ok JSON while static HTML bypasses PHP. The final `down` removes the lab resources.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

Connect the ideas

Build reproducible layered images with a minimal runtime and non-root user. Produce an SBOM, sign the artifact, and deploy by digest. Separate readiness from liveness, use canary or blue/green rollback, and deploy migrations with forward/backward compatibility.

Practice cycle

Write your prediction before running the example and record the output. Introduce one controlled failure, collect evidence from logs or metrics, repair the cause, and rerun the check to prove the fix handles the fault instead of hiding it.

Try it yourself

Deploy a canary failing readiness and prove it receives no traffic.

14. Dependency and supply-chain security

Composer audit, policies, platform checks, lock files, plugins, scripts, and CI verification.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **CLI:** A text-based interface controlled by typed commands.

The lock file is a build contract

Commit `composer.lock` for applications and use `composer install` in CI and production. `update` selects new versions and belongs in a reviewed, tested change—not deployment.

Audit and platform checks

```
composer validate --strict
composer audit --locked
composer check-platform-reqs --lock --no-dev
composer outdated --direct
```

Do not ignore an advisory without a documented reason, expiry, and reachability review.

Plugins and scripts

Composer plugins and scripts execute code with the Composer process permissions. Review and allow-list them:

```
{
  "config": {
    "allow-plugins": {
      "trusted/package": true,
      "*": false
    }
  }
}
```

Do not run Composer as root on untrusted packages. `--no-plugins --no-scripts` reduces execution but may also disable required build steps.

SBOM, provenance, and signing

Generate the SBOM from the final artifact, not only `composer.json`, because an image also contains operating-system packages, extensions, and files. Connect provenance to the commit, builder, workflow, and digest. A signature proves artifact origin and integrity; it does not prove absence of vulnerabilities.

The acceptance policy must identify who approves an exception, its expiry, whether affected code is reachable, and which environment runs the vulnerable digest. Test the path from CVE to lock file, image, and deployment.

CI and secrets

Reduce default workflow permissions, pin third-party actions to a reviewed commit, and keep untrusted pull requests away from secrets. Scan history and rotate any exposed credential; deleting it from the latest commit does not revoke it.

```
source commit -> locked dependencies -> SBOM -> signed image digest -> deployment record
```

Policy and incident response

Use deliberate SemVer constraints, test supported ranges for libraries, review transitive dependencies, update regularly, and remove unused packages. Protect CI tokens and logs and record which artifact was built from which commit.

For a vulnerability, assess reachability, patch or mitigate, run the suite, deploy, and monitor. If a package or script exposed a secret, rotate it; uninstalling the package does not revoke the secret.

References

- Composer CLI: audit
- Composer plugins and scripts safety

Operational problem

▼ What should follow a dependency advisory?

Confirm affected versions and reachable usage, update and test, and apply mitigation rather than ignoring or blindly upgrading.

Run and verify

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Run this checkpoint inside `examples/php-labs` or the extracted lab package:

```
composer audit --locked
composer check-platform-reqs --lock --no-dev
```

Success criterion: The current lock has no unaccepted known vulnerability; every exception has an owner, reason, and expiry date.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

Connect the ideas

Add an SBOM, provenance, and artifact signing, and inspect typosquatting plus maintainer changes rather than CVEs alone. Secret scanning keeps tokens out of source and history. Use VEX or equivalent exploitability records, with an owner, reason, and expiry for every exception.

Practice cycle

Write your prediction before running the example and record the output. Introduce one controlled failure, collect evidence from logs or metrics, repair the cause, and rerun the check to prove the fix handles the fault instead of hiding it.

Try it yourself

Trace one package from commit to artifact and deployment and prove its provenance.

PHP runtime and production

15. Profiling, SLI/SLO, and OpenTelemetry

Performance profiling, flame graphs, service objectives, and trace-context propagation across services.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Runtime:** The period when a program is actually running.
- **HTTP:** The rules used to exchange requests and responses on the web.
- **API:** A defined interface through which one program requests data or actions from another.
- **Function:** A named, reusable block of code with one defined job.

Measure before optimizing

Start with measurable evidence: a slow endpoint, high CPU, memory growth, or a rising error rate. Averages hide tails, so inspect `p50`, `p95`, and `p99` together with request volume and error count. Establish a repeatable baseline before changing code.

Sampling and instrumentation

A sampling profiler periodically captures call stacks with relatively low overhead. Instrumentation times selected operations precisely but adds code and runtime cost. Use production-like data, representative load, and an appropriate warm-up.

```
$started = hrtime(true);
try {
    $result = $service->buildReport($request);
} finally {
    $logger->info('report.completed', [
        'duration_ms' => (hrtime(true) - $started) / 1_000_000,
        'memory_peak_bytes' => memory_get_peak_usage(true),
    ]);
}
```

This times one boundary; it is not a full profiler. Avoid manually timing every function when a profiler or APM can preserve the complete call context.

Reading a flame graph

Frame width represents sample share, not the duration of one invocation. Vertical depth represents call stacks. Investigate wide repeated stacks, then determine whether the cause is CPU, serialization, regex, autoloading, repeated queries, or I/O wait that a CPU profile may not expose. Compare before and after under the same workload.

SLI, SLO, and error budget

- An **SLI** is a measurement, such as the fraction of valid requests completed successfully below 300ms.
- An **SLO** is a target, such as 99.9% over 30 days.
- The **error budget** is the permitted shortfall; exhausting it should shift attention from feature velocity to reliability.

Define valid traffic and success precisely. Exclude health checks or synthetic traffic only through a documented policy.

OpenTelemetry and propagation

OpenTelemetry correlates traces, metrics, and logs through context. An incoming request starts a span; database and outbound HTTP operations become children. Propagate W3C `traceparent` to downstream services instead of starting an unrelated trace.

```
Browser -> API span -> DB span
                    -> HTTP span -- traceparent --> payment span
```

Prefer auto-instrumentation or established libraries. Do not place secrets or personal data in baggage: it crosses service boundaries and may be exported with telemetry.

Investigating a slow request

Start with a slow trace, locate the longest span, correlate logs by trace ID, inspect a query plan or third-party latency, and use a CPU profile only when time is spent inside application computation. Verify the fix under load and watch the SLI after deployment.

A real SDK and OTLP path

The lab installs `open-telemetry/sdk` and `open-telemetry/exporter-otlp`. Environment configuration sends OTLP/HTTP to a Collector, whose debug exporter prints the span.

```
docker compose -f production/compose.yaml -f production/compose.full.yaml up --build -d
curl -fsS http://127.0.0.1:8080/telemetry
docker compose -f production/compose.yaml -f production/compose.full.yaml logs otel-collector
```

Verify that the response contains a nonzero trace ID and the logs contain `lesson.telemetry` and `service.name=php-runtime-lab`. Add low-cardinality attributes such as route, method, and status. Never use a user ID or full URL as a metric label.

A batch processor suits production, but short-lived processes must flush before exit. Define a sampling policy aware of errors and latency, and monitor dropped spans and export failures; request success does not prove telemetry export.

Check your understanding

01 **Why is average latency insufficient?**

> **Answer:** It can hide a slow tail affecting many users; inspect percentiles, traffic, and errors together.

02 **What does a wide flame-graph frame mean?**

> **Answer:** That stack appeared in many samples; it deserves investigation but does not prove every call was slow.

03 **How do SLI and SLO differ?**

> **Answer:** The SLI is the measurement; the SLO is the target for it over a defined window.

04 **How does one trace continue across two services?**

> **Answer:** Inject trace context into the outgoing request and extract it downstream, preferably through tested instrumentation.

Run and verify

The fixed sample yields p50=10ms, p95=100ms, p99=100ms, and mean=28ms. Live measurement and trace export are a separate service exercise.

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Execute this checkpoint inside the lesson environment:

```
php profiling-lab.php
```

Extended integration exercise target: Compare p50, p95, and p99 before and after the change and connect a trace to a slow operation; the average alone does not satisfy acceptance.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

Connect the ideas

Add real instrumentation for HTTP, database, and queues with an exporter and resource attributes. Histograms need suitable buckets, and percentiles must not be averaged. Connect flame-graph improvements to an SLI change rather than one faster function.

Practice cycle

Write your prediction before running the example and record the output. Introduce one controlled failure, collect evidence from logs or metrics, repair the cause, and rerun the check to prove the fix handles the fault instead of hiding it.

Try it yourself

Export a trace and metric from staging and connect one span to elevated p95.

PHP runtime and production

16. Resilience and worker management

Circuit breakers, bulkheads, backpressure, process signals, and worker supervision with systemd or Supervisor.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Queue:** A line of background jobs waiting to be processed.
- **Worker:** A background process that takes jobs from a queue and runs them.
- **CLI:** A text-based interface controlled by typed commands.
- **Function:** A named, reusable block of code with one defined job.

Timeouts come first

Every dependency needs connection and total timeouts. Without them, PHP workers can accumulate while waiting until the pool is exhausted. Retry only transient failures on safe or idempotent operations, with bounded exponential backoff and jitter.

Circuit breaker

A breaker is closed while calls flow, open while failures are rejected quickly, and half-open while a small number of probes test recovery. Do not count domain failures such as `422` as dependency outages. Classify connection errors, timeouts, and selected `5xx` results. Decide whether state is process-local or shared across workers.

Bulkheads and backpressure

A bulkhead isolates capacity through a separate pool or concurrency limit. Backpressure slows or rejects producers when consumers cannot keep up. Use bounded queues, `429` or `503` with `Retry-After`, or reduced concurrency. An unbounded queue delays collapse while consuming memory.

Monitor queue depth, oldest-message age, processing throughput, and failure rate together.

Worker lifecycle and signals

A long-running CLI worker should handle `SIGTERM`, stop reserving new work, finish or safely release the current job, close resources, and exit. `pcntl` applies to Unix-like environments; Windows service management requires a different mechanism.

```
<?php
declare(strict_types=1);

$running = true;
pcntl_async_signals(true);
pcntl_signal(SIGTERM, static function () use (&$running): void {
    $running = false;
});

while ($running) {
    $job = reserveJob(timeoutSeconds: 5);
    if ($job === null) continue;
    try {
        handle($job);
        acknowledge($job);
    } catch (Throwable $error) {
        releaseOrDeadLetter($job, $error);
    }
}
```

Recycle workers after a documented job or memory limit, and release locks and connections during shutdown.

Supervisor and systemd

A process manager starts workers at boot, restarts crashes, supplies the correct environment, and captures output. Prevent rapid restart loops with delay, attempt limits, and alerts.

```
[Service]
ExecStart=/usr/bin/php /srv/app/bin/worker.php
WorkingDirectory=/srv/app
Restart=on-failure
RestartSec=5
TimeoutStopSec=30
KillSignal=SIGTERM
```

Health means more than a live process: observe the ability to reserve work, last successful completion, oldest-message age, and error rate.

A complete systemd unit

```
[Unit]
Description=PHP queue worker
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=app
Group=app
WorkingDirectory=/srv/app/current
EnvironmentFile=/etc/app/worker.env
ExecStart=/usr/bin/php bin/worker.php
Restart=on-failure
RestartSec=5s
TimeoutStopSec=30s
KillSignal=SIGTERM
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ProtectHome=true
ReadWritePaths=/srv/app/shared
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

Run `systemd-analyze verify`, then stop the service during a long job and prove that it drains within `TimeoutStopSec`. Do not put a secret in the unit or command line; load it from a permission-restricted file or secret manager.

Check your understanding

01 **Why can retry without a timeout amplify failure?**

> **Answer:** Each attempt can wait indefinitely, consuming workers and adding pressure to the failing dependency.

02 **What is half-open state for?**

> **Answer:** It probes recovery with limited traffic before restoring the full load.

03 **How does a bulkhead differ from a circuit breaker?**

> **Answer:** A bulkhead isolates capacity; a breaker temporarily stops calls after a failure pattern.

04 **What should a worker do on SIGTERM?**

> **Answer:** Stop taking new work, finish or release the current job within the grace period, close resources, and exit clearly.

Run and verify

This file runs duplicate-effect and rollback tests. A circuit-breaker test requires closed/open/half-open states and a controllable clock; local success does not claim that coverage.

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Execute this checkpoint inside the lesson environment:

```
php resilience-lab.php
```

Extended integration exercise target: Two requests with the same idempotency key produce one business effect, and the circuit breaker opens at the threshold then probes recovery gradually.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

Connect the ideas

Retries need exponential backoff, jitter, and a retry budget to avoid storms. Shutdown stops fetching new jobs, gives current work a deadline, and releases leases. Distributed workers need ownership tokens or fencing so stale workers cannot write after lock loss, and backpressure must reach producers.

Practice cycle

Write your prediction before running the example and record the output. Introduce one controlled failure, collect evidence from logs or metrics, repair the cause, and rerun the check to prove the fix handles the fault instead of hiding it.

Try it yourself

Simulate a worker losing its lease and returning, and prove fencing blocks stale writes.

17. Disaster recovery and restore testing

RPO, RTO, backups, restore order, runbooks, and practical recovery exercises.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **DNS:** The service that translates a site name into a numeric network address.
- **Worker:** A background process that takes jobs from a queue and runs them.

A backup is not a recovery plan

A backup is an artifact. Recovery defines what is restored, in which order, who decides, and how correctness is proven. An untested restore is only an assumption. Protect databases, uploaded files, critical configuration, and encryption keys through explicit policies.

RPO and RTO

RPO is the maximum tolerable data-loss window; **RTO** is the maximum tolerable service-restoration time. They are business decisions that determine backup frequency, replication, automation, and cost. Replication is not backup because logical deletion or corruption can replicate immediately.

Backup policy

Use encrypted copies, separate permissions, documented retention, and a copy outside the primary account or region. Verify checksums, alert on failed jobs, and ensure decryption keys remain available during a disaster without storing them beside the backup.

Restore order

Declare the incident, stop or isolate damaging writes, select a recovery point, restore into a clean environment, apply only compatible migrations, verify counts and constraints, send traffic gradually, and compare actual recovery with the RPO and RTO targets.

A real restore exercise

Restore periodically into an isolated environment. Disable production email and webhooks, use non-production credentials, run smoke and consistency checks, and time every step. A useful game day exposes missing permissions, keys, documentation, and manual dependencies before a real incident.

PHP application behavior during recovery

Support a documented read-only mode when writes are unsafe. Do not run an old worker against a newer incompatible schema. Retain deployable artifacts, versioned non-secret configuration, and a compatibility map between application and database versions.

Check your understanding

01 **Why is replication not a backup?**

> **Answer:** Deletion, logical corruption, or ransomware changes may immediately propagate to replicas.

02 **How do RPO and RTO differ?**

> **Answer:** RPO limits acceptable data loss; RTO limits restoration time.

03 **How is a successful restore proven?**

> **Answer:** Through consistency checks, constraints, counts, checksums, smoke tests, and business scenarios—not merely a running database.

04 **What is a recovery game day for?**

> **Answer:** It measures the plan and exposes missing permissions, dependencies, and manual steps before a real disaster.

Run and verify

The file creates a teaching SQLite snapshot and prints snapshot=ok and verification time. That duration is not production RTO and does not measure RPO; measure data loss and complete-service recovery in the recovery exercise.

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Execute this checkpoint inside the lesson environment:

```
php restore-verify.php
```

Extended integration exercise target: A known backup restores into an isolated environment, integrity checks pass, and actual RPO and RTO are measured rather than merely stated.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

Connect the ideas

Add point-in-time recovery for transaction logs and test the recoverable boundary. Encrypted backups need a key-recovery plan stored separately. Order restoration of DNS, identity, secrets, queues, and third parties, and run a game day including communications and actual timing.

Practice cycle

Write your prediction before running the example and record the output. Introduce one controlled failure, collect evidence from logs or metrics, repair the cause, and rerun the check to prove the fix handles the fault instead of hiding it.

Try it yourself

Perform a restore without the primary production account and measure actual RPO/RTO.

PHP runtime and production

18. Sessions and shared state at scale

Session storage, locking, horizontal scaling, expiration, garbage collection, and failure behavior.

Before you start

A session connects a user's requests. File storage can appear sufficient on one host, but storage location, locking, and expiration become operational decisions as soon as the application has multiple replicas.

New terms

- **Session handler:** The component that reads and writes session data.
- **Horizontal scaling:** Running more than one application replica.
- **Sticky session:** Routing a user to the same replica using a cookie or address.
- **Lock contention:** A request waiting because another request holds the same lock.

Why file sessions fail across replicas

The default `files` handler writes to local disk. A later request routed to another replica cannot see that file unless you add shared storage or sticky routing. Shared disks add latency and a failure dependency. Sticky routing restricts load distribution and does not preserve state when a replica disappears.

A shared store such as Redis lets every replica see the same session. It also becomes critical infrastructure: an outage may block login or checkout, so use short timeouts, monitoring, and an explicit failure policy.

Locking and parallel requests

The file handler locks a user's session from `session_start()` until request shutdown or `session_write_close()`. A slow AJAX request can therefore block that user's later requests even while other FPM workers are idle.

```
session_start();
$userId = $_SESSION['user_id'] ?? null;
session_write_close();

$report = buildSlowReport($userId);
```

Close the session after its final required mutation. Check the locking semantics of the selected Redis handler; disabling locks can produce lost updates.

Safe configuration

```
session.use_strict_mode=1
session.use_only_cookies=1
session.cookie_httponly=1
session.cookie_secure=1
session.cookie_samesite=Lax
session.gc_maxlifetime=1800
```

Align store TTL with cookie lifetime and logout policy. Use native key expiry for a central store instead of probabilistic request-time garbage collection. Regenerate the ID after login or privilege change, and avoid serialized object graphs tied to one code release.

Deployment compatibility

During a rolling deployment, new code can read a session written by old code. Use a small versioned schema and accept both versions during migration. Test logout, expiry, and key rotation rather than accidentally logging out every user during deployment.

Lab

Start the complete environment:

```
docker compose -f production/compose.yaml -f production/compose.full.yaml up --build -d
curl -c cookies.txt -b cookies.txt http://127.0.0.1:8080/session
curl -c cookies.txt -b cookies.txt http://127.0.0.1:8080/session
docker compose -f production/compose.yaml -f production/compose.full.yaml exec redis redis-cli -n 1 scan 0
```

The `visits` value should rise from 1 to 2 and a session key should appear in Redis. Add a temporary delay, issue two parallel requests, then close the session early and compare wait time.

Check your understanding

▼ Why do file sessions fail with two replicas?

The file is local, so the next request may reach a replica that does not have it.

▼ When should session_write_close run?

After the final session read or write and before slow work that does not need the session.

▼ Are sticky sessions a complete shared-store replacement?

No. They reduce scheduling freedom and do not preserve state when a replica disappears.

▼ What should be monitored?

Read and write latency, errors, lock wait, key count, expirations, and memory use.

Practice cycle

Write your prediction before running the example and record the output. Introduce one controlled failure, collect evidence from logs or metrics, repair the cause, and rerun the check to prove the fix handles the fault instead of hiding it.

Try it yourself

Predict the result, run both requests, stop Redis and observe the HTTP status, then restore it and prove the application does not silently fall back to local state.

PHP runtime and production

19. Database connections and runtime operations

Connection budgets, PDO timeouts, persistent state, transactions, replicas, migrations, and failure readiness.

Before you start

Schema and query design have their own track. This lesson focuses on the lifetime of a connection from a PHP worker to the database and on behavior during latency, failure, and deployment.

New terms

- **Connection budget:** The database capacity allocated to each service.
- **Persistent connection:** A connection reused inside a PHP process.
- **Pooler:** A proxy managing a set of database connections.
- **Replica lag:** Delay between a read replica and the primary.

Budget connections before adding workers

Six replicas with 30 FPM workers and one possible connection per worker can request 180 connections before queue workers, cron, and administration. Keep `pm.max_children` and autoscaling within the database budget.

```
database budget
= web replicas × possible connections per replica
+ queue workers
+ scheduled jobs
+ administration reserve
```

Readiness can fail when the application cannot establish a required connection. Liveness should not restart every replica because of one shared database outage.

PDO and deadlines

```
$pdo = new PDO($dsn, $user, $password, [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_TIMEOUT => 2,
    PDO::ATTR_PERSISTENT => false,
]);
```

`ATTR_TIMEOUT` does not control every timeout for every driver. Configure connection, statement, and lock timeouts at the driver or server level and enforce an overall operation deadline. Never expose a DSN or exception trace to the client.

Persistent connections

A persistent connection is cached per process, not pooled across every replica. It may retain session settings, temporary tables, locks, or transactions from a previous request. Enable it only after measuring connection cost and understanding driver cleanup. A dedicated pooler is often easier to govern centrally.

Every transaction must reach commit or rollback on a guaranteed path. Retry deadlocks or serialization failures with a limit and jitter only when the operation is idempotent.

Read replicas

A read immediately following a primary write may see older data on a replica. Read-your-writes paths must use the primary or a supported consistency token. Monitor lag and stop routing critical reports to a replica beyond the accepted threshold.

Migrations and deployment

Use Expand/Contract: add compatible structure, deploy code that understands both forms, backfill in bounded batches, then enforce constraints and remove old fields later. Monitor lock duration, transaction size, and replication delay.

PostgreSQL lab

```
docker compose -f production/compose.yaml -f production/compose.full.yaml up --build -d
curl -fsS http://127.0.0.1:8080/database
docker compose -f production/compose.yaml -f production/compose.full.yaml exec postgres psql -U app -d app -c
"select * from runtime_probe"
```

The HTTP and PostgreSQL `probe_key` values must match. Stop Postgres and verify that the endpoint returns 503 without credentials, then restart it and measure recovery.

Check your understanding

▼ Why can FPM consume more connections than expected?

Every worker in every replica may hold a separate connection, in addition to workers and administration.

▼ What can persist on a persistent connection?

A transaction, lock, temporary table, or connection-level setting can reach a later request.

▼ When is a deadlock retry valid?

After rollback, with a bound and jitter, and only when the operation is safe to repeat.

▼ Why can a replica miss a recent write?

Replication can be asynchronous and the replica may lag behind the primary.

Practice cycle

Write your prediction before running the example and record the output. Introduce one controlled failure, collect evidence from logs or metrics, repair the cause, and rerun the check to prove the fix handles the fault instead of hiding it.

Try it yourself

Predict peak connection use, observe it under a small load, lower the PostgreSQL connection limit until saturation occurs, then repair the FPM and worker budget instead of adding unlimited retries.

20. Long-running PHP and worker runtimes

Safely operating FrankenPHP, RoadRunner, Swoole, and Octane with state reset, memory bounds, signals, and deployment.

Before you start

With FPM, a request begins with mostly clean PHP state and most memory disappears when the request ends. FrankenPHP Worker Mode, RoadRunner, Swoole, and Octane keep the process and application alive across requests. Bootstrap becomes cheaper, but the programming contract changes.

New terms

- **Long-running runtime:** A PHP process handling many requests.
- **State leakage:** Data from one request becoming visible to a later request.
- **Reset hook:** A step returning a service to a clean state after each request.
- **Worker recycling:** Replacing a worker after a configured bound.

What survives?

Static properties, singletons, globals, connections, and in-memory caches can survive. Do not place a request, user, or locale in a long-lived service. Keep request state in a context created and discarded for every cycle.

```
final class RequestContext
{
    public function __construct(
        public readonly string $requestId,
        public readonly ?int $userId,
    ) {}
}

function handle(ServerRequestInterface $request): ResponseInterface
{
    $context = new RequestContext(bin2hex(random_bytes(8)), null);
    return dispatch($request, $context);
}
```

Send request A with sensitive identity and request B anonymously on the same worker. B must never observe A's values. Repeat enough times to exercise worker reuse.

Memory and resources

A small leak per request becomes permanent growth. Record RSS every N requests and configure max jobs or max memory as a guard while fixing the cause. Close streams and cursors, remove temporary listeners, and detach context and tracing scopes in `finally`.

Signals and deployment

On SIGTERM, stop accepting work, finish in-flight work within the grace period, flush telemetry, and exit. The platform deadline must exceed the intended drain time. A rolling deployment also requires session schemas, queue payloads, and database changes that work with two releases simultaneously.

Choosing a model

FPM provides straightforward request isolation. A long-running runtime helps when bootstrap is expensive or the service needs concurrency or WebSockets, but requires reset support and leak tests. A hello-world benchmark is not enough evidence.

```
FPM: request -> fresh execution -> response -> cleanup
Worker runtime: boot once -> request -> reset -> request -> reset -> recycle
```

Check whether extensions and clients are safe with threads or coroutines. Blocking I/O inside an event loop can stall many requests, and a connection does not become concurrency-safe because its API looks asynchronous.

Local leakage exercise

```
php memory-lab.php
php resilience-lab.php
php long-running-state-lab.php
```

Then run the same application on a long-running runtime, send 1000 requests with changing identities, and watch RSS and static state. Acceptance requires no identity leak, stable memory after warm-up, and correct SIGTERM behavior.

Check your understanding

▼ Why is a singleton more dangerous here?

It survives requests and may retain a previous user's data or request configuration.

▼ Does recycling fix a memory leak?

It limits impact but does not remove the cause; measurement and repair remain necessary.

▼ What does a reset hook do?

It clears request-specific state before the next request uses the service.

▼ When is FPM preferable?

When isolation simplicity matters more than bootstrap cost or the application lacks a reliable reset lifecycle.

Practice cycle

Write your prediction before running the example and record the output. Introduce one controlled failure, collect evidence from logs or metrics, repair the cause, and rerun the check to prove the fix handles the fault instead of hiding it.

Try it yourself

Create a deliberate static-array leak, prove RSS growth or cross-request state, then add a reset, a regression test, and a worker recycling limit.

05

TRACK · 22 LESSONS

Security, identity, and authorization

Progressive explanations, practical examples, and corrections that take each idea into real code.

Security, identity, and authorization

Security, Authentication & Authorization

Practical PHP security covering sessions, passwords, authorization, JWT, OAuth 2.0, OpenID Connect, and SSO.

This path begins with protecting a user session, then moves through sign-in, permissions, and advanced security protocols. Each part explains the risk, the defense, and how to verify that the defense works.

1. Session security
2. Input validation
3. CSRF protection
4. XSS protection
5. Password hashing
6. Encryption and HTTP authentication
7. Authentication and authorization
8. JWT and secure validation
9. Federated identity, SSO, SAML, and OIDC
10. OAuth 2.0 concepts and tokens
11. Modern OAuth flows
12. OAuth and OIDC security
13. Account lifecycle
14. MFA, TOTP, and passkeys
15. API keys and machine identity
16. ABAC, ReBAC, and tenant isolation
17. Rate limiting and abuse resistance
18. Security logging and audit trails
19. Secrets and key management
20. Threat modeling and authorization testing — Assets, trust boundaries, roles, ownership, and tenant matrices.
21. Devices, sessions, breached passwords, and SCIM — Session revocation, password screening, and enterprise provisioning.
22. PAR, JAR, RAR, and token incidents — Advanced OAuth requests and incident runbooks for leaked credentials.

Treat browser, API, cookie, header, uploaded, and legacy database data as untrusted. Validate at the boundary and encode for the exact output context.

Standards

- JWT - RFC 7519 and JWT BCP - RFC 8725
- OAuth 2.0 - RFC 6749 and Security BCP - RFC 9700

- PKCE - RFC 7636 and Device Authorization - RFC 8628
- OpenID Connect Core
- SAML 2.0 Technical Overview

Follow one account through the boundaries

Run `php auth-journey.php` from the local lab. Read it after sessions, password hashing, authorization, and account recovery. It uses SQLite 3.35+ (`DELETE … RETURNING`) and the shared inventory schema. The program creates a temporary account, rejects a wrong password, issues a random opaque session, checks expiry, buys an order with the authenticated user ID, denies another user ownership, retries the same order, resets the password once, and revokes earlier sessions. The outbox still contains one event. A structured audit event goes to stderr without any password, session, or reset token.

The raw session/reset token is a 32-byte random secret encoded as hex; only its SHA-256 digest is stored. Unlike a human password, such a high-entropy token does not need a deliberately slow password hash. Reset consumption, password change, and session revocation share a transaction; if any write fails, all are rolled back. An atomic DELETE returning the user ID prevents two consumers from both taking the same token.

Trace the boundary: login establishes identity; the query `WHERE id=? AND user_id=?` enforces ownership; an idempotency key prevents duplicate business effects; the audit event records the outcome without credentials. A random session is freshly issued after password verification instead of accepting a caller-selected authenticated session ID.

This is a complete local state-flow example, not a deployable authentication server. Token delivery is simulated in memory; no email is sent. HTTP cookies, CSRF, rate limiting, account-enumeration resistance, password policy, and provider integration belong to the corresponding lessons and must be implemented before exposing endpoints. The length check here is a small explicit demo policy, not a full password policy. Exercise: move `$now` beyond reset expiry and verify reset fails without changing the old password or sessions. Then inject an exception before commit and verify rollback preserves every prior state.

1. Session security

Secure lifecycle, fixation and hijacking defenses, ID rotation, expiry, and cookies.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **HTTP:** The rules used to exchange requests and responses on the web.
- **IP:** A numeric address that identifies a device or network interface.
- **Session:** Temporary server-side state used to recognize a user across requests.
- **Cookie:** A small value stored by the browser and sent with matching requests.
- **Worker:** A background process that takes jobs from a queue and runs them.

Beginner bridge

A session is the server's way to connect several independent HTTP requests to one signed-in browser. The browser normally carries only an opaque session ID; the valuable state stays on the server. Security therefore depends on protecting that identifier as if it were a temporary password.

Keep three threats separate: fixation chooses an ID before login, hijacking steals a valid ID after login, and weak expiry leaves an old ID useful for too long. Cookie flags reduce exposure, but rotation, server-side invalidation, time limits, and reauthentication are separate controls.

The server stores session state while the browser normally holds a random identifier cookie. That ID is a credential: stealing it can impersonate the user.

Session fixation makes a victim use an attacker-known ID; hijacking steals a valid ID through XSS, insecure transport, logs, or a compromised device.

```
session_start([
    'use_strict_mode' => true,
    'use_only_cookies' => true,
    'cookie_secure' => true,
    'cookie_httponly' => true,
    'cookie_samesite' => 'Lax',
]);
```

Use HTTPS, never put IDs in URLs, and regenerate after authentication or privilege changes before writing elevated state:

```
session_regenerate_id();
$_SESSION['user_id'] = $user['id'];
$_SESSION['auth_time'] = time();
```

Immediate deletion through `session_regenerate_id(true)` can race concurrent requests or unstable networks. Sensitive systems use a short timestamp-based transition and retire obsolete IDs deliberately.

Implement both idle and absolute expiry; do not confuse garbage collection with access policy. Treat IP/User-Agent changes as risk signals rather than strict identity because mobile networks, NAT, and VPNs change. Close session locks early with `session_write_close()`.

Logout must clear `$_SESSION`, expire the cookie with matching options, destroy storage, and revoke server-side refresh tokens or recorded sessions where applicable.

Security scenario

▼ Why rotate the session ID after login?

To prevent session fixation; create a new session identity instead of keeping an attacker-known ID.

Threat drill

Scenario: An attacker captured a pre-login session identifier and tries to reuse it after the victim signs in.

Negative test: Sign in, record the new session identifier, then send a sensitive request from a separate client with the old identifier.

Expected result: The old identifier is rejected, account data is unchanged, and authentication produced a fresh identifier.

Verification source

- PHP session security

Connect the ideas

In distributed deployments, use a shared session store or revocation reaching every node rather than worker memory. Apply suitable cookie prefixes such as `__Host-` where the contract fits, with Secure, HttpOnly, SameSite, and Path. Cookie-authenticated state changes still need CSRF protection.

Try it yourself

Revoke a session on one node, try it through another, then run a CSRF test.

2. Input validation

Validation vs sanitization, PHP filters, domain rules, and output encoding boundaries.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **URL:** The complete address of a resource such as a page or API endpoint.

Beginner bridge

Validation answers whether input belongs to the business domain, not whether it looks harmless. Parse into the intended type, reject impossible ranges and unsupported states, and keep the original untrusted value away from decisions until that process succeeds.

Validation does not replace output encoding, prepared statements, or authorization. Each control belongs at a different boundary: validation defines acceptable data, encoding prevents a rendering context from treating data as code, and authorization decides whether the caller may perform the action.

Validation checks whether data satisfies rules without changing it. Normalization standardizes legitimate representation. Sanitization removes/changes characters and may silently corrupt meaning. Output encoding belongs at the exact HTML/URL/JS sink.

```
$email = filter_var($rawEmail, FILTER_VALIDATE_EMAIL);
if ($email === false) {
    throw new InvalidArgumentException('Invalid email');
}

$age = filter_var($rawAge, FILTER_VALIDATE_INT, [
    'options' => ['min_range' => 18, 'max_range' => 120],
]);
if ($age === false) {
    throw new InvalidArgumentException('Invalid age');
}
```

Use `=== false` because zero can be valid. `FILTER_DEFAULT` is `FILTER_UNSAFE_RAW`; it is not automatic safety.

```
$page = filter_input(INPUT_GET, 'page', FILTER_VALIDATE_INT, [
    'options' => ['default' => 1, 'min_range' => 1],
]);
```

Shape validation is insufficient: a valid order ID does not prove ownership. Validate type/range, then authorize the current user against the loaded resource. Use allow-lists for enums, sort fields, and identifiers; prepared SQL and context-aware output encoding remain separate controls.

Security scenario

▼ Does input validation replace output encoding?

No. Validation enforces an input contract; context-specific encoding prevents data from becoming code.

Threat drill

Scenario: A client submits a value that looks numeric but is out of range or carries extra characters to bypass a shallow check.

Negative test: Try a negative value, an over-limit value, and `12x` in the same field.

Expected result: All three inputs are rejected consistently, and none reaches business logic or the database.

Verification source

- OWASP Input Validation Cheat Sheet

Connect the ideas

Canonicalize before validation when multiple representations exist, but never decode indefinitely. Validate nested arrays for depth, count, and size, not only scalar fields. For files, inspect upload status, size, signature/MIME, and content, using allowlists for known values.

Try it yourself

Test double encoding, deeply nested input, and a file whose name conflicts with its content.

Security, identity, and authorization

3. CSRF protection

How authenticated browsers are tricked and how tokens, SameSite, and origin checks help.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Session:** Temporary server-side state used to recognize a user across requests.
- **Cookie:** A small value stored by the browser and sent with matching requests.
- **Token:** A value representing identity or permission without resending a password.

Beginner bridge

CSRF works because a browser automatically attaches ambient credentials, such as cookies, to a request chosen by another site. The attacker does not need to read the response; causing a state-changing action can be enough.

The mental model is request intent. A valid session proves which browser account sent the cookie, while a CSRF token or strict origin check helps prove that the action came through a page and flow the application created. SameSite is useful defense in depth, not a universal replacement for request-specific protection.

CSRF tricks an authenticated browser into sending an unwanted state-changing request. Cookies may be attached automatically, so the server sees a valid session without proof of user intent.

```
session_start();
$_SESSION['csrf_token'] ??= bin2hex(random_bytes(32));
```

Place the value in a hidden form field and verify before changing state:

```
$sent = $_POST['csrf_token'] ?? '';
$origin = $_SERVER['HTTP_ORIGIN'] ?? null;
$allowedOrigins = ['https://app.example.com'];

if (!is_string($sent)
    || $sent === ''
    || !is_string($origin)
    || !in_array($origin, $allowedOrigins, true)
    || !isset($_SESSION['csrf_token'])
    || !hash_equals($_SESSION['csrf_token'], $sent)) {
    http_response_code(403);
    exit('Invalid CSRF token');
}
```

Keep tokens out of URLs/logs. Do not mutate state through GET. Add suitable SameSite cookies, Origin checks for sensitive requests, strict accepted content types, and re-authentication/MFA for high-risk actions.

CORS is not CSRF protection, and simple cross-site requests may avoid preflight. XSS can often read tokens or issue same-origin requests, so XSS prevention is essential. Signed double-submit cookies are possible, but prefer framework-reviewed protection. Per-session and per-form tokens are both valid designs; per-request rotation can break concurrent tabs.

Security scenario

▼ Why is SameSite alone not always enough?

It has limits and bypass scenarios; combine suitable cookies with tokens and origin checks for the threat.

Threat drill

Scenario: A hostile page attempts to change an email address while the browser carries the victim's cookies.

Negative test: Send the request without a CSRF token, with another user's token, and with a valid token from a disallowed Origin.

Expected result: All three attempts are rejected and the email stays unchanged; a valid cookie alone is insufficient.

Verification source

- OWASP CSRF Prevention Cheat Sheet

Connect the ideas

SameSite reduces some paths but does not replace tokens and Origin checks; Lax, Strict, and None differ. Login CSRF can bind a victim to an attacker account before a trusted session exists. APIs without ambient cookies face a different CSRF model but still need token-leak prevention and correct CORS.

Try it yourself

Test login CSRF and cross-site requests under different SameSite modes.

Origin policy

Configure the exact trusted origin, including scheme and non-default port. Never construct this allowlist from the incoming Host header. The shown policy rejects a missing Origin as well as a foreign one; some clients omit it, so test legitimate clients before choosing a documented Referer fallback. Origin is another layer, not authentication: a non-browser client can set this header. Keep the session token check and use POST for changes.

Security, identity, and authorization

4. XSS protection

Reflected, stored, and DOM XSS with context-aware output encoding.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Sanitizer:** An HTML sanitizer parses untrusted markup and removes disallowed elements and attributes while preserving permitted formatting.
- **URL:** The complete address of a resource such as a page or API endpoint.
- **UTF-8:** A common encoding that stores Unicode numbers as bytes.
- **Function:** A named, reusable block of code with one defined job.

Beginner bridge

XSS is a context confusion: text supplied as data reaches a browser position where it is interpreted as HTML, an attribute, a URL, CSS, or JavaScript. The correct defense depends on that final context, so one universal escaping function cannot be safe everywhere.

Prefer APIs that create text nodes or set safe properties, apply context-specific output encoding at the last responsible moment, and treat HTML sanitization as a specialized operation. Content Security Policy limits impact but does not repair unsafe rendering.

XSS occurs when untrusted data is interpreted as browser code. It may be reflected in one response, stored for later victims, or introduced by DOM code using a dangerous sink.

```
function e(string $value): string
{
    return htmlspecialchars($value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}
```

Use this for HTML text and quoted safe attributes. For a URL query value, apply `rawurlencode` first and HTML-encode the final attribute. Encoding does not make a `javascript:` scheme safe; validate protocols and hosts.

Prefer JSON rather than string concatenation for JavaScript data, with `JSON_HEX_TAG | JSON_HEX_AMP | JSON_HEX_APOS | JSON_HEX_QUOT`. In DOM code, prefer `textContent` over `innerHTML`.

If users must author HTML, use a maintained allow-list sanitizer; regex is not an HTML parser. Avoid untrusted data in scripts, styles, comments, tag/attribute names, and event handlers.

Template auto-escaping, CSP, Trusted Types, secure cookies, and correct MIME types provide defense in depth. CSP is not a substitute for encoding and sanitization.

Security scenario

▼ Where should encoding happen?

At output, for the exact HTML, attribute, URL, or JavaScript context; one encoder does not fit all.

Threat drill

Scenario: An attacker stores a comment containing markup or an event handler so JavaScript runs when the page is opened later.

Negative test: Submit text containing `<script>` and an attribute-context attempt, then render it in every context the page uses.

Expected result: It is displayed only as text, no handler runs, and CSP remains a defense-in-depth layer rather than a substitute for contextual encoding.

Verification source

- OWASP XSS Prevention Cheat Sheet

Connect the ideas

DOM XSS flows from an untrusted source into sinks such as innerHTML or eval; server encoding alone is insufficient. Trusted Types can restrict dangerous sinks, and strong CSP uses nonces or hashes rather than broad allowlists. Sanitizing allowed HTML differs from encoding plain text.

Try it yourself

Trace a value from location to a DOM sink, then replace the sink or enforce a policy.

5. Password hashing

Safe storage with password_hash, password_verify, Argon2id, bcrypt, and rehashing.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **API:** A defined interface through which one program requests data or actions from another.
- **Session:** Temporary server-side state used to recognize a user across requests.
- **Token:** A value representing identity or permission without resending a password.

Beginner bridge

Password storage is designed for an eventual database leak. The system must verify a password without keeping a reversible copy, and each guess should be deliberately expensive for an attacker. General-purpose fast hashes solve a different problem and are therefore unsuitable.

Use the platform password API, let it manage salts, tune cost for the deployment, and rehash after successful login when policy changes. Rate limiting protects the online login endpoint; a slow password hash protects against offline guessing after stolen hashes.

Passwords should be verified, not recovered. Store a slow salted password hash, not reversible encryption or a fast general-purpose hash.

```
<?php
declare(strict_types=1);

$password = 'a long example passphrase';
// One policy for registration AND rehashing after successful login.
$algorithm = defined('PASSWORD_ARGON2ID') ? PASSWORD_ARGON2ID : PASSWORD_DEFAULT;
$options = $algorithm === PASSWORD_DEFAULT ? [] : [
    'memory_cost' => 64 * 1024, 'time_cost' => 3, 'threads' => 2,
];
$storedHash = password_hash($password, $algorithm, $options);
echo password_verify($password, $storedHash) ? "valid\n" : "invalid\n";
echo password_verify('wrong password', $storedHash) ? "valid\n" : "invalid\n";
if (password_verify($password, $storedHash)
    && password_needs_rehash($storedHash, $algorithm, $options)) {
    $storedHash = password_hash($password, $algorithm, $options);
    // Persist using a conditional update against the previous hash.
}
```

Use a `VARCHAR(255)` column because `PASSWORD_DEFAULT` may evolve. Argon2id is available in compatible builds; benchmark memory/time costs on production-like hardware. PHP embeds salt and options in the result; do not create salts or compare hashes manually.

Allow long passphrases, avoid silent trimming/truncation, rate-limit attempts, add MFA for sensitive accounts, and never log passwords. bcrypt historically only uses the first 72 bytes.

A server-side pepper is optional and adds key-management complexity. Keep it in a secret manager. Password-reset tokens must be random, short-lived, one-time, and stored hashed; revoke relevant sessions after a successful reset.

Security scenario

▼ Why use Argon2id or bcrypt instead of SHA-256?

Password hashes are deliberately slow and salted; a fast hash lets attackers test more guesses.

Threat drill

Scenario: The user database leaks; passwords should remain expensive to crack and no response or log should expose sensitive values.

Negative test: Verify that the same password produces different hashes twice, a wrong password fails, and `password_needs_rehash` detects an old policy.

Expected result: Only the correct password verifies, plaintext is never stored, and a successful login upgrades an obsolete hash when needed.

Verification source

- PHP password hashing functions

Connect the ideas

Choose Argon2 or bcrypt parameters by measuring latency and memory on production-class hardware, then version policy through needs_rehash. Check breached passwords with privacy and rate controls, and bind reset tokens to expiry, single use, hashed storage, and session revocation policy.

Try it yourself

Benchmark two hash settings, choose a budget, then test a reset token twice.

Read the result and failure behavior

The complete program prints `valid` then `invalid`. `PASSWORD_DEFAULT` is a policy that PHP may change; storing its result does not pin your application to Argon2id. Choose and benchmark a policy explicitly. PHP 8 throws on hashing failures (`ValueError` for invalid parameters or `Error` for certain failures); a `=== false` branch is obsolete for this baseline. Let a central error handler return a generic failure and avoid logging passwords. The database update is application-specific: use a conditional update to avoid overwriting a concurrent password reset.

Exercise: change only `time_cost` after creating the hash. `password_verify` still succeeds because the stored hash contains the old parameters, while `password_needs_rehash` becomes true for the new policy.

6. Encryption and HTTP authentication

Authenticated encryption, key management, Basic auth, Bearer tokens, and hashing differences.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **HTTP:** The rules used to exchange requests and responses on the web.
- **TLS:** An encryption layer that protects data while it moves between two parties.
- **Token:** A value representing identity or permission without resending a password.
- **Scope:** A named permission requested or granted to a client, such as orders:read; it does not by itself prove ownership of an order.

Beginner bridge

Encryption, hashing, MACs, and digital signatures answer different questions. Encryption hides content, a hash fingerprints it, a MAC proves integrity to parties sharing a secret, and a signature allows verification with a public key. Choosing a primitive before naming the threat usually creates a fragile design.

HTTP authentication schemes also differ in what travels on every request. Basic authentication merely encodes credentials and therefore requires TLS. Bearer credentials grant access to whoever possesses them, so transport protection, narrow scope, short lifetime, and careful logging are essential.

Use encryption when plaintext must be recovered; passwords use hashing. Prefer authenticated encryption APIs that provide confidentiality and tamper detection.

```
$nonce = random_bytes(SODIUM_CRYPTO_SECRETBOX_NONCEBYTES);
$cipher = sodium_crypto_secretbox($plainText, $nonce, $key);
$stored = base64_encode($nonce . $cipher);
```

The key must be securely random with the required length and stored in a secret manager. Base64 is encoding, not encryption. Keep key versions with ciphertext, restrict decrypt permission, audit use, and plan rotation/revocation.

HTTP Basic sends Base64-encoded `username:password` on each request:

```
Authorization: Basic dXNlcjpwYXNz
```

It requires HTTPS and rate limiting. PHP may expose credentials through `PHP_AUTH_USER` and `PHP_AUTH_PW`, depending on server configuration. Basic can suit constrained internal tools but has limited lifecycle/logout controls.

A Bearer token grants access to whoever holds it. Treat it as a secret and design expiry, scope, rotation, and revocation. Not every token is JWT, and JWT is neither encrypted nor automatically revocable.

Security scenario

▼ Does encryption replace signing?

No. Encryption hides content; signing proves integrity and origin. A system may need both.

Threat drill

Scenario: A network attacker attempts to read Basic Authentication credentials or force the client onto an unencrypted connection.

Negative test: Send HTTP without credentials, then incorrect test credentials over HTTPS; verify TLS is established before attaching secrets.

Expected result: HTTP is redirected or rejected, bad credentials fail, and secrets appear in neither URLs nor logs.

Verification source

- OWASP Transport Layer Security Cheat Sheet

Connect the ideas

Use AEAD with a unique nonce per key and associated data for context; do not invent key derivation or formats. Separate data from encryption keys and backups. Basic sends credentials on every request and always needs TLS, while a Bearer token grants its holder access; scope, audience, lifetime, and storage are part of the contract.

Try it yourself

Test wrong associated data with the complete AEAD example below. Nonce reuse is unsafe but does not automatically produce a library exception.

Authenticate the context with AEAD

`secretbox` authenticates the ciphertext but has no associated-data argument. AEAD (authenticated encryption with associated data) also binds unencrypted context to it: a note for user 42 must not decrypt as a note for user 99. The context is not secret. Build it from trusted account/record identifiers and a stable format. This standalone Sodium

example prints `private note`, then `wrong context rejected`.

```
<?php
declare(strict_types=1);



function seal(string $plain, string $context, string $key): string
{
    $nonce = random_bytes(SODIUM_CRYPTO_AEAD_XCHACHA20POLY1305_IETF_NPUBBYTES);
    $cipher = sodium_crypto_aead_xchacha20poly1305_ietf_encrypt($plain, $context, $nonce, $key);
    return base64_encode($nonce . $cipher);
}

function openSealed(string $encoded, string $context, string $key): string
{
    $payload = base64_decode($encoded, true);
    $nonceBytes = SODIUM_CRYPTO_AEAD_XCHACHA20POLY1305_IETF_NPUBBYTES;
    if ($payload === false || strlen($payload) < $nonceBytes + SODIUM_CRYPTO_AEAD_XCHACHA20POLY1305_IETF_ABYTES) {
        throw new RuntimeException('Invalid encrypted payload');
    }
    $plain = sodium_crypto_aead_xchacha20poly1305_ietf_decrypt(
        substr($payload, $nonceBytes), $context, substr($payload, 0, $nonceBytes), $key,
    );
    if ($plain === false) {
        throw new RuntimeException('Authentication failed');
    }
    return $plain;
}

$key = sodium_crypto_aead_xchacha20poly1305_ietf_keygen();
$encoded = seal("private note", "user:42:note:v1", $key);
echo openSealed($encoded, "user:42:note:v1", $key), PHP_EOL;
try {
    openSealed($encoded, "user:99:note:v1", $key);
} catch (RuntimeException) {
    echo "wrong context rejected", PHP_EOL;
}
```

Generate a fresh random 24-byte nonce for each encryption with this algorithm. Uniqueness is a caller obligation; a repeated nonce need not raise an exception. This teaching envelope has no key-ID/rotation protocol; add a versioned, validated envelope before persistent production storage.

An HTTP-to-HTTPS redirect cannot protect Basic credentials already sent in the first HTTP request. Configure HTTPS at the client before attaching credentials, enforce TLS at the edge, and avoid forwarding Authorization across redirects to other origins. HSTS helps supporting browsers after policy is learned (or preloaded); it does not repair past disclosure.

PHP XChaCha20-Poly1305

7. Authentication and authorization

Secure login, RBAC, ownership policies, IDOR prevention, and least privilege.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **HTTP:** The rules used to exchange requests and responses on the web.
- **API:** A defined interface through which one program requests data or actions from another.
- **Session:** Temporary server-side state used to recognize a user across requests.
- **CLI:** A text-based interface controlled by typed commands.
- **Scope:** A named permission requested or granted to a client, such as orders:read; it does not by itself prove ownership of an order.
- **Function:** A named, reusable block of code with one defined job.

Beginner bridge

Authentication establishes an identity claim; authorization evaluates whether that identity may perform one action on one resource in the current context. A successful login never implies universal access.

Place authorization at the resource boundary and evaluate ownership, tenant, role or policy, and action together. Hiding a button is user experience, not enforcement. Server-side checks must run for every route, API operation, background job, and indirect object reference.

Authentication asks who the user is; authorization asks whether that identity may perform this action on this resource; auditing records what happened. Login success never grants universal access, and hiding a UI button is not authorization.

A login flow validates input, loads the account, verifies the password, applies rate/risk controls, regenerates the session ID, stores minimal identity state, and redirects. Return generic credential errors and never log passwords or tokens.

RBAC handles broad roles. Ownership and policy checks remain necessary:

```
function canUpdatePost(array $user, array $post): bool
{
    return $user['role'] === 'admin'
        || ($user['role'] === 'editor'
            && $post['author_id'] === $user['id']);
}
```

Where suitable, scope the query itself:

```
SELECT * FROM posts
WHERE id = :post_id AND author_id = :user_id
```

This helps prevent IDOR/BOLA. Middleware may enforce authentication and broad permission, while domain rules must still apply to HTTP, CLI, and queued jobs.

Use 401 for missing/invalid authentication, 403 for authenticated but forbidden, and sometimes 404 to conceal a resource's existence. Deny by default, apply least privilege, use MFA/step-up auth for sensitive actions, revoke sessions after security changes, protect audit logs, and test cross-user resource access.

Security scenario

▼ How do authentication and authorization differ?

Authentication establishes identity; authorization decides allowed actions on a specific resource.

Threat drill

Scenario: An authenticated user changes a resource identifier to read another user's invoice.

Negative test: Send the request as the owner, then as another authenticated account that does not own the resource.

Expected result: The owner succeeds; the other request returns 403 or policy-defined 404, independent of whether the UI hides the button.

Verification source

- OWASP Authorization Cheat Sheet

Connect the ideas

Build an authorization matrix with actor, resource, action, context, and expected decision, then convert it into allow and deny tests. Generic middleware is insufficient when ownership changes inside a domain operation. Audit the decision and policy version without leaking resource data, and test direct object references.

Try it yourself

Run the matrix through HTTP and a background job and prove identical decisions.

8. JWT and secure validation

JWT structure, signatures, claims, issuer/audience/time validation, and key management.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **URL:** The complete address of a resource such as a page or API endpoint.
- **TLS:** An encryption layer that protects data while it moves between two parties.
- **API:** A defined interface through which one program requests data or actions from another.
- **Cache:** A temporary copy that reduces waiting and repeated work.
- **Session:** Temporary server-side state used to recognize a user across requests.
- **Token:** A value representing identity or permission without resending a password.
- **Scope:** A named permission requested or granted to a client, such as orders:read; it does not by itself prove ownership of an order.
- **Authorization:** Deciding which actions an identity may perform.
- **Encryption:** A reversible transformation requiring the correct key.
- **XSS:** An attack that runs untrusted JavaScript in a user page.
- **CSRF:** An attack that makes a signed-in browser send an unintended request.
- **JWT:** A signed token format; signing does not encrypt its contents.
- **Secret:** A sensitive value such as an API key or service password.

JWT is not a magic session

A common signed JWT is a JWS with three Base64url segments:

```
BASE64URL(header).BASE64URL(payload).BASE64URL(signature)
```

The signature protects integrity and authenticity when verified with the correct key. Base64url provides no confidentiality: anyone holding the token can read its payload.

Claims to validate

Claim	Meaning
`iss`	exact trusted issuer
`sub`	token subject, not necessarily an email
`aud`	intended recipient API/client
`exp`	expiration
`nbf`	not valid before
`iat`	issued at
`jti`	unique token identifier

Short-lived tokens limit stale authorization. Roles embedded in a token do not automatically update when the account changes.

Algorithms

`HS256` uses a shared secret, so every verifier with that key can also issue tokens. Asymmetric signatures separate a private signing key from public verification keys. Configure an explicit allowed algorithm; never trust the token's `alg` choice by itself or accept `none` accidentally.

```
composer require firebase/php-jwt
```

```php
use Firebase\JWT\JWT;
use Firebase\JWT\Key;

$payload = [
    'iss' => 'https://id.example.com',
    'sub' => 'user_123',
    'aud' => 'https://api.example.com',
    'iat' => time(),
    'nbf' => time(),
    'exp' => time() + 300,
    'scope' => 'orders:read',
];

$token = JWT::encode($payload, $privateKey, 'RS256', 'key-2026-09');
$claims = (array) JWT::decode($token, new Key($publicKey, 'RS256'));

validateAccessClaims($claims, 'https://id.example.com', 'https://api.example.com', time());

if (($claims['iss'] ?? null) !== 'https://id.example.com') {
    throw new RuntimeException('Invalid issuer');
}

if (!in_array('https://api.example.com', (array) ($claims['aud'] ?? []), true)) {
    throw new RuntimeException('Invalid audience');
}
```

The library verifies cryptography and time claims, while the application still owns issuer, audience, token type, and authorization policy checks.

Rotation and JWKS

`kid` is untrusted input. Resolve it only inside a key set already bound to a trusted issuer. Never turn it into an arbitrary filename or follow a token-supplied `jku` URL. Cache JWKS with bounded refresh and retain retiring verification keys until outstanding tokens expire.

Checklist

- Require TLS.
- Allowlist algorithms and expected token type.
- Validate signature, issuer, audience, expiration, and not-before.
- Use a small, explicit clock skew.
- Separate ID-token and access-token validation rules and keys.
- Never log complete tokens.
- Choose short lifetimes, a targeted denylist, or opaque tokens where immediate revocation is required.

Security scenario

▼ Why not put secrets in a JWT?

The payload is usually encoded, not encrypted, and token holders can read it; minimize claims and scope.

Threat drill

Scenario: An attacker changes a JWT header or claim, or replays a token issued for another issuer or audience.

Negative test: Test a modified signature, an unexpected `alg`, a wrong `aud`, and an expired token.

Expected result: Every case is rejected before business logic; algorithm, issuer, audience, and lifetime are server-side policy, not token-controlled choices.

Verification source

- OWASP JSON Web Token Cheat Sheet

Connect the ideas

A JWT access token has no inherent instant revocation; use short lifetimes plus rotation, introspection, or a denylist according to risk. Define browser storage against the XSS/CSRF model. Pin algorithm, issuer, and audience in configuration, manage JWKS caching/rotation, and avoid nested or encrypted tokens without a supported need.

Try it yourself

Rotate a signing key and prove the old key works only during the overlap window.

Required claims are an application contract

The example above is an integration fragment: load Composer autoloading and supply managed RSA keys. Add this function in the same file. Call it only after the library verifies the signature with a trusted key and fixed algorithm. This access-token profile requires `iss`, `sub`, `aud`, and integer `exp`; JWT itself does not require every registered claim. A library may validate expiration only when the field exists. Optional `nbf` must also be an integer here. This strict teaching profile uses zero clock tolerance; production tolerance must be explicit and bounded.

```
function validateAccessClaims(array $claims, string $issuer, string $audience, int $now): void
{
    // Only call AFTER a JOSE library has verified the signature and fixed algorithm.
    if (($claims['iss'] ?? null) !== $issuer
        || !is_string($claims['sub'] ?? null) || $claims['sub'] === ''
        || !is_int($claims['exp'] ?? null) || $claims['exp'] <= $now) {
        throw new InvalidArgumentException('Missing or invalid required claim');
    }
    $aud = $claims['aud'] ?? null;
    if (is_string($aud)) {
        $aud = [$aud];
    }
    if (!is_array($aud) || !array_is_list($aud) || $aud === []
        || count(array_filter($aud, 'is_string')) !== count($aud)
        || !in_array($audience, $aud, true)) {
        throw new InvalidArgumentException('Invalid audience');
    }
    if (array_key_exists('nbf', $claims)
        && (!is_int($claims['nbf']) || $claims['nbf'] > $now)) {
        throw new InvalidArgumentException('Invalid not-before claim');
    }
}
```

Test a correctly signed token issued without `exp`: it must fail policy validation. Removing a field from an existing signed token instead tests signature tampering. The local `security` tests validate this policy, not the third-party signature implementation.

9. Federated identity, SSO, SAML, and OIDC

Federation and single sign-on, IdP/SP roles, SAML assertions, and OpenID Connect identity.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **API:** A defined interface through which one program requests data or actions from another.
- **Session:** Temporary server-side state used to recognize a user across requests.
- **Token:** A value representing identity or permission without resending a password.
- **Scope:** A named permission requested or granted to a client, such as orders:read; it does not by itself prove ownership of an order.
- **Authentication:** Confirming that a user owns an identity.
- **Authorization:** Deciding which actions an identity may perform.
- **OAuth:** A delegation protocol granting limited access without sharing a password.
- **OIDC:** An identity layer over OAuth that identifies who signed in.
- **SAML:** A standard for exchanging sign-in information between services.

Federation is not one protocol

Federated identity lets an application rely on an external identity provider. Single Sign-On is the experience of reaching multiple applications after a central sign-in.

```
User -> Application / Service Provider
        -> Identity Provider
        <- signed identity result
     <- local session
```

Separate the protocols

Technology	Primary purpose	Typical result
OAuth 2.0	delegated API authorization	access token
OpenID Connect	authentication layered on OAuth	ID token and identity claims
SAML 2.0	XML identity/assertion exchange, common in enterprise SSO	SAML response/assertion

OAuth alone is not a login protocol. Inferring identity from an arbitrary access token endpoint creates unsafe pseudo-authentication; use OIDC.

SAML browser SSO

The Identity Provider authenticates the user and issues an assertion. The Service Provider validates it and creates its own session.

```
Browser -> SP: protected page
SP -> Browser -> IdP: AuthnRequest
IdP authenticates user
IdP -> Browser -> SP ACS: SAMLResponse
SP validates response/assertion and creates a session
```

Validate the XML signature with trusted IdP metadata, issuer, audience restriction, recipient/destination, time conditions, and `InResponseTo` where applicable. Detect replay by assertion ID. Use a mature SAML library and safe XML parsing; do not implement XML signatures yourself.

OpenID Connect

The `openid` scope makes an OAuth authorization request an OIDC request:

```
scope=openid profile email
```

- ID token: authentication result for the client.
- Access token: authorization credential for an API.
- UserInfo: optional additional claims obtained with an access token.

Never substitute an ID token as the API's access token.

Validate an ID token

1. Use metadata rooted in a preconfigured trusted issuer.
2. Verify the signature with that issuer's JWKS.
3. Exactly match `iss` and require the client ID in `aud`.
4. Check expiration and relevant time claims.
5. Match the transaction-specific `nonce`.
6. Apply `azp` rules for multiple audiences.
7. Require UserInfo `sub` to match the ID-token subject.

An email can change and may be unverified. Use the pair `iss + sub` as the stable external identity key.

Choose OIDC for modern web/mobile login, SAML where enterprise federation requires it, and OAuth for delegated API access. Security depends on exact validation and trust configuration, not the protocol name alone.

Security scenario

▼ Who authenticates the user in federation?

The identity provider authenticates; the relying party validates the assertion or ID token and creates its session.

Threat drill

Scenario: A correctly signed identity response arrives for a different client or login transaction.

Negative test: Test a different `audience`, a stale `nonce`, and an unregistered redirect URI.

Expected result: All cases fail, and the response is bound to the local request before a session is created.

Verification source

- OpenID Connect Core 1.0

Connect the ideas

For SAML, validate metadata, entity ID, Destination, Audience, InResponseTo, time, and the signature at the expected element to resist XML signature wrapping; do not hand-roll XML security. For OIDC, use trusted discovery/JWKS and pin issuer, client ID, redirect URI, and a small clock-skew allowance.

Try it yourself

Test a signed assertion wrapping the wrong element and an ID token from another issuer.

10. OAuth 2.0 concepts and tokens

Delegated authorization, roles, scopes, consent, client types, access tokens, and refresh tokens.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **TLS:** An encryption layer that protects data while it moves between two parties.
- **API:** A defined interface through which one program requests data or actions from another.
- **Token:** A value representing identity or permission without resending a password.
- **Scope:** A named permission requested or granted to a client, such as orders:read; it does not by itself prove ownership of an order.
- **Authentication:** Confirming that a user owns an identity.
- **Authorization:** Deciding which actions an identity may perform.
- **Encryption:** A reversible transformation requiring the correct key.
- **JWT:** A signed token format; signing does not encrypt its contents.
- **OAuth:** A delegation protocol granting limited access without sharing a password.
- **OIDC:** An identity layer over OAuth that identifies who signed in.
- **Secret:** A sensitive value such as an API key or service password.

The problem OAuth solves

OAuth 2.0 is a delegated authorization framework. It lets a client obtain limited access to a resource server without receiving the resource owner's password.

Four roles

1. Resource Owner: can grant access, often a user.
2. Client: application requesting access.
3. Authorization Server: issues tokens after authorization.

4. Resource Server: protected API accepting access tokens.

```
Resource Owner -> Authorization Server: approve
Client -> Authorization Server: grant/code
Authorization Server -> Client: access token
Client -> Resource Server: token + API request
```

Client registration and types

Registration commonly defines a public `client_id`, exact redirect URIs, allowed grants, and authentication methods.

- Public clients cannot keep a secret, including browser JavaScript and native apps.
- Confidential clients run on a backend capable of protecting credentials.

Putting a client secret into a distributed application does not make it confidential.

Scope and consent

Scopes describe requested capabilities:

```
orders:read orders:write profile
```

Use least privilege and clear capability names. Consent is the resource owner's approval experience, not a replacement for authorization-server policy.

Access tokens

An access token is a credential presented to a resource server. It can be opaque or a JWT; OAuth does not require one format. Restrict it by lifetime, audience/resource, and scope. An API must enforce both intended audience and permitted action.

Refresh tokens

A refresh token goes only to the authorization server's token endpoint to obtain a new access token. It never goes to the resource API.

```
Client --refresh token--> Authorization Server
Client <--new access token (+ rotated refresh token)--
```

For public clients, use rotation with reuse detection or sender-constraining as appropriate.

API keys differ

An API key often identifies a project or caller for quota and service access. It does not automatically model a user, consent, or dynamic scopes. Protect it as a secret, but do not call it user authentication.

```
{
  "access_token": "opaque-or-structured-value",
  "token_type": "Bearer",
  "expires_in": 300,
  "refresh_token": "long-lived-secret",
  "scope": "orders:read"
}
```

Protect token responses with TLS and `Cache-Control: no-store`; never log them.

OAuth is not authentication, JWT is only a token format, a client ID is not secret, and token validity never replaces object ownership and domain-policy checks.

Security scenario

▼ Is an access token proof of application login?

Not necessarily; it delegates API access. Use OIDC and a validated ID token for user identity.

Threat drill

Scenario: A client presents an access token meant for another service or lacking the required permission.

Negative test: Send a token with the wrong audience, then a valid token without the required scope.

Expected result: The first is rejected as invalid and the second as insufficient scope; an access token is not treated as proof of user login.

Verification source

- RFC 9700: OAuth 2.0 Security Best Current Practice

Connect the ideas

A resource indicator or audience identifies the intended API and prevents a token for one service from working at another. Introspection supports opaque tokens or central decisions, while revocation invalidates a grant with a defined propagation delay. Consent does not replace least privilege, and scopes are not automatically internal roles.

Try it yourself

Send a token from the same issuer but for another resource and confirm rejection.

11. Modern OAuth flows

Authorization Code with PKCE, Client Credentials, Device Authorization, refresh tokens, and obsolete grants.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **HTTP:** The rules used to exchange requests and responses on the web.
- **Token:** A value representing identity or permission without resending a password.
- **CLI:** A text-based interface controlled by typed commands.
- **Scope:** A named permission requested or granted to a client, such as orders:read; it does not by itself prove ownership of an order.
- **Function:** A named, reusable block of code with one defined job.
- **Authentication:** Confirming that a user owns an identity.
- **Authorization:** Deciding which actions an identity may perform.
- **OAuth:** A delegation protocol granting limited access without sharing a password.
- **OIDC:** An identity layer over OAuth that identifies who signed in.
- **SAML:** A standard for exchanging sign-in information between services.
- **MFA:** Using more than one independent sign-in factor.
- **Secret:** A sensitive value such as an API key or service password.

Match the flow to the client

```
User + web/browser/native -> Authorization Code + PKCE
Service to service        -> Client Credentials
TV/CLI without browser    -> Device Authorization
Renew previous access     -> Refresh Token
```

Authorization Code + PKCE

```
Browser        Client            Authorization Server
  |              |                          |
  |--- start --->|-- authorize + challenge|
  |<-------------- redirect to login -------|
  |------------- login/consent ------------>|
  |<---- redirect: code + state ------------|
  |-- callback -->|-- code + verifier ------>|
  |              |<-- access/refresh token--|
```

```
function base64Url(string $bytes): string
{
    return rtrim(strtr(base64_encode($bytes), '+/', '-_'), '=');
}

$codeVerifier = base64Url(random_bytes(32));
$codeChallenge = base64Url(hash('sha256', $codeVerifier, true));
$state = base64Url(random_bytes(32));
$nonce = base64Url(random_bytes(32));

$_SESSION['oauth'] = [
    'state' => hash('sha256', $state),
    'verifier' => $codeVerifier,
    'nonce' => $nonce,
    'created_at' => time(),
];
```

Send `response_type=code`, exact redirect URI, scopes, state, OIDC nonce, code challenge, and `code_challenge_method=S256`. At callback, consume the bound state once, then exchange the code with the verifier through the token endpoint.

PKCE binds a code to the client instance that started the transaction. Use a new high-entropy verifier and S256 for every attempt.

Client Credentials

Use this for a confidential service acting as itself:

```
POST /oauth/token HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Authorization: Basic BASE64(client_id:client_secret)

grant_type=client_credentials&scope=reports:write
```

There is no end user. Prefer independent workload identities and consider mTLS or `private_key_jwt` for stronger client authentication.

Device Authorization

A constrained device obtains a secret `device_code`, a human `user_code`, and a verification URI. The user approves on a second device while the original device polls.

Respect the returned polling interval. Increase it by five seconds on `slow_down`, and stop on denial or expiration.

Refresh Token

Send refresh tokens only to the token endpoint and never request broader scope. Rotation replaces each used refresh token and detects reuse of an older member of the token family.

Assertion grants

RFC 7522 can exchange a SAML bearer assertion as an OAuth grant or use it for client authentication. This is a federation bridge, not the default SAML browser SSO flow. Validate signature, issuer, audience, time, and replay.

Do not start new systems with these

- The Implicit Grant exposes access tokens through the front channel; use Authorization Code + PKCE.
- The Resource Owner Password Credentials grant exposes user passwords to clients and is incompatible with modern MFA/WebAuthn. The OAuth Security BCP says it must not be used.

Security scenario

▼ What does PKCE prevent?

It binds an authorization code to the initiating client, so an interceptor cannot redeem it without the verifier.

Threat drill

Scenario: An attacker intercepts an authorization code for a public client and tries to redeem it first.

Negative test: Start a PKCE flow, then redeem the code without `code_verifier` and with a value that does not match the challenge.

Expected result: Both attempts fail; only the original verifier with the exact redirect URI is accepted.

Verification source

- RFC 7636: Proof Key for Code Exchange

Connect the ideas

Require PKCE S256 rather than plain, binding code to client, redirect URI, and verifier. Device flow is vulnerable to user-code phishing, so display service/client identity and enforce interval plus expiry. Rotate refresh tokens and detect reuse to revoke the family; do not start new implicit or password-grant designs.

Try it yourself

Test a wrong verifier, expired device code, and reused refresh token.

12. OAuth and OIDC security

State, nonce, PKCE, redirect URIs, browser token storage, BFF, revocation, and monitoring.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Cache:** A temporary copy that reduces waiting and repeated work.
- **Session:** Temporary server-side state used to recognize a user across requests.
- **Cookie:** A small value stored by the browser and sent with matching requests.
- **Token:** A value representing identity or permission without resending a password.
- **Authorization:** Deciding which actions an identity may perform.
- **CSRF:** An attack that makes a signed-in browser send an unintended request.
- **OAuth:** A delegation protocol granting limited access without sharing a password.
- **OIDC:** An identity layer over OAuth that identifies who signed in.
- **Secret:** A sensitive value such as an API key or service password.

Different controls protect different boundaries

Control	Primary purpose
session-bound `state`	callback CSRF and transaction swapping
OIDC `nonce`	authentication-response replay/substitution
PKCE	authorization-code theft or injection
exact redirect URI	redirecting results to an attacker

Use transaction-specific values and consume them once.

Callback skeleton

```
session_start();

$tx = $_SESSION['oauth'] ?? null;
unset($_SESSION['oauth']);

if (!is_array($tx) || time() - $tx['created_at'] > 600) {
    throw new RuntimeException('OAuth transaction expired');
}

$returnedState = $_GET['state'] ?? '';
if (
    !is_string($returnedState)
    || !hash_equals($tx['state'], hash('sha256', $returnedState))
) {
    throw new RuntimeException('Invalid state');
}

if (isset($_GET['error'])) {
    throw new RuntimeException('Authorization was not completed');
}

$code = $_GET['code'] ?? null;
if (!is_string($code) || $code === '') {
    throw new RuntimeException('Missing authorization code');
}
```

Do not log the callback query. Use `Referrer-Policy: no-referrer` and avoid third-party content on callback pages.

Redirect safety

Register and exactly match complete HTTPS redirect URIs. Avoid production wildcards and open redirects. Store post-login destinations server-side or restrict them to internal paths. Bind the transaction to the expected issuer when one client supports multiple authorization servers.

Browser architectures

A Backend for Frontend keeps tokens server-side:

```
Browser --Secure HttpOnly session cookie--> BFF
BFF --access token--> APIs
```

Protect the cookie session against CSRF and fixation. In browser-only clients, memory limits persistence; long-lived `localStorage` tokens are available to any JavaScript executing in that origin. Never ship a client secret to a SPA.

Keep tokens out of URLs, history, logs, and analytics.

Lifecycle

- Keep access tokens short-lived and resource/audience restricted.
- Store refresh tokens with stronger protection.
- Rotate refresh tokens and detect reuse.
- Revoke the token family on compromise, logout, or security changes.
- Use revocation or introspection where immediate control is required.
- Consider mTLS or DPoP for sender-constrained tokens in higher-risk systems.

OIDC checklist

1. Start discovery from a preconfigured trusted issuer.
2. Use Authorization Code + PKCE S256.
3. Bind state/PKCE and nonce to one browser transaction.
4. Require an exact registered redirect URI.
5. Validate signature, issuer, audience, expiration, and nonce.
6. Link accounts by `iss + sub`.
7. Do not turn email/domain claims into local privileges without an explicit policy.
8. Create a fresh local session and rotate its identifier.

Log issuer, client ID, flow, outcome, error code, and correlation ID, but never tokens, codes, verifiers, secrets, or assertions. Monitor invalid grants, refresh-token reuse, issuer/redirect mismatch, signature/audience failures, unusual scopes, and device-flow polling.

Security scenario

▼ Why validate state and nonce?

State binds the response and resists CSRF; nonce binds the ID token to the flow and resists replay.

Threat drill

Scenario: An attacker changes the redirect URI or replays `state` from an earlier transaction to mix login sessions.

Negative test: Try a similar but non-identical redirect URI, a missing state, and an already-consumed state.

Expected result: All three fail; redirect matching is exact and state is single-use and bound to the initiating browser.

Verification source

- RFC 9700: OAuth 2.0 Security Best Current Practice

Connect the ideas

Prevent mix-up by pinning issuer and binding the response to the expected authorization server; prevent code injection by binding code to state, nonce, PKCE, and session. Sender-constrained tokens such as DPoP or mTLS reduce stolen-token value but need replay cache and key lifecycle. Never log codes or tokens.

Try it yourself

Swap issuer at callback and replay a DPoP proof and confirm both fail.

13. Account lifecycle

Registration, verification, password recovery, identity changes, reauthentication, and session revocation.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Session:** Temporary server-side state used to recognize a user across requests.
- **Token:** A value representing identity or permission without resending a password.

Registration and enumeration

Normalize identifiers according to a documented policy, validate product rules, rate-limit abuse, hash passwords, and delay sensitive privileges until required contact details are verified. Use similar public responses whether an account exists while keeping useful internal security logs.

Verification token

```
$token = bin2hex(random_bytes(32));
$tokenHash = hash('sha256', $token);
// Store hash + user_id + expires_at + used_at.
// Send the raw token once over an HTTPS link.
```

Hash the received value, verify expiry and one-time status, then consume it inside a transaction. Do not store raw bearer tokens.

Password recovery

Use a short-lived one-time random token, a generic public response, and no guessable security questions. After success, rotate session identifiers and revoke other sessions or refresh tokens according to policy. Send a security notification.

Sensitive changes

Require recent authentication or MFA before changing password, email, recovery factors, or performing high-risk actions. Track `auth_time`; possession of an old session is not sufficient.

Verify a new email, notify the old address, and enforce uniqueness in the database. Define suspension, deactivation, deletion, export, and retention explicitly. Test races such as double-clicked links and concurrent resets.

Security scenario

▼ What should happen when an employee is disabled?

Block login, revoke sessions, tokens, and keys, and review ownership; deleting one user row is insufficient.

Threat drill

Scenario: A former employee has a still-live session or refresh token after the account is disabled.

Negative test: Disable the account, then attempt a new login, token refresh, and use of an existing session.

Expected result: All three paths fail within the defined revocation window, and the actor plus reason are audited without sensitive data.

Verification source

- OWASP Authentication Cheat Sheet

Connect the ideas

Normalize email under an explicit policy without assuming every provider behaves alike, and enforce uniqueness in the database. Make registration and recovery resistant to account enumeration. Define dormant-account handling, retention, deletion, and legal hold, and audit transitions plus actor.

Try it yourself

Test reset for existing and absent emails with equivalent response shape and acceptable timing.

14. MFA, TOTP, and passkeys

Authentication factors, TOTP, recovery codes, WebAuthn/passkeys, step-up, and safe recovery.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

This lesson introduces no extra technical label that needs memorizing; its new ideas are explained where they first appear.

Factors and TOTP

Factors are something known, possessed, or inherent/local user verification. Two steps from the same category are not strong MFA. SMS may improve password-only login but is exposed to phishing and SIM swap.

For TOTP, generate a random secret, prove setup with a code before enabling it, encrypt the secret under a managed key, use a small clock window, rate-limit checks, and avoid logging codes or secrets.

Recovery codes

Generate random one-time codes, show them once, store strong hashes, consume each atomically, notify the user, and require reauthentication to regenerate the set.

WebAuthn and passkeys

During registration the server sends a challenge and stores the resulting credential ID and public key. During authentication it verifies a fresh challenge, signature, origin, RP ID, user-verification policy, and relevant authenticator metadata. The server never stores the private key.

Passkeys resist phishing because credentials are scoped to the relying party and origin.

Step-up and recovery

Ask for stronger assurance before changing recovery factors, high-value payments, or secret access—not every click. Bind the result to a short-lived action context.

Recovery must not silently bypass MFA. Use risk-appropriate evidence, delays and notifications for sensitive changes, audited support procedures, and explicit revocation of lost credentials.

Reference

- OWASP Authentication Cheat Sheet

Security scenario

▼ Is SMS the strongest MFA?

No. It is exposed to SIM swaps and phishing; passkeys are generally phishing-resistant, and TOTP is stronger than a password alone.

Threat drill

Scenario: An attacker knows the password and tries to bypass the second factor or replay a passkey challenge for another origin.

Negative test: Test a wrong and reused OTP, then a WebAuthn assertion with a different challenge or origin.

Expected result: Every case fails, challenges are single-use, and account recovery cannot silently downgrade stronger protection.

Verification source

- W3C Web Authentication Level 3

Connect the ideas

Enrollment is sensitive and needs reauthentication, notification, and a credential inventory. Plan device loss and recovery without downgrading to weak questions. Understand synced passkeys versus device-bound credentials and attestation policy; do not require attestation without a reason. Step-up depends on action risk and authentication age.

Try it yourself

Enroll and revoke a passkey, then test recovery without the original device.

Reject a successfully used OTP

A valid TOTP is not reusable within its accepted time window. After verifying it, atomically consume the matched time step for that factor before granting access. A unique database constraint on `(factor_id, time_step)` lets only one concurrent request succeed. Track the matched step, not just the current server step when a clock window is allowed. Keep records until that step cannot be accepted again; factor rotation changes the factor ID. Test two simultaneous submissions: exactly one may succeed. Recovery codes need the same one-use guarantee.

RFC 6238, section 5.2

15. API keys and machine identity

Issuing, storing, scoping, rotating, and revoking API keys, service accounts, and mTLS.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **IP:** A numeric address that identifies a device or network interface.
- **TLS:** An encryption layer that protects data while it moves between two parties.
- **API:** A defined interface through which one program requests data or actions from another.
- **Scope:** A named permission requested or granted to a client, such as orders:read; it does not by itself prove ownership of an order.

A key is not a user identity

An API key usually identifies an application or integration. Bind it to a clear service/client principal with owner, purpose, environment, and least privilege.

Use a recognizable public prefix/identifier plus a random secret. Look up by identifier and store only a hash of the secret. Display the secret once.

Lifecycle

- Small deny-by-default scopes.
- Expiry where practical.
- Safe last-used metadata.
- Rotation with a short overlap.
- Immediate revocation.
- Separate keys per system and environment.

Send keys in an authorization or dedicated header over TLS, never in a query string. Apply per-key and per-tenant quotas. Treat IP restrictions only as an additional layer.

Service accounts and mTLS

Machine accounts should have no interactive login unless required, minimum privileges, and attributable audit events. Prefer short-lived workload identity over static key files when available.

Mutual TLS authenticates both endpoints but still requires certificate lifecycle management and operation-level authorization.

Security scenario

▼ How should an API key be stored?

Show it once, store a hash or use a secret manager, and add scope, expiry, rotation, and audit.

Threat drill

Scenario: A broadly privileged service API key leaks and is used from an unexpected environment.

Negative test: Try the key after revocation, from a disallowed source, and against an operation outside its scope.

Expected result: All three attempts fail, rotation is possible without downtime, and logs contain a key identifier rather than the secret value.

Verification source

- OWASP Secrets Management Cheat Sheet

Connect the ideas

Prefer workload identity federation or short-lived credentials over static secrets when infrastructure supports it. Signed client assertions need audience, time, jti, and replay prevention. HSM/KMS can keep private keys non-exportable, and rotation needs an overlap window plus version identifier without downtime.

Try it yourself

Rotate a live service credential and prove the old one expires after a defined overlap.

Security, identity, and authorization

16. ABAC, ReBAC, and tenant isolation

Attribute and relationship policies, multi-tenant boundaries, deny-by-default decisions, and testing.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **HTTP:** The rules used to exchange requests and responses on the web.
- **Cache:** A temporary copy that reduces waiting and repeated work.
- **Queue:** A line of background jobs waiting to be processed.
- **CLI:** A text-based interface controlled by typed commands.

Beyond RBAC

RBAC uses roles, ABAC uses subject/resource/context attributes, and ReBAC uses relationships such as owner, member, or manager. Adopt complexity only when it represents real policy.

Centralize decisions in policy objects or an authorization service. A UI may hide a button, but the server must authorize every HTTP, CLI, and queue path.

Tenant isolation

Derive tenant context from trusted identity, not only request data. Enforce it in queries, cache keys, object paths, queue messages, search indexes, exports, and administrative tooling.

Shared database, schema-per-tenant, and database-per-tenant models have different isolation and operational trade-offs; none is secure automatically.

Deny and test

Deny unknown actions by default. Define allow/deny precedence and record policy version plus an internal decision reason without exposing sensitive details.

Test same-tenant owners, unrelated users, cross-tenant ID collisions, privileged roles, missing resources, and every background path. Explicitly test IDOR by swapping identifiers.

Reference

- OWASP Authorization Cheat Sheet

Security scenario

▼ Where must a tenant boundary be enforced?

In every query, cache key, job, and object path, preferably with a data-layer control that cannot be forgotten.

Threat drill

Scenario: A user from tenant A supplies tenant B's identifier in the route or request body.

Negative test: Repeat both read and write requests with a valid resource identifier owned by another tenant.

Expected result: No data is returned and no update occurs; tenant context comes from trusted identity and is enforced inside the query.

Verification source

- OWASP Authorization Cheat Sheet

Connect the ideas

A policy engine needs versioned policy, explainable decisions, and cache invalidation when relationships or roles change. Model hierarchies explicitly rather than scattering conditions. Derive tenant from trusted identity and enforce it in queries, storage keys, cache keys, and logs, including side-channel tests for counts, timing, and 404 behavior.

Try it yourself

Change a relationship while authorization is cached and prove the old decision disappears.

17. Rate limiting and abuse resistance

Fixed/sliding windows, token buckets, login defenses, credential stuffing, CAPTCHA, and failure policy.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **IP:** A numeric address that identifies a device or network interface.
- **Token:** A value representing identity or permission without resending a password.

Choose a key per operation

IP alone is insufficient because users share NAT and attackers distribute traffic. Combine signals: account and network for login, contact and network for reset, key/tenant/endpoint for APIs, and user plus cost for expensive searches. Keep a global capacity limit.

Algorithms

Fixed windows are simple but burst at boundaries. Sliding windows are more precise and expensive. Token buckets allow controlled bursts; leaky buckets smooth output. Updates must be atomic in shared storage across servers.

Return `429 Too Many Requests` and an appropriate `Retry-After` without revealing account existence.

Login abuse

Avoid permanent lockouts that let attackers deny access to victims. Use progressive delay, layered limits, breach-aware password policy, MFA/passkeys, and user notification for unusual activity.

CAPTCHA adds friction but is not complete proof of humanity. Trigger it from risk signals and account for accessibility and privacy.

Define fail-open or fail-closed behavior when the limiter store is unavailable. A privileged login and a public read endpoint need different policy.

Security scenario

▼ Is IP-only rate limiting enough?

No. Shared addresses and botnets make IP only one signal; combine identity, device, route, and operation cost.

Threat drill

Scenario: An attacker distributes login attempts across many IP addresses to evade a simple per-IP counter.

Negative test: Send a burst against one account from different sources, then a legitimate request for another account.

Expected result: The attack is delayed or blocked using account-aware signals while the other user still works; the defense does not become a global denial of service.

Verification source

- OWASP Denial of Service Cheat Sheet

Connect the ideas

A distributed limiter needs atomic shared state or an approximation-tolerant design, including clock skew and failover behavior. Trust X-Forwarded-For only from known proxies. Add cost-based limits for expensive work across account, device, and IP, while ensuring the defense does not become denial of service for legitimate users.

Try it yourself

Run the limiter on two nodes and simulate shared-store loss plus IP rotation.

18. Security logging and audit trails

Security events, structured schemas, redaction, tamper resistance, alerting, and investigation.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **API:** A defined interface through which one program requests data or actions from another.
- **Session:** Temporary server-side state used to recognize a user across requests.

Different records

Operational logs support debugging, security events support detection, and an audit trail records who did what, when, and to which resource with stronger integrity and retention requirements.

Use a structured schema containing UTC time, event name, request ID, actor, tenant, action, resource, result, and a stable reason code. Prefer identifiers over unnecessary personal values and version the schema.

Events and exclusions

Record login/MFA/recovery outcomes, identity and role changes, key lifecycle, authorization denial, administrative activity, exports/deletion, secret access, and policy changes.

Never record passwords, raw tokens, session IDs, API secrets, or recovery codes.

Integrity and detection

Send records to centralized restricted storage, separate write and deletion authority, define retention and backup, synchronize clocks, and detect ingestion failure or tampering.

Alert on patterns such as distributed failures, privilege escalation, a new key followed by a large export, or unusual recovery. Give every alert an owner and runbook, and preserve request/trace IDs for investigation.

Reference

- OWASP Logging Cheat Sheet

Security scenario

▼ What belongs in a security event?

Actor, action, resource, outcome, time, and correlation ID without passwords, tokens, or excess sensitive data.

Threat drill

Scenario: An attacker tries to forge log entries with newline characters or flood telemetry with secrets.

Negative test: Submit a value containing a newline and a token-shaped string, then inspect the stored event and correlation chain.

Expected result: The event remains one structured record, secrets are redacted, and actor, time, outcome, and searchable correlation ID are present.

Verification source

- OWASP Logging Cheat Sheet

Connect the ideas

Audit trails need append-only or tamper-evident storage with write/delete separation, synchronized time, and a clear timestamp source. Define retention, legal access, and SIEM export, and test log injection plus redaction. Every alert needs an owner, runbook, and noise threshold.

Try it yourself

Modify a stored event or inject a newline and prove tampering is detected.

19. Secrets and key management

Secret managers, envelope encryption, rotation, revocation, access policy, break-glass, and incident response.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Runtime:** The period when a program is actually running.
- **API:** A defined interface through which one program requests data or actions from another.
- **Cache:** A temporary copy that reduces waiting and repeated work.
- **Scope:** A named permission requested or granted to a client, such as orders:read; it does not by itself prove ownership of an order.

Identify the secret

Database passwords, API keys, private keys, encryption keys, and signing secrets are sensitive. Public environment names and URLs usually are not. A `.env` file is a development loading mechanism, not a secret manager, and Base64 is not encryption.

Lifecycle

```
generate -> store -> distribute -> use -> rotate -> revoke -> destroy
```

Every secret needs an owner, purpose, consumers, lifetime, and rotation policy. Avoid sharing one credential across many services.

Secret managers and envelope encryption

Let the workload authenticate with least privilege and prefer short-lived or dynamic credentials. Cache only briefly in memory and never write values to logs.

With envelope encryption, a KMS/HSM master key protects a data-encryption key, while the DEK encrypts application data. Store ciphertext, encrypted DEK, algorithm, and key version. Use authenticated encryption with correct nonces and contextual associated data.

Rotation

Store a key ID/version. Create a new key, write new data with it, keep old versions readable, re-encrypt where needed, then revoke after a safe window. Destroying an encryption key may permanently destroy access to data.

Access and incident response

Apply least privilege, separate administration from runtime, audit access and changes, provide controlled break-glass procedures, alert on unusual access, and test encrypted backup recovery.

After exposure, restrict use, rotate or revoke, determine scope from audit records, fix the source, and inspect history, artifacts, and logs. Removing a value from Git does not revoke it.

Reference

- OWASP Secrets Management Cheat Sheet

Security scenario

▼ What is the first response to a leaked secret?

Revoke or rotate it immediately, determine usage from logs, fix the source, and monitor abuse.

Threat drill

Scenario: An old encryption-key version leaks while some stored data still depends on it.

Negative test: Rotate the key, encrypt new data, decrypt legacy data during the migration window, then disable the old version.

Expected result: New writes immediately use the new key, controlled migration succeeds, and the retired key cannot perform new operations.

Verification source

- OWASP Secrets Management Cheat Sheet

Connect the ideas

Define a cryptoperiod for each key and use KMS/HSM for non-exportable keys or separation of duties. Encrypted backups need separate key backup and restore testing. Key destruction is a documented irreversible decision, and rotation differs from re-encryption; use dual-read/single-write during migration.

Try it yourself

Rotate a key, restore an old backup, and prove required key availability matches policy.

Security, identity, and authorization

20. Threat modeling and authorization testing

Model login threats and build authorization test matrices for roles, ownership, and tenant isolation.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **HTTP:** The rules used to exchange requests and responses on the web.
- **API:** A defined interface through which one program requests data or actions from another.
- **Session:** Temporary server-side state used to recognize a user across requests.

Assets and trust boundaries

A threat model is not a generic vulnerability list. Identify assets—credentials, sessions, tokens, user data, and audit records—then actors such as users, administrators, internal services, identity providers, and attackers. Every browser, API, IdP, and database transition is a trust boundary requiring independent validation.

Model the login flow

At each step ask whether identity can be spoofed, parameters such as `return_to` or `tenant_id` can be altered, secrets reach logs, sensitive actions lack evidence, errors reveal account existence, or recovery bypasses MFA. Convert every material threat into a control, test, and telemetry event.

Session fixation, for example, maps to session-ID regeneration after login, a test proving the identifier changed and the old one fails, and an audit event that never records the identifier value.

Authorization test matrix

Subject	Resource	Tenant	Action	Expected
Owner	Own invoice	Same	read	Allow
User	Other invoice	Same	read	Deny
Admin	Invoice	Same	refund	Policy-dependent allow
Admin	Invoice	Other	read	Deny
Suspended user	Own invoice	Same	read	Deny or documented read-only

Add missing identity, stale role, deleted resource, and random identifier cases. Unit-test the policy and integration-test the endpoint so routing and middleware cannot bypass it.

Deny by default

An action without an explicit policy is denied. A cross-tenant lookup may return `404` rather than `403` to avoid revealing resource existence, while recording the internal reason. Hiding a button is never authorization; the server decides from trusted subject, action, resource, and context.

Review after change

Revisit the model when adding an identity provider, webhook, role, tenant boundary, or recovery path. Every threat needs an owner, decision, test, and documented residual risk.

Check your understanding

01 **How do an asset and a trust boundary differ?**

> **Answer:** An asset is protected value; a boundary is a transition where trust assumptions change and validation must occur.

02 **Why is an administrator happy path insufficient?**

> **Answer:** Serious failures appear in ownership, tenant crossing, suspended users, and stale privileges.

03 **When can 404 be preferable to 403?**

> **Answer:** When revealing the existence of an unauthorized resource is itself sensitive, while the internal denial reason remains audited.

04 **Turn session fixation into a control and test.**

> **Answer:** Regenerate after login and invalidate the old ID; verify the old session fails and the new one has exactly the intended privileges.

Threat drill

Scenario: Tests cover only the happy path while a secondary entry point performs the same action without an authorization check.

Negative test: Turn every asset and trust boundary in the model into allow and deny tests, then repeat them across HTTP and background jobs.

Expected result: Every sensitive action has evidence of success for the allowed actor and rejection for the disallowed actor at every entry point.

Verification source

- OWASP Threat Modeling

Connect the ideas

Use STRIDE as a lens rather than a checklist: spoofing, tampering, repudiation, information disclosure, denial of service, and elevation map to assets and trust boundaries. Turn each abuse case into a test or control with an owner, and revisit the model when data flow, identity, or a third party changes.

Try it yourself

Choose one flow and record a threat, test, and owner for each applicable category.

21. Devices, sessions, breached passwords, and SCIM

Manage device sessions, screen breached passwords, and implement safe enterprise provisioning through SCIM.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **HTTP:** The rules used to exchange requests and responses on the web.
- **IP:** A numeric address that identifies a device or network interface.
- **Session:** Temporary server-side state used to recognize a user across requests.
- **Token:** A value representing identity or permission without resending a password.
- **Scope:** A named permission requested or granted to a client, such as orders:read; it does not by itself prove ownership of an order.
- **Function:** A named, reusable block of code with one defined job.

Manageable session inventory

For each session, store a server-side hash of its identifier, creation and last-use times, expiry, an approximate device label, and minimized IP metadata where operationally and legally justified. User-Agent data is mutable and does not prove device identity.

An “Your devices” page lists active sessions and revokes one or all. “Log out all devices” can increment a session version or revoke sessions and refresh tokens. Require re-authentication for this sensitive operation.

```
function revokeAllSessions(PDO $pdo, int $userId): void
{
    $statement = $pdo->prepare(
        'UPDATE sessions SET revoked_at = CURRENT_TIMESTAMP
         WHERE user_id = :user_id AND revoked_at IS NULL'
    );
    $statement->execute(['user_id' => $userId]);
}
```

Notify the user after password changes or unusual new sessions without placing revocation credentials in email or logs.

Breached-password screening

Length rules and `password_hash` do not reveal whether a chosen password appeared in a breach. Screen during registration and change through a trusted dataset or service. Never send the raw password; use a privacy-preserving protocol such as k-anonymity where available, with explicit timeout and outage behavior.

Allow password managers and paste. Do not force periodic changes without evidence of compromise.

SCIM provisions accounts; it does not log users in

SCIM is an HTTP protocol for managing users and groups between an identity system and an application. It complements rather than replaces SAML or OIDC. Core actions include user creation, attribute updates, `active` disablement, and group membership.

Bind every SCIM client to one tenant, scope its credential, validate media type and schema, and use versions or ETags to prevent lost concurrent updates.

Deprovisioning is critical

When `active=false`, block new login, revoke relevant sessions and tokens, remove group-derived privileges, preserve or transfer data under retention policy, and audit actor, source, and correlation ID. Test duplicate, replayed, and out-of-order updates; make disablement idempotent.

Check your understanding

01 **Why is a displayed device name not identity proof?**

> **Answer:** It is usually inferred from mutable, shared User-Agent and IP data; use it for display and risk hints, not authentication.

02 **What should “log out all devices” do?**

> **Answer:** Revoke applicable sessions and refresh tokens, audit the action, and notify the user without leaking credentials.

03 **Why does SCIM not replace OIDC?**

> **Answer:** SCIM manages account and group lifecycle; OIDC authenticates a user during login.

04 **Why must deprovisioning be idempotent?**

> **Answer:** Providers retry requests; repetition must not duplicate deletion, transfer, or another dangerous side effect.

Threat drill

Scenario: An administrator disables a user through SCIM while sessions remain active on several devices.

Negative test: Deliver the disable event twice, then try every session, token refresh, and a login using a breached password.

Expected result: Processing is idempotent, sessions and tokens are revoked within policy, and the breached password is rejected with privacy and abuse controls.

Verification source

- RFC 7644: SCIM Protocol

Connect the ideas

Device binding is a risk signal rather than absolute truth because browsers and devices change. Show users session inventory, last activity, approximate location, and individual/global revocation. For SCIM, test PATCH, groups, idempotency, and out-of-order events, and make deprovisioning revoke sessions and tokens within a monitored SLO.

Try it yourself

Send duplicate and delayed SCIM disable events and verify final state plus revocation.

22. PAR, JAR, RAR, and token incidents

Advanced OAuth authorization requests and practical response to leaked access tokens, refresh tokens, or signing credentials.

Before you start

Read this lesson in three passes: understand the problem, follow the example, then try the final check yourself. The terms below are explained before they are used in detail.

New terms in this lesson

- **Session:** Temporary server-side state used to recognize a user across requests.
- **Token:** A value representing identity or permission without resending a password.
- **Scope:** A named permission requested or granted to a client, such as orders:read; it does not by itself prove ownership of an order.

The authorization-request problem

A normal authorization request passes through the browser with `client_id`, `redirect_uri`, `scope`, `state`, and possibly sensitive details. Signed or directly pushed requests reduce manipulation and leakage but do not replace PKCE, state, nonce, or exact redirect validation.

PAR

With Pushed Authorization Requests, the client sends details directly to an authenticated PAR endpoint and receives a short-lived `request_uri`. The browser then carries the reference to the authorization endpoint. Bind the reference to its client and enforce its expiration and reuse policy.

JAR

JWT-Secured Authorization Request packages parameters in a signed Request Object. Validate signature, `iss`, `aud`, `exp`, and conflicts between JWT and query parameters. A signature protects integrity and authenticity; it does not hide content unless encryption is separately applied.

RAR

Rich Authorization Requests place structured detail in `authorization_details` rather than relying only on a broad scope. Payment type, amount, currency, and recipient can support precise policy and consent. Validate a registered schema and prevent clients from requesting unregistered power.

PAR, JAR, and RAR solve different problems and can be combined: RAR describes rich authority, JAR protects a Request Object, and PAR pushes it through a back channel.

Incident: a leaked access token

Activate incident command, identify whether exposure affects one token, a client, or a signing key, revoke the grant or session where possible, stop continued leakage from logs, preserve restricted evidence, and rotate a broader secret when necessary. Hunt suspicious use by `jti`, client, or subject without logging the raw token. Notify affected parties under policy and law, fix the source, and add detection and regression tests.

A leaked refresh token requires family or grant revocation and reuse investigation. A leaked signing key may require rejecting all tokens signed by it, publishing new JWKS material, and coordinating safe retirement of the old key.

A tested runbook

Document contacts, revocation authority, safe commands, evidence sources, communication templates, and closure criteria. Run a tabletop exercise for a token found in Git history, an error trace, or a frontend bundle. Measure detection and containment time.

Check your understanding

01 **How do PAR and JAR differ?**

> **Answer:** PAR pushes parameters through a back channel and returns a reference; JAR packages them in a signed Request Object.

02 **Does a JAR signature hide its data?**

> **Answer:** No. It protects integrity and authenticity; confidentiality requires separate encryption and key policy.

03 **What does RAR add beyond a broad scope?**

> **Answer:** Structured transaction details that support more precise consent and policy decisions.

04 **What follows a leaked refresh token?**

> **Answer:** Revoke its family or grant, investigate reuse, contain the leak, rotate broader secrets if needed, and hunt suspicious use.

Threat drill

Scenario: An attacker alters authorization parameters in the browser or replays a stolen token during incident response.

Negative test: Change a parameter outside the protected request, then revoke a refresh-token family and try to use it again.

Expected result: The server rejects parameters not bound to the protected request, detects replay, and revokes the family without issuing a new token.

Verification source

- RFC 9126: Pushed Authorization Requests

Connect the ideas

Draw a sequence for browser, client, authorization server, and resource server, identifying parameters protected by PAR/JAR and validations at each party. Validate request-object issuer, audience, time, jti, signature, and replay protection. Run a token-incident tabletop covering detection, containment, revocation, evidence, and postmortem.

Try it yourself

Replay the same request object and alter an external parameter, proving both replay and conflict are rejected.

06

TRACK · 6 LESSONS

Databases and PDO

Progressive explanations, practical examples, and corrections that take each idea into real code.

Databases & PDO

Secure connections, prepared statements, ORM, transactions, ACID, isolation, locks, and indexes.

This path starts with the first database connection, then explains safe reads and writes, transactions, indexes, and migrations step by step.

0. SQL tables, constraints and joins.
1. PDO and prepared statements
2. ORM and data-access patterns
3. Transactions and ACID
4. Isolation, locking, and deadlocks
5. Schema, indexes, and migrations

Learn SQL and PDO before relying on an ORM: abstractions do not remove the need to understand queries, indexes, and transactions.

SQL foundations before PDO and ORM

Tables, constraints, joins, grouping and safe changes.

Before PDO or an ORM, understand the question you ask the database. Build learners and attempts, then report every learner's attempt count and average score, including learners with no attempts.

Model two tables

A table contains rows with defined columns. A row represents one learner or one attempt. A primary key identifies a row; names are insufficient because two people can share a name. A foreign key links an attempt to an existing learner. This is a one-to-many relationship: one learner has many attempts, each attempt belongs to one learner.

`NOT NULL` requires a value, while `CHECK` enforces a rule such as a score from zero through 100. PHP validation provides a helpful message; database constraints also protect writes made by other programs.

State the question before SQL

Start with all learners, attach their attempts, then summarize each learner's group. `SELECT` chooses output columns; `FROM` selects the starting table. `LEFT JOIN` preserves learners without attempts, while `INNER JOIN` excludes them. `ON` states which rows match; `AS` provides a short table alias or a clear output name.

```
CREATE TABLE learners (id INTEGER PRIMARY KEY, name TEXT NOT NULL);
CREATE TABLE attempts (
    id INTEGER PRIMARY KEY,
    learner_id INTEGER NOT NULL REFERENCES learners(id),
    score INTEGER NOT NULL CHECK(score BETWEEN 0 AND 100)
);
INSERT INTO learners VALUES (1, 'Omar'), (2, 'Mona'), (3, 'Ali');
INSERT INTO attempts VALUES (1, 1, 80), (2, 1, 100), (3, 2, 60);
SELECT l.id, l.name, COUNT(a.id) AS attempts, AVG(a.score) AS average
FROM learners AS l
LEFT JOIN attempts AS a ON a.learner_id = l.id
GROUP BY l.id, l.name
ORDER BY l.id;
```

Run the complete implementation with `php sql-foundations.php` in the lab. It creates an in-memory SQLite database, enables foreign-key enforcement, executes the same SQL, and prints JSON. PDO is PHP's database connection interface; the next lesson explains connection setup and prepared queries.

Learner	Attempts	Average
Omar	2	90
Mona	1	60
Ali	0	NULL

`GROUP BY` groups learner rows before `COUNT` and `AVG`. `COUNT(a.id)` ignores the missing attempt in a LEFT JOIN result; `COUNT(*)` would count Ali's preserved row as one attempt. `AVG` ignores NULL and returns NULL, not zero, when no scores exist. `ORDER BY` guarantees the requested ordering; insertion order does not guarantee query output order.

Filter and modify

`WHERE` filters rows before grouping; `HAVING` filters groups afterward. Add `HAVING AVG(a.score) > 70` after GROUP BY and before ORDER BY: only Omar remains. A WHERE condition on the optional table after LEFT JOIN can accidentally remove learners with no attempts.

`NULL` means missing or unknown. Test it with `IS NULL`, not `= NULL`: a comparison with NULL does not yield true, and WHERE retains only true conditions.

```
UPDATE attempts SET score = 85 WHERE id = 1;
DELETE FROM attempts WHERE id = 3;
```

Run these only after creating the tables. The first changes one attempt; the second deletes one. Before important mutations, inspect a SELECT using the same WHERE, and use a transaction when several changes must succeed together. Omitting WHERE can change every row. In application code, bind values through placeholders rather than concatenating user input into SQL.

An index is not a correctness rule

An index is an extra structure that can speed up lookups at the cost of storage and writes. An index on `attempts(learner_id)` may help the join. A normal index does not prevent duplicates; UNIQUE enforces uniqueness according to the engine's rules. Inspect the execution plan on realistic data before judging performance.

Try and explain

Add a fourth learner without attempts: expect count zero and average NULL. Insert score 101 or `learner_id=999`: constraints reject both. Change LEFT JOIN to INNER JOIN: Ali disappears. Trace rows before joining, after joining, and after grouping instead of memorizing syntax.

SQLite makes this lab easy to run. Column types, foreign-key activation, identity syntax, locking, and isolation differ across engines; continue with PDO and transaction lessons before moving the example to MySQL or PostgreSQL.

SQLite transactions

1. PDO and prepared statements

PDO connections, error handling, prepared statements, fetching, and injection prevention.

Beginner bridge

Treat a database connection as a boundary with two contracts: connection configuration and query execution. Configure the character set, exception mode, and fetch behavior explicitly so the same code does not change meaning across environments.

Prepared statements separate SQL structure from data values. They do not validate business rules and placeholders normally cannot represent table or column names. Dynamic identifiers must come from a small allowlist, while values should remain bound parameters.

PDO provides one interface for several database drivers, but SQL and capabilities remain driver-specific.

```
$pdo = new PDO(
    'mysql:host=127.0.0.1;dbname=shop;charset=utf8mb4',
    getenv('DB_USER'),
    getenv('DB_PASSWORD'),
    [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false,
    ]
);
```

Keep credentials out of Git, specify charset in the DSN, and never expose exception details to users.

```
$stmt = $pdo->prepare(
    'SELECT id, name FROM users
     WHERE email = :email AND status = :status'
);
$stmt->execute(['email' => $email, 'status' => 'active']);
$user = $stmt->fetch();
```

Placeholders separate values from SQL structure. They cannot represent table/column names, keywords, sort direction, or a whole `IN` list. Allow-list identifiers and create one placeholder per list value.

`fetch()` returns one row or false; `fetchAll()` may use substantial memory; `fetchColumn()` returns one column. `rowCount()` behavior for SELECT is driver-specific. Stream large results and select only needed columns.

`bindValue` binds the current value; `bindParam` binds a variable by reference. Passing an array to `execute` is often simplest. Log database details securely while returning a generic error and request ID.

Lesson-specific problems

▼ Why not rely on escaping instead of prepared statements?

Prepared statements separate SQL structure from values; manual escaping is error-prone and context-sensitive.

▼ Can a parameter represent a table name?

Normally no; placeholders represent values. Select identifiers from an allowlist.

Run and verify

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Execute this checkpoint inside the lesson environment:

```
php pdo-lab.php
```

Success criterion: Valid input succeeds, a value such as `1 OR 1=1` remains data rather than SQL, and only the expected row count is returned.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

2. ORM and data-access patterns

Active Record, Data Mapper, relationships, N+1 queries, and abstraction limits.

Beginner bridge

An ORM maps application objects to database operations, but it does not remove SQL costs or transactional boundaries. Every convenient property access can still become a query, and object graphs can hide large result sets.

Use repositories or query services to make important access patterns visible. Inspect generated SQL, count queries, and decide deliberately between lazy loading, eager loading, joins, and projections. The model used for writing a transaction does not have to be the best model for reading a report.

An ORM maps rows and relationships to objects. It reduces repetitive SQL but does not remove query cost or database semantics.

Active Record combines row state and persistence; Laravel Eloquent is a PHP example. It is convenient for CRUD but can couple domain and storage concerns.

Data Mapper keeps entities closer to domain logic while a mapper/repository handles persistence; Doctrine is a PHP example. It offers stronger separation with more concepts such as Unit of Work and Identity Map. Django is a Python comparison, not a PHP package.

Model one-to-one, one-to-many, and many-to-many relationships while preserving database foreign-key constraints. Configure cascades deliberately.

N+1 occurs when loading a relation triggers one query per parent. Use eager loading or a tailored join/select, but measure because eager-loading everything can over-fetch.

Never mass-assign raw `$_POST`; allow-list validated fields so attackers cannot set `is_admin`. Do not serialize entities containing hashes/secrets.

Use direct SQL or a query builder for complex reports, bulk operations, database-specific features, and measured hot paths. ORM, query builder, and PDO can coexist.

Lesson-specific problems

▼ What is the N+1 problem?

One query loads a list and one more runs per item; use suitable eager loading or joins and monitor query count.

▼ Does an ORM remove the need to understand SQL?

No. Query plans, indexes, transactions, and boundaries still determine correctness and performance.

Run and verify

The PDO model demonstrates N+1 with naive_queries=21 and batch_queries=2; it does not test a particular ORM's lazy-loading behavior.

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Execute this checkpoint inside the lesson environment:

```
php orm-query-lab.php
```

Extended integration exercise target: The query counter proves that loading 20 records does not issue 21 queries, and no lazy loading occurs after the unit of work closes.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

3. Transactions and ACID

Auto-commit, commit, rollback, ACID, and safe multi-step operations.

Beginner bridge

A transaction protects one database invariant across several statements. Commit makes the group durable; rollback discards the group when a required step fails. Atomicity does not mean that emails, HTTP calls, or another independent database automatically roll back with it.

Write the invariant first, keep the transaction short, and include every dependent database change. For external effects, store an outbox event in the same transaction and deliver it later with idempotent processing.

A transaction prevents partially completed multi-step changes. Under normal auto-commit, each statement is independent until an explicit transaction begins.

ACID means atomicity (all or rollback), consistency (constraints/rules remain valid), isolation (concurrent work behaves according to the selected level), and durability (committed data survives under system guarantees).

```
$pdo->beginTransaction();

try {
    $debit->execute(['amount' => $amount, 'id' => $from]);
    $credit->execute(['amount' => $amount, 'id' => $to]);
    $pdo->commit();
} catch (Throwable $e) {
    if ($pdo->inTransaction()) {
        $pdo->rollBack();
    }
    throw $e;
}
```

For balance transfer, lock/check both accounts consistently and update both before commit. Exact locking syntax is database-specific. Keep transactions short; validate first and avoid waiting for external APIs or email while holding locks.

Database rollback cannot undo an already-sent email. Use a transactional outbox: store an event in the same transaction and publish it later.

Some databases implicitly commit DDL such as `CREATE TABLE` or `DROP TABLE`; never assume all schema changes roll back. Constraints remain necessary even inside transactions.

Oracle deserves the same warning: DML transactions provide ACID semantics, while DDL normally issues implicit commits before and after the statement. Do not mix schema changes into a business transaction expecting one rollback boundary.

Lesson-specific problems

▼ When should a transaction roll back?

When any step within one invariant fails, preventing partial writes.

▼ Does a database transaction make an API call atomic?

No. The database cannot normally roll back another service; use an outbox or failure-aware workflow.

Run and verify

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Execute this checkpoint inside the lesson environment:

```
php transaction-lab.php
```

Success criterion: On success every required table changes; on a mid-operation exception none changes. Repeat after a simulated connection loss.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

4. Isolation, locking, and deadlocks

Concurrency anomalies, isolation levels, pessimistic/optimistic locking, retries, and savepoints.

Beginner bridge

Concurrent transactions can each be locally correct and still produce a wrong combined result. Isolation describes which intermediate states one transaction may observe, while locks and multiversion techniques are mechanisms used to provide that behavior.

Start from the anomaly the business rule cannot tolerate: dirty read, non-repeatable read, phantom, or lost update. Then choose an isolation level and explicit locking strategy, keep a consistent lock order, and treat deadlock retry as a normal bounded path.

Transactions can overlap; isolation controls what each sees. Anomalies include dirty reads, non-repeatable reads, phantoms, lost updates, and write skew.

Levels commonly range from Read Uncommitted through Read Committed and Repeatable Read to Serializable. Exact MVCC and locking behavior differs across MySQL, PostgreSQL, and other systems; verify your database rather than relying on a generic table.

Pessimistic locking uses database-specific syntax such as `SELECT ... FOR UPDATE`. Good indexes keep the lock scope focused.

Optimistic locking updates only when a version matches:

```
UPDATE products
SET stock = :new_stock, version = version + 1
WHERE id = :id AND version = :old_version
```

Zero affected rows indicates a conflict.

Deadlocks are expected concurrency outcomes. Access resources in consistent order, keep transactions short, index queries, and use bounded retry with backoff/jitter for retryable errors. Make the operation idempotent or use an idempotency key.

PDO has no portable true nested transactions. Some databases support savepoints, but rolling back to one does not finish the outer transaction. Choose isolation from business invariants and combine it with constraints, atomic updates, and deliberate locking.

Lesson-specific problems

▼ Why does a deadlock occur?

Transactions hold different resources and each waits for the other; keep lock order consistent and retry safely.

▼ Is the highest isolation always best?

No. It prevents more anomalies but may increase waiting and conflicts; choose for the invariant and workload.

Run and verify

Two real processes compete on SQLite: one purchases and one rejects insufficient stock, leaving orders=1 stock=2. Test deadlocks and isolation separately on the production engine.

Use the downloadable lab for supplied scripts. Commands for Composer, FPM, Docker, or a real server run inside the corresponding configured project, not an empty folder.

Execute this checkpoint inside the lesson environment:

```
php locking-lab.php
```

Extended integration exercise target: Two concurrent operations produce the expected final state without a lost update, and deadlock retry terminates within an explicit bound.

Record the exit code and observed evidence. If reality differs, explain the environmental or design assumption that failed instead of editing the expectation to match a defect.

5. Schema, indexes, and migrations

Types, constraints, relationships, composite indexes, query plans, pagination, and safe schema change.

Beginner bridge

A schema is an executable statement of allowed data: types, nullability, keys, uniqueness, and relationships. Application validation improves messages, while database constraints protect every writer, including scripts and future services.

Indexes are workload-specific copies that accelerate selected reads at the cost of space and write work. Migrations must remain compatible with old and new application versions during deployment; use expand, migrate data, switch readers and writers, then contract in a later release.

Protect invariants in the database as well as application code. Use `NOT NULL`, `UNIQUE`, `CHECK`, foreign keys, and appropriate types. Store money as integer minor units or an appropriate decimal type, not binary float; define a timezone policy.

Indexes speed lookup, join, and ordering at the cost of storage and write work:

```
CREATE INDEX idx_orders_user_status_created
ON orders (user_id, status, created_at);
```

Column order matters. Indexes must be chosen for measured query patterns, not added to every column.

Use `EXPLAIN` and, when supported, actual-analysis plans. Test production-like volumes and monitor slow queries and latency percentiles. Watch for N+1, unnecessary `SELECT *`, deep OFFSET pagination, casts/functions blocking indexes, and excessive round trips.

Keyset pagination often scales better:

```
SELECT *
FROM orders
WHERE id < :last_seen_id
ORDER BY id DESC
LIMIT 20
```

Migrations should be versioned, reviewed, automated, and tested with realistic data. Back up and test restoration, monitor locks, separate heavy backfills, and do not assume DDL rollback. Expand/contract deployments add compatible structure, migrate readers/writers and data, then remove old structure later.

Keep schema migrations distinct from development/reference seeds, make seeds idempotent, and never copy production personal data into fixtures.

Lesson-specific problems

▼ Why not index every column?

Indexes consume space and slow writes; design for real queries and inspect execution plans.

▼ How do you deploy a risky migration safely?

Use expand/migrate/contract: add compatible structure, move data, update code, and remove old structure later.

Cumulative project: inventory that cannot oversell

Design `products`, `orders`, and `order_items`, then implement one purchase transaction through PDO. The database must protect rules that remain true even when application code is wrong: positive quantities, valid foreign keys, and unique order identifiers.

Concurrent scenario

1. Begin a transaction and read the product row using the lock appropriate for your engine.
2. Reject the operation when stock is below requested quantity.
3. Create the order and its items, update stock, and commit.
4. Roll back every change on failure; never leave an empty order or a deduction without an order.

Deterministic verification data

```
initial_stock=5
buyer_A_requests=4
buyer_B_requests=4
expected_successes=1
expected_final_stock=1
expected_negative_stock_rows=0
```

Run both purchases concurrently several times. Two successes or negative stock proves a real race condition, not a cosmetic test failure.

Measure the index and migration

Save `EXPLAIN` output for lookup by `user_id, created_at` before and after the index, recording estimated or examined rows instead of claiming it “became faster.” Apply the migration through expand/migrate/contract: add compatible structure, backfill in resumable batches, switch reads and writes, and defer old-field removal to a later release after proving it is unused.

Acceptance criteria

- Every client value is passed as a prepared parameter.

- Integration tests prove commit, rollback, and the two-buyer race.
- Constraints reject impossible states inside the database.
- A rollback or roll-forward plan exists, and backup restoration is tested rather than assumed.